Học LLM, Attention, VLM và Inference

Lộ trình 75 bài

Ghi chú học tập bằng tiếng Việt

Tháng 11, 2026

Mục lục

Học LLM, Attention, VLM và Inference Optimization từ đầu: lộ trình 74 ghi chú . . . 5
Transformer từ đầu: attention là một phép tra cứu mềm mà ta có thể tính tay . . . 10
Khối Transformer: nhiều đầu attention, đường residual, chuẩn hoá và FFN cùng nhau tạo một lớp . . . 19
Mã hoá vị trí: attention không biết thứ tự, nên ta phải đưa vị trí vào bằng cách quay (RoPE) hoặc trừ điểm theo khoảng cách (ALiBi) . . . 27
Scaling laws và tính toán: đếm tham số, FLOPs và bộ nhớ trước khi huấn luyện . . . 36
KV cache và chi phí suy luận: vì sao sinh từng token bị chặn bởi bộ nhớ . . . 44
Attention tuyến tính, attention thưa và SSM: đổi chi phí bậc hai lấy trạng thái cố định . . . 52
Mixture of Experts và định tuyến: nhiều chuyên gia, nhưng mỗi token chỉ gọi vài người . . . 61
Lượng tử hoá và bộ nhớ GPU: bớt bit cho trọng số, bớt chuyến đi cho dữ liệu . . . 69
Mô hình thị giác-ngôn ngữ: biến ảnh thành token để LLM đọc . . . 77
00 Preface: Toán nền tảng và hướng dẫn đọc (LLM, VLM, suy luận) . . . 85
01 Attention Is All You Need (2017) . . . 100
02 Language Models are Few-Shot Learners (2020) . . . 106
03 Scaling Laws for Neural Language Models (2020) . . . 114
04 Training Compute-Optimal Large Language Models (2022) . . . 121
05 LLaMA: Open and Efficient Foundation Language Models (2023) . . . 128
06 Mistral 7B (2023) . . . 135
07 Training language models to follow instructions with human feedback (2022) . . . 142
08 LoRA: Low-Rank Adaptation of Large Language Models (2021) . . . 150
09 On Layer Normalization in the Transformer Architecture (2020) . . . 156
10 Root Mean Square Layer Normalization (2019) . . . 162
11 GLU Variants Improve Transformer (2020) . . . 168
12 RoFormer: Enhanced Transformer with Rotary Position Embedding (2021) . . . 173
13 Train Short, Test Long: Attention with Linear Biases Enables Input Length Extrapolation (2021) . . . 180
14 Fast Transformer Decoding: One Write-Head is All You Need (2019) . . . 186
15 GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints (2023) . . . 192
16 DeepSeek-V2: A Strong, Economical, and Efficient Mixture-of-Experts Language Model (2024) . . . 196
17 Longformer: The Long-Document Transformer (2020) . . . 204
18 Transformers are RNNs: Fast Autoregressive Transformers with Linear Attention (2020) . . . 211
19 Retentive Network: A Successor to Transformer for Large Language Models (2023) . . . 217
20 Gated Linear Attention Transformers with Hardware-Efficient Training (2024) . . . 224
21 Mamba: Linear-Time Sequence Modeling with Selective State Spaces (2023) . . . 233
22 Transformers are SSMs: Generalized Models and Efficient Algorithms Through Structured State Space Duality (2024) . . . 243
23 Parallelizing Linear Transformers with the Delta Rule over Sequence Length (2024) . . . 251
24 Gated Delta Networks: Improving Mamba2 with Delta Rule (2025) . . . 258
25 Jamba: A Hybrid Transformer-Mamba Language Model (2024) . . . 268
26 Kimi Linear: An Expressive, Efficient Attention Architecture (2025) . . . 274
27 Switch Transformers: Scaling to Trillion Parameter Models with Simple and Efficient Sparsity (2021) . . . 282
28 Mixtral of Experts (2024) . . . 289
29 An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale (2020) . . . 296
30 Learning Transferable Visual Models From Natural Language Supervision (CLIP, 2021) . . . 302

31 Sigmoid Loss for Language Image Pre-Training (SigLIP, 2023) . . . 309
32 Flamingo: a Visual Language Model for Few-Shot Learning (2022) . . . 315
33 BLIP-2: Bootstrapping Language-Image Pre-training with Frozen Image Encoders and Large Language Models (2023) . . . 322
34 Visual Instruction Tuning (LLaVA) (2023) . . . 327
35 Improved Baselines with Visual Instruction Tuning (LLaVA-1.5) (2024) . . . 333
36 How Far Are We to GPT-4V? Closing the Gap to Commercial Multimodal Models with Open-Source Suites (InternVL 1.5) (2024) . . . 339
37 Qwen2-VL: Enhancing Vision-Language Model's Perception of the World at Any Resolution (2024) . . . 345
38 SmolVLM: Redefining small and efficient multimodal models (2025) . . . 352
39 OCR-free Document Understanding Transformer (2022) . . . 358
40 Nougat: Neural Optical Understanding for Academic Documents (2023) . . . 364
41 General OCR Theory: Towards OCR-2.0 via a Unified End-to-end Model (2024) . . . 370
42 olmOCR: Unlocking Trillions of Tokens in PDFs with Vision Language Models (2025) . . . 375
44 FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness (2022) . . . 381
45 FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning (2023) . . . 389
46 Efficient Memory Management for Large Language Model Serving with PagedAttention (2023) . . . 394
47 SGLang: Efficient Execution of Structured Language Model Programs (2023/2024) . . . 400
48 Fast Inference from Transformers via Speculative Decoding (2022/2023) . . . 407
49 Medusa: Simple LLM Inference Acceleration Framework with Multiple Decoding Heads (2024) . . . 415
50 EAGLE: Speculative Sampling Requires Rethinking Feature Uncertainty (2024) . . . 422
51 LLM.int8(): 8-bit Matrix Multiplication for Transformers at Scale (2022) . . . 428
52 SmoothQuant: Accurate and Efficient Post-Training Quantization for Large Language Models (2023) . . . 434
53 GPTQ: Accurate Post-Training Quantization for Generative Pre-trained Transformers (2023) . . . 441
54 AWQ: Activation-aware Weight Quantization for On-Device LLM Compression and Acceleration (2024) . . . 448
55 Efficient Streaming Language Models with Attention Sinks (2024) . . . 455
56 H2O: Heavy-Hitter Oracle for Efficient Generative Inference of Large Language Models (2023) . . . 461
57 An Image is Worth 1/2 Tokens After Layer 2: Plug-and-Play Acceleration for VLLM Inference (FastV) (2024) . . . 468
58 Generating Long Sequences with Sparse Transformers (2019) . . . 474
59 Reformer: The Efficient Transformer (2020) . . . 482
60 Big Bird: Transformers for Longer Sequences (2020) . . . 489
61 Quest: Query-Aware Sparsity for Efficient Long-Context LLM Inference (2024) . . . 498
62 MInference 1.0: Accelerating Pre-filling for Long-Context LLMs via Dynamic Sparse Attention (2024) 504
63 SeerAttention: Learning Intrinsic Sparse Attention in Your LLMs (2024) . . . 510
64 Native Sparse Attention: Hardware-Aligned and Natively Trainable Sparse Attention (2025) . . . 516
65 MoBA: Mixture of Block Attention for Long-Context LLMs (2025) . . . 523
66 The Sparse Frontier: Sparse Attention Trade-offs in Transformer LLMs (2025) . . . 529
67 Gated Attention for Large Language Models: Non-linearity, Sparsity, and Attention-Sink-Free (2025) . . . 535
68 DeepSeek-V3.2: Pushing the Frontier of Open Large Language Models (2025) . . . 541
69 GLM-5: from Vibe Coding to Agentic Engineering (2026) . . . 548
70 MiniCPM-SALA: Hybridizing Sparse and Linear Attention for Efficient Long-Context Modeling (2026) . . . 556
71 DeepSeek-V4: Towards Highly Efficient Million-Token Context Intelligence (2026) . . . 563
72 MiniMax Sparse Attention (2026) . . . 572
73 On the Design of Qwen3.8-Next Architecture: Evaluation, Efficiency, and Training Stability (2026) . . . 580

74 DeepSeek-V4.1-Flash: Pushing the Limits of KV Cache Compression (2026) 588

Học LLM, Attention, VLM và Inference Optimization từ đầu: lộ trình 74 ghi chú

Thư mục này chứa ghi chú học tập bằng tiếng Việt, đi từ Transformer gốc (2017) đến các kiến trúc linear attention và hybrid (GatedDeltaNet, Kimi Linear), VLM, model OCR và các kỹ thuật tăng tốc inference. Thứ tự được sắp cho **người mới**. Bộ tài liệu gồm một bài nhập môn và 73 ghi chú về các công trình nghiên cứu; số thứ tự các bài được giữ nguyên.

> Đây là các ghi chú diễn giải, không phải bản dịch. Các phép tính được ghi là "tự tính" là ví dụ minh họa, không phải kết quả trực tiếp của bài báo.

Giai đoạn 0: Chuẩn bị

#	File	Nội dung
00	`00_preface_toan_va_huong_dan.md`	Token, attention với shape cụ thể, causal mask, O(n^2), prefill và decode, công thức KV cache, đếm tham số và FLOPs, GPU (HBM/SRAM, roofline, vì sao decode batch-1 bị nghẽn băng thông bộ nhớ), định dạng số, linear attention/SSM, cơ bản về VLM, bảng thuật ngữ, câu tự kiểm tra. **Đọc trước tiên.**
01	`nen01_transformer_tu_dau_attention_tinh_tay.md`	Chương nền: Transformer từ đầu. attention là một phép tra cứu mềm mà ta có thể tính tay.
02	`nen02_khoi_transformer_residual_chuan_hoa_ffn.md`	Chương nền: Khối Transformer. nhiều đầu attention, đường residual, chuẩn hoá và FFN cùng nhau tạo một lớp.
03	`nen03_ma_hoa_vi_tri_rope_alibi.md`	Chương nền: Mã hoá vị trí. attention không biết thứ tự, nên ta phải đưa vị trí vào bằng cách quay (RoPE) hoặc trừ điểm theo khoảng cách (ALiBi).
04	`nen04_scaling_laws_tham_so_flops_bo_nho.md`	Chương nền: Scaling laws và tính toán. đếm tham số, FLOPs và bộ nhớ trước khi huấn luyện.
05	`nen05_kv_cache_va_chi_phi_suy_luan.md`	Chương nền: KV cache và chi phí suy luận. vì sao sinh từng token bị chặn bởi bộ nhớ.
06	`nen06_attention_tuyen_tinh_thua_va_ssm.md`	Chương nền: Attention tuyến tính, attention thưa và SSM. đổi chi phí bậc hai lấy trạng thái cố định.
07	`nen07_mixture_of_experts_va_dinh_tuyen.md`	Chương nền: Mixture of Experts và định tuyến. nhiều chuyên gia, nhưng mỗi token chỉ gọi vài người.
08	`nen08_luong_tu_hoa_bo_nho_gpu_flashattention.md`	Chương nền: Lượng tử hoá và bộ nhớ GPU. bớt bit cho trọng số, bớt chuyến đi cho dữ liệu.
09	`nen09_mo_hinh_thi_giac_ngon_ngu.md`	Chương nền: Mô hình thị giác-ngôn ngữ. biến ảnh thành token để LLM đọc.

Giai đoạn 1: Nền tảng LLM

#	File	Bài	Ý chính
01	`01_2017_attention_is_all_you_need.md`	Attention Is All You Need	Kiến trúc Transformer gốc.
02	`02_2020_gpt3_few_shot_learners.md`	GPT-3	Mở rộng quy mô; in-context learning.
03	`03_2020_scaling_laws_kaplan.md`	Scaling Laws (Kaplan)	Loss giảm theo luật lũy thừa của N, D và compute.
04	`04_2022_chinchilla.md`	Chinchilla	Phân bổ compute tối ưu: khoảng 20 token cho mỗi tham số.
05	`05_2023_llama.md`	LLaMA	Công thức "chuẩn": Pre-RMSNorm, SwiGLU, RoPE.
06	`06_2023_mistral_7b.md`	Mistral 7B	Sliding-window attention, rolling buffer cache, GQA.
07	`07_2022_instructgpt_rlhf.md`	InstructGPT	SFT → reward model → PPO (RLHF).
08	`08_2021_lora.md`	LoRA	Fine-tune bằng cập nhật hạng thấp.

Giai đoạn 2: Các thành phần của Transformer hiện đại

#	File	Bài	Ý chính
09	`09_2020_pre_ln_transformer.md`	On Layer Normalization in the Transformer	Pre-LN và Post-LN; vì sao Pre-LN train ổn định.
10	`10_2019_rmsnorm.md`	RMSNorm	Chuẩn hóa chỉ dùng RMS.
11	`11_2020_glu_variants_swiglu.md`	GLU Variants	SwiGLU trong FFN.
12	`12_2021_rope_roformer.md`	RoPE	Vị trí mã hóa bằng phép quay; tính chất vị trí tương đối.
13	`13_2021_alibi.md`	ALiBi	Bias tuyến tính theo khoảng cách; ngoại suy độ dài.

Giai đoạn 3: Các loại attention (từ MHA đến linear, SSM và hybrid)

#	File	Bài	Ý chính
14	`14_2019_multi_query_attention.md`	MQA	Một K/V cho mọi head; KV cache nhỏ.
15	`15_2023_gqa.md`	GQA	Nhóm head dùng chung K/V; uptrain từ checkpoint MHA.
16	`16_2024_mla_deepseek_v2.md`	MLA (DeepSeek-V2)	Nén KV xuống latent hạng thấp; decoupled RoPE.
17	`17_2020_longformer.md`	Longformer	Attention cửa sổ trượt kết hợp global.
18	`18_2020_linear_transformers_rnns.md`	Transformers are RNNs	Linear attention = RNN với trạng thái ma trận.
19	`19_2023_retnet.md`	RetNet	Retention: song song, hồi quy, theo chunk.
20	`20_2024_gated_linear_attention.md`	Gated Linear Attention	Cổng phụ thuộc dữ liệu; train theo chunk hiệu quả trên phần cứng.
21	`21_2023_mamba.md`	Mamba	Selective SSM.

#	File	Bài	Ý chính
22	`22_2024_mamba2_ssd.md`	Mamba-2 / SSD	Đối ngẫu giữa SSM và attention.
23	`23_2024_deltanet_parallel.md`	DeltaNet (song song hóa delta rule)	Cập nhật trạng thái kiểu delta rule; biểu diễn WY.
24	`24_2025_gated_deltanet.md`	**Gated DeltaNet**	Cổng kết hợp với delta rule trong attention tuyến tính.
25	`25_2024_jamba.md`	Jamba	Hybrid Transformer-Mamba (+MoE).
26	`26_2025_kimi_linear.md`	Kimi Linear	Hybrid linear attention thế hệ mới.

Giai đoạn 4: Mixture of Experts

#	File	Bài	Ý chính
27	`27_2021_switch_transformer_moe.md`	Switch Transformer	Routing top-1, cân bằng tải.
28	`28_2024_mixtral.md`	Mixtral	Sparse MoE top-2.

Giai đoạn 5: Vision encoder và VLM

#	File	Bài	Ý chính
29	`29_2020_vit.md`	ViT	Ảnh chia thành patch rồi đưa vào Transformer.
30	`30_2021_clip.md`	CLIP	Contrastive ảnh-chữ; zero-shot.
31	`31_2023_siglip.md`	SigLIP	Loss sigmoid thay cho softmax.
32	`32_2022_flamingo.md`	Flamingo	Perceiver resampler, gated cross-attention.
33	`33_2023_blip2.md`	BLIP-2	Q-Former nối encoder đóng băng với LLM.
34	`34_2023_llava.md`	LLaVA	Projector đơn giản, visual instruction tuning.
35	`35_2024_llava_15.md`	LLaVA-1.5	MLP projector, công thức baseline mạnh.
36	`36_2024_internvl_15.md`	InternVL 1.5	Chia tile động độ phân giải cao, pixel shuffle.
37	`37_2024_qwen2_vl.md`	Qwen2-VL	Độ phân giải động, M-RoPE.
38	`38_2025_smolvlm.md`	SmolVLM	Thiết kế VLM nhỏ: cân bằng encoder/LM, ngân sách token.

Giai đoạn 6: Document AI và OCR

#	File	Bài	Ý chính
39	`39_2022_donut.md`	Donut	Hiểu tài liệu không cần OCR, end-to-end.
40	`40_2023_nougat.md`	Nougat	PDF học thuật thành markup; xử lý lặp lại và hallucination.
41	`41_2024_got_ocr2.md`	GOT-OCR2.0	“OCR-2.0” thống nhất end-to-end.
42	`42_2025_olmocr.md`	olmOCR	Pipeline dữ liệu, document anchoring, benchmark.

Giai đoạn 7: Inference optimization

#	File	Bài	Ý chính
44	`44_2022_flashattention.md`	FlashAttention	Tiling, online softmax, phân tích IO.
45	`45_2023_flashattention2.md`	FlashAttention-2	Song song hóa và chia việc tốt hơn.
46	`46_2023_pagedattention_vllm.md`	PagedAttention (vLLM)	KV cache chia trang như bộ nhớ ảo.
47	`47_2024_sglang_radixattention.md`	SGLang	RadixAttention: dùng lại prefix.
48	`48_2023_speculative_decoding.md`	Speculative Decoding	Draft model nhỏ đoán trước, model lớn kiểm tra; phân phối output không đổi.
49	`49_2024_medusa.md`	Medusa	Nhiều decoding head, tree attention.
50	`50_2024_eagle.md`	EAGLE	Speculative ở mức feature.
51	`51_2022_llm_int8.md`	LLM.int8()	Outlier feature; nhân ma trận 8-bit.
52	`52_2023_smoothquant.md`	SmoothQuant	Chuyển độ khó quantization từ activation sang weight.
53	`53_2023_gptq.md`	GPTQ	Quantization dựa trên OBS (cùng họ với SparseGPT).
54	`54_2024_awq.md`	AWQ	Quantization weight có tính đến activation.
55	`55_2024_streamingllm_attention_sinks.md`	StreamingLLM	Attention sink; cache cửa sổ cộng sink.
56	`56_2023_h2o_kv_cache.md`	H2O	Loại bớt KV cache theo “heavy hitter”.
57	`57_2024_fastv_visual_token_pruning.md`	FastV	Cắt visual token sau layer 2.

Giai đoạn 8: Sparse attention

#	File	Bài	Ý chính
58	`58_2019_sparse_transformer.md`	Sparse Transformer	Attention phân rã (strided, fixed), $O(n\sqrt{n})$. Mở đầu giai đoạn, có bảng phân loại sparse attention.
59	`59_2020_reformer.md`	Reformer	LSH attention: gom query/key giống nhau vào cùng bucket.
60	`60_2020_bigbird.md`	BigBird	Random + window + global; lý thuyết xấp xỉ phổ quát.
61	`61_2024_quest.md`	Quest	Chọn trang KV theo query khi decode (không loại bỏ KV).
62	`62_2024_minference.md`	MInference	Pattern thưa động cho prefill 1M token.
63	`63_2024_seerattention.md`	SeerAttention	Gate học được dự đoán block thưa, distill từ full attention.
64	`64_2025_nsa_native_sparse_attention.md`	**NSA (DeepSeek)**	Nén + chọn block + cửa sổ trượt; train được end-to-end, kernel theo phần cứng.
65	`65_2025_moba.md`	**MoBA (Kimi)**	Mixture of Block Attention: gate chọn top-k block.
66	`66_2025_sparse_frontier.md`	The Sparse Frontier	Nghiên cứu thực nghiệm lớn về trade-off sparse attention không cần train.

#	File	Bài	Ý chính
67	`67_2025_gated_attention_qwen.md`	Gated Attention (Qwen)	Gate sau SDPA: phi tuyến, thưa theo query, hết attention sink.
68	`68_2025_deepseek_v32_dsa.md`	**DeepSeek-V3.2 (DSA)**	Lightning indexer + chọn top-k token; dense warm-up rồi sparse training.
69	`69_2026_glm5_dsa.md`	GLM-5 (DSA)	GLM-5 chuyển sang DSA; kèm phần ngoài paper về GLM-5.3 và GLM-5.3-Flash.
70	`70_2026_minicpm_sala.md`	MiniCPM-SALA	Kết hợp sparse attention và linear attention.
71	`71_2026_deepseek_v4_csa_hca.md`	**DeepSeek-V4**	Compressed Sparse Attention + Heavily Compressed Attention cho 1M token.
72	`72_2026_minimax_sparse_attention.md`	MiniMax Sparse Attention (MSA)	Sparse theo block trên GQA: indexer nhẹ chọn top-k block riêng cho từng nhóm GQA, học bằng KL; kernel tối ưu cho top-k.
73	`73_2026_qwen38_next_qsa.md`	**Qwen3.8-Next (QSA)**	Qwen Sparse Attention: indexer mức block, hybrid với Gated DeltaNet.
74	`74_2026_deepseek_v41_flash.md`	DeepSeek-V4.1-Flash	Đẩy giới hạn nén KV cache; kèm phần ngoài paper về bản cập nhật Flash-1017.

Cách đọc gợi ý

- **Người mới hoàn toàn:** 00 → 01 → 05 → 09–12 → 14, 15 → 29, 34, 35 → 44 → 48.
- **Tìm hiểu decoder hybrid:** 00 → 01 → 14, 15 → 18 → 20 → 21 → 23 → **24** → 25 → 26.
- **Hiểu phần vision và OCR:** 29 → 31 → 35 → 36 → 37 → 38 → 39–42 → 57.
- **Tối ưu suy luận:** 00 (roofline, KV cache) → 44 → 48–50 → 51–54 → 55–57.
- **Sparse attention (từ cổ điển đến 2026):** 17 → 58 → 60 → 55, 56 → 61 → 62 → 64 → 65 → 68 → 71 → 73. MLA (16) nên đọc trước 68, 71, 74; hybrid sparse + linear: 24 → 26 → 70 → 73.

Liên kết với các chủ đề khác

- Các ghi chú về pruning, distillation và interpretability bổ sung góc nhìn về nén và phân tích mô hình.
- Bài 07 giới thiệu RLHF trong huấn luyện mô hình ngôn ngữ.

Nền tảng 01

Transformer từ đầu: attention là một phép tra cứu mềm mà ta có thể tính tay

Loại: chương nền tảng (không phải ghi chú paper)

Đọc trước: vector và ma trận, tích vô hướng ở math nền tảng 02; softmax và xác suất ở math nền tảng 08; mạng nơ-ron nhiều lớp ở ML cơ bản nền tảng 05

Dùng ở: llm 01 (Attention Is All You Need), llm 02 (GPT-3, mô hình chỉ có decoder), llm 14 (multi-query attention), llm 15 (GQA), và mọi bài attention về sau (sparse, linear, KV cache)

> Chương này giải thích từ đầu, không phải bản dịch một paper. Các phép tính ghi "tự tính" và các hình minh hoạ đều đã kiểm bằng numpy.

1. Vì sao cần chương này

Attention là phép tính cho mỗi vị trí trong câu quyết định nên "nhìn" vào những vị trí nào, rồi lấy trung bình có trọng số thông tin ở đó. Gần như mọi mô hình ngôn ngữ lớn trong cuốn sách đều xây trên phép tính này, nên nếu nó còn mơ hồ thì các bài sau (nhóm truy vấn, cache, attention thưa, attention tuyến tính) chỉ còn là chuỗi ký hiệu.

Chương đi chậm theo thứ tự: câu chữ biến thành số như thế nào (token và embedding), rồi attention một đầu tính tay trên ví dụ ba token với vector hai chiều, rồi vì sao gọi nó là "tra cứu mềm", rồi mặt nạ nhân quả, rồi vai trò của nhiệt độ và hệ số chia $\sqrt{d}$. Cuối chương có bài tập tự tính. Chương kế tiếp (nen02 của cuốn này) đặt attention vào một khối Transformer hoàn chỉnh; chương nen03 giải thích cách mô hình biết thứ tự các từ.

2. Nhắc nền: token, embedding và tích vô hướng

2.1 Token: mảnh nhỏ của văn bản

Mạng nơ-ron chỉ nhận số. Bước đầu tiên là cắt văn bản thành các **token**, mỗi token là một mảnh (một từ, một phần của từ, hoặc một ký tự đặc biệt) lấy từ một danh sách cố định gọi là **từ vựng**, có kích thước V. Mỗi token có một số nguyên gọi là chỉ số (id) trong danh sách đó. Cách cắt cụ thể (BPE và các biến thể) nằm ngoài chương này; điều cần nhớ chỉ là: câu có T token được biểu diễn bằng dãy T số nguyên.

Ví dụ minh hoạ (giả định, không phải từ vựng thật): câu "mèo ăn cá" được cắt thành ba token với id lần lượt là $17, 4, 9$. Ở các phần sau ta gọi ba token ấy là token 1, token 2, token 3.

2.2 Embedding: tra bảng để đổi số nguyên thành vector

Embedding là một bảng có V hàng, mỗi hàng là một vector d chiều; token có id i nhận về hàng thứ i. Bảng này là tham số học được của mô hình, nên các token có vai trò giống nhau sẽ dần có vector gần nhau. Đổi id thành vector không có phép tính nào ngoài việc đọc hàng.

Để tính tay được, ta lấy $d = 2$ và đặt ba hàng (giả định, chọn cho dễ tính):

- "mèo" $\to (1, 0)$
- "ăn" $\to (0, 1)$
- "cá" $\to (1, 1)$

Xếp ba hàng thành ma trận X cỡ 3×2, mỗi hàng là một token:

$$X = \begin{pmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \end{pmatrix}.$$

Trong mô hình thật d cỡ hàng nghìn, nhưng mọi phép tính dưới đây giữ nguyên hình dạng.

2.3 Tích vô hướng đo độ giống nhau

Tích vô hướng của hai vector cùng chiều là tổng các tích từng cặp thành phần: với $u = (u_1, u_2)$ và $v = (v_1, v_2)$ thì $u \cdot v = u_1 v_1 + u_2 v_2$. Nếu hai vector cùng hướng thì tích này dương và lớn; nếu vuông góc thì bằng 0; nếu ngược hướng thì âm. Ví dụ (tự tính): $(1, 2) \cdot (1, 1) = 1 \cdot 1 + 2 \cdot 1 = 3$ còn $(1, 2) \cdot (2, -1) = 0$.

Attention dùng đúng tính chất này: tích vô hướng giữa một "câu hỏi" và một "nhãn" cho điểm số về mức độ phù hợp.

2.4 Softmax đổi điểm số thành trọng số

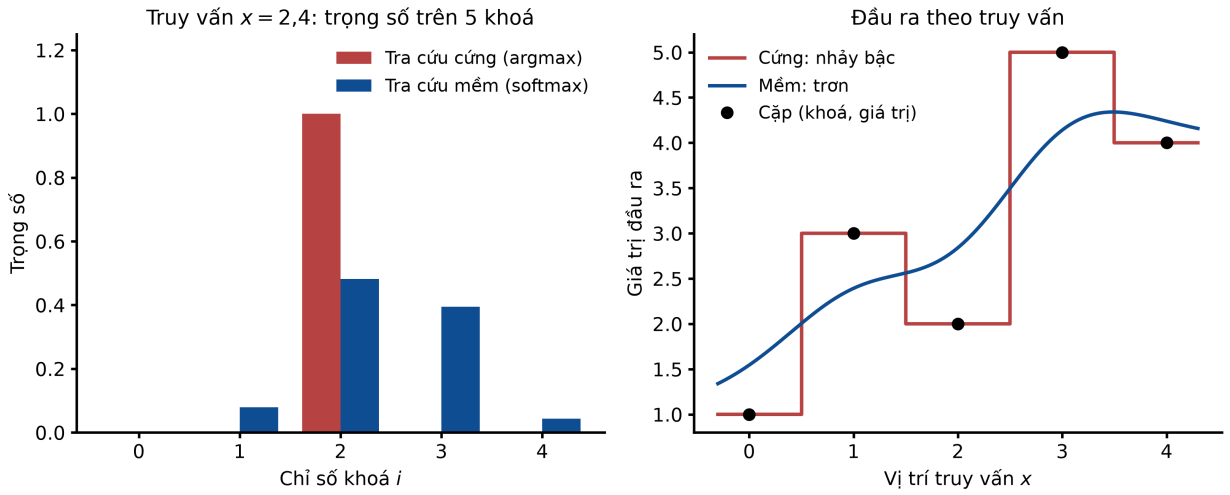
Cho điểm số $s_1, \ldots, s_n$, softmax cho ra $w_j = \dfrac{e^{s_j}}{\sum_k e^{s_k}}$. Mọi w_j dương và cộng lại bằng 1, nên coi được như tỉ lệ chia sự chú ý. Điểm càng lớn thì trọng số càng lớn, nhưng không bao giờ đúng bằng 0 hoặc 1 trừ khi có điểm bằng $\pm\infty$.

Ví dụ (tự tính): điểm $(0, \ln 2, \ln 4)$ cho $e^s = (1, 2, 4)$, tổng 7, nên trọng số $(\frac{1}{7}, \frac{2}{7}, \frac{4}{7}) \approx (0{,}143; 0{,}286; 0{,}571)$.

3. Trực giác bằng hình: tra cứu cứng và tra cứu mềm

Hãy nghĩ đến từ điển Python. Ta có các cặp (khoá, giá trị); truy vấn một khoá thì từ điển trả về đúng một giá trị, hoặc báo lỗi nếu khoá không có. Đó là **tra cứu cứng**: kết quả nhảy bậc, một khoá thắng hoàn toàn, các khoá khác không đóng góp gì.

Tra cứu mềm làm khác: truy vấn được so với tất cả khoá, mỗi khoá nhận một trọng số, và kết quả là **trung bình có trọng số** của tất cả giá trị. Khoá giống truy vấn nhất đóng góp nhiều nhất, nhưng khoá gần giống cũng đóng góp một phần. Hai điều xảy ra nhờ vậy: kết quả thay đổi trơn khi truy vấn thay đổi nhẹ (nên tính được đạo hàm và học bằng gradient), và truy vấn không khớp hoàn toàn với khoá nào vẫn nhận được một câu trả lời hợp lý là pha trộn của các khoá gần nó.



Hình 1: Tra cứu cứng (một-nóng) so với tra cứu mềm (softmax) trên 5 khoá, và đầu ra theo vị trí truy vấn (minh hoạ, tự tính)

Cách đọc hình: năm khoá nằm ở vị trí $0, 1, 2, 3, 4$ với các giá trị lần lượt là $1, 3, 2, 5, 4$ (số giả định để vẽ). Điểm số của khoá i với truy vấn x được đặt là $-(x - i)^2$, tức khoá càng gần x điểm càng cao; đây là một cách đo độ giống đơn giản để minh hoạ, không phải điểm số thật của Transformer. Bên trái, truy vấn $x = 2{,}4$: tra cứu cứng chỉ chọn khoá 2 (trọng số 1), còn tra cứu mềm chia trọng số khoảng $0{,}00; 0{,}08; 0{,}48; 0{,}39; 0{,}04$ và cho đầu ra khoảng $3{,}35$ (tự tính). Bên phải, đường đỏ nhảy bậc khi truy vấn vượt qua điểm giữa hai khoá, đường xanh trơn nên gradient chảy qua được; đường xanh cũng không đi chính xác qua các chấm đen vì khoá lân cận kéo nó đi.

4. Attention một đầu: từng bước

4.1 Ba vai trò: truy vấn, khoá, giá trị

Mỗi token đóng đồng thời ba vai trò, và mỗi vai trò dùng một ma trận trọng số riêng (tham số học được):

- **Truy vấn** q: "tôi đang cần tìm thông tin gì?"
- **Khoá** k: "tôi chứa loại thông tin gì, để người khác đối chiếu?"
- **Giá trị** v: "nếu ai đó chú ý đến tôi, đây là nội dung tôi trao đi."

Với ma trận đầu vào X cỡ $T \times d$, ta tính

$$Q = XW_Q, \qquad K = XW_K, \qquad V = XW_V.$$

Mỗi hàng của Q, K, V tương ứng một token. Việc tách riêng khoá và giá trị là chi tiết đáng nhớ: thông tin dùng để **tìm** (khoá) không bắt buộc giống thông tin dùng để **lấy** (giá trị).

4.2 Công thức

Điểm số, trọng số và đầu ra:

$$S = \frac{QK^\top}{\sqrt{d_k}}, \qquad A = \text{softmax}(S) \text{ (theo từng hàng)}, \qquad O = AV.$$

Ở đây d_k là chiều của vector truy vấn và khoá. Phần tử S_{ij} là điểm số mà token i dành cho token j. Hàng i của A là các trọng số mà token i chia cho mọi token, cộng lại bằng 1. Hàng i của O là trung bình có trọng số các hàng của V với trọng số ấy.

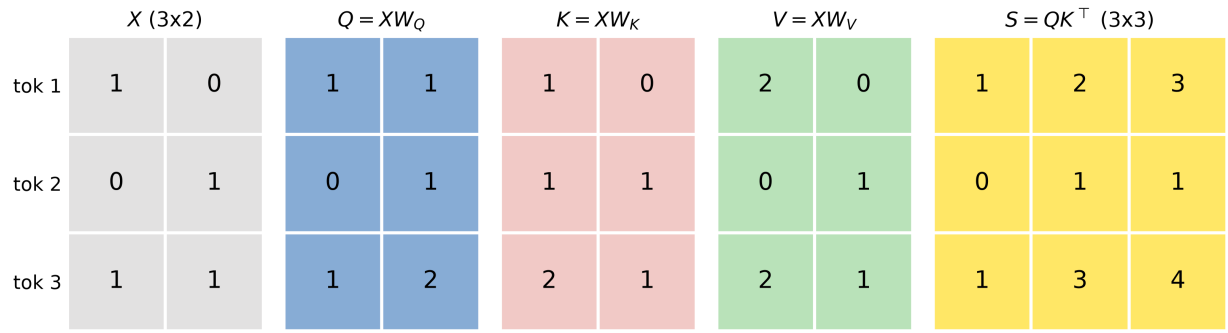
4.3 Ví dụ tay: ba token "mèo ăn cá", $d = 2$

Dùng X ở mục 2.2 và ba ma trận trọng số được chọn cho gọn (giả định):

$$W_Q = \begin{pmatrix} 1 & 1 \\ 0 & 1 \end{pmatrix}, \quad W_K = \begin{pmatrix} 1 & 0 \\ 1 & 1 \end{pmatrix}, \quad W_V = \begin{pmatrix} 2 & 0 \\ 0 & 1 \end{pmatrix}.$$

Bước 1: tạo Q, K, V. Nhân từng hàng của X với ma trận (tự tính). Ví dụ hàng 3 của X là $(1, 1)$ nên $q_3 = (1 \cdot 1 + 1 \cdot 0,\ 1 \cdot 1 + 1 \cdot 1) = (1, 2)$. Kết quả:

$$Q = \begin{pmatrix} 1 & 1 \\ 0 & 1 \\ 1 & 2 \end{pmatrix}, \quad K = \begin{pmatrix} 1 & 0 \\ 1 & 1 \\ 2 & 1 \end{pmatrix}, \quad V = \begin{pmatrix} 2 & 0 \\ 0 & 1 \\ 2 & 1 \end{pmatrix}.$$



Hình 2: Các ma trận X, Q, K, V và điểm số S của ví dụ ba token (minh hoạ, tự tính)

Cách đọc hình: mỗi ô là một số, mỗi hàng là một token; cùng một hàng ở X, Q, K, V là cùng một token nhìn qua ba "ống kính" khác nhau. Ma trận vàng ngoài cùng bên phải là S trước khi chia $\sqrt{d_k}$, chính là bước tiếp theo.

Bước 2: điểm số thô $QK^\top$. Phần tử (i, j) là $q_i \cdot k_j$ (tự tính):

$$QK^\top = \begin{pmatrix} 1 & 2 & 3 \\ 0 & 1 & 1 \\ 1 & 3 & 4 \end{pmatrix}.$$

Ví dụ phần tử hàng 3, cột 2 là $q_3 \cdot k_2 = (1, 2) \cdot (1, 1) = 3$. Hàng 3 nói: token "cá" thấy token 3 phù hợp nhất (điểm 4), rồi đến token 2 (điểm 3), cuối cùng là token 1 (điểm 1).

Bước 3: chia $\sqrt{d_k} = \sqrt{2} \approx 1{,}414$. Hàng 3 thành $(0{,}707;\ 2{,}121;\ 2{,}828)$ (tự tính). Lý do cho phép chia này sẽ nói ở mục 7.1.

Bước 4: softmax từng hàng. Làm kỹ cho hàng 3: $e^{0{,}707} = 2{,}028$; $e^{2{,}121} = 8{,}342$; $e^{2{,}828} = 16{,}919$; tổng $27{,}289$. Các trọng số là $0{,}074;\ 0{,}306;\ 0{,}620$ (tự tính). Làm tương tự cho hai hàng còn lại:

$$A = \begin{pmatrix} 0{,}140 & 0{,}284 & 0{,}576 \\ 0{,}198 & 0{,}401 & 0{,}401 \\ 0{,}074 & 0{,}306 & 0{,}620 \end{pmatrix} \quad \text{(tự tính).}$$

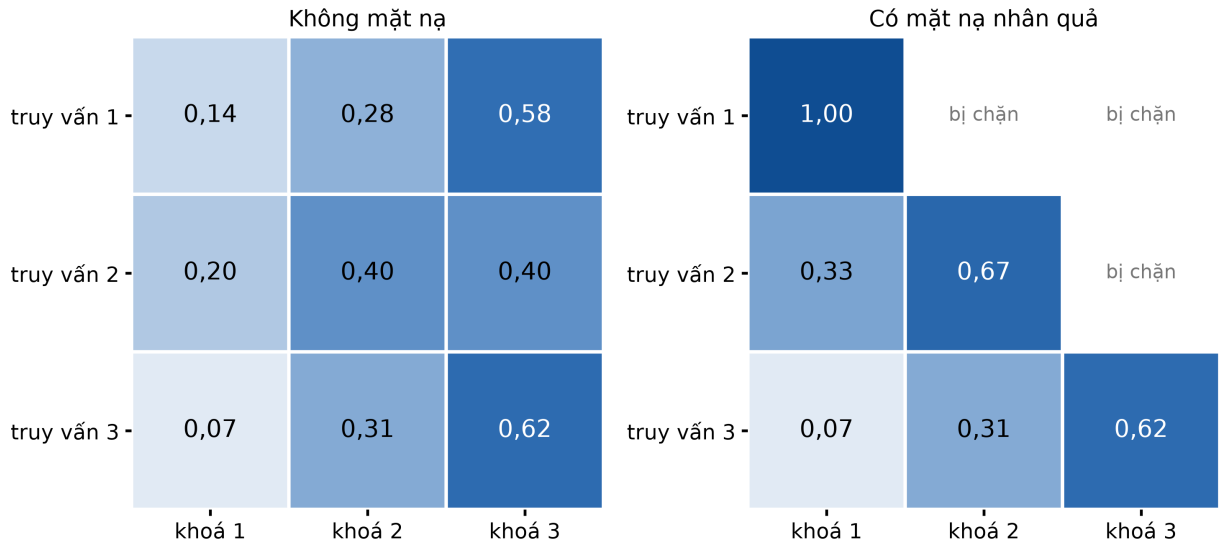
Bước 5: trung bình có trọng số các giá trị. Hàng 3 của O là

$$0{,}074 \cdot (2, 0) + 0{,}306 \cdot (0, 1) + 0{,}620 \cdot (2, 1) \approx (1{,}389;\ 0{,}926) \quad \text{(tự tính).}$$

Hai hàng còn lại cho $o_1 \approx (1{,}432;\ 0{,}860)$ và $o_2 \approx (1{,}198;\ 0{,}802)$ (tự tính). Ba token vào, ba vector ra, cùng cỡ 2 chiều.

Cách đọc hình: hàng là token đang đặt câu hỏi, cột là token bị hỏi; mỗi hàng cộng lại bằng 1. Bảng bên trái chính là ma trận A vừa tính: token 3 dồn $0{,}62$ sự chú ý cho chính nó. Bảng bên phải là phiên bản có mặt nạ, giải thích ngay dưới đây.

5. Mặt nạ nhân quả: không được nhìn trộm tương lai



Hình 3: Ma trận trọng số attention của ví dụ ba token, không mặt nạ (trái) và có mặt nạ nhân quả (phải) (minh hoạ, tự tính)

5.1 Vì sao cần

Mô hình ngôn ngữ được huấn luyện để đoán token kế tiếp. Nếu token 2 được nhìn thấy token 3 (chính là đáp án khi đoán sau token 2) thì bài toán trở nên tầm thường và mô hình học được cách chép đáp án, không học được cách dự đoán. Vì vậy ta ép token i chỉ được chú ý các token $j \le i$.

5.2 Cách làm

Trước softmax, đặt điểm số của các ô $j > i$ bằng $-\infty$ (trong phần mềm là một số âm rất lớn). Vì $e^{-\infty} = 0$, các ô đó nhận trọng số 0 sau softmax, còn các ô còn lại tự chuẩn hoá lại để tổng hàng vẫn bằng 1. Đây là **mặt nạ nhân quả** (causal mask): ma trận điểm số bị cắt thành hình tam giác dưới.

Hàng 1: chỉ còn token 1 nên trọng số $(1, 0, 0)$ và $o_1 = v_1 = (2, 0)$. Hàng 2: còn điểm số $(0;\ 0{,}707)$, $e^0 = 1$ và $e^{0{,}707} = 2{,}028$, tổng $3{,}028$, trọng số $(0{,}330;\ 0{,}670)$ và $o_2 = 0{,}330 \cdot (2, 0) + 0{,}670 \cdot (0, 1) \approx (0{,}660;\ 0{,}670)$ (tự tính). Hàng 3: không có ô nào bị chặn nên giữ nguyên.

Ghi nhận quan trọng: ở bước đoán token kế tiếp, mọi vị trí của câu được tính **song song** trong một lần nhân ma trận, nhưng nhờ mặt nạ mỗi vị trí chỉ dùng thông tin trước nó, giống như câu được đọc từng token. Khi sinh chữ thật sự, token mới được thêm vào cuối và hàng mới của K, V được thêm vào bộ nhớ đệm; các bài về KV cache (llm 56 và các bài inference) dựa trên chính cấu trúc tam giác này.

Một số mô hình khác (như bộ mã hoá kiểu BERT, hoặc phần bộ mã hoá của Transformer dịch máy) không dùng mặt nạ, để mỗi token nhìn được cả hai phía. Mô hình chỉ có bộ giải mã (decoder-only), loại phổ biến cho LLM, luôn dùng mặt nạ nhân quả.

6. Định nghĩa chính thức

Cho chuỗi T token với ma trận embedding $X \in \mathbb{R}^{T \times d}$. Với các tham số $W_Q, W_K \in \mathbb{R}^{d \times d_k}$ và $W_V \in \mathbb{R}^{d \times d_v}$, attention một đầu có mặt nạ là

$$O = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}} + M\right) V, \qquad M_{ij} = \begin{cases} 0 & j \le i \\ -\infty & j > i \end{cases}$$

với $Q = XW_Q$, $K = XW_K$, $V = XW_V$. Số tham số của một đầu là $2d\,d_k + d\,d_v$; nếu $d_k = d_v = d$ thì là $3d^2$. Chi phí tính và bộ nhớ của ma trận A tăng theo T^2: với $T = 4096$ và mỗi số lưu bằng 2 byte, một ma trận A chiếm $4096^2 \cdot 2 \approx 33{,}6$ MB (tự tính), và có một ma trận như vậy cho mỗi đầu ở mỗi lớp. Chính chi phí T^2 này thúc đẩy nhiều bài về attention thưa và tuyến tính ở các phần sau của sách.

7. Biến thể và chi tiết hay bị bỏ qua

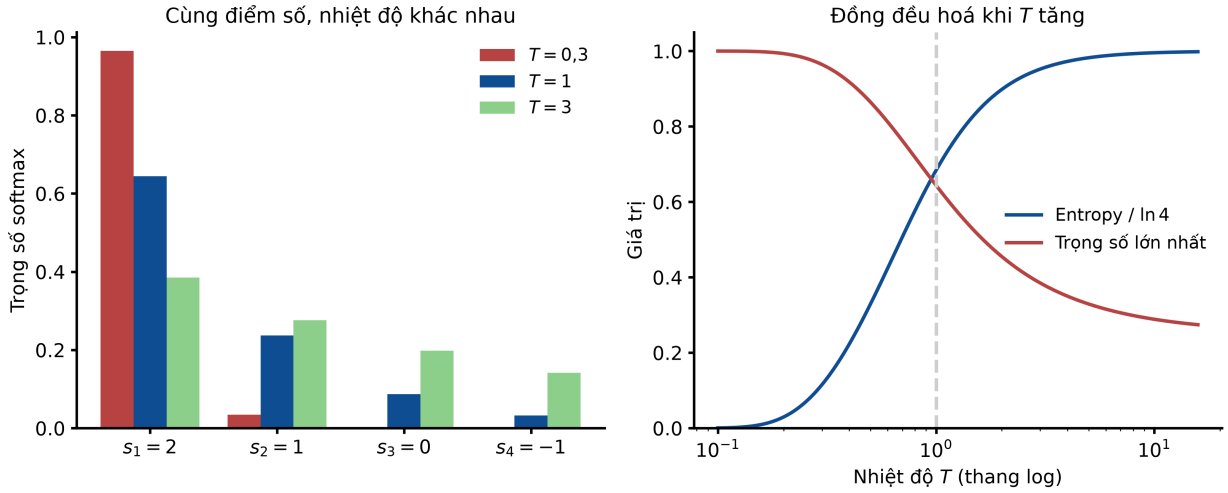
7.1 Vì sao chia $\sqrt{d_k}$

Khi hai vector truy vấn và khoá có các thành phần độc lập, trung bình 0 và phương sai 1, tích vô hướng $q \cdot k$ là tổng của d_k số hạng, nên độ lệch chuẩn của nó cỡ $\sqrt{d_k}$. Mô phỏng (giả định: thành phần Gauss chuẩn, $d_k = 64$, 100000 cặp, seed 0) cho độ lệch chuẩn $\approx 8{,}0$ trước khi chia, và $\approx 1{,}0$ sau khi chia cho $\sqrt{64} = 8$ (tự tính). Nếu không chia, điểm số có độ lớn cỡ 8 khiến softmax rất nhọn (gần một-nóng), gradient qua softmax nhỏ và việc học chậm. Đây là lý do thực dụng, và đây là suy luận của chương dựa trên phép đo mô phỏng; paper gốc nêu lập luận tương tự ở chú thích 4 (theo ghi chú llm 01).

7.2 Nhiệt độ softmax

Chia điểm số cho một số $T > 0$ trước softmax gọi là đổi **nhiệt độ**: $w_j \propto e^{s_j/T}$. Chia cho $\sqrt{d_k}$ thực chất là chọn $T = \sqrt{d_k}$ cố định. Với điểm số $(2, 1, 0, -1)$ (giả định) ta có (tự tính):

- $T = 0{,}3$: trọng số $(0{,}964;\ 0{,}034;\ 0{,}001;\ 0{,}000)$, rất nhọn, gần tra cứu cứng.
- $T = 1$: $(0{,}644;\ 0{,}237;\ 0{,}087;\ 0{,}032)$.
- $T = 3$: $(0{,}385;\ 0{,}276;\ 0{,}198;\ 0{,}142)$, đã khá đều.



Hình 4: Ảnh hưởng của nhiệt độ softmax lên trọng số và độ đồng đều (minh hoạ, tự tính)

Cách đọc hình: bên trái là ba bộ thanh cho ba nhiệt độ trên cùng bốn điểm số; thanh đỏ (T nhỏ) dồn gần hết vào điểm cao nhất, thanh xanh lá (T lớn) trải đều. Bên phải theo dõi liên tục: khi T nhỏ dần, trọng số lớn nhất tiến về 1 và entropy (chia $\ln 4$ để thang là 0 đến 1) về 0 (tra cứu cứng); khi T lớn dần, trọng số lớn nhất tiến về $1/4$ và entropy về giá trị cực đại (trung bình đều). Tra cứu cứng và tra cứu mềm vì vậy chỉ là hai đầu của cùng một núm vặn.

7.3 Tính hoán vị và vì sao attention “mù thứ tự”

Quan sát điều sau trên ví dụ không mặt nạ (tự tính): đổi chỗ token 1 và token 2 trong X thì vector ra của token 3 **không đổi**, vẫn là $(1{,}389;\ 0{,}926)$; hai vector ra còn lại chỉ đổi chỗ cho nhau. Lý do: o_3 là tổng có trọng số theo token, tổng không phụ thuộc thứ tự cộng. Tính chất này gọi là hoán vị-đồng biến: thay đổi thứ tự đầu vào chỉ làm đổi thứ tự đầu ra tương ứng. Hệ quả là “chó cắn người” và “người cắn chó” trông giống hệt nhau

với attention thuần. Mặt nạ nhân quả phá tính đối xứng này một phần (vì mỗi token thấy một tập khác nhau), nhưng không đủ để mã hoá khoảng cách. Vì vậy cần thêm thông tin vị trí, nội dung của chương nen03.

7.4 Thêm các phần quanh attention

Attention chỉ là một phần của khối Transformer. Một khối thật còn có nhiều đầu chạy song song (multi-head), kết nối residual, chuẩn hoá và mạng FFN; chương nen02 trình bày các phần đó. Ở ví dụ trên ta cũng bỏ qua lớp chiếu đầu ra W_O và các hằng số lệch (bias), không ảnh hưởng đến ý chính.

7.5 Hình dạng và đếm tham số

Khi tự cài đặt hoặc đọc code, điều đầu tiên cần kiểm là hình dạng. Với một câu T token: X cỡ $T \times d$; W_Q, W_K cỡ $d \times d_k$; Q, K cỡ $T \times d_k$; $QK^\top$ cỡ $T \times T$; W_V cỡ $d \times d_v$; V cỡ $T \times d_v$; $O = AV$ cỡ $T \times d_v$. Ở ví dụ ba token: $T = 3$, $d = d_k = d_v = 2$, nên mọi thứ chỉ gồm các khối 3×2 và một khối 3×3.

Đếm tham số của một đầu với giả định $d = 4096$ và $d_k = d_v = 4096$: ba ma trận cỡ 4096×4096 có $3 \cdot 4096^2 = 50\,331\,648$ tham số (tự tính). Trong mô hình thật ta không dùng một đầu khổng lồ như vậy mà chia d thành nhiều đầu nhỏ, nội dung của chương nen02.

Chú ý rằng số tham số của attention **không phụ thuộc** T: cùng một bộ W_Q, W_K, W_V áp dụng cho mọi vị trí và mọi độ dài câu. Chỉ có chi phí tính toán và bộ nhớ của ma trận A mới phụ thuộc T. Nhờ đó mô hình huấn luyện ở một độ dài vẫn chạy được ở độ dài khác về mặt hình dạng, dù chất lượng không được đảm bảo; vấn đề này là chủ đề của chương nen03.

8. Nối sang các bài paper của cuốn

- **llm 01 (Attention Is All You Need):** paper định nghĩa chính xác công thức ở mục 4.2 và 6, kể cả phép chia $\sqrt{d_k}$ và mặt nạ ở bộ giải mã. Sau chương này, ghi chú đó đọc như một chuỗi các bước đã tính tay.
- **llm 02 (GPT-3):** mô hình chỉ có bộ giải mã, mọi lớp attention đều có mặt nạ nhân quả như mục 5. Cách dùng "học vài ví dụ" (few-shot) chỉ là đặt thêm token vào ngữ cảnh, tức thêm các hàng khoá và giá trị mà những token sau có thể tra cứu (đây là cách hiểu của chương, xem ghi chú llm 02 cho mô tả của paper).
- **llm 14 và 15 (multi-query và GQA):** hai bài đổi số bộ khoá và giá trị so với số bộ truy vấn, để bộ nhớ đệm K, V nhỏ hơn. Công thức không đổi, chỉ khác số hàng của K, V dùng chung giữa các đầu.
- **llm 44 đến 56 (FlashAttention, PagedAttention, KV cache):** các bài về hiệu năng đều xoay quanh việc tránh lưu trọn ma trận $T \times T$ ở mục 6, hoặc tái dùng K, V của các token trước nhờ cấu trúc tam giác của mục 5.
- **llm 17 và 58 đến 73 (attention thưa):** thay vì mọi ô của tam giác dưới đều được tính, các bài này chỉ giữ một số ô; mô hình tra cứu mềm vẫn như mục 3 nhưng chỉ trên một tập khoá con.

Các bài trên không được dùng số liệu của paper trong chương này.

9. Sai lầm hay gặp

- Nghĩ rằng Q, K, V là các đầu vào khác nhau. Chúng đều sinh ra từ cùng một X qua ba ma trận trọng số khác nhau (với self-attention).
- Quên rằng softmax chạy **theo từng hàng** (trên các khoá), không phải theo cột hay trên cả ma trận. Mỗi hàng của A cộng bằng 1, cột thì không.
- Cho rằng trọng số attention luôn giải thích được "mô hình hiểu gì". Trọng số chỉ nói đầu ra lấy bao nhiêu từ mỗi giá trị; nội dung của giá trị cũng quyết định đầu ra, nên đây chỉ là gợi ý, không phải bằng chứng đầy đủ.
- Đặt mặt nạ **sau** softmax rồi mới chuẩn hoá lại thủ công hoặc quên chuẩn hoá. Cách đúng là cộng $-\infty$ **trước** softmax để các ô còn lại tự chuẩn hoá.

- Nhầm khoá và giá trị. Khoá để so khớp với truy vấn; giá trị mới là thứ được cộng vào đầu ra. Hai thứ có ma trận trọng số riêng.
- Quên hệ số $\sqrt{d_k}$ hoặc chia nhầm cho d_k (thay vì căn) khi đối chiếu code với paper.

10. Bảng tóm tắt

Ý	Công thức hoặc quy tắc
Embedding	hàng thứ id của bảng $V \times d$
Truy vấn, khoá, giá trị	$Q = XW_Q$, $K = XW_K$, $V = XW_V$
Điểm số	$S = QK^\top / \sqrt{d_k}$
Trọng số	$A = \text{softmax}(S + M)$ theo hàng
Đầu ra	$O = AV$
Mặt nạ nhân quả	$M_{ij} = -\infty$ nếu $j > i$
Nhiệt độ	$w_j \propto e^{s_j/T}$; $T \to 0$ là tra cứu cứng, $T \to \infty$ là trung bình đều
Chi phí	ma trận A cỡ $T \times T$, tăng theo T^2

11. Bài tập tự tính

Bài 1. Softmax cơ bản

Tính softmax của điểm số $(0,\ \ln 2,\ \ln 4)$.

Đáp án (đã kiểm numpy): $e^s = (1, 2, 4)$, tổng 7, trọng số $(1/7,\ 2/7,\ 4/7) \approx (0{,}143;\ 0{,}286;\ 0{,}571)$.

Bài 2. Truy vấn mới trên cùng ba khoá

Dùng ba khoá $k_1 = (1, 0)$, $k_2 = (0, 1)$, $k_3 = (1, 1)$ và truy vấn $q = (1, 2)$. Tính điểm số $q \cdot k_j$, rồi trọng số softmax khi **không** chia $\sqrt{d_k}$ và khi chia.

Đáp án (đã kiểm numpy): điểm số $(1, 2, 3)$. Không chia: $(0{,}090;\ 0{,}245;\ 0{,}665)$. Chia $\sqrt{2}$: $(0{,}140;\ 0{,}284;\ 0{,}576)$, đều hơn một chút vì điểm số nhỏ đi.

Bài 3. Trung bình có trọng số

Với giá trị $v_1 = (2, 0)$, $v_2 = (0, 1)$, $v_3 = (2, 1)$, tính đầu ra khi trọng số là $(0{,}5;\ 0{,}5;\ 0)$ và khi dùng trọng số không chia ở bài 2.

Đáp án (đã kiểm numpy): $(0{,}5)(2, 0) + (0{,}5)(0, 1) = (1;\ 0{,}5)$. Với trọng số $(0{,}090;\ 0{,}245;\ 0{,}665)$: $\approx (1{,}511;\ 0{,}910)$.

Bài 4. Đếm ô số của mặt nạ nhân quả

Một chuỗi có T token. Bao nhiêu ô của ma trận A khác 0 khi có mặt nạ nhân quả? Tính cho $T = 1024$ và so với không mặt nạ.

Đáp án (đã kiểm numpy): hàng i có i ô khác 0, tổng $T(T+1)/2$. Với $T = 1024$: 524800 ô, so với $1024^2 = 1048576$ ô không mặt nạ, tức khoảng một nửa (đúng hơn là $50{,}05\%$).

Bài 5. Nhiệt độ

Với điểm số $(2, 1, 0, -1)$, tính trọng số softmax khi $T = 2$ và khi $T = 0{,}5$. Nhiệt độ nào gần tra cứu cứng hơn?

Đáp án (đã kiểm numpy): $T = 2$: $(0{,}455;\ 0{,}276;\ 0{,}167;\ 0{,}102)$. $T = 0{,}5$: $(0{,}865;\ 0{,}117;\ 0{,}016;\ 0{,}002)$. Nhiệt độ nhỏ $T = 0{,}5$ gần tra cứu cứng hơn.

Tài liệu tham khảo nền: Vaswani và cộng sự, Attention Is All You Need (ghi chú llm 01 trong sách); Radford và cộng sự, Language Models are Unsupervised Multitask Learners; Brown và cộng sự, Language Models are Few-Shot Learners (ghi chú llm 02); các ghi chú llm 14, 15 và 44 trong sách.

Nền tảng 02

Khối Transformer: nhiều đầu attention, đường residual, chuẩn hoá và FFN cùng nhau tạo một lớp

Loại: chương nền tảng (không phải ghi chú paper)

Đọc trước: attention một đầu tính tay ở nen01 của cuốn này; trung bình và phương sai ở math nền tảng 08; đạo hàm và chain rule ở math nền tảng 06, 07; mạng nơ-ron nhiều lớp ở ML cơ bản nền tảng 05; chuẩn hoá và khởi tạo ở optimization nền 08

Dùng ở: llm 01 (khối Transformer gốc), llm 09 (Pre-LN so với Post-LN), llm 10 (RMSNorm), llm 11 (GLU và SwiGLU), llm 05 (LLaMA) và các bài kiến trúc về sau

> Chương này giải thích từ đầu, không phải bản dịch một paper. Các phép tính ghi "tự tính" và các hình minh hoạ đều đã kiểm bằng numpy.

1. Vì sao cần chương này

Một khối Transformer là đơn vị lặp lại: attention để các token trao đổi thông tin với nhau, FFN để mỗi token tự xử lý thông tin ấy, residual để giữ lại đầu vào và chuẩn hoá để giữ con số không phình ra hay biến mất. Một LLM chỉ là hàng chục đến hàng trăm khối giống hệt cấu trúc, xếp chồng lên nhau, nên hiểu một khối là hiểu gần hết kiến trúc.

Các bài paper sau mỗi bài đụng đến đúng một chi tiết của khối này: bài 09 hỏi chuẩn hoá nên đặt ở đâu, bài 10 hỏi có thể chuẩn hoá rẻ hơn không, bài 11 hỏi FFN dùng hàm nào, bài 14 đến 16 hỏi cách chia nhóm các đầu attention. Chương này đặt các chi tiết ấy vào một sơ đồ chung và tính tay hai thứ: chuẩn hoá một vector nhỏ và đếm tham số của một khối.

2. Nhắc nền

2.1 Chuyện đã biết từ nen01

Ở nen01 ta đã tính $O = \text{softmax}(QK^\top/\sqrt{d_k} + M)V$ cho một đầu. Đầu vào là ma trận X cỡ $T \times d$, đầu ra cũng cỡ $T \times d_v$. Nhớ rằng attention trộn thông tin **giữa các vị trí** (qua trung bình có trọng số các hàng của V), nhưng với mỗi hàng nó chỉ biến đổi tuyến tính các giá trị, nên chưa có phi tuyến nào đáng kể bên trong mỗi token.

2.2 Trung bình, phương sai, độ lệch chuẩn của một vector

Cho vector $x = (x_1, \ldots, x_n)$. Trung bình $\mu = \frac{1}{n}\sum_i x_i$; phương sai $\sigma^2 = \frac{1}{n}\sum_i (x_i - \mu)^2$; độ lệch chuẩn $\sigma = \sqrt{\sigma^2}$. Hai thứ cần nhớ cho chương này: trừ μ thì vector có trung bình 0; chia cho σ thì độ lớn cỡ 1. Chú ý đây là các thống kê **trên các thành phần của một vector**, không phải trên một lô dữ liệu.

Ví dụ (tự tính): $x = (1, 3, -1, 5)$ có $\mu = 2$, độ lệch $(-1, 1, -3, 3)$, $\sigma^2 = \frac{1+1+9+9}{4} = 5$, $\sigma \approx 2{,}236$.

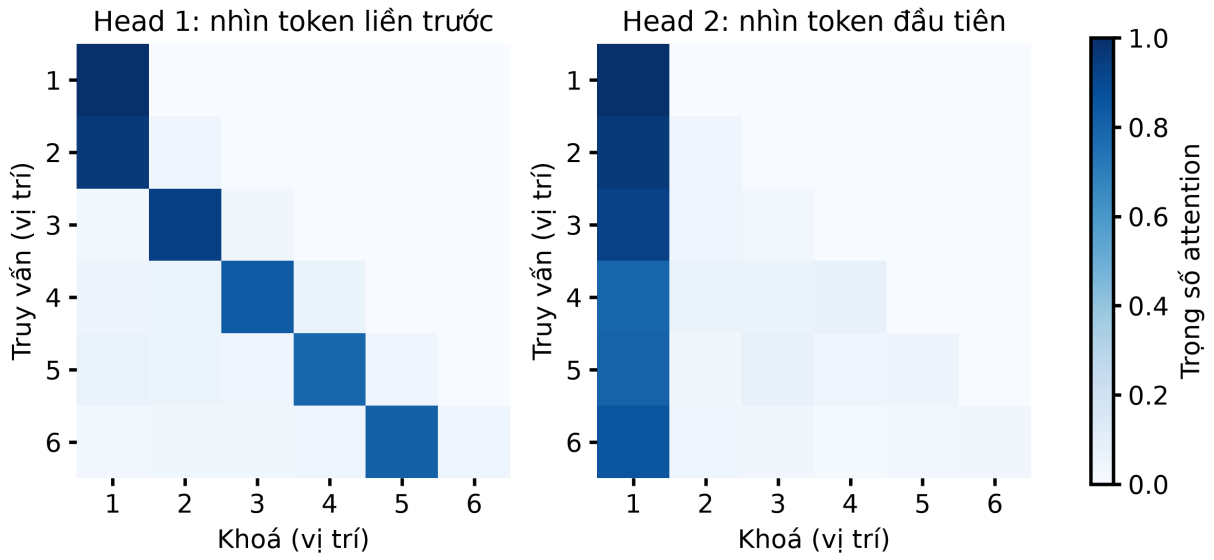
2.3 Hàm sigmoid

$\sigma(t) = \dfrac{1}{1 + e^{-t}}$ nằm trong khoảng $(0, 1)$, gần 0 khi t rất âm và gần 1 khi t rất dương. Ta dùng nó như một "cổng": nhân một số với $\sigma(t)$ nghĩa là cho qua một phần tỉ lệ. Ví dụ (tự tính): $\sigma(1) \approx 0{,}731$.

3. Nhiều đầu attention

3.1 Trực giác: mỗi đầu hỏi một kiểu câu hỏi

Một đầu attention chỉ có một ma trận trọng số A, nên mỗi token chỉ có một cách chia sự chú ý. Nhưng một từ có thể cần cùng lúc nhiều thứ: nhìn từ ngay trước nó (ngữ pháp), nhìn từ chủ ngữ ở xa (quan hệ), nhìn token đầu câu (làm mốc). **Multi-head attention chạy nhiều đầu song song, mỗi đầu có W_Q, W_K, W_V riêng và một chiều nhỏ hơn, rồi ghép kết quả lại.**



Hình 5: Hai đầu attention với hai kiểu chú ý khác nhau trên 6 token có mặt nạ nhân quả (minh hoạ, tự tính)

Cách đọc hình: đây là hình **mô phỏng**, điểm số được đặt tay theo kiểu mong muốn (cộng nhiễu nhỏ, seed cố định), không phải kết quả của một mô hình đã học. Đầu 1 dồn trọng số vào token liền trước (đường chéo phụ), đầu 2 dồn vào token đầu tiên (cột đầu). Cả hai đều có hình tam giác dưới vì mặt nạ nhân quả. Mô hình thật học được những kiểu chú ý như vậy, nhưng cũng học nhiều kiểu khó mô tả bằng lời.

3.2 Định nghĩa

Với h đầu và chiều mô hình d, mỗi đầu dùng chiều nhỏ $d_h = d/h$. Đầu r có $W_Q^{(r)}, W_K^{(r)}, W_V^{(r)} \in \mathbb{R}^{d \times d_h}$, cho đầu ra $O^{(r)}$ cỡ $T \times d_h$. Ghép các đầu theo chiều cột được ma trận cỡ $T \times d$, rồi nhân với ma trận chiếu đầu ra $W_O \in \mathbb{R}^{d \times d}$:

$$\mathrm{MHA}(X) = \mathrm{concat}\big(O^{(1)}, \ldots, O^{(h)}\big)\, W_O.$$

Phép chia điểm số trong mỗi đầu dùng $\sqrt{d_h}$ (không phải $\sqrt{d}$), vì tích vô hướng chỉ cộng d_h số hạng.

3.3 Ví dụ tay: đếm tham số với $d = 12$, $h = 3$

Mỗi đầu có $d_h = 12/3 = 4$. Một đầu có ba ma trận 12×4, tức $3 \cdot 48 = 144$ tham số. Ba đầu có $3 \cdot 144 = 432$ tham số. Ma trận W_O cỡ 12×12 thêm 144. Tổng (tự tính): $432 + 144 = 576 = 4d^2$.

So sánh: một đầu duy nhất với $d_k = d_v = d = 12$ có ba ma trận 12×12 cộng W_O cũng là $4 \cdot 144 = 576$. **Số tham số của lớp attention là $4d^2$, không phụ thuộc số đầu h** (khi $d_h = d/h$). Việc chia đầu không làm mô hình nặng hơn, chỉ chia nhỏ chiều. Bài llm 14 và 15 (multi-query và GQA) sẽ thay đổi quy tắc này: các đầu dùng chung khoá và giá trị nên W_K, W_V nhỏ đi.

4. Đường residual

4.1 Ý tưởng

Thay vì đầu ra $= F(x)$, khối tính **đầu ra** $= x + F(x)$: đầu vào được cộng thẳng vào đầu ra của nhánh tính toán F (attention hoặc FFN). Hình dung: x là một bảng ghi chú chung đi suốt mô hình; mỗi khối đọc bảng, tính một **phần bổ sung**, rồi cộng phần bổ sung đó vào bảng. Bảng không bao giờ bị viết lại hoàn toàn.

4.2 Vì sao có ích: đường đi của gradient

Từ $y = x + F(x)$ có $\dfrac{\partial y}{\partial x} = I + \dfrac{\partial F}{\partial x}$ với I là ma trận đơn vị. Qua L khối, gradient truyền ngược là tích của L ma trận dạng $I + J_l$ (đây là suy luận từ chain rule, math nền tảng 07). Khai triển tích đó có một số hạng là chính ma trận đơn vị, nghĩa là luôn có một đường đi mà gradient truyền nguyên vẹn đến các lớp đầu, dù các nhánh F có làm nó nhỏ đi hay không. Nếu không có residual, gradient phải đi qua toàn bộ L nhánh liên tiếp và dễ co về 0 hoặc phình ra.

4.3 Dòng dư (residual stream)

Cách nhìn thứ hai là coi x qua các khối là một **dòng dư** duy nhất, mỗi khối đọc và ghi vào đó. Với L khối, đầu ra cuối là x_0 cộng tổng các phần bổ sung. Giả định đơn giản (suy luận, không phải số đo): mỗi phần bổ sung có độ dài 1 và vuông góc với nhau và với x_0 có độ dài 1, thì độ dài dòng dư sau L khối là $\sqrt{1+L}$ (tự tính: $L = 24$ cho 5). Dòng dư vì thế phình dần theo độ sâu, và đây là lý do thứ hai cần chuẩn hoá.

5. Chuẩn hoá

5.1 LayerNorm

LayerNorm chuẩn hoá từng token riêng lẻ: trừ trung bình các thành phần của vector rồi chia độ lệch chuẩn, sau đó nhân và cộng hai vector học được.

$$\mathrm{LN}(x) = \gamma \odot \frac{x - \mu}{\sqrt{\sigma^2 + \epsilon}} + \beta.$$

μ, σ^2 tính trên d thành phần của vector đó; γ (nhân) và β (cộng) là hai vector d chiều học được; ϵ nhỏ (ví dụ 10^{-5}) tránh chia 0; $\odot$ là nhân từng phần tử. Khác với BatchNorm (thống kê qua các mẫu trong lô), LayerNorm chỉ nhìn một token nên cho kết quả giống nhau dù lô lớn hay nhỏ và dù dùng khi huấn luyện hay khi sinh từng token.

5.2 RMSNorm

RMSNorm bỏ bước trừ trung bình, chỉ chia cho căn bậc hai của trung bình bình phương (RMS).

$$\mathrm{RMSNorm}(x) = \gamma \odot \frac{x}{\sqrt{\frac{1}{n}\sum_i x_i^2 + \epsilon}}.$$

Ít phép tính hơn (không có μ, không có β). Ghi chú llm 10 cho biết paper RMSNorm báo thời gian chạy nhanh hơn từ 7% đến 64% tuỳ mô hình và framework mà chất lượng tương đương; đây là số của paper, không do chương này đo. Khi $\mu = 0$, RMS bằng đúng σ nên RMSNorm trùng LayerNorm.

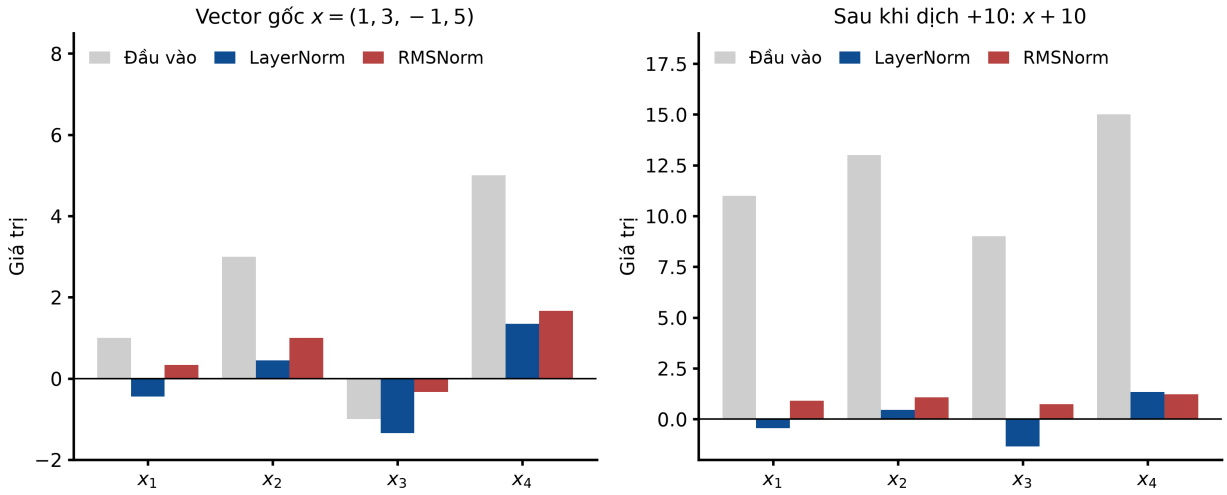
5.3 Ví dụ tay: cùng một vector, hai cách chuẩn hoá

Lấy $x = (1, 3, -1, 5)$ và bỏ ϵ, $\gamma = 1$, $\beta = 0$ cho dễ tính.

LayerNorm. $\mu = 2$, $\sigma = \sqrt{5} \approx 2{,}236$. Kết quả $\dfrac{(-1,\ 1,\ -3,\ 3)}{2{,}236} = (-0{,}447;\ 0{,}447;\ -1{,}342;\ 1{,}342)$ (tự tính). Trung bình bằng 0 và độ lớn $\sqrt{\text{tổng bình phương}} = 2 = \sqrt{n}$.

RMSNorm. $\frac{1}{n}\sum x_i^2 = \frac{1+9+1+25}{4} = 9$, RMS $= 3$. Kết quả $x/3 = (0{,}333;\ 1;\ -0{,}333;\ 1{,}667)$ (tự tính). Trung bình **không** bằng 0 (bằng $0{,}667$), và độ lớn cũng là $\sqrt{n} = 2$ vì ta chia đúng cho RMS.

Thêm γ. Nếu $\gamma = (1,\ 0{,}5,\ 2,\ 1)$ thì RMSNorm cho $(0{,}333;\ 0{,}5;\ -0{,}667;\ 1{,}667)$ (tự tính): γ cho phép mô hình tự co giãn từng chiều.



Hình 6: So sánh LayerNorm và RMSNorm trên vector (1, 3, -1, 5) và trên cùng vector cộng thêm 10 (minh hoạ, tự tính)

Cách đọc hình: thanh xám là đầu vào, xanh là LayerNorm, đỏ là RMSNorm. Bên trái, hai kết quả có cùng độ lớn nhưng LayerNorm cân quanh 0 còn RMSNorm giữ nguyên dấu và tỉ lệ giữa các thành phần. Bên phải ta cộng 10 vào mọi thành phần: LayerNorm cho **đúng kết quả cũ** $(-0{,}447;\ 0{,}447;\ -1{,}342;\ 1{,}342)$ vì phép trừ trung bình triệt tiêu phần dịch, còn RMSNorm cho $(0{,}901;\ 1{,}065;\ 0{,}737;\ 1{,}229)$ (tự tính), khác hẳn. Cả hai đều **bất biến khi nhân cả vector với một hằng dương** (bỏ ϵ): nhân x với 10 vẫn cho đúng kết quả cũ. Giả thuyết của bài llm 10 là chính tính bất biến theo tỉ lệ này mới là phần cốt lõi, còn tính bất biến theo dịch chuyển là thừa; đó là giả thuyết của paper được ghi chú ở llm 10, kèm kết quả thực nghiệm ở đó.

5.4 Đặt chuẩn hoá ở đâu: Post-LN và Pre-LN

Transformer gốc đặt chuẩn hoá **sau** phép cộng residual (Post-LN):

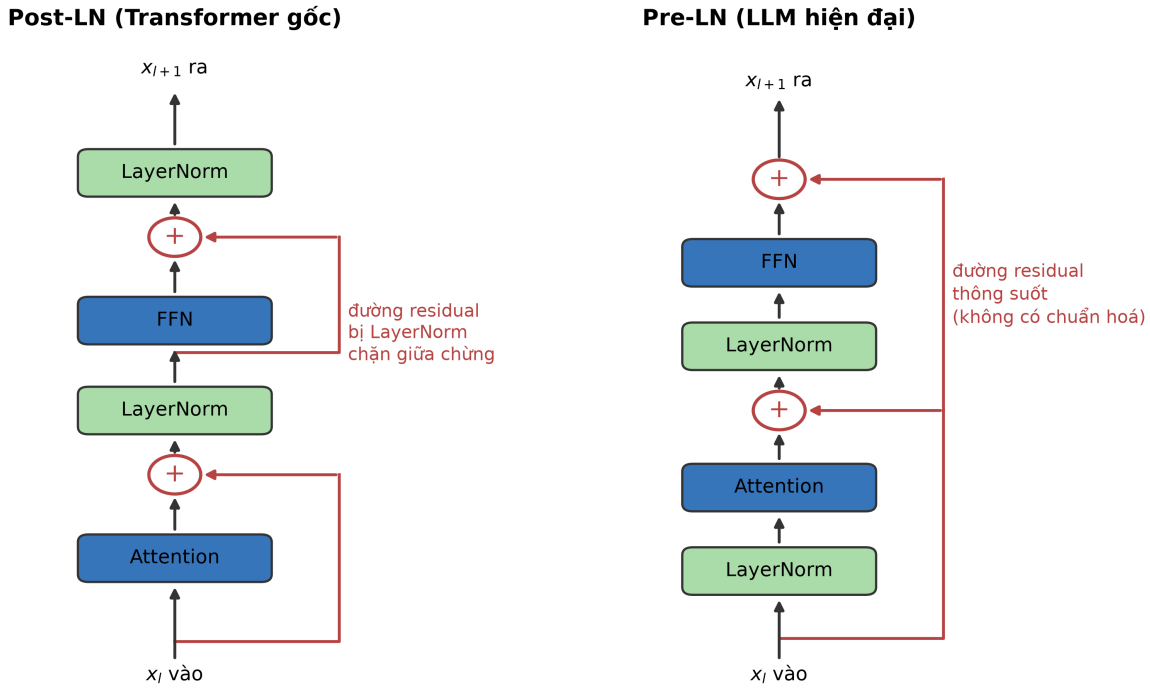
$$x_{l+1} = \mathrm{LN}\big(x_l + F(x_l)\big).$$

Các LLM hiện đại thường đặt **trước** nhánh tính toán (Pre-LN), và thêm một chuẩn hoá cuối trước lớp dự đoán:

$$x_{l+1} = x_l + F\big(\mathrm{LN}(x_l)\big).$$

Cách đọc hình: đi từ dưới lên. Đường đỏ là residual, đường đen là nhánh tính toán. Ở Post-LN, mỗi vòng cộng residual đều bị LayerNorm chặn ngay sau đó nên dòng dư bị chuẩn hoá ở **mọi** khối. Ở Pre-LN, đường đỏ chạy thông suốt từ đầu đến cuối, chuẩn hoá chỉ nằm trên nhánh bên trong; hình vẽ bỏ chuẩn hoá cuối cùng của toàn mạng để gọn.

Đây là điểm mấu chốt về gradient: ở Pre-LN, khai triển $x_L = x_0 + \sum_l F_l(\mathrm{LN}(x_l))$ có một đường đi đồng nhất từ đầu ra về đầu vào (suy luận từ công thức). Theo ghi chú llm 09, phân tích trung bình-trường của paper chỉ ra rằng ở Post-LN gradient ở các tham số gần đầu ra lớn và không giảm theo số lớp lúc khởi tạo nên cần learning rate warm-up, còn ở Pre-LN thì có thể bỏ warm-up.



Hình 7: Sơ đồ khối Post-LN (trái) và Pre-LN (phải) với đường residual màu đỏ (minh hoạ, tự tính)

Ví dụ tay (tự tính) thấy sự khác biệt: lấy $x = (1, 3, -1, 5)$ và nhánh giả định $F(y) = 0{,}5\,y$. Post-LN: $\mathrm{LN}(x + 0{,}5x) = \mathrm{LN}(1{,}5x) = (-0{,}447;\ 0{,}447;\ -1{,}342;\ 1{,}342)$, tức là **kết quả như không có nhánh**, độ lớn bị ép về 2 (thông tin về độ lớn 6 của x mất). Pre-LN: $x + 0{,}5\,\mathrm{LN}(x) = (0{,}776;\ 3{,}224;\ -1{,}671;\ 5{,}671)$, độ lớn $6{,}78$; đầu vào được giữ lại và nhánh chỉ thêm một phần. Đây là ví dụ tay một bước với nhánh đơn giản, không chứng minh hành vi ở mạng sâu.

Cái giá của Pre-LN: dòng dư không bị chuẩn hoá nên phình dần theo độ sâu (mục 4.3), và người ta thường thêm một chuẩn hoá cuối trước khi dự đoán.

6. Mạng FFN và các biến thể GLU

6.1 FFN: xử lý từng token riêng

Sau attention, mỗi token qua cùng một mạng hai lớp **độc lập với các token khác**:

$$\mathrm{FFN}(x) = \phi(xW_1)\,W_2, \qquad W_1 \in \mathbb{R}^{d \times d_{ff}},\ W_2 \in \mathbb{R}^{d_{ff} \times d}.$$

Chiều ẩn d_{ff} thường là $4d$. Attention trộn **giữa** các token, FFN biến đổi **trong** từng token, bằng cách mở rộng ra chiều d_{ff} lớn, áp phi tuyến ϕ, rồi nén về d. Theo ghi chú llm 11, khoảng 2/3 tham số mỗi khối nằm ở FFN khi $d_{ff} = 4d$. Các bài về MoE (llm 27, 28) thay đúng phần này bằng nhiều chuyên gia.

6.2 Hàm kích hoạt ReLU, GELU, SiLU

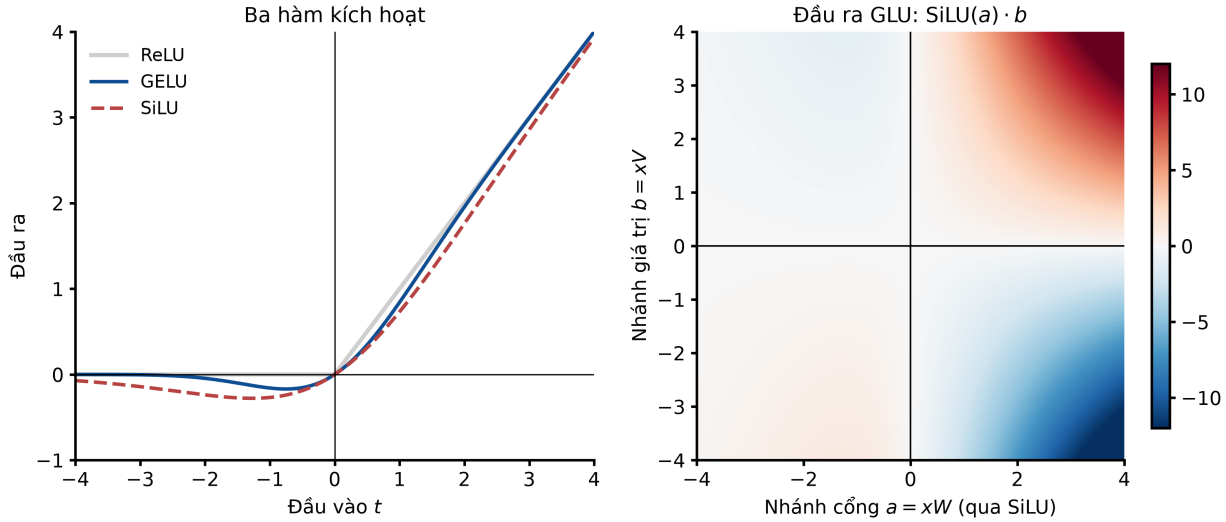
ReLU: $\max(t, 0)$. GELU: $t \cdot \Phi(t)$ với Φ là hàm phân phối chuẩn tắc, một đường cong trơn giống ReLU. SiLU (còn gọi swish): $t \cdot \sigma(t)$. Cả ba gần nhau ở xa gốc; khác ở gần 0, nơi GELU và SiLU trơn và hơi âm một chút. Giá trị tại $t = 1$ (tự tính): GELU $0{,}841$, SiLU $0{,}731$; tại $t = -1$: GELU $-0{,}159$, SiLU $-0{,}269$. SiLU có đáy thấp nhất khoảng $-0{,}278$ tại $t \approx -1{,}28$ (tự tính).

6.3 GLU và SwiGLU: nhân với một cổng

GLU (gated linear unit) dùng hai phép chiếu song song và nhân từng phần tử một nhánh "cổng" với nhánh "giá trị". Dạng gốc: $(xW) \odot \sigma(xV)$ với σ sigmoid. **SwiGLU** thay sigmoid bằng SiLU và thêm phép chiếu đầu ra:

$$\text{SwiGLU-FFN}(x) = \Big(\text{SiLU}(xW_1) \odot (xV)\Big) W_2.$$

Số ma trận từ hai thành ba (W_1, V, W_2). Ví dụ tay một thành phần (tự tính): nhánh cổng $a = 2$, nhánh giá trị $b = 3$: $\text{SiLU}(2) = 1{,}762$ nên đầu ra $1{,}762 \cdot 3 = 5{,}285$. Nếu $a = -2$: $\text{SiLU}(-2) = -0{,}238$ và đầu ra $-0{,}715$, nhỏ và bị "đóng cổng". Với GLU sigmoid gốc, $\sigma(2) \cdot 3 = 2{,}642$ và $\sigma(-2) \cdot 3 = 0{,}358$.



Hình 8: Ba hàm kích hoạt ReLU, GELU, SiLU (trái) và đầu ra của cổng SiLU(a) nhân b (phải) (minh hoạ, tự tính)

Cách đọc hình: bên trái là ba hàm một biến; GELU và SiLU gần trùng nhau và đều nhẵn quanh 0. Bên phải là đầu ra của một thành phần SwiGLU như hàm của hai biến: trục ngang là nhánh cổng a (đã qua SiLU), trục dọc là nhánh giá trị b; màu đỏ là đầu ra dương, xanh là âm. Khi a rất âm (nửa trái), toàn bộ đầu ra gần 0 bất kể b: cổng đóng. Khi a dương, đầu ra tỉ lệ với b: cổng mở. Mạng học cách mở hay đóng cổng tuỳ nội dung của từng token.

6.4 Đếm tham số để giữ cân bằng

Vì SwiGLU có ba ma trận thay vì hai, nếu giữ nguyên $d_{ff} = 4d$ thì FFN nặng hơn 1,5 lần. Để giữ số tham số xấp xỉ, ta giảm chiều ẩn xuống $d'_{ff} = \frac{2}{3} \cdot 4d = \frac{8}{3}d$. Kiểm tra: $3\, d\, \frac{8}{3}d = 8d^2 = 2\, d\, (4d)$ (tự tính). Ghi chú llm 11 ghi cụ thể $3072 \to 2048$ trong paper GLU, và ghi chú llm 05 ghi rằng LLaMA dùng quy tắc $\frac{2}{3}4d$ này.

Ví dụ tay: một khối với $d = 12$. Giả định (không phải mô hình thật): không có bias, Pre-LN với RMSNorm có γ (d tham số mỗi chuẩn hoá), hai chuẩn hoá mỗi khối.

- Attention: $4d^2 = 4 \cdot 144 = 576$.
- FFN dạng thông thường với $d_{ff} = 4d = 48$: $2 \cdot 12 \cdot 48 = 1152$.
- FFN dạng SwiGLU với $d'_{ff} = \frac{8}{3} \cdot 12 = 32$: $3 \cdot 12 \cdot 32 = 1152$ (tự tính), bằng đúng số trên.
- Hai RMSNorm: $2 \cdot 12 = 24$.
- Tổng một khối: $576 + 1152 + 24 = 1752$ (tự tính).

Ví dụ có cỡ thật hơn (giả định cấu hình): $d = 4096$, $d'_{ff} = 11008$ (đây là $\frac{8}{3}d \approx 10922{,}7$ làm tròn

lên bội số của 256, đúng với cách mô tả ở ghi chú llm 11). Attention: $4 \cdot 4096^2 = 67\,108\,864$. SwiGLU: $3 \cdot 4096 \cdot 11008 = 135\,266\,304$. Hai RMSNorm: 8192. Tổng 202 383 360 (tự tính); FFN chiếm khoảng 66,8% và gấp khoảng 2,02 lần attention. Phần lớn tham số của một khối nằm ở FFN, đúng như quan sát ở ghi chú llm 11.

7. Biến thể và chi tiết hay bị bỏ qua

- **Bỏ bias.** Nhiều triển khai LLM không dùng hằng số lệch trong các phép chiếu; số tham số ở trên đã bỏ bias, nếu có bias thì cộng thêm vài chục đến vài trăm tham số cho mỗi ma trận, rất nhỏ so với d^2.
- **Chuẩn hoá cuối.** Pre-LN cần một chuẩn hoá sau khối cuối cùng, trước lớp dự đoán, vì dòng dư không được chuẩn hoá bởi nhánh.
- **Chuẩn hoá trên từng đầu.** Một số mô hình chuẩn hoá thêm Q và K trước khi tính điểm số để tránh điểm số quá lớn; đây là chi tiết thêm không có trong sơ đồ, nằm ở các bài sau.
- **Thứ tự song song.** Một số kiến trúc tính attention và FFN song song trên cùng một đầu vào thay vì nối tiếp; công thức khác nhau về chi tiết nhưng ý dòng dư vẫn như mục 4.
- **Phần mềm và độ chính xác thấp.** Phép bình phương và cộng trong RMSNorm có thể tràn nếu dùng số chấm động độ chính xác thấp; nhiều triển khai tính chuẩn hoá bằng độ chính xác cao hơn rồi đổi lại (xem optimization nền 08).

8. Nối sang các bài paper của cuốn

- **llm 01:** khối gốc là Post-LN với ReLU trong FFN và nhiều đầu attention; sau chương này sơ đồ bên trái ở mục 5.4 chính là khối đó.
- **llm 09:** trả lời câu hỏi ở mục 5.4, vì sao Pre-LN dễ huấn luyện hơn. Công thức Pre-LN và Post-LN trong chương là công thức dùng trong bài đó.
- **llm 10:** RMSNorm ở mục 5.2; tính bất biến theo tỉ lệ và việc bỏ trừ trung bình là chủ đề chính, kèm thử nghiệm tốc độ.
- **llm 11:** GLU và SwiGLU ở mục 6.3; việc hạ d_{ff} xuống $\frac{2}{3}$ ở mục 6.4 được bài này đo lường.
- **llm 05 (LLaMA):** kết hợp Pre-LN với RMSNorm, SwiGLU và RoPE (nen03); chương này và nen03 cùng cho đủ các thành phần.
- **llm 14, 15, 16:** thay đổi cách chia W_K, W_V giữa các đầu (mục 3.3).
- **llm 27, 28:** thay khối FFN ở mục 6 bằng nhiều chuyên gia chọn có điều kiện.

9. Sai lầm hay gặp

- Nghĩ số đầu h làm tăng tham số attention. Với $d_h = d/h$, tổng vẫn là $4d^2$; chỉ khi d_h giữ cố định và tăng h thì tham số mới tăng.
- Nhầm LayerNorm với BatchNorm. LayerNorm tính thống kê trên các **thành phần của một token**, không qua các mẫu trong lô.
- Cho rằng RMSNorm "bằng" LayerNorm. Chúng chỉ trùng khi trung bình các thành phần bằng 0; với vector có trung bình khác 0 (như ví dụ cộng 10) kết quả khác nhau.
- Đặt chuẩn hoá nhầm vị trí khi chuyển công thức paper sang code: $\mathrm{LN}(x + F(x))$ và $x + F(\mathrm{LN}(x))$ là hai mô hình khác nhau.
- Quên chiều ẩn d_{ff} phải giảm khi dùng SwiGLU nếu muốn so sánh công bằng số tham số.
- Nghĩ residual chỉ là "mẹo tối ưu". Nó thay đổi cách nhìn của cả khối: khối học phần **bổ sung** chứ không học lại toàn bộ biểu diễn.

10. Bảng tóm tắt

Thành phần	Công thức	Ghi chú
Nhiều đầu	concat$(O^{(r)})W_O$, $d_h = d/h$	tham số $4d^2$, độc lập h
Residual	$y = x + F(x)$	gradient có đường đồng nhất
LayerNorm	$\gamma \odot (x-\mu)/\sqrt{\sigma^2+\epsilon} + \beta$	theo từng token
RMSNorm	$\gamma \odot x/\sqrt{\text{mean}(x^2)+\epsilon}$	bỏ μ và β
Post-LN	LN$(x + F(x))$	cần warm-up (theo llm 09)
Pre-LN	$x + F(\text{LN}(x))$	thêm LN cuối
FFN	$\phi(xW_1)W_2$	$2\,d\,d_{ff}$ tham số
SwiGLU	$(\text{SiLU}(xW_1) \odot xV)W_2$	$3\,d\,d'_{ff}$, $d'_{ff} = \frac{8}{3}d$

11. Bài tập tự tính

Bài 1. Chuẩn hoá một vector

Tính LayerNorm và RMSNorm (bỏ ϵ, $\gamma = 1$, $\beta = 0$) của $x = (2, 4, 6, 8)$.

Đáp án (đã kiểm numpy): $\mu = 5$, $\sigma^2 = 5$, LayerNorm $= (-1{,}342;\ -0{,}447;\ 0{,}447;\ 1{,}342)$. RMS $= \sqrt{30} \approx 5{,}477$, RMSNorm $= (0{,}365;\ 0{,}730;\ 1{,}095;\ 1{,}461)$.

Bài 2. Đếm tham số

Với $d = 24$, tính số tham số attention ($4d^2$), FFN thông thường với $d_{ff} = 4d$ và FFN SwiGLU với $d'_{ff} = \frac{8}{3}d$ (bỏ bias và chuẩn hoá).

Đáp án (đã kiểm numpy): attention $4 \cdot 576 = 2304$; FFN thông thường $2 \cdot 24 \cdot 96 = 4608$; SwiGLU với $d'_{ff} = 64$: $3 \cdot 24 \cdot 64 = 4608$, bằng nhau.

Bài 3. Cổng SwiGLU

Một thành phần có nhánh cổng $a = 1$ và nhánh giá trị $b = -2$. Tính đầu ra của SwiGLU (một thành phần, bỏ W_2).

Đáp án (đã kiểm numpy): SiLU$(1) = 0{,}731$ nên đầu ra $= 0{,}731 \cdot (-2) = -1{,}462$.

Bài 4. Độ lớn dòng dư

Dùng giả định ở mục 4.3 (đầu vào độ dài 1, mỗi khối thêm một vector độ dài 1 vuông góc với mọi thứ trước). Tính độ dài dòng dư sau $L = 4$ và $L = 24$ khối.

Đáp án (đã kiểm numpy): $\sqrt{1+L}$: $L = 4$ cho $\sqrt{5} \approx 2{,}236$; $L = 24$ cho 5.

Bài 5. Pre-LN so với Post-LN

Với $x = (1, 3, -1, 5)$ và nhánh $F(y) = 0{,}5y$, tính độ lớn (chuẩn Euclid) của đầu ra Post-LN LN$(x + F(x))$ và Pre-LN $x + F(\text{LN}(x))$, bỏ γ, β, ϵ. Cái nào còn giữ thông tin về độ lớn của x?

Đáp án (đã kiểm numpy): $\|x\| = 6$. Post-LN: LN$(1{,}5x) =$ LN(x) có độ lớn 2 (luôn bằng $\sqrt{n}$), mất thông tin về độ lớn 6. Pre-LN: $x + 0{,}5\,\text{LN}(x) = (0{,}776;\ 3{,}224;\ -1{,}671;\ 5{,}671)$ có độ lớn $\approx 6{,}78$, giữ thông tin về độ lớn của x.

Tài liệu tham khảo nền: Vaswani và cộng sự, Attention Is All You Need; Ba, Kiros và Hinton, Layer Normalization; Xiong và cộng sự, On Layer Normalization in the Transformer Architecture; Zhang và Sennrich, Root Mean Square Layer Normalization; Shazeer, GLU Variants Improve Transformer; các ghi chú llm 01, 05, 09, 10 và 11 trong sách.

Nền tảng 03

Mã hoá vị trí: attention không biết thứ tự, nên ta phải đưa vị trí vào bằng cách quay (RoPE) hoặc trừ điểm theo khoảng cách (ALiBi)

Loại: chương nền tảng (không phải ghi chú paper)

Đọc trước: attention một đầu tính tay và tính hoán vị-đồng biến ở nen01 của cuốn này; khối Transformer ở nen02 của cuốn này; vector và phép quay 2 chiều ở math nền tảng 02, 03; tích vô hướng ở math nền tảng 02

Dùng ở: llm 12 (RoPE), llm 13 (ALiBi), llm 05 (LLaMA dùng RoPE), llm 16 (MLA có phần khoá mang RoPE tách riêng) và các bài về ngữ cảnh dài (llm 55, 61 đến 73)

> Chương này giải thích từ đầu, không phải bản dịch một paper. Các phép tính ghi "tự tính" và các hình minh hoạ đều đã kiểm bằng numpy.

1. Vì sao cần chương này

Mã hoá vị trí là cách làm cho mô hình biết token nào đứng trước, token nào đứng sau, và hai token cách nhau bao xa. Attention thuần chỉ nhìn vào nội dung của các token, không nhìn vào chỗ đứng của chúng; không có thông tin vị trí thì "chó cắn người" và "người cắn chó" trông giống nhau. Mọi LLM đều phải chọn một cách mã hoá vị trí, và lựa chọn đó quyết định mô hình xử lý được chuỗi dài đến mức nào.

Chương đi theo thứ tự: tại sao thiếu vị trí là vấn đề (nhắc nen01), cách đầu tiên (cộng vector sinusoidal vào embedding), rồi hai cách hiện đại là RoPE (quay vector truy vấn và khoá theo vị trí, để tích vô hướng chỉ phụ thuộc hiệu vị trí) và ALiBi (trừ điểm số tuyến tính theo khoảng cách). Phần cuối nói về **ngoại suy độ dài**: huấn luyện ở độ dài ngắn nhưng dùng ở độ dài dài hơn. Hai ví dụ tay chính là chuỗi quay hai chiều ở RoPE và ma trận thiên lệch ở ALiBi.

2. Nhắc nền

2.1 Phép quay trong mặt phẳng

Quay vector (x, y) một góc α (ngược chiều kim đồng hồ) cho vector mới

$$R(\alpha)\begin{pmatrix} x \\ y \end{pmatrix} = \begin{pmatrix} \cos\alpha & -\sin\alpha \\ \sin\alpha & \cos\alpha \end{pmatrix}\begin{pmatrix} x \\ y \end{pmatrix}.$$

Ba tính chất ta sẽ dùng: (a) phép quay giữ nguyên độ dài vector; (b) hai phép quay liên tiếp cộng góc, $R(\alpha)R(\beta) = R(\alpha+\beta)$; (c) chuyển vị của phép quay là phép quay ngược, $R(\alpha)^\top = R(-\alpha)$. Ví dụ (tự tính): quay $(1, 0)$ một góc $90°$ cho $(0, 1)$; quay $(1, 0)$ một góc $45°$ cho $(0{,}707;\ 0{,}707)$.

2.2 Tính hoán vị-đồng biến của attention

Ở nen01, mục 7.3, ta thấy: đổi chỗ hai token đầu trong ví dụ ba token thì vector ra của token thứ ba không đổi (tự tính: vẫn $(1{,}389;\ 0{,}926)$), và hai vector kia chỉ đổi chỗ cho nhau. Lý do là đầu ra của mỗi token là tổng có trọng số theo token, và tổng không phụ thuộc thứ tự. Kết luận: nếu chỉ đưa embedding vào attention thì mô hình **không phân biệt được thứ tự**. Mặt nạ nhân quả giúp một phần (mỗi token thấy một tập khác nhau) nhưng không nói được "token này cách token kia bao xa".

2.3 Hai cách đưa vị trí vào

Có hai hướng. Hướng thứ nhất: **đổi đầu vào**, cộng một vector vị trí p_m vào embedding của token ở vị trí m rồi đưa tất cả qua mô hình như cũ. Hướng thứ hai: **đổi điểm số attention**, sửa cách tính $q \cdot k$ sao cho nó phụ thuộc vị trí. RoPE và ALiBi thuộc hướng thứ hai, và có điểm chung quan trọng: chỉ đưa vị trí vào truy vấn và khoá, không đưa vào giá trị.

3. Cách thứ nhất: vector sinusoidal cộng vào embedding

3.1 Công thức

Transformer gốc cộng vào embedding ở vị trí m một vector p_m có các cặp thành phần sin, cos ở nhiều tần số. Với chiều d và chỉ số cặp $i = 0, 1, \ldots, d/2 - 1$:

$$p_{m,2i} = \sin(m\,\omega_i), \qquad p_{m,2i+1} = \cos(m\,\omega_i), \qquad \omega_i = 10000^{-2i/d}.$$

Tần số $\omega_0 = 1$ (chu kỳ $2\pi \approx 6{,}3$ token) là nhanh nhất; ω_i giảm dần theo i nên các cặp sau có chu kỳ dài hơn rất nhiều. Hình dung đồng hồ có nhiều kim: kim giây quay nhanh, kim giờ quay chậm, đọc cùng lúc tất cả các kim thì xác định được vị trí.

3.2 Ví dụ tay: $d = 4$

Ở $d = 4$: $\omega_0 = 1$ và $\omega_1 = 10000^{-2/4} = 0{,}01$. Vector vị trí là $p_m = (\sin m,\ \cos m,\ \sin 0{,}01m,\ \cos 0{,}01m)$. Ba vị trí đầu (tự tính):

- $p_0 = (0;\ 1;\ 0;\ 1)$
- $p_1 = (0{,}8415;\ 0{,}5403;\ 0{,}0100;\ 1{,}0000)$
- $p_2 = (0{,}9093;\ -0{,}4161;\ 0{,}0200;\ 0{,}9998)$

Tích vô hướng giữa các vector vị trí chỉ phụ thuộc hiệu vị trí: theo công thức cộng góc, $p_m \cdot p_n = \sum_i \cos\big((m-n)\omega_i\big)$. Kiểm tra bằng số (tự tính): $p_0 \cdot p_1 = 0{,}5403 + 0{,}99995 = 1{,}5403$ và $p_1 \cdot p_2 = 0{,}7651 - 0{,}2248 + 0{,}0002 + 0{,}9998 = 1{,}5403$, đúng bằng $\cos 1 + \cos 0{,}01 = 1{,}5403$. Hai cặp cùng cách nhau 1 cho cùng kết quả.

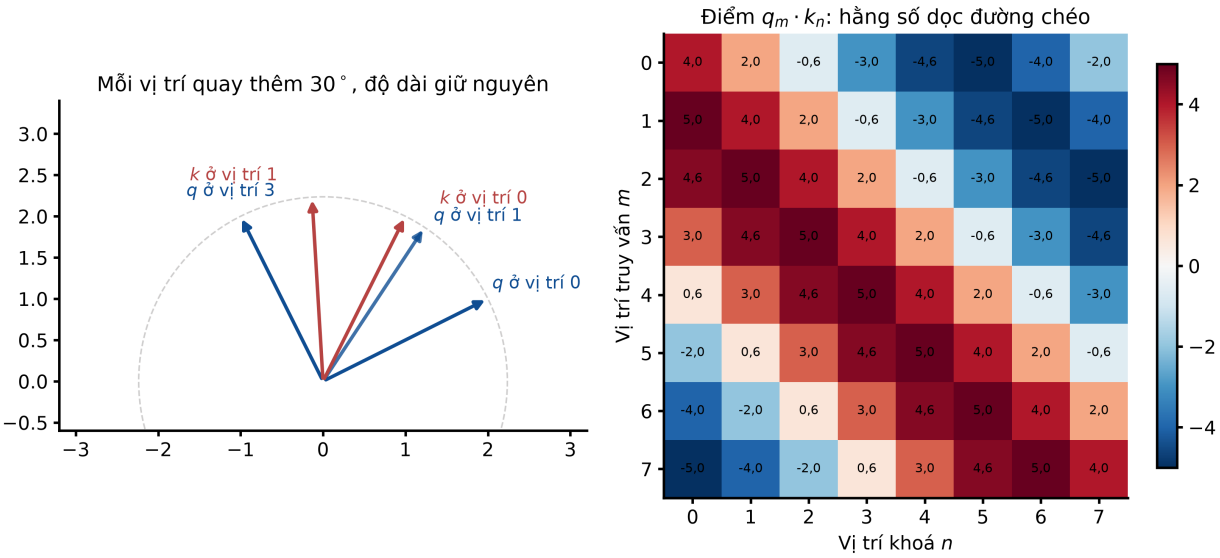
3.3 Giới hạn

Điều trên đúng cho các vector p_m **thô**. Nhưng trong mô hình, vector vào attention là $x_m + p_m$ và còn đi qua W_Q, W_K; khai triển $(x_m + p_m)W_Q W_K^\top (x_n + p_n)^\top$ cho bốn số hạng, trong đó $p_m W_Q W_K^\top p_n^\top$ với ma trận $W_Q W_K^\top$ bất kỳ không bảo đảm chỉ phụ thuộc $m - n$ (đây là suy luận của chương từ khai triển, không phải khẳng định của một paper). Vì vậy thông tin khoảng cách bị trộn lẫn nội dung và mô hình phải tự học cách tách ra. Theo ghi chú llm 13, thí nghiệm của paper ALiBi còn cho thấy sinusoidal ngoại suy rất yếu: hiệu năng ngừng cải thiện sau khoảng 20 đến 50 token vượt độ dài huấn luyện rồi xấu đi nhanh. Đây chính là động lực cho RoPE và ALiBi.

4. RoPE: quay vector theo vị trí

4.1 Trực giác bằng hình

RoPE (rotary position embedding) không cộng gì vào embedding mà quay vector truy vấn và vector khoá một góc tỉ lệ với vị trí của chúng. Token ở vị trí m bị quay một góc $m\theta$ với θ là một hằng số. Khi tính tích vô hướng giữa truy vấn ở vị trí m và khoá ở vị trí n, hai phép quay chồng lên nhau: một bên quay $m\theta$, bên kia quay $n\theta$, và tích vô hướng chỉ thấy **hiệu** $(n - m)\theta$, vì tích vô hướng không đổi khi ta quay cả hai vector cùng một góc.



Hình 9: Truy vấn và khoá bị quay theo vị trí (trái), và ma trận điểm q_m . k_n có giá trị hằng dọc mỗi đường chéo (phải) (minh hoạ, tự tính)

Cách đọc hình: bên trái, một vector truy vấn $q = (2, 1)$ được quay $30°$ cho mỗi bước vị trí; ta thấy q ở các vị trí 0, 1, 3 (xanh) và khoá $k = (1, 2)$ ở các vị trí 0, 1 (đỏ), mọi mũi tên nằm trên cùng một vòng tròn vì phép quay giữ độ dài. Bên phải là điểm $q_m \cdot k_n$ cho mọi cặp (m, n) từ 0 đến 7: các ô **cùng một đường chéo có cùng giá trị** (ví dụ cả đường chéo chính đều là $4{,}0$). Giá trị chỉ phụ thuộc $n - m$, không phụ thuộc m hay n riêng lẻ. Màu xanh và đỏ lặp tuần hoàn dọc theo khoảng cách vì góc quay đi qua một vòng tròn đầy đủ sau 12 bước (do $12 \times 30° = 360°$).

4.2 Định nghĩa trong hai chiều

Ở chiều $d = 2$, với $q, k \in \mathbb{R}^2$ và vị trí m, n, RoPE quay theo góc $m\theta$ và $n\theta$:

$$\tilde{q}_m = R(m\theta)\, q, \qquad \tilde{k}_n = R(n\theta)\, k.$$

Điểm số attention (chưa chia $\sqrt{d_k}$) là

$$\tilde{q}_m \cdot \tilde{k}_n = q^\top R(m\theta)^\top R(n\theta)\, k = q^\top R\big((n-m)\theta\big)\, k.$$

Chỉ còn hiệu vị trí $n - m$ trong công thức, đây là "tích vô hướng chỉ phụ thuộc khoảng cách tương đối". Chú ý dấu: điểm số phụ thuộc $n - m$ **có dấu**, nên "khoá cách truy vấn 2 về phía trước" và "2 về phía sau" cho hai kết quả khác nhau (ví dụ bên dưới).

4.3 Ví dụ tay: $q = (2, 1)$, $k = (1, 2)$, $\theta = 30°$

Ta tính điểm số cho cặp $(m, n) = (3, 1)$ (hiệu $n - m = -2$) rồi cho cặp $(m, n) = (5, 3)$ (cũng hiệu -2), để thấy chúng bằng nhau. Dùng $\cos 30° = 0{,}866$ và $\sin 30° = 0{,}5$.

Cặp $(3, 1)$**.** Quay q một góc $3 \cdot 30° = 90°$: $R(90°)(2, 1) = (0 \cdot 2 - 1 \cdot 1,\ 1 \cdot 2 + 0 \cdot 1) = (-1,\ 2)$. Quay k một góc $30°$: $(1 \cdot 0{,}866 - 2 \cdot 0{,}5,\ 1 \cdot 0{,}5 + 2 \cdot 0{,}866) = (-0{,}134;\ 2{,}232)$. Tích vô hướng $= (-1)(-0{,}134) + 2 \cdot 2{,}232 = 0{,}134 + 4{,}464 = 4{,}598$ (tự tính).

Cặp $(5, 3)$**.** Quay q một góc $150°$ cho $(-2{,}232;\ 0{,}134)$. Quay k một góc $90°$ cho $(-2,\ 1)$. Tích vô hướng $= (-2{,}232)(-2) + 0{,}134 \cdot 1 = 4{,}464 + 0{,}134 = 4{,}598$ (tự tính). **Hai kết quả bằng nhau**, dù vị trí tuyệt đối

khác hẳn.

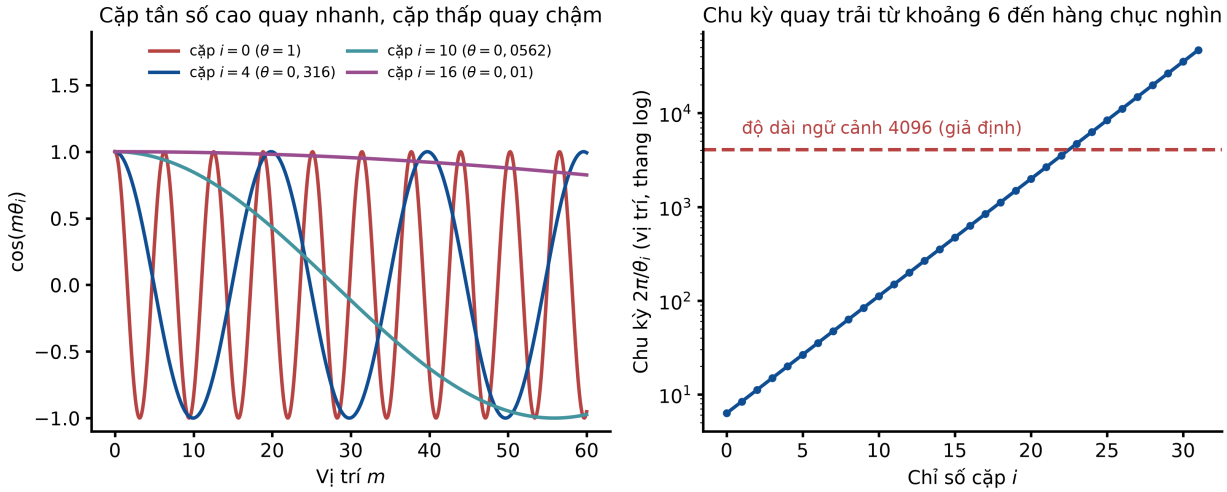
Công thức gọn. Khai triển $q^\top R(\alpha)k$ với $q = (2,1)$, $k = (1,2)$ cho $4\cos\alpha - 3\sin\alpha$ (tự tính). Với $\alpha = (n-m)\cdot 30°$: $n-m=0$ cho 4; 1 cho 1,96; 2 cho $-0{,}60$; 3 cho $-3{,}0$; 4 cho $-4{,}60$. Và với dấu ngược: $n-m=-2$ cho $4\cdot 0{,}5 + 3\cdot 0{,}866 = 4{,}598$, đúng bằng kết quả ở trên. Điểm số tuần hoàn theo khoảng cách với biên độ $\sqrt{4^2+3^2} = 5$, bằng tích độ dài $\|q\|\|k\| = \sqrt{5}\cdot\sqrt{5} = 5$.

4.4 Từ hai chiều lên d chiều

Với d chiều (chẵn) ta ghép các thành phần thành $d/2$ **cặp** và quay mỗi cặp theo tần số riêng của nó:

$$\theta_i = 10000^{-2i/d}, \qquad i = 0, 1, \ldots, \tfrac{d}{2} - 1.$$

Cặp thứ i ở vị trí m bị quay góc $m\theta_i$. Nói cách khác, ma trận quay toàn phần là khối chéo gồm $d/2$ ma trận 2×2. Đây là cùng họ tần số với sinusoidal (mục 3.1). Điểm số $\tilde{q}_m \cdot \tilde{k}_n$ là tổng của $d/2$ số hạng, mỗi số hạng như công thức hai chiều với góc $(n-m)\theta_i$; vì từng số hạng chỉ phụ thuộc $n-m$ nên tổng cũng vậy.



Hình 10: Các tần số RoPE với d=64: cos(mθ_i) theo vị trí (trái) và chu kỳ của từng cặp (phải) (minh hoạ, tự tính)

Cách đọc hình: lấy $d = 64$ (giả định), 32 cặp. Bên trái vẽ $\cos(m\theta_i)$ theo vị trí m cho bốn cặp: cặp 0 ($\theta = 1$) dao động nhanh, cặp 4 chậm hơn, cặp 10 chậm hơn nữa, cặp 16 ($\theta = 0{,}01$) gần như phẳng trong 60 vị trí. Bên phải là chu kỳ $2\pi/\theta_i$ theo chỉ số cặp trên thang log: từ $2\pi \approx 6{,}3$ token ở cặp 0 đến khoảng $47\,000$ token ở cặp cuối ($i = 31$) (tự tính). Đường đỏ chấm là độ dài ngữ cảnh 4096 giả định: các cặp tần số thấp có chu kỳ vượt xa độ dài đó, các cặp tần số cao quay nhiều vòng trong đó. Cặp tần số cao phân biệt vị trí **gần nhau** (chu kỳ ngắn, thay đổi nhanh), cặp tần số thấp phân biệt vị trí **xa nhau** (thay đổi chậm). Ghi chú llm 12 cũng nêu quan sát này.

4.5 Điều ít được nói rõ

- **Chỉ quay truy vấn và khoá, không quay giá trị.** Vị trí chỉ đi vào điểm số; đầu ra AV vẫn là trung bình có trọng số của các giá trị nguyên bản.
- **Không có tham số mới.** Các góc $m\theta_i$ cố định, nên RoPE không thêm tham số học được.
- **Áp dụng ở mọi lớp.** Mỗi lớp attention quay lại truy vấn và khoá từ đầu, không cộng vector vị trí vào dòng dư đầu vào.
- **Về "suy giảm theo khoảng cách".** Paper RoPE cho thấy bằng hình số học rằng một chặn trên của điểm số có xu hướng giảm khi khoảng cách tăng; theo ghi chú llm 12, đó là chặn trên được trình bày

bằng hình chứ không phải chứng minh chặt, và không có nghĩa điểm số thực tế luôn giảm. Đồ thị bên dưới là một trường hợp minh hoạ, không phải bằng chứng chung.

5. ALiBi: thiên lệch tuyến tính theo khoảng cách

5.1 Ý tưởng

ALiBi (attention with linear biases) không dùng embedding vị trí nào cả; nó cộng vào điểm số một số âm tỉ lệ với khoảng cách giữa truy vấn và khoá. Token càng xa bị trừ điểm càng nhiều, tạo xu hướng ưu tiên các token gần. Mỗi đầu attention dùng một độ dốc m riêng cố định (không học), nên có đầu nhìn rất gần và có đầu nhìn xa hơn.

5.2 Định nghĩa

Điểm số của truy vấn ở vị trí i với khoá ở vị trí $j \le i$ là

$$s_{ij} = \frac{q_i \cdot k_j}{\sqrt{d_k}} - m\,(i-j).$$

Với n đầu, độ dốc của đầu h là $m_h = 2^{-8h/n}$, $h = 1, \dots, n$ (theo ghi chú llm 13). Với $n = 8$: $\frac{1}{2}, \frac{1}{4}, \dots, \frac{1}{256}$ (tự tính). Với $n = 4$ thì là $\frac{1}{4}, \frac{1}{16}, \frac{1}{64}, \frac{1}{256}$.

Một chi tiết hay: softmax không đổi khi cộng cùng một hằng số vào mọi phần tử của một hàng. Vì $-m(i-j) = -mi + mj$ và $-mi$ cùng một giá trị cho cả hàng i, ta có thể thay bằng $+mj$, chỉ phụ thuộc vị trí khoá (đây là suy luận đại số của chương, cho cùng trọng số softmax).

5.3 Ví dụ tay: một hàng với độ dốc $\frac{1}{2}$

Xét truy vấn ở vị trí 4 trong chuỗi bốn token và giả định điểm số nội dung $q \cdot k/\sqrt{d_k}$ bằng 0 cho cả bốn khoá (để chỉ nhìn tác dụng của thiên lệch). Khoảng cách $i - j = 3, 2, 1, 0$ cho thiên lệch $-1{,}5;\ -1;\ -0{,}5;\ 0$. Lấy mũ: $0{,}2231;\ 0{,}3679;\ 0{,}6065;\ 1{,}0$, tổng $2{,}1975$. Trọng số: $0{,}101;\ 0{,}167;\ 0{,}276;\ 0{,}455$ (tự tính). Token gần nhất nhận $45{,}5\%$ sự chú ý, token xa nhất chỉ $10{,}1\%$, dù nội dung của chúng "giống nhau" như nhau.

Với độ dốc nhỏ hơn nhiều, ví dụ $\frac{1}{256}$ (đầu thứ tám với $n = 8$), thiên lệch cho cùng cấu hình là $(-3, -2, -1, 0)/256$ và trọng số $\approx (0{,}248;\ 0{,}250;\ 0{,}251;\ 0{,}252)$ (tự tính), gần như đều: đầu này gần như bỏ qua khoảng cách ở độ dài ngắn.

Cách đọc hình: bên trái là ma trận thiên lệch cho một đầu có độ dốc $\frac{1}{2}$, mỗi ô (i, j) ghi số cộng vào điểm số; ô chéo chính là 0 và càng xuống góc dưới trái thiên lệch càng âm (ví dụ ô $(8, 1)$ là $-3{,}5$); nửa trên trống vì có mặt nạ nhân quả. Bên phải là cùng thiên lệch dưới dạng hàm của khoảng cách $i - j$ cho tám độ dốc: đường dốc nhất (độ dốc $\frac{1}{2}$) đi xuống tới -16 ở khoảng cách 32, đường nông nhất gần như phẳng. Tám đầu cho tám "tầm nhìn" khác nhau.

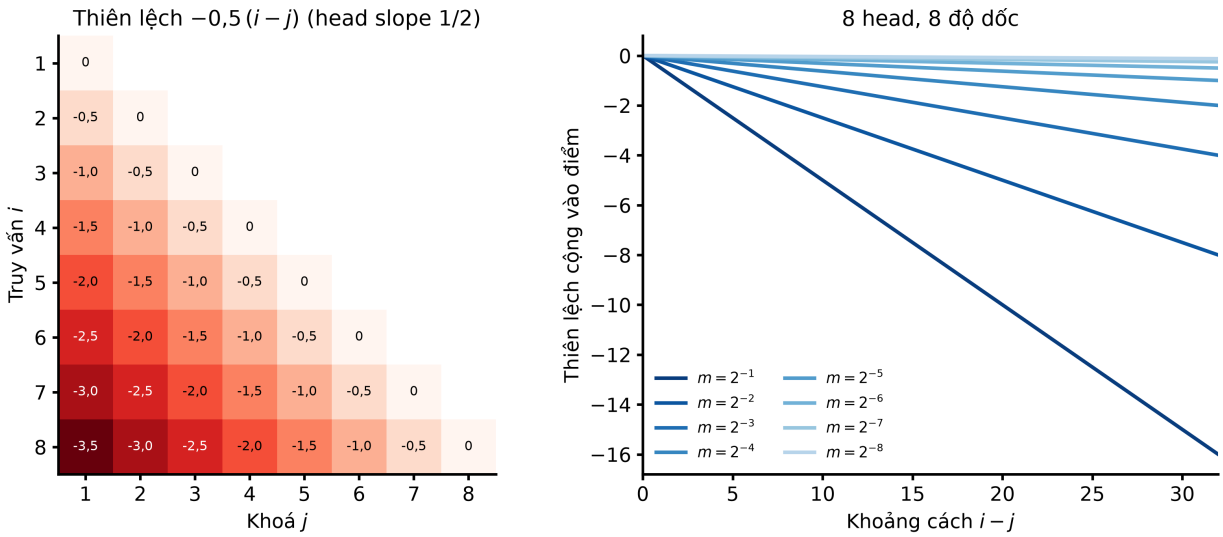
6. Ngoại suy độ dài

6.1 Vấn đề

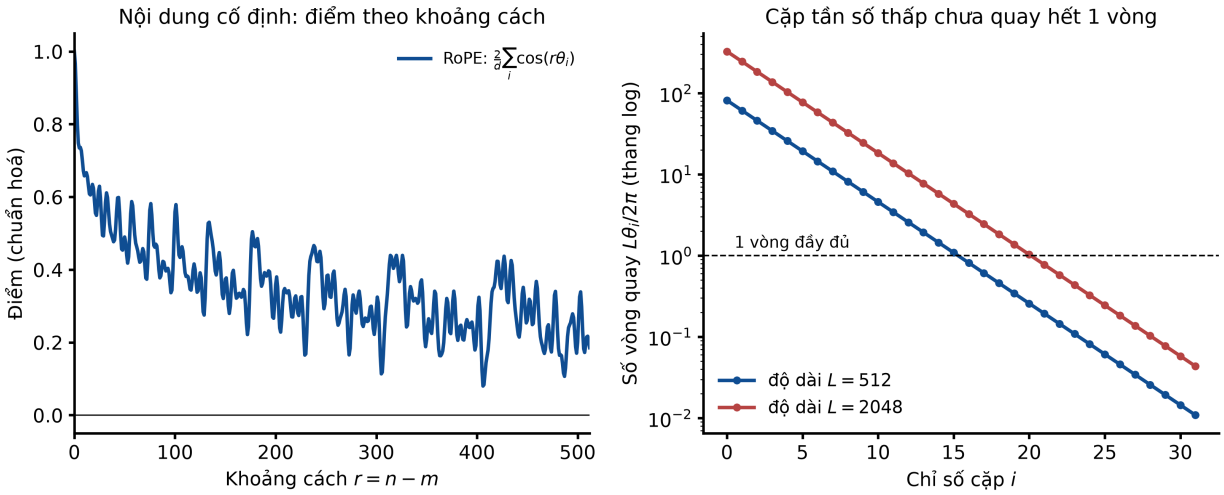
Huấn luyện trên chuỗi dài tốn kém vì chi phí attention tăng theo T^2. Câu hỏi: nếu huấn luyện ở độ dài L thì khi dùng ở độ dài $L' > L$ mô hình còn hoạt động không? Khả năng đó gọi là **ngoại suy độ dài**. Vị trí học được (một bảng embedding cho mỗi vị trí) không ngoại suy được vì không có hàng nào cho vị trí $> L$.

6.2 Vì sao RoPE gặp khó

Với RoPE, góc $m\theta_i$ ở vị trí $m > L$ là một giá trị mà cặp i **chưa từng gặp** khi huấn luyện nếu cặp đó chưa quay hết một vòng trong L vị trí. Cặp tần số cao quay nhiều vòng trong L nên mọi góc đều đã thấy; nhưng cặp tần số thấp chưa quay đủ một vòng.



Hình 11: Ma trận thiên lệch ALiBi 8 token độ dốc 1/2 (trái) và thiên lệch theo khoảng cách cho 8 độ dốc (phải) (minh hoạ, tự tính)



Hình 12: Điểm số RoPE theo khoảng cách với nội dung cố định (trái) và số vòng quay của mỗi cặp ở độ dài 512 và 2048 (phải) (minh hoạ, tự tính)

Cách đọc hình: bên trái lấy $d = 64$ và nội dung cố định $q = k = (1, 1, \ldots, 1)$ (giả định), điểm số chuẩn hoá bằng trung bình của $\cos(r\theta_i)$ trên 32 cặp, theo khoảng cách r; điểm bằng 1 ở $r = 0$, giảm nhanh ban đầu rồi dao động quanh mức thấp. Đây là một **ví dụ minh hoạ cho nội dung cố định**, không phải quy luật cho nội dung bất kỳ. Bên phải: trục dọc là số vòng quay $L\theta_i/2\pi$ trên thang log; đường ngang đứt là 1 vòng. Với $L = 512$ có 16 trên 32 cặp ($i \geq 16$) chưa quay hết một vòng; với $L = 2048$ còn 11 cặp (tự tính). Cặp cuối ($i = 31$) chỉ quay khoảng $0{,}011$ vòng trong 512 vị trí.

Một hệ quả (suy luận của chương): khi đưa vào chuỗi dài hơn L, các cặp tần số thấp nhận những góc mà chúng chưa thấy, và điểm số có thể lệch khỏi những gì mô hình đã học. Ghi chú llm 12 ghi rằng paper RoPE không thử ngoại suy độ dài và thực tế RoPE ngoại suy kém nếu không có can thiệp; ghi chú đó nhắc đến các kỹ thuật như position interpolation, NTK scaling và YaRN, được xem là kiến thức ngoài paper. Ý của position interpolation (suy luận, ví dụ minh hoạ): nén vị trí lại để mọi góc rơi vào miền đã thấy, chẳng hạn nhân vị trí với $512/2048 = 0{,}25$ để vị trí 2048 có góc như vị trí 512; cái giá là hai token liền kề giờ cách nhau góc nhỏ hơn bốn lần.

6.3 Vì sao ALiBi ngoại suy tốt hơn

ALiBi không có góc hay bảng nào gắn với vị trí tuyệt đối; thiên lệch $-m(i - j)$ chỉ là một hàm của khoảng cách, và khoảng cách 200 chỉ là một giá trị lớn hơn khoảng cách 100 trên cùng một đường thẳng. Theo ghi chú llm 13, thí nghiệm của paper (mô hình huấn luyện ở $L = 512$, bảng perplexity trên tập dev) cho:

Độ dài khi suy luận	Sinusoidal	Rotary	T5 bias	ALiBi
512	20,05	20,07	19,65	19,73
2512	132,41	31,58	20,41	18,41
15512	406,01	79,25	hết bộ nhớ ở 13512 trở lên	18,31

(Perplexity càng thấp càng tốt; số lấy từ ghi chú llm 13, không do chương này tính.) Ở độ dài huấn luyện các phương pháp tương đương; khi độ dài tăng, sinusoidal và rotary xấu đi nhanh còn ALiBi giữ ổn định. Cũng theo ghi chú đó, paper chỉ đánh giá bằng perplexity, không có thí nghiệm truy xuất chính xác ở khoảng cách xa, nên không suy ra rằng mô hình **dùng** được thông tin ở xa: thiên lệch tuyến tính còn làm giảm trọng số của các token xa.

7. Biến thể và chi tiết hay bị bỏ qua

- **Chỉ quay một phần chiều.** Một số mô hình chỉ áp RoPE cho một phần của chiều đầu (phần còn lại giữ nguyên); chi tiết này thuộc về từng mô hình cụ thể. Bài llm 16 (MLA) tách riêng một phần khoá mang RoPE khỏi phần nén; theo ghi chú đó, YaRN chỉ áp dụng cho khoá mang RoPE tách riêng.
- **Cơ số 10000 không bắt buộc.** Đây là hằng số kế thừa từ sinusoidal (theo ghi chú llm 12); đổi cơ số làm đổi chu kỳ của các cặp và vì vậy đổi cách mô hình xử lý ngữ cảnh dài, nhưng ảnh hưởng này không được paper RoPE thử nghiệm.
- **Quy ước tần số.** Có tài liệu đánh chỉ số cặp từ 1 và viết $\theta_i = 10000^{-2(i-1)/d}$; đó là cùng một dãy tần số với chỉ số $i = 0, \ldots$ như ở đây. Cặp cuối có chu kỳ $2\pi \cdot 10000^{62/64} \approx 47\,000$ khi $d = 64$ (tự tính), còn nếu viết gọn $\theta_{d/2} \approx 10000^{-1}$ thì chu kỳ xấp xỉ $62\,800$; hai cách viết khác nhau do d hữu hạn.
- **Vị trí trong KV cache.** Vì điểm số chỉ cần hiệu vị trí, khoá có thể được quay một lần khi tính rồi lưu vào cache (xoay theo vị trí tuyệt đối của chính nó); truy vấn mới quay theo vị trí hiện tại, và tích vô hướng tự động chứa hiệu vị trí.
- **Tính đối xứng.** Với RoPE, điểm số không đối xứng theo $n - m$ (ví dụ $-0{,}598$ và $4{,}598$ cho ± 2); với ALiBi bản nhân quả chỉ định nghĩa $i \geq j$, nên điều này không xuất hiện.

8. Nối sang các bài paper của cuốn

- **llm 12 (RoFormer, RoPE):** nội dung mục 4. Bài đó chứng minh $\tilde{q}_m \cdot \tilde{k}_n$ chỉ phụ thuộc $n - m$ và đo hiệu quả trên các bài toán cụ thể; ghi chú nhắc paper chưa thử ngoại suy.
- **llm 13 (ALiBi):** mục 5 và 6.3; bài đó đặt câu hỏi ngoại suy và so sánh bốn cách đặt vị trí. Trong ghi chú đó, thứ tự khả năng ngoại suy ở thí nghiệm WikiText-103 là sinusoidal < rotary < T5 bias < ALiBi.
- **llm 01 (Attention Is All You Need):** sinusoidal ở mục 3; chương này giải thích vì sao bài gốc phải thêm vector vị trí.
- **llm 05 (LLaMA):** dùng RoPE cùng với khối Pre-LN, RMSNorm và SwiGLU (nen02); chương này cho phần vị trí còn thiếu.
- **llm 55 (StreamingLLM) và các bài ngữ cảnh dài (llm 61 đến 73):** những bài này làm việc với chuỗi dài hơn độ dài huấn luyện hoặc chọn ra một tập con khoá; hiểu vì sao góc RoPE ở vị trí xa là "lạ" giúp đọc các phần về vị trí trong đó.
- **llm 16 (MLA):** bài tách riêng phần khoá mang RoPE vì RoPE phụ thuộc vị trí và cản trở việc nén khoá, theo mô tả trong ghi chú llm 16.

9. Sai lầm hay gặp

- Nghĩ rằng RoPE cộng một vector vị trí vào embedding. RoPE **quay** truy vấn và khoá, không cộng gì, và không quay giá trị.
- Nghĩ rằng vị trí tuyệt đối biến mất hoàn toàn. Mỗi vector được quay theo vị trí tuyệt đối của nó; chỉ có **điểm số** là phụ thuộc hiệu vị trí.
- Quên rằng khác hiệu có dấu cho điểm số khác nhau; $(n - m) = +2$ và -2 không cho cùng một giá trị.
- Nhầm độ dốc ALiBi của các đầu: đầu có độ dốc lớn nhìn gần, đầu có độ dốc nhỏ nhìn xa hơn.
- Kết luận từ perplexity phẳng rằng mô hình "hiểu" ngữ cảnh dài. Perplexity phẳng chỉ cho biết khả năng dự đoán không xấu đi; không chứng minh mô hình dùng được thông tin ở xa.
- Dùng RoPE ở độ dài lớn hơn nhiều so với lúc huấn luyện mà không thêm bước điều chỉnh (nội suy vị trí hoặc huấn luyện thêm) rồi ngạc nhiên khi chất lượng giảm.

10. Bảng tóm tắt

Phương pháp	Cách đưa vị trí vào	Công thức chính	Ghi chú
Sinusoidal	cộng p_m vào embedding	$p_{m,2i} = \sin(m\omega_i)$, $p_{m,2i+1} = \cos(m\omega_i)$	ngoại suy yếu (theo llm 13)
RoPE	quay q và k	$\tilde{q}_m \cdot \tilde{k}_n = q^\top R((n-m)\theta)k$	không thêm tham số
Tần số RoPE	cặp thứ i	$\theta_i = 10000^{-2i/d}$	chu kỳ từ 2π đến hàng chục nghìn
ALiBi	trừ điểm theo khoảng cách	$s_{ij} = q_i \cdot k_j/\sqrt{d_k} - m(i-j)$	độ dốc $m_h = 2^{-8h/n}$, không học
Ngoại suy	độ dài suy luận $>$ độ dài huấn luyện	RoPE cần can thiệp; ALiBi ổn định hơn	theo ghi chú llm 12, 13

11. Bài tập tự tính

Bài 1. Quay theo vị trí

Cho $\theta = 45°$ và $q = (1, 0)$. Tính $R(m\theta)q$ ở vị trí $m = 2$ và $m = 3$.

Đáp án (đã kiểm numpy): vị trí 2 quay $90°$: $(0,\ 1)$. Vị trí 3 quay $135°$: $(-0{,}707;\ 0{,}707)$.

Bài 2. Điểm số chỉ phụ thuộc hiệu vị trí

Cho $q = k = (1, 0)$, $\theta = 60°$. Tính điểm số $\tilde{q}_m \cdot \tilde{k}_n$ khi $n - m = 0, 1, 2, 3$, rồi kiểm tra cặp $(m, n) = (2, 5)$ và $(7, 10)$.

Đáp án (đã kiểm numpy): điểm số $= \cos\big((n-m)\theta\big)$ nên $n - m = 0, 1, 2, 3$ cho $1;\ 0{,}5;\ -0{,}5;\ -1$. Cặp $(2, 5)$ và $(7, 10)$ đều có $n - m = 3$ nên đều cho -1.

Bài 3. Công thức gọn của ví dụ tay

Dùng $q = (2, 1)$, $k = (1, 2)$, $\theta = 30°$ và điểm số $= 4\cos\alpha - 3\sin\alpha$ với $\alpha = (n - m)\theta$. Tính điểm số khi $(m, n) = (5, 2)$.

Đáp án (đã kiểm numpy): $n - m = -3$, $\alpha = -90°$, $4\cos(-90°) - 3\sin(-90°) = 0 + 3 = 3$.

Bài 4. Hàng ALiBi

Chuỗi năm token, truy vấn ở vị trí 5, điểm số nội dung bằng 0 cho mọi khoá, độ dốc $\frac{1}{4}$. Tính thiên lệch cho mỗi khoá và trọng số softmax. Thiên lệch giữa truy vấn 5 và khoá 2 với độ dốc $\frac{1}{256}$ là bao nhiêu?

Đáp án (đã kiểm numpy): thiên lệch $-(5 - j)/4 = (-1;\ -0{,}75;\ -0{,}5;\ -0{,}25;\ 0)$. Trọng số $(0{,}114;\ 0{,}146;\ 0{,}188;\ 0{,}241;\ 0{,}310)$. Với độ dốc $\frac{1}{256}$ và khoảng cách 3: $-3/256 \approx -0{,}0117$.

Bài 5. Chu kỳ và số vòng quay

Với $d = 64$ và cơ số 10000, tính θ_{20}, chu kỳ $2\pi/\theta_{20}$, và số vòng quay của cặp này ở độ dài $L = 512$ và $L = 4096$.

Đáp án (đã kiểm numpy): $\theta_{20} = 10000^{-40/64} = 10000^{-0{,}625} \approx 0{,}003162$ rad. Chu kỳ ≈ 1987 token. Ở $L = 512$: $512/1987 \approx 0{,}258$ vòng (chưa đủ một vòng). Ở $L = 4096$: $\approx 2{,}06$ vòng.

Tài liệu tham khảo nền: Vaswani và cộng sự, Attention Is All You Need; Su và cộng sự, RoFormer: Enhanced Transformer with Rotary Position Embedding; Press, Smith và Lewis, Train Short, Test Long: Attention with Linear Biases Enables Input Length Extrapolation; các ghi chú llm 01, 05, 12, 13 và 16 trong sách.

Nền tảng 04

Scaling laws và tính toán: đếm tham số, FLOPs và bộ nhớ trước khi huấn luyện

Loại: chương nền tảng (không phải ghi chú paper)

Đọc trước: vector và ma trận ở math nền tảng 02; hàm luỹ thừa và logarit (nhắc lại ngay trong chương); mạng nhiều lớp ở ML cơ bản nền tảng 05; Adam và bộ nhớ optimizer ở optimization nền 05, 08; nen01 đến nen03 của cuốn này

Dùng ở: llm 03 (luật luỹ thừa của Kaplan), llm 04 (Chinchilla, chia ngân sách giữa N và D), llm 05 (huấn luyện mô hình nhỏ với nhiều token)

> Chương này giải thích từ đầu, không phải bản dịch một paper. Các phép tính ghi "tự tính" đã kiểm bằng numpy; các đường trong hình là mô phỏng từ công thức, không phải số đo thật. Số lấy từ paper luôn ghi rõ "theo ghi chú llm 03" hoặc "theo ghi chú llm 04".

1. Vì sao cần khái niệm này

Trước khi huấn luyện một mô hình ngôn ngữ lớn, ta phải trả lời ba câu hỏi bằng số: mô hình có bao nhiêu tham số, huấn luyện tốn bao nhiêu phép tính, và cần bao nhiêu bộ nhớ. Ba câu hỏi này quyết định mua bao nhiêu GPU, chạy bao lâu, và quan trọng nhất: với ngân sách tính toán cho trước, nên làm mô hình lớn huấn luyện ít, hay mô hình nhỏ huấn luyện nhiều.

Chương dùng để đọc hai bài scaling laws (llm 03 và llm 04). Hai bài đó đầy công thức dạng $N^{-0{,}076}$ hay $C = 6ND$; nếu chưa từng tự đếm tham số một Transformer nhỏ, các con số ấy chỉ là chữ. Ở đây ta làm từng bước với cấu hình nhỏ đến mức tính tay được, rồi mới nhân lên quy mô lớn.

2. Nhắc nền cần thiết

2.1 Tham số, token, loss

Tham số là các số thực mà quá trình huấn luyện được phép thay đổi (trọng số các ma trận, vector chuẩn hoá). Tổng số tham số ký hiệu N. **Token** là đơn vị văn bản sau khi cắt thành mảnh; D là tổng số token mô hình đã đọc trong huấn luyện (một token được tính mỗi lần nó xuất hiện, nên đọc lại dữ liệu hai lần thì D tính hai lần). **Loss** L là cross-entropy trung bình trên mỗi token (xem math nền tảng 11): càng thấp càng tốt.

Ví dụ nhỏ (tự tính): mô hình có $N = 1000$ tham số đọc $D = 50$ token thì tích $ND = 50\,000$. Con số tích ND sẽ xuất hiện liên tục, vì mỗi token đi qua mỗi tham số một vài lần.

2.2 FLOP và FLOPs

FLOP là một phép cộng hoặc nhân số thực. Một phép nhân-cộng $a \cdot b + c$ tính là 2 FLOP. **FLOPs** (viết hoa, số nhiều) trong chương này nghĩa là *tổng số FLOP* của một việc, không phải tốc độ. Tốc độ phần cứng ghi là FLOP/s. Ví dụ: nhân vector độ dài d với ma trận $d \times d$ cần d^2 phép nhân và khoảng d^2 phép cộng, tức $2d^2$ FLOP.

2.3 Luật luỹ thừa và trục log-log

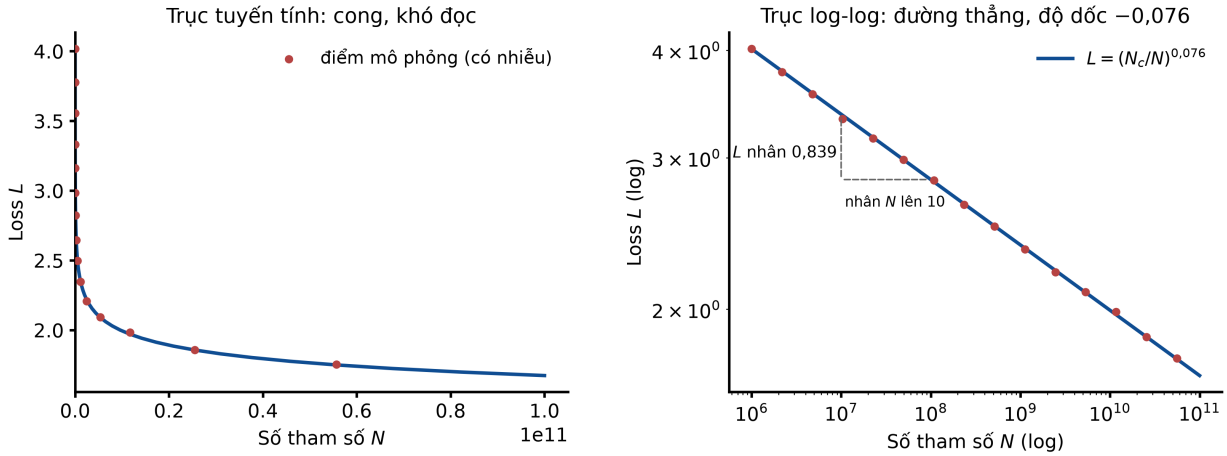
Hàm **luỹ thừa** có dạng $y = c\,x^a$. Lấy logarit hai vế: $\log y = \log c + a \log x$. Vậy trên giấy mà cả hai trục là logarit, hàm luỹ thừa là một **đường thẳng có độ dốc** a. Đó là lý do mọi hình scaling laws đều vẽ log-log.

Ví dụ (tự tính): $y = x^{-0{,}5}$. Khi x nhân 100, y nhân $100^{-0{,}5} = 0{,}1$. Mỗi lần nhân x với cùng một hệ số, y cũng bị nhân với một hệ số cố định: đó là đặc trưng của luỹ thừa.

2.4 Byte mỗi số

Một số float32 chiếm 4 byte, bfloat16 (bf16) hoặc float16 chiếm 2 byte, int8 chiếm 1 byte (xem optimization nền 08). Bộ nhớ của N tham số lưu bf16 là $2N$ byte.

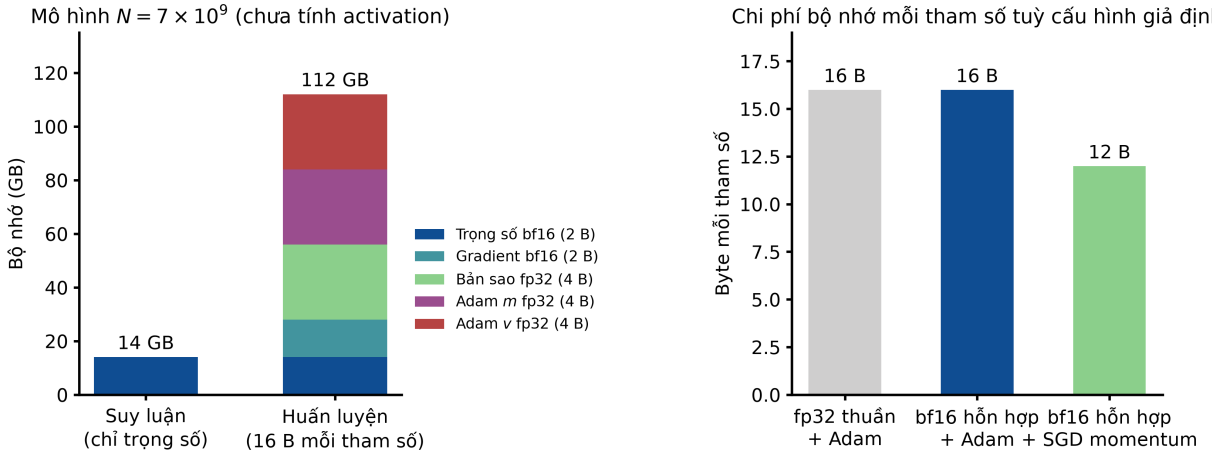
3. Trực giác bằng hình trước



Hình 13: Luật luỹ thừa trên trục tuyến tính và log-log (minh hoạ, tự tính)

Cách đọc hình: bên trái và bên phải là **cùng một hàm** $L = (N_c/N)^{0{,}076}$ với hằng số $N_c = 8{,}8 \times 10^{13}$ lấy từ ghi chú llm 03, cộng chút nhiễu mô phỏng (seed cố định). Trục tuyến tính ép toàn bộ đường cong vào sát trục dọc, không thể nhìn ra quy luật. Trục log-log cho một đường thẳng; tam giác nét đứt cho thấy nhân N lên 10 làm L nhân với $10^{-0{,}076} \approx 0{,}839$ (tự tính), nghĩa là loss giảm khoảng 16% mỗi mười lần số tham số. Đường thẳng này là mô phỏng từ công thức, không phải đo trên mô hình thật.

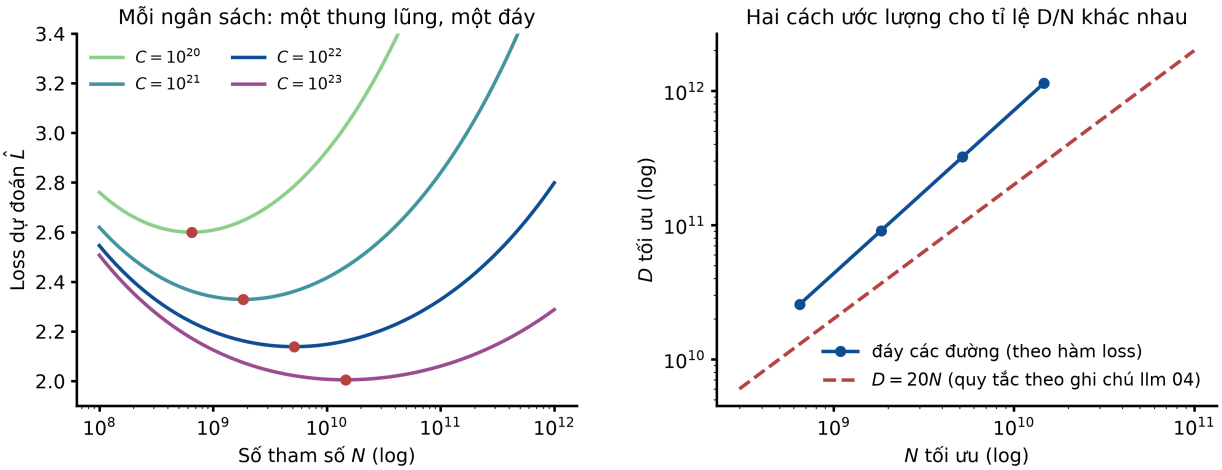
Một lưu ý quan trọng: loss giảm đều đặn theo luỹ thừa **không có nghĩa** là mô hình giỏi hơn đều đặn trên mọi tác vụ. Nó chỉ nói về cross-entropy trung bình.



Hình 14: Phân rã bộ nhớ khi huấn luyện mô hình 7 tỷ tham số (minh hoạ, tự tính)

Cách đọc hình: cột trái so sánh bộ nhớ chỉ lưu trọng số (suy luận, 14 GB) với bộ nhớ khi huấn luyện (112 GB, chưa tính activation). Cột xếp chồng chia 16 byte mỗi tham số thành năm phần. Bên phải so sánh ba cấu hình giả định; các con số là cấu hình tiêu biểu được chọn để minh hoạ, thực tế khác nhau tuỳ thư viện.

Cách đọc hình: mỗi đường màu là một ngân sách C cố định; trên đường đó, ta thử mọi N và để $D = C/(6N)$



Hình 15: Đường đẳng ngân sách (isoFLOP) và đáy của chúng (minh hoạ, tự tính)

tự điều chỉnh sao cho tích $6ND$ không đổi. Loss dự đoán rơi xuống một đáy rồi tăng lại: N quá nhỏ thì mô hình thiếu sức chứa, N quá lớn thì không đủ token để huấn luyện. Chấm đỏ là đáy. Bên phải vẽ các đáy trên trục log-log cùng đường $D = 20N$; hàm loss dùng ở đây là hàm tham số của ghi chú llm 04, và đáy của nó nằm ở tỉ lệ D/N lớn hơn 20, khác một chút với quy tắc thực hành (mục 8 giải thích).

4. Đếm tham số một Transformer

4.1 Một khối Transformer có những ma trận nào

Xem bài llm 01 cho khối đầy đủ. Ở đây ta chỉ cần danh sách ma trận, kiểu decoder, với chiều ẩn d (gọi là d_{model}) và chiều trong của FFN là d_{ff}:

- **Attention:** bốn ma trận $d \times d$: W_Q, W_K, W_V, W_O. Tổng $4d^2$ tham số (khi số đầu nhân chiều mỗi đầu bằng d và mọi đầu có K, V riêng; nếu dùng GQA/MQA thì W_K, W_V nhỏ hơn, xem nen05 của cuốn này).
- **FFN hai tầng:** $W_1 \in \mathbb{R}^{d \times d_{\text{ff}}}$ và $W_2 \in \mathbb{R}^{d_{\text{ff}} \times d}$. Tổng $2\,d\,d_{\text{ff}}$.
- **Chuẩn hoá (LayerNorm hoặc RMSNorm) và bias:** vài vector độ dài d, bé so với các ma trận; trong chương ta bỏ qua (nên con số là xấp xỉ).

Với quy ước kinh điển $d_{\text{ff}} = 4d$: tham số mỗi lớp là $4d^2 + 8d^2 = 12d^2$. Đây là suy luận từ cấu trúc, không phải số của một paper cụ thể.

4.2 Embedding

Bảng embedding có V hàng (từ vựng) và d cột: Vd tham số. Nếu lớp cuối (chiếu ra logit) **dùng chung** bảng này (tied embedding) thì không đếm lần hai; nếu không dùng chung thì cộng thêm Vd.

4.3 Công thức tổng

$$N \;\approx\; L\left(4d^2 + 2\,d\,d_{\text{ff}}\right) + Vd \;\overset{d_{\text{ff}}=4d}{=}\; 12\,L\,d^2 + Vd.$$

Ở mô hình lớn, số hạng $12Ld^2$ áp đảo, nên người ta thường nói "$N \approx 12Ld^2$". Chú ý bài Chinchilla (theo ghi chú llm 04) đếm N **gồm cả embedding**, còn bài Kaplan (llm 03) đếm không gồm; khi đối chiếu hai bài phải nhớ khác biệt này.

5. Ví dụ tay: đếm tham số và FLOPs một cấu hình nhỏ

Cấu hình giả định: $d = 64$, số lớp $L = 4$, $d_{\text{ff}} = 256$, từ vựng $V = 1000$, embedding dùng chung với lớp cuối.

Bước 1, attention mỗi lớp. $4d^2 = 4 \cdot 64^2 = 4 \cdot 4096 = 16\,384$ (tự tính).

Bước 2, FFN mỗi lớp. $2\,d\,d_{\text{ff}} = 2 \cdot 64 \cdot 256 = 32\,768$ (tự tính).

Bước 3, tổng mỗi lớp. $16\,384 + 32\,768 = 49\,152$ (tự tính); kiểm tra nhanh $12d^2 = 12 \cdot 4096 = 49\,152$, khớp.

Bước 4, bốn lớp. $4 \cdot 49\,152 = 196\,608$ (tự tính).

Bước 5, embedding. $Vd = 1000 \cdot 64 = 64\,000$ (tự tính).

Bước 6, tổng. $N = 196\,608 + 64\,000 = 260\,608$ tham số (tự tính, bỏ norm và bias).

Bước 7, FLOPs huấn luyện (công thức ở mục 6) cho $D = 10^6$ token: $6ND = 6 \cdot 260\,608 \cdot 10^6 \approx 1{,}56 \times 10^{12}$ FLOP (tự tính). Một GPU làm 10^{14} FLOP/s thực tế chỉ cần vài giây lý thuyết cho việc này (con số hiệu suất thật còn thấp hơn đỉnh, đây chỉ là cỡ độ lớn).

Bước 8, bộ nhớ. Mỗi tham số 16 byte (mục 7): $16 \cdot 260\,608 \approx 4{,}17$ MB (tự tính). Chỉ trọng số bf16: $2 \cdot 260\,608 = 521\,216$ byte, khoảng $0{,}52$ MB.

Nhận xét: ở cỡ nhỏ này embedding chiếm $64\,000/260\,608 \approx 24{,}6\%$ tổng (tự tính). Với mô hình hàng tỷ tham số tỉ lệ ấy nhỏ hơn nhiều, vì $12Ld^2$ lớn lên theo d^2 còn Vd chỉ theo d.

6. FLOPs huấn luyện: vì sao xấp xỉ $6ND$

6.1 Một lượt forward

Phần lớn tính toán nằm ở các phép nhân ma trận với vector hoặc ma trận (xem math nền tảng 02). Mỗi tham số của một ma trận tham gia đúng một phép nhân và một phép cộng cho mỗi token. Vậy forward tốn khoảng $2N$ FLOP **mỗi token** (nhân và cộng).

6.2 Backward gấp đôi forward

Backprop (math nền tảng 07, ML cơ bản nền tảng 06) tính hai thứ cho mỗi ma trận: gradient theo đầu vào (để truyền ngược) và gradient theo chính ma trận (để cập nhật). Mỗi thứ tốn chừng bằng một forward. Vậy backward khoảng $4N$ FLOP mỗi token.

6.3 Tổng

$$C \approx \underbrace{2N}_{\text{forward}} \cdot D + \underbrace{4N}_{\text{backward}} \cdot D = 6\,N\,D$$

Bài Chinchilla (theo ghi chú llm 04) tính chi tiết hơn (kể cả embedding và attention theo độ dài), nhận hệ số 2 cho nhân-cộng và backward gấp 2 lần forward, nhưng thấy chênh lệch so với $6ND$ rất nhỏ nên dùng $6ND$ trong suy diễn.

Bỏ gì khi dùng $6ND$: phần attention $QK^\top$ tốn thêm khoảng $O(n \cdot d)$ mỗi token tuỳ độ dài ngữ cảnh n; với n nhỏ hơn nhiều d thì không đáng kể, với ngữ cảnh rất dài thì có thể đáng kể (xem nen06). Phép tính cũng bỏ qua tái tính activation (gradient checkpointing, thêm khoảng một forward nữa) và hiệu suất phần cứng (MFU, tỉ lệ FLOP dùng được so với đỉnh).

6.4 Suy luận cũng theo $2N$

Sinh một token tốn khoảng $2N$ FLOP (chỉ forward). Đây là lý do suy luận nghiêng về chi phí bộ nhớ hơn tính toán, đề tài của nen05.

7. Bộ nhớ: trọng số, gradient, optimizer

Với Adam trong huấn luyện hỗn hợp (một cấu hình phổ biến, dùng làm ví dụ; bài tối ưu có nói về bản sao fp32, xem optimization nền 05 và 08):

Thành phần	Kiểu số	Byte/tham số
Trọng số dùng để tính (forward, backward)	bf16	2
Gradient	bf16	2
Bản sao chính của trọng số	fp32	4
Moment bậc một m của Adam	fp32	4
Moment bậc hai v của Adam	fp32	4
Tổng		**16**

Ví dụ (tự tính): $N = 7 \times 10^9$. Suy luận chỉ lưu trọng số bf16: $2 \cdot 7 \times 10^9 = 14 \times 10^9$ byte $= 14$ GB. Huấn luyện: $16 \cdot 7 \times 10^9 = 112$ GB, chưa tính **activation** (giá trị trung gian lưu lại cho backward, phụ thuộc batch và độ dài). Một GPU 80 GB không chứa nổi 112 GB, nên phải chia trạng thái ra nhiều GPU (ZeRO, FSDP...). Con số 16 byte là *giả định* về cấu hình; fp32 thuần với Adam cũng là $4+4+4+4 = 16$, còn SGD momentum với bản sao fp32 là $2 + 2 + 4 + 4 = 12$.

Hệ quả: bộ nhớ huấn luyện gấp khoảng 8 lần bộ nhớ suy luận (tự tính: $16/2 = 8$) cho cùng N. Đó cũng là một lý do người ta nghiên cứu các optimizer ít trạng thái (xem optimization 12 cho Adafactor).

8. Scaling laws: luật luỹ thừa và Chinchilla

8.1 Luật luỹ thừa của Kaplan

Theo ghi chú llm 03: khi dữ liệu đủ lớn, loss giảm theo N như

$$L(N) = \left(\frac{N_c}{N}\right)^{\alpha_N}, \qquad \alpha_N \approx 0{,}076, \quad N_c \approx 8{,}8 \times 10^{13}.$$

Cũng theo ghi chú đó, $\alpha_D \approx 0{,}095$ cho dữ liệu khi mô hình đủ lớn, và các hằng số fit đồng thời hơi khác các hằng số riêng lẻ (ghi chú nêu $\alpha_D \approx 0{,}103$ cho fit chung). Ý nghĩa: nhân đôi N làm L nhân $2^{-0{,}076} \approx 0{,}949$ (tự tính), tức giảm khoảng $5{,}1\%$. Vì mũ nhỏ, muốn giảm loss 10% cần nhân N lên $0{,}9^{-1/0{,}076} \approx 4{,}0$ lần (tự tính).

Điểm cần nhớ: đây là **quan hệ thực nghiệm** đo trên một dải kích thước và một họ mô hình; ngoại suy ra xa là giả thuyết, không phải định luật vật lý.

8.2 Bài toán chia ngân sách

Ngân sách C cố định, $C \approx 6ND$. Chọn N lớn thì $D = C/(6N)$ nhỏ, và ngược lại. Bài Chinchilla đặt câu hỏi: với C cho trước, cặp (N, D) nào cho loss thấp nhất?

Theo ghi chú llm 04, bài này dùng ba cách ước lượng độc lập và cả ba cho $N_{\text{opt}} \propto C^a$, $D_{\text{opt}} \propto C^b$ với a và b gần $0{,}5$ (cách 1: $0{,}50$ và $0{,}50$; cách 2: $0{,}49$ và $0{,}51$; cách 3: $0{,}46$ và $0{,}54$; Kaplan: $0{,}73$ và $0{,}27$). Nghĩa là khi ngân sách tăng gấp bốn, cả N và D nên tăng gấp đôi.

8.3 Hàm loss tham số

Cách 3 của bài Chinchilla mô hình hoá

$$\hat{L}(N, D) = E + \frac{A}{N^\alpha} + \frac{B}{D^\beta},$$

với $E = 1{,}69$, $A = 406{,}4$, $B = 410{,}7$, $\alpha = 0{,}34$, $\beta = 0{,}28$ (theo ghi chú llm 04). Đọc từng số hạng: E là sàn không thể xuống thấp hơn (entropy của văn bản), A/N^{α} là phần mất vì mô hình hữu hạn, B/D^{β} là phần mất vì dữ liệu hữu hạn. Đường đẳng ngân sách trong hình 3 là hàm này với $D = C/(6N)$.

8.4 Quy tắc thực hành $D \approx 20N$ và ví dụ tính tay

Theo ghi chú llm 04, tỉ lệ khoảng 20 token mỗi tham số **không** được paper phát biểu như quy tắc, mà được rút ra từ bảng các cặp tối ưu của cách 1 (ví dụ dòng 400 triệu tham số, $1{,}92 \times 10^{19}$ FLOPs, 8,0 tỷ token, tỉ lệ 20), và đó là mốc xấp xỉ chứ không phải hằng số phổ quát. Với quy tắc này và $C = 6ND$:

$$C = 6N \cdot 20N = 120\,N^2 \;\Rightarrow\; N = \sqrt{\frac{C}{120}}, \qquad D = 20N.$$

Ví dụ tính tay (theo ghi chú llm 04 cho quy tắc 20; số còn lại tự tính). Ngân sách $C = 1{,}2 \times 10^{22}$ FLOP.

1. $C/120 = 10^{20}$.
2. $N = \sqrt{10^{20}} = 10^{10}$, tức 10 tỷ tham số.
3. $D = 20N = 2 \times 10^{11}$, tức 200 tỷ token.
4. Kiểm tra: $6ND = 6 \cdot 10^{10} \cdot 2 \times 10^{11} = 1{,}2 \times 10^{22}$. Khớp.

Kiểm tra với ví dụ của chính paper theo ghi chú llm 04: $C = 5{,}76 \times 10^{23}$ cho $N \approx 6{,}9 \times 10^{10}$ và $D \approx 1{,}39 \times 10^{12}$, khớp Chinchilla 70B, 1,4 nghìn tỷ token.

Hệ quả tỉ lệ: vì $N \propto \sqrt{C}$, ngân sách tăng 100 lần thì N tăng 10 lần và D cũng tăng 10 lần (tự tính).

Chú ý về hình thứ ba. Nếu lấy hàm loss tham số ở mục 8.3 và tìm đáy số học trên mỗi đường đẳng ngân sách, đáy nằm ở D/N cỡ 40 đến 78 tuỳ ngân sách (mô phỏng, tự tính: ngân sách 10^{20} cho khoảng 40, ngân sách 10^{23} cho khoảng 78), tức nhiều token hơn quy tắc 20. Điều này nhất quán với ghi chú llm 04: cách 3 cho mô hình tối ưu nhỏ hơn hai cách còn lại ở compute lớn, và các hằng số được làm tròn ở đây không phải fit đầy đủ. Đây là *suy luận của chương* từ các hằng số làm tròn, không phải khẳng định của paper; mục đích là nhắc rằng "20 token mỗi tham số" là mốc xấp xỉ, không phải điểm chính xác.

9. Biến thể và chi tiết hay bị bỏ qua

Tối ưu cho huấn luyện khác với tối ưu cho phục vụ. Chinchilla tối ưu chi phí huấn luyện. Nếu mô hình sẽ được suy luận rất nhiều lần, một mô hình nhỏ hơn huấn luyện với nhiều token hơn mức 20 mỗi tham số có thể rẻ hơn về tổng chi phí vòng đời. Ghi chú llm 04 nêu điều này (chi phí phục vụ có thể làm thay đổi lựa chọn); hướng này giải thích vì sao llm 05 huấn luyện mô hình nhỏ với nhiều token.

Tham số "hoạt động" khác tham số tổng. Với mô hình MoE (llm 27, 28), mỗi token chỉ đi qua một phần chuyên gia; FLOPs mỗi token tính theo tham số hoạt động, còn bộ nhớ tính theo tổng. Công thức $6ND$ khi đó dùng N hoạt động.

Lặp lại dữ liệu. D đếm token đã thấy; đọc lại cùng dữ liệu nhiều epoch không cho thông tin mới tương đương token mới. Các luật trong llm 03 và llm 04 xét chủ yếu trường hợp dữ liệu ít lặp.

Hằng số phụ thuộc môi trường. Các hằng số E, A, B, α, β fit trên một tập dữ liệu và một tokenizer cụ thể (ghi chú llm 04 nêu MassiveText với tokenizer Gopher). Đổi tokenizer làm đổi đơn vị của loss; so sánh con số giữa các bài phải cẩn thận.

Loss thấp không bảo đảm năng lực. Các luật mô tả cross-entropy; năng lực trên một tác vụ cụ thể có thể nhảy hoặc phẳng.

10. Nối sang các bài paper của cuốn

- **llm 03 (Kaplan):** dùng đúng khái niệm luỹ thừa của mục 2.3 và 8.1. Các hằng số $\alpha_N = 0{,}076$, $\alpha_D = 0{,}095$ trong hình 1 là lấy từ ghi chú này. Bài còn nói $N_{\text{opt}} \propto C^{0{,}73}$, tức nên tăng mô hình nhanh hơn dữ liệu.
- **llm 04 (Chinchilla):** phản biện kết luận trên: N và D nên tăng cùng tỉ lệ (mục 8.2). Hàm loss tham số ở mục 8.3 và ví dụ 70B với 1,4 nghìn tỷ token ở mục 8.4 đều từ ghi chú này.
- **llm 05 (LLaMA):** đặt vấn đề huấn luyện mô hình nhỏ hơn với nhiều token hơn để phục vụ rẻ hơn; liên hệ với mục 9. Chương này không trích số của bài llm 05.
- **Các bài sau về suy luận (llm 14 đến 16, 46):** chi phí $2N$ mỗi token ở mục 6.4 là nền cho chương nen05.

11. Sai lầm hay gặp

- Đọc $C = 6ND$ như thể C là thời gian. C là tổng số FLOP; thời gian còn phụ thuộc số GPU và hiệu suất sử dụng, thường thấp hơn đỉnh nhiều.
- Quên rằng N có hai quy ước (có hoặc không tính embedding), làm các con số của hai bài không so sánh trực tiếp được.
- Tính bộ nhớ huấn luyện chỉ bằng $2N$ byte (chỉ trọng số). Gradient và trạng thái optimizer thường chiếm phần lớn, và activation còn nằm ngoài con số 16 byte.
- Xem "20 token mỗi tham số" là định luật. Đó là mốc xấp xỉ rút từ bảng thực nghiệm, phụ thuộc điều kiện huấn luyện.
- Ngoại suy đường thẳng log-log ra ngoài dải đã đo rồi trình bày như dự đoán chắc chắn.
- Lẫn "tham số tổng" với "tham số hoạt động" ở mô hình MoE khi tính FLOPs.

12. Tóm tắt công thức

Đại lượng	Công thức
Tham số mỗi lớp (FFN $4d$)	$12d^2$
Tham số tổng xấp xỉ	$12Ld^2 + Vd$
FLOPs forward mỗi token	$\approx 2N$
FLOPs huấn luyện tổng	$C \approx 6ND$
Bộ nhớ huấn luyện (Adam hỗn hợp, giả định)	$\approx 16N$ byte (chưa activation)
Bộ nhớ suy luận (bf16)	$2N$ byte
Luật luỹ thừa	$L = (N_c/N)^{\alpha}$, log-log là đường thẳng
Hàm loss tham số (llm 04)	$\hat{L} = E + A/N^{\alpha} + B/D^{\beta}$
Quy tắc thực hành (llm 04)	$D \approx 20N$, $N = \sqrt{C/120}$

13. Bài tập tự tính

Bài 1. Một Transformer có $d = 32$, $L = 2$, $d_{\text{ff}} = 128$, từ vựng $V = 500$, embedding dùng chung. Bỏ qua norm và bias. Tính N.

Đáp án: attention mỗi lớp $4 \cdot 32^2 = 4096$; FFN $2 \cdot 32 \cdot 128 = 8192$; tổng $12\,288$ mỗi lớp; hai lớp $24\,576$; embedding $500 \cdot 32 = 16\,000$; $N = 40\,576$ (tự tính).

Bài 2. Huấn luyện mô hình $N = 10^9$ trên $D = 2 \times 10^{10}$ token. Tính FLOPs theo $6ND$.

Đáp án: $6 \cdot 10^9 \cdot 2 \times 10^{10} = 1{,}2 \times 10^{20}$ FLOP (tự tính).

Bài 3. Mô hình $N = 10^9$: bộ nhớ trọng số bf16 và bộ nhớ huấn luyện ở 16 byte mỗi tham số (giả định như mục 7)?

Đáp án: $2 \cdot 10^9 = 2$ GB cho trọng số; $16 \cdot 10^9 = 16$ GB cho trọng số, gradient, bản sao fp32 và hai moment Adam, chưa tính activation (tự tính).

Bài 4. Với luật $L \propto N^{-0{,}076}$, muốn loss giảm 10% (tức L nhân $0{,}9$) thì nhân N lên bao nhiêu lần?

Đáp án: giải $x^{-0{,}076} = 0{,}9$, ra $x = 0{,}9^{-1/0{,}076} \approx 4{,}0$ (tự tính, so với nhân đôi N chỉ đạt hệ số $0{,}949$).

Bài 5. Theo quy tắc $D \approx 20N$ (ghi chú llm 04), ngân sách $C = 4{,}8 \times 10^{22}$ FLOP cho N và D bằng bao nhiêu? Nếu ngân sách tăng 100 lần thì N tăng bao nhiêu lần?

Đáp án: $C/120 = 4 \times 10^{20}$, $N = \sqrt{4 \times 10^{20}} = 2 \times 10^{10}$ (20 tỷ), $D = 4 \times 10^{11}$ (400 tỷ); kiểm tra $6 \cdot 2 \times 10^{10} \cdot 4 \times 10^{11} = 4{,}8 \times 10^{22}$. Tăng 100 lần thì N tăng $\sqrt{100} = 10$ lần (tự tính).

Tài liệu tham khảo nền: ghi chú llm 03 (scaling laws Kaplan), ghi chú llm 04 (Chinchilla), math nền tảng 11 (cross-entropy), optimization nền 05 và 08 (Adam, độ chính xác số), nen05 và nen06 của cuốn này.

Nền tảng 05

KV cache và chi phí suy luận: vì sao sinh từng token bị chặn bởi bộ nhớ

Loại: chương nền tảng (không phải ghi chú paper)

Đọc trước: nhân ma trận ở math nền tảng 02; attention và khối Transformer ở llm 01; đếm tham số và FLOPs ở nen04 của cuốn này; số byte mỗi kiểu số ở optimization nền 08

Dùng ở: llm 14 (MQA), llm 15 (GQA), llm 16 (MLA), llm 46 (PagedAttention), llm 47 (dùng lại tiền tố KV)

> Chương này giải thích từ đầu, không phải bản dịch một paper. Các phép tính ghi "tự tính" đã kiểm bằng numpy. Cấu hình mô hình và tham số phần cứng trong ví dụ và hình đều là **giả định để minh hoạ**, không phải số đo thật; số lấy từ paper ghi "theo ghi chú llm NN".

1. Vì sao cần khái niệm này

KV cache là bộ nhớ lưu lại key và value của các token đã xử lý để lần sinh token sau không phải tính lại. Nó đổi bộ nhớ lấy thời gian tính, và gần như mọi hệ thống phục vụ LLM đều dùng. Nhưng chính bộ nhớ ấy trở thành nút cổ chai: cache lớn theo độ dài và theo số yêu cầu chạy đồng thời, và mỗi bước sinh phải đọc lại toàn bộ.

Cả năm bài của dải suy luận (llm 14 đến 16, 46, 47) đều là các cách giảm kích thước KV cache hoặc dùng nó hiệu quả hơn: chia sẻ K, V giữa các đầu (MQA, GQA), nén thành vector tiềm ẩn (MLA), cấp phát theo khối (PagedAttention), dùng lại tiền tố chung (RadixAttention). Không có hiểu biết về cache và về giới hạn băng thông, năm bài này chỉ là năm mẹo rời rạc.

2. Nhắc nền cần thiết

2.1 Attention nhân quả cho một query

Xem llm 01 cho công thức đầy đủ. Với một token ở vị trí t trong mô hình decoder: ta có query q_t, và các key $k_1, \dots, k_t$, value $v_1, \dots, v_t$ của mọi token từ đầu đến t (mặt nạ nhân quả chặn các token sau). Đầu ra là trung bình có trọng số của value, trọng số là softmax của $q_t \cdot k_j$ (chia $\sqrt{d_h}$):

$$o_t = \sum_{j \le t} \mathrm{softmax}_j\left(\frac{q_t \cdot k_j}{\sqrt{d_h}}\right) v_j .$$

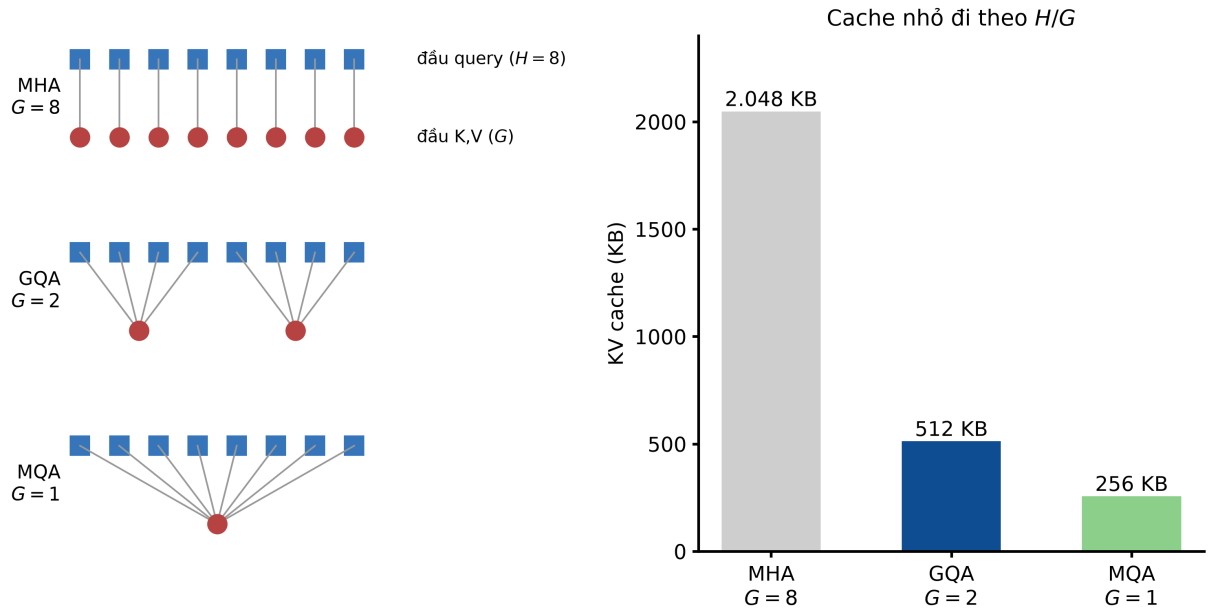
Điều quan trọng cho chương này: k_j và v_j của token j **chỉ phụ thuộc token j và các token trước nó**, không phụ thuộc token sinh sau. Một khi đã tính, chúng không đổi.

2.2 Nhiều đầu và nhiều lớp

Một lớp attention có H đầu; mỗi đầu có q, k, v chiều d_h (thường $H \cdot d_h = d_{\text{model}}$). Có L lớp xếp chồng, và mỗi lớp có bộ K, V riêng. Vì vậy một token để lại $L \times H$ cặp (key, value) nếu mọi đầu đều có K, V riêng.

2.3 Byte mỗi số

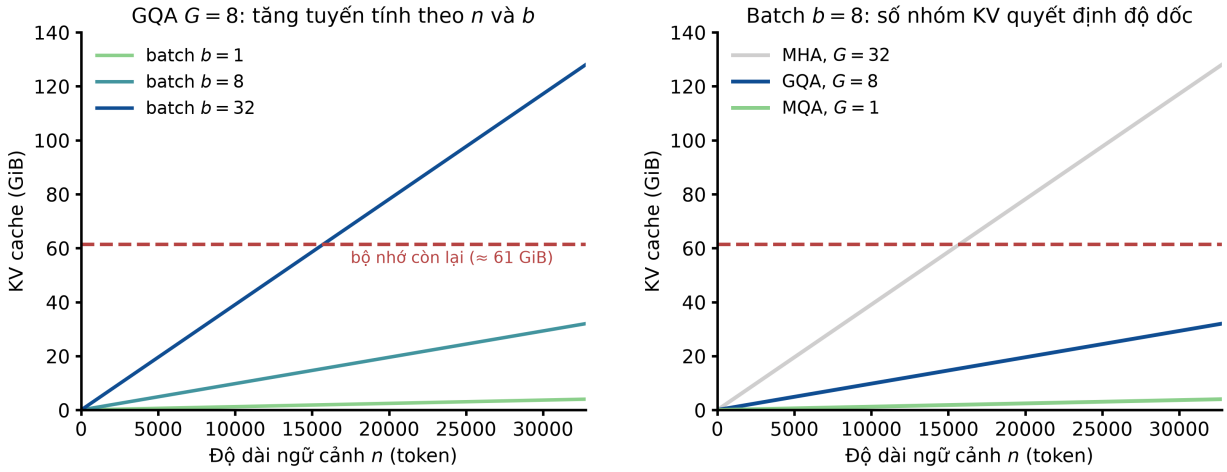
Lưu bf16 hay fp16: $s = 2$ byte mỗi phần tử. Ví dụ (tự tính): một vector $d_h = 16$ bf16 chiếm $16 \cdot 2 = 32$ byte.



Hình 16: So sánh số nhóm KV của MHA, GQA, MQA (minh hoạ, tự tính)

3. Trực giác bằng hình trước

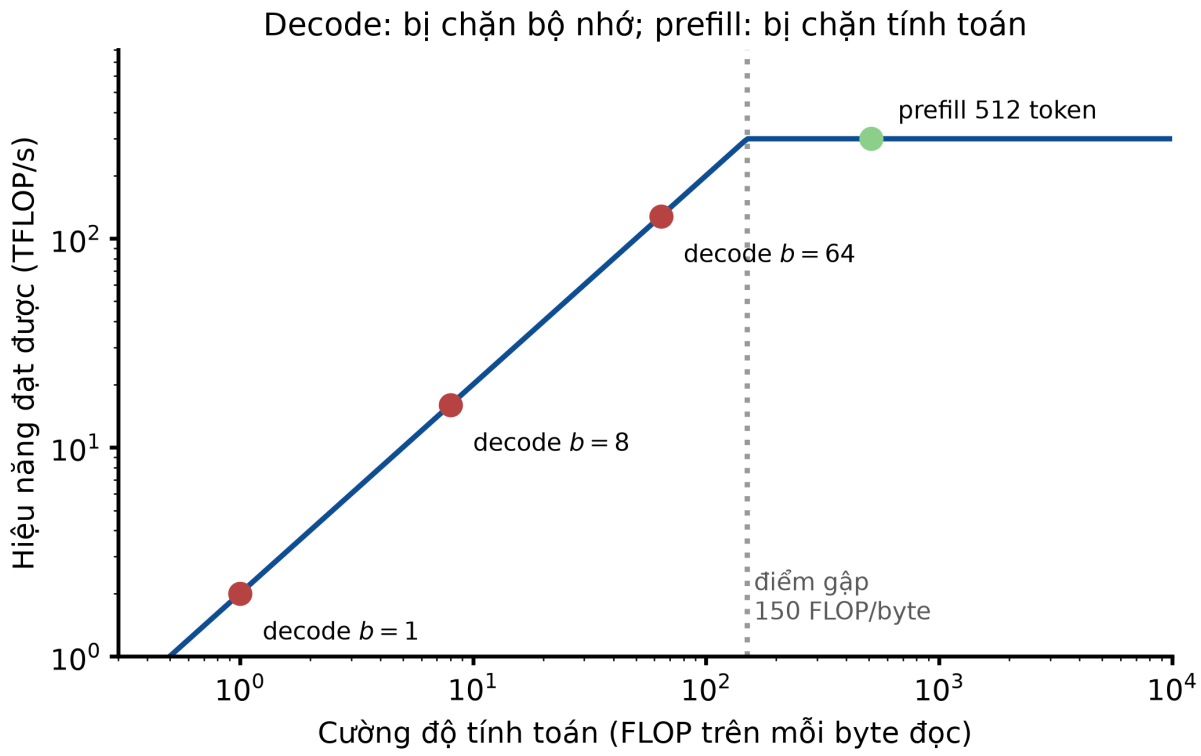
Cách đọc hình: ô vuông xanh là 8 đầu query, chấm đỏ là các đầu K,V thực sự được lưu. Hàng MHA: mỗi query có K,V riêng (8 nhóm). Hàng GQA: các query chia thành 2 nhóm, mỗi nhóm dùng chung một cặp K,V. Hàng MQA: tất cả 8 query dùng chung một cặp. Cột bên phải là KV cache tính cho cấu hình giả định $L = 4$, $H = 8$, $d_h = 16$, $n = 1000$, bf16, một chuỗi: $2\,048\,000$, $512\,000$ và $256\,000$ byte (tự tính, xem mục 6). Số query không đổi, chỉ số K,V cần lưu thay đổi.



Hình 17: KV cache theo độ dài ngữ cảnh và batch (minh hoạ, tự tính)

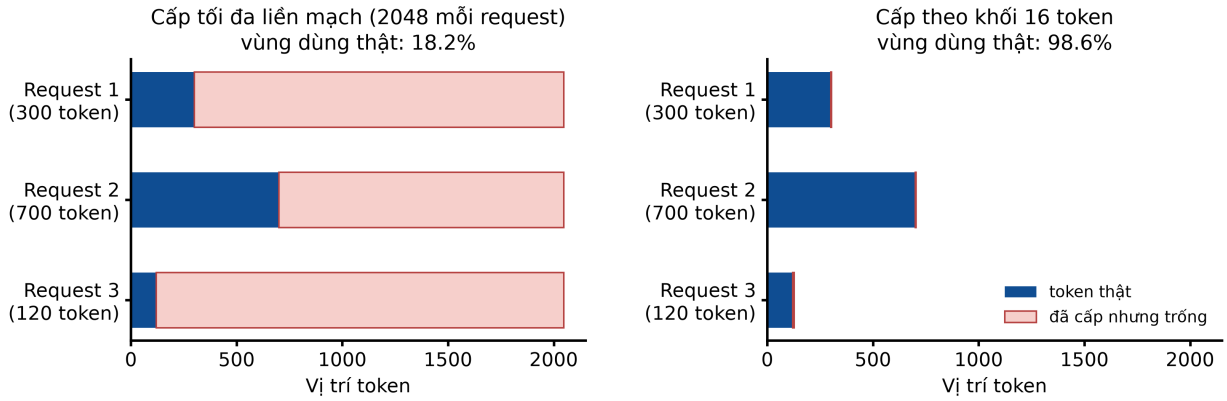
Cách đọc hình: cấu hình giả định $L = 32$, $d_h = 128$, bf16, trọng số 14 GB, GPU 80 GB (nên còn khoảng 61 GiB cho cache; tất cả là giả định). Trái: với GQA 8 nhóm, cache tăng **tuyến tính** theo n và theo batch b; batch 32 chạm trần bộ nhớ ở khoảng $n \approx 15\,700$. Phải: cùng batch 8, MHA (32 nhóm) vượt trần ở n cỡ $16\,000$, GQA và MQA thì còn rất xa. Số nhóm KV quyết định độ dốc.

Cách đọc hình: đường xanh là "mái nhà" của hiệu năng đạt được: bên trái nó tăng theo cường độ tính toán (bị băng thông giới hạn), bên phải phẳng (bị đỉnh tính toán giới hạn). Giả định: đỉnh 300 TFLOP/s, băng thông 2



Hình 18: Roofline đơn giản: prefill và decode (minh hoạ, tự tính)

TB/s, nên điểm gập ở 150 FLOP/byte. Ba chấm đỏ là decode với batch 1, 8, 64: cả ba nằm ở sườn dốc, nghĩa là máy chờ bộ nhớ chứ không bận tính. Chấm xanh là prefill 512 token: nằm ở phần phẳng, bị tính toán giới hạn. Mục 5 giải thích vì sao.



Hình 19: Cấp phát liền mạch tối đa so với cấp theo khối (minh hoạ, tự tính)

Cách đọc hình: ba request có độ dài thật 300, 700 và 120 token, độ dài tối đa 2048. Bên trái cấp liền mạch 2048 vị trí cho mỗi request: chỉ khoảng 18,2% dùng thật. Bên phải cấp theo khối 16 token: chỉ lãng phí phần lẻ cuối mỗi request, dùng thật khoảng 98,6% (cả hai tính ở mục 8.1). Đây là ví dụ số minh hoạ, không phải đo hệ thống.

4. Prefill và decode

Phục vụ một yêu cầu có hai pha, có tính chất rất khác nhau:

- **Prefill:** xử lý cả prompt (giả sử n_0 token) **một lần song song**. Các token của prompt đã biết hết, nên mọi q, k, v tính cùng lúc bằng các phép nhân ma trận lớn. Kết quả: ta có K, V của cả prompt, lưu vào cache, và có token đầu tiên của câu trả lời.
- **Decode:** sinh từng token một, **tuần tự**, vì token $t+1$ chỉ biết sau khi sinh xong token t. Mỗi bước chỉ tính q, k, v cho **một** token mới, thêm k, v mới vào cache, rồi attend lên toàn bộ cache.

Không có cache, mỗi bước decode phải chạy lại cả chuỗi dài từ đầu để có K, V của mọi token: chi phí mỗi bước tăng theo độ dài và tổng chi phí cả câu tăng như bình phương. Có cache, mỗi bước chỉ tính cho một token mới. Đó chính là món hời "đổi bộ nhớ lấy tính toán".

Ví dụ tay, đếm phép tính chiều K (suy luận của chương, bỏ hằng số). Câu trả lời dài T token. Không cache: bước t tính lại K cho t token, tổng $1 + 2 + \cdots + T = T(T+1)/2$ lần tính K cho một token; với $T = 100$ là 5050 (tự tính). Có cache: mỗi bước tính 1 lần, tổng 100. Tỉ lệ $50{,}5$ lần. Cái giá: phải giữ 100 cặp (K, V) mỗi lớp trong bộ nhớ.

5. Vì sao decode bị chặn bởi bộ nhớ

5.1 Cường độ tính toán

Cường độ tính toán (arithmetic intensity) là số FLOP thực hiện chia cho số byte phải đọc từ bộ nhớ GPU. Phần cứng có hai giới hạn: đỉnh tính toán P (FLOP/s) và băng thông B (byte/s). Thời gian tối thiểu của một việc là lớn hơn giữa FLOP$/P$ và byte$/B$. Tỉ số P/B chính là **điểm gập** của roofline: việc có cường độ thấp hơn điểm gập thì bị băng thông chặn.

5.2 Decode có cường độ rất thấp

Mỗi bước decode của một chuỗi: phải đọc **toàn bộ trọng số** ($2N$ byte ở bf16) và toàn bộ KV cache của chuỗi đó, chỉ để làm khoảng $2N$ FLOP cho một token (xem nen04, mục 6.4). Nếu phục vụ b chuỗi cùng lúc, trọng số chỉ đọc **một lần** cho cả batch, còn FLOP nhân b:

$$\text{cường độ} \approx \frac{2N \cdot b}{2N} = b \ \text{ FLOP/byte},$$

khi bỏ qua việc đọc KV (KV chỉ làm cường độ thấp hơn nữa, vì mỗi chuỗi có KV riêng không chia sẻ được). Với giả định $P = 300$ TFLOP/s và $B = 2$ TB/s, điểm gập ở 150 FLOP/byte; decode với batch dưới khoảng 150 nằm ở sườn dốc của hình roofline, tức bị băng thông chặn. Các con số P, B là giả định cỡ độ lớn, chứ không phải của một GPU cụ thể.

5.3 Ví dụ tay: thời gian một bước decode (giả định)

Mô hình $N = 7 \times 10^9$ bf16 (14 GB), băng thông 2 TB/s, đỉnh 300 TFLOP/s, mọi con số giả định.

1. **Thời gian đọc trọng số:** $14 \times 10^9 / (2 \times 10^{12}) = 7 \times 10^{-3}$ s $= 7$ ms (tự tính).
2. **Thời gian tính cho batch 1:** $2N = 1{,}4 \times 10^{10}$ FLOP chia 3×10^{14} FLOP/s $\approx 4{,}7 \times 10^{-5}$ s $\approx 0{,}047$ ms (tự tính). Nhỏ hơn thời gian đọc khoảng 150 lần.
3. **Thêm KV cache** của batch 8, mỗi chuỗi dài 4096, GQA 8 nhóm (cấu hình ở mục 6): 4 GiB, đọc mất $4\,294\,967\,296/(2 \times 10^{12}) \approx 2{,}15$ ms (tự tính).
4. **Tổng một bước cho batch 8:** khoảng $7 + 2{,}15 = 9{,}15$ ms, tức khoảng $8/0{,}00915 \approx 875$ token/giây tổng (tự tính, mô hình rất đơn giản: bỏ qua activation, độ trễ kernel, giao tiếp).

Bài học: thời gian bị thống trị bởi **đọc bộ nhớ**. Muốn nhanh hơn ta hoặc đọc ít byte hơn (cache nhỏ hơn, lượng tử hoá, xem llm 51 đến 54), hoặc dùng mỗi byte đọc cho nhiều token hơn (batch lớn, speculative decoding, xem llm 48 đến 50). Batch lớn bị chặn bởi chính KV cache: đó là mối nối giữa chương này và llm 46.

5.4 Prefill khác hẳn

Prefill xử lý n_0 token cùng lúc với cùng một lượt đọc trọng số, nên cường độ khoảng n_0 FLOP/byte; với prompt 512 token vượt điểm gập 150 (giả định), nên bị tính toán chặn. Vì thế prefill tận dụng GPU tốt, còn decode thì không. (Suy luận của chương, đủ cho cỡ độ lớn; prompt rất dài còn chịu thêm chi phí attention $O(n^2)$, xem nen06.)

6. Định nghĩa chính thức: kích thước KV cache

Gọi L số lớp, G số **nhóm KV** (số đầu K,V thực sự lưu mỗi lớp; MHA có $G = H$), d_h chiều mỗi đầu, n độ dài đã lưu, b số chuỗi (batch), s số byte mỗi phần tử:

$$\boxed{\text{KV bytes} = 2 \cdot L \cdot G \cdot d_h \cdot n \cdot b \cdot s}$$

Hệ số 2 là một cho K và một cho V. Công thức này trùng với công thức trong ghi chú llm 14 và llm 15. Cache **tăng tuyến tính** theo L, G, d_h, n, b.

Cache mỗi token (một chuỗi): $2 \cdot L \cdot G \cdot d_h \cdot s$ byte.

6.1 Ví dụ tay: một bước decode có cache

Xét một đầu, $d_h = 2$, bỏ hệ số $1/\sqrt{d_h}$ để số gọn. Cache đang giữ hai token đã xử lý:

- $k_1 = (1, 0)$, $v_1 = (2, 0)$;
- $k_2 = (0, 1)$, $v_2 = (0, 4)$.

Token mới (vị trí 3) cho $q_3 = (1, 1)$, $k_3 = (1, 1)$, $v_3 = (1, 1)$.

Bước 1, thêm vào cache. Cache nay có ba cặp. Bước này chỉ *ghi* k_3, v_3, không tính lại k_1, k_2, v_1, v_2.

Bước 2, điểm. $q_3 \cdot k_1 = 1$, $q_3 \cdot k_2 = 1$, $q_3 \cdot k_3 = 2$.

Bước 3, softmax. $e^1 = 2{,}718$, $e^1 = 2{,}718$, $e^2 = 7{,}389$; tổng $12{,}825$; trọng số $(0{,}2119;\ 0{,}2119;\ 0{,}5761)$ (tự tính).

Bước 4, trung bình value. $0{,}2119 \cdot (2, 0) + 0{,}2119 \cdot (0, 4) + 0{,}5761 \cdot (1, 1) = (0{,}4238 + 0{,}5761,\ 0{,}8476 + 0{,}5761) \approx (1{,}0;\ 1{,}424)$ (tự tính).

Nhận xét: bước 2 đến 4 phải **đọc cả ba key và ba value** (toàn bộ cache), còn chỉ tính ba tích vô hướng nhỏ. Với chuỗi dài hàng nghìn token và nhiều đầu, nhiều lớp, đó là lượng đọc khổng lồ so với lượng tính. Cùng một lượt như vậy diễn ra ở mọi lớp và mọi đầu.

Hai độ đo thường gặp khi nói về hai pha (định nghĩa chuẩn trong phục vụ LLM): thời gian tới token đầu tiên (TTFT) chủ yếu là chi phí prefill; thời gian mỗi token sau đó (TPOT) chủ yếu là chi phí decode, tức một lần đọc trọng số và cache cho mỗi token.

7. Ví dụ tay: MHA, GQA, MQA

Cấu hình giả định nhỏ: $L = 4$ lớp, $H = 8$ đầu query, $d_h = 16$, độ dài $n = 1000$, batch $b = 1$, bf16 ($s = 2$).

MHA ($G = 8$). $2 \cdot 4 \cdot 8 \cdot 16 \cdot 1000 \cdot 1 \cdot 2$. Nhân dần: $2 \cdot 4 = 8$; $8 \cdot 8 = 64$; $64 \cdot 16 = 1024$; $1024 \cdot 1000 = 1\,024\,000$; nhân 2 byte: $2\,048\,000$ byte (tự tính).

GQA với $G = 2$. Chỉ thay 8 bằng 2: $2 \cdot 4 \cdot 2 \cdot 16 \cdot 1000 \cdot 2 = 512\,000$ byte (tự tính), nhỏ hơn MHA $H/G = 4$ lần.

MQA ($G = 1$). $2 \cdot 4 \cdot 1 \cdot 16 \cdot 1000 \cdot 2 = 256\,000$ byte (tự tính), nhỏ hơn MHA 8 lần.

Cấu hình giả định "cỡ thật hơn": $L = 32$, $d_h = 128$, $n = 4096$, $b = 8$, bf16.

Kiểu	G	KV cache (GiB, tự tính)
MHA	32	16
GQA	8	4
MQA	1	0,5

Ví dụ: GQA, $2{\cdot}32{\cdot}8{\cdot}128{\cdot}4096{\cdot}8{\cdot}2 = 4\,294\,967\,296$ byte $= 4$ GiB. Mỗi token của một chuỗi: $2{\cdot}32{\cdot}8{\cdot}128{\cdot}2 = 131\,072$ byte $= 128$ KiB. Mô hình chạy chuỗi 4096 token thì cache của riêng chuỗi ấy $0{,}5$ GiB.

Cái giá của việc chia sẻ. MQA ép mọi đầu query nhìn vào cùng một bộ K,V, nên mô hình mất đa dạng và có thể giảm chất lượng; GQA giữ vài nhóm để cân bằng. Chi tiết chất lượng (số liệu từ paper) nằm ở ghi chú llm 14 và llm 15; chương này chỉ cho thấy phía bộ nhớ. Theo ghi chú llm 15, GQA còn có công thức chuyển một checkpoint MHA có sẵn bằng mean-pooling các đầu K,V rồi huấn luyện tiếp một lượng nhỏ.

8. Biến thể và chi tiết hay bị bỏ qua

8.1 PagedAttention ở mức khái niệm

Hệ thống cũ cấp cho mỗi yêu cầu **một khối liền mạch** đủ cho độ dài tối đa, vì không biết trước câu trả lời dài bao nhiêu. Phần chưa dùng thành lãng phí. **PagedAttention** mượn ý tưởng bộ nhớ ảo của hệ điều hành: cắt cache thành các **khối cố định** (ví dụ 16 token), cấp khối **khi cần**, và dùng một **bảng khối** ánh xạ vị trí logic sang vị trí vật lý, nên các khối không cần nằm liền nhau. Chi tiết và số liệu ở ghi chú llm 46.

Ví dụ tính tay cho hình 4 (tự tính): ba request dài 300, 700, 120 token. Cấp tối đa 2048 mỗi cái: dùng thật $(300 + 700 + 120)/(3 \cdot 2048) = 1120/6144 \approx 18{,}2\%$. Cấp khối 16: cần $\lceil 300/16 \rceil = 19$, $\lceil 700/16 \rceil = 44$, $\lceil 120/16 \rceil = 8$ khối, tức $304 + 704 + 128 = 1136$ vị trí; dùng thật $1120/1136 \approx 98{,}6\%$. Lãng phí chỉ còn phần lẻ, tối đa dưới một khối mỗi request. Theo ghi chú llm 46, các hệ thống trước chỉ có khoảng $20{,}4\%$ đến $38{,}2\%$ bộ nhớ KV chứa token thật (số đo của paper, không phải của ví dụ trên).

Vì ánh xạ gián tiếp, nhiều chuỗi có thể trỏ **cùng một khối vật lý** khi dùng chung tiền tố (prompt hệ thống chung, beam search), kèm cơ chế sao chép khi ghi. Ý này mở rộng thành cây tiền tố ở llm 47.

8.2 MLA: nén tiềm ẩn

Thay vì lưu K và V cho từng nhóm, **MLA** lưu **một vector tiềm ẩn nhỏ** cho mỗi token (nén từ trạng thái ẩn), và khôi phục K, V cho từng đầu khi cần bằng các ma trận chiếu lên. Trực giác: K và V của các đầu khác nhau chứa nhiều thông tin chồng lấn, nên nén được. Theo ghi chú llm 16, cache mỗi token chỉ còn $(d_c + d_h^R) \cdot l$ phần tử (với l là số lớp; d_c chiều vector nén; d_h^R phần nhỏ riêng cho RoPE), tương đương GQA với $2{,}25$ nhóm. Ví dụ giả định (tự tính, chọn số khớp với tỉ lệ đó): $d_c = 512$, $d_h^R = 64$, $l = 32$, bf16: $(512 + 64) \cdot 32 \cdot 2 = 36\,864$ byte mỗi token; so với GQA 8 nhóm ở mục 7 (131 072 byte mỗi token) thì nhỏ hơn khoảng $3{,}56$ lần. Điểm khác MQA và GQA: MLA không giảm số "đầu" mà nén biểu diễn, nên vẫn còn H đầu với ma trận chiếu riêng.

8.3 Việc không giảm cache nhưng giảm đọc

Nhiều kỹ thuật không làm cache nhỏ hơn mà làm mỗi bước **đọc ít hơn** (attention thưa, cắt token ít quan trọng, xem llm 55 đến 57 và nen06) hoặc đọc bằng kiểu số nhỏ hơn (cache lượng tử hoá). Cùng mục tiêu: giảm byte trên mỗi token sinh.

8.4 Những thứ khác chiếm bộ nhớ

Ngoài cache còn có trọng số, activation tạm và bộ đệm của framework. Ghi chú llm 46 cho ví dụ mô hình 13B trên A100 40GB với khoảng 65% bộ nhớ là trọng số và gần 30% là KV cache, phần còn lại là activation; con số đó là ví dụ của paper, còn tỉ lệ trong hệ thống của bạn tuỳ cấu hình.

8.5 Ví dụ tay: bao nhiêu chuỗi chạy được cùng lúc

Ghép các mảnh của chương. Giả định: GPU 80 GB (tức 80×10^9 byte), trọng số 14 GB, bỏ qua activation và bộ đệm khác, mỗi chuỗi dài 4096 token, mô hình $L = 32$, $d_h = 128$, bf16.

1. Bộ nhớ còn lại cho cache: $80 - 14 = 66$ GB $= 66 \times 10^9$ byte.
2. Cache mỗi chuỗi với GQA 8 nhóm: $131\,072 \cdot 4096 = 536\,870\,912$ byte.
3. Số chuỗi: $66 \times 10^9 / 536\,870\,912 \approx 122{,}9$, tức tối đa 122 chuỗi (tự tính).
4. Với MHA ($G = 32$), cache mỗi chuỗi gấp 4 lần ($2\,147\,483\,648$ byte): $66 \times 10^9 / 2\,147\,483\,648 \approx 30{,}7$, tức tối đa 30 chuỗi (tự tính).

Cùng một GPU, đổi MHA sang GQA 8 nhóm cho chạy gần 4 lần chuỗi đồng thời. Theo mục 5.2 (giả định điểm gập 150), batch 122 đã tiến sát điểm gập, nên thông lượng decode tăng gần tuyến tính theo batch cho đến lúc đó; đó là lý do thu nhỏ cache chuyển trực tiếp thành thông lượng. Đây là mô hình đơn giản hoá để thấy chiều của hiệu ứng, không phải dự đoán số đo cho hệ thống thật.

9. Nối sang các bài paper của cuốn

- **llm 14 (MQA):** ý tưởng nền: giảm KV bằng cách mọi đầu query dùng chung một K, V. Chương này cho thấy vì sao quan trọng: cache nhỏ H lần và việc đọc cache mỗi bước nhẹ hơn. Cường độ tính toán ở mục 5 là khái niệm mà bài này gọi là arithmetic intensity.
- **llm 15 (GQA):** nội suy giữa MHA và MQA bằng G nhóm, kích thước cache tỉ lệ G/H so với MHA (mục 6 và 7).
- **llm 16 (MLA):** nén tiềm ẩn thay cho chia sẻ (mục 8.2).
- **llm 46 (PagedAttention):** giải quyết lãng phí và phân mảnh khi cấp phát cache (mục 8.1), nhờ đó batch lớn hơn và thông lượng cao hơn (theo ghi chú llm 46, 2 đến 4 lần ở cùng mức độ trễ).
- **llm 47 (RadixAttention):** chia sẻ khối cho tiền tố chung, mở rộng ý chia sẻ khối ở mục 8.1.

10. Sai lầm hay gặp

- Nghĩ KV cache làm **giảm** bộ nhớ. Nó **tăng** bộ nhớ để giảm tính toán lặp.
- Quên nhân hệ số 2 (K và V) hoặc quên nhân s byte mỗi phần tử, làm cache nhỏ đi 2 hoặc 4 lần so với thực tế.
- Dùng H (số đầu query) thay cho G (số nhóm KV) trong công thức cache ở GQA và MQA.
- Cho rằng decode chậm vì "tính nhiều". Thực ra decode tính ít và đọc nhiều; tăng sức tính của GPU không cứu được nếu băng thông không tăng.
- Tính thông lượng bằng cách nhân đơn giản số chuỗi, quên rằng tăng batch cũng tăng cache và cuối cùng chạm trần bộ nhớ.
- Nhầm prefill với decode: prefill song song và bị tính toán chặn, decode tuần tự và bị bộ nhớ chặn; tối ưu cho pha này có thể không giúp pha kia.

11. Tóm tắt công thức

Đại lượng	Công thức
KV cache tổng	$2\,L\,G\,d_h\,n\,b\,s$ byte
KV cache mỗi token	$2\,L\,G\,d_h\,s$ byte

Đại lượng	Công thức
Giảm so với MHA	H/G lần
Cường độ tính toán decode (bỏ KV)	$\approx b$ FLOP/byte
Điểm gập roofline	P/B FLOP/byte
Thời gian đọc	byte$/B$
MLA (theo llm 16)	$(d_c + d_h^R)\, l$ phần tử mỗi token
Số khối PagedAttention	$\lceil$độ dài$/$kích thước khối$\rceil$

12. Bài tập tự tính

Bài 1. Cấu hình $L = 2$, $G = 4$, $d_h = 32$, $n = 512$, $b = 1$, bf16. Tính KV cache theo byte.

Đáp án: $2 \cdot 2 \cdot 4 \cdot 32 \cdot 512 \cdot 1 \cdot 2 = 524\,288$ byte, đúng $0{,}5$ MiB (tự tính).

Bài 2. Mô hình $L = 32$, $G = 8$, $d_h = 128$, bf16. Cache mỗi token là bao nhiêu byte? Nếu dành 20 GiB cho cache và mỗi chuỗi dài 4096 token, chứa được bao nhiêu chuỗi? Nếu dùng MHA ($G = 32$) thì sao?

Đáp án: mỗi token $2 \cdot 32 \cdot 8 \cdot 128 \cdot 2 = 131\,072$ byte; mỗi chuỗi $131\,072 \cdot 4096 = 2^{29}$ byte $= 0{,}5$ GiB; $20/0{,}5 = 40$ chuỗi. MHA gấp 4 lần cache mỗi chuỗi nên chứa 10 chuỗi (tự tính).

Bài 3. Dùng giả định $P = 300$ TFLOP/s, $B = 2$ TB/s. Cường độ decode xấp xỉ b FLOP/byte. Batch tối thiểu để vượt điểm gập là bao nhiêu (bỏ qua KV)? Với $N = 7 \times 10^9$ bf16, thời gian đọc trọng số một bước là bao nhiêu?

Đáp án: điểm gập $300 \times 10^{12}/(2 \times 10^{12}) = 150$ nên cần $b \gtrsim 150$; thời gian đọc $14 \times 10^9/(2 \times 10^{12}) = 7$ ms (tự tính; với KV cache thực tế, điểm bão hoà còn cao hơn).

Bài 4. Một cấu hình MLA giả định có $d_c = 512$, $d_h^R = 64$, $l = 32$, bf16. Cache mỗi token là bao nhiêu byte và nhỏ hơn GQA 8 nhóm ($L = 32$, $d_h = 128$) bao nhiêu lần?

Đáp án: $(512 + 64) \cdot 32 \cdot 2 = 36\,864$ byte; GQA $131\,072$ byte; tỉ lệ $131\,072/36\,864 \approx 3{,}56$ (tự tính).

Bài 5. Ba request dài 100, 250 và 33 token. Cấp khối 16 token cần bao nhiêu khối, bao nhiêu vị trí được cấp, lãng phí bao nhiêu vị trí?

Đáp án: $\lceil 100/16 \rceil = 7$, $\lceil 250/16 \rceil = 16$, $\lceil 33/16 \rceil = 3$, tổng 26 khối $= 416$ vị trí; dùng thật 383 nên lãng phí $416 - 383 = 33$ vị trí (tự tính).

Tài liệu tham khảo nền: ghi chú llm 14 (MQA), llm 15 (GQA), llm 16 (MLA), llm 46 (PagedAttention), llm 47 (RadixAttention); nen04 và nen06 của cuốn này.

Nền tảng 06

Attention tuyến tính, attention thưa và SSM: đổi chi phí bậc hai lấy trạng thái cố định

Loại: chương nền tảng (không phải ghi chú paper)

Đọc trước: nhân ma trận và tính kết hợp ở math nền tảng 02; attention ở llm 01; KV cache và chi phí suy luận ở nen05 của cuốn này; đếm FLOPs ở nen04 của cuốn này

Dùng ở: llm 17 (cửa sổ trượt), llm 18 (attention tuyến tính), llm 19 (RetNet), llm 20 (GLA), llm 21 và llm 22 (Mamba), llm 23 đến llm 26 (DeltaNet và các mô hình lai), llm 58 đến llm 66 (attention thưa)

> Chương này giải thích từ đầu, không phải bản dịch một paper. Các phép tính ghi "tự tính" đã kiểm bằng numpy; các đường chi phí và đường truy hồi trong hình là mô phỏng từ công thức hoặc từ dữ liệu ngẫu nhiên có seed cố định, không phải số đo trên mô hình thật. Số lấy từ paper ghi "theo ghi chú llm NN".

1. Vì sao cần khái niệm này

Softmax attention so sánh mỗi token với mọi token trước nó, nên chi phí tăng bình phương theo độ dài. Khi ngữ cảnh từ vài nghìn lên hàng trăm nghìn token, chi phí ấy là điểm nghẽn. Hai nhánh lớn của nghiên cứu cố gắng vượt qua: giữ attention nhưng **bỏ bớt cặp** (attention thưa, cửa sổ trượt), hoặc **bỏ softmax** để viết lại attention thành một mạng hồi quy với trạng thái cố định (attention tuyến tính, SSM như Mamba).

Chương này là nền cho hai nhóm bài lớn nhất của cuốn: llm 17 đến 26 (từ cửa sổ trượt qua attention tuyến tính, RetNet, GLA, Mamba, DeltaNet, các mô hình lai) và llm 58 đến 66 (attention thưa hiện đại). Mọi bài trong các nhóm ấy trả lời cùng một câu hỏi theo cách khác nhau: bỏ cái gì, giữ cái gì, và mất gì.

2. Nhắc nền cần thiết

2.1 Attention nhân quả là một ma trận $n \times n$

Với chuỗi n token, $Q, K, V \in \mathbb{R}^{n \times d}$ (một đầu, chiều d). Ma trận điểm $S = QK^\top \in \mathbb{R}^{n \times n}$: ô (i, j) là độ liên quan của query i với key j. Nhân quả nghĩa là chỉ giữ $j \le i$. Softmax theo từng hàng cho ma trận trọng số A, rồi đầu ra là $O = AV$ (xem llm 01).

2.2 Tính kết hợp của phép nhân ma trận

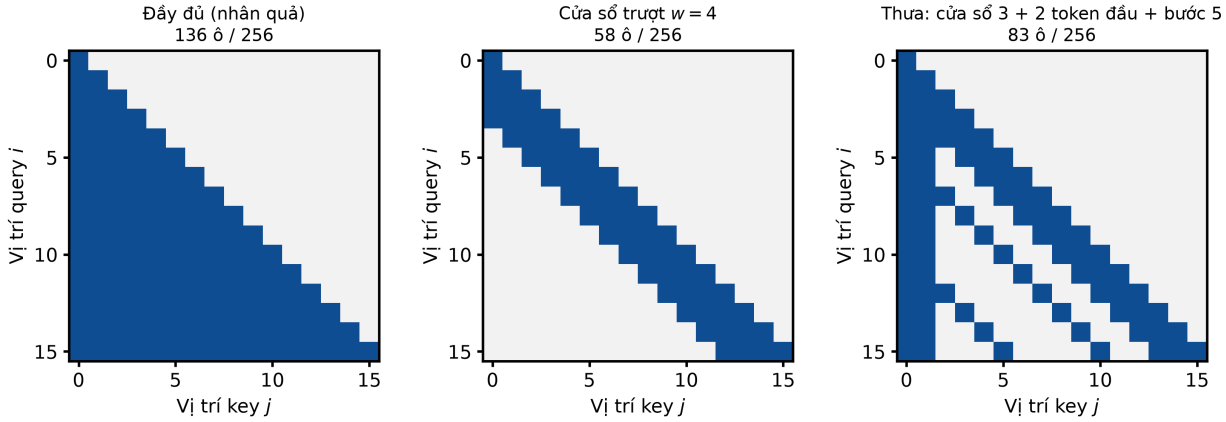
Với ba ma trận A, B, C có số chiều ghép được: $(AB)C = A(BC)$, **kết quả giống nhau nhưng chi phí có thể khác**. Ví dụ (tự tính): $A \in \mathbb{R}^{1000 \times 10}$, $B \in \mathbb{R}^{10 \times 1000}$, $C \in \mathbb{R}^{1000 \times 10}$. Tính $(AB)C$ tạo ma trận trung gian 1000×1000 (một triệu ô), tốn khoảng $1000 \cdot 10 \cdot 1000 + 1000 \cdot 1000 \cdot 10 = 2 \times 10^7$ phép nhân. Tính $A(BC)$ tạo ma trận trung gian 10×10, tốn khoảng $10 \cdot 1000 \cdot 10 + 1000 \cdot 10 \cdot 10 = 2 \times 10^5$ phép nhân, nhỏ hơn 100 lần. Toàn bộ "mánh" của attention tuyến tính nằm ở đây.

2.3 Trạng thái ẩn của một mạng hồi quy

Mạng hồi quy (RNN, xem ML cơ bản nền tảng 07) đọc từng token và giữ một **trạng thái** S_t có kích thước cố định, tóm tắt lịch sử. Bộ nhớ của nó không lớn lên theo t. Đổi lại, nó phải **nén** lịch sử vào một chỗ có sức chứa hữu hạn.

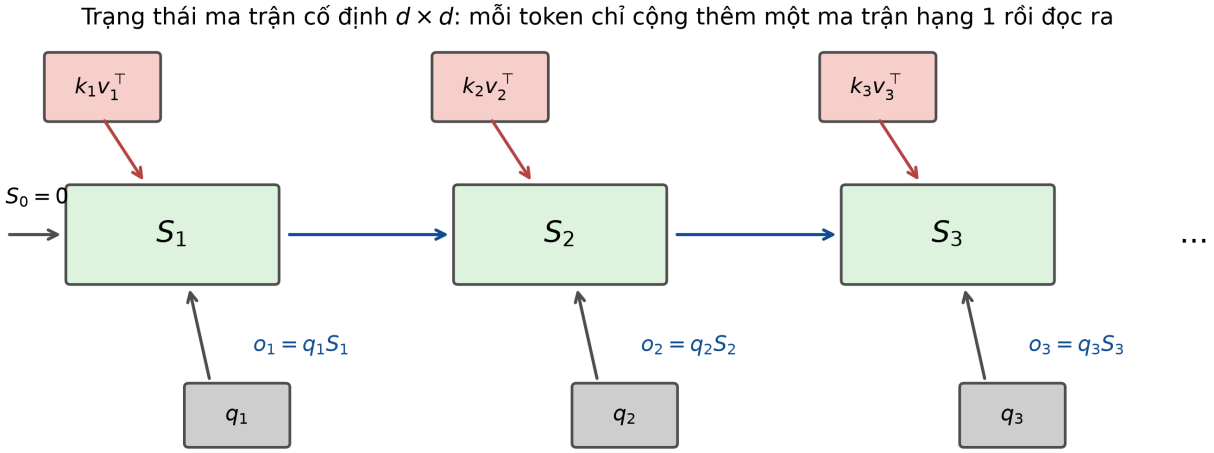
3. Trực giác bằng hình trước

Cách đọc hình: mỗi ô xanh (i, j) nghĩa là query i được phép nhìn key j; ô xám là bị chặn. Chuỗi $n = 16$. Bên trái, mặt nạ nhân quả đầy đủ có 136 ô ($16 \cdot 17/2$). Giữa, cửa sổ trượt $w = 4$ chỉ nhìn 4 token gần nhất



Hình 20: Mẫu mặt nạ attention: đầy đủ, cửa sổ trượt và thưa (minh hoạ, tự tính)

(kể cả chính nó): 58 ô. Bên phải, mẫu thưa gồm cửa sổ 3, hai token đầu tiên luôn được nhìn và các vị trí cách nhau bội của 5: 83 ô (cấu hình này do chương chọn để minh hoạ). Khi n lớn, số ô xanh của mặt nạ đầy đủ tăng như n^2, còn của cửa sổ chỉ tăng như $n \cdot w$.

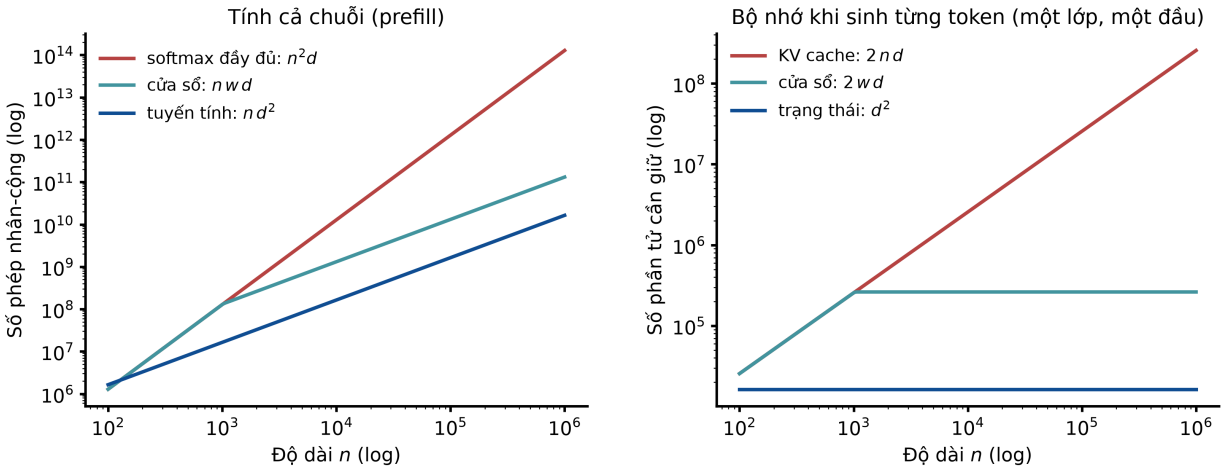


Hình 21: Sơ đồ trạng thái đệ quy của attention tuyến tính (minh hoạ, tự tính)

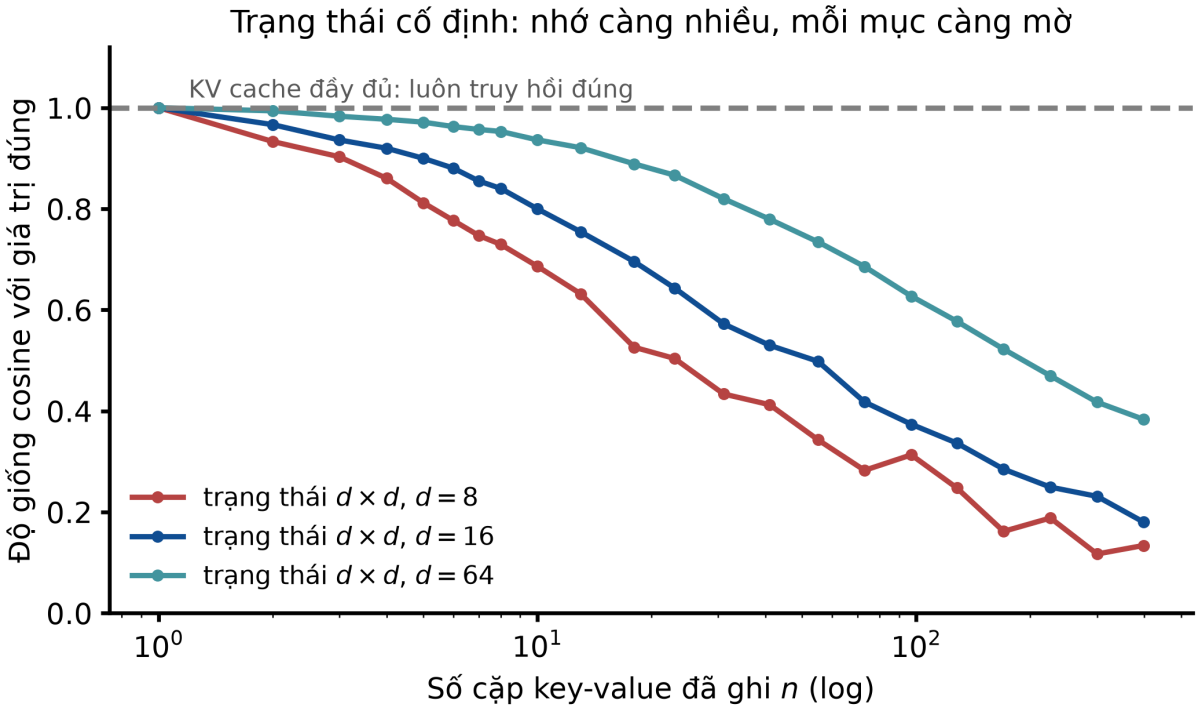
Cách đọc hình: đọc từ trái sang phải. Mỗi token đến cho một ma trận hạng một $k_t v_t^\top$ (ô đỏ), được **cộng** vào trạng thái S_{t-1} để thành S_t (ô xanh lục). Query q_t (ô xám) rồi **đọc** trạng thái này để cho đầu ra $o_t = q_t S_t$. Trạng thái luôn là ma trận $d \times d$ bất kể đã đọc bao nhiêu token. Đây là sơ đồ khái niệm cho dạng không cổng, không suy giảm; mục 6 thêm cổng và suy giảm.

Cách đọc hình: cả hai trục log-log nên luỹ thừa thành đường thẳng; độ dốc 2 nghĩa là bậc hai, độ dốc 1 là tuyến tính, độ dốc 0 là hằng. Tham số **giả định**: $d = 128$, cửa sổ $w = 1024$, bỏ mọi hằng số. Trái: số phép nhân-cộng khi xử lý cả chuỗi: softmax $n^2 d$, cửa sổ nwd (trùng softmax khi $n \le w$, sau đó bẻ gập thành tuyến tính), tuyến tính nd^2. Phải: số phần tử phải giữ khi sinh từng token: KV cache $2nd$ tăng theo n, cửa sổ chặn ở $2wd$, trạng thái ma trận hằng d^2. Đây là bậc của công thức, không phải thời gian đo.

Cách đọc hình: ta ghi n cặp (key, value) ngẫu nhiên vào trạng thái $S = \sum_j k_j v_j^\top$ rồi hỏi lại bằng key đầu tiên, và đo độ giống cosine giữa kết quả với giá trị đúng (mô phỏng 200 lần mỗi điểm, seed cố định, key chiều d chuẩn hoá). KV cache đầy đủ (đường nét đứt) luôn truy hồi đúng. Trạng thái ma trận càng nhiều cặp càng mờ, và mờ chậm hơn khi d lớn. Đây là hình ảnh trực quan của "đánh đổi giữa bộ nhớ cố định và khả năng nhớ" ở mục 8.



Hình 22: Chi phí theo độ dài: softmax, cửa sổ, tuyến tính (minh hoạ, tự tính)



Hình 23: Trạng thái cố định: nhớ càng nhiều cặp, truy hồi mỗi cặp càng mờ (minh hoạ, tự tính)

4. Vì sao softmax attention là $O(n^2)$

4.1 Đếm ô

Ma trận điểm nhân quả có $n(n+1)/2$ ô. Mỗi ô cần một tích vô hướng chiều d ($\approx d$ phép nhân-cộng), nên tính điểm tốn $\approx n^2 d/2$ và nhân với V tốn tương tự. Gộp lại, đến hằng số, chi phí cả chuỗi là $O(n^2 d)$.

Ví dụ tay (tự tính): $n = 4$ có $4 \cdot 5/2 = 10$ ô; $n = 8$ có 36 ô. Nhân đôi độ dài làm số ô nhân khoảng $3{,}6$ lần ở cỡ nhỏ và dần tiến đến 4 lần: $n = 1000$ có $500\,500$ ô, $n = 2000$ có $2\,001\,000$ ô, tỉ lệ $3{,}998$.

4.2 Hai chi phí khác nhau

- **Thời gian tính** khi prefill hoặc huấn luyện: $O(n^2 d)$.
- **Bộ nhớ** nếu lưu cả ma trận điểm: $O(n^2)$. Các cài đặt như FlashAttention (llm 44, 45) tránh lưu cả ma trận này nên bộ nhớ chỉ $O(n)$, nhưng **thời gian vẫn** $O(n^2)$.
- **Decode** (nen05): mỗi token mới cần so với toàn bộ cache t token: $O(td)$ mỗi bước và cache $O(nd)$ tăng theo n.

Vì lõi $O(n^2)$ nằm ở "mọi cặp", hai hướng thoát tự nhiên: **bớt cặp** (mục 5), hoặc **không xét từng cặp** mà gom lịch sử vào một trạng thái (mục 6 và 7).

5. Attention thưa và cửa sổ trượt

5.1 Cửa sổ trượt

Mỗi token chỉ nhìn w token gần nhất. Số ô: $\approx n\,w$, tuyến tính theo n khi w cố định. KV cache khi decode chỉ cần giữ w token gần nhất, bộ nhớ hằng. Câu hỏi hiển nhiên: thông tin ở xa hơn w thì sao? Vì các lớp xếp chồng, thông tin có thể truyền qua nhiều lớp: sau ℓ lớp, một token có thể bị ảnh hưởng bởi token cách xa tối đa khoảng $\ell \cdot w$ (đây là suy luận về trường nhìn, theo ghi chú llm 17 cho Longformer; việc có dùng được hiệu quả hay không tuỳ bài toán).

Ví dụ tay (tự tính): $n = 8$, $w = 3$. Hàng i có $\min(i+1, 3)$ ô: $1, 2, 3, 3, 3, 3, 3, 3$, tổng 21 ô; so với mặt nạ đầy đủ 36 ô. Tỉ lệ $21/36 \approx 58\%$. Khi n tăng thì tỉ lệ giảm: $n = 1000$, $w = 100$: khoảng $100\,000$ ô so với $500\,500$, còn khoảng 20%.

5.2 Token toàn cục và mẫu thưa cố định

Cửa sổ thuần mất khả năng tổng hợp toàn chuỗi. Longformer (theo ghi chú llm 17) thêm vài token **toàn cục** nhìn tất cả và được tất cả nhìn, số token này nhỏ và độc lập với n, nên tổng vẫn $O(n)$. Sparse Transformer (theo ghi chú llm 58) dùng các mẫu cố định (strided, fixed) để đạt $O(n\sqrt{n})$ mà mọi cặp vẫn liên thông qua vài chặng. Hình mặt nạ ở trên gộp ba thành phần: cửa sổ gần, token đầu tiên luôn thấy được, và các vị trí cách đều.

5.3 Thưa động

Mẫu cố định đặt trước thì không biết nội dung. Các bài sau (llm 61 đến 65 và 68 trở đi) chọn ô theo **nội dung**: dùng một bộ chọn rẻ để tìm các khối key quan trọng với từng query rồi chỉ tính attention trên đó. Chương này không đi vào chi tiết; chỉ cần nhớ hai điểm: (1) tính chất của mặt nạ (có nhân quả, có truy cập ngẫu nhiên hiệu quả trên phần cứng không) quyết định tốc độ thật nhiều hơn số ô; (2) bỏ ô là **xấp xỉ**, có thể bỏ sót thông tin quan trọng.

6. Attention tuyến tính: đổi thứ tự phép nhân

6.1 Ý tưởng

Softmax đặt hàm mũ lên từng $q_i \cdot k_j$ rồi chuẩn hoá, nên không tách được Q và K. Thay $\exp(q \cdot k)$ bằng một **kernel phân rã được** $\phi(q) \cdot \phi(k)$ với ϕ là hàm đặc trưng cho giá trị không âm (ghi chú llm 18 dùng

$\phi(x) = \text{elu}(x) + 1$). Khi đó (dạng không nhân quả):

$$o_i = \frac{\phi(q_i)^\top \left(\sum_j \phi(k_j)\, v_j^\top \right)}{\phi(q_i)^\top \sum_j \phi(k_j)}.$$

Ma trận $\sum_j \phi(k_j) v_j^\top \in \mathbb{R}^{d \times d}$ tính **một lần** và dùng chung cho mọi query. Nói bằng ma trận: thay $\big(\phi(Q)\phi(K)^\top\big)V$ (có ma trận $n \times n$) bằng $\phi(Q)\big(\phi(K)^\top V\big)$ (ma trận $d \times d$). Theo mục 2.2, chi phí từ $O(n^2 d)$ xuống $O(nd^2)$.

6.2 Dạng nhân quả là một mạng hồi quy

Với mặt nạ nhân quả, tổng chỉ chạy $j \le t$, nên đặt

$$S_t = S_{t-1} + \phi(k_t)\, v_t^\top, \qquad z_t = z_{t-1} + \phi(k_t), \qquad o_t = \frac{\phi(q_t)^\top S_t}{\phi(q_t)^\top z_t},$$

với $S_0 = 0$, $z_0 = 0$. Đây là sơ đồ ở hình hai. **Trạng thái S_t là ma trận** $d \times d$, mỗi bước cập nhật là một phép cộng ma trận hạng một. Theo ghi chú llm 18, đó là sự tương đương giữa Transformer nhân quả và RNN: huấn luyện chạy dạng song song (đủ nhanh trên GPU), suy luận chạy dạng hồi quy với chi phí và bộ nhớ hằng mỗi token.

6.3 Ví dụ tay: ba token, softmax và tuyến tính

Một đầu, $d = 2$, ba token. Để tính tay gọn, bỏ hệ số $1/\sqrt{d}$, và với tuyến tính chọn ϕ là hàm đồng nhất (hợp lệ ở đây vì mọi phần tử của q, k đều không âm, nên điểm không âm). Đây là lựa chọn đơn giản hoá để minh hoạ, không phải cấu hình thật (ghi chú llm 18 dùng elu $+1$).

- $q_1 = (1, 0)$, $q_2 = (0, 1)$, $q_3 = (1, 1)$
- $k_1 = (1, 0)$, $k_2 = (0, 1)$, $k_3 = (1, 1)$
- $v_1 = (1, 2)$, $v_2 = (3, 0)$, $v_3 = (0, 1)$

Điểm $q_i \cdot k_j$ (tự tính): hàng 1: $(1, 0, 1)$; hàng 2: $(0, 1, 1)$; hàng 3: $(1, 1, 2)$. Dùng mặt nạ nhân quả nên chỉ lấy $j \le i$.

Softmax, token 3. Điểm $(1, 1, 2)$, hàm mũ $(2{,}718;\ 2{,}718;\ 7{,}389)$, tổng $12{,}825$, trọng số $(0{,}212;\ 0{,}212;\ 0{,}576)$. Đầu ra $0{,}212 \cdot (1, 2) + 0{,}212 \cdot (3, 0) + 0{,}576 \cdot (0, 1) \approx (0{,}848;\ 1{,}000)$ (tự tính).

Tuyến tính, token 3. Điểm $(1, 1, 2)$, tổng 4, trọng số $(0{,}25;\ 0{,}25;\ 0{,}5)$. Đầu ra $0{,}25 \cdot (1, 2) + 0{,}25 \cdot (3, 0) + 0{,}5 \cdot (0, 1) = (1{,}0;\ 1{,}0)$ (tự tính). Trọng số tuyến tính **phẳng hơn** softmax: softmax khuếch đại chênh lệch điểm bằng hàm mũ, tuyến tính chỉ chuẩn hoá tỉ lệ. Đây là cách trực giác cho thấy "tập trung" kém hơn, một lý do vì sao attention tuyến tính thường cần thêm cơ chế (cổng, quy tắc delta ở llm 20, 23, 24) để chọn lọc.

Kiểm tra bằng dạng hồi quy. Chạy từng bước với ϕ đồng nhất:

t	cập nhật	S_t	z_t	$q_t S_t$	$q_t \cdot z_t$	o_t
1	$k_1 v_1^\top$	$\begin{pmatrix} 1 & 2 \\ 0 & 0 \end{pmatrix}$	$(1, 0)$	$(1, 2)$	1	$(1, 2)$
2	$k_2 v_2^\top$	$\begin{pmatrix} 1 & 2 \\ 3 & 0 \end{pmatrix}$	$(1, 1)$	$(3, 0)$	1	$(3, 0)$
3	$k_3 v_3^\top$	$\begin{pmatrix} 1 & 3 \\ 3 & 1 \end{pmatrix}$	$(2, 2)$	$(4, 4)$	4	$(1, 1)$

Bước $t=3$: $k_3 v_3^\top = \begin{pmatrix}1\\1\end{pmatrix}\begin{pmatrix}0 & 1\end{pmatrix} = \begin{pmatrix}0 & 1\\0 & 1\end{pmatrix}$, cộng vào S_2 cho $S_3 = \begin{pmatrix}1 & 3\\3 & 1\end{pmatrix}$. Đầu ra $o_3 = (4,4)/4 = (1,1)$, **đúng bằng** dạng song song ở trên (tự tính). Hai cách tính cùng một kết quả, nhưng cách hồi quy chỉ giữ một ma trận 2×2 và một vector 2 chiều, không giữ cả ba cặp (k, v).

6.4 Thêm suy giảm và cổng

Trạng thái chỉ cộng dồn thì mọi token cũ đóng góp như nhau và trạng thái bão hoà dần. Các bước cải tiến chung một dạng:

$$S_t = \gamma_t\, S_{t-1} + k_t v_t^\top,$$

với $\gamma_t \in (0,1]$ là hệ số quên. Theo ghi chú llm 19 (RetNet), γ là hằng số cho từng đầu (trọng số token cách m bước là γ^m); theo ghi chú llm 20 (GLA), cổng phụ thuộc dữ liệu, làm quên có chọn lọc. **Ví dụ tay (tự tính):** $\gamma = 0{,}9$ thì đóng góp của một token cách đây 10 bước còn $0{,}9^{10} \approx 0{,}349$; cách 50 bước còn $0{,}9^{50} \approx 0{,}005$. Nhớ gần mạnh, nhớ xa yếu dần.

DeltaNet và Gated DeltaNet (llm 23, 24) thay "cộng thêm" bằng quy tắc **delta** (sửa giá trị đang lưu cho key ấy thay vì chồng thêm), giúp ghi đè hiệu quả hơn; chi tiết nằm ở các bài đó.

7. SSM và Mamba ở mức trực giác

Mô hình không gian trạng thái (SSM) đến từ lý thuyết điều khiển: một trạng thái ẩn h_t được cập nhật tuyến tính theo đầu vào, và đầu ra là một phép chiếu của trạng thái:

$$h_t = \bar{A}\, h_{t-1} + \bar{B}\, x_t, \qquad y_t = C\, h_t.$$

So với $S_t = \gamma S_{t-1} + k_t v_t^\top$: cùng cấu trúc "trạng thái nhân một hệ số quên rồi cộng thông tin mới, đọc ra bằng một phép chiếu". Điểm khác là cách đặt tên và cách tham số hoá, nhưng **họ hàng gần**. Ghi chú llm 21 mô tả Mamba (SSM chọn lọc) là hội tụ về cùng cấu trúc, và ghi chú llm 22 (Mamba-2) chứng minh sự đối ngẫu giữa SSM chọn lọc và attention tuyến tính có cổng.

Điểm mới của Mamba là tính chọn lọc. Nếu $\bar{A}, \bar{B}, C$ là hằng (SSM bất biến theo thời gian), mô hình đối xử mọi token như nhau và không thể "chọn" token nào đáng nhớ. Mamba làm các tham số phụ thuộc đầu vào: token nào quan trọng thì $\bar{B}$ lớn (ghi nhiều vào trạng thái), token nhiễu thì được bỏ qua hoặc làm trạng thái quên nhanh. Ghi chú llm 21 nêu các tác vụ tổng hợp (selective copying, induction heads) để chứng minh sự cần thiết của tính chọn lọc.

Về trực giác: **trạng thái ẩn là một bản tóm tắt nén của lịch sử**, kích thước cố định. Mamba học cách viết bản tóm tắt thông minh. Phần "thuật toán scan nhận biết phần cứng" trong llm 21 là cách tính hồi quy tuần tự đó nhanh trên GPU; chương này không đi vào.

8. Đánh đổi: bộ nhớ trạng thái cố định và khả năng nhớ

8.1 Đếm bộ nhớ

KV cache (một lớp, một đầu): $2nd$ phần tử, tăng theo n. Trạng thái ma trận: d^2 phần tử, hằng.

Ví dụ tay (tự tính): $d = 64$. Trạng thái: $64^2 = 4096$ phần tử. KV cache với $n = 1000$: $2\cdot 1000\cdot 64 = 128\,000$ phần tử, gấp $31{,}25$ lần trạng thái. Điểm hoà vốn $2nd = d^2$ tại $n = d/2 = 32$ token: dưới 32 token KV cache nhỏ hơn, trên 32 token trạng thái nhỏ hơn.

8.2 Cái giá: sức chứa

Một trạng thái $d \times d$ chỉ có d^2 số, không thể chứa chính xác $n \gg d$ cặp (key, value) độc lập. Hình bốn mô phỏng điều này: key chiều d, n cặp ngẫu nhiên, truy hồi giá trị gắn với key đầu tiên. Quan sát mô phỏng (seed cố định, $d = 16$, 2000 lần lặp): độ giống cosine $\approx 1{,}0$ khi $n = 1$; $0{,}92$ khi $n = 4$; $0{,}72$ khi $n = 16$; $0{,}44$ khi $n = 64$; $0{,}23$ khi $n = 256$. Số này gần với công thức xấp xỉ $1/\sqrt{1 + (n-1)/d}$ (suy luận của chương: nhiễu từ $n - 1$ cặp khác cộng vào theo kiểu tổng các số ngẫu nhiên, nên tỉ lệ tín hiệu trên nhiễu giảm theo $\sqrt{(n-1)/d}$; đây là phép tính cỡ độ lớn cho key ngẫu nhiên, không phải định lý cho dữ liệu thật). Tăng d giúp, và key có cấu trúc (gần trực giao, hoặc được cổng chọn lọc) thì tốt hơn key ngẫu nhiên.

Vì vậy ngữ cảnh dài mà **cần truy hồi chính xác** một chi tiết cụ thể từ xa (ví dụ trích một con số ở đầu tài liệu) là điểm yếu tự nhiên của trạng thái cố định; ngược lại, nhiệm vụ cần tóm tắt xu hướng thì ổn. Đây là phát biểu cỡ độ lớn, kết quả thực nghiệm cụ thể có ở từng bài (llm 21, 25, 26).

8.3 Mô hình lai

Vì mỗi loại có điểm mạnh riêng, nhiều hệ thống **trộn**: phần lớn lớp dùng SSM hoặc attention tuyến tính (rẻ, bộ nhớ hằng), một số ít lớp dùng softmax attention đầy đủ (truy hồi chính xác). Jamba (llm 25) và Kimi Linear (llm 26) là các ví dụ của hướng này; tỉ lệ trộn và kết quả cụ thể nằm ở các ghi chú đó.

8.4 Bảng tổng hợp đánh đổi

Cơ chế	Thời gian cả chuỗi	Bộ nhớ khi decode	Mất gì
Softmax đầy đủ	$O(n^2 d)$	$O(nd)$	không mất, nhưng đắt
Cửa sổ trượt w	$O(nwd)$	$O(wd)$	thông tin ngoài cửa sổ phải truyền qua nhiều lớp
Thưa có chọn lọc	giữa hai cực	tuỳ cơ chế	có thể bỏ sót cặp quan trọng
Tuyến tính / SSM	$O(nd^2)$	$O(d^2)$	sức chứa trạng thái hữu hạn, trọng số kém nhọn

9. Biến thể và chi tiết hay bị bỏ qua

- **Thứ tự phép nhân không áp dụng được cho softmax chính xác.** Hàm mũ và chuẩn hoá theo từng hàng làm ma trận $n \times n$ không thể tránh. Attention tuyến tính là một **mô hình khác**, không phải cách tính nhanh hơn của cùng một mô hình.
- **Hằng số ẩn.** nd^2 nhỏ hơn $n^2 d$ khi $n > d$ (ví dụ $d = 128$, $n = 4096$: tỉ lệ $n/d = 32$ lần theo bậc), nhưng với $n < d$ ngược lại. Ở ngữ cảnh ngắn, attention thường vẫn nhanh hơn.
- **Hiệu quả phần cứng.** Nhân ma trận lớn tận dụng tốt GPU; hồi quy tuần tự thì không, nên các bài dùng dạng chunkwise (llm 19, 20): song song trong từng khúc, hồi quy giữa các khúc.
- **Thưa chưa chắc nhanh.** Mặt nạ thưa không đều làm truy cập bộ nhớ rời rạc; các bài attention thưa hiện đại chọn theo **khối** để hợp phần cứng (llm 62, 64).
- **Cửa sổ cố định trong KV cache.** Dùng cửa sổ thì cache bị chặn ở w token, nhưng cần xử lý cẩn thận phần đầu chuỗi (xem llm 55 về attention sinks).

10. Nối sang các bài paper của cuốn

- **llm 17 (Longformer):** cửa sổ trượt và token toàn cục: mục 5.1 và 5.2.
- **llm 18 (Linear Transformers are RNNs):** dạng kernel $\phi(q)^\top \phi(k)$, đổi thứ tự phép nhân, dạng hồi quy của mục 6.1 và 6.2 là đúng nội dung bài này.
- **llm 19 (RetNet), llm 20 (GLA):** thêm suy giảm γ và cổng phụ thuộc dữ liệu (mục 6.4), dạng chunkwise.
- **llm 21, 22 (Mamba, Mamba-2):** SSM chọn lọc và đối ngẫu với attention tuyến tính có cổng (mục 7).

- **llm 23, 24 (DeltaNet, Gated DeltaNet):** quy tắc delta cho việc ghi đè trạng thái.
- **llm 25, 26 (Jamba, Kimi Linear):** mô hình lai (mục 8.3).
- **llm 58 đến 66:** attention thưa theo mẫu cố định hoặc chọn theo nội dung (mục 5.2 và 5.3), và khảo sát tổng quan của llm 66.

11. Sai lầm hay gặp

- Nghĩ FlashAttention giảm độ phức tạp thời gian. Nó giảm lưu trữ và truy cập bộ nhớ; thời gian vẫn $O(n^2)$.
- Gọi attention tuyến tính là "softmax attention tính nhanh". Nó dùng kernel khác nên là một mô hình khác, với biểu diễn khác.
- Nói "bộ nhớ hằng nên nhớ được vô hạn". Bộ nhớ hằng đồng nghĩa với sức chứa hữu hạn: càng ghi nhiều, từng mục càng mờ.
- Đếm số ô thưa mà quên rằng phần cứng bị chi phối bởi cách truy cập bộ nhớ, không chỉ số phép tính.
- Dùng ϕ đồng nhất như ở ví dụ tay cho dữ liệu có phần tử âm: mẫu số có thể bằng 0 hoặc âm, làm trọng số mất ý nghĩa.
- Quên rằng SSM và attention tuyến tính có cùng khung $S_t = \gamma_t S_{t-1} + k_t v_t^\top$, rồi coi chúng là hai thế giới tách biệt.

12. Tóm tắt công thức

Đại lượng	Công thức
Số ô nhân quả	$n(n+1)/2$
Chi phí softmax cả chuỗi	$O(n^2 d)$
Chi phí cửa sổ trượt	$O(nwd)$
Attention tuyến tính (song song)	$\phi(Q)\big(\phi(K)^\top V\big)$, $O(nd^2)$
Dạng hồi quy	$S_t = S_{t-1} + \phi(k_t)v_t^\top$, $o_t = \phi(q_t)^\top S_t / (\phi(q_t)^\top z_t)$
Thêm suy giảm	$S_t = \gamma S_{t-1} + k_t v_t^\top$
SSM	$h_t = \bar{A}h_{t-1} + \bar{B}x_t$, $y_t = Ch_t$
Bộ nhớ decode	KV: $2nd$; trạng thái: d^2
Hoà vốn bộ nhớ	$n = d/2$

13. Bài tập tự tính

Bài 1. Chuỗi $n = 8$. Đếm số ô của mặt nạ nhân quả đầy đủ và của cửa sổ trượt $w = 3$ (kể cả chính nó).

Đáp án: đầy đủ $8 \cdot 9/2 = 36$; cửa sổ $1 + 2 + 3 \cdot 6 = 21$ (tự tính).

Bài 2. Attention tuyến tính với ϕ đồng nhất, $d = 2$, hai token: $k_1 = (1, 0)$, $v_1 = (2, 0)$, $k_2 = (0, 1)$, $v_2 = (0, 4)$, query $q_2 = (1, 1)$. Tính S_2 và o_2 (có chuẩn hoá bằng $q_2 \cdot z_2$).

Đáp án: $S_1 = \begin{pmatrix} 2 & 0 \\ 0 & 0 \end{pmatrix}$, $S_2 = \begin{pmatrix} 2 & 0 \\ 0 & 4 \end{pmatrix}$, $z_2 = (1, 1)$; $q_2 S_2 = (2, 4)$, $q_2 \cdot z_2 = 2$, nên $o_2 = (1, 2)$. Kiểm bằng dạng song song: trọng số $(1, 1)/2$, $0{,}5 \cdot (2, 0) + 0{,}5 \cdot (0, 4) = (1, 2)$ (tự tính).

Bài 3. $d = 128$, $n = 4096$. Bỏ hằng số, tỉ lệ chi phí $n^2 d$ so với nd^2 là bao nhiêu? Với $n = 64$ thì sao?

Đáp án: tỉ lệ bằng n/d: $4096/128 = 32$ lần nghiêng về tuyến tính; $n = 64$ thì $64/128 = 0{,}5$, tức tuyến tính **không** rẻ hơn (tự tính).

Bài 4. $d = 64$. So số phần tử của KV cache $2nd$ ở $n = 1000$ với trạng thái d^2. Từ độ dài nào trạng thái nhỏ hơn KV cache?

Đáp án: $2 \cdot 1000 \cdot 64 = 128\,000$; $64^2 = 4096$; tỉ lệ $31{,}25$; hoà vốn $n = d/2 = 32$ (tự tính).

Bài 5. Trong $S_t = \gamma S_{t-1} + k_t v_t^\top$ với $\gamma = 0{,}9$, token cách đây 10 bước đóng góp còn bao nhiêu phần so với lúc vừa ghi? Cần bao nhiêu bước để còn dưới 1%?

Đáp án: $0{,}9^{10} \approx 0{,}349$. Giải $0{,}9^m < 0{,}01$: $m > \ln 0{,}01 / \ln 0{,}9 \approx 43{,}7$, tức 44 bước (tự tính).

Tài liệu tham khảo nền: ghi chú llm 17 (Longformer), llm 18 (Linear Transformers), llm 19 (RetNet), llm 20 (GLA), llm 21 và llm 22 (Mamba), llm 58 (Sparse Transformer); nen04 và nen05 của cuốn này.

Nền tảng 07

Mixture of Experts và định tuyến: nhiều chuyên gia, nhưng mỗi token chỉ gọi vài người

Loại: chương nền tảng (không phải ghi chú paper)

Đọc trước: xác suất và softmax ở math nền tảng 08; entropy ở math nền tảng 11; mạng nơ-ron nhiều lớp ở ML cơ bản nền tảng 05; khối Transformer và FFN ở nen01 đến nen03 của cuốn này (có thể đọc thêm llm 01 và llm 11)

Dùng ở: llm 27 (Switch Transformer), llm 28 (Mixtral), và các bài có lớp MoE như llm 16, llm 25

> Chương này giải thích từ đầu, không phải bản dịch một paper. Các phép tính ghi "tự tính" đã kiểm bằng numpy; các hình là minh hoạ hoặc mô phỏng với dữ liệu tự sinh, không phải số đo từ một hệ thống thật.

1. Vì sao cần Mixture of Experts

Mixture of Experts (MoE) là cách làm một mô hình có rất nhiều tham số nhưng mỗi token chỉ dùng một phần nhỏ trong số đó. Trong Transformer thông thường (gọi là mô hình dày, dense), mọi tham số đều chạy cho mọi token, nên tăng tham số đồng nghĩa với tăng phép tính. MoE tách hai thứ ấy ra: số tham số tăng, còn phép tính mỗi token gần như giữ nguyên.

Hãy hình dung một bệnh viện lớn. Một bệnh viện dày là một bác sĩ biết hết mọi khoa, mọi bệnh nhân đều gặp người đó. Một bệnh viện MoE có tám bác sĩ chuyên khoa và một lễ tân; lễ tân nhìn bệnh nhân rồi gửi đến hai bác sĩ phù hợp. Mỗi bệnh nhân chỉ tốn thời gian của hai bác sĩ, nhưng cả bệnh viện chứa kiến thức của tám người. Phép ví von chỉ để lấy trực giác: ở mô hình thật, các "bác sĩ" không nhất thiết chuyên theo chủ đề mà ta gọi tên được (mục 7).

Trong LLM, MoE là con đường chính để có mô hình lớn mà vẫn phục vụ được. Chương này dạy bốn ý: tham số tổng so với tham số kích hoạt, bộ định tuyến top-k, cân bằng tải bằng loss phụ, và đánh đổi giữa bộ nhớ với tính toán.

2. Nhắc nền: FFN, softmax và top-k

2.1 FFN là chỗ chứa phần lớn tham số

Một khối Transformer có hai phần: attention (trộn thông tin giữa các token) và FFN (feed-forward network, xử lý từng token độc lập). FFN nhận vector ẩn $x \in \mathbb{R}^d$, nhân với ma trận mở rộng lên chiều d_{ff} lớn hơn, qua một hàm phi tuyến, rồi nhân với ma trận thu về d chiều. Phiên bản SwiGLU dùng ba ma trận, nên một FFN có khoảng $3\,d\,d_{ff}$ tham số (xem bài 11).

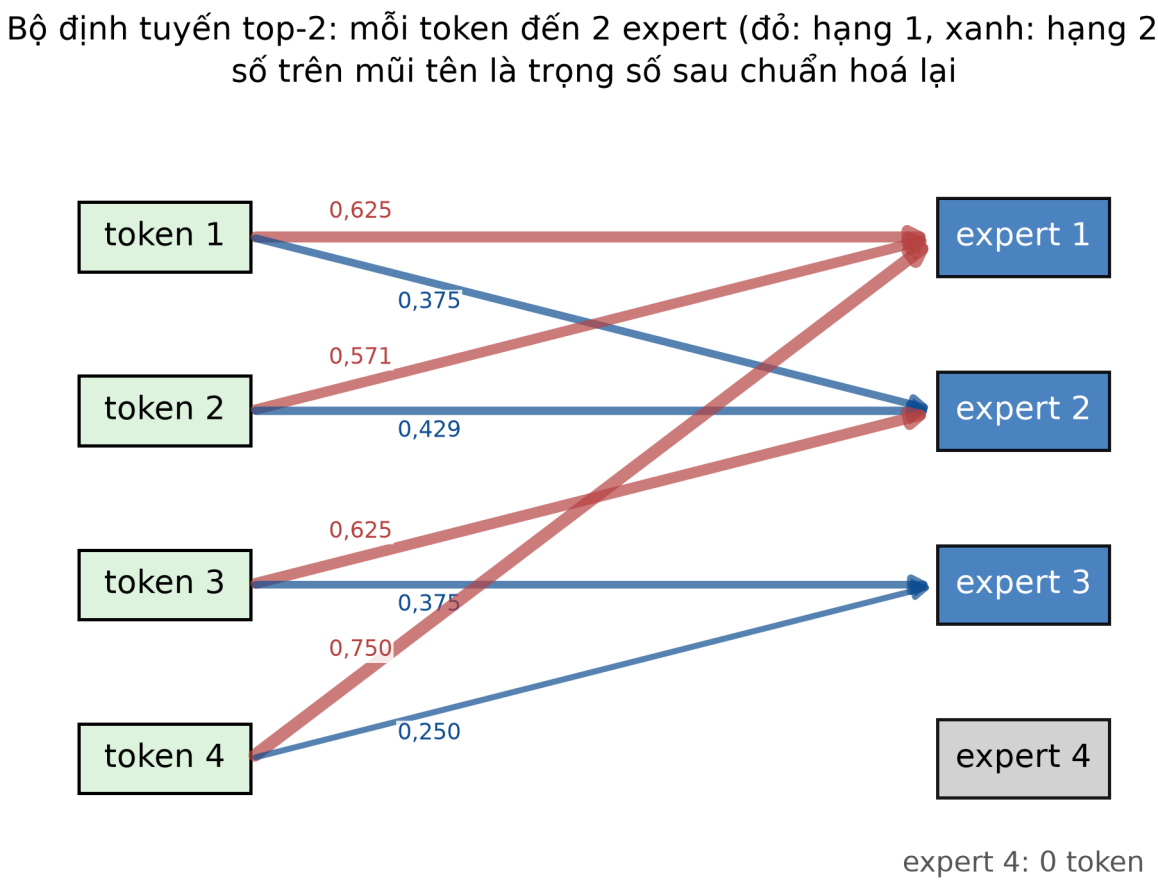
Ví dụ (tự tính) với hình dạng Mixtral: $d = 4096$, $d_{ff} = 14336$ thì một FFN có $3 \cdot 4096 \cdot 14336 \approx 176{,}2$ triệu tham số. Attention của một lớp cùng cỡ chỉ khoảng $41{,}9$ triệu (theo ghi chú llm 28). Như vậy FFN chiếm áp đảo, và đó là lý do MoE nhân bản đúng khối này: nhân bản FFN là cách rẻ nhất để thêm tham số.

2.2 Softmax và top-k

Với vector điểm $h = (h_1, \dots, h_N)$, softmax cho xác suất $p_i = e^{h_i} / \sum_j e^{h_j}$: mọi $p_i > 0$ và tổng bằng 1. Phép **top-k** chọn k chỉ số có p_i lớn nhất. Ví dụ (tự tính): $h = (2, 1, 0, -1)$ cho $p \approx (0{,}644;\ 0{,}237;\ 0{,}087;\ 0{,}032)$; top-2 chọn chỉ số 1 và 2. Chuẩn hoá lại trên hai chỉ số được chọn: $(0{,}644;\ 0{,}237)/0{,}881 \approx (0{,}731;\ 0{,}269)$.

Chú ý: top-k là phép rời rạc, không khả vi theo chỉ số được chọn. Nhưng trọng số p_i của expert được chọn vẫn là số liên tục và được nhân vào đầu ra, nên gradient chảy được vào bộ định tuyến qua các trọng số ấy.

3. Trực giác bằng hình: token đi đâu



Hình 24: Bộ định tuyến top-2 gửi mỗi token đến 2 trong 4 expert, kèm trọng số sau chuẩn hoá lại (minh hoạ, tự tính)

Cách đọc hình: bên trái là bốn token, bên phải là bốn expert (mỗi expert là một FFN riêng). Mỗi token phát ra hai mũi tên: mũi tên đỏ đến expert có xác suất cao nhất, mũi tên xanh đến expert hạng hai; độ dày mũi tên tỉ lệ với trọng số, và con số ghi cạnh mũi tên chính là trọng số ấy sau khi chuẩn hoá lại trên hai expert được chọn. Để ý expert 4 không nhận token nào, còn expert 1 và 2 nhận ba token mỗi bên: đó là mất cân bằng, vấn đề trung tâm của mục 5.

Sơ đồ này dùng đúng các số của ví dụ tay ở mục 6. Toàn bộ quá trình xảy ra ở mỗi lớp MoE, độc lập với các lớp khác: token 1 có thể đi expert 1 và 2 ở lớp này, nhưng đi expert 5 và 7 ở lớp kế.

4. Định nghĩa chính thức

4.1 Lớp MoE

Một lớp MoE gồm N expert $E_1, \ldots, E_N$ (mỗi cái là một FFN) và một bộ định tuyến (router) là một ma trận nhỏ $W_r \in \mathbb{R}^{d \times N}$. Với token x:

$$h(x) = xW_r, \qquad p(x) = \text{softmax}(h(x)), \qquad \mathcal{T}(x) = \text{top-}k \text{ của } p(x).$$

Đầu ra là tổng có trọng số của các expert được chọn:

$$y(x) = \sum_{i \in \mathcal{T}(x)} g_i(x)\, E_i(x).$$

Cách đặt trọng số g_i có hai biến thể. Switch Transformer với $k = 1$ dùng $g_i = p_i$ (xác suất gốc, vì chuẩn hoá lại trên một phần tử sẽ luôn bằng 1 và làm mất gradient của router). Mixtral với $k = 2$ lấy softmax chỉ trên hai điểm số được chọn, tức tương đương chuẩn hoá lại p trên tập top-2 (theo ghi chú llm 28). Chi phí router là $O(d \cdot N)$ mỗi token, rất nhỏ so với một FFN.

4.2 Tham số tổng và tham số kích hoạt

- **Tham số tổng**: mọi tham số của mô hình, kể cả mọi expert. Quyết định lượng bộ nhớ cần để giữ mô hình.
- **Tham số kích hoạt** (active): tham số thực sự tham gia tính cho một token, gồm phần dùng chung (attention, embedding, router) cộng k expert mỗi lớp. Quyết định lượng phép tính mỗi token.

Gọi S là tham số dùng chung, e là tổng tham số của một "ô expert" qua mọi lớp (một expert ở mỗi lớp, cộng qua L lớp), thì

$$P_{\text{tổng}} = S + N\, e, \qquad P_{\text{kích hoạt}} = S + k\, e.$$

Tăng N làm tổng tăng tuyến tính, còn kích hoạt không đổi khi k cố định.

4.3 Loss phụ cân bằng tải

Nếu chỉ huấn luyện theo loss chính, router thường thiên về vài expert vì expert nào được gọi nhiều thì học nhanh hơn, nên lại được gọi nhiều hơn (vòng lặp tự củng cố). Các expert còn lại gần như không được huấn luyện. Cách chữa là thêm loss phụ. Với batch gồm T token, định nghĩa

$$f_i = \frac{\text{số lượt token được gửi đến expert } i}{T\, k}, \qquad P_i = \frac{1}{T} \sum_x p_i(x).$$

f_i là tỉ lệ lượt định tuyến thực tế (mỗi token đóng góp k lượt, nên chia cho Tk để $\sum_i f_i = 1$); P_i là xác suất trung bình router dành cho expert i. Loss phụ:

$$\mathcal{L}_{\text{aux}} = \alpha\, N \sum_{i=1}^{N} f_i\, P_i.$$

Khi tải đều, $f_i = P_i = 1/N$, tổng bằng $1/N$, nhân N ra 1, đó là mức thấp nhất khi f và P bám theo nhau (đúng với router thật, vì expert nào được ưu tiên xác suất thì cũng nhận nhiều lượt). Khi mọi thứ dồn vào một expert, giá trị là N. f_i không khả vi (do đếm), nên gradient đi qua P_i: giảm P_i ở expert đang bị gọi nhiều (f_i lớn). Switch Transformer dùng $\alpha = 10^{-2}$ và $k = 1$ (theo ghi chú llm 27); việc chia cho Tk khi $k > 1$ là cách tổng quát hoá của chương này, các hệ thống khác có thể chuẩn hoá khác.

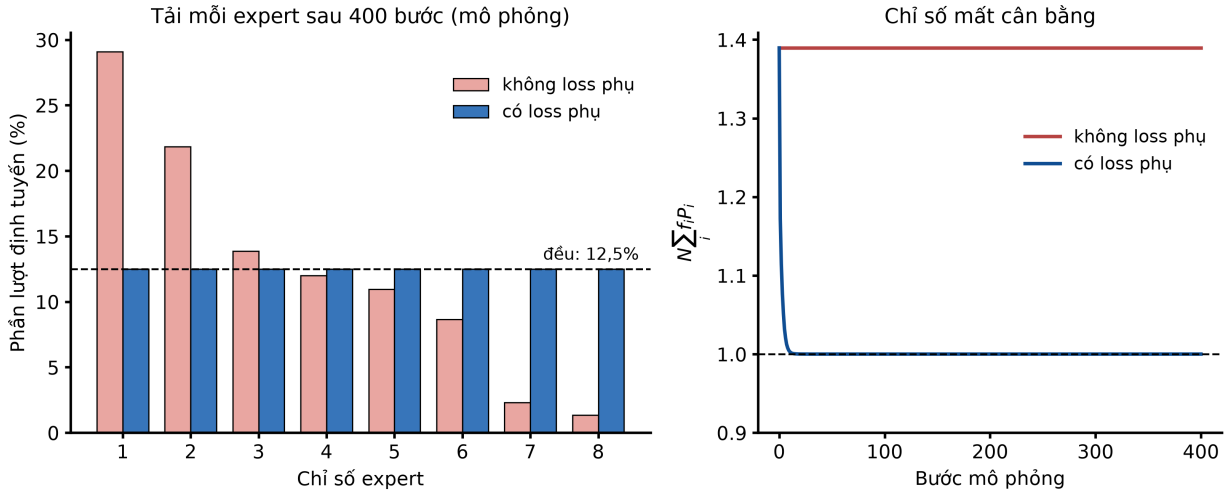
4.4 Dung lượng expert (capacity)

Trên phần cứng cần kích thước tensor cố định, nên mỗi expert nhận tối đa

$$\text{capacity} = \frac{\text{số lượt định tuyến}}{N} \times \text{capacity factor}.$$

Lượt nào vượt dung lượng bị bỏ (drop): token ấy bỏ qua lớp MoE và đi tiếp qua đường residual. Capacity factor lớn hơn 1 chừa chỗ đệm cho mất cân bằng nhẹ, nhưng tốn tính toán cho các ô trống.

5. Cân bằng tải bằng hình



Hình 25: Tải mỗi expert khi không có và khi có loss phụ, và chỉ số mất cân bằng theo bước mô phỏng (minh hoạ, tự tính)

Cách đọc hình: đây là mô phỏng, không phải số đo. Chúng tôi sinh 4096 token ngẫu nhiên (chiều 16), $N = 8$ expert, top-2, với bộ định tuyến cố ý khởi tạo lệch (bias ưu tiên các expert đầu, giả định). Cột đỏ bên trái là tải ở bước đầu, khi chưa có gì chữa: expert 1 nhận 29,1% lượt, expert 8 chỉ 1,3%, so với mức đều 12,5% (đường nét đứt). Cột xanh là tải sau 400 bước chỉ tối ưu loss phụ: cả tám expert về 12,5%. Bên phải, chỉ số $N \sum_i f_i P_i$ giảm từ 1,389 xuống 1,000 khi có loss phụ, và đứng yên ở 1,389 khi không có (vì mô phỏng không có tín hiệu nào khác để thay đổi router).

Mô phỏng này rút gọn: không có loss chính, chỉ cho thấy loss phụ làm đúng việc của nó. Trong huấn luyện thật, loss phụ cạnh tranh với loss chính: α quá lớn ép tải đều nhưng làm router mất quyền chọn expert phù hợp; α quá nhỏ thì vẫn lệch.

6. Ví dụ tay: 4 token, 4 expert, top-2

Giả định router đã tính xong xác suất softmax cho bốn token (mỗi hàng là một token, mỗi cột là một expert):

Token	p_1	p_2	p_3	p_4
1	0,5	0,3	0,1	0,1
2	0,4	0,3	0,2	0,1
3	0,1	0,5	0,3	0,1
4	0,6	0,1	0,2	0,1

Bước 1, chọn top-2 mỗi token (tự tính). Token 1: expert 1 và 2. Token 2: expert 1 và 2. Token 3: expert 2 và 3. Token 4: expert 1 và 3.

Bước 2, chuẩn hoá lại trọng số trên hai expert được chọn (tự tính). Token 1: $0{,}5/0{,}8 = 0{,}625$ và $0{,}3/0{,}8 = 0{,}375$. Token 2: $0{,}4/0{,}7 \approx 0{,}571$ và $0{,}429$. Token 3: $0{,}5/0{,}8 = 0{,}625$ và $0{,}375$. Token 4: $0{,}6/0{,}8 = 0{,}75$ và $0{,}25$. Đây chính là các con số trên hình ở mục 3. Đầu ra của token 1 là $0{,}625\, E_1(x_1) + 0{,}375\, E_2(x_1)$.

Bước 3, đếm tải. Tổng cộng có $4 \times 2 = 8$ lượt. Expert 1 nhận 3 lượt, expert 2 nhận 3, expert 3 nhận 2, expert 4 nhận 0. Vậy

$$f = \left(\tfrac{3}{8}, \tfrac{3}{8}, \tfrac{2}{8}, 0\right) = (0{,}375;\ 0{,}375;\ 0{,}25;\ 0) \quad \text{(tự tính)}.$$

Bước 4, xác suất trung bình (trung bình từng cột, tự tính): $P_1 = (0{,}5 + 0{,}4 + 0{,}1 + 0{,}6)/4 = 0{,}4$; $P_2 = 0{,}3$; $P_3 = 0{,}2$; $P_4 = 0{,}1$. Tổng bằng 1, đúng yêu cầu.

Bước 5, loss cân bằng đơn giản (đặt $\alpha = 1$ cho gọn):

$$N \sum_i f_i P_i = 4\,(0{,}375 \cdot 0{,}4 + 0{,}375 \cdot 0{,}3 + 0{,}25 \cdot 0{,}2 + 0 \cdot 0{,}1) = 4 \cdot 0{,}3125 = 1{,}25.$$

Nếu tải đều hoàn toàn ($f_i = P_i = 0{,}25$) thì giá trị là $4 \cdot 4 \cdot 0{,}0625 = 1$. Con số $1{,}25 > 1$ báo rằng đang có mất cân bằng, và vì $P_4 = 0{,}1$ nhỏ còn $f_4 = 0$, gradient sẽ đẩy xác suất khỏi expert 1 và 2 (đang bị gọi nhiều) sang expert 4.

Bước 6, dung lượng. Với capacity factor 1,0: capacity $= 8/4 \times 1 = 2$ lượt mỗi expert. Expert 1 nhận 3 lượt nên một lượt bị drop (lượt nào bị bỏ phụ thuộc quy tắc ưu tiên của hệ thống, thường theo thứ tự token hoặc theo xác suất). Với capacity factor 1,5: capacity $= 3$, không drop nào, nhưng expert 4 vẫn trống: phần cứng vẫn dành chỗ cho nó. Ví dụ rất nhỏ nên mất cân bằng nhìn ra ngay; ở batch hàng nghìn token, các con số dao động nhỏ hơn.

7. Biến thể và chi tiết hay bị bỏ qua

Expert có chuyên môn hoá không? Trực giác "expert toán, expert code" rất hấp dẫn nhưng không được dữ liệu ủng hộ nhiều. Theo ghi chú llm 28, Mixtral đo phân bố expert trên các tập con của The Pile (ArXiv, PubMed, PhilPapers, GitHub, ...) ở lớp 0, 15, 31 và không thấy chuyên môn hoá theo chủ đề rõ ràng; định tuyến mang tính cú pháp và cục bộ theo thời gian (ví dụ token thụt lề trong code thường đi cùng một expert). Kết luận đó là quan sát trên ba lớp, không phải bằng chứng cho mọi tầng. Vì vậy trong chương này, "chuyên môn hoá" nên hiểu là sự phân chia công việc do huấn luyện tự tìm ra, chứ không phải nhãn ngữ nghĩa ta đặt trước.

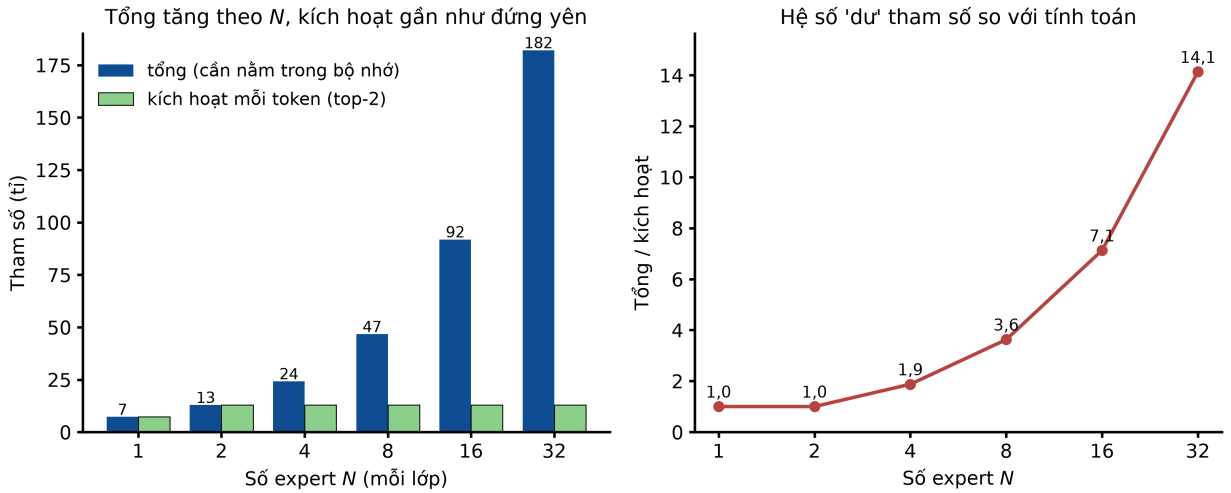
Top-1 hay top-2? Switch Transformer dùng $k = 1$ để giảm chi phí giao tiếp và đơn giản hoá cài đặt, bác bỏ giả định cũ rằng phải so sánh ít nhất hai expert thì router mới học được (theo ghi chú llm 27). Mixtral dùng $k = 2$. Hai lựa chọn đều hoạt động; đánh đổi nằm ở chi phí mỗi token (k FFN) so với độ phong phú của tổ hợp.

Mô hình dày so với MoE ở cùng ngân sách. Cùng số tham số kích hoạt, MoE thường cho chất lượng tốt hơn vì có thêm dung lượng "lưu kiến thức". Cùng số tham số tổng, mô hình dày thường mạnh hơn mỗi token vì mọi tham số đều được dùng. Hai so sánh cho kết luận ngược nhau, nên khi đọc "MoE tốt hơn" phải hỏi: cố định đại lượng nào?

Expert dùng chung. Một số kiến trúc (như DeepSeek) thêm một vài expert luôn được gọi, kèm các expert định tuyến. Ghi chú trong repo ở llm 16 và 25 nêu MoE trong ngữ cảnh MLA và hybrid; chương này không đi sâu vào các biến thể ấy.

Ổn định huấn luyện. Router là softmax trên điểm số nhạy với số học độ chính xác thấp. Theo ghi chú llm 27, Switch Transformer chạy riêng router bằng float32, khởi tạo nhỏ hơn và tăng dropout bên trong expert khi tinh chỉnh. Đây là chi tiết triển khai hay bị bỏ qua nhưng quyết định việc có huấn luyện được hay không.

Loss phụ không phải là tất cả. Phương pháp khác dùng bias điều chỉnh hoặc chọn expert theo kiểu "expert chọn token". Chương này chỉ dạy kiểu loss phụ vì nó xuất hiện trong llm 27.



Hình 26: Tham số tổng và tham số kích hoạt khi tăng số expert (hình dạng Mixtral, top-2) cùng tỉ số tổng trên kích hoạt (minh hoạ, tự tính)

8. Đánh đổi bộ nhớ so với tính toán

Cách đọc hình: dùng hình dạng của Mixtral 8x7B (32 lớp, $d = 4096$, $d_{ff} = 14336$, SwiGLU, attention nhóm 1024 cho K và V). Phần dùng chung $S \approx 1{,}60$ tỉ (attention cộng embedding và đầu ra), một ô expert qua 32 lớp là $e \approx 5{,}64$ tỉ (đều là tự tính). Cột xanh đậm là $S + Ne$, cột xanh nhạt là $S + \min(k, N)\, e$ với $k = 2$. Tại $N = 8$ ta được tổng khoảng $46{,}7$ tỉ và kích hoạt khoảng $12{,}9$ tỉ, khớp với "47B và 13B" trong ghi chú llm 28. Các điểm $N \neq 8$ là suy luận của chương (giả định giữ nguyên hình dạng, chỉ đổi số expert), không phải mô hình có thật. Bên phải: tỉ số tổng trên kích hoạt tăng từ 1,0 lên 14,1 khi N từ 1 lên 32.

Đọc hai con số cùng nhau để thấy đánh đổi:

- **Tính toán mỗi token** tỉ lệ với tham số kích hoạt: khoảng $2 \times 12{,}9 \approx 25{,}8$ tỉ phép nhân-cộng mỗi token (quy ước: một tham số tương ứng khoảng 2 phép tính dấu phẩy động mỗi token, bỏ qua attention theo độ dài, tự tính). Mô hình dày 7,24 tỉ chỉ khoảng một nửa số đó.
- **Bộ nhớ** tỉ lệ với tham số tổng: ở fp16 (2 byte mỗi tham số), $46{,}7$ tỉ cần khoảng $93{,}4$ GB chỉ cho trọng số (tự tính), so với khoảng $14{,}5$ GB cho mô hình dày 7,24 tỉ. Dù mỗi token chỉ chạm hai expert mỗi lớp, mọi expert đều phải nằm sẵn vì token kế tiếp có thể cần bất kỳ expert nào.
- **Batch nhỏ so với batch lớn.** Với batch rất nhỏ, mỗi bước giải mã phải đọc trọng số của các expert được gọi từ bộ nhớ; chi phí đọc bộ nhớ chiếm ưu thế, nên lợi thế tính toán của MoE ít thể hiện. Với batch lớn, các expert được chia đều token và phần cứng chạy hiệu quả hơn. Theo ghi chú llm 28, paper Mixtral nói SMoE hợp với tải theo batch có cường độ số học tốt và thừa nhận overhead của định tuyến, nhưng không đưa số đo tốc độ.
- **Huấn luyện phân tán.** Expert thường đặt trên các thiết bị khác nhau, nên token phải được gửi đi (giao tiếp all-to-all) rồi gửi về. Capacity factor, loss phụ và kích thước nhóm expert đều phản ánh việc giữ tải đều giữa các thiết bị.

Tóm lại, MoE đổi bộ nhớ lấy tính toán: bỏ thêm dung lượng nhớ để cùng một lượng phép tính cho chất lượng cao hơn. Ở nơi bộ nhớ khan hiếm (thiết bị đầu cuối), đánh đổi đó có thể không còn đáng.

9. Nối sang các bài paper của cuốn

llm 27 (Switch Transformer). Bài đặt $k = 1$, dùng loss phụ $\alpha N \sum f_i P_i$ với $\alpha = 10^{-2}$, capacity factor và việc bỏ token vượt dung lượng, theo ghi chú llm 27. Ví dụ trong ghi chú (batch 4096 token, 8 expert, capacity factor 1,25 cho capacity 640 mỗi expert) cùng công thức với mục 4.4 của chương này: $4096/8 \times 1{,}25 = 640$.

llm 28 (Mixtral). Bài dùng 8 expert, top-2, softmax trên hai điểm số được chọn; ghi chú llm 28 tự kiểm số học để ra $\approx 46{,}7$ tỉ tổng và $\approx 12{,}9$ tỉ kích hoạt, trùng với hình ở mục 8. Hãy đọc kỹ phần phân tích định tuyến (không chuyên môn hoá theo chủ đề) cùng với mục 7.

llm 25 (Jamba) và llm 16 (MLA, DeepSeek-V2). Hai bài dùng MoE như một thành phần trong kiến trúc lớn hơn. Ghi chú llm 25 so sánh Mixtral 46,7B tổng và 12,9B kích hoạt với Jamba, và nhấn mạnh rằng bộ nhớ KV cache là chiều chi phí khác, tách khỏi chiều tham số.

Các bài suy luận. Bài về lượng tử hoá và phục vụ (xem nen08 của cuốn này) trở nên quan trọng vì MoE làm bộ nhớ trọng số là nút thắt; giảm trọng số xuống 8 hoặc 4 bit là cách tự nhiên để chứa mô hình MoE.

10. Sai lầm hay gặp

- **Nhầm "47B" là chi phí tính toán.** 47 tỉ là bộ nhớ; chi phí tính toán theo 13 tỉ kích hoạt. Ngược lại, nói "mô hình 13B" mà quên rằng vẫn phải nạp 47 tỉ tham số vào bộ nhớ.
- **Cho rằng mỗi expert là một chủ đề.** Dữ liệu trong ghi chú llm 28 cho thấy định tuyến mang tính cú pháp, không chia theo chủ đề; đừng tự gán nhãn "expert Toán" khi chưa đo.
- **Quên chuẩn hoá lại trọng số top-k.** Với $k = 2$, nếu dùng thẳng p_i gốc, tổng hai trọng số không bằng 1 và quy mô đầu ra lệch; phải chuẩn hoá lại (hoặc softmax trên hai điểm số) theo thiết kế của từng mô hình.
- **Đọc loss phụ nhỏ là tải đã đều.** Loss phụ có mức tham chiếu thấp nhất là α (khi $N \sum f_i P_i = 1$), không phải 0. Ngoài ra một lớp có thể cân bằng trung bình nhưng lệch theo từng chuỗi cụ thể.
- **Coi capacity factor lớn là miễn phí.** Hệ số đệm làm tăng phép tính và bộ nhớ cho các ô trống; token bị drop thì lớp ấy không xử lý token đó.
- **Đánh đồng f_i và P_i.** f_i là đếm thực sự (không khả vi), P_i là xác suất mềm (khả vi). Gradient của loss phụ đi qua P_i, còn f_i chỉ đóng vai hệ số.

11. Tóm tắt công thức

Đại lượng	Công thức	Ghi chú
Xác suất router	$p = \text{softmax}(xW_r)$	$O(dN)$ mỗi token
Đầu ra MoE	$y = \sum_{i \in \mathcal{T}} g_i E_i(x)$	g_i là p_i (top-1) hoặc đã chuẩn hoá lại (top-2)
Tham số tổng	$S + Ne$	quyết định bộ nhớ
Tham số kích hoạt	$S + ke$	quyết định phép tính mỗi token
Tỉ lệ tải	$f_i =$ lượt đến $i/(Tk)$	không khả vi
Loss cân bằng	$\alpha N \sum_i f_i P_i$	mức thấp nhất α khi tải đều
Dung lượng expert	$\frac{\text{lượt}}{N} \times$ capacity factor	vượt thì drop

12. Bài tập tự tính

Bài 1. Router cho điểm $h = (2, 1, 0, -1)$ trên 4 expert. Tìm top-2 và trọng số sau chuẩn hoá lại.

Đáp án (đã kiểm numpy): softmax $\approx (0{,}644;\ 0{,}237;\ 0{,}087;\ 0{,}032)$; top-2 là expert 1 và 2; trọng số chuẩn hoá lại $\approx (0{,}731;\ 0{,}269)$.

Bài 2. Một lớp MoE có batch 1024 token, $N = 8$, $k = 2$, capacity factor $1{,}25$. Tính capacity mỗi expert.

Đáp án (đã kiểm numpy): số lượt $= 1024 \times 2 = 2048$; $2048/8 \times 1{,}25 = 320$ lượt mỗi expert.

Bài 3. Top-1 với $N = 4$: $f = (0{,}5;\ 0{,}5;\ 0;\ 0)$ và $P = (0{,}4;\ 0{,}4;\ 0{,}1;\ 0{,}1)$. Tính $N \sum f_i P_i$ (đặt $\alpha = 1$).

Đáp án (đã kiểm numpy): $4\,(0{,}5 \cdot 0{,}4 + 0{,}5 \cdot 0{,}4) = 4 \cdot 0{,}4 = 1{,}6$, lớn hơn mức đều là 1.

Bài 4. Dùng hình dạng ở mục 8 ($S \approx 1{,}60$ tỉ, $e \approx 5{,}637$ tỉ). Tính tham số tổng và kích hoạt nếu $N = 16$, $k = 2$, rồi bộ nhớ fp16 của trọng số.

Đáp án (đã kiểm numpy): tổng $\approx 1{,}60 + 16 \cdot 5{,}637 \approx 91{,}8$ tỉ; kích hoạt $\approx 1{,}60 + 2 \cdot 5{,}637 \approx 12{,}9$ tỉ (không đổi so với $N = 8$); bộ nhớ fp16 $\approx 91{,}8 \times 2 \approx 183{,}6$ GB.

Bài 5. Trong ví dụ tay ở mục 6, giả sử router được chỉnh sao cho mỗi token chọn expert khác nhau và mỗi expert nhận đúng 2 lượt (tải đều hoàn toàn, $f_i = 0{,}25$), còn $P_i = 0{,}25$. Tính loss cân bằng và so với ví dụ lệch.

Đáp án (đã kiểm numpy): $4 \cdot (4 \cdot 0{,}25 \cdot 0{,}25) = 1$. Đây là mức tham chiếu của tải đều; ví dụ lệch cho $1{,}25$ cao hơn. Khi f và P bám theo nhau như trong router thật, mọi lệch khỏi đều đều làm tổng $\sum f_i P_i$ lớn hơn $1/N$, nên giá trị 1 là mức thấp nhất đạt được.

Tài liệu tham khảo nền: Shazeer et al. 2017 (Sparsely-Gated MoE), GShard (Lepikhin et al. 2020), ghi chú llm 27 (Switch Transformer) và llm 28 (Mixtral) trong cuốn này.

Nền tảng 08

Lượng tử hoá và bộ nhớ GPU: bớt bit cho trọng số, bớt chuyến đi cho dữ liệu

Loại: chương nền tảng (không phải ghi chú paper)

Đọc trước: số học máy tính và làm tròn ở math nền tảng 12; chuẩn và điều kiện số ở math nền tảng 13; vector và ma trận ở math nền tảng 02; softmax ở math nền tảng 08; precision ở optimization nền 08

Dùng ở: llm 44 (FlashAttention), llm 45 (FlashAttention-2), llm 51 (LLM.int8), llm 52 (SmoothQuant), llm 53 (GPTQ), llm 54 (AWQ)

> Chương này giải thích từ đầu, không phải bản dịch một paper. Các phép tính ghi "tự tính" đã kiểm bằng numpy; các hình là minh hoạ hoặc mô phỏng với dữ liệu tự sinh, không phải số đo từ phần cứng thật.

1. Vì sao cần hai khái niệm này

Chạy một LLM tốn hai thứ: **bộ nhớ** để chứa trọng số và dữ liệu trung gian, và **thời gian** để tính toán. Chương này gộp hai nửa của cùng một câu chuyện. Nửa thứ nhất là **lượng tử hoá**: biểu diễn mỗi số bằng ít bit hơn (8 hoặc 4 thay vì 16) để mô hình nhỏ đi và đọc nhanh hơn. Nửa thứ hai là **phân cấp bộ nhớ GPU và FlashAttention**: nhận ra rằng nhiều phép tính bị chậm không phải vì thiếu sức tính mà vì dữ liệu phải đi đi về về giữa các tầng bộ nhớ, rồi sắp xếp lại phép tính để bớt những chuyến đi đó.

Hai ý này đi cùng nhau vì chúng cùng trả lời một câu: dữ liệu cần ở đâu, với bao nhiêu bit, và bao nhiêu lần phải di chuyển. Các bài llm 44 và 45 giải quyết nửa thứ hai cho attention; các bài llm 51 đến 54 giải quyết nửa thứ nhất cho trọng số và activation.

2. Nhắc nền: số nguyên, làm tròn và byte

2.1 Bao nhiêu byte cho một số

Số thực thường được lưu bằng fp16 (16 bit, 2 byte) hoặc bf16 (16 bit). Số nguyên có dấu 8 bit (int8) biểu diễn các giá trị $-128, \dots, 127$; số nguyên 4 bit (int4) có 16 mã. Ví dụ (tự tính): một mô hình 7 tỉ tham số cần khoảng 14 GB ở fp16, 7 GB ở int8 và $3{,}5$ GB ở int4, chỉ tính trọng số (GB hiểu là 10^9 byte). Mô hình càng nhỏ thì càng dễ chứa trong một GPU và càng ít byte phải đọc cho mỗi token sinh ra.

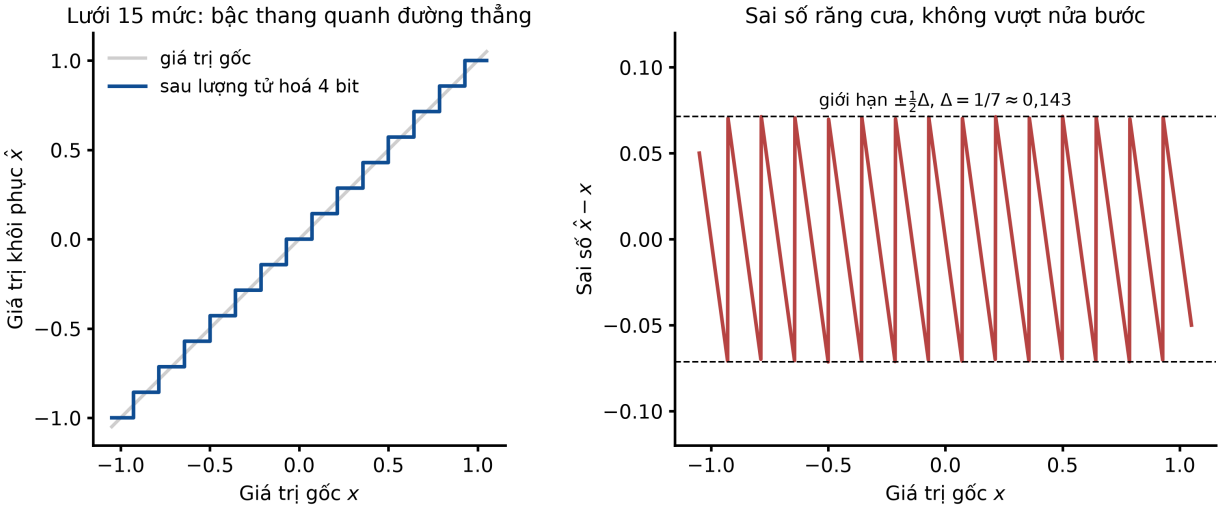
2.2 Làm tròn là nguồn của sai số

Đổi một số thực thành số nguyên là làm tròn: $\text{round}(2{,}6) = 3$, $\text{round}(-1{,}2) = -1$. Sai số của một lần làm tròn nằm trong khoảng $[-0{,}5;\ 0{,}5]$ đơn vị nguyên. Muốn sai số tuyệt đối nhỏ, phải làm cho "một đơn vị nguyên" tương ứng với một lượng thực nhỏ, và đó là việc của thang đo (scale) ở mục sau.

3. Trực giác bằng hình: lưới và bậc thang

Ý tưởng cơ bản: lấy đoạn giá trị $[-m, m]$ của dữ liệu, chia thành một lưới đều các mức, rồi mỗi số thực được thay bằng mức gần nhất trên lưới. Lưu chỉ số mức (một số nguyên nhỏ) cùng một số thực chung cho cả nhóm là thang đo.

Cách đọc hình: bên trái, đường xám là giá trị gốc, đường xanh là giá trị sau khi lượng tử hoá 4 bit đối xứng với 15 mức ($-7, \dots, 7$ nhân với bước $\Delta = 1/7 \approx 0{,}143$). Mỗi bậc thang là các giá trị gốc bị làm tròn về cùng một mức. Bên phải là sai số $\hat{x} - x$: đường răng cưa chạy trong dải giữa hai đường nét đứt $\pm\Delta/2 \approx \pm 0{,}0714$;



Hình 27: Lưới lượng tử hoá 4 bit trên đoạn từ âm một đến một: đồ thị bậc thang và sai số răng cưa không vượt nửa bước (minh hoạ, tự tính)

số đo trong mô phỏng là $0{,}0714$ (tự tính). Giá trị ngoài đoạn $[-1, 1]$ bị cắt về mức biên, đó là chỗ sai số có thể lớn hơn nửa bước.

Điều quan trọng: **sai số tối đa tỉ lệ với bước Δ, mà bước tỉ lệ với khoảng giá trị phải phủ.** Tăng số bit làm bước nhỏ đi (8 bit có 255 mức thay vì 15), nhưng khoảng phủ lớn (do ngoại lai) làm bước to ra. Cuộc đấu giữa hai điều này là toàn bộ nội dung lượng tử hoá.

4. Định nghĩa chính thức

4.1 Lượng tử hoá đối xứng (absmax)

Cho vector thực x. Gọi $m = \max_i |x_i|$ và $q_{\max} = 2^{b-1} - 1$ (với b bit: 127 khi $b = 8$, 7 khi $b = 4$). Định nghĩa thang đo $s = q_{\max}/m$ và

$$q_i = \operatorname{round}(s\, x_i), \qquad \hat{x}_i = \frac{q_i}{s}.$$

Số nguyên q_i là thứ được lưu (cùng s), còn $\hat{x}_i$ là giá trị khôi phục. Một số tài liệu gọi $\Delta = 1/s$ là kích thước bước (step) hay scale; cần đọc kỹ quy ước là nhân hay chia. Sai số mỗi phần tử $|\hat{x}_i - x_i| \le \Delta/2 = m/(2q_{\max})$ miễn không bị cắt.

4.2 Lượng tử hoá có điểm không (zero-point)

Khi dữ liệu không đối xứng quanh 0 (ví dụ kết quả sau hàm kích hoạt luôn dương), lưới đối xứng lãng phí một nửa mã. Dùng khoảng $[a, c]$ với $a = \min x$, $c = \max x$, và 256 mã (int8 từ -128 đến 127):

$$\Delta = \frac{c - a}{255}, \qquad z = \operatorname{round}\!\left(-\frac{a}{\Delta}\right) - 128, \qquad q = \operatorname{clip}\!\left(\operatorname{round}\!\left(\frac{x}{\Delta}\right) + z,\ -128,\ 127\right), \qquad \hat{x} = (q - z)\Delta.$$

z là điểm không: mã nguyên ứng với giá trị thực 0, để số 0 được biểu diễn chính xác (quan trọng vì padding và ReLU sinh nhiều số 0). Hai số (Δ, z) là "tham số lượng tử hoá".

4.3 Độ mịn: theo tensor, theo kênh, theo nhóm

Một thang đo cho cả ma trận (per-tensor) rẻ nhưng nhạy với ngoại lai. Thang đo riêng mỗi hàng hoặc cột (per-channel, vector-wise) chính xác hơn. Thang đo riêng mỗi nhóm g giá trị liên tiếp (group-wise, thường $g = 64$ hoặc 128) là điểm giữa. Chi phí lưu thang đo: một scale fp16 mỗi nhóm 128 trọng số tốn thêm $16/128 = 0{,}125$ bit mỗi trọng số (tự tính), nên "4 bit" thực tế là khoảng $4{,}125$ bit; thêm một điểm không 16 bit nữa thành khoảng $4{,}25$ bit (tự tính).

4.4 Lượng tử hoá trọng số và activation

Trọng số cố định sau khi huấn luyện, nên có thể lượng tử hoá một lần và cất. **Activation** (đầu ra các lớp) thay đổi theo từng đầu vào, nên phải lượng tử hoá động mỗi lần chạy hoặc dùng thang đo ước lượng sẵn. Hầu hết phương pháp "chỉ trọng số" (weight-only, 4 bit hoặc 3 bit) giữ activation ở fp16; phương pháp W8A8 lượng tử hoá cả hai để dùng được phép nhân số nguyên.

5. Ví dụ tay: lượng tử hoá một vector sang int8

Cho $x = (0{,}3,\ -1{,}2,\ 0{,}5,\ 2{,}0,\ -0{,}8)$. Mọi số dưới đây là tự tính (đã kiểm numpy).

Bước 1, thang đo. $m = 2{,}0$; int8 có $q_{\max} = 127$; $s = 127/2 = 63{,}5$.

Bước 2, nhân và làm tròn. $s\,x = (19{,}05;\ -76{,}2;\ 31{,}75;\ 127;\ -50{,}8)$, làm tròn ra $q = (19,\ -76,\ 32,\ 127,\ -51)$.

Bước 3, khôi phục. $\hat{x} = q/s = (0{,}2992;\ -1{,}1969;\ 0{,}5039;\ 2{,}0;\ -0{,}8031)$.

Bước 4, sai số. $\hat{x} - x = (-0{,}0008;\ 0{,}0031;\ 0{,}0039;\ 0;\ -0{,}0031)$. Sai số tuyệt đối lớn nhất $0{,}0039$, nhỏ hơn nửa bước $\Delta/2 = 1/127 \approx 0{,}0079$ đúng như lý thuyết. Căn quân phương (RMS): $\sqrt{(7{,}2 \times 10^{-6})} \approx 0{,}0027$.

Cùng vector, nhưng int4 đối xứng ($q_{\max} = 7$): $s = 3{,}5$, $q = (1,\ -4,\ 2,\ 7,\ -3)$, $\hat{x} = (0{,}286;\ -1{,}143;\ 0{,}571;\ 2{,}0;\ -0{,}857)$. Sai số lớn nhất $0{,}0714$ (đúng bằng $\Delta/2 = 1/(2 \cdot 3{,}5)$) và RMS $\approx 0{,}049$. Bớt 4 bit làm sai số lớn khoảng 18 lần ($0{,}0714/0{,}0039$, tự tính).

Thêm một ngoại lai. Cho $x' = (0{,}1,\ -0{,}5,\ 2{,}0,\ 0{,}03,\ 60)$ ở int8: $m = 60$, $s = 127/60 \approx 2{,}117$; $s\,x' = (0{,}21;\ -1{,}06;\ 4{,}23;\ 0{,}06;\ 127)$, làm tròn $q = (0,\ -1,\ 4,\ 0,\ 127)$; khôi phục $(0;\ -0{,}472;\ 1{,}890;\ 0;\ 60)$. Hai giá trị nhỏ $0{,}1$ và $0{,}03$ biến mất thành 0, và $2{,}0$ thành $1{,}89$ (sai $0{,}11$). Con số 60 chiếm 127 mức, 4 số còn lại chia sẻ vài mức. Đó là bệnh của ngoại lai.

Ví dụ có điểm không. Dữ liệu trong $[-1, 3]$: $\Delta = 4/255 \approx 0{,}01569$, $z = \text{round}(1/0{,}01569) - 128 = 64 - 128 = -64$. Giá trị 0 ánh xạ đúng đến mã -64 và khôi phục đúng 0; giá trị 3 ánh xạ đến mã 127 (khôi phục $2{,}996$); giá trị -1 ánh xạ đến mã -128 (khôi phục $-1{,}0039$) (tự tính).

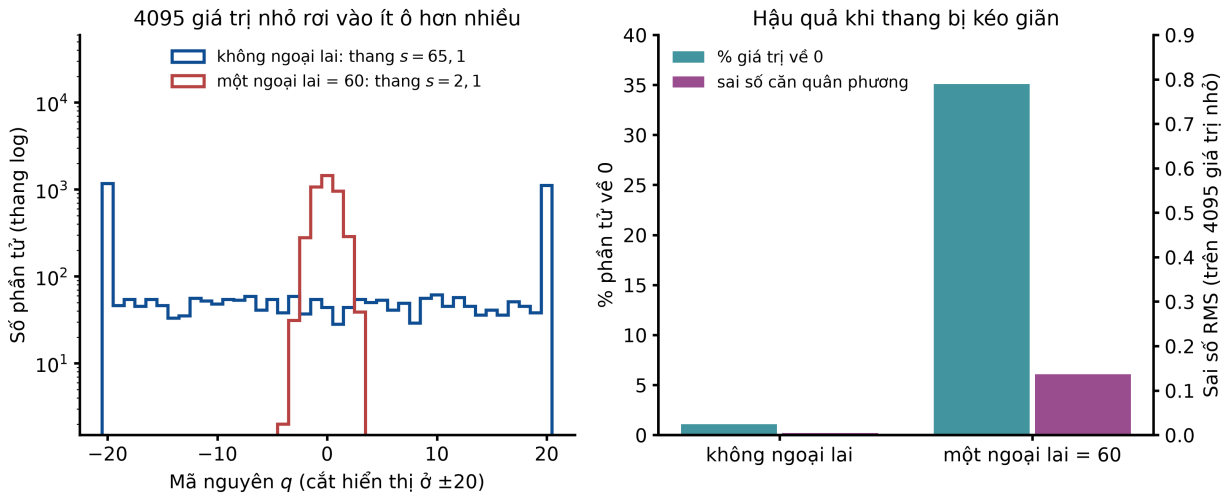
6. Ngoại lai (outlier) trong LLM

6.1 Một ngoại lai kéo giãn cả thang đo

Cách đọc hình: mô phỏng (seed cố định) với 4096 giá trị Gauss độ lệch chuẩn $0{,}5$, lượng tử hoá int8 theo cả vector. Đường xanh (không ngoại lai) có thang $s \approx 65{,}1$: các mã trải rộng đều (phần tụ ở ± 20 là các mã bị cắt hiển thị). Đường đỏ có một giá trị bị thay bằng 60: thang tụt xuống $s \approx 2{,}1$, mã của 4095 giá trị còn lại co cụm quanh $-3, \ldots, 3$ (chỉ vài mức). Bên phải: tỉ lệ giá trị nhỏ bị làm tròn về 0 tăng từ khoảng $1{,}1\%$ lên khoảng $35{,}1\%$; sai số RMS trên các giá trị nhỏ tăng từ khoảng $0{,}0044$ lên khoảng $0{,}137$, tức hơn 30 lần (tự tính trong mô phỏng).

6.2 Ngoại lai thực sự xuất hiện trong LLM

Theo ghi chú llm 51, ở LLM từ khoảng 6,7 tỉ tham số, một vài chiều đặc trưng của hidden state có độ lớn cực lớn (từ 6 trở lên, tới khoảng 20 lần các chiều khác lúc mới xuất hiện), xuất hiện ở hầu hết token nhưng ở vài



Hình 28: Một giá trị ngoại lai kéo giãn thang đo int8: phân bố mã nguyên và hậu quả cho các giá trị nhỏ (minh hoạ, tự tính)

cột cố định, và hiện tượng này trùng với lúc lượng tử hoá int8 thông thường bắt đầu hỏng. LLM.int8() tách các cột ngoại lai (khoảng 0,1% chiều) ra nhân bằng fp16, phần còn lại bằng int8 (theo ghi chú llm 51).

Theo ghi chú llm 52, activation khó lượng tử hoá hơn trọng số vì ngoại lai cố định theo kênh; SmoothQuant dịch bớt độ khó từ activation sang trọng số bằng phép co giãn theo kênh $s_j = \max(|X_j|)^{\alpha} / \max(|W_j|)^{1-\alpha}$, một biến đổi tương đương toán học trước khi làm tròn. Theo ghi chú llm 54, AWQ cũng nhân kênh quan trọng trước khi lượng tử hoá, nhưng cho nén chỉ trọng số.

Ý chung: ngoại lai làm bước lưới quá thô cho phần đông giá trị bình thường. Có ba cách xử lý, đều xuất hiện trong cuốn này: tách ngoại lai ra xử lý riêng (llm 51), biến đổi để san bằng biên độ giữa các kênh (llm 52 và 54), hoặc lượng tử hoá có bù sai số từng lớp (llm 53).

7. Phân cấp bộ nhớ GPU

7.1 Hai tầng bộ nhớ

GPU có nhiều tầng bộ nhớ; hai tầng cần nhớ:

- **HBM** (high bandwidth memory): bộ nhớ chính của GPU, lớn (hàng chục GB) nhưng tương đối chậm.
- **SRAM** (bộ nhớ trên chip, gồm shared memory và thanh ghi): rất nhỏ (cỡ vài trăm KB mỗi đơn vị tính) nhưng nhanh hơn nhiều.

Theo ghi chú llm 44, trên A100 HBM có 40 đến 80 GB với băng thông khoảng 1,5 đến 2,0 TB/s, còn SRAM mỗi SM có 192 KB (108 SM) và nhanh hơn cỡ một bậc (paper ước khoảng 19 TB/s). Đó là các con số phần cứng cụ thể của một đời GPU, chỉ để thấy chênh lệch cỡ lớn, không phải hằng số chung.

Phép ví von: đầu bếp là bộ tính toán, mặt bàn bếp nhỏ là SRAM, kho lạnh xa là HBM. Nấu từng bước mà mỗi bước đều chạy ra kho lấy nguyên liệu rồi cất kết quả vào kho thì thời gian chủ yếu mất ở việc chạy. Cách chữa là mang đủ nguyên liệu cho cả công đoạn lên bàn, nấu xong mới cất.

7.2 Memory-bound và compute-bound

Một phép toán là **bị giới hạn bởi tính toán** (compute-bound) nếu thời gian chủ yếu là các phép nhân cộng, và **bị giới hạn bởi bộ nhớ** (memory-bound) nếu thời gian chủ yếu là đọc ghi dữ liệu. Tỉ số số phép tính trên số byte phải di chuyển gọi là cường độ số học. Các phép phần tử (softmax, dropout, cộng) có cường độ rất thấp nên gần như luôn memory-bound. Khi sinh từng token, nhân ma trận với vector cũng memory-bound:

mỗi trọng số chỉ được dùng một lần mà phải đọc từ HBM. **Đây chính là lý do lượng tử hoá trọng số làm suy luận nhanh hơn: ít byte phải đọc**, dù tính toán vẫn là số lần nhân cộng như trước.

8. Attention: vì sao ma trận $n \times n$ là gánh nặng

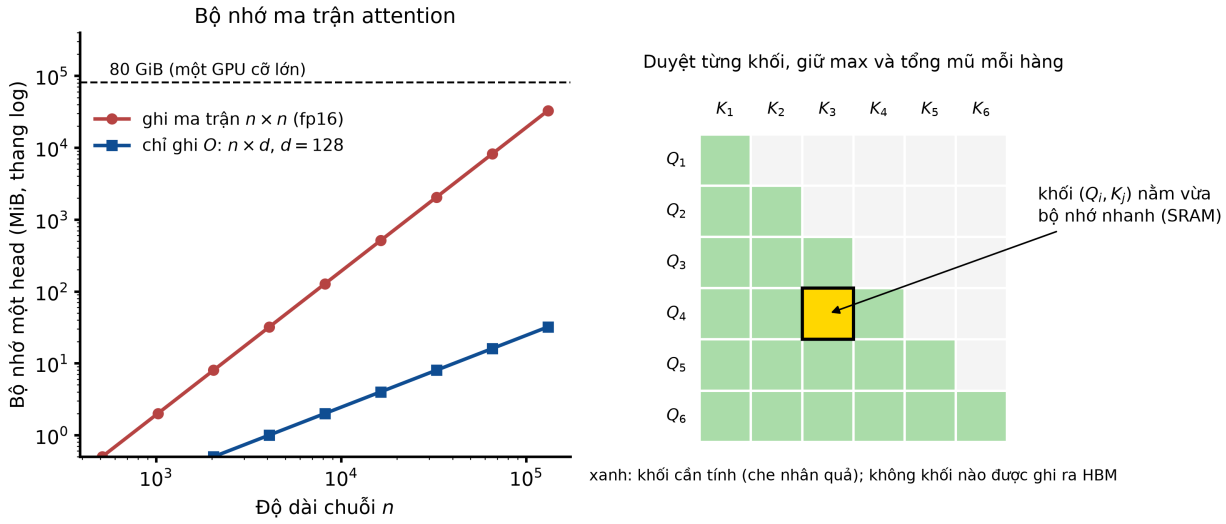
Attention chuẩn tính $S = QK^\top$ (ma trận $n \times n$ điểm số), $P = \text{softmax}(S)$, rồi $O = PV$. Cách làm ngây thơ ghi S ra HBM, đọc lại để tính P, ghi P, đọc lại để nhân V. Ma trận $n \times n$ này lớn lên theo bình phương độ dài chuỗi.

Ví dụ tay: bộ nhớ của ma trận $n \times n$ (fp16, 2 byte mỗi phần tử, một head, một chuỗi, tự tính):

n	số phần tử	bộ nhớ
1024	1.048.576	2 MiB
4096	16.777.216	32 MiB
32768	1.073.741.824	2 GiB
131072	17.179.869.184	32 GiB

Với 32 head ở $n = 32768$ thì riêng ma trận điểm số đã khoảng $32 \times 2 = 64$ GiB (tự tính). Nhân đôi độ dài thì bộ nhớ nhân bốn. Dù bộ nhớ tổng có đủ, thời gian ghi và đọc các ma trận này qua HBM cũng chiếm phần lớn thời gian của attention.

9. FlashAttention ở mức trực giác



Hình 29: Bộ nhớ ma trận attention theo độ dài chuỗi và sơ đồ tính theo khối: khối Q và K vừa SRAM, không ghi ma trận n nhân n ra HBM (minh hoạ, tự tính)

Cách đọc hình: bên trái, đường đỏ là bộ nhớ nếu ghi ma trận $n \times n$ (một head, fp16), đường xanh là bộ nhớ nếu chỉ ghi kết quả O cỡ $n \times d$ với $d = 128$ (giả định để minh hoạ): hai đường lệch nhau khi n tăng, một đường dốc gấp đôi đường kia trên thang log vì n^2 so với n. Bên phải, ma trận được chia thành lưới khối: ô vàng là một khối (Q_i, K_j) đủ nhỏ để nằm trong SRAM; các khối xanh là những khối cần tính (tam giác dưới do che nhân quả). Thuật toán đi qua từng khối, không bao giờ cần cả ma trận cùng lúc.

9.1 Tính theo khối và softmax trực tuyến

FlashAttention tính **chính xác** cùng kết quả như attention chuẩn, nhưng chia Q, K, V thành khối, nạp từng khối vào SRAM, tính điểm, softmax và nhân với V ngay tại chỗ, chỉ ghi O ra HBM (theo ghi chú llm 44).

Khó khăn: softmax cần chuẩn hoá trên cả hàng, mà mỗi lần ta chỉ thấy một khối. Cách chữa là **softmax trực tuyến**: với mỗi hàng giữ hai số, giá trị lớn nhất đã thấy m và tổng mũ $\ell = \sum e^{s-m}$, và một tích lũy $o = \sum e^{s-m} v$. Khi thấy khối mới có m' lớn hơn, nhân kết quả cũ với $e^{m-m'}$ để đổi về thang mới.

9.2 Ví dụ tay: softmax trực tuyến trên 2 khối

Một hàng có bốn điểm $s = (2, 1, 3, 0)$ và các giá trị $v = (1, 2, 3, 4)$ (số vô hướng cho gọn). Đáp án chuẩn: $\sum_i \text{softmax}(s)_i v_i \approx 2{,}4711$ (tự tính).

Khối 1 $= \{2, 1\}$ với $v = \{1, 2\}$: $m = 2$; $\ell = e^0 + e^{-1} = 1{,}3679$; $o = 1 \cdot 1 + e^{-1} \cdot 2 = 1{,}7358$. Kết quả tạm $o/\ell \approx 1{,}2689$ (chưa phải đáp án).

Khối 2 $= \{3, 0\}$ với $v = \{3, 4\}$: max mới $m' = 3$. Hệ số đổi thang $e^{m-m'} = e^{-1} \approx 0{,}3679$. $\ell' = \ell\, e^{-1} + e^0 + e^{-3} = 0{,}5032 + 1 + 0{,}0498 = 1{,}5530$; $o' = o\, e^{-1} + 3 \cdot e^0 + 4 \cdot e^{-3} = 0{,}6386 + 3 + 0{,}1991 = 3{,}8377$. Cuối cùng $o'/\ell' = 3{,}8377/1{,}5530 \approx 2{,}4711$ (tự tính), **trùng đúng đáp án chuẩn**, dù ta không bao giờ giữ cả bốn điểm cùng lúc. Chuẩn hoá một lần ở cuối là đủ.

9.3 Cái được và cái trả giá

Số phép tính (FLOPs) gần như không đổi, thậm chí tăng nhẹ vì phần backward tính lại các điểm số thay vì lưu chúng. Cái giảm mạnh là số lần đọc ghi HBM, và bộ nhớ phụ trợ giảm từ bậc hai xuống bậc tuyến tính theo n (theo ghi chú llm 44). Vì chuỗi các bước vẫn là phép tính chính xác, FlashAttention không phải là xấp xỉ attention: nó chỉ đổi thứ tự và nơi diễn ra phép tính. Đây là ví dụ mẫu của tư duy "bớt chuyến đi hơn là bớt phép tính".

Kích thước khối được chọn sao cho các khối Q, K, V và khối điểm số vừa SRAM; ghi chú llm 44 nêu công thức $B_c = \lceil M/4d \rceil$ với M là kích thước SRAM tính theo phần tử.

10. Biến thể và chi tiết hay bị bỏ qua

Làm tròn gần nhất so với làm tròn ngẫu nhiên. Chương dùng làm tròn gần nhất. Có kỹ thuật làm tròn thích nghi theo dữ liệu (ví dụ GPTQ bù sai số từng lớp, theo ghi chú llm 53) cho kết quả tốt hơn làm tròn gần nhất ở 3 đến 4 bit.

Cắt (clipping). Chọn m nhỏ hơn giá trị tuyệt đối lớn nhất, chấp nhận cắt vài ngoại lai để bước lưới mịn hơn cho phần còn lại. Đánh đổi sai số làm tròn với sai số cắt. Hình ở mục 6 là trường hợp không cắt.

Hạt (granularity) và phần cứng. Thang đo theo kênh trong chiều tích vô hướng làm phép nhân số nguyên khó chạy bằng kernel chuẩn (theo ghi chú llm 52: scale chỉ áp được ở các chiều ngoài), nên SmoothQuant tìm cách đưa ngoại lai về trọng số. Tương thích kernel quyết định nhiều chọn lựa thiết kế.

Lượng tử hoá KV cache. Ngoài trọng số, bộ nhớ đệm KV khi sinh văn bản cũng chiếm nhiều bộ nhớ; kỹ thuật tương tự áp dụng được, nhưng chương này không đi sâu.

Nén trọng số không cứu được attention. Lượng tử hoá giảm bộ nhớ trọng số; FlashAttention giảm bộ nhớ và truy cập của activation attention. Hai việc độc lập và cộng được với nhau.

11. Nối sang các bài paper của cuốn

llm 44 (FlashAttention). Bài trình bày mô hình IO hai tầng (HBM và SRAM), tính theo khối, softmax trực tuyến, tính lại trong backward và phân tích số lần truy cập HBM. Mục 7 đến 9 của chương này là phần nền cần có trước khi đọc.

llm 45 (FlashAttention-2). Cùng ý tưởng, tối ưu cách chia việc giữa các đơn vị tính; chương này chỉ cung cấp trực giác chung.

llm 51 (LLM.int8). Bài chẩn đoán ngoại lai hệ thống từ khoảng 6,7 tỉ tham số và giải bằng tách cột ngoại lai fp16, theo ghi chú llm 51. Ví dụ ngoại lai 60 ở mục 5 trùng ví dụ của ghi chú.

llm 52 (SmoothQuant). Bài chuyển độ khó lượng tử hoá từ activation sang trọng số để dùng kernel INT8 thuần.

llm 53 (GPTQ) và llm 54 (AWQ). Hai bài về nén chỉ trọng số xuống 3 đến 4 bit; chương này cung cấp khái niệm scale, nhóm và điểm không, cùng chi phí lưu thang đo (mục 4.3).

Liên hệ MoE. MoE (nen07 của cuốn này) làm bộ nhớ trọng số trở thành nút thắt; lượng tử hoá là một công cụ chính để chứa chúng.

12. Sai lầm hay gặp

- **Cho rằng 4 bit nghĩa là sai số bằng một phần tư của 8 bit.** Số mức giảm từ 255 xuống 15, nên bước lớn hơn khoảng 17 lần: trong ví dụ mục 5, sai số lớn nhất tăng từ $0{,}0039$ lên $0{,}0714$.
- **Quên rằng thang đo cũng tốn bộ nhớ.** Nhóm nhỏ cho độ chính xác tốt hơn nhưng làm số bit hiệu dụng cao hơn (nhóm 128 với scale fp16 thêm $0{,}125$ bit).
- **Lượng tử hoá cả vector theo giá trị lớn nhất mà không nhìn phân bố.** Một ngoại lai làm hàng chục phần trăm giá trị còn lại về 0 (mô phỏng mục 6).
- **Nhầm FlashAttention là attention xấp xỉ.** Nó chính xác; thay đổi nằm ở thứ tự tính và việc không ghi S, P ra HBM.
- **Nghĩ rằng giảm FLOPs là cách duy nhất làm nhanh.** Với phép memory-bound như attention và nhân ma trận-vector khi sinh token, giảm byte phải đọc mới là chìa khoá.
- **Dùng số của một đời GPU làm quy luật chung.** 1,5 đến 2,0 TB/s và 192 KB SRAM là số của A100 theo ghi chú llm 44; các thế hệ sau khác.

13. Tóm tắt công thức

Đại lượng	Công thức	Ghi chú
Thang đo đối xứng	$s = q_{\max}/m$, $q_{\max} = 2^{b-1}-1$	m là giá trị tuyệt đối lớn nhất
Mã và khôi phục	$q = \mathrm{round}(sx)$, $\hat{x} = q/s$	sai số $\le 1/(2s)$ khi không bị cắt
Có điểm không	$\Delta = (c-a)/255$, $\hat{x} = (q-z)\Delta$	dữ liệu lệch quanh 0
Chi phí nhóm	$16/g$ bit mỗi trọng số	g là kích thước nhóm, scale fp16
Ma trận điểm số	$2n^2$ byte (fp16, một head)	tăng bậc hai theo n
Softmax trực tuyến	$m' = \max(m, m_{\text{mới}})$, $\ell' = \ell e^{m-m'} + \dots$	cho kết quả chính xác

14. Bài tập tự tính

Bài 1. Lượng tử hoá $x = (0{,}04,\ 0{,}2,\ -0{,}1,\ 1{,}0)$ sang int8 đối xứng. Tìm s, q và $\hat{x}$.

Đáp án (đã kiểm numpy): $m = 1$, $s = 127$; $q = (5,\ 25,\ -13,\ 127)$ (vì $0{,}04 \cdot 127 = 5{,}08$, $0{,}2 \cdot 127 = 25{,}4$, $-0{,}1 \cdot 127 = -12{,}7$); $\hat{x} \approx (0{,}0394;\ 0{,}1969;\ -0{,}1024;\ 1{,}0)$.

Bài 2. Một vector có $\max|x| = 3$. Lượng tử hoá giá trị $x = 1{,}0$ bằng int8 rồi bằng int4 đối xứng. Tính sai số tuyệt đối mỗi trường hợp.

Đáp án (đã kiểm numpy): int8: $s = 127/3 \approx 42{,}33$, $q = 42$, $\hat{x} \approx 0{,}9921$, sai số $\approx 0{,}0079$. int4: $s = 7/3 \approx 2{,}333$, $q = 2$, $\hat{x} \approx 0{,}8571$, sai số $\approx 0{,}1429$ (khoảng 18 lần).

Bài 3. Vector $(0{,}4,\ -0{,}3,\ 0{,}2,\ 8)$ lượng tử hoá int4 đối xứng. Điều gì xảy ra với ba giá trị nhỏ?

Đáp án (đã kiểm numpy): $s = 7/8 = 0{,}875$; $s\,x = (0{,}35;\ -0{,}2625;\ 0{,}175;\ 7)$ làm tròn $(0,\ 0,\ 0,\ 7)$. Cả ba giá trị nhỏ bị xoá thành 0; chỉ ngoại lai 8 được giữ nguyên.

Bài 4. Mô hình 7 tỉ tham số. Tính bộ nhớ trọng số ở fp16, int8 và int4 (GB, 10^9 byte). Với nhóm 128 và scale fp16, số bit trung bình mỗi trọng số ở "4 bit" là bao nhiêu?

Đáp án (đã kiểm numpy): 14 GB, 7 GB, $3{,}5$ GB (chưa tính thang đo); số bit trung bình $4 + 16/128 = 4{,}125$ bit, tức khoảng $3{,}6$ GB nếu tính cả thang đo ($7 \times 10^9 \times 4{,}125/8$ byte $\approx 3{,}61$ GB).

Bài 5. Tính bộ nhớ ma trận điểm số fp16 một head ở $n = 2048, 8192, 16384$, và tổng cho 32 head ở $n = 16384$. Sau đó, nếu mọi giá trị $v_i = 1$ thì kết quả attention cho một hàng bằng bao nhiêu, và vì sao điều này là phép thử tiện lợi cho softmax trực tuyến?

Đáp án (đã kiểm numpy): $2048^2 \cdot 2 = 8$ MiB; $8192^2 \cdot 2 = 128$ MiB; $16384^2 \cdot 2 = 512$ MiB; 32 head: 16 GiB. Nếu mọi $v_i = 1$ thì kết quả bằng 1, vì các trọng số softmax cộng lại bằng 1 bất kể chia khối thế nào. Với softmax trực tuyến, khi $v \equiv 1$ thì o và ℓ được cập nhật bằng cùng một công thức nên $o/\ell = 1$ ở mọi bước: đây là phép thử nhanh cho một cài đặt.

Tài liệu tham khảo nền: Dettmers et al. (LLM.int8), Xiao et al. (SmoothQuant), Frantar et al. (GPTQ), Lin et al. (AWQ), Dao et al. (FlashAttention), và các ghi chú llm 44, 45, 51, 52, 53, 54 trong cuốn này.

Nền tảng 09

Mô hình thị giác-ngôn ngữ: biến ảnh thành token để LLM đọc

Loại: chương nền tảng (không phải ghi chú paper)

Đọc trước: vector và ma trận ở math nền tảng 02; cosine và chuẩn ở math nền tảng 13; softmax, cross-entropy ở math nền tảng 08 và 11; CNN ở ML cơ bản nền tảng 07; attention và Transformer ở bài 01 của cuốn này; nen07 và nen08 của cuốn này (cho chi phí bộ nhớ)

Dùng ở: llm 29 (ViT), llm 30 (CLIP), llm 31 (SigLIP), llm 32, llm 33, llm 34, llm 35, llm 36, llm 37, llm 38 và llm 57 (cắt token ảnh)

> Chương này giải thích từ đầu, không phải bản dịch một paper. Các phép tính ghi "tự tính" đã kiểm bằng numpy; các hình là minh hoạ với dữ liệu tự sinh, không phải ảnh hay số đo thật từ một mô hình.

1. Vì sao cần khái niệm này

Mô hình thị giác-ngôn ngữ (VLM, vision-language model) là mô hình nhận cả ảnh lẫn chữ và trả lời bằng chữ. LLM chỉ đọc được chuỗi token, còn ảnh là một lưới điểm ảnh. Mọi VLM trong cuốn này trả lời cùng một câu hỏi: làm sao biến ảnh thành thứ mà LLM đọc như đọc chữ.

Lời giải chung có ba mảnh. Mảnh một: cắt ảnh thành các ô vuông nhỏ (patch) và coi mỗi ô là một token, rồi dùng Transformer để mã hoá (ViT). Mảnh hai: dạy bộ mã hoá ảnh hiểu ảnh theo ngôn ngữ bằng học tương phản trên hàng loạt cặp ảnh-chữ (CLIP). Mảnh ba: nối đầu ra của bộ mã hoá ảnh vào LLM qua một cầu nối nhỏ gọi là projector. Hệ quả kinh tế là một ảnh biến thành hàng trăm đến hàng nghìn token, và số token ấy chi phối chi phí chạy mô hình, nên độ phân giải là đánh đổi chính.

2. Nhắc nền: ảnh là một mảng số

2.1 Ảnh và byte

Một ảnh màu $H \times W$ có ba kênh (đỏ, xanh lá, xanh dương): mảng $H \times W \times 3$. Ảnh 224×224 có $224 \cdot 224 \cdot 3 = 150.528$ số. Điểm ảnh gần nhau thường tương quan mạnh (một mảng trời xanh có hàng trăm điểm gần như giống nhau), nên cần gom chúng lại thành đơn vị lớn hơn trước khi đưa vào Transformer.

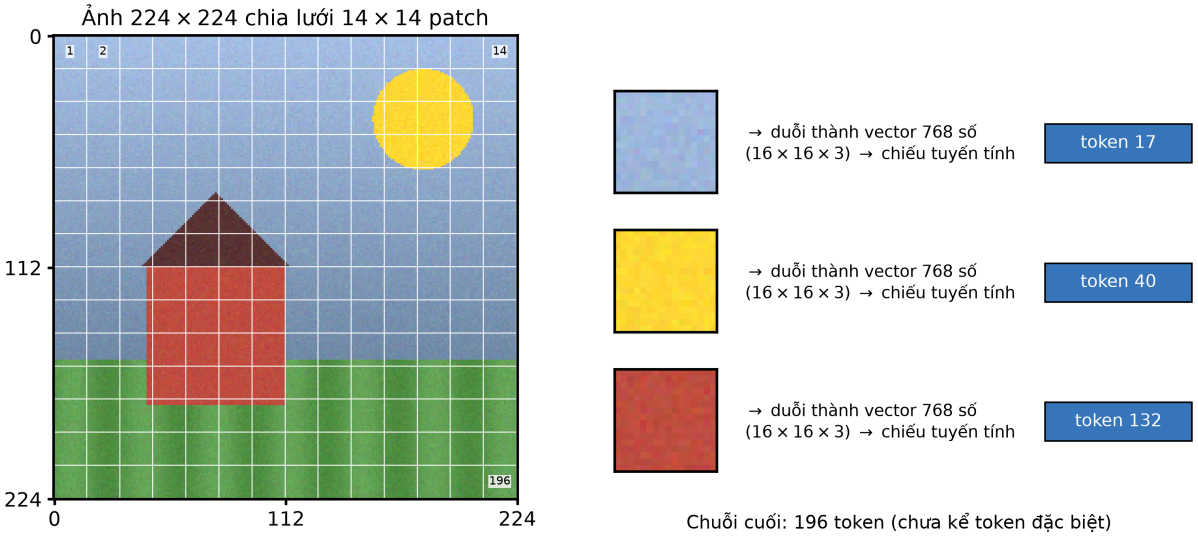
2.2 Vì sao không coi mỗi điểm ảnh là một token

Attention có chi phí bậc hai theo số token. Nếu mỗi điểm ảnh là một token, ảnh 224×224 có 50.176 token và ma trận attention có khoảng $2{,}5$ tỉ phần tử (theo ghi chú llm 29). Tự tính: $50176^2 \approx 2{,}52 \times 10^9$ phần tử, tức khoảng 5 GB ở fp16 cho một head của một lớp (nhắc lại bài học ở nen08: ma trận $n \times n$ phình theo bình phương). Cắt thành patch là cách hạ n xuống.

2.3 Cosine giữa hai vector

Cosine của hai vector u, v là $u^\top v / (\|u\| \|v\|)$; nếu cả hai đã chuẩn hoá về độ dài 1 thì chỉ là tích vô hướng. Giá trị trong $[-1, 1]$: càng gần 1 thì hai vector càng cùng hướng. CLIP dùng cosine để đo "ảnh này khớp câu chữ này đến mức nào".

3. Từ ảnh thành token: cắt patch (ViT)



Hình 30: Ảnh 224 nhân 224 chia thành lưới 14 nhân 14 patch 16 nhân 16; mỗi patch duỗi thành vector 768 số rồi chiếu thành một token (minh hoạ, tự tính)

3.1 Trực giác bằng hình

Cách đọc hình: ảnh tổng hợp (trời, mặt trời, nhà, cỏ) được kẻ lưới 14×14, mỗi ô là một patch 16×16 điểm ảnh; các ô được đánh số từ 1 đến 196 theo thứ tự đọc (trái sang phải, trên xuống dưới), chỉ vài số được in. Bên phải lấy ba patch (một ô trời, một ô mặt trời, một ô tường nhà), mỗi patch có $16 \cdot 16 \cdot 3 = 768$ số, được duỗi thành một vector rồi nhân với một ma trận để thành một token. Hình dung đơn giản: mỗi patch là một "từ", cả ảnh là một "câu" gồm 196 từ.

3.2 Định nghĩa

Ảnh $H \times W \times C$ chia thành các patch $P \times P$; số patch là $N = (H/P) \cdot (W/P)$. Mỗi patch được duỗi thành vector độ dài P^2C rồi nhân với ma trận học được $E \in \mathbb{R}^{(P^2C) \times D}$ để ra token chiều D ("patch embedding"). Thêm một token đặc biệt (`[class]`) và embedding vị trí học được, rồi cho chuỗi qua Transformer encoder chuẩn (theo ghi chú llm 29). Vì mỗi patch được chiếu giống nhau, bước này cài đặt được bằng một phép tích chập có nhân $P \times P$ và bước nhảy P (nhận xét trong ghi chú llm 29, paper mô tả là chiếu tuyến tính).

3.3 Ví dụ tay: đếm token và tham số

Ảnh 224×224, patch 16×16 (tất cả tự tính):

- Số patch mỗi chiều: $224/16 = 14$; tổng $N = 14 \cdot 14 = 196$ token (chưa kể token `[class]`).
- Mỗi patch có $16 \cdot 16 \cdot 3 = 768$ số. Với $D = 768$ (cỡ Base), ma trận E có $768 \cdot 768 = 589.824$ trọng số, cộng 768 bias là 590.592 tham số.
- Patch 32×32: $224/32 = 7$, $N = 49$ token. Patch 14×14: $224/14 = 16$, $N = 256$ token (khớp ghi chú llm 29 nêu 49, 196, 256).
- Kích thước ma trận attention mỗi head: $196^2 = 38.416$ phần tử, khoảng $0{,}07$ MiB ở fp16, so với 5 GB của cách coi mỗi điểm ảnh là token (tỉ lệ khoảng 65.000 lần, tự tính).

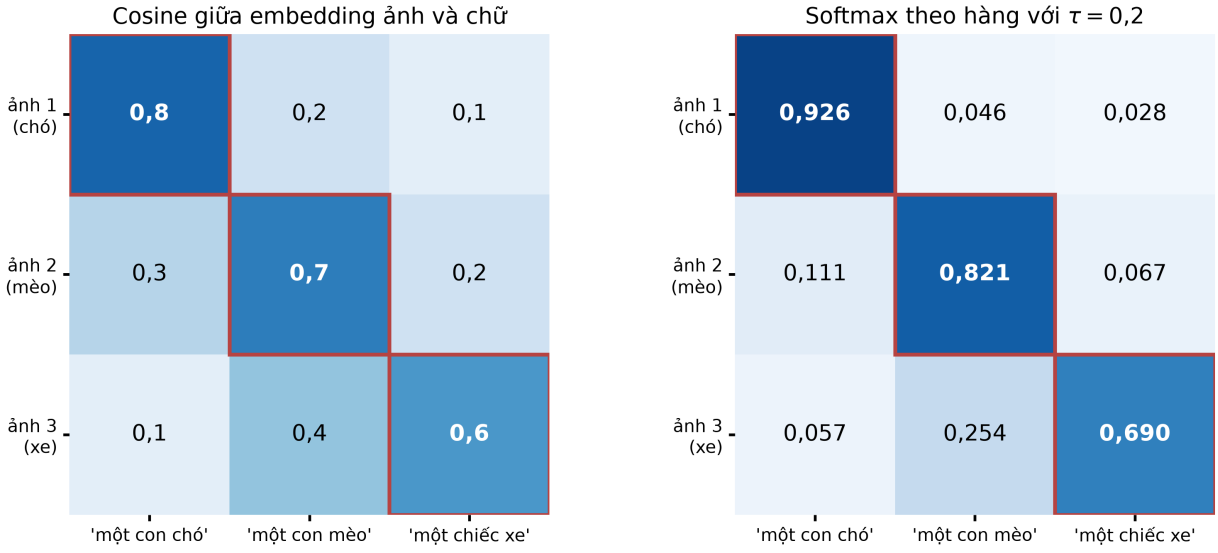
Patch càng nhỏ thì token càng nhiều và đắt hơn, nhưng mịn hơn: đánh đổi chính của cắt patch. Một chi tiết hay bị bỏ qua: sau khi cắt patch, ViT không còn biết sẵn "hai patch kề nhau trong ảnh"; quan hệ 2D phải học qua embedding vị trí và attention (theo ghi chú llm 29, cấu trúc lân cận 2D chỉ được dùng lúc cắt patch và lúc nội suy embedding vị trí khi đổi độ phân giải).

4. Học tương phản: CLIP

4.1 Mục tiêu

Muốn một bộ mã hoá ảnh biết "ảnh này nói về gì" mà không cần nhãn thủ công cho từng lớp. CLIP học từ rất nhiều cặp (ảnh, câu mô tả) thu thập từ Internet: hai bộ mã hoá (một cho ảnh, một cho chữ) chiếu vào cùng một không gian vector, sao cho ảnh và câu mô tả của nó nằm gần nhau, ảnh và câu mô tả của ảnh khác nằm xa nhau. Theo ghi chú llm 30, CLIP dùng 400 triệu cặp và batch 32.768.

4.2 Trực giác bằng hình: ma trận tương đồng



Hình 31: Ma trận cosine giữa 3 ảnh và 3 câu mô tả, và xác suất softmax theo hàng với nhiệt độ 0,2; đường chéo là cặp đúng (minh hoạ, tự tính)

Cách đọc hình: các số cosine ở hình trái là giả định, chọn để ví dụ gọn, không đo từ mô hình thật. Mỗi hàng là một ảnh, mỗi cột là một câu mô tả; ô trên đường chéo (viền đỏ) là cặp thật (ảnh chó với "một con chó", v.v.). Hình phải là softmax theo từng hàng, nhiệt độ $\tau = 0{,}2$: nó biến cosine thành "xác suất" rằng ảnh này khớp với câu nào. Mục tiêu huấn luyện là kéo các ô chéo lên gần 1 và các ô ngoài chéo xuống gần 0. Hàng 3 (xe) có xác suất đúng thấp nhất ($0{,}690$) vì cosine với "con mèo" ($0{,}4$) khá gần cosine đúng ($0{,}6$).

4.3 Định nghĩa: loss tương phản đối xứng

Với batch N cặp, gọi u_i là embedding ảnh đã chuẩn hoá L2, v_j là embedding chữ đã chuẩn hoá, $S_{ij} = u_i^\top v_j$ và $\tau > 0$ là nhiệt độ (CLIP học nhiệt độ, theo ghi chú llm 30). Loss là cross-entropy theo cả hai chiều với nhãn là đường chéo:

$$\mathcal{L}_{\text{ảnh}\to\text{chữ}} = -\frac{1}{N}\sum_{i=1}^{N} \log \frac{e^{S_{ii}/\tau}}{\sum_j e^{S_{ij}/\tau}}, \qquad \mathcal{L} = \frac{1}{2}\left(\mathcal{L}_{\text{ảnh}\to\text{chữ}} + \mathcal{L}_{\text{chữ}\to\text{ảnh}}\right),$$

trong đó $\mathcal{L}_{\text{chữ}\to\text{ảnh}}$ chuẩn hoá theo cột thay vì theo hàng. Mỗi ảnh coi $N-1$ câu còn lại trong batch là mẫu âm; vì vậy batch lớn cho nhiều mẫu âm hơn (theo ghi chú llm 30, mỗi ảnh có 32.767 mẫu âm ở batch 32.768). Nhiệt độ nhỏ làm softmax "nhọn" hơn, phạt nặng những ô sai có cosine cao.

4.4 Ví dụ tay: loss tương phản cho 3 cặp

Dùng ma trận cosine ở hình, $\tau = 0{,}2$, nên logit $= S/\tau$ (nhân 5):

$$S = \begin{pmatrix} 0{,}8 & 0{,}2 & 0{,}1 \\ 0{,}3 & 0{,}7 & 0{,}2 \\ 0{,}1 & 0{,}4 & 0{,}6 \end{pmatrix}, \qquad S/\tau = \begin{pmatrix} 4 & 1 & 0{,}5 \\ 1{,}5 & 3{,}5 & 1 \\ 0{,}5 & 2 & 3 \end{pmatrix}.$$

Hàng 1 (ảnh chó): mẫu số $e^4 + e^1 + e^{0,5} \approx 54{,}60 + 2{,}72 + 1{,}65 = 58{,}97$; xác suất đúng $54{,}60/58{,}97 \approx 0{,}926$; loss $-\ln 0{,}926 \approx 0{,}0769$ (tự tính). **Hàng 2:** xác suất đúng $\approx 0{,}821$, loss $\approx 0{,}1967$. **Hàng 3:** xác suất đúng $\approx 0{,}690$, loss $\approx 0{,}3715$. Trung bình theo hàng: $\mathcal{L}_{\text{ảnh}\to\text{chữ}} \approx 0{,}2151$.

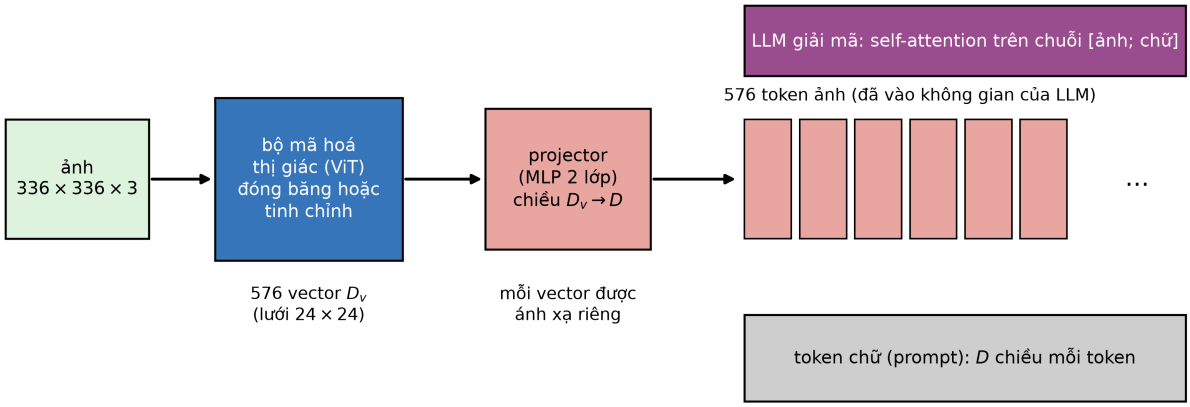
Theo cột (chữ chọn ảnh): loss ba cột lần lượt $\approx 0{,}1064;\ 0{,}2664;\ 0{,}1967$, trung bình $\approx 0{,}1898$. **Loss tổng** $\approx (0{,}2151 + 0{,}1898)/2 \approx 0{,}2025$ (tự tính). Tham chiếu: nếu mô hình không biết gì và đoán đều, mỗi xác suất là $1/3$ và loss là $\ln 3 \approx 1{,}0986$. Giá trị $0{,}2025$ nhỏ hơn nhiều, tương ứng với ma trận gần chéo. Nếu đối τ từ $0{,}2$ lên 1, softmax phẳng hơn và loss sẽ lớn hơn dù cosine không đổi.

4.5 Dùng CLIP

Sau huấn luyện, phân loại ảnh "không cần dữ liệu nhãn" bằng cách viết tên lớp thành câu ("một bức ảnh của một con chó"), mã hoá câu, và chọn câu có cosine cao nhất với ảnh (theo ghi chú llm 30). Quan trọng hơn cho VLM: bộ mã hoá ảnh của CLIP, vì đã học theo ngôn ngữ, cho các vector đặc trưng "gần với chữ", nên là nguyên liệu tốt để nối vào LLM.

5. Nối bộ mã hoá ảnh vào LLM: projector

5.1 Trực giác bằng hình



Hình 32: Đường đi của ảnh qua bộ mã hoá thị giác, projector rồi vào chuỗi cùng token chữ trong LLM (minh hoạ, tự tính)

Cách đọc hình: từ trái sang phải. Ảnh 336×336 vào bộ mã hoá thị giác (ViT) và ra một lưới vector (ở patch 14 thì $24 \times 24 = 576$ vector, mỗi vector D_v chiều). Projector biến từng vector D_v chiều thành vector D chiều của LLM; 576 vector sau đó trở thành 576 "token ảnh" đặt cạnh các token chữ của câu hỏi trong cùng một chuỗi. LLM không cần biết token nào đến từ ảnh: nó chạy self-attention trên toàn chuỗi như bình thường. Hình là sơ đồ khái niệm theo kiểu LLaVA, không mô tả đúng mọi chi tiết của từng mô hình.

5.2 Định nghĩa

Gọi $Z_v \in \mathbb{R}^{n \times D_v}$ là đầu ra của bộ mã hoá ảnh (n vector). Projector f biến Z_v thành $H_v = f(Z_v) \in \mathbb{R}^{n \times D}$ (cùng chiều với embedding chữ của LLM). Theo ghi chú llm 34, LLaVA dùng một lớp tuyến tính $H_v =$

WZ_v; theo ghi chú llm 35, LLaVA-1.5 thay bằng MLP hai lớp $H_v = W_2\sigma(W_1 Z_v)$. Chuỗi đưa vào LLM là $[H_v;\ H_{\text{chữ}}]$, và loss ngôn ngữ chỉ tính trên các token trả lời. Mỗi vector ảnh được chiếu độc lập: projector không trộn thông tin giữa các vị trí ảnh.

5.3 Ví dụ tay: tham số của projector

Giả định minh hoạ (không lấy từ paper): $D_v = 1024$ và $D = 4096$ cho MLP hai lớp $1024 \to 4096 \to 4096$ với bias (tự tính):

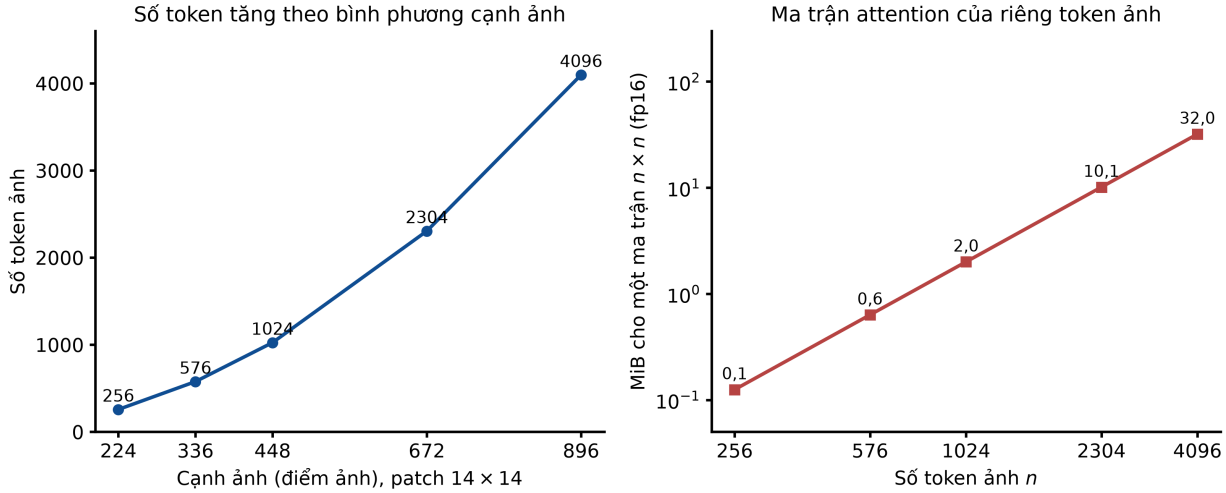
$$1024 \cdot 4096 + 4096 + 4096 \cdot 4096 + 4096 = 4.194.304 + 4.096 + 16.777.216 + 4.096 = 20.979.712.$$

Khoảng 21 triệu tham số, rất nhỏ so với LLM vài tỉ. Đó là lý do nhiều công thức huấn luyện hai giai đoạn: giai đoạn đầu chỉ huấn luyện projector (đóng băng bộ mã hoá ảnh và LLM) để căn chỉnh hai không gian, giai đoạn sau tinh chỉnh thêm với dữ liệu chỉ dẫn; theo ghi chú llm 34 giai đoạn đầu đóng băng cả ViT và LLM, giai đoạn sau đóng băng ViT.

5.4 Các cách nối khác

Ghi chú trong repo mô tả ba họ chính. **Chiếu thẳng rồi nối vào chuỗi** (LLaVA, nêu trên): đơn giản, giữ nguyên số token. **Nút thắt học được** (BLIP-2, llm 33): một Q-Former với 32 query học được chỉ cho ra 32×768 vector, bất kể ảnh to hay nhỏ (theo ghi chú llm 33). **Cross-attention chèn vào LLM** (Flamingo, llm 32): token ảnh không nối vào chuỗi mà được LLM truy cập qua lớp cross-attention. Hai họ đầu cho chi phí khác nhau: nút thắt cho số token cố định và nhỏ, còn chiếu thẳng giữ nhiều chi tiết hơn nhưng số token tăng theo độ phân giải.

6. Số token ảnh và chi phí



Hình 33: Số token ảnh theo cạnh ảnh với patch 14 và kích thước ma trận attention tương ứng (minh hoạ, tự tính)

Cách đọc hình: trục ngang bên trái là cạnh ảnh vuông (điểm ảnh), patch 14×14; số token là $(\text{cạnh}/14)^2$ nên tăng theo bình phương cạnh: $224 \to 256$, $336 \to 576$, $448 \to 1024$, $672 \to 2304$, $896 \to 4096$ (tự tính). Bên phải là kích thước ma trận attention $n \times n$ cho riêng các token ảnh ở fp16 (một head, một lớp): $0{,}125$ MiB ở $n = 256$, $0{,}63$ MiB ở 576, 2 MiB ở 1024, $10{,}1$ MiB ở 2304 và 32 MiB ở 4096. Nhớ rằng mỗi lớp và mỗi head có một ma trận như thế (kể cả khi FlashAttention không ghi chúng ra HBM, FLOPs vẫn tỉ lệ n^2).

6.1 Ví dụ tay: câu hỏi ngắn kèm một ảnh

LLaVA-1.5 dùng ảnh 336 ($24 \times 24 = 576$ token) theo ghi chú llm 35. Giả định câu hỏi và câu trả lời dài khoảng 100 token chữ (giả định). Tổng chuỗi: $576 + 100 = 676$ token (tự tính). Token ảnh chiếm $576/676 \approx 85\%$ chuỗi. Với attention đầy đủ, số cặp điểm số là $676^2 = 456.976$, so với $100^2 = 10.000$ nếu chỉ có chữ: gấp khoảng 46 lần (tự tính). Ảnh vì thế tốn bộ nhớ, thời gian prefill và KV cache nhiều hơn rất nhiều so với chữ cùng ngữ cảnh.

6.2 Độ phân giải: được và mất

Ảnh to hơn cho chi tiết hơn (chữ nhỏ trong tài liệu, vật nhỏ), nhưng số token tăng bậc hai. Các bài trong cuốn đưa ra các cách khác nhau để cân bằng, nêu theo ghi chú:

- **LLaVA-1.5 (llm 35):** nâng ViT từ 224 lên 336 px, số token từ 256 lên 576 (patch 14, tự tính), thời gian huấn luyện khoảng gấp đôi; thêm biến thể HD chia ảnh thành lưới.
- **InternVL 1.5 (llm 36):** độ phân giải động, chia ảnh thành 1 đến 12 tile 448×448 kèm một thumbnail; pixel shuffle đưa mỗi tile từ $32 \times 32 = 1024$ token xuống 256 token, tức còn $1/4$ (nhận định "1/4" và 256 theo ghi chú llm 36).
- **Qwen2-VL (llm 37):** độ phân giải gần gốc, số token thay đổi theo ảnh, và một MLP gộp 2×2 token kề thành 1 (ví dụ ảnh 224×224, patch 14: 256 patch, gộp còn 64, cộng 2 token đặc biệt là 66, theo ghi chú llm 37).
- **SmolVLM (llm 38):** nén token mạnh hơn (pixel shuffle hệ số 4 so với 2 ở VLM lớn) để mô hình nhỏ chạy được.
- **FastV (llm 57):** cắt bớt token ảnh trong lúc suy luận. Theo ghi chú llm 57, ở lớp sâu token ảnh nhận rất ít attention nên cắt 50% token ảnh sau lớp 2 giảm khoảng 45% FLOPs mà gần như không mất điểm (kết quả của paper cho LLaVA-1.5-13B, không phải suy luận của chương này).

Pixel shuffle (space-to-depth) là phép gộp một khối $r \times r$ vị trí thành một vị trí có số kênh nhân r^2: số token giảm r^2 lần, mỗi token rộng hơn r^2 lần, rồi projector đưa về chiều của LLM. Thông tin không bị bỏ, chỉ được nén vào chiều sâu.

7. Biến thể và chi tiết hay bị bỏ qua

CLIP so với SigLIP. SigLIP thay softmax toàn batch bằng loss sigmoid theo từng cặp (theo ghi chú llm 31), tránh việc chuẩn hoá trên cả batch và đỡ phụ thuộc batch khổng lồ. Ma trận tương đồng vẫn là ma trận $N \times N$, nhưng mỗi ô được xét độc lập là "khớp hay không".

Bộ mã hoá thị giác đóng băng hay tinh chỉnh. Đóng băng giữ nguyên chất lượng biểu diễn ảnh, rẻ, nhưng giới hạn khả năng thích nghi (ví dụ với tài liệu, OCR). Tinh chỉnh toàn bộ tốn hơn và có thể làm hỏng biểu diễn nếu dữ liệu ít.

Token đặc biệt và vị trí. Ảnh thường được bọc bằng token đánh dấu bắt đầu và kết thúc ảnh; mô hình còn cần mã hoá vị trí 2D của patch (embedding vị trí học được trong ViT; 2D-RoPE và M-RoPE trong Qwen2-VL, theo ghi chú llm 37).

Số token ảnh không đồng nghĩa với lượng thông tin. Pixel shuffle và nút thắt học được cho thấy có thể nén token mạnh mà vẫn giữ nhiều chất lượng, nhưng nén quá tay làm mất chi tiết (chữ nhỏ). Ở mức chương này chỉ nêu đánh đổi, không đưa số điểm benchmark.

Chi phí nằm ở cả hai phía. Bộ mã hoá thị giác cũng tốn tính toán, và mô hình nhỏ vẫn có thể ngốn bộ nhớ nếu giữ encoder lớn và nhiều token ảnh: theo ghi chú llm 38, các tác giả SmolVLM đo Qwen2-VL-2B cần 13,7 GB VRAM, InternVL2-2B cần 10,5 GB và SmolVLM-2.2B cần 4,9 GB, và nhấn mạnh số tham số không đủ để dự báo chi phí.

8. Nối sang các bài paper của cuốn

llm 29 (ViT). Bài chính cho mục 3: cắt patch, embedding vị trí, token `[class]`, quy mô dữ liệu tiền huấn luyện. Ví dụ 196 token ở đây trùng ví dụ của ghi chú.

llm 30 và 31 (CLIP, SigLIP). Mục 4: loss tương phản đối xứng và biến thể sigmoid. Hãy so ví dụ 3 cặp ở đây với ma trận $N \times N$ trong ghi chú.

llm 32, 33, 34, 35 (Flamingo, BLIP-2, LLaVA, LLaVA-1.5). Mục 5: ba cách nối bộ mã hoá ảnh với LLM, từ cross-attention đến nút thắt học được đến projector đơn giản. LLaVA-1.5 là bài chứng minh MLP projector cộng ViT độ phân giải cao đủ mạnh.

llm 36, 37, 38 (InternVL 1.5, Qwen2-VL, SmolVLM). Mục 6: các cách xử lý độ phân giải và nén token.

llm 57 (FastV). Mục 6: cắt token ảnh khi suy luận. Chương này cho thấy vì sao token ảnh đáng cắt: chúng chiếm phần lớn độ dài chuỗi (ví dụ mục 6.1).

Liên hệ nen08. Chi phí bộ nhớ của ma trận attention và ý tưởng tính theo khối đều áp dụng cho token ảnh; các token ảnh làm cho prefill dài ra theo cùng cách như prompt chữ dài.

9. Sai lầm hay gặp

- **Nghĩ rằng LLM "nhìn" ảnh trực tiếp.** LLM chỉ thấy các vector sau projector; mọi thông tin phải đi qua bộ mã hoá ảnh và cầu nối.
- **Quên rằng số token tăng theo bình phương cạnh.** Tăng đôi cạnh ảnh nhân bốn số token, và chi phí attention nhân khoảng mười sáu.
- **Nhầm patch 16×16 với 16 patch.** 16×16 là kích thước mỗi patch tính bằng điểm ảnh; số patch là $(H/P)(W/P)$.
- **Nhầm hướng loss tương phản.** Cross-entropy theo hàng chọn chữ cho mỗi ảnh, theo cột chọn ảnh cho mỗi chữ; CLIP dùng cả hai và lấy trung bình.
- **Đọc cosine như xác suất.** Cosine nằm trong $[-1, 1]$ và chỉ thành xác suất sau khi chia nhiệt độ rồi softmax.
- **Coi projector là nơi "hiểu" ảnh.** Với khoảng 21 triệu tham số (ví dụ mục 5.3), projector chủ yếu căn chỉnh không gian; phần hiểu nằm ở bộ mã hoá ảnh và LLM.

10. Tóm tắt công thức

Đại lượng	Công thức	Ghi chú
Số patch	$N = (H/P)(W/P)$	224, $P = 16$ cho 196
Số giá trị mỗi patch	P^2C	768 khi $P = 16$, $C = 3$
Patch embedding	$E \in \mathbb{R}^{(P^2C) \times D}$	một ma trận dùng chung mọi patch
Logit CLIP	S_{ij}/τ, $S_{ij} = u_i^\top v_j$	u, v chuẩn hoá L2
Loss CLIP	trung bình của CE theo hàng và theo cột	nhãn là đường chéo
Projector	$H_v = f(Z_v)$, f tuyến tính hoặc MLP	cùng chiều D với LLM
Chuỗi vào LLM	$[H_v;\ H_{\text{chữ}}]$	self-attention chung
Chi phí attention	tỉ lệ n^2, n = token ảnh + token chữ	ảnh thường chiếm đa số n

11. Bài tập tự tính

Bài 1. Ảnh 448×448, patch 14×14. Tính số token, rồi số token sau khi pixel shuffle với $r = 2$.

Đáp án (đã kiểm numpy): $448/14 = 32$, $N = 32 \cdot 32 = 1024$ token; sau pixel shuffle $r = 2$: $1024/4 = 256$ token (khớp 256 token mỗi tile 448×448 trong ghi chú llm 36).

Bài 2. Patch 16×16 trên ảnh $224 \times 224 \times 3$, $D = 768$. Tính số tham số của patch embedding có bias, và số token nếu patch là 32×32.

Đáp án (đã kiểm numpy): $16 \cdot 16 \cdot 3 = 768$ giá trị mỗi patch; $768 \cdot 768 = 589.824$ trọng số, cộng 768 bias là 590.592 tham số. Patch 32×32: $(224/32)^2 = 49$ token.

Bài 3. Một VLM nhận ảnh cho 576 token ảnh và prompt chữ 100 token. Tính tổng độ dài chuỗi, tỉ lệ token ảnh, và tỉ số số cặp điểm số attention so với khi chỉ có chữ.

Đáp án (đã kiểm numpy): tổng 676; tỉ lệ ảnh $576/676 \approx 85{,}2\%$; số cặp $676^2 = 456.976$ so với $100^2 = 10.000$, gấp $\approx 45{,}7$ lần.

Bài 4. Hai cặp ảnh-chữ có cosine $S = \begin{pmatrix} 0{,}9 & 0{,}1 \\ 0{,}2 & 0{,}8 \end{pmatrix}$ và $\tau = 1$. Tính loss theo hàng và theo cột, rồi loss tổng. So với mức đoán đều $\ln 2$.

Đáp án (đã kiểm numpy): hàng 1: $-\ln \frac{e^{0,9}}{e^{0,9}+e^{0,1}} = \ln(1 + e^{-0,8}) \approx 0{,}3711$; hàng 2: $\ln(1 + e^{-0,6}) \approx 0{,}4375$; trung bình hàng $\approx 0{,}4043$. Cột 1: $\ln(1 + e^{0,2-0,9}) \approx 0{,}4032$; cột 2: $\ln(1 + e^{0,1-0,8}) \approx 0{,}4032$; trung bình cột $\approx 0{,}4032$. Loss tổng $\approx (0{,}4043 + 0{,}4032)/2 \approx 0{,}4037$. Mức đoán đều $\ln 2 \approx 0{,}6931$, nên mô hình khá hơn đoán.

Bài 5. Projector là MLP hai lớp $D_v = 768 \to D = 2048 \to D = 2048$ với bias. Tính số tham số. Nếu nhân đôi cạnh ảnh thì số token ảnh và số tham số projector thay đổi thế nào?

Đáp án (đã kiểm numpy): $768 \cdot 2048 + 2048 + 2048 \cdot 2048 + 2048 = 1.572.864 + 2.048 + 4.194.304 + 2.048 = 5.771.264$ tham số. Nhân đôi cạnh ảnh: số token ảnh nhân 4; số tham số projector không đổi vì projector chiếu từng vector độc lập và không phụ thuộc số token. Chi phí thay đổi nằm ở LLM (chuỗi dài hơn), không phải ở projector.

Tài liệu tham khảo nền: Dosovitskiy et al. (ViT), Radford et al. (CLIP), Zhai et al. (SigLIP), Liu et al. (LLaVA, LLaVA-1.5), và các ghi chú llm 29 đến 38 và 57 trong cuốn này.

Bài 00

Preface: Toán nền tảng và hướng dẫn đọc (LLM, VLM, suy luận)

Tác giả: Ghi chú tổng hợp từ kiến thức chuẩn của lĩnh vực (không dựa vào một paper cụ thể).
Venue / arXiv: không áp dụng.
Nguồn: các paper trong thư mục này (xem lịch đọc ở mục 14); công thức tham chiếu chính từ 1706.03762 (bài 01) và 2001.08361 (bài 03).
Chủ đề: token hoá, embedding, next-token prediction, attention và các biến thể phức tạp, KV cache, FLOPs/bộ nhớ, GPU roofline, độ chính xác số, attention tuyến tính/SSM, VLM.
Vị trí trong lộ trình: đọc TRƯỚC mọi bài khác. Bài này giới thiệu kiến thức nền để đọc 73 bài nghiên cứu tiếp theo. Sau đó đi theo thứ tự 01 → 04, rồi theo lịch ở mục 14.

> Ghi chú nhập môn bằng tiếng Việt, dùng các ví dụ tính toán có giả định để giải thích khái niệm và chi phí tính toán.

1. Token và tokenization (BPE)

Token là đơn vị rời rạc mà mô hình thực sự đọc và sinh. Văn bản (chuỗi byte/ký tự) được **tokenizer** cắt thành chuỗi số nguyên $x_1, \ldots, x_n \in \{0, \ldots, V-1\}$, với V là kích thước từ vựng (vocabulary), thường 32k đến 250k.

Vì sao không dùng ký tự hay từ nguyên?

- Ký tự: chuỗi quá dài (n lớn, mà attention tốn $O(n^2)$).
- Từ nguyên: từ vựng quá lớn, từ lạ/lỗi chính tả thành "unknown".
- **Subword** (mảnh từ) là điểm giữa: từ phổ biến là 1 token, từ hiếm tách thành vài mảnh.

1.1 Thuật toán BPE (Byte-Pair Encoding)

Ý tưởng: bắt đầu từ các ký tự (hoặc byte), lặp lại việc **gộp cặp liền kề xuất hiện nhiều nhất** thành một ký hiệu mới, cho đến khi đủ V ký hiệu.

```
Huấn luyện BPE(corpus, V_target):
  vocab <- tất cả byte/ký tự đơn
  tokens <- corpus tách thành các ký hiệu trong vocab
  while |vocab| < V_target:
      (a, b) <- cặp liền kề xuất hiện nhiều nhất trong tokens
      ghi lại luật gộp (a, b) -> ab
      vocab <- vocab U {ab}
      thay mọi cặp (a, b) liền kề trong tokens bằng ab
Mã hoá(text): áp dụng các luật gộp theo đúng thứ tự đã học.
```

Ví dụ nhỏ: corpus "low low low lower lowest". Ban đầu: `l o w`. Cặp (`l`,`o`) xuất hiện nhiều nhất, gộp thành `lo`; sau đó (`lo`,`w`) thành `low`. Từ "lower" thành `low` + `e` + `r`. Hai luật gộp đầu đã biến "low" thành 1 token.

Byte-level BPE (GPT-2 trở đi): ký hiệu gốc là 256 byte nên không bao giờ có "unknown". Bài 01 dùng BPE với từ vựng chung ~37.000 (EN-DE), bài 03 dùng $V = 50257$.

1.2 Hệ quả cho OCR

Số token cần sinh quyết định số bước decode tuần tự; độ trễ xấp xỉ tổng thời gian của các bước đó. Chữ số, ký hiệu toán, bảng markdown bị cắt thành nhiều token nhỏ. Từ vựng lớn làm ma trận embedding/LM head lớn ($V \cdot d$, mục 9).

---

2. Embedding

Embedding là một bảng tra $E \in \mathbb{R}^{V\times d}$: token x_t được thay bằng hàng $E[x_t] \in \mathbb{R}^d$. Về mặt toán, nó bằng $\text{onehot}(x_t)^\top E$ nhưng cài đặt là tra bảng (không nhân ma trận thật).

$$h_t^{(0)} = E[x_t] \quad (\text{+ thông tin vị trí nếu dùng cộng, xem mục 7}), \qquad h^{(0)} \in \mathbb{R}^{n\times d}.$$

- $d = d_{model}$ là chiều của **residual stream** (luồng dữ liệu chung chạy xuyên các tầng).
- Cuối mạng, **LM head** $W_{out} \in \mathbb{R}^{d\times V}$ biến vector cuối thành **logits** $z \in \mathbb{R}^V$. **Weight tying** dùng chung $W_{out} = E^\top$ để tiết kiệm $V \cdot d$ tham số (bài 01 cũng làm vậy: "share the same weight matrix between the two embedding layers and the pre-softmax linear transformation").
- Với ảnh, "embedding" là phép chiếu tuyến tính các patch (mục 13).

---

3. Next-token prediction, cross-entropy, perplexity

Mô hình ngôn ngữ tự hồi quy (autoregressive LM) phân rã xác suất của chuỗi bằng quy tắc chuỗi:

$$p_\theta(x_1, \ldots, x_n) = \prod_{t=1}^{n} p_\theta(x_t \mid x_{<t}).$$

Tại vị trí t mạng xuất logits $z_t \in \mathbb{R}^V$, rồi **softmax**:

$$p_\theta(v \mid x_{<t}) = \frac{\exp(z_{t,v})}{\sum_u \exp(z_{t,u})}.$$

Cross-entropy (CE) trung bình trên token (đơn vị nats nếu dùng ln):

$$\boxed{\mathcal{L}(\theta) = -\frac{1}{n}\sum_{t=1}^{n} \log p_\theta(x_t \mid x_{<t}).}$$

Perplexity: $\text{PPL} = \exp(\mathcal{L})$. Hiểu trực quan: mô hình "bối rối như thể phải chọn đều giữa PPL token". Mô hình ngẫu nhiên đều trên $V = 50257$ có $\mathcal{L} = \ln 50257 \approx 10.82$ nats, $\text{PPL} = V$.

Ví dụ: CE = 2.64 nats (một con số hợp lý cho LM cỡ 1B) tương ứng $\text{PPL} = e^{2.64} \approx 14$ (tự tính).

Teacher forcing: khi huấn luyện, đầu vào là chuỗi thật dịch một vị trí, nên loss mọi vị trí tính song song trong một lần forward (nhờ causal mask, mục 5); khi suy luận phải sinh tuần tự (mục 8).

Lưu ý: PPL và CE phụ thuộc tokenizer. Bài 01 ghi rõ "perplexities are per-wordpiece... should not be compared to per-word perplexities". Bài 03 và 04 dùng CE (nats/token) hoặc bits-per-byte để so sánh.

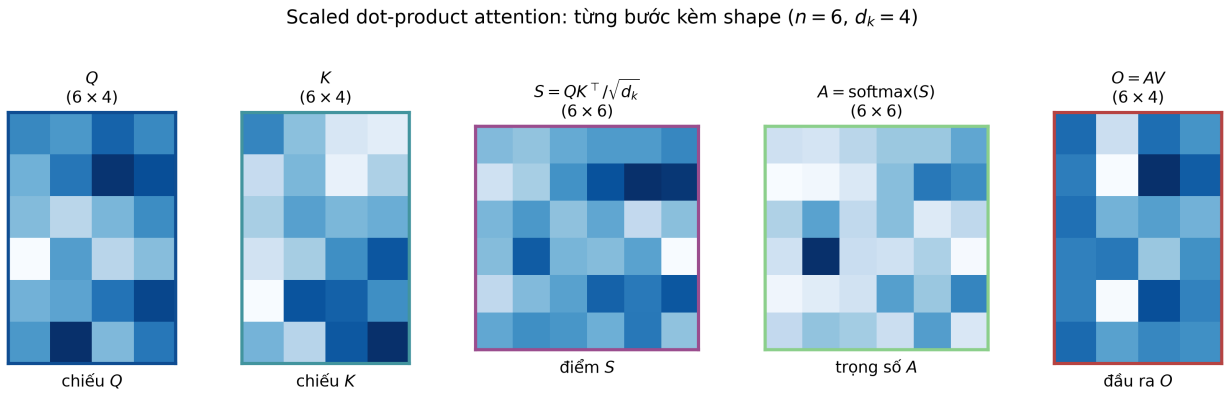
---

4. Scaled dot-product attention và multi-head, kèm shape

Ký hiệu: batch B (đặt $B = 1$ cho gọn), độ dài chuỗi n, $d = d_{model}$, số head h, $d_k = d_v = d/h$.

4.1 Một head

Đầu vào $X \in \mathbb{R}^{n \times d}$. Ba phép chiếu tuyến tính học được:



Luồng scaled dot-product attention với shape từng bước: $Q, K \in \mathbb{R}^{n \times d_k}$, $S = QK^\top/\sqrt{d_k}$, $A = \text{softmax}(S)$, $O = AV$ ($n = 6$, $d_k = 4$). (minh hoạ, tự tính)

$$Q = XW_Q, \quad K = XW_K, \quad V = XW_V, \qquad W_Q, W_K \in \mathbb{R}^{d \times d_k},\ W_V \in \mathbb{R}^{d \times d_v}.$$

Shape: $Q, K \in \mathbb{R}^{n \times d_k}$, $V \in \mathbb{R}^{n \times d_v}$.

$$\boxed{\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V}$$

Từng bước với shape:

Bước	Biểu thức	Shape
Điểm	$S = QK^\top/\sqrt{d_k}$	$n \times n$
Mask (mục 5)	$S \leftarrow S + M$	$n \times n$
Trọng số	$A = \text{softmax}_{\text{theo hàng}}(S)$	$n \times n$, mỗi hàng tổng = 1
Đầu ra	$O = AV$	$n \times d_v$

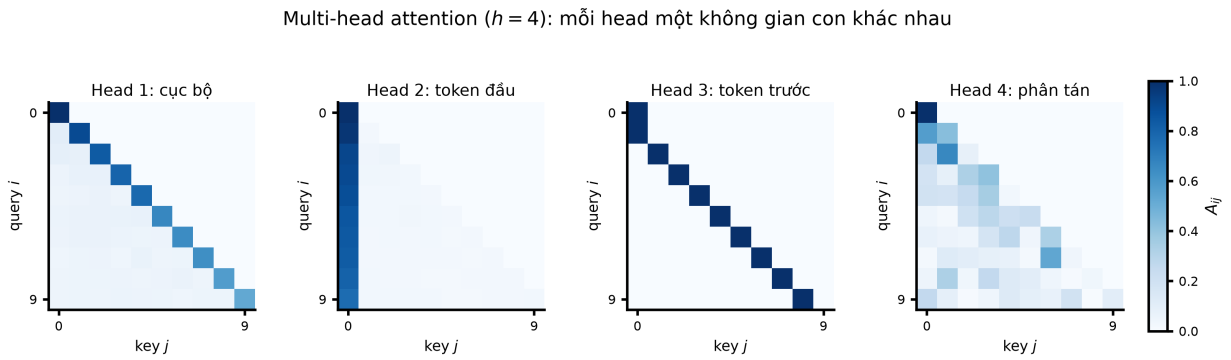
Diễn giải: query của vị trí i so khớp với key của mọi vị trí j; trọng số A_{ij} quyết định lấy bao nhiêu value v_j. $O_i = \sum_j A_{ij} v_j$ là **trung bình có trọng số** của value.

4.2 Vì sao chia $\sqrt{d_k}$

Nếu các thành phần của q, k độc lập, kỳ vọng 0, phương sai 1 thì $q \cdot k = \sum_{i=1}^{d_k} q_i k_i$ có phương sai d_k (bài 01, chú thích 4). Điểm lớn đẩy softmax vào vùng bão hoà, gradient rất nhỏ. Chia $\sqrt{d_k}$ đưa phương sai về 1.

4.3 Multi-head attention (MHA)

Chạy h head song song với phép chiếu riêng, ghép lại rồi chiếu ra:



Bốn head attention giả lập (cục bộ, nhìn token đầu, token trước, phân tán) dưới causal mask, minh họa nhiều không gian con song song. (minh hoạ, tự tính)

$$\mathrm{MHA}(X) = \mathrm{Concat}(\mathrm{head}_1, \dots, \mathrm{head}_h)\, W_O, \quad \mathrm{head}_i = \mathrm{Attention}(XW_Q^i, XW_K^i, XW_V^i),$$

$W_O \in \mathbb{R}^{hd_v \times d}$. Trong thực tế gộp h head thành một ma trận lớn $W_Q \in \mathbb{R}^{d \times d}$ rồi reshape:

```
# X: (B, n, d); h heads; dk = d // h
Q = (X @ Wq).view(B, n, h, dk).transpose(1, 2)   # (B, h, n, dk)
K = (X @ Wk).view(B, n, h, dk).transpose(1, 2)   # (B, h, n, dk)
V = (X @ Wv).view(B, n, h, dk).transpose(1, 2)   # (B, h, n, dk)
S = Q @ K.transpose(-1, -2) / dk**0.5            # (B, h, n, n)
S = S.masked_fill(causal_mask, float('-inf'))
A = S.softmax(dim=-1)                            # (B, h, n, n)
O = (A @ V).transpose(1, 2).reshape(B, n, d)     # (B, n, d)
Y = O @ Wo                                       # (B, n, d)
```

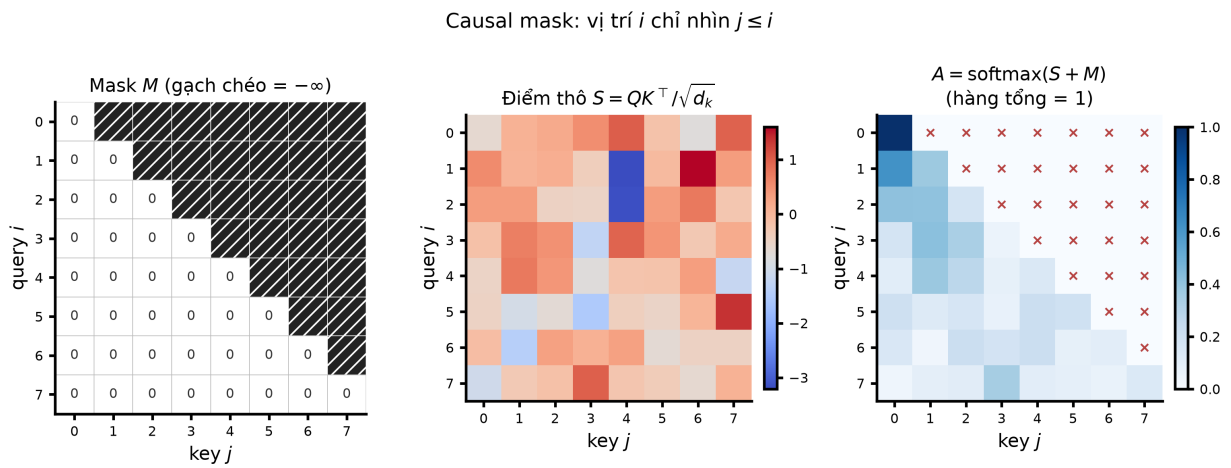
Chi phí tham số của MHA: $4d^2$ (W_Q, W_K, W_V, W_O mỗi cái d^2). Bài 01 cho thấy chia d thành h head nhỏ thì tổng tính toán gần như bằng một head chiều đầy đủ, nhưng cho phép chú ý nhiều "không gian con" khác nhau.

---

5. Causal mask và chi phí bậc hai của attention

5.1 Causal mask

Với mô hình sinh, vị trí i chỉ được nhìn $j \le i$. Cộng ma trận mask M trước softmax:



Causal mask $M_{ij} = -\infty$ khi $j > i$: điểm thô S và trọng số $A = \text{softmax}(S + M)$ chỉ còn nửa dưới tam giác (hàng tổng bằng 1). (minh hoạ, tự tính)

$$M_{ij} = \begin{cases} 0 & j \le i \\ -\infty & j > i \end{cases} \qquad \Rightarrow \qquad A_{ij} = 0 \text{ khi } j > i.$$

Nhờ đó một lần forward trên cả chuỗi tính được loss cho mọi vị trí song song mà không "nhìn trộm" tương lai. Bài 01 cài mask bằng cách đặt $-\infty$ ở đầu vào softmax. Encoder (và ViT) dùng attention **hai chiều** (không mask).

5.2 Độ phức tạp

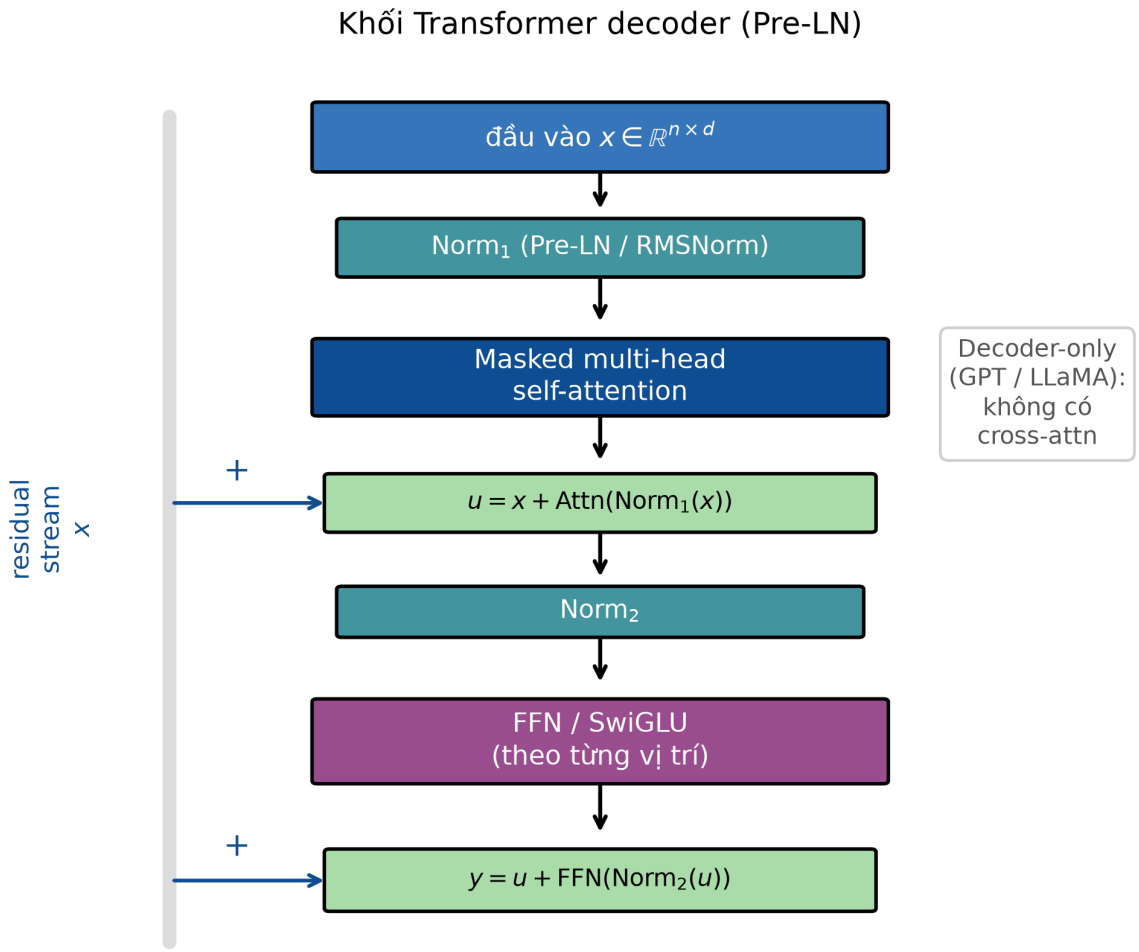
Ma trận điểm S có n^2 phần tử, mỗi phần tử là tích vô hướng d_k chiều.

- **Thời gian**: $QK^\top$ tốn $2n^2 d_k$ FLOPs mỗi head, AV tốn thêm $2n^2 d_v$. Tổng một tầng (mọi head): $\approx 4n^2 d$ FLOPs, tức $O(n^2 d)$ (khớp bảng 1 bài 01: self-attention $O(n^2 \cdot d)$).
- **Bộ nhớ**: nếu lưu S/A thì $O(h\, n^2)$ phần tử mỗi tầng. Với $n = 2048$, $h = 16$, FP16: $16 \cdot 2048^2 \cdot 2 \approx 134$ MB mỗi tầng (tự tính). Đây là lý do FlashAttention (bài 44) tính theo khối trong SRAM và không bao giờ ghi S ra HBM, đưa bộ nhớ phụ về $O(n)$.
- So với phần còn lại: phần chiếu tuyến tính + FFN tốn $\approx 24nd^2$ FLOPs mỗi tầng (nhân n tuyến tính). Attention $4n^2 d$ chiếm ưu thế khi $4n^2 d > 24nd^2$, tức $n > 6d$. Với $d = 1024$ thì $n \gtrsim 6000$. Với chuỗi ngắn hơn ngưỡng này, attention chưa chiếm ưu thế về FLOPs; KV cache và băng thông vẫn cần được tính đến (mục 8).

---

6. Khối residual: normalization, FFN, vị trí

Một **Transformer block** (kiểu decoder hiện đại, "Pre-LN"):



Sơ đồ khối decoder Pre-LN: Norm → masked MHA → residual → Norm → FFN/SwiGLU → residual trên residual stream (không cross-attn). (minh hoạ, tự tính)

$$u = x + \text{Attn}(\text{Norm}_1(x)),$$
$$y = u + \text{FFN}(\text{Norm}_2(u)).$$

Mọi sub-layer **cộng vào** residual stream x thay vì thay thế nó. Hệ quả: $x^{(L)} = x^{(0)} + \sum_l \Delta x^{(l)}$; gradient có đường tắt, và mỗi tầng chỉ "đọc rồi ghi thêm" một cập nhật (cách nhìn này trùng với notes về Transformer Circuits).

6.1 Normalization

- **LayerNorm**: $\text{LN}(x) = \gamma \odot \frac{x-\mu}{\sqrt{\sigma^2+\epsilon}} + \beta$, với μ, σ tính trên chiều d của từng token. Bài 01 dùng **Post-LN**: $\text{LN}(x + \text{Sublayer}(x))$.
- **Pre-LN** (norm trước sub-layer, bài 09): huấn luyện ổn định hơn, ít cần warmup. Đa số LLM hiện nay dùng Pre-LN.
- **RMSNorm** (bài 10): bỏ phép trừ trung bình, $\text{RMS}(x) = \gamma \odot x / \sqrt{\frac{1}{d}\sum x_i^2 + \epsilon}$. Rẻ hơn, hiệu quả tương đương.

6.2 FFN (feed-forward network, còn gọi MLP)

Áp dụng độc lập cho từng token. Bản gốc (bài 01): $\text{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2$ với $d_{ff} = 4d$. Hiện đại hoá: **SwiGLU** (bài 11)

$$\mathrm{FFN}(x) = \big(\mathrm{SiLU}(xW_g) \odot xW_u\big)W_d, \quad W_g, W_u \in \mathbb{R}^{d\times d_{ff}},\ W_d \in \mathbb{R}^{d_{ff}\times d}.$$

Ba ma trận nên thường chọn $d_{ff} \approx \frac{8}{3}d$ để giữ ~$8d^2$ tham số như FFN hai ma trận với $d_{ff} = 4d$.

FFN chiếm khoảng 2/3 số tham số của một tầng (xem mục 9) và thường được xem là nơi lưu "kiến thức". Khi thay đổi chiều ẩn của SwiGLU, các ma trận của hai nhánh đầu vào và phép chiếu đầu ra phải được điều chỉnh nhất quán.

---

7. Thông tin vị trí

Attention là hàm **hoán vị-bất biến** (đổi thứ tự token chỉ đổi thứ tự hàng đầu ra), nên phải đưa vị trí vào.

Cách	Ý tưởng	Bài
Sinusoidal cộng vào embedding	$PE_{(pos,2i)} = \sin(pos/10000^{2i/d})$, $PE_{(pos,2i+1)} = \cos(\cdot)$	01
Learned absolute	bảng $P \in \mathbb{R}^{n_{max}\times d}$	01 (kết quả gần như như nhau), GPT
RoPE	xoay q, k theo góc $\propto$ vị trí; tích $q_i^\top k_j$ chỉ phụ thuộc $i-j$	12
ALiBi	cộng $-m\cdot(i-j)$ vào điểm attention	13
Ngầm (recurrence)	trạng thái RNN/SSM tự mang thứ tự	18 đến 24

Công thức RoPE cho một cặp chiều $(2m, 2m+1)$: xoay bởi góc $i\theta_m$ với $\theta_m = 10000^{-2m/d_k}$; vì phép xoay trực giao, $\langle R_i q, R_j k\rangle = \langle q, R_{j-i}k\rangle$, tức chỉ phụ thuộc khoảng cách tương đối.

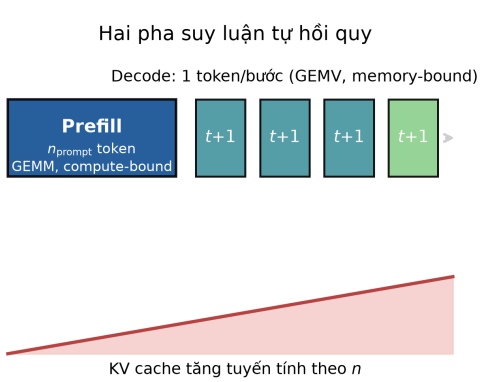
Ảnh 2D cần vị trí 2D; Qwen2-VL (bài 37) mở rộng RoPE sang nhiều chiều.

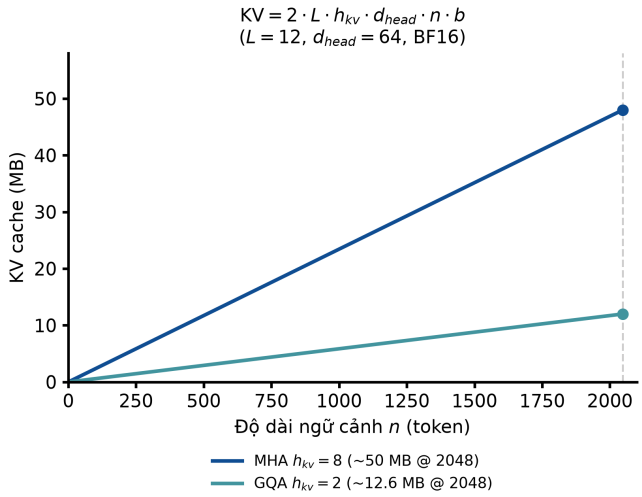
---

8. Sinh tự hồi quy: prefill và decode, KV cache

8.1 Hai pha của suy luận

Sinh văn bản là vòng lặp: đưa prompt vào, chọn token tiếp theo (argmax hoặc lấy mẫu), nối vào chuỗi, lặp lại.





Prefill (GEMM) rồi decode từng token (GEMV); kích thước KV cache $2 \cdot L \cdot h_{kv} \cdot d_{head} \cdot n \cdot b$ tăng tuyến tính, so MHA $h_{kv} = 8$ với GQA $h_{kv} = 2$. (minh hoạ, tự tính)

	Prefill	**Decode**
Làm gì	xử lý toàn bộ prompt trong một lần	sinh 1 token mỗi bước
Số token xử lý mỗi lượt	n_{prompt} (hàng trăm đến nghìn)	1
Phép toán chính	GEMM lớn (ma trận x ma trận)	GEMV (ma trận x vector)
Giới hạn bởi	**tính toán** (compute-bound)	**băng thông bộ nhớ** (memory-bound), xem mục 10
Chỉ số	TTFT (time to first token)	TPOT/ITL (time per output token)

Với OCR, prefill xử lý ảnh và token đầu vào một lần; decode cần một bước nối tiếp cho mỗi token đầu ra. Tỉ lệ thời gian giữa hai pha phụ thuộc độ dài và cách triển khai.

8.2 KV cache

Ở bước decode t, query mới q_t cần $K_{1..t}$ và $V_{1..t}$. Các k_s, v_s của $s < t$ **không đổi** (nhờ causal mask), nên ta lưu lại thay vì tính lại. Mỗi bước chỉ tính q_t, k_t, v_t cho token mới, ghi k_t, v_t vào cache, rồi attention với cache.

```
decode_step(token_t, cache):
    x = embed(token_t)
    for layer l:
        q, k, v = proj(norm(x))           # chỉ 1 token
        cache[l].K.append(k); cache[l].V.append(v)
        x = x + Wo @ softmax(q @ cache[l].K^T / sqrt(dk)) @ cache[l].V
        x = x + FFN(norm(x))
    return logits(x)
```

Công thức kích thước KV cache (một chuỗi, batch 1):

$$\boxed{\text{KV bytes} = 2 \cdot L \cdot h_{kv} \cdot d_{head} \cdot n \cdot b}$$

với số 2 là K và V, L số tầng **có full attention**, h_{kv} số KV head, d_{head} chiều mỗi head, n độ dài (tổng token đã ở trong ngữ cảnh), b byte mỗi phần tử.

Ví dụ (tự tính): Với 12 tầng attention, 8 KV head, mỗi head 64 chiều, 2.048 token và BF16, cache chiếm $2 \cdot 12 \cdot 8 \cdot 64 \cdot 2048 \cdot 2 = 50{,}331{,}648$ byte, khoảng 50 MB. Với cùng số tầng và chiều head, GQA dùng 2 KV head giảm còn khoảng 12,6 MB.

Cache tăng tuyến tính theo độ dài ngữ cảnh và batch. Khi decode, dữ liệu cache còn phải được đọc để tính attention; với VLM, token ảnh cũng đóng góp vào độ dài ngữ cảnh.

---

9. Số tham số và FLOPs mỗi token

9.1 Đếm tham số (decoder chuẩn, MHA, FFN $d_{ff} = 4d$)

Mỗi tầng: attention $4d^2$ + FFN $2 \cdot d \cdot 4d = 8d^2$ (bỏ bias, norm):

$$\boxed{N_{\text{non-emb}} \approx 12\, L\, d^2}$$

(đúng như Kaplan phương trình 2.1, bài 03). Thêm embedding $V \cdot d$ (và LM head $V \cdot d$ nếu không tie). Ví dụ GPT-2 small-ish: $L = 12, d = 768$: $12 \cdot 12 \cdot 768^2 \approx 85$M.

Với GQA: W_K, W_V nhỏ đi nên attention còn $\approx 2d^2 + 2d \cdot h_{kv} d_{head}$. Với SwiGLU: 3 ma trận $d \times d_{ff}$.

Cảnh báo cho mô hình nhỏ: với $d = 1024$, $V \approx 150$k thì $V \cdot d \approx 154$M, có thể chiếm phần đáng kể tổng số tham số của một mô hình nhỏ. Khi cắt, hãy luôn tách riêng "embedding" khỏi "thân Transformer" (Kaplan cũng loại embedding khỏi N, bài 03).

9.2 FLOPs mỗi token

Một phép nhân ma trận–vector với ma trận $m \times k$ tốn $2mk$ FLOPs (một nhân và một cộng cho mỗi trọng số). Vì mỗi trọng số tham gia một lần:

$$\boxed{\text{FLOPs}_{\text{forward}}/\text{token} \approx 2N} \qquad \text{(huấn luyện: } \approx 6N \text{ vì backward} \approx 2 \times \text{forward).}$$

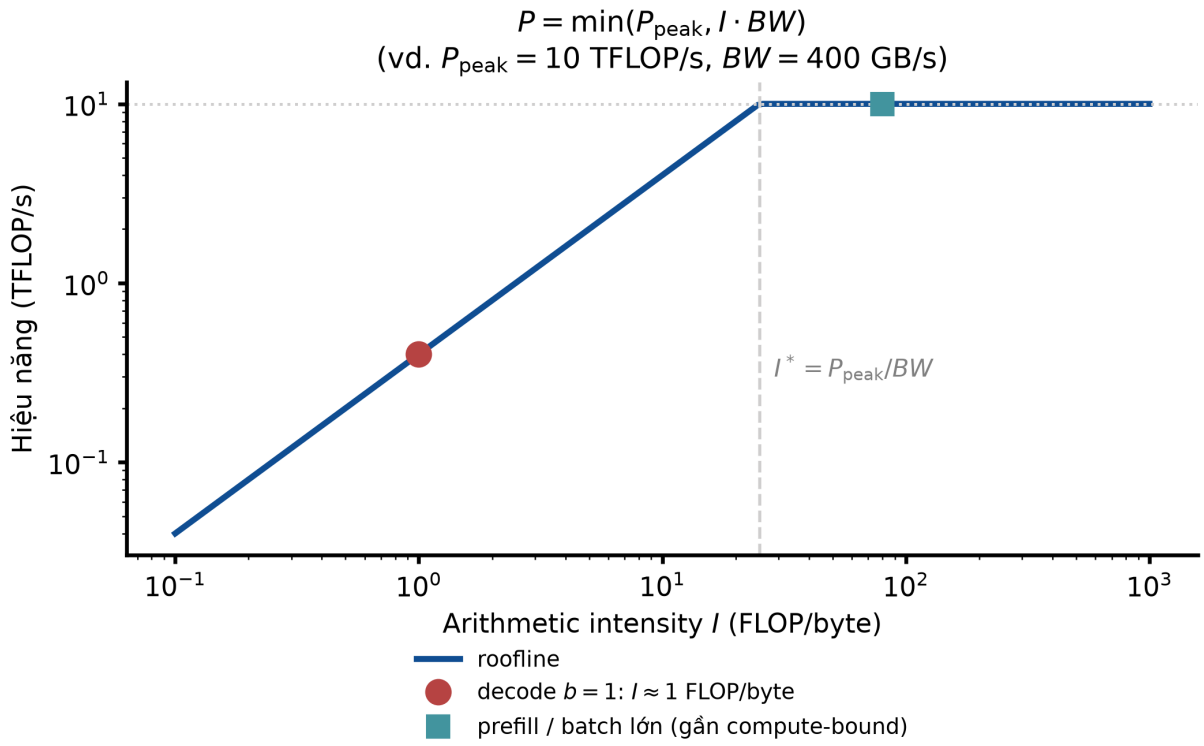
Cộng thêm phần phụ thuộc ngữ cảnh $\approx 2L\, n\, d$ (Kaplan 2.2, bài 03: $C_{forward} \approx 2N + 2n_{layer} n_{ctx} d_{attn}$, với n_{ctx} là độ dài ngữ cảnh).

Ví dụ (tự tính): với $N = 10^9$, forward cần xấp xỉ 2 GFLOPs/token; 2.000 token tương ứng khoảng 4 TFLOPs theo ước lượng đơn giản này. Prefill có thể xử lý nhiều token song song, còn decode tự hồi quy gồm các bước nối tiếp. Chi phí huấn luyện trên D token xấp xỉ $6ND$ (bài 03, 04).

Cắt tham số thì FLOPs/token giảm tỷ lệ, nhưng **latency batch-1 phụ thuộc byte đọc chứ không phải FLOPs** (mục 10).

---

10. GPU: HBM, SRAM, băng thông, roofline



Roofline $P = \min(P_{\text{peak}}, I \cdot BW)$: decode b=1 có $I \approx 1$ FLOP/byte (memory-bound). (minh hoạ, tự tính)

10.1 Phân cấp bộ nhớ

Tầng	Dung lượng (cỡ)	Băng thông (cỡ)	Ghi chú
Thanh ghi / **SRAM** (shared mem, L1/L2) trên chip	vài chục KB mỗi SM; tổng ~chục MB	rất cao (hàng chục TB/s)	nhanh, nhỏ
HBM/VRAM (bộ nhớ chính của GPU)	8 GB đến 80 GB	~200 GB/s (card nhỏ) đến ~3 TB/s (data center)	trọng số, KV cache, kích hoạt nằm đây
RAM máy chủ qua PCIe	lớn	~vài chục GB/s	chậm

(Các con số là bậc độ lớn, tuỳ GPU; hãy tra thông số card đang dùng.)

Phần lớn tối ưu suy luận (FlashAttention, quantization, KV cache nén, speculative decoding) đều là **giảm số byte di chuyển giữa HBM và chip**, không phải giảm FLOPs.

10.2 Arithmetic intensity và roofline

Arithmetic intensity I = FLOPs thực hiện / byte truyền từ bộ nhớ (FLOP/byte). **Roofline model**: hiệu năng đạt được

$$\boxed{P = \min\left(P_{peak},\ I \cdot BW\right)}$$

với P_{peak} đỉnh tính toán (FLOP/s), BW băng thông bộ nhớ (byte/s). Điểm gãy (ridge) $I^* = P_{peak}/BW$: dưới I^* là memory-bound, trên là compute-bound.

10.3 Vì sao decode batch-1 bị memory-bound

Một bước decode với batch 1 là GEMV: mỗi trọng số (2 byte BF16) được đọc **một lần** để thực hiện đúng 2 FLOPs (1 nhân + 1 cộng).

$$I_{decode,b=1} \approx \frac{2N}{2N \text{ byte}} = 1 \text{ FLOP/byte (BF16)}, \quad \text{vs. } I^* \sim 30 \text{ đến } 300 \text{ FLOP/byte}.$$

Ví dụ (tự tính): Giả sử mô hình có 1 tỷ tham số BF16 và GPU đạt băng thông 400 GB/s. Đọc 2 GB trọng số mỗi token mất ít nhất $2/400 = 5$ ms, tương ứng cận trên lý tưởng 200 token/s. Tính toán 2 GFLOPs trên thiết bị đạt 10 TFLOP/s chỉ mất 0,2 ms. Đây là mô hình đơn giản: KV cache, kích hoạt, kernel và đồng bộ đều làm thời gian thực tế lớn hơn.

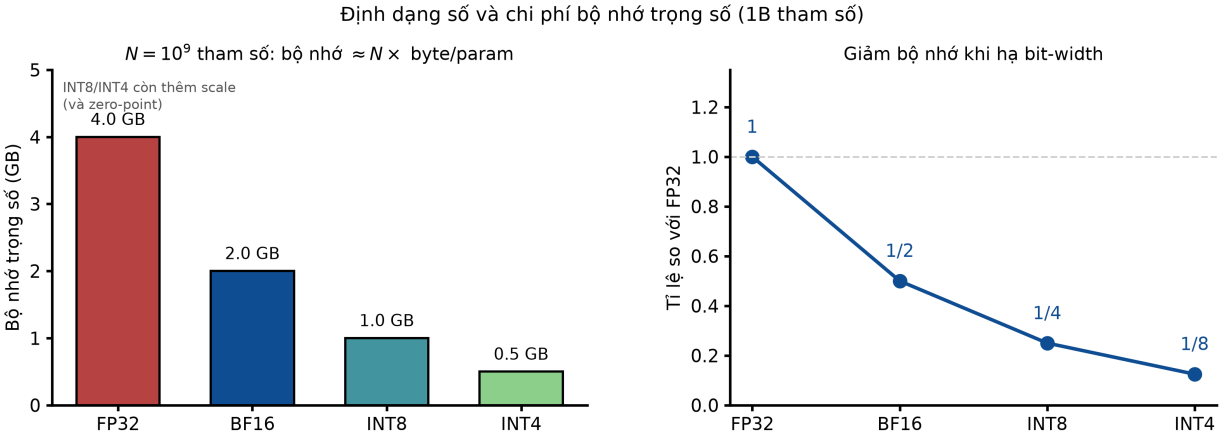
Tăng batch có thể tăng lượng tính toán trên mỗi lần đọc trọng số. Với độ trễ của một chuỗi, giảm lượng dữ liệu phải đọc hoặc số bước tuần tự là những hướng tối ưu cần đo thực nghiệm; speculative decoding được trình bày ở bài 48–50.

---

11. Định dạng số và chi phí bộ nhớ

Định dạng	Bit	Cấu trúc (dấu/mũ/định trị)	Ghi chú
FP32	32	1/8/23	chuẩn huấn luyện cũ; giữ “master weights” và optimizer state
BF16	16	1/8/7	cùng dải mũ với FP32, ít chính xác; ổn định, phổ biến cho train và infer

Định dạng	Bit	Cấu trúc (dấu/mũ/định trị)	Ghi chú
FP16	16	1/5/10	chính xác hơn BF16 nhưng dải hẹp, cần loss scaling
INT8	8	số nguyên	quantization sau huấn luyện (bài 51, 52)
INT4	4	số nguyên	cần quantization nhóm/hiệu chỉnh (bài 53, 54)



Bộ nhớ trọng số 1B tham số theo bit-width: FP32 4 GB, BF16 2 GB, INT8 1 GB, INT4 0.5 GB (tỉ lệ $1, 1/2, 1/4, 1/8$ so với FP32). (minh hoạ, tự tính)

Bộ nhớ trọng số xấp xỉ $N \times$ byte/param. Chẳng hạn, một tỷ tham số chiếm khoảng 4 GB ở FP32, 2 GB ở BF16, 1 GB ở INT8 và 0,5 GB ở INT4 (tự tính). Lượng tử hoá thực tế còn cần lưu scale và có thể cả zero-point. Khi kết hợp với cắt tỉa, cần đánh giá sai số sau từng bước.

---

12. Attention tuyến tính, RNN, SSM: trạng thái kích thước cố định

12.1 Vấn đề của softmax attention

Softmax attention cần giữ **toàn bộ** K, V (kích thước $\propto n$) và tốn $O(n)$ mỗi bước decode (so khớp q với n key), tổng $O(n^2)$.

12.2 Attention tuyến tính = RNN có trạng thái ma trận

Bỏ softmax, thay bằng kernel: $A_{ij} \propto \phi(q_i)^\top \phi(k_j)$. Khi đó đầu ra có thể viết dạng hồi quy (bài 18):

$$S_t = S_{t-1} + v_t k_t^\top, \qquad o_t = S_t q_t, \qquad S_t \in \mathbb{R}^{d_v \times d_k}.$$

S_t là **bộ nhớ liên kết (associative memory)** có kích thước **cố định** bất kể n: mỗi bước decode tốn $O(d_k d_v)$, không phụ thuộc n. Song song hoá lúc huấn luyện bằng thuật toán chunk (chia chuỗi thành khối).

12.3 Các cải tiến (đúng thứ tự thư mục)

- **Gating/decay** (RetNet 19, GLA 20, Mamba 21, Mamba-2 22): $S_t = \alpha_t S_{t-1} + v_t k_t^\top$, với $\alpha_t \in (0, 1)$ cho phép "quên".
- **Delta rule** (DeltaNet 23): sửa lỗi thay vì chỉ cộng dồn:

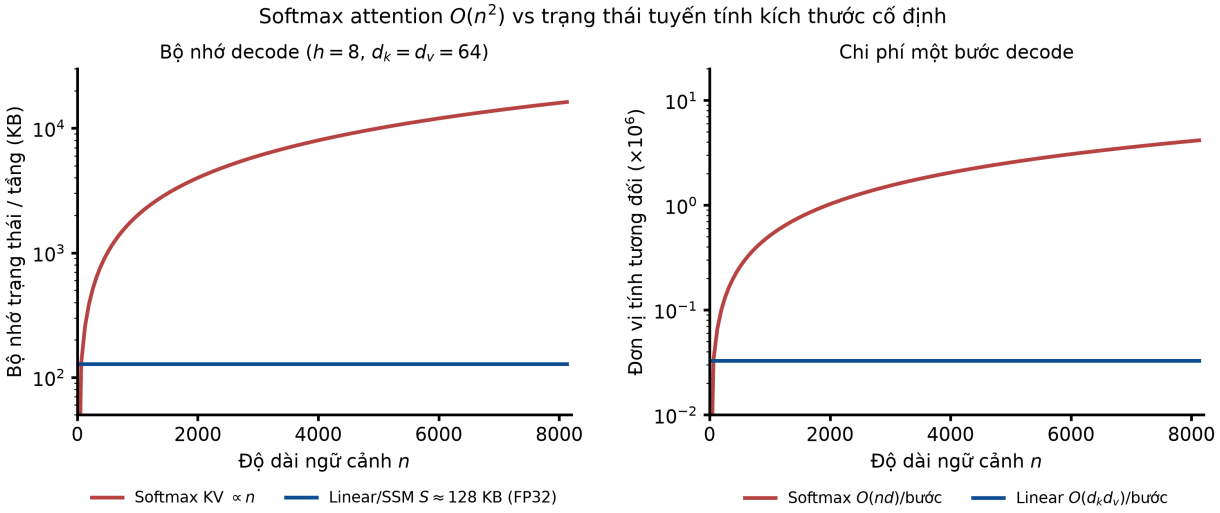
$$S_t = S_{t-1}\left(I - \beta_t k_t k_t^\top\right) + \beta_t v_t k_t^\top,$$

tức trước khi ghi cặp (k_t, v_t) thì xoá phần giá trị cũ gắn với k_t. **Gated DeltaNet** (24) kết hợp cả quên toàn cục α_t và delta rule.

- **SSM** (state-space model, Mamba): cùng họ hồi quy tuyến tính với trạng thái cố định, tham số hoá khác (bài 21, 22 chỉ ra quan hệ đối ngẫu với attention).

12.4 So sánh chi phí

	Softmax attention	Linear/SSM/DeltaNet
Trạng thái khi decode	KV cache, $\propto n$	S cố định ($d_k \times d_v$ mỗi head)
Thời gian một bước decode	$O(n\,d)$	$O(d_k d_v)$
Huấn luyện	$O(n^2 d)$	$O(n\,d^2)$ dạng chunk
Truy hồi chính xác (copy/lookup dài)	mạnh	yếu hơn (nén vào S hữu hạn)



So sánh bộ nhớ/chi phí decode: Softmax KV $\propto n$ và $O(nd)$/bước vs trạng thái tuyến tính/SSM cố định $S \approx 128$ KB/tầng ($h = 8$, $d_k = d_v = 64$, FP32). (minh hoạ, tự tính)

Các kiến trúc hybrid như Jamba (25) và Kimi Linear (26) xen kẽ tầng hồi quy với attention để cân bằng chi phí và khả năng truy hồi.

Ví dụ (tự tính): Với 8 head, $d_k = d_v = 64$ và trạng thái FP32, một tầng lưu $8 \cdot 64 \cdot 64 \cdot 4 = 131{,}072$ byte trạng thái. Kích thước này không tăng theo độ dài chuỗi; các tầng attention đầy đủ vẫn cần KV cache riêng.

---

13. Kiến thức nền về VLM

VLM (vision-language model) = bộ mã hoá ảnh + (bộ chiếu) + mô hình ngôn ngữ. Với OCR: ảnh trang → văn bản.

13.1 Từ ảnh thành token (ViT, bài 29)

1. Chia ảnh $H \times W$ thành các **patch** $P \times P$ (ví dụ $P = 14$ hoặc 16): có $\frac{H}{P} \cdot \frac{W}{P}$ patch.
2. Mỗi patch duỗi thành vector $P^2 \cdot 3$ chiều, nhân với ma trận chiếu → embedding d_v chiều (thực chất một conv stride P).
3. Cộng embedding vị trí (hoặc dùng RoPE 2D), đưa qua M khối Transformer **hai chiều** (full attention, không mask).
4. Ra một chuỗi **visual token** (đặc trưng thị giác).

13.2 Projector (cầu nối)

Visual token sống trong không gian của ViT; phải chiếu sang không gian embedding của LLM (d khác). Các lựa chọn: tuyến tính/MLP 2 lớp (LLaVA, bài 34, 35), Q-Former (BLIP-2, bài 33), cross-attention (Flamingo, bài 32). Nhiều mô hình còn **gộp token** (ví dụ gộp 2x2 patch thành 1) để giảm số visual token.

13.3 Visual token đi vào decoder

Chuỗi đầu vào của decoder $=$ [visual tokens] $+$ [prompt tokens], rồi sinh text. Visual token chỉ xuất hiện ở **prefill** nhưng chiếm chỗ trong KV cache suốt quá trình decode.

13.4 Độ phân giải động (dynamic resolution)

Tài liệu và trang OCR cần độ phân giải cao (chữ nhỏ). Thay vì ép về 224x224, các mô hình mới (InternVL 1.5 bài 36, Qwen2-VL bài 37) cho **số visual token thay đổi theo kích thước ảnh**: ảnh lớn hơn nhiều token hơn.

Ví dụ (tự tính, giả định): ảnh 896×896, $P = 14$: $64 \times 64 = 4096$ patch; gộp 2x2 $\rightarrow$ 1024 visual token. ViT xử lý 4096 token (attention $n^2 = 16.8$M điểm mỗi head mỗi khối), decoder nhận 1024 token ảnh.

---

14. Lịch đọc theo số thư mục

Mỗi mục là gợi ý khoảng một buổi (1 đến 2 giờ/bài ngắn; bài lớn như GPT-3, Flamingo chỉ đọc phần lõi).

Giai đoạn	Bài	Mục tiêu
A. Nền Transformer	01, 09, 10, 11, 12	Attention, Pre-LN, RMSNorm, SwiGLU, RoPE: các khối trong decoder hiện đại
B. Quy mô và huấn luyện	02, 03, 04, 05, 06, 07, 08	GPT-3, scaling laws, Chinchilla, LLaMA, Mistral, RLHF, LoRA
C. Giảm KV và ngữ cảnh	13, 14, 15, 16, 17	ALiBi, MQA, GQA, MLA, sliding window
D. Attention tuyến tính và hybrid	18, 19, 20, 21, 22, 23, 24, 25, 26	Từ Linear Transformer đến Gated DeltaNet và hybrid
E. MoE	27, 28	Switch, Mixtral và định tuyến chuyên gia
F. Thị giác và VLM	29, 30, 31, 32, 33, 34, 35, 36, 37, 38	ViT, CLIP, SigLIP, ... , Qwen2-VL, SmolVLM
G. OCR	39, 40, 41, 42	Donut, Nougat, GOT và olmOCR
H. Suy luận hiệu quả	44, 45, 46, 47, 48, 49, 50	FlashAttention, PagedAttention, SGLang, speculative decoding
I. Lượng tử hoá và nén KV	51, 52, 53, 54, 55, 56, 57	LLM.int8, SmoothQuant, GPTQ, AWQ, StreamingLLM, H2O, FastV

Lộ trình nhập môn gợi ý: 00 $\rightarrow$ 01 $\rightarrow$ 03 $\rightarrow$ 04 $\rightarrow$ 09–12 $\rightarrow$ 14–15 $\rightarrow$ 18 $\rightarrow$ 23–25 $\rightarrow$ 29 $\rightarrow$ 34, 37 $\rightarrow$ 44 $\rightarrow$ 48 $\rightarrow$ 53–54 $\rightarrow$ 57.

---

15. Bảng thuật ngữ

Thuật ngữ	Nghĩa ngắn
Token	Đơn vị rời rạc (mảnh từ/byte/patch) mà mô hình xử lý
BPE	Thuật toán gộp cặp phổ biến để tạo từ vựng subword
Embedding	Vector d chiều tra bảng cho mỗi token
Cross-entropy	$-\log p$(token đúng) trung bình; hàm loss của LM
Residual stream	Vector d chiều mỗi token, mọi tầng đọc và cộng vào
Attention	Trung bình có trọng số của value theo độ khớp query-key
Decode	Pha sinh từng token
KV cache	K, V đã tính của các token trước, lưu để dùng lại
MQA / GQA / MLA	Chia sẻ hoặc nén KV để giảm cache
Arithmetic intensity	FLOPs / byte truyền
Roofline	Mô hình $P = \min(P_{peak}, I \cdot BW)$
Quantization	Lưu trọng số bằng ít bit hơn
Linear attention	Attention không softmax, viết được dạng hồi quy
SSM	State-space model, họ hồi quy tuyến tính (Mamba)
Hybrid	Trộn tầng tuyến tính và full attention
ViT	Vision Transformer, ảnh → patch → token
Distillation	Học sinh bắt chước đầu ra giảng viên

---

16. Sáu câu tự kiểm tra

Câu 1. Với $d_k = 64$, vì sao chia cho $\sqrt{d_k}$ và điều gì xảy ra nếu bỏ? **Đáp án.**

Nếu thành phần q, k độc lập, trung bình 0, phương sai 1 thì $q \cdot k$ có phương sai $d_k = 64$, độ lệch chuẩn 8. Điểm lớn đẩy softmax thành gần one-hot, gradient qua softmax rất nhỏ, huấn luyện chậm hoặc kẹt. Chia $\sqrt{d_k} = 8$ đưa phương sai về 1.

Câu 2. Một mô hình 12 tầng, 8 KV head, $d_{head} = 64$, BF16. KV cache cho 2.048 token là bao nhiêu? Nếu chỉ 6 tầng là full attention thì sao? **Đáp án.**

$2 \cdot 12 \cdot 8 \cdot 64 \cdot 2048 \cdot 2 = 50{,}331{,}648$ B ≈ 50.3 MB. Với 6 tầng, cache bằng một nửa, khoảng 25.2 MB. Các tầng tuyến tính chỉ thêm trạng thái cố định.

Câu 3. Vì sao decode batch 1 của mô hình một tỷ tham số BF16 có thể bị giới hạn bởi băng thông bộ nhớ? **Đáp án.**

Nếu phải đọc 2 GB trọng số cho mỗi token, băng thông 400 GB/s cho thời gian đọc lý tưởng 5 ms. Chi phí tính khoảng 2 GFLOPs mỗi token nhỏ hơn thời gian đọc trên thiết bị đạt 10 TFLOP/s. Đó là ước lượng roofline: tốc độ thực tế còn chịu ảnh hưởng của cache, kích hoạt và overhead.

Câu 4. Tại sao attention là $O(n^2)$ thời gian nhưng FlashAttention vẫn cho kết quả toán học giống hệt? **Đáp án.**

Thời gian $O(n^2 d)$ vì cần mọi cặp (i, j) của $QK^\top$ và AV. FlashAttention không giảm số FLOPs mà tính theo khối (tiling) trong SRAM với softmax trực tuyến (online softmax), không bao giờ ghi ma trận $n \times n$ ra HBM. Kết quả chính xác vì chỉ đổi thứ tự tính và cách truy cập bộ nhớ, giảm bộ nhớ phụ về $O(n)$ và số byte HBM.

Câu 5. Ghi một cặp (k, v) vào trạng thái S bằng quy tắc cộng dồn khác gì quy tắc delta? Tại sao delta tốt hơn cho truy hồi? **Đáp án.**

Cộng dồn: $S_t = S_{t-1} + v_t k_t^\top$, không bao giờ xoá nên khi nhiều key tương quan thì nhiễu chồng chất, S bị bão hoà. Delta: $S_t = S_{t-1}(I - \beta_t k_t k_t^\top) + \beta_t v_t k_t^\top$, trước tiên loại bỏ giá trị cũ gắn với k_t rồi mới ghi giá trị mới (như một bước SGD sửa lỗi hồi quy tuyến tính), nên bộ nhớ hữu hạn được dùng hiệu quả hơn cho việc tra cứu theo key (bài 23, 24).

Câu 6. Ảnh 1008×1008, patch 14, gộp 2x2. Bao nhiêu patch vào ViT và bao nhiêu visual token vào decoder? Điều này ảnh hưởng gì tới prefill và KV cache? **Đáp án.**

$1008/14 = 72$, nên $72^2 = 5184$ patch vào ViT. Gộp 2x2 cho $36^2 = 1296$ visual token vào decoder. ViT tốn attention n^2 trên 5184 token (compute-bound, một lần/trang). Decoder: prefill xử lý 1296 + prompt token; mỗi token ảnh để lại KV ở mọi tầng full-attention (cache ban đầu $\propto 1296$), và mỗi bước decode phải đọc chúng. Giảm token ảnh (gộp mạnh hơn, FastV) giảm prefill và KV nhưng có thể mất chữ nhỏ.

Kết luận ngắn: Ba ý cần mang theo: (1) mọi thứ xoay quanh attention $\text{softmax}(QK^\top/\sqrt{d_k})V$ với chi phí $O(n^2)$ và KV cache $\propto n$; (2) decode batch-1 bị chặn bởi **byte đọc**, không phải FLOPs, nên mọi tối ưu cho GPU nhỏ là bài toán giảm byte và số bước tuần tự; (3) attention tuyến tính/delta đổi KV cache tăng dần thành trạng thái cố định, đổi lại khả năng truy hồi chính xác, nên hybrid là điểm giữa.

Bài 01

Attention Is All You Need (2017)

Tác giả: Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, Illia Polosukhin (Google Brain / Google Research / Univ. of Toronto).
Venue / arXiv: arXiv:1706.03762 (bản v7, 2023); NIPS 2017 (31st Conference on Neural Information Processing Systems).
Nguồn: https://arxiv.org/abs/1706.03762
Chủ đề: kiến trúc Transformer, scaled dot-product attention, multi-head attention, mã hoá vị trí sinusoidal, so sánh chi phí với RNN/CNN.
Vị trí trong lộ trình: đọc sau bài 00 (preface). Đây là nền của mọi bài còn lại. Bài này bổ sung: bỏ hẳn recurrence/convolution, chỉ dùng attention. Tiếp theo: 02 (GPT-3, decoder-only quy mô lớn), 03 và 04 (scaling laws), 09 đến 12 (cải tiến từng khối), 29 (ViT đưa Transformer sang ảnh).

> Tài liệu học tập bằng tiếng Việt, ghi chú diễn giải, không phải bản dịch. Mọi con số lấy từ paper (trừ khi ghi "tự tính"). Các ví dụ đánh dấu "tự tính" là phép tính minh họa, không phải kết quả trực tiếp của bài báo.

1. Bối cảnh

Trước 2017, mô hình chuỗi (dịch máy, mô hình ngôn ngữ) chủ yếu là **RNN/LSTM/GRU** ("recurrent neural network": đọc từng token, giữ một trạng thái ẩn $h_t = f(h_{t-1}, x_t)$). Có hai vấn đề:

1. **Tuần tự**: h_t cần h_{t-1}, nên không song song hoá được trong một chuỗi. Paper nói rõ điều này "precludes parallelization within training examples", càng tệ với chuỗi dài vì bộ nhớ hạn chế kích thước batch.
2. **Phụ thuộc xa khó học**: tín hiệu giữa hai vị trí cách xa phải đi qua nhiều bước, gradient dễ tắt.

Attention ("chú ý": cho mỗi vị trí đầu ra tự chọn xem nên lấy thông tin từ những vị trí đầu vào nào) đã được dùng cùng RNN (Bahdanau và cộng sự, 2014) để giải vấn đề thứ hai, nhưng RNN vẫn còn đó. Các hướng khác (ConvS2S, ByteNet) dùng **tích chập** để song song hoá; nhưng số phép tính để nối hai vị trí xa tăng theo khoảng cách (tuyến tính với ConvS2S, logarit với ByteNet).

Câu hỏi của paper: có thể bỏ cả recurrence lẫn convolution, chỉ dùng attention, mà vẫn ra kết quả tốt và huấn luyện nhanh hơn không?

2. Ý tưởng chính

Transformer xếp chồng các tầng gồm hai việc: (i) **self-attention** (mỗi token nhìn mọi token khác trong cùng chuỗi và lấy trung bình có trọng số thông tin của chúng) và (ii) một mạng **feed-forward** áp riêng cho từng vị trí. Vì mỗi cặp vị trí nối với nhau trong **một** phép tính (độ dài đường đi $O(1)$) và mọi vị trí tính cùng lúc, huấn luyện song song rất tốt. Vì không còn thứ tự ngầm của RNN, phải **cộng mã hoá vị trí** vào embedding. Multi-head attention cho phép nhiều "góc nhìn" attention chạy song song. Kết quả: chất lượng dịch tốt hơn với chi phí huấn luyện thấp hơn nhiều.

3. Phương pháp chi tiết

3.1 Kiến trúc tổng thể

Encoder-decoder (như dịch máy):

- **Encoder**: $N = 6$ tầng giống nhau; mỗi tầng = multi-head self-attention + FFN theo vị trí. Mỗi sub-layer bọc bằng residual và LayerNorm: $\text{LayerNorm}(x + \text{Sublayer}(x))$ (đây là **Post-LN**, so sánh với bài 09).

- **Decoder**: $N = 6$ tầng, mỗi tầng thêm sub-layer thứ ba là **encoder-decoder (cross) attention** (query từ decoder, key/value từ đầu ra encoder). Self-attention của decoder có **causal mask** để vị trí i chỉ nhìn $j \le i$; kết hợp với việc dịch đầu ra một vị trí để giữ tính tự hồi quy.
- Mọi sub-layer và embedding xuất chiều $d_{model} = 512$ để residual cộng được.
- Embedding đầu vào, đầu ra và phép chiếu tuyến tính trước softmax **dùng chung trọng số** (weight tying); embedding nhân $\sqrt{d_{model}}$.

3.2 Scaled dot-product attention

Với $Q \in \mathbb{R}^{n \times d_k}$, $K \in \mathbb{R}^{m \times d_k}$, $V \in \mathbb{R}^{m \times d_v}$:

$$\boxed{\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V}$$

Lý do chia $\sqrt{d_k}$ (chú thích 4 của paper): nếu các thành phần của q, k độc lập, trung bình 0, phương sai 1, thì $q \cdot k = \sum_{i=1}^{d_k} q_i k_i$ có trung bình 0 và phương sai d_k. Với d_k lớn, điểm lớn đẩy softmax vào vùng gradient cực nhỏ. Paper so sánh với **additive attention** (mạng một tầng ẩn tính độ tương thích): lý thuyết tương đương, nhưng dot-product nhanh và tiết kiệm bộ nhớ hơn vì dùng được phép nhân ma trận tối ưu; additive tốt hơn dot-product *không scale* khi d_k lớn, nên họ thêm hệ số scale.

Mask (decoder): đặt $-\infty$ vào các điểm tương ứng kết nối bất hợp lệ trước softmax.

3.3 Multi-head attention

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \ldots, \text{head}_h) W^O, \quad \text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$$

với $W_i^Q, W_i^K \in \mathbb{R}^{d_{model} \times d_k}$, $W_i^V \in \mathbb{R}^{d_{model} \times d_v}$, $W^O \in \mathbb{R}^{hd_v \times d_{model}}$. Paper dùng $h = 8$, $d_k = d_v = d_{model}/h = 64$, nên tổng tính toán gần bằng một head đầy đủ chiều. Lý do: một head duy nhất phải "lấy trung bình" nên mất thông tin từ nhiều không gian con; nhiều head cho phép "jointly attend to information from different representation subspaces at different positions".

Ba cách dùng attention trong mô hình: encoder-decoder attention, encoder self-attention, decoder masked self-attention.

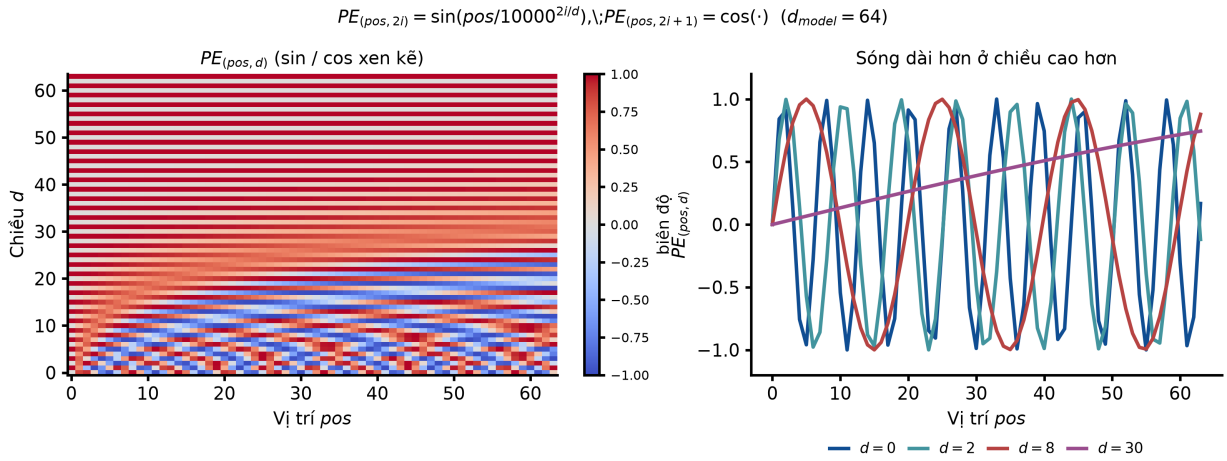
3.4 FFN theo vị trí

$$\text{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2, \qquad d_{model} = 512,\ d_{ff} = 2048.$$

Giống hai tích chập kích thước 1. Cùng tham số cho mọi vị trí, khác nhau giữa các tầng.

3.5 Mã hoá vị trí sinusoidal

Không có recurrence/conv nên phải đưa thứ tự vào:



Mã hóa vị trí sinusoidal $PE_{(pos,2i)} = \sin(pos/10000^{2i/d})$, $PE_{(pos,2i+1)} = \cos(\cdot)$: heatmap toàn bộ chiều và vài sóng theo pos ($d_{model} = 64$). (minh hoạ, tự tính)

$$PE_{(pos,2i)} = \sin\big(pos/10000^{2i/d_{model}}\big), \qquad PE_{(pos,2i+1)} = \cos\big(pos/10000^{2i/d_{model}}\big).$$

Bước sóng tạo thành cấp số nhân từ 2π đến $10000 \cdot 2\pi$. Giả thuyết của tác giả: với mọi độ lệch cố định k, PE_{pos+k} là hàm tuyến tính của PE_{pos} nên mô hình dễ học chú ý theo vị trí tương đối. Họ cũng thử embedding vị trí học được và thấy kết quả "nearly identical" (Bảng 3, hàng E); chọn sinusoidal vì có thể ngoại suy sang chuỗi dài hơn lúc huấn luyện (đây là kỳ vọng, không được kiểm chứng trong bài).

3.6 Vì sao self-attention? (Mục 4, Bảng 1)

Loại tầng	Độ phức tạp mỗi tầng	Số phép tuần tự	Độ dài đường đi tối đa
Self-attention	$O(n^2 \cdot d)$	$O(1)$	$O(1)$
Recurrent	$O(n \cdot d^2)$	$O(n)$	$O(n)$
Convolutional	$O(k \cdot n \cdot d^2)$	$O(1)$	$O(\log_k n)$
Self-attention (hạn chế, láng giềng r)	$O(r \cdot n \cdot d)$	$O(1)$	$O(n/r)$

Ở đây n là độ dài chuỗi, d chiều biểu diễn, k kích thước kernel. Self-attention nhanh hơn recurrent khi $n < d$ (thường đúng với biểu diễn word-piece/BPE trong dịch máy). Để xử lý chuỗi rất dài, đề xuất (chưa thử) giới hạn attention trong vùng bán kính r (tiền thân của sliding window, bài 17). Phụ lục còn nói self-attention có thể dễ giải thích hơn.

3.7 Huấn luyện

- Dữ liệu: WMT 2014 EN-DE (~4.5M cặp câu, BPE chung ~37.000 token) và EN-FR (36M câu, word-piece 32.000). Mỗi batch ~25.000 token nguồn và 25.000 token đích.
- Phần cứng: 1 máy 8 GPU NVIDIA P100. Mô hình base: 0,4 s/bước, 100.000 bước (12 giờ). Mô hình big: 1,0 s/bước, 300.000 bước (3,5 ngày).
- Optimizer Adam ($\beta_1 = 0.9, \beta_2 = 0.98, \epsilon = 10^{-9}$) với lịch learning rate:

$$lrate = d_{model}^{-0.5} \cdot \min\big(step^{-0.5},\; step \cdot warmup^{-1.5}\big), \quad warmup = 4000.$$

Tăng tuyến tính 4000 bước đầu, sau đó giảm theo $1/\sqrt{step}$.

- Regularization: dropout $P_{drop} = 0.1$ (lên sub-layer output và tổng embedding+PE); **label smoothing** $\epsilon_{ls} = 0.1$ (làm tệ perplexity nhưng tăng accuracy và BLEU).
- Suy luận: beam size 4, length penalty $\alpha = 0.6$; trung bình 5 checkpoint cuối (base) hoặc 20 (big).

Pseudo-code một tầng encoder (Post-LN như paper):

```
def encoder_layer(x):                       # x: (n, d_model)
    a = multi_head_self_attention(x)        # Q=K=V=x, không mask
    x = layer_norm(x + dropout(a))
    f = relu(x @ W1 + b1) @ W2 + b2         # d_ff = 2048
    return layer_norm(x + dropout(f))
# decoder: thêm masked self-attn trước, và cross-attn(Q=decoder, K,V=encoder_out) ở giữa
```

Ví dụ đếm tham số (tự tính, d=512, bỏ bias/norm): self-attention $4 \cdot 512^2 \approx 1.05$M; FFN $2 \cdot 512 \cdot 2048 \approx 2.1$M; một tầng encoder ≈ 3.15M, 6 tầng ≈ 18.9M; mỗi tầng decoder thêm một attention → ≈ 4.2M, 6 tầng ≈ 25.2M; embedding chung $37000 \cdot 512 \approx 18.9$M. Tổng ≈ 63M, khớp với 65M ghi trong Bảng 3.

4. Thí nghiệm và kết quả chính

Dịch máy (Bảng 2, newstest2014). BLEU và chi phí huấn luyện (FLOPs, ước lượng bằng thời gian x số GPU x công suất FP32 duy trì: 2.8, 3.7, 6.0, 9.5 TFLOPS cho K80, K40, M40, P100):

Mô hình	BLEU EN-DE	BLEU EN-FR	FLOPs EN-DE	FLOPs EN-FR
GNMT + RL	24.6	39.92	2.3·10^19	1.4·10^20
ConvS2S	25.16	40.46	9.6·10^18	1.5·10^20
MoE	26.03	40.56	2.0·10^19	1.2·10^20
GNMT + RL (ensemble)	26.30	41.16	1.8·10^20	1.1·10^21
ConvS2S (ensemble)	26.36	41.29	7.7·10^19	1.2·10^21
Transformer (base)	27.3	38.1	3.3·10^18	(cột trích bị lỗi)
Transformer (big)	**28.4**	**41.8**	2.3·10^19	(cột trích bị lỗi)

Lưu ý về tính nhất quán: Bảng 2 và abstract ghi 41.8 BLEU cho EN-FR (big), nhưng đoạn văn mục 6.1 ghi "41.0". Ghi chú dùng số từ Bảng 2 và abstract; đây là điểm không nhất quán trong bài báo. Trên EN-DE, big vượt kết quả trước đó (kể cả ensemble) hơn 2.0 BLEU; ngay cả base cũng vượt mọi mô hình đã công bố với chi phí nhỏ hơn. Với EN-FR, big đạt state-of-the-art đơn mô hình với chưa đến 1/4 chi phí huấn luyện của mô hình tốt nhất trước đó (theo paper).

Constituency parsing (Bảng 4, WSJ section 23, F1). Transformer 4 tầng, $d_{model} = 1024$: 91.3 (chỉ WSJ, ~40K câu) và 92.7 (bán giám sát, ~17M câu). So: Dyer et al. 2016 (discriminative) 91.7; Vinyals & Kaiser 2014 bán giám sát 92.1; RNN Grammar (generative) 93.3. Paper nói Transformer tốt hơn BerkeleyParser ngay cả khi chỉ dùng 40K câu WSJ, và hầu hết các mô hình trước ngoại trừ RNNG.

5. Phân tích / ablation

Bảng 3 (EN-DE dev newstest2013, base: PPL 4.92, BLEU 25.8, 65M tham số; PPL tính theo word-piece):

Biến thể	PPL	BLEU	Ghi chú
(A) 1 head ($d_k = 512$)	5.29	24.9	single head kém 0.9 BLEU so với tốt nhất
(A) 4 head ($d_k = 128$)	5.00	25.5	

Biến thể	PPL	BLEU	Ghi chú
(A) 16 head ($d_k = 32$)	4.91	25.8	ngang base
(A) 32 head ($d_k = 16$)	5.01	25.4	quá nhiều head cũng giảm
(B) $d_k = 16$	5.16	25.1	giảm d_k làm tệ đi
(B) $d_k = 32$	5.01	25.4	
(C) $N = 2$	6.11	23.7	36M tham số
(C) $N = 4$	5.19	25.3	
(C) $N = 8$	4.88	25.5	80M
(C) $d_{model} = 256$	5.75	24.5	28M
(C) $d_{model} = 1024$	4.66	26.0	168M
(C) $d_{ff} = 1024$	5.12	25.4	
(C) $d_{ff} = 4096$	4.75	26.2	90M
(D) dropout 0.0 / 0.2	5.77 / 4.95	24.6 / 25.5	dropout giúp chống overfit
(D) $\epsilon_{ls} = 0.0$ / 0.2	4.67 / 5.47	25.3 / 25.7	label smoothing: PPL tệ, BLEU tốt
(E) positional embedding học được	4.92	25.7	gần như bằng sinusoidal
big (N=6, 1024, 4096, h=16, drop 0.3, 300K bước)	4.33	26.4	213M

Kết luận của tác giả: mô hình lớn hơn tốt hơn, dropout rất hữu ích; giảm d_k hại chất lượng ("determining compatibility is not easy", gợi ý hàm tương thích tinh vi hơn có thể tốt hơn dot-product); vị trí học được và sinusoidal tương đương. Mục Phụ lục (không có trong text trích) còn hiển thị các head attention làm những việc khác nhau, một số liên quan tới cấu trúc cú pháp/ngữ nghĩa (đó là quan sát định tính).

6. Giới hạn

Do tác giả nêu:

- Self-attention tốn $O(n^2)$; với chuỗi rất dài cần attention hạn chế (hướng tương lai).
- Cần mở rộng sang các modality khác ngoài văn bản (ảnh, âm thanh, video) và các cơ chế attention cục bộ.
- Muốn làm cho việc sinh "less sequential" (decoder vẫn tự hồi quy).
- Mất độ phân giải hiệu dụng do lấy trung bình có trọng số (giảm nhẹ bằng multi-head).

Nhận xét bổ sung:

- Chỉ đánh giá dịch máy và parsing; không bàn đến mô hình ngôn ngữ quy mô lớn hay pre-training (đó là việc của bài 02 và các bài sau).
- Không có KV cache hay phân tích chi phí suy luận (chỉ phân tích huấn luyện). Tại suy luận tự hồi quy, chi phí thật bị chặn bởi băng thông bộ nhớ (bài 00, mục 10).
- Số liệu thiếu nhất quán nhỏ (41.0 so với 41.8). Ngoại suy chuỗi dài của PE sinusoidal chỉ là kỳ vọng; các bài sau (RoPE, ALiBi: 12, 13) cho thấy vấn đề này khó hơn.
- Post-LN cần warmup cẩn thận (lịch lr 4000 bước warmup); bài 09 phân tích kỹ.

7. Liên hệ với các bài khác

- **00** (preface): mọi công thức và shape của bài này được giải nghĩa chi tiết ở đó.
- **02** GPT-3: giữ khối Transformer, bỏ encoder, dùng decoder-only phóng to.
- **03, 04**: scaling laws dùng đúng công thức tham số $N \approx 12\, L\, d^2$ và $C \approx 6ND$ rút ra từ kiến trúc này.

- **09** Pre-LN, **10** RMSNorm, **11** SwiGLU, **12** RoPE, **13** ALiBi: thay từng chi tiết của bài này (vị trí LayerNorm, loại norm, FFN, mã hoá vị trí).
- **14, 15, 16**: giảm KV cache của multi-head attention.
- **17**: hiện thực hoá ý "restricted self-attention" mà bài này dự định.
- **18 đến 26**: thay softmax attention $O(n^2)$ bằng hồi quy tuyến tính, trạng thái cố định.
- **29** ViT: đưa chính kiến trúc này sang ảnh.
- **39, 40**: Donut, Nougat dùng encoder-decoder kiểu bài này cho OCR.
- **44, 45**: tối ưu cách tính đúng công thức attention ở mục 3.2 trên GPU.

8. Gợi ý áp dụng

Công thức $12Ld^2$ cho decoder chuẩn cho phép ước lượng số tham số của các tầng Transformer, chưa tính embedding. Khi so sánh kiến trúc, cần tính thêm các biến thể như GQA hoặc SwiGLU và phân biệt chi phí attention với phép chiếu và FFN. Chi phí suy luận thực tế còn phụ thuộc vào việc đọc trọng số, KV cache và cách triển khai; các phép tính trong bài 00 chỉ là ước lượng.

9. Thí nghiệm gợi ý

1. **Kiểm tra hệ số $\sqrt{d_k}$.** Lấy $q, k \sim \mathcal{N}(0, I)$ với $d_k \in \{16, 64, 256, 1024\}$; đo độ lệch chuẩn của $q \cdot k$ và entropy của softmax có và không chia cho $\sqrt{d_k}$. So sánh với phương sai lý thuyết d_k.
2. **So sánh chi phí theo độ dài.** Dùng profiler đo attention và phần chiếu cùng FFN theo n, rồi đối chiếu với các ước lượng $4n^2d$ và $24nd^2$. Theo mô hình đơn giản này, hai chi phí bằng nhau tại $n \approx 6d$; thời gian thực tế còn phụ thuộc phần cứng và kernel.
3. **Khảo sát các attention head.** Tắt từng head trong mô hình thử nghiệm và đo thay đổi trên tập đánh giá. Đối chiếu với ablation về số head của bài báo, nhưng không xem hai phép thử là tương đương.

10. Tóm tắt công thức / khái niệm

Khái niệm	Công thức / giá trị
Attention	$\text{softmax}(QK^\top/\sqrt{d_k})V$
Multi-head	$\text{Concat}(\text{head}_i)W^O$, $h = 8$, $d_k = d_v = 64$
FFN	$\max(0, xW_1 + b_1)W_2 + b_2$, $d_{ff} = 2048$
PE	$\sin$ / $\cos(pos/10000^{2i/d})$
Learning rate	$d^{-0.5} \min(s^{-0.5}, s \cdot 4000^{-1.5})$
Độ phức tạp self-attn	$O(n^2 d)$, đường đi $O(1)$, tuần tự $O(1)$
Base / big	65M / 213M tham số; 28.4 / 41.8 BLEU (big)
Chi phí	base 12 giờ, big 3.5 ngày trên 8 P100

11. Kết luận ngắn

Transformer thay recurrence bằng attention, cho phép tính song song các vị trí trong huấn luyện và rút ngắn đường đi giữa các vị trí xuống $O(1)$. Mô hình đạt kết quả dịch máy hàng đầu với chi phí huấn luyện thấp hơn trong các so sánh của bài báo. Chi phí $O(n^2)$ và thiếu phân tích suy luận là hai điểm các bài sau (hiệu quả attention, KV cache, hybrid) lần lượt giải quyết.

Nguồn gốc: https://arxiv.org/abs/1706.03762

Bài 02

Language Models are Few-Shot Learners (2020)

Tác giả: Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell và cộng sự (OpenAI; tổng cộng 31 tác giả).
Venue / arXiv: arXiv:2005.14165 (v4, 22/07/2020; bản nộp 28/05/2020). Paper này được biết đến là bản NeurIPS 2020, nhưng bản text không ghi venue nên chỉ nêu arXiv.
Nguồn: https://arxiv.org/abs/2005.14165
Chủ đề: Mô hình ngôn ngữ tự hồi quy 175B tham số, in-context learning (zero/one/few-shot), scaling theo kích thước mô hình, ô nhiễm benchmark (data contamination).
Vị trí trong lộ trình: Đọc sau *01 Attention Is All You Need* (kiến trúc Transformer). Bài này cho thấy mở rộng Transformer decoder và dữ liệu giúp mô hình thực hiện nhiều tác vụ qua prompt mà không cần fine-tune. Nó dùng kết quả của *03 Kaplan scaling laws* (quyết định train mô hình rất lớn, ít token), và bị *04 Chinchilla* phản biện (GPT-3 train thiếu dữ liệu so với kích thước). Tiếp theo: *05 LLaMA* (làm ngược lại: mô hình nhỏ hơn, nhiều token hơn), *07 InstructGPT* (tinh chỉnh GPT-3 theo chỉ dẫn).

> Ghi chú diễn giải bằng tiếng Việt, không phải bản dịch. Các số liệu "tự tính" là ví dụ minh họa, không phải kết quả trực tiếp của bài báo.

1. Bối cảnh

Mô hình hóa ngôn ngữ (language modeling): huấn luyện mạng dự đoán token tiếp theo từ các token trước. Không cần nhãn người gán, nên có thể tận dụng các tập văn bản rất lớn.

Paradigm trước GPT-3: pre-train rồi fine-tune. Pre-train một Transformer lớn trên văn bản thô (GPT-1/2, BERT, T5), sau đó *fine-tune* (tiếp tục huấn luyện, cập nhật trọng số) trên một tập có nhãn riêng cho từng tác vụ, thường hàng nghìn đến hàng trăm nghìn mẫu. Kiến trúc thì dùng chung, nhưng *dữ liệu và bước fine-tune thì phải làm lại cho mỗi tác vụ*.

Paper nêu ba lý do vì sao đó là hạn chế:

1. **Thực tế:** có vô số tác vụ ngôn ngữ hữu ích; gom bộ dữ liệu có nhãn cho từng cái là không khả thi.
2. **Tương quan giả (spurious correlations):** mô hình lớn fine-tune trên tập hẹp dễ bám vào đặc điểm tình cờ của tập đó, nên điểm benchmark "ngang người" có thể phóng đại năng lực thật.
3. **So với con người:** người chỉ cần một câu chỉ dẫn hoặc vài ví dụ là làm được tác vụ mới.

Ý tưởng tiền đề: GPT-2 đã thử đưa tác vụ vào *văn bản đầu vào* (prompt) và để mô hình tự điền tiếp, nhưng kết quả còn kém xa fine-tune (paper nhắc: GPT-2 chỉ đạt 4% trên Natural Questions). Cùng lúc, các nghiên cứu scaling (Kaplan et al., bài 03) cho thấy loss giảm mượt theo kích thước. Giả thuyết của GPT-3: *khả năng học từ ngữ cảnh cũng sẽ tăng theo quy mô*.

---

2. Ý tưởng chính

Huấn luyện một Transformer decoder khổng lồ (175 tỷ tham số, gấp 10 lần mô hình dày lớn nhất trước đó theo paper) chỉ với mục tiêu "đoán token tiếp theo" trên ~300 tỷ token. Sau đó, **không cập nhật trọng số nào**: ta chỉ viết vào đầu vào một mô tả tác vụ và/hoặc vài ví dụ mẫu (demonstrations) rồi để mô hình điền tiếp. Cách dùng này gọi là **in-context learning** (học trong ngữ cảnh): "việc học" xảy ra trong một lượt forward pass, trên chính chuỗi đầu vào, chứ không phải bằng gradient.

Điểm quan trọng nhất của paper là *quan sát thực nghiệm*: khoảng cách giữa zero-shot, one-shot và few-shot **tăng lên cùng kích thước mô hình**; mô hình càng lớn càng khai thác tốt các ví dụ trong ngữ cảnh. Paper gọi đây là dạng "meta-learning" ngầm: vòng ngoài là pre-train (học nhiều kỹ năng và nhận dạng mẫu), vòng trong là in-context learning lúc suy luận.

---

3. Phương pháp chi tiết

3.1 Bốn chế độ sử dụng (Hình 2.1)

Chế độ	Có cập nhật trọng số?	Trong ngữ cảnh có gì	Số mẫu
Fine-tuning (FT)	Có	(không dùng ngữ cảnh)	nghìn đến trăm nghìn mẫu có nhãn
Few-shot (FS)	Không	mô tả (tuỳ chọn) + K ví dụ + 1 truy vấn	K thường 10 đến 100
One-shot (1S)	Không	mô tả + 1 ví dụ + truy vấn	1
Zero-shot (0S)	Không	chỉ mô tả bằng ngôn ngữ tự nhiên + truy vấn	0

Paper **không fine-tune GPT-3** (để lại cho tương lai), chỉ đánh giá FS/1S/0S. K bị giới hạn bởi cửa sổ ngữ cảnh $n_{ctx} = 2048$.

Ví dụ prompt few-shot dịch Anh-Pháp (minh họa, theo tinh thần Hình 2.1):

```
Translate English to French:
sea otter => loutre de mer
peppermint => menthe poivrée
plush giraffe => girafe peluche
cheese =>            <- mô hình điền tiếp
```

Cách chấm điểm (mục 2.4):

- **Trắc nghiệm:** đưa K ví dụ (ngữ cảnh + đáp án đúng), rồi ngữ cảnh của câu hỏi; so sánh *likelihood* của từng phương án, thường chuẩn hoá theo số token. Với ARC, OpenBookQA, RACE chuẩn hoá theo $\frac{P(\text{completion}|\text{context})}{P(\text{completion}|\text{"Answer: "})}$.
- **Sinh tự do:** beam search, beam width 4, length penalty $\alpha = 0.6$; chấm bằng F1, BLEU hoặc exact match.

3.2 Kiến trúc

Paper nói gọn: *"same model and architecture as GPT-2"* (decoder-only Transformer, khởi tạo cải tiến, **pre-normalization**, tokenizer BPE ở mức byte) với **một khác biệt**: các lớp **xen kẽ attention dày (dense) và attention thưa kiểu dải băng cục bộ (locally banded sparse)**, giống Sparse Transformer. Paper không mô tả chi tiết dải băng này (độ rộng, lớp nào dày lớp nào thưa), nên đừng suy diễn thêm.

Giải thích thuật ngữ:

- **Decoder-only:** chỉ có khối giải mã với causal mask (mỗi token chỉ nhìn các token trước).
- **Pre-normalization (pre-LN):** áp dụng LayerNorm *trước* mỗi sub-layer (attention/FFN) thay vì sau; huấn luyện ổn định hơn ở mô hình sâu (xem bài 09).
- **BPE (byte-pair encoding):** gộp các cặp ký tự/byte hay đi cùng nhau thành token.
- **Locally banded sparse attention:** mỗi token chỉ attend vào một dải w token gần nhất, thay vì toàn bộ quá khứ → tính toán $O(nw)$ thay vì $O(n^2)$ (cùng ý tưởng sliding-window ở bài 17 và Mistral, bài 06).

Một block (đúng theo kiểu GPT-2, pre-LN):

$$x \leftarrow x + \text{Attn}(\text{LN}(x)), \qquad x \leftarrow x + \text{FFN}(\text{LN}(x)), \qquad d_{ff} = 4\, d_{model}$$

Các cấu hình (Bảng 2.1, nguyên văn từ paper; tất cả train trên **300B token**, $n_{ctx} = 2048$):

Mô hình	n_{params}	n_{layers}	d_{model}	n_{heads}	d_{head}	Batch (token)	LR
GPT-3 Small	125M	12	768	12	64	0.5M	6.0×10^{-4}
GPT-3 Medium	350M	24	1024	16	64	0.5M	3.0×10^{-4}
GPT-3 Large	760M	24	1536	16	96	0.5M	2.5×10^{-4}
GPT-3 XL	1.3B	24	2048	24	128	1M	2.0×10^{-4}
GPT-3 2.7B	2.7B	32	2560	32	80	1M	1.6×10^{-4}
GPT-3 6.7B	6.7B	32	4096	32	128	2M	1.2×10^{-4}
GPT-3 13B	13.0B	40	5140	40	128	2M	1.0×10^{-4}
GPT-3 175B	175.0B	96	12288	96	128	3.2M	0.6×10^{-4}

Lưu ý: $d_{model} = 5140$ cho bản 13B là đúng như bảng trong paper (bản text), con số hơi lạ (không chia hết cho 128) nên ghi chú giữ nguyên. Quy luật rõ ràng: **mô hình càng lớn → batch càng lớn, learning rate càng nhỏ** (paper dựa vào gradient noise scale để chọn batch).

Kiểm tra số tham số (tự tính): mỗi layer Transformer có $\approx 4d^2$ (attention: Q,K,V,O) $+8d^2$ (FFN với $d_{ff} = 4d$) $= 12d^2$ tham số. Với 175B: $12 \times 96 \times 12288^2 \approx 1.74 \times 10^{11}$, khớp 175B (chưa tính embedding, nhỏ so với tổng).

3.3 Dữ liệu (Bảng 2.2)

Dataset	Token	Trọng số trong mix huấn luyện	Số epoch qua 300B token
Common Crawl (đã lọc)	410B	60%	0.44
WebText2	19B	22%	2.9
Books1	12B	8%	1.9
Books2	55B	8%	0.43
Wikipedia	3B	3%	3.4

Ba bước cải thiện chất lượng: (1) lọc Common Crawl bằng bộ phân loại "giống corpus chất lượng cao"; (2) khử trùng lặp mờ ở mức tài liệu (MinHashLSH, giảm trung bình ~10% kích thước); (3) trộn thêm corpus chất lượng cao. Quan trọng: dataset **không lấy mẫu tỉ lệ với kích thước** mà ưu tiên nguồn tốt (Wikipedia thấy 3.4 lần, Common Crawl chưa đến 1 lần) - chấp nhận chút overfit để đổi lấy chất lượng. Common Crawl: 45TB nén → 570GB sau lọc (~400B token BPE). Ngôn ngữ: tiếng Anh chiếm 93% theo số từ.

3.4 Huấn luyện (Phụ lục B)

- Adam với $\beta_1 = 0.9,\ \beta_2 = 0.95,\ \epsilon = 10^{-8}$; clip gradient norm toàn cục ở 1.0.
- LR: warmup tuyến tính trong 375M token đầu, sau đó cosine decay xuống 10% giá trị gốc trong 260B token, rồi giữ 10%.
- Batch size tăng tuyến tính từ 32k token lên giá trị đầy đủ trong 4-12B token đầu.
- Weight decay 0.1; lấy mẫu không hoàn lại (sampling without replacement).

- **Packing:** luôn train chuỗi dài đủ 2048 token, nối nhiều tài liệu ngắn vào một chuỗi, ngăn cách bằng token end-of-text, *không* mask đặc biệt giữa các tài liệu.
- Hạ tầng: GPU V100, cụm của Microsoft; song song hoá mô hình theo cả chiều sâu (giữa các layer) lẫn chiều rộng (trong từng phép nhân ma trận).

3.5 Chi phí tính toán (Phụ lục D)

Paper ước lượng tính toán huấn luyện bỏ qua attention (dưới 10% tổng): mỗi token cần 2 FLOPs/tham số cho forward và nhân 3 cho forward+backward:

$$\boxed{C_{train} \approx 6\,N\,D}$$

với N là số tham số, D số token. Với GPT-3 175B: $6 \times 175 \times 10^9 \times 300 \times 10^9 \approx 3.15 \times 10^{23}$ FLOPs (tự tính), khớp bảng D.1 của paper: 3.14×10^{23} FLOPs = **3.64E+03 petaflop/s-day** (1 PF-day = 8.64×10^{19} FLOPs). Paper cũng so sánh: GPT-3 3B (~50 PF-day) tốn ngang RoBERTa-Large 355M (Hình 2.2): họ cố ý "train mô hình rất lớn trên ít token hơn thông thường", theo hướng của Kaplan et al.

3.6 Pseudo-code: in-context learning lúc suy luận

```
def few_shot_predict(model, task_description, demos, query, K):
    # demos: list[(context_i, completion_i)] lấy ngẫu nhiên từ tập train
    prompt = task_description + "\n"
    for c, y in random.sample(demos, K):        # K sao cho vừa 2048 token
        prompt += f"{c} {y}\n"
    prompt += f"{query}"
    # KHÔNG có gradient, KHÔNG sửa trọng số
    if multiple_choice:
        return argmax_over_options(model.logprob(option | prompt) / len_norm)
    else:
        return model.generate(prompt, beam=4, length_penalty=0.6)
```

---

4. Thí nghiệm và kết quả chính

Chọn lọc; mọi số liệu lấy từ các bảng trong paper (GPT-3 175B nếu không ghi khác).

4.1 Language modeling và cloze

Benchmark	SOTA trước đó	0-shot	1-shot	Few-shot
PTB (perplexity, thấp tốt hơn)	35.8	20.5	n/a	n/a
LAMBADA acc	68.0	76.2	72.5	86.4
LAMBADA ppl	8.63	3.00	3.35	1.92
StoryCloze acc	91.8	83.2	84.7	87.7
HellaSwag acc	85.6	78.9	78.1	79.3

LAMBADA là ví dụ đẹp về giá trị của few-shot: có ví dụ dạng điền chỗ trống thì mô hình hiểu "đáp án chính xác là một từ", nhảy từ 76.2% lên 86.4%.

4.2 Hỏi đáp closed-book (không cho tài liệu tham khảo)

Setting	NaturalQS	WebQS	TriviaQA
T5-11B+SSM (fine-tuned)	36.6	44.7	60.5
RAG (fine-tuned, open-domain)	44.5	45.5	68.0
GPT-3 0-shot	14.6	14.4	64.3
GPT-3 1-shot	23.0	25.3	68.0
GPT-3 few-shot	29.9	41.5	71.2

4.3 Dịch máy (BLEU, few-shot dùng 64 ví dụ theo paper)

	En→Fr	Fr→En	En→De	De→En	En→Ro	Ro→En
GPT-3 0-shot	25.2	21.2	24.6	27.2	14.1	19.9
GPT-3 1-shot	28.3	33.7	26.2	30.4	20.6	38.6
GPT-3 few-shot	32.6	39.2	29.7	40.6	21.0	39.5

Dịch *sang* tiếng Anh tốt hơn nhiều so với dịch *từ* tiếng Anh (dữ liệu 93% tiếng Anh). Paper nghi tokenizer BPE của GPT-2 (thiết kế cho tiếng Anh) là một nguyên nhân gây En→Ro yếu.

4.4 Winograd và SuperGLUE

	Winograd	Winogrande (XL)
Fine-tuned SOTA	90.1	84.6
GPT-3 0-shot / 1-shot / few-shot	88.3* / 89.7* / 88.6*	70.2 / 73.2 / 77.7

(* = có nghi ngờ nhiễm dữ liệu.)

SuperGLUE (few-shot, 32 ví dụ/tác vụ, test set): trung bình **71.8** so với 69.0 của BERT-Large fine-tune và 89.0 của SOTA fine-tune. Điểm mạnh: COPA 92.0, ReCoRD 90.2/91.1, WSC 80.1. Điểm yếu: **WiC 49.4% (ngang đoán bừa)**, RTE 69.0, CB F1 52.0, MultiRC acc 30.5. Paper nhận xét: GPT-3 yếu ở các tác vụ "so sánh hai đoạn văn" (WiC, ANLI, RTE).

4.5 Tác vụ tổng hợp: số học (Bảng 3.9, 175B, độ chính xác %)

	2D+	2D-	3D+	3D-	4D+	4D-	5D+	5D-	2Dx	1DC
0-shot	76.9	58.0	34.2	48.3	4.0	7.5	0.7	0.8	19.8	9.8
1-shot	99.6	86.4	65.5	78.7	14.0	14.0	3.5	3.8	27.4	14.3
few-shot	100.0	98.9	80.4	94.2	25.5	26.8	9.3	9.9	29.2	21.3

(Văn bản của paper ghi 3D+ few-shot là 80.2%, bảng ghi 80.4%; ghi chú dùng số trong bảng.) Kiểm tra ghi nhớ: trong 2000 bài cộng 3 chữ số chỉ có 17 bài (0.8%) xuất hiện nguyên văn trong dữ liệu train; lỗi thường gặp là "quên nhớ 1" → mô hình có vẻ đang *tính*, không chỉ thuộc lòng. Mô hình 13B chỉ giải được cộng/trừ 2 chữ số khoảng một nửa số lần; nhảy vọt xảy ra ở 175B.

Các tác vụ khác: xáo trộn chữ (cycle letters: 37.9% few-shot; anagram bỏ chữ đầu-cuối: 15.1%; chèn ngẫu nhiên: 67.2%), dùng từ mới sau khi chỉ thấy định nghĩa một lần, sửa ngữ pháp tiếng Anh (định tính).

4.6 Sinh bài báo và khả năng phân biệt của con người

Điều kiện: 3 bài báo thật làm few-shot context + tiêu đề/phụ đề mới. Người tham gia (~80 người Mỹ mỗi mô hình) phân biệt bài người viết và bài máy viết:

Mô hình	Độ chính xác của người
Control (mô hình cố tình tệ)	86%
GPT-3 Small	76%
GPT-3 13B	55%
GPT-3 175B	52% (gần đoán bừa 50%)

Bài ~500 từ: 88% (control) so với 52% (GPT-3 175B).

---

5. Phân tích / ablation

(a) Scaling mượt theo compute. Cross-entropy validation loss giảm theo luật lũy thừa theo compute (Hình 3.1), kéo dài thêm 2 bậc độ lớn so với Kaplan et al. mà chỉ lệch nhỏ khỏi đường dự đoán. Và loss thấp hơn chuyển thành điểm các tác vụ downstream tốt hơn một cách nhất quán.

(b) Khoảng cách 0/1/few-shot tăng theo kích thước (Hình 1.3, 3.8). Trên 42 benchmark tính tổng hợp, zero-shot tăng đều theo kích thước, few-shot tăng nhanh hơn. SuperGLUE: GPT-3 cần chưa đến 8 ví dụ mỗi tác vụ để vượt BERT-Large fine-tune, và chênh lệch giữa BERT-Large và BERT++ gần tương đương chênh lệch giữa 1 ví dụ và 8 ví dụ trong ngữ cảnh. Đây là bằng chứng chính cho tuyên bố "mô hình lớn là meta-learner tốt hơn".

(c) Mô tả bằng ngôn ngữ tự nhiên + số lượng ví dụ K đều giúp (Hình 1.2), và độ dốc "đường cong học trong ngữ cảnh" lớn hơn ở mô hình lớn (tác vụ thử: xóa ký hiệu thừa khỏi từ).

(d) Ô nhiễm benchmark (mục 4). Do một lỗi trong bước lọc, một phần dữ liệu test lọt vào tập train mà không thể train lại. Paper tạo phiên bản "sạch" của mỗi benchmark (loại mẫu có overlap 13-gram) và so điểm:

- Hơn 1/4 benchmark có tỉ lệ nghi nhiễm trên 50%, nhưng *điểm hầu như không đổi* và không thấy tương quan giữa mức nhiễm và chênh lệch điểm.
- Chỉ PIQA và Winograd có bằng chứng nhiễm đáng kể → đánh dấu sao (*) trong kết quả.
- Đọc hiểu (QuAC, SQuAD2, DROP): >90% mẫu bị gắn cờ nhưng khi kiểm tay chỉ văn bản nguồn xuất hiện, không có cặp hỏi-đáp.
- Training/validation gap nhỏ (Hình 4.1) → GPT-3 không overfit đáng kể dù lớn.

(e) In-context learning thật sự "học" hay chỉ "nhận ra" tác vụ? Paper thừa nhận là câu hỏi mở: dịch máy chắc chắn học trong pre-train; các tác vụ tổng hợp (xáo chữ, từ vô nghĩa) có vẻ gần với "học mới hoàn toàn" hơn.

Paper **không** làm ablation kiến trúc (không thử bỏ sparse attention, không so sánh biến thể), vì kiến trúc coi như cố định và biến chính là kích thước.

---

6. Giới hạn

Do tác giả nêu:

- Sinh văn bản dài vẫn lặp ý, mất mạch lạc, tự mâu thuẫn; yếu về "vật lý thường thức" ("bỏ phô mai vào tủ lạnh thì có tan không?").

- Gần ngang đoán bừa ở các tác vụ so sánh (WiC, ANLI) và một số đọc hiểu (QuAC, RACE). Giả thuyết: do mô hình tự hồi quy một chiều (không bidirectional) không phù hợp với việc "xem lại, so sánh hai đoạn".
- Mục tiêu pre-train coi mọi token như nhau, không có khái niệm "token nào quan trọng"; không có grounding (hình ảnh, tương tác vật lý).
- Hiệu quả mẫu khi pre-train kém (thấy nhiều văn bản hơn một đời người).
- Chưa rõ few-shot là học mới hay nhận dạng tác vụ đã gặp.
- **Suy luận đắt và bất tiện ở quy mô này**; paper tự gợi ý *distillation* ở quy mô trăm tỷ tham số là hướng nghiên cứu.
- Thiên kiến (giới tính, chủng tộc, tôn giáo), khả năng lạm dụng sinh văn bản giả (mục 6), tiêu thụ năng lượng: chi phí train là hàng nghìn PF-day, nhưng sinh 100 trang văn bản chỉ tốn khoảng 0.4 kW-hr (theo paper).
- Lỗi lọc ô nhiễm không sửa được.

Nhận xét bổ sung:

- Cửa sổ ngữ cảnh 2048 token giới hạn số ví dụ in-context ($K \approx$ 10-100) và độ dài tài liệu.
- Mô hình 175B với KV-cache lớn (xem 8.1) khiến chi phí in-context learning ở lúc suy luận không hề nhỏ: mỗi ví dụ thêm vào prompt là token phải prefill và lưu cache.
- Chưa có so sánh trực tiếp "few-shot vs fine-tune cùng mô hình" vì paper không fine-tune GPT-3.
- Kích thước tối ưu theo quan điểm sau này (Chinchilla, bài 04): 300B token cho 175B tham số là rất ít (khoảng 1.7 token/tham số, tự tính).

---

7. Liên hệ với các bài khác

- **01 Attention Is All You Need:** GPT-3 dùng chỉ phần decoder, bỏ cross-attention, chồng 96 layer.
- **03 Kaplan scaling laws:** cơ sở lý thuyết cho việc "phình mô hình, giữ token ở 300B"; GPT-3 xác nhận luật lũy thừa thêm 2 bậc độ lớn compute.
- **04 Chinchilla:** chứng minh GPT-3 train thiếu dữ liệu; mô hình nhỏ hơn mà nhiều token hơn thì tốt hơn cho cùng compute.
- **05 LLaMA:** trực tiếp tuyên bố LLaMA-13B vượt GPT-3 175B trên hầu hết benchmark dù nhỏ hơn 10 lần.
- **07 InstructGPT:** GPT-3 prompt thế nào cũng phải "thiết kế prompt"; RLHF làm mô hình làm theo chỉ dẫn trực tiếp.
- **09 Pre-LN:** lý do kỹ thuật cho pre-normalization.
- **17 Longformer / 06 Mistral:** attention dải băng cục bộ; GPT-3 xen kẽ dày/thưa, Mistral dùng sliding window ở *mọi* layer.
- **48 Speculative decoding, 46 PagedAttention:** các kỹ thuật phục vụ mô hình cỡ GPT-3 ra đời để giải bài toán chi phí suy luận mà paper nêu ở mục Giới hạn.
- **08 LoRA:** thay thế rẻ cho full fine-tune mà GPT-3 muốn tránh (LoRA nhắm đúng mô hình cỡ GPT-3 175B).

---

8. Gợi ý áp dụng

GPT-3 minh họa sự khác biệt giữa thay đổi prompt và cập nhật trọng số. Khi đánh giá in-context learning, cần giữ cố định dữ liệu, mẫu prompt và cách chọn ví dụ để có thể so sánh zero-shot với few-shot. Bài báo cũng so sánh kết quả trên toàn bộ tập kiểm tra với tập đã loại các mẫu nghi nhiễm dữ liệu huấn luyện; phép

kiểm tra này giúp đánh giá mức độ đáng tin của điểm benchmark, dù không bảo đảm phát hiện mọi trường hợp trùng lặp.

Chi phí cũng tăng cùng quy mô. Với GPT-3 175B, giả sử FP16, 96 tầng và chiều 12.288, KV cache cho 2.048 token chiếm khoảng 9,66 GB theo công thức $2Ldnb$ (tự tính); trọng số FP16 khoảng 350 GB. Các con số chưa tính bộ nhớ phụ và cách phân bố trên thiết bị.

9. Thí nghiệm gợi ý

1. So sánh zero-shot, one-shot và few-shot trên cùng tập kiểm tra; thay đổi thứ tự và cách chọn ví dụ để đo độ nhạy của kết quả với prompt.
2. Đo mức trùng n-gram giữa tập kiểm tra và tập huấn luyện, sau đó báo cáo điểm trên toàn bộ tập và trên phần không phát hiện trùng. Phân tích thêm các trường hợp trùng để tránh coi mọi trùng lặp là ô nhiễm thực sự.

10. Tóm tắt công thức / khái niệm

Khái niệm	Ý nghĩa ngắn gọn
In-context learning	Học tác vụ từ prompt trong một forward pass, không cập nhật trọng số
Zero/One/Few-shot	0 / 1 / K (10-100) ví dụ trong ngữ cảnh + mô tả tác vụ
Fine-tuning (FT)	Cập nhật trọng số trên tập có nhãn cho từng tác vụ (không dùng cho GPT-3)
$n_{ctx} = 2048$	Độ dài ngữ cảnh tối đa, giới hạn K
$d_{ff} = 4d_{model}$	Kích thước hidden của FFN
Alternating dense/banded sparse attention	Xen kẽ attention toàn cục và attention dải băng cục bộ giữa các layer
$N \approx 12Ld^2$	Số tham số xấp xỉ của Transformer decoder (tự tính)
$\boxed{C \approx 6ND}$	FLOPs huấn luyện: 2 forward + 4 backward mỗi tham số mỗi token
3.64E+03 PF-day	Compute huấn luyện GPT-3 175B (Bảng D.1)
300B token	Số token train mọi kích thước GPT-3
Data contamination	Dữ liệu test lọt vào train; kiểm bằng 13-gram overlap và so điểm clean/all
Gradient noise scale	Số đo dùng để chọn batch size (mô hình lớn: batch lớn, LR nhỏ)

---

11. Kết luận ngắn

GPT-3 không phát minh kiến trúc mới; đóng góp là bằng chứng thực nghiệm rằng *quy mô + mục tiêu đoán token tiếp theo* đủ cho một mô hình làm được rất nhiều tác vụ chỉ qua prompt, và khả năng học từ ví dụ trong ngữ cảnh tăng theo kích thước. Cái giá là chi phí khổng lồ (3640 PF-day train, hàng trăm GB trọng số) và những điểm yếu rõ rệt (so sánh câu, lặp ý). Chi phí này thúc đẩy nghiên cứu các mô hình nhỏ hơn được huấn luyện trên nhiều token hơn, cũng như các phương pháp nén mô hình.

Nguồn gốc: https://arxiv.org/abs/2005.14165

Bài 03

Scaling Laws for Neural Language Models (2020)

Tác giả: Jared Kaplan, Sam McCandlish, Tom Henighan, Tom B. Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, Dario Amodei (Johns Hopkins University, OpenAI).
Venue / arXiv: arXiv:2001.08361 (v1, 23/01/2020); paper không nêu venue hội nghị.
Nguồn: https://arxiv.org/abs/2001.08361
Chủ đề: quy luật luỹ thừa giữa loss của Transformer LM và số tham số N, lượng dữ liệu D, compute C; overfitting; batch size tới hạn; phân bổ compute tối ưu.
Vị trí trong lộ trình: đọc sau 01 (kiến trúc) và 02 (GPT-3, ứng dụng trực tiếp tư tưởng mô hình lớn). Bài này đặt ra khung "loss là hàm luỹ thừa của quy mô". Tiếp theo là 04 (Chinchilla), sửa lại kết luận về phân bổ mô hình và dữ liệu.

> Ghi chú diễn giải bằng tiếng Việt, không phải bản dịch. Các số liệu "tự tính" là ví dụ minh họa, không phải kết quả trực tiếp của bài báo.

1. Bối cảnh

Đến 2019, các mô hình ngôn ngữ ngày càng lớn (GPT-2, BERT, ...) nhưng quyết định "huấn luyện model cỡ nào, bao nhiêu dữ liệu, bao lâu" chủ yếu dựa trên trực giác. Mỗi lần huấn luyện một mô hình lớn rất đắt, nên cần **dự đoán được** kết quả từ các lần chạy nhỏ.

Khái niệm cần nhớ (xem thêm bài 00):

- **Loss** ở đây là cross-entropy L (nats/token) trên dữ liệu kiểm tra, trung bình trên ngữ cảnh 1024 token.
- **Luật luỹ thừa (power law)** $y = (x_c/x)^{\alpha}$: vẽ trên log-log là đường thẳng với độ dốc $-\alpha$; nhân x lên k lần thì y nhân với $k^{-\alpha}$.
- **Compute** C đo bằng PF-days: 1 PF-day $= 10^{15} \times 24 \times 3600 = 8.64 \times 10^{19}$ FLOPs.

Câu hỏi: loss phụ thuộc thế nào vào kiến trúc, kích thước mô hình, kích thước dữ liệu, và compute? Có thể phân bổ ngân sách compute cố định một cách tối ưu không?

2. Ý tưởng chính

Tác giả huấn luyện rất nhiều Transformer decoder-only ở nhiều kích thước (768 đến 1.5 tỷ tham số không tính embedding), nhiều kích thước dữ liệu (22 triệu đến 23 tỷ token), và thấy loss giảm theo **luật luỹ thừa mượt** theo từng yếu tố (khi hai yếu tố kia không là nút thắt), kéo dài hơn 6 đến 7 bậc độ lớn. Hình dạng mô hình (sâu hay rộng, số head) ít quan trọng; **quy mô** mới quan trọng. Từ các luật này họ suy ra: với compute cố định, nên **tăng kích thước mô hình thật nhanh, tăng dữ liệu rất chậm, và dừng huấn luyện rất sớm (xa hội tụ)**. Kết luận "big models may be more important than big data" ảnh hưởng mạnh đến GPT-3 (và bị Chinchilla, bài 04, sửa lại).

3. Phương pháp chi tiết

3.1 Thiết lập

- Dữ liệu: **WebText2** (mở rộng WebText: link ra từ Reddit đến 10/2018, tối thiểu 3 karma): 20.3M tài liệu, 96 GB, 1.62×10^{10} từ, 2.29×10^{10} token; giữ 6.6×10^{8} token làm tập kiểm tra. Cũng kiểm tra trên Books Corpus, Common Crawl, English Wikipedia, Internet Books.
- Tokenizer: BPE, $n_{vocab} = 50257$. Ngữ cảnh $n_{ctx} = 1024$ (đa số).
- Mô hình: Transformer decoder-only; ngoài ra có LSTM và Universal Transformer để so.

- Huấn luyện: Adam, 2.5×10^5 bước, batch 512 chuỗi x 1024 token ($\approx 2^{19}$ token); các mô hình >1B dùng Adafactor; lr: warmup 3000 bước rồi cosine về 0. Dropout 10% khi nghiên cứu overfitting, kèm early stopping.

3.2 Tham số và FLOPs (Mục 2.1, Bảng 1)

Quan trọng: N trong paper là số tham số **không tính embedding** (không có $n_{vocab}d_{model}$ và vị trí). Với $d_{attn} = d_{ff}/4 = d_{model}$:

$$\boxed{N \approx 2\, d_{model}\, n_{layer}\, (2d_{attn} + d_{ff}) = 12\, n_{layer}\, d_{model}^2}$$

FLOPs forward mỗi token (cộng nhân = 2 phép):

$$C_{forward} \approx 2N + 2\, n_{layer}\, n_{ctx}\, d_{model}.$$

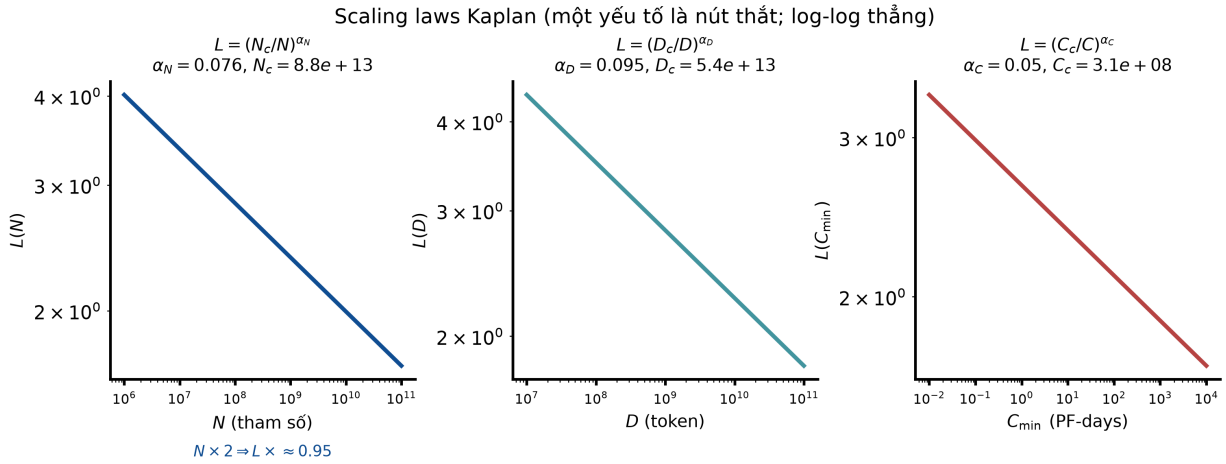
Khi $d_{model} > n_{ctx}/12$, thành phần phụ thuộc ngữ cảnh nhỏ nên bỏ. Cộng backward (~2 lần forward):

$$\boxed{C \approx 6N\, B\, S = 6ND \;\; (\text{FLOPs trên } D = BS \text{ token})}$$

Phép toán	Tham số	FLOPs mỗi token (Bảng 1)
Embed	$(n_{vocab} + n_{ctx})d_{model}$	$4d_{model}$
Attention QKV	$n_{layer}d_{model}3d_{attn}$	$2n_{layer}d_{model}3d_{attn}$
Attention mask	không	$2n_{layer}n_{ctx}d_{attn}$
Attention project	$n_{layer}d_{attn}d_{model}$	$2n_{layer}d_{attn}d_{embd}$
Feed-forward	$n_{layer}2d_{model}d_{ff}$	$2n_{layer}2d_{model}d_{ff}$
De-embed	không	$2d_{model}n_{vocab}$
Tổng (không embedding)	$N = 2d_{model}n_{layer}(2d_{attn} + d_{ff})$	$C_{forward} = 2N + 2n_{layer}n_{ctx}d_{attn}$

3.3 Các định luật (Mục 1.2 và Phụ lục A)

Ba luật cơ bản khi chỉ một yếu tố là nút thắt:



Ba luật lũy thừa Kaplan $L(N)$, $L(D)$, $L(C_{min})$ trên trục log-log với $\alpha_N = 0.076$, $\alpha_D = 0.095$, $\alpha_C = 0.050$ và các hằng N_c, D_c, C_c lấy từ note. (minh hoạ, tự tính)

$$L(N) = \left(\frac{N_c}{N}\right)^{\alpha_N},\ \alpha_N \approx 0.076,\ N_c \approx 8.8 \times 10^{13};$$

$$L(D) = \left(\frac{D_c}{D}\right)^{\alpha_D},\ \alpha_D \approx 0.095,\ D_c \approx 5.4 \times 10^{13} \text{ token};$$

$$L(C_{min}) = \left(\frac{C_c^{min}}{C_{min}}\right)^{\alpha_C^{min}},\ \alpha_C^{min} \approx 0.050,\ C_c^{min} \approx 3.1 \times 10^8 \text{ PF-days}.$$

Điều kiện: $L(N)$ là mô hình hội tụ trên dữ liệu đủ lớn; $L(D)$ là mô hình đủ lớn với early stopping; $L(C_{min})$ là compute dùng tối ưu, dữ liệu đủ, mô hình kích thước tối ưu và batch nhỏ. Doubling N làm loss nhân với $2^{-0.076} \approx 0.95$ (paper ghi 0.95; tự tính 0.9487). Các giá trị N_c, D_c, C_c phụ thuộc tokenizer nên không có ý nghĩa tuyệt đối.

Hợp nhất N và D (phương trình 1.5/4.1):

$$\boxed{L(N, D) = \left[\left(\frac{N_c}{N}\right)^{\alpha_N/\alpha_D} + \frac{D_c}{D}\right]^{\alpha_D}}$$

Ba nguyên tắc chọn dạng này: (1) đổi từ vựng/tokenization chỉ nhân loss với hệ số, có thể hấp thụ vào N_c, D_c; (2) $D \to \infty$ về $L(N)$ và $N \to \infty$ về $L(D)$; (3) giải tích theo $1/D$ (khai triển nguyên). Fit 4 tham số (Bảng 2): $\alpha_N = 0.076$, $\alpha_D = 0.103$, $N_c = 6.4 \times 10^{13}$, $D_c = 1.8 \times 10^{13}$ (hơi khác các giá trị riêng lẻ ở trên vì fit đồng thời).

Mức overfitting $\delta L = L(N, D)/L(N, \infty) - 1$ chỉ phụ thuộc tổ hợp $N^{\alpha_N/\alpha_D}/D$ (Hình 9). Để overfitting nằm trong ngưỡng nhiễu seed (~0.02):

$$\boxed{D \gtrsim (5 \times 10^3)\, N^{0.74}}$$

tức mỗi lần tăng N lên 8 lần chỉ cần tăng D khoảng 5 lần (tự tính: $8^{0.74} \approx 4.7$).

Loss theo N và số bước (dữ liệu vô hạn):

$$L(N, S_{min}) = \left(\frac{N_c}{N}\right)^{\alpha_N} + \left(\frac{S_c}{S_{min}}\right)^{\alpha_S}, \quad \alpha_N = 0.077,\ \alpha_S = 0.76,\ N_c = 6.5 \times 10^{13},\ S_c = 2.1 \times 10^3.$$

3.4 Batch size tới hạn (Mục 5.1)

Critical batch size B_{crit} (McCandlish et al. 2018): batch lớn tới mức tăng thêm cho lợi ích giảm dần. Mô hình bước/mẫu: $(S/S_{min} - 1)(E/E_{min} - 1) = 1$, $B_{crit} = E_{min}/S_{min}$. Nó chỉ phụ thuộc vào loss mục tiêu:

$$B_{crit}(L) = \frac{B_*}{L^{1/\alpha_B}}, \quad B_* \approx 2 \times 10^8 \text{ token},\ \alpha_B \approx 0.21.$$

Huấn luyện tại B_{crit} tốn $2S_{min}$ bước và $2E_{min}$ mẫu. Các đại lượng hiệu chỉnh:

$$S_{min}(S) = \frac{S}{1 + B_{crit}(L)/B}, \qquad C_{min}(C) = \frac{C}{1 + B/B_{crit}(L)}.$$

Paper cho: B_{crit} khoảng 1 đến 2 triệu token tại hội tụ ở mô hình lớn nhất, và gấp đôi khi loss giảm khoảng 13%.

3.5 Phân bổ compute tối ưu (Mục 6 và Phụ lục B)

Thay $S = C/(6NB)$ vào $L(N, S)$ và tối ưu theo N với C cố định (đặt $\partial L/\partial N|_C = 0$). Kết quả đóng:

$$\alpha_C^{min} = \frac{1}{1/\alpha_S + 1/\alpha_B + 1/\alpha_N}, \qquad N \propto C^{\alpha_C^{min}/\alpha_N},\ B \propto C^{\alpha_C^{min}/\alpha_B},\ S \propto C^{\alpha_C^{min}/\alpha_S},\ D = B \cdot S.$$

Thay giá trị: $\alpha_C^{min} = 1/(1.316 + 4.762 + 13.158) \approx 0.052$ (tự tính). Thực nghiệm (Bảng 6):

$$\boxed{N_{opt} \propto C_{min}^{0.73}, \quad B \propto C_{min}^{0.24}, \quad S_{min} \propto C_{min}^{0.03}, \quad D_{opt} \propto C_{min}^{0.27}}$$

với $N_e = 1.3 \times 10^9$, $B_e = 2.0 \times 10^6$ token, $S_e = 5.4 \times 10^3$, $D_e = 2 \times 10^{10}$ token. **Tăng compute 10 lần** thì: N tăng $10^{0.73} \approx 5.4\times$, batch $\approx 1.7\times$, số bước gần như không đổi ($10^{0.03} \approx 1.07$), dữ liệu $\approx 1.9\times$ (tự tính). Paper nói hầu hết phần tăng đi vào mô hình lớn hơn, phần tăng dữ liệu chủ yếu dùng cho batch lớn hơn (song song).

Hệ quả "dừng sớm": huấn luyện tối ưu compute dừng khi loss còn cao hơn loss hội tụ khoảng $\alpha_N/\alpha_S \approx 10\%$ (phương trình B.5). So với huấn luyện kiểu "gần hội tụ" ($f' = 2\%$), huấn luyện hiệu quả dùng 2.7 lần tham số, 7.7 lần ít bước, 65% ít compute hơn để đạt cùng loss (B.3).

Mô hình không tối ưu (B.4): mô hình có kích thước từ 0.6 đến 2.2 lần kích thước tối ưu chỉ tốn thêm khoảng 20% compute; mô hình lớn hơn 2.2 lần cần ít hơn 45% số bước, tốn 20% thêm compute. Paper chỉ ra rằng chọn mô hình nhỏ hơn có ích khi tính chi phí suy luận.

Pseudo-code dự báo cho một ngân sách:

```
def plan(C_pfdays):                          # C_min, đã điều chỉnh về B << B_crit
    N = 1.3e9 * C_pfdays**0.73               # tham số không embedding
    B = 2.0e6 * C_pfdays**0.24               # token mỗi batch
    S = 5.4e3 * C_pfdays**0.03               # số bước (cận dưới)
    D_epoch1 = 2e10 * C_pfdays**0.27         # token nếu 1 epoch
    L = (3.1e8 / C_pfdays)**0.050            # loss dự báo (nats/token, WebText2, BPE
    ↪  GPT-2)
    return N, B, S, D_epoch1, L
```

4. Thí nghiệm và kết quả chính

Phạm vi: hơn tám bậc độ lớn C_{min}, sáu bậc N, hơn hai bậc D. Các điểm chính (copy từ paper):

Phát hiện	Nội dung
Hình dạng ít quan trọng	Với N cố định, loss chỉ dao động vài % trên dải lớn của tỉ lệ d_{ff}/d_{model}, d_{model}/n_{layer} (aspect ratio), d_{model}/n_{head}. Aspect ratio thay đổi tới 40 lần ảnh hưởng nhẹ; $(n_{layer}, d_{model}) = (6, 4288)$ đạt loss trong vòng 3% so với $(48, 1600)$.
Chỉ tính non-embedding	Gộp embedding vào N làm xu hướng bị nhiễu theo số tầng; loại embedding thì mọi độ sâu hội tụ về một đường. Hàm ý: ma trận embedding có thể nhỏ hơn mà không hại.
Transfer	Loss trên phân phối khác (Books, Wikipedia, Common Crawl, Internet Books) tăng song song với loss WebText2, chênh lệch gần hằng số; chỉ phụ thuộc loss trên phân phối huấn luyện, không phụ thuộc giai đoạn huấn luyện hay độ sâu.
Sample efficiency	Mô hình lớn đạt cùng loss với ít bước và ít mẫu hơn.

Phát hiện	Nội dung
Hợp nhất	$L(N, D)$ khớp tốt trừ khi D giảm 1024 lần (~2×10^7 token; một epoch chỉ 40 bước cập nhật), khi đó fit kém.
Phạm vi N	768 đến 1.5 tỷ tham số không embedding; D: 22 triệu đến 23 tỷ token.
Mô hình lớn gặp overfit nhẹ	Với 22 tỷ token WebText2 thì mô hình $< 10^9$ tham số hầu như không overfit; mô hình lớn nhất gặp overfit nhẹ.

Bảng tham số fit (Phụ lục A, Bảng 5 và 6):

Hằng số	Giá trị
α_N, N_c	0.076, 8.8×10^{13}
α_D, D_c	0.095, 5.4×10^{13}
α_C (naive, batch cố định), C_c	0.057, 1.6×10^7 PF-days
α_C^{min}, C_c^{min}	0.050, 3.1×10^8 PF-days
α_B, B_*	0.21, 2.1×10^8 token
α_S, S_c	0.76, 2.1×10^3 bước

Ví dụ tự tính: Theo $L(N)$ trong Phụ lục A, tăng số tham số không tính embedding từ 10^8 lên 10^9 cho tỉ lệ loss dự đoán $10^{-0.076} \approx 0.84$. Đây là phép nội suy cho WebText2 và tokenizer được dùng trong nghiên cứu, không phải dự báo phổ quát cho mọi tập dữ liệu.

5. Phân tích / ablation

- **LSTM so với Transformer (Hình 7):** LSTM ngang Transformer ở các token đầu ngữ cảnh nhưng **không** bắt kịp ở token sau (LSTM bão hoà sau dưới 100 token); Transformer cải thiện suốt cả ngữ cảnh. Phù hợp với việc attention tận dụng ngữ cảnh dài tốt hơn.
- **Universal Transformer (chia sẻ tham số):** tốt hơn một chút theo N nhưng kém hơn một chút theo compute (tái sử dụng tham số).
- **Batch tới hạn B_{crit}:** không phụ thuộc trực tiếp vào kích thước mô hình, chỉ qua loss đạt được; khớp với ước lượng gradient noise scale.
- **Độ dốc ngữ cảnh:** theo vị trí token, loss cũng theo luỹ thừa (Phụ lục D.5), mô hình lớn có số mũ lớn hơn, gợi ý nhận ra mẫu nhanh hơn.
- **Hessian:** luỹ thừa theo S_{min} có thể phản ánh phổ Hessian gần như độc lập với kích thước mô hình (suy đoán của tác giả).

Mâu thuẫn và giả thuyết (Mục 6.3). Ngoại suy $L(C_{min})$ với dữ liệu chỉ tăng $D \propto C^{0.27}$ (một epoch) cắt $L(D)$ khi $C^* \sim 10^4$ PF-days, $N^* \sim 10^{12}$, $D^* \sim 10^{12}$ token, $L^* \sim 1.7$ nats/token (rất bất định, thay đổi cỡ một bậc độ lớn theo số mũ). Tác giả đoán nó có thể là ước lượng entropy của ngôn ngữ, chỉ là giả thuyết.

6. Giới hạn

Do tác giả nêu (Phụ lục C):

- Chưa có lý thuyết giải thích các luật; “not solid theoretical understanding”.
- Không tự tin về $B_{crit}(L)$ khi loss ra ngoài dải đã thử.
- Chưa kiểm kỹ vùng dữ liệu nhỏ (fit $L(N, D)$ kém cho D nhỏ nhất); chưa thử regularization/data augmentation.
- $C \approx 6NBS$ bỏ qua phần phụ thuộc n_{ctx} nên sai lệch với ngữ cảnh rất dài ($n_{ctx} \gtrsim 12 d_{model}$).
- Có thể chưa tinh chỉnh hết siêu tham số (khởi tạo, momentum); lr tối ưu phụ thuộc loss mục tiêu.

- Thấy mâu thuẫn ở quy mô cực lớn (mục 6.3 trên).

Nhận xét bổ sung:

- Mọi mô hình dùng một lịch cosine cố định (độ dài ~2.5×10^5 bước) cho tất cả; bài 04 chỉ ra điều này làm đánh giá thấp mô hình huấn luyện ngắn, và ảnh hưởng đến kết luận về phân bổ N so với D.
- Đa số mô hình trong thí nghiệm nhỏ (<100M tham số) nên ngoại suy lên cỡ hàng chục tỷ có độ lệch (Chinchilla thấy đường cong biên hơi cong).
- Chỉ trên văn bản tự nhiên, từ vựng GPT-2; không áp dụng trực tiếp cho OCR hay đa phương thức.
- Loại embedding khỏi N đẹp cho phân tích, nhưng với một số mô hình nhỏ embedding có thể chiếm tỉ lệ lớn nên luật này kém hữu dụng nếu không điều chỉnh (suy luận; xem mục 8).
- Số liệu có vài chỗ lệch nhỏ giữa các phần: α_D là 0.095 (tóm tắt) so với 0.103 (fit chung); α_C^{min} là 0.050 (thực nghiệm), 0.054 (6.4), 0.052 (B.7); $N \propto C^{0.73}$ (thực nghiệm) so với $C^{0.71}$ (dự báo).

7. Liên hệ với các bài khác

- **01** Attention Is All You Need: kiến trúc được khảo sát; công thức $N = 12\, n_{layer} d^2$ lấy từ cấu trúc ở đó.
- **02** GPT-3: áp dụng trực tiếp tinh thần bài này (model rất lớn, ~300B token).
- **04** Chinchilla: kiểm lại cùng câu hỏi, kết luận ngược về hệ số ($N \propto C^{0.5}$, $D \propto C^{0.5}$ so với $0.73/0.27$ ở đây).
- **05** LLaMA, **06** Mistral: huấn luyện nhỏ hơn nhưng nhiều dữ liệu hơn (đi ngược gợi ý bài này vì chi phí suy luận).
- **08** LoRA, **48 đến 50**: giảm chi phí triển khai thay vì chi phí huấn luyện, hướng bài này không xét.
- **27, 28** MoE: thay đổi quan hệ giữa N (tổng) và FLOPs mỗi token (bài này giả định dense, $C \propto N$).

8. Gợi ý áp dụng

Các luật này được khớp trên mô hình huấn luyện từ đầu và dữ liệu văn bản cụ thể. Chúng là công cụ dự báo trong miền đã khảo sát, không bảo đảm dự báo chất lượng sau cắt tỉa hoặc chưng cất. Khi đối chiếu quy mô, cần chú ý N của bài báo không tính embedding; phần này vẫn tiêu tốn bộ nhớ và có thể chiếm tỉ lệ lớn trong mô hình nhỏ.

Ước lượng $C \approx 6ND$ giúp lập ngân sách huấn luyện sơ bộ. Công thức bỏ qua chi phí không thuộc forward và backward của mô hình đang xét; ví dụ, chưng cất còn cần tính thêm lượt forward của teacher. Kết luận về phân bổ N và D cần đọc cùng Chinchilla (bài 04).

9. Thí nghiệm gợi ý

1. Huấn luyện một họ mô hình với nhiều kích thước trên cùng phân phối dữ liệu; vẽ loss kiểm tra theo số tham số không tính embedding trên thang log–log và so sánh độ dốc trong miền quan sát với luật của bài báo.
2. Với số tham số gần nhau, so sánh mô hình sâu và hẹp với mô hình nông và rộng; báo cáo cả loss và thời gian suy luận. Sự tương đồng về loss không đồng nghĩa với chi phí triển khai giống nhau.

10. Tóm tắt công thức / khái niệm

Khái niệm	Công thức / giá trị
Tham số	$N \approx 12\, n_{layer} d_{model}^2$ (không embedding)
Compute huấn luyện	$C \approx 6ND$; 1 PF-day $= 8.64 \times 10^{19}$ FLOPs
$L(N)$	$(N_c/N)^{0.076}$, $N_c = 8.8 \times 10^{13}$

Khái niệm	Công thức / giá trị
$L(D)$	$(D_c/D)^{0.095}$, $D_c = 5.4 \times 10^{13}$
$L(C_{min})$	$(C_c^{min}/C_{min})^{0.050}$
$L(N, D)$	$[(N_c/N)^{\alpha_N/\alpha_D} + D_c/D]^{\alpha_D}$
Chống overfit	$D \gtrsim 5 \times 10^3 N^{0.74}$
B_{crit}	$B_*/L^{1/\alpha_B}$, $B_* = 2.1 \times 10^8$, $\alpha_B = 0.21$
Phân bổ tối ưu	$N \propto C^{0.73}$, $B \propto C^{0.24}$, $S \propto C^{0.03}$, $D \propto C^{0.27}$
Dừng sớm	huấn luyện hiệu quả dừng ~10% trên loss hội tụ
Mô hình lệch kích thước	0.6 đến 2.2 lần kích thước tối ưu: +20% compute

11. Kết luận ngắn

Loss của Transformer LM tuân theo luật luỹ thừa mượt theo N, D, C với rất ít phụ thuộc vào hình dạng; từ đó suy ra phân bổ compute nghiêng mạnh về mô hình lớn ($N \propto C^{0.73}$) và dừng sớm. Kết luận về tỉ lệ model/data sau này bị bài 04 sửa; phần còn bền gồm công thức $C \approx 6ND$, $N \approx 12Ld^2$, tư duy dự báo bằng luật luỹ thừa và việc tách embedding khỏi N.

Nguồn gốc: https://arxiv.org/abs/2001.08361

Bài 04

Training Compute-Optimal Large Language Models (2022)

Tác giả: Jordan Hoffmann, Sebastian Borgeaud, Arthur Mensch, Elena Buchatskaya, Trevor Cai, Eliza Rutherford, Diego de Las Casas, Lisa Anne Hendricks, Johannes Welbl, Aidan Clark, Tom Hennigan, Eric Noland, Katie Millican, George van den Driessche, Bogdan Damoc, Aurelia Guy, Simon Osindero, Karen Simonyan, Erich Elsen, Jack W. Rae, Oriol Vinyals, Laurent Sifre (DeepMind). Ba tác giả đầu và Sifre đóng góp ngang nhau.
Venue / arXiv: arXiv:2203.15556 (v1, 29/03/2022); paper không nêu venue hội nghị.
Nguồn: https://arxiv.org/abs/2203.15556
Chủ đề: phân bổ tối ưu ngân sách compute giữa kích thước mô hình N và số token huấn luyện D; mô hình Chinchilla 70B.
Vị trí trong lộ trình: đọc sau 03 (Kaplan), vì bài này phản biện trực tiếp. Bổ sung: kết luận mới "model và data tăng cùng tỉ lệ", cùng bằng chứng thực nghiệm. Tiếp theo: 05 (LLaMA) và 06 (Mistral), các mô hình nhỏ huấn luyện nhiều token, và các bài về suy luận hiệu quả.

> Ghi chú diễn giải bằng tiếng Việt, không phải bản dịch. Các số liệu "tự tính" là ví dụ minh họa, không phải kết quả trực tiếp của bài báo.

1. Bối cảnh

Bài 03 (Kaplan) kết luận rằng khi tăng compute 10 lần thì nên tăng kích thước mô hình 5.5 lần nhưng chỉ tăng số token 1.8 lần (cách diễn đạt của paper này). Làm theo đó, các LLM lớn nhất được huấn luyện với khoảng 300 tỷ token dù kích thước tăng vọt:

Mô hình	Tham số	Token huấn luyện
LaMDA	137 tỷ	168 tỷ
GPT-3	175 tỷ	300 tỷ
Jurassic	178 tỷ	300 tỷ
Gopher	280 tỷ	300 tỷ
MT-NLG 530B	530 tỷ	270 tỷ
Chinchilla	**70 tỷ**	**1.4 nghìn tỷ**

Huấn luyện một LLM thường chỉ làm được một lần, và ngân sách (số accelerator x thời gian) thường biết trước. Nên câu hỏi then chốt là: **với tổng FLOPs cố định C, chọn N và D thế nào để loss nhỏ nhất?**

$$N_{opt}(C),\ D_{opt}(C) = \arg \min_{N,D \text{ s.t. } \mathrm{FLOPs}(N,D)=C} L(N, D).$$

(Giải nghĩa: N số tham số, D số token đã thấy; "compute-optimal" = tối ưu cho tổng huấn luyện, chưa tính chi phí suy luận.)

2. Ý tưởng chính

Tác giả huấn luyện hơn 400 mô hình (70 triệu đến hơn 16 tỷ tham số, 5 tỷ đến hơn 400 tỷ token) và ước lượng N_{opt}, D_{opt} bằng **ba cách độc lập**. Cả ba cho cùng kết luận: $N_{opt} \propto C^{\approx 0.5}$ và $D_{opt} \propto C^{\approx 0.5}$, tức mỗi lần nhân đôi kích thước mô hình thì cũng phải nhân đôi số token. Hệ quả: các LLM đương thời **quá lớn và huấn luyện thiếu dữ liệu**. Để kiểm chứng, họ huấn luyện **Chinchilla** (70B, 1.4T token) bằng đúng ngân sách compute

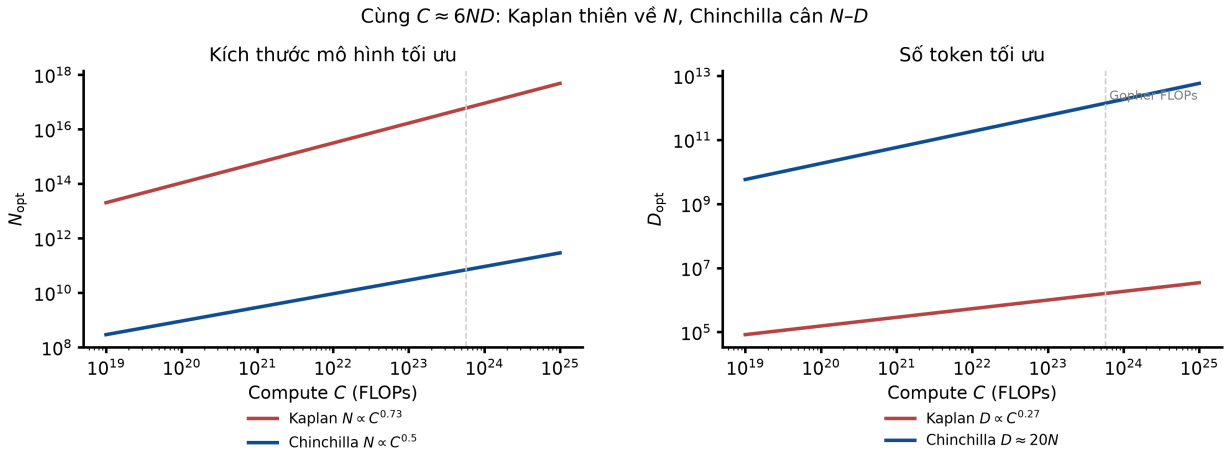
của Gopher (280B, 300B token); Chinchilla thắng Gopher, GPT-3, Jurassic-1, MT-NLG trên hầu hết tác vụ và còn rẻ hơn khi suy luận/fine-tune vì nhỏ hơn 4 lần.

3. Phương pháp chi tiết

3.1 Vì sao kết luận khác Kaplan (Mục 2)

Paper nêu hai khác biệt phương pháp:

1. **Lịch learning rate.** Kaplan dùng số token và lịch lr cố định cho mọi mô hình. Nhóm này thấy: đặt độ dài chu kỳ cosine **khớp với số token huấn luyện** cho loss cuối tốt nhất (Hình A1). Nếu dùng lịch cosine dài cố định tới 130B token thì loss trung gian (ở $D' \ll 130$B) bị **đánh giá cao** so với mô hình thực sự huấn luyện với lịch ngắn hơn; điều này làm đánh giá thấp hiệu quả huấn luyện ít dữ liệu và góp phần dẫn đến kết luận "N nên tăng nhanh hơn D". Khi cosine vượt quá số bước huấn luyện hơn 25% thì hiệu năng giảm rõ (Phụ lục B, D.1).
2. **Dải kích thước mô hình.** Chinchilla dùng mô hình đến 16B (đa số trên 500M tham số), vì thấy đường biên FLOP-loss hơi cong (Phụ lục E); Kaplan chủ yếu dưới 100M.



Kaplan $N \propto C^{0.73}$ vs Chinchilla $N \propto C^{0.5}$, $D \approx 20N$. (minh hoạ, tự tính)

3.2 Đếm FLOPs

Paper tính FLOPs đầy đủ (kể cả embedding), hệ số 2 cho nhân-cộng, backward = 2 lần forward (Phụ lục F), nhưng so với xấp xỉ quen thuộc $C = 6ND$ thì chênh lệch rất nhỏ (Bảng A4) nên dùng FLOPs$(N, D) \approx 6ND$ trong suy diễn. Khác Kaplan: N ở đây **bao gồm** embedding.

3.3 Cách 1: giữ cỡ mô hình, đổi số token

Với mỗi N (70M đến hơn 10B) huấn luyện 4 độ dài (độ dài chu kỳ cosine khác nhau tới 16 lần, lr giảm 10 lần). Làm trơn và nội suy đường loss thành hàm FLOPs → loss. Tại 1500 mức FLOPs cách đều log, chọn mô hình có loss nhỏ nhất, từ đó có (N, D) tối ưu theo C (Hình 2). Fit luỹ thừa: $a = 0.50$, $b = 0.50$. Mọi điểm được chọn nằm trong 15% cuối quá trình huấn luyện, củng cố nguyên tắc "chu kỳ cosine ≈ độ dài huấn luyện".

3.4 Cách 2: IsoFLOP profiles

Cố định 9 mức FLOPs (từ 6×10^{18} đến 3×10^{21}), với mỗi mức huấn luyện nhiều kích thước N (tới 16B) sao cho $D = C/(6N)$ (độ dài cosine khớp). Vẽ loss cuối theo N: được **thung lũng** rõ ràng (Hình 3). Fit parabol để tìm N tối ưu từng mức, rồi fit luỹ thừa: $a = 0.49$, $b = 0.51$.

3.5 Cách 3: hàm loss tham số

Mô hình hoá loss cuối như tổng ba số hạng (dựa trên phân rã rủi ro cổ điển):

$$\boxed{\hat{L}(N, D) = E + \frac{A}{N^{\alpha}} + \frac{B}{D^{\beta}}}$$

- E: loss của quá trình sinh lý tưởng, tương ứng entropy của văn bản tự nhiên.
- A/N^{α}: một Transformer N tham số huấn luyện hoàn hảo vẫn kém quá trình lý tưởng (hạn chế của không gian hàm).
- B/D^{β}: chưa hội tụ do chỉ có số bước hữu hạn, một epoch trên mẫu hữu hạn.

Fit bằng tối thiểu Huber ($\delta = 10^{-3}$) giữa $\log \hat{L}$ và $\log L$ quan sát, dùng L-BFGS với lưới khởi tạo (để tránh cực tiểu địa phương). Huber chống ngoại lai; δ lớn hơn làm mô hình khớp quá vùng compute nhỏ. Kết quả (Phụ lục D.2):

$$L(N, D) = 1.69 + \frac{406.4}{N^{0.34}} + \frac{410.7}{D^{0.28}}, \quad E = 1.69,\ A = 406.4,\ B = 410.7,\ \alpha = 0.34,\ \beta = 0.28.$$

Với ràng buộc $6ND = C$, cực tiểu có dạng đóng (phương trình 4):

$$N_{opt}(C) = G\Big(\frac{C}{6}\Big)^{a}, \quad D_{opt}(C) = G^{-1}\Big(\frac{C}{6}\Big)^{b}, \quad G = \Big(\frac{\alpha A}{\beta B}\Big)^{\frac{1}{\alpha+\beta}},\ a = \frac{\beta}{\alpha+\beta},\ b = \frac{\alpha}{\alpha+\beta}.$$

Dễ thấy $a + b = 1$, và vì $\alpha \approx \beta$ nên $a \approx b \approx 0.5$. Với $\alpha = 0.34$, $\beta = 0.28$: $a \approx 0.45$, $b \approx 0.55$ (tự tính từ các hằng số làm tròn; paper báo cáo 0.46 và 0.54 từ fit đầy đủ). Các điểm có $C \leq 10^{21}$ có phần dư lớn hơn nên Huber coi như ngoại lai; do đường biên cong âm, cách 3 dự đoán mô hình tối ưu **nhỏ hơn** hai cách còn lại ở compute lớn.

3.6 Tổng hợp (Bảng 2)

Cách	a ($N_{opt} \propto C^a$)	b ($D_{opt} \propto C^b$)
1. Minimum over training curves	0.50 (0.488, 0.502)	0.50 (0.501, 0.512)
2. IsoFLOP profiles	0.49 (0.462, 0.534)	0.51 (0.483, 0.529)
3. Parametric modelling of the loss	0.46 (0.454, 0.455)	0.54 (0.542, 0.543)
Kaplan et al. (2020)	0.73	0.27

(Trong ngoặc: phân vị 10 và 90 qua bootstrap, lấy mẫu 80% dữ liệu 100 lần.)

Pseudo-code để dùng kết quả cách 1 và 3:

```
def chinchilla_plan(C):                 # C: tổng FLOPs huấn luyện
    # Quy tắc thực hành rút từ Bảng 3: D ≈ 20 * N  (tự tính, xem dưới)
    N = (C / (6 * 20)) ** 0.5           # từ C = 6*N*D với D = 20*N
    D = 20 * N
    return N, D
# ví dụ: C = 5.76e23 (ngân sách Gopher) -> N ≈ 69e9, D ≈ 1.4e12  (khớp Chinchilla)

def parametric_loss(N, D):
    return 1.69 + 406.4 / N**0.34 + 410.7 / D**0.28   # trên MassiveText, tokenizer
    ↪  Gopher
```

Kiểm tra bằng số (tự tính): $N = \sqrt{5.76 \times 10^{23}/120} = \sqrt{4.8 \times 10^{21}} \approx 6.9 \times 10^{10}$, $D = 20N \approx 1.39 \times 10^{12}$. Khớp Chinchilla (70B, 1.4T). Tỉ lệ ~20 token/tham số **không** được paper phát biểu như quy tắc; ta rút ra từ Bảng 3 (xem ngay dưới).

4. Thí nghiệm và kết quả chính

4.1 Bảng 3: (N, FLOPs, token) tối ưu theo cách 1

Tham số	FLOPs	Token	Token/tham số (tự tính)
400 triệu	1.92×10^{19}	8.0 tỷ	20
1 tỷ	1.21×10^{20}	20.2 tỷ	20
10 tỷ	1.23×10^{22}	205.1 tỷ	20.5
67 tỷ	5.76×10^{23}	1.5 nghìn tỷ	22
175 tỷ	3.85×10^{24}	3.7 nghìn tỷ	21
280 tỷ	9.90×10^{24}	5.9 nghìn tỷ	21
520 tỷ	3.43×10^{25}	11.0 nghìn tỷ	21
1 nghìn tỷ	1.27×10^{26}	21.2 nghìn tỷ	21
10 nghìn tỷ	1.30×10^{28}	216.2 nghìn tỷ	21.6

Cột "FLOPs (Gopher unit)": mô hình 67B ứng với 1 đơn vị Gopher (5.76×10^{23}); GPT-3 175B tối ưu cần 6.7 đơn vị. Phần chữ của paper (tổng hợp các cách) nêu: mô hình 175B nên huấn luyện với 4.41×10^{24} FLOPs và trên 4.2 nghìn tỷ token (khác chút so với Bảng 3); Gopher-like 280B tối ưu ở khoảng 10^{25} FLOPs với 6.8 nghìn tỷ token; trừ khi có 10^{26} FLOPs, mô hình 1 nghìn tỷ tham số khó là tối ưu. (Các cách 2 và 3 cho số khác một chút, xem Bảng A3.) Dự đoán cho ngân sách Gopher (5.76×10^{23}): mô hình tối ưu nhỏ hơn 4 lần (khoảng 40 đến 70 tỷ) và huấn luyện nhiều hơn 4 lần token. Cách 3 ước lượng 40B (Hình 4); paper chọn 70B cho Chinchilla, tức đầu cao của dải 40 đến 70B, "vì lý do dữ liệu và hiệu quả tính toán".

Mô hình Chinchilla vs Gopher (Bảng 4): cùng ngân sách FLOPs, cùng kiến trúc cơ bản.

	Layers	Heads	Key/Value size	d_{model}	Max LR	Batch (token)
Gopher 280B	80	128	128	16,384	4×10^{-5}	3M → 6M
Chinchilla 70B	80	64	128	8,192	1×10^{-4}	1.5M → 3M

FFN luôn $4 \times d_{model}$. Khác biệt huấn luyện: dùng AdamW thay Adam (cải thiện loss và hiệu năng sau fine-tune; AdamW chỉ vượt Adam khoảng 80% đường cosine); tokenizer SentencePiece sửa đổi không NFKC (94.15% token giống Gopher, tốt hơn cho toán/hoá học); forward/backward bfloat16 nhưng giữ bản sao float32 của trọng số trong optimizer. Huấn luyện trên MassiveText với tỉ lệ nhóm dữ liệu hơi khác, JAX + Haiku, TPUv3/v4.

4.2 Kết quả đánh giá

Tác vụ	Chinchilla	Gopher	Ghi chú
MMLU (5-shot, trung bình 57 tác vụ)	67.6% (Bảng 6)	60.0%	GPT-3 43.9%; chuyên gia người 89.8%; abstract ghi 67.5% (lệch nhỏ với bảng)
BIG-bench (62 tác vụ)	65.1%	54.4%	tăng 10.7%; kém hơn Gopher ở 4/62 tác vụ

Tác vụ	Chinchilla	Gopher	Ghi chú
Wikitext103 (perplexity)	7.16	7.75	cảnh báo rò rỉ dữ liệu do 4x dữ liệu
LAMBADA zero-shot	77.4%	74.5%	GPT-3 76.2%; MT-NLG 530B 76.6%
RACE-m / RACE-h few-shot	86.8% / 82.3%	75.1% / 71.6%	GPT-3: 58.1 / 46.8 (prompt khác)
HellaSwag / PIQA / Winogrande / SIQA / BoolQ (zero-shot)	80.8 / 81.8 / 74.9 / 51.3 / 83.7	79.2 / 81.8 / 70.1 / 50.6 / 79.3	MT-NLG 530B: 80.2 / 82.0 / 73.0 / - / 78.2
TruthfulQA (0 / 5 / 10-shot)	43.6 / 58.5 / 66.7	29.5 (0-shot), 43.7 (10-shot)	
Natural Questions 5-shot / 64-shot	31.5% / 35.5%	24.5% / 28.2% (Bảng 9)	text ghi Gopher 21% và 28%, lệch với bảng
TriviaQA (unfiltered, test) 0 / 5 / 64-shot	67.0 / 73.2 / 72.3	52.8 / 63.6 / 61.3	
Winogender (tất cả)	78.3%	71.4%	cải thiện không đều: nam +3.2%, nữ +8.3%, trung tính +9.2%

Trên MMLU, Chinchilla tốt hơn Gopher ở 51/57 tác vụ, bằng ở 2, kém ở 4 (college_mathematics, econometrics, moral_scenarios, formal_logic); đạt trên 90% ở 4 tác vụ (high_school_gov_and_politics, international_law, sociology, us_foreign_policy). Trên The Pile, Chinchilla tốt hơn Gopher ở mọi tập con (bits-per-byte), và hơn Jurassic-1 trừ dm_mathematics và ubuntu_irc. Toxicity (25.000 mẫu, PerspectiveAPI): trung bình 0.087 (Chinchilla) so với 0.081 (Gopher), percentile 95 là 0.238 so với 0.230: khác biệt không đáng kể.

5. Phân tích / ablation

- **Ba cách đồng thuận:** các hệ số a, b xấp xỉ 0.5 và rất khác Kaplan (0.73/0.27); đây là bằng chứng chính, vì ba phương pháp dùng các mô hình và cách fit khác nhau.
- **Kiểm chứng trực tiếp** ở quy mô lớn: Chinchilla vs Gopher cùng FLOPs; bằng chứng bổ sung là so sánh đối đầu ở 10^{21} FLOPs giữa kích thước theo Kaplan và theo đề xuất này (Phụ lục D.4), kích thước đề xuất thắng rõ.
- **Nhất quán giữa các bộ dữ liệu (Phụ lục C):** lặp lại IsoFLOP trên C4 và GitHub code cho kết luận tương tự (tăng N và D cân bằng).
- **Lịch lr quan trọng:** cosine dài hơn mục tiêu hơn 25% gây suy giảm; vì vậy mọi so sánh phải dùng độ dài khớp.
- **AdamW:** loss cuối tốt hơn Adam, dù chỉ bắt kịp ở khoảng 80% đường cosine (chú thích 8, Hình A7).
- **Độ cong:** đường biên FLOP-loss cong âm ở compute cao (Phụ lục E, Hình A5): fit riêng ba đoạn đầu/giữa/cuối cho độ dốc khác nhau, nên có thể vẫn đang **đánh giá quá cao** kích thước tối ưu cho các ngân sách rất lớn. Đây là lý do cách 3 dự đoán mô hình nhỏ hơn.
- **Tác động đến lợi ích hạ nguồn:** vì nhỏ hơn 4 lần, bộ nhớ và chi phí suy luận của Chinchilla thấp hơn Gopher tương ứng; tác giả nhấn mạnh năng lượng huấn luyện được "amortized" qua suy luận và fine-tuning.

6. Giới hạn

Do tác giả nêu:

- Chỉ có hai lần chạy cỡ lớn so sánh được (Chinchilla và Gopher), không có thí nghiệm trung gian ở quy mô giữa.

- Giả định đường biên là luỹ thừa; thực tế thấy độ cong (có thể vẫn đang ước lượng kích thước tối ưu quá lớn).
- Tất cả huấn luyện dưới một epoch; chưa xét chế độ nhiều epoch.
- Cần thận trọng với rò rỉ train/test khi so sánh trên các benchmark ngôn ngữ vì Chinchilla thấy 4 lần dữ liệu.
- Không đủ dữ liệu chất lượng cao: kết quả cho thấy cần các tập dữ liệu lớn hơn và có chất lượng cao mới tiếp tục scale được.
- Đánh giá bias và toxicity chỉ là mẫu nhỏ, không toàn diện; cải thiện bias không đều giữa các nhóm.

Nhận xét bổ sung:

- Tối ưu theo **compute huấn luyện**, không phải tổng chi phí vòng đời. Nếu mô hình phục vụ rất nhiều truy vấn, nên huấn luyện mô hình nhỏ hơn với nhiều token hơn mức này (đi theo hướng LLaMA, bài 05; ý này nằm ngoài paper, dù paper có nhắc lợi ích suy luận).
- Hệ số A, B, α, β, E gắn với MassiveText, tokenizer của Gopher và kiến trúc cụ thể; không chuyển trực tiếp sang dữ liệu khác (ví dụ OCR).
- Một số số liệu không đồng nhất giữa phần chữ và bảng (MMLU 67.5% so với 67.6%; NQ của Gopher). Ngoài ra bảng đối chiếu cách 3 (Bảng A3) có vài ô khó đọc trong bản trích văn bản nên ta chỉ dùng Bảng 3 (cách 1).
- Chỉ xét dense Transformer; MoE và retrieval chỉ nhắc ở phần liên quan.

7. Liên hệ với các bài khác

- **03** Kaplan: bài được phản biện; so sánh $0.73/0.27$ với $0.5/0.5$, và công thức $C \approx 6ND$ lấy từ đó. Khác biệt chính (theo paper) nằm ở lịch lr và dải kích thước mô hình; ngoài ra quy ước N cũng khác (Chinchilla đếm cả embedding, Kaplan thì không).
- **02** GPT-3: ví dụ điển hình của mô hình lớn huấn luyện 300B token (theo Bảng 1 của bài này, lẽ ra GPT-3 nên có nhiều token hơn nhiều).
- **05** LLaMA, **06** Mistral: áp dụng bài học này và đi xa hơn (huấn luyện mô hình nhỏ nhiều token vì tối ưu suy luận).
- **27, 28** MoE: Chinchilla nhắc rằng lợi ích tính toán của routed model giảm ở quy mô lớn (Clark et al. 2022).
- **44 đến 57** (suy luận hiệu quả, lượng tử hoá, nén KV): tiếp nối ý "mô hình nhỏ hơn thì rẻ hơn khi triển khai".

8. Gợi ý áp dụng

Tỉ lệ khoảng 20 token cho mỗi tham số là một mốc rút từ các điều kiện và dữ liệu của nghiên cứu, không phải hằng số phổ quát. Nó nhắm tới tối ưu chi phí huấn luyện; khi một mô hình được suy luận nhiều lần, chi phí phục vụ có thể làm thay đổi lựa chọn. Bài báo không kiểm chứng trực tiếp quy tắc này cho cắt tỉa, chưng cất hay dữ liệu đa phương thức.

9. Thí nghiệm gợi ý

Ở mỗi ngân sách FLOPs đã chọn, huấn luyện nhiều kích thước mô hình với lượng dữ liệu tương ứng và vẽ loss theo kích thước. Lặp lại cho nhiều ngân sách để khảo sát điểm tối ưu theo cách tiếp cận IsoFLOP. Giữ lịch learning rate phù hợp với độ dài từng lượt chạy; các điều kiện dữ liệu và kiến trúc cũng cần được báo cáo để việc so sánh có ý nghĩa.

10. Tóm tắt công thức / khái niệm

Khái niệm	Công thức / giá trị
Bài toán	$\min L(N, D)$ s.t. $6ND = C$
Hàm loss tham số	$\hat{L} = E + A/N^{\alpha} + B/D^{\beta}$; $E = 1.69$, $A = 406.4$, $B = 410.7$, $\alpha = 0.34$, $\beta = 0.28$
Nghiệm đóng	$N_{opt} = G(C/6)^{a}$, $D_{opt} = G^{-1}(C/6)^{b}$, $a = \beta/(\alpha+\beta)$, $b = \alpha/(\alpha+\beta)$
Hệ số thực nghiệm	$a \approx 0.50/0.49/0.46$, $b \approx 0.50/0.51/0.54$ (ba cách)
Kaplan	$a = 0.73$, $b = 0.27$
Quy tắc thực hành (tự rút)	$D \approx 20N$
Chinchilla	70B tham số, 1.4T token, cùng FLOPs với Gopher (280B, 300B token)
Lịch lr	cosine khớp số token, giảm 10 lần
Kết quả nổi bật	MMLU 67.6% (Gopher 60.0%); BIG-bench 65.1% (Gopher 54.4%)

11. Kết luận ngắn

Với compute cố định, tham số và token nên được tăng cùng tỉ lệ (khoảng 20 token mỗi tham số theo Bảng 3), và các LLM đương thời bị quá cỡ. Chinchilla 70B huấn luyện 1.4T token thắng Gopher 280B cùng ngân sách và rẻ hơn khi suy luận. Giá trị lâu dài là phương pháp (ba cách ước lượng, lịch lr khớp độ dài, hàm loss tham số) và thông điệp “dữ liệu cũng quan trọng ngang kích thước”.

Nguồn gốc: https://arxiv.org/abs/2203.15556

Bài 05

LLaMA: Open and Efficient Foundation Language Models (2023)

Tác giả: Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, Aurelien Rodriguez, Armand Joulin, Edouard Grave, Guillaume Lample (Meta AI).
Venue / arXiv: arXiv:2302.13971 (v1, 27/02/2023). Bản text không ghi venue.
Nguồn: https://arxiv.org/abs/2302.13971
Chủ đề: Họ mô hình ngôn ngữ 7B/13B/33B/65B train trên dữ liệu công khai, "train lâu hơn mức compute-optimal để suy luận rẻ"; công thức kiến trúc Transformer hiện đại (RMSNorm pre-norm + SwiGLU + RoPE).
Vị trí trong lộ trình: Đọc sau *02 GPT-3*, *03 Kaplan*, *04 Chinchilla* (bối cảnh scaling), và nên có nền về *09 Pre-LN*, *10 RMSNorm*, *11 SwiGLU*, *12 RoPE* (LLaMA chỉ "ráp" các mảnh này; ghi chú này giải thích đủ để đọc độc lập). Bài này cho thấy công thức decoder chuẩn mà gần như mọi LLM mở sau đó đều dùng. Tiếp theo: *06 Mistral 7B* (thêm GQA + sliding window), *08 LoRA*.

> Ghi chú diễn giải bằng tiếng Việt, không phải bản dịch. Các số liệu "tự tính" là ví dụ minh họa, không phải kết quả trực tiếp của bài báo.

1. Bối cảnh

Đến đầu 2023, các LLM mạnh nhất (GPT-3 175B, Gopher 280B, PaLM 540B) đều **đóng**: trọng số không công bố, và dữ liệu dùng nhiều nguồn không công khai hoặc không được ghi chép (paper ví dụ "Books – 2TB", "Social media conversations"). Mô hình mở (OPT, GPT-NeoX, BLOOM, GLM) chưa cạnh tranh được với PaLM-62B hay Chinchilla.

Scaling laws và điểm mù về chi phí suy luận. *Scaling law* là công thức dự đoán loss theo số tham số N và số token D. Chinchilla (bài 04) chỉ ra: với *ngân sách compute huấn luyện cố định*, mô hình tối ưu là mô hình nhỏ hơn train trên nhiều dữ liệu hơn (khoảng 20 token/tham số, theo cách hiểu phổ biến của Chinchilla). Paper LLaMA lưu ý rằng mục tiêu này **bỏ qua chi phí phục vụ (inference)**: khi bạn phục vụ mô hình cho hàng triệu lượt, thứ cần tối ưu là mô hình *nhanh nhất lúc suy luận cho một mức chất lượng cho trước*. Với mục tiêu đó, nên train mô hình nhỏ **lâu hơn nhiều** mức Chinchilla-optimal. Ví dụ paper đưa ra: Chinchilla khuyên 10B mô hình với 200B token, nhưng họ thấy mô hình 7B vẫn tiếp tục cải thiện sau 1T token.

---

2. Ý tưởng chính

LLaMA là một họ Transformer decoder (7B, 13B, 33B, 65B) có hai điểm khác biệt về triết lý: (1) **chỉ dùng dữ liệu công khai** (CommonCrawl, C4, GitHub, Wikipedia, sách, ArXiv, StackExchange) để kết quả tái lập và mở được; (2) **train trên rất nhiều token** (1.0T token cho 7B/13B, 1.4T cho 33B/65B) để mô hình nhỏ đạt chất lượng của mô hình lớn hơn nhiều. Về kiến trúc, họ lắp ghép ba cải tiến đã có: RMSNorm pre-normalization (từ GPT-3), SwiGLU (từ PaLM), rotary embeddings (từ GPTNeo). Kết quả gây chú ý: **LLaMA-13B vượt GPT-3 175B trên hầu hết benchmark dù nhỏ hơn hơn 10 lần**, chạy được trên một GPU; LLaMA-65B ngang Chinchilla-70B và PaLM-540B.

---

3. Phương pháp chi tiết

3.1 Dữ liệu (Bảng 1)

Nguồn	Tỉ lệ lấy mẫu	Epoch (khi train 1.4T token)	Kích thước đĩa
CommonCrawl	67.0%	1.10	3.3 TB
C4	15.0%	1.06	783 GB
GitHub	4.5%	0.64	328 GB
Wikipedia	4.5%	2.45	83 GB
Books	4.5%	2.23	85 GB
ArXiv	2.5%	1.06	92 GB
StackExchange	2.0%	1.03	78 GB

Tổng ~1.4T token sau tokenize. Hầu hết dữ liệu chỉ thấy 1 lần; riêng Wikipedia và Books khoảng 2 epoch.

Xử lý đáng chú ý:

- **CommonCrawl:** 5 bản dump 2017-2020 qua pipeline CCNet (khử trùng lặp ở mức dòng, nhận dạng ngôn ngữ bằng fastText để bỏ trang không phải tiếng Anh, lọc chất lượng bằng mô hình n-gram), thêm bộ phân loại tuyến tính "trang giống nguồn tham chiếu của Wikipedia hay trang ngẫu nhiên" để loại trang kém.
- **GitHub:** chỉ giữ giấy phép Apache/BSD/MIT; lọc bằng heuristic độ dài dòng, tỉ lệ ký tự chữ-số; khử trùng lặp mức file.
- **Wikipedia:** dump tháng 6-8/2022, 20 ngôn ngữ (chữ Latin hoặc Cyrillic).
- **Books:** Gutenberg + Books3; bỏ sách trùng >90% nội dung.
- **ArXiv:** LaTeX, bỏ phần trước section đầu tiên và bibliography, bỏ comment, mở macro.
- **Tokenizer:** BPE qua SentencePiece; **tách mọi số thành từng chữ số**; ký tự UTF-8 lạ thì rơi về byte.

3.2 Kiến trúc: ba thay đổi so với Transformer gốc

Mỗi block là decoder-only (causal), nhưng với:

(i) Pre-normalization + RMSNorm. Chuẩn hoá *đầu vào* của mỗi sub-layer thay vì đầu ra (ổn định huấn luyện). Dùng **RMSNorm** (Zhang & Sennrich, 2019): bỏ bước trừ trung bình của LayerNorm, chỉ chia theo căn bậc hai trung bình bình phương:

$$\boxed{\text{RMSNorm}(x) = \frac{x}{\sqrt{\frac{1}{d}\sum_{i=1}^{d} x_i^2 + \epsilon}} \odot g}$$

Rẻ hơn LayerNorm (không tính mean, không bias). Bài 10 phân tích chi tiết.

(ii) SwiGLU thay ReLU trong FFN. FFN thường: $W_2\,\text{ReLU}(W_1 x)$ với hidden $4d$. SwiGLU là một biến thể của *GLU* (gated linear unit: nhân từng phần tử giữa một nhánh "cổng" qua hàm kích hoạt và một nhánh tuyến tính):

$$\boxed{\text{FFN}_{\text{SwiGLU}}(x) = W_2(\text{SiLU}(W_1 x) \odot W_3 x)}, \qquad \text{SiLU}(z) = z\,\sigma(z)$$

Vì có **3 ma trận** thay vì 2, để giữ số tham số xấp xỉ, hidden được giảm xuống $\frac{2}{3} \cdot 4d$ (paper: "$\frac{2}{3}4d$ instead of $4d$ as in PaLM"). Kiểm tra: $3 \cdot d \cdot \frac{8}{3}d = 8d^2$, bằng $2 \cdot d \cdot 4d = 8d^2$ (tự tính). Bài 11 phân tích sâu hơn.

(iii) Rotary Embeddings (RoPE) thay positional embedding tuyệt đối. Thay vì cộng vector vị trí vào embedding đầu vào, RoPE **xoay** vector query và key theo góc tỉ lệ với vị trí, *ở mỗi layer*. Với từng cặp chiều

(x_{2j}, x_{2j+1}) ở vị trí m, xoay góc $m\theta_j$; khi đó tích vô hướng $q_m^\top k_n$ chỉ phụ thuộc vị trí *tương đối* $m-n$. Paper chỉ nói "add rotary positional embeddings at each layer"; cơ chế trên là kiến thức nền từ RoFormer (bài 12). Lưu ý: RoPE chỉ tác động lên Q, K (không có tham số học thêm), nên không tốn tham số.

Một block (tự viết từ mô tả):

$$h = x + \text{Attn}_{\text{RoPE}}(\text{RMSNorm}(x)), \qquad y = h + \text{FFN}_{\text{SwiGLU}}(\text{RMSNorm}(h))$$

Attention là multi-head thông thường (n_{heads} đầu, mỗi đầu có K,V riêng): **LLaMA-1 chưa dùng GQA/MQA** (bài này không nhắc, khác với Mistral).

3.3 Cấu hình (Bảng 2)

Tham số	dim	n_{heads}	n_{layers}	Learning rate	Batch	Số token
6.7B	4096	32	32	3.0×10^{-4}	4M	1.0T
13.0B	5120	40	40	3.0×10^{-4}	4M	1.0T
32.5B	6656	52	60	1.5×10^{-4}	4M	1.4T
65.2B	8192	64	80	1.5×10^{-4}	4M	1.4T

(Paper gọi là 7B/13B/33B/65B; bảng ghi chính xác 6.7B/13.0B/32.5B/65.2B.)

Kiểm tra số tham số cho 6.7B (tự tính): mỗi layer có attention $4d^2$ + FFN SwiGLU $8d^2 = 12d^2$; $12 \times 4096^2 \times 32 \approx 6.44$B, cộng embedding đầu vào + đầu ra (giả sử không chia sẻ, vocab khoảng 32k, con số vocab này **không** nằm trong bản văn bản bài báo mà là kiến thức bên ngoài) ≈ 0.26B → khoảng 6.7B, khớp.

3.4 Tối ưu hóa (mục 2.3)

AdamW với $\beta_1 = 0.9$, $\beta_2 = 0.95$; cosine schedule, LR cuối = 10% LR đỉnh; weight decay 0.1; gradient clipping 1.0; 2000 bước warmup; LR và batch thay đổi theo kích thước (Bảng 2).

3.5 Triển khai hiệu quả (mục 2.4)

- Attention nhân quả tiết kiệm bộ nhớ (xformers): **không lưu trọng số attention** và **không tính các cặp query-key bị mask** (cùng tinh thần FlashAttention, bài 44).
- Giảm tính lại activation (activation checkpointing có chọn lọc): lưu các activation đắt như output của linear layer; tự viết backward cho transformer layer thay vì dùng autograd.
- Model parallelism + sequence parallelism, chồng lấp tính toán với truyền thông giữa GPU.
- Thông lượng: ~**380 token/s/GPU** trên 2048 GPU A100-80GB cho mô hình 65B; 1.4T token mất khoảng **21 ngày**. Kiểm tra (tự tính): $1.4 \times 10^{12}/(380 \times 2048) \approx 1.8 \times 10^6$ giây ≈ 20.8 ngày, khớp.

3.6 Đánh giá

Zero-shot (mô tả + câu hỏi) và few-shot (1 đến 64 ví dụ). Trắc nghiệm: chọn phương án có likelihood cao nhất, chuẩn hoá theo số ký tự (hầu hết) hoặc theo $P(\text{completion} \mid \text{context})/P(\text{completion} \mid \text{"Answer:"})$ (OpenBookQA, BoolQ). Tổng 20 benchmark.

---

4. Thí nghiệm và kết quả chính

Số liệu copy từ các bảng của paper; GPT-3/Gopher/Chinchilla/PaLM là số các paper tương ứng, theo paper này trích.

4.1 Suy luận thường thức, zero-shot (Bảng 3)

Mô hình	BoolQ	PIQA	SIQA	HellaSwag	WinoGrande	ARC-e	ARC-c	OBQA
GPT-3 175B	60.5	81.0	-	78.9	70.2	68.8	51.4	57.6
Chinchilla 70B	83.7	81.8	51.3	80.8	74.9	-	-	-
PaLM 540B	88.0	82.3	-	83.4	81.1	76.6	53.0	53.4
LLaMA 7B	76.5	79.8	48.9	76.1	70.1	72.8	47.6	57.2
LLaMA 13B	78.1	80.1	50.4	79.2	73.0	74.8	52.7	56.4
LLaMA 33B	83.1	82.3	50.4	82.8	76.0	80.0	57.8	58.6
LLaMA 65B	85.3	82.8	52.3	84.2	77.0	78.9	56.0	60.2

LLaMA-13B vượt GPT-3 175B trên 5/7 cột có số liệu của cả hai (thua nhẹ PIQA 80.1 vs 81.0 và OBQA 56.4 vs 57.6); LLaMA-65B vượt Chinchilla-70B ở mọi cột trừ BoolQ (theo paper).

4.2 Hỏi đáp closed-book (exact match)

NaturalQuestions: LLaMA-65B 23.8 / 31.0 / 35.0 / 39.9 (0/1/5/64-shot), so với Chinchilla-70B 16.6 / - / 31.5 / 35.5 và GPT-3 175B 14.6 / 23.0 / - / 29.9.

TriviaQA (filtered dev): LLaMA 7B 50.0 / 53.4 / 56.3 / 57.6; 13B 56.6 / 60.5 / 63.1 / 64.0; 33B 65.1 / 67.9 / 69.9 / 70.4; **65B 68.2 / 71.6 / 72.6 / 73.0**; Chinchilla-70B 55.4 / - / 64.1 / 64.6.

4.3 Đọc hiểu RACE (zero-shot): middle / high

GPT-3 175B 58.4 / 45.5; PaLM 540B 68.1 / 49.1; LLaMA 7B 61.1 / 46.9; 13B 61.6 / 47.2; 33B 64.1 / 48.3; **65B 67.9 / 51.6**.

4.4 Toán (Bảng 7) và Code (Bảng 8)

	MATH	MATH+maj1@k	GSM8k	GSM8k+maj1@k
PaLM 62B	4.4	-	33.0	-
PaLM 540B	8.8	-	56.5	-
Minerva 62B	27.6	43.4	52.4	68.5
LLaMA 7B	2.9	6.9	11.0	18.1
LLaMA 13B	3.9	8.8	17.8	29.3
LLaMA 33B	7.1	15.2	35.6	53.1
LLaMA 65B	10.6	20.5	50.9	69.7

Minerva là họ PaLM đã fine-tune trên 38.5B token toán/ArXiv; LLaMA không fine-tune toán nhưng 65B vẫn vượt Minerva-62B trên GSM8k.

Code (pass@1 / pass@100 hoặc @80 trên HumanEval / MBPP): PaLM 62B 15.9 / 46.3* / 21.4 / 63.2*; LLaMA 7B 10.5 / 36.5 / 17.7 / 56.2; 13B 15.8 / 52.5 / 22.0 / 64.0; 33B 21.7 / 70.7 / 30.2 / 73.4;* ***65B 23.7 / 79.3 / 37.7 / 76.8****. (* = đọc từ hình.) LLaMA-13B+ vượt LaMDA 137B (14.0 / 14.8 pass@1).

4.5 MMLU (5-shot, Bảng 9) và instruction finetuning

	Humanities	STEM	Social Sci.	Other	Trung bình
GPT-3 175B	40.8	36.7	50.4	48.8	43.9

	Humanities	STEM	Social Sci.	Other	Trung bình
Chinchilla 70B	63.6	54.9	79.3	73.9	67.5
PaLM 540B	77.0	55.6	81.0	69.6	69.3
LLaMA 7B	34.0	30.5	38.3	38.1	35.1
LLaMA 13B	45.0	35.8	53.8	53.3	46.9
LLaMA 33B	55.8	46.0	66.7	63.4	57.8
LLaMA 65B	61.8	51.7	72.9	67.4	63.4

Đây là chỗ LLaMA-65B **thua** Chinchilla-70B và PaLM-540B vài điểm. Giả thuyết của tác giả: lượng sách và bài báo khoa học trong dữ liệu chỉ ~177GB (ArXiv + Gutenberg + Books3) so với tới 2TB sách ở các mô hình kia.

Một lượt fine-tune instruction rất ngắn (LLaMA-I 65B, theo giao thức Flan) đưa MMLU lên **68.9%** (so 63.4% bản gốc), vượt OPT-IML-Max 30B (43.2) và Flan-PaLM 62B (59.6), nhưng vẫn kém SOTA (77.4 của GPT code-davinci-002).

4.6 Chi phí huấn luyện và carbon (Bảng 15, A100-80GB, 400W, PUE 1.1)

Mô hình	GPU-hours	Điện	CO2
OPT-175B	809,472	356 MWh	137 tCO2eq
BLOOM-175B	1,082,880	475 MWh	183
LLaMA-7B	82,432	36 MWh	14
LLaMA-13B	135,168	59 MWh	23
LLaMA-33B	530,432	233 MWh	90
LLaMA-65B	1,022,362	449 MWh	173

Toàn bộ quá trình phát triển: khoảng 2048 A100 trong ~5 tháng, ước tính 2,638 MWh và 1,015 tCO2eq.

4.7 Thiên kiến / độc hại (tóm tắt)

RealToxicityPrompts: độc hại tăng theo kích thước (basic 0.106 ở 7B lên 0.128 ở 65B). CrowS-Pairs: trung bình 66.6 (LLaMA-65B) so với 67.2 (GPT-3) và 69.5 (OPT), thiên kiến tôn giáo cao (+10 so với OPT-175B). WinoGender: model làm tốt hơn với đại từ trung tính “their/them/someone” (81.7 ở 65B) so với “her/her/she” (78.8) và “his/him/he” (72.1), và tệ hơn trên các câu “gotcha” → bắt được thiên kiến nghề nghiệp-giới tính. TruthfulQA (truthful*informative): 65B đạt 0.53, so với GPT-3 175B 0.25 nhưng tỉ lệ đúng vẫn thấp, tức vẫn dễ hallucinate.

---

5. Phân tích / ablation

Paper này **gần như không có ablation kiến trúc**: ba thay đổi (RMSNorm, SwiGLU, RoPE) không được thử bỏ từng cái; lý do họ ghi là “lấy cảm hứng từ” các mô hình khác. Những quan sát có giá trị phân tích:

- **Mô hình nhỏ vẫn cải thiện sau mức Chinchilla-optimal.** Hình 1 (training loss theo token): 7B và 13B train đến 1T, 33B/65B đến 1.4T; Hình 2 cho thấy điểm TriviaQA, HellaSwag, NQ, PIQA tăng đều theo số token. Ngoại lệ: SIQA dao động mạnh (tác giả nghi benchmark không đáng tin) và WinoGrande không tương quan tốt với perplexity (33B và 65B gần bằng nhau).

- **Tỉ lệ token/tham số (tự tính):** 7B với 1T token ≈ 143 token/tham số; 65B với 1.4T ≈ 21.5 token/tham số. Tức mô hình 7B được train gấp ~7 lần mức "20 token/tham số", còn 65B xấp xỉ mức Chinchilla. Đây là cách LLaMA "mua" chất lượng bằng compute huấn luyện thêm để có inference rẻ.
- **Quy mô vs chất lượng:** 13B ≈ GPT-3 175B nhưng chi phí train chỉ 135,168 GPU-hours.
- **Dữ liệu mảng sách/học thuật quyết định MMLU:** giải thích tác giả đưa ra (giả thuyết, không kiểm chứng bằng ablation).
- Hiệu ứng instruction-tuning: chỉ một thí nghiệm duy nhất (MMLU +5.5 điểm ở 65B), tác giả nói rõ "không phải trọng tâm".

---

6. Giới hạn

Do tác giả nêu:

- Vẫn thiên kiến và độc hại; độc hại tăng theo kích thước trong cùng họ; TruthfulQA cho thấy hay hallucinate.
- MMLU thua Chinchilla/PaLM do ít dữ liệu sách/học thuật.
- Đánh giá thiên kiến/độc hại "không đủ để hiểu hết rủi ro".
- Chi phí carbon lớn.
- Chưa phân tích kỹ instruction finetuning; hứa hẹn mô hình lớn hơn.

Nhận xét bổ sung:

- Không công bố tỉ lệ pha trộn dữ liệu dẫn đến công thức này đã được tối ưu hay chưa, và không có ablation cho bất kỳ lựa chọn kiến trúc nào, nên "công thức LLaMA" là kết quả kết hợp chứ không phải được chứng minh từng phần.
- Không có đánh giá đa ngôn ngữ ngoài Wikipedia 20 ngôn ngữ; tiếng Việt không được nhắc đến trong bản đọc.
- Attention vẫn là MHA đầy đủ → KV-cache lớn, chưa tối ưu cho phục vụ (xem 8.1); Mistral và Llama 2 70B về sau mới thêm GQA.
- So sánh với mô hình đóng dùng số liệu từ paper khác với giao thức khác nhau; một số ô trống trong bảng.
- Cửa sổ ngữ cảnh không được nêu trong bản này (không có phần dài ngữ cảnh).

---

7. Liên hệ với các bài khác

- **02 GPT-3:** LLaMA là phản đề trực tiếp: 13B thắng 175B nhờ train nhiều token và kiến trúc cải tiến; pre-norm lấy từ GPT-3.
- **03 Kaplan / 04 Chinchilla:** LLaMA áp dụng *tinh thần* Chinchilla (nhiều dữ liệu) nhưng mở rộng mục tiêu sang chi phí suy luận, train vượt mức compute-optimal.
- **09 Pre-LN, 10 RMSNorm, 11 SwiGLU, 12 RoPE:** bốn mảnh ghép. Đọc 10-12 để hiểu từng cải tiến.
- **44 FlashAttention:** LLaMA dùng kernel attention tiết kiệm bộ nhớ cùng họ (xformers, backward theo Dao et al.).
- **06 Mistral 7B:** giữ nguyên nền LLaMA, thêm GQA + sliding window; **14 MQA / 15 GQA** là hướng giảm KV-cache mà LLaMA-1 chưa dùng.
- **08 LoRA:** LLaMA là mô hình nền chính cho LoRA/fine-tune cộng đồng.
- **33–38 (BLIP-2 đến SmolVLM):** backbone ngôn ngữ của nhiều VLM hiện đại thừa hưởng cấu trúc block RMSNorm + SwiGLU + RoPE này.

---

8. Gợi ý áp dụng

LLaMA làm rõ sự khác nhau giữa tối ưu ngân sách huấn luyện và tối ưu tổng chi phí sử dụng: huấn luyện một mô hình nhỏ lâu hơn có thể đáng cân nhắc khi cần phục vụ nhiều lượt. Đây là đánh đổi phụ thuộc khối lượng suy luận và chất lượng yêu cầu, không phải quy tắc mô hình nhỏ luôn tốt hơn.

Với SwiGLU, mỗi chiều ẩn gắn với một cặp nhánh cổng và giá trị. Khi thay đổi độ rộng FFN, các chiều tương ứng của hai phép chiếu đầu vào và phép chiếu đầu ra phải nhất quán về kích thước. Bài báo không đánh giá trực tiếp cắt tỉa; đây là hệ quả về cấu trúc, không phải kết quả thực nghiệm của bài báo.

9. Thí nghiệm gợi ý

So sánh các checkpoint theo số token huấn luyện, đồng thời đo loss, chất lượng tác vụ và chi phí suy luận. Sau đó đối chiếu các mô hình có chất lượng gần nhau để khảo sát đánh đổi giữa thời gian huấn luyện và chi phí phục vụ; cần giữ dữ liệu và phương pháp đánh giá nhất quán.

10. Tóm tắt công thức / khái niệm

Khái niệm	Công thức / ý nghĩa
RMSNorm	$x/\sqrt{\frac{1}{d}\sum x_i^2+\epsilon}\odot g$ (không trừ mean)
Pre-normalization	Chuẩn hoá đầu vào mỗi sub-layer: $x+\text{Sub}(\text{Norm}(x))$
SwiGLU FFN	$W_2(\text{SiLU}(W_1x)\odot W_3x)$, hidden $\frac{2}{3}\cdot 4d$
RoPE	Xoay Q, K theo vị trí; tích vô hướng chỉ phụ thuộc $m-n$
Tham số / layer (xấp xỉ)	$4d^2$ (attn) $+8d^2$ (SwiGLU) $=12d^2$ (tự tính)
KV-cache	$2\,L\,n_{kv}\,d_{head}\,T\,b$ (tự tính)
Dữ liệu	1.4T token, 7 nguồn công khai, CommonCrawl 67%
Train	AdamW (0.9, 0.95), cosine → 10%, wd 0.1, clip 1.0, warmup 2000, batch 4M token
Thông lượng	~380 token/s/GPU, 2048 A100, 65B, 1.4T token ≈ 21 ngày
Triết lý	Tối ưu chi phí *suy luận*: mô hình nhỏ train rất lâu

---

11. Kết luận ngắn

LLaMA chứng minh rằng một công thức Transformer decoder gọn (RMSNorm pre-norm, SwiGLU, RoPE) train trên 1.0 đến 1.4 nghìn tỷ token công khai cho ra mô hình nhỏ vừa mạnh vừa rẻ khi phục vụ: 13B đánh bại GPT-3 175B, 65B ngang Chinchilla/PaLM. Kiến trúc attention của LLaMA-1 (MHA đầy đủ, KV-cache lớn) là điểm Mistral, GQA và các biến thể attention tuyến tính sau đó cải tiến.

Nguồn gốc: https://arxiv.org/abs/2302.13971

Bài 06

Mistral 7B (2023)

Tác giả: Albert Q. Jiang, Alexandre Sablayrolles, Arthur Mensch, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Florian Bressand, Gianna Lengyel, Guillaume Lample, Lucile Saulnier, Lélio Renard Lavaud, Marie-Anne Lachaux, Pierre Stock, Teven Le Scao, Thibaut Lavril, Thomas Wang, Timothée Lacroix, William El Sayed (Mistral AI).
Venue / arXiv: arXiv:2310.06825 (v1, 10/10/2023). Bản text không ghi venue.
Nguồn: https://arxiv.org/abs/2310.06825
Chủ đề: Mô hình 7B hiệu quả: Grouped-Query Attention (GQA), Sliding Window Attention (SWA), Rolling Buffer Cache, pre-fill theo chunk; so sánh với Llama 2/Llama 1.
Vị trí trong lộ trình: Đọc sau *05 LLaMA* (kiến trúc nền) và nên biết *15 GQA* (Ainslie et al.) và *17 Longformer* (sliding window). Bài này là bản "kỹ thuật phục vụ" của LLaMA: cùng quy mô nhưng KV-cache nhỏ hơn rất nhiều. Tiếp theo: *28 Mixtral* (cùng nhà, thêm MoE), *55 StreamingLLM* (vì sao cắt cửa sổ làm mất token đầu), *46 PagedAttention* (quản lý KV-cache trên GPU).

> Ghi chú diễn giải bằng tiếng Việt, không phải bản dịch. Các số liệu "tự tính" là ví dụ minh họa, không phải kết quả trực tiếp của bài báo.

1. Bối cảnh

Hai bài toán chi phí khi phục vụ LLM:

1. **Attention thuần (full causal attention) tốn $O(n^2)$ tính toán và $O(n)$ bộ nhớ cache theo độ dài chuỗi n.** Khi sinh token thứ i, mô hình phải so khớp query mới với *toàn bộ* i key trước đó, và phải **giữ KV-cache** (key và value của mọi token đã xử lý, để khỏi tính lại) trong bộ nhớ GPU. Chuỗi dài → cache lớn → batch nhỏ → thông lượng thấp, độ trễ cao.
2. **Multi-head attention (MHA) lưu K,V riêng cho từng đầu**, nên cache nặng. Multi-query (MQA, bài 14) và Grouped-query (GQA, bài 15) cho nhiều đầu query dùng chung ít đầu K/V hơn.

LLaMA (bài 05) cho thấy mô hình nhỏ train nhiều token có thể đánh bại mô hình lớn, nhưng vẫn dùng MHA và full attention. Mistral hỏi: ở quy mô 7B, có thể vừa tăng chất lượng vừa cắt mạnh chi phí suy luận không? Paper tóm gọn quan điểm này trong kết luận: bài toán không phải 2 chiều (năng lực vs chi phí huấn luyện) mà là **3 chiều: năng lực, chi phí huấn luyện, chi phí suy luận**.

---

2. Ý tưởng chính

Mistral 7B là một Transformer decoder kiểu LLaMA với hai chỉnh sửa attention nhằm suy luận nhanh và rẻ. Thứ nhất, **GQA**: 32 đầu query chia sẻ chỉ 8 đầu key/value, làm KV-cache nhỏ đi 4 lần và giải mã nhanh hơn. Thứ hai, **sliding window attention (SWA)**: mỗi token chỉ attend vào $W = 4096$ token gần nhất ở mỗi layer; nhờ xếp chồng nhiều layer, thông tin vẫn lan xa hơn W (sau k layer, tới $k \times W$ token). Vì phạm vi attention cố định, cache có thể là **bộ đệm vòng (rolling buffer)** kích thước cố định W, không tăng theo độ dài chuỗi. Kết quả: 7B tham số vượt Llama 2 13B trên mọi benchmark paper thử và vượt Llama 1 34B ở toán, code, suy luận.

---

3. Phương pháp chi tiết

3.1 Cấu hình kiến trúc (Bảng 1)

Tham số	Giá trị
dim	4096
n_layers	32
head_dim	128
hidden_dim (FFN)	14336
n_heads	32
n_kv_heads	8
window_size	4096
context_len	8192
vocab_size	32000

Paper nói "Mistral 7B is based on a transformer architecture" và chỉ liệt kê "những thay đổi so với Llama"; các phần còn lại (RMSNorm, SwiGLU, RoPE) không được mô tả lại trong bài, nên ở đây chỉ coi là *giả định hợp lý theo họ Llama*, không phải điều paper xác nhận. Không có thông tin về dữ liệu huấn luyện, số token, hay hyper-parameter huấn luyện trong paper.

Kiểm tra số tham số (tự tính): mỗi layer: attention Q: 4096×4096, O: 4096×4096, K, V: mỗi cái $4096 \times (8 \cdot 128) = 4096 \times 1024$; tổng attention $= 2 \cdot 16.78\text{M} + 2 \cdot 4.19\text{M} = 41.9\text{M}$. FFN SwiGLU 3 ma trận: $3 \times 4096 \times 14336 = 176.2\text{M}$. Mỗi layer ≈ 218.1M, 32 layer ≈ 6.98B; thêm embedding vào + ra $2 \times 32000 \times 4096 = 0.26\text{B}$ → **≈ 7.24B** (khớp mức "7B"; lưu ý GQA tiết kiệm K, V rõ rệt so với MHA là $2 \times 16.8\text{M}$ mỗi layer).

3.2 Grouped-Query Attention (GQA)

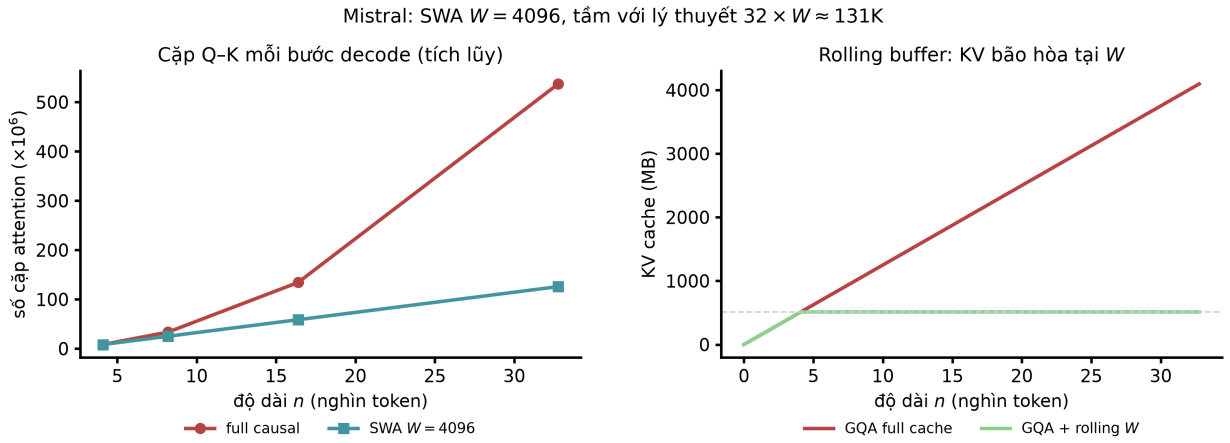
Với $h_q = 32$ đầu query và $h_{kv} = 8$ đầu K/V, mỗi nhóm $g = h_q / h_{kv} = 4$ đầu query dùng chung một cặp (K,V). MHA là trường hợp $h_{kv} = h_q$; MQA là $h_{kv} = 1$.

$$\text{head}_j = \text{softmax}\Big(\frac{q_j \, K_{\lceil j/g \rceil}^{\top}}{\sqrt{d_{head}}}\Big) V_{\lceil j/g \rceil}, \qquad j = 1..h_q$$

Hệ quả: tham số W_K, W_V nhỏ đi g lần và **KV-cache nhỏ đi $g = 4$ lần** so với MHA cùng số đầu query. Paper chỉ nói GQA "significantly accelerates the inference speed, and also reduces the memory requirement during decoding, allowing for higher batch sizes hence higher throughput"; không đưa số đo riêng cho GQA hay ablation so MHA.

3.3 Sliding Window Attention (SWA)

Thay vì token i attend vào mọi $j \le i$, chỉ attend vào $j \in [i - W,\ i]$:



Mistral SWA $W{=}4096$: giảm cặp Q–K và rolling KV bão hòa tại W. (minh hoạ, tự tính)

$$\boxed{A_{ij} \neq 0 \iff i - W \leq j \leq i}$$

Chi phí tính toán mỗi layer giảm từ $\sim n^2/2$ xuống $\sim nW$ cặp (query, key).

Vì sao thông tin vẫn lan xa? Trạng thái ẩn $h_i^{(k)}$ ở layer k attend các $h_j^{(k-1)}$ với $j \in [i-W, i]$, mà mỗi $h_j^{(k-1)}$ lại đã tích hợp thông tin từ $[j-W, j]$ ở layer trước, v.v. Vì vậy sau k layer, token i có thể "thấy gián tiếp" token đầu vào ở khoảng cách tới $k \times W$:

$$\boxed{\text{tầm với lý thuyết} = k \times W = 32 \times 4096 = 131{,}072 \ (\approx 131\text{K})}$$

Đây là *trần lý thuyết*, như paper nói "theoretical attention span of approximately 131K tokens"; paper không đo khả năng thực sự dùng thông tin ở khoảng cách đó (ví dụ không có thí nghiệm needle-in-a-haystack). Trong thực tế thông tin bị pha loãng qua mỗi bước truyền.

Minh họa với $W = 3$: token ở vị trí 10 ở layer 1 thấy các vị trí 7-10; ở layer 2 thấy hidden state của vị trí 7-10, mà vị trí 7 ở layer 1 thấy 4-7, nên tầm với trực tiếp lan tới vị trí 4 sau 2 layer, tức $2 \times 3 = 6$ bước lùi (hình 1 của paper).

Về triển khai: chỉnh sửa trong FlashAttention và xFormers cho tốc độ nhanh gấp **2 lần** baseline attention thường với độ dài chuỗi 16K và $W = 4096$. Kiểm tra (tự tính): số cặp (query,key) nhân quả đầy đủ $n(n+1)/2 \approx 134.2$M; với SWA $\approx nW - W^2/2 \approx 58.7$M; tỉ lệ ≈ 2.29 lần, cùng bậc với con số 2x của paper.

3.4 Rolling Buffer Cache

Vì phạm vi attention là W, ta chỉ cần giữ KV của W token gần nhất. Cache có kích thước cố định W; K,V của vị trí i lưu vào ô i mod W. Khi $i \geq W$, ghi đè giá trị cũ nhất.

Ví dụ $W = 4$ (Hình 2): vị trí 0,1,2,3 vào ô 0,1,2,3; vị trí 4 ghi đè ô 0 (vì 4 mod $4 = 0$, token 0 đã ra khỏi cửa sổ), vị trí 5 ghi đè ô 1, v.v.

```
class RollingKVCache:
    def __init__(self, W, n_layers, n_kv, d_head):
        self.W = W
        self.K = zeros(n_layers, W, n_kv, d_head)
        self.V = zeros(n_layers, W, n_kv, d_head)

    def write(self, layer, pos, k, v):
```

```
        slot = pos % self.W          # vị trí i lưu ở ô i mod W
        self.K[layer, slot] = k      # ghi đè token cũ nhất khi pos >= W
        self.V[layer, slot] = v

    def read(self, layer, pos):
        n = min(pos + 1, self.W)     # tối đa W token hợp lệ
        # trả về K,V của n token gần nhất (cần ghi nhớ thứ tự theo pos,
        # vì RoPE/mask dựa trên vị trí tuyệt đối thật của từng ô)
        return self.K[layer, :n], self.V[layer, :n]
```

(Pseudo-code tự viết; paper chỉ mô tả ý tưởng, không đưa code.) Lưu ý: ô cache không còn theo thứ tự thời gian sau khi vòng lại, nên cần theo dõi vị trí thật của từng ô để áp mask/positional encoding đúng.

Hiệu quả bộ nhớ: trên chuỗi 32k token, giảm bộ nhớ cache **8 lần** ($32768/4096 = 8$) mà không ảnh hưởng chất lượng theo paper.

3.5 Pre-fill và chunking

Khi sinh, các token sinh ra phải tuần tự, nhưng **prompt đã biết trước**, nên có thể nạp (pre-fill) KV-cache cho prompt song song. Nếu prompt quá dài, cắt thành các **chunk** (chọn kích thước chunk $= W$) và pre-fill lần lượt. Với mỗi chunk, attention gồm hai phần: (a) attention *vào cache* (các token của chunk trước, theo cửa sổ trượt), (b) attention *trong chính chunk* với mask nhân quả. Hình 3: chia "The cat sat on the mat and saw the dog go to" thành 3 chunk; ở chunk thứ ba ("the dog go to"), nó attend chính nó bằng mask nhân quả, attend cache qua cửa sổ trượt, và không attend các token đã ra khỏi cửa sổ.

---

4. Thí nghiệm và kết quả chính

Paper **chạy lại mọi benchmark bằng pipeline đánh giá của chính họ** cho công bằng (nên số Llama có thể khác số trong paper Llama). Khác biệt giao thức: MBPP dùng tập con đã kiểm bằng tay; TriviaQA không cung cấp ngữ cảnh Wikipedia.

Nhóm benchmark: suy luận thường thức 0-shot (HellaSwag, WinoGrande, PIQA, SIQA, OpenbookQA, ARC-e/c, CommonsenseQA); kiến thức thế giới 5-shot (NQ, TriviaQA); đọc hiểu 0-shot (BoolQ, QuAC); toán GSM8K 8-shot maj@8 và MATH 4-shot maj@4; code HumanEval 0-shot, MBPP 3-shot; tổng hợp MMLU 5-shot, BBH 3-shot, AGI Eval 3-5-shot.

4.1 So với Llama (Bảng 2, base models)

Mô hình	MMLU	HellaSwag	WinoG	PIQA	Arc-e	Arc-c	NQ	TriviaQA	HumanEval	MBPP	MATH	GSM8K
LLaMA 2 7B	44.4	77.1	69.5	77.9	68.7	43.2	24.7	63.8	11.6	26.1	3.9	16.0
LLaMA 2 13B	55.6	80.7	72.9	80.8	75.2	48.8	29.0	69.6	18.9	35.4	6.0	34.3
Code-Llama 7B (fine-tuned)	36.9	62.9	62.3	72.8	59.4	34.5	11.0	34.9	31.1	52.5	5.2	20.8
Mistral 7B	**60.1**	**81.3**	**75.3**	**83.0**	**80.0**	**55.5**	28.8	**69.9**	**30.5**	47.5	**13.1**	**52.2**

Đơn vị %. Mistral 7B vượt Llama 2 13B ở hầu hết cột; ở NQ gần ngang (28.8 vs 29.0). Đặc biệt MMLU +4.5 điểm so với 13B, GSM8K 52.2 so với 34.3, MATH 13.1 so với 6.0. Paper nêu Mistral gần với Code-Llama

7B về code (30.5 vs 31.1 HumanEval) mà không hy sinh benchmark phi-code, đồng thời Code-Llama 7B kém trên mọi benchmark ngoài code.

4.2 "Kích thước tương đương" (Hình 5)

Paper tính "equivalent model size" của họ Llama 2: trên suy luận, đọc hiểu và MMLU/STEM, Mistral 7B tương đương Llama 2 **lớn hơn hơn 3 lần**; trên benchmark kiến thức chỉ **1.9 lần**, được giải thích (giả thuyết) là do số tham số hạn chế dung lượng kiến thức có thể lưu.

4.3 Instruction finetuning (Bảng 3)

Chỉ fine-tune trên các dataset instruction công khai của Hugging Face, "không dữ liệu độc quyền hay mẹo huấn luyện":

Mô hình	Chatbot Arena ELO	MT-Bench
WizardLM 13B v1.2	1047	7.2
Mistral 7B Instruct	1031	6.84 ± 0.07
Llama 2 13B Chat	1012	6.65
Vicuna 13B	1041	6.57
Llama 2 7B Chat	985	6.27
Vicuna 7B	997	6.17
Alpaca 13B	914	4.53

Đánh giá người độc lập trên llmboxing.com (tính đến 6/10/2023): đầu ra Mistral 7B được ưa thích 5020 lần so với 4143 lần của Llama 2 13B.

4.4 Guardrails

Với system prompt đề xuất, mô hình từ chối 100% trong 175 prompt không an toàn; MT-Bench: không system prompt 6.84 ± 0.07, system prompt của Mistral 6.58 ± 0.05, của Llama 2 6.38 ± 0.07. Tự kiểm duyệt nội dung (self-reflection): precision 99.4% tại recall 95.6% trên tập tự dựng.

---

5. Phân tích / ablation

Đáng nói thẳng: **paper không có ablation nào** cho GQA, SWA hay rolling buffer. Không có bảng "với/không có SWA", không có đo độ chính xác theo độ dài ngữ cảnh, không có so sánh GQA với MHA, và không có thông tin dữ liệu huấn luyện. Những gì được hỗ trợ bằng số liệu:

- Chất lượng chung: Bảng 2 (so Llama).
- Tốc độ SWA: tăng gấp 2 lần ở chuỗi 16K, $W = 4096$ nhờ sửa đổi FlashAttention/xFormers.
- Bộ nhớ cache giảm 8 lần ở 32k token (chỉ phép chia, không đo chất lượng riêng; câu "without impacting the model quality" là tuyên bố của tác giả).
- Điểm yếu về kiến thức (1.9x vs 3x) là quan sát có số liệu, nguyên nhân là giả thuyết.

Vì vậy mọi đóng góp riêng của GQA/SWA đối với điểm số benchmark là **không tách được** từ paper; chất lượng cao hơn Llama 2 có thể đến từ dữ liệu và huấn luyện (không công bố) chứ không nhất thiết từ attention.

---

6. Giới hạn

Do tác giả nêu:

- Dung lượng kiến thức hạn chế do số tham số nhỏ (kiến thức chỉ nén 1.9x).
- Mô hình Instruct chỉ là "demonstration đơn giản và sơ bộ", chưa có cơ chế moderation tích hợp ngoài system prompt.

Nhận xét bổ sung:

- Không có dữ liệu/ablation, không mô tả huấn luyện → khó tái lập và khó quy kết nguyên nhân.
- Tầm với 131K chỉ là trần lý thuyết; mô hình không được kiểm chứng khả năng truy hồi thông tin ở xa (context_len trong Bảng 1 chỉ 8192).
- SWA làm mất truy cập trực tiếp tới token cũ (kể cả token đầu, nơi thường chứa "attention sink" mà StreamingLLM, bài 55, chỉ ra là quan trọng); phải dựa vào lan truyền gián tiếp qua nhiều layer, có thể mất chi tiết chính xác (ví dụ chép lại một chuỗi dài từ rất xa).
- Rolling buffer làm hỏng khả năng tái sử dụng prefix cache theo kiểu RadixAttention (bài 47) vì cache bị ghi đè.
- Số liệu Llama tự chạy lại nên có thể lệch so với số công bố của Meta; không có khoảng tin cậy.
- Chỉ có model 7B; không rõ cách các kỹ thuật này scale.

---

7. Liên hệ với các bài khác

- **05 LLaMA:** nền kiến trúc; Mistral so sánh trực tiếp với Llama 1/2 và nhắc "Compared to Llama, it introduces a few changes".
- **15 GQA, 14 MQA:** nguồn gốc của kỹ thuật nhóm đầu K/V. **16 MLA (DeepSeek-V2):** cách khác để nén KV-cache (nén low-rank), mạnh hơn GQA.
- **17 Longformer:** nguồn gốc của sliding-window attention; paper cũng trích Sparse Transformer (Child et al.). GPT-3 (bài 02) xen kẽ attention dày và dải băng cục bộ, còn Mistral dùng cửa sổ ở mọi layer.
- **44/45 FlashAttention:** Mistral cần chỉnh kernel FlashAttention để hỗ trợ SWA.
- **46 PagedAttention (vLLM):** paper chọn vLLM làm server; PagedAttention quản lý KV-cache theo trang, bổ sung cho rolling buffer.
- **55 StreamingLLM / 56 H2O:** hai hướng khác để giới hạn KV-cache (giữ attention sink + cửa sổ; loại bỏ token ít quan trọng).
- **18-26 (attention tuyến tính / SSM / hybrid):** thay vì cắt cửa sổ, các mô hình này nén quá khứ vào state cố định; SWA là phương án đơn giản giữ softmax attention nhưng bị chặn.
- **28 Mixtral:** cùng nhóm tác giả, thêm Mixture-of-Experts lên nền kiến trúc này.

---

8. Gợi ý áp dụng

8.1 Phân tích chi phí KV-cache (tự tính)

$$\boxed{\text{KV bytes} = 2 \cdot L \cdot n_{kv} \cdot d_{head} \cdot T_{\text{cache}} \cdot b}$$

với $T_{\text{cache}} = \min(T, W)$ khi dùng SWA + rolling buffer, $b = 2$ (fp16/bf16).

Mistral 7B ($L = 32,\ n_{kv} = 8,\ d_{head} = 128$): 131,072 B = **128 KiB/token** với GQA.

Cấu hình, $T = 32{,}768$	Cache
MHA, full attention (kiểu LLaMA-7B: $n_{kv} = 32$)	16 GiB

Cấu hình, $T = 32{,}768$	Cache
GQA, full attention	4 GiB
GQA + SWA ($W = 4096$, rolling buffer)	**512 MiB**

Tức riêng SWA giảm 8x (khớp paper), GQA thêm 4x, tổng 32x so với MHA full attention. (Lưu ý: so sánh đến từ phép tính, không phải đo.)

Chi phí tính toán attention: mỗi token mới tính $\min(i, W)$ tích vô hướng ($\times$ số đầu) thay vì i. Ở chuỗi $n = 16{,}384$, $W = 4096$: ≈ 58.7M cặp so với ≈ 134.2M, tỉ lệ ≈ 2.3.

8.2 Lưu ý về cửa sổ attention

Cửa sổ chỉ giảm số vị trí nhìn trực tiếp khi chuỗi dài hơn cửa sổ. Trong VLM đặt token ảnh ở đầu chuỗi, một cửa sổ nhỏ có thể khiến token sinh sau không còn truy cập trực tiếp ảnh; ảnh hưởng thực tế phụ thuộc kiến trúc và cần đo trên tác vụ cụ thể.

9. Thí nghiệm gợi ý

So sánh nhiều độ rộng cửa sổ về bộ nhớ, độ trễ và chất lượng. Với tác vụ truy hồi, kiểm tra khả năng sao chép thông tin ở khoảng cách lớn hơn W nhưng nhỏ hơn trường nhìn lý thuyết của nhiều tầng; đường đi lý thuyết không bảo đảm thông tin được bảo toàn.

10. Tóm tắt công thức / khái niệm

Khái niệm	Công thức / ý nghĩa
GQA	$h_q = 32$ đầu query, $h_{kv} = 8$ đầu K/V, nhóm $g = 4$
SWA	Token i attend $j \in [i - W, i]$, $W = 4096$
Tầm với lý thuyết	$k \times W = 32 \times 4096 \approx 131$K
Rolling buffer	Ghi KV vị trí i vào ô i mod W; cache kích thước cố định W
Giảm cache	8x ở 32k token (SWA); +4x từ GQA so với MHA (tự tính)
Pre-fill + chunk	Nạp cache cho prompt theo chunk cỡ W; chunk attend cache (cửa sổ) + chính nó (mask nhân quả)
KV bytes	$2\, L\, n_{kv}\, d_{head}\, \min(T, W)\, b$ (tự tính)
Mistral 7B	32 layer, dim 4096, FFN 14336, head_dim 128, vocab 32000, ≈7.2B tham số (tự tính)
Tốc độ SWA	~2x ở chuỗi 16K, $W = 4096$ (FlashAttention/xFormers đã sửa)

---

11. Kết luận ngắn

Mistral 7B là minh chứng cho quan điểm "tối ưu chi phí suy luận ngay trong thiết kế": GQA thu nhỏ KV-cache theo chiều đầu, SWA + rolling buffer thu nhỏ theo chiều độ dài, và kết quả benchmark vượt Llama 2 13B. Điểm cần nhớ khi đọc kỹ: paper không có ablation hay thông tin dữ liệu, nên không tách được đóng góp từng thành phần.

Nguồn gốc: https://arxiv.org/abs/2310.06825

Bài 07

Training language models to follow instructions with human feedback (2022)

Tác giả: Long Ouyang, Jeff Wu, Xu Jiang, Diogo Almeida, Carroll L. Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, John Schulman, Jacob Hilton, Fraser Kelton, Luke Miller, Maddie Simens, Amanda Askell, Peter Welinder, Paul Christiano, Jan Leike, Ryan Lowe (OpenAI).
Venue / arXiv: arXiv:2203.02155 (v1, 4/3/2022). Paper không nêu venue trong bản đã đọc.
Nguồn: https://arxiv.org/abs/2203.02155
Chủ đề: RLHF, alignment, supervised fine-tuning (SFT), reward model (RM), PPO, KL penalty, "alignment tax", PPO-ptx.
Vị trí trong lộ trình: Đọc sau GPT-3 (02) và Kaplan/Chinchilla (03–04) vì InstructGPT chính là GPT-3 được fine-tune tiếp. Bài này thêm *bước hậu huấn luyện* biến một mô hình "đoán token tiếp theo" thành mô hình "làm theo chỉ dẫn". Phần LoRA (08) là kỹ thuật fine-tune rẻ có thể dùng thay cho full fine-tune trong các bước kiểu SFT. Hướng đi tiếp: các bài sau trong lộ trình chủ yếu là kiến trúc/suy luận, nên bài này là điểm duy nhất về hậu huấn luyện.

> Ghi chú diễn giải bằng tiếng Việt, không phải bản dịch. Các số liệu "tự tính" là ví dụ minh họa, không phải kết quả trực tiếp của bài báo.

1. Bối cảnh

Vấn đề. GPT-3 (02) được huấn luyện để *dự đoán token tiếp theo trên văn bản internet*. Mục tiêu đó khác với mục tiêu người dùng thực sự muốn: "làm theo chỉ dẫn của tôi một cách hữu ích và an toàn". Paper gọi đây là **misaligned** (lệch mục tiêu): mô hình có thể bịa sự thật, sinh nội dung độc hại, hoặc đơn giản là không làm theo chỉ dẫn (ví dụ: bạn yêu cầu viết truyện, nó lại viết thêm nhiều yêu cầu tương tự, như ví dụ tiếng Pháp trong Figure 8 của paper).

Hướng cũ. (i) *Prompt engineering / few-shot*: ghép vài ví dụ vào đầu prompt để "gợi" mô hình. (ii) *Instruction tuning trên dataset NLP công khai* (FLAN, T0): fine-tune trên nhiều tác vụ NLP có kèm chỉ dẫn. (iii) *RLHF* đã được dùng cho robot/Atari (Christiano 2017) và cho tóm tắt văn bản (Ziegler 2019, Stiennon 2020). Paper này là "ứng dụng trực tiếp RLHF lên một *phân phối rộng* các tác vụ ngôn ngữ".

Ba thuật ngữ nền tảng (giải thích cho người mới):

- **Alignment (căn chỉnh):** làm cho mô hình hành xử đúng ý người dùng. Paper dùng bộ tiêu chí của Askell et al.: **helpful** (hữu ích), **honest** (trung thực), **harmless** (vô hại).
- **RLHF (Reinforcement Learning from Human Feedback):** dùng phán đoán của con người làm *tín hiệu thưởng* để huấn luyện bằng học tăng cường. Con người không viết đáp án chuẩn; họ chỉ *so sánh* các đáp án mà mô hình tự sinh.
- **Policy (chính sách):** trong ngữ cảnh này chính là mô hình ngôn ngữ $\pi(y \mid x)$: cho prompt x, nó phân phối xác suất trên câu trả lời y.

2. Ý tưởng chính

Paper làm ba bước nối tiếp, bắt đầu từ GPT-3 đã pretrain:

1. **SFT:** thuê người viết câu trả lời mẫu cho các prompt, fine-tune GPT-3 bằng học có giám sát (cross-entropy thông thường) trên các cặp (prompt, đáp án mẫu).

2. **Reward model (RM):** cho mô hình SFT sinh nhiều đáp án cho cùng một prompt, nhờ người xếp hạng; huấn luyện một mạng khác để chấm điểm vô hướng sao cho đáp án được ưa thích có điểm cao hơn.
3. **PPO:** dùng điểm RM làm phần thưởng, tối ưu chính sách (mô hình SFT) bằng thuật toán PPO, kèm *phạt KL* để chính sách không đi quá xa mô hình SFT, và (ở biến thể PPO-ptx) trộn thêm gradient của hàm mất mát pretraining để giảm "quên".

Kết quả gây chú ý nhất: mô hình 1.3B tham số sau RLHF được người đánh giá ưa thích hơn GPT-3 175B (ít hơn >100 lần tham số), trong khi kiến trúc giống hệt nhau.

3. Phương pháp chi tiết

3.1 Dữ liệu và nhãn viên

- **Prompt:** chủ yếu là prompt do khách hàng gửi qua OpenAI API Playground tới các bản InstructGPT sớm (đã được SFT trên một phần dữ liệu demo); thêm prompt do nhãn viên tự viết để "mồi" ban đầu (ba kiểu: *plain*, *few-shot*, *user-based*). Loại trùng lặp theo tiền tố chung, giới hạn 200 prompt/người dùng, tách train/valid/test *theo user ID*, lọc thông tin cá nhân (PII). Dữ liệu >96% là tiếng Anh.
- **Phân bố loại tác vụ (Table 1, trên API prompt dataset):** Generation 45.6%, Open QA 12.4%, Brainstorming 11.2%, Chat 8.4%, Rewrite 6.6%, Summarization 4.2%, Classification 3.5%, Other 3.5%, Closed QA 2.6%, Extract 1.9%.
- **Kích thước tập dữ liệu (Table 6, số prompt):**

Tập	Train (labeler + customer)	Valid
SFT	11,295 + 1,430 (~13k)	1,550 + 103
RM	6,623 + 26,584 (~33k)	3,488 + 14,399
PPO (không có nhãn người)	31,144 (chỉ customer)	16,185

- **Nhãn viên:** ~40 contractor (Upwork, ScaleAI), chọn qua bài kiểm tra sàng lọc về độ nhạy với các nhóm nhân khẩu và khả năng nhận diện nội dung có hại. **Độ đồng thuận giữa nhãn viên:** 72.6 ± 1.5% (nhãn viên train), 77.3 ± 1.3% (nhãn viên held-out); so sánh: 73 ± 4% giữa researcher trong Stiennon et al. 2020.
- Khi huấn luyện, ưu tiên *hữu ích*; khi đánh giá cuối, nhãn viên được yêu cầu ưu tiên *trung thực và vô hại*.

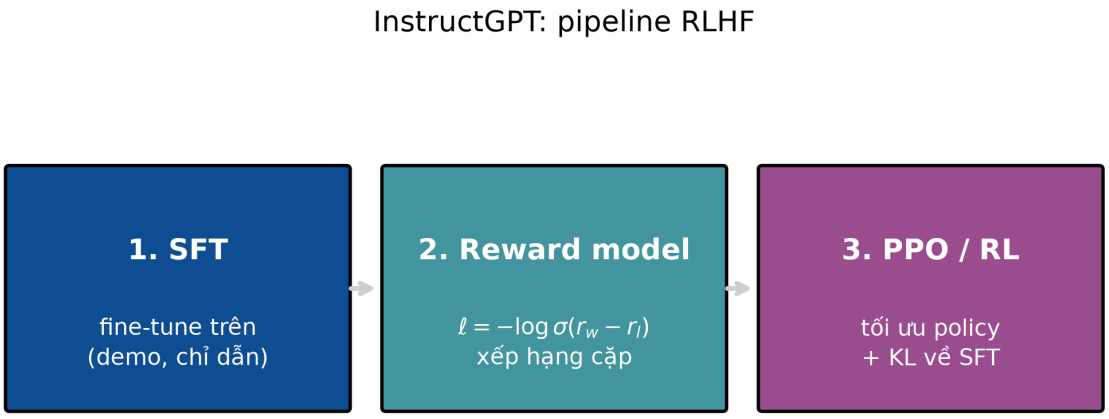
3.2 Bước 1: SFT

Fine-tune GPT-3 (3 cỡ: 1.3B, 6B, 175B) trên demonstration của nhãn viên bằng mục tiêu ngôn ngữ chuẩn.

- 16 epoch, cosine LR decay xuống 10%, residual dropout 0.2, không warmup.
- LR: 9.65e-6, batch 32 (1.3B và 6B); 5.03e-6, batch 8 (175B).
- Điểm đáng chú ý: mô hình SFT **overfit validation loss sau 1 epoch**, nhưng huấn luyện nhiều epoch hơn vẫn **tăng điểm RM và điểm ưa thích của người**. Vì vậy, họ chọn checkpoint theo *điểm RM trên tập valid* (paper nói chỉ số này dự báo ưa thích của người tốt hơn validation loss), không theo loss.

3.3 Bước 2: Reward model

Kiến trúc. Lấy mô hình SFT, **bỏ lớp unembedding cuối** (lớp ánh xạ ra từ vựng) và thay bằng đầu ra vô hướng: $r_\theta(x, y) \in \mathbb{R}$. Chỉ dùng RM cỡ **6B** cho mọi cỡ policy (RM 175B huấn luyện không ổn định, và tốn kém khi dùng làm value function).



Ba giai đoạn căn chỉnh theo phản hồi người

Ba giai đoạn RLHF: SFT → reward model (xếp hạng cặp) → PPO (+KL). (minh hoạ, tự tính)

Dữ liệu so sánh. Với mỗi prompt, người xem $K \in [4, 9]$ đáp án và xếp hạng. Từ đó có $\binom{K}{2}$ cặp so sánh. Ví dụ $K = 4 \Rightarrow 6$ cặp; $K = 9 \Rightarrow 36$ cặp.

Hàm mất mát (Eq. 1):

$$\boxed{\text{loss}(\theta) = -\frac{1}{\binom{K}{2}}\,\mathbb{E}_{(x,y_w,y_l)\sim D}\Big[\log \sigma(r_\theta(x, y_w) - r_\theta(x, y_l))\Big]}$$

trong đó y_w là đáp án được ưa thích ("winner"), y_l là đáp án kém hơn ("loser"), σ là sigmoid. Ý nghĩa: hiệu điểm $r_w - r_l$ được coi là *log-odds* rằng người chọn y_w (mô hình Bradley–Terry), và loss là cross-entropy với nhãn "người chọn y_w".

Ví dụ số nhỏ (tự tính):

$r_w - r_l$	$\sigma(\cdot)$	loss $= -\log\sigma$
+1.0	0.731	0.313
0.0	0.500	0.693
−1.0	0.269	1.313

Nghĩa là RM bị phạt nặng khi xếp ngược thứ tự người chọn, và loss giảm khi khoảng cách điểm đúng chiều tăng.

Mẹo kỹ thuật quan trọng. Nếu xáo trộn mọi cặp so sánh thành một dataset phẳng, các cặp cùng prompt rất tương quan, nên chỉ một epoch đã overfit (mỗi đáp án xuất hiện trong $K - 1$ cập nhật gradient). Thay vào đó, họ đặt **toàn bộ $\binom{K}{2}$ cặp của một prompt vào cùng một batch element**. Lợi ích: chỉ cần **một lần forward cho mỗi đáp án** (K lần) thay vì $2\binom{K}{2}$ lần, và validation accuracy/log-loss tốt hơn hẳn.

Huấn luyện RM: 1 epoch, LR 9e-6 cosine (xuống 10%), batch 64 *prompt* (tức tối đa $64 \times 36 = 2{,}304$ cặp). RM khá không nhạy với LR (đổi 50% cho kết quả tương tự) nhưng rất nhạy với số epoch (nhiều epoch → overfit nhanh). Khởi tạo từ GPT-3 6B đã fine-tune trên nhiều bộ NLP công khai (vì lý do lịch sử; khởi tạo

từ GPT-3 hay SFT cho kết quả tương tự). Cuối cùng RM được cộng một bias sao cho đáp án demo của nhãn viên có điểm trung bình 0 trước khi chạy RL (vì loss bất biến theo phép tịnh tiến điểm).

3.4 Bước 3: PPO với KL penalty và PPO-ptx

Môi trường. Là một *bandit*: lấy một prompt ngẫu nhiên, policy sinh cả câu trả lời, RM cho điểm, episode kết thúc. Không có phần thưởng trung gian. (Tuy vậy, phạt KL được tính *theo từng token*.)

PPO (Proximal Policy Optimization): thuật toán policy-gradient cắt tỉ số xác suất (clip) để mỗi lần cập nhật không làm policy thay đổi quá nhiều. Value function (ước lượng phần thưởng kỳ vọng, dùng để giảm phương sai) khởi tạo từ chính RM 6B.

Hàm mục tiêu (Eq. 2):

$$\text{objective}(\phi) = \mathbb{E}_{(x,y)\sim D_{\pi_\phi^{RL}}}\Big[r_\theta(x,y) - \beta \log \frac{\pi_\phi^{RL}(y \mid x)}{\pi^{SFT}(y \mid x)}\Big] + \gamma\, \mathbb{E}_{x\sim D_{pretrain}}\big[\log \pi_\phi^{RL}(x)\big]$$

Giải nghĩa từng phần:

- $r_\theta(x,y)$: điểm RM, cái ta muốn **tối đa hoá**.
- $-\beta \log \frac{\pi^{RL}}{\pi^{SFT}}$: **phạt KL**, tức một ước lượng lấy mẫu của $\beta \cdot \mathrm{KL}(\pi^{RL} \| \pi^{SFT})$ lấy dấu âm. Nó giữ policy gần mô hình SFT để không "khai thác" lỗ hổng của RM (**reward over-optimization**: RM chỉ là xấp xỉ sở thích người; tối ưu quá mạnh sẽ cho đáp án điểm cao nhưng thực chất tệ).
- $\gamma\, \mathbb{E}[\log \pi^{RL}(x)]$: gradient *pretraining* trộn vào, để giữ khả năng NLP chung. $\gamma = 0$ là "PPO" thuần; $\gamma > 0$ là **PPO-ptx**. "InstructGPT" trong paper mặc định chỉ PPO-ptx.

Siêu tham số (Phụ lục C.3–C.4):

- Khởi tạo policy: GPT-3 + SFT 2 epoch trên demo, có trộn 10% dữ liệu pretraining. Cũng dùng chính mô hình này tính KL.
- $\beta = 0.02$; $\gamma = 27.8$; dữ liệu pretraining dùng gấp **8 lần** số episode RL.
- 256k episode (~31k prompt duy nhất); batch mỗi vòng 512, minibatch 64 (tức 8 minibatch, 1 inner epoch); clip ratio 0.2; nhiệt độ lấy mẫu rollout 1; không chiết khấu khi tính GAE.
- LR không đổi, warmup 10 vòng đầu (bắt đầu 1/10 LR đỉnh); EMA trọng số với decay 0.992; LR value function 9e-6 (policy 1.3B, 6B), 5e-6 (175B).
- Chi phí tính toán (paper): SFT 175B ≈ 4.9 petaflops/s-days; PPO-ptx 175B ≈ 60 petaflops/s-days; so với GPT-3 pretrain 3,640 petaflops/s-days.

3.5 Pseudo-code toàn quy trình

```
# Đầu vào: GPT-3 pretrain, tập prompt P, nhãn viên H
# ---- Bước 1: SFT ----
D_demo  = { (x, y*) : x ~ P, y* = H.viết_đáp_án(x) }          # ~13k prompt
pi_sft  = finetune(GPT3, D_demo, loss = -log p(y*|x), epochs=16)
          # chọn checkpoint theo điểm RM trên valid, không theo valid loss

# ---- Bước 2: Reward model ----
for x in P_rm:                                                 # ~33k prompt
    ys = [pi_sft.sample(x) for _ in range(K)]                  # K in [4, 9]
    rank = H.xếp_hạng(x, ys)
RM = pi_sft_without_unembedding + linear_head → scalar         # 6B
for batch of 64 prompts:                                       # 1 epoch
    r = [RM(x, y) for y in ys]                                 # K forward / prompt
    loss = -mean over C(K,2) pairs (w,l) of  log sigmoid(r[w]-r[l])
    update RM
```

```
RM.bias -= mean(RM(x, demo y*))                                  # chuẩn hoá: demo có
↪  điểm TB = 0

# ---- Bước 3: PPO(-ptx) ----
pi = copy(pi_sft_2epoch_with_10%_pretrain); V = copy(RM)         # value init từ RM
repeat until 256k episodes:
    X = sample 512 prompts from P_ppo (không nhãn người)
    Y = pi.sample(X)                                             # rollout, T = 1
    reward_t = [per-token  -beta * (log pi(y_t) - log pi_sft(y_t))]
    reward_T += RM(X, Y)                                         # điểm RM cộng ở token
    ↪  cuối
    A = GAE(reward, V)                                           # lambda, gamma_disc = 1
    for 8 minibatch(64):
        g_ppo = grad PPO_clip_loss(pi, A, clip=0.2)
        g_ptx = gamma * grad( -log pi(x_pretrain) )              # x_pretrain: 8x số
        ↪  episode
        update pi with g_ppo + g_ptx;  update V
```

(Việc cộng phạt KL theo token và điểm RM ở token cuối là cách triển khai chuẩn của Stiennon et al. mà paper nói "theo cùng cách"; paper diễn đạt: "per-token KL penalty from the SFT model at each token".)

4. Thí nghiệm và kết quả chính

Mô hình & baseline. Ba cỡ 1.3B / 6B / 175B (kiến trúc GPT-3). So sánh: GPT-3, GPT-3 + prompt few-shot ("prompted"), SFT, PPO, PPO-ptx, và GPT-3 175B fine-tune trên FLAN/T0 (~1 triệu mẫu mỗi bộ). Baseline để tính tỉ lệ thắng: SFT 175B.

Đánh giá chính (ưa thích của người trên prompt API held-out):

Phát hiện (paper)	Số liệu
175B InstructGPT ưa thích hơn 175B GPT-3	85 ± 3%
175B InstructGPT ưa thích hơn GPT-3 few-shot	71 ± 4%
1.3B PPO-ptx so với 175B GPT-3	1.3B được ưa thích hơn (Figure 1)
175B InstructGPT ưa thích hơn FLAN / T0	78 ± 4% / 79 ± 4%
Win rate (so với SFT baseline) của InstructGPT vs T0, FLAN (Mục 1)	73.4 ± 2% vs 26.8 ± 2% (T0) và 29.8 ± 2% (FLAN)
Hallucination trên tác vụ closed-domain (API)	21% (InstructGPT) vs 41% (GPT-3)
Độc hại (RealToxicityPrompts, khi được nhắc "respectful")	ít hơn ~25% so với GPT-3
TruthfulQA	trung thực và có thông tin ~gấp đôi GPT-3 (paper); riêng 1.3B PPO-ptx kém nhẹ hơn GPT-3 cùng cỡ

Các điểm khác:

- Thứ tự cải thiện nhìn thấy trong Figure 1: GPT-3 < GPT-3 prompted < SFT < PPO (≈ PPO-ptx). Thêm gradient pretraining không làm đổi điểm ưa thích nhiều.
- **Tổng quát hoá sang nhãn viên khác:** nhãn viên held-out (không tạo dữ liệu huấn luyện) cũng ưa InstructGPT với tỉ lệ tương tự. RM chạy 5-fold theo nhóm nhãn viên đạt **69.6 ± 0.9%** trên nhóm bị giữ lại so với **72.4 ± 0.4%** trên nhóm đã thấy.
- **Tổng quát hoá ngoài phân phối:** làm theo chỉ dẫn tiếng khác và hỏi-đáp/tóm tắt code, dù dữ liệu rất hiếm (định tính, không có số).
- **Bias:** không cải thiện đáng kể trên Winogender và CrowS-Pairs.

- **Chi phí:** RLHF rẻ so với pretrain (60 so với 3,640 petaflops/s-days cho 175B); paper kết luận tăng cường alignment "hiệu quả chi phí hơn" việc làm mô hình lớn hơn 100 lần, *trên phân phối khách hàng của họ*.

5. Phân tích / ablation

- **Alignment tax và PPO-ptx.** PPO thuần gây suy giảm trên SQuADv2, DROP, HellaSwag, dịch Pháp→Anh WMT15. Trộn gradient pretraining (PPO-ptx) khắc phục phần lớn, thậm chí vượt GPT-3 trên HellaSwag; vẫn còn kém GPT-3 trên DROP, SQuADv2 và dịch.
- **Pretraining mix tốt hơn tăng KL.** Tăng hệ số KL làm giảm mạnh reward valid và không bao giờ phục hồi hoàn toàn DROP/SQuAD; ngược lại tồn tại một giá trị γ vừa đảo ngược suy giảm vừa chỉ giảm reward rất ít.
- **Lượng dữ liệu pretraining (E.11):** tỉ lệ 4 thì log-prob pretraining thường tăng (tức xấu đi) trong quá trình học; 32 cho Likert tốt hơn nhưng thời gian huấn luyện tăng "vài lần"; chọn 8 (gấp đôi thời gian so với không trộn).
- **Hạn chế số episode:** với 1.3B, huấn luyện >256k episode không giúp (khi có pretraining mix).
- **Batch size PPO** (thử 64–1024 trên 1.3B): 512 tốt nhất theo đánh giá của người; minibatch thử 8, 16, 32, 64.
- **Kích thước RM:** RM 6B đủ ổn định và cho PPO mạnh tương đương; RM 175B có thể đạt loss thấp hơn nhưng không ổn định.
- **FLAN/T0 kém hơn:** lý do paper đưa ra: (1) tác vụ NLP công khai chủ yếu là phân loại/QA (~18% ứng dụng thật), trong khi sinh tự do + brainstorm chiếm ~57%; (2) khó đạt độ đa dạng input như người dùng thật.
- **Hành vi sai còn lại (định tính):** chấp nhận tiền đề sai, rào đón quá mức (paper *nghi ngờ* do nhãn viên được dặn thưởng "khiêm tốn tri thức", RM học theo), giảm chất lượng khi có nhiều ràng buộc tường minh.

6. Giới hạn

Do tác giả nêu:

- Hành vi bị quyết định bởi ~40 nhãn viên chủ yếu nói tiếng Anh ở Mỹ/Đông Nam Á, không đại diện cho mọi người dùng; hầu hết so sánh chỉ được 1 người gán nhãn (vì chi phí).
- Mô hình chưa hoàn toàn an toàn/căn chỉnh; vẫn bịa sự thật, sinh nội dung độc hại/tình dục/bạo lực khi không bị nhắc; **nghiêm trọng nhất**: nó làm theo chỉ dẫn kể cả khi có hại, và khi được yêu cầu "thiên kiến tối đa" thì độc hơn GPT-3 cùng cỡ.
- Alignment chỉ nhắm tới sở thích của một nhóm cụ thể (nhãn viên, nhà nghiên cứu, khách hàng API), không phải "giá trị con người" nói chung.
- Pretraining mix không loại bỏ hết regression và có thể làm tăng hành vi xấu có sẵn trong dữ liệu pretrain.

Nhận xét bổ sung:

- Phần thưởng là một proxy học được (RM chỉ ~70% đồng thuận với người), nên mọi cải thiện đo bằng chính nhãn viên có nguy cơ đo "độ hợp gu nhãn viên" hơn là chất lượng thực.
- Chỉ tiết lộ một thiết lập (GPT-3, API prompt); không có ablation riêng cho việc bỏ SFT hoặc đổi thuật toán RL, nên không biết PPO có thực sự cần thiết (paper tự nhắc các phương pháp đơn giản hơn như expert iteration, behavior cloning trên một phần dữ liệu so sánh là hướng chưa thử).
- PPO cần chạy đồng thời nhiều mô hình (policy, SFT tham chiếu, RM, value), bộ nhớ lớn; nên đòi hỏi bộ nhớ lớn.

7. Liên hệ với các bài khác

- **02 GPT-3:** mô hình khởi đầu; InstructGPT dùng đúng kiến trúc GPT-3, chỉ khác dữ liệu/huấn luyện hậu kỳ. So sánh "1.3B vs 175B" của paper này là một ví dụ rằng *dữ liệu/mục tiêu đúng có thể quan trọng hơn quy mô* (đối chiếu với 03–04 về scaling laws, vốn nói về loss ngôn ngữ chứ không phải sở thích).
- **05 LLaMA, 06 Mistral:** các mô hình nền mở; bản "chat/instruct" của chúng được tạo bằng quy trình cùng họ với bài này (hiểu biết ngoài paper).
- **08 LoRA:** có thể thay full fine-tune trong bước SFT để rẻ hơn rất nhiều.
- **48–50 Speculative decoding/Medusa/EAGLE:** RLHF thay đổi phân phối đầu ra của mô hình; draft model cần khớp với mô hình đã căn chỉnh (suy luận).
- **Distillation:** phạt KL tới mô hình tham chiếu về bản chất giống một số hạng ép phân phối gần một "teacher" (xem phần 8).

8. Gợi ý áp dụng

Ba bước SFT, reward model và PPO giải quyết các vấn đề khác nhau: học từ lời giải mẫu, học sở thích tương đối và tối ưu chính sách theo tín hiệu thưởng. Phạt KL giúp hạn chế độ lệch so với mô hình tham chiếu; hướng KL trong PPO của bài báo khác với hướng thường dùng trong chưng cất, nên không thể coi hai mục tiêu là tương đương.

Việc trộn gradient pretraining trong PPO-ptx gợi ý một cách giảm suy thoái trên dữ liệu gốc. Khi chọn checkpoint, nên theo dõi cả chỉ số tác vụ và loss, vì hai thước đo có thể biến thiên khác nhau. Các nhận xét này không thay thế việc đánh giá chất lượng và thiên lệch trên nhóm người dùng cụ thể.

9. Thí nghiệm gợi ý

Huấn luyện một reward model nhỏ trên các cặp câu trả lời đã được xếp hạng, rồi đo khả năng dự đoán thứ tự trên các cặp chưa thấy. Có thể so sánh PPO và PPO-ptx trên cả tập chỉ dẫn lẫn tập đánh giá ngôn ngữ chung, theo dõi đồng thời điểm thưởng, độ lệch KL và đánh giá của con người.

10. Tóm tắt công thức / khái niệm

Khái niệm	Nội dung
SFT	Fine-tune có giám sát trên demo của người; 16 epoch, chọn checkpoint theo điểm RM
RM	GPT-3 6B bỏ unembedding, đầu ra vô hướng $r_\theta(x, y)$
Loss RM	$-\frac{1}{\binom{K}{2}}\mathbb{E}[\log \sigma(r_\theta(x, y_w) - r_\theta(x, y_l))]$; mọi cặp của một prompt cùng một batch element
PPO objective	$r_\theta - \beta \log \frac{\pi^{RL}}{\pi^{SFT}} + \gamma\, \mathbb{E}_{pretrain} \log \pi^{RL}$
PPO vs PPO-ptx	$\gamma = 0$ vs $\gamma = 27.8$ (dữ liệu pretrain gấp 8 lần số episode), $\beta = 0.02$
Alignment tax	Mất điểm trên benchmark NLP công khai sau RLHF; PPO-ptx giảm phần lớn
Dữ liệu	~13k SFT, ~33k RM (K=4–9), ~31k PPO prompt
Chi phí 175B	SFT 4.9; PPO-ptx 60; GPT-3 3,640 (petaflops/s-days)
Kết quả chốt	1.3B InstructGPT được ưa hơn 175B GPT-3; 85±3% thắng GPT-3 175B; hallucination 21% vs 41%

11. Kết luận ngắn

Bài báo chứng minh một công thức ba bước (SFT → RM từ so sánh cặp → PPO có phạt KL, kèm trộn gradient pretraining) có thể biến một mô hình ngôn ngữ thô thành mô hình làm theo chỉ dẫn tốt hơn nhiều, thậm chí

hơn cả việc tăng 100 lần số tham số, với chi phí nhỏ so với pretraining. Hạn chế nằm ở chỗ nó chỉ căn chỉnh theo sở thích của một nhóm nhãn viên nhỏ và vẫn còn lỗi.

Link gốc: https://arxiv.org/abs/2203.02155

Bài 08

LoRA: Low-Rank Adaptation of Large Language Models (2021)

Tác giả: Edward Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, Weizhu Chen (Microsoft).
Venue / arXiv: arXiv:2106.09685 (bản v2, 16/10/2021). Paper không nêu venue trong bản đã đọc.
Nguồn: https://arxiv.org/abs/2106.09685
Chủ đề: Parameter-efficient fine-tuning (PEFT), cập nhật trọng số hạng thấp, gộp (merge) khi suy luận.
Vị trí trong lộ trình: Cần đọc trước: Attention Is All You Need (01) để biết W_q, W_k, W_v, W_o và MLP; GPT-3 (02) để hiểu vì sao fine-tune toàn bộ 175B tham số là khó. Bài này thêm *cách thích nghi một mô hình đã pretrain mà chỉ train rất ít tham số và không tăng độ trễ suy luận*. Bài tiếp theo trong chuỗi: InstructGPT (07) là ví dụ fine-tune cỡ lớn dùng full fine-tune; LoRA là phương án rẻ thay thế. Các bài lượng tử hoá (51–54) sau này kết hợp được với LoRA (ngoài phạm vi paper).

> Ghi chú diễn giải bằng tiếng Việt, không phải bản dịch. Các ví dụ "tự tính" không phải kết quả trực tiếp của bài báo.

1. Bối cảnh

Mô hình thích nghi chuẩn. Quy trình phổ biến: pretrain một mô hình lớn trên dữ liệu tổng quát, rồi **fine-tune** (cập nhật toàn bộ tham số) cho từng tác vụ. Nhược điểm: mỗi tác vụ cho ra một bản sao đầy đủ của mô hình. Với GPT-3 175B, mỗi bản là ~350 GB; lưu và phục vụ hàng trăm bản là không thực tế.

Hai hướng "chỉ thêm ít tham số" có sẵn và lý do chúng chưa đủ tốt (Mục 3 của paper):

- **Adapter layers** (Houlsby 2019...): chèn các khối nhỏ (nén xuống chiều thấp rồi nở ra lại, "bottleneck") vào giữa các lớp Transformer. Dù ít tham số/FLOPs, chúng **phải chạy tuần tự** với phần còn lại, nên với batch nhỏ (batch-1 online) chúng làm chậm thấy rõ vì GPU không tận dụng được song song. Paper đo (Table 1, GPT-2 medium, độ trễ một forward pass): ở batch 1, độ dài 128, fine-tune/LoRA 19.8 ± 2.7 ms; AdapterL 23.9 ms (+20.7%); AdapterH 25.8 ms (+30.3%). Khi batch 32/độ dài 512 thì chỉ +2.2% / +3.0%.
- **Prefix/prompt tuning:** học các embedding/activation "ảo" ghép vào đầu chuỗi. Khó tối ưu (hiệu năng thay đổi không đơn điệu theo số tham số) và **chiếm mất một phần độ dài chuỗi** dành cho tác vụ.

Cảm hứng. Aghajanyan et al. (2020) cho thấy mô hình quá tham số hoá thực ra sống trong một không gian có "chiều nội tại" (intrinsic dimension) thấp. LoRA đặt giả thuyết tương tự cho *phần thay đổi trọng số khi fine-tune*.

Hạng (rank) của ma trận — giải thích cho người mới: rank là số hướng độc lập mà ma trận thực sự dùng. Ma trận $d \times k$ có rank $\leq \min(d, k)$; ma trận hạng r có thể viết thành tích hai ma trận mỏng $B \in \mathbb{R}^{d \times r}$ và $A \in \mathbb{R}^{r \times k}$, chỉ cần $r(d + k)$ số thay vì dk.

2. Ý tưởng chính

Đóng băng trọng số pretrain W_0. Với mỗi ma trận trọng số cần thích nghi, **học phần cập nhật ΔW dưới dạng tích hạng thấp** BA, với $r \ll \min(d, k)$. Đầu ra là $h = W_0 x + BAx$: nhánh gốc đóng băng cộng một nhánh hạng thấp có thể huấn luyện. Vì B khởi tạo bằng 0, lúc bắt đầu mô hình đúng bằng mô hình gốc. Sau khi train xong, **gộp** $W = W_0 + BA$ thành một ma trận duy nhất nên **suy luận không có thêm độ trễ**. Muốn đổi tác vụ thì trừ BA cũ và cộng $B'A'$ mới.

3. Phương pháp chi tiết

3.1 Bài toán (Eq. 1–2)

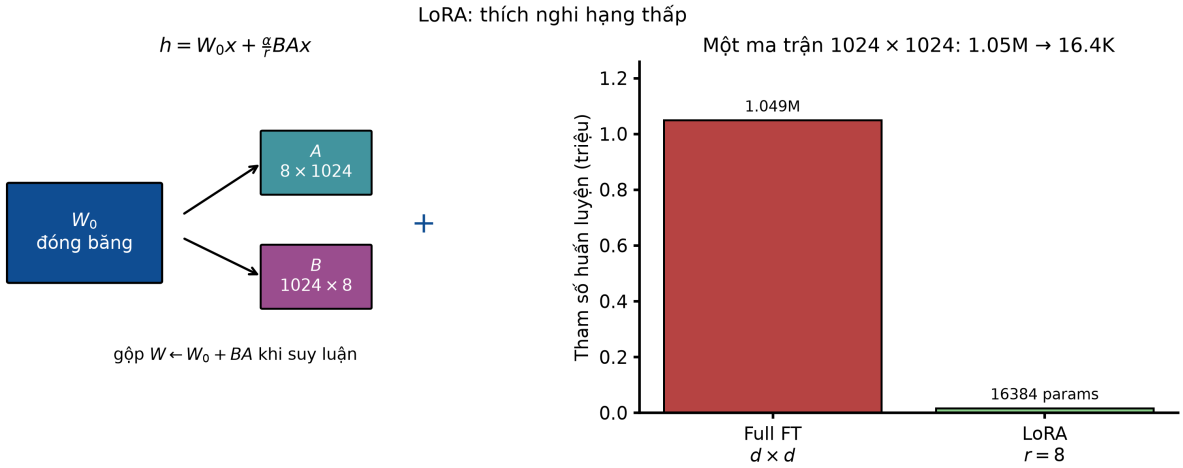
Fine-tune đầy đủ tối đa hoá log-likelihood có điều kiện trên tập $Z = \{(x_i, y_i)\}$:

$$\max_{\Phi} \sum_{(x,y)\in Z} \sum_{t=1}^{|y|} \log P_\Phi(y_t \mid x, y_{<t}), \qquad \Phi = \Phi_0 + \Delta\Phi,\ |\Delta\Phi| = |\Phi_0|.$$

LoRA mã hoá $\Delta\Phi = \Delta\Phi(\Theta)$ bằng bộ tham số nhỏ Θ với $|\Theta| \ll |\Phi_0|$ và tối ưu trên Θ. Với GPT-3 175B, $|\Theta|$ có thể chỉ ~0.01% của $|\Phi_0|$.

3.2 Tham số hoá hạng thấp (Eq. 3)

Với $W_0 \in \mathbb{R}^{d\times k}$:



LoRA $h = W_0x + (\alpha/r)BAx$; ví dụ d=1024, r=8: ~1.05M → 16384 tham số/ma trận. (minh hoạ, tự tính)

$$h = W_0x + \Delta W\, x = W_0x + \frac{\alpha}{r} BA\, x, \qquad B \in \mathbb{R}^{d\times r},\ A \in \mathbb{R}^{r\times k},\ r \ll \min(d,k)$$

- **Khởi tạo:** $A \sim \mathcal{N}(0, \sigma^2)$ (Gaussian ngẫu nhiên), $B = 0$ nên $\Delta W = 0$ lúc đầu. Lưu ý phải phá đối xứng: nếu cả hai bằng 0 thì gradient đều bằng 0; còn với $B = 0$, $A \neq 0$ thì B có gradient khác 0 ngay bước đầu.
- **Hệ số tỉ lệ** α/r (α là hằng số theo r): giúp không phải chỉnh lại siêu tham số khi đổi r. Khi dùng Adam, chỉnh α gần như chỉnh learning rate; paper đặt α bằng giá trị r đầu tiên thử và không tinh chỉnh.
- **Tổng quát hoá fine-tune:** nếu áp dụng LoRA cho mọi ma trận trọng số, train cả bias, và đặt r bằng hạng đầy đủ của W_0, thì khôi phục gần đúng khả năng biểu diễn của fine-tune đầy đủ. Khi tăng tham số, LoRA hội tụ về train mô hình gốc, adapter hội tụ về một MLP, prefix hội tụ về mô hình không nhận được chuỗi dài.

3.3 Áp dụng vào Transformer (Mục 4.2)

Trong self-attention có 4 ma trận (W_q, W_k, W_v, W_o) và 2 ma trận trong MLP. Paper **chỉ thích nghi các ma trận attention, đóng băng MLP** (vì đơn giản và tiết kiệm tham số), và trong hầu hết thí nghiệm chỉ dùng W_q và W_v. Việc thích nghi MLP/LayerNorm/bias được để cho công trình sau. Số tham số khi áp dụng cho $\hat{L}$ ma trận:

$$\boxed{|\Theta| = 2 \times \hat{L}_{LoRA} \times d_{model} \times r}$$

3.4 Gộp khi suy luận

Triển khai: tính sẵn và lưu $W = W_0 + BA$ (cùng kích thước $d \times k$), suy luận như bình thường: *không thêm độ trễ theo cấu trúc*. Đổi tác vụ: $W_0 = W - BA$, rồi cộng $B'A'$ (thao tác nhanh, ít bộ nhớ thêm).

3.5 Pseudo-code (PyTorch-style, đơn giản hoá)

```
class LoRALinear(nn.Module):
    def __init__(self, W0, r, alpha):            # W0: (d_out, d_in) đã đóng băng
        self.W0 = W0.requires_grad_(False)
        self.A = nn.Parameter(torch.randn(r, d_in) * sigma)   # Gaussian
        self.B = nn.Parameter(torch.zeros(d_out, r))          # zero -> ΔW = 0 lúc đầu
        self.scale = alpha / r
    def forward(self, x):
        return x @ self.W0.T + self.scale * (x @ self.A.T) @ self.B.T
    def merge(self):                              # trước khi triển khai
        self.W0 += self.scale * (self.B @ self.A)
# Train: optimizer chỉ nhận {A, B}; không cần gradient/optimizer-state cho W0.
```

4. Phân tích chi phí (tham số, bộ nhớ, độ trễ)

Số tham số huấn luyện. $r(d + k)$ cho mỗi ma trận so với dk. Trên GPT-3 175B ($d_{model} = 12{,}288$, 96 lớp): $r = 4$, thích nghi W_q và W_v → số ma trận $\hat{L} = 2 \times 96 = 192$ và $|\Theta| = 2 \times 192 \times 12288 \times 4 \approx 18.9$M (tự tính; paper nêu ngân sách 18M, ≈35 MB ở FP16). Checkpoint giảm từ 350 GB xuống 35 MB (~10,000×).

Bộ nhớ khi huấn luyện. Với Adam, mỗi tham số cần gradient + 2 trạng thái tối ưu; vì chỉ phải lưu những thứ này cho A, B, VRAM giảm đến 2/3 khi $r \ll d_{model}$. Paper báo trên GPT-3 175B: **1.2 TB → 350 GB**. Tốc độ train +25%: 32.5 → 43.1 token/s mỗi GPU V100 (full FT → LoRA).

Độ trễ. Không tăng nếu gộp. Hạn chế: nếu muốn **gộp rồi batch nhiều tác vụ khác nhau trong một lần forward** thì không được (mỗi tác vụ có W riêng); muốn thế phải giữ A, B riêng và chọn động, chấp nhận chậm hơn.

Ví dụ tự tính: với một ma trận vuông 1024×1024 và hạng $r = 8$, LoRA dùng $8(1024 + 1024) = 16{,}384$ tham số, so với khoảng 1,05 triệu tham số của ma trận gốc. Số này chưa tính các thành phần khác của mô hình.

5. Thí nghiệm và kết quả chính

Phần cứng: NVIDIA Tesla V100. Baseline: Fine-tune (FT), BitFit (chỉ train bias), Prefix-embedding, Prefix-layer, Adapter (H, L, P, D — các biến thể vị trí/nén khác nhau). Các số sau chép từ paper.

RoBERTa / DeBERTa trên GLUE (Table 2, trích):

Mô hình & phương pháp	#tham số train	Trung bình GLUE
RoBERTa-base FT	125.0M	86.4
RoBERTa-base BitFit	0.1M	85.2
RoBERTa-base AdptD	0.3M / 0.9M	84.4 / 85.4
RoBERTa-base LoRA	0.3M	**87.2**
RoBERTa-large FT	355.0M	88.9
RoBERTa-large LoRA	0.8M	89.0

Mô hình & phương pháp	#tham số train	Trung bình GLUE
DeBERTa-XXL FT	1500.0M	91.1
DeBERTa-XXL LoRA	4.7M	**91.3**

(Với cấu hình "†" công bằng với Adapter, RoBERTa-large LoRA 0.8M đạt 88.6, hơn AdptP 88.4 (3.0M) và AdptH 87.8 (6.0M).)

GPT-2 trên E2E NLG (Table 3, trích; BLEU): GPT-2 M: FT 68.2 (354.92M); LoRA 70.4 ± 0.1 (0.35M). GPT-2 L: FT 68.5 (774.03M); LoRA 70.4 ± 0.1 (0.77M).

GPT-3 175B (Table 4):

Phương pháp	#tham số	WikiSQL acc	MNLI-m acc	SAMSum R1/R2/RL
Fine-tune	175,255.8M	73.8	89.5	52.0/28.0/44.5
BitFit	14.2M	71.3	91.0	51.3/27.4/43.5
PreEmbed	3.2M	63.1	88.6	48.3/24.2/40.5
PreLayer	20.2M	70.1	89.5	50.8/27.3/43.5
AdapterH	7.1M	71.9	89.8	53.0/28.9/44.8
AdapterH	40.1M	73.2	91.5	53.2/29.0/45.1
LoRA	4.7M	73.4	**91.7**	**53.8/29.8/45.9**
LoRA	37.7M	**74.0**	91.6	53.4/29.2/45.1

LoRA ngang hoặc hơn fine-tune đầy đủ trên cả ba tập, và scale tốt hơn theo số tham số (Figure 2); prefix-embedding tụt mạnh khi >256 token đặc biệt, prefix-layer khi >32 (paper *nghi ngờ* do phân phối đầu vào lệch khỏi phân phối pretrain).

6. Phân tích / ablation: hiểu cập nhật hạng thấp

Tất cả thí nghiệm dưới đây ở GPT-3 175B.

(a) Nên áp LoRA vào ma trận nào? (Table 5, ngân sách 18M)

Ma trận	W_q	W_k	W_v	W_o	W_q, W_k	W_q, W_v	W_q, W_k, W_v, W_o
Hạng r	8	8	8	8	4	4	2
WikiSQL	70.4	70.0	73.0	73.2	71.4	73.7	73.7
MultiNLI	91.0	90.8	91.0	91.3	91.3	91.3	91.7

Kết luận của paper: với cùng ngân sách, **thích nghi nhiều loại ma trận với hạng nhỏ tốt hơn dồn hạng lớn vào một loại**; W_q, W_v là lựa chọn tốt.

(b) Hạng tối ưu r? (Table 6)

Ma trận	$r = 1$	2	4	8	64
WikiSQL, W_q	68.8	69.6	70.5	70.4	70.0
WikiSQL, W_q, W_v	73.4	73.3	73.7	73.8	73.5
WikiSQL, 4 loại	74.1	73.7	74.0	74.0	73.9

Ma trận	$r = 1$	2	4	8	64
MultiNLI, W_q, W_v	91.3	91.4	91.3	91.6	91.4

Gây ngạc nhiên: $r = 1$ **đã đủ** cho $\{W_q, W_v\}$, còn chỉ W_q thì cần r lớn hơn. Tăng r lên 64 không giúp gì thêm.

(c) Vì sao hạng thấp đủ? Độ tương đồng không gian con. Phân tích SVD của $A_{r=8}$ và $A_{r=64}$ (cùng mô hình gốc), đo độ chồng lấn không gian con bằng $\varphi = \|U^{i\top}_{A_{r=8}} U^{j}_{A_{r=64}}\|_F^2 / \min(i, j) \in [0, 1]$. Phát hiện: hướng kỳ dị top-1 chồng lấn mạnh (>0.5) giữa hai hạng, các hướng sau thì không; giữa hai seed khác nhau cũng chỉ top vài hướng chung (ngẫu nhiên Gaussian thì không chung gì). Diễn giải của paper: các hướng top là "hữu ích", các hướng còn lại phần lớn là nhiễu tích luỹ khi train; ΔW_q có hạng nội tại cao hơn ΔW_v.

(d) ΔW so với W (Table 7, lớp 48 của GPT-3, W_q): chiếu W_q lên không gian con hạng-r của ΔW: $\|U^\top W_q V^\top\|_F = 0.32$ ($r = 4$) so với 21.67 (khi dùng top-r singular của W_q) và 0.02 (ngẫu nhiên); $\|W_q\|_F = 61.95$, $\|\Delta W_q\|_F = 6.91$ ($r = 4$), 3.57 ($r = 64$). Kết luận của paper: (1) ΔW tương quan với W hơn ngẫu nhiên; (2) nhưng nó **khuếch đại các hướng không được nhấn mạnh trong** W chứ không lặp lại hướng top của W; (3) hệ số khuếch đại lớn ($\approx 6.91/0.32 \approx 21.5$ với $r = 4$). Giả thuyết của paper: LoRA khuếch đại các đặc trưng *đã học nhưng chưa được nhấn mạnh* khi pretrain.

7. Giới hạn

Do tác giả nêu:

- Không dễ batch các đầu vào thuộc nhiều tác vụ (nhiều A, B khác nhau) trong một forward pass nếu đã gộp trọng số.
- Việc chọn ma trận nào áp LoRA mang tính heuristic; chưa thử MLP, LayerNorm, bias.
- Không kỳ vọng r nhỏ phù hợp mọi tác vụ: ví dụ nếu tác vụ ở một ngôn ngữ khác hẳn ngôn ngữ pretrain, train toàn bộ ($\approx$ LoRA với $r = d_{model}$) có thể tốt hơn.
- Cơ chế vì sao fine-tune/LoRA hoạt động "còn xa mới rõ".

Nhận xét bổ sung:

- Toàn bộ đánh giá là NLU/NLG tương đối nhỏ (GLUE, E2E, WikiSQL, SAMSum); không có tác vụ đa phương thức, suy luận nhiều bước, hay miền xa pretrain.
- Phân tích hạng thấp chỉ làm trên một mô hình (GPT-3), vài tác vụ, vài lớp; kết luận "r=1 đủ" không nên suy rộng.
- Hiệu quả bộ nhớ chỉ đúng với phần *tham số tối ưu*; **activation vẫn phải lưu** đầy đủ để lan truyền gradient qua các lớp đóng băng, nên LoRA không giảm chi phí activation.

8. Liên hệ với các bài khác

- **01 Attention Is All You Need:** cung cấp ký hiệu W_q, W_k, W_v, W_o mà LoRA cập nhật.
- **02 GPT-3:** mô hình 175B làm động lực (và là bài thử scale chính).
- **07 InstructGPT:** SFT/PPO trong đó là full fine-tune; về nguyên tắc có thể thay bằng LoRA để giảm chi phí.
- **14–16 MQA/GQA/MLA:** MLA cũng dùng nén hạng thấp, nhưng cho K/V cache (nén *kích hoạt ở* suy luận), khác với LoRA (nén *cập nhật trọng số* khi train). Đây là hai ý "hạng thấp" khác nhau, đừng nhầm.
- **51–54 Lượng tử hoá (LLM.int8, SmoothQuant, GPTQ, AWQ):** cho phép nén trọng số gốc; LoRA đặt thêm nhánh nhỏ độ chính xác cao lên trên (ghép lại là hướng phổ biến nhưng ngoài paper này).
- **Pruning:** LoRA và pruning đều khai thác tính dư thừa, nhưng không tương đương nhau.

9. Gợi ý áp dụng

LoRA có thể thích nghi một mô hình với số tham số huấn luyện nhỏ. Việc gộp BA vào trọng số gốc tránh nhánh tính toán bổ sung khi suy luận, nhưng mỗi bản gộp gắn với một bộ adapter; nếu cần chọn adapter động trong cùng batch, cách triển khai và độ trễ sẽ khác.

10. Thí nghiệm gợi ý

So sánh full fine-tune với LoRA ở nhiều hạng r trên cùng dữ liệu và ngân sách; đo chất lượng, bộ nhớ huấn luyện và độ trễ trước và sau khi gộp. Phổ giá trị kỳ dị của cập nhật trọng số có thể giúp khảo sát giả thuyết hạng thấp, nhưng phổ của một bài toán không bảo đảm kết quả tương tự trên bài toán khác.

11. Tóm tắt công thức / khái niệm

Khái niệm	Nội dung
Biểu thức chính	$h = W_0x + \frac{\alpha}{r}BAx$, $B \in \mathbb{R}^{d\times r}$, $A \in \mathbb{R}^{r\times k}$
Khởi tạo	A Gaussian, $B = 0 \Rightarrow \Delta W = 0$ ban đầu
Số tham số	$2\hat{L}d_{model}r$ (khi áp cho $\hat{L}$ ma trận vuông)
Gộp	$W = W_0 + \frac{\alpha}{r}BA$; không thêm độ trễ
Đổi tác vụ	trừ BA cũ, cộng $B'A'$
Bộ nhớ train	tiết kiệm đến 2/3 VRAM (không cần optimizer state cho W_0); 1.2 TB → 350 GB (GPT-3 175B)
Kết quả nổi bật	GPT-3 175B: 4.7M tham số, WikiSQL 73.4, MNLI-m 91.7, vs FT 73.8 / 89.5
Chọn ma trận	W_q, W_v tốt; nhiều ma trận hạng nhỏ > một ma trận hạng lớn
Hạng	$r = 1$–4 thường đủ; tăng lên 64 không giúp
Cơ chế (paper)	ΔW khuếch đại hướng "ẩn" của W (hệ số ≈21.5 với $r = 4$)

12. Kết luận ngắn

LoRA coi cập nhật trọng số khi fine-tune là hạng thấp và chỉ học hai ma trận mỏng song song với trọng số đóng băng, rồi gộp lại khi triển khai. Kết quả: số tham số huấn luyện giảm đến ~10,000 lần, bộ nhớ train giảm ~3 lần, không thêm độ trễ suy luận, và chất lượng ngang hoặc hơn full fine-tune trên các benchmark của paper.

Link gốc: https://arxiv.org/abs/2106.09685

Bài 09

On Layer Normalization in the Transformer Architecture (2020)

Tác giả: Ruibin Xiong, Yunchang Yang, Di He, Kai Zheng, Shuxin Zheng, Chen Xing, Huishuai Zhang, Yanyan Lan, Liwei Wang, Tie-Yan Liu.
Venue / arXiv: arXiv:2002.04745 (v2, 29/6/2020); Proceedings of the 37th International Conference on Machine Learning (ICML), PMLR 108, 2020.
Nguồn: https://arxiv.org/abs/2002.04745
Chủ đề: Vị trí của Layer Normalization (Post-LN vs Pre-LN), gradient lúc khởi tạo, learning rate warm-up, độ ổn định huấn luyện Transformer.
Vị trí trong lộ trình: Đọc sau Attention Is All You Need (01), nơi Transformer gốc đặt LayerNorm *sau* phép cộng residual. Bài này giải thích bằng lý thuyết vì sao kiểu đó cần warm-up và vì sao đặt LayerNorm *bên trong* nhánh residual (Pre-LN) thì không. Tiếp theo: RMSNorm (10) thay chính phép chuẩn hoá bên trong khối Pre-LN bằng bản rẻ hơn; SwiGLU (11), RoPE (12) hoàn thiện "khối Transformer hiện đại" (như LLaMA 05).

> Ghi chú diễn giải, không phải bản dịch. Chứng minh đầy đủ nằm ở phụ lục và không có trong bản văn bản dùng để lập ghi chú; phần dưới trình bày phát biểu và trực giác.

1. Bối cảnh

Layer Normalization (LayerNorm): với vector $v \in \mathbb{R}^d$, chuẩn hoá bằng trung bình và độ lệch chuẩn *của chính vector đó* rồi co giãn bằng tham số học được:

$$\text{LayerNorm}(v) = \gamma \odot \frac{v - \mu}{\sigma} + \beta, \quad \mu = \tfrac{1}{d}\sum_k v_k, \;\; \sigma^2 = \tfrac{1}{d}\sum_k (v_k - \mu)^2.$$

Nó giúp giữ thang giá trị của kích hoạt ổn định, nên là một thành phần then chốt của Transformer.

Residual connection (kết nối tắt): $y = x + F(x)$, cho phép thông tin và gradient đi thẳng qua "đường cao tốc" x, giúp huấn luyện mạng sâu.

Vấn đề. Transformer gốc đặt LayerNorm *giữa các khối residual* (**Post-LN**): $x_{l+1} = \text{LN}(x_l + F(x_l))$. Kiến trúc này cho kết quả rất tốt nhưng khó huấn luyện: bắt buộc có **learning rate warm-up**, tức bắt đầu với learning rate rất nhỏ rồi tăng dần đến lr_{max} trong T_{warmup} bước:

$$lr(t) = \frac{t}{T_{warmup}}\, lr_{max}, \quad t \le T_{warmup}.$$

Warm-up làm chậm tối ưu và thêm siêu tham số nhạy cảm (Popel & Bojar 2018 cho thấy kết quả nhạy với lr_{max} và số bước warm-up). Giải thích trước đó (Liu et al. 2019, RAdam) cho rằng warm-up giảm phương sai của learning rate thích nghi trong Adam. Paper này đặt câu hỏi: **có thể bỏ warm-up an toàn không, và tại sao nó cần?**

2. Ý tưởng chính

Bằng lý thuyết trường trung bình (mean field) tại **thời điểm khởi tạo**, paper chứng minh: ở Post-LN, gradient của các tham số gần lớp đầu ra có độ lớn **lớn và không giảm theo số lớp** L, nên dùng learning rate lớn ngay

từ đầu làm huấn luyện bất ổn; warm-up là cách thực tế tránh điều đó. Ngược lại, ở **Pre-LN** (LayerNorm nằm *bên trong* nhánh residual, trước attention/FFN, cộng thêm một LayerNorm cuối trước dự đoán), độ lớn gradient nhỏ dần theo $\sqrt{L}$ và gần như đồng đều giữa các lớp, nên có thể bỏ warm-up. Thực nghiệm: Pre-LN không cần warm-up, hội tụ nhanh hơn, cho chất lượng tương đương.

3. Phương pháp chi tiết

3.1 Hai kiến trúc (Table 1, Figure 1)

Ký hiệu $x_{l,i}$ là đầu vào của lớp l tại vị trí i, d là chiều ẩn, L là số lớp.

Post-LN (Transformer gốc):

$$
\begin{aligned}
x_{l,i}^{post,1} &= \text{MultiHeadAtt}(x_{l,i}, [x_{l,1}, \ldots, x_{l,n}]) \\
x_{l,i}^{post,2} &= x_{l,i} + x_{l,i}^{post,1} \\
x_{l,i}^{post,3} &= \text{LN}(x_{l,i}^{post,2}) \\
x_{l,i}^{post,4} &= \text{ReLU}(x_{l,i}^{post,3} W^{1,l} + b^{1,l}) W^{2,l} + b^{2,l} \\
x_{l,i}^{post,5} &= x_{l,i}^{post,3} + x_{l,i}^{post,4} \\
x_{l+1,i} &= \text{LN}(x_{l,i}^{post,5})
\end{aligned}
$$

Pre-LN:

$$
\begin{aligned}
x_{l,i}^{pre,1} &= \text{LN}(x_{l,i}) \\
x_{l,i}^{pre,2} &= \text{MultiHeadAtt}(x_{l,i}^{pre,1}, [x_{l,1}^{pre,1}, \ldots, x_{l,n}^{pre,1}]) \\
x_{l,i}^{pre,3} &= x_{l,i} + x_{l,i}^{pre,2} \\
x_{l,i}^{pre,4} &= \text{LN}(x_{l,i}^{pre,3}) \\
x_{l,i}^{pre,5} &= \text{ReLU}(x_{l,i}^{pre,4} W^{1,l} + b^{1,l}) W^{2,l} + b^{2,l} \\
x_{l+1,i} &= x_{l,i}^{pre,3} + x_{l,i}^{pre,5}, \qquad \text{cuối cùng: } x_{Final,i} = \text{LN}(x_{L+1,i})
\end{aligned}
$$

Điểm khác biệt cốt lõi: ở Post-LN, mọi thứ đi qua LayerNorm sau mỗi phép cộng, kể cả "đường cao tốc" residual. Ở Pre-LN, đường residual **không bị chuẩn hoá**: $x_{L+1} = x_0 + \sum_l (\text{đầu ra attention} + \text{FFN})$, tức một tổng cộng dồn, nên giữ nguyên tính chất "gradient đi thẳng" của residual.

3.2 Cài đặt cho phân tích lý thuyết (Mục 3.3)

Để phân tích được, paper đơn giản hoá: dùng single-head; mọi ma trận W^Q, W^K, W^V, W^1, W^2 đều $d \times d$; **khởi tạo** $W^Q = W^K = 0$ (nên attention là phân phối đều, MultiHeadAtt $\to \frac{1}{n}\sum_j x_j W^V$); khởi tạo Xavier ($\mathcal{N}(0, \frac{2}{n_{in}+n_{out}})$) cho phần còn lại, bias 0, $\gamma = 1$; đầu vào là Gaussian. Giả thiết chuẩn bình phương của trạng thái ẩn "tập trung" (ϵ, δ)-bounded: xác suất $\geq 1 - \delta$ thì $\frac{|Z - \mathbb{E}Z|}{\mathbb{E}Z} \leq \epsilon$).

3.3 Kết quả lý thuyết

Lemma 1. Nếu $X \sim \mathcal{N}(0, \sigma^2 I_d)$ thì $\mathbb{E}\|\text{ReLU}(X)\|_2^2 = \frac{1}{2}\sigma^2 d$.

Lemma 2 (độ lớn trạng thái ẩn lúc khởi tạo).

- Post-LN: $\mathbb{E}\|x_{l,i}^{post,5}\|_2^2 = \frac{3}{2}d$ với mọi $l > 0$, **không phụ thuộc độ sâu**.
- Pre-LN: $(1 + \frac{l}{2})d \leq \mathbb{E}\|x_{l,i}^{pre}\|_2^2 \leq (1 + \frac{3l}{2})d$, tức **tăng tuyến tính theo độ sâu** l.

Lemma 3. $\|J_{LN}(x)\|_2 = O(\frac{\sqrt{d}}{\|x\|_2})$ với $J_{LN} = \partial \mathrm{LN}(x)/\partial x$. Trực giác: LayerNorm chia cho độ lớn của đầu vào, nên gradient bị chia cùng hệ số; đầu vào càng lớn thì gradient truyền ngược qua LN càng nhỏ.

Theorem 1 (gradient của lớp FFN cuối). Với xác suất cao (bản trích xuất ghi cận $0.99 - \delta - \epsilon^{0.9+\epsilon}$; nên đối chiếu PDF):

$$\text{Post-LN: } \left\|\frac{\partial \tilde{\mathcal{L}}}{\partial W^{2,L}}\right\|_F \le O(d\sqrt{\ln d}) \qquad \text{Pre-LN: } \left\|\frac{\partial \tilde{\mathcal{L}}}{\partial W^{2,L}}\right\|_F \le O\Big(d\sqrt{\tfrac{\ln d}{L}}\Big)$$

Diễn giải: Post-LN có chuẩn gradient lớp cuối **không phụ thuộc** L vì đầu vào của LN cuối có độ lớn không đổi (Lemma 2). Pre-LN thì đầu vào của LN cuối có độ lớn bình phương $\sim L$, nên gradient bị chia $\sqrt{L}$ (qua Lemma 3).

Mở rộng (phân tích sơ bộ, kết quả nêu không chứng minh đầy đủ trong thân bài): ở Post-LN, chuẩn gradient lớn ở tham số gần đầu ra và có xu hướng giảm khi chỉ số lớp l giảm; ở Pre-LN gần như giữ nguyên với mọi lớp l.

3.4 Từ lý thuyết đến thực hành

Nếu gradient gần đầu ra quá lớn thì bước cập nhật $lr \cdot g$ quá lớn nếu lr lớn → bất ổn → cần warm-up. Với Pre-LN gradient "ngoan" ngay từ đầu → bỏ warm-up, dùng lr lớn ngay.

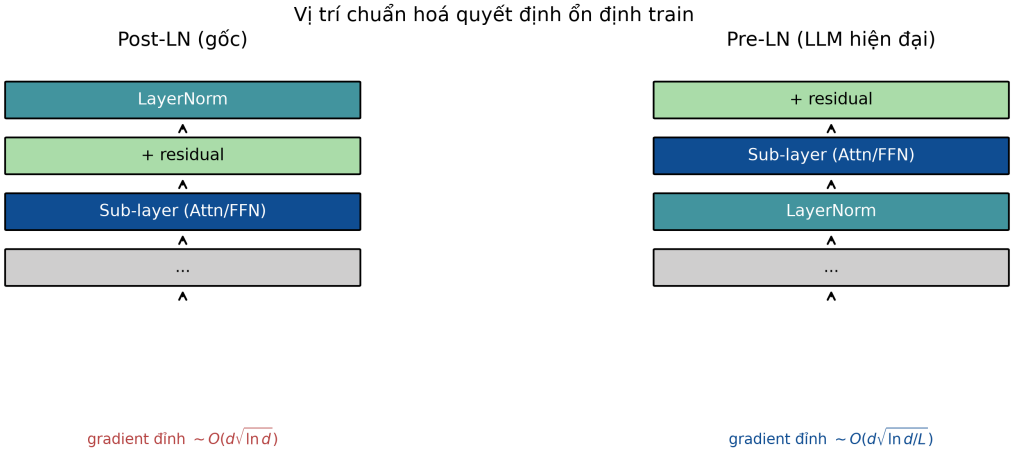
Pseudo-code hai khối:

```
def block_post_ln(x):                    # Transformer gốc
    x = LN(x + Attn(x))
    x = LN(x + FFN(x))
    return x

def block_pre_ln(x):                     # Pre-LN
    x = x + Attn(LN(x))
    x = x + FFN(LN(x))
    return x
# Toàn mạng Pre-LN:  h = x0; for l in layers: h = block_pre_ln(h)
#                    logits = W_emb @ LN(h)        # LN cuối bắt buộc
```

3.5 Ví dụ số nhỏ (tự tính từ Lemma 2)

Với $L = 12$ lớp, $d = 1$ (chuẩn hoá theo d): Post-LN có $\mathbb{E}\|x\|^2/d = 1.5$ ở mọi lớp. Pre-LN ở lớp $l = 12$: giữa $1 + 12/2 = 7$ và $1 + 3 \cdot 12/2 = 19$ lần d. Tức trạng thái ẩn Pre-LN lúc khởi tạo có chuẩn bình phương lớn hơn ~5–13 lần Post-LN ở lớp 12; LN cuối chuẩn hoá lại, và gradient truyền ngược qua nó bị chia khoảng $\sqrt{\text{tỉ số}} \approx 2.2$–$3.6$ lần so với Post-LN (xấp xỉ thô, chỉ để cảm nhận bậc độ lớn, không phải số của paper).



Post-LN vs Pre-LN: vị trí Norm và cận gradient đỉnh theo note. (minh hoạ, tự tính)

4. Thí nghiệm và kết quả chính

Nhiệm vụ. IWSLT14 Đức → Anh (De-En), WMT14 Anh → Đức (En-De, cấu hình Transformer base), và tiền huấn luyện BERT (base) trên English Wikipedia + BookCorpus (~3.4B từ, tỉ lệ train:valid 199:1) rồi kiểm tra trên MRPC, RTE. Tất cả dùng Adam với $\beta = (0.9, 0.98)$ trừ khi nói khác.

(1) Warm-up là thiết yếu với Post-LN (IWSLT14, Mục 3.2):

- Không warm-up, Adam: BLEU chỉ **8.45**; có warm-up ($T_{warmup} = 4000$): khoảng **34** BLEU.
- Với SGD không warm-up: BLEU chỉ hơn 0 một chút sau 15 epoch.
- Nhạy cảm với T_{warmup}: $T_{warmup} = 500$ cho BLEU **31.16** ($lr_{max} = 5\text{e-}4$) và **2.77** ($lr_{max} = 1\text{e-}3$).
- Warm-up cũng giúp SGD, nên (theo tác giả) lợi ích của nó không riêng cho Adam.

(2) Xác minh lý thuyết: chuẩn bình phương trạng thái ẩn khi khởi tạo thoả giả thiết "tập trung" ($(0.1, 0.125)$-bounded). Chuẩn gradient của $W^{2,L}$ khi số lớp tăng (6-6 đến 14-14): Post-LN giữ ~**1.6**, Pre-LN **giảm**. Trong mô hình 6-6, chuẩn gradient kỳ vọng ở Post-LN tăng theo chỉ số lớp, ở Pre-LN gần như phẳng.

(3) Huấn luyện Pre-LN không warm-up: IWSLT14: $lr = 5\text{e-}4$, giảm ×0.1 ở epoch 8; WMT14: $lr = 7\text{e-}4$ hoặc $1.5\text{e-}3$, giảm ở epoch 6 rồi dùng lịch inverse-sqrt. Baseline Post-LN: warm-up 4000 bước. Kết quả: IWSLT14 BLEU ≈ 34 và validation loss ≈ 4, tương đương Post-LN. Ở cùng lr_{max}, **Pre-LN hội tụ nhanh hơn**: checkpoint epoch 9 của Pre-LN ≈ checkpoint epoch 15 của Post-LN (IWSLT14).

(4) So với RAdam: RAdam giúp huấn luyện Post-LN không cần warm-up, nhưng với Pre-LN gần như không khác Adam: thay đổi vị trí LN "chi phối" thay đổi bộ tối ưu.

(5) BERT: Post-LN theo Devlin: warm-up 10k bước, $lr_{max} = 1\text{e-}4$ (thử $3\text{e-}4$ thì phân kỳ). Pre-LN: $3\text{e-}4$, giảm tuyến tính, không warm-up. Paper nêu Post-LN đạt validation loss 1.69 ở 500k cập nhật và Pre-LN đạt tương tự ở 700k cập nhật, kèm kết luận "40% speed-up". *Lưu ý:* đã đối chiếu với PDF: chính bản gốc viết đúng như vậy (Post-LN 500k, Pre-LN 700k). Thứ tự này mâu thuẫn với kết luận "40% speed-up", nên nhiều khả năng đây là lỗi đánh máy trong paper: ý đúng là Pre-LN đạt loss đó ở 500k còn Post-LN cần 700k. Khi trích dẫn, hãy nêu rõ chỗ không nhất quán này. Pre-LN cũng hội tụ nhanh hơn trên MRPC và RTE.

5. Phân tích / ablation

- **Cấu trúc quyết định, không phải bộ tối ưu.** Chỉ đổi vị trí LN thì loại bỏ được nhu cầu warm-up; RAdam chỉ giải quyết Post-LN.

- **Fixed small LR** (10^{-4}) cho Post-LN từ đầu: giảm bớt vấn đề nhưng hội tụ chậm hơn nhiều (kết quả trong phụ lục, được tóm tắt trong thân bài), ủng hộ giả thuyết "gradient quá lớn khi LR lớn".
- **Sau warm-up**, gradient của Post-LN đã nhỏ (Figure 3a–b) và mô hình train được với LR lớn: điều đó ủng hộ cách hiểu warm-up là để vượt qua giai đoạn khởi tạo xấu.
- **LN cuối bắt buộc** trong Pre-LN (kiến trúc của paper có "final layer normalization before prediction").

6. Giới hạn

Do tác giả nêu/hàm ý:

- Lý thuyết chỉ ở lúc khởi tạo, với nhiều đơn giản hoá (single-head, $W^Q = W^K = 0$, kích thước ma trận $d \times d$, ReLU FFN, đầu vào Gaussian). Các tác giả thừa nhận cần kiểm chứng bằng thực nghiệm.
- Việc nghiên cứu các vị trí LN khác và tối ưu Transformer về mặt lý thuyết được đưa vào "tương lai".

Nhận xét bổ sung:

- Chỉ đo chất lượng cuối cùng ở quy mô nhỏ-vừa (IWSLT14, WMT14 base, BERT base); không có thí nghiệm LLM hàng chục lớp trở lên. Paper không nói về hiệu năng cuối cùng của Pre-LN khi rất sâu; một số công trình sau (ngoài paper này) báo rằng Pre-LN có thể kém Post-LN một chút về chất lượng cuối khi so với Post-LN được huấn luyện tốt, và rằng chuẩn trạng thái ẩn Pre-LN phình theo độ sâu. Đây là hiểu biết ngoài paper, nên hãy coi là gợi ý cần kiểm tra.
- Xuất hiện nhiều tham số/biến thể (bộ tối ưu, lịch LR) giữa các nhánh so sánh; chênh lệch nhỏ nên đọc kỹ đường cong trước khi kết luận.
- Phụ lục chứa chứng minh nên người đọc không kiểm tra được ở bản văn bản thuần này.

7. Liên hệ với các bài khác

- **01 Attention Is All You Need:** bản Transformer gốc là Post-LN; bài này là nguyên nhân lý thuyết cho chuyện "phải warm-up".
- **02 GPT-3, 05 LLaMA, 06 Mistral:** các LLM hiện đại dùng chuẩn hoá *trước* mỗi sub-layer (kiểu Pre-LN, với RMSNorm thay LayerNorm) — theo các nguồn ngoài bài báo; xác nhận trong từng bài tương ứng.
- **10 RMSNorm:** thay LayerNorm trong khối Pre-LN bằng bản bỏ trừ trung bình, rẻ hơn.
- **11 SwiGLU, 12 RoPE:** hai thành phần còn lại của khối LLM hiện đại.
- **44 FlashAttention:** tối ưu phép attention bên trong khối; chuẩn hoá và phép cộng residual là các phép elementwise, thường gộp (fuse) trong kernel suy luận.

8. Gợi ý áp dụng

Trong Pre-LN, đường residual cho phép viết trạng thái dưới dạng tổng các đóng góp của các lớp. Cách viết này giúp phân tích luồng gradient, nhưng kết quả khởi tạo của bài báo không tự động áp dụng cho mọi mô hình đã huấn luyện hoặc mọi lịch học.

9. Thí nghiệm gợi ý

So sánh chuẩn gradient theo độ sâu ở Post-LN và Pre-LN khi khởi tạo; thử cùng và khác lịch warm-up, rồi đo loss và độ ổn định qua nhiều seed. Giữ kiến trúc và ngân sách tương đương để tách ảnh hưởng của vị trí chuẩn hoá.

10. Tóm tắt công thức / khái niệm

Khái niệm	Nội dung
Post-LN	$x \leftarrow \text{LN}(x + F(x))$ sau mỗi sub-layer
Pre-LN	$x \leftarrow x + F(\text{LN}(x))$, thêm LN cuối trước dự đoán
Warm-up	$lr(t) = \frac{t}{T_{warmup}} lr_{max}$
Lemma 2	Post-LN: $\mathbb{E}\|x\|^2 = \frac{3}{2}d$ cố định; Pre-LN: $(1 + \frac{l}{2})d \leq \cdot \leq (1 + \frac{3l}{2})d$
Lemma 3	$\|J_{LN}(x)\|_2 = O(\sqrt{d}/\|x\|_2)$
Theorem 1	Gradient $W^{2,L}$: Post $O(d\sqrt{\ln d})$; Pre $O(d\sqrt{\ln d/L})$
Kết quả chốt	Post-LN không warm-up: BLEU 8.45 vs ~34; Pre-LN không warm-up đạt ~34; BERT nhanh hơn ~40% (theo paper)

11. Kết luận ngắn

Paper chỉ ra rằng vị trí của LayerNorm quyết định độ lớn gradient lúc khởi tạo: Post-LN có gradient lớn gần đầu ra, độc lập với số lớp, nên cần warm-up; Pre-LN chia nhỏ gradient theo $\sqrt{L}$ và đồng đều, nên huấn luyện được không cần warm-up, nhanh hơn và cho chất lượng tương đương trên dịch máy và BERT.

Link gốc: https://arxiv.org/abs/2002.04745

Bài 10

Root Mean Square Layer Normalization (2019)

Tác giả: Biao Zhang (University of Edinburgh), Rico Sennrich (University of Edinburgh / University of Zurich).
Venue / arXiv: arXiv:1910.07467 (v1, 16/10/2019); NeurIPS 2019 (33rd Conference on Neural Information Processing Systems, Vancouver).
Nguồn: https://arxiv.org/abs/1910.07467
Chủ đề: Chuẩn hoá lớp rẻ hơn: bỏ re-centering, giữ re-scaling (RMSNorm), phiên bản partial (pRMSNorm).
Vị trí trong lộ trình: Cần biết LayerNorm và khối Pre-LN từ 09 (On Layer Normalization in the Transformer). Bài này thay thế phép chuẩn hoá bên trong khối đó bằng một công thức ngắn hơn. RMSNorm đã trở thành chuẩn trong các LLM hiện đại như LLaMA (05) (hiểu biết ngoài paper). Tiếp theo: SwiGLU (11), RoPE (12).

> Ghi chú diễn giải bằng tiếng Việt, không phải bản dịch. Các ví dụ "tự tính" không phải kết quả trực tiếp của bài báo.

1. Bối cảnh

Chuẩn hoá (normalization) là kỹ thuật co giãn các kích hoạt trong mạng để giữ thang giá trị ổn định, giúp huấn luyện nhanh và ổn định hơn. Hai họ chính: **BatchNorm** (thống kê tính trên cả batch; không hợp chuỗi có độ dài thay đổi trong RNN) và **LayerNorm** (thống kê tính trên các neuron của *một* mẫu, độc lập giữa các mẫu).

LayerNorm (Ba et al.). Với tổng có trọng số đầu vào $a \in \mathbb{R}^n$ (đây là "summed inputs", tức $a_i = \sum_j w_{ij} x_j$):

$$\bar{a}_i = \frac{a_i - \mu}{\sigma} g_i, \quad y_i = f(\bar{a}_i + b_i), \quad \mu = \frac{1}{n}\sum_i a_i, \;\; \sigma = \sqrt{\frac{1}{n}\sum_i (a_i - \mu)^2}.$$

g là gain (khởi tạo 1), b là bias, f là hàm kích hoạt.

Vấn đề của paper. LayerNorm hiệu quả về số bước huấn luyện nhưng **tốn thêm tính toán cho mỗi bước**, nhất là với RNN (có nhiều phép chuẩn hoá tuần tự theo độ dài chuỗi). Figure 1 của paper (RNNSearch dựa GRU): khi loss baseline (không chuẩn hoá) đạt 7.0, LayerNorm đạt 5.4 sau cùng số bước, nhưng chỉ 5.9 sau cùng thời gian chạy, vì mỗi bước chậm hơn.

Hai tính chất bất biến của LayerNorm: *re-centering invariance* (không đổi khi dịch đầu vào/trọng số một lượng) và *re-scaling invariance* (không đổi khi nhân đầu vào/trọng số với hằng số). Cộng đồng thường cho rằng cả hai đều giúp ổn định.

2. Ý tưởng chính

Giả thuyết của paper: **chỉ re-scaling invariance mới là lý do LayerNorm thành công, re-centering (trừ trung bình) là thừa.** Do đó bỏ bước trừ μ và chỉ chia cho **căn bậc hai của trung bình bình phương (RMS)**. Công thức ngắn hơn, ít phép tính hơn (không cần tính trung bình rồi tính phương sai quanh trung bình), thường chạy nhanh hơn 7%–64% tuỳ mô hình/framework, chất lượng tương đương. Ngoài ra, nếu các neuron cùng phân phối thì có thể ước lượng RMS chỉ từ một phần nhỏ (ví dụ 6.25%) các neuron (**pRMSNorm**).

3. Phương pháp chi tiết

3.1 Công thức

$$\bar{a}_i = \frac{a_i}{\mathrm{RMS}(a)}\, g_i, \qquad \mathrm{RMS}(a) = \sqrt{\frac{1}{n}\sum_{i=1}^{n} a_i^2} \tag{Eq. 4}$$

- Khi $\mu = 0$ thì RMSNorm trùng khít LayerNorm (vì khi đó $\sigma = \mathrm{RMS}$).
- RMS bằng chuẩn Euclid chia $\sqrt{n}$: $\mathrm{RMS}(a) = \|a\|_2/\sqrt{n}$, nghĩa là ép vector lên mặt cầu bán kính $\sqrt{n}$. Paper thử dùng **L2-norm thuần** (không chia $\sqrt{n}$) thì thấy **không cải thiện** (Table 2); giả thuyết của tác giả: co giãn bán kính theo kích thước vector giúp chuẩn hoá bền vững qua các vector khác chiều.
- Dạng tổng quát (Eq. 5): $y = f\big(\frac{Wx}{\mathrm{RMS}(a)} \odot g + b\big)$.

Giải nghĩa trực giác: LayerNorm = "đưa vector về tâm 0 rồi đưa độ dài về chuẩn". RMSNorm = "chỉ đưa độ dài về chuẩn, giữ nguyên tâm".

3.2 Bất biến (Table 1)

Do tính tuyến tính $\mathrm{RMS}(\alpha x) = \alpha\,\mathrm{RMS}(x)$ (Eq. 6), nhân ma trận trọng số $W' = \delta W$ cho cùng đầu ra:

$$y' = f\Big(\frac{\delta W x}{\delta\,\mathrm{RMS}(a)} \odot g + b\Big) = y.$$

Phương pháp	Ma trận W: re-scale	Ma trận W: re-center	Vector trọng số: re-scale	Dataset: re-scale	Dataset: re-center	Một mẫu: re-scale
BatchNorm	có	không	có	có	có	không
WeightNorm	có	không	có	không	không	không
LayerNorm	có	có	không	có	không	có
RMSNorm	có	không	không	có	không	có
pRMSNorm	có	không	không	có	không	có

(Trong bản trích xuất, "có/không" được ký hiệu bằng hai ký tự bị mã hoá lại; các ký hiệu được diễn giải theo ngữ cảnh, vì vậy nên đối chiếu bản PDF gốc.) Điểm chính: RMSNorm mất bất biến re-centering so với LayerNorm nhưng giữ re-scaling.

3.3 Gradient (Mục 4.2)

Gọi v là toàn bộ biểu thức bên trong $f(\cdot)$. Gradient theo tham số vector:

$$\frac{\partial L}{\partial b} = \frac{\partial L}{\partial v}, \qquad \frac{\partial L}{\partial g} = \frac{\partial L}{\partial v} \odot \frac{Wx}{\mathrm{RMS}(a)}.$$

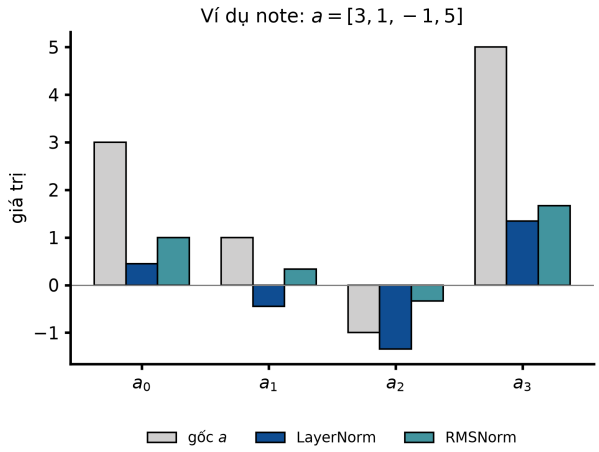
Cả hai bất biến với phép co giãn x và W; gradient của g tỉ lệ với đầu vào *đã chuẩn hoá* nên độ lớn g ổn định. Gradient theo W phức tạp hơn (Eq. 9):

$$\frac{\partial L}{\partial W} = \sum_{i=1}^{n} x^\top \otimes \Big(\frac{\partial L}{\partial v}\Big)_i \mathrm{diag}(g) \times R, \qquad R = \frac{1}{\mathrm{RMS}(a)}\Big(I - \frac{(Wx)(Wx)^\top}{n\,\mathrm{RMS}(a)^2}\Big).$$

Khi co giãn $x' = \delta x$ hoặc $W' = \delta W$ thì $R' = \frac{1}{\delta} R$ (Eq. 10). Hệ quả (theo paper): gradient theo W bất biến với co giãn *đầu vào*, và **tương quan âm với co giãn trọng số**: trọng số càng lớn thì gradient càng nhỏ. Tác giả diễn giải đây là một **bộ điều chỉnh learning rate ngầm**, ngăn chuẩn của ma trận trọng số phình to.

3.4 pRMSNorm

Giả sử các neuron trong một lớp cùng phân phối (i.i.d.), ước lượng RMS từ $k = \lceil n \cdot p \rceil$ phần tử đầu tiên:



Công thức

$$\text{LN}: \ \bar{a}_i = \gamma_i \frac{a_i - \mu}{\sigma} + b_i$$

$$\text{RMSNorm}: \ \bar{a}_i = g_i \frac{a_i}{\text{RMS}(a)}$$

$\text{RMS} = \sqrt{\text{mean}(a^2)} = 3$
(giữ tâm; chỉ re-scale)

RMSNorm vs LayerNorm trên $a = [3, 1, -1, 5]$; chỉ re-scale theo RMS. (minh hoạ, tự tính)

$$\overline{\text{RMS}}(a) = \sqrt{\frac{1}{k}\sum_{i=1}^{k} a_i^2}.$$

Tính tuyến tính của RMS vẫn đúng nên bất biến giữ nguyên (Table 1). Tuy nhiên đây là ước lượng chệch, và tác giả quan sát **gradient có xu hướng bùng nổ khi k nhỏ**; thực tế $p = 6.25\%$ vẫn hội tụ tốt.

3.5 Pseudo-code

```
class RMSNorm(nn.Module):
    def __init__(self, d, eps=1e-8):
        self.g = nn.Parameter(torch.ones(d))      # chỉ có gain, KHÔNG có bias
        self.eps = eps
    def forward(self, a):                          # a: (..., d)
        rms = a.pow(2).mean(dim=-1, keepdim=True).add(self.eps).sqrt()
        return a / rms * self.g

class LayerNorm_ref(nn.Module):                    # để so sánh
    def forward(self, a):
        mu  = a.mean(-1, keepdim=True)
        var = (a - mu).pow(2).mean(-1, keepdim=True)
        return (a - mu) / (var + eps).sqrt() * self.g + self.b
```

(Hằng ϵ để tránh chia 0 là chi tiết triển khai; trong bản văn bản đã đọc, công thức Eq. 4 không có ϵ.)

3.6 Ví dụ số nhỏ (tự tính)

Với $a = [3,\ 1,\ -1,\ 5]$, $n = 4$, $g = 1$:

- LayerNorm: $\mu = 2$, độ lệch $[1, -1, -3, 3]$, $\sigma^2 = (1 + 1 + 9 + 9)/4 = 5$, $\sigma \approx 2.236$, đầu ra $\approx [0.447, -0.447, -1.342, 1.342]$ (trung bình 0).
- RMSNorm: $\text{RMS} = \sqrt{(9 + 1 + 1 + 25)/4} = \sqrt{9} = 3$, đầu ra $= [1,\ 0.333,\ -0.333,\ 1.667]$ (trung bình $\neq 0$). Hai kết quả khác nhau vì $\mu \neq 0$; nếu a có trung bình 0 thì hai công thức trùng nhau.

4. Phân tích chi phí

Đối với một vector d chiều (mỗi token, mỗi vị trí chuẩn hoá):

	LayerNorm	RMSNorm
Phép rút gọn (reduction)	2 (trung bình, phương sai quanh trung bình)	1 (trung bình của bình phương)
Phép elementwise	trừ μ, chia σ, nhân γ, cộng β	chia RMS, nhân g
Tham số	$2d$ (γ, β)	d (g)
FLOPs (xấp xỉ, tự tính)	~$5d$–$7d$	~$3d$–$4d$

Cả hai đều có độ phức tạp $O(d)$ trên mỗi token, **FLOPs của chuẩn hoá nhỏ so với các phép nhân ma trận** ($O(d^2)$ hoặc $O(d\,d_{ff})$ mỗi token mỗi lớp). Lợi ích thực tế đến từ việc **ít lượt đọc/ghi bộ nhớ và ít kernel hơn**, nhất là với phần cứng bị giới hạn bởi băng thông bộ nhớ hoặc độ trễ khởi chạy kernel.

So sánh chi phí: RMSNorm dùng một vector gain có d tham số, còn LayerNorm thông thường có $2d$ tham số gain và bias. Cả hai thường có ít tham số hơn các phép chiếu trong cùng lớp; tiết kiệm thời gian thực tế cần được đo theo cài đặt.

5. Thí nghiệm và kết quả chính

Thực hiện trên TensorFlow, PyTorch, Theano; phần lớn thời gian đo trên một GPU TITAN X (Pascal), trung bình 3 lần chạy. Các mô hình: RNNSearch (GRU, dịch máy), Transformer, attentive reader (CNN/Daily Mail), order-embedding (ảnh–chú thích, MS COCO), ConvPool-CNN-C (CIFAR-10).

Dịch máy WMT14 En-De, RNNSearch (TensorFlow/Nematus, Table 2, SacreBLEU):

Mô hình	Test14	Test17	Thời gian / 1k bước
Baseline	21.7	23.4	399 ± 3.40 s
LayerNorm	22.6	23.6	665 ± 32.5 s
L2-Norm	20.7	22.0	482 ± 19.7 s
RMSNorm	22.4	23.7	501 ± 11.8 s (nhanh hơn 24.7%)
pRMSNorm	22.6	23.1	493 ± 10.7 s (nhanh hơn 25.9%)

LayerNorm và RMSNorm đều giảm ~50% số bước tới hội tụ so với baseline; LayerNorm chậm hơn baseline khoảng 67% trong TensorFlow RNN.

RNNSearch trong Theano và PyTorch (Table 3): RMSNorm nhanh hơn LayerNorm 34.0% (Theano: 652 s vs 988 s) và 11.0% (PyTorch: 763 s vs 857 s) với BLEU tương đương (Theano: Test14 22.5 vs 22.3). pRMSNorm đôi khi chậm hơn RMSNorm, tác giả quy cho cài đặt slice tensor chưa tối ưu.

Transformer (base, 300k bước, Table 4):

Mô hình	Test14	Test17	Thời gian / 1k bước
Baseline (không chuẩn hoá)	không train được (BLEU 0)	không train được	210 ± 0.23 s
LayerNorm	26.6	27.7	248 ± 1.31 s
RMSNorm	26.8	27.7	231 ± 0.04 s (nhanh hơn 6.9%)
pRMSNorm	26.5	27.8	225 ± 1.63 s (nhanh hơn 9.3%)

Chi phí chuẩn hoá tương đối thấp hơn ở Transformer vì ít phép chuẩn hoá tuần tự hơn RNN.

Các tác vụ khác (chọn lọc):

- Attentive reader (CNN/Daily Mail): thời gian / 0.1k bước: LayerNorm 392 s, RMSNorm 333 s (nhanh hơn 15.1%), pRMSNorm 330 s (15.8%); Baseline 315 s. LayerNorm và RMSNorm hội tụ nhanh hơn BatchNorm và đạt tỉ lệ lỗi valid thấp hơn.
- Order-embedding (MS COCO): thời gian / 0.1k bước: Baseline 2.11 s, LayerNorm 12.02 s, RMSNorm 7.12 s (nhanh hơn 40.8%), pRMSNorm 4.34 s (63.9%). Chất lượng recall tương đương, RMSNorm hơi nhỉnh hơn LayerNorm trên một số chỉ số.
- CIFAR-10 (ConvPool-CNN-C, Table 10): test error: Baseline 8.96%, BatchNorm 8.25%, WeightNorm 8.28%, LayerNorm 10.49%, RMSNorm 8.83% (31 s/epoch, nhanh hơn 20.5%), pRMSNorm 10.37%. LayerNorm tệ hơn baseline trên test (không tổng quát hoá), RMSNorm tốt hơn.

6. Phân tích / ablation

- **Re-centering có cần không?** Kết quả đa tác vụ cho thấy RMSNorm tương đương LayerNorm, ủng hộ giả thuyết "re-scaling là cốt lõi". (Không phải chứng minh; đây là bằng chứng thực nghiệm trên các mô hình trong paper.)
- **L2-Norm thuần thất bại** (Table 2: 20.7/22.0 BLEU, thấp hơn cả baseline 21.7/23.4), nên hệ số $1/\sqrt{n}$ trong RMS không chỉ là chuyện quy ước.
- **Thống kê ẩn (Table 5):** ở RNNSearch, độ lệch chuẩn của trạng thái ẩn được cả LayerNorm và RMSNorm ổn định. Trung bình của RMSNorm không được chuẩn hoá nhưng *thực tế ổn định hơn baseline* (ví dụ trung bình tổng thể: baseline −1.60, LayerNorm −0.51, RMSNorm −0.73).
- **Độ bền với khởi tạo (Figure 4):** dịch chuyển tâm khởi tạo trọng số lên 0.2: LayerNorm rất bất ổn, RMSNorm bền hơn (cả hai đều tệ hơn khởi tạo gốc).
- **Tỉ lệ p của pRMSNorm (Figure 3):** trên RNNSearch, BLEU ít bị ảnh hưởng bởi p; các thí nghiệm sau dùng $p = 6.25\%$. Tuy nhiên ở CIFAR-10 pRMSNorm kém (10.37% lỗi) và tác giả nêu gradient có thể bùng nổ với k nhỏ.

7. Giới hạn

Do tác giả nêu:

- Tăng tốc thực tế phụ thuộc framework, phần cứng, kiến trúc và tỉ lệ chi phí của các phần khác; dải 7%–64% rộng.
- pRMSNorm không cho tăng tốc nhất quán trong thực tế (cần mã mức thấp tốt hơn).
- RMS là ước lượng chệch khi chỉ lấy một phần; gradient có thể bất ổn khi p quá nhỏ.
- Cần phân tích thêm vì sao RMSNorm thành công; chưa khám phá chuẩn khác (ℓ_1...).

Nhận xét bổ sung:

- Các mô hình thử nghiệm đều nhỏ-vừa và cũ (RNN GRU/LSTM, Transformer base cho dịch máy, CNN trên CIFAR); **chưa có LLM, chưa có tiền huấn luyện quy mô lớn**. Việc RMSNorm thành chuẩn trong LLM đến từ các công trình sau (ngoài paper này).
- Tăng tốc đo trên *huấn luyện* và trên cài đặt không fuse; không đo suy luận batch-1 hay kernel đã tối ưu, nơi khác biệt LayerNorm/RMSNorm có thể rất nhỏ.
- Chênh lệch BLEU/recall giữa RMSNorm và LayerNorm thường ≤ 0.3–0.5, nằm trong khoảng nhiễu seed thông thường, không có kiểm định thống kê.
- Giả thuyết "giá trị nằm ở re-scaling" được ủng hộ gián tiếp; nó không loại trừ rằng re-centering có ích ở các kiến trúc/độ sâu khác.

8. Liên hệ với các bài khác

- **09 Pre-LN Transformer:** RMSNorm là "chuẩn hoá" đặt vào vị trí Pre-LN: $x \leftarrow x + F(\text{RMSNorm}(x))$.
- **05 LLaMA, 06 Mistral:** dùng RMSNorm trước mỗi sub-layer (hiểu biết ngoài paper; xác nhận trong từng bài).
- **11 SwiGLU:** thành phần MLP đi kèm trong cùng họ khối "RMSNorm + SwiGLU + RoPE".
- **44–45 FlashAttention:** fuse tốt phép attention; còn RMSNorm thường được fuse với phép cộng residual trong các kernel suy luận (hiểu biết ngoài paper).
- **18–24 (linear attention, DeltaNet, Gated DeltaNet):** các lớp này thường có thêm chuẩn hoá riêng (ví dụ chuẩn hoá đầu ra per-head); loại chuẩn hoá cụ thể xem trong từng bài.

9. Gợi ý áp dụng

RMSNorm chuẩn hoá theo bình phương trung bình trên toàn bộ chiều ẩn, vì thế thay đổi các chiều đầu vào cũng thay đổi hệ số chuẩn hoá. Chi phí FLOPs là $O(d)$; hiệu năng thực tế còn phụ thuộc lưu lượng bộ nhớ và khả năng gộp kernel. Không nên suy ra tốc độ của hệ thống khác trực tiếp từ số đo huấn luyện trong bài báo.

10. Thí nghiệm gợi ý

Đo thời gian và bộ nhớ của LayerNorm và RMSNorm trên cùng hình dạng tensor, độ chính xác số và phần cứng, cả khi dùng kernel riêng lẫn kernel gộp. So sánh thêm chất lượng khi huấn luyện lại mô hình với từng loại chuẩn hoá.

11. Tóm tắt công thức / khái niệm

Khái niệm	Nội dung
LayerNorm	$\bar{a}_i = \frac{a_i - \mu}{\sigma} g_i$ (+ bias)
RMSNorm	$\bar{a}_i = \frac{a_i}{\text{RMS}(a)} g_i$, $\text{RMS}(a) = \sqrt{\frac{1}{n}\sum a_i^2}$
Liên hệ	$\mu = 0 \Rightarrow$ RMSNorm = LayerNorm
RMS vs L2	$\text{RMS} = \|a\|_2/\sqrt{n}$; L2-Norm thuần không hiệu quả (Table 2)
Bất biến	RMSNorm: re-scaling (đầu vào, ma trận W, dataset) có; re-centering không
Gradient W	$R = \frac{1}{\text{RMS}}(I - \frac{(Wx)(Wx)^\top}{n\text{RMS}^2})$; $R' = \frac{1}{\delta} R$ khi co giãn $\rightarrow$ learning rate ngầm
pRMSNorm	RMS từ $k = \lceil np \rceil$ phần tử đầu; $p = 6.25\%$
Tham số	RMSNorm: d; LayerNorm: $2d$
Kết quả chốt	Tăng tốc 7%–64% với chất lượng tương đương; Transformer base: 6.9% (RMSNorm), 9.3% (pRMSNorm)

12. Kết luận ngắn

RMSNorm giả định rằng bất biến re-scaling mới là phần quan trọng của LayerNorm và bỏ re-centering, chỉ chia vector cho RMS rồi nhân gain học được. Trên dịch máy (RNN và Transformer), đọc hiểu, truy hồi ảnh–chú thích và CIFAR-10, nó đạt chất lượng tương đương LayerNorm, với thời gian huấn luyện giảm 7%–64% tuỳ cài đặt.

Link gốc: https://arxiv.org/abs/1910.07467

Bài 11

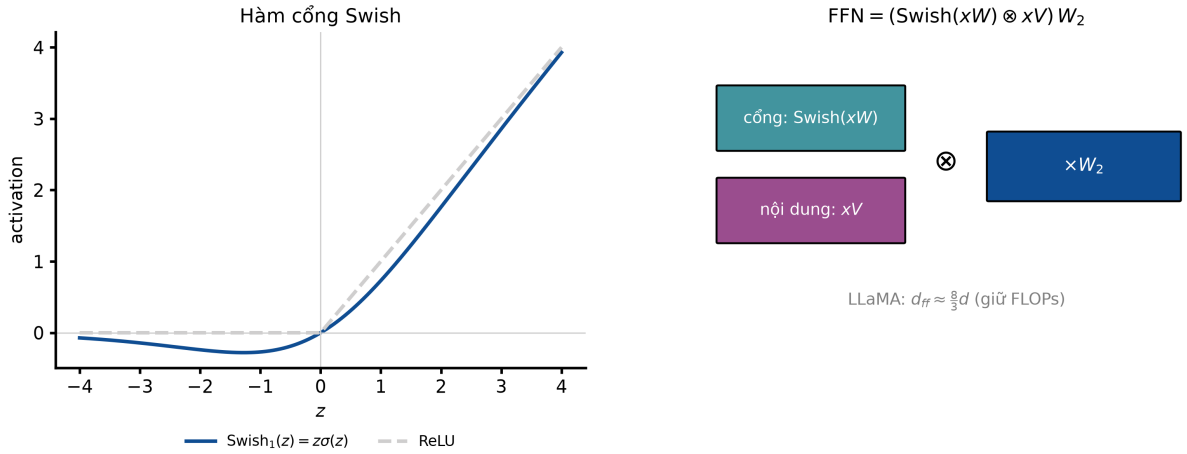
GLU Variants Improve Transformer (2020)

Tác giả: Noam Shazeer (Google)
Venue / arXiv: arXiv:2002.05202 (cs.LG, 12/2/2020), bản v1; paper không nêu venue hội nghị.
Nguồn: https://arxiv.org/abs/2002.05202
Chủ đề: Thay khối FFN của Transformer bằng biến thể Gated Linear Unit (GLU): ReGLU, GEGLU, SwiGLU, Bilinear.
Vị trí trong lộ trình: Đọc sau 01 (Attention Is All You Need, nơi định nghĩa FFN) và 05 (LLaMA dùng SwiGLU), 10 (RMSNorm, cùng "bộ đôi" cải tiến của LLaMA). Paper này bổ sung: lý do FFN hiện đại có **3 ma trận** thay vì 2 và tỉ lệ hidden $\approx \frac{8}{3}d$ thay vì $4d$. Tiếp theo: 12 (RoPE) hoàn thiện công thức "decoder kiểu LLaMA".

> Ghi chú diễn giải bằng tiếng Việt, không phải bản dịch. Các ví dụ "tự tính" không phải kết quả trực tiếp của bài báo.

1. Bối cảnh

Mỗi khối Transformer gồm hai phần: **attention** (trộn thông tin giữa các token) và **FFN** (feed-forward network, xử lý từng vị trí độc lập, "position-wise"). FFN gốc là hai phép tuyến tính với một hàm kích hoạt ở giữa:



SwiGLU: Swish$(xW) \otimes xV$ rồi W_2; $d_{ff} \approx \frac{8}{3}d$. (minh hoạ, tự tính)

$$\text{FFN}(x) = \max(0,\, xW_1 + b_1)W_2 + b_2.$$

- $x \in \mathbb{R}^{d_{model}}$ là biểu diễn ẩn của một token, $W_1 \in \mathbb{R}^{d_{model} \times d_{ff}}$, $W_2 \in \mathbb{R}^{d_{ff} \times d_{model}}$.
- Phần lớn tham số của một Transformer nằm ở FFN (khoảng 2/3 tham số mỗi khối với $d_{ff} = 4d_{model}$), nên đây là chỗ đáng tối ưu.
- T5 bỏ bias: $\text{FFN}_{ReLU}(x) = \max(xW_1, 0)W_2$. Các công trình sau thay ReLU bằng GELU ($x\Phi(x)$, Φ là CDF của chuẩn tắc) hoặc Swish ($x\sigma(\beta x)$, còn gọi SiLU khi $\beta = 1$).

Gated Linear Unit (GLU) do Dauphin et al. (2016) đề xuất cho mô hình ngôn ngữ dùng tích chập: lấy tích từng phần tử ($\otimes$, Hadamard) của hai phép chiếu tuyến tính, một trong hai đi qua sigmoid làm "cổng" (gate). Shazeer hỏi: thay khối "tuyến tính → kích hoạt" đầu của FFN bằng GLU hoặc biến thể thì có tốt hơn không?

2. Ý tưởng chính

Thay vì "một đường": $\text{act}(xW_1)$, ta dùng "hai đường nhân nhau": $\text{act}(xW) \otimes (xV)$. Đường thứ nhất tính ra một **cổng** (mỗi chiều ẩn được nhân với một hệ số mềm), đường thứ hai là **nội dung**. Tích hai đường làm mỗi neuron ẩn có thể được bật/tắt phụ thuộc đầu vào theo cách giàu hơn một hàm kích hoạt đơn lẻ. Vì có thêm một ma trận, tác giả giảm d_{ff} để giữ nguyên số tham số và FLOPs, rồi so sánh công bằng. Kết quả: GEGLU và SwiGLU cho perplexity tốt nhất, và nhiều tác vụ downstream tốt hơn ReLU/GELU. Paper **không giải thích vì sao** ("divine benevolence").

3. Phương pháp chi tiết

3.1 Họ GLU

Với $W, V \in \mathbb{R}^{d_{model} \times d_{ff}}$ (có bias b, c trong định nghĩa gốc):

$$\begin{aligned}
\text{GLU}(x) &= \sigma(xW + b) \otimes (xV + c) \\
\text{Bilinear}(x) &= (xW + b) \otimes (xV + c) \\
\text{ReGLU}(x) &= \max(0, xW + b) \otimes (xV + c) \\
\text{GEGLU}(x) &= \text{GELU}(xW + b) \otimes (xV + c) \\
\text{SwiGLU}(x) &= \text{Swish}_\beta(xW + b) \otimes (xV + c)
\end{aligned}$$

Bilinear là trường hợp không có hàm kích hoạt (cổng là phép tuyến tính). Mỗi biến thể chỉ khác hàm áp lên nhánh cổng.

3.2 FFN dùng GLU (bỏ bias, như T5)

$$\boxed{\text{FFN}_{\text{SwiGLU}}(x, W, V, W_2) = (\text{Swish}_1(xW) \otimes xV)W_2}$$

Tương tự: $\text{FFN}_{GEGLU} = (\text{GELU}(xW) \otimes xV)W_2$, $\text{FFN}_{ReGLU} = (\max(0, xW) \otimes xV)W_2$, $\text{FFN}_{Bilinear} = (xW \otimes xV)W_2$, $\text{FFN}_{GLU} = (\sigma(xW) \otimes xV)W_2$.

Trong đó $\text{Swish}_1(z) = z\,\sigma(z)$ (= SiLU). Trong LLaMA (bài 05) gọi: W = `gate_proj`, V = `up_proj`, W_2 = `down_proj`.

3.3 Giữ nguyên ngân sách tham số

FFN gốc có 2 ma trận cỡ $d_{model} \times d_{ff}$: số tham số $2\,d_{model}d_{ff}$. FFN-GLU có 3 ma trận (W, V, W_2): $3\,d_{model}d'_{ff}$. Để bằng nhau:

$$3\,d'_{ff} = 2\,d_{ff} \;\Rightarrow\; d'_{ff} = \tfrac{2}{3}d_{ff}.$$

Paper viết "giảm theo một hệ số" (bản trích PDF bị lỗi chữ, hiển thị "32"), nhưng số liệu cụ thể trong 3.1 cho thấy đó là **2/3**: $d_{ff} = 3072 \to 2048$. Với $d_{ff} = 4d_{model}$ thì $d'_{ff} = \frac{8}{3}d_{model}$; đó là lý do LLaMA dùng hidden $\approx \frac{8}{3}d$ (ví dụ làm tròn lên bội số của 256) — đây là kiến thức ngoài paper này, xem bài 05.

3.4 Pseudo-code (PyTorch)

```
class SwiGLUFFN(nn.Module):
    def __init__(self, d_model, d_ff):          # d_ff ~ 8/3 * d_model
        super().__init__()
        self.w  = nn.Linear(d_model, d_ff, bias=False)   # gate
        self.v  = nn.Linear(d_model, d_ff, bias=False)   # content
        self.w2 = nn.Linear(d_ff, d_model, bias=False)   # down
```

```
def forward(self, x):
    return self.w2(F.silu(self.w(x)) * self.v(x))
```

3.5 Ví dụ số nhỏ

$d_{model} = 2$, $x = (1, -1)$; một neuron ẩn có $xW = 2$, $xV = 3$. $\text{Swish}_1(2) = 2\sigma(2) \approx 1.76$; đầu ra neuron $= 1.76 \times 3 = 5.28$. Nếu $xW = -4$: $\text{Swish}_1(-4) \approx -0.07$, gần như tắt neuron dù xV lớn. Cổng là hàm của x nên "tắt/mở" theo từng token, khác với ReLU nơi cùng đại lượng xW vừa quyết định tắt vừa là giá trị.

4. Thí nghiệm và kết quả chính

Thiết lập (đúng theo T5 của Raffel et al., 2019): encoder-decoder, mỗi bên 12 layer, $d_{model} = 768$, $h = 12$, $d_k = d_v = 64$; FFN gốc $d_{ff} = 3072$, biến thể GLU $d_{ff} = 2048$ (cùng tham số và phép tính). Tiền huấn luyện 524,288 bước trên C4 với mục tiêu điền đoạn bị xóa (span-filling), batch 128 mẫu (input 512 token, output 114 token), Adafactor, lịch học inverse-sqrt, **không dropout** khi pre-train (khác T5 gốc). Sau đó fine-tune một lần trên hỗn hợp SQuAD + GLUE + SuperGLUE (131,072 bước, lr 10^{-3}, dropout 0.1).

Bảng 1: log-perplexity trên tập held-out (thấp hơn là tốt). Số trong ngoặc là độ lệch chuẩn giữa 4 lần chạy ngắn.

FFN	65,536 bước	524,288 bước
ReLU (baseline)	1.997 (0.005)	1.677
GELU	1.983 (0.005)	1.679
Swish	1.994 (0.003)	1.683
GLU	1.982 (0.006)	1.663
Bilinear	1.960 (0.005)	1.648
GEGLU	**1.942 (0.004)**	**1.633**
SwiGLU	1.944 (0.010)	1.636
ReGLU	1.953 (0.003)	1.645

Điểm trung bình downstream (dev):

FFN	GLUE avg	SuperGLUE avg	SQuAD EM / F1
ReLU	83.80	72.76	83.18 / 90.87
GELU	83.86	72.98	83.09 / 90.79
Swish	83.60	72.40	83.25 / 90.76
GLU	84.20	73.95	82.88 / 90.69
GEGLU	84.12	73.96	83.55 / 91.12
Bilinear	83.79	73.81	83.82 / 91.06
SwiGLU	84.36	**74.56**	83.42 / 91.03
ReGLU	**84.67**	73.66	83.53 / 91.18
T5 gốc (Raffel)	83.28	71.36	80.88 / 88.81

Độ lệch chuẩn giữa các lần chạy theo Raffel et al.: GLUE avg 0.235, SuperGLUE avg 0.416, SQuAD EM 0.343 / F1 0.226.

5. Phân tích / ablation

- **Cổng giúp**: mọi biến thể có nhân tích (GLU, Bilinear, ReGLU, GEGLU, SwiGLU) đều có log-perplexity thấp hơn cả ba baseline không cổng (ReLU, GELU, Swish) ở 524k bước. Khoảng cách

tốt nhất: $1.677 \rightarrow 1.633$ (GEGLU) khoảng 0.044, lớn hơn nhiều so với độ lệch giữa các lần chạy (~0.005 ở 65k bước).

- **Hàm kích hoạt của cổng ít quan trọng hơn bản thân việc có cổng**: ngay cả Bilinear (không phi tuyến) đạt 1.648, tốt hơn ReLU/GELU/Swish. Đây là cách diễn giải từ bảng; tác giả không phân tích.
- GEGLU và SwiGLU tốt nhất theo perplexity; trên downstream thì "nhiễu" (các biến thể thắng ở các tác vụ khác nhau). Tác giả nói "the results are noisy" nhưng GLU-variants thắng ở phần lớn tác vụ.
- Việc T5 gốc kém hơn baseline ReLU của tác giả (dù cùng mô hình) được giải thích bằng dropout khi pre-train (giả thuyết của tác giả, không kiểm chứng riêng).

6. Giới hạn

Do tác giả nêu: không có lời giải thích cơ chế; kết quả downstream nhiễu; mỗi cấu hình fine-tune một lần.

Nhận xét bổ sung:

- Chỉ một quy mô (T5-base, ~220M tham số cỡ) và một họ mô hình (encoder-decoder, span-filling). Không có thí nghiệm decoder-only LM hay quy mô lớn trong paper; sự phổ biến sau này (PaLM, LLaMA) là bằng chứng bên ngoài.
- Chỉ so sánh ở cùng số tham số; không báo cáo tốc độ thực tế (3 matmul nhỏ hơn so với 2 matmul lớn có thể khác nhau về hiệu quả phần cứng). Paper chỉ nói "no apparent computational drawbacks".
- Không có số liệu về độ ổn định huấn luyện hay khác biệt khi lượng tử hóa.

7. Liên hệ với các bài khác

- **01 (Attention Is All You Need):** FFN gốc được thay thế ở đây.
- **05 (LLaMA), 06 (Mistral 7B):** dùng SwiGLU với hidden $\approx \frac{8}{3}d$; cùng RMSNorm (10) và RoPE (12).
- **27/28 (Switch, Mixtral):** mỗi "expert" của MoE chính là một FFN, trong Mixtral là SwiGLU.
- **10 (RMSNorm), 12 (RoPE):** ba thay đổi kiến trúc kinh điển của họ LLaMA.
- **24 (Gated DeltaNet):** cũng có ý tưởng "cổng nhân" nhưng ở mức attention/trạng thái, không phải FFN.

8. Gợi ý áp dụng

Mỗi chiều ẩn của GLU gắn với hai nhánh chiếu và một chiều tương ứng trong phép chiếu đầu ra. Nếu thay đổi độ rộng, các chiều đó phải được điều chỉnh nhất quán. Điều này là hệ quả của cấu trúc phép tính; bài báo không thử cắt tỉa các chiều ẩn.

9. Thí nghiệm gợi ý

Huấn luyện các biến thể SwiGLU, GEGLU và ReLU với ngân sách tham số và dữ liệu tương đương; dùng nhiều seed và báo cáo cả loss lẫn chất lượng tác vụ để đánh giá mức chênh lệch.

10. Tóm tắt công thức / khái niệm

Khái niệm	Công thức / ý nghĩa
FFN gốc	$\max(0, xW_1)W_2$ (không bias)
GLU	$\sigma(xW) \otimes xV$
SwiGLU	$\text{Swish}_1(xW) \otimes xV$, $\text{Swish}_1(z) = z\sigma(z)$
GEGLU / ReGLU / Bilinear	cổng bằng GELU / ReLU / không hàm
FFN-SwiGLU	$(\text{Swish}_1(xW) \otimes xV)W_2$
Giữ ngân sách	$d'_{ff} = \frac{2}{3}d_{ff}$ (3072 → 2048)
Kết quả chính	log-ppl 1.677 (ReLU) → 1.636 (SwiGLU), 1.633 (GEGLU)

11. Kết luận ngắn

Một thay đổi nhỏ, rẻ, không tốn thêm tham số: nhân hai đường chiếu trong FFN thay vì dùng một đường có hàm kích hoạt. Trên T5-base nó giảm perplexity đáng kể và cải thiện nhiều tác vụ downstream, và về sau trở thành mặc định của các LLM (LLaMA, Mistral, Qwen). Paper chỉ cung cấp bằng chứng thực nghiệm, không có lý thuyết.

Nguồn: https://arxiv.org/abs/2002.05202

Bài 12

RoFormer: Enhanced Transformer with Rotary Position Embedding (2021)

Tác giả: Jianlin Su, Yu Lu, Shengfeng Pan, Ahmed Murtadha, Bo Wen, Yunfeng Liu (Zhuiyi Technology, Shenzhen)
Venue / arXiv: arXiv:2104.09864 (cs.CL), bản v5 ngày 8/11/2023; paper không nêu venue hội nghị.
Nguồn: https://arxiv.org/abs/2104.09864
Chủ đề: Mã hóa vị trí (position encoding) bằng phép xoay (Rotary Position Embedding, RoPE); tính chất tương đối, suy giảm theo khoảng cách, tương thích linear attention.
Vị trí trong lộ trình: Đọc sau 01 (Attention Is All You Need: sinusoidal position) và 11 (SwiGLU). Paper bổ sung: cách đưa vị trí vào attention bằng nhân (xoay) thay vì cộng, cho phép $q_m^\top k_n$ chỉ phụ thuộc $m-n$. Tiếp theo: 13 (ALiBi, một hướng khác: cộng bias vào score), 16 (MLA, nơi RoPE gây rắc rối với nén KV), 18-24 (linear attention, nơi RoPE được nhắc tới ở §3.3).

> Ghi chú diễn giải bằng tiếng Việt, không phải bản dịch. Các ví dụ "tự tính" không phải kết quả trực tiếp của bài báo.

1. Bối cảnh

Vấn đề. Self-attention là phép toán trên *tập hợp*: nếu hoán vị token đầu vào thì đầu ra cũng chỉ hoán vị theo, nó không tự biết thứ tự (paper dẫn Yun et al. 2020: "position-agnostic"). Muốn mô hình biết "từ nào đứng trước từ nào" phải tiêm thông tin vị trí.

Các cách đã có (paper §2):

1. **Vị trí tuyệt đối cộng vào embedding:** $f_t(x_i, i) = W_t(x_i + p_i), t \in \{q, k, v\}$. Hoặc p_i học được (BERT, GPT-2), hoặc sinusoidal: $p_{i,2t} = \sin(i/10000^{2t/d})$, $p_{i,2t+1} = \cos(i/10000^{2t/d})$ (Vaswani 2017).
2. **Vị trí tương đối** (Shaw 2018, Transformer-XL, T5, DeBERTa...): đưa $m-n$ vào điểm attention, thường bằng cách khai triển $q_m^\top k_n$ thành 4 số hạng (nội dung-nội dung, nội dung-vị trí, vị trí-nội dung, vị trí-vị trí) rồi thay p_n bằng embedding tương đối $\tilde{p}_{m-n}$, hoặc cộng một bias học được $b_{i,j}$ (T5).

Hạn chế mà paper nhắm tới: các cách trên đều **cộng** thông tin vị trí vào biểu diễn hoặc vào score; paper cho rằng chúng vì thế khó dùng cho **linear attention** (nơi không tính ma trận score $N \times N$ để cộng bias vào). Paper muốn: (i) $\langle q_m, k_n \rangle$ phụ thuộc vị trí **chỉ qua** $m-n$, (ii) không cần tham số học thêm, (iii) hoạt động với độ dài tuỳ ý, (iv) tương thích linear attention.

2. Ý tưởng chính

Thay vì cộng một vector vị trí, **xoay** vector query/key một góc tỉ lệ với vị trí token: token ở vị trí m bị xoay $m\theta$. Khi tính tích vô hướng giữa query ở m và key ở n, hai phép xoay "triệt tiêu một phần" và chỉ còn lại phép xoay góc $(n-m)\theta$: vị trí tuyệt đối biến mất, chỉ còn khoảng cách tương đối. Vì vector d chiều được chia thành $d/2$ cặp chiều (mỗi cặp là một mặt phẳng 2D) và mỗi cặp xoay với tốc độ θ_i khác nhau (từ nhanh đến chậm), ta thu được một "bộ đồng hồ" nhiều kim mã hóa khoảng cách. Phép xoay không đổi độ dài vector, không thêm tham số, và áp dụng được cho cả softmax attention lẫn linear attention.

3. Phương pháp chi tiết (suy diễn từng bước)

3.1 Phát biểu bài toán

Cho $q_m = f_q(x_m, m)$, $k_n = f_k(x_n, n)$. Mong muốn (paper Eq. 11):

$$\langle f_q(x_m, m),\ f_k(x_n, n)\rangle = g(x_m, x_n, m-n).$$

Nghĩa là tích vô hướng chỉ phụ thuộc hai nội dung x_m, x_n và **hiệu vị trí**. Cần tìm f_q, f_k thỏa điều đó, với điều kiện ban đầu $f_q(x,0) = W_q x$, $f_k(x,0) = W_k x$ (vị trí 0 = chưa mã hóa gì).

3.2 Ôn nhanh: số phức và phép xoay 2D

Vector 2D $(a,b) \leftrightarrow$ số phức $z = a + ib$. Nhân với $e^{i\phi} = \cos\phi + i\sin\phi$ là **xoay** z góc ϕ ngược chiều kim đồng hồ, giữ nguyên độ dài $|z|$. Dạng ma trận:

$$e^{i\phi}\cdot(a+ib) \longleftrightarrow \begin{pmatrix}\cos\phi & -\sin\phi\\ \sin\phi & \cos\phi\end{pmatrix}\begin{pmatrix}a\\ b\end{pmatrix} = R(\phi)\begin{pmatrix}a\\ b\end{pmatrix}.$$

Tính chất cần dùng: $R(\phi)^\top = R(-\phi)$ (xoay ngược), $R(\alpha)R(\beta) = R(\alpha+\beta)$ (xoay liên tiếp cộng góc). Tích vô hướng 2D của hai vector tương ứng $\langle u, v\rangle = \mathrm{Re}(u\,\bar{v})$, với $\bar{v}$ là liên hợp phức.

3.3 Suy ra nghiệm ở $d = 2$ (paper §3.4.1)

Viết dạng cực: $f_q(x_q, m) = R_q e^{i\Theta_q}$, $f_k(x_k, n) = R_k e^{i\Theta_k}$, $g = R_g e^{i\Theta_g}$ (đây R là *bán kính*, Θ là *góc*, không phải ma trận). Yêu cầu $\langle f_q, f_k\rangle = g(\cdot, n-m)$, tức $\mathrm{Re}[f_q \bar{f}_k]$ khớp với g (paper làm việc với dạng phức này), cho hai ràng buộc (Eq. 24):

$$R_q(x_q,m)\,R_k(x_k,n) = R_g(x_q,x_k,n-m), \qquad \Theta_k(x_k,n) - \Theta_q(x_q,m) = \Theta_g(x_q,x_k,n-m).$$

Bước 1: bán kính. Đặt $m = n$ và dùng điều kiện đầu $R_q(\cdot,0) = \|q\|$, $R_k(\cdot,0) = \|k\|$ (Eq. 26a, 27): $R_q(x_q,m)R_k(x_k,m) = R_g(\cdot,0) = \|q\|\|k\|$. Nghiệm đơn giản: bán kính **không phụ thuộc vị trí**, $R_q = \|q\|$, $R_k = \|k\|$. (Đây là lý do phép xoay không đổi độ dài.)

Bước 2: góc. Cũng với $m = n$ (Eq. 26b): $\Theta_k(x_k,m) - \Theta_q(x_q,m) = \theta_k - \theta_q$ (hiệu góc ban đầu), suy ra $\Theta_q(x_q,m) - \theta_q = \Theta_k(x_k,m) - \theta_k$. Đại lượng này không phụ thuộc nội dung, chỉ phụ thuộc vị trí; gọi là $\phi(m)$ (Eq. 28): $\Theta_{q,k}(x,m) = \phi(m) + \theta_{q,k}$.

Bước 3: ϕ là cấp số cộng. Thay $n = m+1$ vào Eq. 24 và dùng Eq. 28: $\phi(m+1) - \phi(m) = \Theta_g(x_q,x_k,1) + \theta_q - \theta_k$ (Eq. 29). Vế phải không phụ thuộc m, nên $\phi(m) = m\theta + \gamma$ (Eq. 30) với hằng số $\theta \neq 0$, γ.

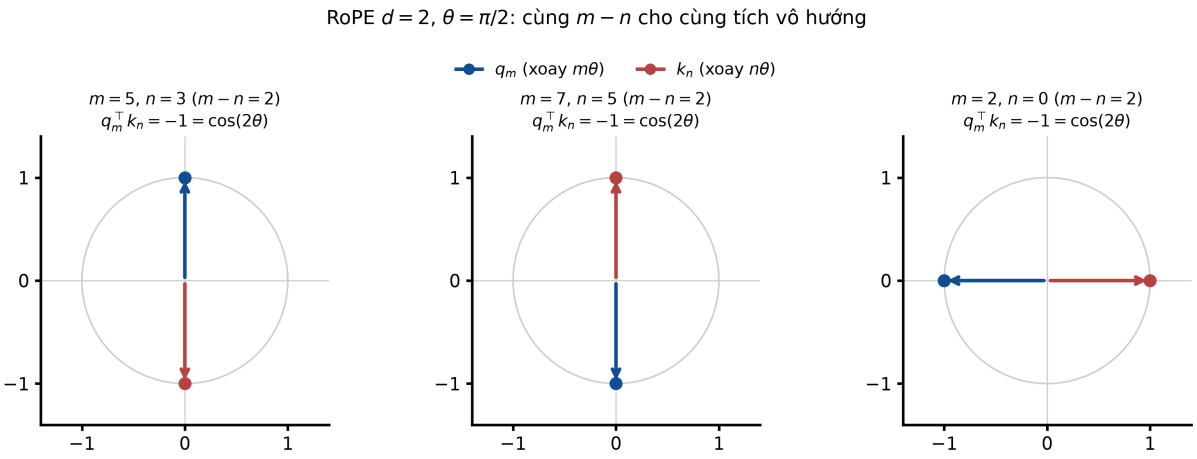
Bước 4: kết quả. Chọn $\gamma = 0$:

$$\boxed{f_q(x_m,m) = (W_q x_m)\,e^{im\theta}, \qquad f_k(x_n,n) = (W_k x_n)\,e^{in\theta}}$$

Tức là: lấy $q = W_q x_m$ (xem như số phức), xoay thêm $m\theta$. (Lưu ý: "suy ra" này đi qua giả thiết nghiệm đơn giản ở Bước 1 và chọn $\gamma = 0$; paper trình bày đây là *một* nghiệm, không chứng minh tính duy nhất.)

3.4 Tính chất tương đối (trọng tâm)

Tích vô hướng giữa q_m và k_n ở $d = 2$:



RoPE $d = 2$, $\theta = \pi/2$: cùng hiệu vị trí $m - n = 2$ cho cùng tích $q_m^\top k_n = \cos(2\theta) = -1$ dù vị trí tuyệt đối khác nhau. (minh hoạ, tự tính)

$$q_m^\top k_n = \mathrm{Re}[(W_q x_m e^{im\theta})\,\overline{(W_k x_n e^{in\theta})}] = \mathrm{Re}[(W_q x_m)\,\overline{(W_k x_n)}\,e^{i(m-n)\theta}].$$

Hai thừa số $e^{im\theta}$ và $\overline{e^{in\theta}} = e^{-in\theta}$ gộp thành $e^{i(m-n)\theta}$: **chỉ còn hiệu vị trí** (Eq. 12). Dạng ma trận: với $R_m \equiv R(m\theta)$,

$$q_m^\top k_n = (R_m W_q x_m)^\top (R_n W_k x_n) = x_m^\top W_q^\top \underbrace{R_m^\top R_n}_{=R_{n-m}} W_k x_n,$$

vì $R_m^\top = R(-m\theta)$ và $R(-m\theta)R(n\theta) = R((n-m)\theta)$. (Paper Eq. 16 viết $R_{\Theta,n-m}$.)

3.5 Tổng quát cho d chiều

Giả sử d chẵn. Chia vector thành $d/2$ cặp chiều (x_{2i-1}, x_{2i}), mỗi cặp xoay với tần số riêng θ_i (Eq. 14, 15). Ma trận xoay là khối đường chéo:

$$R^d_{\Theta,m} = \mathrm{diag}(R(m\theta_1), R(m\theta_2), \ldots, R(m\theta_{d/2})), \qquad \boxed{\theta_i = 10000^{-2(i-1)/d},\ i = 1, \ldots, d/2}$$

$R^d_{\Theta,m}$ trực giao ($R^\top R = I$), nên giữ nguyên chuẩn của q, k ("ổn định" theo paper). Do tính tuyến tính của tích vô hướng, tích của hai vector tổng quát là tổng của $d/2$ tích 2D:

$$\boxed{q_m^\top k_n = (R^d_{\Theta,m} W_q x_m)^\top (R^d_{\Theta,n} W_k x_n) = x_m^\top W_q^\top R^d_{\Theta,n-m} W_k x_n}$$

với $R^d_{\Theta,n-m} = (R^d_{\Theta,m})^\top R^d_{\Theta,n}$. Mỗi cặp i đóng góp một số hạng chỉ phụ thuộc $(n-m)\theta_i$. **Lưu ý:** RoPE chỉ áp lên q và k, **không** áp lên v (vị trí tương đối chỉ đi vào score). Vị trí tuyệt đối vẫn "có mặt" trong q_m, k_n riêng lẻ, nhưng chỉ lộ ra qua score dưới dạng hiệu.

3.6 Cài đặt hiệu quả (Eq. 34)

Ma trận $R^d_{\Theta,m}$ rất thưa, không nhân trực tiếp. Dùng nhân từng phần tử:

$$R^d_{\Theta,m}x = \begin{pmatrix} x_1 \\ x_2 \\ \vdots \\ x_{d-1} \\ x_d \end{pmatrix} \otimes \begin{pmatrix} \cos m\theta_1 \\ \cos m\theta_1 \\ \vdots \\ \cos m\theta_{d/2} \\ \cos m\theta_{d/2} \end{pmatrix} + \begin{pmatrix} -x_2 \\ x_1 \\ \vdots \\ -x_d \\ x_{d-1} \end{pmatrix} \otimes \begin{pmatrix} \sin m\theta_1 \\ \sin m\theta_1 \\ \vdots \\ \sin m\theta_{d/2} \\ \sin m\theta_{d/2} \end{pmatrix}$$

```python
def rope(x, pos, base=10000.0):           # x: [..., T, d], pos: [T]
    d = x.shape[-1]
    inv_freq = base ** (-torch.arange(0, d, 2) / d)       # theta_i, [d/2]
    ang = pos[:, None] * inv_freq[None, :]                # [T, d/2], m*theta_i
    cos, sin = ang.cos(), ang.sin()
    x1, x2 = x[..., 0::2], x[..., 1::2]                  # cặp (x_{2i-1}, x_{2i}), kiểu
↪  paper
    out = torch.empty_like(x)
    out[..., 0::2] = x1 * cos - x2 * sin
    out[..., 1::2] = x1 * sin + x2 * cos
    return out
# q = rope(q, pos); k = rope(k, pos); scores = q @ k.transpose(-1,-2) / sqrt(d)
```

Ghi chú triển khai (ngoài paper): LLaMA/HF thường ghép cặp theo kiểu "chia đôi" $(x_i, x_{i+d/2})$ với hàm `rotate_half` thay vì cặp xen kẽ (x_{2i-1}, x_{2i}) như trên. Hai cách tương đương nếu hoán vị cố định các chiều của W_q, W_k, nhưng **không** thay thế lẫn nhau khi chuyển checkpoint nếu không hoán vị lại trọng số.

3.7 Ví dụ số nhỏ (tự tính)

$d = 2$, $\theta = \pi/2$ (90° mỗi bước, chỉ để dễ tính). Query nội dung $q = (1, 0)$, key nội dung $k = (1, 0)$.

m (query)	n (key)	q_m	k_n	$q_m^\top k_n$
5	3	xoay $450° \equiv 90°$: $(0, 1)$	xoay $270°$: $(0, -1)$	-1
7	5	xoay $630° \equiv 270°$: $(0, -1)$	xoay $450° \equiv 90°$: $(0, 1)$	-1
2	0	$(-1, 0)$	$(1, 0)$	-1

Cả ba cặp có $m - n = 2$ và đều cho $-1 = \cos(2\theta) = \cos 180°$. Vị trí tuyệt đối thay đổi, kết quả không đổi.

3.8 Suy giảm theo khoảng cách (paper §3.3, §3.4.3)

Nhóm các cặp chiều, Eq. 35:

$$q_m^\top k_n = \mathrm{Re}\Big[\sum_{i=0}^{d/2-1} h_i\, e^{i(m-n)\theta_i}\Big], \quad h_i = q_{[2i:2i+1]}\, k^*_{[2i:2i+1]}.$$

Đặt $S_j = \sum_{i<j} e^{i(m-n)\theta_i}$. Biến đổi Abel: $\big|\sum_i h_i e^{i(m-n)\theta_i}\big| = \big|\sum_i S_{i+1}(h_{i+1} - h_i)\big| \le \big(\max_i |h_{i+1} - h_i|\big) \sum_i |S_{i+1}|$. Paper khẳng định (và vẽ ở Fig. 2) rằng với $\theta_i = 10000^{-2i/d}$, đại lượng $\frac{1}{d/2}\sum_i |S_i|$ **giảm khi khoảng cách** $|m - n|$ **tăng**. Đây là một chặn trên trung bình, được trình bày bằng hình số học chứ không có chứng minh giải tích đầy đủ trong paper; nó cũng không có nghĩa score thực tế luôn giảm với khoảng cách (nội dung h_i vẫn quyết định).

3.9 RoPE với linear attention (§3.3, Eq. 17-19)

Dạng tổng quát $\mathrm{Attn}_m = \frac{\sum_n \mathrm{sim}(q_m, k_n) v_n}{\sum_n \mathrm{sim}(q_m, k_n)}$. Linear attention thay $\mathrm{sim} = \varphi(q_m)^\top \phi(k_n)$ với φ, ϕ không âm. Paper đề xuất xoay *sau* hàm đặc trưng:

$$\text{Attn}_m = \frac{\sum_n \left(R^d_{\Theta,m}\varphi(q_m)\right)^\top \left(R^d_{\Theta,n}\phi(k_n)\right)v_n}{\sum_n \varphi(q_m)^\top \phi(k_n)}.$$

Mẫu số **giữ nguyên không xoay** để tránh chia cho số âm/0; tử số có thể có số hạng âm nên trọng số không còn là xác suất chuẩn hóa. Paper nói vẫn mô hình hóa được độ quan trọng của các value.

4. Thí nghiệm và kết quả chính

Tất cả chạy trên hai máy chủ 4×V100 (theo paper). Quy mô nhỏ; không có LLM.

Dịch máy, WMT 2014 En→De (so với Transformer-base):

Model	BLEU
Transformer-base (Vaswani 2017)	27.3
RoFormer	27.5

Tiền huấn luyện MLM (BERT-base, BookCorpus + Wikipedia, 100k bước, batch 64, độ dài 512, AdamW lr 1e-5): RoFormer hội tụ nhanh hơn (chỉ có đường cong ở Fig. 3, không có bảng số).

Fine-tune GLUE (Bảng 2 của paper):

Model	MRPC	SST-2	QNLI	STS-B	QQP	MNLI (m/mm)
BERT	88.9	93.5	90.5	85.8	71.2	84.6/83.4
RoFormer	89.5	90.7	88.0	87.0	86.4	80.2/79.8

RoFormer hơn trên MRPC, STS-B, QQP, kém trên SST-2, QNLI, MNLI. Paper chỉ nhấn mạnh "outperform BERT in three out of six datasets".

Performer (linear attention) + RoPE, Enwik8, 12 layer, d=768, 12 head, độ dài 1024: loss thấp hơn và hội tụ nhanh hơn khi có RoPE (Fig. 3 phải, chỉ là đường cong).

Dữ liệu tiếng Trung (RoFormer sửa từ WoBERT, ~34GB): tiền huấn luyện nhiều giai đoạn; độ chính xác tăng khi độ dài tối đa tăng (stage 1: len 512, loss 1.73, acc 65.0%; stage 2: len 1536, loss 1.61, acc 66.8%; stage 5: len 1536, loss 1.58, acc 67.4%). Lưu ý các giai đoạn có batch và số bước khác nhau nên không so sánh sạch.

CAIL2019-SCM (độ chính xác %):

Model	Validation	Test
BERT-512	64.13	67.77
WoBERT-512	64.07	68.10
RoFormer-512	64.13	68.29
RoFormer-1024	66.07	69.79

Paper: ở 1024 RoFormer hơn WoBERT 1.5% tuyệt đối (so với kết quả 68.10 thì 69.79 − 68.10 ≈ 1.7; con số 1.5% là cách nói của paper, ta chép lại nguyên văn).

5. Phân tích / ablation

- Paper **không có ablation** kiểu bỏ từng thành phần (ví dụ đổi base 10000, thử không xoay một số chiều). Bằng chứng gồm các so sánh tổng thể ở trên.
- Bằng chứng ủng hộ "tổng quát hóa độ dài": bảng tiếng Trung cho thấy độ chính xác tăng khi dùng context dài hơn khi tiền huấn luyện. Đây vẫn *không phải* thí nghiệm ngoại suy (huấn luyện ngắn, kiểm tra dài) của các bài sau như ALiBi (13).
- Hướng "tần số" cho thấy sự đánh đổi mà paper chưa khai thác: $\theta_1 = 1$ rad/token (chu kỳ ≈ 6.3 token) đến $\theta_{d/2} \approx 10000^{-1}$ (chu kỳ ≈ 62,800 token). Cặp chiều tần số cao mã hóa vị trí gần, cặp tần số thấp mã hóa vị trí xa (diễn giải từ công thức).

6. Giới hạn

Do tác giả nêu (§4.5.5): chưa có giải thích kỹ càng vì sao hội tụ nhanh hơn các phương pháp vị trí khác; chưa có lời giải thích chắc chắn vì sao hiệu quả hơn trên văn bản dài dù tính chất suy giảm tương tự các phương pháp khác; cần tài nguyên tiền huấn luyện.

Nhận xét bổ sung:

- Quy mô thí nghiệm nhỏ (BERT-base, T-base); GLUE hỗn hợp (kém hơn BERT ở 3/6 tập). Sức mạnh thực sự của RoPE trong LLM là bằng chứng ngoài paper (GPT-NeoX, LLaMA, PaLM...).
- Paper không thử ngoại suy độ dài ngoài độ dài huấn luyện. Thực tế, RoPE ngoại suy kém nếu không có can thiệp (position interpolation, NTK scaling, YaRN) vì các góc $m\theta_i$ ở vị trí chưa từng thấy là phân phối lạ (nội dung ngoài paper; xem ALiBi 13 để so sánh).
- Tính chất "long-term decay" chỉ là chặn trên gợi ý bằng hình, không chứng minh chặt.
- Tần số cố định (không học), base 10000 là lựa chọn kế thừa từ sinusoidal.

7. Liên hệ với các bài khác

- **01:** sinusoidal tuyệt đối cộng vào embedding; RoPE dùng cùng họ tần số $10000^{-2i/d}$ nhưng *nhân* theo kiểu xoay.
- **13 (ALiBi):** cộng bias tuyến tính $-m|i-j|$ vào score, không có tham số, nhằm ngoại suy độ dài; RoPE xoay q,k. So sánh hai phương án là chủ đề chính của bài 13.
- **05, 06 (LLaMA, Mistral):** dùng RoPE; **16 (MLA):** RoPE không "hấp thụ" được vào ma trận nén KV nên DeepSeek phải tách riêng phần "decoupled RoPE".
- **18-24 (linear attention, RetNet, GLA, Mamba, DeltaNet):** §3.3 là một trong những công trình đầu tiên dùng xoay với linear attention; RetNet dùng một dạng tương tự ("xPos").
- **37 (Qwen2-VL):** mở rộng RoPE thành M-RoPE đa chiều (thời gian/cao/rộng) cho ảnh và video.

8. Gợi ý áp dụng

RoPE ghép các chiều query và key thành cặp để thực hiện phép quay; cài đặt cần giữ cùng quy ước ghép cặp và tần số ở hai phía. Key trong KV cache thường được lưu sau khi áp dụng phép quay theo vị trí. Khi thay đổi chiều hoặc cách mã hoá vị trí, cần đánh giá lại mô hình thay vì giả định hành vi được bảo toàn.

9. Thí nghiệm gợi ý

Với q, k cố định, kiểm tra bằng số rằng tích vô hướng sau phép quay không đổi khi cùng dịch cả hai vị trí đi một lượng; ví dụ so $(m, n) = (5, 3), (105, 103), (1005, 1003)$. Vẽ các tần số khác nhau theo khoảng cách để thấy đóng góp của mỗi cặp chiều.

10. Tóm tắt công thức / khái niệm

Khái niệm	Công thức / ý nghĩa
Yêu cầu	$\langle f_q(x_m, m), f_k(x_n, n)\rangle = g(x_m, x_n, m-n)$
Nghiệm 2D	$f_q = (W_q x_m)e^{im\theta}$, $f_k = (W_k x_n)e^{in\theta}$
Ma trận xoay	$R^d_{\Theta,m} = \text{diag}(R(m\theta_1), \ldots, R(m\theta_{d/2}))$
Tần số	$\theta_i = 10000^{-2(i-1)/d}$
Tính tương đối	$q_m^\top k_n = x_m^\top W_q^\top R^d_{\Theta,n-m} W_k x_n$
Tính chất	trực giao (giữ chuẩn), không tham số, không áp lên v
Suy giảm	$\frac{2}{d}\sum \lvert S_i\rvert$ giảm khi $\lvert m-n\rvert$ tăng (hình số, Fig. 2)
Linear attention	xoay sau φ, ϕ, mẫu số không xoay
Cài đặt	$x \otimes \cos + \text{rot}(x) \otimes \sin$ (Eq. 34)

11. Kết luận ngắn

RoPE mã hóa vị trí bằng cách xoay từng cặp chiều của q và k một góc tỉ lệ với vị trí, nhờ đó tích vô hướng chỉ phụ thuộc khoảng cách $m-n$. Suy diễn gọn (nghiệm của phương trình chức năng ở 2D, mở rộng bằng tổng khối) và cài đặt rẻ, không tham số. Thí nghiệm trong paper nhỏ và không có ngoại suy độ dài, nhưng ý tưởng đã trở thành chuẩn của LLM hiện đại và là nền cho nhiều biến thể (M-RoPE, decoupled RoPE trong MLA).

Nguồn: https://arxiv.org/abs/2104.09864

Bài 13

Train Short, Test Long: Attention with Linear Biases Enables Input Length Extrapolation (2021)

Tác giả: Ofir Press, Noah A. Smith, Mike Lewis (Univ. of Washington, Allen Institute for AI, Facebook AI Research)
Venue / arXiv: arXiv:2108.12409 (v2, 22/4/2022); ICLR 2022 (theo đầu trang paper "Published as a conference paper at ICLR 2022").
Nguồn: https://arxiv.org/abs/2108.12409
Chủ đề: Ngoại suy độ dài (length extrapolation) của Transformer; mã hóa vị trí bằng bias tuyến tính trong score attention (ALiBi).
Vị trí trong lộ trình: Đọc sau 01 (sinusoidal) và 12 (RoPE). Paper này đặt câu hỏi "huấn luyện ngắn, suy luận dài" và đưa ra cách thay thế rất đơn giản. Tiếp theo: 14-16 (giảm KV-cache, vấn đề khác nhưng cũng nhắm tới ngữ cảnh dài), 17 (Longformer), 55 (StreamingLLM: attention sinks, liên quan đến việc chạy ngữ cảnh rất dài).

> Ghi chú diễn giải bằng tiếng Việt, không phải bản dịch. Các ví dụ "tự tính" không phải kết quả trực tiếp của bài báo.

1. Bối cảnh

Huấn luyện trên chuỗi dài L rất đắt (bộ nhớ và thời gian tăng theo L, attention là $O(L^2)$). Nếu mô hình huấn luyện ở L ngắn rồi dùng được ở độ dài suy luận $L_{valid} > L$ thì tiết kiệm lớn. Khả năng đó gọi là **extrapolation** (ngoại suy): hiệu năng không xấu đi khi số token tại lúc kiểm tra vượt số token lúc huấn luyện.

Về lý thuyết các hàm trong Transformer không phụ thuộc độ dài đầu vào, nhưng thực tế:

- **Vị trí học được (learned absolute):** không có embedding cho vị trí $> L$ nên không ngoại suy được.
- **Sinusoidal (Vaswani 2017):** được kỳ vọng ngoại suy, nhưng paper chứng minh thực nghiệm là **rất yếu**: hiệu năng ngừng cải thiện sau ~20-50 token vượt L rồi xấu đi nhanh (Fig. 1).
- **Rotary (RoPE, bài 12):** tốt hơn một chút (cải thiện thêm được khoảng $k = 200$ token với $L = 512$) nhưng vẫn xấu đi, và chậm hơn sinusoidal.
- **T5 bias** (bias học được, phụ thuộc khoảng cách, gộp bucket ở khoảng cách xa): ngoại suy tốt hơn (cải thiện tới $k = 600$ với $L = 512$, $k = 800$ với $L = 1024$) nhưng **chậm gấp ≥2 lần** và tốn bộ nhớ/tham số (T5 bias hết bộ nhớ 32GB GPU ở độ dài ~13,000).

Nên cần một phương pháp vừa ngoại suy tốt vừa rẻ như sinusoidal.

2. Ý tưởng chính

Bỏ hẳn embedding vị trí. Ở mỗi head, cộng vào score $q_i \cdot k_j$ một **hình phạt tuyến tính theo khoảng cách** $-m\,(i - j)$, trong đó m là một hệ số (slope) cố định, không học, khác nhau giữa các head. Token càng xa càng bị trừ điểm nhiều, tức là một thiên kiến "ưu tiên gần" (recency bias). Vì hình phạt chỉ phụ thuộc $i - j$ và tăng đều, nó áp dụng được cho bất kỳ độ dài nào, không có "vị trí lạ" như embedding chưa thấy. Mỗi head có độ dốc khác nhau nên có head nhìn rất gần (dốc lớn), head nhìn xa hơn (dốc nhỏ).

3. Phương pháp chi tiết

3.1 Công thức

Với query thứ i ($1 \le i \le L$) của một head, $q_i \in \mathbb{R}^{1\times d}$, và i key đầu tiên $K \in \mathbb{R}^{i\times d}$ (có causal mask), Transformer gốc:

$$\text{softmax}(q_i K^\top)$$

(paper bỏ hệ số $1/\sqrt{d}$ và các phép chiếu để cho gọn). ALiBi:

$$\boxed{\text{softmax}\big(q_i K^\top + m \cdot [-(i-1), \ldots, -2, -1, 0]\big)}$$

Tức là score giữa query i và key $j \le i$ là $q_i \cdot k_j - m\,(i-j)$. Key cùng vị trí ($j = i$) không bị phạt; key cách Δ bước bị trừ $m\Delta$. Ghi chú nhỏ của paper (chú thích 10): bias ALiBi **không** nhân với $1/\sqrt{d_k}$.

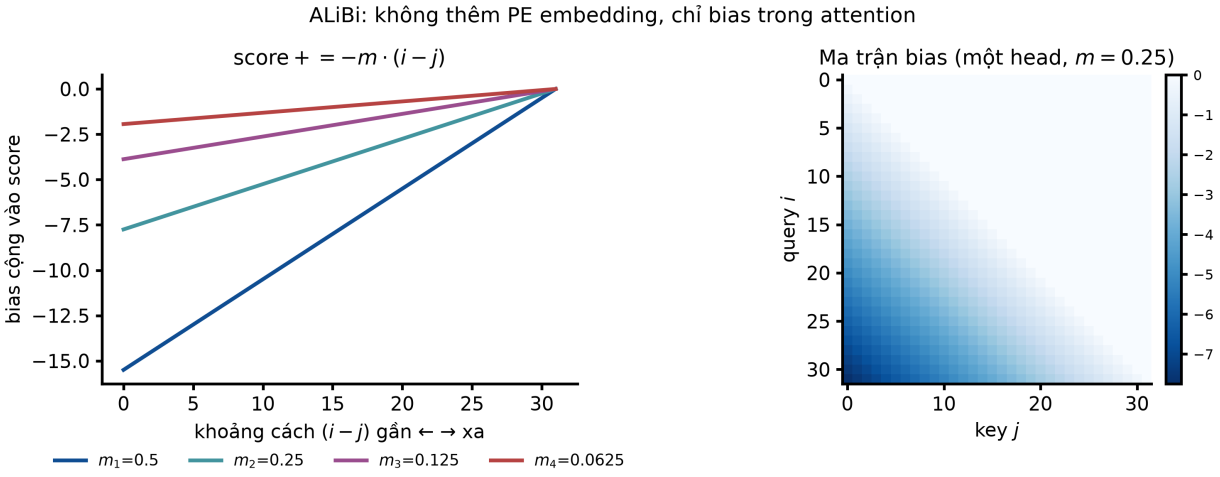
Hình dạng ma trận bias cho $L = 5$ (cho một head, chưa nhân m):

$$\begin{pmatrix} 0 & & & & \\ -1 & 0 & & & \\ -2 & -1 & 0 & & \\ -3 & -2 & -1 & 0 & \\ -4 & -3 & -2 & -1 & 0 \end{pmatrix}$$

ALiBi **không thêm** embedding vị trí ở bất cứ đâu (ở đầu mạng cũng không). Giống RoPE và T5 bias, vị trí được tiêm ở **mọi layer** vào q-k và **không** vào value (paper giả thuyết điều này có lợi cho ngoại suy).

3.2 Chọn slope

Với n head, slope là cấp số nhân bắt đầu và có công bội $2^{-8/n}$:



ALiBi: bias $-m(i-j)$ theo khoảng cách; nhiều slope head. (minh hoạ, tự tính)

$$m_h = 2^{-8h/n}, \quad h = 1, \ldots, n.$$

- $n = 8$: $\frac{1}{2^1}, \frac{1}{2^2}, \ldots, \frac{1}{2^8}$ (tức $\frac{1}{2}, \frac{1}{4}, \ldots, \frac{1}{256}$).
- $n = 16$: nội suy bằng trung bình nhân giữa các cặp liên tiếp: $\frac{1}{2^{0.5}}, \frac{1}{2^1}, \frac{1}{2^{1.5}}, \ldots, \frac{1}{2^8}$ (công bội $2^{-1/2}$, đúng với công thức $2^{-8/16}$).

Các slope này được chọn sau khi thử thủ công khoảng 10 bộ slope. Nhận xét của paper: bộ tốt là các slope trong khoảng $(0, 1)$, mật độ dày hơn gần 0. Slope học được cho kết quả ngoại suy không mạnh (và làm huấn luyện chậm thêm 3%). Slope lấy ngẫu nhiên từ phân phối mũ đôi khi cũng chạy tốt nhưng phương sai cao. Bộ slope này được dùng nguyên cho mọi thí nghiệm sau (sách, 1.3B), không tinh chỉnh lại.

3.3 Ví dụ số (tự tính)

Head có $m = 1/2$, key cách query 10 token: bias $= -5$, hệ số nhân trên xác suất attention $= e^{-5} \approx 0.0067$. Head có $m = 1/256$: bias $= -0.039$, hệ số ≈ 0.96 (gần như không phạt). Với khoảng cách 1,000: $m = 1/256$ cho bias ≈ -3.9 (hệ số ≈ 0.02); $m = 1/2$ cho -500 (loại bỏ hoàn toàn). Như vậy các head sau (slope nhỏ) có tầm nhìn xa, các head đầu chỉ nhìn rất gần.

3.4 Cài đặt

```
def alibi_slopes(n_heads):                        # n_heads là lũy thừa của 2 (đơn giản hóa)
    start = 2 ** (-8 / n_heads)
    return torch.tensor([start ** (h + 1) for h in range(n_heads)])

def alibi_bias(n_heads, L):
    i = torch.arange(L)[:, None]; j = torch.arange(L)[None, :]
    dist = (i - j).clamp(min=0)                    # i - j cho j <= i
    return -alibi_slopes(n_heads)[:, None, None] * dist    # [n_heads, L, L]

# attention:
scores = q @ k.transpose(-1, -2)                   # [B, n_heads, L, L]
scores = scores + alibi_bias(n_heads, L) + causal_mask
attn = scores.softmax(-1)
```

Paper: cài đặt bằng cách cộng bias vào mask nên **không thêm phép tính** nào vào mạng. Chi phí duy nhất: mask từ $L \times L$ thành $n \times L \times L$ (n là số head), tốn thêm tới ~100MB bộ nhớ trong một số thí nghiệm (0-0.7% theo giới thiệu). Với FlashAttention hiện đại thì bias được tính trong kernel nên không phải vật liệu hóa ma trận (nội dung ngoài paper).

4. Thí nghiệm và kết quả chính

4.1 WikiText-103 (mô hình Baevski & Auli: 16 layer, d=1024, 8 head, FFN 4096; ~103M token huấn luyện)

Tốc độ / bộ nhớ (Bảng 1 phụ lục, 1 GPU V100; từ/giây khi train và eval; bộ nhớ khi train):

Phương pháp	L train	Train WPS	Eval WPS	Bộ nhớ
Sinusoidal	512	28.5k	82.1k	15.3 GB
Sinusoidal	1024	26.0k	77.8k	19.2 GB
Rotary	512	20.0k	43.4k	17.8 GB
T5 bias	512	14.4k	21.8k	16.9 GB
T5 bias	3072	4.3k	4.9k	15.9 GB
ALiBi	512	28.3k	85.8k	15.3 GB
ALiBi	1024	25.8k	76.4k	19.3 GB
ALiBi	3072	15.5k	42.2k	15.2 GB

(Chú thích paper: sau này được báo là code rotary có thể tối ưu để chỉ chậm hơn sinusoidal 2%.)

Perplexity trên tập dev khi ngoại suy (một phần Bảng 2, $L = 512$):

L_{valid}	Sinusoidal	Rotary	T5 bias	ALiBi
512	20.05	20.07	19.65	19.73
1012	43.54	21.37	18.79	18.73
2512	132.41	31.58	20.41	18.41
5512	240.44	43.14	29.54	18.36
12512	373.17	69.70	64.94	18.31
15512	406.01	79.25	OOM ở 13512+	18.31

(Ở $L_{valid} \geq 13512$ T5 bias báo OOM.) Mô hình ALiBi huấn luyện ở 512 token đạt 19.73 ở độ dài 512 và 18.40 khi ngoại suy lên 3072, và paper nhấn mạnh con số này tốt hơn mô hình sinusoidal **huấn luyện** ở 3072 (18.67).

Train L=3072, $L_{valid} = 3072$ **(Bảng 4):** Sinusoidal 18.67, Rotary 18.57, T5 bias 18.01, ALiBi 17.60.

ALiBi ngoại suy (Bảng 5, dev, một phần):

Train L eval	512	1024	2048	3072
512	19.73	18.81	18.48	18.40
1024	–	18.66	18.05	17.96
2048	–	–	17.91	17.64
3072	–	–	–	17.60
Sinusoidal (L=Lvalid)	20.05	19.34	18.87	18.67

Mô hình ALiBi L=512 huấn luyện nhanh hơn 1.84 lần so với sinusoidal L=3072 mà vẫn có perplexity thấp hơn khi đánh giá ở 3072.

4.2 Toronto Book Corpus

Dùng nguyên slope, kết quả "chuyển giao hoàn toàn" sang miền sách (Phụ lục A.3; không chép bảng số ở đây phần này không trích bảng số).

4.3 CC100+RoBERTa (461 GB), mô hình 1.3B (25 layer, 16 head, d=2048, FFN 8192), 1 epoch = 50k updates trên 128 V100

- ALiBi $L = 512$ vs sinusoidal $L = 1024$ (đánh giá ở 1024): nhanh hơn 7%, ít hơn 1.6 GB bộ nhớ, khi kết thúc chỉ chênh 0.06 perplexity.
- ALiBi $L = 1024$ vs sinusoidal $L = 2048$ (đánh giá ở 2048): ALiBi tốt hơn 0.09 perplexity, ít hơn 3.1 GB bộ nhớ, đạt cùng perplexity trung bình nhanh hơn 11%.
- Mô hình $L = 512$ (9.79 ở $L_{valid} = 512$) đạt tốt nhất 9.3 ở 1012 token; mô hình $L = 1024$ (9.16 ở 1024) đạt tốt nhất 8.9 ở 2024 token. Sinusoidal không ngoại suy được chút nào ở cấu hình này.
- Khi cùng độ dài L (không ngoại suy), ALiBi tương đương sinusoidal trên bộ dữ liệu lớn (Bảng 12 phụ lục), trong khi trên dữ liệu nhỏ ALiBi tốt hơn rõ.

5. Phân tích / ablation

- **Extrapolation phụ thuộc vào phương pháp vị trí** (thay đổi duy nhất giữa các mô hình WikiText-103): sinusoidal < rotary < T5 bias < ALiBi.
- **Tốt nhất khoảng** $2L$**:** trên 1.3B, hiệu năng tốt nhất ở ~2L, sau đó phẳng hoặc hơi kém nhưng không sụp (vẫn ổn tới 10,000 token). Giải thích (giả thuyết của tác giả): với $L_{valid} \leq 2L$, ít nhất nửa số dự đoán nằm ở ngữ cảnh dài bằng những gì đã thấy.

- **Early token curse:** paper (§4.2, Phụ lục B) kết luận lợi thế của ALiBi so với sinusoidal phần lớn do tránh được "early token curse" (token đầu mỗi đoạn có ít ngữ cảnh nên khó dự đoán); mô hình chưa khai thác hiệu quả ngữ cảnh dài hơn nữa.
- **Slope:** học được slope không giúp ngoại suy; nhân (thay vì cộng) bias làm giảm chất lượng (nhận xét trong phần Related Work, không trình bày số).
- Chi phí so với sinusoidal cùng độ dài: tốc độ chênh trong 1% khi train, 3% khi inference; bộ nhớ +0-0.7%.

6. Giới hạn

Do tác giả nêu: hiệu năng đạt đỉnh quanh $2L$ và không tiếp tục cải thiện nhiều; mô hình chưa khai thác hiệu quả ngữ cảnh dài; tối ưu với bộ slope được chọn sau thử khoảng 10 bộ (không tối ưu hóa hệ thống); không so sánh được các kỹ thuật trực giao (kNN-LM, Routing Transformer) vì chúng đạt kết quả cao hơn trên WikiText-103 và có thể kết hợp.

Nhận xét bổ sung:

- Chỉ đánh giá bằng **perplexity**, không có thí nghiệm tác vụ yêu cầu truy xuất chính xác ở xa (needle-in-haystack, v.v.). Perplexity phẳng có thể che giấu việc mô hình thực ra chỉ dùng ngữ cảnh gần: bias tuyến tính thực sự **triệt tiêu** dần mọi token xa với head có slope lớn, và các nghiên cứu sau cho thấy ngoại suy perplexity ≠ khả năng sử dụng ngữ cảnh dài (nội dung ngoài paper, nên xem như nhận định cần kiểm tra).
- Thiên kiến "ưu tiên gần" cứng nhắc: tác vụ cần nhìn lại nội dung xa (ví dụ OCR cần đối chiếu tiêu đề với phần cuối trang) có thể khó hơn.
- Thí nghiệm chỉ với mô hình ngôn ngữ nguyên nhân (causal LM) thuần văn bản, không có encoder hai chiều hay đa phương thức.
- Về sau các LLM lớn chủ yếu chọn RoPE (có position interpolation) thay vì ALiBi (BLOOM, MPT dùng ALiBi; LLaMA, Qwen, Mistral dùng RoPE), nên đây là kiến thức bối cảnh, ngoài paper.

7. Liên hệ với các bài khác

- **01 (Attention Is All You Need):** sinusoidal là baseline thất bại trong ngoại suy.
- **12 (RoPE):** cùng thuộc nhóm "tiêm vị trí vào q-k ở mọi layer, không vào v"; khác nhau ở cách tiêm: RoPE nhân (xoay), ALiBi cộng bias.
- **17 (Longformer):** sliding window là cách cứng để giới hạn tầm nhìn; ALiBi là cách mềm (phạt tuyến tính).
- **06 (Mistral 7B):** sliding window attention + RoPE.
- **55 (StreamingLLM):** phân tích vì sao chạy ngữ cảnh rất dài cần giữ "attention sink"; liên quan khi thấy perplexity phẳng ở ngữ cảnh rất dài.
- **18-24 (linear attention, Mamba, DeltaNet):** các mô hình hồi quy có "độ dài ngoại suy" tự nhiên vì trạng thái kích thước cố định (ALiBi cố gắng đạt hiệu ứng tương tự bằng một bias phân rã theo khoảng cách; cho thấy phân rã (decay) là thiên kiến quy nạp hữu ích).

8. Gợi ý áp dụng

ALiBi đặt bias khoảng cách cố định với độ dốc riêng theo head, làm các head có xu hướng ưu tiên vùng gần ở mức khác nhau. Nó không tự thu nhỏ KV cache. Ngoại suy perplexity là một chỉ dấu hữu ích nhưng chưa đủ để kết luận mô hình truy hồi hiệu quả thông tin ở mọi khoảng cách.

9. Thí nghiệm gợi ý

Huấn luyện các mô hình nhỏ với ALiBi và các phương pháp vị trí khác trên cùng độ dài chuỗi; đo perplexity ở các độ dài trong và ngoài miền huấn luyện. Bổ sung một bài kiểm tra truy hồi theo khoảng cách để xem xu hướng perplexity có đi kèm khả năng sử dụng thông tin xa không.

10. Tóm tắt công thức / khái niệm

Khái niệm	Công thức / ý nghĩa
Score ALiBi	$q_i \cdot k_j - m\,(i-j),\ j \le i$
Slope	$m_h = 2^{-8h/n}$, $h = 1..n$ (n=8: 1/2..1/256)
Không embedding vị trí	chỉ bias trong score, mọi layer, không vào v
Extrapolation	$L_{valid} > L_{train}$ mà hiệu năng không xấu đi
Kết quả chính	train 512 → 3072: 18.40 (so với sinusoidal train 3072: 18.67)
Hiệu quả 1.3B	train 1024 ≈ sinusoidal train 2048: nhanh hơn 11%, ít hơn 11% bộ nhớ
Chi phí	không phép tính thêm; mask $n \times L \times L$ (≤100MB thêm)
Điểm tốt nhất	khoảng $2L$

11. Kết luận ngắn

ALiBi chứng minh rằng ngoại suy độ dài phụ thuộc vào cách mã hóa vị trí, và một bias tuyến tính theo khoảng cách (slope khác nhau theo head, cố định) là đủ để huấn luyện ngắn, suy luận dài với chi phí gần bằng không. Bằng chứng chỉ ở perplexity trên mô hình ngôn ngữ; hiệu quả sử dụng ngữ cảnh dài thực sự không được kiểm chứng. Trong thực hành sau này RoPE + kỹ thuật mở rộng ngữ cảnh chiếm ưu thế, nhưng ý tưởng “decay theo khoảng cách theo từng head” vẫn xuất hiện trong các kiến trúc hồi quy hiện đại.

Nguồn: https://arxiv.org/abs/2108.12409

Bài 14

Fast Transformer Decoding: One Write-Head is All You Need (2019)

Tác giả: Noam Shazeer (Google)
Venue / arXiv: arXiv:1911.02150 (cs.NE, v1, 6/11/2019); paper không nêu venue hội nghị.
Nguồn: https://arxiv.org/abs/1911.02150
Chủ đề: Multi-Query Attention (MQA): cả h head dùng chung một bộ K và V để giảm kích thước KV-cache và băng thông bộ nhớ khi giải mã tuần tự.
Vị trí trong lộ trình: Đọc sau 01 (multi-head attention) và 05/06 (LLaMA, Mistral, nơi KV-cache trở thành vấn đề thực tế). Paper này đặt nền cho phân tích "decode bị giới hạn bởi băng thông bộ nhớ" và đưa ra giải pháp cực đoan (1 KV head). Tiếp theo: 15 (GQA, điểm giữa MHA và MQA), 16 (MLA, nén KV bằng low-rank), 44-46 (FlashAttention, PagedAttention: các góc nhìn khác về bộ nhớ attention).

> Ghi chú diễn giải bằng tiếng Việt; các ví dụ đánh dấu "tự tính" là phép tính minh họa.

1. Bối cảnh

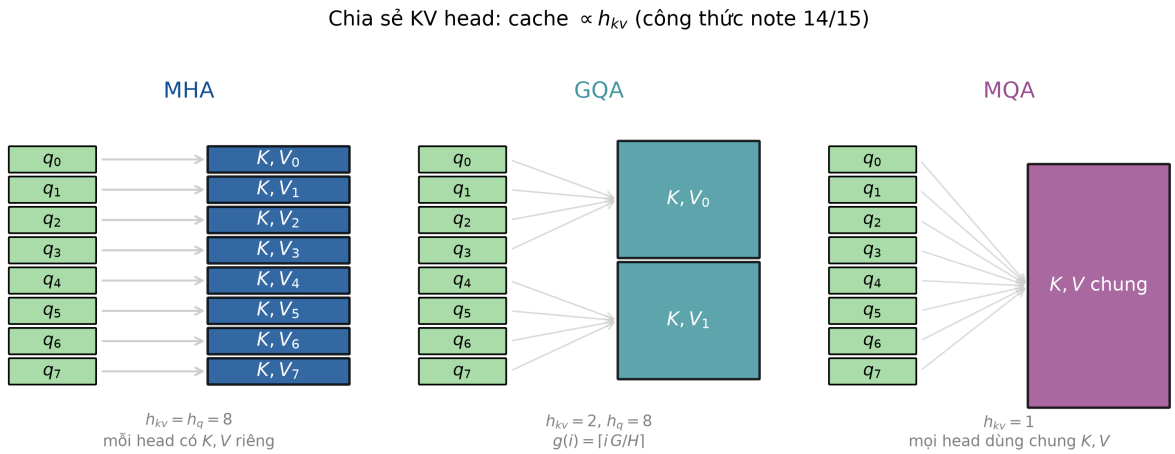
Huấn luyện song song, sinh chữ tuần tự. Khi huấn luyện, ta biết cả chuỗi đích nên tính attention cho mọi vị trí cùng lúc (một matmul lớn). Khi **sinh** (autoregressive decoding), token $t+1$ phụ thuộc vào đầu ra bước t nên mỗi bước chỉ xử lý **một** token mới. Để không tính lại, ta lưu **KV-cache**: các vector key và value của mọi token trước đó ở mọi layer; mỗi bước chỉ tính K, V cho token mới rồi nối vào cache.

Vấn đề của bước tuần tự. Mỗi bước, GPU phải đọc lại toàn bộ KV-cache (kích thước tăng theo độ dài n) và toàn bộ trọng số, nhưng chỉ làm ít phép tính (một token). Khái niệm cần nhớ: **arithmetic intensity** = số phép tính / số byte phải đọc từ bộ nhớ. GPU/TPU hiện đại có sức tính gấp hàng trăm lần băng thông bộ nhớ (paper: "two orders of magnitude"), nên nếu tỉ lệ này thấp thì máy **chờ dữ liệu** chứ không bận tính: gọi là **memory-bandwidth-bound**.

2. Ý tưởng chính

Trong multi-head attention (MHA) mỗi head có K, V riêng, nên cache có chiều "head" h. Multi-query attention (MQA) giữ h **query** head khác nhau nhưng cho cả h head **dùng chung một bộ K và một bộ V**. Cache nhỏ đi đúng h lần, việc đọc cache ở mỗi bước nhẹ đi h lần. Trong thí nghiệm của bài báo, decode nhanh hơn và chất lượng giảm nhẹ. Cách cài đặt cần thay đổi các phép chiếu K, V và cách lưu cache.

3. Phương pháp chi tiết



Chia sẻ KV head: MHA ($h_{kv} = h_q$), GQA ($g(i) = \lceil iG/H \rceil$), MQA ($h_{kv} = 1$); cache tỉ lệ với h_{kv}. (minh hoạ, tự tính)

3.1 Multi-head attention (một bước, bỏ batch)

Ký hiệu của paper: d = chiều mô hình; h = số head; k, v = chiều key/value mỗi head; m = số vị trí trong bộ nhớ (độ dài ngữ cảnh). Với đầu vào $x \in \mathbb{R}^d$ và $M \in \mathbb{R}^{m \times d}$:

$$q_h = xP_{q,h}, \quad K_h = MP_{k,h}, \quad V_h = MP_{v,h}, \quad o_h = \text{softmax}(q_h K_h^\top)V_h, \quad y = \sum_h o_h P_{o,h}$$

(paper bỏ hệ số $1/\sqrt{d_k}$ vì gộp vào P_q). $P_q, P_k \in \mathbb{R}^{h \times d \times k}$, $P_v, P_o \in \mathbb{R}^{h \times d \times v}$. Cache lưu K, V cỡ $[b, h, m, k]$.

3.2 Multi-query attention

$$\boxed{K = MP_k,\ V = MP_v \ \ (P_k \in \mathbb{R}^{d \times k},\ P_v \in \mathbb{R}^{d \times v} \text{ không có chiều } h); \quad o_h = \text{softmax}(q_h K^\top)V}$$

Mỗi head vẫn có $P_{q,h}$ và $P_{o,h}$ riêng nên các head vẫn "hỏi" khác nhau, chỉ là cùng nhìn vào một bộ khóa/giá trị. Cache cỡ $[b, m, k]$ thay cho $[b, h, m, k]$.

3.3 Pseudo-code (PyTorch tương đương với einsum trong paper)

```
# x: [B, d] (một token mới), prev_K/prev_V: [B, m, dk] (MQA) hoặc [B, h, m, dk] (MHA)
q = einsum('bd,hdk->bhk', x, Pq)                # h query head
k_new = einsum('bd,dk->bk', x, Pk)              # MQA: KHÔNG có chữ h
v_new = einsum('bd,dv->bv', x, Pv)
K = cat([prev_K, k_new[:, None]], dim=1)        # [B, m+1, dk]
V = cat([prev_V, v_new[:, None]], dim=1)
logits = einsum('bhk,bmk->bhm', q, K)           # mọi head dùng chung K
w = softmax(logits, -1)
o = einsum('bhm,bmv->bhv', w, V)
y = einsum('bhv,hdv->bd', o, Po)
```

Trong PyTorch thực tế: K,V có `num_kv_heads=1` rồi `expand`/broadcast sang h head, không sao chép bộ nhớ.

3.4 Phân tích hiệu năng của paper (đơn giản hóa $m = n$, $k = v = d/h$, $n \le d$)

MHA, huấn luyện / xử lý theo lô (§2.3.1): số phép tính $\Theta(bnd^2)$; bộ nhớ truy cập $O(bnd + bhn^2 + d^2)$; tỉ lệ bộ nhớ/tính $O(\frac{1}{k} + \frac{1}{bn})$, rất thấp, tốt.

MHA, giải mã tăng dần (§2.4.1): qua n lần gọi, tính $\Theta(bnd^2)$; bộ nhớ truy cập $\Theta(bn^2d + nd^2)$ (số hạng đầu do đọc lại K,V ở mỗi bước, mỗi lần cỡ $bhmk = bnd$); tỉ lệ

$$\text{memory/compute (MHA, incremental)} = \Theta\Big(\frac{n}{d} + \frac{1}{b}\Big).$$

Khi $n \approx d$ hoặc $b \approx 1$ tỉ lệ ≈ 1, tức nút cổ chai băng thông. Số hạng $1/b$ giảm bằng cách tăng batch; số hạng n/d do cache KV.

MQA (§3.1): bộ nhớ truy cập $\Theta(bnd + bn^2k + nd^2)$ (K,V nhỏ hơn h lần vì $k = d/h$): tỉ lệ

$$\boxed{\text{memory/compute (MQA, incremental)} = \Theta\Big(\frac{1}{d} + \frac{n}{dh} + \frac{1}{b}\Big)}$$

Số hạng "xấu" n/d giảm h lần. Lưu ý: số hạng $1/b$ (đọc trọng số d^2 cho mỗi bước) không được MQA giải quyết; nó chỉ giảm khi tăng batch (chi tiết ở §4 dưới đây về hệ quả với batch 1).

4. Phân tích KV-cache và băng thông (tự tính)

4.1 Công thức kích thước KV-cache

Cho L layer, h_{kv} số KV head, $d_k = d_v = d_h$ chiều mỗi head, độ dài n, batch b, s byte/phần tử:

$$\boxed{\text{KV bytes} = 2 \cdot L \cdot h_{kv} \cdot d_h \cdot n \cdot b \cdot s}$$

(hệ số 2 cho K và V). MHA: $h_{kv} = h$. MQA: $h_{kv} = 1$ (thu nhỏ h lần).

4.2 Kích thước cache và băng thông

Với L tầng, H_{kv} head key/value, chiều d_h, n token và s byte mỗi phần tử, KV cache của một chuỗi có $2LH_{kv}d_hns$ byte. Nếu giữ nguyên các yếu tố khác, MQA dùng một KV head thay vì H head nên cache nhỏ hơn H lần so với MHA.

Ở batch lớn, cache tăng theo số chuỗi trong khi trọng số có thể được dùng chung. Vì vậy mức tiết kiệm bộ nhớ có thể quan trọng hơn khi batch hoặc ngữ cảnh tăng; thời gian thực tế cần được đo trên hệ thống cụ thể.

5. Thí nghiệm và kết quả chính

Thiết lập (§4.1):

- Dịch WMT 2014 En-De: Transformer encoder-decoder 6 layer, $d_{model} = 1024$, $d_{ff} = 4096$, $h = 8$, $d_k = d_v = 128$, 211M tham số, 100k bước (~20 epoch), batch 128 mẫu (input 256 token, target 256 token), TPUv3 32 core (~2 giờ mỗi mô hình).
- Mô hình MQA: thay **mọi** lớp attention (encoder self-attn, decoder self-attn, encoder-decoder attn) bằng MQA; nới d_{ff} 4096 → 5440 để giữ nguyên số tham số.
- Biến thể "local": decoder self-attention chỉ nhìn vị trí hiện tại và 31 vị trí trước.
- Baseline giảm kích thước KV bằng cách giảm h và d_k, d_v (nới d_{ff} lên 6784 để cùng tham số).
- Mô hình ngôn ngữ decoder-only trên Billion-Word benchmark: 6 layer, $d_{model} = 1024$, $d_{ff} = 8192$, $h = 8$, $d_k = d_v = 128$, 192M tham số, 136k bước.

Bảng 1: WMT14 En-De

Attention	h	d_k, d_v	d_{ff}	ln(PPL) dev	BLEU dev	BLEU test (beam 1 / 4)
multi-head	8	128	4096	1.424	26.7	27.7 / 28.4
multi-query	8	128	5440	1.439	26.5	27.5 / 28.5
multi-head local	8	128	4096	1.427	26.6	27.5 / 28.3
multi-query local	8	128	5440	1.437	26.5	27.6 / 28.2
multi-head	1	128	6784	1.518	25.8	–
multi-head	2	64	6784	1.480	26.2	26.8 / 27.9
multi-head	4	32	6784	1.488	26.1	–
multi-head	8	16	6784	1.513	25.8	–

Bảng 3: Billion-Word LM (perplexity theo từ, dev)

Attention	h	d_k, d_v	d_{ff}	dev-PPL
multi-head	8	128	8192	29.9
multi-query	8	128	9088	30.2
multi-head	1	128	9984	31.2
multi-head	2	64	9984	31.1
multi-head	4	32	9984	31.0
multi-head	8	16	9984	30.9

Bảng 2: Tốc độ (TPUv2-micro giây trên mỗi token đầu ra; độ dài chuỗi 128; suy luận batch 1024 chuỗi, tức 128 mỗi core)

Attention	Train (enc+dec)	Inference (enc + dec)	Beam-4 (enc + dec)
multi-head	13.2	1.7 + 46	2.0 + 203
multi-query	13.0	1.5 + 3.8	1.6 + 32
multi-head local	13.2	1.7 + 23	1.9 + 47
multi-query local	13.0	1.5 + 3.3	1.6 + 16

Chi tiết (§4.3): mỗi bước decoder của baseline 47 ms, của MQA 3.9 ms (batch 1024 chuỗi); encoder 222 ms vs 195 ms. Bước huấn luyện 433 ms (multi-head) vs 425 ms (MQA).

Tóm lại: decode nhanh hơn ≈ 12 lần (46 → 3.8 µs/token) với độ giảm chất lượng nhỏ (BLEU dev 26.7 → 26.5; PPL LM 29.9 → 30.2), còn hơn nhiều so với cách giảm h hoặc d_k.

6. Phân tích / ablation

- **So với "giảm kích thước K,V bằng cách giảm h hoặc d_k" (cùng tham số):** những biến thể đó kém hơn MQA rõ rệt ở cả dịch (ln PPL 1.48-1.52 so với 1.439) lẫn LM (30.9-31.2 so với 30.2). Nghĩa là giữ nhiều **query head** và chia sẻ K,V hiệu quả hơn là giảm chung số head.
- **Trực giao với local attention:** MQA + local (Bảng 2) nhanh nhất (3.3 µs ở decoder), chứng tỏ hai kỹ thuật cộng dồn.
- **Beam search được lợi nhiều nhất:** 203 → 32 µs (decoder), vì beam nhân KV-cache lên.
- Huấn luyện không chậm đi (13.0 vs 13.2).
- Để giữ cùng số tham số, MQA phải mở rộng FFN (4096 → 5440): một phần khác biệt chất lượng có thể bù bởi FFN lớn hơn. Đây là điểm thiết kế cần để ý khi so sánh công bằng.

7. Giới hạn

Do tác giả nêu: chất lượng giảm "slightly" so với baseline; paper không thêm giải thích sâu; cấu hình huấn luyện "to be added before publication" (chưa đủ chi tiết tái lập).

Nhận xét bổ sung:

- Quy mô nhỏ (~200M tham số), tác vụ dịch và Billion-Word LM; chưa rõ với LLM nhiều tỷ tham số và ngữ cảnh dài. Các bài sau (GQA, bài 15) báo cáo MQA có thể gây mất ổn định huấn luyện và giảm chất lượng ở mô hình lớn (nội dung ngoài paper, xem bài 15).
- Đo tốc độ trên TPUv2 với batch 1024 chuỗi,; tốc độ "12 lần" không thể chuyển thẳng sang cấu hình khác.
- Có cài đặt giả định thêm: padding đến độ dài tối đa nên mỗi bước decode tốn thời gian như nhau; cài đặt tối ưu có thể nhanh hơn ở đầu chuỗi (chú thích 1 trong paper).
- Chia sẻ K,V làm giảm dung lượng biểu diễn của attention (tất cả head đọc cùng một "bộ nhớ"): chưa có phân tích về mất mát khả năng.
- Giải mã suy đoán (bài 48-50) và batch lớn thay đổi cân bằng tính toán/băng thông, nên lợi ích MQA tương đối thay đổi theo bối cảnh.

8. Liên hệ với các bài khác

- **01:** multi-head attention gốc; MQA là một biến thể chia sẻ K,V.
- **15 (GQA):** nội suy giữa MHA (h KV head) và MQA (1 KV head) bằng g nhóm; cho phép "uptrain" từ checkpoint MHA.
- **16 (MLA):** nén KV bằng low-rank (còn tốt hơn MQA về cache với chất lượng ≥ MHA theo DeepSeek).
- **46 (PagedAttention/vLLM):** quản lý bộ nhớ KV-cache; **56 (H2O)**, **55 (StreamingLLM)**: bỏ bớt token trong cache; **44-45 (FlashAttention):** giảm truy cập bộ nhớ khi tính attention (bài toán khác: bộ nhớ activation, không phải cache).
- **05, 06:** LLaMA 2 70B, Mistral 7B dùng GQA (kiến thức ngoài paper này).
- **18-24:** các mô hình hồi quy/linear attention loại bỏ hẳn KV-cache tăng theo n (trạng thái cố định), mục tiêu giống MQA nhưng bằng cách khác.

9. Gợi ý áp dụng

MQA chia sẻ K và V giữa các query head, nên giảm kích thước KV cache theo số head khi các chiều còn lại cố định. Hiệu quả độ trễ phụ thuộc độ dài, batch, phần cứng và kernel: giảm cache không đồng nghĩa toàn bộ thời gian suy luận giảm cùng tỉ lệ. GQA (bài 15) khảo sát điểm trung gian giữa MHA và MQA.

10. Thí nghiệm gợi ý

Đo thời gian decode và bộ nhớ theo độ dài ngữ cảnh với MHA và MQA trên cùng phần cứng, sau đó đối chiếu với công thức cache. Báo cáo batch, độ chính xác số và cách triển khai, vì các yếu tố này ảnh hưởng so sánh.

11. Tóm tắt công thức / khái niệm

Khái niệm	Công thức / ý nghĩa
KV-cache	lưu K,V các token trước để không tính lại
Kích thước KV	$2\,L\,h_{kv}\,d_h\,n\,b\,s$ byte
MQA	$K = MP_k, V = MP_v$ dùng chung cho mọi head; query vẫn h head
Tỉ lệ bộ nhớ/tính (MHA, tăng dần)	$\Theta(n/d + 1/b)$
Tỉ lệ (MQA, tăng dần)	$\Theta(1/d + n/(dh) + 1/b)$

Khái niệm	Công thức / ý nghĩa
Giảm cache	đúng h lần (h_{kv}: $h \to 1$)
Tỉ lệ cache	Với các chiều khác cố định, MQA dùng $1/h$ cache của MHA
Kết quả	decode 46 → 3.8 µs/token; BLEU dev 26.7 → 26.5; LM PPL 29.9 → 30.2

12. Kết luận ngắn

MQA nhận ra rằng chi phí giải mã tăng dần là chi phí **đọc bộ nhớ**, đặc biệt đọc lại KV-cache, và loại bỏ chiều "head" của K,V để giảm chi phí đó đúng h lần. Đánh đổi chất lượng nhỏ trên thí nghiệm của paper. Đây là gốc của GQA và MLA; mức cải thiện thực tế phụ thuộc tỉ lệ chi phí đọc cache và trọng số.

Nguồn: https://arxiv.org/abs/1911.02150

Bài 15

GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints (2023)

Tác giả: Joshua Ainslie, James Lee-Thorp, Michiel de Jong (đóng góp ngang nhau), Yury Zemlyanskiy, Federico Lebrón, Sumit Sanghai (Google Research)
Venue / arXiv: arXiv:2305.13245 (cs.CL, v3, 23/12/2023); văn bản paper không nêu tên hội nghị.
Nguồn: https://arxiv.org/abs/2305.13245
Chủ đề: Grouped-Query Attention (GQA) và quy trình "uptraining" biến checkpoint multi-head (MHA) thành multi-query (MQA) hoặc GQA với ~5% chi phí tiền huấn luyện.
Vị trí trong lộ trình: Đọc sau 14 (MQA: nút cổ chai băng thông KV-cache) và 01. Paper này bổ sung: (1) nội suy giữa MHA và MQA bằng G nhóm; (2) công thức chuyển checkpoint có sẵn bằng mean-pooling rồi huấn luyện tiếp một lượng nhỏ. Tiếp theo: 16 (MLA, hướng nén KV khác), 05/06 (LLaMA-2 70B, Mistral dùng GQA), 56 (H2O, cắt KV-cache).

> Ghi chú diễn giải, không phải bản dịch. Bài báo thử nghiệm trên T5 encoder-decoder; việc dùng GQA ở decoder-only là thực hành ngoài phạm vi thí nghiệm đó.

1. Bối cảnh

Như bài 14: khi sinh chữ tuần tự, mỗi bước phải đọc toàn bộ trọng số và toàn bộ **KV-cache** (key/value của các token trước), nên tốc độ bị giới hạn bởi băng thông bộ nhớ. **MQA** (1 cặp K/V cho mọi query head) giảm cache H lần nhưng:

- Chất lượng có thể giảm và **huấn luyện có thể mất ổn định** (paper nêu "quality degradation and training instability").
- Huấn luyện một mô hình riêng chỉ để suy luận nhanh thì tốn kém; nhiều mô hình công khai (T5, LLaMA) vốn là MHA.
- Mô hình lớn thường tăng số head, nên MQA (luôn về 1 KV head) là một cú cắt càng lúc càng "cực đoan" về dung lượng. GQA giữ được tỉ lệ giảm theo quy mô mô hình.

Hai câu hỏi: làm sao có MQA mà không phải huấn luyện từ đầu? Và có điểm trung gian giữa MHA và MQA tốt hơn không?

2. Ý tưởng chính

(1) **Uptraining:** lấy checkpoint MHA, gộp các head K/V bằng cách lấy **trung bình ma trận chiếu** thành 1 head (MQA) hoặc G head (GQA), rồi huấn luyện tiếp một phần nhỏ α (như 5%) số bước tiền huấn luyện gốc theo đúng công thức cũ. (2) **GQA:** chia H query head thành G nhóm; mỗi nhóm chia sẻ một cặp K/V. $G = H$ là MHA, $G = 1$ là MQA. Với G trung gian (paper chọn 8), chất lượng gần MHA và tốc độ gần MQA.

3. Phương pháp chi tiết

3.1 Định nghĩa GQA

Có H query head $q_1..q_H$ và G cặp K/V $(K_1, V_1), \ldots, (K_G, V_G)$ (G chia hết H). Query head i thuộc nhóm $g(i) = \lceil iG/H \rceil$ và dùng $(K_{g(i)}, V_{g(i)})$:

$$o_i = \text{softmax}\big(q_i K_{g(i)}^\top / \sqrt{d_h}\big) V_{g(i)}, \qquad g(i) = \lceil iG/H \rceil$$

- $G = H$: mỗi head có K/V riêng (MHA).
- $G = 1$: tất cả head chung K/V (MQA).
- Số query head mỗi nhóm: H/G.

KV-cache theo công thức bài 14: $2 \cdot L \cdot G \cdot d_h \cdot n \cdot b \cdot s$ byte. So với MHA, cache nhỏ đi H/G lần.

3.2 Chuyển checkpoint MHA → GQA (uptraining)

Bước 1, **chuyển đổi:** với mỗi nhóm g, ma trận chiếu K (và V) của nhóm là **trung bình** các ma trận chiếu của các head gốc trong nhóm:

$$W_K^{(g)} = \frac{G}{H} \sum_{i \in \text{nhóm } g} W_{K,i}, \qquad W_V^{(g)} = \frac{G}{H} \sum_{i \in \text{nhóm } g} W_{V,i}$$

Với $G = 1$ đó là trung bình mọi head (MQA). Bước 2, **huấn luyện tiếp** α phần tổng số bước ban đầu, cùng dữ liệu và công thức tiền huấn luyện.

Paper so ba cách khởi tạo head gộp: **mean pooling** (tốt nhất), chọn **head đầu tiên**, hoặc **khởi tạo ngẫu nhiên**. Trực giác của paper: thứ hạng theo mức độ bảo toàn thông tin của mô hình tiền huấn luyện.

Quan trọng: ma trận chiếu query W_Q, ma trận ra W_O và phần còn lại **giữ nguyên**; chỉ phần K/V thay đổi và mô hình thích nghi trong lúc huấn luyện tiếp.

3.3 Pseudo-code

```
def mha_to_gqa(Wk, Wv, H, G):
    """Wk, Wv: [H, d_model, d_h]  ->  [G, d_model, d_h] bằng mean-pooling theo nhóm."""
    Wk = Wk.view(G, H // G, *Wk.shape[1:]).mean(dim=1)
    Wv = Wv.view(G, H // G, *Wv.shape[1:]).mean(dim=1)
    return Wk, Wv

# Uptraining: model.load(mha_ckpt); mha_to_gqa trên mọi attention layer (decoder self +
↪  cross);
# rồi train thêm alpha * T_pretrain bước (alpha=0.05) với dữ liệu + lịch học của tiền huấn
↪  luyện gốc.

def gqa_attention(q, k, v, H, G):        # q:[B,H,T,d]  k,v:[B,G,S,d]
    k = k.repeat_interleave(H // G, dim=1)   # broadcast; thực tế dùng expand/kernel hỗ trợ
↪  GQA
    v = v.repeat_interleave(H // G, dim=1)
    return softmax(q @ k.transpose(-1, -2) / d**0.5) @ v
```

3.4 Chi phí theo số nhóm

Cache của GQA tỉ lệ với số nhóm KV G: so với MHA có H head KV, giữ nguyên các chiều khác thì tỉ lệ kích thước là G/H. Các phép chiếu K và V cũng giảm tương ứng, nhưng tổng số tham số mô hình còn phụ thuộc FFN và các thành phần khác.

4. Thí nghiệm và kết quả chính

Thiết lập: kiến trúc T5.1.1 (encoder-decoder); T5 Large và XXL MHA gốc; XXL uptrain thành MQA và GQA-8 với $\alpha = 0.05$ (~600 TPUv3 chip-day). GQA/MQA áp dụng cho decoder self-attention và cross-attention, **không** cho encoder self-attention (encoder tính song song, băng thông không phải nút cổ chai). Adafactor, lịch học như T5. Fine-tune: lr cố định 0.001, batch 128, dropout 0.1; inference dùng greedy decoding. Đo thời gian bằng xprof, 8 TPU, batch lớn nhất vừa (tối đa 32/TPU). Dữ liệu: tóm tắt (CNN/DailyMail, arXiv, PubMed, MediaSum, MultiNews), dịch WMT En→De, hỏi đáp TriviaQA.

Bảng 1 của paper (thời gian/mẫu T_{infer} tính bằng giây mỗi mẫu theo paper, điểm dev):

Model	T_{infer}	Avg	CNN R1	arXiv R1	PubMed R1	MediaSum R1	MultiNews R1	WMT BLEU	TriviaQA F1
MHA-Large	0.37	46.0	42.9	44.6	46.2	35.5	46.6	27.7	78.2
MHA-XXL	1.51	47.2	43.8	45.6	47.5	36.4	46.9	28.4	81.9
MQA-XXL	0.24	46.6	43.0	45.0	46.9	36.1	46.5	28.5	81.3
GQA-8-XXL	0.28	47.1	43.5	45.4	47.7	36.3	47.2	28.4	81.6

(Đơn vị của T_{infer} trong bản trích đã mất; hình 3 ghi "ms" trên trục, bảng ghi "s". Ta chỉ so sánh tỉ lệ, không dùng giá trị tuyệt đối.)

Đọc kết quả: GQA-8-XXL (47.1) gần MHA-XXL (47.2) với thời gian ≈ 0.28 so với 1.51, tức nhanh ≈ 5.4 lần (tự tính 1.51/0.28); MQA-XXL (46.6) nhanh nhất (0.24) nhưng kém hơn 0.6 điểm trung bình; cả hai tốt hơn và nhanh hơn MHA-Large (46.0, 0.37).

Ablation số nhóm (Hình 6): thời gian/mẫu của GQA-XXL (đầu vào 2048, đầu ra 512) tăng nhẹ từ 1 (MQA) lên 8 nhóm, rồi tăng nhanh hơn khi tiến tới 64 (MHA). Paper chọn 8 nhóm. (Chỉ có đồ thị, không có bảng số.)

5. Phân tích / ablation

- **Cách khởi tạo khi chuyển (Hình 4, T5-Large → MQA, α=0.05):** mean pooling > chọn head đầu > ngẫu nhiên; trục điểm trong khoảng 54.4-55.6 (chỉ có đồ thị, sai khác chỉ vài phần mười điểm trên một tập con tác vụ gồm CNN/DM, MultiNews, TriviaQA).
- **Số bước uptraining (Hình 5, T5 XXL):** GQA đã có chất lượng "reasonable" ngay sau khi chuyển đổi (trước khi huấn luyện thêm), còn MQA **cần** uptraining. Cả hai được lợi từ 5% uptraining, và thêm lên 10% lợi ích giảm dần.
- **Số nhóm:** từ MQA lên GQA-8 chỉ tăng chi phí nhẹ; đi gần MHA thì chi phí tăng dần. Lý do (paper): ở mô hình lớn, băng thông KV-cache ít hạn chế hơn (cache tăng theo d_{model}, FLOPs và tham số theo d^2_{model}), nhưng số head nhiều nên mức giảm KV lớn.
- **Sharding:** cách chia mô hình lớn thường sao chép K/V đơn lẻ của MQA cho từng phân vùng (partition); GQA có G head nên tránh lãng phí đó (lập luận của paper, chưa có thí nghiệm riêng).

6. Giới hạn

Do tác giả nêu:

- Đánh giá tóm tắt bằng ROUGE là không hoàn hảo, nên khó chắc chắn trade-off.
- Chưa so sánh GQA-XXL uptrained với mô hình cùng cấu hình **huấn luyện từ đầu** (hạn chế tính toán), nên không biết uptraining so với huấn luyện từ đầu thế nào.
- Chỉ đánh giá encoder-decoder; mô hình decoder-only (không có cross-attention) được kỳ vọng GQA có lợi thế lớn hơn so với MQA.
- Chủ yếu tối ưu băng thông KV ở chuỗi dài; chất lượng sinh dài khó đánh giá.

Nhận xét bổ sung:

- Chênh lệch giữa các phương án nhỏ (46.6 / 47.1 / 47.2) và không có độ lệch chuẩn giữa các seed.
- Thời gian đo trên TPU với batch lớn; không phản ánh batch 1 trên GPU nhỏ.

- Không báo cáo tương tác với kỹ thuật khác (lượng tử hóa, speculative decoding) dù nhắc ở related work.
- Mean-pooling ma trận chiếu cho K và V là heuristic; không có phân tích vì sao đúng (trực giác "bảo toàn thông tin").

7. Liên hệ với các bài khác

- **14 (MQA):** trường hợp $G = 1$; GQA giải quyết hạn chế chất lượng/ổn định.
- **16 (MLA):** hướng thay thế: nén K/V bằng low-rank chung thay vì chia sẻ head; DeepSeek-V2 so sánh trực tiếp MLA với MHA/GQA/MQA.
- **05 (LLaMA), 06 (Mistral):** LLaMA-2 70B và Mistral 7B dùng GQA (kiến thức ngoài paper này; Mistral 7B dùng 8 KV head, 32 query head).
- **08 (LoRA), 48-50:** uptraining điều chỉnh mô hình sau khi thay đổi cấu trúc attention.
- **27 (Switch):** ý tưởng "uptraining/upcycling" (Komatsuzaki 2022) là nguồn cảm hứng, từ dense sang MoE.
- **46 (PagedAttention), 56 (H2O):** cũng nhằm giảm bộ nhớ KV nhưng ở mức quản lý/loại bỏ token.

8. Gợi ý áp dụng

GQA cho thấy một cách chuyển checkpoint MHA: lấy trung bình trọng số của các KV head trong từng nhóm rồi huấn luyện thêm. Nên xem 5% ngân sách trong bài báo là thiết lập của thí nghiệm trên T5, không phải mức phục hồi bảo đảm cho mọi mô hình hay dữ liệu.

9. Thí nghiệm gợi ý

So sánh khởi tạo bằng trung bình, chọn một head và khởi tạo ngẫu nhiên; đo loss trước và sau huấn luyện tiếp với cùng ngân sách. Đo thêm kích thước KV cache theo số nhóm và đối chiếu với công thức.

10. Tóm tắt công thức / khái niệm

Khái niệm	Công thức / ý nghĩa
GQA-G	H query head chia G nhóm, mỗi nhóm 1 cặp K/V
Nhóm của head i	$g(i) = \lceil iG/H \rceil$
Trường hợp biên	$G = H$: MHA; $G = 1$: MQA
KV-cache	$2\,L\,G\,d_h\,n\,b\,s$ B; nhỏ hơn MHA H/G lần
Chuyển checkpoint	$W_K^{(g)} = \frac{G}{H}\sum_{i\in g} W_{K,i}$ (mean pooling), V tương tự
Uptraining	huấn luyện tiếp $\alpha = 5\%$ bước gốc (~600 TPUv3 chip-day cho T5-XXL)
Kết quả	GQA-8-XXL 47.1 vs MHA-XXL 47.2 vs MQA-XXL 46.6 (avg); thời gian 0.28 / 1.51 / 0.24
Chỉ áp dụng	decoder self-attn và cross-attn, không phải encoder self-attn

11. Kết luận ngắn

GQA cho thấy có thể biến checkpoint MHA thành mô hình ít KV head bằng cách gộp trung bình các head và huấn luyện thêm chừng 5% chi phí ban đầu, và nhóm trung gian (8 nhóm) gần giữ nguyên chất lượng MHA trong khi tốc độ gần MQA. Bằng chứng giới hạn ở T5 encoder-decoder, nhưng GQA đã trở thành chuẩn trong nhiều LLM decoder-only.

Nguồn: https://arxiv.org/abs/2305.13245

Bài 16

DeepSeek-V2: A Strong, Economical, and Efficient Mixture-of-Experts Language Model (2024)

Phạm vi ghi chú: DeepSeek-V2 là báo cáo kỹ thuật rất dài (SFT, RL, đánh giá hàng chục benchmark). Ghi chú này **tập trung vào Multi-head Latent Attention (MLA)**: nén KV bằng low-rank, decoupled RoPE, so sánh KV-cache; phần DeepSeekMoE, huấn luyện, căn chỉnh chỉ tóm tắt ngắn.

Tác giả: DeepSeek-AI (research@deepseek.com); nhóm tác giả được liệt kê theo vai trò trong phần cuối paper, ghi nhận Huazuo Gao và Wangding Zeng có "key innovations" cho MLA.
Venue / arXiv: arXiv:2405.04434 (cs.CL, v5, 19/6/2024); văn bản không nêu venue hội nghị.
Nguồn: https://arxiv.org/abs/2405.04434
Chủ đề: MLA (nén KV low-rank chung cho K và V), decoupled RoPE, DeepSeekMoE, mô hình 236B tổng / 21B kích hoạt, ngữ cảnh 128K.
Vị trí trong lộ trình: Đọc sau 12 (RoPE), 14 (MQA), 15 (GQA); trước 27/28 (MoE) nếu muốn hiểu phần MoE. Paper bổ sung: cách đạt cache nhỏ hơn cả GQA mà chất lượng ≥ MHA, và vì sao RoPE gây khó cho nén KV.

> Ghi chú diễn giải, không phải bản dịch. Các phép tính và suy diễn đại số được ghi rõ khi không phải kết quả trực tiếp của bài báo.

1. Bối cảnh

LLM mạnh hơn khi lớn hơn, nhưng đắt khi huấn luyện và chậm khi phục vụ. Hai nút cổ chai mà DeepSeek-V2 nhắm vào:

1. **KV-cache của MHA.** Mỗi token mỗi layer phải lưu $2n_h d_h$ phần tử; cache này "limits the maximum batch size and sequence length" (paper). Giải pháp cũ là MQA (bài 14) và GQA (bài 15) nhưng "their performance does not match MHA": paper trích kết quả ablation dense 7B (xem §6).
2. **Chi phí FFN dày.** Giải pháp là Mixture-of-Experts (MoE): FFN chia thành nhiều "expert", mỗi token chỉ dùng vài expert nên chỉ kích hoạt một phần tham số.

Khái niệm nền cần biết: **KV-cache** (lưu K,V của token trước khi sinh tuần tự), **low-rank** (ma trận lớn xấp xỉ bằng tích hai ma trận hẹp), **RoPE** (bài 12: xoay q,k theo vị trí, nên q·k chỉ phụ thuộc khoảng cách).

2. Ý tưởng chính

MLA không giảm số KV head như GQA. Thay vào đó, nó **nén** hidden state h_t của mỗi token thành một vector latent nhỏ $c_t^{KV} \in \mathbb{R}^{d_c}$ (chung cho cả K lẫn V), và chỉ lưu vector này. Khi cần, các ma trận "up-projection" khôi phục K và V cho từng head từ latent; vì mỗi head có ma trận up-projection riêng, vẫn có n_h head khác nhau (khác với MQA/GQA nơi head chia sẻ nguyên K/V). Điểm tinh tế: khi suy luận, các ma trận up-projection có thể **gộp vào** ma trận của query và của output, nên không phải dựng lại K và V đầy đủ. RoPE phá vỡ phép gộp này, nên paper tách một phần nhỏ của q và k riêng cho RoPE ("decoupled RoPE"), chỉ phần này cần cache thêm. Kết quả: cache mỗi token chỉ $(d_c + d_h^R)\, l$ phần tử, tương đương GQA với 2.25 nhóm, mà ablation cho thấy chất lượng còn tốt hơn MHA.

3. Phương pháp chi tiết: MLA

Ký hiệu của paper: d chiều embedding; n_h số head; d_h chiều mỗi head; $h_t \in \mathbb{R}^d$ input attention của token t; l số layer.

3.1 MHA chuẩn (Eq. 1-8)

$$q_t = W^Q h_t,\ k_t = W^K h_t,\ v_t = W^V h_t\ (\in \mathbb{R}^{d_h n_h}), \quad o_{t,i} = \sum_{j \le t} \text{softmax}_j\Big(\frac{q_{t,i}^\top k_{j,i}}{\sqrt{d_h}}\Big) v_{j,i}, \quad u_t = W^O[o_{t,1}; \ldots; o_{t,n_h}]$$

Cache MHA: $2 n_h d_h l$ phần tử mỗi token.

3.2 Nén KV low-rank chung (Eq. 9-11)

$$\boxed{c_t^{KV} = W^{DKV} h_t, \qquad k_t^C = W^{UK} c_t^{KV}, \qquad v_t^C = W^{UV} c_t^{KV}}$$

- $c_t^{KV} \in \mathbb{R}^{d_c}$ với $d_c \ll d_h n_h$: latent nén.
- $W^{DKV} \in \mathbb{R}^{d_c \times d}$: **down-projection** (nén).
- $W^{UK}, W^{UV} \in \mathbb{R}^{d_h n_h \times d_c}$: **up-projection** cho key và value.
- Khi suy luận chỉ cache c_t^{KV}: $d_c\, l$ phần tử mỗi token.

Vì K và V đều sinh từ **cùng** latent, ma trận $[W^{UK}; W^{UV}]$ có hạng $\le d_c$: giả định rằng thông tin cần cho cả key và value của mọi head nằm trong một không gian con d_c chiều. W^{UK} cắt latent thành n_h khối key (mỗi head một khối), tương tự cho value.

Paper cũng nén **query** (Eq. 12-13) $c_t^Q = W^{DQ} h_t$, $q_t^C = W^{UQ} c_t^Q$ với $c_t^Q \in \mathbb{R}^{d'_c}$, nhưng chỉ để giảm **bộ nhớ activation khi huấn luyện**, không giảm KV-cache.

3.3 Hấp thụ ma trận khi suy luận (tự suy diễn từ phát biểu của paper)

Paper nói: "W^{UK} can be absorbed into W^Q, and W^{UV} can be absorbed into W^O". Diễn giải từng bước, cho head i (ký hiệu $W_i^{UK} \in \mathbb{R}^{d_h \times d_c}$ là khối hàng của head i; W_i^Q tương tự):

Với key. Score (chưa RoPE) giữa query token t và key token j:

$$q_{t,i}^\top k_{j,i}^C = (W_i^Q h_t)^\top (W_i^{UK} c_j^{KV}) = h_t^\top \underbrace{(W_i^Q)^\top W_i^{UK}}_{\text{tính trước, cỡ } d \times d_c} c_j^{KV}.$$

Gộp $\tilde{W}_i = (W_i^Q)^\top W_i^{UK}$ (offline), ta tính $\tilde{q}_{t,i} = \tilde{W}_i^\top h_t \in \mathbb{R}^{d_c}$ rồi $\tilde{q}_{t,i}^\top c_j^{KV}$ trực tiếp với **latent đã cache**; không bao giờ phải tạo k_j^C cho các token tiền tố.

Với value. Đầu ra head i: $o_{t,i} = \sum_j a_{t,j,i} W_i^{UV} c_j^{KV} = W_i^{UV}(\sum_j a_{t,j,i} c_j^{KV})$, rồi chiếu ra bằng W_i^O (khối cột của W^O): đóng góp vào u_t là $W_i^O W_i^{UV}(\sum_j a_{t,j,i} c_j^{KV})$, tức gộp $W_i^O W_i^{UV}$ offline. Chỉ cần lấy tổng có trọng số của **latent**.

Hệ quả: khi sinh, attention hoạt động **trong không gian latent** d_c (cộng thêm phần RoPE), như một MQA với "head dim" d_c mà mọi head dùng chung dữ liệu cache nhưng khác ma trận gộp. (Nhận xét từ phép biến đổi.)

3.4 Vì sao RoPE xung đột, và decoupled RoPE (§2.1.3)

Nếu áp RoPE lên key $k_j^C = W^{UK} c_j^{KV}$ thì score trở thành

$$q_t^\top R_t^\top R_j\, W^{UK} c_j^{KV} \;=\; h_t^\top (W^Q)^\top R_{j-t}\, W^{UK}\, c_j^{KV}$$

(dùng tính chất bài 12: $R_t^\top R_j = R_{j-t}$). Ma trận xoay R_{j-t} **phụ thuộc vị trí tương đối** nằm *giữa* W^Q và W^{UK}; phép nhân ma trận không giao hoán nên không thể gộp $(W^Q)^\top W^{UK}$ trước thành một ma trận cố định như ở §3.3. Hậu quả: phải dựng lại key của **mọi** token tiền tố ở mỗi bước, mất lợi ích cache nhỏ ("significantly hinder the inference efficiency").

Giải pháp: decoupled RoPE. Thêm các thành phần riêng chỉ để mang vị trí (Eq. 14-19):

$$q_t^R = \text{RoPE}(W^{QR} c_t^Q) \quad (n_h \text{ head, mỗi head } d_h^R \text{ chiều}), \qquad k_t^R = \text{RoPE}(W^{KR} h_t) \quad (\textbf{một} \text{ key chung, } d_h^R \text{ chiều})$$
$$q_{t,i} = [\, q_{t,i}^C;\, q_{t,i}^R \,], \qquad k_{t,i} = [\, k_{t,i}^C;\, k_t^R \,]$$
$$o_{t,i} = \sum_{j \le t} \text{softmax}_j\!\left(\frac{q_{t,i}^\top k_{j,i}}{\sqrt{d_h + d_h^R}} \right) v_{j,i}^C, \qquad u_t = W^O [o_{t,1}; \ldots; o_{t,n_h}]$$

Tích vô hướng của hai vector ghép bằng **tổng**: $q_{t,i}^\top k_{j,i} = \underbrace{q_{t,i}^{C\top} k_{j,i}^C}_{\text{không RoPE, hấp thụ được}} + \underbrace{q_{t,i}^{R\top} k_j^R}_{\text{có RoPE, cỡ } d_h^R}$. Phần nội dung vẫn hấp thụ được, phần vị trí nhỏ (d_h^R chiều, key dùng chung giữa các head giống MQA). Cache thêm k_t^R. Tổng cache:

$$\boxed{\text{KV cache của MLA} = (d_c + d_h^R)\, l \text{ phần tử/token}}$$

Công thức đầy đủ (Phụ lục C, Eq. 37-47): $c^Q = W^{DQ} h$; $q^C = W^{UQ} c^Q$; $q^R = \text{RoPE}(W^{QR} c^Q)$; $q = [q^C; q^R]$; $c^{KV} = W^{DKV} h$; $k^C = W^{UK} c^{KV}$; $k^R = \text{RoPE}(W^{KR} h)$; $k = [k^C; k^R]$; $v^C = W^{UV} c^{KV}$; attention như trên. Vector cần cache: c^{KV} và k^R.

3.5 Hằng số của DeepSeek-V2 (§3.1.2)

60 layer, $d = 5120$, $n_h = 128$, $d_h = 128$, $d_c = 512$ (=4d_h), $d_c' = 1536$, $d_h^R = 64$ (=$d_h/2$). Thêm **RMSNorm** sau các latent nén và nhân hệ số tỉ lệ tại các điểm thắt cổ chai (latent, hidden của expert) để huấn luyện ổn định. YaRN mở rộng ngữ cảnh 4K → 128K chỉ áp dụng cho khóa decoupled k_t^R (nơi mang RoPE).

3.6 Pseudo-code MLA (bản đơn giản hóa, huấn luyện / prefill)

```
class MLA(nn.Module):
    def __init__(s, d, n_h, d_h, d_c, d_cq, d_r):
        s.W_dq  = Linear(d, d_cq);          s.W_uq = Linear(d_cq, n_h * d_h)
        s.W_qr  = Linear(d_cq, n_h * d_r)                       # q^R, RoPE
        s.W_dkv = Linear(d, d_c);           s.norm_kv = RMSNorm(d_c)
        s.W_uk  = Linear(d_c, n_h * d_h);   s.W_uv = Linear(d_c, n_h * d_h)
        s.W_kr  = Linear(d, d_r)                                 # k^R chung, RoPE
        s.W_o   = Linear(n_h * d_h, d)
    def forward(s, h, pos, cache):
        cq = s.W_dq(h)
        q  = cat([s.W_uq(cq).view(n_h, d_h), rope(s.W_qr(cq).view(n_h, d_r), pos)], -1)
        ckv = s.norm_kv(s.W_dkv(h));  kr = rope(s.W_kr(h), pos)   # CHỈ (ckv, kr) vào cache
        cache.append(ckv, kr)
        C, KR = cache.all()                                      # [S, d_c], [S, d_r]
        k = cat([s.W_uk(C).view(S, n_h, d_h), KR[:, None].expand(S, n_h, d_r)], -1)
        v = s.W_uv(C).view(S, n_h, d_h)
        o = softmax(q @ k.T / sqrt(d_h + d_r)) @ v
        return s.W_o(o.reshape(-1))
# Khi decode: gộp W_uk vào W_uq (tính q trong không gian d_c) và W_uv vào W_o (xem §3.3);
# khi đó không bao giờ tạo k, v đầy đủ từ cache.
```

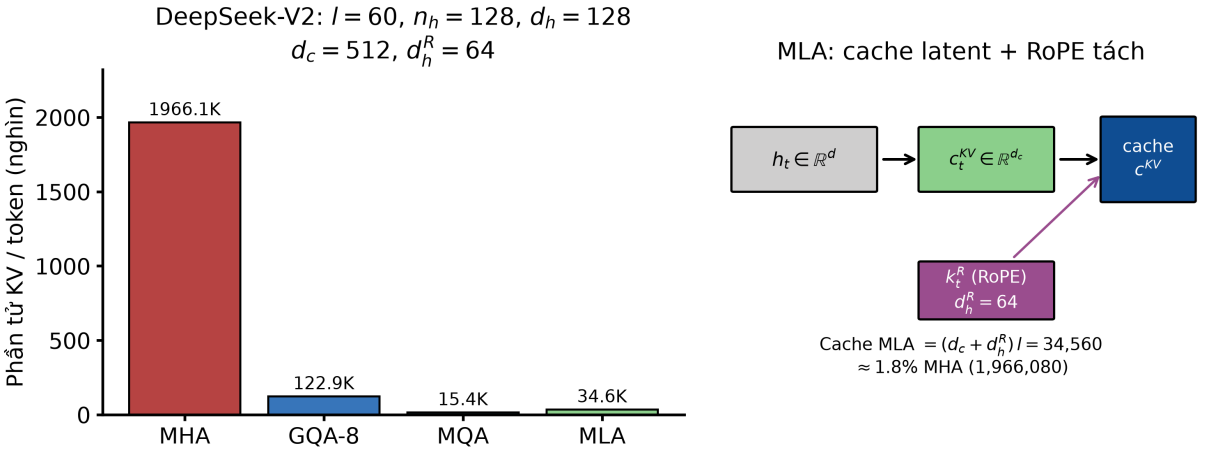
3.7 So sánh theo cấu hình

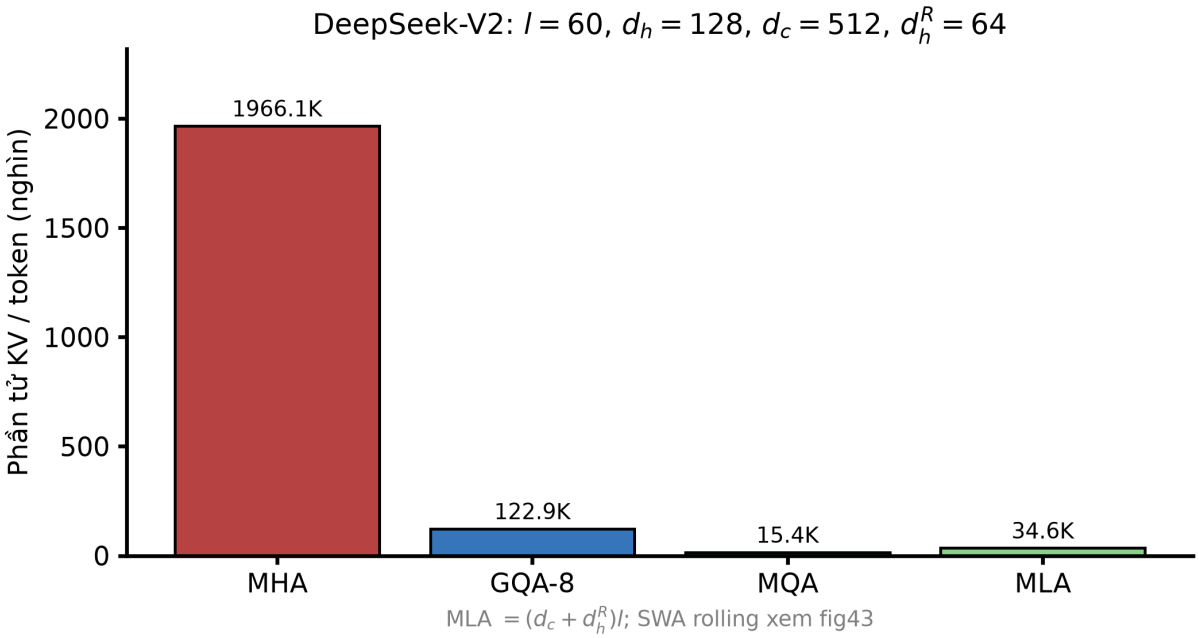
Với l tầng và cùng độ chính xác số, MHA lưu $2 n_h d_h l$ phần tử mỗi token; MLA lưu $(d_c + d_h^R) l$. Tỉ lệ giữa chúng phụ thuộc chiều latent và chiều RoPE, nên cần tính theo từng cấu hình. Số tham số của các phép chiếu

cũng phải tính riêng với cache.

4. So sánh KV-cache

Bảng 1 của paper (số phần tử mỗi token, không phụ thuộc độ chính xác lưu trữ):





So KV phần tử/token (DeepSeek-V2): MHA 1966K, GQA-8 123K, MQA 15.4K, MLA 34.6K; SWA xem fig43. (minh hoạ, tự tính)

So phần tử KV/token DeepSeek-V2: MHA 1.97M, GQA-8, MQA, MLA $(d_c + d_h^R)l = 34560$ ($\approx 1.8\%$ MHA). (minh hoạ, tự tính)

Cơ chế	KV cache/token	Khả năng (theo paper)
MHA	$2n_h d_h l$	Strong
GQA	$2n_g d_h l$	Moderate
MQA	$2d_h l$	Weak
MLA	$(d_c + d_h^R)l \approx \frac{9}{2} d_h l$	**Stronger**

Với $d_c = 4d_h$, $d_h^R = d_h/2$: $(4 + 0.5)d_h l = \frac{9}{2} d_h l = 2 \cdot 2.25\, d_h l$, tức **bằng GQA với 2.25 nhóm**.

Số cụ thể DeepSeek-V2 (tự tính từ cấu hình paper): MHA cùng kích thước sẽ cần $2 \cdot 128 \cdot 128 \cdot 60 = 1{,}966{,}080$ phần tử/token; MLA cần $(512 + 64) \cdot 60 = 34{,}560$; tỉ lệ ≈ 56.9 lần (MLA ≈ 1.8% của MHA). Con số "giảm 93.3%" trong phần tóm tắt là so với **DeepSeek 67B** (mô hình dense dùng GQA), không phải so với MHA.

Bảng 9 của paper (so MLA/MHA, MoE nhỏ và lớn):

	Small MoE MHA	Small MoE MLA	Large MoE MHA	Large MoE MLA
Tham số kích hoạt	2.5B	2.4B	25.0B	21.5B
Tổng tham số	15.8B	15.7B	250.8B	247.4B
KV cache/token (số phần tử)	110.6K	15.6K	860.2K	34.6K

Kiểm tra: small MLA $(512+64) \cdot 27 = 15{,}552 \approx 15.6$K và small MHA $2 \cdot 16 \cdot 128 \cdot 27 = 110{,}592 \approx 110.6$K khớp cấu hình V2-Lite (27 layer, 16 head, $d_h = 128$); large MLA $34{,}560 \approx 34.6$K khớp V2. Paper tóm tắt: MLA chỉ cần 14% (MoE nhỏ) và 4% (MoE lớn) cache so với MHA.

Hiệu ứng đối với suy luận (tự tính): vì cache MLA của V2 ≈ 34.6K phần tử/token, ở fp16 tức ≈ 69 KB/token, ngữ cảnh 128K cần ≈ 8.8 GB cho một chuỗi (so với ≈ 252 GB nếu là MHA: $1{,}966{,}080 \times 2\ \text{B} \times 131{,}072$). Thực tế paper còn lượng tử hóa cache xuống trung bình 6 bit/phần tử.

5. DeepSeekMoE và phần còn lại (tóm tắt)

- **DeepSeekMoE** (Eq. 20-22): thay FFN bằng N_s expert chia sẻ (luôn bật) cộng N_r expert định tuyến (top-K_r); mỗi expert nhỏ ("fine-grained") để chuyên môn hóa cao hơn và các expert chia sẻ giữ kiến thức chung (giảm dư thừa). Đầu ra: $h'_t = u_t + \sum_i \text{FFN}_i^{(s)}(u_t) + \sum_i g_{i,t}\text{FFN}_i^{(r)}(u_t)$, $g_{i,t} = s_{i,t}$ nếu $s_{i,t} \in$ Topk, ngược lại 0; $s_{i,t} = \text{Softmax}_i(u_t^\top e_i)$. V2: tất cả FFN trừ layer đầu là MoE; mỗi layer 2 expert chia sẻ + 160 expert định tuyến (hidden 1536/expert), 6 expert được kích hoạt mỗi token.
- **Device-limited routing:** mỗi token chỉ được gửi tới tối đa $M = 3$ thiết bị để giới hạn chi phí truyền thông.
- **Ba loss cân bằng tải** (expert, device, communication: $\alpha_1 = 0.003, \alpha_2 = 0.05, \alpha_3 = 0.02$) và **token-dropping** ở mức thiết bị khi huấn luyện (không bỏ token khi đánh giá).
- **Tiền huấn luyện:** 8.1T token, tokenizer BBPE 100K, độ dài 4K, AdamW ($\beta_1 = 0.9, \beta_2 = 0.95$), mở rộng ngữ cảnh lên 128K bằng YaRN + 1000 bước ở độ dài 32K. **Căn chỉnh:** SFT 1.5M mẫu (2 epoch, lr 5×10^{-6}) rồi RL bằng GRPO; paper ghi nhận "alignment tax" (RL làm giảm điểm một số benchmark chuẩn như BBH).
- **DeepSeek-V2-Lite** (Phụ lục B): 15.7B tổng / 2.4B kích hoạt, 27 layer, $d = 2048$, 16 head, $d_h = 128$, $d_c = 512$, **không nén query**, $d_h^R = 64$, 2 expert chia sẻ + 64 expert định tuyến (hidden 1408), 6 kích hoạt.

6. Thí nghiệm và kết quả chính

Hiệu quả (tóm tắt): so với DeepSeek 67B: tiết kiệm 42.5% chi phí huấn luyện (172.8K vs 300.6K GPU-giờ trên mỗi nghìn tỷ token, H800), giảm 93.3% KV-cache, thông lượng sinh tối đa gấp 5.76 lần (trên một node 8×H800 vượt 50K token/s; prompt >100K token/s). Triển khai dùng FP8 cho tham số và KV-cache lượng tử hóa trung bình 6 bit.

Ablation attention, dense 7B, cùng 1.33T token (Bảng 8):

Benchmark	MQA (7.1B)	GQA 8 nhóm (6.9B)	MHA (6.9B)
BBH (EM, 3-shot)	33.2	35.6	37.0
MMLU (Acc., 5-shot)	37.9	41.2	45.2
C-Eval (Acc., 5-shot)	30.0	37.7	42.9
CMMLU (Acc., 5-shot)	34.6	38.4	43.5

(Paper cân bằng số tham số bằng cách chỉnh số layer.) **MLA vs MHA (Bảng 9):**

Benchmark	Small MHA	Small MLA	Large MHA	Large MLA
BBH (EM)	37.9	39.0	46.6	50.7
MMLU (Acc.)	48.7	50.0	57.5	59.0
C-Eval (Acc.)	51.6	50.9	57.9	59.2
CMMLU (Acc.)	52.3	53.4	60.7	62.5

Small MoE huấn luyện trên 1.33T token, Large MoE (~250B) trên 420B token. MLA hơn MHA trong 7/8 ô, ngoại lệ C-Eval ở quy mô nhỏ (50.9 so với 51.6). Lưu ý: số tham số kích hoạt của Large MLA (21.5B) và MHA (25.0B) không bằng nhau.

Kết quả mô hình đầy đủ (Bảng 2, một phần; base model, 21B kích hoạt / 236B tổng):

Benchmark	DeepSeek 67B	Mixtral 8x22B	LLaMA 3 70B	DeepSeek-V2
MMLU (5-shot)	71.3	77.6	78.9	78.5
BBH (3-shot)	68.7	78.9	81.0	78.9
GSM8K (8-shot)	63.4	80.3	83.0	79.2
MATH (4-shot)	18.7	42.5	42.2	43.6
HumanEval (Pass@1)	45.1	53.1	48.2	48.8
C-Eval (5-shot)	66.1	59.6	67.5	81.7

Ngữ cảnh dài: thử nghiệm “Needle In A Haystack” cho kết quả tốt trên mọi độ dài tới 128K (Hình 4; chỉ có đồ thị).

V2-Lite (Bảng 6, một phần): MMLU 58.3 (so với 48.2 của DeepSeek 7B dense MHA và 45.0 của DeepSeek-MoE 16B MHA), BBH 44.1; lưu ý V2-Lite huấn luyện 5.7T token so với 2T của hai mô hình kia nên không so sánh sạch kiến trúc.

7. Phân tích / ablation

- **MHA > GQA > MQA** ở quy mô dense 7B (Bảng 8): cắt KV head là đổi chất lượng lấy cache, đúng như bài 14, 15.
- **MLA ≥ MHA** ở hai quy mô MoE (Bảng 9) với cache nhỏ hơn rất nhiều. Paper không cắt nghĩa chắc chắn vì sao một cơ chế nén lại tốt hơn MHA; điều có thể suy luận: nén low-rank hoạt động như bộ điều chuẩn hoặc mỗi head vẫn có K,V riêng nên không mất dung lượng “đa dạng head” như MQA/GQA (đây là suy đoán).
- **Decoupled RoPE là cần thiết** cho cache nhỏ (lập luận bằng đại số ở §3.4); paper không báo cáo ablation số liệu cho riêng thành phần này.
- Nén query chỉ để tiết kiệm activation khi huấn luyện (V2-Lite bỏ nén query).
- Cần RMSNorm sau latent và hệ số tỉ lệ để ổn định: nhận xét thực hành của paper.

8. Giới hạn

Do tác giả nêu: hạn chế chung của LLM (kiến thức không cập nhật, có thể sinh thông tin sai, hallucination); dữ liệu chủ yếu Trung/Anh nên yếu ở ngôn ngữ khác; "alignment tax" của RL; nhấn mạnh online RL tốt hơn offline nhưng chưa so sánh sâu. Với MoE, thừa nhận phần lớn phát hiện về kiến trúc chỉ ở quy mô thử nghiệm.

Nhận xét bổ sung:

- Ablation MLA vs MHA không cô lập từng thành phần (nén KV, nén query, decoupled RoPE) và số tham số hoạt động không khớp hoàn toàn giữa MLA và MHA trong Bảng 9.
- Ablation MHA/GQA/MQA ở dense 7B và MLA/MHA ở MoE là hai bộ thí nghiệm khác nhau, nên không có so sánh trực tiếp MLA với GQA cùng điều kiện.
- Lợi ích tính toán của hấp thụ ma trận chỉ rõ khi memory-bound; ở decode, attention chạy trong không gian $d_c + d_h^R$ ($= 576$ ở V2) cho mỗi head, tức số phép tính attention mỗi head lớn hơn MHA ($d_h = 128$) cỡ 4.5 lần (tự tính) nhưng lượng dữ liệu đọc nhỏ hơn rất nhiều; kernel cần được thiết kế riêng (paper nói MLA được tối ưu trên bản FlashAttention-2 cải tiến).
- Không có kết quả cho mô hình nhỏ cỡ dưới 1B hay cho bài toán đa phương thức.
- Khó chuyển checkpoint MHA có sẵn sang MLA (không có công thức "uptraining" như GQA); các nghiên cứu sau đã đề xuất nhưng ngoài paper này.

9. Liên hệ với các bài khác

- **12 (RoPE):** nền tảng để hiểu tại sao RoPE "cản" việc hấp thụ ma trận; decoupled RoPE là cách giải.
- **14 (MQA), 15 (GQA):** baseline; MLA đạt cache cỡ GQA-2.25 nhóm mà chất lượng ≥ MHA.
- **10 (RMSNorm):** dùng sau latent nén.
- **27 (Switch), 28 (Mixtral):** MoE; DeepSeekMoE thêm expert chia sẻ và expert nhỏ mịn.
- **46 (PagedAttention), 56 (H2O), 55 (StreamingLLM):** quản lý hoặc cắt bỏ cache, trực giao với việc nén theo chiều đặc trưng của MLA.
- **18-24 (linear attention, DeltaNet):** cách khác để tránh cache tăng theo n (trạng thái cố định) thay vì nén theo chiều đặc trưng.
- **08 (LoRA):** cùng nguyên tắc low-rank (ma trận lớn = tích hai ma trận hẹp).

10. Gợi ý áp dụng

MLA tách phần mã hoá vị trí khỏi nhánh latent để cho phép hấp thụ các ma trận chiếu khi suy luận. Kích thước cache và chi phí tính toán phụ thuộc chiều latent, phần RoPE và cách triển khai; tỉ lệ tiết kiệm của DeepSeek-V2 không tự động áp dụng cho mọi cấu hình.

11. Thí nghiệm gợi ý

Dùng tensor ngẫu nhiên để so sánh đầu ra của cách dựng K, V tường minh với cách hấp thụ ma trận trong không gian latent. Kiểm tra sai số số học, rồi khảo sát vì sao đưa phép quay phụ thuộc vị trí vào nhánh nội dung làm thay đổi phép biến đổi đại số.

12. Tóm tắt công thức / khái niệm

Khái niệm	Công thức / ý nghĩa
Nén KV	$c_t^{KV} = W^{DKV} h_t$, $k^C = W^{UK} c^{KV}$, $v^C = W^{UV} c^{KV}$
Cache chỉ latent	$d_c l$ phần tử/token (chưa tính RoPE)
Hấp thụ	$(W_i^Q)^\top W_i^{UK}$ và $W_i^O W_i^{UV}$ tính trước, không dựng K,V
Vì sao RoPE gây xung đột	R_{j-t} nằm giữa W^Q và W^{UK}, phụ thuộc vị trí, không gộp được

Khái niệm	Công thức / ý nghĩa
Decoupled RoPE	$q_{t,i} = [q^C_{t,i}; q^R_{t,i}]$, $k_{t,i} = [k^C_{t,i}; k^R_t]$, k^R chung cho mọi head
Cache MLA	$(d_c + d^R_h)\, l$ phần tử/token
V2	$d_c = 512, d^R_h = 64, n_h = d_h = 128, l = 60$: 34,560 phần tử/token
So với GQA	$\frac{9}{2} d_h l$ = GQA 2.25 nhóm
Kết quả	giảm 93.3% cache (vs 67B), 42.5% chi phí huấn luyện, thông lượng ×5.76
Ablation	MHA>GQA>MQA (7B dense); MLA≥MHA (Bảng 9, cache 14% / 4%)
MoE	2 shared + 160 routed expert, 6 kích hoạt; 236B tổng / 21B kích hoạt

13. Kết luận ngắn

MLA nén K và V của mọi head vào một latent chung nhỏ, giữ nguyên số head và dùng phép gộp ma trận để suy luận trong không gian latent; RoPE được tách thành nhánh nhỏ riêng để không phá phép gộp đó. Theo paper, cache chỉ bằng ~GQA 2.25 nhóm nhưng chất lượng không thua (và ở thí nghiệm của họ còn hơn) MHA.

Nguồn: https://arxiv.org/abs/2405.04434

Bài 17

Longformer: The Long-Document Transformer (2020)

Tác giả: Iz Beltagy, Matthew E. Peters, Arman Cohan (Allen Institute for AI)
Venue / arXiv: arXiv:2004.05150 (bản v2, 12/2020); paper không nêu venue trong bản text đã đọc
Nguồn: https://arxiv.org/abs/2004.05150
Chủ đề: Sparse attention (cửa sổ trượt + global attention), độ phức tạp tuyến tính theo độ dài chuỗi, tiền huấn luyện tiếp từ RoBERTa, biến thể encoder-decoder LED.
Vị trí trong lộ trình: Đọc sau 01 (Attention Is All You Need) và 09/12 (Pre-LN, RoPE) để nắm self-attention và vị trí. Bài này mở đầu nhánh "làm attention rẻ hơn theo độ dài": ở đây bằng cách **cắt bớt các cặp (query, key)** (sparse). Nhánh còn lại, *nén toàn bộ quá khứ vào một trạng thái cố định*, bắt đầu từ 18 (Linear Transformers). Cửa sổ trượt xuất hiện lại trong 06 (Mistral 7B), 55 (StreamingLLM) và trong các lớp full-attention/hybrid ở 25, 26.

Ghi chú diễn giải bằng tiếng Việt, không phải bản dịch. Các phép tính đánh dấu "tự tính" là ví dụ minh họa.

1. Bối cảnh

Vấn đề. Self-attention chuẩn (bài 01) tính điểm cho *mọi cặp* vị trí, nên thời gian và bộ nhớ là $O(n^2)$ theo độ dài chuỗi n. Mô hình BERT/RoBERTa vì vậy bị giới hạn ở 512 token. Với tài liệu dài (bài báo khoa học, nhiều đoạn Wikipedia), người ta phải làm các việc tạm bợ:

- **Cắt cụt** tài liệu (mất thông tin).
- **Chia khúc 512 token** rồi ghép kết quả bằng một mô hình riêng cho từng tác vụ (mất tương tác giữa các khúc).
- **Hai giai đoạn** (retrieve rồi đọc) — lỗi giai đoạn 1 lan sang giai đoạn 2.

Các hướng trước đó (Bảng 1 trong paper): (i) mô hình đi từ trái sang phải theo từng khúc (Transformer-XL, Compressive Transformer...) — tốt cho language modeling nhưng không dùng được bối cảnh hai chiều cho transfer learning; (ii) định nghĩa một **mẫu attention thưa** (Sparse Transformer, Reformer, BP-Transformer, Blockwise...). Nhiều công trình chỉ đánh giá language modeling, chưa kiểm tra theo kiểu "tiền huấn luyện rồi fine-tune" như BERT.

Khái niệm cần biết trước.

- *Attention pattern* (mẫu attention): tập các cặp (i, j) mà token i được phép nhìn token j. Attention đầy đủ là toàn bộ ma trận $n \times n$; attention thưa chỉ tính một phần.
- *Receptive field* (trường nhìn): tập token ban đầu có thể ảnh hưởng đến một biểu diễn ở lớp cao, giống CNN: xếp nhiều lớp cửa sổ nhỏ thì lớp trên "thấy" xa.
- *BPC (bits per character)*: loss trung bình theo bit trên mỗi ký tự, càng thấp càng tốt (dùng cho language modeling cấp ký tự).

2. Ý tưởng chính

Thay vì để mỗi token nhìn *tất cả* token khác, Longformer cho mỗi token chỉ nhìn **một cửa sổ cố định xung quanh nó** (sliding window, kích thước w), nên chi phí là $O(n \times w)$, tuyến tính theo n. Xếp ℓ lớp thì trường nhìn ở lớp trên là $\ell \times w$, nhờ vậy thông tin vẫn lan đi xa. Cửa sổ không đủ cho các tác vụ cần "tổng hợp toàn chuỗi" (phân loại qua `[CLS]`, hỏi đáp so câu hỏi với tài liệu), nên thêm **global attention** trên *một vài token chọn trước*: token global nhìn tất cả và tất cả nhìn nó (đối xứng). Vì số token global nhỏ và không phụ thuộc n, tổng chi phí vẫn $O(n)$. Tác giả hiện thực kernel CUDA, tiếp tục tiền huấn luyện từ RoBERTa với độ dài

4,096, và cho thấy tốt hơn RoBERTa trên các tác vụ tài liệu dài; cuối cùng thêm biến thể encoder-decoder (LED) cho tóm tắt.

3. Phương pháp chi tiết

3.1 Ba thành phần của mẫu attention

(a) Sliding window. Với cửa sổ w (chẵn), token i nhìn các token j thỏa $|i-j| \le w/2$ (paper: "$\frac{1}{2}w$ tokens on each side").

$$\mathcal{N}(i) = \{\, j : \ |i-j| \le w/2 \,\}, \qquad \text{số cặp} \approx n \cdot (w+1) = O(n\,w).$$

Với ℓ lớp và w cố định, trường nhìn ở lớp trên cùng là $\ell \times w$. Có thể dùng w khác nhau theo từng lớp: nhỏ ở lớp thấp, lớn ở lớp cao.

(b) Dilated sliding window. Cửa sổ có "khe" cách nhau d (giống dilated convolution), cùng chi phí nhưng trường nhìn lớn hơn: $\ell \times d \times w$. Paper dùng dilation theo từng head: một số head không dilate (tập trung bối cảnh gần), số khác dilate (nhìn xa).

(c) Global attention. Chọn tập vị trí G (tùy tác vụ: `[CLS]` cho phân loại, toàn bộ token câu hỏi cho QA). Với $g \in G$: g nhìn mọi token; mọi token nhìn g. Do $|G| \ll n$ và độc lập với n, độ phức tạp tổng hợp vẫn $O(n)$.

Hai bộ phép chiếu riêng:

$$\text{cửa sổ: } Q_s, K_s, V_s; \qquad \text{global: } Q_g, K_g, V_g,$$

Q_g, K_g, V_g được **khởi tạo bằng đúng giá trị** của Q_s, K_s, V_s rồi học riêng. Công thức attention vẫn là softmax chuẩn nhưng chỉ lấy trên các cặp được phép:

$$\text{Attention}(Q,K,V) = \text{softmax}\Big(\tfrac{QK^\top}{\sqrt{d_k}}\Big)V \quad \to \quad o_i = \sum_{j \in \mathcal{N}(i)} \frac{\exp(q_i^\top k_j/\sqrt{d_k})}{\sum_{j' \in \mathcal{N}(i)} \exp(q_i^\top k_{j'}/\sqrt{d_k})}\, v_j.$$

Chú ý: **softmax vẫn có mặt**, chỉ chuẩn hóa trên tập hàng xóm. Đây là khác biệt cốt lõi so với bài 18: ở đây ta bỏ bớt cặp nhưng không bỏ softmax, nên *không* có dạng hồi quy trạng thái hữu hạn (muốn sinh token tự hồi quy vẫn phải giữ KV của w token gần nhất).

3.2 Hiện thực: vì sao cần kernel riêng

Phép khó là $QK^\top$. Longformer chỉ cần một số đường chéo của ma trận này ("banded matrix multiplication"), mà PyTorch/TensorFlow không hỗ trợ trực tiếp. Paper so ba cách (Hình 1):

Cách	Mô tả	Ghi chú
Longformer-loop	tính từng đường chéo trong vòng lặp	tiết kiệm bộ nhớ nhưng "unusably slow", chỉ để test
Longformer-chunks	cắt Q, K thành khối kích thước w chồng lấn $w/2$, nhân khối bằng matmul, mask các phần thừa	chỉ hỗ trợ không dilation; nhanh nhất cho pretrain/finetune; tốn gấp ~2× bộ nhớ lý tưởng vì tính cả vài giá trị 0
Longformer-cuda	kernel CUDA viết bằng TVM	hỗ trợ dilation, tiết kiệm bộ nhớ nhất, "as fast as highly optimized full self-attention"; dùng cho language modeling cấp ký tự

Ghi chú của paper: attention được giữ ở fp32 vì fp16 gây tràn số/NaN ở giai đoạn sau của huấn luyện.

Pseudo-code của cách "chunks" (diễn giải, không phải code gốc):

```
def sliding_window_attn(Q, K, V, w):       # Q,K,V: [n, d]
    # 1. chia Q và K thành các khối kích thước w, chồng lấn w/2 (K, V lấy cùng cách chia)
    Qb = blocks(Q, size=w, stride=w//2)    # [số_khối, w, d]
    Kb = blocks(K, size=w, stride=w//2)
    Vb = blocks(V, size=w, stride=w//2)
    S  = Qb @ Kb.transpose(-1,-2)          # một lần matmul theo khối (mỗi khối chỉ ghép
    ↪  với khối K tương ứng/lân cận)
    S  = mask_outside_window(S, w)         # bỏ các ô ngoài |i-j| <= w/2 (phần tính thừa
    ↪  ~2x bộ nhớ)
    P  = softmax(S, dim=-1)
    return P @ Vb                          # ghép lại các khối thành [n, d]
```

3.3 Language modeling cấp ký tự (autoregressive)

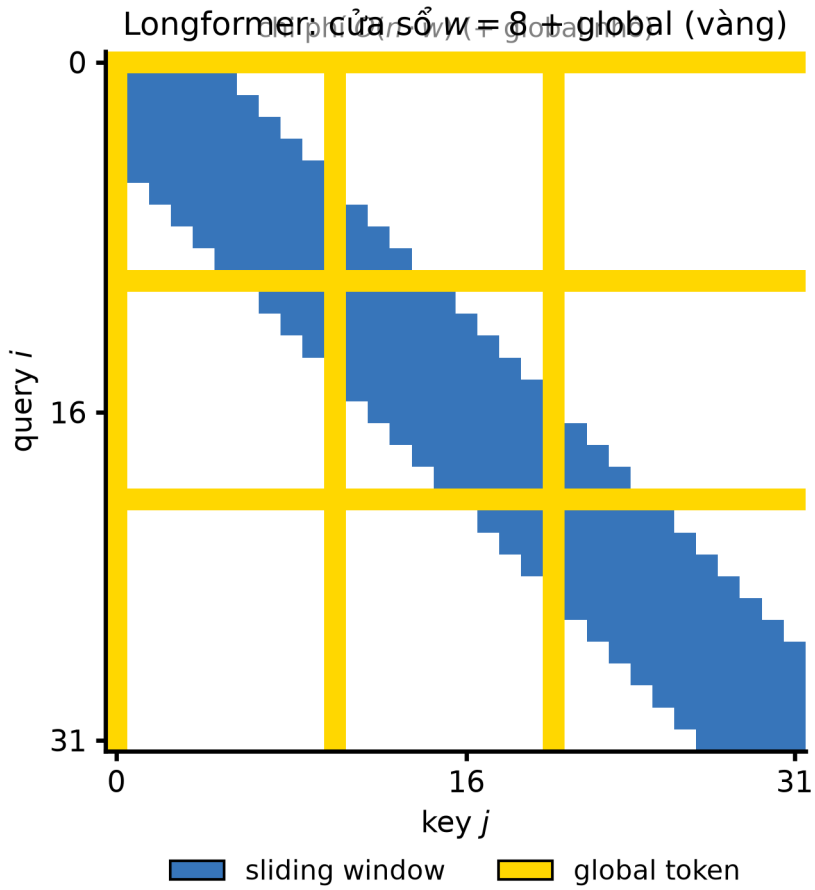
- Dùng cửa sổ *tăng dần theo lớp* (nhỏ ở lớp thấp, lớn ở lớp cao), không dilate ở lớp thấp, và chỉ dilate nhẹ trên 2 head ở lớp cao.
- **Huấn luyện theo giai đoạn**: mô hình cần nhiều bước cập nhật để học bối cảnh cục bộ trước khi dùng được bối cảnh dài, nên bắt đầu với chuỗi và cửa sổ ngắn; mỗi giai đoạn sau **gấp đôi cửa sổ và độ dài chuỗi, giảm nửa learning rate**. 5 giai đoạn, độ dài chuỗi từ 2,048 đến 23,040.
- Đánh giá với chuỗi 32,256, bước nhảy 512, báo cáo trên 512 token cuối.

3.4 Tiền huấn luyện và fine-tune từ RoBERTa

- Thay self-attention của RoBERTa bằng cửa sổ trượt $w = 512$ (**cùng lượng tính toán như RoBERTa**) cộng global attention theo tác vụ. Kiến trúc còn lại không đổi.
- RoBERTa dùng position embedding tuyệt đối học được, tối đa 512. Để lên 4,096, **sao chép** 512 embedding nhiều lần làm khởi tạo. Lý do: phân tích attention của BERT cho thấy thiên hướng mạnh nhìn bối cảnh gần (token trước/sau), sao chép giữ cấu trúc cục bộ ở mọi nơi trừ ranh giới các đoạn sao chép.
- Tiếp tục MLM 65K bước, độ dài 4,096, batch 64 (2^{18} token), LR tối đa 3×10^{-5}, warmup 500 bước, sau đó giảm đa thức bậc 3.

3.5 LED (Longformer-Encoder-Decoder)

Encoder dùng mẫu local+global của Longformer; decoder dùng full attention lên toàn bộ encoder output và các token đã sinh. Khởi tạo từ BART (giữ nguyên số lớp, kích thước ẩn), mở rộng position embedding lên 16K bằng cách sao chép embedding 1K của BART 16 lần. Hai cỡ: LED-base (6+6 lớp), LED-large (12+12 lớp). Với tóm tắt arXiv: cửa sổ encoder 1,024, global attention trên token <s> đầu.



Longformer: sliding window w + vài token global; chi phí $O(nw)$. (minh hoạ, tự tính)

4. Thí nghiệm và kết quả chính

4.1 Language modeling cấp ký tự (BPC, càng thấp càng tốt)

Mô hình nhỏ: 12 lớp, hidden 512 (41M tham số). Mô hình lớn: 30 lớp, hidden 512 (102M).

Mô hình nhỏ	#Param	text8 test	enwik8 test
T12 (Al-Rfou)	44M	1.18	1.11
Adaptive span	38-39M	1.11	1.02
BP-Transformer	38-39M	1.11	1.02
Longformer	41M	**1.10**	**1.00**

Mô hình lớn, enwik8 test BPC: Transformer-XL (18 lớp, 88M) 1.03; Sparse Transformer (~100M) 0.99; Transformer-XL (24 lớp, 277M) 0.99; Adaptive 209M 0.98; Compressive 277M 0.97; Routing ~223M 0.99; **Longformer 102M: 0.99**. Tức khớp mô hình tốt cùng cỡ, kém nhẹ mô hình có hơn 2× tham số.

4.2 Tiền huấn luyện MLM (BPC trên tập dev của kho tài liệu dài)

Cấu hình	base	large
RoBERTa (seqlen 512)	1.846	1.496
Longformer (4,096), position embedding khởi tạo ngẫu nhiên	10.299	8.738

Cấu hình	base	large
+ sao chép position embedding	1.957	1.597
+ 2K bước cập nhật	1.753	1.414
+ 65K bước cập nhật	1.705	1.358
Longformer, chỉ train position embedding mới (đóng băng RoBERTa)	1.850	1.504

4.3 Fine-tune trên tài liệu dài (tập dev, Longformer-base vs RoBERTa-base)

Tác vụ	RoBERTa-base	Longformer-base
WikiHop (acc)	72.4	75.0
TriviaQA (F1)	74.3	75.2
HotpotQA (joint F1)	63.5	64.4
OntoNotes (F1)	78.4	78.6
IMDB (acc)	95.3	95.7
Hyperpartisan (F1)	87.4	94.8

Longformer-large trên bảng xếp hạng lúc nộp (5/2020): WikiHop 81.9 (SOTA trước đó 78.3), TriviaQA 77.3 (73.3), HotpotQA 73.2 (SOTA 74.2). Paper nhận định tăng nhiều nhất ở tác vụ thật sự cần bối cảnh xa (WikiHop, Hyperpartisan), ít ở tác vụ mà bối cảnh cục bộ đã đủ.

4.4 LED trên tóm tắt arXiv

Mô hình	R-1	R-2	R-L
Pegasus	44.21	16.95	38.83
LED-large (seqlen 4,096)	44.40	17.94	39.76
BigBird (seqlen 4,096)	46.63	19.02	41.77
LED-large (seqlen 16,384)	46.63	19.62	41.83

Hình 3 của paper: tăng độ dài đầu vào từ 1K lên 16K cải thiện rõ ROUGE (độ chính xác từng điểm trong hình không đọc được từ text đã trích, nên không ghi ở đây).

5. Phân tích / ablation

Cấu hình cửa sổ (text8, dev BPC, 150K bước ở giai đoạn 1): giảm w từ 512 xuống 32 theo chiều lên lớp: 1.24; cố định $w = 230$: 1.23; **tăng w từ 32 lên 512**: 1.21. Không dilation: 1.21; dilation trên 2 head: 1.20. Kết luận: lớp thấp cửa sổ nhỏ, lớp cao cửa sổ lớn là hợp lý; dilation có ích nhẹ.

Ablation WikiHop (dev acc, Longformer-base): bản chuẩn 73.8 (5 epoch).

Biến thể	Acc	Δ
RoBERTa-base seqlen 512	72.4	-1.4
15 epoch	75.0	+1.2
Longformer seqlen 512, attention đầy đủ n^2	71.7	-2.1
seqlen 2,048	73.1	-0.7
không MLM pretraining	73.2	-0.6

Biến thể	Acc	Δ
không projection riêng cho global	72.2	-1.6
không projection riêng *và* không global attention	65.5	-8.3
chỉ pretrain position embedding mới	73.5	-0.3

Rút ra (theo paper): **global attention và projection riêng cho global là rất quan trọng** (mất 8.3 điểm nếu bỏ cả hai); độ dài dài hơn giúp; khi cấu hình về 512 + attention đầy đủ thì Longformer kém RoBERTa nhẹ, xác nhận cải thiện không đến từ việc tiền huấn luyện thêm.

6. Giới hạn

Do tác giả nêu:

- Global attention là *thủ công theo tác vụ* (phải chọn token nào là global).
- Dilation trên vài head làm giảm hiệu năng khi nạp trọng số RoBERTa (không tương thích với trọng số tiền huấn luyện); có thể cần huấn luyện lại từ đầu.
- Cách "chunks" tốn gấp ~2× bộ nhớ so với lý tưởng; kernel CUDA tối ưu hoàn hảo (nhanh hơn n^2) cần kiến thức lập trình GPU cấp thấp, chưa đạt đầy đủ.
- So sánh với BigBird bị nhiễu do BigBird dùng nhiều tính toán tiền huấn luyện hơn 16×.

Nhận xét bổ sung:

- Cửa sổ trượt *vẫn giữ KV cache* cỡ w token ở thời điểm sinh; bộ nhớ bị chặn trên (tốt) nhưng không "nén" như trạng thái hồi quy ở bài 18-21.
- Thông tin ngoài cửa sổ chỉ đến được qua nhiều lớp (trường nhìn ℓw) hoặc qua token global; với sinh văn bản tự hồi quy không có khái niệm token global "nhìn tương lai".
- Phần lớn lợi ích đo bằng mô hình encoder (MLM) và tác vụ hiểu, không phải LLM sinh tự hồi quy quy mô lớn.
- Mức tiết kiệm phụ thuộc tỉ lệ giữa n và w cũng như chi phí của các phép chiếu.

7. Liên hệ với các bài khác

- **01 Attention Is All You Need:** Longformer giữ nguyên công thức attention, chỉ thay *tập cặp được tính*.
- **06 Mistral 7B:** dùng sliding-window attention ở quy mô LLM, kèm KV cache dạng vòng (rolling buffer) cỡ w; đây là hậu duệ trực tiếp của ý tưởng cửa sổ.
- **12 RoPE, 13 ALiBi:** vấn đề mở rộng vị trí; Longformer chọn "sao chép position embedding" thay vì mã hóa tương đối.
- **18 Linear Transformers:** hướng đối lập: *giữ mọi cặp* nhưng thay softmax để tính được bằng hồi quy. Hai hướng bổ sung nhau: SWA cho bối cảnh gần chính xác, linear/SSM cho bối cảnh xa nén.
- **25 Jamba, 26 Kimi Linear:** hybrid trộn lớp attention (kể cả cửa sổ) với lớp hồi quy.
- **44-45 FlashAttention:** giải quyết bài toán tương tự (băng thông bộ nhớ) cho attention đầy đủ mà không cắt cặp nào.
- **55 StreamingLLM, 56 H2O:** cắt KV cache; StreamingLLM cho thấy cửa sổ thuần túy cần thêm "attention sink".
- **57 FastV:** cắt token thị giác — cùng tinh thần giảm số cặp attention.

8. Gợi ý áp dụng

Cửa sổ cục bộ giảm số cặp attention; các vị trí global tạo thêm đường trao đổi thông tin trên toàn chuỗi. Lựa chọn vị trí global phụ thuộc tác vụ, còn mức tăng tốc thực tế phụ thuộc kernel và độ dài. Longformer là mô hình encoder; không nên chuyển trực tiếp kết luận về cache sang decoder tự hồi quy.

9. Thí nghiệm gợi ý

Trên một tác vụ tài liệu dài, so sánh các kích thước cửa sổ và cách chọn token global về chất lượng, bộ nhớ và thời gian. Giữ dữ liệu và ngân sách huấn luyện cố định; phân tích riêng các mẫu cần thông tin ở xa.

10. Tóm tắt công thức / khái niệm

Khái niệm	Công thức / ý nghĩa
Sliding window	$j \in [i - w/2,\, i + w/2]$; chi phí $O(nw)$; trường nhìn ℓw
Dilated window	khe d; trường nhìn ℓdw
Global attention	token $g \in G$ nhìn tất cả và được tất cả nhìn; $O(n)$ vì $\lvert G\rvert \ll n$
Hai bộ chiếu	(Q_s, K_s, V_s) cho cửa sổ, (Q_g, K_g, V_g) cho global; khởi tạo giống nhau
Khởi tạo vị trí	sao chép 512 embedding RoBERTa thành 4,096 (BART: 1K thành 16K)
Huấn luyện giai đoạn	gấp đôi w và độ dài, giảm nửa LR mỗi giai đoạn
LED	encoder local+global, decoder full attention, khởi tạo từ BART

11. Kết luận ngắn

Longformer là bản "cắt cặp attention" hiệu quả và dễ thay thế (drop-in) cho self-attention, chứng minh được trên cả language modeling và các tác vụ tài liệu dài nhờ ba ý: cửa sổ cục bộ, global attention theo tác vụ, và khởi tạo vị trí bằng sao chép. Cách này là một mốc đối chiếu: sparse (giữ softmax và giới hạn các cặp attention) khác căn bản với hướng hồi quy (18-24) vốn nén quá khứ vào trạng thái cố định.

Nguồn gốc: https://arxiv.org/abs/2004.05150

Bài 18

Transformers are RNNs: Fast Autoregressive Transformers with Linear Attention (2020)

Tác giả: Angelos Katharopoulos, Apoorv Vyas, Nikolaos Pappas, François Fleuret (Idiap/EPFL, U. Washington, U. Geneva)
Venue / arXiv: ICML 2020 (PMLR 119); arXiv:2006.16236 (v3, 8/2020)
Nguồn: https://arxiv.org/abs/2006.16236
Chủ đề: Linear attention bằng kernel feature map, trạng thái hồi quy ma trận $S_t = S_{t-1} + \phi(k_t)v_t^\top$, tương đương Transformer-RNN, suy luận tự hồi quy chi phí hằng số mỗi bước.
Vị trí trong lộ trình: Cần đọc 01 (Attention Is All You Need) trước, đặc biệt công thức softmax attention và causal mask. Bài này là **gốc của cả dòng 18 → 19 → 20 → 22 → 23 → 24**: mọi bài sau chỉ thay *cách cập nhật trạng thái* S_t (thêm suy giảm, thêm cổng phụ thuộc dữ liệu, thêm delta rule). Bài trước (17 Longformer) cắt cặp attention; bài này đổi bản chất phép toán để nén toàn bộ quá khứ.

> Ghi chú diễn giải, không phải bản dịch. Quy ước vector cột: $q_t, k_t \in \mathbb{R}^{d_k}$, $v_t \in \mathbb{R}^{d_v}$ và $S_t \in \mathbb{R}^{d_k \times d_v}$; công thức của bài báo dùng quy ước chuyển vị.

1. Bối cảnh

Hai nỗi đau của Transformer tự hồi quy (bài 01):

1. **Huấn luyện/prefill:** attention tính ma trận $n \times n$ nên thời gian và bộ nhớ là $O(n^2)$.
2. **Sinh từng token (decode):** token thứ t phải attend tới t khóa trước đó; ngay cả khi có KV cache, chi phí một bước là $O(t)$, tổng cả chuỗi $O(T^2)$, và KV cache tăng tuyến tính theo t.

Các hướng giảm chi phí trước đó (paper điểm lại): cắt tỉa, phân rã trọng số, lượng tử hóa, chưng cất; Transformer-XL và Adaptive Span (tăng ngữ cảnh nhưng cùng độ phức tạp tiệm cận); Sparse Transformer ($O(N\sqrt{N})$) và Reformer ($O(N \log N)$, dùng LSH, bắt buộc $K = Q$ nên không dùng được cho decoding). Điểm chung: **các phương pháp này tăng tốc huấn luyện nhưng không tăng tốc suy luận tự hồi quy**. Bài này nhắm vào cả hai.

Khái niệm nền.

- *Kernel / feature map ϕ:* một hàm đưa vector sang không gian khác sao cho tích vô hướng trong không gian mới $\phi(x)^\top\phi(y)$ bằng một hàm tương tự $k(x, y)$ ta muốn dùng. Ví dụ kernel đa thức bậc 2 có ϕ hữu hạn chiều; kernel mũ (của softmax) cần ϕ vô hạn chiều.
- *Tính kết hợp của nhân ma trận:* $(AB)C = A(BC)$ — kết quả giống nhau nhưng chi phí khác nhau. Đây là toàn bộ "mánh" của bài.
- *Causal mask:* chặn token i nhìn $j > i$.

2. Ý tưởng chính

Softmax attention $\text{softmax}(QK^\top)V$ khó tái sắp xếp vì softmax bao quanh tích $QK^\top$. Nếu thay hàm tương tự $\exp(q^\top k)$ bằng một **kernel phân rã được** $\phi(q)^\top\phi(k)$, ta có thể *kết hợp lại phép nhân* thành $\phi(Q)\big(\phi(K)^\top V\big)$: tính trước một ma trận nhỏ $d \times d$ rồi dùng chung cho mọi query, nên chi phí thành $O(N)$ thay vì $O(N^2)$. Với causal mask, ma trận nhỏ này là một **tổng tích lũy** $S_t = \sum_{i \le t} \phi(k_i)v_i^\top$, có thể cập nhật mỗi bước bằng một phép cộng: đó chính là **một mạng hồi quy (RNN) với trạng thái là ma trận**. Vì vậy "mọi Transformer có causal mask đều có thể viết thành RNN", và sinh token tốn chi phí hằng số mỗi bước, bộ nhớ hằng số theo độ dài.

3. Phương pháp chi tiết: từ softmax đến RNN

3.1 Công thức softmax tổng quát hóa

Với $Q, K \in \mathbb{R}^{N\times D}$, $V \in \mathbb{R}^{N\times M}$, softmax attention (có chuẩn hóa $\sqrt{D}$):

$$V_i' = \frac{\sum_{j=1}^{N} \text{sim}(Q_i, K_j)\, V_j}{\sum_{j=1}^{N} \text{sim}(Q_i, K_j)}, \qquad \text{sim}(q,k) = \exp\!\left(\tfrac{q^\top k}{\sqrt{D}}\right).$$

Chỉ ràng buộc duy nhất để công thức này là một "attention" hợp lệ: sim ≥ 0 (để các trọng số cộng lại thành phân phối). Mọi kernel $k(x,y) \geq 0$ đều dùng được.

3.2 Chuyển sang kernel và tái kết hợp (không mask)

Nếu $\text{sim}(q,k) = \phi(q)^\top \phi(k)$ thì (viết cột; ϕ áp dụng theo từng hàng của Q, K):

$$V_i' = \frac{\phi(Q_i)^\top \sum_{j=1}^{N} \phi(K_j) V_j^\top}{\phi(Q_i)^\top \sum_{j=1}^{N} \phi(K_j)}.$$

Hai tổng không phụ thuộc i, nên tính **một lần** rồi dùng lại cho mọi i:

$$\boxed{(\phi(Q)\phi(K)^\top)V \;=\; \phi(Q)\big(\phi(K)^\top V\big)}.$$

Chi phí. Softmax: $O(N^2 \max(D,M))$. Linear với feature map C chiều: tính $\phi(K)^\top V \in \mathbb{R}^{C\times M}$ tốn $O(NCM)$, nhân với $\phi(Q)$ tốn $O(NCM)$; tổng $O(NCM)$.

Chọn feature map. Kernel mũ cần ϕ vô hạn chiều nên không linearize chính xác được. Kernel đa thức bậc 2 có ϕ hữu hạn với chi phí $O(ND^2M)$ — chỉ có lợi khi $N > D^2$. Trong thực nghiệm paper dùng:

$$\boxed{\phi(x) = \text{elu}(x) + 1}$$

với $\text{elu}(x) = x$ nếu $x > 0$, $\alpha(e^x - 1)$ nếu $x \leq 0$ ($\alpha = 1$). Vì $\text{elu}(x) > -1$, nên $\phi(x) > 0 \Rightarrow$ similarity dương. Chọn `elu` thay vì `relu` để **không triệt tiêu gradient khi** $x < 0$. Với ϕ này, $C = D$ và chi phí là $O(NDM)$.

3.3 Thêm causal mask

Mask làm V_i' chỉ tổng đến $j \leq i$:

$$V_i' = \frac{\phi(Q_i)^\top \sum_{j=1}^{i} \phi(K_j) V_j^\top}{\phi(Q_i)^\top \sum_{j=1}^{i} \phi(K_j)}.$$

Điểm tinh tế (rất quan trọng cho các bài sau). Dạng song song $\big(\phi(Q)\phi(K)^\top \odot M\big)V$ với M là mask tam giác **không** tái kết hợp được thành $\phi(Q)(\phi(K)^\top V)$, vì M nằm *giữa* hai phép nhân. Nhưng tổng tích lũy theo i thì tính được dần dần. Định nghĩa

$$S_i = \sum_{j=1}^{i} \phi(K_j) V_j^\top \in \mathbb{R}^{C\times M}, \qquad z_i = \sum_{j=1}^{i} \phi(K_j) \in \mathbb{R}^{C}.$$

Khi đó:

$$\boxed{V_i' = \frac{S_i^\top \phi(Q_i)}{\phi(Q_i)^\top z_i}}.$$

3.4 "Transformer là RNN": dạng hồi quy

S_i và z_i tính từ S_{i-1}, z_{i-1} bằng một phép cộng:

$$\begin{aligned} S_0 &= 0, \quad z_0 = 0, \\ \boxed{S_i} &\boxed{= S_{i-1} + \phi(k_i)\, v_i^\top}, \\ z_i &= z_{i-1} + \phi(k_i), \\ o_i &= \frac{S_i^\top \phi(q_i)}{\phi(q_i)^\top z_i}. \end{aligned}$$

(Paper viết đủ một lớp: $s_i = s_{i-1} + \phi(x_i W_K)(x_i W_V)^\top$, $y_i = f_l\big(\frac{\phi(x_i W_Q)^\top s_i}{\phi(x_i W_Q)^\top z_i} + x_i\big)$, với f_l là FFN. Ở đây RNN có **hai trạng thái ẩn**: "attention memory" s và "normalizer memory" z, và "thời gian" của RNN là *chiều chuỗi*, khác Universal Transformer vốn hồi quy theo chiều sâu.)

Ý nghĩa: S_i là một **bộ nhớ liên tưởng** (associative memory, "fast weights"): mỗi cặp (k_j, v_j) được ghi bằng tích ngoài $\phi(k_j)v_j^\top$ (hạng 1), và truy vấn q đọc bằng phép nhân $S^\top\phi(q)$ — tức là cộng các v_j có trọng số $\phi(q)^\top\phi(k_j)$.

3.5 Ví dụ số nhỏ (đối chiếu hai dạng)

Lấy $\phi =$ identity, bỏ chuẩn hóa, $d_k = d_v = 2$. Cho $k_1 = (1,0)^\top, v_1 = (2,3)^\top$; $k_2 = (0,1)^\top, v_2 = (4,5)^\top$; $q_2 = (1,1)^\top$.

Dạng hồi quy: $S_1 = k_1 v_1^\top = \begin{pmatrix} 2 & 3 \\ 0 & 0 \end{pmatrix}$, $S_2 = S_1 + k_2 v_2^\top = \begin{pmatrix} 2 & 3 \\ 4 & 5 \end{pmatrix}$, $o_2 = S_2^\top q_2 = (2{+}4,\ 3{+}5)^\top = (6,8)^\top$.

Dạng song song: $o_2 = \sum_{j\le 2}(q_2^\top k_j)v_j = 1\cdot(2,3) + 1\cdot(4,5) = (6,8)$. ✓ Hai dạng trùng nhau.

3.6 Gradient với bộ nhớ hằng số

Vì dạng hồi quy cần lưu mọi S_i để backprop, bộ nhớ tăng thêm $\max(D, M)$ lần. Paper dẫn các gradient dưới dạng tổng tích lũy (với $\bar{V}_i$ là tử số, $\nabla_{\bar{V}_i} L$ gradient tương ứng):

$$\nabla_{\phi(Q_i)} L = \nabla_{\bar{V}_i} L \Big(\sum_{j\le i} \phi(K_j) V_j^\top\Big)^\top, \quad \nabla_{\phi(K_i)} L = \Big(\sum_{j\ge i} \phi(Q_j) \nabla_{\bar{V}_j} L^\top\Big) V_i, \quad \nabla_{V_i} L = \Big(\sum_{j\ge i} \phi(Q_j) \nabla_{\bar{V}_j} L^\top\Big)^\top \phi(K_i).$$

(công thức 13-15 của paper, ký hiệu theo hàng; ở đây chỉ nêu cấu trúc). Cả ba là tổng tích lũy (trước hoặc sau) nên tính trong thời gian tuyến tính, bộ nhớ hằng số theo N (phụ thuộc $\max(C, M)$), cài đặt bằng khoảng 200 dòng CUDA. Pseudo-code forward của paper (thu gọn):

```
def forward(phiQ, phiK, V):     # tử số, chưa chia cho z
    S = zeros(C, M)
    for i in 1..N:
        S = S + outer(phiK[i], V[i])       # S_i = S_{i-1} + phi(k_i) v_i^T
        Vbar[i] = S.T @ phiQ[i]
    return Vbar                              # chi phí O(N C M), bộ nhớ O(max(C,M))
```

3.7 Huấn luyện song song và suy luận tuần tự

- **Huấn luyện**: biết toàn bộ chuỗi đích nên song song được theo lớp (FFN và attention), như Transformer.
- **Suy luận**: token i là đầu vào của bước $i+1$ nên bắt buộc tuần tự; softmax-Transformer tốn $O(i)$ (thực ra bình phương nếu không cache), còn linear transformer chỉ cần giữ (S, z) và cập nhật: **thời gian và bộ nhớ mỗi bước không đổi**.

Lưu ý quan trọng ngầm: paper chưa trình bày dạng *chunkwise* — thuật toán tuần tự trên GPU bị giới hạn bởi băng thông; việc tăng tốc huấn luyện thực sự đến ở bài 19/20.

4. Thí nghiệm và kết quả chính

Baseline: softmax Transformer và Reformer (lsh-X = X vòng băm). Mã PyTorch + kernel CUDA cho gradient.

4.1 Chi phí tính toán/bộ nhớ (GTX 1080 Ti 11GB, $N \in \{2^9, \dots, 2^{16}\}$)

Softmax tăng bình phương, Reformer và linear tăng tuyến tính; tối đa chạy được: softmax 4,096; lsh-4/lsh-8 16,384. Paper nói phương pháp của họ nhanh hơn và tốn bộ nhớ ít hơn các baseline ở mọi cấu hình (Hình 1; số liệu từng điểm không có trong text trích).

4.2 Bài toán copy (hội tụ)

Chuỗi tối đa 128, 10 ký hiệu phân cách bằng ký hiệu đặc biệt, 4 lớp 8 head, batch 64, RAdam LR 10^{-3} rồi 10^{-4} sau 3000 bước. Linear hội tụ êm và đạt cùng loss cuối với softmax; Reformer có loss cao hơn do nhiễu băm.

4.3 Sinh ảnh từng pixel

MNIST (784 pixel)	bits/dim	ảnh/giây
Softmax	0.621	0.45 (1×)
LSH-1	0.745	0.68 (1.5×)
LSH-4	0.676	0.27 (0.6×)
Linear	0.644	**142.8 (317×)**

CIFAR-10 (huấn luyện 1 tuần/1 GPU)	bits/dim	ảnh/giây
Softmax	3.47	0.004 (1×)
LSH-1	3.39	0.015 (3.75×)
LSH-4	3.51	0.005 (1.25×)
Linear	3.40	**17.85 (4,462×)**

Cấu hình MNIST: 8 lớp, 8 head, embedding 256, FFN gấp 4 lần, mixture 10 logistics, 250 epoch, batch 10. CIFAR-10: 16 lớp, softmax chỉ chạy batch 1 trên P40 24GB, linear/Reformer batch 4. Linear hoàn thành số epoch nhiều gấp 3 lần softmax trong cùng thời gian (nên perplexity tốt hơn). Paper nhấn mạnh bộ nhớ khi sinh là hằng số (chỉ giữ s_i, z_i), có thể sinh đồng thời 10,000 ảnh MNIST trên một GPU.

4.4 Nhận dạng giọng nói (WSJ 80 giờ, CTC, không tự hồi quy)

Mô hình	PER dev ↓	Thời gian/epoch (s)
Bi-LSTM	10.94	1047
Softmax	5.12	2711
LSH-4	9.33	2250
Linear	8.08	**824**

Linear nhanh hơn softmax hơn 3 lần mỗi epoch nhưng PER kém hơn softmax (8.08 vs 5.12); vượt LSTM và Reformer.

5. Phân tích / ablation

Paper **gần như không có ablation** về feature map. Các quan sát trong bài:

- Kernel đa thức bậc 2 cho chi phí $O(ND^2M)$, chỉ lợi khi $N > D^2$ (phân tích, không thí nghiệm).
- Nếu dùng kernel cho similarity dương lên query/key thì linear attention hội tụ bình thường (nhận xét trong Related Work).
- Về chất lượng: khoảng cách chất lượng với softmax *xuất hiện rõ ở* ASR (8.08 vs 5.12 PER) nhưng nhỏ ở sinh ảnh (MNIST 0.644 vs 0.621 bits/dim). Chưa có thí nghiệm ngôn ngữ tự nhiên quy mô lớn.

6. Giới hạn

Do tác giả nêu / ngụ ý:

- Lựa chọn feature map còn để ngỏ ("choice of feature map for linear attention" là hướng tương lai); ý định gợi ý: xấp xỉ RBF bằng random Fourier features để dùng được mô hình tiền huấn luyện softmax.
- Các thí nghiệm dùng chuỗi vừa phải (tối đa vài nghìn phần tử) và chỉ vài tác vụ (copy, ảnh, ASR); chưa có language modeling văn bản quy mô lớn.

Nhận xét bổ sung:

- **Không có cơ chế quên.** S_i chỉ cộng dồn, không bao giờ giảm; các bài 19-20 thêm suy giảm γ hoặc cổng α_t, và bài 23-24 thêm delta rule để *ghi đè* thay vì chỉ cộng.
- **Dung lượng bị chặn.** $S \in \mathbb{R}^{d_k \times d_v}$ chỉ lưu tối đa $\sim d_k$ cặp độc lập; khi chuỗi dài hơn d_k, các (k, v) chồng chập gây nhiễu, đặc biệt hại cho tác vụ tra cứu chính xác (recall). Bài 20 (Bảng 3) và bài 21 đo điều này.
- **Không có mã hóa vị trí.** Tổng $\sum \phi(k_j)v_j^\top$ không phân biệt thứ tự; RetNet (19) nhận xét "linear attention struggles to effectively encode position information".
- **Tuần tự không tận dụng tensor core.** Cài đặt bằng vòng lặp cộng từng phần tử, tính toán thấp so với truy cập bộ nhớ (low arithmetic intensity), chậm hơn lý thuyết (nội dung bài 20).
- Mẫu số $\phi(q)^\top z$ có thể bất ổn số học (các bài sau, ví dụ TransNormer/RetNet, bỏ hoặc thay bằng chuẩn hóa nhóm).

7. Liên hệ với các bài khác

- **01:** thay `softmax(QK^T)` bằng kernel; giữ nguyên Q, K, V và FFN.
- **17 Longformer:** hướng "cắt cặp" (giữ softmax) đối lập với "đổi kernel + nén".
- **19 RetNet:** thêm suy giảm hằng γ: $S_n = \gamma S_{n-1} + k_n v_n^\top$, bỏ mẫu số, thêm xPos/rotation, giới thiệu dạng *chunkwise*.
- **20 GLA:** suy giảm phụ thuộc dữ liệu $S_t = \mathrm{Diag}(\alpha_t) S_{t-1} + k_t v_t^\top$ và thuật toán chunkwise I/O-aware.
- **21 Mamba / 22 Mamba-2:** cùng họ hồi quy tuyến tính với cổng/suy giảm chọn lọc; 22 chỉ ra "đối ngẫu" với linear attention.

- **23 DeltaNet / 24 Gated DeltaNet:** thay phép *cộng* bằng *delta rule* (sửa lỗi dự đoán $v - S^\top k$ rồi ghi).
- **44 FlashAttention:** cùng vấn đề băng thông bộ nhớ nhưng giữ softmax.

8. Gợi ý áp dụng

Biểu diễn hồi quy giúp hiểu vì sao trạng thái không tăng theo độ dài chuỗi: mỗi head tích luỹ một ma trận kích thước phụ thuộc chiều key và value. Cái giá là lịch sử được nén vào trạng thái hữu hạn; khả năng truy hồi phụ thuộc cả biểu diễn và tác vụ, không chỉ độ phức tạp tiệm cận.

9. Thí nghiệm gợi ý

Viết hai cách tính attention tuyến tính causal, một cách dùng ma trận và một cách cập nhật trạng thái từng bước; so sánh đầu ra và mẫu số với cùng feature map. Sau đó khảo sát tác vụ associative recall khi thay đổi kích thước trạng thái và số cặp cần nhớ.

10. Tóm tắt công thức / khái niệm

Khái niệm	Công thức
Attention tổng quát	$V'_i = \frac{\sum_j \text{sim}(Q_i, K_j) V_j}{\sum_j \text{sim}(Q_i, K_j)}$, $\text{sim} \geq 0$
Softmax	$\text{sim} = \exp(q^\top k / \sqrt{D})$
Kernel hóa	$\text{sim} = \phi(q)^\top \phi(k)$
Tái kết hợp (không mask)	$(\phi(Q)\phi(K)^\top)V = \phi(Q)(\phi(K)^\top V)$
Feature map	$\phi(x) = \text{elu}(x) + 1$
Trạng thái	$S_i = S_{i-1} + \phi(k_i) v_i^\top$, $z_i = z_{i-1} + \phi(k_i)$
Đầu ra	$o_i = S_i^\top \phi(q_i) \,/\, \phi(q_i)^\top z_i$
Chi phí	thời gian $O(NCM)$; bộ nhớ $O(N \max(C, M))$ khi huấn luyện; $O(CM)$ mỗi bước khi suy luận
Bất tiện	mask M chặn tái kết hợp $\Rightarrow$ cần dạng tổng tích lũy / hồi quy

11. Kết luận ngắn

Bài báo cho thấy, chỉ bằng việc thay softmax bằng kernel phân rã, attention có causal mask *chính là* một RNN có trạng thái ma trận $S_t = S_{t-1} + \phi(k_t) v_t^\top$: huấn luyện song song như Transformer, suy luận chi phí hằng số như RNN. Nó chưa giải quyết chất lượng (không quên, dung lượng nhớ hữu hạn, không vị trí) và hiệu quả phần cứng — nhưng toàn bộ chuỗi bài 19-24 là lời giải lần lượt cho các điểm này.

Nguồn gốc: https://arxiv.org/abs/2006.16236

Bài 19

Retentive Network: A Successor to Transformer for Large Language Models (2023)

Tác giả: Yutao Sun, Li Dong, Shaohan Huang, Shuming Ma, Yuqing Xia, Jilong Xue, Jianyong Wang, Furu Wei (Microsoft Research, Tsinghua University)
Venue / arXiv: arXiv:2307.08621 (v4, 8/2023); paper không nêu venue trong bản text đã đọc
Nguồn: https://arxiv.org/abs/2307.08621
Chủ đề: Retention: một cơ chế có **ba dạng tính toán tương đương** (song song, hồi quy, chunkwise); suy giảm mũ theo khoảng cách; multi-scale decay; suy luận $O(1)$.
Vị trí trong lộ trình: Đọc sau 18 (Linear Transformers: $S_t = S_{t-1} + \phi(k_t)v_t^\top$) và 12 (RoPE). RetNet thêm **hệ số suy giảm** γ vào trạng thái, bỏ softmax và mẫu số, và đưa vào **dạng chunkwise** (cầu nối giữa huấn luyện song song và suy luận hồi quy; GLA ghi nhận dạng này đã được dùng trước ở Hua et al. 2022 và Sun et al. 2023a, tức chính RetNet). Bài sau: 20 (GLA) làm cho γ phụ thuộc dữ liệu và làm thuật toán chunkwise nhận biết I/O phần cứng.

> Ghi chú diễn giải, không phải bản dịch. Quy ước vector cột cho q_t, k_t, v_t và $S_t \in \mathbb{R}^{d_k \times d_v}$; công thức của bài báo dùng quy ước chuyển vị.

1. Bối cảnh

Paper đặt vấn đề bằng "**tam giác bất khả thi**": ba thứ mong muốn cùng lúc: (1) **song song hóa khi huấn luyện**, (2) **suy luận chi phí thấp** ($O(1)$ mỗi bước), (3) **chất lượng ngang Transformer**. Theo paper, các hướng trước đó chỉ đạt hai trong ba:

- **Linear attention** (18): suy luận rẻ, huấn luyện song song được, nhưng chất lượng kém Transformer.
- **RNN thuần/ phần tử** (RWKV, AFT): suy luận rẻ, nhưng hy sinh song song hóa hoặc biểu diễn (trạng thái chỉ là vector).
- **SSM kiểu S4, H3, Hyena**: song song bằng tích chập/FFT, nhưng chất lượng chưa vượt Transformer trên ngôn ngữ.

Vấn đề của Transformer khi triển khai (đoạn mở đầu): chi phí $O(N)$ mỗi bước và KV cache bị giới hạn bởi băng thông bộ nhớ ("memory-bound").

Khái niệm nền.

- *Dạng song song / hồi quy / chunkwise* của cùng một hàm: ba cách tính cho ra đúng cùng kết quả, mỗi dạng phù hợp một tình huống. (Mục 3 dẫn từng bước.)
- *Suy giảm mũ (exponential decay):* trọng số γ^{n-m} giảm theo khoảng cách $n-m$ với $0 < \gamma < 1$; token xa đóng góp ít hơn một cách có hệ thống.
- *xPos:* một dạng mã hóa vị trí tương đối cho Transformer có thể suy ra tự nhiên từ công thức hồi quy; họ hàng của RoPE (bài 12).
- *GroupNorm:* chuẩn hóa theo nhóm kênh; ở đây mỗi head là một nhóm.

2. Ý tưởng chính

Xuất phát từ một hệ thống hồi quy tuyến tính $s_n = As_{n-1} + k_n^\top v_n$, $o_n = q_n s_n$, paper **suy ra** dạng song song giống attention khi chéo hóa A và đơn giản hóa thành một số vô hướng γ. Kết quả là retention: $\text{Retention}(X) = (QK^\top \odot D)V$ với $D_{nm} = \gamma^{n-m}$ (mask nhân quả + suy giảm mũ). Cùng hàm đó tính được *tuần tự* với trạng thái $S_n = \gamma S_{n-1} + k_n v_n^\top$ (suy luận $O(1)$), hoặc *theo từng khúc* (chunk): song song

trong khúc, hồi quy giữa các khúc (huấn luyện chuỗi dài tuyến tính). Thêm "multi-scale": mỗi head dùng γ khác nhau (nhớ ngắn/dài), cổng swish, GroupNorm. Kết quả: perplexity tốt hơn Transformer cùng cỡ khi mô hình lớn (>2B), huấn luyện tiết kiệm bộ nhớ, giải mã nhanh và bộ nhớ không đổi theo độ dài.

3. Phương pháp chi tiết

3.1 Từ hồi quy sang dạng song song (suy dẫn)

Bắt đầu từ ánh xạ vô hướng $v_n \mapsto o_n$ qua trạng thái s_n (paper, Eq. 1):

$$s_n = As_{n-1} + K_n^\top v_n, \qquad o_n = Q_n s_n = \sum_{m=1}^{n} Q_n A^{n-m} K_m^\top v_m.$$

Bước 1 — **làm Q, K phụ thuộc nội dung:** $Q = XW_Q,\ K = XW_K$.

Bước 2 — **chéo hóa** $A = \Lambda(\gamma e^{i\theta})\Lambda^{-1}$ (với $\gamma, \theta \in \mathbb{R}^d$); hấp thụ Λ vào W_Q, W_K:

$$o_n = \sum_{m=1}^{n} Q_n(\gamma e^{i\theta})^{n-m} K_m^\top v_m = \sum_{m=1}^{n} \left(Q_n(\gamma e^{i\theta})^n\right)\left(K_m(\gamma e^{i\theta})^{-m}\right)^\top v_m.$$

Hai thừa số có dạng **xPos** (vị trí tương đối).

Bước 3 — **đơn giản hóa γ thành vô hướng**:

$$o_n = \sum_{m=1}^{n} \gamma^{n-m} \left(Q_n e^{in\theta}\right)\left(K_m e^{im\theta}\right)^\dagger v_m,$$

trong đó † là chuyển vị liên hợp. Phần $e^{in\theta}$ đóng vai trò xoay vị trí giống RoPE (bài 12); phần γ^{n-m} là suy giảm. Để dễ nắm bản chất, từ đây bỏ phần xoay (đặt $\theta = 0$): $o_n = \sum_{m \le n} \gamma^{n-m}(q_n^\top k_m)v_m$.

3.2 Ba dạng của retention

(a) Dạng song song (Eq. 5):

$$Q = (XW_Q) \odot \Theta,\ K = (XW_K) \odot \bar{\Theta},\ V = XW_V, \quad D_{nm} = \begin{cases} \gamma^{n-m}, & n \ge m \\ 0, & n < m \end{cases}$$

$$\boxed{\text{Retention}(X) = (QK^\top \odot D)\,V}$$

($\Theta_n = e^{in\theta}$, $\bar{\Theta}$ liên hợp phức). D gộp *mask nhân quả* và *suy giảm mũ theo khoảng cách*. Cấu trúc y như attention nhưng **không softmax** — đó là lý do hồi quy hóa được.

(b) Dạng hồi quy (Eq. 6):

$$\boxed{S_n = \gamma S_{n-1} + k_n v_n^\top}, \qquad \text{Retention}(X_n) = S_n^\top q_n.$$

Chứng minh tương đương (bỏ phần xoay): khai triển $S_n = \sum_{m=1}^{n} \gamma^{n-m} k_m v_m^\top$ (quy nạp: $S_1 = k_1 v_1^\top$; $S_n = \gamma \sum_{m<n} \gamma^{n-1-m} k_m v_m^\top + k_n v_n^\top$). Khi đó $S_n^\top q_n = \sum_m \gamma^{n-m}(k_m^\top q_n)v_m = \sum_m \gamma^{n-m}(q_n^\top k_m)v_m$, trùng dạng song song. Trạng thái chỉ là **một ma trận** $d_k \times d_v$ **mỗi head**; không có KV cache, không có mẫu số.

Ví dụ số (cùng ví dụ bài 18, thêm $\gamma = 0.5$): $k_1 = (1,0)^\top, v_1 = (2,3)^\top$, $k_2 = (0,1)^\top, v_2 = (4,5)^\top$, $q_2 = (1,1)^\top$. Hồi quy: $S_1 = \begin{pmatrix} 2 & 3 \\ 0 & 0 \end{pmatrix}$, $S_2 = 0.5S_1 + k_2 v_2^\top = \begin{pmatrix} 1 & 1.5 \\ 4 & 5 \end{pmatrix}$, $o_2 = S_2^\top q_2 = (1+4,\ 1.5+5)^\top = (5, 6.5)^\top$. Song song: $o_2 = \gamma^1 (q_2^\top k_1) v_1 + \gamma^0 (q_2^\top k_2) v_2 = 0.5 \cdot 1 \cdot (2,3) + 1 \cdot (4,5) = (5, 6.5)$ ✓.

(c) Dạng chunkwise (Eq. 7). Chia chuỗi thành các khúc độ dài B. Dùng chỉ số vị trí trong khúc $j = 1, \ldots, B$ (paper dùng 0-based nên có $B - i - 1$ và $i + 1$; dưới đây là bản 1-based tương đương). Với khúc thứ c, đặt R_c là trạng thái ở **cuối** khúc c (đã gom mọi token từ đầu chuỗi đến hết khúc c):

Trạng thái cuối khúc — mỗi token j trong khúc còn phải suy giảm đến cuối khúc ($B - j$ bước), và trạng thái cũ suy giảm B bước:

$$R_c = \gamma^B R_{c-1} + \sum_{j=1}^{B} \gamma^{B-j} k_j v_j^\top = \gamma^B R_{c-1} + K_{[c]}^\top (V_{[c]} \odot \zeta), \quad \zeta_j = \gamma^{B-j}.$$

Đầu ra token j trong khúc c gồm phần các token **trong cùng khúc** (song song) và phần **các khúc trước** (qua R_{c-1}, suy giảm j bước kể từ đầu khúc):

$$\boxed{o_{[c]} = \underbrace{(Q_{[c]} K_{[c]}^\top \odot D) V_{[c]}}_{\text{trong khúc}} + \underbrace{(Q_{[c]} R_{c-1}) \odot \xi}_{\text{giữa các khúc}}, \qquad \xi_j = \gamma^j}$$

Kiểm chứng phần giữa các khúc: đóng góp của token m ở khúc trước vào token ở vị trí j của khúc c là $\gamma^{(c-1)B+j-m}(q^\top k_m) v_m = \gamma^j \cdot q^\top [\gamma^{(c-1)B-m} k_m v_m^\top]$; cộng theo m cho đúng $\gamma^j\, q^\top R_{c-1}$ (với $R_{c-1} = \sum_{m \le (c-1)B} \gamma^{(c-1)B-m} k_m v_m^\top$) ✓.

Ví dụ kiểm chứng nhanh (dữ liệu như trên, $\gamma = 0.5$, một khúc $B = 2$ gồm token 1 và 2, $R_0 = 0$): đầu ra token 2 chỉ có phần trong khúc, $o_2 = (5, 6.5)^\top$ như đã tính; trạng thái cuối khúc $R_1 = \gamma^1 k_1 v_1^\top + \gamma^0 k_2 v_2^\top = \begin{pmatrix} 1 & 1.5 \\ 4 & 5 \end{pmatrix} = S_2$ ✓ (bằng S_2 của dạng hồi quy).

Chi phí (Eq trong §3.5 paper): $O(dn(b+h))$ với b là kích thước khúc, h là chiều head; khúc song song tốn $O(B^2 d)$ mỗi khúc (tổng $O(nBd)$), phần giữa khúc tốn $O(d_k d_v)$ mỗi token. Chọn $b = 512$, $h = 256$ nên "$b + h$" gần như không đáng kể với mô hình lớn. Mốc tham chiếu: $B = n \Rightarrow$ dạng song song; $B = 1 \Rightarrow$ dạng hồi quy.

Pseudo-code của paper (Hình 4, đã rút gọn, bỏ batch/head):

```
def ParallelRetention(q, k, v, decay_mask):       # decay_mask[n,m] = gamma^(n-m) nếu
↪  n>=m
    r = (q @ k.T) * decay_mask
    return group_norm(r @ v)

def RecurrentRetention(q, k, v, past_kv, decay):  # một token
    kv = decay * past_kv + outer(k, v)            # S_n = gamma S_{n-1} + k v^T
    return group_norm(q @ kv), kv

def ChunkwiseRetention(q, k, v, past_kv, decay_mask, chunk_decay, inner_decay):
    inner = ((q @ k.T) * decay_mask) @ v          # trong khúc
    cross = (q @ past_kv) * inner_decay[:,None]   # từ các khúc trước (xi_j = gamma^j)
    out   = group_norm(inner + cross)
    kv    = chunk_decay * past_kv + k.T @ (v * zeta[:,None])  # R_c ; chunk_decay=gamma^B
    return out, kv
```

(Trong code của paper, phép nhân `zeta` được gộp vào `decay_mask`/`inner_decay`.)

3.3 Gated Multi-Scale Retention (MSR)

Mỗi lớp có $h = d_{model}/d$ head. Điểm mới:

- **Multi-scale decay:** mỗi head i có γ_i riêng, **cố định, không học**. Công thức mặc định (Eq. 8): $\gamma = 1 - 2^{-5-\text{arange}(0,h)} \in \mathbb{R}^h$. Cụ thể, với $h = 8$: $\gamma \in \{1-2^{-5}, 1-2^{-6}, \ldots, 1-2^{-12}\}$, tức γ từ 0.96875 đến ≈ 0.99976. "Chân trời nhớ" xấp xỉ $1/(1-\gamma)$ từ 32 đến 4096 token. Trong thí nghiệm so sánh kích thước, paper dùng $\gamma = 1 - e^{\text{linspace}(\log \frac{1}{32}, \log \frac{1}{512}, h)}$, tức chân trời nhớ từ 32 đến 512 token (giống nhau ở mọi cỡ mô hình).
- **Cổng swish:** $\text{MSR}(X) = (\text{swish}(XW_G) \odot Y) W_O$, với $Y = \text{GroupNorm}_h(\text{Concat}(\text{head}_1, \ldots, \text{head}_h))$.
- **GroupNorm theo head:** các head với γ khác nhau có phương sai khác nhau nên chuẩn hóa từng head riêng (theo ý tưởng Sub-LayerNorm). Tính chất bất biến tỷ lệ của GroupNorm cho phép nhân hằng số tùy ý vào điểm retention mà không đổi kết quả, nên paper chuẩn hóa $QK^\top/\sqrt{d}$, $\tilde{D}_{nm} = D_{nm}/\sqrt{\sum_i D_{ni}}$, và $\tilde{R}_{nm} = R_{nm}/\max(|\sum_i R_{ni}|, 1)$ chỉ để ổn định số, không đổi kết quả.

3.4 Kiến trúc tổng thể

Khối RetNet (pre-LN, residual) gồm MSR và FFN:

$$Y^l = \text{MSR}(\text{LN}(X^l)) + X^l, \qquad X^{l+1} = \text{FFN}(\text{LN}(Y^l)) + Y^l, \quad \text{FFN}(X) = \text{gelu}(XW_1)W_2.$$

Phân bổ tham số (để so công bằng với Transformer, $d = d_{model}$): Transformer có $4d^2$ (attention) $+ 8d^2$ (FFN, trung gian $4d$) $= 12d^2$. RetNet có $W_Q, W_K \in \mathbb{R}^{d\times d}$ ($2d^2$), $W_G, W_V \in \mathbb{R}^{d\times 2d}$ ($4d^2$), $W_O \in \mathbb{R}^{2d\times d}$ ($2d^2$): tổng retention $8d^2$; vì vậy FFN trung gian chỉ $2d$ ($4d^2$). Tổng $12d^2$ ✓. Head dim: 256 cho Q,K và 512 cho V.

Huấn luyện dùng dạng song song và chunkwise; **suy luận** dùng dạng hồi quy.

3.5 So sánh nhanh với các phương pháp khác (theo Bảng 1 của paper)

Kiến trúc	Song song khi train	Chi phí suy luận	Bộ nhớ chuỗi dài	Chất lượng
Transformer	có	$O(N)$	$O(N^2)$	rất tốt (✔✔)
Linear Transformer	có	$O(1)$	$O(N)$	kém
RNN	không	$O(1)$	$O(N)$	kém
RWKV	không	$O(1)$	$O(N)$	tốt
H3/S4	có	$O(1)$	$O(N \log N)$	tốt
Hyena	có	$O(N)$	$O(N \log N)$	tốt
RetNet	có	$O(1)$	$O(N)$	rất tốt (✔✔)

Khác biệt với linear attention (18): (i) **có suy giảm** γ, (ii) **bỏ chuẩn hóa mẫu số** thay bằng GroupNorm, (iii) vị trí tương đối qua xPos, (iv) head lớn và cổng swish. Paper nhận định linear attention "struggles to effectively encode position information".

4. Thí nghiệm và kết quả chính

Thiết lập. Ba cỡ, huấn luyện từ đầu trên Pile + C4 + The Stack, 100B token, batch 4M token, độ dài 2048, AdamW ($\beta = (0.9, 0.98)$), khởi tạo DeepNet, 512 GPU AMD MI200.

Cỡ	Hidden	Lớp	Head	FFN	LR
1.3B	2048	24	8	4096	6×10^{-4}
2.7B	2560	32	10	5120	3×10^{-4}
6.7B	4096	32	16	8192	3×10^{-4}

4.1 Language modeling và downstream

Hình 5: perplexity validation của RetNet gần bằng và với cỡ lớn hơn ~2B tốt hơn Transformer (các giá trị cụ thể không đọc được từ text). Bảng 8 (perplexity, đánh giá 128 token cuối của chunk 2048):

Ngữ cảnh	512	1024	2048
Transformer	13.55	12.56	12.35
RetNet	13.09	12.14	11.98

Bảng 3 (6.7B, độ chính xác):

	HS	BoolQ	COPA	PIQA	Winograd	Winogrande	SC	Avg
Zero-shot Transformer	55.9	62.0	69.0	74.6	69.5	56.5	75.0	66.07
Zero-shot RetNet	60.7	62.2	77.0	75.4	77.2	58.1	76.0	69.51
4-shot Transformer	55.8	58.7	71.0	75.0	71.9	57.3	75.4	66.44
4-shot RetNet	60.5	60.1	78.0	76.0	77.9	59.9	75.9	69.76

(Lưu ý: GLA sau này báo cáo RetNet yếu hơn Transformer++ ở cỡ 340M/1.3B trên các tập khác — xem bài 20; hai thiết lập khác nhau.)

4.2 Chi phí huấn luyện (độ dài 8192, 8×A100-80GB, khúc 512, cài đặt PyTorch thuần)

Cỡ	Bộ nhớ (GB): Trm / Trm+Flash / RetNet	Thông lượng (từ/giây): Trm / Trm+Flash / RetNet
1.3B	74.8 / 38.8 / 34.5	10832.4 / 63965.2 / 73344.8
2.7B	69.6 / 42.1 / 42.0	5186.0 / 34990.2 / 38921.2
6.7B	69.0 / 51.4 / 48.0	2754.4 / 16230.1 / 17458.6
13B	61.4 / 46.3 / 45.9	1208.9 / 7945.1 / 8642.2

Phát biểu của paper (mục giới thiệu): huấn luyện tiết kiệm 25-50% bộ nhớ và nhanh khoảng 7× so với Transformer chuẩn, vẫn cạnh tranh với FlashAttention dù RetNet chưa có kernel hợp nhất.

4.3 Chi phí suy luận (6.7B, A100-80GB)

- **Bộ nhớ:** Transformer tăng tuyến tính theo độ dài do KV cache; RetNet gần như không đổi; bộ nhớ thêm của RetNet khoảng 3% (trọng số chiếm 97%).
- **Thông lượng:** của Transformer giảm khi giải mã dài; RetNet không đổi theo độ dài.
- **Độ trễ:** của Transformer tăng theo batch và độ dài; RetNet “almost the same” qua các batch size và độ dài.
- Con số tổng kết (mục giới thiệu): với 7B và độ dài 8k, RetNet giải mã nhanh 8.4× và tiết kiệm 70% bộ nhớ so với Transformer có KV cache.

4.4 So sánh với các mô hình hiệu quả khác (200M tham số, 16 lớp, hidden 1024, 10k bước, batch 0.5M token)

Perplexity (thấp hơn tốt hơn):

Phương pháp	In-domain	PG22	QMSum	GovReport	SummScreen
RWKV	30.92	51.41	28.17	19.80	25.78
H3	29.97	49.17	24.29	19.19	25.11
Hyena	32.08	52.75	28.18	20.55	26.51
Linear Transformer	40.24	63.86	28.45	25.33	32.02
RetNet	**26.05**	**45.27**	**21.33**	**16.52**	**22.48**

5. Phân tích / ablation

Bảng 6 (200M, perplexity; mỗi dòng bỏ một thành phần):

Biến thể	In-domain	PG22	QMSum	GovReport	SummScreen
RetNet đầy đủ	26.05	45.27	21.33	16.52	22.48
− cổng swish	27.84	49.44	22.52	17.45	23.72
− GroupNorm	27.54	46.95	22.61	17.59	23.73
− suy giảm γ ($\gamma = 1$)	27.86	47.85	21.99	17.49	23.70
− multi-scale decay (mọi head $\gamma = 127/128$)	27.02	47.18	22.08	17.17	23.38
Giảm head dim (256 → 64)	27.68	47.72	23.09	17.46	23.41

Rút ra (theo paper): cả bốn thành phần đều có ích; **cổng swish** và **suy giảm** đáng kể nhất ở in-domain (+1.8 PPL); **head dim lớn hơn** tốt hơn — được diễn giải là kích thước head chính là "dung lượng bộ nhớ" của trạng thái ẩn (công thức hồi quy: trạng thái có kích thước $d_k \times d_v$). Cũng nhấn mạnh: **multi-scale** (nhiều γ) tốt hơn một γ chung.

6. Giới hạn

Do tác giả nêu:

- Thực nghiệm giới hạn ở language modeling, tối đa 6.7B (huấn luyện) / 13B (đo tốc độ); hướng tương lai: mở rộng quy mô, đa phương thức, triển khai trên thiết bị biên.
- Cài đặt PyTorch thuần, chưa có kernel hợp nhất kiểu FlashAttention ("leave kernel fusion ... for future work").
- RetNet bắt đầu vượt Transformer khi kích thước lớn hơn ~2B; ở cỡ nhỏ chỉ ngang bằng.

Nhận xét bổ sung:

- γ **cố định, không phụ thuộc nội dung**: mô hình không thể "quên có chọn lọc" hay "giữ chặt một thông tin quan trọng". Bài 20 (GLA, Bảng 4: 16.55 vs 15.56 vs 14.77 PPL huấn luyện) cho thấy điều này tốn chất lượng; bài 21 giải thích thêm bằng tác vụ tổng hợp.
- Trong thí nghiệm của bài 20, RetNet yếu ở các tác vụ **recall** (FDA 2.9 vs Transformer++ 21.4 ở 340M) — trạng thái nén có dung lượng hữu hạn.
- Số head ít (8-16) và head dim lớn (256/512) làm trạng thái $\approx h \cdot 256 \cdot 512$ phần tử/lớp — lớn so với linear attention thông thường (xem mục 8).
- Chuẩn hóa điểm bằng $\max(|\cdot|, 1)$ và các "mẹo" số học làm khó tái cài đặt đúng từ paper.

7. Liên hệ với các bài khác

- **18 Linear Transformers:** RetNet = linear attention + suy giảm γ + GroupNorm + xPos + cổng swish, bỏ mẫu số; thêm dạng chunkwise.
- **12 RoPE:** phần xoay $e^{in\theta}$ của retention là cùng họ với RoPE; paper nói "Equation (3) presents a similar formulation as xPos and RoPE".
- **13 ALiBi:** $\gamma^{n-m} = e^{-(n-m)\lambda}$ chính là độ lệch tuyến tính trong log-attention (ALiBi), áp lên điểm *không softmax*; GLA (20, chú thích 3) nêu "linear attention with ALiBi".
- **20 GLA:** thay γ bằng α_t phụ thuộc dữ liệu; dạng chunkwise của RetNet là điểm xuất phát của thuật toán FlashLinearAttention.
- **21 Mamba:** Mamba (Phụ lục B) mô tả RetNet như một SSM đặc biệt với $N = 1$ chiều trạng thái trên mỗi kênh, nhưng head dim lớn đóng vai trò mở rộng trạng thái.
- **22 Mamba-2:** dạng "đối ngẫu" song song/hồi quy của retention là trường hợp riêng của khung SSD.
- **24 Gated DeltaNet:** γ trở thành cổng α_t và thêm delta rule.
- **44-45 FlashAttention:** RetNet so sánh trực tiếp tốc độ/bộ nhớ.

8. Gợi ý áp dụng

Ba dạng song song, hồi quy và theo chunk của retention biểu diễn cùng một phép tính, nhưng phù hợp với các chế độ chạy khác nhau. Kích thước trạng thái cố định theo độ dài chuỗi song vẫn tăng theo chiều key và value. Các hệ số suy giảm khác nhau tạo nhiều thang thời gian; không phải mỗi head nhất thiết học được một chức năng riêng.

9. Thí nghiệm gợi ý

Với tensor ngẫu nhiên, so sánh ba dạng tính trong cùng độ chính xác số và kiểm tra sai số làm tròn. Thay đổi hệ số suy giảm và độ dài chuỗi, rồi đo sự đóng góp của thông tin ở các khoảng cách khác nhau.

10. Tóm tắt công thức / khái niệm

Khái niệm	Công thức
Hồi quy gốc	$s_n = As_{n-1} + K_n^\top v_n,\ o_n = Q_n s_n$
Retention song song	$(QK^\top \odot D)V,\ D_{nm} = \gamma^{n-m}\,[n \geq m]$
Retention hồi quy	$S_n = \gamma S_{n-1} + k_n v_n^\top,\ o_n = S_n^\top q_n$
Chunkwise	$R_c = \gamma^B R_{c-1} + K^\top (V \odot \zeta);\ o = (QK^\top \odot D)V + (QR_{c-1}) \odot \xi;$ $\zeta_j = \gamma^{B-j},\ \xi_j = \gamma^j$
Multi-scale	$\gamma_i = 1 - 2^{-5-i}$ (mặc định); thí nghiệm: $32 \to 512$
MSR	$(\text{swish}(XW_G) \odot \text{GN}(\text{concat heads}))W_O$
Tham số/khối	$8d^2$ (retention) $+ 4d^2$ (FFN trung gian $2d$) $= 12d^2$
Chi phí	train: $O(dn(b+h))$; suy luận $O(1)$/token; bộ nhớ trạng thái không đổi

11. Kết luận ngắn

RetNet chứng minh được rằng một **retention có suy giảm mũ** có thể được tính ba cách tương đương, nhờ vậy vừa huấn luyện nhanh (song song/chunkwise) vừa giải mã $O(1)$, với chất lượng sánh Transformer ở cỡ vài tỷ tham số. Hạn chế trung tâm — γ cố định theo thời gian — là động lực trực tiếp cho GLA (bài 20) và Gated DeltaNet.

Nguồn gốc: https://arxiv.org/abs/2307.08621

Bài 20

Gated Linear Attention Transformers with Hardware-Efficient Training (2024)

Tác giả: Songlin Yang, Bailin Wang, Yikang Shen, Rameswar Panda, Yoon Kim (MIT, MIT-IBM Watson AI Lab)
Venue / arXiv: ICML 2024 (PMLR 235); arXiv:2312.06635 (v6, 8/2024)
Nguồn: https://arxiv.org/abs/2312.06635
Chủ đề: Cổng quên phụ thuộc dữ liệu cho linear attention $S_t = \text{Diag}(\alpha_t) S_{t-1} + k_t v_t^\top$; thuật toán chunkwise nhận biết I/O (FlashLinearAttention); chunk hai tầng + log-space để ổn định số và dùng tensor core.
Vị trí trong lộ trình: Đọc sau 18 (linear attention = RNN ma trận) và 19 (RetNet: γ hằng + dạng chunkwise). Bài này (a) làm γ thành α_t **phụ thuộc token**, (b) giải bài toán **làm thế nào huấn luyện hiệu quả trên GPU** khi đã có cổng. Bài sau: 21 (Mamba) và 22 (Mamba-2/SSD) cùng giải bài toán chọn lọc theo cách khác; 23-24 (DeltaNet, Gated DeltaNet) thêm delta rule và đưa vào đúng khung chunkwise của bài này.

> Ghi chú diễn giải, không phải bản dịch. Ký hiệu vector cột: $q_t, k_t \in \mathbb{R}^{d_k}$, $v_t \in \mathbb{R}^{d_v}$, $S_t \in \mathbb{R}^{d_k \times d_v}$ và $o_t = S_t^\top q_t$; quy ước của bài báo tương đương sau chuyển vị.

1. Bối cảnh

Hai vấn đề tồn tại sau bài 18-19 (theo paper):

1. **Hiệu quả phần cứng.** Các cài đặt linear attention "không nhận biết I/O" nên thực tế **chậm hơn FlashAttention-2** ở độ dài vừa phải (1K-4K), dù ít FLOPs hơn. Lý do: GPU hiện đại có tensor core (matmul fp16 nhanh khoảng 16× so với CUDA core trên A100) và phân cấp bộ nhớ (HBM lớn-chậm, SRAM nhỏ-nhanh); thuật toán hồi quy thuần (cộng từng phần tử) có *arithmetic intensity* thấp, không dùng tensor core.
2. **Chất lượng.** Linear attention kém softmax. RetNet/TransNormerLLM cải thiện nhờ suy giảm γ nhưng γ **toàn cục, không phụ thuộc dữ liệu**; trong RNN một chiều, cổng quên phụ thuộc dữ liệu được biết là quan trọng (LSTM, GRU).

Khái niệm nền (nhắc lại).

- *Forget gate / cổng quên:* hệ số $\in (0, 1)$ nhân vào trạng thái cũ trước khi cộng thông tin mới; gần 0 = quên hết, gần 1 = giữ nguyên.
- *Tensor core:* đơn vị tính ma trận chuyên dụng của GPU NVIDIA; chỉ nhanh nếu việc tính được viết dưới dạng matmul nửa chính xác với kích thước phù hợp (bội số 16).
- *Kernel fusion, tiling:* gộp nhiều phép toán vào một kernel và xử lý theo khối để dữ liệu ở lại SRAM, tránh đọc/ghi HBM (ý tưởng của FlashAttention, bài 44).
- *Prefix-sum / parallel scan:* tính tổng tích lũy song song; với trạng thái ma trận, nó đòi hỏi vật chất hóa (materialize) trạng thái của **từng** bước trong HBM, cực tốn I/O.
- *Perplexity (ppl):* hàm mũ của loss trung bình, thấp hơn tốt hơn.

2. Ý tưởng chính

GLA mở rộng linear attention bằng cách cho trạng thái S_t **quên có chọn lọc**: một cổng $\alpha_t \in (0, 1)^{d_k}$ do chính token x_t sinh ra, nhân theo từng chiều khóa. Dạng này tổng quát hóa RetNet ($\alpha_t \equiv \gamma$) và gần với Mamba nhưng dùng cổng theo *chiều khóa* với tham số rất ít (low-rank). Điểm khó là huấn luyện: ma trận suy giảm $D_{nm} = \gamma^{n-m}$ của RetNet không còn là Toeplitz đơn giản mà thành tích tích lũy $\prod \alpha$ theo từng chiều. Giải pháp: (i) **FlashLinearAttention**, thuật toán chunkwise I/O-aware cho linear attention thường; (ii) mở rộng

cho cổng bằng cách tách các hệ số suy giảm khỏi chunk (tích lũy cục bộ trong chunk), cộng **chunk hai tầng** để phần lớn phép tính vẫn là matmul nửa chính xác trên tensor core, chỉ phần đường chéo nhỏ cần log-space fp32. Kết quả: nhanh hơn FlashAttention-2 ngay cả ở 1K, ngang Transformer++/Mamba về chất lượng ở quy mô 340M-1.3B, mạnh về ngoại suy độ dài và các tác vụ cần recall hơn Mamba/RetNet.

3. Phương pháp chi tiết

3.1 Nền: linear attention ở ba dạng (§2 của paper)

Softmax attention có causal mask: $O = \text{softmax}\big((QK^\top)\odot M\big)V$. Linear attention thay $\exp(q^\top k)$ bằng tích của feature map; paper dẫn các kết quả gần đây rằng $\phi =$ **identity và không chuẩn hóa** đã chạy tốt (RetNet), nên:

$$\boxed{S_t = S_{t-1} + k_t v_t^\top, \qquad o_t = S_t^\top q_t} \qquad \text{(hồi quy)},$$

$$O = \big((QK^\top)\odot M\big)V \qquad \text{(song song, } M_{ij} = [i \geq j]\text{, chi phí } O(L^2 d)).$$

Mask M chặn việc tái kết hợp thành $Q(K^\top V)$ — ghi chú đã nêu ở bài 18.

Dạng chunkwise. Chia thành các chunk dài C; $S_{[i]} := S_{iC}$ là trạng thái sau i chunk. Liên chunk (Eq. 2):

$$S_{[i+1]} = S_{[i]} + \sum_{j=iC+1}^{(i+1)C} k_j v_j^\top = S_{[i]} + K_{[i+1]}^\top V_{[i+1]},$$

và đầu ra chunk $i+1$ (viết theo hàng như paper):

$$O_{[i+1]} = \underbrace{Q_{[i+1]} S_{[i]}}_{\text{inter-chunk}} + \underbrace{\big((Q_{[i+1]} K_{[i+1]}^\top)\odot M\big)V_{[i+1]}}_{\text{intra-chunk}}.$$

Chi phí: intra $O(C^2 d + Cd^2)$ mỗi chunk, inter $O(Cd^2)$; tổng L/C chunk:

$$O\Big(\tfrac{L}{C}(C^2 d + Cd^2)\Big) = O(LCd + Ld^2) < O(L^2 d) \text{ khi } L > d.$$

$C = L$ cho dạng song song; $C = 1$ cho dạng hồi quy.

3.2 Nguyên tắc phần cứng (§3.1-3.2)

Dạng	Ưu	Nhược (theo paper)
Hồi quy	FLOPs ít nhất	phép toán theo phần tử, không dùng tensor core, intensity thấp; lưu trạng thái mọi bước vào HBM thì I/O cao; parallel scan phải vật chất hóa trạng thái 2D mọi bước
Song song	dùng matmul, có thể tối ưu I/O như FlashAttention	FLOPs bậc hai theo L
Chunkwise	gần như toàn matmul (bội 16 → tensor core); C là núm vặn giữa FLOPs và song song	các cài đặt trước chưa I/O-aware ⇒ chậm hơn FlashAttention ở 2K-4K

3.3 FlashLinearAttention (§3.3, Alg. 1)

Hai biến thể forward:

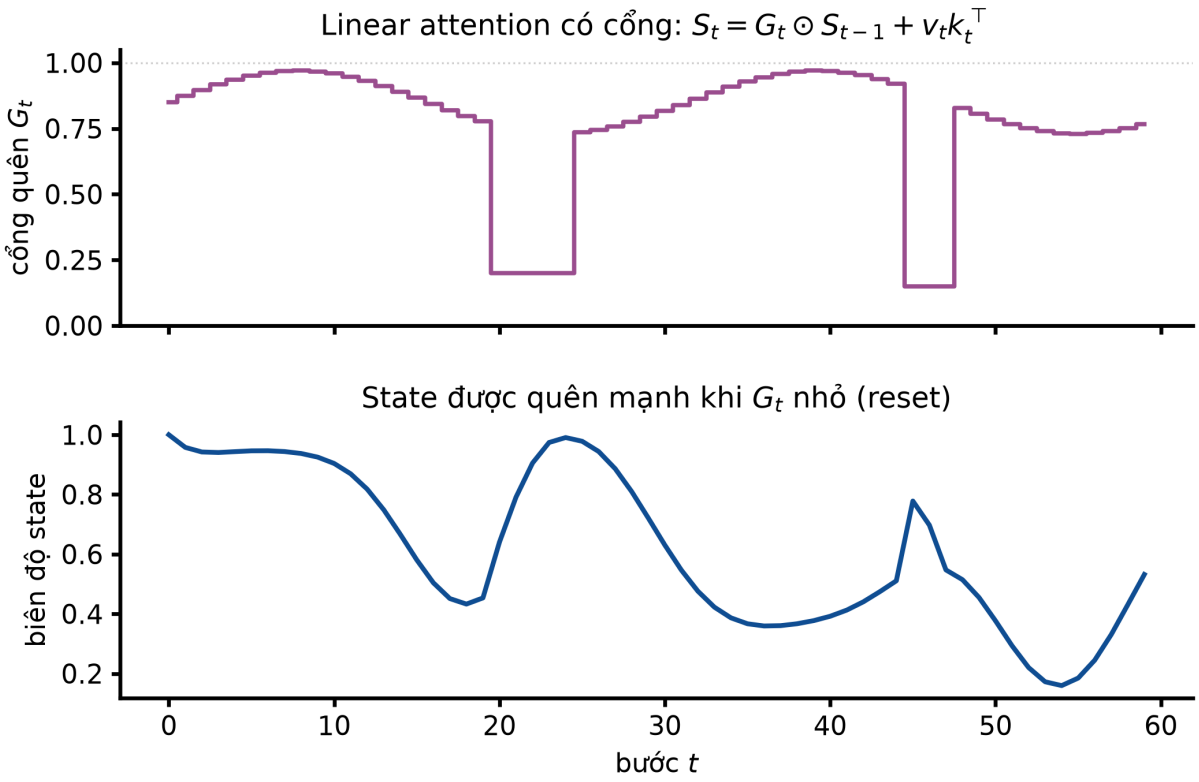
- **Không vật chất hóa (non-materialization):** duyệt các chunk tuần tự, giữ S ở SRAM; song song theo batch/head/chiều head nhưng *không* song song theo chiều chuỗi (đủ khi batch lớn).
- **Vật chất hóa (materialization):** trước tiên chạy hồi quy liên chunk để ghi mọi $S_{[n]}$ vào HBM, rồi tính $O_{[n]}$ song song theo chunk; tăng song song theo chuỗi (hợp khi batch nhỏ, chuỗi dài) với giá +10-20% bộ nhớ, được giảm bằng **recomputation** (bỏ trạng thái sau forward, tính lại ở backward) — là mặc định.

```
# FlashLinearAttention, forward, bản không vật chất hóa (rút gọn Alg.1)
S = 0                                   # nằm trên SRAM
for n in 1..N:                          # N = L / C chunk
    load Q[n], K[n], V[n]  (HBM -> SRAM)
    O[n] = Q[n] @ S + ((Q[n] @ K[n].T) * M) @ V[n]   # inter + intra, toàn matmul
    S    = S + K[n].T @ V[n]
    store O[n]  (SRAM -> HBM)
```

Hình 2: trên H100, batch 32, 16 head, head dim 64, chunk 64, hai biến thể FlashLinearAttention nhanh hơn FlashAttention-2, tuyên bố của paper là “even on short (e.g., 1K) sequences” (giá trị từng điểm không đọc được từ text); bản PyTorch thuần chunkwise chậm hơn rõ — chứng minh giá trị của I/O-awareness.

3.4 Gated Linear Attention: dạng hồi quy (§4.1)

Cổng quên 2D: $G_t \in (0,1)^{d_k \times d_v}$. Dạng tổng quát (Hadamard) $S_t = G_t \odot S_{t-1} + k_t v_t^\top$ bao trùm nhiều mô hình (Bảng 1 dưới). Nhưng tham số hóa G_t đầy đủ tốn $d \cdot d_k \cdot d_v$ tham số. Paper chọn dạng **outer product với vector 1**:



Linear attention có cổng quên G_t: state bị reset khi G nhỏ. (minh hoạ, tự tính)

$$G_t = \alpha_t \mathbf{1}^\top, \quad \alpha_t \in (0,1)^{d_k} \implies \boxed{S_t = \mathrm{Diag}(\alpha_t)\, S_{t-1} + k_t v_t^\top}, \qquad o_t = S_t^\top q_t.$$

Tức mỗi *hàng* của S (ứng với một chiều khóa r) bị nhân cùng một số $\alpha_{t,r}$ — mọi chiều giá trị chia sẻ cùng hệ số quên cho một chiều khóa. Đây là điểm trung gian giữa cổng vô hướng (Mamba-2, mLSTM, gated retention: $G_t = \gamma_t \mathbf{1}\mathbf{1}^\top$) và cổng đầy đủ $\alpha_t \beta_t^\top$ (DFW).

Bảng 1 của paper (một số hàng chính) — cách các mô hình tham số hóa G_t:

Mô hình	G_t	Ghi chú
Mamba	$\exp(-(\Delta_t \mathbf{1}^\top) \odot \exp(A))$, $\Delta_t = \mathrm{softplus}(x_t W_1 W_2)$	$A \in \mathbb{R}^{d_k \times d_v}$ học được; đầy đủ hạng nhưng không viết được dạng matmul
Mamba-2	$\gamma_t \mathbf{1}\mathbf{1}^\top$, $\gamma_t = \exp(-\mathrm{softplus}(x_t W_\gamma)\exp(a))$	vô hướng ⇒ dạng matmul, tensor core
mLSTM	$\gamma_t \mathbf{1}\mathbf{1}^\top$, $\gamma_t = \sigma(x_t W_\gamma)$	vô hướng
Gated Retention	$\gamma_t \mathbf{1}\mathbf{1}^\top$, $\gamma_t = \sigma(x_t W_\gamma)^{1/\tau}$	vô hướng
RWKV-6	$\alpha_t \mathbf{1}^\top$, $\alpha_t = \exp(-\exp(x_t W_\alpha))$	cùng họ với GLA
HGRN-2	$\alpha_t \mathbf{1}^\top$, $\alpha_t = \gamma + (1-\gamma)\sigma(x_t W_\alpha)$	
GLA	$\alpha_t \mathbf{1}^\top$, $\alpha_t = \sigma(x_t W_{\alpha 1} W_{\alpha 2})^{1/\tau}$	$W_{\alpha 1} \in \mathbb{R}^{d \times 16}$, $W_{\alpha 2} \in \mathbb{R}^{16 \times d_k}$

3.5 Dạng song song của GLA (suy dẫn)

Khai triển hồi quy. Với $t \geq i$: ảnh hưởng của $k_i v_i^\top$ lên S_t là $\prod_{j=i+1}^{t} \mathrm{Diag}(\alpha_j)\, k_i v_i^\top$, tức nhân từng chiều r với $\prod_{j=i+1}^{t} \alpha_{j,r}$.

$$S_t = \sum_{i=1}^{t} \Big(\big(\textstyle\prod_{j=i+1}^{t} \alpha_j\big) \odot k_i \Big) v_i^\top.$$

Đặt **tích tích lũy** $b_t := \prod_{j=1}^{t} \alpha_j \in (0,1)^{d_k}$ (nhân từng phần tử). Khi đó $\prod_{j=i+1}^{t} \alpha_j = b_t \oslash b_i$ và

$$o_t = S_t^\top q_t = \sum_{i=1}^{t} v_i \big(q_t \odot b_t\big)^\top \big(k_i \oslash b_i\big) \quad \implies \quad \boxed{O = \Big(\underbrace{\big((Q \odot B)\,(K \oslash B)^\top\big)}_{P} \odot M \Big) V},$$

$B \in (0,1)^{L \times d_k}$ xếp các b_t. Đối chiếu RetNet: nếu $\alpha_t = \gamma \mathbf{1}$ thì $b_t = \gamma^t \mathbf{1}$, và $P_{ti} = \gamma^{t-i} q_t^\top k_i$ — đúng dạng $(QK^\top \odot D)$ của bài 19.

Vấn đề số học. b_t là tích của rất nhiều số < 1 nên cực nhỏ khi t lớn $\Rightarrow K \oslash B$ nổ tung. Cách chữa tạm: tính ở log-space

$$P_{ij} = \sum_{r=1}^{d_k} Q_{ir} K_{jr} \exp(\log B_{ir} - \log B_{jr}), \quad i \geq j,$$

nhưng đây **không còn là matmul chuẩn** nên không dùng được half-precision tensor core.

3.6 Dạng chunkwise của GLA (§4.2, suy dẫn)

Trong chunk thứ c (các vị trí $(c-1)C + 1..cC$), gọi $b_0 := b_{(c-1)C}$ là tích lũy ngay trước chunk, $b_{end} := b_{cC}$ là cuối chunk. Định nghĩa ba đại lượng suy giảm (chỉ phụ thuộc cổng, cục bộ):

$$\Lambda_t = \frac{b_t}{b_0} \text{ (từ đầu chunk đến } t), \qquad \Gamma_t = \frac{b_{end}}{b_t} \text{ (từ } t \text{ đến cuối chunk)}, \qquad \gamma_c = \frac{b_{end}}{b_0} \text{ (cả chunk)}.$$

Đây là các tích của tối đa C cổng nên không nhỏ đến mức tràn số như b_t toàn chuỗi.

Cập nhật trạng thái liên chunk. Token t trong chunk còn phải suy giảm đến cuối chunk bởi $\prod_{j>t}^{cC} \alpha_j = \Gamma_t$; trạng thái cũ bị suy giảm toàn chunk bởi γ_c:

$$\boxed{S_{[c]} = \mathrm{Diag}(\gamma_c)\, S_{[c-1]} + \big(K_{[c]} \odot \Gamma_{[c]}\big)^\top V_{[c]}}$$

(paper: Eq. ở §4.2 với ký hiệu γ_{i+1}, $\Gamma_{[i+1]}$ tương ứng).

Đầu ra inter-chunk. Đóng góp của mọi token *trước chunk* tại vị trí t là $\mathrm{Diag}(\Lambda_t) S_{[c-1]}$, nên

$$O^{\text{inter}}_{[c]} = \big(Q_{[c]} \odot \Lambda_{[c]}\big)\, S_{[c-1]}.$$

Đầu ra intra-chunk. Dùng dạng song song cục bộ: $b_t / b_i = \Lambda_t / \Lambda_i$ (i, t cùng chunk):

$$O^{\text{intra}}_{[c]} = \Big(\big((Q \odot \Lambda)(K \oslash \Lambda)^\top\big) \odot M\Big) V \quad \text{(trong chunk)},$$

với $K \oslash \Lambda$ vẫn có thể bất ổn nếu C lớn và cổng nhỏ — bước tiếp theo xử lý.

Tương đương kiểm chứng: cho $\alpha_t \equiv \gamma$ hằng, $\Lambda_t = \gamma^j$ (j = vị trí trong chunk), $\Gamma_t = \gamma^{C-j}$, $\gamma_c = \gamma^C$ — trùng đúng $\xi_j = \gamma^j$, $\zeta_j = \gamma^{C-j}$ của RetNet (bài 19).

3.7 Hiệu quả phần cứng cho GLA: chunk hai tầng (§4.3)

Tại sao cần: phần intra-chunk có $\exp(\log B_{ir} - \log B_{jr})$ không là matmul. **Secondary-level chunking:** chia mỗi chunk thành sub-chunk. Ma trận P (intra-chunk) chia thành các ô:

- Ô giữa **các sub-chunk khác nhau** (ô cam trong Hình 3): tính bằng matmul nửa chính xác trên tensor core, vì hệ số suy giảm giữa hai sub-chunk có thể tách thành tích Q-phía và K-phía qua các mốc đầu/cuối sub-chunk:
$$P_{[i][j]} = \big(Q_{[i]} \odot \Lambda_{[i]}\big)\Big(K_{[j]} \odot \Gamma_{[j]} \odot \tfrac{b_{iC}}{b_{(j+1)C}}\Big)^\top.$$
- Ô **trong cùng sub-chunk** (ô hồng, đường chéo nhỏ): tính trực tiếp ở log-space, fp32 (ổn định, nhưng không dùng tensor core). Vì chỉ là các khối nhỏ trên đường chéo nên tổng FLOPs không-nửa-chính-xác giảm mạnh.

Hai kỹ thuật khác: (i) **tính** $d \log \alpha_t$ **trong bộ nhớ hằng** — không cần vật chất hóa trạng thái $L \times d \times d$ mọi bước ở backward nhờ công thức đóng $d \log b_t = q_t \odot dq_t - k_t \odot dk_t$, $d \log \alpha_t = \sum_{t \le i \le L} d \log b_i$ (paper, §4.3); (ii) recomputation như trên.

Chi phí tổng quát giống linear attention thường (§3.1) cộng phần đường chéo log-space.

3.8 GLA Transformer (khối đầy đủ, §4.4)

Với H head, mỗi head h:

$$S^h_t = \mathrm{Diag}(\alpha^h_t) S^h_{t-1} + k^h_t (v^h_t)^\top, \qquad o^h_t = (S^h_t)^\top q^h_t, \qquad o'_t = \mathrm{concat}\big(\mathrm{LN}(o^1_t), \ldots, \mathrm{LN}(o^H_t)\big),$$

$$r_t = \text{Swish}(x_t W_r + b_r), \qquad y_t = (r_t \odot o'_t)\, W_O.$$

Cổng quên được tham số hóa **low-rank** và làm "chậm" bằng nhiệt độ $\tau = 16$:

$$\boxed{\alpha_t = \sigma\big(x_t W_{\alpha 1} W_{\alpha 2} + b_\alpha\big)^{1/\tau}}, \quad W_{\alpha 1} \in \mathbb{R}^{d \times 16},\ W_{\alpha 2} \in \mathbb{R}^{16 \times d_k},\ \tau = 16.$$

Lấy lũy thừa $1/\tau$ khiến α_t gần 1 hơn (ví dụ $\sigma = 0.5 \Rightarrow \alpha = 0.5^{1/16} \approx 0.958$), tức *quên chậm theo mặc định*.

Phân bổ tham số: $d_k = d/2$, $d_v = d$ (tổng), 4 head ⇒ mỗi head $d'_k = d/8$, $d'_v = d/4$. Một lớp GLA có xấp xỉ $4d^2$ tham số (như attention softmax): $W_Q, W_K \in \mathbb{R}^{d \times d/2}$, $W_V, W_r, W_O \in \mathbb{R}^{d \times d}$, cộng W_α nhỏ. Khối đầy đủ: $Y = \text{GLA}(\text{LN}(X)) + X$, rồi FFN **SwiGLU** (bài 11) với residual. (Lưu ý: công thức trong paper viết $X^{(l+1)} = \text{SwiGLU}(\text{LN}(Y)) + X$ — đây có vẻ là lỗi in, phải là $+Y$ như residual chuẩn.)

4. Thí nghiệm và kết quả chính

Thiết lập. Dữ liệu: SlimPajama (tập con 100B token), tokenizer Mistral. Hai cỡ: 340M (15B token, batch 0.5M) và 1.3B (100B token, batch 2M). AdamW, LR tối đa 3×10^{-4}, cosine, weight decay 0.01, clip 1.0. Baseline: **Transformer++** (kiến trúc LLaMA: RoPE, SwiGLU, RMSNorm), **RetNet** (thay FFN bằng SwiGLU cho công bằng), **Mamba**; tất cả huấn luyện cùng token, cùng dữ liệu.

4.1 Language modeling và zero-shot (Bảng 2)

Cỡ	Mô hình	Wiki ppl ↓	LMB ppl ↓	LMB acc	PIQA	Hella	Wino	ARC-e	ARC-c	Avg
340M / 15B	Transformer++	28.39	42.69	31.0	63.3	34.0	50.4	44.5	24.2	41.2
	RetNet	32.33	49.19	28.6	63.5	33.5	52.5	44.5	23.4	41.0
	Mamba	28.39	39.66	30.6	65.0	35.4	50.1	46.3	23.6	41.8
	GLA	28.65	43.35	30.3	64.8	34.5	51.4	45.1	22.7	41.5
1.3B / 100B	Transformer++	16.85	13.44	48.9	70.8	49.6	53.6	56.0	26.5	50.9
	RetNet	18.64	17.27	43.3	70.0	47.3	52.5	54.8	25.6	48.9
	Mamba	17.06	13.89	46.2	72.2	40.1	54.1	59.0	28.2	50.0
	GLA	17.22	14.47	46.9	71.8	49.8	53.9	57.2	26.6	**51.0**

(Acc là chuẩn hóa độ dài ở HellaSwag/ARC-c; cột cuối là trung bình các benchmark dùng accuracy.) Kết luận paper: GLA, với cổng phụ thuộc dữ liệu, tốt hơn RetNet trên mọi tác vụ, và ngang Mamba/Transformer++ ở quy mô này.

4.2 Tác vụ cần recall (Bảng 3)

Các tác vụ trích thông tin/đọc hiểu theo Arora et al. 2024 (điểm cao hơn tốt hơn):

Cỡ	Mô hình	FDA	SWDE	SQuAD
340M	Transformer++	21.4	42.2	22.1
	RetNet	2.9	13.3	27.6
	Mamba	2.1	12.4	23.0
	GLA	8.1	18.6	27.2

Cỡ	Mô hình	FDA	SWDE	SQuAD
1.3B	Transformer++	27.4	66.6	31.5
	RetNet	14.3	42.8	34.7
	Mamba	6.2	41.4	35.2
	GLA	19.9	50.6	42.6

Mô hình dưới bậc hai thua Transformer rõ ở FDA, SWDE; GLA là mô hình dưới bậc hai tốt nhất trong nhóm, được cho là nhờ **trạng thái lớn hơn Mamba** và **cơ chế chọn lọc** (so với RetNet). Với tác vụ tổng hợp MQAR (multi-query associative recall; Hình 4), các mô hình trạng thái ma trận (Mamba/RetNet/GLA) vượt Hyena/RWKV-4 và GLA vượt RetNet; attention chuẩn đạt tuyệt đối.

4.3 Ngoại suy độ dài (1.3B, 100B token)

Huấn luyện ở 2K: Transformer không ngoại suy được ngoài độ dài huấn luyện; **Mamba khó ngoại suy quá 4K** (PG19); **GLA/RetNet tổng quát hóa đến ~18K** (SlimPajama test) mà ppl không suy giảm đáng kể. Thử huấn luyện 24K qua truncated BPTT (12 đoạn 2K, truyền trạng thái cuối đoạn trước, không truyền gradient): với GLA chênh lệch ppl rất nhỏ so với huấn luyện trực tiếp, nên TBPTT là cách kinh tế; Mamba hưởng lợi rõ khi huấn luyện 8K và khi đó ngang GLA.

4.4 Hiệu quả huấn luyện (1.3B, H100)

Hình 6: thông lượng và bộ nhớ theo độ dài/batch (2048/8, 4096/4, 8192/2, 16384/1): mọi mô hình có bộ nhớ tuyến tính, chênh lệch bộ nhớ nhỏ; **Mamba tụt sau Transformer++ và GLA về thông lượng, GLA có lợi thế rõ hơn khi độ dài > 4096**. Tuyên bố ở tóm tắt: FlashLinearAttention nhanh hơn FlashAttention-2 ngay ở 1K; GLA có thông lượng cao hơn Mamba cùng cỡ.

5. Phân tích / ablation

Bảng 4 (340M, 7B token, perplexity huấn luyện trung bình 200 bước cuối; thấp hơn tốt hơn):

Biến thể	Train ppl
GLA (4 head)	**14.77**
Không cổng (linear attention thường)	23.21
Cổng độc lập dữ liệu, vô hướng (≈ RetNet)	16.55
Cổng vô hướng phụ thuộc dữ liệu	15.56
8 head (head dim nhỏ)	15.29
1 head (head dim lớn)	14.61

Rút ra (theo paper): (1) **cổng là yếu tố số một** (23.21 → 14.77); (2) *phụ thuộc dữ liệu* tốt hơn *hằng* (16.55 → 15.56); (3) *cổng mịn theo chiều khóa* còn tốt hơn *vô hướng* (15.56 → 14.77) nên cần thiết; (4) head dim lớn hơn tốt hơn nhưng 1 head chỉ nhỉnh hơn một chút (14.61 vs 14.77) với bộ nhớ GPU nhiều hơn rất nhiều, nên chọn 4 head. Thử $G_t = \alpha_t \beta_t^\top$ chỉ cải thiện không đáng kể so với $\alpha_t \mathbf{1}^\top$ (nêu ở chú thích).

6. Giới hạn

Do tác giả nêu:

- Chưa chạy quy mô lớn hơn do hạn chế tài nguyên (chỉ 340M/1.3B, tối đa 100B token); không rõ GLA scale thế nào; tác giả *dự đoán* lợi thế hiệu quả so với Mamba sẽ lớn hơn ở >7B (tensor core, tensor parallelism) — đây là dự đoán, không đo.

- Mamba trong so sánh dùng mã chính thức; Transformer++ dùng FlashAttention-2; có lưu ý so sánh khác biệt cài đặt.
- Hướng tương lai: áp dụng sang các modality khác.

Nhận xét bổ sung:

- Ablation Bảng 4 chỉ ở 340M/7B token; chênh lệch 0.2-1 ppl có thể không giữ ở quy mô lớn.
- Ở tác vụ recall, GLA vẫn thua Transformer++ rõ (FDA 19.9 vs 27.4; SWDE 50.6 vs 66.6 ở 1.3B) — xác nhận hạn chế "trạng thái hữu hạn".
- Cổng chỉ **xóa** thông tin (nhân < 1), không **sửa** nội dung đã lưu cho cùng một khóa. Delta rule (23-24) giải quyết điều này.
- Chunk hai tầng + log-space làm cài đặt phức tạp; cần kernel Triton chuyên dụng (flash-linear-attention).
- Đo thông lượng trên H100; chưa có số liệu cho mọi loại GPU hoặc batch decode.

7. Liên hệ với các bài khác

- **18:** GLA = linear attention + cổng. $\alpha_t \equiv \mathbf{1} \Rightarrow$ bài 18 (không mẫu số).
- **19 RetNet:** $\alpha_t \equiv \gamma\mathbf{1} \Rightarrow$ RetNet; dạng chunkwise của RetNet là trường hợp riêng của §3.6.
- **21 Mamba:** Bảng 1 xếp Mamba vào cùng khung $S_t = G_t \odot S_{t-1} + \ldots$ nhưng G_t đầy đủ hạng, tính bằng scan trong SRAM (không mở rộng trạng thái quá ~16 chiều); GLA dùng trạng thái lớn hơn nhờ matmul/tensor core.
- **22 Mamba-2:** $G_t = \gamma_t \mathbf{1}\mathbf{1}^\top$ vô hướng $\Rightarrow$ dạng matmul thuần; GLA (cổng theo chiều khóa) là tổng quát hơn.
- **23-24 DeltaNet, Gated DeltaNet:** cùng khung chunkwise; thêm delta rule và (ở 24) cổng vô hướng α_t kiểu Mamba-2.
- **26 Kimi Linear:** dùng cổng theo chiều trong khung delta.
- **44-45 FlashAttention:** tư tưởng I/O-awareness và tiling được chuyển sang linear attention.
- **06/17:** hướng sliding-window, bổ sung cho recall chính xác mà GLA yếu.

8. Gợi ý áp dụng

Cổng phụ thuộc dữ liệu cho phép điều chỉnh mức suy giảm của trạng thái theo token. Dạng chunkwise đưa phần lớn công việc về phép nhân ma trận, nhưng hiệu năng còn phụ thuộc cách cài đặt, độ dài chuỗi và độ ổn định số. Cổng theo chiều và cổng vô hướng có khả năng biểu diễn và chi phí khác nhau; nên đối chiếu các ablation thay vì suy ra một lựa chọn luôn tốt hơn.

9. Thí nghiệm gợi ý

So sánh dạng hồi quy với dạng song song và chunkwise trên tensor ngắn. Thử các giá trị cổng nhỏ để quan sát vấn đề ổn định số, rồi so sánh cách tính trực tiếp với phương pháp ổn định hơn được mô tả trong bài.

10. Tóm tắt công thức / khái niệm

Khái niệm	Công thức
Linear attention	$S_t = S_{t-1} + k_t v_t^\top$, $o_t = S_t^\top q_t$
GLA hồi quy	$S_t = \mathrm{Diag}(\alpha_t) S_{t-1} + k_t v_t^\top$
Cổng	$\alpha_t = \sigma(x_t W_{\alpha 1} W_{\alpha 2} + b)^{1/16}$, $W_{\alpha 1} \in \mathbb{R}^{d \times 16}$
Tích tích lũy	$b_t = \prod_{j \le t} \alpha_j$
GLA song song	$O = \big(((Q \odot B)(K \oslash B)^\top) \odot M\big) V$
Log-space	$P_{ij} = \sum_r Q_{ir} K_{jr} e^{\log B_{ir} - \log B_{jr}}$
Chunk decay	$\Lambda_t = b_t / b_0$, $\Gamma_t = b_{end} / b_t$, $\gamma_c = b_{end} / b_0$

Khái niệm	Công thức
Trạng thái liên chunk	$S_{[c]} = \text{Diag}(\gamma_c) S_{[c-1]} + (K \odot \Gamma)^\top V$
Đầu ra chunk	$O = (Q \odot \Lambda) S_{[c-1]} + \text{intra}$
Chi phí	$O(LCd + Ld^2)$; $C = L$ song song, $C = 1$ hồi quy
Chunk hai tầng	ô khác sub-chunk: matmul nửa chính xác; ô đường chéo: log-space fp32
Block	$y = (r \odot \text{LN-heads}(o))W_O$, $r = \text{Swish}(xW_r)$; $d_k = d/2$, $d_v = d$, 4 head

11. Kết luận ngắn

GLA nâng linear attention lên mức cạnh tranh bằng một cổng quên theo chiều khóa phụ thuộc dữ liệu, và quan trọng không kém là **chỉ ra cách huấn luyện nhanh trên GPU**: chunkwise có vật chất hóa + recomputation, chunk hai tầng, log-space cục bộ. Cấu trúc $S_{[c]} = \text{Diag}(\gamma_c) S_{[c-1]} + (K \odot \Gamma)^\top V$ và $O = (Q \odot \Lambda) S_{[c-1]} + \text{intra}$ chính là khung mà DeltaNet và Gated DeltaNet (bài 23-24) kế thừa, là nền để đọc các biến thể hồi quy ở những bài sau.

Nguồn gốc: https://arxiv.org/abs/2312.06635

Bài 21

Mamba: Linear-Time Sequence Modeling with Selective State Spaces (2023)

Tác giả: Albert Gu, Tri Dao (Carnegie Mellon University, Princeton University)
Venue / arXiv: arXiv:2312.00752 (v2, 5/2024); paper không nêu venue trong bản text đã đọc
Nguồn: https://arxiv.org/abs/2312.00752
Chủ đề: Selective state space model (S6): tham số SSM phụ thuộc đầu vào; rời rạc hóa (ZOH); thuật toán scan nhận biết phần cứng; khối Mamba đơn giản hóa; so sánh với Transformer++.
Vị trí trong lộ trình: Đọc sau 18-20 (linear attention, RetNet, GLA). Mamba đến từ nhánh **SSM** (không phải attention) nhưng hội tụ về cùng cấu trúc: trạng thái ẩn + cập nhật hồi quy tuyến tính + cổng phụ thuộc đầu vào. Bài sau: 22 (Mamba-2/SSD) chứng minh sự "đối ngẫu" giữa SSM chọn lọc và linear attention có cổng; 23-24 (DeltaNet, Gated DeltaNet) lấy cổng kiểu Mamba-2 và thêm delta rule; 25 (Jamba) trộn Mamba với attention.

> Ghi chú diễn giải, không phải bản dịch. Bài báo dài, có các phụ lục về DNA và audio; phần dưới tập trung vào cơ chế chọn lọc, rời rạc hoá, scan và các kết quả ngôn ngữ.

1. Bối cảnh

Điểm xuất phát. Các mô hình nền tảng hầu như đều dựa trên Transformer (bài 01), có chi phí bậc hai theo độ dài. Nhiều biến thể dưới bậc hai (linear attention, tích chập có cổng, RNN, SSM có cấu trúc "S4") đã được đề xuất, nhưng **chưa vượt attention trên dữ liệu rời rạc, dày thông tin như văn bản**. Nhóm SSM (S4, S4D, H3, Hyena...) mạnh ở tín hiệu liên tục (audio, thị giác) nhưng yếu ở ngôn ngữ.

Chẩn đoán của paper. Điểm yếu then chốt là thiếu **suy luận theo nội dung (content-based reasoning)**: mọi SSM có cấu trúc trước đó đều **bất biến theo thời gian (LTI)** — các tham số $(\bar{A}, \bar{B}, C)$ là hằng ở mọi bước, vì LTI cho phép tính bằng tích chập toàn cục (FFT) hiệu quả. Hệ quả: không thể *chọn* thông tin nào vào trạng thái, bất kể token là gì.

Khái niệm nền.

- *State space model (SSM):* hệ động lực tuyến tính $h'(t) = Ah(t) + Bx(t)$, $y(t) = Ch(t)$, ánh xạ tín hiệu 1 chiều $x(t)$ qua trạng thái ẩn $h(t) \in \mathbb{R}^N$ sang $y(t)$. Ở đây chữ "SSM" chỉ họ S4 (theo lời paper), không phải mọi mô hình có trạng thái.
- *Rời rạc hóa (discretization):* biến hệ liên tục thành bước rời rạc với bước thời gian Δ.
- *LTI (linear time-invariant):* tham số không đổi theo t. Ý nghĩa tính toán: hồi quy $\Leftrightarrow$ tích chập.
- *Parallel (associative) scan:* thuật toán tính tổng tích lũy / hồi quy tuyến tính bằng cây phép toán kết hợp, độ sâu $O(\log L)$.
- *Kernel fusion, HBM vs SRAM:* như bài 44 và 20.

Hai tác vụ tổng hợp minh họa điểm yếu của LTI:

- *Selective Copying:* sao chép các token "màu" (có nghĩa) nằm ở vị trí ngẫu nhiên, bỏ qua token nhiễu. LTI chỉ biết "đếm vị trí", không phân biệt nội dung $\Rightarrow$ thất bại.
- *Induction Heads:* thấy bigram "Harry Potter" một lần, lần sau gặp "Harry" thì dự đoán "Potter" (liên hệ bài Transformer Circuits trong lộ trình; paper gọi là predictive cho in-context learning).

2. Ý tưởng chính

Nguyên lý do paper đưa ra: **sequence modeling là bài toán nén ngữ cảnh vào trạng thái nhỏ**. Attention hiệu quả vì *không nén* (KV cache giữ hết) nhưng đắt; mô hình hồi quy rẻ vì trạng thái cố định, nhưng chỉ tốt nếu trạng thái giữ đúng thứ cần giữ. Từ đó: cho phép **chọn lọc (selectivity)** — các tham số điều khiển tương tác theo chiều chuỗi (Δ, B, C) trở thành **hàm của đầu vào** x_t. Mô hình khi đó có thể "đóng cửa" với token vô ích (Δ nhỏ $\Rightarrow$ giữ trạng thái, bỏ qua đầu vào) hoặc "đặt lại" (Δ lớn $\Rightarrow$ quên trạng thái, tập trung vào token hiện tại). Cái giá là mất tính LTI $\Rightarrow$ mất dạng tích chập; paper bù bằng một **thuật toán scan nhận biết phần cứng** (kernel fusion, parallel scan, recomputation) tính hồi quy trực tiếp mà không vật chất hóa trạng thái mở rộng ở HBM. Cuối cùng gộp khối SSM với khối MLP thành một **khối Mamba** đồng nhất.

3. Phương pháp chi tiết

3.1 SSM có cấu trúc (S4) — kiến thức nền (§2)

Hệ liên tục:

$$h'(t) = Ah(t) + Bx(t), \qquad y(t) = Ch(t). \tag{1}$$

Tham số (Δ, A, B, C). **Rời rạc hóa** theo quy tắc zero-order hold (ZOH):

$$\boxed{\bar{A} = \exp(\Delta A), \qquad \bar{B} = (\Delta A)^{-1}\big(\exp(\Delta A) - I\big) \cdot \Delta B} \tag{4}$$

Suy dẫn ZOH. Giữ x không đổi trên $[t, t+\Delta)$. Nghiệm của hệ tuyến tính: $h(t+\Delta) = e^{\Delta A}h(t) + \int_0^{\Delta} e^{(\Delta - s)A} B\, x\, ds$. Với x hằng, tích phân $= A^{-1}(e^{\Delta A} - I)Bx$. Vì $(\Delta A)^{-1}(e^{\Delta A} - I)\Delta B = A^{-1}(e^{\Delta A} - I)B$, nên đúng công thức $\bar{B}$ ở trên. Dạng rời rạc:

$$h_t = \bar{A}h_{t-1} + \bar{B}x_t, \qquad y_t = Ch_t. \tag{2}$$

Paper nhấn mạnh về mặt cơ học, rời rạc hóa chỉ là bước đầu tiên của đồ thị tính toán; cũng có thể tham số hóa trực tiếp $(\bar{A}, \bar{B})$.

Tính theo hai cách (khi LTI): (i) hồi quy (2), tuần tự, tốt cho suy luận; (ii) tích chập: khai triển (2) $\Rightarrow$ $y = x * \bar{K}$ với hạt nhân $\bar{K} = (C\bar{B},\, C\bar{A}\bar{B}, \ldots, C\bar{A}^k\bar{B}, \ldots)$ (Eq. 3), tính bằng FFT, song song, tốt cho huấn luyện.

Cấu trúc: A chéo hóa $\Rightarrow$ $A \in \mathbb{R}^{N\times N}$ biểu diễn bằng N số; $B \in \mathbb{R}^{N\times 1}$, $C \in \mathbb{R}^{1\times N}$. Với chuỗi x có batch B_{sz}, độ dài L, D kênh, SSM chạy **độc lập trên từng kênh**, nên tổng trạng thái là $D \cdot N$ mỗi mẫu và tính toán $O(B_{sz}LDN)$ thời gian/bộ nhớ nếu vật chất hóa — "nút cổ chai hiệu quả" gốc.

3.2 Chọn lọc: biến tham số thành hàm của đầu vào (§3.2, Alg. 2)

So sánh hai thuật toán của paper:

	Alg.1 SSM (S4)	Alg.2 SSM + Selection (S6)
B	(D, N) tham số	$(B_{sz}, L, N) \leftarrow s_B(x) = \text{Linear}_N(x)$
C	(D, N) tham số	$(B_{sz}, L, N) \leftarrow s_C(x) = \text{Linear}_N(x)$
Δ	$(D) \leftarrow \tau_\Delta(\text{Param})$	$(B_{sz}, L, D) \leftarrow \tau_\Delta(\text{Param} + s_\Delta(x))$, $s_\Delta(x) = \text{Broadcast}_D(\text{Linear}_1(x))$, $\tau_\Delta = \text{softplus}$
A	(D, N) tham số	vẫn là tham số, không phụ thuộc x
Rời rạc hóa	(D, N)	(B_{sz}, L, D, N)

	Alg.1 SSM (S4)	Alg.2 SSM + Selection (S6)
Tính	hồi quy hoặc tích chập	**chỉ hồi quy (scan)**

Hậu quả: $\bar{A}_t = \exp(\Delta_t A)$, $\bar{B}_t$ **khác nhau ở mỗi bước** $\Rightarrow$ không còn tích chập. Rời rạc hóa theo từng bước t:

$$\bar{A}_t = \exp(\Delta_t A), \quad \bar{B}_t = (\Delta_t A)^{-1}(\exp(\Delta_t A) - I)\,\Delta_t B_t, \quad h_t = \bar{A}_t h_{t-1} + \bar{B}_t x_t, \quad y_t = C_t h_t.$$

Với A chéo, mọi phép toán theo từng phần tử: $\bar{A}_t[d,n] = \exp(\Delta_t[d]\,A[d,n])$ — **hệ số quên khác nhau theo kênh** d **và theo chiều trạng thái** n, đồng thời phụ thuộc token qua $\Delta_t[d]$.

Chọn Δ rộng ra bằng $\text{Broadcast}_D(\text{Linear}_1(x))$ vì nếu một token cần bị bỏ qua thì **mọi kênh** nên bỏ qua; có thể tổng quát hóa thành $\text{Linear}_D(\text{Linear}_R(x))$ (hạng thấp R, §3.6). Paper dùng giá trị thực (không phức) và khởi tạo $A_n = -(n+1)$ (S4D-Real), Δ khởi tạo bởi $\tau_\Delta^{-1}(\text{Uniform}[0.001, 0.1])$.

3.3 Vì sao Δ là cổng RNN (Định lý 1, §3.5.1)

Nếu $N = 1$, $A = -1$, $B = 1$, $s_\Delta = \text{Linear}(x)$, $\tau_\Delta = \text{softplus}$ thì bản hồi quy S6 trở thành

$$g_t = \sigma(\text{Linear}(x_t)), \qquad h_t = (1 - g_t)h_{t-1} + g_t x_t.$$

Chứng minh (phụ lục C, viết lại). $\Delta_t = \text{softplus}(z_t)$ với $z_t = \text{Linear}(x_t)$ (Param gộp vào bias). Khi đó $\bar{A}_t = \exp(-\Delta_t) = \exp(-\text{softplus}(z_t)) = \frac{1}{1+e^{z_t}} = 1 - \sigma(z_t)$. Và $\bar{B}_t = (\Delta A)^{-1}(e^{\Delta A} - 1)\Delta B = -(e^{-\Delta_t} - 1) = 1 - \bar{A}_t = \sigma(z_t) = g_t$. ∎

Ý nghĩa: **Δ tổng quát hóa cổng RNN**; $g_t \to 0$ giữ trạng thái và bỏ qua x_t; $g_t \to 1$ đặt lại trạng thái bằng x_t. Paper xem rời rạc hóa là nền tảng có nguyên lý cho các cổng "heuristic".

Ba hệ quả cơ học (§3.5.2): **variable spacing** (lọc nhiễu giữa các token), **filtering context** (có thể reset để bỏ ngữ cảnh vô ích $\Rightarrow$ hiệu năng lý thuyết đơn điệu theo độ dài ngữ cảnh), **boundary resetting** (khi ghép nhiều tài liệu, reset ở ranh giới; $\Delta \to \infty$).

Về vai trò từng tham số: Δ quyết định cân bằng "tập trung hay bỏ qua đầu vào hiện tại"; A chỉ tác động qua $\exp(\Delta A)$ nên chọn lọc ở Δ là đủ (giả thuyết chưa thử); B, C chọn lọc cho phép điều chỉnh *cái gì vào trạng thái* (B) và *cái gì đọc ra* (C).

3.4 Dạng mở rộng: mối liên hệ với linear attention và GLA (suy dẫn của ghi chú)

Khai triển hồi quy chọn lọc cho một kênh d và chiều trạng thái n:

$$h_t[d,n] = \sum_{i \le t} \Big(\prod_{j=i+1}^{t} \bar{A}_j[d,n] \Big) \bar{B}_i[d,n]\, x_i[d], \qquad y_t[d] = \sum_n C_t[n]\, h_t[d,n].$$

Vì $\prod_{j=i+1}^{t} \exp(\Delta_j[d]A[d,n]) = \exp\big(A[d,n](\sigma_t[d] - \sigma_i[d])\big)$ với $\sigma_t[d] = \sum_{j \le t} \Delta_j[d]$, nên suy giảm là hàm của **hiệu cumsum của Δ**. Nếu xấp xỉ $\bar{B}_t \approx \Delta_t B_t$ (xem ghi chú dưới), có thể đọc

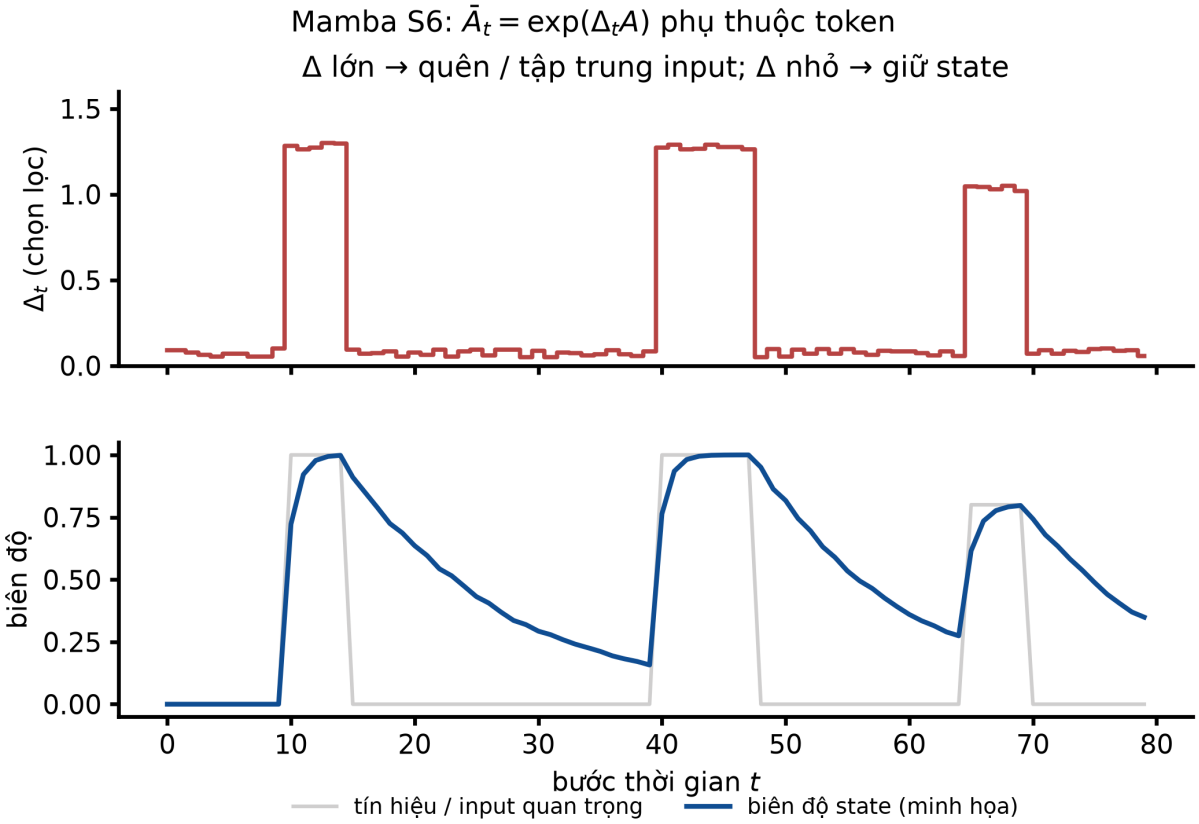
$$S_t = G_t \odot S_{t-1} + B_t\big(\Delta_t \odot x_t\big)^\top, \qquad y_t = S_t^\top C_t, \qquad S_t \in \mathbb{R}^{N \times D},\ (G_t)_{n,d} = e^{\Delta_t[d]A[d,n]},$$

tức **linear attention với "khóa"** $B_t \in \mathbb{R}^N$**, "truy vấn"** C_t**, "giá trị"** $\Delta_t \odot x_t \in \mathbb{R}^D$**, và cổng quên** G_t **phụ thuộc dữ liệu** — đúng dạng G_t trong Bảng 1 của GLA (bài 20: $\exp(-(\Delta_t \mathbf{1}^\top) \odot \exp(A))$, trong đó GLA viết tham số âm dưới dạng $-\exp(A)$). Khác biệt then chốt: Mamba có $d_k = N$ rất nhỏ (16) nhưng $d_v = D$ lớn,

trạng thái *mở rộng* N lần so với đầu vào x. [Ghi chú: $\bar{B}_t \approx \Delta_t B_t$ là xấp xỉ bậc nhất; bản chính xác ZOH có thêm thừa số $(e^{\Delta A} - 1)/(\Delta A)$. Chi tiết cài đặt này không được xác nhận từ phần văn bản bài báo được trích ở đây.]

3.5 Scan: tính hồi quy chọn lọc hiệu quả (§3.3, phụ lục D)

Hồi quy $h_t = a_t h_{t-1} + b_t$ (từng phần tử) có thể tính song song nhờ **phép toán kết hợp**. Biểu diễn mỗi bước là hàm affine $(a, b) : h \mapsto ah + b$. Hợp hai bước (áp dụng (a_1, b_1) trước, rồi (a_2, b_2)):



Mamba S6: Δ_t phụ thuộc input; Δ lớn quên/reset, Δ nhỏ giữ state. (minh hoạ, tự tính)

$$(a_2, b_2) \circ (a_1, b_1) = (a_2 a_1,\ a_2 b_1 + b_2).$$

Phép $\circ$ có tính kết hợp (hợp hàm), nên có thể dùng cây nhị phân/Blelloch scan: **công việc** $O(L)$**, độ sâu** $O(\log L)$.

Ví dụ số. $a = (0.5, 0.5, 0.5, 0.5)$, $b = (1, 2, 3, 4)$, $h_0 = 0$. Tuần tự: $h = 1,\ 2.5,\ 4.25,\ 6.125$. Scan: hợp $(1, 2) \to (0.25,\ 0.5 \cdot 1 + 2 = 2.5)$ (đúng $h_2 = 2.5$); hợp $(3, 4) \to (0.25,\ 0.5 \cdot 3 + 4 = 5.5)$; hợp hai cặp: $(0.25 \cdot 0.25,\ 0.25 \cdot 2.5 + 5.5) = (0.0625,\ 6.125) \Rightarrow h_4 = 6.125$ ✓.

Vấn đề phần cứng. Cách ngây thơ chuẩn bị $\bar{A}, \bar{B}$ kích thước (B_{sz}, L, D, N) ở HBM rồi scan: I/O cỡ $O(B_{sz} LDN)$. Mà phần lớn phép toán (trừ matmul) bị giới hạn bởi băng thông. Giải pháp (ba kỹ thuật cổ điển):

1. **Kernel fusion:** đọc (Δ, A, B, C) kích thước $O(B_{sz} LD + DN)$ từ HBM vào SRAM; rời rạc hóa và scan **ngay trong SRAM**; nhân với C và chỉ ghi y kích thước (B_{sz}, L, D) về HBM $\Rightarrow$ giảm I/O $O(N)$ lần, nhanh hơn 20-40× so với scan PyTorch chuẩn.

2. **Parallel scan** để tránh tuần tự theo L (và chia chuỗi dài thành các đoạn vừa SRAM, tiếp tục scan với trạng thái trung gian).
3. **Recomputation:** không lưu trạng thái trung gian (B_{sz}, L, D, N) cho backward, tính lại khi cần, nên bộ nhớ ngang Transformer dùng FlashAttention.

Hạn chế quan trọng (xem GLA, bài 20): vì trạng thái mở rộng phải nằm gọn trong SRAM, N bị giới hạn (thực tế ≤ 16); scan elementwise không dùng tensor core (bài 22 sửa điều này).

Pseudo-code lớp S6 (hồi quy tuần tự; kernel thật là fused + scan):

```
def selective_ssm(x, A, W_B, W_C, W_dt, dt_bias):     # x: [L, D]; A: [D, N]
    B  = x @ W_B                                       # [L, N]
    C  = x @ W_C                                       # [L, N]
    dt = softplus(dt_bias + broadcast_D(x @ W_dt))     # [L, D]   (Linear_1 rồi
    ↪  broadcast)
    h  = zeros(D, N)
    ys = []
    for t in range(L):                                 # trong thực tế: scan song song,
    ↪  fused, ở SRAM
        Abar = exp(dt[t][:,None] * A)                  # [D, N]
        Bbar = ((Abar - 1) / A) * B[t][None,:]         # ZOH, từng phần tử vì A chéo
        h    = Abar * h + Bbar * x[t][:,None]
        ys.append((h * C[t][None,:]).sum(-1))          # [D]
    return stack(ys)
```

3.6 Khối Mamba (§3.4, Hình 3)

Kết hợp khối H3 (SSM kẹp giữa hai cổng nhân) và khối MLP có cổng (Gated MLP) thành **một khối đồng nhất, lặp lại**:

- Chiếu tuyến tính mở rộng $D \to ED$ (cố định $E = 2$) thành hai nhánh.
- Nhánh chính: tích chập 1 chiều ngắn (xem Hình 3, "Conv") → kích hoạt SiLU/Swish → **SSM chọn lọc**.
- Nhánh cổng: kích hoạt SiLU.
- Nhân hai nhánh (như SwiGLU) → chiếu tuyến tính ra D.
- Xen kẽ với chuẩn hóa (tùy chọn thêm một lớp LayerNorm, lấy cảm hứng từ RetNet) và residual.

Tham số: phần lớn nằm ở các phép chiếu: $3ED^2$ ($2ED^2$ vào, ED^2 ra); phần SSM (Δ, B, C, A) nhỏ hơn nhiều. Với $E = 2$: $6D^2$/khối; **hai khối** Mamba cho $12D^2$, khớp một cặp MHA + MLP của Transformer ($4D^2 + 8D^2$). Khối không có attention và không có MLP riêng.

[Chiều rộng cụ thể của tích chập không được nêu trong text trích; cần đối chiếu mã nguồn trước khi dựa vào kích thước kernel.]

3.7 Chi phí và so sánh định tính

Đại lượng	Transformer (KV cache)	Mamba
Huấn luyện	$O(L^2 D)$	$O(LDN)$ FLOPs, scan fused
Mỗi bước sinh	$O(t)$	$O(DN)$, hằng số
Bộ nhớ khi sinh	KV tăng theo t	trạng thái $ED \cdot N$ mỗi khối, không đổi
Dùng tensor core	có	gần như không (scan elementwise)

Về FLOPs (§3.3.2 của paper): hồi quy ngây thơ tốn $O(B_{sz} LDN)$, tích chập tốn $O(B_{sz} LD \log L)$, và hồi quy có hằng số thấp hơn; vì vậy với chuỗi dài và N không quá lớn, hồi quy thực sự có thể dùng ít FLOPs hơn.

Vấn đề của hồi quy là tuần tự và bộ nhớ, không phải FLOPs.

4. Thí nghiệm và kết quả chính

4.1 Tác vụ tổng hợp

Selective Copying (Bảng 1, accuracy %):

Kiến trúc ngoài	Lớp SSM trong	Acc
không cổng	S4	18.3
không cổng	**S6**	97.0
H3	S4	57.0
H3	Hyena	30.1
H3	**S6**	99.7
Mamba	S4	56.4
Mamba	Hyena	28.4
Mamba	**S6**	**99.8**

Cổng kiến trúc (gating nhân) giúp một phần; **chọn lọc (S4→S6)** giải quyết gần hoàn toàn. Paper lưu ý cổng kiến trúc không tác động dọc trục chuỗi nên không thay được chọn lọc.

Induction Heads: huấn luyện ở độ dài 256 (vocab 16, 2 lớp), thử từ 2^6 đến $2^{20} = 1{,}048{,}576$. Mamba giải **hoàn hảo** và tổng quát hóa tới 1M token, "4000× longer than it saw during training, while no other method goes beyond 2×". Các mô hình attention chỉ thử đến 16384 do bộ nhớ.

4.2 Language modeling

Huấn luyện trên **The Pile**, công thức theo GPT-3, kích thước ~125M đến ~1.3B (scaling law kiểu Chinchilla; Hình 4). Kết luận của paper: Mamba là **mô hình không-attention đầu tiên khớp được Transformer++** (công thức PaLM/LLaMA: RoPE, SwiGLU, RMSNorm...), nhất là khi độ dài chuỗi tăng; RWKV/RetNet thiếu kết quả ở 8K do cài đặt không hiệu quả. Giá trị từng điểm trong Hình 4 không đọc được từ text.

Zero-shot (Bảng 3, một số dòng; Pile ppl ↓, LAMBADA ppl ↓, accuracy ↑, Average ↑):

Mô hình	Pile ppl	LMB ppl	LMB acc	HellaSwag	PIQA	Arc-E	Arc-C	Wino	**Avg**
Pythia-160M	29.64	38.10	33.0	30.2	61.4	43.2	24.1	51.9	40.6
Mamba-130M	10.56	16.07	44.3	35.3	64.5	48.0	24.3	51.9	**44.7**
Pythia-410M	9.95	10.84	51.4	40.6	66.9	52.1	24.6	53.8	48.2
Mamba-370M	8.28	8.14	55.6	46.5	69.5	55.1	28.0	55.3	**50.0**
Pythia-1B	7.82	7.92	56.1	47.2	70.7	57.0	27.1	53.5	51.9
Mamba-790M	7.33	6.02	62.7	55.1	72.1	61.2	29.5	56.1	**57.1**
Pythia-1.4B	7.51	6.08	61.7	52.1	71.0	60.5	28.5	57.2	55.2
RWKV-1.5B	7.70	7.04	56.4	52.5	72.4	60.5	29.4	54.6	54.3

Mô hình	Pile ppl	LMB ppl	LMB acc	HellaSwag	PIQA	Arc-E	Arc-C	Wino	**Avg**
Mamba-1.4B	6.80	5.04	64.9	59.1	74.2	65.5	32.8	61.5	**59.7**
Pythia-2.8B	6.73	5.04	64.7	59.3	74.0	64.1	32.9	59.7	59.1
Pythia-6.9B	6.51	4.45	67.1	64.0	75.2	67.3	35.5	61.3	61.7
GPT-J-6B	–	4.10	68.3	66.3	75.4	67.0	36.6	64.1	63.0
RWKV-7.4B	6.31	4.38	67.2	65.5	76.1	67.8	37.5	61.0	62.5
Mamba-2.8B	6.22	4.23	69.2	66.1	75.2	69.7	36.3	63.5	**63.3**

(Mamba và Pythia cùng tokenizer NeoX, cùng dữ liệu và độ dài huấn luyện 300B token; Mamba/Pythia dùng ngữ cảnh 2048, RWKV 1024.) Paper kết luận: Mamba tốt nhất trong mỗi cỡ và thường ngang mô hình gấp đôi kích thước (Mamba-3B so Pythia-7B).

4.3 DNA và audio (tóm tắt)

- **DNA (HG38, HyenaDNA setup):** Mamba scale tốt hơn HyenaDNA và Transformer++; ở ~40M tham số, khớp Transformer++/HyenaDNA với ít hơn 3-4× tham số. Perplexity **cải thiện khi ngữ cảnh tăng đến 1M**, trong khi HyenaDNA xấu đi (giải thích: LTI không thể lọc thông tin nhiễu). Tác vụ phân loại 5 loài linh trưởng: Mamba tốt hơn khi dùng ngữ cảnh dài (số liệu ở Bảng 13, không trích).
- **Audio (YouTubeMix, SC09):** thay S4+MLP của SaShiMi bằng Mamba; Mamba tốt hơn và cải thiện đến độ dài ~1M mẫu; trên SC09, mô hình Mamba 6.1M tham số đạt FID 0.94, mô hình 24.3M đạt 0.67 (SaShiMi 5.8M: 1.99; DiffWave+SaShiMi 23.0M: 1.42). Đây là thí nghiệm duy nhất dùng tham số hóa **phức**.

4.4 Tốc độ và bộ nhớ (§4.5)

- Scan chọn lọc ($N = 16$, $D = 1024$, batch 1, A100-80GB) **nhanh hơn FlashAttention-2 khi độ dài > 2K**; nhanh hơn scan PyTorch chuẩn 20-40×; phụ lục nêu nhanh hơn attention tới 7× ở độ dài 32K.
- Suy luận: Mamba cho throughput cao hơn Transformer cùng cỡ **4-5×** (prompt 2048, sinh 128 token, batch 1 đến 128, A100); lý do: không có KV cache nên batch lớn hơn nhiều. Mamba-6.9B (chưa huấn luyện) sẽ có throughput cao hơn Transformer-1.3B nhỏ hơn 5×.
- Bộ nhớ huấn luyện 125M, độ dài 2048 (Bảng 15, GB):

Batch	1	2	4	8	16	32
Transformer + FlashAttention-2	4.6	5.2	6.9	11.5	20.7	34.5
Mamba	4.8	5.8	7.3	12.3	23.1	38.2

Tức bộ nhớ huấn luyện của Mamba **tương đương** (hơi cao hơn) Transformer tối ưu.

5. Phân tích / ablation

Kiến trúc và lớp SSM (Bảng 6, perplexity ~350M ở số token Chinchilla):

Kiến trúc	Lớp SSM	PPL
H3	Hyena	10.24
H3	S4 (complex)	10.30
H3	S4 (real)	10.34
H3	**S6**	8.95
Mamba	Hyena	10.75
Mamba	S4 (complex)	10.54
Mamba	S4 (real)	10.56
Mamba	**S6**	**8.69**

Các SSM LTI (S4 phức/thực, Hyena) gần như tương đương nhau; **đổi sang S6 giảm mạnh ~1.5 PPL**; khối Mamba ≈ H3 khi LTI, hơi tốt hơn khi chọn lọc.

Tham số nào nên chọn lọc (Bảng 7):

Δ	B	C	PPL
✗	✗	✗	10.93
✗	✓	✗	10.15
✗	✗	✓	9.98
✓	✗	✗	9.81
✓	✓	✓	**8.71**

Δ quan trọng nhất (do liên hệ cổng RNN, Định lý 1), nhưng dùng cả ba **cộng hưởng**.

Khởi tạo A (Bảng 8): $A_n = -\frac{1}{2} + ni$ (phức) 9.16; $A_n = -\frac{1}{2}$ (thực) 8.85; $A_n = -(n+1)$ (S4D-Real) **8.71**; ngẫu nhiên $\exp(\mathcal{N}(0,1))$ 8.71. Khi SSM chọn lọc, khởi tạo phức chuẩn của S4 lại **kém hơn** thực/ngẫu nhiên trên ngôn ngữ.

Kích thước chiều Δ (Bảng 9): không có phép chiếu 9.12; chiếu xuống 1 chiều rồi broadcast 8.97; tăng dần đến 64 chiều: 8.71 (tham số 358.9M → 371.5M).

Chiều trạng thái N (Bảng 10):

N	B, C hằng	B, C chọn lọc
1	9.88	9.73
2	9.86	9.40
4	9.82	9.09
8	9.82	8.84
16	9.81	**8.71**

(tham số 367.1M → 371.5M). Tăng N chỉ có ích **khi B, C chọn lọc** — cải thiện hơn 1.0 PPL với ~1% tham số thêm. Đây là bằng chứng thực nghiệm cho nguyên lý "mở rộng trạng thái + chọn lọc".

6. Giới hạn

Do tác giả nêu (§5):

- **Không có bữa trưa miễn phí (continuous-discrete):** chọn lọc giúp dữ liệu rời rạc (text, DNA) nhưng có thể làm giảm hiệu năng trên dữ liệu mà SSM LTI vốn mạnh (audio liên tục) — thí nghiệm audio dùng tham số phức để bù.
- **Downstream affordances:** chưa rõ Mamba có các thuộc tính hệ sinh thái của Transformer như fine-tuning, prompting, in-context learning, instruction tuning, RLHF, lượng tử hóa không.
- **Quy mô:** thực nghiệm dừng ở dưới 3B; chưa so ở 7B+ như LLaMA/RWKV/RetNet; có thể cần thêm điều chỉnh kỹ thuật khi scale.

Nhận xét bổ sung:

- Chất lượng **recall** (tra cứu chính xác trong ngữ cảnh) của mô hình trạng thái nhỏ kém Transformer; bài 20 đo trực tiếp (Mamba-1.3B: FDA 6.2 vs Transformer++ 27.4; SWDE 41.4 vs 66.6). Bài 21 không đánh giá các benchmark loại này.
- So sánh với Transformer++ ở quy mô ≤1.3B và độ dài ≤8K.
- Cài đặt phụ thuộc kernel CUDA tùy biến; scan elementwise không tận dụng tensor core, nên khi huấn luyện ở quy mô lớn có thể kém GLA/Mamba-2 (GLA nêu trong bài 20; bài 22 giải quyết).
- Ngoại suy độ dài: GLA báo cáo Mamba khó ngoại suy quá 4K (bài 20), trái với hình ảnh "tổng quát hóa 1M" ở tác vụ tổng hợp — hai thiết lập khác nhau.
- Không có kết quả với dữ liệu đa phương thức ảnh-văn bản.

7. Liên hệ với các bài khác

- **18/19/20:** cùng gia đình hồi quy tuyến tính với trạng thái mở rộng. GLA Bảng 1 xếp Mamba là một trường hợp của $S_t = G_t \odot S_{t-1} + \ldots$. RetNet theo Phụ lục B là SSM đặc biệt $N = 1$ có head dim lớn thay mở rộng trạng thái.
- **22 Mamba-2/SSD:** làm G_t vô hướng, nhờ đó có dạng matmul và chunkwise; chứng minh đối ngẫu SSM–attention.
- **23/24 DeltaNet, Gated DeltaNet:** lấy cổng suy giảm vô hướng kiểu Mamba-2 và thêm delta rule.
- **25 Jamba:** hybrid Mamba + attention (+MoE); động lực: Mamba thiếu recall chính xác.
- **12 RoPE/13 ALiBi:** Mamba không cần mã hóa vị trí tường minh (thứ tự nằm sẵn trong hồi quy).
- **44-45 FlashAttention:** tư tưởng IO-aware (SRAM/HBM, recomputation) là cùng gốc với fused selective scan.
- **Transformer Circuits (Induction Heads):** tác vụ induction head dùng làm bài kiểm tra.

8. Gợi ý áp dụng

Mamba dùng tham số phụ thuộc đầu vào để chọn thông tin được giữ hoặc bỏ trong trạng thái. Kích thước trạng thái không tăng theo độ dài chuỗi, nhưng chi phí đọc ghi và khả năng gộp kernel vẫn ảnh hưởng tốc độ. Thông lượng đo trong bài báo không thể chuyển trực tiếp sang mọi batch hay thiết bị.

9. Thí nghiệm gợi ý

Kiểm tra công thức rời rạc hoá với trường hợp vô hướng, rồi so sánh scan song song với vòng lặp hồi quy. Thay đổi độ dài chuỗi và đo thời gian thực tế để khảo sát ảnh hưởng của lưu lượng bộ nhớ và cách cài đặt.

10. Tóm tắt công thức / khái niệm

Khái niệm	Công thức
SSM liên tục	$h' = Ah + Bx,\ y = Ch$
ZOH	$\bar{A} = e^{\Delta A},\ \bar{B} = (\Delta A)^{-1}(e^{\Delta A} - I)\Delta B$
Hồi quy rời rạc	$h_t = \bar{A}h_{t-1} + \bar{B}x_t,\ y_t = Ch_t$

Khái niệm	Công thức
LTI → tích chập	$y = x * \bar{K}$, $\bar{K} = (C\bar{B}, C\bar{A}\bar{B}, \dots)$
Chọn lọc (S6)	$B_t, C_t = \text{Linear}_N(x_t)$; $\Delta_t = \text{softplus}(\text{Param} + \text{Broadcast}_D(\text{Linear}_1(x_t)))$
Cổng RNN	N=1, A= $-$ 1: $h_t = (1 - g_t)h_{t-1} + g_t x_t$, $g_t = \sigma(z_t)$
Phép scan	$(a_2, b_2) \circ (a_1, b_1) = (a_2 a_1, a_2 b_1 + b_2)$; work $O(L)$, depth $O(\log L)$
Chi phí	$O(B_{sz} LDN)$; I/O giảm N lần nhờ fused scan
Khối Mamba	mở rộng E=2; conv ngắn → SiLU → SSM; nhân cổng SiLU; $3ED^2$ tham số
Tham số khớp	2 khối Mamba $= 12D^2$ = MHA + MLP

11. Kết luận ngắn

Mamba chỉ ra rằng điểm yếu của SSM trên ngôn ngữ là tính bất biến theo thời gian, và sửa bằng cách cho Δ, B, C phụ thuộc đầu vào — thực chất là một cổng quên/ghi/đọc kiểu RNN nhưng có nền tảng rời rạc hóa — kèm một thuật toán scan fused giữ chi phí tuyến tính trên phần cứng. Kết quả ở quy mô ≤ 3B khớp hoặc vượt Transformer++ và có thông lượng suy luận cao hơn khi batch lớn. Cơ chế chọn lọc và kích thước trạng thái là các yếu tố cần khảo sát khi đánh giá những kiến trúc hồi quy sau này.

Nguồn gốc: https://arxiv.org/abs/2312.00752

Bài 22

Transformers are SSMs: Generalized Models and Efficient Algorithms Through Structured State Space Duality (2024)

Tác giả: Tri Dao, Albert Gu.
Venue / arXiv: arXiv:2405.21060 (31/5/2024); ICML 2024 (theo cách bài 24 trích dẫn: PMLR 235). Code: github.com/state-spaces/mamba.
Nguồn: https://arxiv.org/abs/2405.21060
Chủ đề: State space model (SSM), ma trận semiseparable, structured masked attention, "state space duality" (SSD), thuật toán SSD theo khối, kiến trúc Mamba-2, mô hình lai SSM + attention.
Vị trí trong lộ trình: Đọc trước: 18 (Linear Transformers), 19 (RetNet), 21 (Mamba). Bài này **thống nhất** cách nhìn SSM và attention; cho thuật toán chunkwise (khối) mà GDN (24) kế thừa. Đọc tiếp: 23 (DeltaNet song song), 24 (Gated DeltaNet), 25 (Jamba).

> Ghi chú diễn giải, không phải bản dịch. Phần dưới tập trung vào lý thuyết, thuật toán SSD và kiến trúc Mamba-2; các chứng minh chi tiết và phương pháp song song hệ thống chỉ được tóm tắt.

1. Bối cảnh

Hai dòng mô hình chuỗi phát triển gần như tách rời:

- **Transformer**: attention $O(T^2)$ khi huấn luyện, KV-cache tăng tuyến tính khi sinh. Nhưng được hỗ trợ cực tốt về lý thuyết và hệ thống (FlashAttention, tensor parallelism, MQA/GQA...).
- **SSM có cấu trúc** (S4, rồi Mamba): $O(T)$ khi huấn luyện, state cố định khi sinh; Mamba (bài 21) ngang hoặc hơn Transformer cỡ nhỏ-vừa. Nhưng khó hiểu, khó tối ưu: cần "selective scan" kernel riêng, **không dùng nhân ma trận (tensor core)**, state nhỏ ($N = 16$) vì thuật toán scan chậm khi N tăng.

Mục tiêu của paper: xây một cầu lý thuyết giữa hai họ, để (i) chuyển mọi tối ưu của attention sang SSM, (ii) có thuật toán SSM nhanh bằng nhân ma trận, (iii) thiết kế kiến trúc mới (Mamba-2) dễ song song hoá.

Khái niệm nền cần nắm.

- **Sequence transformation (biến đổi chuỗi):** một ánh xạ $X \in \mathbb{R}^{T\times P} \to Y \in \mathbb{R}^{T\times P}$ (chiều T là thời gian). Self-attention và SSM đều là biến đổi chuỗi.
- **Matrix transformation form:** nhiều biến đổi chuỗi viết được dạng $Y = MX$ với $M \in \mathbb{R}^{T\times T}$ phụ thuộc tham số/dữ liệu. Attention: $M = \text{softmax}(QK^\top)$ (nhân quả: tam giác dưới). Đọc "đưa M ra" làm đối tượng nghiên cứu là mấu chốt của paper.
- **Ma trận có cấu trúc (structured matrix):** ma trận có cách lưu và nhân nhanh hơn T^2 (thưa, hạng thấp, Toeplitz...).
- **Đối ngẫu (duality):** cùng một hàm có hai thuật toán tính khác nhau (một "tuyến tính, hồi quy", một "bậc hai, kiểu attention").

---

2. Ý tưởng chính

1. Viết SSM (với tham số phụ thuộc thời gian) thành $Y = MX$. Ma trận M có tên là **ma trận semiseparable**: mọi ma trận con nằm trên/dưới đường chéo đều có hạng $\le N$ (N = chiều state). SSM = nhân với ma trận semiseparable.

2. Nếu hạn chế A_t thành **vô hướng nhân đơn vị** ($A_t = a_t I$), $M = L \circ (CB^\top)$, chính là một "attention có mặt nạ": $C \leftrightarrow Q$, $B \leftrightarrow K$, $X \leftrightarrow V$, mặt nạ L là ma trận 1-semiseparable (tích luỹ của a_t). Dạng bậc hai (nhân thẳng M) = **attention**; dạng tuyến tính (hồi quy) = **SSM**. Đó là **SSD**.
3. Thuật toán nhanh nhất không chọn một trong hai: **chia M thành khối**, đường chéo khối tính bằng dạng attention (nhân ma trận nhỏ), phần ngoài đường chéo là hạng thấp, đi qua state và một scan ngắn trên số khối. Cho FLOPs tuyến tính theo T và gần như toàn phép nhân ma trận: nhanh 2-8× kernel scan của Mamba, vượt FlashAttention-2 từ $T \geq 2$K.
4. Mamba-2 = SSD + chỉnh khối (chiếu song song, thêm norm, head dim lớn kiểu Transformer).

---

3. Phương pháp chi tiết

3.1 SSM là gì

Một SSM rời rạc, tham số phụ thuộc thời gian (ký hiệu paper, state ẩn $h_t \in \mathbb{R}^N$ mỗi kênh):

$$h_t = A_t h_{t-1} + B_t x_t, \qquad y_t = C_t^\top h_t \qquad \text{(Eq. 2)}$$

$x_t \in \mathbb{R}$ (một kênh; mở rộng P kênh bằng cách dùng chung A, B, C cho cả P kênh, gọi là một "head"), $A_t \in \mathbb{R}^{N\times N}$, $B_t, C_t \in \mathbb{R}^N$. N là **chiều state** (còn gọi *state expansion factor* vì nó nhân bộ nhớ lên N lần). "Structured" vì A_t phải có cấu trúc (chéo...) để tính nhanh. Mamba gọi mô hình có A, B, C phụ thuộc đầu vào là **selective SSM** (bài 21). Paper bỏ qua bước rời rạc hoá $(\Delta, \mathring{A}, \mathring{B}) \to (A, B)$ để gọn (Remark 1); theo Mamba thì $a_t = \exp(\Delta_t A)$ với $A < 0$, giá trị nằm trong $(0, 1)$ (đây là kiến thức từ bài 21, không viết lại trong text bài này).

3.2 SSM = nhân với ma trận semiseparable

Khai triển hồi quy (với $h_{-1} = 0$):

$$h_t = \sum_{s\leq t} A_t \cdots A_{s+1}\, B_s x_s, \qquad y_t = \sum_{s\leq t} C_t^\top A_t \cdots A_{s+1} B_s\, x_s.$$

Vậy $y = Mx$ với (Eq. 3)

$$\boxed{M_{ji} = C_j^\top A_j \cdots A_{i+1} B_i \quad (j \geq i), \qquad M_{ji} = 0\ (j < i).}$$

Semiseparable. Một ma trận tam giác dưới là N-*semiseparable* nếu mọi ma trận con nằm hoàn toàn trên hoặc dưới đường chéo có hạng $\leq N$. Lý do M thoả: lấy hàng $j \in [a, b)$, cột $i \in [c, d)$ với $d \leq a$ (khối nằm hẳn dưới đường chéo). Với mọi $j \geq a \geq d > i$ ta có $A_j \cdots A_{i+1} = (A_j \cdots A_d)(A_{d-1} \cdots A_{i+1})$, nên

$$M_{ji} = \underbrace{\left(C_j^\top A_j \cdots A_d\right)}_{1\times N} \underbrace{\left(A_{d-1} \cdots A_{i+1} B_i\right)}_{N\times 1},$$

tức khối con là tích của ma trận $(b-a)\times N$ và ma trận $N \times (d-c)$, hạng $\leq N$ (Lemma 3.3). Ngược lại mọi ma trận N-semiseparable có biểu diễn như vậy (Proposition 3.4). Hiểu trực quan: **thông tin từ quá khứ xa tới hiện tại đi qua "cổ chai" state N chiều**, nên khối ngoài đường chéo có hạng thấp. Khi chia lưới khối (mục 3.5), phân rã này có ba nhân tử: *trái* (C), *giữa* (A qua ranh giới khối), *phải* (B).

Trường hợp 1-semiseparable (state vô hướng $N = 1$, bỏ B, C):

$$L = 1\text{SS}(a) = \begin{pmatrix} 1 & & & \\ a_1 & 1 & & \\ a_2a_1 & a_2 & 1 & \\ \vdots & & & \ddots \end{pmatrix}, \qquad L_{ji} = a_j \cdots a_{i+1}.$$

Nhân $y = Lx$ chính là phép đệ quy vô hướng $y_t = a_t y_{t-1} + x_t$ (cumprodsum, tổng tích luỹ có nhân): dùng nhiều lần làm viên gạch cơ bản.

Hệ quả (Theorem 3.7): mọi SSM với state N tính được trong $O(TN)$ (không tính tiền xử lý) bằng dạng hồi quy; dạng ngây thơ (dựng M, rồi nhân) là bậc hai. Hai thuật toán nhân ma trận có cấu trúc: đây chính là "hai mode" của đối ngẫu.

3.3 Attention có mặt nạ có cấu trúc (SMA)

Attention nhân quả có kernel: $Y = (L \circ QK^\top)V$, L là mặt nạ tam giác dưới toàn 1 (linear attention, bài 18, bỏ softmax). Paper chứng minh gọn bằng *tensor contraction*: $Y = \text{contract}(TN, SN, SP, TS \to TP)(Q, K, V, L)$. Có hai thứ tự co tensor:

- *Bậc hai:* $G = QK^\top$ ($T \times T$), nhân mặt nạ L, rồi nhân V. Là attention thông thường: $O(T^2N)$.
- *Tuyến tính:* $Z = \text{outer}(K, V)$ ($T \times N \times P$), $H = L \cdot Z$ (cumsum theo thời gian, $O(TNP)$), $Y = \text{contract}(Q, H)$. Là dạng RNN của linear attention.

Quan sát then chốt: nút thắt của dạng tuyến tính là phép nhân với mặt nạ L; **chỉ cần L là ma trận có cấu trúc (nhân nhanh)** thì dạng tuyến tính vẫn hiệu quả. Đó là **Structured Masked Attention (SMA)**: $Y = (L \circ QK^\top)V$ với L có cấu trúc bất kỳ. Ví dụ: L toàn 1 → linear attention; $L_{ij} = \gamma^{i-j}$ → **RetNet** (19); L Toeplitz → kiểu mã hoá vị trí tương đối nhân; L Fourier. Định lý 5.2: attention nhân quả với tự hồi quy bậc hữu hạn bắt buộc có L semiseparable.

3.4 Đối ngẫu (SSD): hai thuật toán cho cùng một hàm

Chọn $A_t = a_t I$ (vô hướng nhân đơn vị). Khi đó $M_{ji} = a_j \cdots a_{i+1} \cdot (C_j^\top B_i)$, tức

$$\boxed{M = L \circ (CB^\top), \qquad L = 1\text{SS}(a)} \quad \Longleftrightarrow \quad Y = (L \circ QK^\top)V \text{ với } Q{=}C,\ K{=}B,\ V{=}X.$$

Khung SSM	Khung attention
C_t (chiếu ra)	Q (query)
B_t (chiếu vào)	K (key)
x_t (đầu vào kênh)	V (value)
a_t (decay vô hướng)	mặt nạ $L_{ji} = a_j \cdots a_{i+1}$
h_t (state $N \times P$)	$\sum_{s \le t}(\text{decay}) K_s V_s^\top$

Ghi chú về ký hiệu: trong SSM, $h_t \in \mathbb{R}^{N \times P}$ khi có P kênh: $h_t = a_t h_{t-1} + B_t x_t^\top$. Đó **đúng là** $S_t = \alpha_t S_{t-1} + v_t k_t^\top$ của Mamba2 viết trong GDN (24), chỉ chuyển vị.

Ví dụ kiểm tra ($T = 3$). $L = \begin{pmatrix} 1 & 0 & 0 \\ a_1 & 1 & 0 \\ a_2a_1 & a_2 & 1 \end{pmatrix}$. Dạng hồi quy: $h_0 = B_0x_0$, $h_1 = a_1h_0 + B_1x_1$, $h_2 =$

$a_2h_1 + B_2x_2$; $y_2 = C_2^\top h_2 = a_2a_1(C_2^\top B_0)x_0 + a_2(C_2^\top B_1)x_1 + (C_2^\top B_2)x_2$, khớp hàng thứ ba của $L \circ (CB^\top)$.

Hai dạng ảnh hưởng chi phí: dạng hồi quy $O(TNP)$ (mỗi bước cập nhật state $N \times P$), không dùng tensor core; dạng attention $O(T^2N)$ nhưng toàn nhân ma trận.

3.5 Thuật toán SSD theo khối

Chia T bước thành T/Q khối, mỗi khối Q bước (ví dụ $Q = 64$). Chia M thành lưới khối $Q \times Q$:

- **Khối đường chéo** $M^{(j,j)}$ = SSM trên chính khối đó với state đầu bằng 0 → tính bằng **dạng attention** (nhân ma trận $Q \times Q$, các khối độc lập, song song).
- **Khối ngoài đường chéo** $M^{(j,i)}$, $j > i$: hạng $\leq N$, phân rã ba phần (trái × giữa × phải) như ở 3.2. Tương ứng bốn bước tính:

Bước	Ý nghĩa	Phép tính
1. Trong khối (khối chéo)	Đầu ra do đầu vào cùng khối, state đầu $= 0$	$Y_{\text{diag}} = (L \circ CB^\top)X$ cục bộ
2. State của khối (nhân phải, B)	State cuối khối nếu state đầu $= 0$	$\text{states}_j = \sum_{s \in \text{khối } j} (\text{decay đến cuối khối})\, B_s x_s^\top$
3. Truyền state giữa khối (nhân giữa, A)	State thật đầu mỗi khối	scan 1-SS trên T/Q khối: $H_j = a^{(j)}_{\text{chunk}} H_{j-1} + \text{states}_j$
4. State → đầu ra (nhân trái, C)	Đóng góp của quá khứ vào đầu ra khối	$Y_{\text{off}} = (\text{decay từ đầu khối}) \cdot C\, H_{j-1}$

Đầu ra: $Y = Y_{\text{diag}} + Y_{\text{off}}$. Diễn giải: bước 1 là "attention trong khối", bước 2-4 là "RNN giữa các khối" nhưng chỉ có T/Q bước. Pseudo-code (viết lại, gần Listing 1 của paper, một head, a_t dạng log):

```
def ssd(X, loga, B, C, Q=64):                # X (T,P); loga (T,); B,C (T,N)
    T = X.shape[0]; nc = T // Q
    X, B, C, la = [t.reshape(nc, Q, *t.shape[1:]) for t in (X, B, C, loga)]
    cum = la.cumsum(1)                                          # (nc,Q) log decay trong
↪   khối
    # 1) khối chéo: attention cục bộ với mặt nạ decay
    Lmat = (cum[:, :, None] - cum[:, None, :]).exp().tril()     # (nc,Q,Q)
    Y_diag = torch.einsum('cij,cin,cjn,cjp->cip', Lmat, C, B, X)
    # 2) state cuối mỗi khối (khởi từ 0)
    dec_end = (cum[:, -1:] - cum).exp()                         # (nc,Q)
    states = torch.einsum('cj,cjn,cjp->cnp', dec_end, B, X)     # (nc,N,P)
    # 3) scan giữa khối (ngắn: nc bước)
    H, h = [], torch.zeros_like(states[0])
    chunk_dec = cum[:, -1].exp()
    for c in range(nc):
        H.append(h)                                             # state đầu khối c
        h = chunk_dec[c] * h + states[c]
    H = torch.stack(H)
    # 4) state đầu khối -> đầu ra
    Y_off = torch.einsum('ci,cin,cnp->cip', cum.exp(), C, H)
    return (Y_diag + Y_off).reshape(T, -1)
```

Phiên bản gốc dùng `segsum` + `einsum` đầy đủ batch/head, chỉ khoảng 25 dòng PyTorch, "không cần kernel mức thấp" và vẫn tương đối hiệu quả. Paper cho biết bước 3 là scan độc lập trên NP kênh, chiều dài T/Q, công việc $O(TNP/Q)$ rất nhỏ so với các bước khác.

3.6 Kiến trúc Mamba-2

Từ SSD, thiết kế khối (Hình 6):

- **SSD là hàm của** (A, X, B, C) **cùng lúc.** Mamba-1 sinh B, C, Δ *từ* X sau một phép chiếu (tuần tự). Mamba-2 sinh A, X, B, C **song song** từ đầu vào bằng một phép chiếu lớn, giống Q, K, V trong Transformer: bớt tham số, và hợp với **tensor parallelism** kiểu Megatron (giảm một nửa số điểm đồng bộ mỗi khối).
- **Thêm normalization** (như NormFormer) ngay trước phép chiếu đầu ra, giúp ổn định ở mô hình lớn (giống thêm GroupNorm/LayerNorm sau linear attention ở TransNormer/RetNet).
- **Head dimension** $P = 64$ **hoặc** 128 (Mamba-1 chỉ $P = 1$): làm cho các phép nhân ma trận đủ lớn để tận dụng tensor core. N (state) thường 64-256, lớn hơn nhiều so với Mamba-1 ($N = 16$) vì SSD không chậm đi khi tăng N.
- **Các mẫu "đa đầu"** tương đương attention (Eq. 17-20): *multi-head* (MHA), *multi-contract* (≈ MQA, bài 14: chia sẻ B và X, riêng C mỗi head), *multi-expand* (≈ multi-key), *multi-input* (≈ **multi-value attention, MVA**: chia sẻ B, C qua các head, X riêng mỗi head). Mamba-1 hoá ra là MVA; Mamba-2 dùng MVA, mở rộng thành **grouped-value** (GVA, tương tự GQA bài 15) để chia theo nhóm cho tensor parallel.
- **Hàm kernel/feature map** ψ áp lên nhánh B, C (≈ K, Q): mặc định SiLU/Swish; thử các kernel của linear attention và Performer đều không tốt hơn (mục 5).

3.7 Chi phí

Theo Bảng so sánh của paper (với T = độ dài, N = chiều state/head dim):

	Attention	SSM (scan)	SSD
Kích thước state khi sinh	T	N	N
FLOPs huấn luyện	T^2N	TN^2	TN^2
FLOPs suy luận (mỗi token)	TN	N^2	N^2
Bộ nhớ (thuật toán ngây thơ)	T^2	TN^2	TN
Dùng nhân ma trận (tensor core)	có	không	có

(Định lý 6.1 giả sử $P = N = Q$: huấn luyện $O(TN^2)$ FLOPs, suy luận $O(N^2)$ bộ nhớ và $O(N^2)$ FLOPs mỗi token khi $P = N$.)

Kích thước state mỗi tầng là HNP phần tử; nó không tăng theo chiều dài chuỗi. So sánh thời gian thực tế với attention cần tính đến phép chiếu, lưu lượng bộ nhớ và kernel.

---

4. Thí nghiệm và kết quả chính

Tốc độ (Hình 10, A100 80GB PCIe). SSD nhanh **2-8×** kernel selective scan của Mamba khi $N = 64$, cho phép state lớn hơn (≥ 8× Mamba) với chi phí tăng nhỏ; **nhanh hơn FlashAttention-2 từ độ dài 2K, nhanh hơn 6× ở 16K** (theo mục giới thiệu). Với N tăng, scan của Mamba chậm tuyến tính, SSD hầu như phẳng. Lưu ý (paper): cả mô hình Mamba-2 có thể không nhanh bằng Transformer ở chuỗi ngắn (2K) vì Transformer L layer gồm $L/2$ layer attention và $L/2$ MLP (MLP rất hiệu quả phần cứng).

Khôi phục liên kết đa truy vấn (MQAR, Arora et al.; Hình 8). Mamba-1 gặp khó, Mamba-2 làm tốt ở mọi cấu hình, tốt hơn rõ rệt ngay cả khi so cùng state size ($N = 16$); tăng N lên 64 và 256 cải thiện đều. (Chỉ đồ thị.) Paper "không chắc yếu tố nào chủ yếu".

Mô hình ngôn ngữ (The Pile, GPT-3-like từ 125M đến 2,7B, huấn luyện theo Chinchilla; 300B token cho các mô hình lớn). Scaling law (Hình 9): Mamba-2 Pareto-trội hơn Mamba và Transformer++ về perplexity theo FLOPs và theo thời gian thực (đồ thị). Downstream zero-shot (trích từ Bảng 1):

Mô hình	Pile ppl ↓	LAMBADA ppl ↓	LAMBADA acc ↑	HellaSwag acc ↑	PIQA acc ↑
Pythia-1B	7,82	7,92	56,1	47,2	70,7
Mamba-790M	7,33	6,02	62,7	55,1	72,1
Mamba-2-780M	7,26	5,86	61,7	54,9	72,0
Pythia-1.4B	7,51	6,08	61,7	52,1	71,0
Mamba-1.4B	6,80	5,04	65,0	59,1	74,2
Mamba-2-1.3B	6,66	5,02	65,7	59,9	73,2
Pythia-2.8B	6,73	5,04	64,7	59,3	74,0
Mamba-2.8B	6,22	4,23	69,2	66,1	75,2
Mamba-2-2.7B	6,09	4,10	69,7	66,6	76,4

Paper: Mamba-2 hơn Mamba mỗi cỡ và thường khớp Pythia cỡ gấp đôi (Mamba-2 2,7B cũng vượt Pythia-6,9B theo mục giới thiệu). Các cột khác (ARC, Winogrande, OpenbookQA...) có trong Bảng 10, text bị cắt.

Mô hình lai SSD + attention + MLP (mục 9.2.3).

Bảng 2: 350M, 48 layer, 7B token, số layer attention vs perplexity

Số layer attention	0 (Mamba-2)	1	2	3	4	5	6	7	9	11	15	24	Transformer++
Perplexity ↓	8,60	8,38	8,32	8,29	8,29	8,28	**8,26**	8,27	8,28	8,30	8,34	8,50	8,68

Khoảng 10% layer là attention là tốt nhất; chỉ 1 layer đã giảm từ 8,60 xuống 8,38. Paper chú thích: miễn là các layer attention được rải đều, không ở đầu hoặc cuối, vị trí chính xác không quan trọng nhiều (thử nghiệm quy mô nhỏ).

Bảng 3: 2,7B (64 layer), 300B token, The Pile

Mô hình	Pile ppl	LAMBADA ppl	Avg acc
Transformer++ (32 attn + 32 MLP)	6,13	3,99	60,2
Mamba-2 (64 SSD)	6,09	4,10	60,2
Mamba-2-MLP (32 SSD + 32 MLP)	6,13	4,18	59,6
Mamba-2-Attention (58 SSD + 6 attn)	**5,95**	**3,85**	**61,0**
Mamba-2-MLP-Attention (28 SSD + 4 attn + 32 MLP)	6,00	3,95	60,7

Kết luận paper: Transformer++ và Mamba-2 cùng chất lượng; chỉ cần 6 layer attention đã vượt cả hai. Thêm MLP làm giảm chất lượng nhẹ nhưng nhanh hơn và dễ nâng cấp lên MoE. Giả thuyết của tác giả: SSM là ánh xạ chuỗi tổng quát, attention làm cơ chế *truy hồi* tra token cũ thay vì ép toàn bộ ngữ cảnh vào state.

5. Phân tích / ablation

Thiết kế khối (Bảng 4, 125M). Mamba-1 tuần tự không norm: 129,3M tham số, ppl 11,76; thêm norm: 11,54; chiếu song song không norm: 126,5M, 11,66; **Mamba-2 (song song + norm): 126,5M, 11,49.** Cả hai thay đổi đều giúp nhẹ, chiếu song song bớt tham số và hợp tensor parallel.

Mẫu đầu (Bảng 5, cùng state tổng HPN).

Mẫu head	Analog	125M ppl	360M ppl
Multi-input (MIS)	MVA	**11,66**	**8,73**
Multi-contract (MCS)	MQA	12,62	9,33
Multi-expand (MES)	multi-key	12,59	9,36
Multi-head (MHS)	MHA	12,06	9,01

MVA tốt nhất với chênh lệch lớn "dù tổng state như nhau", điều mà paper nêu là đáng ngạc nhiên và không giải thích được bằng kích thước state.

Kernel/feature map (Bảng 6). Không dùng feature map: 11,58; Swish: 11,66; Exp: 11,62; ReLU: 11,73; cosFormer: 11,97; Random Feature Attention: 11,57; Performer: 12,21. Mở rộng feature kiểu Based/ReBased cũng không cải thiện (Bảng 7, ví dụ 130M: Swish 11,67, Based 12,19, LayerNorm 11,50). Kết luận: **các xấp xỉ kernel của linear attention không giúp**; paper gợi ý có thể bỏ hẳn hàm kích hoạt. Tác giả lưu ý SSD khác linear attention thường ở mặt nạ 1-SS, nên kết quả âm này không đáng ngạc nhiên.

---

6. Giới hạn

Do tác giả nêu:

- SSD giảm nhẹ biểu diễn so với selective SSM của Mamba-1 (A_t từ chéo xuống vô hướng) để đổi lấy tốc độ.
- Mamba-2 toàn mô hình có thể chậm hơn Transformer ở chuỗi ngắn.
- SMA/SSD chỉ nói về một số "vị" attention (nhân quả kiểu linear); softmax thuần không nằm trong khung (chú thích 1: tiêu đề là "homage").
- Tác vụ MQAR: lý do Mamba-2 hơn Mamba-1 chưa rõ.

Nhận xét bổ sung:

- Decay là **vô hướng theo head**: mọi kênh state nhớ cùng tốc độ. GLA (20) và KDA (26) cho thấy có lợi khi mở rộng theo kênh.
- Quy tắc cập nhật chỉ *cộng chồng* ($+B_t x_t^\top$): không có cơ chế sửa/xoá đích danh, nên recall chính xác yếu hơn delta rule (xem Bảng S-NIAH trong bài 24: Mamba2 tụt mạnh khi dài). Đây là động cơ của 23 và 24.
- Hầu hết thí nghiệm đến 2,7B/300B token trên The Pile; không có thử nghiệm ảnh/OCR/đa phương thức.

---

7. Liên hệ với các bài khác

- **18 Linear Transformers:** SSD là linear attention + mặt nạ decay 1-semiseparable phụ thuộc dữ liệu; paper còn đưa bằng chứng tensor contraction đơn giản cho dạng hồi quy của linear attention.
- **19 RetNet:** $L_{ij} = \gamma^{i-j}$ = SMA với mặt nạ decay hằng số; Mamba-2 là bản decay phụ thuộc dữ liệu. **20 GLA:** gate theo kênh (ma trận chuyển chéo đầy đủ) nằm ngoài lớp "vô hướng nhân đơn vị" của SSD nhưng vẫn chunkwise được.

- **21 Mamba:** Mamba-2 = Mamba-1 + (i) A vô hướng, (ii) head dim lớn, (iii) chiếu song song, (iv) norm. Mamba-1 là MVA với $P = 1$.
- **14 MQA / 15 GQA:** SSD cho phép định nghĩa tương tự cho SSM (multi-contract ≈ MQA, grouped-value ≈ GQA).
- **23 DeltaNet / 24 Gated DeltaNet:** dùng cùng khung chunkwise (trong-khối kiểu attention + truyền state), nhưng ma trận chuyển $I - \beta kk^\top$ không phải vô hướng nên *không* nằm trong SSD; đã có bình luận trong bài 23 về việc SSD "có thể không bao quát". Công thức của Mamba2 trong GDN: $S_t = \alpha_t S_{t-1} + v_t k_t^\top$, giống hệt (3.4).
- **25 Jamba / 26 Kimi Linear:** hybrid SSM/linear + attention; mục 9.2.3 ở đây là bằng chứng sớm (tỉ lệ ~10% attention).
- **44 FlashAttention:** so sánh tốc độ; FlashAttention-2 là baseline mạnh nhất, SSD vượt từ 2K.

---

8. Gợi ý áp dụng

Đối ngẫu SSD cho phép nhìn cùng phép tính qua dạng hồi quy và dạng ma trận có cấu trúc. Đây là công cụ để phân tích cũng như xây dựng thuật toán theo chunk; các kích thước khối hiệu quả phụ thuộc phần cứng và kernel, nên giảm FLOPs lý thuyết chưa chắc giảm thời gian.

9. Thí nghiệm gợi ý

Trên chuỗi ngắn, tính cùng đầu ra bằng recurrence, ma trận có cấu trúc và phép phân khối. So sánh sai số làm tròn trước khi benchmark các kích thước chunk khác nhau.

10. Tóm tắt công thức / khái niệm

Khái niệm	Công thức / ý nghĩa
SSM chọn lọc	$h_t = A_t h_{t-1} + B_t x_t,\ y_t = C_t^\top h_t$
Dạng ma trận	$y = Mx,\ M_{ji} = C_j^\top A_j \cdots A_{i+1} B_i$
Semiseparable bậc N	mọi ma trận con trên/dưới chéo có hạng $\le N$
1-SS	$L_{ji} = a_j \cdots a_{i+1}$, nhân = đệ quy vô hướng $y_t = a_t y_{t-1} + x_t$
SMA	$Y = (L \circ QK^\top)V$, L có cấu trúc
SSD	$A_t = a_t I$: $M = L \circ (CB^\top)$, $L = 1\text{SS}(a)$; $C{=}Q, B{=}K, X{=}V$
Thuật toán SSD	khối chéo = attention; ngoài chéo = hạng thấp qua state + scan T/Q bước
Chi phí SSD	$O(TN^2)$ huấn luyện, $O(N^2)$ mỗi token suy luận, state N
Khối Mamba-2	A, X, B, C chiếu song song; norm trước phép chiếu ra; $P \in \{64, 128\}$; MVA/GVA
Hybrid	≈ 10% layer attention tốt nhất (Bảng 2); 58 SSD + 6 attn tốt hơn cả hai (Bảng 3)

11. Kết luận ngắn

Bài này cho thấy attention tuyến tính có mặt nạ và SSM có A_t vô hướng là **cùng một hàm** viết bằng hai cách (đối ngẫu), và rằng SSM là phép nhân ma trận semiseparable. Từ đó có thuật toán SSD theo khối vừa tuyến tính vừa giàu nhân ma trận (2-8× kernel Mamba, qua FlashAttention-2 từ 2K), và kiến trúc Mamba-2. Gated DeltaNet có phép chuyển $I - \beta kk^\top$ nên vượt ra khỏi khung SSD được trình bày ở đây.

Nguồn gốc: https://arxiv.org/abs/2405.21060

Bài 23

Parallelizing Linear Transformers with the Delta Rule over Sequence Length (2024)

Tác giả: Songlin Yang, Bailin Wang, Yu Zhang, Yikang Shen, Yoon Kim.
Venue / arXiv: arXiv:2406.06484; NeurIPS 2024. Code: thư viện `flash-linear-attention`.
Nguồn: https://arxiv.org/abs/2406.06484
Chủ đề: Delta rule trong linear attention (DeltaNet); biểu diễn WY của tích ma trận Householder; thuật toán chunkwise song song theo chiều chuỗi; hybrid DeltaNet + attention.
Vị trí trong lộ trình: Đọc trước: 18 (Linear Transformers), 21 (Mamba), 22 (Mamba-2/SSD). Bài này cho **kỹ thuật song song hoá** mà Gated DeltaNet (24) dùng nguyên xi, và cho cách nhìn "bộ nhớ liên kết sửa lỗi". Đọc tiếp: 24 (thêm gate), 26 (gate theo kênh).

> Ghi chú diễn giải, không phải bản dịch. Các dẫn xuất ở mục 3 là diễn giải từ công thức của bài báo.

1. Bối cảnh

Linear attention (bài 18) viết attention thành RNN với state ma trận $S_t = S_{t-1} + v_t k_t^\top$, $o_t = S_t q_t$. Hai chuyện xảy ra:

1. **Chất lượng:** chỉ cộng dồn nên khi số cặp key–value vượt chiều key d, các key chồng lấn ("key collision", Schlag et al. 2021). Mô hình không thể "dọn chỗ" cho thông tin mới. Kết quả: kém ở các tác vụ cần *truy hồi trong ngữ cảnh* (in-context retrieval), ví dụ RAG.
2. **Delta rule** (Widrow-Hoff, 1960; áp dụng vào Transformer bởi Schlag et al. 2021): trước khi ghi, đọc giá trị cũ gắn với key hiện tại rồi sửa. DeltaNet cho kết quả tốt trên tác vụ tổng hợp và dịch nhỏ. Nhưng **thuật toán huấn luyện gốc tuần tự theo thời gian**: không tận dụng GPU, nên chưa ai huấn luyện được DeltaNet ở quy mô LLM.

Cần giải quyết: tạo thuật toán huấn luyện song song theo chiều chuỗi, giàu phép nhân ma trận (tensor core), không phải dựng state $S_t \in \mathbb{R}^{d\times d}$ cho từng t (tốn I/O).

Ghi chú nền: chunkwise form (dạng theo khối) cho linear attention. Có ba cách tính cùng một hàm:

- *Hồi quy:* $O(Ld^2)$ FLOPs, nhưng L bước tuần tự, toàn phép nhân từng phần tử, không dùng tensor core.
- *Song song:* $O = (QK^\top \odot M)V$, $O(L^2 d)$, một bước, rất hợp GPU nhưng đắt khi L lớn.
- *Chunkwise:* chia chuỗi thành L/C khối cỡ C (64 hay 128). Trong khối dùng dạng song song; giữa khối truyền state. Độ phức tạp $O(LCd + Ld^2)$, L/C bước tuần tự. $C = L$ là dạng song song, $C = 1$ là dạng hồi quy. Phương trình (Eq. 1-2 của paper):

$$S_{[t+1]} = S_{[t]} + V_{[t]}^\top K_{[t]}, \qquad O_{[t]} = Q_{[t]} S_{[t]}^\top + \left(Q_{[t]} K_{[t]}^\top \odot M_C\right) V_{[t]}.$$

---

2. Ý tưởng chính

DeltaNet: $S_t = S_{t-1}(I - \beta_t k_t k_t^\top) + \beta_t v_t k_t^\top$. Mỗi bước nhân state bên phải với một ma trận "đơn vị trừ hạng-1" (Householder tổng quát). Khai triển L bước cho một **tích của L ma trận như vậy**, nhìn qua thì khó song song hoá và khó chạy chunkwise vì state cuối khối phụ thuộc cả tích.

Chìa khoá: tích các Householder có **biểu diễn WY** gọn: $\prod(I - \beta_i k_i k_i^\top) = I - \sum_i w_i k_i^\top$ với các vector w_i cùng kích thước d. Nhờ đó (i) không cần lưu ma trận $d \times d$ cho mỗi bước, chỉ lưu vector; (ii) các w_i (và

"pseudo-value" u_i) tính được cho cả khối bằng **một phép nghịch đảo ma trận tam giác nhỏ** (UT transform), chuyển vòng lặp tuần tự thành phép nhân ma trận. Rồi lắp vào dạng chunkwise của linear attention, chỉ thay V bằng $U - WS^\top$. Kết quả: huấn luyện DeltaNet 1,3B/100B token (và 3B/1T token) lần đầu khả thi, vượt Mamba và GLA cùng quy mô.

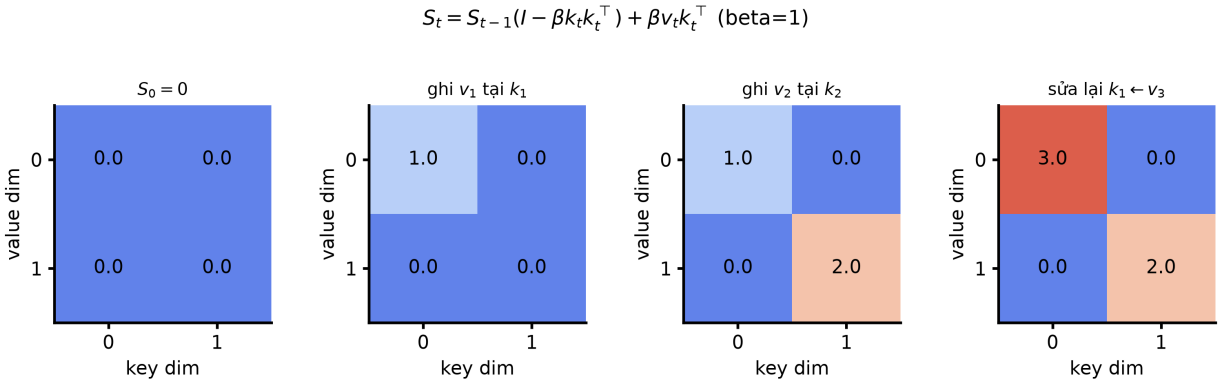
---

3. Phương pháp chi tiết

Ký hiệu: $S_t \in \mathbb{R}^{d_v \times d_k}$ (paper vẽ $d \times d$ cho một head, rồi nói head dim $d_{head} < d$); $o_t = S_t q_t$. Chỉ số $[t]$ = khối thứ t; $x^r_{[t]}$ = phần tử thứ r trong khối; $S^0_{[t]} = S_{[t]}$ = state đầu khối.

3.1 DeltaNet là bộ nhớ liên kết sửa lỗi

State S là bảng tra "key → value" (nhớ liên kết, *associative memory*). **Đọc:** $\hat{v} = Sk$. **Luật delta (Widrow-Hoff / LMS):** nếu dự đoán Sk_t khác đích v_t, dịch S một bước để giảm sai lệch:



DeltaNet $S_t = S_{t-1}(I - \beta k k^\top) + \beta v k^\top$: ghi rồi sửa lại cùng key. (minh hoạ, tự tính)

$$\mathcal{L}_t(S) = \tfrac{1}{2}\|Sk_t - v_t\|^2, \quad S_t = S_{t-1} - \beta_t \nabla \mathcal{L}_t = S_{t-1} - \beta_t (S_{t-1}k_t - v_t)k_t^\top.$$

Viết gọn: $S_t = S_{t-1}(I - \beta_t k_t k_t^\top) + \beta_t v_t k_t^\top$ (Eq. DeltaNet). Cách đọc khác: $v^{\text{old}} = S_{t-1}k_t$, $v^{\text{new}} = \beta_t v_t + (1-\beta_t)v^{\text{old}}$, $S_t = S_{t-1} - v^{\text{old}}k_t^\top + v^{\text{new}}k_t^\top$ ("xoá cũ, ghi mới"). $\beta_t = \sigma(W_\beta x_t) \in (0,1)$ là *cường độ ghi*: $\beta = 1$ thay hẳn, $\beta = 0$ bỏ qua.

Linear attention thuần thì tối ưu hàm mất mát *tuyến tính* $-\langle Sk, v\rangle$: gradient là hằng $vk^\top$, không phụ thuộc state hiện tại, nên chỉ "cộng thêm". Delta rule tối ưu hàm mất mát *bậc hai*: gradient tỉ lệ với sai lệch, nên sửa mạnh khi sai nhiều và không sửa khi đã đúng (chú thích 3 paper). Ví dụ số ($d = 2$, key đơn vị trực giao) xem bài 24 mục 3.2: cộng chồng cho $(11, 12) = v_1 + v_3$, delta rule cho đúng v_3.

Ổn định. $I - \beta k k^\top$ có trị riêng 1 (bội $d-1$) và $1 - \beta\|k\|^2$. Cần $\|k\|_2 = 1$ để trị riêng thuộc $(0, 1]$. Paper dùng **L2-norm** thay cho L1-norm của Schlag et al., vì với $\beta = 1$ thì $I - kk^\top$ là phép chiếu: xoá đúng một chiều, giữ $d-1$ chiều còn lại, "quên có mục tiêu".

3.2 Tái tham số hoá tiết kiệm bộ nhớ: $S_t = \sum_i u_i k_i^\top$

Mệnh đề. $S_t = \sum_{i=1}^{t} u_i k_i^\top$ với

$$u_t = \beta_t \Big(v_t - \sum_{i<t} u_i \, (k_i^\top k_t) \Big).$$

Chứng minh (quy nạp). $t = 1$: $S_1 = \beta_1 v_1 k_1^\top$, $u_1 = \beta_1 v_1$. Bước quy nạp: $S_t = S_{t-1} - \beta_t (S_{t-1}k_t)k_t^\top + \beta_t v_t k_t^\top$ và $S_{t-1}k_t = \sum_{i<t} u_i (k_i^\top k_t)$, nên $S_t = \sum_{i<t} u_i k_i^\top + u_t k_t^\top$ với u_t như trên. □

Ý nghĩa: DeltaNet chính là linear attention với **value giả (pseudo-value)** u_t thay cho v_t. $u_t = \beta_t(v_t - S_{t-1}k_t)$ chính là *vector sửa lỗi*: "giá trị cần ghi trừ giá trị state đang dự đoán". Tính u_t không cần dựng S_{t-1} ($O(d)$ bộ nhớ), nhưng vẫn cần u_i trước đó, tốn $O(L^2 d)$ và vẫn tuần tự. Phần tiếp theo cho phép làm theo khối.

3.3 Dạng chunkwise: WY + UT

Khai triển (Eq. 4): $S_t = \sum_{i=1}^{t} \beta_i (v_i k_i^\top) \prod_{j=i+1}^{t} (I - \beta_j k_j k_j^\top)$. Với khối $[t]$ đặt $P_{[t]}^r = \prod_{i=1}^{r} (I - \beta_{[t]}^i k_{[t]}^i k_{[t]}^{i\top})$ ("hệ số tắt dần" áp lên state đầu khối) và $H_{[t]}^r$ = phần đóng góp của các token trong khối. Khi đó (Eq. 5):

$$S_{[t]}^r = S_{[t]} P_{[t]}^r + H_{[t]}^r.$$

WY representation (Bischof & Van Loan 1985) (Eq. 6-7):

$$P^r = I - \sum_{i=1}^{r} w^i k^{i\top}, \qquad w^r = \beta^r \Big(k^r - \sum_{i<r} w^i \, (k^{i\top} k^r) \Big),$$

$$H^r = \sum_{i=1}^{r} u^i k^{i\top}, \qquad u^r = \beta^r \Big(v^r - \sum_{i<r} u^i \, (k^{i\top} k^r) \Big).$$

Chứng minh P^r: $P^r = P^{r-1}(I - \beta_r k_r k_r^\top) = P^{r-1} - \beta_r (P^{r-1} k_r) k_r^\top$, với $P^{r-1} k_r = k_r - \sum_{i<r} w^i (k^{i\top} k_r)$, suy ra w^r như trên. □ (H^r giống §3.2, nhưng chỉ trong khối.)

Chuyển sang khối (thế P, H vào, Eq. 8-9):

$$S^r = S_{[t]} + \sum_{i \le r} \big(u^i - S_{[t]} w^i \big) k^{i\top}.$$

Số hạng $u^i - S_{[t]} w^i$ chính là vector sửa lỗi *thật* $\beta_i (v_i - S_{i-1} k_i)$: u^i là bản tính với state đầu khối bằng 0, còn $-S_{[t]} w^i$ bù cho những gì state cũ đã "biết". Dạng ma trận:

$$\boxed{S_{[t+1]} = S_{[t]} + \big(U_{[t]} - W_{[t]} S_{[t]}^\top \big)^\top K_{[t]}}$$

$$\boxed{O_{[t]} = Q_{[t]} S_{[t]}^\top + \big(Q_{[t]} K_{[t]}^\top \odot M \big) \big(U_{[t]} - W_{[t]} S_{[t]}^\top \big)}$$

(So với linear attention thuần: chỉ đổi $V \to U - WS^\top$.) Dòng thứ hai: đầu ra token r = đọc state đầu khối + (attention nhân quả trong khối) trên *vector sửa lỗi*.

UT transform (Joffrain et al. 2006) để tính W, U không dùng vòng lặp theo r. Viết hệ $w^r = \beta^r k^r - \beta^r \sum_{i<r} w^i (k^{i\top} k^r)$ ở dạng ma trận: $W = \mathrm{diag}(\beta) K - \mathrm{diag}(\beta)\, \mathrm{strictLower}(KK^\top) W$, tức $\big(I + \mathrm{diag}(\beta) \mathrm{strictLower}(KK^\top) \big) W = \mathrm{diag}(\beta) K$. Vậy (Eq. 10-11):

$$\boxed{T = \Big[I + \mathrm{strictLower}\big(\mathrm{diag}(\beta) KK^\top \big) \Big]^{-1} \mathrm{diag}(\beta), \qquad W = TK, \quad U = TV.}$$

Ma trận cần nghịch đảo là tam giác dưới $C \times C$ với đường chéo bằng 1, giải bằng thế tiến (forward substitution), rẻ và không tuần tự theo L (các khối độc lập). Phần còn lại là toàn phép nhân ma trận.

Pseudo-code (diễn giải từ công thức):

```
def deltanet_chunk(Q, K, V, beta, S, C=64):        # Q,K (L,d) L2-norm; V (L,dv); S (dv,d)
    outs = []
    for s in range(0, Q.shape[0], C):
```

```
        q, k, v, b = Q[s:s+C], K[s:s+C], V[s:s+C], beta[s:s+C]
        I = torch.eye(len(q))
        T = torch.linalg.solve_triangular(I + torch.tril(b[:,None]*(k@k.T), -1),
                                          torch.diag(b), upper=False)
        W, U = T @ k, T @ v
        corr = U - W @ S.T                          # vector sửa lỗi thật của cả khối
        M = torch.tril(torch.ones(len(q), len(q)))
        outs.append(q @ S.T + ((q@k.T) * M) @ corr)
        S = S + corr.T @ k
    return torch.cat(outs), S
```

Thực tế: kernel Triton, state được tính lại trong backward (để tiết kiệm bộ nhớ), không dựng S theo từng bước.

Chi phí. Dạng chunkwise: $O(LCd + Ld^2)$ FLOPs và L/C bước tuần tự (chưa dùng chunk-level parallel scan). Nhân thêm: giải tam giác $C \times C$ mỗi khối (ước lượng khoảng $O(C^2d + C^3)$ cho mỗi khối, paper không nêu). **Dạng hoàn toàn song song** cũng viết được: $A_{ij} = k_j^\top P_i^{j+1} q_i$, $A = (QK^\top \odot M)T$ (Eq. 3,11), nhưng cần nghịch đảo $L \times L$ (lập phương theo L), nên paper không dùng cho huấn luyện, chỉ nói có thể dùng để phân tích/diễn giải "attention map" của RNN.

Thử nghiệm tốc độ (Hình 1): kernel chunkwise vs kernel hồi quy (đã nhanh gấp 2 lần CUDA kernel gốc của Schlag et al.), $d = 2048$ với batch×dài = 16384, head dim 64/128/256. Tăng tốc lớn dần theo L và head dim; trục đồ thị tới 30× (chỉ đọc được từ hình, không có bảng số).

3.4 Khối DeltaNet Transformer

Theo Llama ("Transformer++", bài 05): thay self-attention bằng lớp DeltaNet, giữ SwiGLU, RMSNorm; thêm norm trước phép chiếu đầu ra. Tham số: $4d^2$ cho layer DeltaNet và $8d^2$ cho FFN SwiGLU (phần β nhỏ không đáng kể). q, k: $\frac{\mathrm{SiLU}(Wx)}{\|\mathrm{SiLU}(Wx)\|_2}$; $\beta = \sigma(\cdot)$; có **short convolution** (depthwise, kernel nhỏ, tăng khả năng dịch cục bộ) sau các phép chiếu q, k, v. Thay "ELU+1" của bản gốc bằng SiLU, thay L1-norm bằng L2-norm: cả hai đều có ablation (mục 5).

3.5 Mô hình lai

Linear attention thiếu thông tin vị trí và khó so khớp cục bộ chính xác. Paper thử hai kiểu lai: (1) **xen SWA** (sliding-window attention) mỗi layer thứ hai (theo Griffin/Samba); (2) **hai layer global attention** (layer thứ 2 và layer $N/2 + 1$, theo Fu et al.). Kết quả ở mục 4.

---

4. Thí nghiệm và kết quả chính

Tác vụ tổng hợp. *MQAR* (multi-query associative recall): DeltaNet đạt gần hoàn hảo ở cấu hình khó nhất (không cần conv) và hơn Mamba ở chiều model thấp (Hình 4, chỉ có đồ thị). *MAD benchmark* (Hình 3 paper):

Mô hình	Compress	Fuzzy recall	In-context recall	Memorize	Noisy recall	Selective copy	Trung bình
Transformer	51,6	29,8	94,1	85,2	86,8	99,6	74,5
Mamba	52,7	6,7	90,4	89,5	90,1	86,3	69,3
GLA	38,8	6,9	80,8	63,3	81,6	88,6	60,0
DeltaNet	42,2	**35,7**	**100**	52,8	**100**	**100**	71,8

DeltaNet nổi trội ở các tác vụ recall (kể cả Fuzzy recall vượt cả Transformer) nhưng yếu ở *Memorize* (52,8). Kết quả khác (RegBench, in-context language learning) nằm ở phụ lục, text trích không có số.

Mô hình ngôn ngữ (Bảng 1). Mọi mô hình huấn luyện trên cùng tập con SlimPajama, tokenizer Mistral, đánh giá bằng lm-evaluation-harness. 340M/15B token và 1,3B/100B token. Cột nhiễu do bảng hỏng; dưới đây chỉ các giá trị đọc chắc chắn (Wiki ppl, LAMBADA ppl, và Avg tính được lại khớp):

Quy mô	Mô hình	Wiki ppl ↓	LMB ppl ↓	Avg acc ↑
340M	Transformer++	28,39	42,69	41,2
340M	Mamba (conv)	28,39	39,66	41,8
340M	GLA (không conv)	28,65	43,35	41,5
340M	DeltaNet (conv)	28,24	37,37	42,1
340M	+ Sliding Attn	27,06	38,17	42,1
340M	+ Global Attn (2 layer)	27,51	35,04	42,1
1,3B	Transformer++	16,85	13,44	50,9
1,3B	Mamba (conv)	17,06	13,89	50,0
1,3B	GLA (không conv)	17,22	14,47	51,0
1,3B	DeltaNet (conv)	16,87	12,21	51,6
1,3B	+ Sliding Attn	16,56	11,74	52,1
1,3B	+ Global Attn (2 layer)	16,55	12,40	51,8

Kết luận paper: DeltaNet vượt Mamba/GLA về perplexity và tác vụ downstream; hybrid (SWA hoặc 2 layer global) vượt cả Transformer++. Về tác vụ cần nhiều recall (SWDE, SQuAD, FDA): ở 340M với cùng kích thước state, DeltaNet > GLA (delta rule hiệu quả); ở 1,3B DeltaNet *kém* GLA vì **state scale kém** (head dim bị giới hạn, xem §6); hybrid cải thiện rõ ở các tác vụ này (số liệu chi tiết bị hỏng định dạng trong bản text nên không chép).

Mô hình 3B/1T token (Hình 5, cùng thiết lập Shen et al.):

Mô hình	Avg	MMLU
Llama-3.2-3B	62,8	54,1
PowerLM-3B	62,3	45,0
DeltaNet-3B	**59,8**	40,7
RecurrentGemma-2B	57,9	31,8
RWKV-6-3B	54,9	28,4
Mamba-2.7B	53,3	26,1

DeltaNet 3B hơi kém Transformer cùng thiết lập (PowerLM-3B) nhưng hơn các RNN 2-3B; lưu ý số token huấn luyện của các baseline khác nhau.

Throughput huấn luyện (Hình 6, 1,3B, H100): gần GLA, nhanh hơn Mamba đáng kể; mọi mô hình tuyến tính vượt Transformer khi chuỗi huấn luyện dài hơn (số liệu chỉ ở đồ thị).

---

5. Phân tích / ablation

- **Chuẩn hoá và feature map (340M, Bảng 1 dưới).** Đổi L1-norm → L2-norm cải thiện lớn: ví dụ "L1-norm + 1+ELU" Wiki 31,12 / LMB 55,96 vs "L2-norm + 1+ELU" 28,03 / 37,62. Trong L2-norm, SiLU (mặc định) tốt nhất (28,24 / 37,37 so với ReLU 28,75 / 43,53).
- **Short convolution:** DeltaNet có conv: Wiki 28,24, LMB 37,37, Avg 42,1; không conv: 29,08, 50,87, 41,3. Conv giúp rõ (đặc biệt LAMBADA).

- **Hybrid:** SWA xen kẽ giảm Wiki ppl (28,24 → 27,06 ở 340M; 16,87 → 16,56 ở 1,3B); 2 layer global attention mạnh ở tác vụ truy hồi hơn (paper nói bằng lời).
- **Chunk vs recurrent:** kernel chunkwise càng ưu thế khi L và head dim càng lớn; là nơi sequence-parallelism và tensor core phát huy.

---

6. Giới hạn

Tác giả nêu:

- Tốc độ huấn luyện vẫn kém GLA vì phải "marginalize" trên head dim trong kernel (phụ thuộc state-to-state), khó tile với head dim lớn. Hệ quả: head dim bị chặn nhỏ → state nhỏ → kém ở tác vụ cần recall (ở 1,3B kém GLA). Gợi ý: ma trận Householder khối chéo với khối vừa SRAM (≈128) trong khi head dim tổng thể lớn.
- **Khả năng tổng quát hoá độ dài hạn chế** so với GLA/RetNet/Mamba (không ngoại suy tốt vượt độ dài huấn luyện); suy đoán do **thiếu hệ số decay tường minh**, có thể khắc phục bằng gate. Đây đúng là động cơ của Gated DeltaNet (24).
- Delta rule có giới hạn biểu diễn lý thuyết (Irie et al. 2023); các biến thể mạnh hơn (hồi quy phi tuyến, Mesa layer, TTT, Titans) hy sinh tính song song hoặc cần xử lý khối.
- Mô hình khung "unifying" của paper (Bảng 2) chưa chắc là cách nhìn "đúng"; SSD (22) là ứng viên, nhưng có thể không bao hết lớp mô hình có ma trận chuyển không có cấu trúc.

Nhận xét bổ sung:

- Không có ablation tách riêng tác dụng của delta rule khỏi short conv/L2-norm ở quy mô lớn (chỉ ở 340M).
- Số liệu recall chi tiết không trích được, nên khó đánh giá định lượng "DeltaNet kém GLA ở 1,3B".
- Không có thử nghiệm tác vụ có cấu trúc 2D (ảnh/OCR).

---

7. Liên hệ với các bài khác

- **18 Linear Transformers:** DeltaNet = linear attention + sửa lỗi (loss bậc hai thay loss tuyến tính).
- **19 RetNet, 20 GLA, 21 Mamba, 22 Mamba-2:** họ "decay" (cập nhật phần tử, ma trận chuyển chéo). DeltaNet dùng ma trận chuyển *không chéo* (đơn vị trừ hạng-1), giàu biểu diễn hơn mà vẫn song song nhờ WY. Bảng 2 paper xếp tất cả vào dạng $S_t = S_{t-1} \bullet M_t + v_t k_t^\top$; 22 cho thấy các mô hình vô hướng là trường hợp của SSD (ma trận semiseparable), còn DeltaNet thì không nằm gọn trong đó.
- **24 Gated DeltaNet:** thêm α_t vào đúng khuôn mẫu này; thuật toán chunkwise của 24 là bản mở rộng của §3.3.
- **26 Kimi Linear:** KDA có chung cấu trúc "Householder × gate" với state chuyển vị, đổi gate vô hướng sang theo kênh.
- **25 Jamba:** hybrid Mamba+attention; DeltaNet cho thấy hybrid dùng được với layer delta-rule.

---

8. Gợi ý áp dụng

Delta rule cập nhật bộ nhớ theo sai số dự đoán của trạng thái hiện tại. Chuẩn hoá key và giới hạn hệ số cập nhật có vai trò trong phân tích ổn định; khi thay đổi cách biểu diễn hoặc độ chính xác số, các điều kiện này cần được kiểm tra lại. Thuật toán theo chunk giúp tận dụng phép nhân ma trận trong xử lý chuỗi.

9. Thí nghiệm gợi ý

So sánh đầu ra của recurrence và công thức chunkwise trên tensor ngẫu nhiên, với nhiều kích thước chunk. Theo dõi chuẩn trạng thái khi thay đổi chuẩn của key và hệ số cập nhật để khảo sát điều kiện ổn định được phân tích trong bài.

10. Tóm tắt công thức / khái niệm

Khái niệm	Công thức / ý nghĩa
Delta rule	$S_t = S_{t-1}(I - \beta_t k_t k_t^\top) + \beta_t v_t k_t^\top$ (SGD trên $\frac{1}{2}\|Sk - v\|^2$)
Trị riêng chuyển	1 (bội $d-1$), $1 - \beta\|k\|^2$
Pseudo-value	$S_t = \sum_i u_i k_i^\top$, $u_t = \beta_t(v_t - \sum_{i<t} u_i k_i^\top k_t)$
WY	$\prod(I - \beta_i k_i k_i^\top) = I - \sum w_i k_i^\top$, $w_r = \beta_r(k_r - \sum_{i<r} w_i k_i^\top k_r)$
UT	$T = [I + \text{strictLower}(\text{diag}(\beta)KK^\top)]^{-1}\text{diag}(\beta)$, $W = TK$, $U = TV$
State khối	$S_{[t+1]} = S + (U - WS^\top)^\top K$
Đầu ra khối	$O = QS^\top + (QK^\top \odot M)(U - WS^\top)$
Chi phí chunkwise	$O(LCd + Ld^2)$, L/C bước tuần tự
Hybrid	xen SWA mỗi layer thứ hai / 2 layer global attention

11. Kết luận ngắn

DeltaNet là linear attention có "bộ nhớ sửa lỗi", mạnh ở recall. Đóng góp của bài: dùng biểu diễn WY của tích Householder và biến đổi UT để huấn luyện song song theo chiều chuỗi với cùng khuôn chunkwise của linear attention (chỉ thay V bằng $U - WS^\top$). Kết quả: DeltaNet lần đầu ở quy mô LLM, hơn Mamba/GLA; hybrid còn hơn Transformer++. Điểm yếu (thiếu decay → ngoại suy độ dài kém, state nhỏ) dẫn thẳng đến Gated DeltaNet.

Nguồn gốc: https://arxiv.org/abs/2406.06484

Bài 24

Gated Delta Networks: Improving Mamba2 with Delta Rule (2025)

Tác giả: Songlin Yang (MIT CSAIL, thực hiện khi thực tập ở NVIDIA), Jan Kautz (NVIDIA), Ali Hatamizadeh (NVIDIA).
Venue / arXiv: arXiv:2412.06464 (v3, 6/3/2025); ICLR 2025. Code: github.com/NVlabs/GatedDeltaNet.
Nguồn: https://arxiv.org/abs/2412.06464
Chủ đề: Linear attention có trạng thái ma trận; kết hợp *forget gate* (kiểu Mamba2) với *delta rule* (kiểu DeltaNet); thuật toán chunkwise cho gated delta rule; kiến trúc lai với sliding-window attention (SWA) / Mamba2.
Vị trí trong lộ trình: Đọc trước: 18 (Linear Transformers), 19 (RetNet), 20 (GLA), 21 (Mamba), 22 (Mamba-2/SSD), 23 (DeltaNet song song hoá). Bài này giới thiệu Gated DeltaNet và các biến thể lai với attention cửa sổ. Đọc tiếp: 25 (Jamba, hybrid), 26 (Kimi Linear, KDA = GDN với gate theo kênh).

> Ghi chú diễn giải, không phải bản dịch. Công thức đầu ra chunkwise ở mục 3.4 được dẫn lại từ các phương trình vì bản trích xuất PDF có lỗi định dạng.

1. Bối cảnh

Vấn đề của softmax attention. Self-attention chuẩn (bài 01) tốn $O(L^2)$ khi huấn luyện/prefill và phải giữ **KV-cache** (key, value của mọi token trước) có kích thước tăng tuyến tính theo L khi sinh. Với văn bản dài đó là gánh nặng bộ nhớ và băng thông.

Linear attention (bài 18). Bỏ softmax, dùng tích trong thuần $q^\top k$, rồi đổi thứ tự nhân ma trận để viết thành RNN với **trạng thái ma trận** (gọi là *state, fast weight*, hay *bộ nhớ*):

$$S_t = S_{t-1} + v_t k_t^\top \in \mathbb{R}^{d_v \times d_k}, \qquad o_t = S_t q_t \in \mathbb{R}^{d_v}.$$

Kích thước state **không đổi** theo độ dài chuỗi, nên suy luận là $O(1)$ bộ nhớ mỗi bước.

Cách đọc "bộ nhớ liên kết" (associative memory). $S_t = \sum_{i \le t} v_i k_i^\top$ là tổng các tích ngoài (outer product) "key → value". Khi truy vấn bằng q, ta tính $S_t q = \sum_i v_i (k_i^\top q)$: mỗi value được cộng vào với trọng số bằng độ giống giữa key cũ và query. Nếu các key k_i trực giao đơn vị và $q = k_j$ thì $S_t k_j = v_j$, tức đọc lại đúng value đã ghi. Nhưng trong $\mathbb{R}^{d_k}$ chỉ có tối đa d_k hướng trực giao; khi số cặp key–value $> d_k$ thì các key chồng lấn và việc đọc bị nhiễu: paper gọi là **memory collision** (va chạm bộ nhớ), dẫn Schlag et al. (2021).

Hai hướng cải tiến trước bài này:

1. **Gating / decay** (RetNet 19, GLA 20, Mamba 21, Mamba2 22): $S_t = \alpha_t S_{t-1} + v_t k_t^\top$ với $\alpha_t \in (0, 1)$ phụ thuộc dữ liệu. Quên **đều** mọi liên kết cũ theo cùng một tỉ lệ. Quên nhanh được, nhưng không biết *liên kết nào* cần quên.
2. **Delta rule** (DeltaNet, bài 23): trước khi ghi v_t vào khoá k_t, đọc ra giá trị cũ $S_{t-1} k_t$ và **thay** nó, thay vì cộng chồng. Sửa đúng một liên kết, nhưng không có cách *xoá nhanh hàng loạt* (ví dụ khi đổi chủ đề), nên DeltaNet chỉ "khá" trên dữ liệu thật (Yang et al. 2024b).

Quan sát của paper: hai cơ chế **bổ sung** cho nhau. Gate cho phép xoá nhanh/lọc; delta rule cho phép sửa có mục tiêu. Gộp cả hai thành *gated delta rule* và bổ sung thuật toán huấn luyện song song hiệu quả trên GPU, vì delta rule thuần chỉ có kernel tuần tự cho tới bài 23.

---

2. Ý tưởng chính

Hãy coi state S là một **ma trận trọng số nhỏ** học ngay trong lúc đọc văn bản (một "fast weight" ánh xạ key → value). Ở mỗi token, mô hình (i) **hỏi** bộ nhớ "với key này, hiện tại ta đang nhớ giá trị gì?", (ii) so với giá trị mới cần ghi, (iii) sửa bộ nhớ một phần (mức sửa β_t, như learning rate), và (iv) trước đó **làm mờ** toàn bộ bộ nhớ một lượng α_t (như weight decay). Công thức:

$$\boxed{S_t = S_{t-1}\,\alpha_t\big(I - \beta_t k_t k_t^\top\big) + \beta_t v_t k_t^\top}$$

- $\alpha_t \to 0$: xoá sạch bộ nhớ (chuyển ngữ cảnh). $\alpha_t \to 1$: giữ nguyên, chỉ còn delta rule thuần.
- $\beta_t \to 0$: bỏ qua token này. $\beta_t \to 1$: thay hẳn giá trị cũ bằng v_t.

Về hệ thống: tác giả mở rộng thuật toán chunkwise dựa trên biểu diễn WY của Yang et al. (bài 23) để có gate, với chi phí gần như bằng DeltaNet. Kết quả: vượt Mamba2 và DeltaNet trên ngôn ngữ, suy luận thường thức, truy hồi trong ngữ cảnh, ngoại suy độ dài, LongBench; và các **mô hình lai** (với SWA hoặc Mamba2) còn tốt hơn.

---

3. Phương pháp chi tiết

Quy ước ký hiệu (theo paper GDN/DeltaNet): $S_t \in \mathbb{R}^{d_v \times d_k}$ (hàng = chiều value, cột = chiều key); $q_t, k_t \in \mathbb{R}^{d_k}$, $v_t \in \mathbb{R}^{d_v}$; $o_t = S_t q_t$. Bài Kimi Linear (26) dùng chuyển vị $S \in \mathbb{R}^{d_k \times d_v}$, nhớ đối khi so công thức. Nhiều head chạy độc lập, mỗi head có S riêng.

3.1 Nền: linear attention, Mamba2 và dạng song song

Linear attention thuần: $S_t = S_{t-1} + v_t k_t^\top$. Khai triển: $o_t = \sum_{i \le t} v_i (k_i^\top q_t)$; dạng ma trận $O = (QK^\top \odot M)V$ với M là mặt nạ nhân quả. Đó là **dạng song song** (như attention, $O(L^2)$) và **dạng hồi quy** (như RNN, $O(L)$): hai biểu diễn của cùng một hàm.

Mamba2 thêm decay vô hướng: $S_t = \alpha_t S_{t-1} + v_t k_t^\top$. Gọi $\gamma_j = \prod_{i \le j} \alpha_i$ (tích luỹ), thì

$$o_t = \sum_{i \le t} \frac{\gamma_t}{\gamma_i}\, v_i\,(k_i^\top q_t), \qquad O = \big((QK^\top) \odot \Gamma\big)V, \quad \Gamma_{ij} = \tfrac{\gamma_i}{\gamma_j}\ (i \ge j).$$

Sự tương đương hai dạng này chính là "state space duality" (bài 22). Cả hai dạng đều không tối ưu cho GPU, nên người ta dùng dạng **chunkwise**: chia chuỗi thành khối độ dài C (thường 64 hay 128), tính trong-khối bằng dạng song song (nhân ma trận, dùng tensor core) và truyền thông tin giữa các khối bằng state:

$$S_{[t+1]} = \overrightarrow{S}_{[t]} + \overrightarrow{V}_{[t]}^\top K \ \text{(có decay)}, \qquad O_{[t]} = \overleftarrow{Q}_{[t]} S_{[t]}^\top + \big(Q_{[t]} K_{[t]}^\top \odot \Gamma_{[t]}\big) V_{[t]}.$$

Mũi tên trái/phải trong paper = "decay về vị trí đầu / cuối khối".

3.2 Delta rule = một bước SGD trên bộ nhớ

Xem state như tham số của một bộ hồi quy tuyến tính $\hat{v} = Sk$. Với mỗi token, hàm mất mát hồi quy online:

$$\mathcal{L}_t(S) = \tfrac{1}{2}\|Sk_t - v_t\|^2, \qquad \nabla_S \mathcal{L}_t = (Sk_t - v_t)k_t^\top.$$

Một bước SGD với learning rate β_t:

$$S_t = S_{t-1} - \beta_t(S_{t-1}k_t - v_t)k_t^\top = S_{t-1}\big(I - \beta_t k_t k_t^\top\big) + \beta_t v_t k_t^\top.$$

Cách đọc theo "truy hồi key–value" (paper): giá trị cũ $v_t^{\text{old}} = S_{t-1}k_t$; giá trị mới nội suy $v_t^{\text{new}} = \beta_t v_t + (1-\beta_t)v_t^{\text{old}}$; rồi **xoá cũ, ghi mới**:

$$S_t = S_{t-1} - v_t^{\text{old}}k_t^\top + v_t^{\text{new}}k_t^\top.$$

(Kiểm tra: $-v^{\text{old}}k^\top + \beta vk^\top + (1-\beta)v^{\text{old}}k^\top = -\beta v^{\text{old}}k^\top + \beta vk^\top$, đúng bằng $-\beta S_{t-1}kk^\top + \beta vk^\top$.)

Ma trận chuyển trạng thái là Householder tổng quát. $I - \beta kk^\top$ có trị riêng 1 (bội $d_k - 1$) và $1 - \beta\|k\|^2$ (bội 1). Nếu $\|k\|_2 = 1$ (L2-norm, bước bắt buộc trong paper) và $\beta \in (0,1)$ thì trị riêng nằm trong $(0,1]$: hệ **ổn định**. Khi $\beta = 1$, $I - kk^\top$ là **phép chiếu** vuông góc: xoá đúng thành phần của state dọc hướng k, giữ nguyên $d_k - 1$ hướng còn lại (đây là lý do paper nói L2-norm cho "quên có mục tiêu"). Chú thích 2 của paper: cho $\beta \in (0,2)$ thì trị riêng có thể âm, mở khả năng "state tracking"; GDN gốc dùng $\beta \in (0,1)$.

Ví dụ nhỏ ($d_k = d_v = 2$, $\beta = 1$, key đơn vị). $S_0 = 0$.

- Ghi $(k_1 = e_1,\ v_1 = (2,3))$: $S_1 = \begin{pmatrix} 2 & 0 \\ 3 & 0 \end{pmatrix}$.
- Ghi $(k_2 = e_2,\ v_2 = (5,7))$: $S_2 = \begin{pmatrix} 2 & 5 \\ 3 & 7 \end{pmatrix}$. Đọc $S_2e_1 = (2,3)$, $S_2e_2 = (5,7)$: đúng.
- Ghi lại khoá cũ $k_3 = e_1$ với $v_3 = (9,9)$. **Delta rule:** $S_2(I - e_1e_1^\top) = \begin{pmatrix} 0 & 5 \\ 0 & 7 \end{pmatrix}$, cộng $v_3e_1^\top$ được $S_3 = \begin{pmatrix} 9 & 5 \\ 9 & 7 \end{pmatrix}$: ghi đè *chỉ* khoá 1. **Linear attention** cộng chồng: $S_3 = \begin{pmatrix} 11 & 5 \\ 12 & 7 \end{pmatrix}$, đọc e_1 ra $(11,12) = v_1 + v_3$: va chạm.
- Với $\beta = 0.5$ thì đọc e_1 ra $(5.5, 6) = \frac{1}{2}v_1 + \frac{1}{2}v_3$: đúng công thức nội suy.

3.3 Gated delta rule

$$\boxed{S_t = S_{t-1}\,\alpha_t\big(I - \beta_t k_t k_t^\top\big) + \beta_t v_t k_t^\top} \qquad \text{(Eq. 10)}$$

Hai dạng viết tương đương, tiện cho code (đã kiểm tra bằng số):

$$S_t = \alpha_t\Big(S_{t-1} - \beta_t\,(S_{t-1}k_t)\,k_t^\top\Big) + \beta_t\,v_t k_t^\top.$$

Chú ý: α_t nhân cả phần *đã bị delta-sửa* của state, còn số hạng ghi mới $\beta_t v_t k_t^\top$ **không** bị decay. Tiếp tục ví dụ trên với $\alpha_3 = 0.5, \beta_3 = 1$: $S_3 = 0.5\begin{pmatrix} 0 & 5 \\ 0 & 7 \end{pmatrix} + \begin{pmatrix} 9 & 0 \\ 9 & 0 \end{pmatrix} = \begin{pmatrix} 9 & 2.5 \\ 9 & 3.5 \end{pmatrix}$. Liên kết khoá 2 bị mờ đi một nửa; liên kết khoá 1 bị thay hẳn.

Tham số hoá (theo paper): q, k: Linear → ShortConv → SiLU → L2-norm; v: Linear → ShortConv → SiLU; $\beta_t = \sigma(W_\beta x_t)$ (chỉ Linear); $\alpha_t \in (0,1)$ dùng tham số hoá của Mamba2 (paper nói "dùng Mamba2's parameterization", không viết công thức; theo Mamba thì $\alpha_t = \exp(\Delta_t A)$ với $A < 0$ học được và $\Delta_t = \text{softplus}(\cdot)$ phụ thuộc x_t; phần này lấy từ bài 21, không có trong text GDN).

Góc nhìn học online (Bảng 1 paper). Mỗi luật cập nhật là nghiệm dạng đóng của một bài toán online:

Mô hình	Mục tiêu online	Cập nhật
Linear Attn	$\|S_t - S_{t-1}\|_F^2 - 2\langle S_t k_t, v_t\rangle$	$S_{t-1} + v_t k_t^\top$
Mamba2	$\|S_t - \alpha_t S_{t-1}\|_F^2 - 2\langle S_t k_t, v_t\rangle$	$\alpha_t S_{t-1} + v_t k_t^\top$
DeltaNet	$\|S_t - S_{t-1}\|_F^2 - 2\langle S_t k_t, \beta_t(v_t - S_{t-1}k_t)\rangle$	$S_{t-1}(I - \beta_t k_t k_t^\top) + \beta_t v_t k_t^\top$
Gated DeltaNet	$\|S_t - \alpha_t S_{t-1}\|_F^2 - 2\langle S_t k_t, \beta_t(v_t - \alpha_t S_{t-1}k_t)\rangle$	$S_{t-1}\alpha_t(I - \beta_t k_t k_t^\top) + \beta_t v_t k_t^\top$

Cách đọc: số hạng $\|S_t - \alpha_t S_{t-1}\|^2$ kéo state mới về gần state cũ *đã co lại*; α_t nới "neo" ấy ra (khoan dung với việc quên). Cách đọc khác: **SGD với weight decay thích nghi** trên bộ nhớ, trong đó β_t là learning rate thích nghi, α_t là hệ số weight decay thích nghi. (Linear attention dùng mất mát tuyến tính $-\langle Sk, v\rangle$, gradient hằng số, không sửa lỗi; delta rule dùng mất mát bậc hai nên gradient tỉ lệ với sai lệch, "sửa mạnh khi sai nhiều". Đây là giải thích trực giác trong chú thích của bài 23.)

3.4 Thuật toán chunkwise có gate (WY + UT)

Mục tiêu: tính cả khối C token bằng các phép nhân ma trận, không dựng S_t cho từng t. Trong một khối, đặt chỉ số $r = 1..C$, state đầu khối S_0 (=$S_{[t]}$), $\gamma_r = \prod_{i\le r} \alpha_i$ (tích luỹ **bắt đầu lại ở đầu mỗi khối**), $\Gamma_{ij} = \gamma_i/\gamma_j$ với $i \ge j$.

Bước 1: khai triển hồi quy. Lặp Eq. 10:

$$S_r = S_0 \underbrace{\prod_{i=1}^{r} \alpha_i (I - \beta_i k_i k_i^\top)}_{F_r} + \underbrace{\sum_{i=1}^{r} \beta_i v_i k_i^\top \prod_{j=i+1}^{r} \alpha_j (I - \beta_j k_j k_j^\top)}_{G_r}.$$

Bước 2: biểu diễn WY cho tích Householder. Gate là vô hướng nên rút ra được: $F_r = \gamma_r P_r$ với $P_r = \prod_{i\le r}(I - \beta_i k_i k_i^\top)$ giống hệt DeltaNet. Biểu diễn WY (Bischof & Van Loan 1985) cho phép viết $P_r = I - \sum_{i\le r} w_i k_i^\top$ với

$$w_r = \beta_r \Big(k_r - \sum_{i<r} w_i \, (k_i^\top k_r) \Big).$$

Chứng minh nhanh (quy nạp): $P_r = P_{r-1}(I - \beta_r k_r k_r^\top) = P_{r-1} - \beta_r (P_{r-1}k_r) k_r^\top$ và $P_{r-1}k_r = k_r - \sum_{i<r} w_i (k_i^\top k_r)$. □

Với phần ghi G_r (Phụ lục A paper, chứng minh quy nạp): $G_r = \sum_{i\le r} \frac{\gamma_r}{\gamma_i} \tilde{u}_i k_i^\top$, trong đó

$$\tilde{u}_r = \beta_r \Big(v_r - \sum_{i<r} \tilde{u}_i \, \tfrac{\gamma_r}{\gamma_i} \, (k_i^\top k_r) \Big).$$

(Tức là "pseudo-value": giá trị cần ghi sau khi đã trừ đi những gì các token trước *trong cùng khối* đã làm tới hướng k_r, có kể decay.)

Bước 3: UT transform để thay vòng hồi quy bằng một phép giải tam giác (nhân ma trận được). Gom $w_r, \tilde{u}_r$ thành ma trận $W, \tilde{U}$ theo hàng:

$$T = \Big[I + \text{strictLower}\big(\text{diag}(\beta) K K^\top\big) \Big]^{-1} \text{diag}(\beta), \qquad W = TK,$$

$$\tilde{T} = \Big[I + \text{strictLower}\big(\text{diag}(\beta)(\Gamma \odot K K^\top)\big) \Big]^{-1} \text{diag}(\beta), \qquad \tilde{U} = \tilde{T} V.$$

Ma trận cần nghịch đảo là tam giác dưới kích thước $C \times C$ (với $C = 64$ thì rất nhỏ), nên giải bằng thế tiến (forward substitution) chứ không cần nghịch đảo tổng quát. Lưu ý: W **không phụ thuộc gate** (giống DeltaNet), chỉ $\tilde{U}$ có Γ.

Bước 4: cập nhật state cuối khối và đầu ra. Thay vào bước 1 (tự dẫn, đã kiểm tra số), với $\gamma \in \mathbb{R}^C$ là vector tích luỹ:

$$\boxed{S_{[t+1]} = \gamma_C\, S_{[t]} + \left(\tilde{U} - \mathrm{diag}(\gamma)\, W\, S_{[t]}^\top\right)^\top \overrightarrow{K}, \qquad \overrightarrow{K}_i = \tfrac{\gamma_C}{\gamma_i} k_i}$$

$$\boxed{O_{[t]} = \mathrm{diag}(\gamma)\left[Q\, S_{[t]}^\top - \left(QK^\top \odot M\right) W S_{[t]}^\top\right] + \left(QK^\top \odot \Gamma \odot M\right)\tilde{U}}$$

Cách đọc đầu ra: số hạng 1 = đóng góp từ trạng thái đầu khối (đã bị decay γ_r và đã bị các Householder trong khối "sửa" qua W); số hạng 2 = đóng góp từ các token trong khối (như linear attention với $\tilde{U}$ thay cho V). Trong bài báo, state cuối khối viết là $\overrightarrow{S} + (\tilde{U} - \overleftarrow{W} S^\top)^\top \overrightarrow{K}$ với $\overleftarrow{w}_i = \gamma_i w_i$, khớp với dạng trên.

Ý nghĩa "pseudo-value". $\tilde{U} - \mathrm{diag}(\gamma) W S_0^\top$ chính là *vector sửa thực sự* $\beta_i\big(v_i - \alpha\text{-decayed prediction}\big)$ mà DeltaNet sẽ tính nếu chạy tuần tự: "giá trị cần ghi trừ đi cái state đã dự đoán cho key này". Chunkwise tính tất cả C vector sửa cùng lúc từ S_0 bằng nhân ma trận.

Pseudo-code (PyTorch, một head, diễn giải theo công thức trên):

```
def gated_delta_chunk(Q, K, V, beta, log_alpha, S, C=64):
    # Q,K: (L,dk) đã L2-norm; V: (L,dv); beta,log_alpha: (L,); S: (dv,dk)
    L, outs = Q.shape[0], []
    for s in range(0, L, C):
        q,k,v = Q[s:s+C], K[s:s+C], V[s:s+C]; b = beta[s:s+C]
        cum = log_alpha[s:s+C].cumsum(0)                  # log gamma_r
        gam = cum.exp()                                    # gamma_r (<=1)
        Gam = (cum[:,None]-cum[None,:]).exp().tril()       # Gamma_ij = gamma_i/gamma_j,
↪   i>=j
        KK  = k @ k.T
        I   = torch.eye(C)
        T   = torch.linalg.solve_triangular(I + torch.tril(b[:,None]*KK,-1), torch.diag(b),
↪   upper=False)
        Tt  = torch.linalg.solve_triangular(I + torch.tril(b[:,None]*(Gam*KK),-1),
↪   torch.diag(b), upper=False)
        W, U = T @ k, Tt @ v                               # (C,dk), (C,dv)
        M   = torch.tril(torch.ones(C,C))
        inter = gam[:,None] * (q @ S.T - ((q@k.T)*M) @ (W @ S.T))
        intra = ((q@k.T) * Gam) @ U
        outs.append(inter + intra)
        Kr = (gam[-1]/gam)[:,None] * k                     # decay về cuối khối
        S  = gam[-1]*S + (U - gam[:,None]*(W @ S.T)).T @ Kr
    return torch.cat(outs), S
```

Ghi chú ổn định số: chỉ dùng tỉ số γ_i/γ_j với $i \geq j$ (luôn ≤ 1), tính qua hiệu của log-tích luỹ, nên không có phép chia số rất nhỏ. Suy luận từng token dùng kernel hồi quy (mục 3.6), không dùng chunk.

3.5 Khối Gated DeltaNet và các biến thể lai

Khối cơ bản (Hình 1 phải): theo macro-kiến trúc Llama (khối token-mixer + MLP SwiGLU, bài 05, 10, 11). Trong token-mixer: các nhánh q, k, v qua Linear $\rightarrow$ short conv (depthwise, kernel nhỏ) $\rightarrow$ SiLU; q, k thêm

L2-norm; α, β chỉ Linear; sau cổng hồi quy có **output norm** rồi **output gate** (Linear + SiLU) trước phép chiếu đầu ra.

Lai. Linear attention yếu ở so khớp/dịch cục bộ và có state kích thước cố định nên hạn chế truy hồi chính xác (Arora et al. 2024a). Hai biến thể:

- **GatedDeltaNet-H1:** xen **Gated DeltaNet + SWA** (sliding-window attention, attention chỉ nhìn cửa sổ 2K token gần nhất; xem bài 17 Longformer / Mistral 06) theo kiểu Griffin/Samba.
- **GatedDeltaNet-H2:** xếp **Mamba2 + Gated DeltaNet + SWA**.

(Các biến thể hybrid của bài này dùng linear attention kết hợp sliding-window attention.)

3.6 Chi phí tính toán, kích thước state, KV-cache

Công thức (tổng quát, mỗi head, d_k, d_v = chiều head):

Đại lượng	Softmax attention	Gated DeltaNet
Bộ nhớ khi sinh (mỗi layer)	KV-cache $2\, n_{kv}\, d_h\, T$ phần tử	State $H\, d_k\, d_v$ phần tử, **không phụ thuộc** T
FLOPs sinh 1 token	$\approx 4\, T\, d_h\, H_q$ (đọc cả cache)	$\approx 6\, H\, d_k d_v$ (đọc Sk, cập nhật hạng-1, đọc Sq)
FLOPs huấn luyện/prefill	$O(L^2 d)$	chunkwise: $O(LCd + Ld^2)$ (bài 23) cộng việc giải tam giác $C \times C$ mỗi khối (khoảng C^3/khối, tự suy; paper không nêu số cụ thể cho GDN)
Số bước tuần tự	1	L/C (giữa các khối)

Kích thước state không phụ thuộc độ dài chuỗi, còn KV cache tăng theo số token được giữ. So sánh thời gian thực tế cần tính cả các phép chiếu, lưu lượng bộ nhớ và cách cài đặt kernel.

---

4. Thí nghiệm và kết quả chính

Thiết lập. Mọi mô hình 1,3B tham số, 100B token từ FineWeb-Edu, AdamW lr đỉnh 4e-4, weight decay 0,1, clip 1,0, cosine với warm-up 1B token, batch 0,5M token, tokenizer Llama2 (vocab 32.000), độ dài huấn luyện 4K; Samba và các biến thể lai dùng cửa sổ SWA 2K. Baseline: RetNet, HGRN2, Mamba, Mamba2, Samba, DeltaNet, Transformer++.

Bảng 2: S-NIAH (kim trong đống cỏ), 1,3B, zero-shot

Mô hình	S-NIAH-1 (passkey) 1K/2K/4K/8K	S-NIAH-2 (số) 1K/2K/4K/8K	S-NIAH-3 (uuid) 1K/2K/4K
DeltaNet	97,4 / 96,8 / 99,0 / 98,8	98,4 / 45,6 / 18,6 / 14,4	85,2 / 47,0 / 22,4
Mamba2	99,2 / 98,8 / 65,4 / 30,4	99,4 / 98,8 / 56,2 / 17,0	64,4 / 47,6 / 4,6
Gated DeltaNet	98,4 / 88,4 / 91,4 / 91,8	100,0 / 99,8 / 92,2 / 29,6	86,6 / 84,2 / 27,6

Ba quan sát của paper:

1. **Decay làm hại việc giữ trí nhớ** (S-NIAH-1, ngữ cảnh lặp tổng hợp): Mamba2 tụt mạnh sau 2K vì quên lịch sử quá nhanh; DeltaNet gần hoàn hảo; Gated DeltaNet tụt ít hơn Mamba2 nhờ delta rule.
2. **Gate giúp lọc** (S-NIAH-2/3, ngữ cảnh là bài luận thật): state hữu hạn mà không xoá được thì các thông tin chồng lên nhau; DeltaNet tụt mạnh khi dài; Mamba2 và GDN giữ tốt hơn nhờ gate.

3. **Delta rule giúp ghi nhớ mẫu phức tạp** (S-NIAH-3, value là UUID): Mamba2 tụt nhanh, GDN tốt hơn.

Bảng 3: ngôn ngữ và suy luận thường thức (zero-shot, dòng "Avg." là trung bình các benchmark acc; mỗi khối là số trong text, theo setup 1,3B/100B token)

Mô hình	Wiki ppl ↓	LMB ppl ↓	Avg ↑
RetNet	19,08	17,27	52,02
HGRN2	19,10	17,69	51,79
Mamba	17,92	15,06	53,12
Mamba2	16,56	12,56	54,89
DeltaNet	17,71	16,88	52,14
Gated DeltaNet	**16,42**	**12,17**	**55,32**
Transformer++	18,53	18,32	52,25
Samba	16,13	13,29	54,00
GatedDeltaNet-H1	16,07	12,12	**56,40**
GatedDeltaNet-H2	**15,91**	12,55	56,18

Paper nói Gated DeltaNet vượt các mô hình tuyến tính khác ở cả 400M và 1,3B; bảng 400M không có trong text trích.

Bảng 4: truy hồi thực tế, đầu vào cắt 2K token (acc)

Mô hình	SWDE	SQuAD	FDA	TQA	NQ	Drop	Avg
Mamba2	19,1	33,6	25,3	61,0	20,8	19,2	29,8
DeltaNet	17,9	30,9	18,4	53,9	17,3	18,6	26,2
Gated DeltaNet	25,4	34,8	23,7	60,0	20,0	19,8	30,6
Transformer++	29,5	38,0	52,2	58,3	22,5	21,6	37,0
Samba	33,0	39,2	50,5	57,7	23,5	20,2	37,3
GDN-H1	35,6	39,7	52,0	60,1	24,6	22,2	39,0
GDN-H2	38,2	40,4	50,7	63,3	24,8	23,3	40,1

Điểm đáng nhớ: **mô hình thuần tuyến tính (kể cả GDN: FDA 23,7) vẫn thua xa Transformer (FDA 52,2) ở truy hồi chính xác**; chỉ khi lai với attention (H1/H2: FDA 52,0 và 50,7) mới ngang hoặc hơn. FDA = trích cặp key–value từ PDF, rất gần với nhu cầu OCR/trích xuất. Paper cho rằng khoảng cách giữa GDN và Mamba2 ở bảng này nhỏ hơn ở S-NIAH vì các mô hình nhỏ chưa tinh chỉnh hay lặp lại, lỗi này không phụ thuộc luật cập nhật.

Ngoại suy độ dài (Hình 2, tới 20K token; chỉ có đồ thị, không có số trong text): trên sáu tập dài (GovReport, QMSum, NarrativeQA, Qasper, CodeParrot, PG19), Gated DeltaNet có perplexity thấp nhất trong nhóm RNN, kết quả "hỗn hợp" nhưng bền hơn; lai cải thiện thêm. Paper nói rõ "future work sẽ khảo sát chuỗi dài hơn".

Bảng 5: LongBench (trung bình 14 tác vụ)

Mô hình	Avg
RetNet	13,2
HGRN2	13,5
Mamba	14,6
DeltaNet	13,6

Mô hình	Avg
Mamba2	13,5
Gated DeltaNet	**16,6**
Transformer++	11,0
Samba	15,9
GDN-H1	17,8
GDN-H2	**18,4**

Trong các tác vụ riêng: GDN vượt hẳn ở TREC (30,0 vs 13,0 đến 23,1 cho các RNN khác) và SamSum (23,0 vs 8,4-12,8); đây là điểm mạnh được paper gán cho khả năng truy hồi, học trong ngữ cảnh và theo dõi trạng thái.

Throughput huấn luyện (Hình 3, H100, 1,3B): gated delta rule chỉ thêm overhead nhỏ so với delta rule; Gated DeltaNet ≈ DeltaNet; cả hai chậm hơn Mamba2 khoảng 2-3K token/s do ma trận chuyển trạng thái giàu thông tin hơn. Transformer++ nhanh nhất ở ngữ cảnh 2K nhờ FlashAttention-2; các biến thể lai với SWA 2K thì nhanh hơn mixer đơn lẻ; GDN-H1 giữ throughput tốt ở mọi độ dài. Số liệu chính xác chỉ có trên đồ thị.

---

5. Phân tích / ablation

Bảng S.1: ablation khối GDN (400M, 15B token; Avg-PPL thấp hơn tốt hơn, Avg-Acc cao hơn tốt hơn)

Biến thể	Avg-PPL ↓	Avg-Acc ↑
Gated DeltaNet, head dim 128 (mặc định)	27,35	47,26
thay bằng **naive delta rule** (bỏ gate)	30,87	45,12
bỏ short conv	28,95	46,16
bỏ output gate	29,12	45,46
bỏ output norm	27,55	47,07
L1-norm + ReLU	30,79	45,92
L1-norm + 1+ELU	30,34	46,05
L1-norm + SiLU	30,18	46,09
L2-norm + ReLU	27,67	46,94
L2-norm + 1+ELU	27,58	47,17
head dim 64	28,31	46,35
head dim 256	27,13	47,38

Rút ra (theo paper): (1) bỏ gate là mất lớn nhất (PPL 27,35 → 30,87); (2) **short conv** và **output gate** đều quan trọng, output norm ít; (3) **L2-norm thiết yếu**, loại feature map ít ảnh hưởng nhưng SiLU luôn nhỉnh hơn; (4) head dim 128 là điểm cân bằng, 256 nhỉnh hơn một chút (27,13) nhưng tốn hơn.

Bảng S.2: thứ tự xếp các layer lai (500M, 15B token)

Thứ tự	Wiki ppl	LMB ppl	Avg acc
GDN + SWA + Mamba2	24,02	28,20	47,88
GDN + Mamba2 + SWA	23,69	26,83	47,54
Mamba2 + SWA + GDN	24,14	25,21	47,92

Thứ tự	Wiki ppl	LMB ppl	Avg acc
Mamba2 + GDN + SWA	**23,54**	**24,11**	**48,73**

Paper chọn thứ tự Mamba2 → GDN → SWA cho H2; chênh lệch nhỏ (khoảng 1 điểm acc), chỉ ở quy mô 500M/15B token nên không nên coi là quy luật.

Điều paper không thay đổi bằng chứng: không có ablation riêng "chỉ α vô hướng vs theo kênh" (đó là việc của KDA, bài 26), không có thử nghiệm quy mô > 1,3B trong bài này.

---

6. Giới hạn

Do tác giả nêu:

- Quy mô thử nghiệm: 1,3B/100B token; ngoại suy độ dài mới đến 20K, kết quả "hỗn hợp".
- Truy hồi chính xác trong ngữ cảnh vẫn thua Transformer (Bảng 4), cần lai.
- Throughput huấn luyện vẫn thấp hơn Mamba2 (2-3K token/s) do ma trận chuyển giàu cấu trúc.
- Delta rule có giới hạn biểu diễn lý thuyết (Irie et al. 2023); một số hướng mạnh hơn (hồi quy phi tuyến, TTT/Titans, mesa-layer) đánh đổi tính song song. Hạn chế đã biết nhưng paper chỉ đề xuất hướng $\beta \in (0, 2)$ và nhiều tích Householder (chưa thử).

Nhận xét bổ sung:

- Gate α_t là **vô hướng theo head**: mọi kênh key bị quên cùng tốc độ. Kimi Linear (26) chỉ ra điều này là hạn chế và thay bằng gate theo kênh.
- Bài báo so sánh hybrid linear + SWA, không kiểm chứng trực tiếp hybrid linear + full attention toàn cục.
- Mọi huấn luyện đều từ đầu (pretrain). Không có kết quả về chưng cất, cắt tỉa hay fine-tune lớp GDN.
- Ngữ cảnh là văn bản; không có thử nghiệm đa phương thức hay token ảnh.

---

7. Liên hệ với các bài khác

- **18 Linear Transformers:** nguồn gốc dạng hồi quy $S_t = S_{t-1} + v_t k_t^\top$; GDN thêm hai cơ chế sửa/quên lên đó.
- **19 RetNet / 20 GLA / 21 Mamba / 22 Mamba-2:** họ "decay". α_t trong GDN lấy từ Mamba2 (vô hướng theo head); Mamba2 là $\beta = 0$ về mặt cấu trúc (không có số hạng $-\beta kk^\top$). Bài 22 giải thích dạng song song = nhân với mặt nạ semiseparable; GDN không nằm gọn trong khung đó vì ma trận chuyển không phải vô hướng.
- **23 DeltaNet:** nguồn của WY/UT và delta rule; GDN = bài 23 + α_t (thay đổi chủ yếu ở Γ trong $\tilde{T}$).
- **25 Jamba / 26 Kimi Linear:** hybrid ở quy mô lớn; Kimi Linear xây trực tiếp lên GDN (gate theo kênh, tỉ lệ 3:1 với MLA, NoPE). Qwen3-Next/Qwen3.5 là hướng đi tương tự.
- **12 RoPE / 55 StreamingLLM:** GDN không có mã hoá vị trí tường minh; thông tin vị trí đến từ decay và short conv (Kimi Linear 26 phân tích kỹ hơn).
- **44 FlashAttention / 46 PagedAttention:** các layer full attention trong mô hình lai vẫn cần chúng; layer GDN thay KV-cache bằng state kích thước cố định, nên cấp phát bộ nhớ đơn giản hơn.

---

8. Gợi ý áp dụng

Gated DeltaNet kết hợp suy giảm toàn cục và cập nhật delta có chọn lọc. State là bộ nhớ hữu hạn, còn attention trong các biến thể lai của bài báo dùng cửa sổ trượt; kết quả trên tác vụ truy hồi cần được hiểu theo đúng các cấu hình đã khảo sát. Chi phí thực tế phụ thuộc kích thước state, chiều chuỗi và kernel.

9. Thí nghiệm gợi ý

So sánh recurrence với dạng chunkwise trên chuỗi ngắn và kiểm tra tác động của sai số số học. Trên bài toán truy hồi tổng hợp, thay đổi số cặp cần nhớ và kích thước trạng thái; báo cáo riêng các trường hợp cần truy hồi chính xác và các trường hợp cần quên thông tin cũ.

10. Tóm tắt công thức / khái niệm

Khái niệm	Công thức / ý nghĩa
Linear attention	$S_t = S_{t-1} + v_t k_t^\top,\ o_t = S_t q_t$
Mamba2 (gate)	$S_t = \alpha_t S_{t-1} + v_t k_t^\top$
Delta rule	$S_t = S_{t-1}(I - \beta_t k_t k_t^\top) + \beta_t v_t k_t^\top$ = SGD trên $\frac{1}{2}\|Sk - v\|^2$
Gated delta rule	$\boxed{S_t = S_{t-1}\alpha_t(I - \beta_t k_t k_t^\top) + \beta_t v_t k_t^\top}$
Trị riêng chuyển	1 (bội $d_k - 1$), $1 - \beta\|k\|^2$; cần $\|k\| = 1$
WY	$P_r = I - \sum w_i k_i^\top,\ w_r = \beta_r(k_r - \sum_{i<r} w_i k_i^\top k_r)$
UT	$T = [I + \text{strictLower}(\text{diag}(\beta)KK^\top)]^{-1}\text{diag}(\beta)$, $W = TK$
Pseudo-value có gate	$\tilde{U} = \tilde{T}V$, $\tilde{T}$ dùng $\Gamma \odot KK^\top$
State cuối khối	$\gamma_C S + (\tilde{U} - \text{diag}(\gamma)WS^\top)^\top \overrightarrow{K}$
Đầu ra khối	$\text{diag}(\gamma)[QS^\top - (QK^\top \odot M)WS^\top] + (QK^\top \odot \Gamma \odot M)\tilde{U}$
H1 / H2	GDN+SWA / Mamba2+GDN+SWA
Kích thước state	$H\ d_k\ d_v$ mỗi layer, không phụ thuộc T
Memory collision	số liên kết > d_k khiến key chồng lấn, đọc nhiễu

11. Kết luận ngắn

Gated DeltaNet làm hai việc: về thuật toán, đưa gate vô hướng vào delta rule mà vẫn song song hoá được bằng WY + UT với chi phí gần bằng DeltaNet; về mô hình, cho thấy gate (quên nhanh) và delta rule (sửa có mục tiêu) bổ sung nhau, vượt Mamba2 và DeltaNet trên hầu hết bài kiểm tra, và lai với attention còn tốt hơn.

Nguồn gốc: https://arxiv.org/abs/2412.06464

Bài 25

Jamba: A Hybrid Transformer-Mamba Language Model (2024)

Tác giả: Opher Lieber, Barak Lenz, Hofit Bata, Gal Cohen, Jhonathan Osin, Itay Dalmedigos, Erez Safahi, Shaked Meirom, Yonatan Belinkov, Shai Shalev-Shwartz, Omri Abend, Raz Alon, Tomer Asida, Amir Bergman, Roman Glozman, Michael Gokhman, Avshalom Manevich, Nir Ratner, Noam Rozen, Erez Schwartz, Mor Zusman, Yoav Shoham (AI21 Labs; danh sách theo metadata arXiv).
Venue / arXiv: arXiv:2403.19887 (v2, 3/7/2024). Trọng số: huggingface.co/ai21labs/Jamba-v0.1 (Apache 2.0).
Nguồn: https://arxiv.org/abs/2403.19887
Chủ đề: Mô hình lai Transformer-Mamba-MoE ở quy mô production; tỉ lệ attention:Mamba; KV-cache; ablation về cách kết hợp.
Vị trí trong lộ trình: Đọc trước: 21 (Mamba), 22 (Mamba-2), 27 (Switch/MoE) và 28 (Mixtral) giúp hiểu phần MoE; 15 (GQA). Bài này là **ví dụ hybrid lớn đầu tiên** (52B). Đọc tiếp: 24 (GDN), 26 (Kimi Linear, hybrid 3:1).

> Ghi chú diễn giải, không phải bản dịch. Ghi chú tập trung vào kiến trúc và ablation; công thức KV cache ở mục 3.4 là phép tính bổ sung.

1. Bối cảnh

Transformer có hai nhược điểm khi ngữ cảnh dài (paper nêu): (1) **KV-cache** (bộ nhớ lưu key/value của mọi token trước để không tính lại) lớn, thành giới hạn; (2) không có "state tóm tắt" duy nhất nên mỗi token sinh ra phải đọc toàn bộ ngữ cảnh → chậm, throughput thấp. RNN cổ điển có state nhưng khó huấn luyện song song và khó nhớ xa. SSM kiểu **Mamba** (bài 21) huấn luyện hiệu quả và bắt được phụ thuộc xa tốt hơn RNN, nhưng ở quy mô vừa vẫn **kém Transformer cùng cỡ**.

Trước Jamba đã có nhiều thử nghiệm lai attention + SSM (H3, Hyena/StripedHyena, Mamba xen attention ở ≤1,3B...), nhưng đều nhỏ hoặc không bắt kịp Transformer mạnh như Mistral-7B. Paper tuyên bố Jamba là hybrid Attention-SSM "production-grade" đầu tiên.

Khái niệm cần biết.

- **MoE (mixture of experts, bài 27, 28):** thay MLP bằng n MLP (expert) và một *router* chọn top-K expert cho từng token. Tham số tổng tăng, tham số *hoạt động* (active) mỗi token chỉ là K expert.
- **Active vs total parameters:** Jamba tổng 52B nhưng chỉ 12B hoạt động mỗi token. Chi phí tính toán theo active, chi phí bộ nhớ trọng số theo total.

---

2. Ý tưởng chính

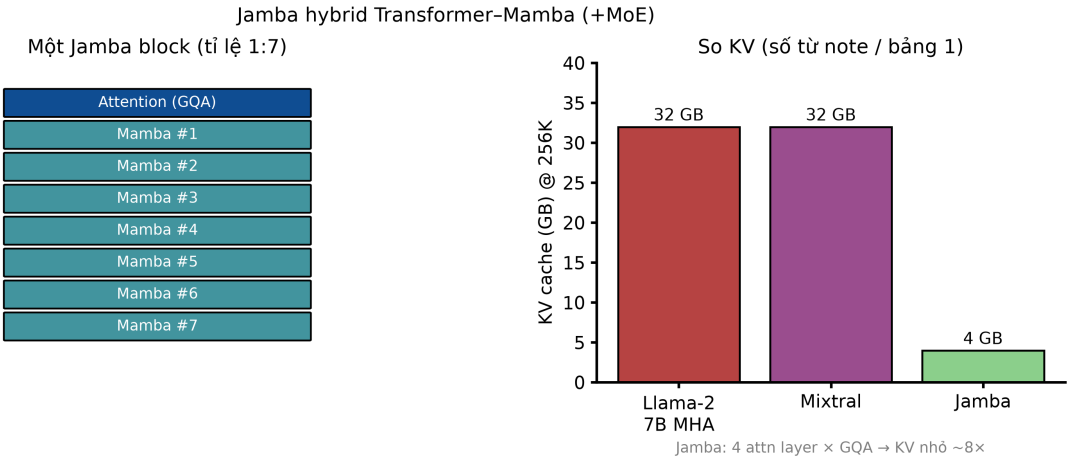
Đan xen các layer Mamba với một số ít layer attention, theo tỉ lệ cố định (**1 attention : 7 Mamba**), cộng MoE ở mỗi layer thứ hai. Mamba xử lý phần lớn các tầng với chi phí $O(L)$ và state cố định; attention vài layer giữ khả năng *truy hồi chính xác* và *học trong ngữ cảnh*. Kết quả: ngang Mixtral-8x7B và Llama-2 70B trên benchmark, KV-cache chỉ 4 GB ở ngữ cảnh 256K (so với 32 GB của Mixtral), throughput gấp 3 lần Mixtral ở ngữ cảnh dài, và chạy vừa một GPU 80 GB.

---

3. Phương pháp chi tiết

3.1 Cấu trúc khối Jamba

Đơn vị cơ bản là **Jamba block**, gồm l layer. Mỗi layer = (một module trộn token: *attention* hoặc *Mamba*) rồi (một module kênh: *MLP* hoặc *MoE*), có RMSNorm (bài 10) và residual. Các bậc tự do:



Jamba 1 attn : 7 Mamba; KV @256K ~4 GB vs ~32 GB Transformer thuần. (minh hoạ, tự tính)

Ký hiệu	Ý nghĩa	Giá trị của Jamba
l	số layer mỗi block	8
$a : m$	tỉ lệ attention : Mamba	1 : 7
e	cứ e layer thì thay MLP bằng MoE	2
n	số expert mỗi layer MoE	16
K	số expert top-K mỗi token	2
số block		4 (tổng 32 layer)

Hệ quả suy ra từ cấu hình: 32 layer = 4 layer attention + 28 layer Mamba; 16 layer dùng MoE, 16 layer dùng MLP thường.

3.2 Các chi tiết thiết kế khác

- Attention dùng **GQA** (bài 15); MLP dùng **SwiGLU** (bài 11); vocab 64K, BPE, mỗi chữ số một token, bỏ "dummy space" của tokenizer Llama/Mistral.
- **Không dùng mã hoá vị trí tường minh** (không RoPE, bài 12). Giả định của tác giả (mục 6.5, từ "presumably"): các Mamba layer đặt trước attention cung cấp thông tin vị trí ngầm; không được chứng minh trực tiếp.
- **RMSNorm trong các phần bên trong của Mamba layer** để ổn định ở quy mô lớn (mục 6.4).
- MoE dùng cân bằng tải (load balancing) chuẩn (bài 27).

3.3 Cấu hình triển khai và lý do

Cấu hình $a : m = 1 : 7$ chọn sau ablation (mục 5): chất lượng tương đương $1 : 3$ nhưng hiệu quả tính toán hơn. Cấu hình MoE ($n = 16, K = 2, e = 2$) chọn sao cho trọng số **int8 vừa một GPU 80 GB** trong khi còn chỗ cho đầu vào; trung bình khoảng 8 expert mỗi layer. Mô hình train trên H100 với framework nội bộ (FSDP, tensor/sequence/expert parallelism); dữ liệu nội bộ (web, sách, code) cập nhật đến 3/2024. Huấn luyện được tới ngữ cảnh 1M token; bản phát hành hỗ trợ 256K.

3.4 Phân tích chi phí (KV-cache, throughput)

Công thức KV-cache (tự tính, độ dài T, n_{attn} layer attention, n_{kv} head KV, d_h chiều head, b byte/phần tử):

$$\boxed{\text{KV} = 2 \cdot n_{attn} \cdot n_{kv} \cdot d_h \cdot T \cdot b}$$

Mamba layer không đóng góp KV-cache (state cố định). Đối chiếu Bảng 1 của paper (256K token, 16-bit, $T = 262{,}144$), nếu giả định 8 KV head × 128 chiều cho Mistral/Jamba và 32 KV head cho Llama-2-7B:

Mô hình	Tham số tổng / active	KV-cache 256K (paper)	Khớp với công thức (tự tính, giả định)
Llama-2	6,7B / 6,7B	128 GB	$32 \cdot 2 \cdot 32 \cdot 128 \cdot 2 \cdot T = 128$ GiB
Mistral	7,2B / 7,2B	32 GB	32 layer, 8 KV head: 32 GiB
Mixtral	46,7B / 12,9B	32 GB	cùng khung attention với Mistral: 32 GiB
Jamba	52B / 12B	**4 GB**	4 layer attention, 8 KV head: 4 GiB

Mức giảm KV của Jamba so với Transformer cùng độ sâu là 8× (= 32/4), đúng bằng tỉ lệ số layer attention (paper cũng nói "8x smaller KV cache"). Lưu ý đây chỉ là KV; Mamba cần thêm state cố định mỗi layer (paper không nêu con số).

Throughput (mục 3.2). Hai kịch bản (A100, kết quả mang tính tương đối vì chưa tối ưu):

1. Một GPU A100 80 GB, int8, ngữ cảnh 8K, sinh 512 token, thay đổi batch: Jamba xử lý được batch lớn, **throughput gấp 3 lần Mixtral** (Mixtral không chứa được batch 16 dù active params tương tự).
2. Batch 1, 4× A100, không lượng tử, thay đổi ngữ cảnh: ở ngữ cảnh ngắn các mô hình tương tự; ở **128K token Jamba nhanh gấp 3 lần Mixtral**; Llama-2 70B không chứa được.

Lý do (paper): ở chuỗi ngắn attention chỉ chiếm phần nhỏ FLOPs, nhưng ở chuỗi dài attention "ngốn" hầu hết tính toán; Mamba hiệu quả hơn, nên tăng tỉ lệ Mamba cải thiện throughput ở chuỗi dài.

Bộ nhớ GPU 80 GB (Hình 2): Jamba chứa được ngữ cảnh dài gấp 2 lần Mixtral và 7 lần Llama-2 70B (kết luận paper: vừa 140K token trên một GPU 80 GB).

---

4. Thí nghiệm và kết quả chính

Benchmark học thuật (Bảng 2; Jamba là base model, chưa alignment).

Mô hình	HellaSwag	WinoGrande	ARC-E	ARC-C	PIQA	NQ	TruthfulQA
Llama-2 13B	80,7	72,8	77,3	59,4	80,5	37,7	37,4
Llama-2 70B	85,3	80,2	80,2	67,3	82,8	46,9	44,9
Gemma	81,2	72,3	81,5	53,2	81,2	32,6	44,8
Mixtral	86,7	81,2	77,6	66	83	44,8	46,8
Jamba	87,1	82,5	73,5	64,4	83,2	45,9	46,4

Mô hình	BoolQ	QuAC	GSM8K	HumanEval	MMLU	BBH
Llama-2 13B	81,7	42,7	34,7	18,3	54,8	39,4
Llama-2 70B	85	42,4	55,3	29,9	69,8	51,2
Gemma	87,2	39,2	54,5	32,3	64,3	55,1

Mô hình	BoolQ	QuAC	GSM8K	HumanEval	MMLU	BBH
Mixtral	88,4	40,9	60,4	34,8	70,6	50,3
Jamba	88,2	40,9	59,9	29,3	67,4	45,4

Nhận xét (paper): ngang Mixtral và Llama-2 70B ở hầu hết tác vụ; chênh rõ ở ARC-E (73,5 vs 77,6), HumanEval (29,3 vs 34,8), BBH (45,4 vs 50,3). Tác giả thận trọng với benchmark ("tương quan một phần với ứng dụng thật").

Ngữ cảnh dài. *Needle-in-a-haystack* (Hình 4): truy hồi tốt đến 256K token dù chỉ có 4 layer attention. *Few-shot phân loại với nhiều ví dụ (tới 128K token)* (Hình 5): Jamba hơn Mixtral ở Trec-Fine và Banking77, ngang ở NLU Intent và CLINC150. *QA ngữ cảnh dài* (Bảng 3, F1, 3-shot, đầu vào trung bình 6K-62K token):

	LongFQA	CUAD	NarrativeQA	NQ	SFiction	Avg
Mixtral	0,42	0,46	0,29	0,58	0,42	0,43
Jamba	0,44	0,44	0,30	0,60	0,40	0,44

5. Phân tích / ablation

5.1 Tỉ lệ attention:Mamba (1,3B, 250B token; Bảng 4; chưa MoE).

Mô hình	OLLM	HellaSwag	WinoGrande	NQ	log-prob C4	Books	Code
Attention thuần	36,4	62,4	59,6	14,5	-0,543	-0,659	-0,331
Mamba thuần	36,1	62,6	59,4	14,5	-0,543	-0,661	-0,334
Jamba 1:3	37,2	65,1	61,7	16,5	-0,533	-0,649	-0,321
Jamba 1:7	37,2	65,1	61,7	16,0	-0,533	-0,650	-0,321

Lai tốt hơn cả hai thuần; **1:3 và 1:7 gần như không khác nhau** → chọn 1:7 vì rẻ hơn. (Lưu ý: "Mamba thuần" ở đây là Mamba xen MLP.) Ở 7B, 50B token (Bảng 5): Attention OLLM 36,1; Mamba 35,3; Jamba (1:7, không MoE) 36,6; lai cũng có loss huấn luyện thấp hơn và throughput hơn Transformer thuần.

5.2 Vì sao lai hiệu quả: học trong ngữ cảnh (ICL). Mamba thuần có perplexity tốt nhưng **kém rõ ở IMDB, QuAC, NarrativeQA** (Bảng 6, 1,3B 250B token):

	IMDB	QuAC	NarrativeQA
Attention	84,1	27,9	45,8
Mamba	48,8	20,2	27,7
Attention-Mamba	90,9	26,6	43,7

Phân tích của tác giả: Mamba thuần thường **không theo đúng định dạng** đáp án (ví dụ trả "Very Good", "3/10" thay vì "Positive/Negative"), gợi ý khó khăn trong in-context learning (ICL). Giả thuyết: ICL liên quan đến *induction heads* (đầu attention sao chép xấp xỉ) ở Transformer; Mamba thiếu attention nên khó phát triển cơ chế này như năng lực nổi lên. Mô hình lai làm ICL tốt dù chỉ 1/8 layer là attention. Bằng chứng giai thoại: Hình 8 cho thấy một head attention từ mô hình lai 1,3B tập trung vào các nhãn của few-shot; tìm được

12 head như vậy ở cả ba layer attention (layer 4, 12, 20). Đây là **giả thuyết và bằng chứng giai thoại**, không phải chứng minh.

5.3 Ảnh hưởng của MoE (Bảng 7, 7B, 50B token). Jamba không MoE: OLLM 36,6, HellaSwag 62,5, NQ 15,4; **Jamba+MoE** 38,1, 66,0, 18,9 (log-prob C4/Books/Code cũng cải thiện): MoE có lợi cho hybrid ở quy mô lớn.

5.4 Ổn định (mục 6.4). Khi mở rộng đến 7B-based (12B/52B) xuất hiện **loss spike**: giá trị kích hoạt lớn bên trong layer Mamba. Thêm RMSNorm vào các kích hoạt nội bộ của Mamba loại bỏ spike (Hình 9). Cùng hướng với việc Mamba-2 (22) thêm một lớp norm trước phép chiếu ra.

5.5 Không cần mã hoá vị trí (Bảng 8, 1,3B, 250B token, có MoE).

	OLLM	HellaSwag	WinoGrande	ARC-C	NarrativeQA	NQ	BoolQ
Jamba	39,6	71,5	64,2	40,7	50,5	22,2	68,9
Jamba + RoPE	40,1	71,8	65,5	40,4	46,2	22,2	67,9

Không có khác biệt đáng kể (NarrativeQA thậm chí tốt hơn khi không RoPE). Lý giải của tác giả: Mamba layer cung cấp thông tin vị trí ngầm. (Kimi Linear, bài 26, đi tới kết luận NoPE cho layer full attention của hybrid, với giải thích dựa trên vai trò của decay/gate.)

---

6. Giới hạn

Do tác giả nêu:

- Chỉ là base model, chưa alignment/instruction tuning, chưa có moderation: không dùng sản xuất.
- Throughput đo "tương đối", Jamba chưa có tối ưu như Transformer sau 6 năm.
- Các ablation ở quy mô tới 7B/250B token; kết luận "ICL của Mamba thuần kém" chỉ là giả thuyết, cần nghiên cứu tiếp ở quy mô lớn.
- Cẩn trọng với benchmark nói chung.

Nhận xét bổ sung:

- Không có ablation "vị trí" của 4 layer attention trong 32 layer (chỉ nói layer 4, 12, 20 có induction head ở bản 1,3B); và không so sánh tỉ lệ ngoài {1:3, 1:7} (Mamba-2 ở bài 22 có đường cong chi tiết hơn).
- Dùng Mamba-1 (selective scan, state nhỏ, không có delta rule) làm layer tuyến tính; GDN/KDA (24, 26) là lựa chọn mạnh hơn về recall.
- Throughput so với Mixtral, còn thiếu so với một Transformer cùng độ sâu có GQA mạnh tương đương (ngoài Mixtral/Llama-2).
- Báo cáo không có phân tích đầy đủ về **kích thước state của Mamba** (phần bộ nhớ cố định), chỉ nhấn mạnh KV.

---

7. Liên hệ với các bài khác

- **21 Mamba / 22 Mamba-2:** Jamba dùng Mamba-1. Bài 22 (mục 9.2.3) độc lập đưa ra kết luận "khoảng 10% layer attention là tốt nhất" (Jamba: 4/32 = 12,5%), hai bằng chứng cùng hướng.
- **15 GQA / 16 MLA:** giảm KV theo *chiều head*; Jamba giảm KV theo *số layer*. Hai trục trực giao và có thể cộng dồn (Jamba đã dùng GQA).

- **27 Switch / 28 Mixtral:** MoE; Jamba so sánh trực tiếp với Mixtral.
- **24 Gated DeltaNet / 26 Kimi Linear:** thế hệ sau thay Mamba bằng layer có delta rule và tăng tỉ lệ tuyến tính lên 3:1 (Kimi) hoặc dùng trong LLM thực tế. Tỉ lệ 1:7 của Jamba là nhẹ attention hơn.
- **12 RoPE:** Jamba bỏ RoPE hoàn toàn; Kimi Linear bỏ RoPE chỉ ở layer full attention của hybrid.
- **46 PagedAttention / 55-56 (KV-cache):** các kỹ thuật quản lý KV vẫn cần cho 4 layer attention, nhưng KV nhỏ đi 8×.

---

8. Gợi ý áp dụng

Jamba kết hợp tầng Mamba, attention và MoE. Ablation trong bài cho thấy vai trò của các thành phần này đối với các tác vụ đã đo; tỉ lệ attention phù hợp không nhất thiết giống nhau ở kiến trúc hay dữ liệu khác. Khi ước lượng bộ nhớ, cần cộng cả trạng thái Mamba, KV cache, trọng số và các bộ đệm thực thi.

9. Thí nghiệm gợi ý

So sánh các tỉ lệ tầng attention trên cùng ngân sách và dữ liệu, theo dõi cả perplexity lẫn truy hồi và làm theo định dạng. Đo bộ nhớ và thông lượng theo chiều dài ngữ cảnh, đồng thời nêu rõ batch và cách triển khai.

10. Tóm tắt công thức / khái niệm

Khái niệm	Ý nghĩa
Jamba block	$l = 8$ layer, tỉ lệ $a : m = 1 : 7$, MoE mỗi $e = 2$ layer, $n = 16$ expert, top-$K = 2$
Tổng cộng	4 block = 32 layer: 4 attention + 28 Mamba; 52B tổng, 12B active
KV-cache	$2\, n_{attn}\, n_{kv}\, d_h\, T\, b$; Mamba layer không có KV
So sánh KV 256K	Llama-2 128 GB, Mistral/Mixtral 32 GB, Jamba 4 GB
Throughput	3× Mixtral ở 128K token (4 A100) và ở batch lớn (1 A100, 8K)
Hybrid tốt hơn thuần	Bảng 4-5: lai > attention thuần, lai > Mamba thuần
ICL ở Mamba thuần	Bảng 6: IMDB 48,8 vs 84,1 (Attention) vs 90,9 (lai)
Không cần vị trí tường minh	Bảng 8: RoPE không giúp
Ổn định	thêm RMSNorm trong Mamba để hết loss spike

11. Kết luận ngắn

Jamba chứng minh hybrid Transformer-Mamba-MoE có thể sánh với Mixtral và Llama-2 70B, với KV-cache nhỏ hơn 8× (4 GB ở 256K) và throughput cao gấp 3× ở ngữ cảnh dài, chỉ với 1/8 layer là attention. Bài học thiết kế: tỉ lệ 1:3 và 1:7 như nhau; Mamba thuần thiếu ICL; MoE giúp; không cần mã hoá vị trí; cần norm bổ sung để ổn định.

Nguồn gốc: https://arxiv.org/abs/2403.19887

Bài 26

Kimi Linear: An Expressive, Efficient Attention Architecture (2025)

Tác giả: Kimi Team (Yu Zhang, Zongyu Lin, Xingcheng Yao, Jiaxi Hu, Fanqing Meng, Chengyin Liu, Xin Men, Songlin Yang, ... Zhilin Yang, Yulun Du và nhiều người khác; danh sách đầy đủ trong paper).
Venue / arXiv: arXiv:2510.26692 (v2, 1/11/2025); technical report. Code/kernel: `fla-org/flash-linear-attention` (fla/ops/kda); checkpoint `moonshotai/Kimi-Linear-48B-A3B-Instruct`.
Nguồn: https://arxiv.org/abs/2510.26692
Chủ đề: Kimi Delta Attention (KDA) = Gated DeltaNet với gate theo từng kênh; hybrid 3:1 KDA:MLA; NoPE cho layer full attention; so sánh công bằng với full attention.
Vị trí trong lộ trình: Đọc trước: 20 (GLA), 23 (DeltaNet), 24 (Gated DeltaNet), 16 (MLA), 25 (Jamba). Đây là **hậu duệ trực tiếp của GDN ở quy mô 48B**; đọc tiếp: chuyển sang phần MoE/VLM (27 trở đi).

> Ghi chú diễn giải, không phải bản dịch. Ký hiệu dùng $S \in \mathbb{R}^{d_k \times d_v}$ và $o_t = S_t^\top q_t$, chuyển vị so với quy ước của bài 23–24.

1. Bối cảnh

Suy luận LLM ngày càng nặng ở các tác vụ **dài**: agent (nhiều lượt dùng công cụ), RL với thời gian suy luận kéo dài (test-time scaling), code cấp kho. Softmax attention có chi phí bậc hai và KV-cache tuyến tính theo độ dài → nút thắt.

Linear attention (bài 18) có state cố định nhưng từng yếu hơn softmax attention. Hai cải tiến thu hẹp khoảng cách: **gate/decay** (RetNet, GLA, Mamba2) và **delta rule** (DeltaNet 23, Gated DeltaNet 24). Tuy vậy, state hữu hạn vẫn hạn chế truy hồi/chép chính xác, nên các mô hình **lai** (vài layer attention toàn cục + nhiều layer tuyến tính) trở thành thoả hiệp thực tế (Jamba 25, Mamba-2 hybrid 22, ...). Lỗ hổng mà paper nhắm tới: các hybrid trước đó thường ở quy mô nhỏ hoặc thiếu đánh giá toàn diện; chưa có bằng chứng "vượt full attention trong so sánh công bằng" trên ngắn, dài và RL.

Khái niệm nền nhắc nhanh.

- **Gate theo head (vô hướng)** như Mamba2/GDN: $\alpha_t \in (0,1)$ một số cho cả state của head: mọi kênh nhớ cùng tốc độ.
- **Gate theo kênh (channel-wise)** như GLA: $\alpha_t \in (0,1)^{d_k}$, mỗi chiều key có tốc độ quên riêng, giống cách RoPE gán mỗi cặp chiều một tần số khác nhau.
- **DPLR (diagonal-plus-low-rank):** ma trận chuyển $D - ab^\top$ (đường chéo trừ hạng-1), tổng quát hoá cả gate theo kênh và delta rule, nhưng tính chunkwise đắt.

---

2. Ý tưởng chính

1. **KDA (Kimi Delta Attention):** lấy gated delta rule của GDN và thay gate vô hướng α_t bằng **gate đường chéo theo kênh** $\mathrm{Diag}(\alpha_t)$:

$$\boxed{S_t = \left(I - \beta_t k_t k_t^\top\right) \mathrm{Diag}(\alpha_t)\, S_{t-1} + \beta_t k_t v_t^\top, \qquad o_t = S_t^\top q_t}$$

   Mỗi kênh key tự quyết mức giữ/quên; phần còn lại giống GDN (ghi đè kiểu delta rule).

2. **Biến thể DPLR bị ràng buộc:** bằng cách buộc hai vector "hạng thấp" a, b đều theo k ($a = \beta k$, $b = k \odot \alpha$), kernel chunkwise bớt phép nhân và tránh vấn đề số, nhanh gần **2×** so với DPLR tổng quát.
3. **Kimi Linear:** hybrid layer-wise, **3 layer KDA : 1 layer MLA (full attention)**, MoE, và layer MLA **không dùng mã hoá vị trí (NoPE)**, để KDA đóng vai trò mã hoá vị trí/recency. Kết quả (so sánh công bằng, cùng 1,4T token, cùng kích thước 48B tổng/3B active): vượt MLA toàn phần trên ngắn, dài và RL, giảm **tới 75% KV-cache** và tăng **tới 6,3×** tốc độ giải mã ở ngữ cảnh 1M.

---

3. Phương pháp chi tiết

3.1 Từ GDN đến KDA

Theo paper (ký hiệu $S \in \mathbb{R}^{d_k \times d_v}$):

Mô hình	Cập nhật
Linear attention	$S_t = S_{t-1} + k_t v_t^\top$
DeltaNet	$S_t = (I - \beta_t k_t k_t^\top) S_{t-1} + \beta_t k_t v_t^\top$
GDN (scalar gate)	$S_t = \alpha_t (I - \beta_t k_t k_t^\top) S_{t-1} + \beta_t k_t v_t^\top$
KDA	$S_t = (I - \beta_t k_t k_t^\top)\,\mathrm{Diag}(\alpha_t)\,S_{t-1} + \beta_t k_t v_t^\top$

(Nếu chuyển vị sang quy ước bài 24 $S' = S^\top$: $S'_t = S'_{t-1}\mathrm{Diag}(\alpha_t)(I - \beta_t k_t k_t^\top) + \beta_t v_t k_t^\top$.) Chú ý **thứ tự**: trước hết decay theo kênh ($\mathrm{Diag}(\alpha_t) S_{t-1}$), rồi mới áp phép chiếu delta, khác một chút so với GDN, nơi α vô hướng nên giao hoán với mọi thứ.

Cách đọc bộ nhớ liên kết. Mỗi *hàng* $S[d, :]$ ứng với một chiều key d. Gate theo kênh quên hàng d với hệ số $\alpha_t[d]$; phép chiếu delta rồi sửa lỗi giữa các hàng (trộn các chiều key bằng $k_t k_t^\top$). Tức là: các chiều key có thể giữ thông tin lâu mỏng khác nhau (một số "nhớ dài", một số "nhớ ngắn"), chống va chạm tốt hơn một gate chung.

Góc nhìn học online (Bảng 7 paper). Với GDN và KDA, cập nhật là một bước SGD trên bộ nhớ *đã bị decay* $\tilde{S}$ (decay vô hướng cho GDN, theo kênh cho KDA) với mất mát $\frac{\beta_t}{2}\|\tilde{S}_{t-1}^\top k_t - v_t\|^2$. Nghĩa là: "weight decay thích nghi" rồi "một bước hồi quy".

Quan hệ với DPLR. KDA viết lại thành $S_t = (\mathrm{Diag}(\alpha_t) - \beta_t k_t k_t^\top \mathrm{Diag}(\alpha_t)) S_{t-1} + \beta_t k_t v_t^\top$, tức dạng $D - a_t b_t^\top$ với $D = \mathrm{Diag}(\alpha_t)$, $a_t = \beta_t k_t$, $b_t = k_t \odot \alpha_t$. So với DPLR tổng quát (như RWKV-7, S4), KDA **gắn** a **và** b **với** k (và chia sẻ α để rút ra ngoài), giảm số tham số/phép tính và vẫn nhất quán với delta rule cổ điển.

3.2 Dẫn xuất chunkwise

Mục tiêu giống 23/24: tính cả khối C token bằng nhân ma trận. Ký hiệu trong khối: $\gamma^r = \prod_{i \le r} \alpha^i$ là vector d_k chiều (tích luỹ theo kênh), $\Gamma^{1 \to r}$ ghép chúng thành ma trận $C \times d_k$, và $\gamma^{i \to r} = \prod_{j=i}^{r} \alpha^j$.

Khai triển (Eq. 2): $S^r = P^r S^0 + H^r$ với $P^r = \prod_{i=1}^{r} (I - \beta_i k_i k_i^\top)\mathrm{Diag}(\alpha_i)$.

WY có gate (Eq. 3-5; theo Comba để khỏi phải thêm nghịch đảo):

$$P^r = \mathrm{Diag}(\gamma^r) - \sum_{i=1}^{r} \mathrm{Diag}(\gamma^{i \to r})\, k_i\, w_i^\top, \qquad H^r = \sum_{i=1}^{r} \mathrm{Diag}(\gamma^{i \to r})\, k_i\, u_i^\top,$$

$$w_r = \beta_r \Big(\mathrm{Diag}(\gamma^r) k_r - \sum_{i<r} w_i \left(k_i^\top \mathrm{Diag}(\gamma^{i \to r}) k_r \right) \Big), \quad u_r = \beta_r \Big(v_r - \sum_{i<r} u_i \left(k_i^\top \mathrm{Diag}(\gamma^{i \to r}) k_r \right) \Big).$$

Điểm khác GDN: tích vô hướng giữa hai key là $k_i^\top \mathrm{Diag}(\gamma^{i\to r}) k_r = \sum_d k_i[d]\, k_r[d]\, \gamma^{i\to r}[d]$, **có trọng số theo kênh**, nên không rút γ ra ngoài thành một tỉ số vô hướng được.

UT transform (Eq. 6-7):

$$M = \Big[I + \mathrm{StrictTril}\Big(\mathrm{Diag}(\beta)\,(\Gamma^{1\to C}\odot K)\,\big(K/\Gamma^{1\to C}\big)^\top\Big)\Big]^{-1}\mathrm{Diag}(\beta), \quad W = M(\Gamma^{1\to C}\odot K),\ U = MV.$$

(phép chia K/Γ theo từng phần tử.) Vẫn là nghịch đảo tam giác $C\times C$ giải bằng thế tiến.

State cuối khối và đầu ra (Eq. 8-9):

$$\boxed{S_{[t+1]} = \mathrm{Diag}(\gamma^C)\, S_{[t]} + \big(\Gamma^{i\to C}\odot K\big)^\top\big(U - W\, S_{[t]}\big)}$$

$$\boxed{O_{[t]} = \underbrace{(\Gamma^{1\to r}\odot Q)\, S_{[t]}}_{\text{liên khối}} + \underbrace{\mathrm{Tril}\Big((\Gamma^{1\to r}\odot Q)\big(K/\Gamma^{1\to r}\big)^\top\Big)\big(U - W\, S_{[t]}\big)}_{\text{trong khối (attention trên "pseudo-value")}}}$$

Ở đây $\Gamma^{i\to C}\odot K$ là K đã được decay tới cuối khối. Ý nghĩa giống bài 24: $U - WS$ là vector sửa lỗi thật cho cả khối (giá trị cần ghi trừ giá trị state đã dự đoán), tính một lần từ $S_{[t]}$. Hai điểm khác GDN: (i) mọi γ là vector theo kênh, (ii) W ở đây đã **gồm cả decay** (nên trong công thức không còn $\mathrm{diag}(\gamma)W$ như ở bài 24 mà là W trực tiếp).

Pseudo-code (tóm tắt Listing 1(b), một head; NumPy-style):

```
def chunk_kda(Q, K, V, beta, g, S, C=64):       # g = log(alpha) (L,dk); S (dk,dv); Kimi
↪  convention
    for chunk in chunks(C):
        gc = g[chunk].cumsum(0)                          # (C,dk) log gamma^{1->r}
        Aqk[i,j] = (Q[i] * K[j] * exp(gc[i]-gc[j])).sum(-1)   # j<=i
        Akk[i,j] = (K[i] * K[j] * exp(gc[i]-gc[j])).sum(-1)   # j<i
        A = solve_tri(I + beta[:,None]*Akk)              # UT: (I + StrictTril(...))^-1
        A = A * beta[None,:]
        W = A @ (exp(gc) * K);  U = A @ V
        O = (Q*exp(gc)) @ S + tril(Aqk) @ (U - W @ S)
        S = S*exp(gc[-1])[:,None] + (K*exp(gc[-1]-gc)).T @ (U - W @ S)   # dùng S cũ
```

Điều đáng chú ý cho kỹ sư: các hệ số đều dạng $\exp(\text{hiệu của log-tích luỹ})$ với chỉ số lớn trừ chỉ số nhỏ $\to \le 1$, ổn định; không chia cho số rất nhỏ.

3.3 Vì sao KDA nhanh hơn DPLR tổng quát

Khi decay theo kênh, thành phần $1/\Gamma$ trong dạng chunkwise (xuất hiện trong K/Γ) có thể gây **mất ổn định số** (chia số nhỏ). GLA giải bằng tính trong miền log và *chunk cấp hai ở độ chính xác đầy đủ* → không dùng được nhân ma trận nửa độ chính xác, chậm. DPLR tổng quát còn cần thêm các ma trận A_{ab}, A_{ak}, A_{qb} (bốn ma trận chunk cấp hai). KDA đặt $a = b = k$ nên:

- số phép tính chunk cấp hai giảm từ **4 xuống 2**;
- loại thêm ba phép nhân ma trận ở khâu liên khối và đầu ra;
- hiệu quả toán tử tăng khoảng **100%** (≈2×) so với DPLR (Hình 2: kernel nhanh gần 2× đến 64K token; batch 1, 16 head).

3.4 Kiến trúc Kimi Linear

Nền: kiến trúc **Moonlight** (MoE kiểu DeepSeek), thay token-mixer.

Tham số hoá lớp KDA (mỗi head h, $d_k = d_v = 128$):

$$q_t^h, k_t^h = \text{L2Norm}\big(\text{Swish}(\text{ShortConv}(W_{q/k}^h x_t))\big), \qquad v_t^h = \text{Swish}(\text{ShortConv}(W_v^h x_t)),$$

$$\alpha_t^h = f\big(W_\alpha^\uparrow W_\alpha^\downarrow x_t\big) \in [0,1]^{d_k}, \qquad \beta_t^h = \text{Sigmoid}(W_\beta^h x_t),$$

$$o_t = W_o\Big(\text{Sigmoid}(W_g^\uparrow W_g^\downarrow x_t) \odot \text{RMSNorm}\big(\text{KDA}(q_t, k_t, v_t, \alpha_t, \beta_t)\big)\Big).$$

- α: **chiếu hạng thấp** (rank = head dim), hàm f giống GDN/Mamba (decay dạng mũ).
- **Output gate sigmoid, hạng thấp** (ablation: sigmoid hơn swish rõ rệt; có gate hơn không gate; paper nói gate còn giúp giảm *attention sink*).
- Head-wise RMSNorm trước gate/projection.

Hybrid: các block gồm một token-mixer rồi MoE. Chọn **theo layer** (không theo head) vì đơn giản hạ tầng và ổn định huấn luyện; tỉ lệ **đều 3:1**: 3 layer KDA rồi 1 layer MLA (Multi-head Latent Attention, bài 16). Layer MLA dùng **NoPE** (không RoPE): KDA chịu trách nhiệm mã hoá vị trí và độ ưu tiên gần đây. Lợi ích NoPE (paper): (i) chuyển MLA sang MQA thuần khi suy luận (hiệu quả, vì không còn RoPE chen vào phép hấp thụ ma trận); (ii) mở rộng ngữ cảnh đơn giản, khỏi chỉnh tần số RoPE/YaRN.

Cấu hình mô hình chính (mục 5.4): 48B tham số tổng, 3B active; MoE 256 expert, 8 active (kể cả 1 shared); thưa 32; layer đầu dense. Pretrain 4096 token, optimizer MuonClip, lịch WSD, batch 32M token, 1,4T token cho so sánh; phiên bản phát hành 5,7T token, ngữ cảnh tới 1M.

3.5 KDA như mã hoá vị trí học được

Phép nhân tích luỹ các ma trận chuyển hoá cho thấy tính chất "vị trí" của layer tuyến tính. Thế attention tổng quát: $s_{t,i} = q_t^\top \big(\prod_{j=i+1}^{t} R_j\big) k_i$. RoPE chọn R_j là ma trận xoay khối chéo (trực giao) nên chỉ phụ thuộc $t - i$. Với gated delta rule:

$$o_t = \sum_{i \le t} q_t^\top \Big(\prod_{j=i+1}^{t} \alpha_j \big(I - \beta_j k_j k_j^\top\big)\Big) k_i \, v_i \quad \text{(Eq. 12)},$$

nghĩa là GDN cũng là mã hoá vị trí *nhân*, nhưng ma trận chuyển **phụ thuộc dữ liệu** và **không bị ràng buộc trực giao** (do đó có thể mạnh hơn và hy vọng ngoại suy độ dài tốt hơn RoPE có tần số cố định). RoPE gán mỗi cặp chiều một tần số khác nhau (như một biến đổi Fourier không đều theo chiều đặc trưng); GDN chỉ có một decay chung. **KDA, với gate theo kênh, khôi phục sự đa dạng "theo chiều" đó**, và vì vậy được kỳ vọng làm vai trò vị trí tốt hơn, là lý do cho thiết kế NoPE ở MLA.

3.6 Chi phí tính toán

FLOPs huấn luyện cho một head (paper, Eq. 13-14; chunk $C = 64$):

$$\text{FLOPs}_{\text{KDA}}(T) = 6Td_h^2 + 3TCd_h + TC^2, \qquad \text{FLOPs}_{\text{Attn}}(T) = 2T^2 d_h.$$

Ví dụ số (tự tính, $d_h = 128$, $C = 64$): hệ số KDA $= 6{\cdot}16384 + 3{\cdot}64{\cdot}128 + 64^2 = 126{,}976$ FLOPs/token/head.

Độ dài T	KDA (mỗi head)	Attention $2T^2 d_h$	Tỉ lệ
2.048	0,26 GFLOPs	1,07 GFLOPs	≈4,1×
4.096	0,52 GFLOPs	4,29 GFLOPs	≈8×
65.536	8,3 GFLOPs	1.100 GFLOPs	≈132×

Điểm hoà vốn $126{,}976\,T = 256\,T^2$ cho $T \approx 500$: ngắn hơn 500 token thì attention thuần rẻ hơn về FLOPs (chưa kể hiệu quả kernel và hằng số causal 1/2, nên chỉ là mức độ).

Suy luận. Prefill: kernel chunkwise (mục 3.2); giải mã từng token: kernel hồi quy (Eq. 2). State KDA cố định $d_k \times d_v = 128 \times 128$ mỗi head, bất kể độ dài. Trong hybrid 3:1, chỉ 1/4 layer có KV-cache → **giảm tới 75% KV** (bằng tỉ lệ layer trong phép tính này). Khi giải mã bị chặn bởi I/O, thời gian tiệm cận chỉ còn 1/4 chi phí đọc KV so với full attention (paper: "hybrid efficiency ratio tối đa 3:1"). Bộ nhớ KV giảm còn cho phép batch lớn hơn → thông lượng cao hơn.

---

4. Thí nghiệm và kết quả chính

Bài tổng hợp (mục 5.1, Hình 4). Mô hình 2 layer, 2 head, head dim 128, tối đa 20.000 bước. *Palindrome* (sao chép ngược chuỗi), *MQAR* (truy hồi liên kết đa truy vấn), *Stack* (theo dõi trạng thái LIFO, 64 ngăn xếp). KDA đạt độ chính xác cao nhất ở mọi độ dài 256 đến 2.048 token và hội tụ nhanh hơn GDN trên Palindrome và MQAR; **Mamba2 thất bại cả ba** trong thiết lập này (chỉ có đồ thị, không có số).

Quy luật scaling (mục 5.3, Bảng 2, Hình 5). Năm mô hình MoE active từ 653M đến 1,7B (8/64 expert, Muon, 4096 ngữ cảnh), lượng token 38,8B đến 128B. Hồi quy:

- MLA: $2{,}3092 \times C^{-0{,}0536}$; Kimi Linear: $2{,}2879 \times C^{-0{,}0527}$ (C = PFLOP/s-ngày).
- Kimi Linear đạt **~1,16× hiệu quả tính toán** so với MLA ở compute-optimal (paper cho rằng với tinh chỉnh siêu tham số sẽ còn tốt hơn).

Kết quả pretrain 1,4T token (Bảng 3; cùng recipe, cùng kích thước):

Benchmark	MLA	GDN-H	Kimi Linear
HellaSwag	81,7	82,2	**82,9**
ARC-challenge	64,6	66,5	**67,3**
Winogrande	78,1	77,9	**78,6**
BBH	71,6	70,6	**72,9**
MMLU	71,6	72,2	**73,8**
MMLU-Pro	47,2	47,9	**51,0**
TriviaQA	68,9	70,1	**71,7**
GSM8K	83,7	81,7	**83,9**
MATH	54,7	54,1	54,7
EvalPlus	59,5	**63,1**	60,2
CRUXEval-I-cot	51,6	56,0	**56,6**
CRUXEval-O-cot	61,5	58,1	**62,0**
CEval	79,3	79,1	**79,5**
CMMLU	79,5	80,7	**80,8**

(GDN-H = hybrid GDN:MLA cùng tỉ lệ, làm baseline "hybrid mạnh".)

Sau SFT (Bảng 4): Kimi Linear dẫn đầu ở hầu hết: ví dụ MMLU 77,0 (MLA 75,7; GDN-H 75,6), MMLU-Pro 67,4 (65,7; 64,8), GPQA-Diamond 62,1 (57,1; 58,6), AIME 2025 21,3 (20,6; 21,1), LiveCodeBench v6 26,0 (25,1; 25,4). Ngoại lệ: MATH500 (81,2 vs GDN-H 83,0), EvalPlus (61,0 vs 62,6 MLA), LiveBench (45,2 vs 46,4 GDN-H).

Ngữ cảnh dài 128K (Bảng 5; cùng 1,4T token):

	RULER	MRCR	HELMET ICL	LongBench v2	Frames	RepoQA	LCA-Lib	LCA-Commit	Avg
MLA	81,3	22,6	88,0	36,1	60,5	63,0	32,8	33,2	52,2
GDN-H	80,5	23,9	85,5	32,6	58,7	63,0	34,7	30,5	51,2
Kimi Linear (RoPE)	78,8	22,0	88,0	35,4	59,9	66,5	31,3	32,5	51,8
Kimi Linear	**84,3**	**29,6**	**90,0**	35,0	58,8	**68,5**	**37,1**	32,7	**54,5**

Điểm đáng nhớ: **GDN-H tụt xuống dưới MLA ở ngữ cảnh dài** (51,2 vs 52,2), còn Kimi Linear dẫn đầu (54,5); **Kimi Linear (RoPE)** ngang trên ngắn nhưng kém trên dài (51,8) → NoPE là quan trọng. Thứ bậc paper kết luận: pretrain/SFT: Kimi Linear > GDN-H > MLA; ngữ cảnh dài: Kimi Linear > MLA > GDN-H.

RL (Hình 6): huấn luyện RLVR toán học cùng thuật toán và siêu tham số; Kimi Linear tăng độ chính xác train và test nhanh hơn và cao hơn MLA đáng kể (MATH500, AIME 2025; đồ thị, không có số).

Hiệu năng suy luận (Hình 1, 7; mô hình cỡ 48B).

- Prefill: gần như trùng GDN-H; ngang MLA ở 4K-16K, nhanh dần từ 128K; **2,3× ở 512K và 2,9× ở 1M** so với MLA.
- Giải mã (TPOT): với batch 1, tăng tốc 2,3× ở 1M (mục 6.3). Khi tận dụng KV nhỏ để tăng batch: TPOT 1,84 ms so với 11,48 ms của MLA tại 1M token (**6,3×**). Paper viết cả "6×" lẫn "2,3×" ở các chỗ khác nhau (tuỳ batch); đọc kỹ điều kiện.
- Hình 1(a): trên RULER 128K, Kimi Linear 84,3 với tăng tốc 3,98×; MMLU-Pro (4K) 51,0 so với 47,2 (MLA) ở tốc độ tương đương.

Bản phát hành 5,7T token (Phụ lục D, Bảng 8, 9): so với Moonlight (16B tổng/3B active, cùng 5,7T): ví dụ MMLU-Pro 54,8 vs 42,4; GSM8K 86,3 vs 77,2; instruct: RULER@128k 95,4, RULER@1M 94,8, MMLU-Pro 72,7, AIME 2025 58,6. (So sánh này **không sạch**: đổi cả kiến trúc lẫn tổng số tham số, nên không dùng làm bằng chứng riêng cho KDA.)

---

5. Phân tích / ablation

Bảng 1 (mô hình scaling nhỏ nhất 16 head 16 layer; PPL thấp hơn tốt hơn):

Biến thể	Train PPL	Val PPL
Hybrid ratio **3:1** (mặc định)	**9,23**	**5,65**
0:1 (toàn MLA)	9,45	5,77
1:1	9,29	5,66
7:1	9,23	5,70
15:1	9,34	5,82
bỏ output gate	9,25	5,67
output gate **swish**	9,43	5,81
bỏ conv	9,29	5,70

Rút ra (paper): 3:1 tốt nhất; 7:1 train PPL ngang nhưng val kém hơn rõ; 1:1 val ngang nhưng tốn suy luận hơn; toàn full attention (0:1) kém (!). Output gate sigmoid (5,65) hơi hơn không gate (5,67) và hơn hẳn swish

(5,81); conv vẫn có ích trong hybrid. (Lưu ý: val PPL tính trên tập có phân phối khác pretrain nên nhấn mạnh khái quát hoá.)

NoPE vs RoPE: đoán của paper: ở bản RoPE, layer toàn cục mang tín hiệu vị trí tường minh mạnh, còn layer tuyến tính cung cấp thiên lệch vị trí ngầm yếu → lệch giữa các tầng, quá nhấn mạnh thứ tự ngắn → tốt ở ngữ cảnh ngắn nhưng kém linh hoạt khi mở rộng sang dài. Với NoPE, KDA gánh toàn bộ → cân bằng hơn qua độ sâu, ngoại suy tốt hơn.

Hybrid layer-wise vs head-wise: paper chọn layer-wise vì hạ tầng và ổn định, không có ablation số riêng trong text; thảo luận trong §7.2: intra-layer hybrid phức tạp, inter-layer thực tế hơn.

---

6. Giới hạn

Do tác giả nêu:

- Linear attention vẫn bị hạn chế bởi cài đặt phần cứng hiện tại và thiếu hạ tầng suy luận tối ưu (đã có vLLM + kernel mở).
- Tương lai: kết hợp với **sparse attention** (NSA, MoBA, DSA) để lấy sức mạnh truy hồi chi tiết; linear attention tuy vẫn bị phê bình về truy hồi/chép chính xác khi cực dài, có thể giảm thiểu qua "state expansion".
- Scaling law chưa tinh chỉnh siêu tham số kỹ cho KDA.

Nhận xét bổ sung:

- Kết quả "vượt full attention" chỉ cho **một họ** (MoE 48B/3B, 1,4T token, MLA làm baseline, không phải GQA/MHA thông thường). Chênh lệch trên nhiều benchmark khá nhỏ (ví dụ MATH 54,7 = 54,7; HellaSwag +1,2).
- Không có ablation tách "gate theo kênh" khỏi các yếu tố khác (NoPE, output gate, conv) ở cùng quy mô lớn: lợi ích của KDA so với GDN-H được thể hiện qua so sánh hệ thống, không phải ablation đơn nhân tố ở 48B; Hình 4 (bài tổng hợp nhỏ) là bằng chứng trực tiếp nhất.
- Không có thử nghiệm đa phương thức/OCR; không có thử nghiệm cắt tỉa hay chưng cất.
- Con số tăng tốc phụ thuộc kernel và batch (xem lưu ý trên); giữ nguyên điều kiện khi so sánh.

---

7. Liên hệ với các bài khác

- **24 Gated DeltaNet:** KDA = GDN với $\alpha_t \in \mathbb{R}^{d_k}$ thay cho vô hướng; cùng WY/UT, cùng L2-norm, short conv. GDN-H là baseline trực tiếp trong paper.
- **20 GLA:** nguồn của gate theo kênh; KDA bổ sung delta rule và kernel chunkwise ổn định hơn.
- **23 DeltaNet:** nền delta rule và WY.
- **16 MLA:** layer toàn cục của Kimi Linear; thêm NoPE → chuyển MQA khi suy luận (bài 14).
- **12 RoPE:** mục 3.5 giải thích vì sao GDN/KDA đóng vai "mã hoá vị trí nhân phụ thuộc dữ liệu".
- **25 Jamba:** hybrid sớm (1:7, Mamba-1, MoE); Kimi Linear tăng tỉ lệ tuyến tính lên 3:1 nhưng dùng layer mạnh hơn và đánh giá cả RL.
- **22 Mamba-2:** bài này cho thấy Mamba2 thất bại ở các bài tổng hợp (không có delta rule).
- **27/28 MoE:** Kimi Linear là MoE 256 expert; không phải trọng tâm.
- **44 FlashAttention / 46 PagedAttention:** vẫn cần cho 1/4 layer MLA; KDA cần kernel riêng (fla).

---

8. Gợi ý áp dụng

KDA dùng gate theo chiều trong quy tắc cập nhật trạng thái, kết hợp với MLA trong mô hình hybrid của nghiên cứu. Các kết quả về tỉ lệ tầng, NoPE và tốc độ gắn với cấu hình, dữ liệu và phần cứng được thử nghiệm; không nên coi chúng là các hằng số thiết kế phổ quát.

9. Thí nghiệm gợi ý

Tái hiện một tác vụ truy hồi tổng hợp với nhiều kích thước trạng thái; so sánh gate theo head và theo chiều ở cùng ngân sách. Đo chất lượng cùng thời gian và bộ nhớ theo độ dài chuỗi để phân biệt độ phức tạp tính toán với hiệu năng kernel.

10. Tóm tắt công thức / khái niệm

Khái niệm	Công thức / ý nghĩa
GDN	$S_t = \alpha_t(I - \beta_t k_t k_t^\top)S_{t-1} + \beta_t k_t v_t^\top$ (quy ước $S \in \mathbb{R}^{d_k \times d_v}$)
KDA	$\boxed{S_t = (I - \beta_t k_t k_t^\top)\text{Diag}(\alpha_t)S_{t-1} + \beta_t k_t v_t^\top}$, $\alpha_t \in [0,1]^{d_k}$
DPLR	$D - a_t b_t^\top$; KDA: $D = \text{Diag}(\alpha)$, $a = \beta k$, $b = k \odot \alpha$
WY có gate	$P^r = \text{Diag}(\gamma^r) - \sum \text{Diag}(\gamma^{i\to r}) k_i w_i^\top$
UT	$M = [I + \text{StrictTril}(\text{Diag}(\beta)(\Gamma \odot K)(K/\Gamma)^\top)]^{-1}\text{Diag}(\beta)$
State khối	$\text{Diag}(\gamma^C)S + (\Gamma^{i\to C} \odot K)^\top (U - WS)$
FLOPs/head	KDA $6Td^2 + 3TCd + TC^2$; attention $2T^2 d$
Hybrid	3 KDA : 1 MLA (NoPE), MoE 256 expert/8 active, 48B/3B
Kết quả chính	MMLU-Pro 51,0 vs 47,2 (MLA); RULER@128k 84,3 vs 81,3; KV −75%; TPOT 6,3× @1M
Scaling	≈1,16× hiệu quả tính toán so với MLA

11. Kết luận ngắn

Kimi Linear = KDA (GDN với gate theo kênh, ràng buộc DPLR để kernel nhanh) + hybrid 3:1 với MLA không mã hoá vị trí. Trong so sánh công bằng cùng 1,4T token, nó nhỉnh hơn full attention và hybrid GDN trên ngắn, dài, RL, đồng thời giảm 75% KV-cache và nhanh tới 6,3× khi giải mã ở 1M token.

Nguồn gốc: https://arxiv.org/abs/2510.26692

Bài 27

Switch Transformers: Scaling to Trillion Parameter Models with Simple and Efficient Sparsity (2021)

Tác giả: William Fedus, Barret Zoph, Noam Shazeer (Google).
Venue / arXiv: arXiv:2101.03961 (v3, 2022); đăng ở Journal of Machine Learning Research 23 (2022) 1-40.
Nguồn: https://arxiv.org/abs/2101.03961
Chủ đề: Mixture of Experts (MoE) thưa, định tuyến top-1 (switch routing), loss cân bằng tải, expert capacity, huấn luyện ổn định với bfloat16, chưng cất mô hình thưa sang mô hình dày.
Vị trí trong lộ trình: Đọc sau 01 (Transformer), 03 (Kaplan scaling laws), 11 (GLU/SwiGLU, vì expert chính là một FFN). Bài này biến MoE từ một kỹ thuật phức tạp thành một thay đổi đơn giản (mỗi token chỉ đi vào *một* expert). Bài kế tiếp là 28 (Mixtral: MoE top-2 trong LLM mã nguồn mở), sau đó 25 (Jamba: hybrid + MoE) và 16 (DeepSeek-V2).

> Ghi chú diễn giải, không phải bản dịch. Phần dưới tập trung vào định tuyến, cân bằng tải, ổn định, scaling và chưng cất; song song hoá hệ thống chỉ được tóm tắt.

1. Bối cảnh

Mô hình dày (dense) và định luật scaling. Trong một Transformer thông thường, *mọi* tham số đều được dùng cho *mọi* token. Kaplan et al. (bài 03) cho thấy loss giảm theo lũy thừa khi tăng số tham số, dữ liệu và compute. Nhưng tăng tham số ở mô hình dày đồng nghĩa với tăng FLOPs (số phép tính dấu phẩy động) tương ứng, nên chi phí tăng theo.

Câu hỏi của paper: nếu ta tăng *số tham số* mà giữ *FLOPs mỗi token* không đổi thì sao? Paper giả thuyết rằng số tham số, tách rời khỏi lượng tính toán, là một **trục scaling thứ tư** (ngoài tham số-và-FLOPs đi cùng nhau, dữ liệu, compute).

Conditional computation (tính toán có điều kiện). Ý tưởng cho mỗi đầu vào đi qua một phần khác nhau của mạng. Dạng quen thuộc nhất là **Mixture of Experts (MoE)**: có nhiều mạng con gọi là *expert*, và một mạng nhỏ gọi là *router* (bộ định tuyến) quyết định mỗi token đi vào expert nào. Shazeer et al. (2017) đưa MoE vào LSTM, GShard (Lepikhin et al., 2020) đưa vào Transformer dịch máy. Nhưng paper nêu ba trở ngại làm MoE chưa được dùng rộng rãi: **phức tạp, chi phí giao tiếp giữa các thiết bị, và huấn luyện không ổn định**.

Giả định cũ bị Switch phản bác. Shazeer 2017 cho rằng phải định tuyến tới k > 1 expert thì router mới có gradient có ý nghĩa ("phải so sánh ít nhất hai expert"). Switch Transformer chỉ dùng k = 1 và vẫn hoạt động tốt hơn.

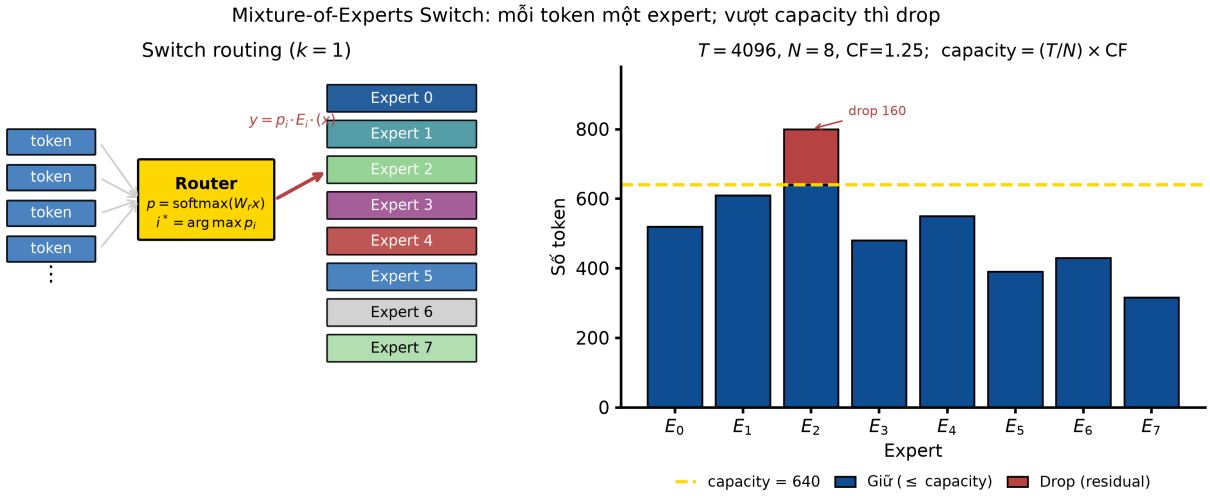
2. Ý tưởng chính

Trong Transformer, thay lớp FFN (feed-forward) bằng một **lớp Switch**: có N bản FFN độc lập (N expert), một router nhỏ chọn *đúng một* expert cho mỗi token, và đầu ra của expert đó được nhân với xác suất mà router gán cho nó. Vì mỗi token chỉ chạy qua một FFN, FLOPs mỗi token bằng mô hình dày gốc, nhưng số tham số tăng gần N lần. Để các expert được dùng đều, paper thêm một loss phụ cân bằng tải; để huấn luyện ổn định, paper dùng độ chính xác float32 cục bộ cho router, khởi tạo nhỏ hơn 10 lần, và dropout lớn hơn trong expert khi fine-tune. Kết quả: cùng FLOPs và cùng thời gian, mô hình thưa đạt chất lượng của mô hình dày sau thời gian nhanh gấp tới 7 lần.

3. Phương pháp chi tiết

3.1 Router và định tuyến top-k

Với token biểu diễn x, router có ma trận W_r, tạo logit $h(x) = W_r x$ và softmax qua N expert:



Switch MoE top-1: router chọn một expert; với batch 4096, 8 expert, CF=1.25 (capacity 640) thì expert quá tải drop 160 token qua residual. (minh hoạ, tự tính)

$$p_i(x) = \frac{e^{h(x)_i}}{\sum_{j=1}^{N} e^{h(x)_j}}.$$

MoE gốc chọn tập $\mathcal{T}$ gồm top-k chỉ số và tính:

$$y = \sum_{i \in \mathcal{T}} p_i(x)\, E_i(x).$$

Switch routing là trường hợp $k = 1$: $\mathcal{T} = \{\arg\max_i p_i(x)\}$, tức

$$\boxed{y = p_{i^*}(x)\, E_{i^*}(x), \qquad i^* = \arg\max_i p_i(x)}$$

Giá trị cổng $p_{i^*}(x)$ vẫn nhân vào đầu ra nên router có gradient (đây là điểm cho phép học định tuyến dù chỉ chọn một expert). Ba lợi ích paper nêu: (1) tính toán router ít hơn; (2) batch size (capacity) của mỗi expert có thể giảm ít nhất một nửa; (3) cài đặt đơn giản, giao tiếp ít hơn.

Chi phí router: $O(d_{model} \times N)$ mỗi token, rất nhỏ so với FFN.

3.2 Expert capacity và token bị bỏ (dropped tokens)

Trên TPU kích thước tensor phải biết trước lúc biên dịch, nên mỗi expert có một **capacity** cố định (số token tối đa nó xử lý):

$$\boxed{\text{expert capacity} = \left(\frac{\text{tokens per batch}}{\text{number of experts}}\right) \times \text{capacity factor}}$$

- Capacity factor = 1.0: mỗi expert vừa đủ nếu phân bố hoàn toàn đều.

- Capacity factor > 1.0: thêm vùng đệm cho trường hợp lệch, nhưng tốn compute/bộ nhớ cho các ô trống (padding).
- Token vượt capacity bị **drop**: layer bỏ qua tính toán, token đi tiếp qua *residual connection* (đầu ra của layer bằng đầu vào). Paper báo tỉ lệ drop thường dưới 1%.

Ví dụ nhỏ (tự tính): batch 4096 token, 8 expert, capacity factor 1.25: capacity mỗi expert = 4096/8 × 1.25 = 640. Nếu router dồn 800 token vào expert 3 thì 160 token bị drop ở layer đó.

3.3 Loss cân bằng tải (load-balancing loss)

Nếu để router tự do, nó thường dồn mọi token vào vài expert ("collapse"), các expert còn lại không được huấn luyện. Paper thêm loss phụ cho mỗi lớp Switch. Với batch $\mathcal{B}$ gồm T token:

$$f_i = \frac{1}{T}\sum_{x\in\mathcal{B}} \mathbb{1}\{\arg\max p(x) = i\}, \qquad P_i = \frac{1}{T}\sum_{x\in\mathcal{B}} p_i(x)$$

$$\boxed{\mathcal{L}_{aux} = \alpha \cdot N \cdot \sum_{i=1}^{N} f_i\, P_i}$$

- f_i: tỉ lệ token *thực sự* được gửi tới expert i (không khả vi vì có argmax).
- P_i: tổng *xác suất* router dành cho expert i (khả vi); gradient chảy qua đây.
- Cực tiểu khi phân bố đều: $f_i = P_i = 1/N$ thì $\sum f_i P_i = 1/N$, nhân N ra 1 (hằng số theo N, nên α không phải chỉnh lại khi đổi số expert).
- Paper dùng $\alpha = 10^{-2}$ (đã quét từ 10^{-1} đến 10^{-5}).

Ví dụ (tự tính): $N = 8$. Nếu phân bố đều, $N\sum f_i P_i = 8 \cdot 8 \cdot \frac{1}{64} = 1$. Nếu toàn bộ token và xác suất dồn vào một expert ($f = P = (1, 0, \ldots, 0)$), giá trị là $8 \cdot 1 = 8$, tức phạt gấp 8 lần.

3.4 Ổn định huấn luyện

1. **Selective precision.** Chạy cả mô hình bằng bfloat16 thì router (softmax trên logit) dễ mất ổn định. Paper chỉ ép *đầu vào router* sang float32, bên trong hàm router; tensor dispatch/combine cast về bfloat16 trước khi giao tiếp đa thiết bị, nên không tốn băng thông float32.
2. **Khởi tạo nhỏ hơn.** Chuẩn truncated normal với $\sigma = \sqrt{s/n}$ (n = fan-in); giảm scale s từ 1.0 xuống 0.1.
3. **Expert dropout.** Khi fine-tune trên tập nhỏ, mô hình nhiều tham số dễ overfit. Giải pháp: dropout thấp (0.1) ở các lớp thường nhưng cao (0.4) *bên trong* FFN của expert.
4. **Exploration trong định tuyến** (Phụ lục C): router chọn rời rạc nên giống bài toán contextual bandit. Thử argmax, sample từ softmax, input dropout, input jitter (nhân nhiễu ngẫu nhiên vào đầu vào router). Paper dùng jitter.

3.5 Pseudo-code lớp Switch (rút gọn từ Phụ lục F, viết lại bằng PyTorch-like)

```
def switch_ffn(x, W_r, experts, capacity):        # x: [T, d]
    logits = (x * jitter()).float() @ W_r          # router chạy float32  [T, N]
    probs  = softmax(logits, -1)
    gate, idx = probs.max(-1)                      # top-1: [T]
    mask = one_hot(idx, N)                         # [T, N]
    pos  = cumsum(mask, 0) * mask                  # vị trí của token trong hàng đợi expert
    keep = (pos <= capacity)                       # token vượt capacity bị drop
    aux  = alpha * N * (mask.float().mean(0) * probs.mean(0)).sum()
    y = zeros_like(x)                              # token bị drop: y=0, residual giữ nguyên
    for i in range(N):                             # thực tế: all-to-all giữa các thiết bị
        sel = (idx == i) & keep.any(-1)
```

```
        y[sel] = gate[sel, None] * experts[i](x[sel])
    return y, aux
```

3.6 Song song hóa (chỉ ý chính, Mục 5)

Mỗi expert nằm trên một thiết bị (core) riêng, nên số tham số tăng theo số thiết bị trong khi bộ nhớ mỗi thiết bị hầu như không đổi. Token được chuyển tới expert bằng phép **all-to-all**, rồi chuyển về. Kết hợp ba kiểu song song (data, model, expert) cho hai mô hình lớn: Switch-XXL (395B tham số, FLOPs/seq bằng T5-XXL) và Switch-C (1,6 nghìn tỉ tham số, 2048 expert, chỉ dùng expert parallelism).

4. Thí nghiệm và kết quả chính

Thiết lập. Tiền huấn luyện trên C4 (Colossal Clean Crawled Corpus) bằng masked language modeling kiểu T5 (che 15% token, thay bằng một sentinel); đo *negative log perplexity* (đơn vị nat, càng cao càng tốt). Switch-Base khớp FLOPs với T5-Base (223M), Switch-Large khớp T5-Large (739M). Các mô hình đặt expert ở một lớp FFN trong hai lớp (expert freq 1/2), trừ Switch-C (mọi lớp).

4.1 Switch so với MoE top-2 và T5 (Bảng 1, 128 expert, 32 core TPUv3)

Mô hình	Capacity factor	Chất lượng sau 100k bước	Thời gian đạt ngưỡng -1.50 (giờ)	Tốc độ (ví dụ/giây)
T5-Base	-	-1.731	không đạt	1600
T5-Large	-	-1.550	131.1	470
MoE-Base	2.0	-1.547	68.7	840
Switch-Base	2.0	-1.554	72.8	860
MoE-Base	1.25	-1.559	80.7	790
Switch-Base	1.25	-1.553	65.0	910
MoE-Base	1.0	-1.572	80.1	860
Switch-Base	1.0	-1.561	62.8	1000
Switch-Base+ (d_model 896, 16 heads)	1.0	-1.534	67.6	780

Đọc bảng: Switch nhanh hơn và tốt hơn MoE top-2 khi capacity factor thấp (1.0, 1.25). Paper tự chú thích một điểm bất thường: MoE chạy chậm đi khi hạ capacity từ 2.0 xuống 1.25 (840 xuống 790), và nhấn mạnh tốc độ phụ thuộc cả cài đặt cấp thấp chứ không chỉ thuật toán.

4.2 Scaling theo số expert

- Giữ FLOPs/token cố định, tăng số expert từ 2 lên 256 (14,7B tham số) làm test loss giảm đều (Hình 4).
- Theo số bước: Switch-Base 64 expert đạt chất lượng T5-Base ở bước 60k (T5-Base) chỉ sau một phần nhỏ số bước (paper nói tăng tốc 7,5 lần theo số bước; câu văn trong bản text bị lỗi nên không trích số bước chính xác).
- Theo thời gian thực: Switch-Base 64 expert đạt chất lượng tương đương T5-Base trong **1/7 thời gian**.
- So với T5-Large (nhiều hơn 3,5 lần FLOPs/token): Switch-Base vẫn hiệu quả mẫu cao hơn và nhanh hơn 2,5 lần theo thời gian.

4.3 Fine-tuning (Bảng 5, trích)

Mô hình	GLUE	SQuAD	SuperGLUE	Winogrande (XL)	XSum	CB Trivia QA
T5-Base	84.3	85.5	75.1	66.6	18.7	24.5

Mô hình	GLUE	SQuAD	SuperGLUE	Winogrande (XL)	XSum	CB Trivia QA
Switch-Base	86.7	87.2	79.5	73.3	20.3	30.7
T5-Large	87.8	88.1	82.7	79.1	20.9	29.5
Switch-Large	88.5	88.6	84.7	83.0	22.3	36.9

SuperGLUE tăng 4,4 điểm (Base) và 2,0 điểm (Large). Ngoại lệ: trên ARC (reasoning) T5-Base thắng Switch-Base ở ARC Challenge (35.5 so với 32.8) và T5-Large thắng Switch-Large ở ARC Easy (68.8 so với 66.0).

4.4 Chưng cất MoE thưa sang mô hình dày (Bảng 6, 7, 8)

Chưng cất (distillation) là huấn luyện mô hình nhỏ ("student") bắt chước đầu ra của mô hình lớn ("teacher"). Paper chưng cất Switch-Base (3,8B) vào T5-Base (223M):

Kỹ thuật	Tham số	Chất lượng (phần trăm chất lượng teacher giữ lại)
T5-Base (baseline)	223M	-1.636
Switch-Base (teacher)	3.800M	-1.444
Chưng cất thường	223M	-1.631 (3%)
+ khởi tạo student bằng trọng số non-expert của teacher	223M	-1.598 (20%)
+ trộn loss 0.75 nhãn thật + 0.25 xác suất teacher	223M	-1.580 (29%)

Hai thủ thuật cộng lại giữ khoảng 30% phần "cải thiện chất lượng" của teacher trong khi nén khoảng 95% tham số. Với các teacher khác kích thước (Bảng 7): teacher 1,1B giữ 37% (nén 82%); 7,4B giữ 27% (nén 97%); 14,7B giữ 28% (nén 99%). Với mô hình đã fine-tune trên SuperGLUE (Bảng 8): T5-Base 74.6, Switch-Base 81.3, T5-Base chưng cất 76.6 (giữ 30%).

Lưu ý: "khởi tạo bằng trọng số non-expert" khả thi vì mô hình khớp FLOPs nên attention và các FFN dày có cùng kích thước; khởi tạo bằng trọng số của expert (chạy bình thường không chưng cất) không cải thiện.

4.5 Đa ngôn ngữ

mSwitch-Base so với mT5-Base trên mC4, 101 ngôn ngữ: cải thiện perplexity ở cả 101 ngôn ngữ; tăng tốc trung bình 5 lần theo số bước, và 91% ngôn ngữ đạt tăng tốc từ 4 lần trở lên.

4.6 Mô hình lớn (Bảng 9)

Mô hình	Tham số	FLOPs/seq	Số expert	Neg. log perp. @250k	@500k
T5-Base	0,2B	124B	-	-1.599	-1.556
T5-XXL	11B	6,3T	-	-1.147	-1.095
Switch-XXL	395B	6,3T	64	-1.086	-1.008
Switch-C	1571B	890B	2048	-1.096	-1.043

Switch-C nhanh hơn T5-XXL 4 lần để đạt cùng perplexity (cùng ngân sách compute). Fine-tune sớm (mới tiền huấn luyện 503B token, khoảng nửa lượng dữ liệu của T5-XXL): SQuAD 89.7 (SOTA lúc đó 91.3), SuperGLUE test 87.5 (SOTA 90.0), ANLI 65.7 (kỷ lục trước đó 49.4), Natural Questions 34.4 (trước đó 32.8), Web Questions 41.0 (trước đó 37.2).

5. Phân tích / ablation

- **Selective precision (Bảng 2).** 32 expert: float32 thuần cho -1.718 ở 1160 ví dụ/giây; bfloat16 thuần *phân kỳ* (-3.780) ở 1390; selective precision cho -1.716 ở 1390. Tức giữ được ổn định của float32 mà gần tốc độ bfloat16.
- **Khởi tạo (Bảng 3).** Sau 3,5k bước (3 seed): init 0.1x cho -2.72 (độ lệch chuẩn 0.01); init 1.0x cho -3.60 (độ lệch chuẩn 0.68). Khởi tạo nhỏ vừa tốt hơn vừa ổn định hơn.
- **Expert dropout (Bảng 4).** Tăng dropout đồng loạt làm xấu đi; dropout 0.1 ở lớp thường + 0.4 ở expert tốt nhất (GLUE 85.2, so với 84.7 khi dropout đều 0.1).
- **Exploration (Bảng 11).** Argmax -1.471; sample softmax -1.570 (kém); input dropout -1.480; input jitter -1.468 (tốt nhất).
- **Switch cho attention (Phụ lục A, Bảng 10).** Đặt expert vào các phép chiếu Q/K/V cho chất lượng tốt hơn khi float32, nhưng *phân kỳ* khi bfloat16, nên không đưa vào bản cuối.
- **No-Token-Left-Behind (Phụ lục B).** Định tuyến lại token tràn sang expert xếp hạng hai; tác giả giả thuyết giúp ích nhưng không thấy lợi ích thực nghiệm.
- **Upstream và downstream (Phụ lục E).** Perplexity tiền huấn luyện tương quan với kết quả downstream, nhưng ở quy mô lớn nhất mô hình thưa không chuyển hết lợi ích sang SuperGLUE (mô hình dày thường tốt hơn ở cùng perplexity), trong khi trên TriviaQA (tri thức) mô hình thưa có vẻ tốt hơn. Paper nói cần thêm dữ liệu để khẳng định.
- **Ít expert vẫn có lợi (Phụ lục D).** Chỉ 2, 4, 8 expert đã cải thiện so với T5-Base dày, nên không cần siêu máy tính.

6. Giới hạn

Do tác giả nêu:

- Mô hình thưa lớn nhất (Switch-XXL) đôi khi mất ổn định; vì vậy không huấn luyện đủ 1M bước. Switch-C (1,6T) không bất ổn.
- Lợi ích upstream chưa chuyển hết sang downstream, nhất là tác vụ suy luận (SuperGLUE, ARC) ở quy mô rất lớn.
- Chưng cất giữ được "chỉ" khoảng 30% phần cải thiện.
- Chi phí giao tiếp all-to-all và việc phải chọn capacity factor là gánh nặng hệ thống; thuật toán chưa dùng được attention thưa theo độ dài chuỗi (phần Related Work nói đó là hướng kết hợp tương lai).
- Paper mô tả quyết định định tuyến cứng (rời rạc) là một nguồn bất ổn và coi việc khám phá expert (exploration) như bài toán bandit; họ chỉ thử vài cách đơn giản (jitter là tốt nhất), không phải lời giải triệt để (nhận xét của người viết).

Do người viết nhận xét:

- Mọi so sánh dựa vào FLOPs/token, bỏ qua **bộ nhớ**: Switch-Base có 7B tham số nhưng chỉ tính như 223M về FLOPs; khi triển khai, toàn bộ 7B phải nằm trong bộ nhớ (hoặc phải offload). Bài toán suy luận trên GPU nhỏ vì thế không được đề cập.
- Tốc độ báo cáo trên TPU với shape tĩnh và capacity cố định; trên GPU với shape động hành vi có thể khác.
- Chất lượng đo chủ yếu qua perplexity trên C4 và các tác vụ fine-tune kiểu T5; không có đánh giá few-shot/instruction kiểu LLM hiện đại.

7. Liên hệ với các bài khác

- **03 Kaplan:** Switch coi số tham số (độc lập FLOPs) là trục scaling mới, mở rộng Kaplan.
- **11 GLU variants:** expert trong Switch dùng FFN GEGLU; trong Mixtral (28) expert là SwiGLU.

- **28 Mixtral:** MoE top-2 trong LLM decoder-only mã nguồn mở, đặt expert ở *mọi* lớp; Switch đặt ở một nửa số lớp và dùng top-1.
- **25 Jamba, 16 DeepSeek-V2:** các kiến trúc về sau kết hợp MoE với thành phần khác (hybrid Mamba/attention; MLA).
- **08 LoRA, 53-54 (GPTQ, AWQ):** MoE nhiều tham số nên bài toán nén/lượng tử hóa càng quan trọng (suy luận của người viết).
- **Chưng cất:** bài báo thử chuyển tri thức từ mô hình thưa sang mô hình dày.

8. Gợi ý áp dụng

MoE phân biệt số tham số được kích hoạt trên mỗi token với tổng số tham số phải lưu. Cân bằng tải và chi phí định tuyến cũng ảnh hưởng hiệu năng thực tế. Thí nghiệm chưng cất từ mô hình thưa sang mô hình dày cho thấy cách khởi tạo và pha loss nhãn thật với tín hiệu teacher có thể ảnh hưởng chất lượng; các con số cụ thể thuộc thiết lập của bài báo.

9. Thí nghiệm gợi ý

Dựng một lớp Switch nhỏ và so sánh phân bố token trên các expert khi bật và tắt loss cân bằng tải. Báo cáo cả chất lượng, tỉ lệ token bị bỏ và tải của từng expert để nhận biết định tuyến mất cân bằng.

10. Tóm tắt công thức / khái niệm

Khái niệm	Công thức / ý nghĩa
Router	$p(x) = \text{softmax}(W_r x)$, chi phí $O(d \cdot N)$
Switch (top-1)	$y = p_{i^*}(x) E_{i^*}(x)$, $i^* = \arg\max p$
MoE top-k	$y = \sum_{i \in \mathcal{T}} p_i(x) E_i(x)$
Expert capacity	$\frac{\text{tokens/batch}}{N} \times$ capacity factor
Dropped token	token vượt capacity, đi tiếp qua residual (thường < 1%)
Loss cân bằng tải	$\alpha N \sum_i f_i P_i$, $\alpha = 10^{-2}$, cực tiểu = α khi đều
Selective precision	router chạy float32 cục bộ, phần còn lại bfloat16
Khởi tạo	scale s giảm 10 lần ($\sigma = \sqrt{s/n}$)
Expert dropout	0.4 trong expert, 0.1 ở nơi khác khi fine-tune
Chưng cất	init non-expert từ teacher + 0.75 nhãn thật / 0.25 teacher: giữ khoảng 30% cải thiện, nén khoảng 95%
Số liệu then chốt	Switch-Base 64 expert: 7 lần nhanh hơn T5-Base theo thời gian; Switch-C 1,6T: 4 lần nhanh hơn T5-XXL

11. Kết luận ngắn

Switch Transformer cho thấy định tuyến một expert mỗi token, cộng loss cân bằng tải đơn giản và vài thủ thuật ổn định (float32 cục bộ cho router, init nhỏ, dropout lớn trong expert), đủ để biến MoE thành kiến trúc huấn luyện được ở quy mô nghìn tỉ tham số, với tốc độ tiền huấn luyện nhanh hơn mô hình dày cùng FLOPs tới 4-7 lần. Cái giá là bộ nhớ (tất cả expert phải được lưu) và độ phức tạp hệ thống.

Liên kết gốc: https://arxiv.org/abs/2101.03961

Bài 28

Mixtral of Experts (2024)

Tác giả: Albert Q. Jiang, Alexandre Sablayrolles, Antoine Roux, Arthur Mensch, Blanche Savary, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Emma Bou Hanna, Florian Bressand, Gianna Lengyel, Guillaume Bour, Guillaume Lample, Lélio Renard Lavaud, Lucile Saulnier, Marie-Anne Lachaux, Pierre Stock, Sandeep Subramanian, Sophia Yang, Szymon Antoniak, Teven Le Scao, Théophile Gervet, Thibaut Lavril, Thomas Wang, Timothée Lacroix, William El Sayed (Mistral AI).
Venue / arXiv: arXiv:2401.04088 (v1, 8/1/2024). Paper không nêu hội nghị.
Nguồn: https://arxiv.org/abs/2401.04088
Chủ đề: Sparse Mixture of Experts (SMoE) trong LLM decoder-only, top-2 routing, tham số tổng (sparse) và tham số hoạt động (active), phân tích định tuyến.
Vị trí trong lộ trình: Đọc sau 06 (Mistral 7B, kiến trúc nền), 11 (SwiGLU) và 27 (Switch Transformer: nguồn gốc MoE hiện đại). Bài này đem MoE vào một LLM mở đạt chất lượng của Llama 2 70B. Bài kế tiếp theo mạch kiến trúc: 29 (ViT) bắt đầu nhánh thị giác; về MoE, xem lại 25 (Jamba) và 16 (DeepSeek-V2).

> Ghi chú diễn giải, không phải bản dịch. Báo cáo kỹ thuật tập trung chủ yếu vào các bảng kết quả.

1. Bối cảnh

Đến cuối 2023, các LLM mã nguồn mở mạnh nhất (Llama 2 70B) đều là mô hình dày (dense): mọi tham số chạy cho mọi token, nên 70B tham số nghĩa là khoảng 70B phép nhân-cộng mỗi token. Mistral 7B (bài 06) cho thấy mô hình nhỏ được thiết kế tốt có thể vượt mô hình lớn hơn, nhưng vẫn bị giới hạn bởi dung lượng.

MoE (Mixture of Experts) từ Switch Transformer (bài 27) giải quyết bằng cách có nhiều "expert" (ở đây là nhiều FFN song song) và một bộ định tuyến (router) chỉ kích hoạt vài expert cho mỗi token. Trước Mixtral, MoE chủ yếu xuất hiện ở các hệ thống đóng hoặc nghiên cứu (GShard, Switch); chưa có LLM mở dùng MoE đạt chất lượng hàng đầu. Mixtral lấp khoảng trống này, đồng thời phát hành trọng số dưới giấy phép Apache 2.0.

2. Ý tưởng chính

Mixtral 8x7B có **cùng kiến trúc với Mistral 7B**, chỉ khác ở chỗ khối FFN của *mọi* lớp được thay bằng 8 FFN (8 expert). Với mỗi token và mỗi lớp, một router chọn 2 trong 8 expert, chạy hai expert đó và cộng đầu ra có trọng số. Vì token mỗi lớp chọn khác nhau, mỗi token "truy cập" tới 47B tham số nhưng chỉ kích hoạt khoảng 13B tham số cho mỗi token. Kết quả: ngang hoặc hơn Llama 2 70B và GPT-3.5 ở hầu hết benchmark, vượt hẳn ở toán, code, đa ngôn ngữ, với khoảng 5 lần ít tham số hoạt động hơn.

3. Phương pháp chi tiết

3.1 Kiến trúc (Bảng 1 của paper)

Tham số	Giá trị
dim (d_{model})	4096
n_layers	32
head_dim	128
hidden_dim (FFN)	14336
n_heads	32
n_kv_heads	8 (GQA, xem bài 15)

Tham số	Giá trị
context_len	32768
vocab_size	32000
num_experts	8
top_k_experts	2

Paper nói Mixtral dùng các cải tiến của Mistral 7B, với điểm khác biệt: hỗ trợ ngữ cảnh *dày đủ* (fully dense) 32k token, và FFN là lớp MoE.

3.2 Lớp MoE

Gọi n là số expert. Đầu ra của lớp MoE với đầu vào x:

$$\sum_{i=0}^{n-1} G(x)_i \cdot E_i(x), \qquad G(x) := \text{Softmax}(\text{TopK}(x \cdot W_g)).$$

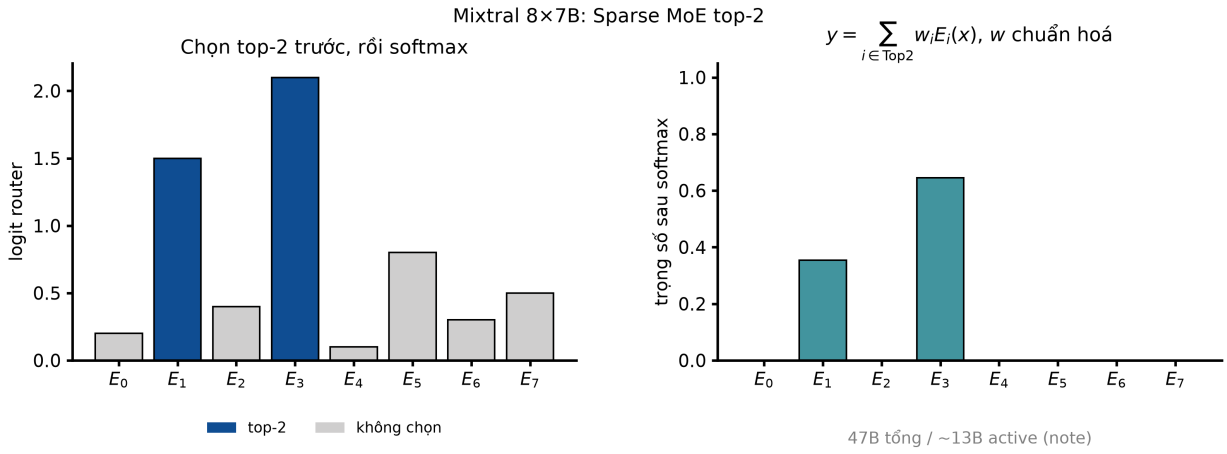
Toán tử $\text{TopK}(\ell)_i = \ell_i$ nếu ℓ_i nằm trong K giá trị lớn nhất, ngược lại bằng $-\infty$. Do $e^{-\infty} = 0$, softmax chỉ phân phối trọng số cho K expert được chọn, nên các expert còn lại không cần tính. Mixtral dùng $K = 2$ và mỗi expert là một khối SwiGLU (bài 11):

$$\boxed{y = \sum_{i=0}^{n-1} \text{Softmax}\big(\text{Top2}(x \cdot W_g)\big)_i \cdot \text{SwiGLU}_i(x)}$$

Khác Switch: Switch lấy softmax trên *toàn bộ* N logit rồi chọn top-1 và nhân xác suất đó; Mixtral chọn top-2 trước rồi softmax *chỉ trên hai logit* nên hai trọng số cộng lại bằng 1. Paper cũng so với GShard: Mixtral thay *tất cả* khối FFN (GShard thay xen kẽ) và dùng cách chọn expert thứ hai đơn giản hơn. Paper không nhắc loss cân bằng tải nào (kết quả huấn luyện chi tiết không được công bố).

3.3 Tham số tổng và tham số hoạt động

- **Sparse (tổng) parameter count:** tăng theo n; quyết định bộ nhớ khi phục vụ (47B).
- **Active parameter count:** tăng theo K; tỉ lệ thuận với chi phí tính toán mỗi token (13B).



Mixtral top-2: chọn 2/8 expert rồi softmax trên hai logit; ~47B tổng / 13B active. (minh hoạ, tự tính)

Kiểm tra bằng số (tự tính từ Bảng 1): một expert SwiGLU có $3 \cdot 4096 \cdot 14336 \approx 176{,}2$M tham số. Tám expert × 32 lớp ≈ 45,1B. Attention mỗi lớp: W_Q, W_O mỗi cái 4096^2 (16,8M), W_K, W_V mỗi cái 4096×1024 (4,2M), tổng khoảng 41,9M; 32 lớp ≈ 1,34B. Embedding và đầu ra: $2 \times 32000 \times 4096 \approx 0{,}26$B (nếu không buộc trọng số). Tổng ≈ 46,7B, khớp "47B". Tham số hoạt động: 2 expert × 32 lớp ≈ 11,3B, cộng 1,34B + 0,26B ≈ 12,9B, khớp "13B".

3.4 Chạy hiệu quả

Paper nêu hai hướng cài đặt: (i) **MegaBlocks** (Gale et al.) viết lại phép tính FFN của MoE thành nhân ma trận thưa-lớn, xử lý tự nhiên trường hợp mỗi expert nhận số token khác nhau; (ii) **Expert Parallelism (EP):** mỗi GPU giữ một số expert, token được gửi tới GPU chứa expert của nó rồi gửi kết quả về. EP gây khó khăn cân bằng tải. Nhóm tác giả đóng góp thay đổi vào vLLM (có MegaBlocks kernel) để chạy Mixtral với stack mã nguồn mở.

3.5 Pseudo-code lớp MoE top-2 (theo công thức của paper, viết lại)

```
def mixtral_moe(x, W_g, experts):           # x: [T, d]; experts: 8 SwiGLU
    logits = x @ W_g                         # [T, 8]
    top_val, top_idx = logits.topk(2, -1)    # [T, 2]
    w = softmax(top_val, -1)                 # chỉ softmax trên 2 logit được chọn
    y = zeros_like(x)
    for e in range(8):
        tok, slot = (top_idx == e).nonzero(as_tuple=True)   # token chọn expert e
        if tok.numel():
            y[tok] += w[tok, slot, None] * experts[e](x[tok])
    return y
```

Khác Switch: không có capacity cố định hay token drop được mô tả trong paper; phần này được MegaBlocks xử lý (token variable-length).

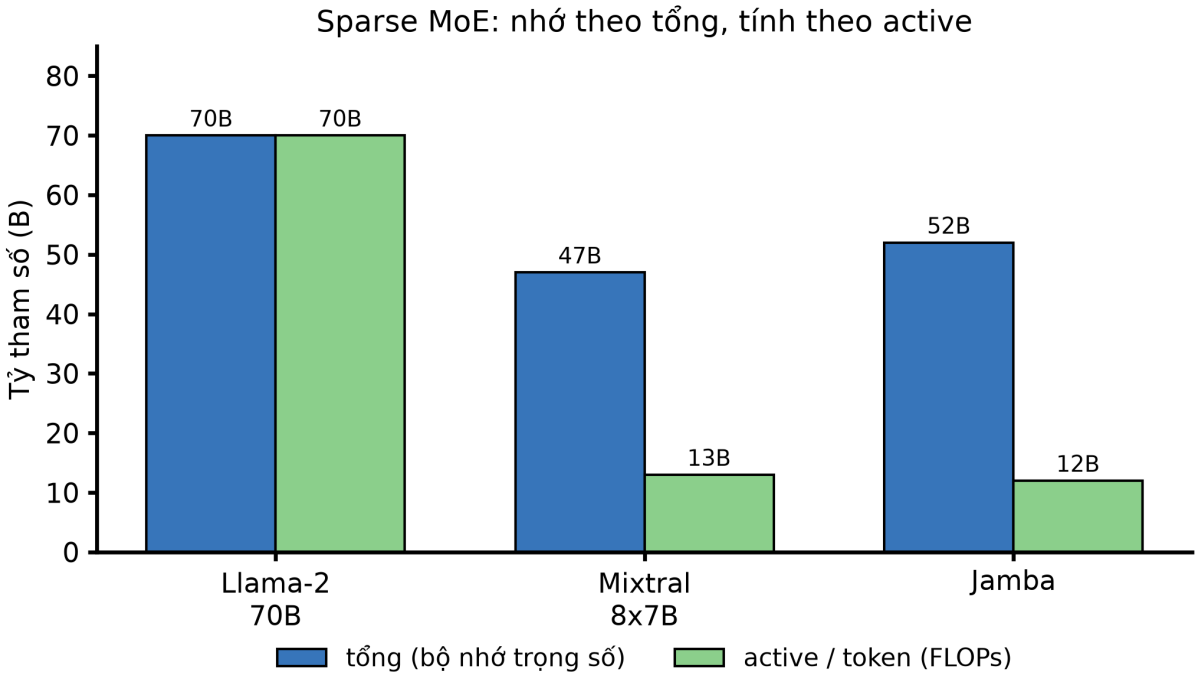
4. Thí nghiệm và kết quả chính

Thiết lập. Tiền huấn luyện với ngữ cảnh 32k, dữ liệu đa ngôn ngữ (tăng tỉ trọng đa ngôn ngữ so với Mistral 7B). Tác giả chạy lại mọi benchmark của Llama bằng pipeline đánh giá riêng của họ để so sánh công bằng. Instruct: SFT rồi DPO (Direct Preference Optimization) trên dữ liệu ưu tiên cặp.

4.1 So với họ Llama (Bảng 2)

Mô hình	Tham số hoạt động	MMLU	HellaSwag	WinoG	PIQA	Arc-e	Arc-c	NQ	TriviaQA	HumanEval	MBPP	MATH	GSM8K
LLaMA 2 7B	7B	44.4	77.1	69.5	77.9	68.7	43.2	17.5	56.6	11.6	26.1	3.9	16.0
LLaMA 2 13B	13B	55.6	80.7	72.9	80.8	75.2	48.8	16.7	64.0	18.9	35.4	6.0	34.3
LLaMA 1 33B	33B	56.8	83.7	76.2	82.2	79.6	54.4	24.1	68.5	25.0	40.9	8.4	44.1
LLaMA 2 70B	70B	69.9	85.4	80.4	82.6	79.9	56.5	25.4	73.0	29.3	49.8	13.8	69.6
Mistral 7B	7B	62.5	81.0	74.2	82.2	80.5	54.9	23.2	62.5	26.2	50.2	12.7	50.0

Mô hình	Tham số hoạt động	MMLU	HellaSwag	WinoGrande	PIQA	Arc-e	Arc-c	NQ	TriviaQA	HumanEval	MBPP	MATH	GSM8K
Mixtral 8x7B	13B	**70.6**	84.4	77.2	83.6	83.1	59.7	30.6	71.5	**40.2**	**60.7**	**28.4**	74.4



So tổng vs active: Llama-2 70B, Mixtral 47/13B, Jamba 52/12B. (minh hoạ, tự tính)

(Đơn vị %.) Mixtral thua Llama 2 70B ở HellaSwag, WinoGrande, TriviaQA (và đọc hiểu, theo paper) nhưng vượt rõ ở code và toán: HumanEval 40.2 so với 29.3, MATH 28.4 so với 13.8.

4.2 So với GPT-3.5 (Bảng 3)

	Llama 2 70B	GPT-3.5	Mixtral 8x7B
MMLU	69.9	70.0	70.6
HellaSwag (10-shot)	87.1	85.5	86.7
ARC Challenge (25-shot)	85.1	85.2	85.8
WinoGrande (5-shot)	83.2	81.6	81.2
MBPP (pass@1)	49.8	52.2	60.7
GSM-8K (5-shot)	53.6	57.1	58.4
MT Bench (Instruct)	6.86	8.32	8.30

Các điều kiện đánh giá khác với paper Llama 2: MBPP dùng tập con đã kiểm tay; TriviaQA không cung cấp ngữ cảnh Wikipedia.

4.3 Đa ngôn ngữ (Bảng 4, tập trung một số cột)

Mixtral vượt Llama 2 70B ở Pháp, Đức, Tây Ban Nha, Ý. Ví dụ ARC-c: Pháp 58.2 so với 49.9, Đức 54.3 so với 47.3; MMLU tiếng Đức 71.5 so với 64.2.

4.4 Ngữ cảnh dài

Truy xuất passkey (tìm một mã bí mật chèn ngẫu nhiên vào văn bản dài): **100%** chính xác với mọi độ dài ngữ cảnh và mọi vị trí. Perplexity trên tập con proof-pile giảm đơn điệu khi tăng độ dài ngữ cảnh.

4.5 Instruct, thiên lệch

Mixtral-Instruct đạt MT-Bench 8.30, "mô hình trọng số mở tốt nhất tháng 12/2023", Arena Elo 1121 (Claude-2.1 1117, GPT-3.5-Turbo tốt nhất 1117, Gemini Pro 1111, Llama-2-70b-chat 1077, theo ảnh chụp bảng xếp hạng LMSys ngày 22/12/2023). BBQ accuracy 56.0% so với 51.5% của Llama 2 70B (ít thiên lệch hơn); điểm cảm xúc BOLD cao hơn.

5. Phân tích định tuyến

Câu hỏi: các expert có chuyên môn hóa theo chủ đề không? Tác giả đo phân bố expert được chọn trên các tập con của The Pile (ArXiv, GitHub, PubMed, PhilPapers, StackExchange, Wikipedia, Gutenberg, DM Mathematics) ở lớp 0, 15, 31.

- **Không thấy chuyên môn hóa theo chủ đề rõ ràng.** Phân bố expert của ArXiv (LaTeX), sinh học (PubMed) và triết học (PhilPapers) rất giống nhau ở mọi lớp. Chỉ DM Mathematics hơi khác, nhiều khả năng do tính tổng hợp (synthetic) và độ phủ ngôn ngữ hẹp, nhất là ở lớp đầu/cuối.
- **Định tuyến mang tính cú pháp.** Token như `self` trong Python hay `Question` trong tiếng Anh thường vào cùng một expert dù thuộc nhiều token; token thụt lề trong code luôn gán cho cùng expert, rõ ở lớp đầu và cuối (nơi hidden state gần embedding vào/ra).
- **Cục bộ theo thời gian (temporal locality).** Token liên tiếp hay chọn cùng expert (Bảng 5). Mức ngẫu nhiên: 12,5% (expert đầu tiên giống nhau) và khoảng 46% (đầu hoặc thứ hai). Ví dụ đo được (expert lựa chọn đầu): lớp 0 khoảng 14%, lớp 15 khoảng 24-28%, lớp 31 khoảng 20-26%; với "đầu hoặc thứ hai": lớp 0 khoảng 45-50%, lớp 15 khoảng 62-67%, lớp 31 khoảng 44-54%. Tức ở lớp sâu lặp lại cao hơn ngẫu nhiên rõ rệt; lớp 0 gần ngẫu nhiên.

Hệ quả theo paper: tính cục bộ có thể làm quá tải một số expert khi dùng Expert Parallelism, nhưng cũng có thể khai thác để **cache** expert (như trong công trình offloading của Eliseev và Mazur).

Về hiệu quả và bộ nhớ (Mục 3, "Size and Efficiency"). Chi phí tính toán tỉ lệ với tham số hoạt động (13B), còn chi phí bộ nhớ tỉ lệ với tham số tổng (47B, vẫn nhỏ hơn 70B). Paper thừa nhận SMoE có overhead do định tuyến và do tải thêm bộ nhớ khi chạy nhiều hơn một expert trên mỗi thiết bị, và phù hợp với tải theo batch có cường độ số học (arithmetic intensity) tốt. Ở phần giới thiệu, paper nói Mixtral cho "inference nhanh hơn ở batch nhỏ và throughput cao hơn ở batch lớn", nhưng không đưa số đo tốc độ.

6. Giới hạn

Do tác giả nêu:

- SMoE có overhead định tuyến và tải bộ nhớ khi nhiều expert trên một thiết bị; hợp với workload có batch lớn.
- Khó cân bằng tải khi dùng Expert Parallelism.
- Chi phí bộ nhớ vẫn theo 47B tham số.

Do người viết nhận xét:

- Paper không công bố dữ liệu huấn luyện, số token, loss phụ hay siêu tham số huấn luyện, nên không tái lập được; và không có ablation về số expert, K, hay MoE-so-với-dày ở cùng compute (không có đối chứng "Mistral 7B dày với cùng số token").
- Các tuyên bố tốc độ ("nhanh hơn ở batch nhỏ") không có số đo trong paper.

- "5 lần ít tham số hoạt động" so với Llama 2 70B không phải so sánh về bộ nhớ hay latency thực tế trên GPU nhỏ.
- Điểm cuối của benchmark do chính tác giả chạy lại, có khác biệt giao thức so với paper Llama 2 (đã nêu).
- Phân tích định tuyến chỉ nhìn ba lớp trên The Pile; kết luận "không chuyên môn hóa theo chủ đề" là quan sát, không phải bằng chứng cho mọi tầng.

7. Liên hệ với các bài khác

- **27 Switch:** top-1, loss cân bằng tải, capacity factor, token drop. Mixtral: top-2, thay mọi FFN, softmax sau TopK.
- **06 Mistral 7B:** kiến trúc nền; Mixtral thêm MoE. Mixtral dùng GQA (8 KV head, bài 15) và hỗ trợ ngữ cảnh dày 32k.
- **11 SwiGLU:** mỗi expert là một SwiGLU.
- **05 LLaMA, 02 GPT-3:** mốc so sánh chất lượng (Llama 2 70B, GPT-3.5).
- **48-50 Speculative decoding, Medusa, EAGLE:** với MoE, kiểm tra nhiều token cùng lúc (verify) kích hoạt nhiều expert hơn trong một bước, làm giảm lợi thế băng thông (suy luận của người viết, paper không bàn).
- **46 vLLM:** Mixtral được tích hợp vào vLLM nhờ MegaBlocks.
- **25 Jamba, 16 DeepSeek-V2:** kiến trúc MoE về sau.

8. Gợi ý áp dụng

Với MoE, tham số hoạt động trên mỗi token, tổng bộ nhớ trọng số và độ trễ là ba chỉ số khác nhau. Số expert được chọn cho biết một phần chi phí tính toán, nhưng không quyết định trực tiếp độ trễ: định tuyến, truyền dữ liệu và phân bố token đều có thể ảnh hưởng.

9. Thí nghiệm gợi ý

Đếm tần suất chọn expert và mức trùng lặp giữa các token liên tiếp trên một tập kiểm tra. So sánh theo loại token và ngữ cảnh, tránh suy ra chuyên môn hoá theo chủ đề chỉ từ tần suất tổng.

10. Tóm tắt công thức / khái niệm

Khái niệm	Công thức / ý nghĩa
SMoE layer	$y = \sum_i G(x)_i E_i(x)$
Gating	$G(x) = \text{Softmax}(\text{TopK}(xW_g))$, K=2, n=8
Expert	khối SwiGLU, hidden 14336, dim 4096
Tham số tổng / hoạt động	47B (bộ nhớ) / 13B (FLOPs mỗi token)
Tham số một expert (tự tính)	$3 \cdot 4096 \cdot 14336 \approx 176\text{M}$
Context	32k token dày đủ, GQA 8 KV head
Locality	token i và i+1 chọn cùng expert: lớp sâu cao hơn ngẫu nhiên (12,5% / khoảng 46%)
Kết quả then chốt	MMLU 70.6, HumanEval 40.2, MATH 28.4, GSM8K 74.4; MT-Bench Instruct 8.30
Giấy phép	Apache 2.0

11. Kết luận ngắn

Mixtral 8x7B chứng minh một LLM MoE mở có thể đạt hoặc vượt Llama 2 70B và GPT-3.5 trên hầu hết benchmark chỉ với 13B tham số hoạt động (47B tổng), đặc biệt mạnh ở toán, code và đa ngôn ngữ. Điều thú

vị về cơ chế là định tuyến không chuyên môn hóa theo chủ đề mà theo cú pháp và có tính cục bộ theo thời gian.

Liên kết gốc: https://arxiv.org/abs/2401.04088

Bài 29

An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale (2020)

Tác giả: Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, Neil Houlsby (Google Research, Brain Team).
Venue / arXiv: arXiv:2010.11929 (v2); đăng ở ICLR 2021.
Nguồn: https://arxiv.org/abs/2010.11929
Chủ đề: Vision Transformer (ViT): cắt ảnh thành patch, coi mỗi patch là một token, dùng Transformer encoder chuẩn; vai trò của dữ liệu tiền huấn luyện lớn thay cho inductive bias của CNN.
Vị trí trong lộ trình: Đọc sau 01 (Transformer) và 09 (Pre-LN, vì ViT dùng LayerNorm trước mỗi khối). Đây là nền tảng của mọi bộ mã hóa ảnh trong VLM: 30 (CLIP) và 31 (SigLIP) huấn luyện ViT bằng văn bản; 33–38 dùng các bộ mã hóa ảnh trong VLM. Bài sau: 30 CLIP.

> Ghi chú diễn giải bằng tiếng Việt, không phải bản dịch. Các phép tính đánh dấu "tự tính" là ví dụ minh họa.

1. Bối cảnh

Năm 2020, trong xử lý ngôn ngữ, Transformer đã thành tiêu chuẩn: tiền huấn luyện trên kho văn bản khổng lồ rồi tinh chỉnh (BERT, GPT-3). Trong thị giác máy tính, CNN (mạng tích chập, kiểu ResNet) vẫn thống trị.

Vì sao không dùng self-attention trực tiếp trên ảnh? Self-attention có chi phí bậc hai theo số phần tử: nếu mỗi *điểm ảnh* là một token, ảnh 224×224 có 50.176 token và ma trận attention có khoảng 2,5 tỉ phần tử. Các nghiên cứu trước chữa bằng cách giới hạn attention trong vùng cục bộ, attention thưa, hoặc attention theo trục (axial); nhưng chúng cần cài đặt đặc biệt nên chưa scale hiệu quả trên phần cứng hiện có (theo paper).

Inductive bias của CNN. Inductive bias là những giả định cấu trúc đã "cài sẵn" vào mô hình. CNN có *tính cục bộ* (mỗi nơ-ron chỉ nhìn vùng lân cận), *cấu trúc lân cận 2D* và *tính bất biến tịnh tiến* (translation equivariance: dịch ảnh thì feature dịch theo). Nhờ vậy CNN học tốt với ít dữ liệu. Transformer không có các giả định này.

Giả thuyết của paper: với đủ dữ liệu (14M-300M ảnh), "large scale training trumps inductive bias": quy mô huấn luyện lấn át inductive bias.

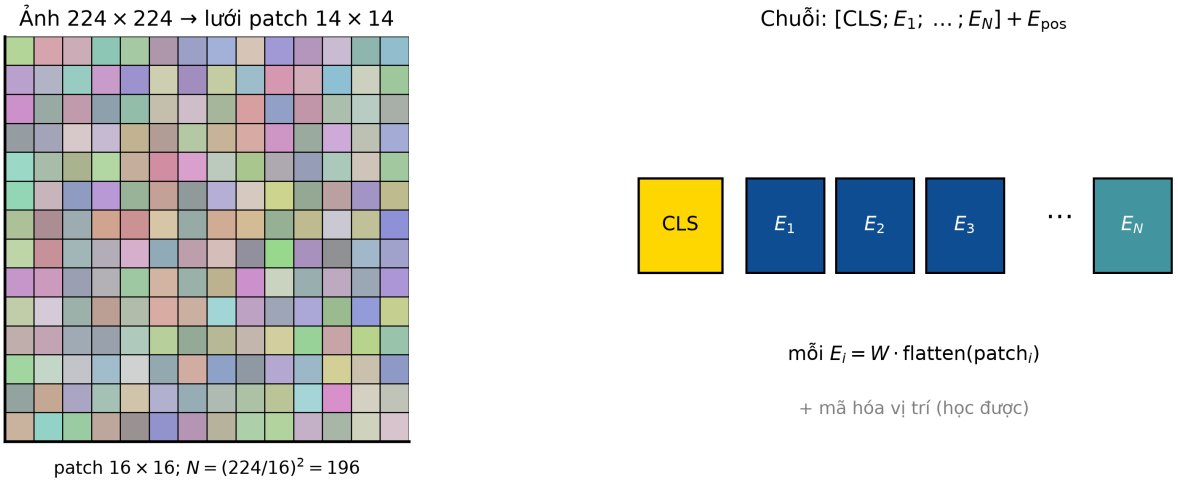
2. Ý tưởng chính

Cắt ảnh thành các ô vuông (patch) cỡ P×P, ví dụ 16×16 điểm ảnh. Duỗi mỗi patch thành một vector, chiếu tuyến tính sang chiều D, và coi chuỗi các vector ấy như chuỗi token từ vựng của một câu ("một bức ảnh đáng giá 16×16 từ"). Thêm một token đặc biệt `[class]` và embedding vị trí học được, rồi đưa chuỗi qua Transformer encoder chuẩn, gần như không đổi so với NLP. Ở quy mô nhỏ (ImageNet) mô hình thua ResNet vài điểm; nhưng tiền huấn luyện trên ImageNet-21k hoặc JFT-300M thì ViT bằng hoặc vượt các CNN tốt nhất với chi phí huấn luyện thấp hơn.

3. Phương pháp chi tiết

3.1 Chuyển ảnh thành chuỗi

Ảnh $x \in \mathbb{R}^{H \times W \times C}$ được cắt thành N patch đã duỗi $x_p \in \mathbb{R}^{N \times (P^2 \cdot C)}$ với



ViT: ảnh 224×224, patch $16 \to 14 \times 14$=196 token; chuỗi $[\text{CLS}; E_i] + E_{\text{pos}}$. (minh hoạ, tự tính)

$$\boxed{N = \frac{HW}{P^2}}$$

N chính là độ dài chuỗi hiệu dụng. Mỗi patch được chiếu bằng ma trận $E \in \mathbb{R}^{(P^2C)\times D}$ ("patch embedding"). Một token x_{class} học được được thêm vào đầu chuỗi (giống `[CLS]` của BERT); trạng thái đầu ra của nó là biểu diễn ảnh. Cộng embedding vị trí $E_{pos} \in \mathbb{R}^{(N+1)\times D}$.

3.2 Bốn phương trình của ViT

$$z_0 = [x_{class};\, x_p^1 E;\, x_p^2 E; \ldots; x_p^N E] + E_{pos}$$

$$z'_\ell = \text{MSA}(\text{LN}(z_{\ell-1})) + z_{\ell-1}, \quad \ell = 1 \ldots L$$

$$z_\ell = \text{MLP}(\text{LN}(z'_\ell)) + z'_\ell, \quad \ell = 1 \ldots L$$

$$\boxed{y = \text{LN}(z_L^0)}$$

- MSA = multi-head self-attention; LN = LayerNorm đặt *trước* mỗi khối (pre-LN, bài 09), residual đặt sau mỗi khối; MLP là hai lớp tuyến tính với phi tuyến GELU.
- Đầu phân loại: MLP một lớp ẩn khi tiền huấn luyện, một lớp tuyến tính khi fine-tune (khởi tạo bằng 0, kích thước $D \times K$).

Về mặt cài đặt, phép "cắt patch rồi chiếu tuyến tính" tương đương một lớp `Conv2d(C, D, kernel_size=P, stride=P)` (nhận xét của người viết; paper mô tả là chiếu tuyến tính của patch đã duỗi).

```
class ViT(nn.Module):
    def __init__(s, P=16, D=768, L=12, heads=12, mlp=3072, n_cls=1000, N=196):
        s.patch = nn.Conv2d(3, D, P, stride=P)            # = E  (patch embedding)
        s.cls   = nn.Parameter(torch.zeros(1, 1, D))
        s.pos   = nn.Parameter(torch.zeros(1, N + 1, D))  # 1D learnable
        s.blocks = nn.ModuleList(Block(D, heads, mlp) for _ in range(L))
        s.norm, s.head = nn.LayerNorm(D), nn.Linear(D, n_cls)
    def forward(s, img):                                   # [B,3,224,224]
        x = s.patch(img).flatten(2).transpose(1, 2)        # [B,196,D]
        x = torch.cat([s.cls.expand(len(x), -1, -1), x], 1) + s.pos
        for b in s.blocks:                                 # pre-LN: x += MSA(LN x); x +=
        ↪  MLP(LN x)
            x = b(x)
```

```
        return s.head(s.norm(x)[:, 0])
```

3.3 Các biến thể kích thước (Bảng 1)

Mô hình	Layers	Hidden D	MLP size	Heads	Params
ViT-Base	12	768	3072	12	86M
ViT-Large	24	1024	4096	16	307M
ViT-Huge	32	1280	5120	16	632M

Ký hiệu **ViT-L/16** nghĩa là cỡ Large, patch 16×16. Độ dài chuỗi tỉ lệ nghịch với P^2 nên patch nhỏ hơn thì đắt hơn nhiều (ví dụ 224×224: P=16 có 196 patch, P=32 chỉ 49, P=14 có 256).

3.4 Inductive bias của ViT

Theo paper, trong ViT chỉ các lớp MLP là cục bộ và bất biến tịnh tiến (vì áp dụng riêng cho từng token); self-attention là toàn cục. Cấu trúc lân cận 2D chỉ được dùng ở hai nơi: lúc cắt patch, và lúc nội suy embedding vị trí khi đổi độ phân giải lúc fine-tune. Embedding vị trí lúc khởi tạo không chứa thông tin về vị trí 2D; mọi quan hệ không gian giữa patch phải học từ đầu.

3.5 Fine-tune ở độ phân giải cao hơn

Thường có lợi khi fine-tune ở độ phân giải lớn hơn lúc tiền huấn luyện. Giữ nguyên kích thước patch nên chuỗi dài hơn; ViT xử lý được chuỗi dài tùy ý (trong giới hạn bộ nhớ), nhưng embedding vị trí đã học mất ý nghĩa, nên paper **nội suy 2D** embedding vị trí theo vị trí của chúng trong ảnh gốc.

3.6 Mô hình lai (hybrid)

Thay vì patch thô, chuỗi đầu vào lấy từ *feature map* của CNN (ResNet): patch 1×1 trên feature map, tức duỗi bản đồ đặc trưng rồi chiếu sang D.

4. Thí nghiệm và kết quả chính

Dữ liệu tiền huấn luyện: ImageNet-1k (1,3M ảnh, 1k lớp), ImageNet-21k (14M ảnh, 21k lớp), JFT-300M (303M ảnh, 18k lớp). Chuyển giao sang ImageNet (nhãn gốc và nhãn ReaL sạch hơn), CIFAR-10/100, Oxford-IIIT Pets, Oxford Flowers-102, và bộ VTAB 19 tác vụ (chuyển giao ít dữ liệu: 1.000 ví dụ mỗi tác vụ).

Huấn luyện: Adam ($\beta_1 = 0.9$, $\beta_2 = 0.999$), batch 4096, weight decay 0.1 (cao, giúp chuyển giao); warmup tuyến tính rồi giảm tuyến tính. Fine-tune bằng SGD momentum, batch 512. ResNet đối chứng là "BiT" (đổi BatchNorm sang GroupNorm + standardized conv).

4.1 So với SOTA (Bảng 2)

	ViT-H/14 (JFT)	ViT-L/16 (JFT)	ViT-L/16 (I21k)	BiT-L (ResNet152x4)	Noisy Student (EfficientNet-L2)
ImageNet	**88.55**	87.76	85.30	87.54	88.4/88.5*
ImageNet ReaL	90.72	90.54	88.62	90.54	90.55
CIFAR-10	99.50	99.42	99.15	99.37	-
CIFAR-100	**94.55**	93.90	93.25	93.51	-
Pets	97.56	97.32	94.67	96.62	-
Flowers-102	99.68	99.74	99.61	99.63	-

	ViT-H/14 (JFT)	ViT-L/16 (JFT)	ViT-L/16 (I21k)	BiT-L (ResNet152x4)	Noisy Student (EfficientNet-L2)
VTAB (19 tác vụ)	**77.63**	76.28	72.72	76.29	-
TPUv3-core-days	2,5k	0,68k	0,23k	9,9k	12,3k

ViT-L/16 trên JFT vượt BiT-L trên mọi tác vụ với chi phí huấn luyện khoảng 1/14 (0,68k so với 9,9k core-days, tự tính tỉ lệ). ViT-L/16 trên I21k đủ rẻ để huấn luyện trên một TPUv3 8 core trong khoảng 30 ngày (theo paper).

4.2 Yêu cầu về dữ liệu tiền huấn luyện

- Trên ImageNet-1k (nhỏ nhất): ViT-Large kém ViT-Base dù đã tối ưu regularization (weight decay, dropout, label smoothing); BiT CNN thắng ViT trên ImageNet.
- Trên ImageNet-21k: ViT-L ngang ViT-B.
- Trên JFT-300M: ViT lớn mới phát huy; ViT vượt BiT.
- Thí nghiệm tập con JFT (9M, 30M, 90M, 300M; không thêm regularization, dùng early stopping): ViT-B/32 hơi nhanh hơn ResNet50 nhưng kém hẳn ở 9M và tốt hơn ở 90M trở lên. Kết luận của paper: **inductive bias tích chập hữu ích khi dữ liệu nhỏ; với dữ liệu lớn, học thẳng từ dữ liệu là đủ, thậm chí tốt hơn**.

4.3 Scaling study (Bảng 6, Hình 5)

Trên JFT-300M, so ViT, ResNet và hybrid theo tổng compute tiền huấn luyện (exaFLOPs). Một số hàng: ViT-B/16 (7 epoch) 224 exaFLOPs, ImageNet 84.15; ViT-L/16 (14 epoch) 1567 exaFLOPs, ImageNet 87.12; ViT-H/14 (14 epoch) 4262 exaFLOPs, 88.08; ResNet152x2 (14 epoch) 1126 exaFLOPs, 85.56; ResNet200x3 (14 epoch) 3306 exaFLOPs, 87.22. Ba kết luận của paper: (1) ViT thắng ResNet về đánh đổi hiệu năng/compute, dùng khoảng 2-4 lần ít compute để đạt cùng kết quả; (2) hybrid hơn ViT một chút ở ngân sách nhỏ nhưng khác biệt biến mất ở mô hình lớn; (3) ViT chưa bão hòa trong dải đã thử.

4.4 Tự giám sát (preliminary)

Dự đoán patch bị che (masked patch prediction, giống MLM của BERT): ViT-B/16 đạt 79.9% ImageNet, +2% so với huấn luyện từ đầu nhưng vẫn kém 4% so với tiền huấn luyện có giám sát.

5. Phân tích / ablation

Nhìn bên trong ViT (Mục 4.5).

- Các filter của lớp chiếu patch đầu tiên giống các hàm cơ sở hợp lý (thành phần chính của "nét mịn" trong patch).
- Embedding vị trí *tự học* cấu trúc 2D: patch gần nhau có embedding giống nhau, cấu trúc hàng-cột xuất hiện, đôi khi có dạng sin ở lưới lớn. Đây là lý do embedding 2D thủ công không cải thiện.
- **Khoảng cách attention (attention distance)**, tương tự receptive field của CNN: một số head ở lớp thấp đã nhìn gần toàn ảnh, số khác chỉ nhìn cục bộ; nhóm cục bộ ít xuất hiện ở mô hình lai (có ResNet phía trước), gợi ý chúng làm việc giống các lớp tích chập đầu. Khoảng cách tăng theo độ sâu.

Kiểu embedding vị trí (Bảng 8, ViT-B/16, ImageNet 5-shot linear).

Loại	Kết quả
Không có	0,61382

Loại	Kết quả
1D (mặc định)	0,64206
2D (X và Y, nối)	0,64001
Tương đối (relative)	0,64032

Có vị trí tốt hơn không có (khoảng 3 điểm), nhưng các cách mã hóa khác nhau cho kết quả tương đương; tác giả giải thích bởi việc mô hình tự học cấu trúc 2D.

Đầu phân loại (Phụ lục D.3). Dùng trung bình toàn cục (GAP) thay token `[class]` ban đầu kém, nhưng nguyên nhân là learning rate cần khác chứ không phải GAP hay token thêm; chỉnh learning rate thì hai cách tương đương.

Hình dạng Transformer (Phụ lục D.2, mô hình cơ sở 8 lớp, D=1024, MLP 2048, patch 32). Tăng độ sâu cải thiện nhiều nhất (rõ tới 64 lớp, giảm lợi ích sau 16 lớp); tăng độ rộng ít thay đổi nhất; giảm patch (tăng độ dài chuỗi) cải thiện ổn định mà không thêm tham số. Paper kết luận compute dự đoán hiệu năng tốt hơn số tham số, và nếu phải chọn thì nên mở rộng chiều sâu; mở rộng đồng đều mọi chiều đều ổn định.

Chi phí thực đo (Phụ lục D.5, TPUv3). Chi phí lý thuyết "bậc hai theo số patch" (tức bậc 4 theo cạnh ảnh) chỉ bắt đầu hiện rõ ở mô hình lớn nhất ở độ phân giải cao nhất; ViT có tốc độ tương đương ResNet cùng cỡ và **dùng bộ nhớ hiệu quả hơn** (batch tối đa mỗi core lớn hơn).

Axial attention (D.6). Axial-ViT-B/32 và B/16 tốt hơn ViT-B tương ứng về chất lượng nhưng tốn compute nhiều hơn (và phiên bản AxialResNet cài đặt chậm trên TPU).

Optimizer (D.1). Với ResNet, Adam tiền huấn luyện nhỉnh hơn SGD (trung bình 89.33 so với 88.79 cho ResNet50; 94.01 so với 93.72 cho ResNet152x2).

6. Giới hạn

Do tác giả nêu: chỉ mới đánh giá phân loại ảnh; cần mở rộng sang phát hiện và phân đoạn; tiền huấn luyện tự giám sát còn kém giám sát có nhãn lớn (79.9% so với khoảng 84%); mô hình còn có thể scale thêm. ViT kém hơn ResNet khi dữ liệu nhỏ vì thiếu inductive bias.

Do người viết nhận xét:

- Kết quả hàng đầu cần JFT-300M, tập dữ liệu nội bộ của Google, không công khai; người ngoài chỉ có ImageNet-21k.
- Chuỗi có độ dài cố định, patch thô: các chi tiết nhỏ hơn patch (ví dụ ký tự nhỏ trong tài liệu) bị trộn trong một vector; lấy mẫu độ phân giải thấp sẽ mất thông tin (không phải điều paper bàn, đây là suy luận từ thiết kế).
- Chi phí attention bậc hai theo N vẫn tồn tại khi tăng độ phân giải (giải pháp: cửa sổ, merge token, v.v. ở các bài sau).
- Không có thí nghiệm đo độ bền trước biến đổi ảnh hay dữ liệu tài liệu.

7. Liên hệ với các bài khác

- **01 Attention Is All You Need:** ViT dùng encoder gần như nguyên bản. **09 Pre-LN:** LN đặt trước mỗi khối.
- **30 CLIP:** CLIP thử cả ResNet và ViT; ViT cho hiệu quả compute cao hơn khoảng 3 lần ở CLIP, tái hiện kết luận của bài này. **31 SigLIP:** dùng ViT làm bộ mã hóa ảnh (B/16, So400m...).
- **12 RoPE:** ViT dùng embedding vị trí học được 1D; các VLM sau (Qwen2-VL, bài 37) dùng vị trí 2D tương đối/xoay để xử lý độ phân giải động.

- **34-38 (LLaVA, InternVL, Qwen2-VL, SmolVLM):** nối đầu ra của ViT vào LLM. **39 Donut:** bộ mã hóa ảnh kiểu Swin cho OCR. **57 FastV:** cắt bớt visual token.
- **44 FlashAttention:** làm self-attention với chuỗi patch dài khả thi về bộ nhớ.

8. Gợi ý áp dụng

Với ảnh $H \times W$ và patch vuông cạnh P, ViT tạo HW/P^2 patch trước các token bổ sung. Tăng độ phân giải làm tăng số token và chi phí attention bậc hai theo số token. Có thể nội suy embedding vị trí để xử lý độ phân giải khác, nhưng chất lượng cần được đo trên dữ liệu đích.

Ví dụ tự tính với ViT-B/16, $D = 768$, $L = 12$: mỗi khối có khoảng $12D^2 \approx 7,08$ triệu tham số, toàn bộ 12 khối khoảng 85 triệu, chưa kể các thành phần khác. Ở ảnh 224×224, có 196 patch; chi phí attention tăng nhanh hơn phần chiếu và MLP khi tăng số patch.

9. Thí nghiệm gợi ý

Đo thời gian, bộ nhớ và chất lượng khi thay đổi độ phân giải; đối chiếu số patch với chi phí quan sát được. Vẽ khoảng cách attention trung bình theo head và tầng để khảo sát mức độ cục bộ của biểu diễn.

10. Tóm tắt công thức / khái niệm

Khái niệm	Công thức / ý nghĩa
Số patch	$N = HW/P^2$ (độ dài chuỗi, +1 cho `[class]`)
Patch embedding	$E \in \mathbb{R}^{(P^2C)\times D}$, tương đương Conv2d stride P
Đầu vào	$z_0 = [x_{class}; x_p^i E] + E_{pos}$
Khối encoder	$z' = \text{MSA}(\text{LN}z) + z$; $z = \text{MLP}(\text{LN}z') + z'$
Biểu diễn ảnh	$y = \text{LN}(z_L^0)$
Cỡ mô hình	B: 12L/768/86M; L: 24L/1024/307M; H: 32L/1280/632M
Vị trí	1D học được; nội suy 2D khi đổi độ phân giải
Inductive bias	ít hơn CNN: chỉ patch và nội suy vị trí dùng cấu trúc 2D
Kết quả then chốt	ViT-H/14 (JFT): ImageNet 88.55; VTAB 77.63; 2,5k TPUv3-core-days
Chi phí attention (tự tính)	$\approx 4N^2D$ FLOPs mỗi lớp, so với $\approx 24ND^2$ cho phần tuyến tính

11. Kết luận ngắn

ViT cho thấy một Transformer chuẩn, chỉ cần cắt ảnh thành patch và thêm embedding vị trí, có thể bằng hoặc hơn CNN tốt nhất khi được tiền huấn luyện trên hàng chục đến hàng trăm triệu ảnh, với chi phí tiền huấn luyện thấp hơn. Mô hình tự học cấu trúc không gian 2D thay vì được cài sẵn.

Liên kết gốc: https://arxiv.org/abs/2010.11929

Bài 30

Learning Transferable Visual Models From Natural Language Supervision (CLIP, 2021)

Tác giả: Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, Gretchen Krueger, Ilya Sutskever (OpenAI).
Venue / arXiv: arXiv:2103.00020 (v1, 26/2/2021). Paper được biết đến là bài ICML 2021, nhưng bản text này không ghi venue.
Nguồn: https://arxiv.org/abs/2103.00020
Chủ đề: Contrastive Language-Image Pre-training (CLIP): huấn luyện song song bộ mã hóa ảnh và văn bản trên 400 triệu cặp (ảnh, chú thích) bằng loss tương phản đối xứng; zero-shot transfer; robustness.
Vị trí trong lộ trình: Đọc sau 29 (ViT, bộ mã hóa ảnh) và 01 (Transformer, bộ mã hóa văn bản). CLIP là nền của hầu hết VLM: 31 (SigLIP) thay softmax bằng sigmoid; 32-38 dùng các bộ mã hóa ảnh kiểu CLIP/SigLIP nối vào LLM.

> Ghi chú diễn giải, không phải bản dịch. Báo cáo dài; phần dưới tập trung vào mục tiêu tương phản, huấn luyện, zero-shot, các kết quả tiêu biểu và giới hạn.

1. Bối cảnh

Hạn chế của thị giác máy tính có giám sát cố định. Các hệ thống thị giác lúc đó học dự đoán một tập nhãn *định sẵn* (ví dụ 1000 lớp ImageNet). Muốn nhận một khái niệm mới phải thu thêm dữ liệu có nhãn và huấn luyện lại bộ phân loại. Trong NLP, ngược lại, các mô hình tiền huấn luyện trên văn bản thô (GPT-3 và tiền bối) có thể làm nhiều tác vụ mới chỉ bằng cách *mô tả tác vụ bằng ngôn ngữ* (zero-shot), không cần đầu ra chuyên biệt cho từng dataset.

Học từ văn bản đi kèm ảnh không mới nhưng chưa thành công ở quy mô lớn. Trước CLIP đã có Visual N-Grams (Li et al., 2017: 11,5% độ chính xác zero-shot trên ImageNet), VirTex, ICMLM, ConVIRT; nhưng chúng huấn luyện trên 100-200 nghìn ảnh trong vài ngày, trong khi các hệ thống giám sát yếu (Instagram hashtag, JFT-300M) huấn luyện trên hàng triệu đến hàng tỉ ảnh nhưng bị giới hạn ở 1000-18291 lớp, dùng softmax cố định, không có cơ chế đầu ra động. CLIP lấp khoảng trống *ở quy mô*.

Prerequisite: học tương phản (contrastive learning). Thay vì dự đoán nhãn, mô hình học một không gian embedding mà các cặp "khớp nhau" có vector gần nhau còn các cặp không khớp có vector xa nhau.

2. Ý tưởng chính

Thu thập 400 triệu cặp (ảnh, văn bản) từ Internet. Huấn luyện hai bộ mã hóa cùng lúc: một cho ảnh, một cho văn bản. Với mỗi batch gồm N cặp, bảng tương đồng N×N (cosine giữa mọi ảnh và mọi văn bản) được tính; nhiệm vụ là làm cho N ô trên đường chéo (cặp thật) có điểm cao nhất trong hàng của nó và trong cột của nó. Khi dùng, muốn phân loại ảnh vào các lớp bất kỳ, ta viết tên lớp thành câu ("a photo of a dog"), mã hóa bằng bộ mã hóa văn bản, và chọn lớp có embedding gần ảnh nhất. Bộ mã hóa văn bản thực chất "sinh ra" một bộ phân loại tuyến tính theo yêu cầu, nên không cần dữ liệu huấn luyện cho tác vụ mới.

3. Phương pháp chi tiết

3.1 Dữ liệu: WIT (WebImageText)

400 triệu cặp (ảnh, văn bản) thu từ nhiều nguồn công khai trên Internet. Để phủ nhiều khái niệm thị giác, tìm các cặp có văn bản chứa một trong 500.000 truy vấn (tất cả từ xuất hiện ≥ 100 lần trong Wikipedia tiếng Anh,

bigram có PMI cao, tên bài Wikipedia đủ lượt tìm kiếm, và WordNet synsets), cân bằng gần đúng bằng cách lấy tối đa 20.000 cặp cho mỗi truy vấn. Tổng số từ ngang WebText của GPT-2. (So sánh: MS-COCO và Visual Genome khoảng 100 nghìn ảnh; YFCC100M lọc còn khoảng 15 triệu.)

3.2 Vì sao dùng mục tiêu tương phản (Hình 2)

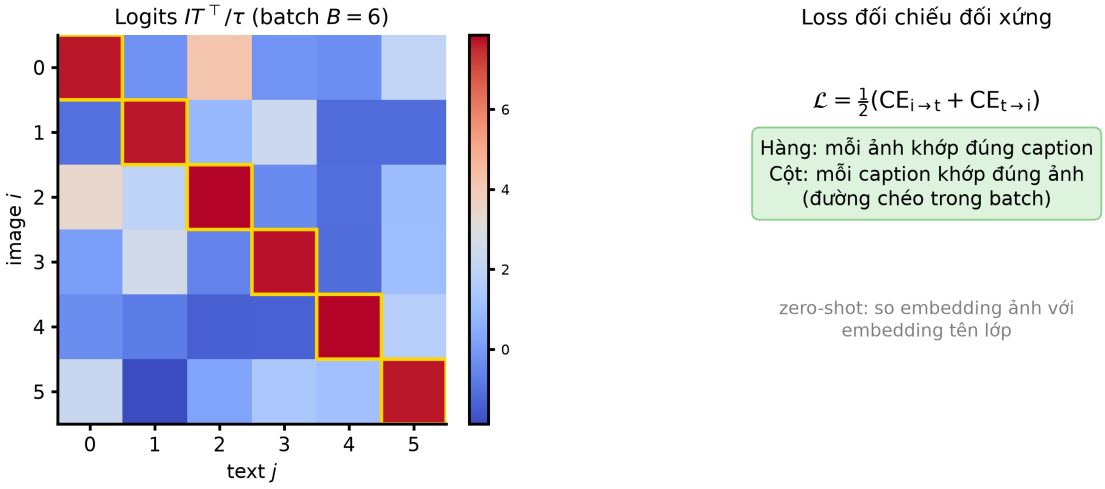
Thử ba cách dự đoán văn bản từ ảnh với tiêu chí *tốc độ đạt độ chính xác zero-shot ImageNet theo số ảnh đã xử lý*:

1. Transformer sinh chú thích (dự đoán chính xác từng từ), giống VirTex: baseline chậm nhất.
2. Dự đoán *túi từ* (bag of words) của chú thích: học nhanh gấp **3 lần** (1).
3. Mục tiêu tương phản (chỉ dự đoán cặp nào khớp với cặp nào, không cần dự đoán từ cụ thể): thêm **4 lần** hiệu quả nữa so với (2).

Lý do (theo paper): dự đoán chính xác từng từ trong chú thích rất khó vì cùng một ảnh có vô số cách mô tả; bài toán "cặp nào đi với cặp nào" dễ hơn và cho biểu diễn tốt hơn. Trọng tâm của phần phương pháp là hiệu quả huấn luyện.

3.3 Loss tương phản đối xứng

Batch gồm N cặp (ảnh I_k, văn bản T_k). Bộ mã hóa ảnh và văn bản cho đặc trưng, sau đó chiếu *tuyến tính* sang không gian embedding chung chiều d_e và chuẩn hóa L2:



CLIP: ma trận logits ảnh–text trong batch và loss CE đối xứng hai chiều. (minh hoạ, tự tính)

$$\mathbf{u}_i = \frac{W_i f_{img}(I_i)}{\|\cdot\|}, \qquad \mathbf{v}_j = \frac{W_t f_{txt}(T_j)}{\|\cdot\|}$$

Ma trận logit $S_{ij} = e^t \cdot \mathbf{u}_i^\top \mathbf{v}_j \in \mathbb{R}^{N\times N}$, trong đó $e^t = 1/\tau$ là nghịch đảo nhiệt độ học được. Hàm mất mát là cross-entropy theo cả hai chiều với nhãn là đường chéo:

$$\mathcal{L} = \frac{1}{2}\Bigg[\underbrace{\tfrac{1}{N}\sum_i -\log\frac{e^{S_{ii}}}{\sum_j e^{S_{ij}}}}_{\text{ảnh}\to\text{văn bản}} + \underbrace{\tfrac{1}{N}\sum_j -\log\frac{e^{S_{jj}}}{\sum_i e^{S_{ij}}}}_{\text{văn bản}\to\text{ảnh}}\Bigg]$$

Mỗi ảnh coi $N-1$ văn bản còn lại trong batch là mẫu âm; có tổng cộng $N^2 - N$ cặp âm. Dạng này gốc từ N-pair loss (Sohn, 2016), được biết đến là InfoNCE (Oord et al., 2018), ConVIRT áp dụng cho ảnh y khoa.

Pseudo-code của paper (Hình 3), viết lại bằng PyTorch:

```
I_f = image_encoder(I)                          # [n, d_i]
T_f = text_encoder(T)                           # [n, d_t]
I_e = F.normalize(I_f @ W_i, dim=1)             # [n, d_e]
T_e = F.normalize(T_f @ W_t, dim=1)
logits = I_e @ T_e.T * t.exp()                  # [n, n], t học được (log-parameterized)
labels = torch.arange(n)
loss = (F.cross_entropy(logits, labels) +       # ảnh -> văn bản (theo hàng)
        F.cross_entropy(logits.T, labels)) / 2 # văn bản -> ảnh (theo cột)
```

Đơn giản hóa so với ConVIRT (theo paper): huấn luyện từ đầu (không khởi tạo bằng ImageNet hay mô hình ngôn ngữ), chỉ dùng phép chiếu *tuyến tính* (không phi tuyến) sang không gian embedding, bỏ việc lấy ngẫu nhiên một câu của văn bản, augmentation chỉ là random square crop. Nhiệt độ τ được tối ưu trực tiếp (tham số hóa theo log), khởi tạo tương đương 0,07, và bị chặn để không làm logit nhân quá 100 (cần cho ổn định).

3.4 Kiến trúc

Bộ mã hóa ảnh (hai họ):

- ResNet-50 sửa đổi (ResNet-D, anti-aliased blur pooling), thay global average pooling bằng **attention pooling** (một lớp attention nhiều đầu kiểu transformer, query lấy từ trung bình toàn cục).
- Vision Transformer (bài 29), theo sát bản gốc, chỉ thêm một LayerNorm lên tổng patch + position embedding trước khi vào transformer và khởi tạo hơi khác.

Bộ mã hóa văn bản: Transformer kiểu GPT-2: cơ sở 12 lớp, rộng 512, 8 head, khoảng 63M tham số; BPE chữ thường, từ vựng 49.152, độ dài chuỗi tối đa 76; chuỗi được bao bởi `[SOS]` và `[EOS]`, và **kích hoạt của lớp cao nhất tại token `[EOS]`** được lấy làm đặc trưng văn bản (rồi layer norm và chiếu tuyến tính). Dùng masked (causal) self-attention nhằm giữ khả năng khởi tạo từ LM hoặc thêm mục tiêu ngôn ngữ sau này.

Scaling: ResNet theo kiểu EfficientNet (tăng đồng đều chiều rộng, sâu, độ phân giải): RN50, RN101 và RN50x4/x16/x64 (khoảng 4/16/64 lần compute RN50). Với văn bản chỉ tăng độ rộng theo tỉ lệ, không tăng độ sâu (vì CLIP ít nhạy với dung lượng bộ mã hóa văn bản). ViT: B/32, B/16, L/14.

3.5 Huấn luyện

Mục	Giá trị (Bảng 18-20 và Mục 2.5)
Epoch	32 (khoảng 12,8 tỉ ảnh nhìn thấy = 400M × 32)
Batch	32.768 (rất lớn)
Optimizer	Adam (decoupled weight decay 0,2), cosine schedule, warmup 2000 bước
Độ chính xác	mixed precision, gradient checkpointing, Adam stats nửa độ chính xác
Tính similarity	chia nhỏ (sharded): mỗi GPU chỉ tính phần ma trận tương ứng batch cục bộ
Chi phí	RN50x64: 18 ngày trên 592 GPU V100; ViT-L/14: 12 ngày trên 256 V100
Mô hình tốt nhất	ViT-L/14@336px (thêm 1 epoch ở độ phân giải 336)
ViT-L/14	24 lớp, rộng 1024, 16 head; embedding 768; ảnh 224

Hai ý quan trọng: (a) quy mô batch quyết định số mẫu âm (32.767 mỗi ảnh); (b) ma trận $N \times N$ lớn nên cần phân tán tính toán; đây chính là điểm mà SigLIP (bài 31) cải tiến.

3.6 Zero-shot transfer: bộ phân loại sinh từ văn bản

Với một dataset, lấy tên các lớp, nhúng bằng bộ mã hóa văn bản được tập $\{\mathbf{v}_c\}$; với ảnh x có **u**, phân phối dự đoán:

$$p(c \mid x) = \text{softmax}_c(e^t \, \mathbf{u}^\top \mathbf{v}_c).$$

Paper diễn giải: đây là hồi quy logistic đa lớp với đầu vào và trọng số chuẩn hóa L2, không bias, có nhiệt độ; bộ mã hóa ảnh là backbone, bộ mã hóa văn bản là **hypernetwork** sinh trọng số cho bộ phân loại tuyến tính từ mô tả văn bản. Bộ phân loại được tính một lần và lưu đệm (cache), nên chi phí sinh nó được chia đều cho mọi dự đoán.

Cách nhìn thú vị: mỗi bước tiền huấn luyện giống tối ưu một "dataset ngẫu nhiên" có 1 ví dụ mỗi lớp và 32.768 lớp (định nghĩa bằng ngôn ngữ).

Prompt engineering và ensembling (Mục 3.1.4).

- Tên lớp đứng trơ trọi thường mơ hồ (đa nghĩa như "crane", và hiếm khi chú thích chỉ là một từ). Dùng mẫu `"A photo of a {label}."` tăng độ chính xác ImageNet 1,3% so với dùng tên trần.
- Có thể thêm ngữ cảnh: `"A photo of a {label}, a type of pet."` (Pets), "satellite photo of..." cho ảnh vệ tinh, và với OCR đặt chữ/số trong ngoặc kép giúp ích.
- Tập hợp (ensemble) 80 mẫu ngữ cảnh khác nhau trên ImageNet cải thiện thêm 3,5% so với một mẫu mặc định. Qua 36 dataset, prompt + ensembling tăng trung bình gần 5 điểm, tương đương việc dùng nhiều gấp 4 lần compute với baseline, nhưng "miễn phí" khi chia đều cho nhiều dự đoán.

4. Thí nghiệm và kết quả chính

4.1 Zero-shot so với Visual N-Grams (Bảng 1)

Dataset	Visual N-Grams	CLIP
aYahoo	72,4	98,4
ImageNet	11,5	**76,2**
SUN	23,0	58,5

CLIP đạt 76,2% top-1 ImageNet zero-shot, ngang ResNet-50 gốc có giám sát (không dùng một trong 1,28 triệu ảnh có nhãn), top-5 là 95% (ngang Inception-V4). Paper lưu ý đây chỉ để đặt vào ngữ cảnh: dữ liệu lớn hơn 10 lần, mô hình dùng nhiều hơn khoảng 100 lần compute mỗi dự đoán, và khoảng hơn 1000 lần compute huấn luyện.

4.2 Zero-shot so với bộ phân loại tuyến tính có giám sát trên ResNet-50 (Hình 5)

Trên bộ 27 dataset, CLIP zero-shot thắng hồi quy logistic có giám sát trên đặc trưng ResNet-50 ở **16 trong 27 dataset**, kể cả ImageNet. Phân hóa mạnh:

- Thắng hơn 20 điểm: Stanford Cars, Food101; thắng rõ trên các tác vụ nhận diện hành động video (Kinetics-700: hơn 14,5 điểm; UCF101 hơn 7,7 điểm).
- Thua hơn 10 điểm: Flowers102, FGVCAircraft (phân loại tinh vi, có thể do ít dữ liệu liên quan trong WIT, theo tác giả dự đoán).
- Rất yếu ở tác vụ chuyên biệt/trừu tượng: ảnh vệ tinh (EuroSAT, RESISC45), biển báo Đức (GTSRB), ước tính khoảng cách xe (KITTI), đếm đối tượng, phân loại khối u hạch (PatchCamelyon); gần ngẫu nhiên với tác vụ lạ.

4.3 Zero-shot so với few-shot (Hình 6, 7)

Zero-shot CLIP **ngang trung bình bộ phân loại tuyến tính 4-shot** trên cùng không gian đặc trưng, và gần bằng 16-shot tốt nhất công khai (BiT-M ResNet-152x2 trên ImageNet-21k). Ước lượng số ví dụ cần cho mỗi

lớp để bộ phân loại tuyến tính khớp zero-shot CLIP: từ dưới 1 đến 184; trung vị 5,4, trung bình 20,8; trên ImageNet tương đương 16-shot. Lý do giả định của tác giả: zero-shot dùng *khái niệm tường minh qua ngôn ngữ*, còn học từ ít ví dụ phải suy khái niệm gián tiếp (mơ hồ nhiều giả thuyết). Hệ quả lạ: chuyển từ zero-shot sang few-shot bằng linear probe lại làm *giảm* hiệu năng ở ít ví dụ, vì mất "prior" ngôn ngữ (bất lợi nêu ở Mục Limitations).

4.4 Scaling

Lỗi zero-shot trung bình theo compute theo xu hướng log-log tuyến tính qua dải compute 44 lần (5 mô hình CLIP); từng dataset riêng nhiễu hơn.

4.5 Linear probe (biểu diễn)

Trên bộ 12 dataset của Kornblith et al.: CLIP nhỏ (RN50, RN101) hơn ResNet ImageNet-1k nhưng kém BiT-M (ImageNet-21k) và EfficientNet cùng compute; nhưng mô hình lớn scale tốt: RN50x64 nhỉnh hơn mô hình tốt nhất hiện có (Noisy Student EfficientNet-L2) cả điểm và hiệu quả compute. **CLIP ViT hiệu quả compute hơn CLIP ResNet khoảng 3 lần**, tái hiện kết luận của bài 29. Mô hình tốt nhất ViT-L/14@336 hơn mô hình tốt nhất trước đó trung bình 2,6 điểm trên 12 dataset; trên bộ rộng 27 dataset CLIP thắng Noisy Student ở 21/27 dataset, cải thiện nhiều nhất ở tác vụ cần OCR (SST2, HatefulMemes), nhận diện hành động và định vị địa lý.

4.6 Robustness với dịch chuyển phân phối tự nhiên (Hình 13, 14)

Mô hình ImageNet có giám sát rơi mạnh khi chuyển sang ImageNetV2/R/Sketch/A/ObjectNet (khác biệt là "robustness gap"). Zero-shot CLIP **thu hẹp khoảng cách này tới 75%** ("effective robustness" cao hơn nhiều so với mô hình ImageNet cùng độ chính xác). Thí nghiệm kiểm chứng: huấn luyện hồi quy logistic có L2 trên đặc trưng CLIP với tập huấn luyện ImageNet làm tăng độ chính xác ImageNet thêm 9,2% (lên 85,4%) nhưng độ chính xác trung bình dưới dịch chuyển phân phối *hơi giảm*. Kết luận mở: robustness đến từ đâu (zero-shot hay dữ liệu đa dạng?) chưa chắc, tác giả nêu rõ "chưa có câu trả lời tự tin".

4.7 OCR và dữ liệu chữ

CLIP học được OCR ở mức ngữ nghĩa với chữ in số hóa (Rendered SST2: tập tác giả tạo bằng cách render câu SST ra ảnh 448×448 để đo năng lực đọc chữ). Nhưng trên chữ số viết tay MNIST chỉ đạt 88% và một baseline hồi quy logistic trên điểm ảnh thô còn hơn (Mục Limitations). Các bảng OCR chi tiết ở phụ lục bị lỗi định dạng trong bản text nên ghi chú này không trích số.

4.8 Chồng lấn dữ liệu (Mục 5)

Phát hiện trùng lặp giữa tập tiền huấn luyện và 35 tập đánh giá: trung vị chồng lấn 2,2%, trung bình 3,2%; độ chính xác tổng thường thay đổi dưới 0,1%; lớn nhất 0,6% ở Birdsnap. Kết luận: nhiễm dữ liệu không giải thích kết quả.

4.9 So với con người (Mục 4, trích)

Trên Oxford Pets với 5 người: so sánh zero-shot, one-shot và two-shot cho thấy người tăng lớn từ zero-shot sang one-shot, còn CLIP lại không (đây là quan sát được nêu ở Limitations, các con số chi tiết bị mangled nên không trích).

5. Phân tích / ablation

- **Mục tiêu tương phản vs dự đoán**: xem 3.2 (3 lần rồi thêm 4 lần hiệu quả).
- **Bộ mã hóa ảnh**: ViT cho hiệu quả compute tốt hơn ResNet (khoảng 3 lần).
- **Bộ mã hóa văn bản**: chỉ tăng độ rộng, không tăng độ sâu; CLIP ít nhạy với dung lượng văn bản (theo tác giả).

- **Prompt/ensembling**: +1,3 và +3,5 điểm trên ImageNet; trung bình gần 5 điểm qua 36 dataset.
- **Tách "tác vụ" và "dịch chuyển phân phối"**: với dataset như CIFAR-10, zero-shot phản ánh khả năng tổng quát hóa miền hơn là tổng quát hóa tác vụ (Mục 3.1.1).
- **Lúc mở rộng:** ước tính cần khoảng 1000 lần compute nữa để zero-shot CLIP đạt SOTA tổng thể, không khả thi với phần cứng hiện tại.

6. Giới hạn

Do tác giả nêu:

- Zero-shot vẫn dưới SOTA có giám sát ở nhiều dataset, cần khoảng 1000 lần compute để đuổi kịp.
- Yếu ở phân loại tinh (mẫu xe, hoa, máy bay), đếm, tác vụ trừu tượng/hệ thống, tác vụ lạ (đo khoảng cách) gần ngẫu nhiên.
- Tổng quát hóa kém với dữ liệu thực sự ngoài phân phối (MNIST 88%).
- Chỉ chọn trong tập khái niệm cho trước (không sinh được đầu ra mới như captioning); đề xuất kết hợp mục tiêu tương phản và sinh.
- Hiệu quả dữ liệu thấp: nhìn 12,8 tỉ ảnh qua 32 epoch, tương đương 405 năm nếu mỗi ảnh một giây.
- Đã dùng validation set đầy đủ để phát triển trong khi nói là zero-shot, và bộ 27 dataset đánh giá được chọn tùy tiện, đồng thích nghi với CLIP.
- Dữ liệu mạng không lọc nên học thiên lệch xã hội (Mục 7 paper phân tích).
- Chuyển từ zero-shot sang few-shot qua linear probe làm giảm hiệu năng, khác với người.

Do người viết nhận xét:

- Bộ mã hóa văn bản gồm 76 token và biểu diễn một vector toàn cục cho cả ảnh; không có biểu diễn cấp patch/pixel cho định vị hay đọc chữ dày đặc.
- Batch 32.768 cùng ma trận N×N làm cho đây là một mục tiêu đòi hỏi quy mô phần cứng lớn; không dễ tái lập ngoài nhóm lớn.
- Dữ liệu WIT không công khai.

7. Liên hệ với các bài khác

- **29 ViT:** bộ mã hóa ảnh; CLIP xác nhận ViT hiệu quả compute hơn ResNet.
- **31 SigLIP:** thay softmax toàn batch bằng sigmoid từng cặp, tránh ma trận N×N và all-gather; cho kết quả tốt hơn ở batch nhỏ.
- **02 GPT-3:** CLIP lấy cảm hứng từ zero-shot/prompting của GPT; **01 Transformer** cho bộ mã hóa văn bản.
- **32 Flamingo, 33 BLIP-2, 34 LLaVA, 35 LLaVA-1.5:** dùng bộ mã hóa ảnh kiểu CLIP (đóng băng) nối với LLM; **36 InternVL, 37 Qwen2-VL, 38 SmolVLM:** các biến thể về sau.
- **07 InstructGPT:** thái độ "mô tả tác vụ bằng ngôn ngữ" tương tự.

8. Gợi ý áp dụng

CLIP học biểu diễn ảnh và văn bản thông qua mục tiêu tương phản toàn cục. Điều đó không đảm bảo mọi chi tiết cục bộ được giữ lại: các tác vụ cần đọc chữ nhỏ hoặc định vị chính xác cần được đánh giá riêng. Độ nhạy với cách diễn đạt nhãn cũng khiến việc chọn prompt quan trọng khi phân loại zero-shot.

9. Thí nghiệm gợi ý

Huấn luyện mô hình ảnh–văn bản nhỏ với loss tương phản, thay đổi batch và nhiệt độ; đo chất lượng truy hồi trên tập giữ riêng. Với zero-shot, so sánh các mẫu mô tả lớp khác nhau để kiểm tra độ nhạy với prompt.

10. Tóm tắt công thức / khái niệm

Khái niệm	Công thức / ý nghĩa
Embedding chuẩn hóa	$\mathbf{u} = \text{norm}(W_i f_{img})$, $\mathbf{v} = \text{norm}(W_t f_{txt})$
Logit	$S_{ij} = e^t\, \mathbf{u}_i^\top \mathbf{v}_j$, t học được, chặn $e^t \le 100$
Loss	cross-entropy đối xứng theo hàng và cột, nhãn là đường chéo
Số cặp âm	$N^2 - N$ (với $N = 32768$, khoảng $1{,}07 \times 10^9$ cặp, tự tính)
Bộ nhớ ma trận logit (tự tính)	$32768^2 \times 4$ byte ≈ 4,3 GB fp32 nếu không chia nhỏ
Zero-shot	$p(c\|x) = \text{softmax}(e^t \mathbf{u}^\top \mathbf{v}_c)$, $\mathbf{v}_c$ từ prompt tên lớp
Prompt	`"A photo of a {label}."` +1,3% ImageNet; ensemble 80 prompt +3,5%
Dữ liệu	WIT: 400M cặp, 500k truy vấn, ≤ 20k cặp/truy vấn
Huấn luyện	32 epoch, batch 32.768, Adam (wd 0,2), cosine
Kết quả then chốt	ImageNet zero-shot 76,2% (so với 11,5% của Visual N-Grams); thắng ResNet-50 linear probe ở 16/27 dataset; thu hẹp robustness gap tới 75%

11. Kết luận ngắn

CLIP cho thấy rằng với đủ dữ liệu (400 triệu cặp ảnh-văn bản), một mục tiêu đơn giản, tức dự đoán cặp ảnh-văn bản nào khớp nhau bằng loss tương phản đối xứng, đủ để huấn luyện bộ mã hóa ảnh có thể chuyển giao zero-shot sang hầu hết tác vụ phân loại bằng cách mô tả lớp bằng ngôn ngữ, với độ bền vững cao hơn mô hình giám sát ImageNet. Giới hạn chính là hiệu quả dữ liệu, tác vụ chuyên biệt và ngoài phân phối, cùng đòi hỏi quy mô batch/phần cứng.

Liên kết gốc: https://arxiv.org/abs/2103.00020

Bài 31

Sigmoid Loss for Language Image Pre-Training (SigLIP, 2023)

Tác giả: Xiaohua Zhai, Basil Mustafa, Alexander Kolesnikov, Lucas Beyer (Google DeepMind, Zürich).
Venue / arXiv: arXiv:2303.15343 (bản text không ghi hội nghị).
Nguồn: https://arxiv.org/abs/2303.15343
Chủ đề: thay loss softmax tương phản của CLIP bằng loss sigmoid theo từng cặp ảnh-văn bản; cài đặt tiết kiệm bộ nhớ (chunked, không all-gather); tác động của batch size; huấn luyện hiệu quả với ít chip; SigLIP đa ngôn ngữ.
Vị trí trong lộ trình: Đọc sau 30 (CLIP) và 29 (ViT). Bài này cải tiến *mục tiêu và cài đặt* của CLIP, không đổi kiến trúc. Bộ mã hóa ảnh SigLIP là lựa chọn phổ biến cho tầng thị giác của VLM (32-38), nhưng điều đó ngoài paper này; xem mục "Ứng dụng".

> Ghi chú diễn giải, không phải bản dịch. Một số bảng trong bản text bị lỗi định dạng; ghi chú chỉ trích số liệu đọc rõ.

1. Bối cảnh

CLIP (bài 30) và ALIGN huấn luyện bộ mã hóa ảnh và văn bản bằng loss **softmax tương phản** (InfoNCE) trên batch rất lớn (CLIP dùng 32.768). Cách làm này có ba bất tiện:

1. **Chuẩn hóa toàn batch.** Softmax cho mỗi ảnh chia cho tổng exp trên *toàn bộ* văn bản trong batch, và ngược lại cho mỗi văn bản. Vì không đối xứng, phải tính hai lần (ảnh sang văn bản, văn bản sang ảnh). Để ổn định số học còn phải trừ giá trị lớn nhất, nghĩa là thêm một lượt duyệt qua toàn batch.
2. **Giao tiếp và bộ nhớ.** Khi dữ liệu chia trên nhiều thiết bị (data parallel), mỗi thiết bị cần embedding của *mọi* mẫu khác ⇒ all-gather đắt, rồi dựng một ma trận tương đồng $|B| \times |B|$ trong bộ nhớ.
3. **Batch size gắn chặt vào định nghĩa bài toán.** Với softmax, số mẫu âm của mỗi mẫu dương là $|B| - 1$ nên batch size là một phần của hàm mục tiêu.

Câu hỏi của paper: có thể dùng một loss đơn giản hơn, xử lý mỗi cặp ảnh-văn bản độc lập, không cần chuẩn hóa toàn cục, cho hiệu quả tốt hơn hoặc bằng không?

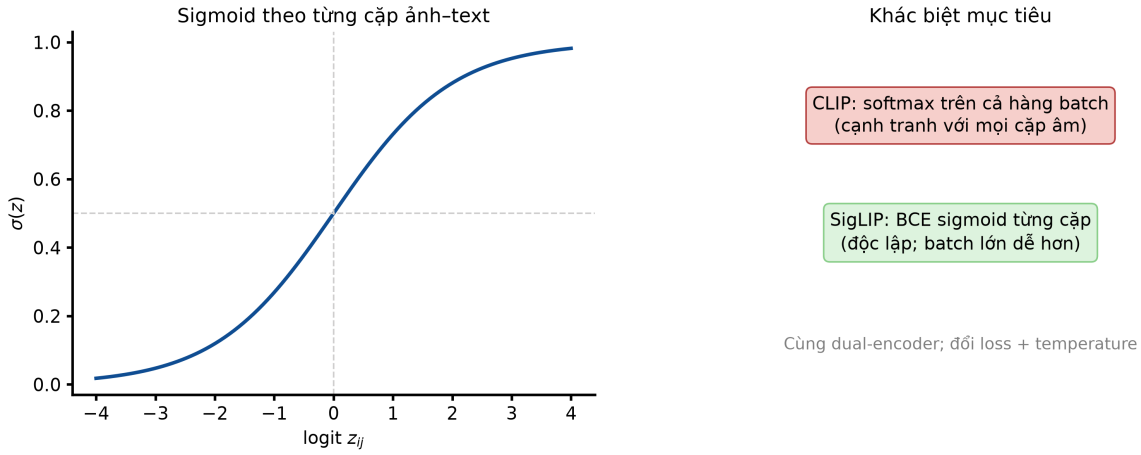
2. Ý tưởng chính

Biến bài toán thành **phân loại nhị phân trên mọi cặp (ảnh i, văn bản j) trong batch**: cặp khớp (i = j) có nhãn +1, mọi cặp còn lại nhãn -1. Mỗi cặp được chấm bằng một sigmoid độc lập: $\sigma(\text{nhãn} \cdot (t \cdot \cos(x_i, y_j) + b))$, trong đó t (nhiệt độ) và b (bias) là hai số vô hướng học được. Vì mỗi cặp là một số hạng riêng trong tổng loss, ta tính được loss theo từng khối nhỏ trên mỗi thiết bị, luân phiên đổi khối văn bản giữa các thiết bị (không all-gather, không dựng ma trận $|B|^2$). Kết quả: loss sigmoid tốt hơn rõ khi batch nhỏ hơn 16k, ngang khi batch lớn, tiết kiệm bộ nhớ nên cho phép batch lớn hơn với cùng số chip; và bất ngờ là hiệu quả bão hòa ở batch khoảng 32k (cả softmax lẫn sigmoid), dù thử tới 1 triệu.

3. Phương pháp chi tiết

3.1 Nhắc lại: loss softmax của CLIP

Với batch $\mathcal{B} = \{(I_1, T_1), \dots\}$, ảnh có embedding chuẩn hóa $x_i = f(I_i)/\|f(I_i)\|$, văn bản $y_i = g(T_i)/\|g(T_i)\|$. Loss softmax (gồm hai hướng):



SigLIP: sigmoid/BCE từng cặp vs CLIP softmax trên batch. (minh hoạ, tự tính)

$$\mathcal{L}_{softmax} = -\frac{1}{2|\mathcal{B}|}\sum_{i=1}^{|\mathcal{B}|}\Bigg(\underbrace{\log\frac{e^{t\,x_i\cdot y_i}}{\sum_j e^{t\,x_i\cdot y_j}}}_{\text{ảnh}\to\text{văn bản}} + \underbrace{\log\frac{e^{t\,x_i\cdot y_i}}{\sum_j e^{t\,x_j\cdot y_i}}}_{\text{văn bản}\to\text{ảnh}}\Bigg)$$

Nhiệt độ $t = \exp(t')$ là tham số học được. Trong paper, ảnh dùng ViT, văn bản dùng Transformer.

3.2 Loss sigmoid

Mỗi cặp (i, j) là một ví dụ phân loại nhị phân với nhãn $z_{ij} = +1$ nếu $i = j$, ngược lại -1:

$$\boxed{\mathcal{L}_{sigmoid} = -\frac{1}{|\mathcal{B}|}\sum_{i=1}^{|\mathcal{B}|}\sum_{j=1}^{|\mathcal{B}|}\log\sigma\left(z_{ij}\left(t\,x_i\cdot y_j + b\right)\right)}$$

Chú thích về dấu: ghi chú dùng dạng của thuật toán 1 trong paper (logit $= t\,x\cdot y + b$, b khởi tạo -10), nhất quán với việc khởi tạo cho "mọi cặp mặc định là âm". Phần cảm ơn của paper nói có sửa lỗi pseudo-code ở phiên bản sau, và biểu thức in trong bản text bị lỗi định dạng, nên nếu bạn đọc bản gốc hãy kiểm tra dấu của b theo cùng quy ước.

Pseudo-code (Thuật toán 1, viết lại bằng PyTorch):

```
def sigmoid_loss(img_emb, txt_emb, t_prime, b):   # [n, dim], [n, dim], scalar, scalar
    n = img_emb.shape[0]
    t = t_prime.exp()
    zimg = F.normalize(img_emb, dim=-1)
    ztxt = F.normalize(txt_emb, dim=-1)
    logits = zimg @ ztxt.T * t + b                # [n, n]
    labels = 2 * torch.eye(n) - 1                 # +1 trên đường chéo, -1 chỗ khác
    return -F.logsigmoid(labels * logits).sum() / n
```

Khởi tạo: $t' = \log 10$ (tức $t = 10$) và $b = -10$.

Vì sao cần bias b? Trong batch có $|\mathcal{B}|$ cặp dương nhưng $|\mathcal{B}|^2 - |\mathcal{B}|$ cặp âm. Ví dụ ở batch 16.384 (paper nêu): 16k cặp dương và khoảng 268 triệu cặp âm. Nếu khởi tạo không có bias, ngay từ đầu loss của hàng trăm triệu cặp âm lấn át, tạo những bước tối ưu lớn đầu tiên chỉ để sửa sự mất cân bằng này ("over-correction"). Bias âm lớn làm mô hình khởi đầu gần với "prior" (hầu hết các cặp là âm).

Ví dụ (tự tính, $n = 16384$, $t = 10$, cosine ≈ 0 lúc đầu):

Khởi tạo	Loss mỗi cặp âm	Tổng loss âm	Tổng loss dương
$b = 0$	$\log 2 \approx 0{,}693$	$\approx 1{,}9 \times 10^{8}$	$\approx 1{,}1 \times 10^{4}$
$b = -10$	$\log(1 + e^{-10}) \approx 4{,}5 \times 10^{-5}$	$\approx 1{,}2 \times 10^{4}$	$\approx 1{,}6 \times 10^{5}$

Với $b = 0$, phần âm lấn át phần dương hơn 4 bậc độ lớn; với $b = -10$, hai phần cùng bậc 10^4-10^5, nên gradient cân bằng hơn.

3.3 Cài đặt "chunked", không all-gather (Hình 1)

Giả sử có D thiết bị, batch toàn cục $|\mathcal{B}|$, mỗi thiết bị giữ $b = |\mathcal{B}|/D$ mẫu (dùng chữ b cho kích thước batch mỗi thiết bị trong đoạn này, khác bias). Vì mỗi cặp là một số hạng riêng:

$$\mathcal{L} = \frac{1}{|\mathcal{B}|} \sum_{d_i=1}^{D} \underbrace{\sum_{d_j=1}^{D} \underbrace{\mathcal{L}_{(d_i,d_j)}}_{\text{khối } b\times b}}_{\text{đối chỗ mẫu âm giữa các thiết bị}}$$

Thuật toán:

1. Mỗi thiết bị tính khối loss của ảnh cục bộ với văn bản cục bộ (b cặp dương, b²-b cặp âm).
2. **Hoán đổi** (collective permute) phần biểu diễn văn bản sang thiết bị bên cạnh; mỗi thiết bị tính khối loss mới với văn bản nhận được (toàn mẫu âm).
3. Lặp $D - 1$ lần cho đến khi mọi ảnh đã gặp mọi văn bản; cuối cùng cộng loss qua các thiết bị.

Bộ nhớ ma trận tương đồng giảm từ $|\mathcal{B}|^2$ xuống b^2 tại một thời điểm. Ví dụ (tự tính): $|\mathcal{B}| = 32768$, $D = 8$ thiết bị $\Rightarrow b = 4096$; ma trận đầy đủ có $1{,}07 \times 10^9$ phần tử (≈ 4,3 GB nếu fp32), ma trận một khối chỉ $1{,}68 \times 10^7$ phần tử (≈ 67 MB fp32), nhỏ hơn $D^2 = 64$ lần. Paper nhận xét D lần hoán đổi tập thể thường nhanh hơn hai all-gather giữa D thiết bị. Cách làm này nhờ tính tách được của sigmoid; softmax cần chuẩn hóa toàn hàng/cột nên khó chia khối theo cách này.

Thực nghiệm (Mục 4.2): với 4 chip TPUv4 và mô hình SigLIP cỡ Base, có thể đặt batch 4096, so với chỉ 2048 cho CLIP tương ứng.

4. Thí nghiệm và kết quả chính

Thiết lập chung. Ảnh dùng ViT (B/16 là chính, độ phân giải 224), văn bản dùng Transformer; tokenizer sentencepiece 32k huấn luyện trên C4 tiếng Anh, tối đa 16 token văn bản (trong các thí nghiệm gốc). Dữ liệu: WebLI (ảnh-văn bản web nhiều ngôn ngữ của Google, không công khai) cho SigLIP; bộ LiT cho SigLiT. Optimizer mặc định là Adafactor kiểu ScalingViT. Đánh giá: zero-shot ImageNet (INet-0) và truy xuất văn bản-ảnh trung bình trên 36 ngôn ngữ của XM3600 (crossmodal-3600).

Hai biến thể:

- **SigLIP**: huấn luyện cả hai tháp (như CLIP) với loss sigmoid.
- **SigLiT**: theo LiT (Locked-image Tuning): *khóa* (đóng băng) bộ mã hóa ảnh đã tiền huấn luyện và tính trước embedding ảnh, chỉ huấn luyện tháp văn bản; tính trước nên huấn luyện rất rẻ.

4.1 Bảng 1: huấn luyện hiệu quả với ít chip

Mô hình	Ảnh	Text	Batch	#TPUv4	Ngày	INet 0-shot
SigLiT	B/8 (khóa)	L*	32k	4	1	79,8 (văn bản bài ghi 79,7)
SigLiT	g/14 (khóa)	L	20k	4	2	84,5
SigLIP	B/16	B	16k	16	3	71,0
SigLIP	B/16	B	32k	32	2	72,1
SigLIP	B/16	B	32k	32	5	73,4

(* biến thể L 12 lớp.) Hai hàng đầu dùng ảnh tiền huấn luyện; hai hàng SigLIP cuối là mô hình khởi tạo ngẫu nhiên. Paper so sánh với FLIP và CLIP (khoảng 5 và 10 ngày trên 256 lõi TPUv3). Hàng 71,0 dùng ảnh B/16 đã tiền huấn luyện và không áp weight decay lên nó (xem 5).

4.2 Ảnh hưởng của batch size (Hình 2, 3)

- **SigLiT** (18 tỉ mẫu thấy được): khi batch < 16k, sigmoid tốt hơn softmax rất nhiều; batch lớn thì hai loss gần nhau. Đã huấn luyện thành công tới batch **1 triệu**, nhưng hiệu quả **bão hòa ở khoảng 32k**; chỉ cải thiện nhẹ ở lớn hơn, và mô hình đạt đỉnh tại khoảng 256k. SigLiT tốt nhất với tháp văn bản cỡ B đạt 84,7% (LiT gốc báo 85,2% với tháp văn bản cỡ g, lớn hơn 10 lần).
- **SigLIP** (9 tỉ mẫu): sigmoid tốt hơn softmax khi batch dưới 32k; đỉnh của sigmoid ở 32k xuất hiện sớm hơn và nhỉnh hơn đỉnh softmax (softmax cần khoảng 98k để tối ưu mà vẫn không vượt sigmoid); batch 307k làm hỏng cả hai. Tác giả khuyên dùng công thức SigLIP cả ở batch lớn vì đơn giản và tiết kiệm.
- **Thời gian huấn luyện** (Hình 3): batch lớn (262k) thắng batch nhỏ (8k) nhưng chỉ khi huấn luyện đủ lâu; lịch ngắn thì batch quá lớn ít bước cập nhật nên không có lợi.

4.3 SigLIP đa ngôn ngữ (mSigLIP)

Dùng cả 100+ ngôn ngữ của WebLI, 30 tỉ mẫu thấy được. Quan sát chính: batch **32k đã đủ** cho đa ngôn ngữ; lớn hơn không cải thiện rõ trên INet-0 và trung bình XM3600 còn xấu đi. So sánh từ vựng đa ngôn ngữ nhỏ (32k token) với lớn (250k token): từ vựng lớn cho cải thiện hơn 1% (đọc từ văn bản bài, câu bị cắt nên chỉ nêu bậc độ lớn). Để giảm chi phí embedding $N \times W$, dùng **embedding thắt cổ chai** $N \times K$ cộng phép chiếu $K \times W$ ($K = 96$ cho $W = 768$) chỉ mất khoảng nửa điểm INet-0. mSigLIP ViT-B đạt kết quả truy xuất XM3600 tốt nhất cho cỡ Base (so với kết quả trước 28,5% của LiT dùng ViT-e 4 tỉ tham số, theo văn bản bài).

4.4 Mô hình công khai (Bảng 3)

Huấn luyện "quá tay" (overtraining): 40 tỉ mẫu ở batch 32k, 256 patch ảnh (độ phân giải 256, patch 16) và 64 token văn bản, sau đó thêm 5 tỉ mẫu ở độ phân giải đích với learning rate nhỏ hơn 100 lần và không weight decay. Paper báo SigLIP B, L và So400m (ViT "shape-optimized" 400M tham số) thắng OpenCLIP, CLIP gốc, EVA-CLIP, CLIPA-v2 về zero-shot (ImageNet, ObjectNet, ImageNet-v2, ReaL) và truy xuất (MSCOCO); đặc biệt So400m vượt cả các mô hình lớn hơn nhiều. Bảng 3 bị lỗi định dạng trong bản text nên ghi chú không trích số cột.

5. Phân tích / ablation

- **Ổn định ở batch lớn.** Khi tăng batch xuất hiện các đột biến chuẩn gradient (gradient norm spikes) làm loss bất ổn. Giảm β_2 của Adam/Adafactor từ 0,999 xuống **0,95** (gợi ý từ MAE và nghiên cứu khác) đủ để ổn định; paper dùng $\beta_2 = 0.95$ trong mọi thí nghiệm. Giải thích trực giác: phản ứng nhanh hơn để hồi phục sau đột biến gradient.
- **Khởi tạo từ ảnh tiền huấn luyện (Hình 4).** Khởi tạo tháp ảnh từ ViT-AugReg-B/16 tiền huấn luyện (không khóa, nhân learning rate của tháp ảnh với 0,1) và huấn luyện cùng tháp văn bản: ban đầu không hoạt động tốt, và độ chính xác 10-shot ImageNet giảm khi weight decay làm suy yếu biểu diễn đã học.

Tắt weight decay trên trọng số tiền huấn luyện (chỉ áp weight decay cho tháp văn bản khởi tạo ngẫu nhiên) làm kết quả ổn định và tốt hơn đáng kể: 71% zero-shot ImageNet với batch 16k trên 16 chip trong 3 ngày.

- **Tỉ lệ dương/âm (Hình 6).** Mô phỏng batch 16k bằng cách che bớt mẫu âm (ngẫu nhiên, giữ khó nhất, giữ dễ nhất) tới tỉ lệ 1:1,6. Kết luận: (1) loại ngẫu nhiên các mẫu âm để cân bằng làm xấu đi; (2) giữ các mẫu âm *dễ* thì hoàn toàn không hiệu quả; (3) giữ các mẫu âm *khó nhất* gần như giữ nguyên chất lượng, và huấn luyện lâu hơn để bù tổng số cặp thấy được còn nhỉnh hơn; tức phần lớn tín hiệu học từ phía âm đến từ các mẫu khó. Do đó mất cân bằng dương/âm không phải mối lo chính, nhưng việc tìm cách đưa nhiều mẫu âm vào một cách hiệu quả vẫn đáng quan tâm.
- **Bias và nhiệt độ (Bảng 4).** Bật bias với khởi tạo -10 và $t' = \log 10$ cải thiện nhất quán trên ImageNet, Oxford-iiit Pet và CIFAR-100 (bảng bị lỗi định dạng nên không trích số cho từng cấu hình). Khởi tạo bias ngẫu nhiên, như 0, không giải quyết vấn đề over-correction và kém hơn rõ, nhất là khi nhiệt độ khởi tạo nhỏ.
- **Bền với nhiễu dữ liệu (Hình 7).** Thêm nhiễu với xác suất p: thay ảnh bằng nhiễu đều, thay văn bản bằng token ngẫu nhiên, ghép cả hai, và xáo trộn căn chỉnh ảnh-văn bản. Mô hình cỡ M huấn luyện với 3,6 tỉ mẫu ở batch 16.384: sigmoid duy trì ưu thế trước softmax khi mức hỏng tăng. Paper liên hệ với việc sigmoid bền hơn với nhiễu nhãn trong phân loại.

6. Giới hạn

Do tác giả nêu:

- Phần lớn kết quả dùng WebLI, tập dữ liệu nội bộ của Google, không công khai. Paper nêu hy vọng nghiên cứu "với nguồn lực hạn chế" nhưng dữ liệu vẫn là điều kiện.
- Batch quá lớn (ví dụ 307k) làm hỏng cả hai loss; hiệu quả bão hòa.
- Tìm cách đưa nhiều mẫu âm khó vào một cách hiệu quả chưa được giải.
- Không có kết quả cho các tác vụ ngoài phân loại zero-shot và truy xuất ảnh-văn bản.

Do người viết nhận xét:

- Lợi thế sigmoid rõ ở batch nhỏ (< 16k) và ở tiết kiệm bộ nhớ; ở batch rất lớn lợi ích về chất lượng nhỏ (nhưng đơn giản hơn).
- So sánh softmax chủ yếu ở các cỡ Base/Large; chưa chắc lặp lại ở quy mô rất lớn hay dữ liệu khác.
- Bảng 3 so sánh công bố chéo nhiều công thức huấn luyện (dữ liệu, số mẫu thấy được khác nhau), nên khác biệt không chỉ do loss.
- Bản text bảng bị lỗi, một số con số chỉ nêu định tính.

7. Liên hệ với các bài khác

- **30 CLIP:** nguồn gốc của softmax loss, batch 32k, cài đặt phân tán sharded. SigLIP giữ nguyên kiến trúc/bài toán nhưng đổi loss.
- **29 ViT:** tháp ảnh; SigLIP dùng ViT-B/16, L, So400m (shape-optimized).
- **32-38 (Flamingo ... SmolVLM):** các VLM thường dùng bộ mã hóa ảnh kiểu CLIP/SigLIP; xem chi tiết từng bài.
- **08 LoRA, 52-54 (lượng tử hóa):** không liên quan trực tiếp.
- **Chưng cất:** loss sigmoid cặp là một dạng "phân loại độc lập mỗi cặp" có thể dùng làm tín hiệu chưng cất embedding (suy luận ngoài bài báo).

8. Gợi ý áp dụng

SigLIP dùng loss sigmoid theo từng cặp ảnh–văn bản thay cho chuẩn hoá softmax trên toàn batch. Tác động của batch, bias và cấu hình tối ưu cần được đánh giá cùng nhau; kết quả của bài báo không tự động chuyển

sang các mục tiêu huấn luyện khác.

9. Thí nghiệm gợi ý

Trên cùng dữ liệu và kiến trúc nhỏ, so sánh loss sigmoid và softmax với nhiều cỡ batch. Theo dõi loss, chất lượng truy hồi và độ ổn định gradient; thay đổi bias khởi tạo để quan sát vai trò của các cặp âm.

10. Tóm tắt công thức / khái niệm

Khái niệm	Công thức / ý nghĩa
Softmax loss (CLIP)	chuẩn hóa theo hàng và cột của ma trận $\|\mathcal{B}\| \times \|\mathcal{B}\|$
Sigmoid loss	$-\frac{1}{\|\mathcal{B}\|}\sum_{i,j}\log\sigma(z_{ij}(t\,x_i\cdot y_j+b))$
Nhãn	$z_{ij}=+1$ nếu $i=j$, ngược lại -1
Khởi tạo	$t'=\log 10$ ($t=10$), $b=-10$
Tỉ lệ dương:âm	$1:(\|\mathcal{B}\|-1)$; ví dụ 16k cặp dương và ≈ 268M cặp âm
Cài đặt chunked	D lần hoán đổi tập thể; bộ nhớ b^2 thay vì $\|\mathcal{B}\|^2$
SigLiT	khóa tháp ảnh đã huấn luyện, tính sẵn embedding, chỉ huấn luyện tháp văn bản
Batch hiệu quả	bão hòa ở khoảng 32k; 1M vẫn huấn luyện được; 307k làm hỏng
Ổn định	Adam/Adafactor $\beta_2=0.95$
Khởi tạo từ ảnh tiền huấn luyện	không weight decay trên trọng số tiền huấn luyện
Số liệu then chốt	SigLiT 4 chip, 1 ngày: 79,7-79,8% INet; SigLiT g/14, 2 ngày: 84,5%; SigLIP B/16 32 chip 5 ngày: 73,4%

11. Kết luận ngắn

SigLIP thay softmax toàn batch của CLIP bằng loss sigmoid trên từng cặp ảnh-văn bản cùng một bias học được, nhờ đó cài đặt được theo khối nhỏ trên từng thiết bị mà không cần all-gather hay ma trận $|\mathcal{B}|^2$, tốt hơn ở batch nhỏ, ngang ở batch lớn và bền hơn với nhiễu dữ liệu. Phát hiện thực dụng là batch khoảng 32k đã đủ; thêm vào đó là hai mẹo huấn luyện (β_2 = 0,95, tắt weight decay cho trọng số tiền huấn luyện).

Liên kết gốc: https://arxiv.org/abs/2303.15343

Bài 32

Flamingo: a Visual Language Model for Few-Shot Learning (2022)

Tác giả: Jean-Baptiste Alayrac, Jeff Donahue, Pauline Luc, Antoine Miech và cộng sự (DeepMind; tổng cộng 27 tác giả, Karen Simonyan là đồng tác giả cấp cao).
Venue / arXiv: arXiv:2204.14198 (v2, 15/11/2022); NeurIPS 2022 (paper ghi "36th Conference on Neural Information Processing Systems").
Nguồn: https://arxiv.org/abs/2204.14198
Chủ đề: Ghép một vision encoder đóng băng với một LLM đóng băng; Perceiver Resampler; gated cross-attention (GATED XATTN-DENSE); dữ liệu xen kẽ ảnh–văn bản; few-shot in-context learning cho ảnh/video.
Vị trí trong lộ trình: Đọc trước: 01 (Transformer, cross-attention), 02 (GPT-3, in-context learning), 04 (Chinchilla, LM nền của Flamingo), 29 (ViT), 30 (CLIP). Bài này là "bản thiết kế gốc" của họ VLM: nó đặt ra câu hỏi *làm sao nối vision và LLM*. Tiếp theo: 33 (BLIP-2 dùng Q-Former thay Resampler), 34–35 (LLaVA bỏ hẳn cross-attention, chỉ dùng projector), 36 (InternVL 1.5).

> Ghi chú diễn giải, không phải bản dịch. Phần dưới tập trung vào Perceiver Resampler, gated cross-attention, masking và các kết quả tiêu biểu của báo cáo dài.

1. Bối cảnh

Vấn đề trước Flamingo. Năm 2022, thị giác máy tính vẫn theo kiểu "pretrain rồi fine-tune cho từng task", cần hàng nghìn mẫu có nhãn và tinh chỉnh siêu tham số riêng cho từng task. Trong khi đó, GPT-3 (bài 02) cho thấy LLM chỉ cần vài ví dụ trong prompt là làm được task mới (**in-context few-shot learning**: đưa vài cặp vào/ra vào prompt, mô hình tự suy ra quy tắc mà không cập nhật trọng số).

Hai hướng vision-language có sẵn đều thiếu:

- **Mô hình đối lập (contrastive) như CLIP/ALIGN** (bài 30): cho điểm tương đồng ảnh–văn bản, làm được zero-shot phân loại nhưng **không sinh được văn bản**, nên không làm được captioning hay VQA mở.
- **Mô hình sinh có điều kiện ảnh**: chưa cho thấy chất lượng tốt trong chế độ ít dữ liệu.

Khó khăn kỹ thuật: muốn tận dụng một LM mạnh đã pretrain, ta phải "tiêm" thông tin ảnh vào mà không phá kiến thức sẵn có (catastrophic forgetting: học task mới làm mô hình quên thứ đã học). Thêm nữa, ảnh độ phân giải cao hoặc video cho ra hàng nghìn đặc trưng, quá nhiều để đưa thẳng vào attention của LM.

Khái niệm nền cần nhớ:

- *Cross-attention*: attention mà query đến từ một chuỗi (ở đây là văn bản) còn key/value đến từ chuỗi khác (ở đây là đặc trưng ảnh).
- *Frozen (đóng băng)*: trọng số không được cập nhật khi huấn luyện, chỉ chạy forward/backward qua nó (gradient vẫn đi qua nếu cần, nhưng không có bước cập nhật).
- *Interleaved data*: văn bản xen kẽ ảnh, như một trang web thực (chữ, ảnh, chữ, ảnh...).

2. Ý tưởng chính

Flamingo lấy **hai mô hình mạnh đã pretrain và đóng băng**: một vision encoder (NFNet-F6) và một LM (Chinchilla 1.4B/7B/70B). Giữa chúng, paper chèn hai loại module huấn luyện từ đầu:

1. **Perceiver Resampler**: nén một lưới đặc trưng thị giác có kích thước tuỳ ý (ảnh to, nhiều khung hình video) thành **đúng 64 "visual token"** bằng một Transformer nhỏ với 64 query học được.
2. **GATED XATTN-DENSE**: các lớp cross-attention + FFN chèn *giữa* các lớp LM đóng băng, để văn bản "nhìn" vào 64 visual token đó. Mỗi lớp được nhân với $\tanh(\alpha)$ với α khởi tạo bằng 0, nên lúc đầu mô hình y hệt LM gốc, rồi dần dần "mở cổng" cho thông tin thị giác.

Huấn luyện bằng một hỗn hợp dữ liệu web (đặc biệt là trang web xen kẽ ảnh–chữ, M3W) với loss next-token thông thường. Kết quả là một mô hình có khả năng few-shot như GPT-3 nhưng cho ảnh/video.

3. Phương pháp chi tiết

3.1 Tổng quan luồng dữ liệu

Flamingo mô hình hoá xác suất của văn bản y khi biết các ảnh/video x xen kẽ (Eq. 1):

$$p(y \mid x) = \prod_{\ell=1}^{L} p\big(y_\ell \mid y_{<\ell},\, x_{\le \ell}\big)$$

trong đó $x_{\le \ell}$ là các ảnh/video đứng *trước* token y_ℓ trong chuỗi xen kẽ. Đây chính là một LM nhân quả, chỉ thêm điều kiện thị giác.

```
ảnh/video --> [Vision Encoder, đóng băng] --> lưới đặc trưng [T*S, d]
          --> [Perceiver Resampler, huấn luyện] --> 64 visual token
văn bản   --> LM block 1 (đóng băng) --> GATED XATTN-DENSE (hỏi 64 token) --> LM block 2
↪  ... --> logits
```

3.2 Vision encoder

- NFNet-F6 (mạng CNN không dùng BatchNorm), **đóng băng**, được pretrain riêng bằng loss contrastive hai chiều kiểu CLIP trên ALIGN + LTIP (ảnh 288x288; trong Flamingo ảnh được resize 320x320).
- Lấy đầu ra tầng cuối: lưới 2D đặc trưng, duỗi thành chuỗi 1D.
- Video: lấy mẫu 1 FPS, mã hoá từng khung độc lập, cộng **embedding thời gian học được**, rồi duỗi thành 1D. Ảnh được coi là video 1 khung.
- Paper không dùng positional embedding không gian rõ ràng, vì CNN đã mã hoá thông tin vị trí ngầm trong kênh (paper nói không thấy cải thiện khi thêm).

3.3 Perceiver Resampler

Mục tiêu: biến $X_f \in \mathbb{R}^{T \cdot S \times d}$ (kích thước thay đổi) thành $X \in \mathbb{R}^{R \times d}$ với $R = 64$ cố định.

Cách làm (giống Perceiver/DETR): giữ R vector **latent học được** (tham số của mô hình) làm query. Mỗi lớp: latent cross-attend vào đặc trưng ảnh, rồi qua FFN. Điểm khác biệt nhỏ so với Perceiver/DETR: **key/value là nối của đặc trưng ảnh và chính các latent** (paper nói cách này tốt hơn chút).

Pseudo-code (bám theo Figure 5 của paper):

```
def perceiver_resampler(x_f,              # [T, S, d] đặc trưng thị giác
                        time_embeddings,  # [T, 1, d]
                        x,                # [R, d] latent học được
                        num_layers):
    x_f = x_f + time_embeddings
    x_f = flatten(x_f)                    # [T*S, d]
    for i in range(num_layers):
        x = x + attention_i(q=x, kv=concat([x_f, x]))   # latent hỏi ảnh (và hỏi nhau)
        x = x + ffw_i(x)
    return x                              # [R, d], R = 64
```

Công thức cho một lớp (theo quy ước của ghi chú):

$$X \leftarrow X + \text{Attn}\big(Q = XW_Q,\ K = [X_f; X]W_K,\ V = [X_f; X]W_V\big), \qquad X \leftarrow X + \text{FFN}(X)$$

Chi phí attention của một lớp Resampler là $O\big(R \cdot (TS + R) \cdot d\big)$: **tuyến tính** theo số đặc trưng đầu vào TS, và số token đầu ra luôn là R. Đây là điểm mấu chốt: video dài hay ảnh to chỉ làm tăng chi phí *ở Resampler*, còn LM luôn chỉ thấy 64 token.

Cấu hình (Table 4 của paper): 6 lớp, $D = 1536$, 16 head, FFN dùng Squared ReLU; tổng ~194M tham số (Table 5), cố định kích thước cho cả ba model.

3.4 GATED XATTN-DENSE

Một khối mới chèn *trước* một lớp LM đóng băng. Pseudo-code từ paper (Figure 4):

```
def gated_xattn_dense(y, x, alpha_xattn, alpha_dense):
    # y: đặc trưng ngôn ngữ; x: visual token (64 token từ Resampler)
    # alpha_*: vô hướng học được, khởi tạo = 0
    y = y + tanh(alpha_xattn) * attention(q=y, kv=x)   # 1. cross-attention có cổng
    y = y + tanh(alpha_dense) * ffw(y)                 # 2. FFN có cổng
    y = y + frozen_attention(q=y, kv=y)                # lớp LM gốc (đóng băng)
    y = y + frozen_ffw(y)
    return y
```

$$\boxed{y \leftarrow y + \tanh(\alpha) \cdot f_{\text{new}}(y, x)}, \qquad \alpha_{\text{init}} = 0$$

Vì sao khởi tạo $\alpha = 0$ quan trọng? Lúc bắt đầu, $\tanh(0) = 0$ nên khối mới bị "tắt": đầu ra của mô hình *đúng bằng* LM gốc. Gradient vẫn chảy vào α và vào trọng số khối mới (vì $\tanh'(0) = 1$), nên cổng từ từ mở ra khi có lợi. Paper thấy: bỏ tanh gating làm điểm tổng giảm 4.2% và gây bất ổn huấn luyện. Hình 6 của paper cho thấy $|\tanh\alpha|$ ở mọi lớp tăng nhanh từ 0 trong huấn luyện (tức mọi lớp LM đều dùng thông tin thị giác), có xu hướng lớn dần theo độ sâu; paper cảnh báo không nên suy diễn mạnh vì thang đo của activation cũng đổi theo độ sâu.

Tần suất chèn: chèn ở mọi lớp thì tốt nhất nhưng đắt. Paper chèn mỗi 1 lớp (3B), mỗi 4 lớp (9B), mỗi 7 lớp (80B).

Cấu hình (Table 4, 5):

Model	LM đóng băng	Vision (đóng băng)	GATED XATTN-DENSE (huấn luyện)	Resampler	Tổng
Flamingo-3B	1.4B	435M	1.2B (mỗi lớp; 24 khối, $D = 2048$)	194M	3.2B
Flamingo-9B	7.1B	435M	1.6B (mỗi 4 lớp; 10 khối, $D = 4096$)	194M	9.3B
Flamingo-80B	70B	435M	10B (mỗi 7 lớp; 12 khối, $D = 8192$)	194M	80B

Kiểm tra nhanh (tự tính): với $D = 2048$, một khối gồm cross-attention ($\approx 4D^2 \approx 16.8$M) + FFN 4D ($\approx 8D^2 \approx 33.5$M) ≈ 50M tham số; nhân 24 khối ≈ 1.2B, khớp với Table 5. Nghĩa là phần tham số huấn luyện của Flamingo-3B *lớn hơn* phần LM đóng băng chút ít: chi phí của cách "chèn cross-attention vào mọi lớp" khá cao.

3.5 Masking cho ảnh xen kẽ

Với chuỗi có nhiều ảnh, Flamingo định nghĩa hàm $\varphi(\ell)$ = chỉ số của ảnh/video **gần nhất đứng trước** token ℓ (0 nếu chưa có ảnh). Tại token ℓ, cross-attention chỉ được nhìn visual token của ảnh $\varphi(\ell)$ (qua mask trên ma trận cross-attention chữ→ảnh), chứ không nhìn mọi ảnh trước đó.

Thông tin từ các ảnh cũ hơn vẫn đến được qua self-attention của LM (vì token ở ảnh trước đã "thấm" thông tin ảnh vào residual stream). Lợi ích: huấn luyện chỉ với tối đa 5 ảnh/chuỗi (M3W) mà lúc suy luận vẫn dùng được tới 32 cặp (ảnh, chữ) làm "shot". Paper khẳng định (Section 2.3) cách này hiệu quả hơn việc cho phép nhìn mọi ảnh trước, nhưng bảng ablation chính (Table 3) không có dòng riêng cho điểm này.

Còn có token đặc biệt: `<image>` đánh dấu vị trí ảnh, `<EOC>` (end of chunk) đặt trước mỗi ảnh và cuối tài liệu, giúp mô hình biết khi nào kết thúc một đoạn (dùng để dừng sinh khi đánh giá).

3.6 Dữ liệu và hàm mục tiêu

Ba loại dữ liệu, đều từ web:

Nguồn	Mô tả
M3W (MultiModal MassiveWeb)	~43 triệu trang web; chèn `<image>` đúng vị trí; lấy ngẫu nhiên đoạn $L = 256$ token, tối đa $N = 5$ ảnh đầu
ALIGN + **LTIP**	1.8 tỷ cặp ảnh–alt-text (ALIGN); LTIP 312 triệu cặp ảnh–caption dài và chất lượng hơn
VTP	27 triệu video ngắn (~22 giây) kèm câu mô tả

Loss là tổng có trọng số của negative log-likelihood theo từng tập (Eq. 2):

$$\sum_{m=1}^{M} \lambda_m \, \mathbb{E}_{(x,y)\sim\mathcal{D}_m} \Big[- \sum_{\ell=1}^{L} \log p(y_\ell \mid y_{<\ell}, x_{\leq \ell}) \Big]$$

Paper nhấn mạnh: chỉnh λ_m là then chốt; **tích luỹ gradient qua tất cả dataset** tốt hơn xoay vòng (round-robin).

3.7 Few-shot prompting

Prompt = các cặp ví dụ (ảnh hoặc video, văn bản) nối tiếp nhau rồi đến ảnh truy vấn, ví dụ: `<BOS><image>Output: A cat wearing sunglasses.<EOC><image>Output: Elephants walking in the savanna.<EOC><image>Output:`. Với VQA: `Question: ... Answer: ...`. Đánh giá:

- **Open-ended**: beam search (beam=3), dừng ở `<EOC>`.
- **Close-ended**: chấm log-likelihood cho từng đáp án rồi xếp hạng.
- **Zero-shot**: dùng 2 ví dụ *chỉ có chữ* (bỏ ảnh); chỉ 1 ví dụ thì mô hình bị lệch bắt chước câu đó.

4. Thí nghiệm và kết quả chính

16 benchmark ảnh/video (5 benchmark DEV dùng để chọn thiết kế: COCO, OKVQA, VQAv2, MSVDQA, VATEX; 11 còn lại để đo không thiên lệch).

Few-shot, Flamingo-80B, chọn các cột đọc được rõ trong Table 1 (một số cột bị lỗi trích xuất PDF nên bỏ qua; mọi số copy từ paper):

Benchmark	0-shot	4-shot	32-shot	Pretrained FT SOTA (số mẫu fine-tune)
OKVQA	50.6	57.4	57.8	54.4 (10K)
VQAv2	56.3	63.1	67.6	80.2 (444K)
COCO (CIDEr)	84.3	103.2	113.8	143.3 (500K)
MSVDQA	35.6	41.7	52.3	47.9 (27K)
VATEX (CIDEr)	46.7	56.0	65.1	76.3 (500K)
VizWiz	31.6	39.6	49.8	57.2 (20K)
Flick30K	67.2	75.1	75.4	67.4 (30K)
MSRVTTQA	17.4	23.9	31.0	46.8 (130K)
YouCook2	60.1	74.5	86.8	138.7 (10K)

(Cột bị bỏ vì khó đọc: TextVQA, VisDial, HatefulMemes, NextQA, STAR, iVQA, RareAct.)

Thông điệp: với 32 shot và không cập nhật trọng số, Flamingo vượt SOTA fine-tune trên 6 trong 16 task (phần giới thiệu và Figure 2 của paper; chú thích Table 1 lại nói "seven tasks", nên con số này hơi lệch giữa các chỗ trong paper), dùng khoảng 1000 lần ít dữ liệu theo từng task. Model lớn hơn và nhiều shot hơn đều tốt hơn (Figure 2).

Fine-tune (Table 2, mở khoá vision backbone, độ phân giải cao hơn): SOTA mới trên 5 task: VQAv2 82.0 (test-dev), VATEX 84.2, VizWiz 65.7, MSRVTTQA 47.4, HatefulMemes 86.6. So 32-shot VQAv2 67.6 -> fine-tune 82.0.

Cỡ mô hình và số shot: Flamingo-3B/9B/80B tăng đều theo kích thước; model lớn tận dụng nhiều shot tốt hơn. Dù huấn luyện chỉ với 5 ảnh/chuỗi, suy luận với 32 shot vẫn lợi.

5. Phân tích / ablation

Ablation trên Flamingo-3B, 4-shot, 5 benchmark DEV (Table 3 của paper). Baseline: điểm tổng **70.7**, 3.2B tham số, step time 1.74 s.

Thay đổi	Điểm tổng	Nhận xét (theo paper)
Bỏ cặp video–văn bản	67.3	Hại các task video
Bỏ cặp ảnh–văn bản	60.9	Cặp ảnh–text rất quan trọng
Cặp ảnh–text -> LAION	66.4	Kém hơn một chút so với ALIGN+LTIP
Bỏ M3W (dữ liệu xen kẽ)	**53.4**	Giảm hơn 17%; dữ liệu xen kẽ là nền tảng của few-shot
Round-robin thay vì tích luỹ gradient	62.9	Tích luỹ gradient tốt hơn
Bỏ tanh gating	66.5	Giảm 4.2% và mất ổn định
Vanilla xattn (không dense)	66.9 (2.4B)	GATED XATTN-DENSE tốt hơn
Grafting (kiểu của [68])	63.1	LM đóng băng + stack học mới phía trên: kém
Chỉ 1 xattn ở giữa	59.8 (2.0B, 0.87s)	Quá ít chỗ tiêm
Xattn mỗi 4 lớp	68.8 (1.02s)	Nhanh hơn ~66%, mất 1.9%
Xattn mỗi 2 lớp	68.2	
Resampler -> MLP	66.6	Perceiver Resampler tốt hơn và nhanh hơn
Resampler -> Transformer thường	66.7	
Vision: CLIP ViT-L/14	64.9	NFNet-F6 hơn 5.8%
Vision: NFNet-F0	62.7	NFNet-F6 hơn 8.0%

Thay đổi	Điểm tổng	Nhận xét (theo paper)
LM không đóng băng, train từ đầu	57.8	Giảm 12.9%
LM không đóng băng, khởi tạo từ pretrain	62.7	Giảm 8.0%: catastrophic forgetting

Rút ra: (i) **dữ liệu xen kẽ** quan trọng hơn mọi thiết kế kiến trúc riêng lẻ; (ii) **đóng băng LM** là lựa chọn tốt hơn là fine-tune toàn bộ LM trên dữ liệu ảnh–văn bản; (iii) **vision encoder mạnh** đáng kể; (iv) tần suất chèn cross-attention là đánh đổi chi phí/chất lượng rõ ràng (mỗi 4 lớp: nhanh hơn ~66% mà mất 1.9%).

6. Giới hạn

Do tác giả nêu:

- Kế thừa điểm yếu của LM đóng băng: ảo giác (hallucination), đoán không có căn cứ; LM tổng quát hoá kém với chuỗi dài hơn lúc huấn luyện; hiệu quả mẫu thấp trong huấn luyện.
- Hiệu năng phân loại ảnh kém các mô hình contrastive SOTA (đoạn thảo luận ở phần 5 của paper).

Nhận xét bổ sung:

- Số tham số huấn luyện và chi phí huấn luyện lớn (80B có 10B tham số xattn; dữ liệu M3W/LTIP/VTP không công khai) nên khó tái lập.
- Mỗi lớp chèn xattn phải chạy trong mọi bước giải mã: chi phí suy luận tăng theo số khối chèn. Điểm lợi là KV của ảnh chỉ có 64 token/ảnh và tính một lần.
- Thiết kế "đóng băng toàn bộ + chèn lớp" khiến việc *cắt tỉa* hay đổi LM nền khó: các khối xattn gắn với kích thước ẩn D cụ thể của LM.

7. Liên hệ với các bài khác

- **02 GPT-3**: Flamingo tái hiện "few-shot bằng prompt" cho đa phương thức. **04 Chinchilla**: LM nền; Flamingo-80B dùng Chinchilla 70B.
- **01 Transformer**: cross-attention gốc (encoder-decoder); Flamingo dùng nó nhưng thêm cổng tanh.
- **29 ViT / 30 CLIP**: Flamingo tự pretrain vision encoder bằng loss CLIP nhưng dùng NFNet; ablation cho thấy CLIP ViT-L/14 kém hơn NFNet-F6 5.8%.
- **33 BLIP-2**: thay Resampler bằng Q-Former (32 query, có pretrain hai pha) và *bỏ* cross-attention trong LM, chỉ nối đầu ra vào đầu vào LM.
- **34–35 LLaVA**: đi xa hơn: không Resampler, không xattn; chỉ projector tuyến tính/MLP, các visual token nối thẳng vào chuỗi đầu vào của LM.
- **36 InternVL 1.5**: cùng triết lý "ViT–projector–LLM", nhưng giảm token bằng pixel shuffle thay vì Resampler.
- **08 LoRA**: một cách khác để thích ứng LM mà không phá nó; Flamingo dùng "thêm tham số gated", LoRA dùng "thêm hiệu low-rank".

8. Gợi ý áp dụng

Perceiver Resampler chuyển các đặc trưng thị giác thành số token cố định trước khi đưa vào cross-attention. Cách này khống chế chiều dài đầu vào thị giác cho từng khối, nhưng đánh đổi giữa mức nén và thông tin còn giữ cần được đánh giá theo tác vụ; kết quả của Flamingo không tự chứng minh chất lượng đọc chữ nhỏ.

Cổng khởi tạo ở 0 giúp nhánh cross-attention bắt đầu mà không làm thay đổi đầu ra của mô hình ngôn ngữ đóng băng. Khi học, cổng cho phép nhánh thị giác dần đóng góp; đây là lựa chọn khởi tạo, không bảo đảm tránh mọi thay đổi hành vi sau huấn luyện.

9. Thí nghiệm gợi ý

Thay đổi số latent của Resampler và đo chất lượng cùng chi phí trên các tác vụ cần thông tin toàn cục và cục bộ. So sánh thêm các cách khởi tạo cổng để đánh giá ảnh hưởng tới giai đoạn đầu huấn luyện.

10. Tóm tắt công thức / khái niệm

Khái niệm	Nội dung
Xác suất	$p(y \mid x) = \prod_\ell p(y_\ell \mid y_{<\ell}, x_{\leq \ell})$
Perceiver Resampler	$R = 64$ latent học được; $X \leftarrow X + \text{Attn}(q = X, kv = [X_f; X])$; $X \leftarrow X + \text{FFN}(X)$; 6 lớp, 194M tham số
Cổng tanh	$y \leftarrow y + \tanh(\alpha) f(y, x)$, $\alpha_0 = 0$
GATED XATTN-DENSE	xattn có cổng + FFN có cổng, chèn trước lớp LM đóng băng; mỗi 1/4/7 lớp tuỳ model
Mask ảnh xen kẽ	token ℓ chỉ nhìn visual token của ảnh $\varphi(\ell)$ (ảnh gần nhất trước nó)
Loss	tổng có trọng số λ_m của NLL theo dataset; tích luỹ gradient
Few-shot	prompt xen kẽ (ảnh, chữ) x shot + truy vấn; open-ended: beam=3; close-ended: log-likelihood
Đóng băng LM	tránh catastrophic forgetting (+8.0% so với fine-tune LM)

11. Kết luận ngắn

Flamingo chứng minh rằng có thể "dạy" một LM nhìn mà không làm hỏng nó: đóng băng cả vision lẫn LM, nén ảnh bằng Perceiver Resampler, rồi tiêm qua cross-attention có cổng khởi tạo bằng 0, và huấn luyện trên dữ liệu web xen kẽ để có few-shot. Các bài sau (BLIP-2, LLaVA) đơn giản hoá phần "nối" nhưng giữ cùng ý tưởng nền: tái dùng hai mô hình đơn phương thức mạnh.

Link gốc: https://arxiv.org/abs/2204.14198

Bài 33

BLIP-2: Bootstrapping Language-Image Pre-training with Frozen Image Encoders and Large Language Models (2023)

Tác giả: Junnan Li, Dongxu Li, Silvio Savarese, Steven Hoi (Salesforce Research).
Venue / arXiv: arXiv:2301.12597 (v3, 15/06/2023). Text paper không ghi tên hội nghị.
Nguồn: https://arxiv.org/abs/2301.12597
Chủ đề: Q-Former (Querying Transformer), tiền huấn luyện hai pha, nối vision encoder đóng băng với LLM đóng băng bằng ít tham số huấn luyện.
Vị trí trong lộ trình: Đọc trước: 01 (Transformer), 29 (ViT), 30 (CLIP), 32 (Flamingo, đối thủ trực tiếp). Bài này thay Perceiver Resampler + cross-attention trong LM bằng *một* module Q-Former và *chỉ* nối đầu ra vào đầu vào của LLM. Tiếp theo: 34 (LLaVA, đơn giản hơn nữa: một lớp tuyến tính), 35 (LLaVA-1.5 so sánh trực tiếp với kiểu Q-Former).

> Ghi chú học tập bằng tiếng Việt, không phải bản dịch.

1. Bối cảnh

Tiền huấn luyện vision-language (VLP) thời đó thường huấn luyện *end-to-end* các mô hình lớn trên hàng trăm triệu cặp ảnh–chữ, rất tốn kém, và khó tái dùng các LLM mạnh đã có. Giải pháp tự nhiên là **đóng băng** cả vision encoder lẫn LLM và chỉ huấn luyện một cầu nối nhỏ. Khó khăn: LLM chưa từng thấy ảnh, nên khi đóng băng nó, việc căn chỉnh (alignment) đặc trưng thị giác vào "không gian chữ" khó. Các phương pháp có sẵn (Frozen, Flamingo, bài 32) chỉ dùng loss sinh ảnh→chữ (language modeling), và BLIP-2 cho rằng loss đó *không đủ* để bắc cầu hai phương thức.

Khái niệm nền:

- *Soft prompt*: vector liên tục (không phải token rời rạc) đặt vào đầu chuỗi đầu vào của LM để điều khiển nó.
- *Image-text contrastive (ITC), image-text matching (ITM), image-grounded text generation (ITG)*: ba loss VLP quen thuộc từ BLIP. ITC kéo embedding ảnh và chữ khớp lại gần nhau (như CLIP); ITM là bài toán nhị phân "cặp này khớp không"; ITG là sinh caption có điều kiện ảnh.
- *Bottleneck (cổ chai thông tin)*: lớp hẹp ép mô hình giữ lại thông tin quan trọng nhất.

2. Ý tưởng chính

BLIP-2 dùng một Transformer nhỏ gọi là **Q-Former** (188M tham số) với **32 query học được**. Các query cross-attend vào đặc trưng của ViT đóng băng (257 x 1024 với ViT-L/14) và cho ra chỉ **32 x 768** vector: đó là cổ chai. Q-Former được huấn luyện hai pha:

1. **Pha 1 (representation learning)**: gắn Q-Former với vision encoder đóng băng, huấn luyện bằng ITC + ITM + ITG để các query học trích xuất thông tin thị giác *liên quan đến văn bản*.
2. **Pha 2 (generative learning)**: nối đầu ra Q-Former qua một lớp fully-connected vào LLM đóng băng như *soft visual prompt*, huấn luyện bằng loss sinh chữ.

Nhờ pha 1, đầu ra Q-Former đã "nói ngôn ngữ" gần với văn bản, nên LLM ít phải làm công việc căn chỉnh và ít bị quên. Kết quả: SOTA zero-shot VQA với số tham số huấn luyện ít hơn Flamingo-80B tới 54 lần.

3. Phương pháp chi tiết

3.1 Kiến trúc Q-Former

Q-Former gồm hai submodule Transformer **dùng chung các lớp self-attention**:

1. **Image transformer**: nhận một tập query học được làm đầu vào, tương tác với nhau qua self-attention, tương tác với đặc trưng ảnh đóng băng qua **cross-attention chèn mỗi hai khối** (every other block).
2. **Text transformer**: hoạt động vừa là text encoder vừa là text decoder; chia sẻ self-attention với image transformer.

- Khởi tạo bằng trọng số **BERT-base**; các lớp cross-attention khởi tạo ngẫu nhiên. Tổng **188M** tham số (query cũng được tính là tham số).
- **32 query, mỗi query 768 chiều** (bằng chiều ẩn của Q-Former). Đầu ra $Z \in \mathbb{R}^{32\times 768}$.
- So sánh kích thước: đặc trưng đóng băng của ViT-L/14 là 257×1024; Z nhỏ hơn nhiều, độc lập với độ phân giải ảnh đầu vào.

Khác với Resampler của Flamingo: Q-Former có nhánh văn bản dùng chung self-attention, cho phép huấn luyện các loss ảnh–chữ, và đầu ra sẽ được nối vào đầu vào LLM, không chèn vào giữa LLM.

3.2 Pha 1: học biểu diễn từ vision encoder đóng băng

Ba loss cùng chia sẻ tham số, mỗi loss dùng **mặt nạ self-attention** khác nhau giữa query (Q) và chữ (T):

Loss	Mask query–text	Làm gì
ITC (contrastive)	*Unimodal*: query và chữ không nhìn thấy nhau	Tính tương đồng giữa từng đầu ra query và embedding t của token [CLS] chữ, **lấy giá trị cao nhất** làm điểm ảnh–chữ; dùng negative trong batch (không dùng momentum queue như BLIP)
ITG (sinh chữ)	*Multimodal causal*: query nhìn nhau, không nhìn chữ; mỗi token chữ nhìn mọi query và các token chữ trước nó	Q-Former phải sinh caption mà chữ **không có đường tới ảnh** ngoài qua query, nên query buộc phải mang đủ thông tin cần để sinh chữ; token [CLS] được thay bằng [DEC]
ITM (khớp)	*Bi-directional*: mọi query và chữ nhìn nhau	Mỗi đầu ra query qua bộ phân loại tuyến tính 2 lớp, **lấy trung bình logit** làm điểm khớp; dùng hard negative mining

Điểm tinh tế: do vision encoder đóng băng, ITG không cho text token nhìn thẳng đặc trưng ảnh. Thông tin phải đi *ảnh -> query -> chữ*. Đó chính là cách "ép" query chứa thông tin liên quan đến văn bản.

3.3 Pha 2: học sinh chữ từ LLM đóng băng

```
ảnh --> [ViT đóng băng] --> [Q-Former] --> Z (32 x 768) --> [FC] --> (32 x d_LLM)
                                                                      |
                                                [soft visual prompt] ++ [text embedding] --> [LLM đóng băng]
                                                ↪  --> chữ
```

$$H_v = \mathrm{FC}(Z) \in \mathbb{R}^{32\times d_{\mathrm{LLM}}}, \qquad \mathrm{input}_{\mathrm{LLM}} = [H_v;\, E(\mathrm{text})]$$

- **LLM chỉ-decoder (OPT)**: loss language modeling thông thường, sinh chữ có điều kiện H_v.
- **LLM encoder–decoder (FlanT5)**: loss *prefix language modeling*: chia chữ thành tiền tố (nối với H_v đưa vào encoder) và hậu tố (mục tiêu của decoder).

Chỉ Q-Former và lớp FC được huấn luyện; ViT và LLM đóng băng (chuyển sang FP16, riêng FlanT5 dùng BFloat16; paper nói không thấy suy giảm so với 32-bit).

Vì Q-Former đã học trích xuất thông tin liên quan văn bản ở pha 1, nó đóng vai trò cổ chai lọc bỏ thông tin thị giác thừa, giảm gánh nặng cho LLM và giảm quên.

3.4 Dữ liệu và huấn luyện

- **129M ảnh**: COCO, Visual Genome, CC3M, CC12M, SBU, và 115M ảnh từ LAION400M.
- **CapFilt** (từ BLIP): sinh 10 caption tổng hợp bằng BLIP_large, xếp hạng cùng caption gốc bằng độ tương đồng của CLIP ViT-L/14, giữ 2 caption tốt nhất, mỗi bước lấy ngẫu nhiên 1.
- Vision encoder: **ViT-L/14 (CLIP)** hoặc **ViT-g/14 (EVA-CLIP)**; bỏ tầng cuối, dùng đặc trưng **tầng áp chót**.
- LLM: OPT (2.7B, 6.7B) và FlanT5 (XL, XXL).
- 250k bước pha 1, 80k bước pha 2. Batch pha 1: 2320/1680 (ViT-L/ViT-g); pha 2: 1920/1520 (OPT/FlanT5).
- AdamW ($\beta_1 = 0.9,\ \beta_2 = 0.98$ [paper ghi nhầm hai lần β_1], weight decay 0.05), cosine decay, lr đỉnh 1e-4, warmup 2k bước; lr tối thiểu pha 2 là 5e-5. Ảnh 224x224.
- Chi phí: mô hình lớn nhất (ViT-g + FlanT5-XXL) trên 1 máy 16 x A100-40G: **dưới 6 ngày cho pha 1 và dưới 3 ngày cho pha 2**.

3.5 Hai chế độ sử dụng

- **Zero-shot có chỉ dẫn**: thêm câu lệnh sau visual prompt (ví dụ "Question: {} Answer:" cho OPT, "Question: {} Short answer:" cho FlanT5), beam=5, length penalty -1.
- **Fine-tune VQA**: tinh chỉnh Q-Former và image encoder, giữ LLM đóng băng; đưa **câu hỏi vào Q-Former** (token câu hỏi tương tác với query qua self-attention) để cross-attention tập trung vùng ảnh liên quan. Đây là điểm "instruction-aware Q-Former", ý tưởng mà InstructBLIP sau đó khai thác (bài 35 nhắc đến).

4. Thí nghiệm và kết quả chính

Zero-shot VQA (Table 2). Số tham số huấn luyện / tổng; số trong bảng:

Model	Params huấn luyện	Tổng	VQAv2 val / test-dev	OK-VQA test	GQA test-dev
Frozen	40M	7.1B	29.6 / -	5.9	-
Flamingo-80B	10.2B	80B	- / 56.3	50.6	-
BLIP-2 ViT-L OPT2.7B	104M	3.1B	50.1 / 49.7	30.2	33.9
BLIP-2 ViT-g OPT6.7B	108M	7.8B	54.3 / 52.6	36.4	36.4
BLIP-2 ViT-g FlanT5-XL	107M	4.1B	63.1 / 63.0	40.7	44.2
BLIP-2 ViT-g FlanT5-XXL	108M	12.1B	65.2 / 65.0	45.9	44.7

BLIP-2 thắng Flamingo-80B 8.7 điểm trên VQAv2 với ít hơn 54 lần tham số huấn luyện; thua trên OK-VQA, mà tác giả đoán là vì OK-VQA cần kiến thức thế giới và Chinchilla-70B biết nhiều hơn FlanT5-XXL 11B.

Captioning (Table 3), BLIP-2 ViT-g, LLM đóng băng, Q-Former + ViT fine-tune trên COCO, đánh giá zero-shot trên NoCaps: overall CIDEr **119.7** (OPT2.7B), **121.0** (OPT6.7B), **121.6** (FlanT5-XL); COCO Karpathy CIDEr 145.8 / 145.2 / 144.5.

VQA fine-tune (Table 4): BLIP-2 ViT-g OPT6.7B test-dev 82.19 / test-std 82.30 (1.2B tham số huấn luyện), so Flamingo-80B 82.00 / 82.10.

Retrieval (Table 5): BLIP-2 ViT-g, Flickr30K zero-shot: Image->Text R@1 97.6, Text->Image R@1 89.7; COCO fine-tune: 85.4 / 68.3.

Tổng quan Table 1: trên 188M tham số huấn luyện, BLIP-2 đạt VQAv2 65.0, NoCaps CIDEr 121.6, Flickr TR@1 97.6.

5. Phân tích / ablation

- **Pha 1 là thiết yếu.** Figure 5: nếu bỏ pha representation learning, cả hai loại LLM đều kém hơn hẳn ở zero-shot VQA; với OPT, hiệu năng thậm chí giảm dần theo huấn luyện (catastrophic forgetting). Paper nói không có pha 1 thì Q-Former chỉ dựa vào loss sinh để bắc cầu, giống Perceiver Resampler của Flamingo.
- **Encoder mạnh hơn hoặc LLM mạnh hơn đều giúp**: ViT-g > ViT-L; LLM lớn hơn > nhỏ hơn trong cùng họ; FlanT5 (instruction-tuned) > OPT trên VQA.
- **ITG giúp cả retrieval** (Table 6): ITC+ITM cho Image->Text R@1 84.5, Text->Image R@1 67.2; thêm ITG lên 85.4 và 68.3.
- Ảnh đầu vào chỉ 224x224 để huấn luyện nhanh.

6. Giới hạn

Do tác giả nêu:

- **Không có in-context learning** cho VQA: đưa thêm ví dụ vào prompt không cải thiện. Tác giả quy cho dữ liệu pretrain chỉ có *một* cặp ảnh–chữ mỗi mẫu, không như M3W của Flamingo.
- Sinh chữ vẫn có thể sai do tri thức LLM không chính xác, lập luận sai, thiếu thông tin cập nhật; kế thừa rủi ro của LLM đóng băng (thiên kiến, nội dung xúc phạm, rò rỉ thông tin riêng).

Nhận xét bổ sung:

- 32 token cổ chai nén mọi thông tin ảnh; paper không đánh giá OCR dày đặc hay chi tiết nhỏ (ảnh chỉ 224x224). Nhiều nghiên cứu sau (bài 35) cho rằng việc nén kiểu này có thể bất lợi khi cần chi tiết.
- Quy trình hai pha và ba loss phức tạp hơn nhiều so với LLaVA; chi phí thiết kế/tinh chỉnh cao.

7. Liên hệ với các bài khác

- **32 Flamingo**: cùng chủ đề đóng băng hai mô hình; Flamingo chèn xattn vào *giữa* LLM, BLIP-2 chỉ nối vào *đầu* LLM. Resampler (64 token) vs Q-Former (32 token).
- **30 CLIP / 31 SigLIP**: BLIP-2 dùng CLIP ViT-L và EVA-CLIP ViT-g làm vision encoder; ITC là loss kiểu CLIP.
- **34 LLaVA**: bỏ cả Q-Former, chỉ dùng một lớp tuyến tính (về sau MLP) trên *toàn bộ* patch token, và fine-tune cả LLM. **35 LLaVA-1.5**: cho thấy một MLP đơn giản + dữ liệu tốt vượt InstructBLIP/Q-Former dù dùng 129M cặp pretrain.
- **08 LoRA**: một cách thích ứng LLM nhẹ khác; BLIP-2 đóng băng hoàn toàn LLM.

8. Gợi ý áp dụng

Q-Former dùng một tập query học được để chuyển đặc trưng ảnh thành số lượng token cố định. Đây là cách kiểm soát chi phí giao tiếp với LLM, nhưng số query và mức giữ chi tiết cần được đánh giá theo tác vụ; nén tốt thông tin ngữ nghĩa không bảo đảm giữ đủ chi tiết ký tự nhỏ.

9. Thí nghiệm gợi ý

So sánh các số lượng query với một baseline gộp patch đơn giản, đo đồng thời chất lượng và chi phí. Phân tích riêng các mẫu cần thông tin cục bộ để tìm giới hạn của phép nén.

10. Tóm tắt công thức / khái niệm

Khái niệm	Nội dung
Q-Former	188M tham số, khởi tạo BERT-base, 32 query x 768, cross-attention mỗi 2 khối
Cổ chai	Z: 32x768 so với đặc trưng ViT-L/14: 257x1024
ITC	tương đồng = max trên query của $\cos(Z_i, t)$; unimodal mask; in-batch negative
ITG	mask multimodal causal; thông tin chỉ đi ảnh -> query -> chữ
ITM	mask hai chiều; trung bình logit của các query; hard negative
Pha 2	$H_v = \text{FC}(Z)$ làm soft prompt cho LLM đóng băng; OPT: LM loss, FlanT5: prefix LM loss
Huấn luyện	250k + 80k bước, 129M ảnh, CapFilt
Kết quả	ZS VQAv2 65.0 (FlanT5-XXL), hơn Flamingo-80B 8.7 điểm, 54x ít tham số huấn luyện

11. Kết luận ngắn

BLIP-2 cho thấy chỉ cần một cầu nối nhỏ (Q-Former 188M) huấn luyện hai pha, giữa ViT và LLM đều đóng băng, là đạt SOTA zero-shot VQA và captioning với chi phí thấp. Đóng góp cốt lõi là *pha học biểu diễn trước* khiến đầu ra cổ chai trở nên "dễ đọc" cho LLM. Hạn chế là mất khả năng in-context và 32 token nén có thể không đủ cho chi tiết nhỏ; các bài sau (LLaVA, InternVL) chọn con đường ngược lại: giữ nhiều token và chi tiết hơn.

Link gốc: https://arxiv.org/abs/2301.12597

Bài 34

Visual Instruction Tuning (LLaVA) (2023)

Tác giả: Haotian Liu, Chunyuan Li, Qingyang Wu, Yong Jae Lee (University of Wisconsin-Madison, Microsoft Research, Columbia University).
Venue / arXiv: arXiv:2304.08485 (v2, 11/12/2023); NeurIPS 2023 (paper ghi "37th Conference on Neural Information Processing Systems").
Nguồn: https://arxiv.org/abs/2304.08485
Chủ đề: Visual instruction tuning; dữ liệu chỉ dẫn đa phương thức do GPT-4 sinh từ caption/bounding box; kiến trúc ViT + một lớp tuyến tính + LLM; huấn luyện hai giai đoạn.
Vị trí trong lộ trình: Đọc trước: 05 (LLaMA, Vicuna dựa trên nó), 07 (InstructGPT, ý tưởng instruction tuning), 29 (ViT), 30 (CLIP), 32–33 (Flamingo, BLIP-2 là những lựa chọn "phức tạp hơn" mà LLaVA cố tình tránh). Bài này giới thiệu một kiến trúc "ViT -> projector -> LLM, nối thẳng token ảnh vào chuỗi" được phát triển trong nhiều VLM mở về sau. Tiếp theo: 35 (LLaVA-1.5, cải tiến đơn giản nhưng hiệu quả), 36 (InternVL 1.5).

> Ghi chú học tập bằng tiếng Việt, không phải bản dịch.

1. Bối cảnh

Trong NLP, **instruction tuning** (tinh chỉnh LLM trên các cặp "chỉ dẫn -> câu trả lời" để nó làm theo yêu cầu của người dùng, như InstructGPT ở bài 07, Alpaca, Vicuna) giúp mô hình tổng quát hoá tốt sang task mới. Nhưng trong thị giác-ngôn ngữ, các mô hình trước đó (Flamingo, BLIP-2) được huấn luyện chủ yếu bằng cặp ảnh–caption, nên chúng *mô tả* ảnh hơn là *làm theo chỉ dẫn* về ảnh. Cản trở lớn: **thiếu dữ liệu chỉ dẫn đa phương thức**, vì thuê người viết rất tốn và khó định nghĩa.

Câu hỏi của paper: có thể (1) tạo dữ liệu chỉ dẫn ảnh–chữ bằng cách nhờ một LLM mạnh chỉ nhận văn bản, và (2) ghép một vision encoder với LLM một cách đơn giản nhất để huấn luyện trên dữ liệu đó không?

2. Ý tưởng chính

1. **Dữ liệu:** GPT-4 (chỉ nhận chữ) không thấy ảnh, nên paper đưa cho nó *biểu diễn ký hiệu* của ảnh COCO: các caption và các bounding box (nhãn lớp + toạ độ). Từ đó GPT-4 sinh 3 loại dữ liệu: hội thoại, mô tả chi tiết, suy luận phức tạp, tổng cộng **158K** mẫu.
2. **Mô hình:** lấy ViT-L/14 của CLIP, lấy đặc trưng patch, nhân với **một ma trận chiếu W (lớp tuyến tính)** để thành các "visual token" cùng chiều với embedding từ của Vicuna, rồi nối vào chuỗi đầu vào của LLM. Không Resampler, không Q-Former, không cross-attention.
3. **Huấn luyện hai giai đoạn:** (a) chỉ train W trên cặp ảnh–caption để căn chỉnh; (b) train W + toàn bộ LLM trên dữ liệu chỉ dẫn.

Kết quả: LLaVA làm theo chỉ dẫn tốt hơn hẳn BLIP-2 và OpenFlamingo trên bộ benchmark tự tạo, và (kết hợp với GPT-4) đạt SOTA 92.53% trên ScienceQA.

3. Phương pháp chi tiết

3.1 Sinh dữ liệu chỉ dẫn bằng GPT-4

Cách "ngây thơ": với ảnh X_v và caption X_c, tạo câu hỏi X_q yêu cầu mô tả ảnh, rồi dùng khuôn `Human: X_q X_v <STOP> Assistant: X_c <STOP>`. Rẻ nhưng thiếu đa dạng và chiều sâu.

Cách của paper: dùng hai loại ngữ cảnh ký hiệu cho mỗi ảnh COCO:

- **Captions** (nhiều caption mô tả cảnh ở các góc nhìn khác nhau).
- **Bounding boxes** (lớp đối tượng + vị trí, dạng `person: [0.681, 0.242, 0.774, 0.694]`).

Tác giả chỉ viết thủ công vài ví dụ mẫu cho mỗi loại (đây là *chú thích người duy nhất*), dùng làm in-context seed để gọi GPT-4. Ba loại phản hồi:

Loại	Mô tả	Số mẫu
Conversation	Hội thoại nhiều lượt: hỏi loại đối tượng, đếm, hành động, vị trí, quan hệ không gian; chỉ giữ câu hỏi có đáp án chắc chắn	58K
Detailed description	Một câu hỏi ngẫu nhiên từ danh sách câu hỏi "mô tả chi tiết" -> GPT-4 viết mô tả phong phú	23K
Complex reasoning	Câu hỏi sâu cần lập luận từng bước	77K

Tổng **158K** mẫu ảnh–chữ. Paper ghi: GPT-4 cho dữ liệu chất lượng cao hơn ChatGPT (đặc biệt suy luận không gian), theo thử nghiệm sớm.

3.2 Kiến trúc

- LLM f_ϕ: **Vicuna** (chọn vì khả năng làm theo chỉ dẫn tốt nhất trong các checkpoint mở lúc đó).
- Vision encoder: **CLIP ViT-L/14**. Đặc trưng lưới $Z_v = g(X_v)$ lấy *trước* hoặc *sau* tầng Transformer cuối (thí nghiệm cả hai).
- Cầu nối: một lớp tuyến tính huấn luyện được.

$$\boxed{H_v = W \cdot Z_v, \qquad Z_v = g(X_v)}$$

H_v là chuỗi visual token có cùng chiều với embedding từ. Paper nói rõ: cách chiếu đơn giản này "lightweight", cho phép lặp thí nghiệm dữ liệu nhanh; các phương án phức tạp hơn (gated cross-attention của Flamingo, Q-Former của BLIP-2) để dành cho tương lai.

Số visual token (tự tính, không có trong paper): ViT-L/14 với ảnh 224x224 cho lưới $16 \times 16 = 256$ patch, tức 256 visual token mỗi ảnh (nếu lấy toàn bộ patch, không tính token CLS). Phiên bản 336 px (bài 35) cho $24 \times 24 = 576$.

3.3 Định dạng huấn luyện

Với mỗi ảnh có hội thoại nhiều lượt $(X_q^1, X_a^1, \ldots, X_q^T, X_a^T)$. Chỉ dẫn lượt t (Eq. 2):

$$X_{\text{instruct}}^t = \begin{cases} \text{ngẫu nhiên } [X_q^1, X_v] \text{ hoặc } [X_v, X_q^1], & t = 1 \\ X_q^t, & t > 1 \end{cases}$$

Chuỗi: `system-message <STOP> Human: X_instruct^1 <STOP> Assistant: X_a^1 <STOP> Human: ...` với `<STOP>` = `###` (theo Vicuna-v0). Loss là next-token **chỉ tính trên token trả lời của Assistant và token dừng** (phần xanh trong Table 2 của paper):

$$p(X_a \mid X_v, X_{\text{instruct}}) = \prod_{i=1}^{L} p_\theta(x_i \mid X_v,\, X_{\text{instruct},<i},\, X_{a,<i})$$

3.4 Hai giai đoạn huấn luyện

	Giai đoạn 1: căn chỉnh đặc trưng	Giai đoạn 2: fine-tune đầu-cuối
Dữ liệu	CC3M lọc còn **595K** cặp ảnh–caption, đổi thành chỉ dẫn "mô tả ngắn ảnh" (đáp án = caption gốc)	LLaVA-Instruct-158K (chatbot) hoặc ScienceQA
Đóng băng	ViT **và** LLM	ViT
Huấn luyện	chỉ W ($\theta = W$)	W **và** toàn bộ LLM ($\theta = \{W, \phi\}$)
Siêu tham số	1 epoch, lr 2e-3, batch 128	3 epoch, lr 2e-5, batch 32

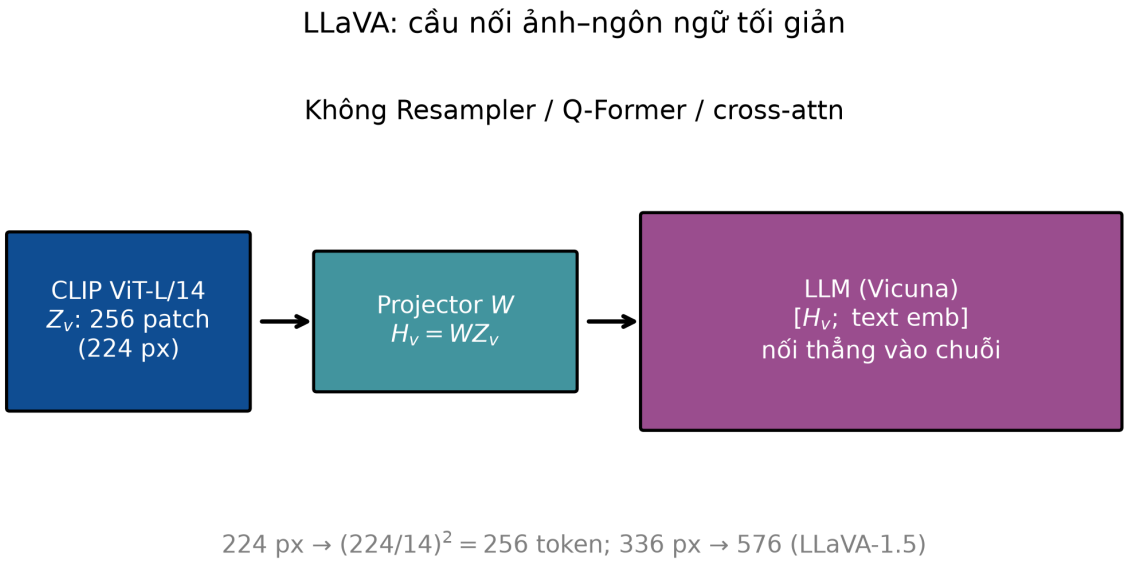
Paper mô tả giai đoạn 1 là "huấn luyện một **visual tokenizer** tương thích với LLM đóng băng". Huấn luyện trên 8 x A100 theo siêu tham số của Vicuna.

```
# Giai đoạn 1: chỉ train W
freeze(vit); freeze(llm)
for img, caption in cc595k:
    z = vit(img)                 # [N_patch, 1024]
    h = W(z)                     # [N_patch, d_llm], W: Linear
    loss = ce(llm([h, prompt]), caption)   # loss chỉ trên caption
# Giai đoạn 2: train W + llm
freeze(vit)
for img, conv in llava_instruct_158k:
    loss = ce(llm([h, turns...]), assistant_tokens_only)
```

4. Thí nghiệm và kết quả chính

4.1 Multimodal chatbot (LLaVA-Bench)

Đánh giá bằng GPT-4 làm giám khảo: với mỗi (ảnh, mô tả ground-truth, câu hỏi), một GPT-4 chỉ-chữ nhận mô tả ground-truth và trả lời để làm "cận trên xấp xỉ", rồi GPT-4 chấm cả hai (thang 1-10: hữu ích, liên quan, chính xác, chi tiết). Báo cáo **điểm tương đối** so với cận trên đó.



LLaVA: CLIP ViT → projector W → nối H_v vào LLM; 256 token @224 px (576 @336). (minh hoạ, tự tính)

Ablation dữ liệu trên LLaVA-Bench (COCO), 30 ảnh x 3 loại câu hỏi = 90 câu (Table 4):

Dữ liệu huấn luyện	Conversation	Detail	Complex reasoning	Tất cả
Đầy đủ	83.1	75.3	96.5	**85.1**
Detail + Complex	81.5	73.3	90.8	81.9
Conv + 5% Detail + 10% Complex	81.0	68.4	91.5	80.5
Chỉ Conversation	76.5	59.8	84.9	73.8
Không instruction tuning	22.0	24.0	18.5	21.5

LLaVA-Bench (In-the-Wild): 24 ảnh, 60 câu hỏi (trong nhà, ngoài trời, meme, tranh, phác thảo...) kèm mô tả thủ công rất chi tiết (Table 5):

Model	Conversation	Detail	Complex	Tất cả
OpenFlamingo	19.3	19.0	19.1	19.1
BLIP-2	54.6	29.1	32.9	38.1
LLaVA	57.3	52.5	81.7	**67.3**

LLaVA hơn BLIP-2 29 điểm và OpenFlamingo 48 điểm (theo paper).

4.2 ScienceQA

21K câu hỏi trắc nghiệm đa phương thức. LLaVA (13B, đặc trưng trước tầng cuối, dự đoán lập luận rồi đáp án, 12 epoch) đạt **90.92%**, SOTA trước đó MM-CoT-Large 91.68%. GPT-4 chỉ-chữ 2-shot đạt 82.69%.

Phương pháp	Trung bình
GPT-3.5 w/ CoT	75.17
LLaMA-Adapter	85.19
MM-CoT Large (SOTA trước)	91.68
GPT-4 (chỉ chữ, 2-shot)	82.69
LLaVA	90.92
LLaVA + GPT-4 (complement)	90.97
LLaVA + GPT-4 (judge)	**92.53**

“Judge”: khi GPT-4 và LLaVA bất đồng, hỏi lại GPT-4 để chốt đáp án dựa trên cả hai kết quả; đây là lần đầu dùng GPT-4 để ensemble theo cách này (theo paper).

5. Phân tích / ablation

Bảng ScienceQA (Table 8):

Biến thể	Độ chính xác
Tốt nhất (13B, đặc trưng trước tầng cuối, lập luận trước)	90.92
Dùng đặc trưng tầng cuối CLIP	89.96 (-0.96)
Đáp án trước lập luận	89.77 (-1.15)
Không pretrain giai đoạn 1 (train từ đầu)	85.81 (-5.11)
Model 7B	89.84 (-1.08)

- **Đặc trưng trước tầng cuối tốt hơn tầng cuối**: giả thuyết của tác giả (chưa kiểm chứng thêm) là tầng cuối của CLIP thiên về thuộc tính toàn cục/trừu tượng, còn tầng trước giữ thuộc tính cục bộ hữu ích cho chi tiết.
- **Giai đoạn 1 quan trọng** (+5.11): căn chỉnh trước giữ tri thức LLM mà vẫn học đặc trưng đa phương thức.
- Lập luận trước đáp án hội tụ nhanh hơn (6 epoch đã 89.77) nhưng ít cải thiện kết quả cuối.
- **Quy mô LLM** giúp (13B > 7B).
- Dữ liệu suy luận phức tạp + mô tả chi tiết cải thiện cả hội thoại thường: "reasoning complements conversation".

6. Giới hạn

Do tác giả nêu:

- Câu hỏi khó cần **kiến thức rộng, đa ngôn ngữ, truy xuất thông tin** (ví dụ tên nhà hàng ramen) hoặc **ảnh độ phân giải cao** (nhận nhãn hiệu sữa chua) vẫn sai.
- LLaVA đôi lúc coi ảnh như "**bag of patches**": ví dụ trả lời "có" cho "có sữa chua vị dâu không?" trong khi tủ lạnh chỉ có sữa chua và dâu riêng. Tức là chưa nắm quan hệ ngữ nghĩa phức tạp.
- Dữ liệu sinh bằng GPT-4 từ ký hiệu có thể mang lỗi/ảo giác của GPT-4; huấn luyện ~80K ảnh unique là nhỏ.

Nhận xét bổ sung:

- Đánh giá dựa vào GPT-4 làm giám khảo trên bộ nhỏ (30 + 24 ảnh) nên nhiễu và thiên lệch; paper tự báo std của 3 lần chạy.
- Độ phân giải cố định 224 px (256 token) không phù hợp với văn bản dày/nhỏ. Bài 35 giải quyết một phần.
- Chưa đánh giá trên các benchmark học thuật câu trả lời ngắn (VQAv2...); bài 35 chỉ ra LLaVA gốc thường trả lời "yes" cho câu hỏi yes/no vì thiếu loại dữ liệu này.

7. Liên hệ với các bài khác

- **07 InstructGPT**: ý tưởng instruction tuning; LLaVA mang nó sang đa phương thức, dùng GPT-4 sinh dữ liệu thay cho người gán nhãn.
- **05 LLaMA**: Vicuna là LLaMA tinh chỉnh bằng hội thoại ShareGPT.
- **30 CLIP**: bộ mã hoá ảnh; nhấn mạnh việc lấy đặc trưng tầng áp chót (cũng như BLIP-2 ở bài 33).
- **32 Flamingo / 33 BLIP-2**: LLaVA thay Resampler/Q-Former bằng một lớp tuyến tính; đánh đổi: nhiều visual token hơn nhưng đơn giản và fine-tune được cả LLM.
- **35 LLaVA-1.5**: nâng lên MLP 2 lớp, ViT 336 px, thêm dữ liệu học thuật + format prompt.
- **08 LoRA**: giai đoạn 2 của LLaVA fine-tune đầy đủ LLM; LoRA là phương án rẻ hơn (LLaVA-LoRA xuất hiện về sau, ngoài phạm vi paper này).

8. Gợi ý áp dụng

LLaVA tách căn chỉnh bộ chiếu ảnh–ngôn ngữ khỏi tinh chỉnh theo chỉ dẫn. Các giai đoạn có mục tiêu và tập dữ liệu khác nhau; để so sánh, cần theo dõi tác động của từng giai đoạn và chất lượng dữ liệu, không chỉ kích thước bộ chiếu.

9. Thí nghiệm gợi ý

So sánh bộ chiếu tuyến tính và MLP trên cùng bộ mã hoá, dữ liệu và ngân sách. Kiểm tra cả khả năng trả lời câu hỏi về ảnh lẫn các mẫu đòi hỏi quan hệ không gian, vì điểm tổng hợp có thể che khuất lỗi theo nhóm.

10. Tóm tắt công thức / khái niệm

Khái niệm	Nội dung
Visual token	$H_v = WZ_v$, Z_v = đặc trưng patch của CLIP ViT-L/14
Dữ liệu	158K = 58K conversation + 23K detailed description + 77K complex reasoning, sinh bởi GPT-4 từ caption + bbox
Loss	next-token, chỉ trên token Assistant: $\prod_i p(x_i \mid X_v, X_{\text{instruct},<i}, X_{a,<i})$
Giai đoạn 1	CC3M-595K, chỉ train W, lr 2e-3, 1 epoch, batch 128
Giai đoạn 2	train W + LLM, lr 2e-5, 3 epoch, batch 32 (ScienceQA: 12 epoch)
Kết quả chính	LLaVA-Bench COCO 85.1; In-the-Wild 67.3; ScienceQA 90.92 / 92.53 (với GPT-4 judge)

11. Kết luận ngắn

LLaVA chứng minh rằng một thiết kế rất đơn giản (ViT đóng băng + lớp tuyến tính + LLM fine-tune) cộng với dữ liệu chỉ dẫn do GPT-4 sinh từ caption và bounding box là đủ để có một trợ lý thị giác làm theo chỉ dẫn tốt. Đóng góp lớn nhất là *dữ liệu* và *công thức hai giai đoạn*, không phải kiến trúc.

Link gốc: https://arxiv.org/abs/2304.08485

Bài 35

Improved Baselines with Visual Instruction Tuning (LLaVA-1.5) (2024)

Tác giả: Haotian Liu, Chunyuan Li, Yuheng Li, Yong Jae Lee (University of Wisconsin-Madison, Microsoft Research).
Venue / arXiv: arXiv:2310.03744 (v2, 15/05/2024). Text paper không ghi tên hội nghị. (Bản đầu đăng tháng 10/2023; ghi chú đặt năm 2024 theo phiên bản v2.)
Nguồn: https://arxiv.org/abs/2310.03744
Chủ đề: Nghiên cứu có kiểm soát các lựa chọn thiết kế của LMM trong khung LLaVA: MLP connector, dữ liệu học thuật + response-format prompt, ViT 336 px, LLaVA-1.5-HD (chia ảnh thành lưới).
Vị trí trong lộ trình: Đọc trước: 34 (LLaVA), 33 (BLIP-2/InstructBLIP, đối tượng so sánh), 30 (CLIP). Bài này chứng minh "đơn giản + dữ liệu đúng" thắng "cầu nối phức tạp + pretrain khổng lồ", và giới thiệu ý tưởng chia ảnh thành lưới độ phân giải cao (tiền thân của dynamic tiling). Tiếp theo: 36 (InternVL 1.5 đẩy dynamic tiling lên 1–40 tile + pixel shuffle), 37 (Qwen2-VL, độ phân giải động gốc).

> Ghi chú học tập bằng tiếng Việt, không phải bản dịch.

1. Bối cảnh

Sau LLaVA (bài 34), cộng đồng có hai hướng khác nhau:

- **LLaVA**: hội thoại tự nhiên rất tốt, nhưng yếu ở benchmark học thuật cần câu trả lời ngắn (một từ); hay trả lời "yes" cho câu yes/no do thiếu loại dữ liệu đó.
- **InstructBLIP**: mạnh ở VQA câu ngắn, nhưng kém trong hội thoại đời thực; nó pretrain Q-Former trên 129M cặp, chỉ fine-tune Q-Former khi instruction tuning, và dễ overfit sang câu trả lời ngắn ngay cả khi người dùng cần mô tả dài (Table 1a của paper).

Chưa rõ nguyên nhân gốc: do lượng dữ liệu, do bộ lấy mẫu lại như Q-Former, hay do cách trộn dữ liệu? Các hệ thống lớn khác (Qwen-VL: 1.4B cặp pretrain, 50M dữ liệu fine-tune nội bộ) thì không tái lập được. Paper này là **nghiên cứu có kiểm soát đầu tiên** về các lựa chọn thiết kế trong khung LLaVA, chỉ dùng dữ liệu công khai.

2. Ý tưởng chính

Bốn thay đổi trong khung LLaVA giúp mô hình đạt kết quả hàng đầu trên 11 benchmark theo báo cáo với **1.2M** dữ liệu công khai, huấn luyện xong cỡ ~1 ngày trên một máy 8 x A100:

1. **Connector MLP 2 lớp** thay lớp tuyến tính.
2. **Dữ liệu học thuật** (VQAv2, GQA, OKVQA, OCR-VQA, TextCaps, A-OKVQA, RefCOCO, Visual Genome) trộn cùng dữ liệu hội thoại, kèm **response-format prompt** (một câu chỉ rõ định dạng đầu ra, ví dụ "Answer the question using a single word or phrase.") để mô hình không nhầm giữa trả lời ngắn và dài.
3. **CLIP-ViT-L-336px** (ảnh 336 thay vì 224) và LLM lớn hơn (13B).
4. **LLaVA-1.5-HD**: chia ảnh thành lưới các mảnh 224 x 224, mã hoá riêng từng mảnh, ghép lại, thêm ảnh thu nhỏ làm ngữ cảnh toàn cục.

Thông điệp: cầu nối đầy đủ kết nối (fully-connected) của LLaVA "hiệu quả và tiết kiệm dữ liệu đáng ngạc nhiên"; không cần Resampler hay pretrain khổng lồ.

3. Phương pháp chi tiết

3.1 Response format prompting

Vấn đề: dữ liệu như `Q: {Question} A: {Answer}` không nói rõ định dạng đầu ra, nên LLM có thể bị lệch sang câu trả lời ngắn cho cả hội thoại tự nhiên. Kèm theo đó, InstructBLIP chỉ fine-tune Q-Former (không fine-tune LLM) nên phải dựa vào token ra của Q-Former để điều khiển độ dài đầu ra như prefix tuning, mà Q-Former có thể thiếu dung lượng để làm tốt.

Giải pháp: nối **một câu chỉ định định dạng rõ ràng** vào cuối câu hỏi VQA khi cần câu ngắn:

Loại	Prompt định dạng (Table 7/8)
VQAv2, GQA, TextVQA, MME, POPE	“Answer the question using a single word or phrase.”
A-OKVQA, ScienceQA, MMBench, SEED	“Answer with the option’s letter from the given choices directly.”
TextCaps	“Provide a one-sentence caption for the provided image.”
VizWiz	“When the provided information is insufficient, respond with ’Unanswerable’. Answer the question using a single word or phrase.”
Visual Genome / RefCOCO	“Provide the bounding box coordinate of the region this sentence describes.” (region-level, toạ độ ở dạng chữ)
LLaVA-Bench, MM-Vet	không dùng prompt định dạng

Kết quả: chỉ thêm VQAv2 + format prompt, MME của LLaVA tăng từ 809.6 lên 1323.8 (Table 2) và vượt InstructBLIP 111 điểm. Mô hình cũng tổng quát hoá format: sau khi học, VizWiz “Unanswerable” tăng 11.1% -> 67.8%.

3.2 MLP connector

Thay $H_v = WZ_v$ bằng MLP hai lớp (có phi tuyến, thường GELU; paper chỉ nói “two-layer MLP”):

$$\boxed{H_v = W_2\,\sigma(W_1 Z_v)}$$

Lý do (theo paper): lấy cảm hứng từ self-supervised learning, nơi đổi từ projection tuyến tính sang MLP cải thiện chất lượng biểu diễn. Trong thực tế paper chỉ báo cáo đây là một dòng cải tiến nhỏ (Table 2: MME 1323.8 -> 1355.2; GQA 46.8 -> 47.3; MM-Vet 26.3 -> 27.8); đồng thời **giảm một nửa learning rate pretrain** (còn 1e-3) vì dùng MLP.

3.3 Dữ liệu (665K mẫu)

Hỗn hợp huấn luyện giai đoạn 2 (Table 7):

Nguồn	Số mẫu
LLaVA-Instruct	158K
ShareGPT (chữ thuần, giữ năng lực ngôn ngữ)	40K
VQAv2	83K
GQA	72K
OKVQA	9K
OCRVQA	80K
A-OKVQA	66K
TextCaps	22K

Nguồn	Số mẫu
RefCOCO	48K
Visual Genome	86K
Tổng	**665K**

Giai đoạn 1 (căn chỉnh) dùng 558K cặp ảnh–chữ (Table 3). Chiến lược giảm chi phí: gộp mọi cặp hỏi–đáp cùng một ảnh thành một hội thoại; cắt hội thoại ShareGPT dài tới 2048 token; mỗi batch chỉ lấy hội thoại từ *một* modality (tăng tốc 25%, không ảnh hưởng kết quả cuối); A-OKVQA nhân lên k lần (k = số lựa chọn) để bù thiếu dữ liệu trắc nghiệm.

Siêu tham số (Table 9): Pretrain: batch 256, lr 1e-3; Finetune: batch 128, lr 2e-5; cosine decay, warmup 0.03, weight decay 0, 1 epoch, AdamW, DeepSpeed ZeRO-2 (pretrain) / ZeRO-3 (finetune). Đánh giá dùng greedy decoding. LLM nền: Vicuna v1.5.

3.4 Tăng độ phân giải: 336 px và LLaVA-1.5-HD

Bước 1: đổi sang CLIP-ViT-L-336px (độ phân giải cao nhất của CLIP công khai lúc đó). Số visual token tăng: $(336/14)^2 = 576$ so với $(224/14)^2 = 256$ (tự tính; ViT-L/14 patch 14). Thời gian huấn luyện khoảng gấp đôi LLaVA: ~6 giờ pretrain + ~20 giờ instruction tuning trên 8 x A100.

Bước 2: LLaVA-1.5-HD vì không có ViT công khai độ phân giải cao hơn, và nội suy positional embedding thường đòi hỏi pretrain thêm trên lượng lớn cặp ảnh–chữ:

```
1. Chọn độ phân giải đích từ tập định sẵn (tối đa 6 lưới:
↪  1x1,1x2,1x3,1x4,1x5,1x6,2x2,2x3 và chuyển vị;
   lớn nhất 672x448 / 448x672). Tiêu chí: giữ chi tiết nhiều nhất VÀ không quá lớn.
2. Pad + resize ảnh tới độ phân giải đích, cắt thành các mảnh 224x224 (kích thước ViT
↪  được train).
3. Mã hoá từng mảnh độc lập bằng CLIP ViT-L/14-224, ghép lại thành một feature map lớn.
4. Hậu xử lý: (a) bỏ token padding thuần, (b) thêm token đặc biệt "cuối hàng" để biết
↪  hình dạng ảnh,
   (c) duỗi (flatten).
5. Nối thêm đặc trưng của ảnh thu nhỏ 224x224 (global context).
```

Không cần pretrain lại connector ở độ phân giải cao; vào thẳng visual instruction tuning. Thành công vì mỗi mảnh khớp đúng độ phân giải mà ViT được huấn luyện.

4. Thí nghiệm và kết quả chính

Lộ trình cải tiến từng bước (Table 2) trên GQA, MME, MM-Vet:

#	Bước	LLM	Res.	GQA	MME	MM-Vet
-	InstructBLIP	14B	224	49.5	1212.8	25.6
0	LLaVA	7B	224	-	809.6	25.5
1	+VQAv2	7B	224	47.0	1197.0	27.7
2	+Format prompt	7B	224	46.8	1323.8	26.3
3	+MLP VL connector	7B	224	47.3	1355.2	27.8
4	+OKVQA/OCR	7B	224	50.0	1377.6	29.6
5	+Region-level VQA	7B	224	50.3	1426.5	30.8
6	+Scale up resolution	7B	336	51.4	1450	30.3
7	+GQA	7B	336	62.0*	1469.2	30.7

#	Bước	LLM	Res.	GQA	MME	MM-Vet
8	+ShareGPT	7B	336	62.0*	1510.7	31.1
9	+Scale up LLM	13B	336	63.3*	1531.3	36.1

(* ảnh huấn luyện GQA đã được thấy khi huấn luyện.) Chỉ với tập con dữ liệu của InstructBLIP, LLaVA đã vượt nó trên cả ba chỉ số (bước 3-4 trở đi).

So sánh SOTA, benchmark học thuật (Table 3):

Model	Res.	Pretrain / Finetune	VQAv2	GQA	VizWiz	SQA-IMG	TextVQA
BLIP-2 (Vicuna-13B)	224	129M / -	65.0	41	19.6	61	42.5
InstructBLIP (13B)	224	129M / 1.2M	-	49.5	33.4	63.1	50.7
IDEFICS-80B	224	353M / 1M	60.0	45.2	36.0	-	30.9
Qwen-VL-Chat	448	1.4B / 50M	78.2*	57.5*	38.9	68.2	61.5*
LLaVA-1.5 7B	336	558K / 665K	78.5*	62.0*	50.0	66.8	58.2
LLaVA-1.5 13B	336	558K / 665K	80.0*	63.3*	53.6	71.6	61.3
LLaVA-1.5-HD 13B	448	558K / 665K	81.8*	64.7*	57.5	71.0	62.5

Benchmark theo chỉ dẫn (Table 4), LLaVA-1.5-13B: POPE (rand/pop/adv) 87.1 / 86.2 / 84.5; MME 1531.3; MMBench 67.7 (en) / 63.6 (cn); SEED-Bench 61.6 (all), 68.2 (img), 42.7 (vid); LLaVA-Wild 72.5; MM-Vet 36.1. LLaVA-7B gốc: MME 809.6, MMBench 38.7, LLaVA-Wild 62.8, MM-Vet 25.5. LLaVA-1.5-13B-HD: MM-Vet 39.4, MMBench 68.8.

Thông điệp: LLaVA-1.5 vượt IDEFICS-80B (kiểu Flamingo với hàng tỷ tham số cầu nối) dù chỉ dùng 558K + 665K mẫu.

5. Phân tích / ablation

- **Global context cho HD** (bảng nhỏ, 7B): chỉ high-res patch: GQA 62.9, MME 1425.8, MM-Vet 31.9; thêm ảnh thu nhỏ toàn cục: 63.8 (+0.9), 1497.5 (+71), 35.1 (+3.2). Ảnh toàn cục khắc phục lỗi của bước chia-mã hoá-ghép và giúp định vị vùng liên quan.
- **LLM nền** (Fig. 3): Vicuna-v1.5 (LLaMA-2) tốt nhất; mô hình dựa LLaMA-2 nói chung tốt hơn dựa LLaMA-1. LLaMA-2-Chat (có RLHF) tổng quát hoá sang tiếng Trung (MMBench-CN) kém hơn Vicuna-1.5 do dữ liệu SFT/RLHF chủ yếu tiếng Anh; tức dữ liệu *ngôn ngữ* của LLM nền ảnh hưởng năng lực đa phương thức.
- **Hiệu quả dữ liệu** (Fig. 4): giữ chỉ 50% mẫu vẫn giữ trên 98% hiệu năng đầy đủ; MMBench, ScienceQA, POPE không giảm khi xuống 50%, thậm chí MMBench nhỉnh hơn; giữ ổn định cả ở 30%. Hàm ý "less is more" cho LMM.
- **Ảo giác** (5.2): ảo giác trong dữ liệu mô tả chi tiết giảm rõ khi tăng độ phân giải lên 448. Giả thuyết tác giả: mô hình khá bền với vài lỗi trong dữ liệu, nhưng khi độ phân giải không đủ để thấy chi tiết mà dữ liệu lại mô tả ở mức chi tiết đó, mô hình học cách bịa.
- **Năng lực tổ hợp** (5.3): thêm ShareGPT cải thiện cả đa ngôn ngữ trong hội thoại ảnh (LLaVA-1.5 đạt 63.6% MMBench-CN, hơn Qwen-VL-Chat 56.7% dù không fine-tune tiếng Trung) và độ dài/độ chi tiết câu trả lời; dữ liệu học thuật cải thiện "visual grounding".

- **Tổng quát hoá format**: chỉ với một vài format prompt, mô hình theo được format khác (JSON ràng buộc, "Unanswerable").

6. Giới hạn

Do tác giả nêu:

- Huấn luyện bản độ phân giải cao kéo dài; **không hiểu nhiều ảnh**; năng lực giải quyết vấn đề hạn chế trong một số lĩnh vực; vẫn có ảo giác, không nên dùng trong ứng dụng quan trọng như y tế (phụ lục).
- Chưa cân bằng được tốt mọi tổ hợp năng lực (đúng thuộc tính trong VQA nhưng mô tả chi tiết cả ảnh chưa chắc đúng), và hội thoại tiếng Hàn vẫn yếu (Section 5.3).
- Nhiều điểm VQAv2/GQA có dấu * nghĩa là ảnh huấn luyện của benchmark đã thấy trong huấn luyện, nên không hoàn toàn zero-shot.

Nhận xét bổ sung:

- Vẫn là ảnh nén vào lưới patch ViT cố định (576 token, hoặc nhiều hơn với HD); với văn bản dày (tài liệu), 448-672 px là chưa đủ so với các hệ sau (bài 36 lên tới 4K).
- Các so sánh chủ yếu giữa các hệ khác dữ liệu/LLM nên "đơn giản thắng phức tạp" là kết luận về cả công thức, không tách riêng được ảnh hưởng của connector MLP vs Q-Former (không có ablation Q-Former trong cùng điều kiện dữ liệu).

7. Liên hệ với các bài khác

- **34 LLaVA**: bài gốc; 1.5 sửa 3 điểm yếu: format ngắn, độ phân giải, connector.
- **33 BLIP-2 / InstructBLIP**: đối tượng so sánh; kết quả đặt dấu hỏi lên nhu cầu Q-Former và pretrain 129M.
- **32 Flamingo**: IDEFICS (bản tái hiện kiểu Flamingo) bị LLaVA-1.5 vượt dù cầu nối của nó có hàng tỷ tham số.
- **36 InternVL 1.5**: dùng cùng "response formatting prompts as LLaVA 1.5" và phát triển HD thành dynamic tiling 1–12 tile (thử nghiệm tới 40) kèm pixel shuffle.
- **12 RoPE, 15 GQA**: ít liên quan trực tiếp, nhưng token ảnh dài kéo theo bài toán độ dài ngữ cảnh/KV-cache mà các bài đó giải quyết.
- **57 FastV**: cắt tỉa visual token, hướng nhắm vào chính chi phí 576+ token này.

8. Gợi ý áp dụng

Độ phân giải, connector và dữ liệu chỉ dẫn cùng ảnh hưởng chất lượng VLM. Chia ảnh thành ô có thể giữ chi tiết cục bộ nhưng làm tăng số token và chi phí của LLM; ảnh thu nhỏ toàn cục có thể bổ sung bối cảnh bố cục. Mức đánh đổi cần đo trên tác vụ và phân phối ảnh cụ thể.

9. Thí nghiệm gợi ý

So sánh các độ phân giải và số ô trên cùng tập đánh giá; báo cáo số token ảnh, thời gian prefill và chất lượng. Với ảnh chia ô, thử có và không có thumbnail để khảo sát tác động lên các câu hỏi cần bối cảnh toàn ảnh.

10. Tóm tắt công thức / khái niệm

Khái niệm	Nội dung
Connector	$H_v = W_2\sigma(W_1 Z_v)$ (MLP 2 lớp) thay WZ_v
Response format prompt	Câu chỉ định định dạng đầu ra, nối cuối câu hỏi ("single word or phrase", "option's letter"...)

Khái niệm	Nội dung
Dữ liệu	665K mẫu (hỗn hợp: LLaVA 158K, ShareGPT 40K, VQAv2 83K, GQA 72K, OKVQA 9K, OCRVQA 80K, A-OKVQA 66K, TextCaps 22K, RefCOCO 48K, VG 86K); căn chỉnh 558K
Số token	224 px: 256; 336 px: 576 (tự tính, patch 14)
HD	lưới tối đa 6 (tối đa 672x448), mảnh 224, thêm token cuối hàng, bỏ padding, + ảnh thu nhỏ toàn cục
Kết quả	11 SOTA; 13B: VQAv2 80.0, GQA 63.3, MME 1531.3, MMBench 67.7, MM-Vet 36.1; ~1 ngày trên 8 x A100

11. Kết luận ngắn

LLaVA-1.5 là bài "thí nghiệm có kiểm soát" cho thấy một connector MLP đơn giản, dữ liệu học thuật có kèm format prompt, ViT độ phân giải cao hơn và chia lưới là đủ để một mô hình mở, rẻ vượt các mô hình dùng hàng trăm triệu đến hàng tỷ cặp pretrain.

Link gốc: https://arxiv.org/abs/2310.03744

Bài 36

How Far Are We to GPT-4V? Closing the Gap to Commercial Multimodal Models with Open-Source Suites (InternVL 1.5) (2024)

Tác giả: Zhe Chen, Weiyun Wang, Hao Tian, Shenglong Ye và cộng sự (Shanghai AI Laboratory, SenseTime, Tsinghua, Nanjing University, Fudan, CUHK); tác giả liên hệ Wenhai Wang.
Venue / arXiv: arXiv:2404.16821 (v2, 29/04/2024); báo cáo kỹ thuật (text paper không ghi tên hội nghị).
Nguồn: https://arxiv.org/abs/2404.16821
Chủ đề: Dynamic high-resolution (chia ảnh thành 1–12 tile 448x448 + thumbnail, test tới 40 tile), pixel shuffle giảm token ảnh còn 1/4, vision encoder lớn InternViT-6B học liên tục, dữ liệu song ngữ Anh–Trung và đường ống dịch dữ liệu.
Vị trí trong lộ trình: Đọc trước: 29 (ViT), 30 (CLIP), 34–35 (LLaVA, LLaVA-1.5-HD là tiền thân của tiling). Bài này là phiên bản "công nghiệp" của công thức ViT–MLP–LLM, đặc biệt mạnh về OCR/tài liệu. Tiếp theo: 37 (Qwen2-VL: độ phân giải động gốc, không tiling), 38 (SmolVLM: pixel shuffle ở mô hình nhỏ), 41–42 (các mô hình OCR).

> Ghi chú diễn giải, không phải bản dịch. Một số cột bảng trích từ PDF bị lỗi; ghi chú chỉ trích các số đọc rõ.

1. Bối cảnh

Đầu 2024 vẫn còn khoảng cách giữa MLLM mã nguồn mở và thương mại (GPT-4V, Gemini, Qwen-VL-Max). Paper chỉ ra ba nguyên nhân:

1. **Quy mô tham số**: mô hình thương mại thường ≥ 100B; mô hình mở phổ biến dùng vision foundation model (VFM) ~300M ghép với LLM 7B/13B.
2. **Độ phân giải ảnh**: mô hình thương mại dùng độ phân giải *động*, giữ tỉ lệ khung hình; mô hình mở thường train ở độ phân giải cố định (336x336, 448x448), hại ảnh có tỉ lệ lạ và tài liệu.
3. **Đa ngôn ngữ**: mô hình mở chủ yếu dùng dữ liệu tiếng Anh, dựa vào zero-shot của LLM cho ngôn ngữ khác, nên yếu ở OCR và hiểu cảnh phi Anh.

Khái niệm nền:

- *VFM (vision foundation model)*: bộ mã hoá ảnh lớn, đã pretrain (ở đây là ViT).
- *Tiling*: chia ảnh thành các ô cố định, mã hoá từng ô độc lập.
- *OCR benchmark*: DocVQA (hỏi–đáp trên tài liệu), ChartQA (biểu đồ), InfoVQA (infographic), TextVQA (chữ trong ảnh cảnh), OCRBench (tổng hợp).

2. Ý tưởng chính

Giữ khung **ViT–MLP–LLM** (như LLaVA) nhưng nâng cấp ba thứ:

1. **Vision encoder lớn, học liên tục**: InternViT-6B (6 tỷ tham số) được tiếp tục pretrain (continuous learning) lên độ phân giải 448, dữ liệu OCR và caption chất lượng cao, đến mức dùng lại được cho nhiều LLM khác nhau.
2. **Dynamic high-resolution**: chọn tỉ lệ khung hình gần nhất trong 35 tỉ lệ định trước (tạo từ 1–12 tile), resize rồi cắt thành các tile 448x448, kèm 1 thumbnail toàn cục. Lúc test có thể tăng tới 40 tile (khoảng 4K) mà không huấn luyện thêm.
3. **Pixel shuffle** giảm số visual token mỗi tile còn 1/4 (còn **256 token / tile 448x448**), giữ chi phí LLM trong tầm kiểm soát.

Cộng thêm bộ dữ liệu song ngữ lớn (có đường ống dịch bằng LLM) để mạnh cả OCR tiếng Anh và tiếng Trung. Mô hình cuối: InternViT-6B + InternLM2-20B = **26B** tham số, đạt SOTA 8/18 benchmark.

3. Phương pháp chi tiết

3.1 Kiến trúc tổng thể

```
ảnh --> [Dynamic High-Resolution: chọn lưới tile + thumbnail]
    --> mỗi ảnh 448x448 --> [InternViT-6B] --> lưới patch (patch 14: 32x32 = 1024 token)
    ↪  *
    --> [Pixel Shuffle] --> 256 token --> [MLP projector, khởi tạo ngẫu nhiên] -->
    ↪  embedding LLM
    --> nối với text embedding --> [InternLM2-Chat-20B]
```

(* số patch 1024 là tự tính từ patch size 14 của ViT; paper nói: "a 448x448 image is represented by 256 visual tokens" sau pixel shuffle, giảm còn một phần tư của lượng gốc.)

- **ViT**: InternViT-6B, đã pretrain từ InternVL gốc, được nâng cấp (xem 3.2).
- **MLP projector**: khởi tạo ngẫu nhiên.
- **LLM**: InternLM2-20B, dùng bản **chat** chứ không phải bản base.

3.2 Strong Vision Encoder: continuous learning

- **InternViT-6B-448px-V1.2** (bản InternVL 1.2): (i) thấy đặc trưng ở **tầng thứ tư tính từ cuối** là tốt nhất cho đa phương thức, nên **bỏ 3 tầng cuối**, từ 48 tầng còn **45 tầng**; (ii) tăng độ phân giải từ 224 lên **448**; (iii) ghép với Nous-Hermes-2-Yi-34B; đồng thời mở khoá ViT và MLP, huấn luyện trên caption + OCR để có khả năng độ phân giải cao và OCR.
- **InternViT-6B-448px-V1.5**: tiếp tục từ V1.2, chuyển từ 448x448 cố định sang **động**: tile 448x448, 1–12 tile; mở rộng quy mô, chất lượng, đa dạng dữ liệu pretrain, cho độ bền (robustness), OCR và xử lý độ phân giải cao.

Điểm đáng chú ý: dù LLM đổi từ Yi-34B sang InternLM2-20B, InternViT vẫn tương thích và mang đi được, tức đặc trưng thị giác học được là tổng quát và không gắn chặt với một LLM cụ thể (theo paper).

3.3 Dynamic high-resolution

Lấy cảm hứng từ UReader. Các bước:

```
Đầu vào: ảnh HxW; tập tỉ lệ R gồm 35 tỉ lệ (i:j) với 1 <= i*j <= 12
1. r = W/H.  Với mỗi (i:j) trong R, tính |r - i/j|; chọn tỉ lệ có sai khác nhỏ nhất.
   Nếu có nhiều ứng viên (vd 1:1 và 2:2), ưu tiên cái KHÔNG vượt quá 2 lần diện tích ảnh
   (tránh phóng đại quá mức ảnh nhỏ).
2. Resize ảnh về (i*448) x (j*448).   Ví dụ: ảnh 800x1300 -> khớp 2:3 -> resize 896x1344.
3. Cắt thành i*j tile 448x448.
4. Thêm 1 thumbnail của cả ảnh thu về 448x448 (ngữ cảnh toàn cục).
```

- Huấn luyện: tối đa **12 tile** do hạn chế tính toán, nên 35 tỉ lệ ứng với mọi tổ hợp 1–12 tile (1:1, 1:2, 2:1, 3:1, ..., 2:6...).
- Số visual token = 256 x (số tile + 1 thumbnail) (tự tính): lúc train paper ghi số visual token nằm trong khoảng 256 đến 3,328 (tối đa $13 \times 256 = 3328$ gồm 12 tile + thumbnail); test tối đa 40 tile: **10,496** token (41×256).
- Ảnh nhỏ: một tile duy nhất (sử dụng như độ phân giải thấp, rẻ); trang tài liệu: nhiều tile. Paper ví điều này như chế độ "low/high" của GPT-4V.

3.4 Pixel shuffle

Mục đích: giảm 4 lần số token mà vẫn giữ thông tin, bằng cách đổi bớt chiều không gian lấy thêm chiều kênh. Paper chỉ nói "simple pixel shuffle operation to reduce the number of visual tokens to one-quarter". Cơ chế chuẩn (không mô tả chi tiết trong paper, ghi theo cách hiểu thông thường):

$$X \in \mathbb{R}^{H\times W\times C} \xrightarrow{\text{gom khối } 2\times 2} X' \in \mathbb{R}^{\frac{H}{2}\times\frac{W}{2}\times 4C}$$

Tức mỗi khối 2x2 patch kề nhau được nối (concatenate) theo chiều kênh thành một token rộng gấp 4, rồi MLP projector ánh xạ $4C \to d_{\text{LLM}}$. So với mean-pooling, không vứt bỏ thông tin cục bộ ở bước này (mất mát nếu có nằm ở MLP). Ví dụ (tự tính): $32 \times 32 = 1024$ patch $\to 16 \times 16 = 256$ token.

3.5 Hai giai đoạn huấn luyện

Giai đoạn	Huấn luyện	Dữ liệu
Pre-training	InternViT-6B và MLP projector (tối ưu trích xuất đặc trưng thị giác)	caption (53.9%), detection/grounding (5.2%), OCR lớn (32.0%), OCR nhỏ (8.9%)
Fine-tuning	Toàn bộ 26B tham số	caption, general QA, science, chart, math, knowledge, OCR, document, grounding, conversation, text-only

Cả hai giai đoạn dùng context length 4096, và dùng cùng response-format prompt như LLaVA-1.5 (bài 35). Đánh giá chủ yếu bằng VLMEvalKit.

Dữ liệu pretrain (Table 1a, tóm tắt): Captioning: Laion-EN/ZH, COYO (zh), GRIT (zh), COCO, TextCaps. Detection: Objects365, GRIT, All-Seeing. OCR lớn: Wukong-OCR (zh), LaionCOCO-OCR (en), Common Crawl PDF; được tạo bằng **PaddleOCR** chạy trên ảnh Wukong (tiếng Trung) và LAION-COCO (tiếng Anh). OCR nhỏ: MMC-Inst, LSVT, ST-VQA, RCTW-17, ReCTs, ArT, SynthDoG, COCO-Text, ChartQA, CTW, DocVQA, TextOCR, PlotQA, InfoVQA.

Dữ liệu fine-tune (Table 1b): ShareGPT4V, VQAv2, GQA, OKVQA, AI2D, ScienceQA, ChartQA, DVQA, GeoQA+, TabMWP, DocVQA, SynthDoG, LLaVA-150K, ALLaVA, và text-only (OpenHermes2.5, Alpaca-GPT4, ShareGPT, COIG-CQIA) để giữ năng lực ngôn ngữ của LLM.

3.6 Đường ống dịch dữ liệu

Dùng LLM mã nguồn mở (hoặc GPT-3.5) dịch dữ liệu tiếng Anh sang tiếng Trung với prompt hệ thống: giữ tên riêng/thuật ngữ bằng tiếng Anh, dùng thành ngữ bản địa, câu tự nhiên tránh "translationese". Chỉ cần đổi `{language}` là mở rộng sang ngôn ngữ khác. Trong Table 1, "zh" gắn với dataset gốc tiếng Anh nghĩa là đã dịch (ví dụ COYO, GRIT). Đây là nguồn chính của khả năng tiếng Trung.

4. Thí nghiệm và kết quả chính

So sánh với mô hình thương mại và mở (Table 2, chọn các số liên quan OCR/chung, đọc rõ):

Model	Tham số	DocVQA	ChartQA	InfoVQA	TextVQA	OCRBench	MMMU	MathVista
GPT-4V	-	88.4	78.5	-	78.0	645	56.8	49.9
Gemini Pro 1.5	-	86.5	81.3	72.7	73.5	-	58.5	52.1

Model	Tham số	DocVQA	ChartQA	InfoVQA	TextVQA	OCRBench	MMMU	MathVista
Claude-3 Opus	-	89.3	80.8	-	-	694	59.4	50.5
Grok-1.5V	-	85.6	76.1	-	78.1	-	-	52.8
DocOwl-1.5	8B	82.2	70.2	50.7	68.6	599	-	-
LLaVA-NeXT	35B	84.0	68.7	51.5	69.5*	574	51.1	46.5
InternVL 1.2	40B	57.7	68.0	39.5	72.5*	569	51.6	47.7
InternVL 1.5	**26B**	**90.9**	**83.8**	**72.5**	**80.6**	**724**	45.2	**53.5**

(* dùng token Rosetta OCR cho TextVQA.) Các số đọc từ PDF, một số mục (dấu "-") không có trong paper.

Các khẳng định của paper: SOTA trên ChartQA và OCRBench (vượt cả mô hình thương mại), DocVQA 90.9, tốt nhất HallusionBench (49.3), cải thiện lớn tiếng Trung (MMBench-CN 82.0, CCBench 69.8); kết quả tổng: **SOTA 8/18 benchmark**. Bốn benchmark mô hình vượt các mô hình thương mại (Grok-1.5V, GPT-4V, Claude-3 Opus, Gemini Pro 1.5) gồm TextVQA, ChartQA, DocVQA, MathVista (Fig. 1).

Một số số khác (Table 2): MME 2187.8, RealWorldQA 66.0, AI2D 80.7, MMBench-EN 82.2, SEED 76.0, MMVet 62.8.

Điểm yếu thấy trong bảng: MMMU 45.2, thấp hơn InternVL 1.2 (51.6); paper cho là do LLM nhỏ hơn (20B so với 34B). Tương tự MMT-Bench overall: 59.0 (1.5) so với 63.4 (1.2). Hội thoại nhiều lượt (ConvBench) dẫn đầu mô hình mở nhưng vẫn kém GPT-4V.

5. Phân tích / ablation

(1) Larger LLMs need larger VFMs. So sánh InternVL 1.2 (VFM 6B + LLM 34B, ảnh cố định 448) với LLaVA-NeXT-34B (VFM ~300M, độ phân giải động 672x672 cao hơn). Sau khi loại 5 tập OCR, ConvBench và RealWorldQA, InternVL 1.2 hơn LLaVA-NeXT ở **9 trong 11** dataset còn lại. Paper nói so sánh "không hoàn toàn công bằng" nhưng ủng hộ giả thuyết: LLM lớn cần VFM lớn.

(2) Dynamic resolution matters (Fig. 6): đo hiệu năng theo số tile (0 -> 40). Quan sát của paper:

- Dù chỉ train 1–12 tile, test **zero-shot** lên tới 40 tile vẫn chạy tốt.
- Tác vụ **OCR** (DocVQA, InfoVQA, TextVQA, OCRBench) **được lợi từ độ phân giải cao hơn**.
- AI2D, MMMU, MMBench, HallusionBench **hơi giảm** khi độ phân giải cao, tức không phải task nào cũng cần nhiều tile.
- Kết luận: mô hình thích ứng độ phân giải theo task, tiết kiệm tài nguyên khi không cần.
- (MMMU nhiều ảnh mỗi mẫu nên chỉ test tối đa 18 tile do hết bộ nhớ.)

Các giá trị điểm cụ thể theo số tile nằm trong biểu đồ nên ghi chú này không trích (PDF mất thông tin đỉnh/vị trí).

(3) Chất lượng định tính (so với GPT-4V): trả lời tổng quát, OCR tiếng Trung, khoa học, văn hoá truyền thống Trung Quốc, định vị đối tượng, hội thoại nhiều ảnh (dù chỉ train ảnh đơn, vẫn có khả năng zero-shot đa ảnh). Một số ví dụ GPT-4V nhận ra ít chi tiết trong cảnh Trung Quốc hơn.

Lưu ý: paper **không có ablation riêng** cho pixel shuffle (so với pooling hay giữ đủ token), cho thumbnail, hay cho việc bỏ 3 tầng cuối; các lựa chọn đó được nêu là quyết định thiết kế, không phải kết quả ablation.

6. Giới hạn

Do tác giả nêu:

- Điểm MMMU giảm so với InternVL 1.2 (do LLM nhỏ hơn); hội thoại nhiều lượt vẫn thua GPT-4V "với khoảng cách đáng kể"; hạn chế tính toán nên tối đa 12 tile khi huấn luyện; MMMU hết bộ nhớ khi nhiều tile.
- Lĩnh vực vẫn nhiều thách thức; tác giả cam kết tiếp tục cải thiện.

Nhận xét bổ sung:

- **Chi phí lớn**: 6B ViT + 20B LLM, và mỗi trang cao phân giải có thể đến hàng nghìn token; đòi hỏi nhiều bộ nhớ thiết bị. Paper không báo cáo latency, bộ nhớ hay tốc độ suy luận.
- Thiếu ablation cho từng thành phần (pixel shuffle, tile, thumbnail), nên không biết mỗi quyết định đóng góp bao nhiêu; kết quả mạnh có thể một phần nhờ dữ liệu OCR khổng lồ (OCR chiếm 32.0% + 8.9% của dữ liệu pretrain) chứ không chỉ kiến trúc.
- Tính zero-shot lên 40 tile không có bảng số đầy đủ trong văn bản; thông tin chủ yếu nằm trong hình.
- Dữ liệu và một phần nhãn OCR do PaddleOCR sinh ra có nhiễu (nhãn "bạc").

7. Liên hệ với các bài khác

- **35 LLaVA-1.5-HD**: cũng chia ảnh thành lưới + ảnh thu nhỏ toàn cục; nhưng dùng mảnh 224 px, tối đa 6 lưới, không giảm token; InternVL dùng tile 448 px và pixel shuffle, tới 12 (train) / 40 (test) tile.
- **29 ViT / 30 CLIP / 31 SigLIP**: InternViT-6B là ViT rất lớn học bằng huấn luyện liên tục; paper so sánh với dùng CLIP/SigLIP ~300M phổ biến.
- **32–33 Flamingo, BLIP-2**: các hướng giảm token bằng Resampler/Q-Former; InternVL chọn pixel shuffle (cố định, không học) thay vì module học.
- **37 Qwen2-VL**: giải quyết cùng bài toán độ phân giải động bằng cách khác: ViT xử lý độ phân giải gốc với 2D-RoPE, gộp token 2x2, không cần tile.
- **38 SmolVLM**: dùng pixel shuffle cho VLM nhỏ.
- **39–42 Donut, Nougat, GOT-OCR2.0, olmOCR**: các mô hình OCR chuyên dụng; InternVL 1.5 cho thấy VLM tổng quát cũng đạt DocVQA 90.9.
- **57 FastV**: cắt tỉa visual token ở lớp sâu, trực tiếp giảm chi phí do số token lớn của tiling.

8. Gợi ý áp dụng

Tiling động điều chỉnh số ô theo ảnh, còn thumbnail bổ sung góc nhìn toàn cục. Pixel shuffle gộp đặc trưng các patch lân cận để giảm số token đưa vào LLM; hiệu quả tính toán và mức mất chi tiết phụ thuộc cách chiếu và tác vụ.

9. Thí nghiệm gợi ý

So sánh pixel shuffle, pooling và không gộp trên cùng bộ mã hoá và tập kiểm tra. Vẽ chất lượng theo số ô và số token, đồng thời đo thời gian và bộ nhớ; phân tích riêng trang thưa, trang dày và bố cục nhiều cột.

10. Tóm tắt công thức / khái niệm

Khái niệm	Nội dung
Kiến trúc	InternViT-6B (45 tầng sau khi bỏ 3 tầng cuối) -> pixel shuffle -> MLP -> InternLM2-Chat-20B; tổng 26B
Token / tile	ảnh 448x448 -> 256 token (sau pixel shuffle, giảm còn 1/4)
Dynamic tiling	35 tỉ lệ (tạo từ 1–12 tile); chọn gần nhất theo $\lvert r - i/j \rvert$; ưu tiên không vượt 2x diện tích; resize rồi cắt tile 448; + thumbnail 448
Token tối đa	train: 256–3,328 (12 tile + thumbnail); test 40 tile: 10,496

Khái niệm	Nội dung
Huấn luyện	GĐ1: ViT + MLP; GĐ2: toàn bộ 26B; context 4096; response-format prompt của LLaVA-1.5
Dữ liệu	pretrain: caption 53.9%, detection 5.2%, OCR lớn 32.0%, OCR nhỏ 8.9%; dịch Anh -> Trung bằng LLM
Kết quả	DocVQA 90.9, ChartQA 83.8, InfoVQA 72.5, TextVQA 80.6, OCRBench 724; SOTA 8/18 benchmark

11. Kết luận ngắn

InternVL 1.5 cho thấy một mô hình mở (26B) có thể sánh với GPT-4V về OCR/tài liệu nhờ ba thứ: ViT rất lớn được học liên tục, tiling động theo tỉ lệ khung hình (1–12 tile lúc train, 40 lúc test) kèm thumbnail, và dữ liệu OCR song ngữ khổng lồ; pixel shuffle giữ số token ở mức 256/tile.

Link gốc: https://arxiv.org/abs/2404.16821

Bài 37

Qwen2-VL: Enhancing Vision-Language Model's Perception of the World at Any Resolution (2024)

Tác giả: Peng Wang, Shuai Bai, Sinan Tan, Shijie Wang, Zhihao Fan, Jinze Bai và cộng sự (Qwen Team, Alibaba Group).
Venue / arXiv: arXiv:2409.12191 (v2, 3/10/2024); bài không nêu venue hội nghị.
Nguồn: https://arxiv.org/abs/2409.12191
Chủ đề: VLM với ảnh độ phân giải tùy ý (naive dynamic resolution), 2D-RoPE trong ViT, gộp 2×2 token thị giác, M-RoPE (RoPE đa phương thức), thống nhất ảnh và video.
Vị trí trong lộ trình: Đọc trước: 12 (RoPE), 29 (ViT), 31 (SigLIP), 34-35 (LLaVA), 36 (InternVL 1.5). Bài này thêm: bỏ việc ép ảnh về kích thước cố định, số token thị giác thay đổi theo ảnh, và vị trí 3 chiều (thời gian, cao, rộng) cho LLM. Tiếp theo: 38 (SmolVLM, nén token mạnh hơn), 57 (FastV, cắt token thị giác).

> Ghi chú học tập, không phải bản dịch. Các số đánh dấu "tự tính" là ví dụ từ công thức, không phải kết quả trực tiếp của bài báo.

1. Bối cảnh

Khung chung của VLM (vision-language model, mô hình xử lý cả ảnh và chữ): `bộ mã hóa thị giác` → `bộ nối (connector)` → `LLM`. Ảnh đi qua ViT thành một dãy vector, bộ nối đưa chúng về không gian embedding của LLM, rồi LLM đọc dãy "token ảnh + token chữ" và sinh chữ bằng dự đoán token kế tiếp.

Vấn đề paper nêu:

1. **Kích thước ảnh cố định.** Nhiều LVLM (large vision-language model) ép mọi ảnh về một cỡ cố định (ví dụ 224×224) bằng co/giãn, hoặc scale rồi pad. Ảnh độ phân giải cao mất chi tiết; ảnh có tỉ lệ cực đoan bị méo hoặc lãng phí pixel đệm. Với tài liệu, chữ nhỏ là thứ đầu tiên biến mất.
2. **ViT đóng băng kiểu CLIP.** Nhiều mô hình giữ nguyên encoder CLIP, nghi ngờ biểu diễn đó chưa đủ cho suy luận phức tạp và chi tiết nhỏ.
3. **Vị trí 1 chiều.** LLM dùng RoPE 1 chiều. Văn bản vốn 1 chiều, nhưng ảnh là 2 chiều và video là 3 chiều (thời gian + 2 chiều không gian). Ép chúng thành chuỗi 1 chiều làm mất cấu trúc.
4. **Video thường bị coi là một modality riêng,** không dùng chung cơ chế với ảnh.

Kiến thức nền cần nhớ:

- **RoPE** (bài 12): mã hóa vị trí bằng cách xoay từng cặp kênh của query/key một góc tỉ lệ với vị trí. Tích vô hướng $q^\top k$ sau xoay chỉ phụ thuộc **khoảng cách tương đối** giữa hai vị trí.
- **Patch/token ảnh** (bài 29): ViT cắt ảnh thành các ô vuông (patch) rồi coi mỗi ô là một token. Với patch 14 pixel, ảnh $H \times W$ có $(H/14)(W/14)$ token.

---

2. Ý tưởng chính

Qwen2-VL giữ khung `ViT` → `connector` → `LLM (Qwen2)`, nhưng sửa ba chỗ. Thứ nhất, **naive dynamic resolution**: ảnh được đưa vào ở độ phân giải gần gốc, số patch tùy ảnh, nên số token thị giác thay đổi theo ảnh (ảnh to nhiều token, ảnh nhỏ ít token). Để làm được, họ bỏ position embedding tuyệt đối của ViT và thay bằng **2D-RoPE**, rồi dùng một MLP gộp mỗi khối 2×2 token kề nhau thành 1 token trước khi vào LLM. Thứ hai, **M-RoPE**: tách RoPE thành ba thành phần (thời gian, chiều cao, chiều rộng); với văn bản cả ba dùng cùng một ID nên tương đương RoPE 1 chiều, với ảnh thì ID cao/rộng thay đổi theo ô, với video thì ID thời gian

tăng theo khung. Thứ ba, **ảnh và video dùng chung một đường xử lý** (ảnh coi như video hai khung giống nhau, dùng conv 3D độ sâu 2).

---

3. Phương pháp chi tiết

3.1 Các cỡ mô hình

Mô hình	Vision encoder	LLM
Qwen2-VL-2B	675M	1.5B
Qwen2-VL-7B	675M	7.6B
Qwen2-VL-72B	675M	72B

(Bảng 1 của paper.) Điểm đáng chú ý: **ViT luôn 675M** cho mọi cỡ LLM, nên chi phí phần thị giác không đổi khi LLM to ra. LLM khởi tạo từ Qwen2; ViT khởi tạo từ ViT của DFN, nhưng position embedding cố định của DFN bị thay bằng RoPE-2D.

3.2 Naive dynamic resolution

Quy trình cho một ảnh, theo mô tả của paper:

1. Ảnh giữ gần độ phân giải gốc (có hai tham số `min_pixels`, `max_pixels` để kẹp tổng số pixel; ablation dùng $100 \times 28 \times 28$ và $16384 \times 28 \times 28$).
2. ViT (patch 14) biến ảnh thành lưới patch. Vì bỏ position embedding tuyệt đối và dùng 2D-RoPE nên lưới có kích thước bất kỳ vẫn chạy được.
3. Khi suy luận, nhiều ảnh có thể được **đóng gói thành một chuỗi duy nhất**, độ dài gói được giới hạn để khống chế bộ nhớ GPU.
4. Một MLP sau ViT **gộp 2×2 token kề nhau thành 1 token**, rồi bọc bằng `<|vision_start|>` và `<|vision_end|>`.

Paper cho ví dụ: ảnh 224×224, patch 14 → lưới 16×16 = 256 patch → gộp 2×2 còn 64, cộng 2 token đặc biệt = **66 token** (số 66 có trong paper; phép chia 256/4 + 2 là ta kiểm lại).

Như vậy mỗi token vào LLM tương ứng một ô **28×28 pixel** của ảnh. Công thức số token cho ảnh $H \times W$ (sau khi đã resize về bội của 28, tự suy từ mô tả trên):

$$\boxed{N_{\text{vis}} \approx \frac{H}{28} \cdot \frac{W}{28}}$$

2D-RoPE trong ViT (ý tưởng)

Paper chỉ nói "2D-RoPE" mà không cho công thức; cách hiểu thông thường: chia các kênh của mỗi head làm hai nửa, nửa đầu xoay theo chỉ số **hàng**, nửa sau xoay theo chỉ số **cột**. Khi đó điểm attention giữa hai patch phụ thuộc vào độ lệch hàng và độ lệch cột. Đây là cách diễn giải trong ghi chú; cách chia cụ thể không nằm trong paper.

3.3 M-RoPE

RoPE 1 chiều gán cho token thứ n một số nguyên vị trí n, và xoay cặp kênh thứ i một góc $n\theta_i$. M-RoPE gán cho mỗi token **bộ ba** ID (t, h, w) và dành mỗi nhóm kênh cho một thành phần:

$$\text{góc xoay của cặp kênh } i = p_{c(i)}(n) \cdot \theta_i, \qquad c(i) \in \{t, h, w\}$$

trong đó $c(i)$ cho biết cặp kênh i thuộc thành phần nào. (Cách chia số kênh cho ba thành phần không được nêu trong paper.)

Quy tắc gán ID (đúng theo paper):

- **Văn bản:** cả ba ID bằng nhau và tăng dần → M-RoPE **tương đương** RoPE 1 chiều.
- **Ảnh:** ID thời gian t **không đổi** cho mọi token của ảnh; ID h và w theo hàng và cột của token.
- **Video:** t **tăng theo từng khung**; h, w gán như ảnh.
- **Nhiều modality:** ID khởi đầu của modality sau = ID lớn nhất của modality trước + 1.

Ví dụ nhỏ (tự dựng theo quy tắc trên). Câu 3 token văn bản, rồi một ảnh có lưới 2×2 token (sau gộp), rồi 2 token văn bản:

Token	loại	t	h	w
a, b, c	chữ	0, 1, 2	0, 1, 2	0, 1, 2
ảnh (0,0)	ảnh	3	3	3
ảnh (0,1)	ảnh	3	3	4
ảnh (1,0)	ảnh	3	4	3
ảnh (1,1)	ảnh	3	4	4
d, e	chữ	5, 6	5, 6	5, 6

ID lớn nhất của ảnh là 4 nên chữ tiếp theo bắt đầu từ 5. Chú ý: một ảnh 40×56 token chỉ làm vị trí tăng thêm khoảng 56 (số lớn nhất của h hoặc w) thay vì 2240 như khi đánh số tuần tự. Đây là lý do paper nói M-RoPE "giảm giá trị ID vị trí của ảnh/video, giúp ngoại suy lên chuỗi dài hơn".

Pseudo-code gán vị trí:

```
pos = 0; ids = []
for segment in sequence:
    if segment is text of length L:
        for k in range(L): ids.append((pos+k, pos+k, pos+k))
        pos += L
    elif segment is image with grid (Hg, Wg):        # sau khi gộp 2x2
        for r in range(Hg): for c in range(Wg):
            ids.append((pos, pos+r, pos+c))
        pos += max(Hg, Wg)
    elif segment is video with T frame-groups, grid (Hg, Wg):
        for f in range(T): for r in ...: for c in ...:
            ids.append((pos+f, pos+r, pos+c))
        pos += max(T, Hg, Wg)
```

(Dòng cập nhật `pos` với video và ảnh là cách ta diễn giải câu "tăng ID lớn nhất của modality trước thêm 1".)

3.4 Ảnh và video thống nhất

- Lấy mẫu video **2 khung/giây**.
- **Conv 3D độ sâu 2**: xử lý "ống 3D" (2 khung × patch) thay vì patch 2D → xử lý được nhiều khung hơn mà không tăng độ dài chuỗi.
- Mỗi ảnh được coi là **hai khung giống hệt nhau**, để cùng đường xử lý.
- Độ phân giải mỗi khung video được chỉnh động sao cho tổng token của một video **không quá 16384**.

3.5 Huấn luyện (tóm tắt)

Ba giai đoạn theo Qwen-VL:

1. Chỉ huấn luyện ViT, trên lượng lớn cặp ảnh-chữ.
2. Mở khóa toàn bộ tham số, dữ liệu đa dạng hơn.
3. Khóa ViT, chỉ tinh chỉnh LLM bằng dữ liệu chỉ dẫn (định dạng ChatML).

Số liệu: giai đoạn pre-train đầu ~600B token, giai đoạn hai thêm 800B token dữ liệu ảnh; tổng **1.4T token**. Chỉ token chữ được giám sát. Dữ liệu gồm cặp ảnh-chữ, **dữ liệu OCR**, văn bản xen kẽ ảnh, VQA, hội thoại video, tri thức ảnh. Bounding box chuẩn hóa vào $[0, 1000)$ và biểu diễn bằng text với token `<|box_start|>`.

---

4. Thí nghiệm và kết quả chính

Bảng 2 (trích, số đúng như paper):

Benchmark	Qwen2-VL-72B	Qwen2-VL-7B	Qwen2-VL-2B	GPT-4o	Claude-3.5 Sonnet
DocVQA (test)	96.5	94.5	90.1	92.8	95.2
InfoVQA (test)	84.5	76.5	65.5	-	-
ChartQA (test)	88.3	83.0	73.5	85.7	90.8
TextVQA (val)	85.5	84.3	79.7	-	-
OCRBench	877	866	809	736	788
MTVQA	30.9	25.6	18.1	27.8	25.7
MMMU (val)	64.5	54.1	41.1	69.1	68.3
MathVista (testmini)	70.5	58.2	43.0	63.8	67.7

Nhận xét của paper: 72B thường ngang hoặc vượt GPT-4o và Claude 3.5 Sonnet, **lợi thế rõ nhất ở hiểu tài liệu**; MMMU vẫn thua GPT-4o.

Điểm đáng chú ý cho người làm OCR: 2B đã đạt OCRBench 809 và DocVQA 90.1. Paper nhận xét (Hình 6a) rằng với tác vụ liên quan OCR, mô hình nhỏ vẫn khá mạnh, trong khi khả năng toán tăng rõ theo kích thước.

OCR đa ngôn ngữ nội bộ (Bảng 3, Qwen2-VL-72B vs GPT-4o): Hàn 94.5 vs 87.8; Nhật 93.4 vs 88.3; Pháp 94.1 vs 89.7; Đức 91.5 vs 88.3; Ý 89.8 vs 74.1; Nga 97.2 vs 96.8; **Việt 73.0 vs 72.0**; Ả Rập 70.7 vs 75.9. Đây là benchmark nội bộ của Qwen, không công khai.

Video (Bảng 4, 72B): MVBench 73.6, PerceptionTest 68.0, EgoSchema 77.9, Video-MME (không/có phụ đề) 71.2/77.8. Khi đánh giá Video-MME, họ giới hạn tối đa 768 khung mỗi video.

---

5. Phân tích / ablation

5.1 Dynamic resolution vs độ phân giải cố định (Bảng 7, Qwen2-VL-7B)

Chiến lược	Số token ảnh TB	InfoVQA val	RealWorldQA	OCRBench	MMMU
Cố định	64	28.85	56.47	572	53.33
Cố định	576	65.72	65.88	828	52.78
Cố định	1600	74.99	69.54	824	52.89
Cố định	3136	77.27	70.59	786	53.44
Động	**1924**	75.89	70.07	**866**	53.44

Cách đọc: ở các tác vụ cần nhìn kỹ (InfoVQA), điểm tăng mạnh theo số token (28.85 → 77.27), tức **tài liệu rất đói token**. Không có mức cố định nào tốt nhất cho mọi benchmark (OCRBench đạt đỉnh 828 ở 576 token rồi tụt còn 786 ở 3136), còn chiến lược động cho điểm hàng đầu với số token trung bình ít hơn mức 3136. Đây là bằng chứng cho "chọn độ phân giải hợp lý cho từng ảnh quan trọng hơn là cứ tăng".

5.2 Phóng to ảnh nhỏ (min_pixels, Hình 4)

Phóng ảnh nhỏ lên trên ngưỡng `min_pixels` cải thiện các tác vụ thiên về nhận thức (InfoVQA, Hallusion-Bench, OCRBench). Nhưng với OCRBench, `min_pixels` quá cao làm điểm **giảm mạnh**: tác giả giải thích (là giả thuyết) rằng OCRBench có nhiều ảnh rất nhỏ, phóng quá đà biến chúng thành out-of-distribution so với dữ liệu huấn luyện. MMMU gần như không đổi, nên điểm nghẽn của nó là khả năng suy luận chứ không phải độ phân giải.

5.3 M-RoPE vs 1D-RoPE (Bảng 8; Qwen2-1.5B + ViT-L, mô hình pre-train)

	MathVista	MMB	MMStar	RWQ	DocVQA	ChartQA	InfoVQA	TextVQA	Perception Test	NextQA	STAR
1D-RoPE	39.2	58.6	36.7	54.5	82.5	68.0	50.8	71.3	46.6	43.9	55.5
M-RoPE	43.4	60.6	36.7	53.7	82.8	68.4	50.3	71.8	47.4	46.0	57.9

Đọc thẳng: M-RoPE giúp rõ ở MathVista (+4.2) và video (NextQA +2.1, STAR +2.4), còn ở ảnh tài liệu thì gần như hòa (DocVQA +0.3, InfoVQA -0.5, RWQ -0.8). Với **OCR trang đơn thì M-RoPE không phải yếu tố quyết định**.

Ngoại suy độ dài (Hình 5): huấn luyện với tối đa 16384 token mỗi video, nhưng 72B vẫn cho kết quả tốt đến độ dài suy luận **80K token** trên Video-MME (loại video trung bình). Paper cho đây là nhờ M-RoPE (ID vị trí nhỏ hơn). Đây là quan sát, không có ablation tách riêng.

5.4 Scaling (Hình 6)

Hiệu năng tăng đều theo cỡ mô hình, đặc biệt ở toán; OCR thì mô hình nhỏ vẫn mạnh. Theo số token huấn luyện (giai đoạn 2, 7B), kết quả AI2D và InfoVQA tăng đều, còn VQA tổng quát dao động.

---

6. Giới hạn

Tác giả nêu:

- MMMU vẫn thua GPT-4o (suy luận phức tạp còn yếu).
- Điều hướng ngôn ngữ-thị giác (VLN) kém xa mô hình chuyên biệt; họ quy cho việc mô hình tạo bản đồ không đầy đủ từ nhiều ảnh.
- Video dài: hạn chế 768 khung khi đánh giá; "future work" mở rộng chuỗi dài hơn.

Ta nhận xét (không phải của tác giả):

- Paper là báo cáo kỹ thuật: không có ablation riêng cho việc gộp 2×2, cho 2D-RoPE trong ViT, hay cho chia kênh của M-RoPE. Ta không biết mỗi thành phần đóng góp bao nhiêu.
- Số token **không bị chặn trên chặt** theo ảnh: `max_pixels` lớn (16384×28×28 ≈ 12.8M pixel trong ablation) có thể cho tới 16384 token cho một ảnh, kéo theo chi phí attention của ViT (bậc hai theo số patch) và KV cache của LLM. Với GPU nhỏ phải tự đặt trần.
- Các benchmark so với GPT-4o/Claude phụ thuộc cấu hình đánh giá của chính tác giả; OCR đa ngôn ngữ là bộ nội bộ không công khai.

---

7. Liên hệ với các bài khác

- **12 (RoPE):** M-RoPE là RoPE chia thành 3 nhóm kênh; 2D-RoPE trong ViT là phiên bản 2 nhóm.
- **29 (ViT), 31 (SigLIP):** Qwen2-VL bỏ position embedding tuyệt đối của ViT (cố định kích thước lưới) để nhận lưới bất kỳ; xem "NaViT" (trích dẫn là Dehghani et al., 2024) là nguồn ý tưởng packing.
- **34-36 (LLaVA, InternVL 1.5):** InternVL 1.5 chia ảnh thành ô 448×448 cố định (tiling) rồi dùng pixel shuffle; Qwen2-VL đi hướng khác: không chia ô, để ViT nhận ảnh nguyên và gộp 2×2.
- **38 (SmolVLM):** SmolVLM dùng chia ô + pixel shuffle với hệ số 4 (nén 16 lần) và chỉ ra rằng Qwen2-VL-2B tốn 13.7 GB VRAM trong cấu hình của họ, nhiều hơn rất nhiều so với SmolVLM-2.2B (4.9 GB). Đây là minh họa trực tiếp cho cái giá của "nhiều token thị giác".
- **41–42 (GOT, olmOCR):** các mô hình OCR dùng cùng họ ý tưởng (ViT + LLM, ảnh trang độ phân giải cao). olmOCR dựa trên Qwen2-VL (xem bài 42).
- **57 (FastV):** cắt token thị giác ở các lớp LLM sâu; có ích khi số token thị giác lớn như ở đây.
- **46 (PagedAttention):** số token thị giác biến thiên làm KV cache có kích thước biến thiên theo ảnh, đúng bài toán quản lý bộ nhớ động.

---

8. Gợi ý áp dụng

Độ phân giải động làm số token thị giác thay đổi theo ảnh. Ví dụ tự tính: với ảnh 1120×1568 và patch cạnh 14, encoder nhận $80 \times 112 = 8960$ patch; sau gộp 2×2 còn $40 \times 56 = 2240$ token. Đây là số lượng hình học, chưa phải phép đo chi phí thực tế của encoder.

M-RoPE mã hoá các trục thời gian, cao và rộng. Lợi ích của từng trục phụ thuộc tác vụ và thiết lập; các ablation của bài báo nên được xem cùng kết quả trên ảnh và video thay vì suy rộng từ một benchmark.

9. Thí nghiệm gợi ý

Đánh giá chất lượng, số token và thời gian prefill ở nhiều mức độ phân giải trên cùng tập dữ liệu. So sánh giảm độ phân giải với gộp token khi có cùng ngân sách token; hai cách có thể làm mất những loại thông tin khác nhau.

10. Tóm tắt công thức / khái niệm

Khái niệm	Nội dung
Naive dynamic resolution	Ảnh vào ở độ phân giải gần gốc; số token biến thiên theo ảnh
2D-RoPE (ViT)	Thay position embedding tuyệt đối; xoay theo hàng và cột (cách chia kênh: cách diễn giải trong ghi chú)
Gộp 2×2	MLP gộp 4 token kề thành 1; mỗi token LLM ứng với ô 28×28 px
Số token ảnh	$N \approx \frac{H}{28} \cdot \frac{W}{28}$; 224×224 → 66 token (gồm 2 token đặc biệt)
M-RoPE	Bộ ba ID (t, h, w); chữ: ba ID bằng nhau; ảnh: t cố định; video: t tăng theo khung
Ngoại suy	Train ≤16384 token/video, vẫn tốt tới 80K lúc suy luận
Video	2 fps, conv 3D độ sâu 2, ảnh = 2 khung giống nhau, trần 16384 token/video
Ablation chính	Động 1924 token: OCRBench 866 vs cố định 3136 token: 786
Kích thước	ViT 675M cho 2B/7B/72B; LLM 1.5B/7.6B/72B

11. Kết luận ngắn

Qwen2-VL cho thấy cách đơn giản để bỏ ràng buộc "ảnh kích thước cố định": bỏ position embedding tuyệt đối, dùng 2D-RoPE, gộp 2×2, để số token thị giác tự thay đổi theo ảnh. Với tài liệu và OCR, bằng chứng mạnh nhất là Bảng 7: điểm tăng theo số token nhưng không đơn điệu, và độ phân giải động đạt điểm hàng đầu với số token trung bình thấp hơn. M-RoPE chủ yếu có ích cho video và chuỗi dài. Để vận hành trên GPU nhỏ, bài này cũng là lời nhắc rằng số token thị giác (chứ không chỉ số tham số) quyết định bộ nhớ và độ trễ.

Link gốc: https://arxiv.org/abs/2409.12191

Bài 38

SmolVLM: Redefining small and efficient multimodal models (2025)

Tác giả: Andrés Marafioti, Orr Zohar, Miquel Farré, Merve Noyan, Elie Bakouch, Pedro Cuenca, Cyril Zakka, Loubna Ben Allal, Anton Lozhkov, Nouamane Tazi, Vaibhav Srivastav, Joshua Lochner, Hugo Larcher, Mathieu Morlon, Lewis Tunstall, Leandro von Werra, Thomas Wolf (Hugging Face, Stanford).
Venue / arXiv: arXiv:2504.05299 (v1, 7/4/2025); bài không nêu venue.
Nguồn: https://arxiv.org/abs/2504.05299
Chủ đề: Thiết kế VLM nhỏ (256M-2.2B): cân bằng encoder/LM, kéo dài ngữ cảnh, pixel shuffle, ngân sách token thị giác, token vị trí học được, công thức dữ liệu.
Vị trí trong lộ trình: Đọc trước: 29 (ViT), 31 (SigLIP), 36 (InternVL 1.5, pixel shuffle), 37 (Qwen2-VL). Bài này thêm: các "finding" thực nghiệm cho VLM nhỏ, và con số bộ nhớ thực tế. Tiếp theo: 41–42 (các mô hình OCR), 57 (FastV, cắt token).

> Ghi chú học tập, không phải bản dịch. Số "tự tính" không có trong paper. Paper phần lớn trình bày kết quả bằng hình (Hình 3, 5, 6, 7) mà bản trích văn bản không đọc được giá trị; ghi chú chỉ dùng những số có trong chữ hoặc bảng.

1. Bối cảnh

Các VLM nhỏ có thể vẫn tiêu tốn nhiều bộ nhớ nếu chỉ thu nhỏ LLM mà giữ encoder thị giác lớn và nhiều token ảnh. Trong phép đo của bài báo, Qwen2-VL-2B cần 13.7 GB VRAM, InternVL2-2B cần 10.5 GB, còn SmolVLM-2.2B cần 4.9 GB. Các tác giả vì vậy nhấn mạnh rằng số tham số không đủ để dự báo chi phí chạy VLM; cần đo cả bộ nhớ sử dụng.

Thuật ngữ cần biết:

- **Token thị giác:** vector đầu ra của ViT được xem như "từ" đưa vào LLM. Số token càng nhiều, ngữ cảnh LLM càng dài, chi phí prefill (bậc hai với attention đầy đủ) và KV cache càng lớn.
- **Pixel shuffle (space-to-depth):** phép sắp xếp lại, gộp một khối $r \times r$ vị trí không gian thành một vị trí với số kênh nhân r^2.
- **SigLIP (bài 31):** họ encoder ảnh dùng loss sigmoid; SmolVLM dùng SigLIP-B/16 (93M) hoặc SigLIP-SO400M (428M, paper ghi 400M ở một chỗ).
- **SmolLM2:** họ LLM nhỏ của Hugging Face (135M, 360M, 1.7B).

---

2. Ý tưởng chính

Paper là một chuỗi **thực nghiệm có hệ thống về thiết kế VLM nhỏ**, rút ra chín "finding". Ý tưởng cốt lõi: khi LLM nhỏ, nên (i) dùng encoder thị giác nhỏ để cân bằng, (ii) nén token thị giác **mạnh hơn** các VLM lớn (pixel shuffle hệ số 4 thay vì 2), đồng thời (iii) kéo dài ngữ cảnh LLM để chứa ảnh độ phân giải cao, (iv) chia ảnh lớn thành ô con kèm ảnh thu nhỏ, và (v) tinh chỉnh công thức dữ liệu và prompt cho phù hợp với dung lượng nhỏ. Kết quả là ba mô hình: 256M, 500M, 2.2B, trong đó mô hình 256M chạy dưới 1 GB VRAM.

---

3. Phương pháp chi tiết

3.1 Kiến trúc (Hình 2)

```
ảnh -> chia ô con (+ ảnh thu nhỏ) -> vision encoder (SigLIP)
    -> pixel shuffle -> linear/MLP projection -> token thị giác
    -> nối/xen kẽ với embedding chữ -> SmolLM2 -> chữ ra
```

Kiểu ghép: **self-attention architecture**, tức token thị giác được nối thẳng vào chuỗi cùng token chữ rồi LLM xử lý chung (khác cross-attention của Flamingo, bài 32). Video: lấy mẫu khung rồi mã hóa như ảnh.

Ba biến thể:

Mô hình	Encoder	LM	Ghi chú của paper
SmolVLM-256M	SigLIP-B/16 93M	SmolLM2-135M	< 1 GB VRAM
SmolVLM-500M	SigLIP-B/16 93M	SmolLM2-360M	cân bằng
SmolVLM-2.2B	SigLIP-SO400M	SmolLM2-1.7B	mạnh nhất

3.2 Cân bằng encoder - LM (mục 2.1, Finding 1)

Họ ghép ba SmolLM2 (135M, 360M, 1.7B) với hai SigLIP (93M, 428M), tức 6 cấu hình. Kết luận trong chữ:

- LM 135M + encoder lớn: hiệu năng **giảm đáng kể**, cân bằng kém.
- LM 360M: encoder lớn tăng điểm **11.6%** nhưng tăng **66% tham số** → chọn encoder nhỏ.
- LM 1.7B: encoder lớn chỉ làm tăng tham số ~10% → đáng dùng.

Finding 1: mô hình đa phương thức gọn nên cân bằng phân bổ tham số giữa encoder và LM, ưu tiên encoder nhỏ khi LM nhỏ.

3.3 Ngữ cảnh dài (mục 2.2, Finding 2)

Vì ghép bằng self-attention, ảnh tiêu tốn nhiều vị trí: một ảnh 512×512 qua SigLIP-B/16 cho $(512/16)^2 = 32^2 = 1024$ token (số có trong paper). SmolLM2 gốc chỉ có ngữ cảnh 2k. Cách xử lý:

- Tăng **RoPE base từ 10k lên 273k**.
- Tinh chỉnh trên hỗn hợp dữ liệu dài (Dolma books, The Stack) và ngắn (FineWeb-Edu, DCLM, toán của SmolLM2).
- Kết quả: LM 1.7B ổn định ở 16k; LM nhỏ (135M, 360M) khó vượt 8k. Chọn **16k cho 2.2B, 8k cho hai mô hình nhỏ**.

3.4 Pixel shuffle (Hình 4, Finding 3)

Đầu vào là bản đồ đặc trưng $X \in \mathbb{R}^{H\times W\times C}$. Pixel shuffle với hệ số r:

$$\boxed{\mathbb{R}^{H\times W\times C} \longrightarrow \mathbb{R}^{(H/r)\times(W/r)\times(Cr^2)}}$$

rồi một lớp tuyến tính/MLP chiếu Cr^2 chiều về chiều của LM. Số token giảm r^2 lần:

$$N_{\text{sau}} = \frac{N_{\text{trước}}}{r^2}.$$

Pseudo-code PyTorch:

```
def pixel_shuffle(x, r):            # x: [B, H, W, C]  (H, W chia hết cho r)
    B, H, W, C = x.shape
    x = x.view(B, H // r, r, W // r, r, C)
    x = x.permute(0, 1, 3, 2, 4, 5)          # [B, H/r, W/r, r, r, C]
```

```
    return x.reshape(B, H // r, W // r, C * r * r)

tokens = proj(pixel_shuffle(vit_out, r=4)).flatten(1, 2)   # [B, N/16, d_lm]
```

Ví dụ số (tự tính từ số liệu paper): ảnh 512×512, SigLIP-B/16 → 1024 token. Với $r = 2$ còn $1024/4 = 256$ token; với $r = 4$ còn $1024/16 = 64$ token.

Tác giả lưu ý: tỉ lệ cao làm một token phải mô tả vùng không gian lớn hơn nên hại cho tác vụ cần định vị chính xác như OCR. InternVL và Idefics3 dùng $r = 2$. Nhưng thực nghiệm của họ (Hình 3, phải) cho thấy **VLM nhỏ lại hưởng lợi từ** $r = 4$, vì ít token hơn làm nhẹ attention và dễ mô hình hóa ngữ cảnh dài. Paper cũng ghi (chú thích Hình 3) rằng hệ số tối ưu thay đổi theo cỡ mô hình (PS=2 vs PS=4).

Finding 3: VLM nhỏ hưởng lợi từ nén token thị giác mạnh hơn.

3.5 Ảnh và video (mục 2.3, Finding 4)

- **Ảnh độ phân giải cao:** chia thành nhiều ảnh con + một ảnh thu nhỏ của toàn bộ (kiểu UReader, SPHINX). Hiệu quả cho tác vụ ảnh.
- **Video:** thử "frame averaging" (trung bình nhiều khung thành một) nhưng làm tệ đi, và càng tệ khi hệ số (2, 4, 8) tăng. Bỏ; thay vào đó chỉ rescale khung về độ phân giải của encoder.

Khi đánh giá, cạnh dài của ảnh được resize về **1920** (256M, 500M) hoặc **1536** (2.2B).

3.6 Token vị trí học được (mục 3.1, Finding 5)

Cần báo cho LM biết ô con nào (hàng, cột). Cách ban đầu: chuỗi chữ như `<row_1_col_2>` — gây ra hiện tượng họ gọi là **"OCR loss plague"**: loss đột ngột tụt nhưng chất lượng OCR không tốt lên (Hình 5 trái). Sửa bằng **token vị trí đặc biệt** có embedding riêng; mô hình lớn khá bền với chuỗi chữ, mô hình nhỏ hưởng lợi rõ (OCR tốt hơn, tổng quát hóa tốt hơn).

3.7 Prompt có cấu trúc (mục 3.2, Finding 6)

- **System prompt** ngắn (ví dụ "You are a visual agent and should provide concise answers"): cải thiện, nhất là ảnh.
- **Token mở/đóng media** ("Here is an image...", "Here are N frames sampled from a video..."; xen kẽ với "Given this image/video..."): giúp mạnh ở video, cũng giúp ảnh.
- **Che prompt người dùng** khi SFT (chỉ tính loss trên phần trả lời): giảm overfit vì câu hỏi QA đa phương thức lặp lại và dễ bị ghi nhớ.

3.8 Công thức dữ liệu (mục 3.3-3.5, 4.1)

- **Finding 7:** thêm dữ liệu văn bản SFT của LLM (SmolTalk) **làm tệ đi** mô hình nhỏ: giảm tới 3.7% ở video và 6.5% ở ảnh. Giữ tỉ lệ text đúng **14%**, dùng dữ liệu text khác.
- **Finding 8:** dữ liệu chain-of-thought chỉ nên rất ít (0.02-0.05%); nhiều hơn làm giảm điểm, đặc biệt ảnh.
- **Finding 9:** tăng độ dài video trung bình khi huấn luyện đến ~3.5 phút thì cải thiện cả video lẫn ảnh, vượt 3.5 phút lợi ích giảm dần.
- Huấn luyện hai giai đoạn: giai đoạn ảnh, rồi giai đoạn video (14% text, 33% video). Giai đoạn ảnh có tài liệu, OCR, biểu đồ, bảng, và bổ sung MathWriting.

---

4. Thí nghiệm và kết quả chính

Bảng 1 (số đúng như paper):

Benchmark	256M	500M	2.2B	Mô hình hiệu quả khác (cột cuối của paper)
OCRBench	52.6%	61.0%	72.9%	54.7% (MolmoE-A1B-7B)
AI2D	46.4%	59.2%	70.0%	71.0% (MolmoE)
ChartQA	55.6%	62.8%	68.7%	48.0% (MolmoE)
TextVQA	50.2%	60.2%	73.0%	61.5% (MolmoE)
DocVQA	58.3%	70.5%	80.0%	77.7% (MolmoE)
ScienceQA	73.8%	80.0%	89.6%	87.5% (MolmoE)
MMMU	29.0%	33.7%	42.0%	33.9% (MolmoE)
MathVista	35.9%	40.1%	51.5%	37.6% (MolmoE)
MMStar	34.6%	38.3%	46.0%	43.1% (MolmoE)
Video-MME	33.7%	42.2%	52.1%	45.0% (InternVL2-2B)
Trung bình	**44.0%**	**51.0%**	**59.8%**	-
RAM, batch 1	**0.8 GB**	**1.2 GB**	**4.9 GB**	27.7 GB (MolmoE)
RAM, batch 64	15.0 GB	16.0 GB	49.9 GB	-

(Bảng còn MLVU, MVBench, WorldSense, TempCompass; bỏ bớt cho gọn. Cột "mô hình hiệu quả khác" lấy MolmoE-A1B-7B cho tác vụ ảnh và InternVL2-2B hoặc Qwen2VL-7B cho video, theo ghi chú trong paper.)

Điểm đáng chú ý:

- Abstract: SmolVLM-256M dùng < 1 GB GPU và vượt Idefics-80B (to hơn ~300 lần).
- Ngoại lệ ở mô hình 256M so với Idefics 80B: MMMU (29.0% vs 42.3%) và AI2D (46.4% vs 56.3%), tức tác vụ cần LLM suy luận mạnh.
- Paper nhận xét OCRBench cũng hưởng lợi rõ khi tăng LM (52.6% → 61.0% từ 256M lên 500M, hai mô hình dùng **cùng** encoder 93M). Tức **chất lượng OCR phụ thuộc LM chứ không chỉ encoder**.

Thông lượng (mục 4.4):

- A100: 256M từ 0.8 ví dụ/giây (batch 1) lên 16.3 ví dụ/giây (batch 64); 500M 0.7 → 9.9; 2.2B 0.6 → 1.7.
- L4: 256M đạt đỉnh 2.7 ví dụ/giây ở batch 8, sau đó giảm do giới hạn bộ nhớ.
- Trình duyệt (WebGPU): 256M đạt tới 80 token giải mã/giây trên MacBook Pro 14 inch (M4 Max).
- Ứng dụng: ColSmolVLM (truy hồi tài liệu) và SmolDocling (256M, chuyển đổi tài liệu end-to-end bằng DocTags).

---

5. Phân tích / ablation (9 findings)

#	Finding	Ghi chú
1	Cân bằng encoder-LM; encoder nhỏ cho LM nhỏ	LM 360M: encoder lớn +11.6% điểm, +66% tham số
2	Ngữ cảnh dài giúp VLM nhỏ	RoPE base 10k → 273k; 16k (2.2B), 8k (nhỏ)
3	Nén token mạnh hơn cho VLM nhỏ	$r = 4$ thay vì $r = 2$
4	Chia ô con tốt cho ảnh; frame averaging hại video	
5	Token vị trí học được > chuỗi chữ	tránh "OCR loss plague"
6	System prompt + token mở/đóng media; che prompt	

#	Finding	Ghi chú
7	Không tái sử dụng text SFT của LLM	-3.7% video, -6.5% ảnh
8	Quá nhiều CoT hại	tối ưu ở 0.02-0.05%
9	Video dài vừa phải (~3.5 phút)	lợi suy giảm sau đó

Cách đọc có phê phán: hầu hết các finding được chứng minh bằng đường cong trong hình (kết quả tuyệt đối khó đọc từ bản trích văn bản), trên điểm trung bình OpenCompass/CIDEr/VQA, và không có khoảng tin cậy. Ví dụ Hình 6 vẽ "violin" ba checkpoint cho mỗi cấu hình. Đáng tin ở xu hướng, kém tin ở độ lớn chính xác.

---

6. Giới hạn

Tác giả nêu (qua nội dung): hiệu năng ở tác vụ cần suy luận ngôn ngữ mạnh (MMMU, AI2D) vẫn thua mô hình lớn; LM nhỏ khó huấn luyện ổn định ở ngữ cảnh > 8k; video dài có lợi ích giảm dần.

Nhận xét:

- Thiết kế được tinh chỉnh cho bộ benchmark chung (OpenCompass), **không phải cho OCR tài liệu dài**. Kết luận "r=4 tốt hơn" có thể đảo ngược với trang chữ dày: chính paper cảnh báo nén mạnh hại OCR.
- Thông số "bộ nhớ" đo trong khung suy luận của họ (VLMEvalKit, ảnh resize cạnh dài 1920/1536); chạy trong stack khác (vLLM, ONNX, quantize) sẽ ra con số khác.
- Ablation ghép nhiều yếu tố cùng lúc (dữ liệu, prompt, kiến trúc) nên khó tách nhân quả.
- Thông lượng ở batch 1 (0.6-0.8 ví dụ/giây trên A100) cho thấy **batch 1 chưa nhanh**: lợi thế lớn nằm ở bộ nhớ và ở batch lớn.

---

7. Liên hệ với các bài khác

- **36 (InternVL 1.5):** nguồn của ý tưởng pixel shuffle trong VLM; SmolVLM đẩy hệ số lên 4 cho mô hình nhỏ.
- **37 (Qwen2-VL):** đối chiếu hai triết lý. Qwen2-VL giữ ảnh gần nguyên độ phân giải, gộp 2×2 (mỗi token = 28×28 px); SmolVLM chia ô cố định rồi nén 4×4 (mỗi token ứng với vùng 64×64 px nếu patch 16: 16×4). Cả hai dùng self-attention (nối token vào chuỗi).
- **31 (SigLIP), 29 (ViT):** encoder; SmolVLM chỉ ra tỉ lệ kích thước encoder/LM mới là biến quan trọng.
- **12 (RoPE):** việc tăng RoPE base (10k → 273k) là cách kéo dài ngữ cảnh đơn giản.
- **41–42 (GOT, olmOCR):** SmolDocling là phiên bản SmolVLM-256M cho tài liệu, cùng thuộc nhóm bài toán này.
- **57 (FastV), 56 (H2O):** giảm token ở trong LLM, bổ sung cho nén token trước LLM kiểu pixel shuffle.

---

8. Tóm tắt công thức / khái niệm

Khái niệm	Nội dung
Pixel shuffle	$\mathbb{R}^{H\times W\times C} \to \mathbb{R}^{(H/r)\times(W/r)\times Cr^2}$, token giảm r^2 lần
Token từ ViT	Ảnh 512×512, patch 16 → 1024 token; $r=2$: 256; $r=4$: 64
Ngữ cảnh	RoPE base 10k → 273k; 16k (2.2B), 8k (256M, 500M)
Cân bằng	LM 360M + encoder lớn: +11.6% điểm, +66% tham số

Khái niệm	Nội dung
Cấu hình	256M = SigLIP 93M + 135M; 500M = 93M + 360M; 2.2B = SO400M + 1.7B
Bộ nhớ batch 1	0.8 / 1.2 / 4.9 GB
Điểm TB	44.0 / 51.0 / 59.8
Text SFT tái dùng	-3.7% (video), -6.5% (ảnh)
CoT	tối ưu 0.02-0.05%
Tỉ lệ text khi train	14%

9. Kết luận ngắn

SmolVLM là công thức thực nghiệm cho VLM nhỏ: encoder nhỏ khi LM nhỏ, nén token thị giác mạnh, ngữ cảnh dài, token vị trí học được, dữ liệu chọn lọc. Điểm giá trị nhất cho người triển khai là con số bộ nhớ (0.8 GB cho 256M, 4.9 GB cho 2.2B so với 13.7 GB của Qwen2-VL-2B trong cùng khung đo). Hạn chế: tối ưu cho benchmark chung chứ không phải OCR dày chữ, và phần lớn bằng chứng nằm trong đường cong không có sai số.

Link gốc: https://arxiv.org/abs/2504.05299

Bài 39

OCR-free Document Understanding Transformer (2022)

Tác giả: Geewook Kim, Teakgyu Hong, Moonbin Yim, Jeongyeon Nam, Jinyoung Park, Jinyeong Yim, Wonseok Hwang, Sangdoo Yun, Dongyoon Han, Seunghyun Park (NAVER CLOVA và đồng nghiệp).
Venue / arXiv: arXiv:2111.15664; ECCV 2022 (theo ghi chú trên trang arXiv, bản v5).
Nguồn: https://arxiv.org/abs/2111.15664
Chủ đề: Hiểu tài liệu ảnh không dùng OCR ngoài: encoder Swin + decoder BART sinh trực tiếp chuỗi token (JSON), tiền huấn luyện bằng "đọc chữ", dữ liệu tổng hợp SynthDoG.
Vị trí trong lộ trình: Đọc trước: 01 (Transformer, encoder-decoder), 29 (ViT). Bài này thêm: mẫu thiết kế "ảnh vào, chuỗi có cấu trúc ra", nền cho OCR end-to-end. Tiếp theo: 40 (Nougat, kế thừa Donut), 41 (GOT-OCR2.0), 42 (olmOCR).

> Ghi chú học tập, không phải bản dịch. Số "tự tính" không có trong paper (ví dụ số token của encoder, vì paper không nêu chi tiết lưới đặc trưng).

1. Bối cảnh

Hiểu tài liệu thị giác (Visual Document Understanding, VDU) gồm các bài toán xử lý ảnh hóa đơn, biên lai, danh thiếp hay biểu mẫu. Những tác vụ thường gặp là phân loại tài liệu, trích xuất thông tin có cấu trúc (information extraction, IE) và hỏi đáp trên tài liệu (DocVQA).

Cách làm truyền thống: hai giai đoạn. Một chương trình OCR đọc chữ (phát hiện vùng chữ, nhận dạng ký tự, trả hộp tọa độ + text); rồi một mô hình ngôn ngữ/bố cục (BERT, LayoutLM, ...) làm phần hiểu trên đầu ra OCR (Hình 1 của paper).

Ba vấn đề tác giả nêu:

1. **Đắt:** OCR chất lượng cao tốn tính toán; huấn luyện OCR riêng cần dữ liệu lớn.
2. **Kém linh hoạt:** OCR có sẵn khó thích ứng ngôn ngữ/miền mới.
3. **Lan truyền lỗi:** lỗi OCR đi vào mô hình sau, nặng hơn với tiếng Hàn, Trung (bộ ký tự phức tạp). Thêm bước sửa lỗi sau OCR làm hệ thống phình ra.

Khái niệm nền:

- **Encoder-decoder transformer:** encoder biến đầu vào thành chuỗi vector; decoder sinh từng token, ở mỗi bước dùng **cross-attention** để "nhìn" các vector của encoder.
- **Teacher forcing:** lúc huấn luyện, đưa chuỗi đáp án đúng (dịch một vị trí) làm đầu vào decoder thay vì đầu ra của chính nó.
- **Swin Transformer:** ViT phân cấp, attention trong cửa sổ cố định, cửa sổ dịch giữa các lớp, và gộp patch giữa các stage để giảm số token.

---

2. Ý tưởng chính

Donut bỏ hẳn OCR: một encoder Transformer (Swin) đọc thẳng ảnh tài liệu, một decoder Transformer (BART) sinh ra chuỗi token. Mọi tác vụ đều quy về "sinh một chuỗi", và chuỗi đó **đảo ngược một-một thành JSON** nhờ cặp thẻ `[START_x]...[END_x]`. Tiền huấn luyện chỉ là một việc đơn giản: "đọc hết chữ trong ảnh theo thứ tự đọc" bằng cross-entropy dự đoán token kế tiếp (pseudo-OCR). Sau đó fine-tune để hiểu tài liệu cho từng tác vụ, bằng cách đổi prompt. Vì mục tiêu tiền huấn luyện chỉ là đọc chữ, có thể dùng **dữ liệu tổng hợp** (SynthDoG) để mở sang ngôn ngữ khác.

---

3. Phương pháp chi tiết

3.1 Kiến trúc (Hình 3)

Encoder. Ảnh $x \in \mathbb{R}^{H \times W \times C}$ được biến thành tập embedding $\{z_i \in \mathbb{R}^d\}_{i=1}^n$, với n là số patch/kích thước bản đồ đặc trưng, d là chiều ẩn. Họ chọn **Swin-B** (so sánh trong nghiên cứu sơ bộ: tốt nhất cho document parsing), với "chỉnh sửa nhẹ": số lớp mỗi stage $\{2, 2, 14, 2\}$, cửa sổ 10.

Decoder. Sinh chuỗi $(y_i)_{i=1}^m$, với $y_i \in \mathbb{R}^v$ one-hot trên từ vựng cỡ v. Dùng **BART**, khởi tạo từ BART đa ngôn ngữ (`asian-bart-ecjk`), nhưng chỉ lấy **4 lớp đầu** của BART để cân bằng tốc độ-độ chính xác.

Prompt. Giống GPT-3: lúc suy luận đưa một prompt (token đặc biệt cho từng tác vụ) rồi decoder sinh tiếp. Ví dụ: `<classification>`, `<parsing>`, hoặc `<vqa><question>...</question><answer>`; với DocVQA, câu hỏi được đặt làm prompt khởi đầu.

Chuyển đầu ra thành JSON. Với mỗi trường cần trích, thêm cặp token `[START_name]`/`[END_name]`. Nếu chuỗi sai cấu trúc (thiếu thẻ đóng), trường đó coi như mất. Cài bằng regex đơn giản. Ví dụ phân loại: `[START class][memo][END class]` ⇔ `{"class": "memo"}`; lớp "memo" có thể là một token đặc biệt riêng.

3.2 Huấn luyện

Pre-training (pseudo-OCR). Tối thiểu hóa cross-entropy dự đoán token kế tiếp, điều kiện hóa trên ảnh và các token trước (đọc theo thứ tự trên-xuống, trái-phải):

$$\boxed{\mathcal{L}_{\text{pre}} = -\sum_{i=1}^{m} \log p_\theta(y_i \mid y_{<i},\ x)}$$

(công thức viết lại theo mô tả trong paper; paper mô tả mục tiêu bằng lời.)

Dữ liệu:

- **IIT-CDIP:** 11 triệu ảnh tài liệu quét tiếng Anh; nhãn chữ giả (pseudo-label) lấy từ API CLOVA OCR thương mại.
- **SynthDoG:** bộ sinh tài liệu tổng hợp. Họ sinh **0.5M mẫu mỗi ngôn ngữ** cho Trung, Nhật, Hàn, Anh (tổng 2M) từ Wikipedia. Mẫu gồm nền (lấy từ ImageNet), kết cấu giấy (ảnh giấy thu thập), chữ (từ Wikipedia), và bố cục (thuật toán luật xếp lưới ngẫu nhiên), cộng các kỹ thuật render để giống thật.
- Mô hình đa ngôn ngữ huấn luyện trên 2M tổng hợp + 11M IIT-CDIP.

Fine-tuning. Mọi tác vụ coi là dự đoán JSON; chuỗi đích được xây theo mẫu `[START_*]...[END_*]`.

Thiết lập (mục 3.2 và phụ lục A.5): 200K bước, 64 GPU A100, mini-batch 196, Adam, lr khởi tạo 10^{-5} đến 10^{-4} (pre-train 10^{-4}), fp16, gradient clipping 0.05-1.0; mất khoảng 2-3 ngày GPU (theo phụ lục, đúng chữ "2-3 GPU days"). **Độ phân giải đầu vào 2560×1920**, độ dài tối đa decoder **1536**. Một số tác vụ hạ xuống 960×1280 (Train Ticket, Business Card). Tốc độ đo trên GPU P40.

Pseudo-code:

```
# pretrain
for (image, text_tokens) in IIT_CDIP + SynthDoG:
    z = SwinB(image)                        # [n, d]
    logits = BART4(decoder_input = [<s>] + text_tokens[:-1], enc = z)
    loss = CE(logits, text_tokens)          # teacher forcing

# finetune (tác vụ T, vd. parsing): target = json_to_tokens(gt)
```

```
# inference
tokens = greedy_or_beam(BART4, prompt=<T>, enc=SwinB(image))
json   = regex_parse(tokens)              # thiếu [END_x] -> trường x coi như mất
```

(Phần inference là cách mô tả của ta; paper nêu rằng sinh theo prompt, không nêu rõ chiến lược giải mã trong đoạn đã đọc.)

---

4. Thí nghiệm và kết quả chính

4.1 Tác vụ, dữ liệu và độ đo

Tác vụ	Dataset	Độ đo
Phân loại	RVL-CDIP (400K ảnh, 16 lớp; 320K/40K/40K)	accuracy
IE	CORD (receipt Latin, 30 trường; 0.8K/0.1K/0.1K), Ticket (vé tàu Trung Quốc, 1.5K train/0.4K test, 8 trường), Business Card (nội bộ, tiếng Nhật, 20K train, 11 trường), Receipt (nội bộ, tiếng Hàn, 40K train, 81 trường)	F1 mức trường, độ chính xác dựa trên Tree Edit Distance (TED)
DocVQA	50K câu hỏi / 12K tài liệu	ANLS

TED-based accuracy: $\max\left(0,\ 1-\text{TED}(pr, gt)/\text{TED}(\varnothing, gt)\right)$, đo cả cấu trúc cây. F1 mức trường khắt khe (sai một ký tự là trượt), không tính trùng một phần và không đo cấu trúc. ANLS dựa trên khoảng cách Levenshtein chuẩn hóa.

4.2 Kết quả

Phân loại RVL-CDIP (Bảng 1):

Mô hình	OCR	#Params	Thời gian (ms)	Accuracy
BERT	có	110M + α	1392	89.81
LayoutLM	có	113M + α	1396	91.78
LayoutLM (w/ image)	có	160M + α	1426	94.42
LayoutLMv2	có	200M + α	1489	95.25
Donut	không	**143M**	**752**	**95.30**

("+α" là tham số của engine OCR, thường > 80M theo paper.)

IE (Bảng 2, trích; F1 / accuracy-TED / thời gian giây):

Mô hình	CORD	Ticket	Business Card	Receipt
LayoutLMv2	78.9 / 82.4 (1.7 s)	87.2 / 90.1 (1.8 s)	52.2 / 83.0 (1.6 s)	72.9 / 78.0 (2.6 s)
BERT	73.0 / 65.5 (1.6 s)	74.3 / 82.4 (1.7 s)	40.8 / 72.1 (1.5 s)	70.3 / 54.1 (2.5 s)
Donut	**84.1 / 90.9 (1.2 s)**	**94.1 / 98.7 (0.6 s)**	**57.8 / 84.4 (1.4 s)**	**78.6 / 88.6 (1.9 s)**

(Bảng gốc còn BROS, LayoutLM, SPADE, WYVERN; hàng ở đây đọc trực tiếp từ bản trích văn bản, các cột có vẻ thẳng hàng nhưng bảng gốc là PDF nên nên đối chiếu bản gốc khi dùng số cho báo cáo.)

DocVQA (Bảng 3):

Mô hình	OCR	#Params	Thời gian (ms)	ANLS test	ANLS chữ viết tay
BERT	có	110M + α	1517	63.5	n/a
LayoutLM	có	113M + α	1519	69.8	n/a
LayoutLMv2	có	200M + α	1610	78.1	n/a
Donut	không	176M	782	67.5	**72.1**
LayoutLMv2-Large-QG (train+dev+QG)	có	390M + α	1698	86.7	67.3

Cách đọc: trên DocVQA Donut **thua** LayoutLMv2 (67.5 vs 78.1), chỉ "cạnh tranh". Nó chỉ nổi bật ở chữ viết tay (72.1 vs 67.3 của LayoutLMv2-Large-QG).

Cỡ mô hình & tốc độ: Hình 2 của paper tóm tắt trên IE: Donut vừa nhanh hơn vừa chính xác hơn so với pipeline OCR + LayoutLMv2 (bộ nhớ và thời gian thấp hơn); độ chính xác tuyệt đối đã nêu ở Bảng 2.

---

5. Phân tích / ablation

5.1 Chọn nhiệm vụ tiền huấn luyện (Hình 7a)

So sánh: không pre-train, phân loại ảnh, image captioning, và đọc chữ. Kết luận trong chữ: **đọc chữ (text reading)** là hiệu quả nhất; các tác vụ kiểu captioning (cung cấp kiến thức chung) ít hiệu quả cho tinh chỉnh VDU. Với đọc chữ, so SynthDoG-only, IIT-CDIP-only, cả hai: tổng hợp **đủ cho IE**, nhưng với **DocVQA thì cần ảnh thật** (có thể vì phân phối ảnh IIT-CDIP gần DocVQA). Giá trị số trên biểu đồ không đọc được từ bản trích.

5.2 Backbone (Hình 7b)

Thử ResNet-152, EfficientNetV2, ViT-B, Swin-B (hai biến thể). **EfficientNetV2 và Swin** tốt nhất; chọn Swin vì khả năng mở rộng của kiến trúc Transformer và điểm cao hơn.

5.3 Độ phân giải đầu vào (Hình 7c)

Hiệu năng **tăng nhanh khi tăng độ phân giải**, rõ nhất ở DocVQA (ảnh lớn, chữ rất nhỏ). Đánh đổi là chi phí tính toán. Tác giả nói có thể dùng attention hiệu quả để tránh, nhưng cố tình giữ Transformer gốc cho đơn giản. Ví dụ: CORD ở 1280×960 cho 0.7 giây/ảnh và accuracy 91.1 (so với 1.2 giây, 90.9 ở 2560×1920 theo Bảng 2). Tức hạ độ phân giải trong tác vụ này **không làm giảm điểm** mà còn nhanh hơn, nhưng độ phân giải lớn giúp khi dữ liệu ít.

5.4 Định vị chữ không cần giám sát (Hình 8)

Cross-attention của decoder tập trung đúng vùng chữ đang sinh, dù không có giám sát định vị: chỉ báo phụ trợ.

5.5 Phụ thuộc vào OCR (Hình 9)

Với 4 engine OCR công khai (EasyOCR, PaddleOCR, MS Azure, CLOVA OCR), tốc độ và độ chính xác của BERT/LayoutLMv2 **dao động mạnh theo engine**. Donut không bị phụ thuộc.

5.6 Ít dữ liệu

Giới hạn tập huấn luyện CORD: Donut vượt độ chính xác của LayoutLMv2 chỉ với **10% dữ liệu (80 mẫu)**; độ phân giải 2560×1920 bền hơn khi cực ít dữ liệu.

---

6. Giới hạn

Tác giả nêu:

- Do giới hạn độ phân giải của pipeline end-to-end, **bỏ sót chữ nhỏ** trong ảnh lớn (ví dụ trong Hình 6, câu hỏi về "Publication No." trả lời 943 thay vì 540); có thể giảm nhẹ bằng tăng kích thước ảnh, nhưng làm đắt hơn.
- Là bước đầu ("first step") với kiến trúc tối giản; không dùng attention hiệu quả.
- Trên DocVQA vẫn còn khoảng cách với mô hình dựa OCR mạnh nhất.

Nhận xét:

- Chi phí encoder tăng bậc hai theo số patch trong mỗi cửa sổ và tuyến tính theo số cửa sổ; cross-attention của decoder phải nhìn toàn bộ token ảnh ở mỗi bước.
- Chỉ so sánh với baseline cùng thời (2021-2022); các so sánh tốc độ đo trên GPU P40 và bao gồm thời gian gọi OCR API, nên khó dịch sang phần cứng hiện đại.
- Một số bộ dữ liệu IE (Business Card, Receipt) là nội bộ, không kiểm chứng lại được.
- Cơ chế parse JSON bằng regex coi trường là "mất" khi thiếu thẻ, không có cơ chế sửa lỗi.
- Phần lớn ablation (Hình 7, 9) chỉ có biểu đồ; ta không có số.

---

7. Liên hệ với các bài khác

- **01 (Attention Is All You Need):** Donut là encoder-decoder Transformer thuần, decoder dùng cross-attention vào encoder.
- **29 (ViT):** encoder Swin là ViT phân cấp với attention cửa sổ.
- **40 (Nougat):** kế thừa gần như nguyên kiến trúc Donut (Swin + mBART), đổi sang đầu ra Markdown cho PDF khoa học, đầu vào 896×672, decoder 4096 token. Đọc ngay sau bài này.
- **41 (GOT-OCR2.0), 42 (olmOCR):** Các hệ sau dùng LLM làm decoder cho bài toán chuyển ảnh trang thành văn bản.
- **34-37 (LLaVA, InternVL, Qwen2-VL):** thay vì cross-attention, VLM kiểu LLaVA nối token ảnh vào chuỗi LLM (self-attention); khác nhau về kiểu ghép, giống nhau ở chỗ đầu ra là chuỗi token.
- **32 (Flamingo):** dùng cross-attention vào LLM, gần với kiểu ghép của Donut hơn LLaVA.

---

8. Tóm tắt công thức / khái niệm

Khái niệm	Nội dung
OCR-free VDU	Một mô hình Transformer đọc ảnh và sinh chuỗi, không OCR ngoài
Encoder	Swin-B, layers {2,2,14,2}, window 10
Decoder	4 lớp đầu BART đa ngôn ngữ
Mục tiêu pre-train	$-\sum_i \log p(y_i \mid y_{<i}, x)$ trên chuỗi chữ theo thứ tự đọc
Dữ liệu	IIT-CDIP 11M (nhãn CLOVA OCR) + SynthDoG 2M (0.5M × 4 ngôn ngữ)
Đầu ra có cấu trúc	`[START_x] ... [END_x]` ⇔ JSON; thiếu thẻ đóng = mất trường
Thiết lập	200K bước, 64 A100, batch 196, 2560×1920, độ dài decoder 1536
RVL-CDIP	95.30 acc, 143M, 752 ms
CORD	84.1 F1 / 90.9 TED-acc, 1.2 s
DocVQA	ANLS 67.5 (chữ viết tay 72.1)
Phát hiện	Đọc chữ là nhiệm vụ pre-train tốt nhất; độ phân giải lớn rất quan trọng

9. Kết luận ngắn

Donut đặt ra mẫu thiết kế mà các bài sau (Nougat, GOT, olmOCR) dùng lại: ảnh vào, chuỗi token có cấu trúc ra, tiền huấn luyện bằng "đọc chữ". Điểm mạnh: đơn giản, nhanh hơn pipeline OCR, bền vững với lỗi OCR và ít dữ liệu. Điểm yếu: phụ thuộc mạnh vào độ phân giải (chữ nhỏ), thua mô hình dựa OCR mạnh nhất ở DocVQA.

Link gốc: https://arxiv.org/abs/2111.15664

Bài 40

Nougat: Neural Optical Understanding for Academic Documents (2023)

Tác giả: Lukas Blecher, Guillem Cucurull, Thomas Scialom, Robert Stojnic (Meta AI).
Venue / arXiv: arXiv:2308.13418 (v1, 25/8/2023); bài không nêu venue.
Nguồn: https://arxiv.org/abs/2308.13418
Chủ đề: Chuyển ảnh trang PDF của bài báo khoa học thành markup (Markdown + LaTeX) bằng một encoder-decoder end-to-end; tạo dữ liệu cặp (trang PDF, nguồn LaTeX); xử lý hiện tượng lặp (repetition) khi sinh.
Vị trí trong lộ trình: Đọc trước: 39 (Donut, kiến trúc gốc), 29 (ViT). Bài này thêm: bài toán ảnh trang → văn bản dài (hàng nghìn token), cách tạo dữ liệu, và cách phát hiện/giảm sụp đổ lặp. Tiếp theo: 41 (GOT-OCR2.0), 42 (olmOCR).

> Ghi chú học tập, không phải bản dịch. Số "tự tính" không có trong paper. Bản văn bản của PDF có một đoạn nhiễu lạc từ bài khác nằm giữa mục 4 và mục 5; ghi chú chỉ dùng phần nội dung thuộc paper Nougat.

1. Bối cảnh

PDF chứa nhiều tài liệu khoa học nhưng thường không biểu diễn đầy đủ cấu trúc ngữ nghĩa của trang. Đặc biệt, một công thức có thể chỉ được lưu thành các ký tự rời và vị trí của chúng. Những công cụ hiện có giải quyết từng phần của bài toán:

- **OCR truyền thống (ví dụ Tesseract):** đọc từng dòng, từng ký tự; không hiểu quan hệ giữa ký tự nên coi chỉ số trên/dưới như chữ thường, hỏng phân số, ma trận.
- **GROBID:** phân tích PDF khoa học sinh ra XML (tập trung vào thông tin thư mục), nhưng không khôi phục ngữ nghĩa công thức.
- **Corpus kiểu S2ORC:** dùng GROBID nên thiếu biểu diễn công thức.

Nougat đề xuất một mô hình **chỉ dựa vào ảnh trang** (không cần văn bản nhúng trong PDF), nên cũng dùng được cho tài liệu quét.

Khái niệm nền:

- **Markup:** ngôn ngữ đánh dấu (ở đây là Markdown "nhẹ" + LaTeX cho công thức/bảng) cho phép biểu diễn tiêu đề, in đậm, công thức, bảng.
- **Greedy decoding:** ở mỗi bước chọn token có xác suất cao nhất; dễ rơi vào vòng lặp.
- **Edit distance (Levenshtein):** số phép chèn/xóa/thay ký tự tối thiểu để biến chuỗi này thành chuỗi kia.

---

2. Ý tưởng chính

Nougat chính là **Donut được áp dụng cho trang bài báo khoa học**: encoder Swin nhận ảnh trang, decoder mBART sinh Markdown autoregressive. Đóng góp chính không phải kiến trúc mà là (i) **pipeline tạo dữ liệu** ghép trang PDF với mã nguồn LaTeX tương ứng (qua LaTeXML → HTML → Markdown, rồi tách theo trang bằng khớp văn bản), (ii) **tăng cường dữ liệu** mô phỏng ảnh quét và **nhiễu token** để chống lặp, và (iii) **phát hiện lặp lúc suy luận** bằng phương sai của logit lớn nhất để dừng sớm. Kết quả: vượt rõ GROBID và văn bản nhúng PDF trên mọi chỉ số.

---

3. Phương pháp chi tiết

3.1 Kiến trúc (Hình 1, mục 3)

- **Encoder:** nhận ảnh $x \in \mathbb{R}^{3\times H_0\times W_0}$, cắt lề, resize vào khung chữ nhật cố định (H, W) và pad nếu nhỏ hơn. Dùng **Swin Transformer** (base), khởi tạo từ trọng số tiền huấn luyện; đầu ra $z \in \mathbb{R}^{d\times N}$.
- **Decoder:** transformer decoder có cross-attention vào z, sinh token autoregressive; đầu ra chiếu về từ vựng thành logit $\ell \in \mathbb{R}^{v}$. Theo Donut, dùng **mBART**, và **tokenizer của Galactica** (Taylor et al., chuyên miền khoa học).
- Giải mã: **greedy**.

3.2 Thông số (mục 3.1)

Mục	Giá trị
Render	96 DPI
Kích thước đầu vào	$(H, W) = (896, 672)$; tỉ lệ nằm giữa US Letter và A4
Độ dài decoder tối đa	$S = 4096$ (chữ dày, bảng tốn token)
Decoder	10 lớp (mBART, "decoder-only" theo cách paper mô tả); tổng **350M** tham số (base)
Bản nhỏ	**250M**, 4 lớp decoder, $S = 3584$
Huấn luyện	AdamW, 3 epoch, batch hiệu dụng 192
Learning rate	khởi đầu $5 \cdot 10^{-5}$, nhân 0.9996 mỗi 15 bước đến $7.5 \cdot 10^{-6}$ (chọn thấp vì bất ổn)

3.3 Tăng cường dữ liệu

Dữ liệu là PDF "sinh số" (digital-born) nên sạch; để mô phỏng bản quét, áp dụng (thư viện Albumentations) bitmap, erosion, dilation, affine, shift-scale-rotate, grid distortion, elastic transform, brightness/contrast, nén ảnh, nhiễu Gauss, làm mờ Gauss (Hình 2). Ngoài ra **nhiễu trên văn bản đích** (xem 5.1).

3.4 Tạo dữ liệu (mục 4)

Nguồn:

- **arXiv:** 1,748,201 bài; lấy mã nguồn LaTeX + PDF đã biên dịch.
- **PMC** (PubMed Central): dùng XML ngữ nghĩa; ít hơn vì công thức/bảng hay là ảnh; chỉ dùng cho giai đoạn pre-train.
- **IDL** (Industry Documents Library): chỉ có text OCR chất lượng cao, không có định dạng; dùng pre-train để học OCR cơ bản trên tài liệu quét.

Bảng A.1: arXiv 7,511,745 trang; PMC 536,319; IDL 446,777; tổng **8,204,754** trang. arXiv chiếm > 91.5%.

Chuẩn hóa nguồn: chạy qua **LaTeXML** → HTML5 để thay macro, chuẩn hóa khoảng trắng, bảng, tham chiếu; rồi chuyển HTML thành Markdown nhẹ (tiêu đề, đậm/nghiêng, thuật toán, công thức LaTeX inline/display, bảng LaTeX) (Hình 3).

Tách theo trang (mục 4.1) — khó vì LaTeX tự quyết định ngắt trang và họ không biên dịch lại:

1. Loại hình và bảng khỏi PDF bằng pdffigures2 (chú thích khớp với XML bằng khoảng cách Levenshtein), rồi chèn lại ở cuối trang.
2. Trích dòng chữ từ PDF (MuPDF), bỏ số trang/đầu cuối trang. Huấn luyện **bag-of-words TF-IDF + SVM tuyến tính** để dự đoán số trang từ dòng chữ; áp mô hình lên từng đoạn của nguồn LaTeX.
3. Dự đoán số trang theo chỉ số đoạn lý tưởng là hàm bậc thang; thực tế nhiễu. Tìm điểm cắt tối ưu bằng độ không thuần Gini:

$$G_{[a,b]}(i) = (b-a)\cdot\Big(1 - p^2_{[a,b]}(i) - p^2_{[a,b]}(i+1)\Big), \qquad \hat{t}_i = \arg\min_t \Big(G_{[a,t]}(i) + G_{[t,b]}(i)\Big)$$

trong đó $p_{[a,b]}(i)$ là xác suất chọn một phần tử có trang dự đoán i trong đoạn $[a,b]$ (công thức theo paper). Tìm lần lượt từng điểm ngắt, cận dưới là điểm ngắt trước.

4. **Fuzzy matching** quanh điểm cắt: so câu cuối trang trước và câu đầu trang sau với văn bản nhúng PDF bằng khoảng cách Levenshtein chuẩn hóa; điểm cắt "chính xác" được 1, nếu lệch thì điểm là $1-$khoảng cách. Một trang chỉ được giữ nếu điểm trung bình hai ranh giới ≥ 0.9. **Tỉ lệ chấp nhận khoảng 47%** số trang.

Nhãn có nhiễu (mục 4.2): LaTeXML để lại tạo tác (đánh số tiểu mục không có trong PDF, thiếu hình/bảng, lệch ranh giới trang, mất ký tự định dạng vô hình như đậm/nghiêng/tiêu đề). Tác giả nhận định: lượng mẫu lớn bù đắp những lỗi nhỏ này.

Pseudo-code pipeline:

```
for paper in arXiv:
    html = LaTeXML(tex)                      # chuẩn hóa macro, bảng, ref
    md   = html_to_markdown(html)
    pdf_lines = mupdf_lines(pdf) minus header/footer
    clf = SVM(TFIDF).fit(pdf_lines, page_no)
    page_of_para = clf.predict(paragraphs(md))
    cuts = min_gini_staircase(page_of_para)         # điểm cắt thô
    cuts = fuzzy_refine(cuts, pdf_text)             # điểm cắt tinh + điểm số
    for page in pages(md, cuts):
        if mean(score_both_boundaries) >= 0.9:
            keep (rasterize(pdf_page), page_markdown)   # ~47% được giữ
```

---

4. Thí nghiệm và kết quả chính

4.1 Độ đo

Edit distance chuẩn hóa (chia tổng số ký tự), BLEU, METEOR, và precision/recall/F1. Tách riêng ba kiểu nội dung: **văn bản thường, công thức, bảng**, vì công thức LaTeX có nhiều cách viết tương đương (thứ tự chỉ số trên/dưới, `\frac` vs `\over`, `\bm` vs `\mathbf`, khoảng trắng, ngoặc thừa), và ranh giới giữa công thức inline và chữ mơ hồ (ví dụ `H_0 1` vs `H_01`), làm điểm công thức/văn bản thấp đi dù hiển thị giống nhau.

4.2 Bảng 1: tập test arXiv (số đúng như paper)

Phương pháp	Loại	Edit dist. ↓	BLEU ↑	METEOR ↑	Prec. ↑	Recall ↑	F1 ↑
PDF (văn bản nhúng)	Tất cả	0.255	65.8	82.1	77.1	81.4	79.2
GROBID	Tất cả	0.312	55.6	71.9	74.0	72.1	73.0
GROBID + LaTeX-OCR	Bảng	0.626	25.1	64.5	61.4	80.7	69.7
	Văn bản thường	0.363	57.4	69.2	82.1	70.5	75.9
	Công thức	0.727	0.3	5.0	11.0	8.6	9.7
Nougat small (250M)	Tất cả	0.073	88.9	92.8	93.6	92.2	92.9
	Bảng	0.220	68.5	78.6	75.0	79.8	77.3

Phương pháp	Loại	Edit dist. ↓	BLEU ↑	METEOR ↑	Prec. ↑	Recall ↑	F1 ↑
	Văn bản thường	0.058	91.0	94.3	96.1	95.3	95.7
	Công thức	0.117	56.0	74.7	77.1	76.8	76.9
Nougat base (350M)	**Tất cả**	**0.071**	**89.1**	**93.0**	**93.5**	**92.8**	**93.1**
	Bảng	0.211	69.7	79.1	75.4	80.7	78.0
	Văn bản thường	0.058	91.2	94.6	96.2	95.3	95.7
	Công thức	0.128	56.9	75.4	76.5	76.6	76.5

Đọc kết quả:

- Edit distance toàn trang giảm từ 0.255 (văn bản nhúng PDF) và 0.312 (GROBID) xuống **0.071** (Nougat base).
- Công thức khó nhất như dự kiến (BLEU 56-57), nhưng GROBID + LaTeX-OCR gần như thất bại (BLEU 0.3), chủ yếu vì GROBID bỏ sót nhiều công thức và hộp bao kém.
- **Mô hình nhỏ (250M) ngang mô hình base (350M)** (edit 0.073 vs 0.071; thậm chí công thức edit 0.117 thấp hơn 0.128). Đây là một điểm dữ liệu trực tiếp về việc cắt decoder (10 → 4 lớp) ít hại. Lưu ý đây là hai mô hình huấn luyện riêng, không phải cắt tỉa.
- Văn bản nhúng PDF tốt hơn GROBID do khác biệt định dạng ở trang tiêu đề và danh mục tham khảo (theo tác giả).

---

5. Phân tích: lặp, hallucination, tốc độ

5.1 Hiện tượng lặp (mục 5.4)

Mô hình có lúc **sụp vào vòng lặp**: lặp câu/đoạn cuối mãi, hoặc luân phiên hai câu (đôi khi đổi vài từ), hoặc thậm chí tự đếm số lần lặp trong danh mục tham khảo. Xảy ra ở **1.5% trang test**, nhiều hơn với tài liệu ngoài miền. Tác giả quan sát nó thường xuất hiện **sau một lỗi** của mô hình và mô hình không tự phục hồi được (đây là quan sát, không phải thí nghiệm nguyên nhân).

Cách 1: nhiễu token lúc huấn luyện (anti-repetition augmentation). Với mỗi mẫu, có xác suất cố định thay một token ngẫu nhiên bằng token ngẫu nhiên khác; tiếp tục lấy mẫu để thay thêm cho đến khi số ngẫu nhiên vượt ngưỡng (10%). Ý nghĩa: mô hình học cách xử lý một token trước đó bị sai (vì lúc suy luận ngữ cảnh chứa token do chính nó sinh, kể cả sai). Tác giả không thấy giảm hiệu năng nhưng **giảm đáng kể lặp: với tài liệu ngoài miền, số trang chuyển đổi thất bại giảm 32%**.

Cách 2: phát hiện lặp lúc suy luận bằng logit. Sau khi sụp, logit lớn nhất $\ell_i = \max \ell_i$ của token thứ i thành tín hiệu gần như phẳng. Định nghĩa với cửa sổ $B = 15$:

$$\operatorname{VarWin}_B[\ell](x) = \frac{1}{B} \sum_{i=x}^{x+B} \left(\ell_i - \frac{1}{B} \sum_{j=x}^{x+B} \ell_j \right)^2$$

và phương sai của tín hiệu đó tính từ x đến hết chuỗi:

$$\mathrm{VarEnd}_B[\ell](x) = \frac{1}{S-x}\sum_{i=x}^{S}\Big(\mathrm{VarWin}_B[\ell](i) - \frac{1}{S-x}\sum_{j=x}^{S}\mathrm{VarWin}_B[\ell](j)\Big)^2$$

Nếu VarEnd rơi **dưới ngưỡng 6.75 và ở dưới đó cho đến cuối chuỗi** thì coi là lặp. Trực giác (diễn giải của ta): khi mô hình lặp, nó rất chắc chắn nên logit lớn nhất ổn định, nên phương sai cửa sổ nhỏ và đồng đều.

Lúc suy luận không biết đuôi chuỗi, nên tính trên **200 token cuối** với **nửa ngưỡng**; sau khi sinh xong, lặp lại quy trình đầy đủ để xác nhận. Mục đích: không lãng phí tài nguyên chờ đến 4096 token khi không có token kết thúc nào xuất hiện.

Pseudo-code:

```
def collapsed(max_logits, B=15, thr=6.75, tail=200, online=True):
    l = max_logits[-tail:] if online else max_logits
    t = thr / 2 if online else thr
    vw = [var(l[i:i+B]) for i in range(len(l) - B)]      # VarWin
    # VarEnd(x) = var(vw[x:]) ; "dưới ngưỡng cho đến hết chuỗi"
    return all(var(vw[x:]) < t for x in range(len(vw)))   # (diễn giải đơn giản)
```

(Dòng cuối là cách diễn giải của ta cho "dưới ngưỡng và ở dưới đến hết chuỗi", dùng đuôi nên sẽ tính O(n²) nếu viết ngây thơ; cài đặt thật có thể dùng tổng tích lũy.)

5.2 Hallucination và thiếu nhất quán giữa trang

Paper không dùng từ hallucination nhiều, nhưng nêu đúng hiện tượng: vì mô hình chỉ xử lý **từng trang độc lập** nên tài liệu ghép lại bị **thiếu nhất quán**, nhất là danh mục tham khảo (nhiều kiểu trích dẫn) và tiêu đề mục: số thứ tự đôi khi **bị bỏ qua hoặc bịa (hallucinated)**. Đó là giới hạn được nêu, chưa có phương pháp giải.

5.3 Tốc độ (mục 5.5)

GPU **A10G 24 GB**: xử lý **6 trang song song**; trung bình khoảng **1400 token/trang**; thời gian sinh trung bình **19.5 giây mỗi batch** cho mô hình base, không tối ưu suy luận. So sánh: GROBID 10.6 PDF/giây; paper thừa nhận Nougat "rất chậm" nhưng xử lý được bản quét và công thức.

Tính nhẩm (tự tính): 1400 token / 19.5 s ≈ **72 token/giây** cho mỗi chuỗi trong batch, và ≈ 3.25 giây/trang theo thông lượng (19.5/6). Đây là cận dưới của tốc độ từng chuỗi (batch kết thúc khi chuỗi dài nhất xong), nên không phải tốc độ batch 1.

---

6. Giới hạn

Tác giả nêu:

- Lặp/sụp đổ là vấn đề chính, để lại cho công việc sau.
- Huấn luyện trên bài báo khoa học nên mạnh nhất trên tài liệu tương tự; hầu hết mẫu là tiếng Anh. Ngôn ngữ Latin khác "tạm ổn" nhưng ký tự đặc biệt bị thay bằng tương đương Latin; **chữ không phải Latin cho kết quả lặp ngay lập tức**.
- Tốc độ chậm so với phương pháp cổ điển.
- Mỗi trang xử lý độc lập → thiếu nhất quán toàn tài liệu.

Nhận xét:

- Đánh giá trên tập test do chính họ tạo từ arXiv với nhãn nhiễu từ LaTeXML; điểm số bị hạn chế bởi mơ hồ viết LaTeX (tác giả cũng nói điều này) và có thể đánh giá thấp mô hình. Tập đã lọc theo điều kiện khớp ≥ 0.9 nên thiên về trang "dễ khớp".
- Không có ablation số liệu riêng cho nhiễu token (chỉ nêu "giảm 32% trang lỗi ngoài miền"); không có đo độ nhạy ngưỡng 6.75 (đây là ngưỡng chọn bằng quan sát).
- Không so với mô hình OCR/LLM hiện đại; GROBID + LaTeX-OCR là baseline yếu cho công thức vì ghép hai hệ thống rời.
- Mô hình chỉ có bản tiếng Anh/khoa học; không chứng minh trên tiếng Việt hay chữ không Latin.

---

7. Liên hệ với các bài khác

- **39 (Donut):** kiến trúc gần như giống (Swin + mBART); khác ở miền (bài báo), đầu ra (Markdown/LaTeX thay vì JSON), kích thước đầu vào (896×672 thay vì 2560×1920) và độ dài đầu ra (4096 thay vì 1536).
- **29 (ViT):** Swin là ViT phân cấp.
- **41 (GOT-OCR2.0), 42 (olmOCR):** Các hệ sau cùng hướng tới chuyển trang thành văn bản; olmOCR dùng VLM thay cho mBART.
- **48-50 (speculative decoding, Medusa, EAGLE):** tăng tốc sinh chuỗi dài; phù hợp với bài toán 1400-4096 token mỗi trang.
- **44-46 (FlashAttention, PagedAttention):** các tối ưu suy luận mà Nougat nói rõ là chưa dùng ("without any inference optimization").
- **55-56 (StreamingLLM, H2O):** quản lý KV cache cho chuỗi dài.

---

8. Tóm tắt công thức / khái niệm

Khái niệm	Nội dung
Kiến trúc	Swin (base) encoder + mBART decoder, tokenizer Galactica, greedy decoding
Đầu vào / đầu ra	896×672 (96 DPI); tối đa $S = 4096$ token (bản nhỏ 3584)
Kích thước	base 350M (10 lớp decoder), small 250M (4 lớp)
Dữ liệu	8,204,754 trang (arXiv 7,511,745; PMC 536,319; IDL 446,777)
Tách trang	TF-IDF + SVM, Gini $G_{[a,b]}(i) = (b-a)(1-p^2(i)-p^2(i+1))$, giữ trang nếu điểm ≥ 0.9 (~47%)
Nhiễu token	thay token ngẫu nhiên, tiếp tục đến ngưỡng 10%; giảm 32% trang lỗi ngoài miền
Phát hiện lặp	$\text{VarEnd}_B[\ell]$, $B = 15$, ngưỡng 6.75; online: 200 token cuối, nửa ngưỡng
Kết quả chính	Edit 0.071 (base) vs 0.255 (văn bản PDF) vs 0.312 (GROBID)
Công thức / bảng	Edit 0.128 / 0.211 (base)
Tốc độ	A10G 24 GB, 6 trang song song, ≈1400 token/trang, 19.5 s/batch
Tần suất lặp	1.5% trang test

9. Kết luận ngắn

Nougat chuyển kiến trúc Donut sang bài toán trang bài báo → Markdown/LaTeX, với đóng góp thực tiễn ở chỗ tạo dữ liệu cặp (PDF, LaTeX) và ở xử lý sụp đổ lặp (nhiễu token lúc train, phát hiện bằng phương sai logit lúc suy luận). Kết quả vượt rõ GROBID và văn bản nhúng PDF; mô hình nhỏ 250M ngang mô hình 350M.

Link gốc: https://arxiv.org/abs/2308.13418

Bài 41

General OCR Theory: Towards OCR-2.0 via a Unified End-to-end Model (2024)

Tác giả: Haoran Wei, Chenglong Liu, Jinyue Chen, Jia Wang, Lingyu Kong, Yanming Xu, Zheng Ge, Liang Zhao, Jianjian Sun, Yuang Peng, Chunrui Han, Xiangyu Zhang (StepFun, Megvii, UCAS, Tsinghua).
Venue / arXiv: arXiv:2409.01704 (v1, 03/09/2024). Paper không nêu venue hội nghị.
Nguồn: https://arxiv.org/abs/2409.01704
Chủ đề: OCR end-to-end, encoder nén mạnh (1024×1024 → 256 token), decoder 0.5B, dữ liệu tổng hợp (data engine), OCR "tổng quát" (công thức, bảng, nhạc, hình học, biểu đồ).
Vị trí trong lộ trình: Đọc trước: 29 (ViT), 34/35 (LLaVA), 39 (Donut), 40 (Nougat). Bài này đưa ra một thiết kế OCR **rất nhỏ** (580M) với số visual token cực ít; nó là mốc tham chiếu để so với hướng "VLM lớn cắt ảnh thành nhiều tile" (36 InternVL 1.5, 37 Qwen2-VL) và với các VLM OCR đời sau (42 olmOCR.

> Đây là ghi chú học tập bằng tiếng Việt, **không phải bản dịch** paper. Các con số đánh dấu "tự tính" được tính từ số liệu trong paper.

1. Bối cảnh

Trong cách phân loại OCR-1.0 của các tác giả, một pipeline OCR truyền thống ghép nhiều mô-đun: phân tích bố cục, phát hiện chữ, cắt vùng và nhận dạng ký tự. Việc tối ưu từng mô-đun riêng có thể dẫn đến lỗi cộng dồn và tăng chi phí bảo trì; các tác vụ như chữ trong cảnh tự nhiên, tài liệu hay công thức cũng thường cần những mạng khác nhau. Theo bài báo, giới hạn về khả năng mở rộng của bộ nhận dạng khiến hệ thống phải xử lý từng mảnh ảnh nhỏ.

LVLM (large vision-language model) làm OCR. Các mô hình kiểu LLaVA gắn một vision encoder kiểu CLIP vào LLM; khả năng OCR "mọc" ra từ đó. Tác giả chỉ ra hai vấn đề:

1. **Xung đột giữa perception và reasoning.** LVLM được thiết kế để suy luận (VQA), nên căn chỉnh *một visual token ≈ một text token*. Với trang tài liệu dày chữ, một token ảnh không chứa đủ ký tự; các mô hình như InternVL-1.5 phải cắt ảnh thành nhiều tile -> hàng nghìn visual token cho một trang A4.
2. **Chi phí huấn luyện/triển khai cao.** Hàng tỷ tham số; thêm một kiểu OCR mới (ví dụ ngôn ngữ mới) mà fine-tune không đủ thì phải pre-train lại, rất tốn.

Thuật ngữ: **visual token** = vector đầu ra của vision encoder được đưa vào LLM như một token. **Linearize** = chuyển cấu trúc 2D (trang) thành chuỗi 1D (văn bản theo thứ tự đọc).

2. Ý tưởng chính

Tác giả đề xuất "General OCR Theory" (OCR-2.0): mô hình OCR nên (i) **end-to-end**, (ii) **rẻ** (không phải chatbot, chỉ cần perception mạnh), (iii) **đa năng** (coi mọi tín hiệu quang học do con người tạo ra là "ký tự": chữ, công thức toán/phân tử, bảng, biểu đồ, nhạc, hình học đơn giản). Hiện thực hoá bằng **GOT**: một encoder nhỏ (~80M) nén ảnh 1024×1024 thành đúng **256 token**, nối qua một lớp linear vào decoder Qwen-0.5B có context 8K. Phần lớn "bí quyết" nằm ở **chiến lược huấn luyện 3 giai đoạn** và **data engine tổng hợp** (render công thức, bảng, nhạc, hình học, biểu đồ bằng các công cụ như LaTeX, Mathpix-markdown-it, Verovio, TikZ, Matplotlib/Pyecharts) để có nhãn miễn phí và chính xác.

3. Phương pháp chi tiết

3.1 Kiến trúc

```
ảnh 1024x1024x3 -> VitDet-base (~80M) -> 256 token x 1024 chiều
                -> Linear (1024x1024)  -> Qwen-0.5B decoder (~500M, 8K ctx) -> văn bản
```

- **Encoder:** VitDet (base). Điểm khác ViT thường: dùng **attention cục bộ (windowed)** nên chi phí với ảnh độ phân giải cao thấp. Hai lớp cuối theo thiết lập Vary-tiny để đưa 1024×1024×3 xuống 256×1024 token.
- **Mọi ảnh được resize cưỡng bức về hình vuông 1024×1024** (thoả hiệp cho nhiều tỉ lệ khung hình).
- **Connector:** một lớp linear (ở giai đoạn 1 là 1024×768 để khớp OPT-125M; giai đoạn 2 là 1024×1024 để khớp Qwen-0.5B).
- **Tổng ~580M tham số**, tác giả nói chạy được trên GPU tiêu dùng 4 GB.

Mức nén (tự tính): nếu patch 16×16 thì 1024^2 cho 64×64 = 4096 patch; còn 256 token tức nén **16 lần**. Với một trang chứa cỡ vài nghìn ký tự (giả định), mỗi visual token phải "gánh" nhiều ký tự; đó chính là lập luận chống lại "1 visual token ≈ 1 text token".

3.2 Ba giai đoạn huấn luyện

Giai đoạn	Huấn luyện gì	Decoder	Dữ liệu	Thiết lập
1. Pre-train encoder	Toàn bộ encoder (decoder nhỏ chỉ để truyền gradient)	OPT-125M	~5M cặp: 3M cảnh tự nhiên + 2M tài liệu	bs 128, 3 epoch, AdamW, cosine, lr 1e-4, max 4096 token
2. Joint-training	Encoder + decoder mới	Qwen-0.5B	OCR thuần (80% dữ liệu gđ1) + Mathpix-markdown + OCR "tổng quát"	max 6000 token, 1 epoch, cùng optimizer
3. Post-train decoder	Chỉ decoder (encoder đóng băng), thêm tính năng mới	Qwen-0.5B	fine-grained, multi-crop, multi-page	max 8192 token, lr bắt đầu 2e-5, 1 epoch

Phần cứng: 8×8 GPU L40s (64 GPU). Mỗi giai đoạn sau lấy lại **80% dữ liệu giai đoạn trước** để không quên (replay).

Ý nghĩa: tách "học nhìn" (encoder) khỏi "học nói" (decoder) -> khi cần tính năng mới chỉ post-train decoder, rẻ hơn nhiều so với pre-train lại cả mô hình.

3.3 Data engine

Giai đoạn 1 (encoder):

- Cảnh tự nhiên: ảnh từ Laion-2B (tiếng Anh) và Wukong (tiếng Trung), nhãn giả bằng PaddleOCR -> 2M cặp (một nửa mỗi ngôn ngữ). Hai kiểu nhãn: (1) ghép chữ trên->dưới, trái->phải; (2) cắt vùng theo bbox thành slice (thêm 1M).
- Tài liệu: PDF từ Common Crawl, trích chữ bằng Fitz -> 1.2M trang + 0.8M slice (dòng/đoạn).

Giai đoạn 2:

- Handwriting: CASIA-HWDB2 (Trung), IAM (Anh), NorHand-v3 (Na Uy); ghép 6-8 dòng vào một trang trắng.
- Công thức toán: ~1M đoạn từ nguồn LaTeX arXiv -> Mathpix-markdown-it -> HTML -> SVG -> PNG (nhanh hơn render LaTeX trực tiếp hơn 20 lần theo paper).
- Phân tử: ChEMBL_25 (2M SMILES) -> ~1M cặp (rdkit + Mathpix-markdown-it).
- Bảng: ~0.3M bảng từ .tex, render bằng LaTeX.

- Trang đầy đủ: ~0.5M Anh (phương pháp Nougat) + 0.5M Trung (theo Vary) + 0.2M in-house gán nhãn bằng Mathpix.
- Nhạc: GrandStaff (**kern, Humdrum) render lại bằng Verovio -> ~0.5M (chỉ một "system", không phải cả trang).
- Hình học: ~1M mẫu TikZ (điểm, đường, hình cơ bản, đường cong hàm số).
- Biểu đồ: 2M (Matplotlib + Pyecharts), nội dung chữ lấy ngẫu nhiên từ corpus NLP, số liệu ngẫu nhiên theo phân phối kiểm soát.

Giai đoạn 3:

- **Fine-grained / tương tác:** prompt kèm bbox (toạ độ chuẩn hoá rồi nhân 1000) hoặc kèm màu khung (đỏ/xanh lá/xanh dương vẽ lên ảnh) để chỉ đọc vùng đó. ~600k mẫu.
- **Multi-crop (ảnh cực lớn):** cắt kiểu InternVL-1.5, tối đa 12 tile 1024×1024, tổng hợp từ ảnh ghép ngang/dọc của trang PDF. ~500k mẫu.
- **Multi-page:** ghép ngẫu nhiên 2-8 trang (mỗi trang < 650 token, tổng không quá 8K) thành một tác vụ -> khỏi phải cắt nhãn theo trang (vấn đề của .tex/Nougat). ~200k mẫu.

3.4 Pseudo-code huấn luyện

```
# Stage 1: encoder pre-training
enc = VitDet_base(); dec_small = OPT_125M(); proj = Linear(1024, 768)
for (img, text) in D1:                      # 5M scene+doc pairs
    loss = CE(dec_small(proj(enc(img))), text); update(enc, proj, dec_small)

# Stage 2: joint
dec = Qwen_0.5B(); proj = Linear(1024, 1024)
for (img, text) in D2:                      # plain OCR + formatted + general OCR
    loss = CE(dec(proj(enc(img))), text); update(enc, proj, dec)

# Stage 3: post-train decoder only
freeze(enc)
for (img, prompt, text) in D3 + 0.8*D2:    # fine-grained / multi-crop / multi-page
    loss = CE(dec(proj(enc(img)), prompt), text); update(proj?, dec)  # paper: "only the
    ↪  decoder"
```

(Dòng cuối: paper nói "chỉ post-train phần decoder"; không nói rõ connector có cập nhật hay không.)

4. Thí nghiệm và kết quả chính

Tất cả benchmark test đã lọc trùng văn bản với tập train (theo paper).

Bảng 1: OCR tài liệu dày (benchmark Fox, trang Anh/Trung; số các mô hình khác lấy từ Fox). Edit distance (thấp tốt) và F1:

Mô hình	Size	Edit en ↓	Edit zh ↓	F1 en	F1 zh
Nougat	250M	0.255	-	0.745	-
Vary	7B	0.092	0.113	0.918	0.952
Qwen-VL-Max	>72B	0.057	0.091	0.964	0.931
Fox	1.8B	0.046	0.061	0.952	0.954
GOT	580M	**0.035**	**0.038**	**0.972**	**0.980**

Bảng 2: Scene text (400 ảnh tự nhiên, 200 Anh + 200 Trung, tự gán nhãn và sửa tay). GOT: edit 0.112 (en) / 0.096 (zh), F1 0.926 / 0.928; Qwen-VL-Max (>72B): 0.182 / 0.168, F1 0.881 / 0.867.

Bảng 3: Tài liệu có định dạng (90 trang, nhãn giả từ Mathpix rồi sửa tay) và OCR tổng quát:

Loại	Edit ↓	F1
All text (single 1024^2)	0.097	0.942
Formula (single)	0.269	0.749
Table (single)	0.254	0.867
All text (multi-crop)	0.086	0.953
Formula (multi-crop)	0.159	0.865
Table (multi-crop)	0.220	0.878
Sheet music (100 mẫu)	0.046	0.939
Geometry (180 mẫu)	0.061	0.884

Fine-grained (Bảng 4): GOT hơn Fox ở cả region-prompt và color-prompt; ví dụ English region edit 0.041 (Fox 0.059), color 0.034 (Fox 0.064).

Chart OCR (Bảng 5): ChartQA-SE AP@strict 0.747 (ChartVLM-7.3B: 0.718, GPT-4V >100B: 0.504); PlotQA-SE AP@strict 0.133 (ChartVLM 0.038). Tác giả khẳng định GOT hơn cả mô hình chuyên biểu đồ.

Số liệu từ bên thứ ba (paper 42 olmOCR, bảng 4): trên olmOCR-Bench, "GOT OCR" đạt **48.3 ± 1.1** overall, trong đó mục Tables chỉ **0.2**, Old Scans 22.1, Long Tiny Text 29.9 (so với Marker 70.1, MinerU 61.5). Điều này cho thấy GOT mạnh trên benchmark của chính nó nhưng yếu trên trang web/PDF đa dạng của olmOCR (chú ý olmOCR chạy GOT ở chế độ "format" mặc định).

5. Phân tích / ablation

Paper **không có bảng ablation** theo nghĩa chặt (không tách tác dụng từng giai đoạn hay từng loại dữ liệu). Những gì có:

- **Multi-crop vs single:** công thức từ F1 0.749 -> 0.865 (edit 0.269 -> 0.159), bảng 0.867 -> 0.878; chữ thường 0.942 -> 0.953. Tức độ phân giải 1024^2 là nút thắt cho chữ nhỏ/công thức/bảng, đúng như trực giác về nén 256 token.
- Phần còn lại là so sánh với baseline khác, không cô lập từng yếu tố nên **không kết luận được** thành phần nào đóng góp bao nhiêu.

6. Giới hạn

Do tác giả nêu:

- Chủ yếu hỗ trợ tiếng Anh và tiếng Trung; cần thêm ngôn ngữ, ký hiệu quang học tổng quát hơn, hình học phức tạp hơn (hình học mới là "thử nghiệm sơ bộ"; nhạc chỉ render đơn "system").
- Đây là mô hình OCR-2.0 "sơ khai" còn nhiều chỗ cải thiện.

Nhận xét (suy luận):

- Benchmark đa phần do chính nhóm tác giả dựng (90 trang, 400 ảnh, 100/180 mẫu nhỏ), nhãn Mathpix; chưa kiểm chứng độc lập. Con số của olmOCR-Bench (48.3) cho thấy khả năng tổng quát hoá ra ngoài phân phối train kém hơn bảng của paper gợi ý.
- Resize cưỡng bức về 1024^2 vuông làm méo trang không vuông và mất chữ nhỏ; phải nhờ multi-crop để bù.
- Không có kiểm soát lặp vô hạn (repetition) hay đánh giá reading order tường minh.
- Không công bố FLOPs/latency/memory đo thực tế, chỉ nói "4 GB GPU".

7. Liên hệ với các bài khác

- **39 Donut, 40 Nougat:** tiền thân end-to-end "ảnh -> văn bản"; GOT mở rộng sang công thức/bảng/nhạc/biểu đồ và dùng LLM nhỏ làm decoder.
- **29 ViT, 36 InternVL 1.5:** GOT dùng VitDet (ViT với attention cục bộ); multi-crop mượn cách cắt tile của InternVL 1.5. Khác biệt triết lý: InternVL/Qwen2-VL tăng số token theo độ phân giải; GOT nén cứng về 256 token mỗi tile.
- **37 Qwen2-VL:** độ phân giải động, số token tỉ lệ với diện tích ảnh; trái ngược với 256 token cố định.
- **42 olmOCR:** dùng GOT làm baseline (48.3) và cho thấy hạn chế tổng quát hoá; olmOCR chọn VLM 7B + nhãn từ GPT-4o thay vì dữ liệu tổng hợp.
- **57 FastV:** hướng khác để giảm visual token (cắt token lúc suy luận thay vì nén ở encoder).

8. Tóm tắt công thức / khái niệm

Khái niệm	Ý nghĩa ngắn
OCR-1.0 / OCR-2.0	pipeline nhiều module / một mô hình end-to-end đa năng
GOT	580M: VitDet ~80M + linear + Qwen-0.5B
Nén ảnh	1024×1024 -> 256 token × 1024 chiều (16× nếu patch 16, tự tính)
3 giai đoạn	pre-train encoder (OPT-125M) -> joint (Qwen-0.5B) -> post-train decoder
Data engine	render nhãn chính xác bằng LaTeX, Mathpix-markdown-it, Verovio, TikZ, Matplotlib/Pyecharts
Fine-grained OCR	prompt kèm bbox (chuẩn hoá ×1000) hoặc màu khung
Multi-crop	tối đa 12 tile 1024^2 kiểu InternVL-1.5
Multi-page	2-8 trang (<650 token/trang) trong một sample, tối đa 8K
Kết quả chính	Fox edit en/zh = 0.035/0.038; olmOCR-Bench (bên thứ ba) 48.3

9. Kết luận ngắn

GOT là một thiết kế "nhỏ mà đa năng": encoder nén mạnh + decoder 0.5B + data engine tổng hợp + huấn luyện 3 giai đoạn có thể cắm thêm tính năng. Điểm mạnh là hiệu quả (580M, 256 token) và tính module; điểm yếu là đánh giá chủ yếu trên benchmark tự xây và khả năng tổng quát hoá hạn chế khi gặp trang thật đa dạng (olmOCR-Bench 48.3).

Link gốc: https://arxiv.org/abs/2409.01704

Bài 42

olmOCR: Unlocking Trillions of Tokens in PDFs with Vision Language Models (2025)

Tác giả: Jake Poznanski, Aman Rangapur, Jon Borchardt, Jason Dunkelberger, Regan Huff, Daniel Lin, Christopher Wilhelm, Kyle Lo, Luca Soldaini (Allen Institute for AI).
Venue / arXiv: arXiv:2502.18443 (v3, 02/07/2025; ngày đầu 25/02/2025). Paper không nêu venue.
Nguồn: https://arxiv.org/abs/2502.18443
Chủ đề: Pipeline dữ liệu OCR bằng VLM, "document anchoring", fine-tune Qwen2-VL-7B, benchmark olmOCR-Bench (unit test pass/fail), chi phí suy luận quy mô lớn.
Vị trí trong lộ trình: Đọc trước: 37 (Qwen2-VL), 34/35 (LLaVA), 41 (GOT-OCR2.0), 46/47 (vLLM, SGLang). Bài này cung cấp (a) công thức "teacher VLM đắt tạo nhãn -> VLM nhỏ chuyên OCR" và (b) **olmOCR-Bench**, benchmark mà dùng làm chuẩn chính; nên là nền cho việc đánh giá model sau khi cắt tỉa.

> Đây là báo cáo kỹ thuật dài; ghi chú này **tập trung vào phần cốt lõi cho việc xây OCR VLM**: pipeline dữ liệu, document anchoring, huấn luyện, benchmark, chi phí. Bỏ qua chi tiết thí nghiệm phụ (ELO, peS2o chỉ tóm tắt) và các prompt dài. Ghi chú học tập, không phải bản dịch; các số "tự tính" là phép tính diễn giải, không phải số liệu của paper.

1. Bối cảnh

PDF chứa lượng lớn văn bản có giá trị cho huấn luyện LLM, từ bài báo, sách đến tài liệu pháp lý. Tuy nhiên, để hiển thị trang, PDF có thể lưu từng glyph cùng tọa độ và font thay vì cấu trúc đoạn văn; ví dụ trong bài báo cho thấy mỗi ký tự có ma trận biến đổi riêng. Thứ tự đọc vì vậy không phải lúc nào cũng có sẵn. Các hướng xử lý hiện có gồm:

- **Pipeline nhiều module** (MinerU, Marker, Grobid...): phức tạp, lỗi cộng dồn.
- **End-to-end** (Nougat, GOT-OCR 2.0): ảnh trang -> văn bản.
- **VLM thương mại** (GPT-4o, Gemini): chất lượng cao nhưng đắt: GPT-4o khoảng 6.240 USD cho 1 triệu trang (batch) và không dùng được nếu tài liệu không được phép gửi API.

Thuật ngữ: **linearization** = làm phẳng trang thành văn bản theo thứ tự đọc tự nhiên. **Born-digital PDF** = PDF tạo thẳng từ phần mềm (có text nhúng), khác với PDF scan chỉ chứa ảnh. **Silver data** = nhãn do mô hình mạnh sinh ra (không phải người gán, nên không hoàn hảo).

2. Ý tưởng chính

(1) Dùng GPT-4o tạo "nhãn bạc" cho 260k trang PDF thật, nhưng **cho GPT-4o xem thêm văn bản thô và toạ độ trích từ chính PDF** (document anchoring) để nó bớt bịa/bỏ sót. (2) Fine-tune Qwen2-VL-7B-Instruct trên các nhãn đó để có một VLM 7B chuyên OCR, chạy rẻ (khoảng 176 USD/triệu trang) và qua SGLang/vLLM. (3) Xây olmOCR-Bench: ~7.000 unit test pass/fail trên 1.400 trang, tránh dùng "LLM làm giám khảo" hay edit-distance mờ. Kết quả: model vượt cả GPT-4o (teacher) trên benchmark này.

3. Phương pháp chi tiết

3.1 Dữ liệu olmOCR-mix-0225

- **Nguồn:** PDF web ngẫu nhiên lấy từ kho nội bộ 240 triệu PDF + sách miền công cộng của Internet Archive (dạng scan ảnh). Lọc: bỏ không phải tiếng Anh (Lingua), bỏ file pypdf không đọc được, từ khoá spam, form điền, văn bản quá ngắn. Lấy tối đa 3 trang ngẫu nhiên mỗi PDF.

- **Quy mô:** 258.641 trang từ 102.825 tài liệu (96.929 doc web = 240.940 trang; 5.896 sách IA = 17.701 trang). Phân loại ước lượng (mẫu 707 trang, phân loại bằng GPT-4o): Academic 55.9%, Brochure 11.2%, Legal 10.2%, Books 6.8%, Table 5.6%, Diagram 4.7%, Slideshow 1.9%, Other 3.7%.

3.2 Document anchoring (lõi kỹ thuật)

Ý tưởng: VLM thường nhận **chỉ ảnh**; nhưng phần lớn PDF có text nhúng. Ta trích bằng `pypdf`: (i) các khối chữ kèm toạ độ, (ii) vị trí ảnh; ghép thành text và đưa vào prompt cùng ảnh raster của trang.

```
Page dimensions: 612.0x792.0
[Image 480x298 to 525x312]
[71x459]Figure 1
[71x398]1 Introduction
...
```

(Mỗi dòng: `[xxy]` toạ độ + nội dung; gốc toạ độ ở góc dưới-trái.) Chi tiết:

- Dữ liệu này **nhiễu**: không giữ thứ tự đọc, lẫn boilerplate và artefact render.
- Chọn khối cho tới giới hạn ký tự, **ưu tiên khối ở đầu và cuối tài liệu**.
- Khi prompt vượt giới hạn token, **tạo lại anchor với giới hạn ký tự giảm theo cấp số nhân**.
- Anchor được lấy mẫu ngẫu nhiên, nên *retry* có thể cho kết quả khác (hữu ích khi một anchor xấu gây lỗi).
- Với PDF scan không có text nhúng thì không có anchor; model chỉ dựa vào ảnh. Paper nói chất lượng vẫn được giữ.

Tác dụng quan sát: ít ảo giác hơn (không tự "hoàn thành" câu dở, không bịa văn bản khi ảnh mờ). Số liệu (bảng 4): GPT-4o có anchor 69.9 vs không 68.9; Gemini Flash 2: 63.8 vs 57.8; olmOCR: 75.5 vs 74.7.

3.3 Sinh nhãn bằng GPT-4o (có cấu trúc)

- Chọn `gpt-4o-2024-08-06` ở chế độ batch (đã loại Gemini 1.5 vì lỗi RECITATION, GPT-4o mini vì ảo giác nhiều, Claude Sonnet 3.5 vì đắt).
- Prompt yêu cầu: trả văn bản thuần như đọc tự nhiên; công thức -> LaTeX; bảng -> markdown; bỏ header/footer nhưng giữ tham chiếu và chú thích; đọc chữ viết tay; giữ nguyên câu nối từ trang trước/sau; nếu không có chữ thì trả null; "Do not hallucinate".
- **Structured output (JSON schema)** buộc model trả trước các trường metadata: `primary_language`, `is_rotation_valid`, `rotation_correction` (0/90/180/270), `is_table`, `is_diagram`, rồi mới tới `natural_text`. Tác giả thấy điều này quan trọng để GPT-4o **không tự viết caption cho ảnh** khi trang không có chữ.

3.4 Huấn luyện

Hạng mục	Giá trị
Base	Qwen2-VL-7B-Instruct, full fine-tune (LoRA cho loss cao hơn nên bỏ)
Tối ưu	AdamW, lr 1e-6, cosine annealing, 10.000 bước, effective batch size 4
Phần cứng	1 node 8×H100 80GB; 1 lần train = 16 node-giờ; tổng mọi thí nghiệm 365 node-giờ
Ảnh	trang render cạnh dài tối đa 1024 px
Prompt	rút gọn, tối đa 6.000 ký tự; thường ~1.000 token ảnh + ~1.800 token anchor ≈ 3.000 token vào
Cắt	mỗi ví dụ cắt về 8.192 token
Loss	chỉ trên token phần trả lời cuối (mask phần prompt)
Đầu ra	giữ nguyên định dạng JSON structured của nhãn

Lưu ý: paper ghi "10.000 bước ≈ 1,2 epoch" nhưng với batch hiệu dụng 4 thì 10.000 bước chỉ là 40.000 mẫu (~0,15 epoch trên 260k); hai con số không khớp, có thể batch thực tế khác. Đừng dựa vào dòng này khi tái lập.

3.5 Suy luận (pipeline)

- Engine SGLang (có hỗ trợ vLLM); gom tài liệu thành work item ~500 trang, hàng đợi trên S3, mở rộng từ 1 tới hàng trăm node.
- **Không ép JSON schema lúc suy luận** (công cụ ép schema không ổn định, schema lệch nhẹ có thể làm model lặp/sụp); vì model đã học schema nên tuân thủ tự nhiên. Parse hỏng -> **retry** từ cùng input.
- **Xoay trang:** nếu model báo `is_rotation_valid`/`rotation_correction`, xoay trang và chạy lại.
- **Lặp vô hạn (repetition)** là lỗi thất bại phổ biến nhất; bị phát hiện khi vượt độ dài tối đa hoặc không parse được. Tăng temperature từ 0.1 tới 0.8 giảm lặp; retry tối đa N lần; fallback sang trích text thuần từ PDF nếu vẫn hỏng. Hạn chế: lặp tới max context ngốn bộ nhớ SGLang và giảm throughput (tác giả định cải tiến bằng phát hiện lặp sớm).
- Tỉ lệ retry đo được: 12%.

3.6 olmOCR-Bench

Mỗi trang có các **unit test** nhị phân, tự động kiểm tra; điểm = % test pass, **macro-average theo loại nguồn**, CI 95% bằng bootstrap 10k mẫu.

Các loại test: **Presence** (đoạn 1-3 câu phải có, cho phép fuzzy), **Absence** (đoạn phải *không* có, dành cho header/footer/số trang), **Reading order** (đoạn A xuất hiện trước B), **Table** (có ô chứa giá trị X và ô kề cạnh thoả quan hệ, dùng markdown hoặc HTML; rowspan/colspan chỉ giữ được với HTML), **Formula** (render LaTeX bằng KaTeX, lấy bbox các ký hiệu, kiểm tra các ký hiệu trong output có cùng quan hệ vị trí tương đối), **Baseline** (có chữ-số, không lặp n-gram > 30 ở cuối, không chứa ký tự CJK/Emoji).

Nguồn (7 loại)	Test	Cách tạo
arXiv Math (AM/AR)	2.927 formula	khớp TeX nguồn với PDF, kiểm KaTeX, duyệt tay
Old Scans Math (OSM)	458 formula	sách toán cũ IA, gán tay
Tables (TA)	1.020	Gemini-Flash-2.0 sinh quan hệ ô, duyệt tay
Old Scans (OS)	526 (279 presence, 70 absence, 177 order)	thư/văn bản đánh máy ở Library of Congress
Headers Footers (HF)	753 absence	DocLayout-YOLO tìm header/footer
Multi Column (MC)	884 order	trang nhiều cột, Claude-Sonnet-3.7 render HTML
Long Tiny Text (LTT)	442 presence	từ điển, danh mục tham khảo chữ nhỏ dày
Tổng	**7.010 test, 1.402 PDF**	

Benchmark được dựng *sau* khi train model và loại trùng URL với olmOCR-mix.

4. Thí nghiệm và kết quả chính

Bảng 4: olmOCR-Bench (pass rate %, ± CI 95%). Cột: AR = arXiv Math, OSM, TA = Tables, OS = Old Scans, HF = Headers/Footers, MC = Multi Column, LTT = Long Tiny Text, Base = baseline.

Model	AR	OSM	TA	OS	HF	MC	LTT	Base	Overall
GOT OCR	52.7	52.0	0.2	22.1	93.6	42.0	29.9	94.0	48.3 ± 1.1
Marker v1.7.5	76.0	57.9	57.6	27.8	84.9	72.9	84.6	99.1	70.1 ± 1.1
MinerU v1.3.10	75.4	47.4	60.9	17.3	96.6	59.0	39.1	96.6	61.5 ± 1.1
Mistral OCR API	77.2	67.5	60.6	29.3	93.6	71.3	77.1	99.4	72.0 ± 1.1
GPT-4o (No Anchor)	51.5	75.5	69.1	40.9	94.2	68.9	54.1	96.7	68.9 ± 1.1
GPT-4o (Anchored)	53.5	74.5	70.0	40.7	93.8	69.3	60.6	96.8	69.9 ± 1.1
Gemini Flash 2 (Anchored)	54.5	56.1	72.1	34.2	64.7	61.5	71.5	95.6	63.8 ± 1.2
Qwen 2 VL (No Anchor)	19.7	31.7	24.2	17.1	88.9	8.3	6.8	55.5	31.5 ± 0.9
Qwen 2.5 VL (No Anchor)	63.1	65.7	67.3	38.6	73.6	68.3	49.1	98.3	65.5 ± 1.2
olmOCR v0.1.75 (No Anchor)	71.5	71.4	71.4	42.8	94.1	77.7	71.0	97.8	74.7 ± 1.1
olmOCR v0.1.75 (Anchored)	74.9	71.2	71.0	42.2	94.5	78.3	73.3	98.3	**75.5 ± 1.0**

Đáng chú ý: Qwen2-VL "thô" (cùng base) chỉ 31.5, fine-tune đưa lên 75.5; theo chú thích hình 1 của paper, chênh lệch hiệu năng là do fine-tune trên dữ liệu olmOCR-mix (cùng chi phí suy luận 7B).

Chi phí (Bảng 6; L40S 0,79 USD/giờ, H100 2,69 USD/giờ, giá RunPod tháng 2/2025):

Hệ thống	Phần cứng	Tokens/s	Trang/USD	USD/triệu trang
GPT-4o API / Batch	API	-	80 / 160	12.480 / 6.240
Marker	H100	332	674	1.484
Mistral OCR	API	-	1.000	1.000
MinerU	L40S	238	1.678	596
Gemini Flash 2 API / Batch	API	-	2.004 / 4.008	499 / 249
olmOCR	L40S	906	5.697	**176**
olmOCR	H100	3.050	5.632	178

Từ chi tiết phụ lục (tự tính): olmOCR trên L40S xử lý 1.288 trang trong 17 phút 10 giây, ~906 token/s => khoảng 933k token sinh ra, tức **~725 token đầu ra/trang** (đã gồm 12% retry), so với ~647 token/trang của GPT-4o (833.599 token / 1.288 trang). Tốc độ ≈ 1,25 trang/giây/L40S và ≈ 4,2 trang/giây/H100 (tự tính).

Đánh giá phụ:

- *Downstream (continued pretraining OLMo-2-7B, 50B token):* thay peS2o (Grobid + luật) bằng cùng PDF xử lý bởi olmOCR: trung bình **53.9 -> 55.2 (+1.3 điểm)** trên MMLU, ARC-C, DROP, HellaSwag, NQ, WinoGrande; riêng HellaSwag 57.4 -> 62.6, MMLU không đổi 61.1.
- *Độ bám teacher:* alignment 0.875 (temperature 0.1) so với nhãn GPT-4o; GPT-4o tự so với chính nó 0.954; GPT-4o mini 0.833.

- *ELO người đánh giá (452 phán đoán, 11 người):* olmOCR > 1800 ELO; thắng MinerU 55/22 (71.4%), Marker 49/31 (61.3%), GOT-OCR 41/29 (58.6%).

5. Phân tích / ablation

Paper có ít ablation chính thức; những gì xác nhận được:

- **Anchor có ích nhưng nhỏ với model đã fine-tune** (+0.8 điểm: 74.7 -> 75.5), lớn hơn với model chưa fine-tune (Gemini Flash 2: +6.0). Với GPT-4o (+1.0). Tức anchor chủ yếu là công cụ **tạo nhãn tốt hơn** và phụ trợ nhẹ lúc suy luận.
- **Full fine-tune thắng LoRA** về validation loss (hình 4, 5; không có con số trong phần chữ).
- **Temperature cao hơn (0.8) giảm lặp** nhưng làm phân phối alignment rộng hơn (nhiều trang "low match" hơn: 195 vs 158 / 1.221 trang theo bảng 8).
- Không có ablation về kích thước dữ liệu (260k trang), độ phân giải 1024 hay kích thước model.

6. Giới hạn

Do tác giả nêu:

- Lặp vô hạn là lỗi thường gặp; retry kéo throughput xuống và phát hiện lặp mới chỉ ở max context (kế hoạch cải thiện sau).
- Dữ liệu và benchmark chủ yếu **tiếng Anh**; trang không phải Anh bị lọc/bỏ qua trong đánh giá người.
- Nhãn là "silver" từ GPT-4o nên kế thừa lỗi/ảo giác của teacher; model chỉ khớp teacher ở 0.875.
- Hầu hết test do GPT-4o/Gemini/Claude sinh rồi người duyệt (tạo thiên lệch tiềm ẩn).

Nhận xét:

- Benchmark chấm *điểm sạch về nội dung* theo định nghĩa riêng (ví dụ **thưởng cho việc loại bỏ header/footer**); các hệ thống muốn phiên âm đầy đủ trang bị phạt.
- Chỉ so các mô hình tại thời điểm 02/2025.

7. Liên hệ với các bài khác

- **37 Qwen2-VL:** base model và cơ chế độ phân giải động (số token ảnh tỉ lệ diện tích; 1024 px ≈ 1.000 token ở đây).
- **41 GOT-OCR2.0:** baseline yếu (48.3), điển hình cho "dữ liệu tổng hợp + benchmark tự dựng" khó tổng quát.
- **46 PagedAttention (vLLM), 47 SGLang:** engine suy luận trong pipeline; lặp vô hạn làm KV cache phình, liên quan RadixAttention/chia sẻ prefix.
- **57 FastV:** prefill ~3.000 token/trang ở olmOCR là chỗ cắt visual token có lợi nhất.
- **08 LoRA:** paper thử và loại bỏ vì loss tệ hơn full fine-tune.

8. Tóm tắt công thức / khái niệm

Khái niệm	Ý nghĩa ngắn
Document anchoring	thêm text + toạ độ trích từ PDF vào prompt cùng ảnh trang
olmOCR-mix-0225	258.641 trang, nhãn GPT-4o 4o-2024-08-06
Huấn luyện	Qwen2-VL-7B, full FT, lr 1e-6, 10k bước, batch 4, ảnh ≤1024 px, ~3.000 token vào
Điểm bench	Overall $= \frac{1}{N}\sum_s \text{Score}(s)$ macro theo loại nguồn
Kết quả	75.5 ± 1.0 (v0.1.75, anchored)
Chi phí	176 USD / triệu trang (L40S), 5.697 trang/USD

Khái niệm	Ý nghĩa ngắn
Lỗi chính	lặp vô hạn; xử lý bằng temperature 0.8, retry, fallback

9. Kết luận ngắn

olmOCR chứng minh rằng với **dữ liệu nhãn chất lượng từ teacher mạnh có hướng dẫn bằng metadata PDF**, một VLM 7B fine-tune đơn giản vượt cả teacher trên benchmark unit-test, với chi phí thấp hơn GPT-4o batch hơn 32 lần (theo paper).

Link gốc: https://arxiv.org/abs/2502.18443

Bài 44

FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness (2022)

Tác giả: Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, Christopher Ré.
Venue / arXiv: arXiv:2205.14135 (v2, 23/06/2022).
Nguồn: https://arxiv.org/abs/2205.14135
Chủ đề: Attention chính xác (không xấp xỉ) nhưng tính theo khối (tiling) để giảm số lần đọc/ghi HBM; online softmax; phân tích IO complexity; block-sparse FlashAttention.
Vị trí trong lộ trình: Đọc trước: 01 (Attention Is All You Need) để nắm softmax$(QK^\top)V$; 17, 18 (attention thưa / tuyến tính) để thấy các cách giảm FLOPs mà bài này phản biện. Bài này thêm: góc nhìn "attention bị nghẽn bởi bộ nhớ, không phải FLOPs" và một kernel chính xác, nhanh hơn. Đọc tiếp: 45 (FlashAttention-2, cải thiện phân chia công việc trên GPU), 46 (PagedAttention, quản lý KV cache khi phục vụ), 14-15 (MQA/GQA, giảm KV cache).

> Đây là ghi chú học tập bằng tiếng Việt, diễn giải lại chứ không phải bản dịch. Các số liệu lấy từ paper (có ghi rõ). Paper dài (~18k từ, nhiều phụ lục); ghi chú tập trung vào thuật toán, chứng minh IO và các kết quả chính.

1. Bối cảnh

Với cách hiện thực thông thường, self-attention tạo ma trận điểm $N \times N$, dẫn đến chi phí thời gian và bộ nhớ bậc hai theo độ dài chuỗi N. Trước FlashAttention, nhiều phương pháp giảm FLOPs bằng attention thưa, xấp xỉ hạng thấp như Linformer hoặc attention tuyến tính như Performer. Bài báo chỉ ra rằng giảm phép tính không nhất thiết làm giảm thời gian chạy: chi phí đọc và ghi bộ nhớ (IO) cũng có thể chi phối hiệu năng. Các phép xấp xỉ còn có thể làm giảm chất lượng.

Quan sát then chốt: trên GPU hiện đại, tốc độ tính toán tăng nhanh hơn tốc độ bộ nhớ, nên đa số phép toán trong Transformer bị nghẽn bởi việc đọc/ghi bộ nhớ (memory-bound), không phải bởi số phép nhân-cộng.

Thuật ngữ nền:

- **HBM (High Bandwidth Memory):** bộ nhớ chính của GPU (ví dụ 40-80 GB trên A100), băng thông ~1.5-2.0 TB/s. Lớn nhưng "chậm" tương đối.
- **SRAM (on-chip, shared memory):** bộ nhớ trên chip, rất nhỏ (A100: 192 KB mỗi SM, 108 SM) nhưng nhanh hơn HBM cỡ một bậc (paper ước ~19 TB/s).
- **Kernel:** một hàm chạy trên GPU; mỗi kernel đọc input từ HBM, tính, ghi output về HBM.
- **Kernel fusion:** gộp nhiều phép toán vào một kernel để input chỉ đọc một lần.
- **Arithmetic intensity:** số phép tính trên mỗi byte truy cập bộ nhớ. Thấp thì memory-bound, cao thì compute-bound.
- **Memory-bound vs compute-bound:** thời gian chạy do truy cập bộ nhớ quyết định vs do số phép tính quyết định. Bài báo nêu: nhân ma trận có chiều trong lớn là compute-bound; elementwise (activation, dropout) và reduction (softmax, layer norm) là memory-bound.

2. Ý tưởng chính

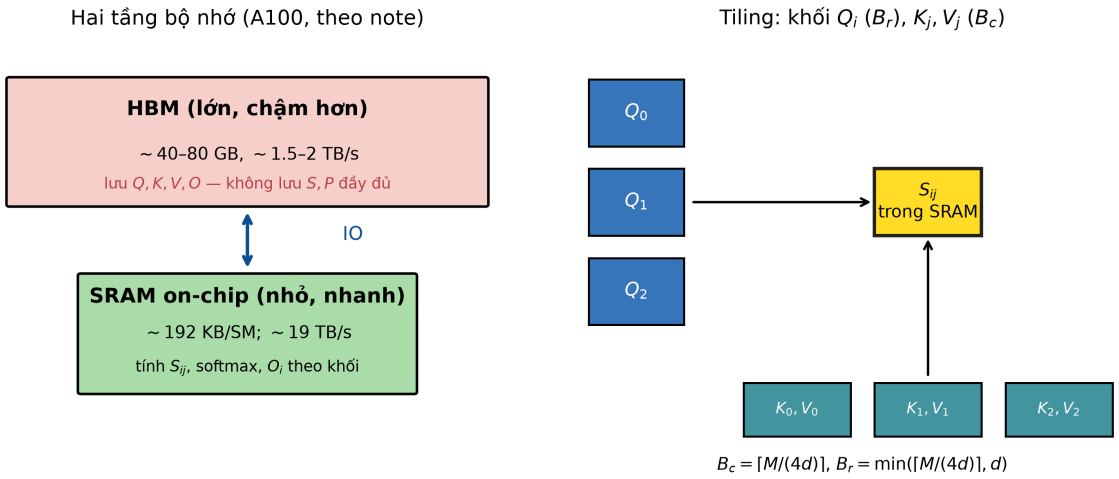
Tính chính xác $O = \text{softmax}(QK^\top)V$ nhưng **không bao giờ ghi ma trận** $N \times N$ **ra HBM**. Chia Q, K, V thành các khối nhỏ vừa SRAM, nạp từng khối vào SRAM, tính điểm, softmax và nhân với V ngay trong SRAM, rồi chỉ ghi kết quả cuối O ($N \times d$). Vì softmax cần cả hàng để chuẩn hoá, ta dùng **online softmax**: giữ vài thống kê nhỏ (max và tổng mũ) cho mỗi hàng và "chỉnh lại tỉ lệ" kết quả cũ mỗi khi thấy khối mới. Ở

backward, thay vì lưu S, P ($N \times N$), chỉ lưu O và thống kê softmax rồi **tính lại** S, P từng khối trong SRAM. FLOPs tăng nhẹ (do tính lại) nhưng IO giảm mạnh, nên chạy nhanh hơn và bộ nhớ chỉ tuyến tính theo N.

3. Phương pháp chi tiết

3.1 Kiến thức nền: HBM, SRAM, memory-bound

Mô hình IO của paper: có hai tầng bộ nhớ, SRAM (nhỏ, kích thước M phần tử) và HBM (lớn). Chi phí = số phần tử di chuyển giữa hai tầng. Paper dùng mô hình IO complexity cổ điển (Aggarwal & Vitter 1988).



FlashAttention: HBM ↔ SRAM tiling; khối Q_i (B_r), K_j, V_j (B_c); không ghi S đầy đủ ra HBM. (minh hoạ, tự tính)

Ví dụ trực giác: bạn là đầu bếp (đơn vị tính), SRAM là mặt bàn bếp nhỏ, HBM là kho lạnh ở xa. Nếu mỗi bước nấu phải chạy ra kho lấy nguyên liệu rồi chạy về cất kết quả, bạn tốn thời gian chạy chứ không phải thời gian nấu. Fusion và tiling là cách mang đủ nguyên liệu cho cả công đoạn lên bàn, nấu xong mới cất.

3.2 Attention chuẩn và chi phí IO

Với $Q, K, V \in \mathbb{R}^{N \times d}$ (N độ dài chuỗi, d head dim):

$$S = QK^\top \in \mathbb{R}^{N \times N}, \quad P = \text{softmax}(S) \in \mathbb{R}^{N \times N}, \quad O = PV \in \mathbb{R}^{N \times d}.$$

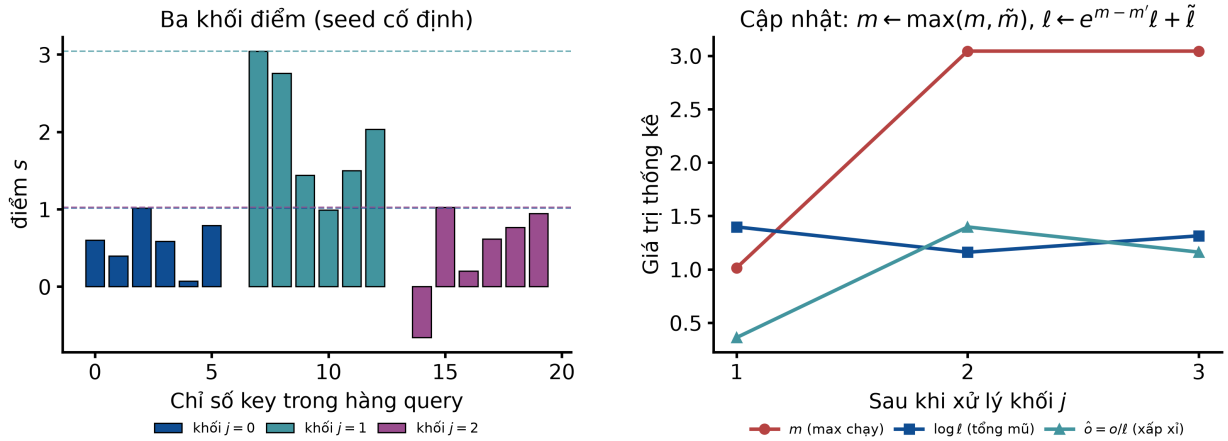
Triển khai chuẩn (Algorithm 0 của paper), mỗi bước là một kernel riêng:

1. Đọc Q, K, tính S, **ghi** S **ra HBM**.
2. **Đọc** S, tính P, **ghi** P **ra HBM**.
3. Đọc P, V, tính O, ghi O.

Hệ quả (Theorem 2): attention chuẩn cần $\Theta(Nd + N^2)$ truy cập HBM và $O(N^2)$ bộ nhớ. Vì thường $N \gg d$ (GPT-2: $N = 1024$, $d = 64$), số hạng N^2 chi phối. Thêm masking/dropout lên S, P còn tăng số lần đọc/ghi.

3.3 Online softmax: suy ra từng bước

Softmax ổn định số học. Với vector $x \in \mathbb{R}^B$:



Online softmax theo khối: cập nhật max m và tổng mũ ℓ, rescale kết quả trung gian. (minh hoạ, tự tính)

$$m(x) = \max_i x_i, \quad f(x) = [e^{x_1 - m(x)}, \dots, e^{x_B - m(x)}], \quad \ell(x) = \sum_i f(x)_i, \quad \text{softmax}(x) = \frac{f(x)}{\ell(x)}.$$

Trừ $m(x)$ để tránh tràn số mũ; kết quả không đổi vì tử và mẫu cùng nhân e^{-m}.

Ghép hai khối. Cho $x = [x^{(1)}, x^{(2)}]$. Khi đó $m(x) = \max(m(x^{(1)}), m(x^{(2)}))$ và

$$\ell(x) = e^{m(x^{(1)}) - m(x)}\, \ell(x^{(1)}) + e^{m(x^{(2)}) - m(x)}\, \ell(x^{(2)}).$$

Lý do: $\ell(x^{(1)}) = \sum_{i \in 1} e^{x_i - m^{(1)}}$ nên $e^{m^{(1)} - m} \ell(x^{(1)}) = \sum_{i \in 1} e^{x_i - m}$. Tức ta chỉ "đổi mốc trừ" từ $m^{(1)}$ sang m bằng nhân một hệ số vô hướng. Vậy chỉ cần giữ (m, ℓ) cho mỗi hàng là gộp được từng khối một. (Paper gọi đây là *algebraic aggregation*.)

Thêm phép nhân với V. Xét một hàng truy vấn q, sau j khối cột đã xử lý, đặt các đại lượng sau (đúng như phần chứng minh bằng quy nạp trong Phụ lục C):

$$m^{(j)} = \max_{\text{cột} \le j} s, \quad \ell^{(j)} = \sum_{\text{cột} \le j} e^{s - m^{(j)}}, \quad o^{(j)} = \sum_{\text{cột} \le j} \frac{e^{s - m^{(j)}}}{\ell^{(j)}} v.$$

Khối mới có $\tilde{m} = \max$ cục bộ, $\tilde{\ell} = \sum e^{s - \tilde{m}}$, và $\tilde{P}V = \sum e^{s - \tilde{m}} v$ (chưa chia). Cập nhật:

$$m^{\text{new}} = \max(m^{(j)}, \tilde{m}), \qquad \ell^{\text{new}} = e^{m^{(j)} - m^{\text{new}}} \ell^{(j)} + e^{\tilde{m} - m^{\text{new}}} \tilde{\ell},$$

$$\boxed{o^{\text{new}} = \frac{1}{\ell^{\text{new}}} \left(\ell^{(j)} e^{m^{(j)} - m^{\text{new}}} o^{(j)} + e^{\tilde{m} - m^{\text{new}}} \tilde{P}V \right)}$$

Kiểm chứng: $\ell^{(j)} o^{(j)} = \sum_{\le j} e^{s - m^{(j)}} v$; nhân $e^{m^{(j)} - m^{\text{new}}}$ cho $\sum_{\le j} e^{s - m^{\text{new}}} v$. Số hạng khối mới tương tự. Tổng hai phần là $\sum_{\text{tất cả}} e^{s - m^{\text{new}}} v$, chia cho ℓ^{new} ra đúng softmax-weighted average. Nghĩa là sau khối cuối, o **bằng đúng** kết quả attention chuẩn (không xấp xỉ).

Ví dụ số nhỏ. Một hàng có điểm $[1, 3]$ (khối 1) và $[2]$ (khối 2), giá trị v vô hướng $[10, 20, 30]$. Khối 1: $m = 3$, $f = [e^{-2}, 1] = [0.135, 1]$, $\ell = 1.135$, $o = (0.135 \cdot 10 + 1 \cdot 20)/1.135 = 18.81$. Khối 2: $\tilde{m} = 2$, $\tilde{\ell} = 1$,

$\tilde{P}V = 30$. $m^{\text{new}} = 3$; $\ell^{\text{new}} = 1.135 + e^{-1} \cdot 1 = 1.503$; $o^{\text{new}} = (1.135 \cdot e^{0} \cdot 18.81 + e^{-1} \cdot 30)/1.503 = (21.35 + 11.04)/1.503 = 21.55$. Trực tiếp: trọng số $\propto [e^1, e^3, e^2] = [2.718, 20.09, 7.389]$, tổng 30.2; $o = (27.18 + 401.7 + 221.7)/30.2 = 21.55$. Khớp (tự tính).

3.4 Thuật toán FlashAttention (forward)

Kích thước khối (Algorithm 1): $B_c = \lceil M/4d \rceil$, $B_r = \min(\lceil M/4d \rceil, d)$, với M là kích thước SRAM tính theo phần tử. Khối Q_i ($B_r \times d$), K_j, V_j ($B_c \times d$); S_{ij} ($B_r \times B_c$) phải vừa SRAM, nên mới có hệ số 4 ở mẫu.

```
Input: Q,K,V (N x d) trong HBM; SRAM kích thước M
Chọn Bc = ceil(M/4d), Br = min(ceil(M/4d), d)
Khởi tạo trong HBM: O = 0 (N x d), l = 0 (N), m = -inf (N)
Tr = ceil(N/Br), Tc = ceil(N/Bc)

for j = 1..Tc:                          # vòng ngoài: khối K,V
    nạp K_j, V_j  HBM -> SRAM
    for i = 1..Tr:                      # vòng trong: khối Q
        nạp Q_i, O_i, l_i, m_i  HBM -> SRAM
        S_ij  = Q_i K_j^T                          # trên chip
        m~_ij = rowmax(S_ij)
        P~_ij = exp(S_ij - m~_ij)                  # theo từng phần tử
        l~_ij = rowsum(P~_ij)
        m_new = max(m_i, m~_ij)
        l_new = exp(m_i - m_new) * l_i + exp(m~_ij - m_new) * l~_ij
        O_i   = diag(l_new)^-1 ( diag(l_i) exp(m_i - m_new) O_i
                               + exp(m~_ij - m_new) P~_ij V_j )   # ghi về HBM
        l_i, m_i <- l_new, m_new                                  # ghi về HBM
return O
```

Theorem 1: thuật toán trả đúng $O = \text{softmax}(QK^\top)V$ với $O(N^2 d)$ FLOPs và chỉ cần thêm $O(N)$ bộ nhớ (cho m, ℓ). Phiên bản đầy đủ (Phụ lục B.3) xử lý thêm scale τ, mask và dropout (lưu trạng thái RNG để tái tạo ở backward).

Ghi chú vòng lặp: vòng ngoài chạy theo K, V, vòng trong theo Q, nên O, ℓ, m bị đọc/ghi lại ở mỗi vòng ngoài. FlashAttention-2 (bài 45) đảo thứ tự này.

3.5 Backward: tính lại thay vì lưu

Backward chuẩn cần $P \in \mathbb{R}^{N \times N}$ để tính dQ, dK, dV. FlashAttention chỉ lưu O và (m, ℓ) rồi **tính lại** S_{ij}, P_{ij} theo khối trong SRAM (đây là một dạng gradient checkpointing chọn lọc). Phụ lục B.2 tự suy ra gradient với bộ nhớ tuyến tính:

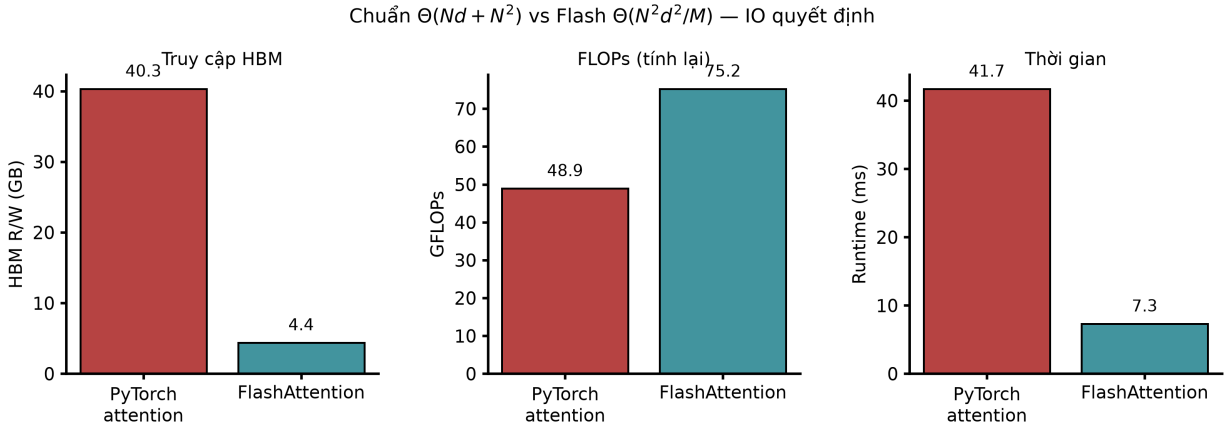
- $dV = P^\top dO$, nên $dv_j = \sum_i P_{ij}\, do_i$.
- $dP = dO\, V^\top$, nên $dP_{ij} = do_i^\top v_j$.
- Jacobian của softmax là $\text{diag}(y) - yy^\top$, nên $dS_{i:} = P_{i:} \circ dP_{i:} - (P_{i:}^\top dP_{i:}) P_{i:}$.
- Đặt $D_i = P_{i:}^\top dP_{i:} = \sum_j \frac{e^{q_i^\top k_j}}{L_i} do_i^\top v_j = do_i^\top o_i$ (đẹp ở chỗ D_i chỉ cần do_i và o_i, kích thước d).

$$\boxed{dS_{ij} = P_{ij}\,(dP_{ij} - D_i)}, \qquad dq_i = \sum_j dS_{ij} k_j, \qquad dk_j = \sum_i dS_{ij} q_i.$$

Nhờ $D_i = do_i^\top o_i$, không cần có P nguyên ma trận để tính tổng hàng. Paper so sánh với Rabe & Staats (2021): họ cũng tránh lưu $N \times N$ nhưng tập trung vào tổng bộ nhớ, không phải IO, nên không nhanh hơn attention chuẩn; còn FlashAttention cập nhật O tăng dần (chỉ một bản sao) và đơn giản hoá backward bằng giải tích thay vì checkpoint từng khối.

3.6 Phân tích IO complexity

Đếm. Mỗi vòng ngoài (một khối K_j, V_j cỡ $\Theta(M)$ phần tử) quét toàn bộ Q và O (mỗi cái Nd phần tử). Số vòng ngoài $T_c = N/B_c = \Theta(Nd/M)$. Tổng truy cập HBM:



Bảng IO: HBM 40.3 → 4.4 GB, FLOPs tăng nhẹ, runtime 41.7 → 7.3 ms. (minh hoạ, tự tính)

$$\underbrace{\Theta(Nd)}_{K,V \text{ đọc một lần}} + T_c \cdot \Theta(Nd) = \Theta\Big(Nd + Nd \cdot \frac{Nd}{M}\Big) = \Theta\Big(\frac{N^2 d^2}{M}\Big) \quad (\text{vì } M \le Nd).$$

$$\boxed{\text{Chuẩn: } \Theta(Nd + N^2) \qquad \text{FlashAttention: } \Theta\Big(\frac{N^2 d^2}{M}\Big)} \quad (d \le M \le Nd)$$

Với $d = 64$-128 và $M \sim 100$ KB, $d^2 \ll M$ nên FlashAttention ít truy cập hơn nhiều lần (paper nói tới ~9 lần ít hơn, Hình 2 trái). Backward cũng $\Theta(N^2 d^2 M^{-1})$.

Cận dưới (Proposition 3). Không tồn tại thuật toán attention chính xác nào đạt $o(N^2 d^2 M^{-1})$ truy cập HBM **cho mọi** $M \in [d, Nd]$. Chứng minh: với $M = \Theta(Nd)$ mọi thuật toán phải đọc ít nhất $\Omega(Nd)$ dữ liệu vào. Đây là cận dưới trên một khoảng của M (kiểu bài toán streaming); paper nhận là chưa chứng minh cận tham số hoá theo M đầy đủ. Nghĩa "tối ưu" ở đây: tối ưu tiệm cận trên một dải kích thước SRAM, không phải tối ưu từng hằng số.

Ví dụ số (tự tính, bỏ qua hằng số trong Θ). Lấy $N = 2500$ (gần độ dài một trang OCR gồm prompt + output, giả định), $M = 100$ KB $= 50{,}000$ phần tử fp16.

d	N^2 (triệu phần tử)	$N^2 d^2 / M$ (triệu phần tử)	Tỉ lệ
64	6.25	0.51	~12x ít hơn
128	6.25	2.05	~3x ít hơn

Nhận xét: lợi thế giảm khi d tăng (vì d^2/M tăng). Số thực tế còn phụ thuộc hằng số (attention chuẩn thực sự đọc/ghi S, P nhiều lần, nên lợi thế thực lớn hơn tỉ lệ thô này). Số đo thực của paper: trên GPT-2 medium ($N = 1024$, $d = 64$, 16 head, batch 64, A100), forward+backward:

	GFLOPs	HBM R/W (GB)	Runtime (ms)
Attention chuẩn	66.6	40.3	41.7

	GFLOPs	HBM R/W (GB)	Runtime (ms)
FlashAttention	75.2	4.4	7.3

FLOPs của Flash **cao hơn** (do tính lại), nhưng HBM giảm ~9x và runtime giảm ~5.7x. Đây là bằng chứng trực tiếp cho luận điểm "IO quyết định thời gian".

3.7 Block-sparse FlashAttention

Cho mặt nạ khối $M \in \{0,1\}^{N/B_r \times N/B_c}$, thuật toán giống Algorithm 1 nhưng **bỏ qua khối bằng 0**. Proposition 4: IO là $\Theta(Nd + N^2d^2M^{-1}s)$ với s là tỉ lệ khối khác 0. Với $s = N^{-1/2}$ hoặc $N^{-1}\log N$ thì IO còn $\Theta(N\sqrt{N})$ hoặc $\Theta(N \log N)$. Thí nghiệm dùng mẫu thưa butterfly cố định. Đây là bằng chứng "proof of concept" rằng FlashAttention là một primitive cho attention xấp xỉ.

4. Thí nghiệm và kết quả chính

Phần cứng: chủ yếu 8×A100 (huấn luyện), một A100 40GB (benchmark).

Tốc độ huấn luyện

Bài	Kết quả (paper)
BERT-large (seq 512), tới 72.0% MLM, 8×A100	MLPerf 1.1 của Nvidia: 20.0 ± 1.5 phút; FlashAttention: 17.4 ± 1.4 phút (nhanh hơn 15%)
GPT-2 small, OpenWebText	HuggingFace 9.5 ngày (1.0x); Megatron-LM 4.7 ngày (2.0x); FlashAttention 2.7 ngày (3.5x); perplexity đều 18.2
GPT-2 medium	HF 21.0 ngày; Megatron 11.5 ngày (1.8x); Flash 6.9 ngày (3.0x); ppl 14.2 / 14.3 / 14.3

(Phần tóm tắt của paper nói "tới 3x so với HF"; bảng cho 3.5x với small và 3.0x với medium. Ghi nguyên số bảng.)

Chất lượng nhờ ngữ cảnh dài hơn

GPT-2 small, Megatron vs Flash	ctx	ppl	thời gian huấn luyện
Megatron-LM	1k	18.2	4.7 ngày (1.0x)
FlashAttention	1k	18.2	2.7 ngày (1.7x)
FlashAttention	2k	17.6	3.0 ngày (1.6x)
FlashAttention	4k	17.5	3.6 ngày (1.3x)

Ngữ cảnh 4k vẫn nhanh hơn Megatron ngữ cảnh 1k cỡ 30% mà perplexity tốt hơn 0.7.

- **Long-Range Arena (1k-4k):** FlashAttention 2.4x so với Transformer chuẩn, độ chính xác trung bình 59.8 so với 59.3; block-sparse 2.8x, 59.6. Để so sánh: Linformer 2.5x nhưng chỉ 54.9; Performer 1.8x, 58.9; Reformer 1.3x, 57.6. Tức FlashAttention giữ nguyên độ chính xác của attention chuẩn mà tốc độ ngang các phương pháp xấp xỉ.
- **Phân loại tài liệu dài:** MIMIC-III: 52.8 (512) tăng lên 57.1 (16k); ECtHR: 72.2 (512) tăng lên 80.7 (8k), rồi 79.2 (16k).
- **Path-X (16K) và Path-256 (64K):** FlashAttention đạt 61.4 trên Path-X; block-sparse đạt 56.0 trên Path-X và 63.1 trên Path-256. Các Transformer khác trong bảng là chance/OOM. Đây là lần đầu Transformer vượt mức ngẫu nhiên.

- **Benchmark attention:** FlashAttention nhanh hơn tới ~3x so với PyTorch cho $N \in [128, 2K]$; bộ nhớ tuyến tính, tới 20x ít hơn các baseline chính xác; hội tụ với các phương pháp xấp xỉ ở N khoảng 512-1024 (xấp xỉ bắt đầu nhanh hơn từ đó); block-sparse nhanh hơn mọi baseline xấp xỉ biết được.

5. Phân tích / ablation

- **IO là biến quyết định thời gian** (Hình 2 trái): FLOPs cao hơn nhưng HBM thấp hơn 9x thì nhanh hơn rõ rệt.
- **Kích thước khối** (Hình 2 giữa): tăng B_c giảm số lượt đọc Q, giảm HBM và giảm runtime; vượt khoảng 256 thì bị nghẽn bởi tính toán, và khối quá lớn không vừa SRAM.
- **Độ thưa** (Hình 2 phải): runtime của block-sparse giảm tỉ lệ với độ thưa ($N = 4K$).
- **Độ ổn định số:** do không đổi định nghĩa mô hình, đường cong validation perplexity trùng với baseline (Phụ lục E).

6. Giới hạn

Do tác giả nêu:

- **Phải viết CUDA riêng cho mỗi biến thể attention**, tốn công, khó chuyển giữa các kiến trúc GPU; mong muốn có compiler từ ngôn ngữ cấp cao (kiểu Halide).
- IO-aware nên được áp dụng cho các module khác, chưa làm.
- Mới tối ưu trên một GPU; nhiều GPU thêm tầng IO giữa các GPU.

Do ta nhận thấy:

- Phân tích tối ưu là tiệm cận trên một dải M; không nói gì về hằng số, và thực tế GPU còn nhiều yếu tố khác (số warp, shared memory). Bài 45 chỉ ra FlashAttention v1 mới đạt 25-40% FLOPs lý thuyết.
- Toàn bộ benchmark tập trung huấn luyện/prefill với Q dài. Với giải mã (decode) từng token, Q chỉ có 1 hàng nên ma trận S vốn đã nhỏ ($1 \times N$) và lợi thế chính (tránh ghi $N \times N$) gần như biến mất. Paper không đánh giá decode.
- Không đề cập attention tuyến tính/recurrent (như GatedDeltaNet), nơi không có ma trận $N \times N$ để tránh.

7. Liên hệ với các bài khác

- **01 (Attention Is All You Need):** đây là cùng phép toán, chỉ đổi thứ tự tính toán.
- **17 (Longformer), 18 (Linear Transformers), 19-24:** những hướng giảm FLOPs bằng xấp xỉ/đổi cơ chế. Bài này cho thấy vì sao chúng cần kernel IO-aware mới nhanh thật (và các bài như 20, 24 đều có kernel chunk-wise kiểu tiling tương tự, cùng triết lý).
- **14-15 (MQA, GQA):** giảm KV cache khi decode; trực giao với FlashAttention (FA2 hỗ trợ GQA bằng chỉ số head).
- **45 (FlashAttention-2):** cùng thuật toán, cải thiện phân chia công việc để đạt ~2x.
- **46 (PagedAttention):** FlashAttention tối ưu *tính* attention; PagedAttention tối ưu *chỗ để* KV cache khi phục vụ nhiều request.
- **36-37 (InternVL, Qwen2-VL), 29 (ViT):** ViT có chuỗi patch dài, attention hai chiều đầy đủ; FlashAttention là cách thực tế để chạy.

8. Tóm tắt công thức / khái niệm

Khái niệm	Công thức / ý nghĩa
Attention	$O = \text{softmax}(QK^\top)V$

Khái niệm	Công thức / ý nghĩa
Softmax ổn định	$\text{softmax}(x) = f(x)/\ell(x)$, $f = e^{x-m}$
Ghép khối	$\ell = e^{m_1-m}\ell_1 + e^{m_2-m}\ell_2$
Cập nhật output	$o^{new} = \frac{1}{\ell^{new}}(\ell e^{m-m^{new}} o + e^{\tilde{m}-m^{new}} \tilde{P}V)$
Block size	$B_c = \lceil M/4d \rceil$, $B_r = \min(B_c, d)$
IO chuẩn	$\Theta(Nd + N^2)$
IO Flash	$\Theta(N^2d^2/M)$
Bộ nhớ thêm	$O(N)$ (m, ℓ)
FLOPs	$O(N^2d)$
Backward	$D_i = do_i^\top o_i$, $dS_{ij} = P_{ij}(dP_{ij} - D_i)$
Block-sparse IO	$\Theta(Nd + N^2d^2M^{-1}s)$
Recomputation	lưu O, m, ℓ; tính lại S, P theo khối

9. Kết luận ngắn

FlashAttention chuyển trọng tâm từ "giảm FLOPs" sang "giảm truy cập HBM" và chứng minh điều này bằng một thuật toán chính xác (tiling + online softmax + recomputation), có phân tích IO $\Theta(N^2d^2/M)$ và cận dưới. Kết quả: nhanh hơn 2-4x, bộ nhớ tuyến tính, cho phép ngữ cảnh dài.

Link gốc: https://arxiv.org/abs/2205.14135

Bài 45

FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning (2023)

Tác giả: Tri Dao.
Venue / arXiv: arXiv:2307.08691 (v1, 17/07/2023).
Nguồn: https://arxiv.org/abs/2307.08691
Chủ đề: Tối ưu kernel attention chính xác trên GPU: giảm FLOPs không phải matmul, song song theo chiều độ dài chuỗi, chia việc giữa các warp; GQA/MQA; causal masking theo khối.
Vị trí trong lộ trình: Đọc trước: 44 (FlashAttention, tiling + online softmax + IO complexity), 01. Bài này thêm: cùng thuật toán và cùng kết quả, nhưng khai thác phần cứng tốt hơn (~2x). Đọc tiếp: 46 (PagedAttention), 15 (GQA, vì FA2 hỗ trợ GQA/MQA), 48 (speculative decoding, nơi batch-1 decode có đặc điểm khác hẳn).

> Đây là ghi chú học tập bằng tiếng Việt, diễn giải lại chứ không phải bản dịch. Các con số lấy từ paper (có ghi rõ). Paper là loại trung bình (~7k từ), nhiều số liệu nằm trong biểu đồ nên chỉ trích các số xuất hiện trong chữ hoặc bảng.

1. Bối cảnh

FlashAttention (bài 44) giảm IO và mức sử dụng bộ nhớ, đạt tốc độ gấp 2–4 lần attention thông thường. Tuy vậy, trên A100, bản đầu chỉ đạt 30–50% thông lượng lý thuyết ở forward và 25–35% ở backward, trong khi phép nhân ma trận GEMM tối ưu đạt 80–90%. Qua phân tích hiệu năng, các tác giả cho rằng cách chia việc giữa thread block và warp còn chưa tốt: GPU chưa được sử dụng hết hoặc phát sinh đọc ghi shared memory dư thừa.

Thuật ngữ GPU:

- **Thread block:** nhóm thread chạy trên một SM (streaming multiprocessor); A100 có 108 SM.
- **Warp:** 32 thread chạy đồng bộ; thread trong warp trao đổi nhanh bằng shuffle; warp trong block trao đổi qua shared memory (SRAM), chậm hơn.
- **Occupancy:** mức độ tận dụng tài nguyên GPU. Ít thread block hơn số SM thì có SM bị bỏ không.
- **Tensor Core:** đơn vị chuyên nhân ma trận (fp16/bf16). A100: 312 TFLOPs/s cho matmul fp16/bf16 nhưng chỉ 19.5 TFLOPs/s cho FP32 không phải matmul, nghĩa là mỗi FLOP không phải matmul "đắt" gấp ~16 lần.
- **Split-K:** cách chia việc theo chiều rút gọn (K), các warp tính riêng rồi phải cộng lại qua shared memory.

2. Ý tưởng chính

Giữ nguyên kết quả toán học của FlashAttention nhưng làm ba việc: (1) sửa công thức online softmax để **giảm phép toán không phải matmul** (vì chúng đắt gấp 16 lần), (2) **đảo vòng lặp** và song song hoá cả theo chiều độ dài chuỗi (không chỉ batch và head) để lấp đầy GPU khi chuỗi dài, batch nhỏ, (3) trong mỗi thread block, **chia Q giữa các warp thay vì chia K, V** để các warp khỏi phải giao tiếp qua shared memory. Kết quả khoảng 2x nhanh hơn FlashAttention, đạt 50-73% thông lượng đỉnh trên A100.

3. Phương pháp chi tiết

3.1 Tối ưu 1: bớt FLOPs không phải matmul

Online softmax của FlashAttention (bài 44) với hai khối (xét một hàng, $S = [S^{(1)}, S^{(2)}]$):

$$O^{(2)} = \text{diag}\big(\ell^{(1)}/\ell^{(2)}\big)^{-1} O^{(1)} + \text{diag}(\ell^{(2)})^{-1} e^{S^{(2)} - m^{(2)}} V^{(2)}.$$

Mỗi vòng đều phải chia (rescale) bằng $\text{diag}(\ell)^{-1}$ cho cả hai số hạng. FA2 nhận ra không cần: giữ $\tilde{O}$ **chưa chuẩn hoá**:

$$\tilde{O}^{(1)} = e^{S^{(1)} - m^{(1)}} V^{(1)}, \qquad \tilde{O}^{(2)} = \text{diag}\big(e^{m^{(1)} - m^{(2)}}\big) \tilde{O}^{(1)} + e^{S^{(2)} - m^{(2)}} V^{(2)},$$

và chỉ chia **một lần ở cuối vòng lặp**:

$$\boxed{O = \text{diag}\big(\ell^{(\text{last})}\big)^{-1} \tilde{O}^{(\text{last})}}.$$

Tối ưu thứ hai: backward chỉ cần **logsumexp** $L = m + \log \ell$ cho mỗi hàng thay vì lưu cả m và ℓ. Khi đó $P_{ij} = \exp(S_{ij} - L_i)$ tính lại trực tiếp. Chứng minh: $\exp(S - m)/\ell = \exp(S - m - \log \ell) = \exp(S - L)$.

Thuật toán forward của FA2 (Algorithm 1), **vòng ngoài là khối hàng Q_i, vòng trong là khối cột K_j, V_j**:

```
for i = 1..Tr:                              # mỗi khối hàng Q_i (song song giữa các
↪  thread block)
    nạp Q_i vào SRAM
    O~ = 0, l = 0, m = -inf                 # trên chip
    for j = 1..Tc:
        nạp K_j, V_j vào SRAM
        S    = Q_i K_j^T
        m_new = max(m, rowmax(S))
        P~   = exp(S - m_new)
        l    = exp(m - m_new) * l + rowsum(P~)
        O~   = diag(exp(m - m_new)) * O~ + P~ V_j     # KHÔNG chia cho l ở đây
        m    = m_new
    O_i = diag(l)^-1 * O~                   # chia một lần
    L_i = m + log(l)
    ghi O_i, L_i về HBM
```

So với FA1 (bài 44): O, ℓ, m không còn bị đọc/ghi về HBM ở mỗi vòng trong; mỗi khối hàng ghi đúng một lần. Paper nói thứ tự vòng lặp này (ngoài là hàng, trong là cột) đã được Phil Tillet gợi ý và cài trong Triton trước.

Độ đúng và chi phí không đổi: kết quả chính xác, $O(N^2 d)$ FLOPs, thêm $O(N)$ bộ nhớ (lưu L).

3.2 Tối ưu 2: song song theo chiều độ dài chuỗi

FA1 dùng **1 thread block cho mỗi (batch, head)**, tổng batch×heads block. Hiệu quả khi con số này lớn (paper nói ≥ 80, vì A100 có 108 SM). Nhưng với chuỗi dài thường batch nhỏ, head ít, nên nhiều SM bỏ không.

- **Forward:** vòng ngoài (theo khối hàng của Q) là "embarrassingly parallel", mỗi khối hàng giao cho một thread block riêng, không cần giao tiếp. Thêm vào song song theo batch và head như cũ.
- **Backward:** các khối cột chỉ chia sẻ phép cập nhật dQ (Algorithm 2: $dQ_i \leftarrow dQ_i + dS_i^{(j)} K_j$). Giao mỗi khối cột cho một thread block và dùng **atomic add** để các block cộng dồn vào dQ.

Backward (Algorithm 2) giữ công thức của bài 44: $P = \exp(S - L)$, $dV_j \mathrel{+}= P^\top dO_i$, $dP = dO_i V_j^\top$, $dS = P \circ (dP - D_i)$ với $D = \text{rowsum}(dO \circ O)$, $dK_j \mathrel{+}= dS^\top Q_i$.

3.3 Tối ưu 3: chia việc giữa các warp

Mỗi thread block thường có 4 hoặc 8 warp.

- **FA1 ("split-K"):** chia K, V cho 4 warp, mọi warp thấy Q. Mỗi warp tính một lát $QK^\top$, rồi nhân với lát V tương ứng, nhưng kết quả phải **cộng lại giữa các warp**: mỗi warp ghi kết quả trung gian vào shared memory, đồng bộ, rồi cộng. Các lần đọc/ghi shared memory này làm chậm.
- **FA2:** chia Q cho 4 warp, mọi warp thấy K, V. Mỗi warp tính hàng của nó với toàn bộ K, V và tự ra lát output của mình, **không cần giao tiếp**.

Backward cũng tránh split-K nhưng vẫn cần một số đồng bộ vì phụ thuộc phức tạp giữa $Q, K, V, O, dO, dQ, dK, dV$ (backward có 5 phép matmul so với 2 của forward).

3.4 Causal mask, MQA/GQA, block size

- **Causal masking:** vì đã chia khối, các khối có toàn bộ chỉ số cột lớn hơn chỉ số hàng (khoảng nửa số khối khi chuỗi dài) được **bỏ qua hoàn toàn**, cho tốc độ khoảng 1.7-1.8x so với không mask. Các khối có chỉ số hàng chắc chắn nhỏ hơn cột thì không cần áp mask; mỗi hàng chỉ cần áp mask ở một khối (nếu khối vuông).
- **MQA/GQA (bài 14-15):** thay vì nhân đôi K, V cho các query head dùng chung, FA2 **thao tác chỉ số head ngầm** để dùng lại. Backward phải cộng dồn dK, dV qua các head chia sẻ.
- **Block size:** thường chọn $\{64, 128\} \times \{64, 128\}$ tuỳ head dim và shared memory. Khối lớn giảm đọc/ghi shared memory nhưng cần nhiều register hơn; vượt ngưỡng thì register spilling hoặc không đủ shared memory để chạy. Phải chỉnh tay cho từng head dim (chỉ có ~4 lựa chọn).

3.5 Phân tích chi phí

Công thức FLOPs dùng trong paper:

- Forward: $4 \cdot \text{seqlen}^2 \cdot \text{head dim} \cdot \text{số head}$ (hai matmul, mỗi cái $2N^2d$); chia 2 nếu causal.
- Backward: $2.5\times$ forward (5 matmul so với 2, vì có tính lại).
- Huấn luyện end-to-end: $6 \cdot \text{seqlen} \cdot \text{số tham số} + 12 \cdot \text{số layer} \cdot \text{hidden} \cdot \text{seqlen}^2$ (kiểu Megatron-LM, không chia 2 cho causal).

Về occupancy (suy luận từ Mục 3.2): với decode batch 1, 16 head thì chỉ có $1 \times 16 = 16$ thread block cho 108 SM (A100) nếu chỉ song song theo (batch, head); độ dài query là 1 nên cũng không thể song song thêm theo chiều query. Đây là lý do cộng đồng sau này có kỹ thuật chia theo chiều khoá/giá trị (split-KV, thường gọi Flash-Decoding), nhưng điều này **không nằm trong paper này**; ghi chú để tra cứu.

4. Thí nghiệm và kết quả chính

Thiết lập benchmark: A100 80GB SXM4; độ dài chuỗi 512, 1k, ..., 16k, batch sao cho tổng 16k token; hidden 2048, head dim 64 (32 head) hoặc 128 (16 head); với và không có causal mask.

Các kết luận trong chữ của paper:

Nội dung	Kết quả
FA2 vs FlashAttention (v1)	nhanh hơn 1.7-3.0x
FA2 vs FlashAttention trong Triton	1.3-2.5x (forward 1.3-1.5x, backward ~2x)
FA2 vs attention chuẩn PyTorch	3-10x (tới 10x ở một số cấu hình)
Thông lượng đỉnh	tới 230 TFLOPs/s (73% lý thuyết) forward; backward tới 63%
H100 (chạy cùng code, chưa dùng TMA/Tensor Core thế hệ 4)	tới 335 TFLOPs/s
Phần trăm đỉnh của FA1 (so sánh)	forward 30-50%, backward 25-35%

Huấn luyện end-to-end (Bảng 1 của paper), GPT-style, 8×A100 80GB, TFLOPs/s mỗi GPU:

Mô hình	Không FlashAttention	FlashAttention	FlashAttention-2
GPT3-1.3B, ctx 2k	142	189	196
GPT3-1.3B, ctx 8k	72	170	220
GPT3-2.7B, ctx 2k	149	189	205
GPT3-2.7B, ctx 8k	80	175	225

Paper tóm: tới 2.8x so với không dùng FlashAttention, 1.3x so với FlashAttention v1, tới 225 TFLOPs/s mỗi GPU A100 (72% MFU, "model FLOPs utilization", tỉ lệ FLOPs hữu dụng của mô hình trên đỉnh phần cứng). Kết luận thực tế mà tác giả nêu: cùng chi phí huấn luyện, có thể dùng ngữ cảnh 16k thay vì 8k.

5. Phân tích / ablation

Paper **không có bảng ablation riêng** từng tối ưu. Mức đóng góp của ba tối ưu chỉ được lập luận qua phân tích và qua kết quả tổng. Những gì paper nêu cụ thể:

- Causal mask bỏ qua khối cho speedup ~1.7-1.8x so với không mask.
- Block size lớn hơn giảm truy cập shared memory nhưng tốn register; quá ngưỡng thì chậm hoặc không chạy.
- FLOPs không phải matmul chiếm tỉ lệ nhỏ nhưng tốn thời gian nhiều vì thông lượng ~16 lần thấp hơn.

Ta nhận xét (không phải paper): độ lệch giữa speedup do thuật toán (giảm non-matmul) và do lập lịch (song song hoá, chia warp) không được tách riêng, nên khó biết yếu tố nào quan trọng nhất cho từng cấu hình.

6. Giới hạn

Do tác giả nêu:

- Mới tối ưu cho A100; trên H100 chưa dùng lệnh mới (TMA, Tensor Core thế hệ 4, FP8), kỳ vọng còn 1.5-2x nữa.
- Block size phải chỉnh tay cho từng head dim; mong muốn auto-tuning.
- Cần hợp tác với compiler để dễ lập trình, kết hợp với attention thưa/cục bộ cho ngữ cảnh rất dài.

Do ta nhận thấy:

- Toàn bộ benchmark là chuỗi dài, nhiều token cùng lúc (huấn luyện/prefill). Không có số liệu về decode với query dài 1, mà đó mới là chế độ chiếm phần lớn thời gian ở suy luận batch 1.
- Chỉ so sánh trên NVIDIA GPU datacenter (A100/H100); GPU tiêu dùng ≤ 8 GB (kiến trúc khác, SRAM và băng thông khác) không được đo.
- Mô hình chỉ GPT-style kích thước 1.3B-2.7B trong thí nghiệm end-to-end.

7. Liên hệ với các bài khác

- **44 (FlashAttention):** cùng thuật toán; FA2 chỉ đổi cách chạy trên GPU. Công thức IO không đổi.
- **14, 15 (MQA, GQA):** FA2 hỗ trợ trực tiếp bằng chỉ số head; hữu ích khi GQA giảm KV cache.
- **46, 47 (vLLM, SGLang):** các hệ thống phục vụ gọi kernel FlashAttention (hoặc FlashInfer) cho phần prefill; PagedAttention có kernel riêng cho decode.
- **48 (speculative decoding):** verify $\gamma + 1$ token cùng lúc biến decode từ query dài 1 thành query dài $\gamma + 1$, gần với chế độ mà FA2 tối ưu hơn một chút.
- **20, 23, 24 (GLA, DeltaNet, Gated DeltaNet):** các kernel chunk-wise của họ cùng tinh thần "tiling + giữ trạng thái trên chip + song song theo chiều chuỗi". Không phải trích dẫn của paper; đây là điểm tương đồng ta nhận ra.

8. Tóm tắt công thức / khái niệm

Khái niệm	Công thức / ý nghĩa
Output không chuẩn hoá	$\tilde{O}^{(j)} = \text{diag}(e^{m^{(j-1)}-m^{(j)}})\tilde{O}^{(j-1)} + e^{S^{(j)}-m^{(j)}}V^{(j)}$
Chuẩn hoá một lần	$O = \text{diag}(\ell^{(last)})^{-1}\tilde{O}^{(last)}$
Logsumexp	$L = m + \log \ell$, $P = \exp(S - L)$
FLOPs forward	$4\,N^2\,d\,H$ (chia 2 nếu causal)
FLOPs backward	$2.5\times$ forward
FLOPs huấn luyện	$6\,N$ #params $+ 12$ #layer $\cdot$ hidden $\cdot\, N^2$
Matmul vs non-matmul (A100)	312 vs 19.5 TFLOPs/s, ~16x
Song song	FA1: batch×head; FA2: thêm chiều độ dài chuỗi
Warp	FA1: split-K (chia K, V); FA2: chia Q
Backward song song	theo khối cột, atomic add vào dQ

9. Kết luận ngắn

FlashAttention-2 không đổi thuật toán toán học mà đổi cách ánh xạ vào GPU: ít phép non-matmul hơn, song song cả theo chiều chuỗi, chia Q giữa warp. Kết quả ~2x so với v1, đạt tới 73% đỉnh trên A100 và 225 TFLOPs/s huấn luyện end-to-end.

Link gốc: https://arxiv.org/abs/2307.08691

Bài 46

Efficient Memory Management for Large Language Model Serving with PagedAttention (2023)

Tác giả: Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, Ion Stoica.
Venue / arXiv: arXiv:2309.06180 (v1, 12/09/2023); SOSP '23 (theo trang bản quyền trong paper).
Nguồn: https://arxiv.org/abs/2309.06180
Chủ đề: Quản lý bộ nhớ KV cache khi phục vụ LLM nhiều request (serving); PagedAttention; vLLM; continuous batching; chia sẻ KV cache (parallel sampling, beam search, shared prefix).
Vị trí trong lộ trình: Đọc trước: 01 (self-attention, KV cache), 44-45 (FlashAttention: tính attention nhanh), 14-16 (MQA/GQA/MLA: giảm kích thước KV cache). Bài này thêm: cách **lưu và cấp phát** KV cache để nhồi được nhiều request hơn vào GPU. Đọc tiếp: 47 (SGLang/RadixAttention, tái sử dụng KV theo tiền tố), 55-56 (StreamingLLM, H2O: nén/loại bỏ KV cache).

> Đây là ghi chú học tập bằng tiếng Việt, diễn giải lại chứ không phải bản dịch. Số liệu lấy từ paper; nhiều biểu đồ bị mất nhãn khi trích PDF nên chỉ dùng số xuất hiện trong chữ. Paper dài vừa (~13k từ); ghi chú tập trung vào cơ chế cốt lõi và kết quả chính.

1. Bối cảnh

Chi phí phục vụ LLM có thể rất lớn: bài báo dẫn một ước tính rằng một yêu cầu LLM có thể đắt gấp mười lần một truy vấn từ khóa. Một cách giảm chi phí là gom nhiều yêu cầu vào batch, để chia sẻ chi phí đọc cùng bộ trọng số.

Khái niệm nền:

- **Hai pha sinh văn bản.** *Prompt (prefill):* xử lý toàn bộ prompt song song bằng phép nhân ma trận-ma trận, dùng GPU hiệu quả. *Autoregressive generation (decode):* sinh từng token một, mỗi bước chỉ có 1 token vào, dùng nhân ma trận-vector, bị nghẽn bởi bộ nhớ (memory-bound), chiếm phần lớn latency của một request.
- **KV cache:** các vector key k_i và value v_i của mọi token đã xử lý, lưu lại để các bước sau không phải tính lại. Dùng cho attention $a_{ij} = \frac{\exp(q_i^\top k_j/\sqrt{d})}{\sum_t \exp(q_i^\top k_t/\sqrt{d})}$, $o_i = \sum_j a_{ij} v_j$.
- **Continuous (iteration-level) batching:** thay vì chờ cả batch xong, sau **mỗi bước** đưa request xong ra và nhận request mới vào (Orca, cellular batching).

Vấn đề: số request đưa được vào batch bị **giới hạn bởi bộ nhớ chứa KV cache**, tức thông lượng bị nghẽn bởi bộ nhớ. Ví dụ paper (Hình 1): mô hình 13B trên A100 40GB, khoảng 65% bộ nhớ là trọng số, gần 30% là KV cache, phần còn lại là activation.

Các hệ thống trước (FasterTransformer, Orca) lưu KV cache của mỗi request trong **một khối liền mạch** (vì framework đòi tensor liền mạch) với kích thước **tối đa** có thể (ví dụ 2048 token), dù độ dài thật chưa biết. Kết quả: chỉ 20.4%-38.2% bộ nhớ KV chứa token thật (Hình 2 của paper).

2. Ý tưởng chính

Mượn **bộ nhớ ảo + phân trang (paging)** của hệ điều hành: chia KV cache của mỗi chuỗi thành **khối (block) kích thước cố định** (ví dụ 16 token). Các khối **không cần nằm liền nhau** trong bộ nhớ vật lý; một **block table** ánh xạ khối logic của chuỗi sang khối vật lý, giống bảng trang. Khối được cấp phát **theo nhu cầu**, nên lãng phí tối đa dưới một khối cho mỗi chuỗi. Vì khối giữ nguyên kích thước nên hết phân mảnh ngoài, và

vì ánh xạ là gián tiếp nên nhiều chuỗi có thể **trỏ cùng một khối vật lý** (chia sẻ prompt, beam search, tiền tố chung) với cơ chế **copy-on-write**. PagedAttention là kernel attention đọc KV qua block table; vLLM là hệ thống phục vụ xây trên đó. Kết quả: thông lượng cao hơn 2-4x ở cùng mức latency, không đổi đầu ra.

3. Phương pháp chi tiết

3.1 KV cache và công thức kích thước

Tổng quát, KV cache cho một token:

$$\boxed{\text{bytes}_{\text{token}} = 2\ (K, V) \times n_{kv}\, d_{head} \times L_{\text{attn}} \times \text{bytes per element}}$$

Ví dụ của paper (OPT-13B): 2×5120 (hidden) $\times\ 40$ (layer) $\times\ 2$ (fp16) $= 800$ KB mỗi token. Một request dài tối đa 2048 token cần tới 1.6 GB. Bộ nhớ GPU chỉ vài chục GB nên chỉ nhồi được vài chục request. Paper lưu ý tốc độ tính toán GPU tăng nhanh hơn dung lượng bộ nhớ (A100 sang H100: FLOPS tăng hơn 2x nhưng bộ nhớ vẫn tối đa 80 GB), nên bộ nhớ ngày càng là nút thắt.

3.2 Ba nguồn lãng phí bộ nhớ

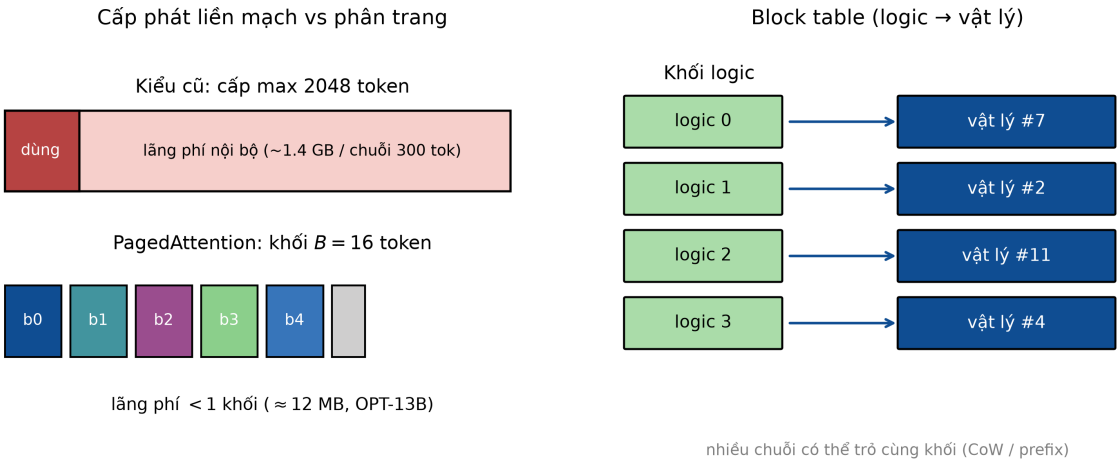
Khi cấp phát trước một khối liền mạch theo độ dài tối đa:

1. **Reserved (dự trữ):** chỗ cho token tương lai, sẽ được dùng nhưng chiếm chỗ suốt đời request, các request khác không dùng được.
2. **Internal fragmentation (phân mảnh trong):** cấp phát theo mức tối đa nhưng độ dài thật ngắn hơn; phần thừa không bao giờ dùng.
3. **External fragmentation (phân mảnh ngoài):** khoảng trống giữa các khối do bộ cấp phát (ví dụ buddy allocator) để lại.

Nén lại (compaction) không khả thi vì KV cache quá lớn. Ngoài ra, khối liền mạch riêng cho từng chuỗi ngăn việc chia sẻ KV giữa các chuỗi trong cùng một request (parallel sampling, beam search).

3.3 PagedAttention

Chia KV của chuỗi thành khối B token: $K_j = (k_{(j-1)B+1}, \ldots, k_{jB})$, V_j tương tự. Attention viết lại theo khối:



PagedAttention: cấp phát khối $B{=}16$ token + block table logic → vật lý; giảm lãng phí so với cấp max liền mạch. (minh hoạ, tự tính)

$$A_{ij} = \frac{\exp(q_i^\top K_j / \sqrt{d})}{\sum_{t=1}^{\lceil i/B \rceil} \exp(q_i^\top K_t \mathbf{1} / \sqrt{d})}, \qquad o_i = \sum_{j=1}^{\lceil i/B \rceil} V_j A_{ij}^\top,$$

với A_{ij} là vector hàng điểm attention trên khối j. Về mặt toán học **giống hệt** attention thường; điểm khác là kernel lấy từng khối key/value từ địa chỉ vật lý bất kỳ (qua block table), tính điểm với khối đó, rồi nhân với khối value. Ví dụ Hình 5: truy vấn "forth" đọc 3 khối nằm rải rác trong bộ nhớ.

Paper không nói rõ cách gộp softmax giữa các khối trong kernel (xem Giới hạn); về mặt thực hành nó cùng kiểu online softmax như bài 44, nhưng điều đó ta suy ra chứ paper không trình bày.

3.4 KV cache manager và block table

- Khối **logic** của một chuỗi được lấp từ trái sang phải; khối cuối còn chỗ trống dự trữ cho token sau.
- Trên GPU worker, *block engine* cấp một vùng DRAM liền mạch lớn và chia thành **khối vật lý** (cũng có vùng tương tự trên RAM CPU để swap).
- **Block table** cho mỗi chuỗi: danh sách (khối vật lý, số vị trí đã đầy).

Ví dụ (Hình 6): prompt 7 token, $B = 4$. Khối logic 0 và 1 ánh xạ tới khối vật lý 7 và 1; khối 1 có 3/4 chỗ đầy. Bước decode đầu tiên ghi token mới vào chỗ trống còn lại. Bước tiếp theo khối logic 1 đã đầy, nên cấp phát khối vật lý mới (khối 3) và thêm một dòng vào bảng.

Thuật toán mỗi vòng lặp (giả mã theo mô tả §4.3 của paper):

```
mỗi iteration:
    chọn tập chuỗi ứng viên để batch (FCFS)       # §4.5
    với mỗi chuỗi cần khối logic mới:
        cấp phát khối vật lý, thêm vào block table
        nếu hết khối: preempt chuỗi đến sau cùng (swap hoặc recompute)
    ghép token đầu vào của mọi chuỗi (prompt và decode) thành một "chuỗi" duy nhất
    chạy mô hình; kernel PagedAttention đọc KV theo block table
    ghi KV mới vào khối vật lý đúng chỗ
    chuỗi xong -> giải phóng khối
```

Lãng phí chỉ xảy ra ở **khối cuối** của mỗi chuỗi, nên nhỏ hơn 1 khối/chuỗi. Paper báo cáo gần như không lãng phí (Hình 2 của paper cho vLLM; vì nhãn biểu đồ bị lẫn khi trích, ta chỉ khẳng định "gần 0" như tác giả nói).

3.5 Chia sẻ: parallel sampling, beam search, shared prefix

- **Parallel sampling** (nhiều mẫu từ một prompt): các mẫu trỏ chung khối vật lý của prompt; mỗi khối vật lý có **reference count**. Khi một mẫu cần ghi vào khối có count > 1 thì **copy-on-write**: cấp khối mới, sao chép nội dung, giảm count ở khối cũ. Tất cả khối của prompt được chia sẻ, trừ khối cuối (CoW).
- **Beam search:** các beam chia sẻ cả phần prompt lẫn phần sinh ra ban đầu; nhánh bị loại thì giải phóng khối khi count về 0. Hệ thống tránh sao chép KV cache như các cài đặt cũ (phải copy cho mỗi beam), chỉ copy một khối khi cần.
- **Shared prefix:** dịch vụ có thể đặt trước KV của tiền tố chung (ví dụ ví dụ few-shot, system prompt); request mới ánh xạ khối logic vào các khối đã cache (khối cuối đánh dấu CoW). Chỉ phần đầu vào riêng của người dùng phải tính prefill. Đây là tiền thân của RadixAttention (bài 47), nhưng ở vLLM phải khai báo tiền tố trước.
- **Mixed decoding:** lớp ánh xạ ẩn mô hình sharing, nên các request có kiểu giải mã khác nhau vẫn batch chung được. Các hàm cài đặt: `fork`, `append`, `free`.

3.6 Lập lịch và preemption

FCFS, preempt chuỗi đến sau cùng khi hết khối GPU. Hai câu hỏi cổ điển:

- **Loại khối nào?** Vì mọi khối của một chuỗi được truy cập cùng nhau, vLLM dùng **all-or-nothing**: loại hết hoặc không loại; các chuỗi cùng một request (ví dụ các beam) là một *sequence group*, được preempt/lên lịch lại cùng nhau.
- **Khôi phục thế nào?** (a) *Swapping:* chép khối sang RAM CPU (số khối swap không vượt tổng số khối GPU). (b) *Recomputation:* tính lại KV khi chuỗi được lên lịch lại, bằng cách ghép token đã sinh vào prompt và chạy một lượt prefill; không có tương đương trong OS.

3.7 Kernel, phân tán, cài đặt

- Ba kernel hợp nhất: (1) ghi KV mới vào khối (reshape + ghi), (2) đọc khối và attention (dựa trên kernel FasterTransformer, mỗi warp đọc một khối để truy cập liên tục), (3) sao chép khối hàng loạt cho CoW (một kernel thay cho nhiều `cudaMemcpyAsync`).
- Phân tán kiểu Megatron-LM tensor parallel: mọi shard dùng chung một KV manager và cùng block table; mỗi worker chỉ lưu KV cho các head của mình.
- Cài đặt: 8.5K dòng Python + 2K dòng C++/CUDA; hỗ trợ GPT, OPT, LLaMA.

3.8 Phân tích chi phí

Lãng phí tối đa theo định nghĩa: với khối B token, mỗi chuỗi lãng phí tối đa $B-1$ vị trí (khối cuối), tức $\leq (B-1) \times \text{bytes}_{\text{token}}$. Cấp phát trước kiểu cũ lãng phí tới $L_{\text{max}} - L_{\text{thật}}$ vị trí.

Ví dụ (OPT-13B của paper, tự tính từ số trong paper): $B = 16$: lãng phí tối đa 15×800 KB ≈ 12 MB/chuỗi; cấp phát tối đa 2048 token là 1.6 GB/chuỗi. Với chuỗi thật 300 token: kiểu cũ lãng phí $\approx 1748 \times 0.8$ MB ≈ 1.4 GB so với < 12 MB.

Chi phí của chỉ mục gián tiếp: kernel PagedAttention có thêm truy cập block table, nhánh điều kiện và xử lý độ dài biến thiên: paper đo **kernel attention chậm hơn 20-26%** so với FasterTransformer, nhưng chỉ ảnh hưởng toán tử attention nên tổng thể vẫn thắng.

4. Thí nghiệm và kết quả chính

Thiết lập: OPT 13B/66B/175B và LLaMA-13B; GPU A100 trên Google Cloud; workload tổng hợp từ ShareGPT (đầu vào dài hơn 8.4x và đầu ra dài hơn 5.8x so với Alpaca) và Alpaca, đến theo phân phối Poisson. Chỉ số: độ trễ chuẩn hoá (end-to-end latency chia độ dài đầu ra, s/token); hệ thống tốt là hệ thống giữ độ trễ thấp ở request rate cao. Baseline: FasterTransformer (có bộ lập lịch dynamic batching tự viết) và ba biến thể Orca (Oracle biết trước độ dài đầu ra; Pow2 dự trữ làm tròn lên luỹ thừa của 2; Max dự trữ 2048).

Cấu hình (Bảng 1):

	13B	66B	175B
GPU	A100	4×A100	8×A100-80GB
Bộ nhớ GPU tổng	40 GB	160 GB	640 GB
Tham số	26 GB	132 GB	346 GB
Bộ nhớ cho KV	12 GB	21 GB	264 GB
Số slot KV tối đa	15.7K	9.7K	60.1K

Kết quả chính (từ chữ của paper):

Nội dung	Kết quả
ShareGPT, request rate duy trì được	1.7-2.7x so với Orca (Oracle); 2.7-8x so với Orca (Max)
So với FasterTransformer	tới 22x

Nội dung	Kết quả
OPT-13B ShareGPT, số request chạy cùng lúc	2.2x so với Orca (Oracle); 4.3x so với Orca (Max)
Parallel sampling / beam search (OPT-13B, Alpaca)	vượt Orca (Oracle) từ 1.3x (lấy mẫu cơ bản) tới 2.3x (beam search, độ rộng 6)
Tiết kiệm bộ nhớ nhờ chia sẻ, Alpaca	parallel sampling 6.1%-9.8%; beam search 37.6%-55.2%
Tiết kiệm bộ nhớ nhờ chia sẻ, ShareGPT	parallel sampling 16.2%-30.5%; beam search 44.3%-66.3%
Shared prefix (LLaMA-13B, dịch WMT16 En-De)	1.67x (1-shot, 80 token) và 3.58x (5-shot, 341 token) so với Orca (Oracle)
Chatbot (OPT-13B, cắt prompt 1024 token)	duy trì request rate cao gấp 2x so với ba Orca

Lưu ý: với Alpaca + OPT-175B, lợi thế của vLLM nhỏ hơn vì cấu hình có rất nhiều bộ nhớ KV mà chuỗi lại ngắn, nên hệ thống thiên về bị nghẽn bởi tính toán (compute-bound), không phải bộ nhớ.

5. Phân tích / ablation

- **Kernel (7.1):** chỉ mục gián tiếp làm kernel attention chậm hơn 20-26%, không ảnh hưởng toán tử khác như Linear.
- **Block size (7.2):** quá nhỏ thì không dùng hết song song của GPU khi đọc KV; quá lớn thì phân mảnh trong tăng và xác suất chia sẻ giảm. ShareGPT: 16-128 là tốt nhất; Alpaca: 16 và 32 tốt, lớn hơn kém đi vì chuỗi ngắn hơn khối. **Mặc định 16.**
- **Swap vs recompute (7.3):** swap tốn kém khi khối nhỏ (nhiều lần chép nhỏ qua PCIe); recompute có chi phí không phụ thuộc kích thước khối và không bao giờ vượt 20% độ trễ của swap. Khối cỡ vừa 16-64: hai cách tương đương về hiệu năng end-to-end.
- Paper không có ablation riêng cho từng thành phần của vLLM (ví dụ tắt chia sẻ) ngoài các so sánh trên.

6. Giới hạn

Do tác giả nêu:

- Kỹ thuật phù hợp khi bộ nhớ là nút thắt và cấp phát động, như serving LLM; **không phù hợp** huấn luyện (hình dạng tensor tĩnh) hay phục vụ DNN không phải LLM (compute-bound, chỉ thêm overhead gián tiếp).
- Kernel bị overhead 20-26%.

Do ta nhận thấy:

- Lợi ích đo ở chế độ **thông lượng với nhiều request đồng thời**; tác động lên latency của **một request độc lập** (batch 1) không được đo và về nguyên tắc gần bằng 0 (thậm chí âm do overhead kernel).
- Mô hình thí nghiệm là OPT/LLaMA full attention, KV cache lớn. Không xét kiến trúc hồi quy/lai có trạng thái cố định.
- FCFS có thể bất lợi cho request dài; không có phân tích công bằng.
- Paper không mô tả chi tiết cách kernel kết hợp softmax qua các khối (công thức viết theo mẫu số tổng quát).

7. Liên hệ với các bài khác

- **44, 45 (FlashAttention):** FlashAttention tối ưu *cách tính* (IO) khi KV liền mạch; PagedAttention tối ưu *cách lưu*. Chúng bổ sung nhau; thực tế vLLM dùng kernel khác nhau cho prefill và decode.
- **14, 15, 16 (MQA, GQA, MLA):** giảm n_{kv} trong công thức kích thước, tức giảm KV theo token; PagedAttention giảm lãng phí theo cấp phát. Hai trục độc lập, nhân vào nhau.

- **47 (SGLang):** RadixAttention là bước tổng quát hoá của "shared prefix" ở đây thành cây cơ số tự động với LRU; SGLang xây trên bộ nhớ phân trang kiểu này (kích thước trang 1 token).
- **18-24, 26 (attention tuyến tính, hồi quy):** trạng thái cố định kích thước, không cần phân trang theo độ dài, nhưng khó chia sẻ tiền tố.
- **55, 56 (StreamingLLM, H2O):** giảm số token cần giữ; ghép với cấp phát theo khối.
- **48 (speculative decoding):** các token nháp bị từ chối phải thu hồi KV; với quản lý theo khối điều này là giải phóng/ghi đè khối.

8. Tóm tắt công thức / khái niệm

Khái niệm	Ý nghĩa / công thức
KV cache theo token	$2\, n_{kv} d_{head} L_{attn}$ · bytes (OPT-13B: 800 KB)
Block (trang)	B token, mặc định 16
Block table	ánh xạ khối logic → khối vật lý + số vị trí đã đầy
Lãng phí tối đa	$< B$ vị trí mỗi chuỗi
Attention theo khối	$o_i = \sum_j V_j A_{ij}^\top$, $A_{ij} \propto \exp(q_i^\top K_j / \sqrt{d})$
Copy-on-write	đếm tham chiếu; ghi vào khối có count > 1 thì sao chép
Sequence group	các chuỗi cùng request, preempt cùng nhau
Swap vs recompute	chép sang CPU vs tính lại bằng một lượt prefill
Overhead kernel	+20-26% thời gian attention
Kết quả chính	thông lượng 2-4x cùng độ trễ

9. Kết luận ngắn

PagedAttention đưa ý tưởng bộ nhớ ảo vào KV cache: khối kích thước cố định, cấp phát theo nhu cầu, ánh xạ gián tiếp, chia sẻ copy-on-write. Nhờ đó gần như không lãng phí bộ nhớ, batch lớn hơn, thông lượng tăng 2-4x.

Link gốc: https://arxiv.org/abs/2309.06180

Bài 47

SGLang: Efficient Execution of Structured Language Model Programs (2023/2024)

Tác giả: Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Sun, Jeff Huang, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E. Gonzalez, Clark Barrett, Ying Sheng.
Venue / arXiv: arXiv:2312.07104 (v1 12/2023; bản v2 06/2024, ghi "Preprint. Under review").
Nguồn: https://arxiv.org/abs/2312.07104
Chủ đề: Hệ thống chạy "chương trình LLM" gồm nhiều lần gọi mô hình; RadixAttention (tái sử dụng KV cache theo tiền tố bằng cây cơ số + LRU + lập lịch nhận biết cache); giải mã có ràng buộc bằng FSM nén; speculative execution cho API.
Vị trí trong lộ trình: Đọc trước: 46 (PagedAttention/vLLM, KV cache phân trang, shared prefix thủ công), 44-45 (FlashAttention). Bài này thêm: tái sử dụng KV cache **tự động** giữa các lần gọi, và cách lập lịch để tăng tỉ lệ trúng cache. Đọc tiếp: 48 (speculative decoding, một cách khác để giảm số bước chạy mô hình), 55-56 (quản lý/loại bỏ KV).

> Đây là ghi chú học tập bằng tiếng Việt, diễn giải lại chứ không phải bản dịch. Số liệu lấy từ paper; nhiều biểu đồ bị mất nhãn khi trích PDF nên chỉ dùng số xuất hiện trong chữ hoặc bảng. Paper dài vừa (~12k từ); ghi chú tập trung vào RadixAttention (phần liên quan nhất tới suy luận), nén FSM ở mức vừa phải, phần ngôn ngữ lập trình lướt qua.

1. Bối cảnh

Nhiều ứng dụng kết hợp các lần gọi LLM với điều khiển luồng và dữ liệu có cấu trúc; bài báo gọi chúng là chương trình LLM (LM program). Ví dụ gồm ReAct, tree-of-thought, self-consistency, few-shot, RAG và hội thoại nhiều lượt. Tác giả nêu hai khó khăn:

1. **Lập trình khó** (xử lý chuỗi thủ công, parse đầu ra mong manh, song song hoá).
2. **Thực thi kém hiệu quả:** các engine như vLLM, TGI, TensorRT-LLM tối ưu độ trễ/thông lượng mà không biết cấu trúc workload. Ví dụ rõ nhất là **KV cache bị bỏ sau mỗi request** dù nhiều lần gọi chia sẻ cùng tiền tố (system prompt, ví dụ few-shot, lịch sử chat). Một ví dụ khác: giải mã ràng buộc (JSON) vẫn sinh từng token một dù nhiều đoạn chỉ có một lựa chọn hợp lệ.

Thuật ngữ:

- **Tiền tố (prefix):** phần đầu chung của hai chuỗi token. KV cache của một token chỉ phụ thuộc các token **trước** nó, nên hai request có cùng tiền tố có cùng KV cho phần tiền tố đó.
- **Cây tiền tố (trie) / cây cơ số (radix tree):** cấu trúc lưu tập chuỗi theo tiền tố chung. Cây cơ số là trie nén: một cạnh mang nhãn là một dãy phần tử (không chỉ một).
- **LRU:** loại phần tử ít được dùng gần đây nhất.
- **FSM (máy trạng thái hữu hạn):** dạng tính toán của biểu thức chính quy (regex); mỗi trạng thái cho biết ký tự nào được phép tiếp theo.

2. Ý tưởng chính

SGLang gồm hai phần: **ngôn ngữ phía trước (embedded trong Python)** với các primitive `gen`, `select`, `fork`, `join`,... và **runtime** với ba tối ưu. Quan trọng nhất cho suy luận: **RadixAttention** giữ KV cache của cả prompt lẫn kết quả sinh ra trong một **cây cơ số** trên bộ nhớ GPU; request mới tìm tiền tố dài nhất đã có trong cây và dùng lại KV của nó, bộ nhớ cạn thì loại lá ít dùng gần đây nhất (LRU). Bộ lập lịch ưu tiên request có tiền tố khớp dài nhất để tăng tỉ lệ trúng cache; paper chứng minh thứ tự này là tối ưu (trong trường hợp

offline). Hai tối ưu còn lại: **FSM nén** để giải mã nhiều token một bước khi ràng buộc chỉ cho một đường đi, và **speculative execution** để giảm số lần gọi API cho mô hình chỉ có API.

3. Phương pháp chi tiết

3.1 Mô hình lập trình (lướt)

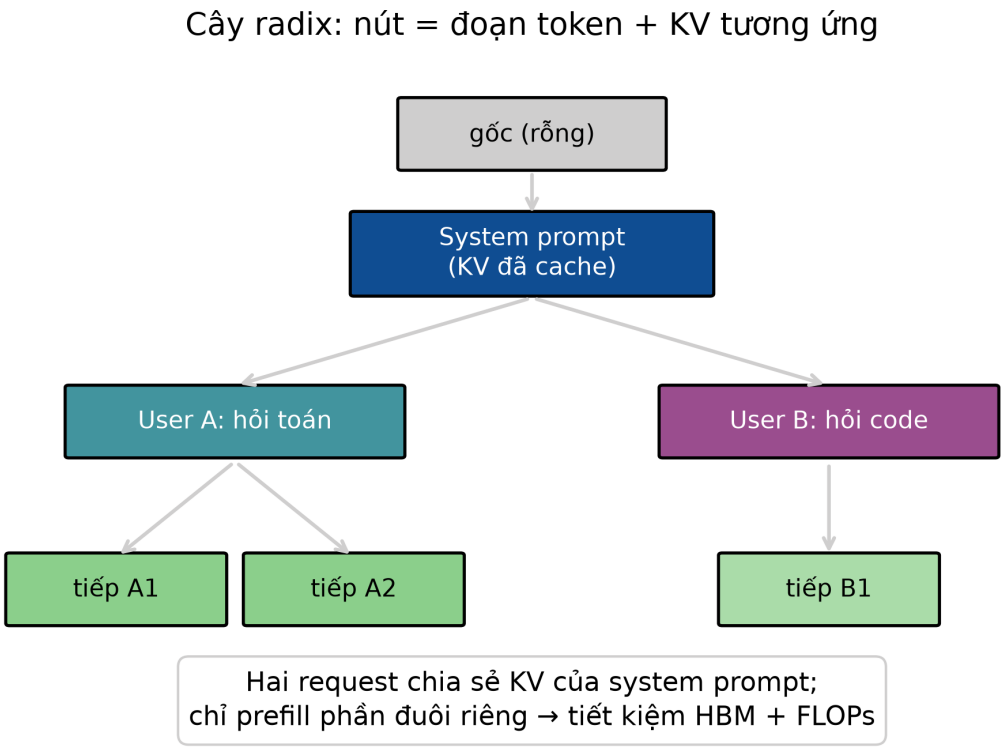
Ví dụ (Hình 2): bộ chấm bài luận về ảnh theo kiểu branch-solve-merge:

```
@function
def multi_dimensional_judge(s, path, essay):
    s += system("Evaluate an essay about an image.")
    s += user(image(path) + "Essay:" + essay)
    s += assistant(select("related", choices=["yes", "no"]))   # chọn lựa chọn xác suất cao
↪   nhất
    forks = s.fork(3)                                          # 3 nhánh chạy song song,
↪   chia sẻ tiền tố
    for f, dim in zip(forks, dimensions):
        f += user("Evaluate based on: " + dim)
        f += assistant("Judgment:" + gen("judgment", stop="END"))
    ...
    s += assistant(gen("output", regex=schema))                # giải mã ràng buộc bằng
↪   regex
```

Primitive: `gen` (sinh, có thể có `regex`), `select`, `+=`/`extend`, `fork`/`join`, `image`/`video`. Trình thông dịch xử lý prompt như một luồng bất đồng bộ (giống phóng kernel CUDA không chặn). Paper nói cùng chương trình viết bằng API kiểu OpenAI tốn ~2.1x số dòng. Phần này ít liên quan suy luận một mô hình nên lướt.

3.2 RadixAttention: cây cơ số quản lý KV cache

Cấu trúc dữ liệu: ánh xạ **dãy token → tensor KV tương ứng**. Cây lưu trên CPU (chi phí duy trì không đáng kể); tensor KV nằm trên GPU, theo bố cục **phân trang với kích thước trang = 1 token** (không liền mạch, giống PagedAttention ở bài 46).



Cây radix tái dùng KV theo tiền tố chung (system prompt / few-shot); chỉ prefill phần đuôi riêng. (minh hoạ, tự tính)

Thao tác:

- **Khớp tiền tố (match):** duyệt cây theo token của request mới, lấy tiền tố dài nhất đã có; KV của phần này dùng lại, chỉ cần tính prefill cho phần còn lại.
- **Chèn (insert):** sau khi xử lý, dãy mới (prompt và cả đầu ra) được thêm vào cây; nếu nó tách giữa một cạnh, cạnh bị **chia đôi** thành hai nút (ví dụ hai phiên chat chia sẻ system prompt, Hình 3 bước 4).
- **Loại bỏ (evict):** LRU, **lá trước**; vì thế các tổ tiên chung được giữ cho tới khi trở thành lá.
- **Đếm tham chiếu:** mỗi nút có reference counter số request đang chạy dùng nó; nút có count 0 mới loại được (không loại thứ batch đang chạy cần).
- **Không cấp bộ nhớ cố định cho cache:** token cache và request đang chạy dùng chung một bể bộ nhớ; khi có nhiều request chờ, hệ thống loại cache để mở rộng batch.

Tương thích với continuous batching, PagedAttention và tensor parallelism (mỗi GPU giữ một phần KV, thao tác cây giống hệt, không cần đồng bộ thêm). Với mô hình đa phương thức, nó lấy **hash của ảnh làm khoá** trong cây, nên các câu hỏi khác nhau về cùng một ảnh dùng lại KV của token ảnh.

"Frontend hint": khi `fork`, frontend gửi tiền tố trước để chắc nó được chèn vào cây, rồi mới gửi phần còn lại.

3.3 Lập lịch nhận biết cache và định lý tối ưu

Tỉ lệ trúng cache:

$$\text{hit rate} = \frac{\text{số token prompt lấy từ cache}}{\text{số token prompt}}.$$

Nếu bộ lập lịch xen kẽ các request không liên quan thì cache bị đuổi liên tục (thrashing). Thuật toán 1 của paper:

```
mỗi vòng lặp:
    requests <- lấy toàn bộ hàng chờ
    với mỗi req: (req.prefix_node, req.prefix_len) <- T.match_prefix(req.input_tokens)
    sắp xếp requests theo prefix_len giảm dần      # tiền tố khớp dài nhất trước
    available <- T.evictable_size() + P.available_size()
    new_batch <- []
    với mỗi req theo thứ tự:
        nếu req.size() + current_size < available:
            new_batch.append(req); tăng ref counter của req.prefix_node
    B.merge(new_batch)                              # vào batch đang chạy
    cấp phát bộ nhớ; nếu thiếu thì T.evict(needed)
    chạy B
    với mỗi request xong: giảm ref counter, T.insert(req)
```

Định lý 3.1. Với một batch request (trường hợp offline), duyệt cây cơ số theo thứ tự **tìm kiếm theo chiều sâu (DFS)** cho tỉ lệ trúng cache tối ưu, với điều kiện kích thước cache ≥ độ dài request lớn nhất; và thứ tự "tiền tố chung dài nhất trước" tương đương DFS.

Phác chứng minh (Phụ lục A.3): với mỗi cạnh e của cây, KV của nó phải tính ít nhất một lần, nên chi phí tính toán $C \geq \sum_e |e|$. Duyệt DFS: lần đầu tính cạnh e thì tính luôn cả cây con dưới nó, trong lúc đó e liên tục được truy cập nên không bị loại (cache đủ chứa đường dài nhất), và sau khi xong cây con thì e không bao giờ được quay lại. Vậy mỗi cạnh được tính đúng một lần: $C = \sum_e |e|$, đạt cận dưới. Khi đó

$$\text{hit rate} = 1 - \frac{C}{\sum_{r \in R} \#\text{prefill tokens}(r)}.$$

Phần tương đương DFS dùng quy nạp. Trường hợp online thì thứ tự DFS bị phá vỡ nhưng lịch vẫn xấp xỉ DFS trên phần cây tăng thêm. Lưu ý chú thích 2 của paper: thực tế tính toán không hoàn toàn như trong chứng minh vì số token đầu ra không biết trước có thể dẫn tới tính lại KV.

Ví dụ số (tự tính). Bài few-shot: tiền tố chung 1,000 token (các ví dụ), 14 câu hỏi mỗi câu 100 token. Không cache: $14 \times 1100 = 15{,}400$ token prefill. Có cache (DFS): $1000 + 14 \times 100 = 2{,}400$ token, hit rate $= 1 - 2400/15400 \approx 84\%$.

Phân tán: tensor parallelism đã nêu; với data parallelism nhiều worker, phụ lục A.4 bàn riêng (không đi sâu ở đây).

3.4 FSM nén cho giải mã có ràng buộc

Giải mã có ràng buộc (ví dụ ép đầu ra khớp regex JSON): hệ thống thường chuyển regex thành FSM, mỗi bước lấy các token hợp lệ từ trạng thái hiện tại và đặt xác suất token không hợp lệ bằng 0, sinh **một token mỗi bước**. Nhưng nếu một đoạn như `{"summary": "` chỉ có một đường hợp lệ thì có thể "điền" cả đoạn trong **một lượt chạy mô hình** (prefill nhiều token). SGLang phân tích FSM và **nén các cạnh chỉ có một chuyển tiếp liền kề thành một cạnh**, để giải mã nhiều token trong một bước. Điểm kỹ thuật: cần tiền xử lý FSM một lần rồi dùng lại cho cả batch (nếu tiền xử lý mỗi request, thông lượng giảm 2.4x).

3.5 Speculative execution cho API

Với mô hình chỉ có API (GPT-4/3.5), chỉ gọi được hộp đen. Với mẫu `gen("name", stop="\n") + "job:" + gen("job", stop="\n")`, bình thường cần hai lời gọi và trả phí token đầu vào hai lần. SGLang cho lời gọi đầu **bỏ qua điều kiện dừng và sinh thêm vài token**, giữ phần dư và khớp lại với primitive sau; nếu khớp

thì tiết kiệm một lời gọi. (Chú ý: đây là "speculative" theo nghĩa suy đoán khuôn mẫu, **khác** speculative decoding ở bài 48 vốn là thuật toán lấy mẫu giữ nguyên phân phối.)

3.6 Phân tích chi phí

- **Thời gian prefill tiết kiệm:** với tiền tố chung dài L_p và n request dùng lại, FLOPs phần tuyến tính tiết kiệm xấp xỉ $2 \cdot \#\text{params} \cdot L_p \cdot (n-1)$ (cùng cấu trúc đếm như Mục 3.3), cộng bớt phần attention bậc hai. Bộ nhớ KV cũng được **chia sẻ** thay vì nhân bản n lần, nên batch lớn hơn.
- **Overhead của cây:** thử trên ShareGPT không có cơ hội tái sử dụng: chạy 100 request mất 74.3 s, quản lý cấu trúc RadixAttention chỉ 0.2 s (<0.3%), vì thao tác cây tuyến tính theo độ dài và nhỏ. Do đó bật mặc định không hại.
- **Chi phí bộ nhớ:** cache chiếm chung bể bộ nhớ KV với request đang chạy; khi áp lực cao, hệ thống ưu tiên batch hơn cache.

4. Thí nghiệm và kết quả chính

Thiết lập: Llama-2 (7B-70B), Mixtral-8x7B, LLaVA-v1.5-7B (ảnh), LLaVA-NeXT-34B (video), GPT-3.5 (API). Chủ yếu AWS G5 với **A10G 24GB**, một số thí nghiệm A100G 80GB; fp16. Baseline: Guidance (llama.cpp), vLLM v0.2.5, LMQL. Workload: MMLU 5-shot, HellaSwag 20-shot, ReAct, generative agents, tree-of-thought (GSM-8K), skeleton-of-thought, LLM judge (BSM), JSON decoding bằng regex, chat nhiều lượt (4 lượt, đầu vào mỗi lượt 256-512 token; ngắn: 4-8 token đầu ra, dài: 256-512), RAG qua DSPy. Chỉ số: thông lượng (chương trình/giây) và độ trễ khi chạy từng chương trình không batch.

Nội dung	Kết quả (paper)
Thông lượng tổng quát	tới **6.4x** so với hệ thống tốt nhất
Độ trễ	giảm tới 3.7x
Tỉ lệ trúng cache của các benchmark	50% đến 99%
Tỉ lệ trúng cache của lập lịch nhận biết cache so với tối ưu	trung bình 96%
Mô hình lớn (Mixtral-8x7B, Llama-70B, tensor parallelism)	xu hướng speedup tương tự mô hình nhỏ
LLaVA-v1.5-7B (ảnh, llava-bench-in-the-wild)	bản gốc của tác giả mô hình 0.18 ảnh/s; SGLang 1.15 ảnh/s
LLaVA-NeXT-34B (video, ActivityNet)	gốc 0.02 khung/s; SGLang 0.10 khung/s (5x theo bảng)
Chatbot Arena sau một tháng	trúng cache 52.4% với LLaVA-NeXT-34B, 74.1% với Vicuna-33B; giảm thời gian token đầu tiên trung bình 1.7x với Vicuna-33B
Chat nhiều lượt, đầu ra dài	gần như không tăng tốc (không có nhiều chia sẻ và decode chi phối)
API (GPT-3.5, trích 3 trường từ trang Wikipedia)	giảm chi phí token đầu vào khoảng 3 lần

Nguyên nhân tăng tốc theo benchmark (paper): MMLU/HellaSwag/RAG tái sử dụng KV của ví dụ few-shot và câu hỏi chung; agent tái sử dụng khuôn mẫu và các lần gọi trước; ToT/SoT song song hoá trong một chương trình; JSON dùng FSM nén; chat nhiều lượt tái sử dụng lịch sử, tác dụng rõ nhất khi đầu ra ngắn vì lúc đó prefill chiếm phần lớn thời gian.

5. Phân tích / ablation

- **Tỉ lệ trúng cache vs hiệu năng** (trên tree-of-thought, cách tắt có chọn lọc các token khớp): trúng cache cao hơn dẫn tới batch lớn hơn, thông lượng cao hơn, độ trễ (cả token đầu tiên và tổng) thấp hơn.

- **Thành phần RadixAttention** (Hình 8c): "No Cache", "No Tree-Structure" (bảng tra đơn giản thay cho cây), "FCFS Schedule", "Random Schedule", "No Frontend Parallelism", "No Frontend Hint", "Full". Kết luận của paper: **mỗi thành phần đều cần thiết** để đạt hiệu năng tốt nhất, và việc tắt song song/hint của frontend làm giảm hiệu năng, nên đồng thiết kế frontend-runtime là có ích. Số cụ thể nằm trong biểu đồ, không trích được.
- **Overhead:** <0.3% (Mục 3.6).
- **FSM nén:** tăng thông lượng 1.6x trên JSON decoding; tiền xử lý lại cho từng request làm thông lượng thấp hơn 2.4x.

6. Giới hạn

Do tác giả nêu (Future Directions):

- Mở rộng đa phương thức đầu ra; tối ưu RadixAttention qua nhiều tầng bộ nhớ (DRAM, đĩa); khớp mờ theo ngữ nghĩa trong RadixAttention; primitive cấp cao hơn.
- Lập lịch tham lam theo tiền tố có thể gây **đói (starvation)**; cần kết hợp lập lịch công bằng.
- Compiler của SGLang chưa có tối ưu tĩnh nâng cao (lập lịch, quy hoạch bộ nhớ).
- Trong thực tế, số token đầu ra không biết trước làm tính toán khác với chứng minh định lý (chú thích 2).

Do ta nhận thấy:

- Định lý tối ưu giả định cache đủ chứa request dài nhất và chỉ nói về trường hợp offline; trường hợp online chỉ là xấp xỉ.
- Cần tiền tố **trùng khít từng token** (kể cả cách tách từ). Hai prompt khác nhau một ký tự ở đầu mất toàn bộ chia sẻ phía sau.
- Chỉ đánh giá kiến trúc Transformer attention thuần (Llama, Mixtral, LLaVA); không bàn đến mô hình có trạng thái hồi quy (Mamba, GatedDeltaNet), nơi tái sử dụng tiền tố khác bản chất.
- Không đo GPU nhỏ ≤ 8 GB; mọi thí nghiệm trên A10G 24GB trở lên.
- Một số baseline (Guidance, LMQL) bị loại khỏi vài benchmark vì chậm hoặc thiếu chức năng (tác giả nói), nên mức so sánh không đồng đều; số 6.4x là "tới", không phải trung bình.

7. Liên hệ với các bài khác

- **46 (PagedAttention):** vLLM chỉ hỗ trợ shared prefix khai báo trước; SGLang tổng quát hoá thành cây tự động có LRU và lập lịch. Hai kỹ thuật chồng lên nhau (SGLang dùng trang 1 token).
- **44, 45 (FlashAttention):** kernel tính; SGLang dùng kernel FlashInfer và Triton (theo paper) cho attention trên bố cục không liền mạch.
- **48 (speculative decoding):** hướng khác để giảm thời gian; SGLang "API speculative execution" là suy đoán khuôn mẫu chứ không phải lấy mẫu chấp nhận/từ chối.
- **14-16 (MQA/GQA/MLA):** KV nhỏ hơn thì cache được nhiều tiền tố hơn trong cùng bộ nhớ.
- **18-24 (attention tuyến tính, hồi quy):** KV cache theo token không còn là đối tượng duy nhất; tái sử dụng tiền tố cần lưu **trạng thái hồi quy tại mốc tiền tố**.
- **32, 34, 37 (Flamingo, LLaVA, Qwen2-VL):** Token ảnh là một phần của tiền tố; SGLang dùng hash ảnh làm khóa.
- **55, 56 (StreamingLLM, H2O):** các chính sách giữ/loại KV; LRU theo cạnh của cây là chính sách cấp request, H2O/StreamingLLM là cấp token.

8. Tóm tắt công thức / khái niệm

Khái niệm	Ý nghĩa / công thức
RadixAttention	cây cơ số: dãy token → KV; khớp/chèn/loại (LRU lá)
Reference counter	số request chạy đang dùng nút; 0 thì có thể loại
Hit rate	token prompt lấy từ cache / token prompt
Cận dưới tính toán	$C \geq \sum_{e \in \text{edges}(T)} \lvert e \rvert$
Lập lịch tối ưu (offline)	DFS trên cây = tiền tố chung dài nhất trước; hit $= 1 - C/\sum$ prefill
FSM nén	gộp các cạnh một chuyển tiếp để giải mã nhiều token một bước
API speculative execution	sinh dư để khớp lời gọi sau (không phải speculative decoding)
Overhead cây	0.2 s trên 74.3 s (<0.3%)
Kết quả	thông lượng tới 6.4x, độ trễ giảm tới 3.7x

9. Kết luận ngắn

SGLang biến KV cache từ "rác bỏ sau mỗi request" thành một **bộ nhớ đệm có cấu trúc cây** với LRU và lập lịch ưu tiên tiền tố dài, có chứng minh tối ưu trong trường hợp offline, cộng với giải mã ràng buộc nhanh hơn nhờ FSM nén. Lợi ích lớn khi nhiều lần gọi chia sẻ tiền tố (few-shot, agent, chat, nhiều câu hỏi về một ảnh).

Link gốc: https://arxiv.org/abs/2312.07104

Bài 48

Fast Inference from Transformers via Speculative Decoding (2022/2023)

Tác giả: Yaniv Leviathan, Matan Kalman, Yossi Matias (Google Research).
Venue / arXiv: arXiv:2211.17192 (v1 30/11/2022; v2 18/05/2023); ICML 2023 (PMLR 202, theo trang đầu của paper).
Nguồn: https://arxiv.org/abs/2211.17192
Chủ đề: Giải mã suy đoán (speculative decoding): dùng mô hình nhỏ đề xuất nhiều token, mô hình lớn kiểm tra song song bằng speculative sampling, **giữ nguyên phân phối đầu ra**; công thức số token kỳ vọng, tăng tốc, số phép toán và chọn γ.
Vị trí trong lộ trình: Đọc trước: 01 (Transformer, giải mã tự hồi quy), 44-46 (decode bị nghẽn bởi bộ nhớ, KV cache). Bài này thêm: cách giảm **số lần chạy mô hình lớn** mà không đổi đầu ra. Đọc tiếp: 49 (Medusa) và 50 (EAGLE): thay mô hình nháp riêng bằng đầu dự đoán/ngoại suy đặc trưng ngay trong mô hình; và để hiểu mô hình nháp tốt từ đâu, xem lại các bài distillation của lộ trình.

> Đây là ghi chú học tập bằng tiếng Việt, diễn giải lại chứ không phải bản dịch. Số liệu lấy từ paper (có ghi rõ). Paper loại trung bình (~8k từ); đây là bài có toán rõ ràng nên ghi chú đi sâu vào chứng minh.

1. Bối cảnh

Một mô hình tự hồi quy lớn như GPT-3, PaLM, LaMDA hay T5-XXL cần K lượt chạy tuần tự để sinh K token. Mỗi lượt của mô hình lớn cũng thường chậm hơn một lượt của mô hình nhỏ. Trước speculative decoding, các hướng giảm chi phí gồm:

- **Giảm chi phí cho mọi token như nhau:** distillation, thưa hoá, lượng tử hoá, đổi kiến trúc (MQA...). Thường phải huấn luyện lại và **đổi đầu ra**.
- **Tính toán thích ứng (adaptive computation):** early exit, dùng mô hình nhỏ cho bước dễ. Cũng thường phải đổi kiến trúc/huấn luyện và không giữ đầu ra như cũ.
- **Blockwise Parallel Decoding (Stern et al. 2018):** giải mã song song nhiều token nhưng chỉ hỗ trợ greedy, cần huấn luyện mô hình riêng. **Shallow Aggressive Decoding** chỉ chép đầu vào sang đầu ra.

Quan sát then chốt của paper: (1) có bước "khó" cần mô hình lớn và bước "dễ" mô hình nhỏ xấp xỉ tốt; (2) suy luận mô hình lớn thường **bị nghẽn bởi băng thông bộ nhớ và giao tiếp chứ không phải số phép toán**, nên còn dư năng lực tính toán để chạy song song nhiều vị trí. Đây là cùng hiện tượng memory-bound ở bài 44-46.

Thuật ngữ:

- **Speculative execution (thực thi suy đoán):** kỹ thuật bộ xử lý, ví dụ dự đoán rẽ nhánh: thực hiện công việc trước khi biết có cần không, rồi kiểm tra; trả giá bằng tính song song tăng.
- **Lấy mẫu loại bỏ (rejection sampling):** lấy mẫu từ phân phối q dễ, chấp nhận ngẫu nhiên để thu được mẫu theo phân phối p khó.
- **Total variation (TV):** khoảng cách giữa hai phân phối $\frac{1}{2}\sum_x |p(x) - q(x)|$.

2. Ý tưởng chính

Dùng mô hình nhỏ M_q (nhanh) sinh nháp γ token liên tiếp. Rồi chạy **một lượt** mô hình lớn M_p trên cả $\gamma + 1$ vị trí song song (nhờ tính song song giữa các vị trí, giống prefill), thu được các phân phối $p_1, \ldots, p_{\gamma+1}$ của mô hình lớn. Với từng token nháp $x_i \sim q_i$, chấp nhận nó với xác suất $\min(1, p_i(x_i)/q_i(x_i))$; token đầu tiên

bị từ chối được **lấy mẫu lại từ phân phối hiệu chỉnh** $\text{norm}(\max(0, p - q))$; nếu tất cả được chấp nhận, thêm một token thưởng từ $p_{\gamma+1}$. Quy tắc này **chứng minh** cho ra đúng phân phối của M_p, nên không đổi đầu ra. Mỗi lượt chạy mô hình lớn cho ra **từ 1 tới** $\gamma + 1$ **token** (không bao giờ ít hơn giải mã thường). Không cần huấn luyện lại, không đổi kiến trúc.

3. Phương pháp chi tiết

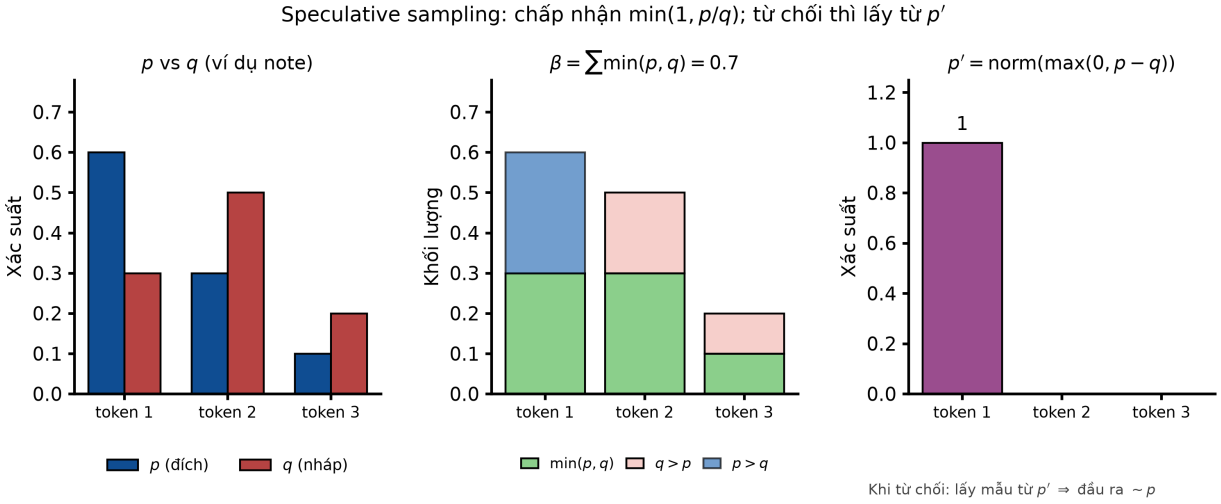
Ký hiệu: M_p mô hình đích (target) với phân phối $p(x_t \mid x_{<t})$; M_q mô hình nháp (approximation/draft) với $q(x_t \mid x_{<t})$; $\gamma \in \mathbb{Z}^+$ số token nháp.

3.1 Chuẩn hoá về lấy mẫu từ một phân phối

Các cách lấy mẫu (argmax, top-k, nucleus, nhiệt độ) đều quy về **lấy mẫu chuẩn từ một phân phối đã hiệu chỉnh**. Ví dụ argmax = đặt bằng 0 các phần tử không phải max rồi chuẩn hoá (phân phối one-hot). Từ đây p và q là phân phối **sau khi** áp phương pháp lấy mẫu, và ta chỉ cần chứng minh cho lấy mẫu chuẩn.

3.2 Speculative sampling: quy tắc chấp nhận

Mục tiêu: lấy mẫu $x \sim p(x)$ trong khi chỉ lấy mẫu rẻ từ q.



Speculative sampling trên ví dụ note $p = [0.6, 0.3, 0.1]$, $q = [0.3, 0.5, 0.2]$: $\beta = \sum \min(p, q) = 0.7$ và phân phối hiệu chỉnh p' khi từ chối. (minh hoạ, tự tính)

1. Lấy $x \sim q(x)$ và $r \sim U(0, 1)$.
2. Nếu $q(x) \le p(x)$: **giữ** x (luôn chấp nhận). Nếu $q(x) > p(x)$: chấp nhận với xác suất $p(x)/q(x)$, tức nhận nếu $r \le p(x)/q(x)$. Gộp lại: chấp nhận với xác suất $\min(1, \frac{p(x)}{q(x)})$.
3. Nếu bị từ chối: lấy mẫu lại từ phân phối **hiệu chỉnh** (residual)

$$p'(x) = \frac{\max(0,\ p(x) - q(x))}{\sum_{x'} \max(0,\ p(x') - q(x'))}.$$

Trực giác: q có thể "quá nhiều" ở một số token (nơi $q > p$) và "quá ít" ở token khác ($q < p$). Việc từ chối bớt các token quá nhiều, rồi bù phần thiếu bằng p', chính xác là phần dư của p chưa được q phủ.

3.3 Chứng minh phân phối không đổi

Đặt acceptance probability tổng thể $\beta = \Pr[\text{chấp nhận}] = \mathbb{E}_{x\sim q}[\min(1, p(x)/q(x))] = \sum_x \min(q(x), p(x))$ (vì $q(x)\min(1, p/q) = \min(q, p)$).

Chuẩn hoá của p'. Vì $\max(0, p-q) = p - \min(p,q)$,

$$\sum_x \max(0, p(x) - q(x)) = \sum_x p(x) - \sum_x \min(p(x), q(x)) = 1 - \beta,$$

nên $p'(x) = \dfrac{p(x) - \min(p(x), q(x))}{1-\beta}$.

Xác suất đầu ra bằng x_0:

$$\Pr[x = x_0] = \underbrace{\Pr[\text{chấp nhận}, x = x_0]}_{\text{nháp } x_0 \text{ được nhận}} + \underbrace{\Pr[\text{từ chối}, x = x_0]}_{\text{lấy lại từ } p'}.$$

- Số hạng đầu: $q(x_0) \cdot \min(1, \frac{p(x_0)}{q(x_0)}) = \min\left(q(x_0), p(x_0)\right)$.
- Số hạng hai: xác suất từ chối $= 1-\beta$, rồi chọn x_0 theo p': $(1-\beta)\, p'(x_0) = p(x_0) - \min\left(p(x_0), q(x_0)\right)$.

Cộng lại:

$$\boxed{\Pr[x = x_0] = \min(p(x_0), q(x_0)) + p(x_0) - \min(p(x_0), q(x_0)) = p(x_0).}$$

Điều này đúng với **mọi** p, q (miễn $q(x) > 0$ ở chỗ cần chia; token nào $q = 0$ thì không bao giờ được đề xuất và do p' hay phần thiếu bù đắp). Do đó chất lượng mô hình nháp **không ảnh hưởng tính đúng đắn**, chỉ ảnh hưởng tốc độ.

Ví dụ số (tự tính). $p = [0.6, 0.3, 0.1]$, $q = [0.3, 0.5, 0.2]$.

- $\min(p, q) = [0.3, 0.3, 0.1]$ nên $\beta = 0.7$.
- $\max(0, p - q) = [0.3, 0, 0]$, chuẩn hoá theo $1 - \beta = 0.3$ được $p' = [1, 0, 0]$.
- Xác suất ra token 1: $\min = 0.3$ (nhận) $+ (1 - \beta)p'(1) = 0.3 \cdot 1 = 0.3$, tổng 0.6 = $p(1)$. Token 2: $0.3 + 0 = 0.3$. Token 3: $0.1 + 0 = 0.1$. Khớp p.

So với rejection sampling cổ điển (Phụ lục A.2). Rejection sampling dùng hằng số $M = \max_x p(x)/q(x)$ và nhận $x \sim q$ khi $r < \frac{p(x)}{Mq(x)}$, nếu từ chối thì lặp lại. Xác suất nhận của nó là $\mathbb{E}_{x\sim q}[\frac{p(x)}{Mq(x)}] = \sum_x p(x) \min_{x'} \frac{q(x')}{p(x')} \le \sum_x p(x) \min\left(1, \frac{q(x)}{p(x)}\right) = \sum_x \min(p, q) = \alpha$, tức **thấp hơn** (có thể thấp hơn nhiều) xác suất nhận của speculative sampling. Speculative sampling cũng không lặp vô hạn: khi từ chối nó lấy mẫu một lần từ p'.

Đến cả dãy token. Với γ token nháp, xét theo thứ tự: tại vị trí i (điều kiện trên tiền tố gồm các token đã chấp nhận), token được phát ra có phân phối p_i theo chứng minh trên. Khi gặp từ chối đầu tiên tại vị trí $n+1$, ta bỏ mọi token nháp và tính toán phía sau (nó dựa trên tiền tố không còn đúng). Quy nạp theo vị trí: mỗi token phát ra có phân phối đúng $p(\cdot \mid \text{tiền tố đã phát})$, nên toàn dãy có phân phối đúng $\prod_t p(x_t \mid x_{<t})$. Nếu cả γ token được nhận, token thưởng thứ $\gamma + 1$ lấy trực tiếp từ $p_{\gamma+1}$ (phân phối của M_p trên tiền tố đầy đủ).

3.4 Thuật toán giải mã nhiều token

```
SpeculativeDecodingStep(Mp, Mq, prefix, gamma):
    # 1) Mq nháp gamma token, tuần tự
    for i = 1..gamma:
        q_i(x) <- Mq(prefix + [x_1..x_{i-1}])
        x_i    ~ q_i
    # 2) Mp đánh giá song song gamma+1 vị trí (một lượt chạy)
    p_1..p_{gamma+1} <- Mp(prefix), Mp(prefix+[x_1]), ..., Mp(prefix+[x_1..x_gamma])
    # 3) Số token nháp được chấp nhận
    r_1..r_gamma ~ U(0,1)
    n <- min({ i-1 : r_i > p_i(x_i)/q_i(x_i) } U {gamma})
    # 4) Hiệu chỉnh phân phối cho token cuối
```

```
    p'(x) <- p_{n+1}(x)
    if n < gamma:
        p'(x) <- norm(max(0, p_{n+1}(x) - q_{n+1}(x)))
    t ~ p'
    return prefix + [x_1..x_n, t]            # sinh n+1 token, từ 1 đến gamma+1
```

Hình 1 của paper (mô hình 97M đích, nháp 6M, lm1b): câu 38 token được sinh bằng chỉ 9 lượt chạy tuần tự của mô hình lớn (dòng đầu: một lượt, 5 token).

Hiệu quả vì sao có: một lượt chạy M_p trên $\gamma+1$ vị trí tốn gần bằng một lượt trên 1 vị trí khi bị nghẽn bộ nhớ (đọc trọng số và KV một lần), nhưng cho ra nhiều token.

3.5 Tỉ lệ chấp nhận α và độ lệch D_{LK}

Định nghĩa 3.1. $\beta_{x_{<t}}$ là xác suất chấp nhận $x_t \sim q$ cho tiền tố đã cho; $\alpha = \mathbb{E}(\beta)$ (kỳ vọng trên tiền tố). α là thước đo mô hình nháp xấp xỉ mô hình đích tốt đến đâu.

Định nghĩa 3.2. $D_{LK}(p,q) = \sum_x |p(x) - M(x)| = \sum_x |q(x) - M(x)|$ với $M = \frac{p+q}{2}$.

Bổ đề 3.3. $D_{LK}(p,q) = 1 - \sum_x \min(p(x), q(x))$. *Chứng minh:* $D_{LK} = \sum_x \frac{|p-q|}{2}$; dùng $|p-q| = p + q - 2\min(p,q)$ được $D_{LK} = \sum_x (\frac{p+q}{2} - \min(p,q)) = 1 - \sum_x \min(p,q)$ (vì $\sum_x \frac{p+q}{2} = 1$). Hệ quả: D_{LK} đối xứng, trong $[0,1]$, bằng 0 khi $p = q$ và bằng 1 khi hai phân phối có giá không giao nhau.

Định lý 3.5. $\beta = 1 - D_{LK}(p,q)$. Do đó (Hệ quả 3.6) $\alpha = 1 - \mathbb{E}[D_{LK}(p,q)] = \mathbb{E}[\sum_x \min(p,q)]$.

Nhận xét: D_{LK} chính là **khoảng cách total variation** $\mathrm{TV}(p,q) = \frac{1}{2}\sum |p-q|$ (một cách tính nữa: $\frac{1}{2}\sum |p-q| = 1 - \sum \min(p,q)$). Nên $\alpha = 1 - \mathbb{E}[\mathrm{TV}(p,q)]$. Đây là liên hệ trực tiếp với việc chưng cất mô hình.

Trường hợp greedy (p one-hot tại $a = \arg\max p$ sau hiệu chỉnh): nếu q cũng được hiệu chỉnh thành one-hot tại b, thì $\beta = 1$ nếu $a = b$ và 0 nếu khác. Tức là nháp được nhận khi **argmax hai mô hình trùng nhau**.

3.6 Số token kỳ vọng và công thức tăng tốc

Giả sử β độc lập, cùng phân phối với kỳ vọng α (xấp xỉ đơn giản hoá). Số token nháp được nhận liên tiếp n có $\Pr[n \geq k] = \alpha^k$ ($k \leq \gamma$), và mỗi lượt sinh $n+1$ token:

$$\mathbb{E}[\#\text{token}] = 1 + \sum_{k=1}^{\gamma} \alpha^k = \sum_{k=0}^{\gamma} \alpha^k = \boxed{\frac{1-\alpha^{\gamma+1}}{1-\alpha}} \qquad (1)$$

(paper mô tả nó là biến hình học bị chặn với giới hạn $\gamma+1$). Khi $\gamma \to \infty$ giá trị tiến tới $\frac{1}{1-\alpha}$.

Hệ số chi phí. Gọi c = (thời gian một lượt M_q)/(thời gian một lượt M_p) — phụ thuộc phần cứng và cài đặt (trong thí nghiệm của paper $c < 0.05$, thường gần 0). Một lượt thuật toán tốn $T(c\gamma+1)$ (với T là thời gian một lượt M_p). Giả sử đủ tài nguyên để chạy $\gamma+1$ vị trí song song không tăng thời gian. Thời gian mỗi token $= \frac{T(c\gamma+1)(1-\alpha)}{1-\alpha^{\gamma+1}}$, chia cho T của giải mã thường:

$$\boxed{\text{Speedup}(\alpha,\gamma,c) = \frac{1-\alpha^{\gamma+1}}{(1-\alpha)(\gamma c+1)}} \qquad \text{(Định lý 3.8)}$$

Điều kiện có lợi. Hệ quả 3.9: nếu $\alpha > c$ tồn tại γ cho tăng tốc, và hệ số tối thiểu là $\frac{1+\alpha}{1+c}$ (lấy $\gamma = 1$: $\frac{1-\alpha^2}{(1-\alpha)(1+c)} = \frac{1+\alpha}{1+c}$; nếu có lợi với γ thì cũng có lợi với mọi $\gamma^* < \gamma$). Trường hợp nháp miễn phí ($c = 0$): tăng tốc $= \frac{1-\alpha^{\gamma+1}}{1-\alpha} \leq \frac{1}{1-\alpha}$.

Kiểm tra với bảng của paper (Bảng 4; $\gamma = 7$, $\alpha = 0.75$, $c = 0.02$, T5-small): $\frac{1-0.75^8}{0.25\,(7\cdot 0.02+1)} = \frac{0.8999}{0.285} \approx 3.16$, paper ghi dự đoán 3.2 (đo được 3.4). Kiểm tra bigram: $\alpha \approx 0.2$, $\gamma = 3$, $c \approx 0$: $\frac{1-0.2^4}{0.8} = 1.248 \approx 1.25\times$, đúng như paper.

3.7 Số phép toán và chọn γ

Gọi $\hat{c}$ là tỉ số số phép toán số học mỗi token của M_q so với M_p. Một lượt thuật toán: γ lần M_q và $\gamma + 1$ lần M_p song song, tức $\hat{T}(\hat{c}\gamma + \gamma + 1)$ phép toán ($\hat{T}$ là phép toán của một lượt M_p). Chia cho số token kỳ vọng:

$$\text{Hệ số tăng phép toán} = \frac{(1-\alpha)(\gamma\hat{c} + \gamma + 1)}{1 - \alpha^{\gamma+1}} \qquad \text{(Định lý 3.11)}$$

Khi nháp bị từ chối thì phần tính toán dư bị phí; khi nháp được nhận thì sự song song là "miễn phí". Còn **số lần truy cập bộ nhớ có thể giảm**: trọng số và KV cache của M_p được đọc một lần mỗi lượt thay vì mỗi token, nên số lần đọc giảm hệ số $\frac{1-\alpha^{\gamma+1}}{1-\alpha}$. Cũng nhận xét: tổng phép toán của M_p (không tính M_q) bị chặn bởi một lượt chạy bộ mã hoá cùng kích thước.

Một số cặp ($c = \hat{c} = 0$) trong Bảng 1 của paper:

α	γ	Phép toán	Tốc độ
0.6	2	1.53x	1.96x
0.7	3	1.58x	2.53x
0.8	2	1.23x	2.44x
0.8	5	1.63x	3.69x
0.9	2	1.11x	2.71x
0.9	10	1.60x	6.86x

Chọn γ: cực đại hoá Định lý 3.8 theo γ nguyên (tìm số, Hình 3). γ tăng thì số token kỳ vọng tăng nhưng chi phí nháp γc và phí tính toán cũng tăng. Vì β không cố định, thay đổi γ động theo β dự đoán có thể cho thêm tới ~60% cải thiện so với γ cố định (cận trên giả định oracle và đủ tài nguyên; paper để dành cho tương lai).

3.8 Chọn mô hình nháp, beam search, lenience

- **Mô hình nháp:** tính đúng đắn không phụ thuộc lựa chọn; thực tế chọn mô hình nhỏ hơn đích cỡ hai bậc độ lớn để cân bằng α và c. Các loại: Transformer nhỏ cùng kiến trúc; **mô hình chi phí bằng không** ($c \approx 0$) như n-gram (tra bảng) hoặc heuristic chép từ ngữ cảnh khi chuỗi hay lặp (tóm tắt, hội thoại lặp), hay mô hình không tự hồi quy; cả mô hình chọn token ngẫu nhiên vẫn được cải thiện nhỏ.
- **Beam search (Phụ lục A.4):** áp dụng được với chi phí hơn: dùng beam rộng $u \geq w$ ở M_q trong γ bước, kiểm bằng $w + u\gamma$ lượt M_p; chấp nhận khi $\text{top}_w(M_p) \subseteq \text{top}_u(M_q)$.
- **Lenience (Phụ lục A.5):** nhân q với $l \in [0, 1]$ trước khi so với p để tăng α đổi lấy sai khác nhỏ; vẫn đảm bảo không token nào có xác suất lớn hơn $p(x)/l$. Mọi kết quả chính của paper dùng phiên bản nghiêm ngặt ($l = 1$).

4. Thí nghiệm và kết quả chính

Tăng tốc thực đo trên T5-XXL (11B) với M_q là T5-small (77M, "small f"), T5-base (250M), T5-large (800M); batch size 1, một TPU-v4, so với T5X. Hai tác vụ: dịch Anh-Đức (WMT EnDe) và tóm tắt CNN/DM. Ghi chú: temp=0 là argmax, temp=1 là lấy mẫu chuẩn.

Tác vụ	M_q	Temp	γ	α	Tốc độ
EnDe	T5-small	0	7	0.75	3.4x
EnDe	T5-base	0	7	0.8	2.8x
EnDe	T5-large	0	7	0.82	1.7x
EnDe	T5-small	1	7	0.62	2.6x
EnDe	T5-base	1	5	0.68	2.4x
EnDe	T5-large	1	3	0.71	1.4x
CNNDM	T5-small	0	5	0.65	3.1x
CNNDM	T5-base	0	5	0.73	3.0x
CNNDM	T5-large	0	3	0.74	2.2x
CNNDM	T5-small	1	5	0.53	2.3x
CNNDM	T5-base	1	3	0.55	2.2x
CNNDM	T5-large	1	3	0.56	1.7x

Kết luận của paper: tăng tốc **2x-3x** (đỉnh 3.4x) với đầu ra giống hệt. T5-small cho tốc độ cao nhất dù α không cao nhất, vì cân bằng α và c.

Dự đoán lý thuyết vs đo thực (Bảng 4, c ước từ profiler):

Tác vụ	M_q	Temp	γ	α	c	Dự đoán	Đo
EnDe	small	0	7	0.75	0.02	3.2	3.4
EnDe	base	0	7	0.8	0.04	3.3	2.8
EnDe	large	0	7	0.82	0.11	2.5	1.7
EnDe	large	1	3	0.71	0.11	2.0	1.4
CNNDM	small	0	5	0.65	0.02	2.4	3.1
CNNDM	large	1	3	0.56	0.11	1.6	1.7

(Chọn 6 hàng; bảng gốc có 12.) Lệch do khác biệt tối ưu hoá giữa cài đặt của tác giả và baseline và do giả định β độc lập chỉ gần đúng.

Giá trị α (Bảng 3, ước lượng trên 10K token):

M_p	M_q	T=0	T=1
GPT-like 97M	unigram / bigram / GPT-like 6M	0.03 / 0.05 / 0.88	0.03 / 0.05 / 0.89
T5-XXL (EnDe)	unigram / bigram	0.08 / 0.20	0.07 / 0.19
T5-XXL (CNNDM)	unigram / bigram	0.13 / 0.23	0.08 / 0.16
LaMDA 137B	LaMDA 100M / 2B / 8B	0.61 / 0.71 / 0.75	0.57 / 0.71 / 0.74

Quan sát của paper: nháp nhỏ hơn đích cỡ hai bậc độ lớn thường cho $\alpha \in [0.5, 0.9]$; α tăng theo kích thước nháp; phân phối càng "nhọn" ($T = 0$) thì α càng cao; n-gram tầm thường vẫn có α khác 0. Một thực nghiệm độc lập khác (Chen et al. 2023) được paper dẫn: 2x-2.5x trên Chinchilla 70B.

5. Phân tích / ablation

- **Kích thước nháp:** M_q lớn hơn thì α cao hơn nhưng c lớn hơn; T5-small tốt nhất về tốc độ, T5-large kém nhất (Bảng ở Mục 4).

- **Nhiệt độ:** $T = 0$ cho α và tốc độ cao hơn $T = 1$ (EnDe: 3.4x so với 2.6x với T5-small).
- **Chọn γ:** tối ưu phụ thuộc (α, c); γ trong thí nghiệm là 3-7.
- **Phép toán so với tốc độ:** Bảng 1 cho thấy tăng tốc kèm tăng số phép toán (ví dụ $\alpha = 0.9$, $\gamma = 10$: 6.86x tốc độ, 1.60x phép toán).
- Paper không có ablation riêng ngoài các biến trên.

6. Giới hạn

Do tác giả nêu:

- Giả định có sẵn tài nguyên tính toán để chạy $\gamma + 1$ vị trí song song không tăng thời gian; nếu không thì không có lợi.
- Tổng số phép toán **tăng** (kể cả khi tốc độ tăng).
- Giả định β i.i.d. chỉ là xấp xỉ.
- Chỉ cài đặt và đo thực cho T5; các mô hình khác chỉ đo α. Chưa thử miền khác ngoài văn bản (ví dụ ảnh).
- Chưa khai thác γ thích ứng; chưa thử các phép biến đổi phân phối của nháp khác.

Do ta nhận thấy:

- Tất cả thí nghiệm tốc độ ở **batch size 1** trên TPU, các mô hình encoder-decoder T5 và một mô hình GPT nhỏ; chưa kiểm tra decoder-only lớn, GPU tiêu dùng, hay mô hình có kiến trúc hồi quy.
- Khi batch lớn, decode chuyển dần sang compute-bound và giả định "dư tài nguyên" mất đi; lợi ích giảm. Paper chủ yếu khẳng định cho trường hợp nghẽn bộ nhớ.
- Tính đúng đắn của phân phối giả định tính toán chính xác (số học lý tưởng); trong thực tế fp16/bf16 và khác biệt số học giữa lượt song song và lượt tuần tự có thể làm kết quả không giống từng bit.

7. Liên hệ với các bài khác

- **44-46 (FlashAttention, PagedAttention):** cùng gốc "decode là memory-bound"; speculative decoding tận dụng điều đó bằng cách tăng số token mỗi lượt đọc trọng số.
- **49 (Medusa), 50 (EAGLE):** thay mô hình nháp độc lập bằng các đầu dự đoán thêm hoặc dự đoán đặc trưng tầng cuối trong chính mô hình đích, thường dùng xác minh dạng cây; cùng khung chấp nhận/loại.
- **47 (SGLang):** "API speculative execution" ở đó là suy đoán khuôn mẫu, không phải thuật toán giữ phân phối như bài này.
- **Các bài distillation/pruning trong lộ trình:** mô hình nháp tốt chính là mô hình nhỏ có phân phối gần mô hình lớn; $\alpha = 1 - \mathbb{E}[\text{TV}]$ là chỉ số trực tiếp.
- **55, 56 (StreamingLLM, H2O):** nén KV làm đổi phân phối; speculative decoding thì không đổi phân phối, hai cái đối lập về tính "lossless".

8. Tóm tắt công thức / khái niệm

Khái niệm	Công thức / ý nghĩa
Chấp nhận	nhận $x \sim q$ với xác suất $\min(1, p(x)/q(x))$
Phân phối hiệu chỉnh	$p' = \text{norm}(\max(0, p - q))$, hằng số chuẩn hoá $1 - \beta$
Tính đúng	$\Pr[x] = \min(p, q) + p - \min(p, q) = p$
Xác suất chấp nhận	$\beta = \sum_x \min(p, q) = 1 - D_{LK}(p, q)$; $D_{LK} = \text{TV}$
α	$\mathbb{E}[\beta]$
Token kỳ vọng mỗi lượt	$\frac{1-\alpha^{\gamma+1}}{1-\alpha}$
Tăng tốc	$\frac{1-\alpha^{\gamma+1}}{(1-\alpha)(\gamma c+1)}$
Chi phí nháp	c = thời gian M_q / thời gian M_p

Khái niệm	Công thức / ý nghĩa
Điều kiện có lợi	$\alpha > c$; tối thiểu $\frac{1+\alpha}{1+c}$
Tăng phép toán	$\frac{(1-\alpha)(\gamma c+\gamma+1)}{1-\alpha^{\gamma+1}}$
Cận trên ($c=0$, $\gamma \to \infty$)	$\frac{1}{1-\alpha}$
Số token mỗi lượt	từ 1 đến $\gamma+1$

9. Kết luận ngắn

Speculative decoding dùng mô hình nháp rẻ và một lượt kiểm tra song song bằng mô hình đích, với quy tắc chấp nhận $\min(1, p/q)$ cộng phân phối hiệu chỉnh $\text{norm}(\max(0, p-q))$ được chứng minh giữ nguyên phân phối đầu ra. Tăng tốc kỳ vọng $\frac{1-\alpha^{\gamma+1}}{(1-\alpha)(\gamma c+1)}$ với $\alpha = 1 - \mathbb{E}[\text{TV}]$; thực nghiệm 2x-3.4x trên T5-XXL ở batch 1.

Link gốc: https://arxiv.org/abs/2211.17192

Bài 49

Medusa: Simple LLM Inference Acceleration Framework with Multiple Decoding Heads (2024)

Tác giả: Tianle Cai, *Yuhong Li*, Zhengyang Geng, Hongwu Peng, Jason D. Lee, Deming Chen, Tri Dao (Princeton, Together AI, UIUC, CMU, UConn; * đóng góp ngang nhau)
Venue / arXiv: arXiv:2401.10774 (v3, 14/6/2024); ICML 2024 (PMLR 235)
Nguồn: https://arxiv.org/abs/2401.10774
Chủ đề: Tăng tốc decode của LLM bằng cách gắn thêm nhiều "đầu giải mã" (decoding heads) dự đoán nhiều token tương lai song song, kiểm chứng bằng tree attention; không cần draft model riêng.
Vị trí trong lộ trình: Đọc sau 48 (Speculative decoding: ý tưởng draft-rồi-verify) và nên nắm 14/15 (MQA/GQA: vì sao decode bị chặn bởi bộ nhớ) và 46 (PagedAttention). Bài này thay "draft model" bằng "head gắn trên hidden state cuối". Bài kế tiếp: **50 (EAGLE)** sửa điểm yếu độ chính xác của draft ở Medusa; **51–53** chuyển sang cách giảm bytes phải đọc mỗi bước (lượng tử hoá).

> Đây là ghi chú học tập bằng tiếng Việt, không phải bản dịch. Paper dài ~14k từ; ghi chú tập trung vào phương pháp lõi, kết quả chính và phụ lục G (mô hình phần cứng).

1. Bối cảnh: vì sao decode chậm

Giải mã tự hồi quy sinh từng token: token ở bước $t+1$ phụ thuộc kết quả bước t, nên mỗi token cần một lượt forward qua mạng.

Paper nhắc lại luận điểm cổ điển: decode là **memory-bandwidth-bound** (bị chặn bởi băng thông bộ nhớ, không phải bởi phép tính). Mỗi forward phải kéo *toàn bộ trọng số* từ HBM (bộ nhớ GPU) lên cache/thanh ghi, nhưng chỉ sinh ra **một** token, nên khả năng tính toán của GPU có thể chưa được tận dụng hết.

Một số thuật ngữ:

- **Arithmetic intensity** (cường độ số học) = FLOPs / bytes di chuyển. Decode batch-1 chỉ ~1 FLOP/byte (mỗi trọng số FP16 = 2 byte, dùng cho 2 FLOP: 1 nhân + 1 cộng). GPU hiện đại cần hàng chục–trăm FLOP/byte mới bão hoà tính toán.
- **Roofline model**: thời gian ≈ max(FLOPs / đỉnh tính toán, bytes / băng thông). Dưới "điểm gập" (ridge point) thì bị chặn bởi bộ nhớ.
- **Speculative decoding** (bài 48): một draft model nhỏ đoán trước γ token, mô hình lớn kiểm chứng *cả cụm* trong một forward. Vì forward kiểm chứng nhiều token có thể tăng ít thời gian khi còn bị giới hạn bởi băng thông bộ nhớ, nếu nhiều token được chấp nhận thì tiết kiệm được nhiều bước.

Hai nỗi đau của speculative decoding mà bài báo nêu: (1) khó kiếm/huấn luyện một draft model đủ nhỏ mà vẫn giống mô hình đích (ví dụ SpecInfer báo cáo 275 giờ A100 để pre-train draft); (2) phải chạy 2 mô hình trong hệ thống phân tán, dễ lệch phân phối (distribution shift) giữa draft và target.

2. Ý tưởng chính

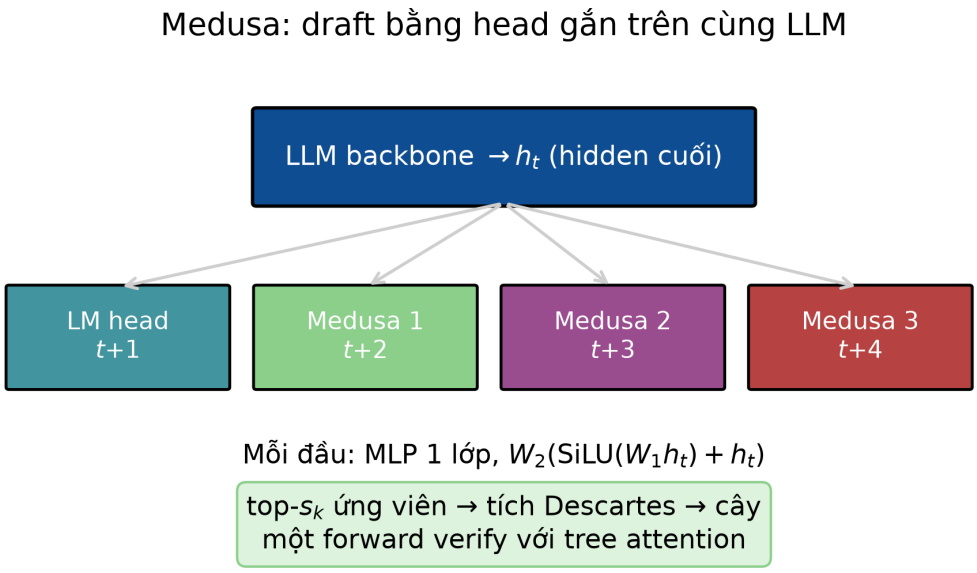
Thay vì một draft model riêng, Medusa gắn **K đầu nhỏ (MLP 1 lớp)** lên hidden state cuối cùng của chính LLM. Đầu thứ k dự đoán token ở vị trí $t+k+1$ (đầu LM gốc dự đoán $t+1$). Mỗi đầu trả về top-s_k ứng viên; tích Descartes của các ứng viên tạo thành một **cây** các chuỗi ứng viên. Cây được kiểm chứng đồng thời trong **một** forward của mô hình gốc nhờ **tree attention** (mặt nạ attention theo cấu trúc cây). Token nào khớp thì được nhận; lấy tiền tố hợp lệ dài nhất. Nhờ vậy mỗi bước sinh trung bình ~3 token thay vì 1, với overhead mỗi bước chỉ ~1.2×.

Có hai chế độ huấn luyện: **Medusa-1** (đóng băng backbone, chỉ train các đầu, "lossless" cho backbone) và **Medusa-2** (train chung với backbone bằng công thức đặc biệt, speedup cao hơn).

3. Phương pháp chi tiết

3.1 Medusa head

Gọi $h_t \in \mathbb{R}^d$ là hidden state cuối tại vị trí t, V là kích thước từ vựng. Đầu thứ k:



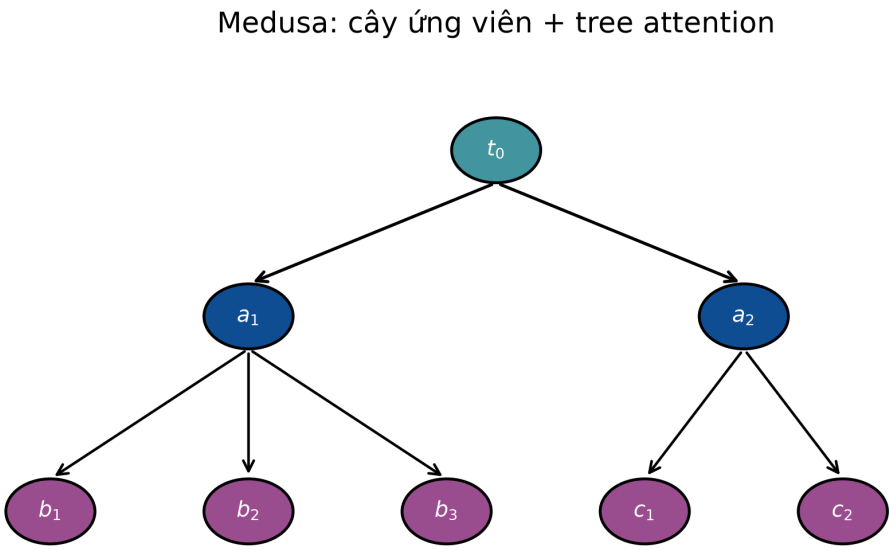
Medusa: nhiều decoding head MLP trên h_t; top-s_k → cây ứng viên; verify một forward. (minh hoạ, tự tính)

$$\boxed{p_t^{(k)} = \text{softmax}\big(W_2^{(k)}\,(\text{SiLU}(W_1^{(k)} h_t) + h_t)\big), \quad W_1^{(k)} \in \mathbb{R}^{d\times d},\ W_2^{(k)} \in \mathbb{R}^{d\times V}}$$

(paper viết $W_2 \cdot (\text{SiLU}(W_1 h_t) + h_t)$, tức một FFN một lớp có residual). **Khởi tạo:** $W_2^{(k)}$ copy từ LM head gốc, $W_1^{(k)} = 0$, nên lúc bắt đầu mỗi đầu cho đúng phân phối của LM head gốc (và residual làm đầu ra ban đầu bằng $p^{(0)}$). Paper khuyến nghị tối đa **5 đầu**; đôi khi 3–4 đầu là đủ khi dùng cây tối ưu.

3.2 Tree attention

Với top-s_k ứng viên từ đầu k, tập ứng viên là tích Descartes. Ví dụ Hình 2 của paper: $s_1 = 2,\ s_2 = 3$ cho $2 \times 3 = 6$ chuỗi ứng viên. Tổng số token mới cần xử lý (theo công thức của paper) là



Cây Medusa ví dụ s_1=2, s_2=3 → 8 nút; tree attention verify. (minh hoạ, tự tính)

$$\sum_{k=1}^{K} \prod_{i=1}^{k} s_i \quad \text{(ví dụ } 2 + 2 \cdot 3 = 8 \text{ token, chưa kể token gốc từ LM head)}.$$

Mặt nạ cây: mỗi token chỉ được attend tới *tổ tiên* của nó trong cây (và toàn bộ ngữ cảnh trước đó), không attend sang nhánh khác. Chỉ số vị trí (positional index) cũng được đặt theo độ sâu trong cây. Như vậy nhiều ứng viên được xử lý trong **cùng một batch-1 forward**, không cần nhân batch.

3.3 Chấp nhận ứng viên

- **Rejection sampling** như speculative decoding gốc (giữ đúng phân phối), nhưng paper nói khi nhiệt độ tăng thì hiệu quả giảm, và không giúp thêm tăng tốc.
- **Typical acceptance** (đề xuất mới): chấp nhận token x_{n+k} nếu

$$p_{\text{orig}}(x_{n+k} \mid x_{<n+k}) > \min\big(\epsilon,\ \delta \exp(-H(p_{\text{orig}}(\cdot \mid x_{<n+k})))\big)$$

với H là entropy, ϵ ngưỡng cứng, δ ngưỡng phụ thuộc entropy (lấy cảm hứng từ truncation sampling của Hewitt et al., 2022; trong thí nghiệm đặt $\alpha = \sqrt{\epsilon}$ theo Hewitt). Token đầu luôn nhận bằng greedy để mỗi bước chắc chắn tiến ít nhất 1 token. Ở nhiệt độ 0 nó thoái hoá thành greedy decoding. **Lưu ý:** typical acceptance *không* đảm bảo giữ đúng phân phối của mô hình gốc (paper nói rõ là không cần khớp chính xác).

3.4 Huấn luyện

Medusa-1 (backbone đóng băng). Loss cross-entropy theo từng đầu, có trọng số λ_k (đặt $\lambda_k = 0.8^k$ vì đầu xa khó hơn):

$$\boxed{\mathcal{L}_{\text{Medusa-1}} = \sum_{k=1}^{K} -\lambda_k \log p_t^{(k)}(y_{t+k+1})}$$

Vì backbone chỉ cung cấp hidden state, có thể dùng backbone lượng tử hoá 4-bit (kiểu QLoRA) để train trên một GPU phổ thông; Vicuna-7B + 60k mẫu ShareGPT: ~5 giờ trên 1 A100 PCIe.

Medusa-2 (train chung). Cần ba mẹo để không phá năng lực backbone:

1. *Combined loss:* $\mathcal{L}_{\text{Medusa-2}} = \mathcal{L}_{\text{LM}} + \lambda_0 \mathcal{L}_{\text{Medusa-1}}$ (giữ cross-entropy của LM head gốc).
2. *Differential learning rates:* lr của các đầu lớn hơn backbone (phụ lục B: lớn gấp 4 lần).
3. *Heads warmup:* hai giai đoạn: train các đầu trước như Medusa-1, sau đó train chung (hoặc tăng dần λ_0), vì loss ban đầu của đầu rất lớn sẽ làm lệch backbone (viện dẫn Kumar et al., 2022).

Self-distillation (khi không có dữ liệu train gốc / mô hình đã qua RLHF). Dùng chính mô hình sinh câu trả lời cho các prompt từ một tập hạt giống (ShareGPT, UltraChat), rồi train đầu trên dữ liệu đó. Với Medusa-2 còn cần loss chưng cất cho backbone:

$$\mathcal{L}_{\text{LM-distill}} = \text{KL}\big(p^{(0)}_{\text{orig},t} \,\|\, p^{(0)}_t\big)$$

Mẹo bộ nhớ: dùng **LoRA** cho backbone thì "mô hình gốc" = mô hình tắt adapter, không cần giữ hai bản trọng số. Paper khuyên dùng LoRA không lượng tử hoá trong trường hợp này (nếu không, "giáo viên" là bản lượng tử hoá, chất lượng thấp hơn).

3.5 Tối ưu cấu trúc cây

Cây Descartes đầy đủ lãng phí. Với ngân sách tổng số nút cố định, ước lượng độ chính xác $a_k^{(i)}$ của dự đoán top-i ở đầu k trên tập calibration (độc lập giả định). Kỳ vọng độ dài chấp nhận của một cây là tổng, qua mọi nút, của tích độ chính xác dọc đường từ gốc:

$$\mathbb{E}[\text{accept length}] = \sum_{[i_1,\ldots,i_k]\in\mathcal{I}} \prod_{j=1}^{k} a_j^{(i_j)}$$

Thêm nút theo **tham lam**: mỗi lần thêm nút kề cây hiện tại có tích độ chính xác lớn nhất. Cây Hình 6: 64 nút, sâu 4 (4 đầu), calibrate trên Alpaca-Eval.

3.6 Pseudo-code một bước decode

```
# trạng thái: KV cache của ngữ cảnh đã được nhận
def medusa_step(model, heads, context, tree):   # tree: cấu trúc (s_1..s_K) hoặc cây thưa
    # (1) Sinh ứng viên - dùng hidden state cuối của bước TRƯỚC
    t0 = argmax(LM_head(h_last))                  # token gốc
    cand = []
    for k in 1..K: cand_k = topk(heads[k](h_last), s_k)
    tokens = build_tree_tokens(t0, cand_k, tree)  # danh sách nút + mặt nạ cây + vị trí
    # (2) Một forward duy nhất với tree attention
    logits, h_all = model.forward(tokens, attn_mask=tree.mask, pos=tree.pos, kv=cache)
    # (3) Chấp nhận (greedy / typical acceptance)
    best = longest_prefix_accepted(tokens, logits, tree)   # so với p_orig
    cache.keep(best.path)                          # giữ KV của nhánh được nhận
    h_last = h_all[best.last_node]                 # dùng cho bước kế (không tốn thêm
    ↪  forward)
    return best.tokens                             # >= 1 token
```

Điểm tinh tế: logits của bước (2) dùng luôn cho vừa kiểm chứng vừa sinh ứng viên của bước sau, nên mỗi bước chỉ **một** forward lớn.

4. Thí nghiệm và kết quả chính

Thiết lập: benchmark **MT-Bench** (hội thoại nhiều lượt, GPT-4 chấm 0–10); **batch size 1** (đại diện chạy LLM cục bộ); baseline = triển khai HuggingFace mặc định. Mô hình: Vicuna-7B/13B (huấn luyện trên ShareGPT công khai), Vicuna-33B (dữ liệu riêng), Zephyr-7B (SFT + alignment).

Định nghĩa (Phụ lục B.1): acceleration rate = số token trung bình mỗi bước giải mã (baseline = 1.0); overhead = latency mỗi bước so với vanilla; **Speedup = Acceleration rate / Overhead**.

Kết quả (Hình 3, Mục 3.1)	Medusa-1	Medusa-2
Vicuna-7B	2.18×	2.83×
Vicuna-13B	2.33×	2.83×

Theo hạng mục MT-Bench cho Medusa-2 Vicuna-7B: coding 3.29×, extraction 3.62× (cao nhất).

Bảng 1 (Medusa-2, self-distillation cho 33B và Zephyr):

	Vicuna-7B	Zephyr-7B	Vicuna-13B	Vicuna-33B
Acc. rate (token/bước)	3.47	3.14	3.51	3.01
Overhead	1.22	1.18	1.23	1.27
Chất lượng MT-Bench (Δ so với gốc)	6.18 (+0.01)	7.25 (−0.07)	6.43 (−0.14)	7.18 (+0.05)
Speedup Medusa	2.83	2.66	2.83	2.35
Speedup speculative decoding (draft rời)	1.47	-	1.56	1.60

Kiểm tra: 3.47 / 1.22 = 2.84 ≈ 2.83 (khớp công thức speedup).

Phụ lục F, AlpacaEval, Medusa-2 (tokens/s): Vicuna-7B 37.07 → 106.76 (2.88×, acc. 3.23); Vicuna-13B 29.01 → 91.54 (3.16×, 3.28); Vicuna-33B 17.87 → 40.43 (2.26×, 2.85); Zephyr-7B 34.21 → 99.50 (2.91×, 3.08).

Paper tóm tắt: Medusa-1 > 2.2×, không đổi chất lượng; Medusa-2 2.3–2.8×.

5. Phân tích / ablation

Đóng góp của từng kỹ thuật (Bảng 3, Vicuna-7B):

Kỹ thuật	Speedup
Đầu Medusa-1, chưa có tree attention	~1.5×
+ tree attention	~1.9×
+ cấu trúc cây tối ưu	~2.2×
+ huấn luyện Medusa-2	~2.8×

Cây thưa vs. cây đặc (Hình 4): cây thưa 64 nút có acceleration rate cao hơn cây đặc 256 nút. Số token ứng viên tăng thì acceleration rate tăng theo kiểu logarit (chậm dần), còn tốc độ thực (token/s) giảm vì phép nhân ma trận bắt đầu thành compute-bound: “nếu mức tăng acceleration nhỏ hơn overhead thì chậm đi tổng thể”.

Hai giai đoạn (Bảng 2, Vicuna-7B, chất lượng MT-Bench): gốc 6.17; fine-tune trực tiếp cả backbone + đầu: **5.925** (xuống); Medusa-1: 6.23 (speedup 2.18×); Medusa-2: 6.18 (2.83×). Train thẳng không có công thức đặc biệt làm hỏng chất lượng.

Typical acceptance (Hình 5): tăng ϵ thì chất lượng tăng nhưng acceleration rate giảm (đánh đổi rõ). Với tác vụ sáng tạo, random sampling tốt hơn greedy, và typical sampling tiệm cận random sampling khi ϵ lớn. (Số cụ thể trong hình không đọc được từ bản trích text.)

Self-distillation: Vicuna-33B có speedup thấp hơn (2.35×) nhưng giữ chất lượng; tác giả giả thuyết do dữ liệu train ẩn (private) khác dữ liệu self-distillation.

6. Phân tích bộ nhớ và độ trễ (batch 1)

6.1 Chi phí của Medusa ở batch 1

Thời gian một bước decode ≈ $\max(\text{FLOPs}/P,\ \text{bytes}/B)$. Với batch 1, bytes ≈ kích thước trọng số S_w (cộng KV cache), FLOPs ≈ $2 \cdot N_{\text{params}} \cdot n_{\text{tok}}$ với n_{tok} là số token trong forward. Kiểm chứng cây gồm n token chỉ nhân FLOPs lên n lần nhưng **bytes trọng số gần như giữ nguyên**. Chừng nào $2Nn/P < S_w/B$ (vẫn dưới ridge point), thời gian bước gần như không đổi. Phụ lục G của paper dựng roofline cho Llama-7B/13B/33B trên A100-80GB-PCIe, A40, A6000, và Bảng 5 liệt kê độ phức tạp tính toán/không gian của các toán tử khi prefill, decode, và Medusa decode (với q là độ dài ứng viên). Kết luận định tính của họ: các phép nhân ma trận trong linear/attention là nguồn overhead chính; cây càng lớn càng sớm chạm trần compute (phù hợp Hình 4b).

Chi phí các đầu dự đoán cũng cần được tính đến: mỗi đầu có ma trận đầu ra $W_2 \in \mathbb{R}^{d \times V}$, nên số tham số của nhiều đầu tăng theo tích KdV. Với từ vựng lớn hoặc mô hình nhỏ, chi phí này có thể đáng kể so với backbone; lợi ích tốc độ phải được đo cùng overhead của các đầu.

6.2 Batch lớn

Paper nêu ý tưởng khái quát cho batch lớn và nói thư viện như TensorRT, HuggingFace TGI đã hỗ trợ sau đó, nhưng **thí nghiệm của paper chỉ ở batch 1**. Khi batch tăng, GPU trở nên compute-bound sớm hơn nên cây ứng viên "miễn phí" ít đi (bài EAGLE báo cáo speedup giảm khi tăng batch, xem 50).

7. Giới hạn

Tác giả nêu:

- Hiệu năng gắn với dữ liệu huấn luyện đầu: khi dùng self-distillation (Vicuna-33B) speedup thấp hơn do khác phân phối với dữ liệu gốc.
- Cây phức tạp hơn làm tăng overhead vì matmul chuyển sang compute-bound; tăng acceleration chậm theo kiểu logarit.
- Typical acceptance không bảo đảm đúng phân phối mô hình gốc (chỉ "plausible").
- Ý kiến đạo đức: khả năng bị lạm dụng khi sinh văn bản nhanh hơn.

Nhận xét:

- Dự đoán đầu k chỉ dựa vào h_t, **không biết token nào đã được chọn ở các vị trí giữa**; độ chính xác các đầu xa thấp và độc lập (chính là điểm EAGLE chỉ ra ở bài 50).
- Giả định độc lập khi tối ưu cây là xấp xỉ thô.
- Chi phí tham số của đầu tỉ lệ $K \cdot d \cdot V$ (mục 6.2).
- Chất lượng chỉ đo bằng MT-Bench/GPT-4 (nhiễu); không có OCR/ngôn ngữ ít tài nguyên.
- Số speedup phụ thuộc phần cứng, baseline HuggingFace (chưa tối ưu như gpt-fast/CUDA graphs); với baseline mạnh hơn, tỉ lệ có thể khác.

8. Liên hệ với các bài khác

- **48 (Speculative decoding):** Medusa dùng cùng khung draft–verify nhưng thay draft model bằng các đầu; tree attention từ SpecInfer/Spector & Ré (độc lập).
- **50 (EAGLE):** sửa hai điểm yếu: đưa token đã chọn ở bước trước vào bộ dự đoán (giải bất định) và dự đoán *feature* thay vì token. EAGLE báo cáo nhanh hơn Medusa 1.5–1.6×.
- **14/15 (MQA, GQA):** giảm KV cache để chạy batch lớn; Medusa tăng utilization bằng cách giảm số bước, hai hướng bổ trợ nhau.
- **46 (PagedAttention/vLLM), 47 (SGLang):** tầng hệ thống phục vụ; Medusa nói ở batch lớn phải tích hợp vào các hệ thống đó.
- **51–53 (lượng tử hoá):** giảm bytes mỗi bước; Medusa giảm *số bước*. Hai trục nhân nhau (paper cũng nói train đầu trên backbone lượng tử hoá).
- **Chưng cất (nhóm pruning/distillation):** self-distillation của Medusa-2 dùng KL(gốc‖mới) với LoRA.

9. Tóm tắt công thức / khái niệm

Khái niệm	Công thức / ý
Medusa head	$p^{(k)} = \text{softmax}(W_2^{(k)}(\text{SiLU}(W_1^{(k)}h) + h))$; W_2 copy LM head, $W_1 = 0$
Số token cây	$\sum_k \prod_{i \le k} s_i$
Loss Medusa-1	$\sum_k -\lambda_k \log p_t^{(k)}(y_{t+k+1})$, $\lambda_k = 0.8^k$
Loss Medusa-2	$\mathcal{L}_{\text{LM}} + \lambda_0 \mathcal{L}_{\text{Medusa-1}}$ (+ KL chưng cất khi self-distill)
Typical acceptance	$p > \min(\epsilon, \delta e^{-H})$
Kỳ vọng độ dài chấp nhận	$\sum_{\text{nút}} \prod a_j^{(i_j)}$ (tối ưu cây tham lam)
Speedup	acceleration rate / overhead
Tham số thêm (tự tính)	$\approx K(d^2 + dV)$

10. Kết luận ngắn

Medusa chứng minh rằng một vài MLP nhỏ trên hidden state cuối + tree attention đủ cho speedup 2.2–2.8× ở batch 1, không cần draft model, với train rẻ (Medusa-1: vài giờ một GPU). Điểm yếu: các đầu độc lập nhau nên đoán xa kém chính xác (EAGLE, bài 50, sửa), và chi phí tham số $K \cdot d \cdot V$ có thể không còn nhỏ với mô hình nhỏ có từ vựng lớn.

Link gốc: https://arxiv.org/abs/2401.10774

Bài 50

EAGLE: Speculative Sampling Requires Rethinking Feature Uncertainty (2024)

Tác giả: Yuhui Li, Fangyun Wei, Chao Zhang, Hongyang Zhang (Peking University; Microsoft Research; University of Waterloo; Vector Institute)
Venue / arXiv: arXiv:2401.15077 (v3, 4/3/2025). Paper (bản này) không ghi venue trong text trích.
Nguồn: https://arxiv.org/abs/2401.15077
Chủ đề: Speculative sampling với draft model rất nhẹ hoạt động ở mức *feature* (hidden state áp chót), có đưa token đã chọn (dịch một bước) vào đầu vào để giải bất định; giữ nguyên phân phối đầu ra.
Vị trí trong lộ trình: Đọc sau 48 (Speculative decoding) và 49 (Medusa). Bài này giữ tính "lossless" của speculative sampling (Medusa typical acceptance thì không) nhưng đạt độ chính xác draft cao hơn nhiều. Tiếp theo: 51–53 (giảm bytes mỗi bước bằng lượng tử hoá), bổ trợ cho EAGLE (paper có thí nghiệm EAGLE + gpt-fast int4).

> Đây là ghi chú học tập bằng tiếng Việt, không phải bản dịch.

1. Bối cảnh

Speculative sampling (bài 48): giai đoạn *draft* tạo γ token ứng viên $\hat{t}_{j+1:j+\gamma}$ cùng phân phối $\hat{p}$ của chúng bằng mô hình rẻ; giai đoạn *verify* chạy MỘT forward của mô hình đích để có phân phối p tại mọi vị trí. Token $\hat{t}_{j+i}$ được chấp nhận với xác suất

$$\min\Big(1,\ \frac{p_{j+i}(\hat{t}_{j+i})}{\hat{p}_{j+i}(\hat{t}_{j+i})}\Big),$$

khi bị từ chối thì bỏ hết token sau và lấy mẫu lại từ phân phối đã hiệu chỉnh $\text{norm}(\max(0,\ p_{j+i} - \hat{p}_{j+i}))$. Đây được chứng minh tương đương với lấy mẫu trực tiếp từ mô hình đích (Leviathan et al.), nên **phân phối văn bản không đổi** ở cả greedy lẫn non-greedy.

Muốn nhanh hơn cần (a) giảm chi phí draft, (b) tăng **tỉ lệ chấp nhận** (acceptance rate). Các phương pháp trước:

- Draft model nhỏ cùng họ: khó kiếm (ví dụ không có model nhỏ hơn LLaMA2-7B; TinyLLaMA khác template instruct), và 7B làm draft cho 13B quá nặng nên có khi chậm hơn vanilla (đánh dấu N/A trong Hình 1 của paper).
- **Lookahead** (n-gram + Jacobi) và **Medusa** (MLP trên feature áp chót): draft rẻ, nhưng độ chính xác thấp, paper nói Medusa ≈ 0.6, Lookahead thấp hơn, còn EAGLE ≈ 0.8.

Thuật ngữ: **feature** ở đây = hidden state áp chót (đầu vào của LM head), kí hiệu f; **token** t → embedding e → ... → feature f_j → $p_{j+1} = \text{LMHead}(f_j)$ → token t_{j+1}.

2. Ý tưởng chính

Hai quan sát:

1. **Dự đoán tự hồi quy ở mức feature dễ hơn ở mức token.** Chuỗi feature "đều đặn" hơn chuỗi token (token là biến đổi rời rạc của ngôn ngữ). Draft dự đoán feature kế tiếp rồi dùng LM head *của mô hình đích* để ra token. Thử nghiệm Hình 4: speedup 1.9× (feature) so với 1.5× (token).

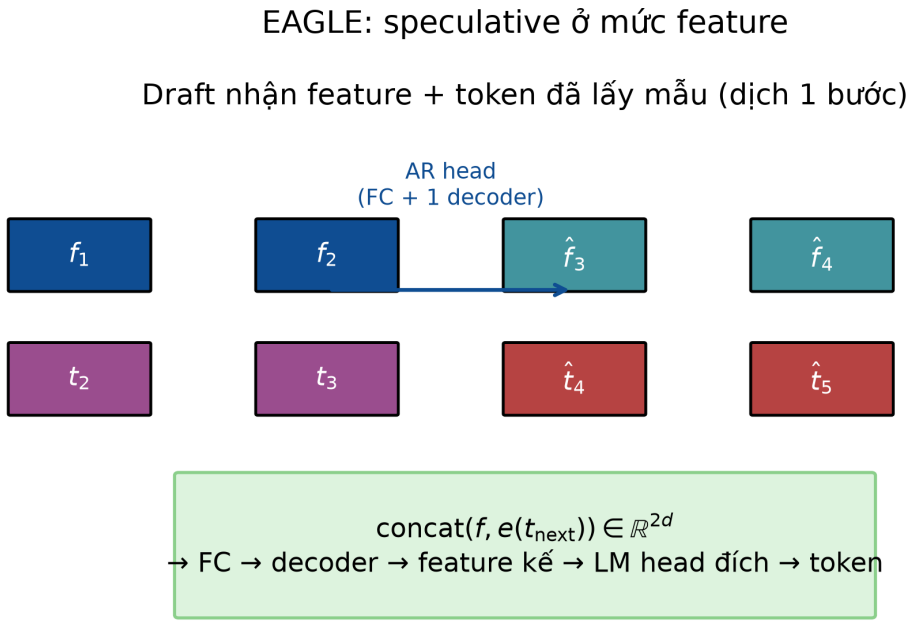
2. **Bất định do lấy mẫu giới hạn dự đoán feature.** Từ f_I, token kế có thể là "am" hoặc "always" tuỳ kết quả lấy mẫu, và hai nhánh cho hai chuỗi feature khác nhau; chỉ từ f_I không xác định được feature kế tiếp. Medusa bị đúng vấn đề này khi đoán token cách xa. **Giải pháp:** cho draft nhận thêm *chuỗi token đã được lấy mẫu dịch trước một bước* (t_{am} hoặc t_{always}). Speedup tăng từ 1.9× lên 2.8× (Vicuna-7B, Hình 4).

Kết quả: draft chỉ là **một lớp decoder + một lớp FC** mượn embedding và LM head của mô hình đích; xác suất chấp nhận cao hơn Medusa/Lookahead và vẫn giữ đúng phân phối.

3. Phương pháp chi tiết

3.1 Giai đoạn draft

Draft model gồm: (i) lớp Embedding và (ii) LM Head, *dùng chung và đóng băng* từ mô hình đích; (iii) **Autoregression Head** = FC + một decoder layer. Đầu vào là chuỗi feature $(bs,\ seq,\ d)$ và chuỗi token **dịch trước một bước** $(bs,\ seq)$:



Note: acceptance ~0.8 (vs Medusa ~0.6); giữ đúng phân phối p

EAGLE: draft tự hồi quy ở mức feature + token dịch một bước; LM head đích; acceptance ~0.8. (minh hoạ, tự tính)

$$\hat{f}_{i+1} = \text{AR-Head}\Big(\text{FC}\big([\, f_{1:i}\ ;\ \text{Emb}(t_{2:i+1})\,]\big)\Big), \qquad p_{i+2} = \text{softmax}(\text{LMHead}(\hat{f}_{i+1}))$$

Concat trên chiều kênh cho tensor $(bs,\ seq,\ 2d)$; FC nén về d; decoder layer dự đoán feature kế. Ví dụ Hình 5/6: dự đoán f_3 từ (f_1, f_2) và (t_2, t_3); từ $p_4 = \text{LMHead}(f_3)$ lấy mẫu t_4; ghép (f_3, t_4) vào đầu vào để dự đoán f_4, v.v.

Draft dạng cây: EAGLE sinh cây ứng viên bằng tree attention: cây có độ sâu m nhưng nhiều hơn m token qua m forward của draft. Ví dụ Hình 6: 10 token qua 3 forward. Cấu trúc thực dùng trong mọi thí nghiệm

(Hình 9, Phụ lục A.1): **5 forward của draft**, gốc có $k = 4$ nhánh; cây chọn theo trực giác (nhánh xác suất cao nên sâu và rộng hơn), **chưa tối ưu chặt**. Greedy: lấy top-k làm con; non-greedy: lấy mẫu k token.

3.2 Huấn luyện draft (mô hình đích đóng băng)

Hồi quy feature (Smooth L1) cộng phân loại token qua LM head cố định:

$$\mathcal{L}_{\text{reg}} = \text{SmoothL1}\big(f_{i+1},\ \text{Draft}(T_{2:i+1}, F_{1:i})\big), \quad \mathcal{L}_{\text{cls}} = \text{CE}(p_{i+2}, \hat{p}_{i+2}), \quad \boxed{\mathcal{L} = \mathcal{L}_{\text{reg}} + w_{\text{cls}}\mathcal{L}_{\text{cls}},\ w_{\text{cls}} = 0.1}$$

($\mathcal{L}_{\text{cls}}$ lớn hơn $\mathcal{L}_{\text{reg}}$ khoảng một bậc độ lớn nên trọng số 0.1.) Chi tiết:

- Dữ liệu: **dataset cố định ShareGPT, 68,000 lượt hội thoại**, không cần sinh lại bằng mô hình đích (ablation cho thấy ít nhạy cảm).
- Chống tích luỹ sai số feature (draft tự hồi quy trên feature dự đoán): **thêm nhiễu** $U(-0.1, 0.1)$ vào feature thật khi huấn luyện (theo NEFTune, Jain et al. 2023).
- lr 3×10^{-5}, AdamW $(\beta_1, \beta_2) = (0.9, 0.95)$, gradient clip 0.5.
- Tham số huấn luyện: **0.24B / 0.37B / 0.56B / 0.99B** cho 7B / 13B / 33B / 70B; Mixtral 8x7B: 0.28B. Train 1–2 ngày trên 4×A100-40G cho 70B; 7B/13B/33B có thể train trên node RTX 3090 trong 1–2 ngày. Chi phí train rất thấp so với TinyLLaMA (3,000B token, so với 2–4B token của EAGLE).

3.3 Giai đoạn verify

Mô hình đích tính xác suất của mọi nút cây trong một forward (tree attention). Tại mỗi nút áp dụng đệ quy speculative sampling đa vòng (Phụ lục A.2, như SpecInfer): sau khi một ứng viên bị từ chối, *gọi lại thuật toán A* với ứng viên anh em kế; chỉ khi tất cả bị từ chối mới lấy mẫu trực tiếp từ phân phối hiệu chỉnh. Nhờ đó **phân phối đầu ra không đổi** (bảo đảm lý thuyết, cả greedy và non-greedy). Các token được chấp nhận và feature của chúng được lưu để dùng cho vòng draft tiếp theo.

3.4 Pseudo-code

```
def eagle_cycle(target, ar_head, state):          # state: KV cache, (f_1..f_i),
↪  (t_2..t_{i+1})
    # --- DRAFT: m = 5 forward nhẹ, xây cây ---
    tree = {root: (f_i, t_{i+1})}
    for depth in 1..5:
        for node in frontier(tree):
            f_hat = ar_head(FC(concat(node.f, Emb(node.t_next))))   # 1 FC + 1 decoder
            ↪  layer
            p_hat = softmax(LM_head(f_hat))                          # LM head của
            ↪  target (cố định)
            for tok in topk_or_sample(p_hat, k=branching[depth]):    # greedy: top-k;
            ↪  else: sample
                tree.add_child(node, token=tok, feature=f_hat)
    # --- VERIFY: 1 forward của target ---
    p_tree = target.forward(tree.tokens, attn_mask=tree.mask)        # tree attention
    accepted = multi_round_speculative_sampling(tree, p_tree)        # giữ nguyên phân
    ↪  phối
    state.update(accepted.tokens, accepted.target_features)          # feature THẬT dùng
    ↪  cho chu kỳ sau
    return accepted.tokens                                           # trung bình tau ~
    ↪  3.2-4.5
```

4. Thí nghiệm và kết quả chính

Thiết lập: Vicuna 7B/13B/33B, LLaMA2-Chat 7B/13B/70B, Mixtral 8x7B Instruct; MT-Bench (hội thoại), HumanEval (code), GSM8K (toán), Alpaca (làm theo hướng dẫn); **batch 1** là chính (theo speculative sampling gốc); *một* bộ trọng số EAGLE huấn luyện chỉ trên ShareGPT dùng cho mọi tác vụ (zero-shot). Chỉ so với các phương pháp speculative không cần fine-tune backbone (Lookahead, Medusa, speculative sampling gốc, DistillSpec). Chất lượng sinh không đánh giá vì bảo đảm lý thuyết không đổi.

Metric: walltime speedup, **average acceptance length** τ (số token chấp nhận mỗi forward của target), acceptance rate n-α (khi đầu vào draft có n feature dự đoán, có thể sai).

Bảng 1 (speedup / τ; T = nhiệt độ; V = Vicuna; LC = LLaMA2-Chat):

	HumanEval	GSM8K	Alpaca
T=0, V 7B	3.33× / 4.29	3.01× / 4.00	2.79× / 3.86
T=0, V 13B	3.58× / 4.39	3.08× / 3.97	3.03× / 3.95
T=0, V 33B	3.67× / 4.28	3.25× / 3.94	2.97× / 3.61
T=0, LC 7B	3.17× / 4.24	2.91× / 3.82	2.78× / 3.71
T=0, LC 13B	3.76× / 4.52	3.20× / 4.03	3.01× / 3.83
T=0, LC 70B	3.52× / 4.42	3.03× / 3.93	2.97× / 3.77
T=1, V 7B	2.39× / 3.43	2.34× / 3.29	2.21× / 3.30
T=1, LC 13B	2.89× / 3.78	2.82× / 3.67	2.66× / 3.55
T=1, LC 70B	2.92× / 3.76	2.74× / 3.58	2.65× / 3.47

(Bảng đầy đủ có 12 dòng; trên là một phần.) Tác vụ code (HumanEval) nhanh nhất do nhiều khuôn mẫu cố định. Paper: nhanh hơn Lookahead 1.70–2.08× và Medusa 1.47–1.60×; LLaMA2-Chat 70B đạt 2.7–3.5× latency speedup, gấp đôi throughput. Speculative sampling gốc: không có draft hợp lý cho 7B; 13B không nhanh hơn; 33B 1.12×, 70B 1.88×.

Acceptance (Bảng 2, MT-Bench): T=0 Vicuna 7B: $\tau = 3.94$, 0-$\alpha = 0.79$, 1-$\alpha = 0.74$, 2-$\alpha = 0.72$, 3-$\alpha = 0.73$, 4-$\alpha = 0.67$. Sự chênh nhỏ giữa 1-α…4-α cho thấy EAGLE **bền với sai số feature** (ít tích luỹ lỗi).

Mixtral 8x7B Instruct (Bảng 3): speedup 1.50×, $\tau = 3.25$. Chậm hơn dense vì MoE chỉ đọc 2 expert mỗi token ở decode thường, còn khi verify nhiều token có thể phải đọc nhiều expert hơn.

EAGLE + gpt-fast (Bảng 4, LLaMA2-Chat 7B, MT-Bench T=0, 1 RTX 3090):

Precision	FP16	int4
Vanilla (HuggingFace)	24.5 tok/s	N/A
gpt-fast	55.1	106.9
EAGLE + gpt-fast	100.2	**160.4**

Batch lớn (Bảng 7, MT-Bench T=0):

Batch	1	2	3	4	Throughput (bs lớn nhất)
Vicuna 7B	2.90×	2.87×	2.65×	2.76×	1.97×
LLaMA2-Chat 70B	3.01×	2.81×	2.50×	2.40×	1.99×

Trên RTX 3090 24 GB với Vicuna 7B, bs tối đa của vanilla là 8, của EAGLE là 7 (EAGLE tốn thêm chút bộ nhớ CUDA).

5. Phân tích / ablation

Tree attention (Bảng 5, Hình 7): cây tăng τ thêm 0.6–0.8 (Vicuna 7B: chain 3.20 → tree 3.94, +0.74; LLaMA2-Chat 7B: 3.00 → 3.62, +0.62) nhưng speedup chỉ tăng ~0.3–0.5 vì nhiều token hơn mỗi forward. Chưa dùng cây EAGLE vẫn ~2.3–2.7×. Cây không tăng số forward, chỉ tăng số token mỗi forward.

Đầu vào của draft (Hình 8, Vicuna 7B), bốn kiểu: feature&shifted-token (EAGLE), feature&unshifted-token, token, feature. Ba nhận xét của paper:

1. Khi draft ít tham số, dùng feature nhỉnh hơn token.
2. Gộp feature + token (không dịch) chỉ tăng nhẹ; token rời rạc, không lỗi, giúp giảm tích luỹ sai số (1-α cải thiện, 0-α giống feature-only).
3. **Giải quyết tính ngẫu nhiên của lấy mẫu bằng dịch token một bước là cải thiện lớn nhất**, "không thêm độ phức tạp".

Dữ liệu huấn luyện (Bảng 6, LLaMA2-Chat 7B, T=0): ShareGPT cố định 2.78× ($\tau = 3.62$) vs. câu trả lời do chính target sinh 2.88× ($\tau = 3.75$). Chênh nhỏ ⇒ ít nhạy cảm với dữ liệu, chọn dataset cố định để rẻ.

Distillation draft (so với DistillSpec): distill cải thiện tỉ lệ chấp nhận nhưng nút cổ chai của speculative sampling là *overhead của draft*, nên cải thiện hạn chế.

6. Phân tích bộ nhớ và độ trễ (batch 1)

6.1 Mô hình chi phí một chu kỳ

Một chu kỳ = m forward của draft + 1 forward của target, sinh trung bình τ token:

$$\text{Speedup} \approx \frac{\tau}{1+\rho}, \qquad \rho = \frac{m \cdot T_{\text{draft}}}{T_{\text{target}}} \ (+ \text{ overhead cây})$$

Khi bị chặn bởi bộ nhớ, $T \approx$ bytes đọc$/B$. Kiểm tra bằng số LLaMA2-7B (tự tính): draft layer ≈ FC $2d \to d$ (8192×4096 = 33.5M) + decoder layer (attention $4d^2$ = 67M + MLP $3 \cdot d \cdot 11008$ = 135M) ≈ 0.236B, khớp con số 0.24B paper nêu. Mỗi forward draft đọc ≈ 0.47 GB (layer) + LM head (4096×32000 = 131M tham số = 0.26 GB) ≈ 0.73 GB; 5 forward ≈ 3.7 GB; target ≈ 13.5 GB ⇒ $\rho \approx 0.27$; với $\tau = 3.94$ cho speedup ≈ 3.1, so với 2.90× đo được (phần chênh là overhead cây, KV, launch). Hợp lý.

6.2 Batch

Paper nói rõ cơ chế: suy luận LLM bị chặn bởi bộ nhớ nên speculative sampling tận dụng tài nguyên tính toán dư; **tăng batch thì chỗ dư giảm và speedup giảm** (Bảng 7: 2.90× → ~2.7× ở bs 2–4; throughput gấp ~2×). Tác giả gợi ý cây nhỏ hơn khi batch lớn. Batch 1 là chế độ thuận lợi nhất.

7. Giới hạn

Tác giả nêu:

- Cây chưa được tối ưu chặt; cấu trúc tối ưu có thể phụ thuộc ngữ cảnh và batch (Phụ lục A.1).
- Với MoE (Mixtral) tăng tốc khiêm tốn (1.5×) vì verify nhiều token phải đọc nhiều expert.
- Speedup giảm khi tăng batch; EAGLE cần thêm chút bộ nhớ CUDA.
- Chỉ so với các phương pháp không fine-tune backbone, nên không đo Medusa-2/lossy.

Nhận xét:

- Chất lượng sinh không đánh giá thực nghiệm (dựa vào chứng minh lý thuyết); bảo đảm "không đổi phân phối" nói về toán học, còn khác biệt số học FP16 giữa batch-shape khác nhau vẫn có thể làm lệch lựa chọn greedy ở các token sát nhau (nhận xét này chưa được paper đo).

- Huấn luyện chỉ trên ShareGPT/chat; chưa có dữ liệu OCR, đa phương thức, ngôn ngữ không phải tiếng Anh.
- Draft phụ thuộc feature của một kiến trúc Transformer chuẩn; chưa thử mô hình linear attention/hybrid.
- Baseline của một số so sánh (Medusa, Lookahead) lấy speedup từ báo cáo gốc của họ, không chạy lại cùng phần cứng (paper tự ghi trong chú thích Hình 1).

8. Liên hệ với các bài khác

- **48 (Speculative decoding):** EAGLE giữ nguyên toàn bộ bảo đảm lý thuyết, chỉ thay phần draft.
- **49 (Medusa):** cùng "gắn draft vào target" và tree attention. Khác: Medusa dự đoán token $t+k+1$ chỉ từ h_t, EAGLE tự hồi quy trên feature và có token đã chọn; Medusa typical acceptance không lossless. EAGLE nhanh hơn 1.5–1.6×.
- **16 (MLA), 14/15 (MQA/GQA):** giảm KV cache; trực giao với EAGLE.
- **51–53 (lượng tử hoá):** cộng gộp được, minh chứng EAGLE+gpt-fast int4 đạt 160.4 tok/s (Bảng 4).
- **44–47 (FlashAttention, PagedAttention, SGLang):** tầng kernel/serving mà EAGLE cần tích hợp khi batch lớn.

9. Tóm tắt công thức / khái niệm

Khái niệm	Công thức / ý
Xác suất chấp nhận	$\min(1,\ p(\hat{t})/\hat{p}(\hat{t}))$; từ chối $\Rightarrow$ lấy mẫu từ $\text{norm}(\max(0, p-\hat{p}))$
Đầu vào draft	$[f_{1:i};\ \text{Emb}(t_{2:i+1})] \in \mathbb{R}^{seq\times 2d}$ (token dịch trước 1 bước)
Loss	$\mathcal{L}_{\text{reg}}$ (Smooth L1) $+ 0.1\,\mathcal{L}_{\text{cls}}$ (CE qua LM head)
Nhiễu huấn luyện	$U(-0.1, 0.1)$ cộng vào feature
τ	số token chấp nhận trung bình mỗi forward của target
Speedup (xấp xỉ)	$\tau/(1+\rho)$
Kích thước draft	0.24B / 0.37B / 0.56B / 0.99B cho 7B / 13B / 33B / 70B

10. Kết luận ngắn

EAGLE chỉ ra rằng (a) dự đoán ở mức feature dễ hơn mức token và (b) cần đưa token vừa được lấy mẫu vào đầu vào của draft để loại bất định. Với một lớp decoder nhẹ, đạt $\tau \approx 3.2$–4.5, speedup 2.1–3.8× trên MT-Bench/HumanEval/GSM8K ở batch 1, giữ nguyên phân phối đầu ra, và cộng gộp được với lượng tử hoá (160.4 tok/s trên RTX 3090).

Link gốc: https://arxiv.org/abs/2401.15077

Bài 51

LLM.int8(): 8-bit Matrix Multiplication for Transformers at Scale (2022)

Tác giả: Tim Dettmers, Mike Lewis, Younes Belkada, Luke Zettlemoyer (University of Washington; Facebook AI Research; Hugging Face; ENS Paris-Saclay)
Venue / arXiv: arXiv:2208.07339 (v2, 10/11/2022); NeurIPS 2022 (36th Conference)
Nguồn: https://arxiv.org/abs/2208.07339
Chủ đề: Lượng tử hoá INT8 sau huấn luyện cho LLM đến 175B tham số: lượng tử hoá theo vector (vector-wise) + tách các chiều đặc trưng ngoại lai (outlier features) ra nhân ma trận 16-bit (mixed-precision decomposition).
Vị trí trong lộ trình: Bài nền cho nhóm lượng tử hoá. Cần biết 01 (Transformer), 05 (LLaMA, để hiểu kích thước mô hình) và nên đọc cùng 21 (survey nén LLM, nhóm pruning). Tiếp theo: **52 (SmoothQuant)** dùng chính hiện tượng outlier này nhưng giải bằng phép biến đổi tương đương thay vì tách FP16; **53 (GPTQ)** lượng tử hoá *chỉ trọng số* xuống 3–4 bit; 54 (AWQ).

> Đây là ghi chú học tập bằng tiếng Việt, không phải bản dịch. Phần "Kiến thức nền về lượng tử hoá" được bổ sung cho người mới học; công thức zero-point viết theo dạng chuẩn vì bản trích text của paper bị lỗi ký hiệu ở phương trình (2)–(3). Số "tự tính" là phép tính diễn giải.

1. Kiến thức nền về lượng tử hoá

Lượng tử hoá (quantization): thay số thực độ chính xác cao (FP16/FP32) bằng số nguyên ít bit (INT8: 256 mức; INT4: 16 mức) cộng một vài hằng số để khôi phục độ lớn. Lợi ích: (a) trọng số nhỏ hơn (INT8 = ½ FP16), (b) ít byte phải đọc từ HBM, (c) có thể dùng lệnh nhân số nguyên nhanh (INT8 Tensor Core). Cái giá: **sai số làm tròn**.

Lượng tử hoá đối xứng theo giá trị tuyệt đối lớn nhất (absmax). Với tensor X FP16, hằng số tỉ lệ (**scale**) $s = 127/\max|X|$, và

$$X_{i8} = \text{round}(s \cdot X), \qquad \hat{X} = X_{i8}/s \text{ (khôi phục)}.$$

Đây là công thức (1) của paper: $X_{i8} = \lfloor 127 \cdot X_{f16}/\|X_{f16}\|_\infty \rceil$. "Bước lượng tử" là $\Delta = 1/s = \max|X|/127$; sai số làm tròn tối đa mỗi phần tử là $\Delta/2$. Thuật ngữ: round = làm tròn tới số nguyên gần nhất; $\|\cdot\|_\infty$ = giá trị tuyệt đối lớn nhất.

Lượng tử hoá bất đối xứng với zero-point. Nếu phân phối lệch (ví dụ đầu ra ReLU toàn dương) thì absmax bỏ phí một nửa dải $[-127, 0)$. Giải pháp: co giãn *và dịch* sao cho $[\min X, \max X]$ phủ kín $[-127, 127]$. Dạng chuẩn (viết lại theo dạng chuẩn):

$$nd = \frac{2 \cdot 127}{\max X - \min X}, \qquad X_{i8} = \text{round}\big(nd \cdot (X - \min X)\big) - 127,$$

zero-point là số nguyên mà giá trị thực 0 (hoặc min) ánh xạ vào. Nhược điểm thực tế mà bài báo nêu: nhân hai tensor có zero-point cần khai triển $(A + zp_a)(B + zp_b) = AB + A\,zp_b + B\,zp_a + zp_a zp_b$ (phương trình 5), trên GPU/TPU không có lệnh nhân int16 sẵn nên chậm; vì thế absmax là cách phổ biến nhất.

Ví dụ số (tự tính), tensor $x = [0.1,\ -0.5,\ 2.0,\ 0.03]$:

- absmax: $s = 127/2 = 63.5$; $q = \text{round}([6.35, -31.75, 127, 1.9]) = [6, -32, 127, 2]$; khôi phục $[0.094, -0.504, 2.0, 0.031]$: sai số nhỏ.
- Thêm một outlier: $x = [0.1, -0.5, 2.0, 0.03, \mathbf{60}]$: $s = 127/60 = 2.117$; $q = \text{round}([0.21, -1.06, 4.23, 0.06, 127]) = [0, -1, 4, 0, 127]$. Khôi phục $0.1 \to 0$, $0.03 \to 0$, $-0.5 \to -0.47$, $2.0 \to 1.89$. **Một outlier kéo giãn thang đo, khiến phần lớn giá trị nhỏ bị làm tròn về 0.** Đây chính là bệnh mà cả bài này xử lý.

Độ chi tiết (granularity): dùng *một* scale cho cả tensor (per-tensor), hay mỗi hàng/cột/nhóm một scale. Càng mịn càng chính xác nhưng càng tốn scale và kernel phức tạp.

Ma trận nhân INT8. $C = XW$ với X_{i8}, W_{i8} nhân bằng số nguyên, tích luỹ vào **INT32** (tránh tràn), rồi **dequantize** bằng cách nhân với $1/(s_x s_w)$ trả về FP16: $C_{f16} \approx \frac{1}{s_x s_w}(X_{i8} W_{i8})$ (phương trình 6 của paper).

2. Bối cảnh

Theo bài báo, ở LLM từ 6.7B tham số trở lên, các lớp FFN và phép chiếu của attention chiếm khoảng 95% tham số và 65–85% phép tính. Những nghiên cứu lượng tử hóa INT8 trước đó chủ yếu xét mô hình dưới 350M tham số, thường cần tinh chỉnh lại; khi mở rộng lên khoảng 2.7B–6.7B, perplexity bắt đầu xấu đi. Bài báo tìm hiểu ngưỡng và nguyên nhân của hiện tượng này, đồng thời hướng tới chuyển checkpoint 16/32-bit của mô hình tới 175B sang INT8 mà không làm giảm hiệu năng đáng kể.

3. Ý tưởng chính

Paper tìm ra hai điều. (1) Cần độ chính xác cao hơn khi lên ≥ 1B tham số: dùng **một scale riêng cho mỗi tích vô hướng** (mỗi hàng của X, mỗi cột của W), gọi là vector-wise quantization. (2) Từ khoảng **6.7B** tham số, trong gần như mọi lớp xuất hiện một vài **chiều đặc trưng cực lớn** (outlier features, biên độ ≥ 6, lên tới ~20× các chiều khác khi mới xuất hiện) xuất hiện ở mọi token nhưng chỉ ở vài cột cố định của hidden state. Vector-wise không cứu được vì scale hàng bị outlier kéo. Giải pháp: **mixed-precision decomposition**, nhân phần cột outlier (~0.1% chiều) bằng FP16 và 99.9% còn lại bằng INT8. Hai phần kết hợp là LLM.int8().

4. Phương pháp chi tiết

4.1 Vector-wise quantization

Với $X \in \mathbb{R}^{s \times h}$ (s token, h kênh) và $W \in \mathbb{R}^{h \times o}$: mỗi hàng của X (một token) có $c_x^{(i)} = \max_j |X_{ij}|$, mỗi cột của W (một kênh đầu ra) có $c_w^{(k)}$. Mỗi phần tử $C_{ik} = \sum_j X_{ij} W_{jk}$ là một tích vô hướng, nên có thể khử chuẩn riêng:

$$\boxed{C_{f16} \approx \frac{1}{c_x \otimes c_w} \cdot C_{i32} = S \cdot \left(X_{i8} W_{i8}\right)}$$

với $\otimes$ là tích ngoài (outer product) của vector scale hàng và scale cột (phương trình 7). Với absmax, c là giá trị tuyệt đối lớn nhất của hàng/cột (nhân $127/c$ khi lượng tử). Thuận lợi phần cứng: scale chỉ ở các chiều ngoài (s và o), áp được *sau* khi nhân INT8 xong.

4.2 Mixed-precision decomposition

Outlier nằm ở một số **cột** (chiều h) của X, xuất hiện ở hầu hết token. Vì scale hàng bao phủ toàn bộ cột của hàng đó nên nó vẫn bị outlier chi phối. Cách làm: gọi $O = \{h \mid \exists \text{ phần tử trong cột } h \text{ có } |\cdot| \geq \alpha\}$, $\alpha = 6.0$ (paper: đủ để mất mát gần 0). Khi đó:

$$\boxed{C_{f16} \approx \sum_{h \in O} X^h_{f16} W^h_{f16} \;+\; S_{f16} \cdot \sum_{h \notin O} X^h_{i8} W^h_{i8}} \quad (8)$$

Phần $h \in O$: nhân FP16; phần còn lại: INT8 với vector-wise. Số chiều outlier $|O| \leq 7$ cho mô hình đến 13B, nên phần FP16 chỉ tốn khoảng **0.1% bộ nhớ** thêm; bộ nhớ vẫn giảm ≈ 50% (BLOOM-176B giảm **1.96×**).

4.3 Pseudo-code (một linear layer)

```
def llm_int8_linear(X_fp16, W_fp16, alpha=6.0):          # X: [s,h], W: [h,o]
    O = {h : max_i |X[i,h]| >= alpha}                     # chọn cột outlier TỪ
    ↪  ACTIVATION, lúc chạy
    # nhánh FP16: các cột outlier của X và hàng tương ứng của W
    out_fp16 = X_fp16[:, O] @ W_fp16[O, :]
    # nhánh INT8: phần còn lại
    Xr, Wr = X_fp16[:, ~O], W_fp16[~O, :]
    cx = Xr.abs().max(dim=1) / 127                        # scale từng hàng (token), [s]
    cw = Wr.abs().max(dim=0) / 127                        # scale từng cột (kênh ra),
    ↪  [o]
    Xq = round(Xr / cx[:,None]); Wq = round(Wr / cw[None,:])
    out_i32 = int8_matmul(Xq, Wq)                         # tích luỹ INT32
    out_fp16_2 = out_i32 * (cx[:,None] * cw[None,:])      # dequant bằng tích ngoài
    return out_fp16 + out_fp16_2                          # cộng ở FP16
```

(Trọng số W_{i8} có thể lượng tử hoá một lần và cất ở INT8; activation phải làm *động* mỗi lần chạy, kể cả việc tìm cột outlier. Đó là nguồn của overhead ở mục 7.)

4.4 Cách tìm outlier (Mục 4.1)

Một đặc trưng (chiều h_i) được tính là outlier hệ thống nếu: độ lớn ≥ 6.0, xuất hiện trong ≥ 25% số lớp, và ở ≥ 6% số vị trí chuỗi. Hai ngưỡng sau đặt sao cho mô hình nhỏ nhất (125M) chỉ có một outlier. Phân tích áp lên đầu vào các lớp K/Q/V/O của attention và lớp FFN expansion (lớp FFN contraction và hàm attention không đưa vào phân tích này).

5. Thí nghiệm và kết quả chính

Thiết lập. (i) Perplexity C4 validation trên transformer dày dạng fairseq 125M–13B (đủ nhạy để so baseline lượng tử); (ii) độ chính xác zero-shot của OPT (125M–175B) trên EleutherAI LM evaluation harness (WinoGrande, HellaSwag, PIQA, LAMBADA). GPU A40 cho phần đo perplexity.

Bảng 1: perplexity C4 (thấp hơn là tốt).

Phương pháp	125M	1.3B	2.7B	6.7B	13B
32-bit float	25.65	15.91	14.43	13.30	12.45
Int8 absmax	87.76	16.55	15.11	14.59	19.08
Int8 zeropoint	56.66	16.24	14.76	13.49	13.94
Int8 absmax row-wise	30.93	17.08	15.24	14.13	16.49
Int8 absmax vector-wise	35.84	16.82	14.98	14.13	16.48
Int8 zeropoint vector-wise	25.72	15.94	14.36	13.38	13.47
Absmax LLM.int8() (vector-wise + decomp)	25.83	15.93	14.44	13.24	**12.45**
Zeropoint LLM.int8()	25.69	15.92	14.43	13.24	12.45

Ở 13B, mọi baseline INT8 đều tệ hơn cả 6.7B (ví dụ absmax 19.08 > 6.7B absmax 14.59), còn LLM.int8() khớp FP32 (12.45). Zero-shot (Hình 1): OPT từ 125M đến 175B, LLM.int8() giữ độ chính xác 16-bit, còn baseline vector-wise 8-bit trượt dần về mức ngẫu nhiên khi outlier xuất hiện (~6.7B). **Hình 1** cũng cho thấy mốc 6.7B: “once systematic outliers occur at a scale of 6.7B parameters, regular quantization methods fail”.

Khả năng tiếp cận phần cứng (Bảng 2 của paper):

Phần cứng	Bộ nhớ GPU	Mô hình lớn nhất, 8-bit	Mô hình lớn nhất, 16-bit
8×A100	80 GB	OPT-175B / BLOOM	OPT-175B / BLOOM
8×A100	40 GB	OPT-175B / BLOOM	OPT-66B
8×RTX 3090	24 GB	OPT-175B / BLOOM	OPT-66B
4×RTX 3090	24 GB	OPT-66B	OPT-30B
Colab Pro	15 GB	OPT-13B	GPT-J-6B
Colab (free)	12 GB	T0/T5-11B	GPT-2 1.3B

6. Phân tích outlier / ablation

Bốn kết luận định lượng của Mục 4.2 (outlier là "hiện tượng nổi lên theo quy mô"):

1. **Theo số tham số**, outlier xuất hiện ở mọi lớp đột ngột giữa **6B và 6.7B**: số lớp bị ảnh hưởng tăng 65% → 100%, số vị trí chuỗi bị ảnh hưởng 35% → 75%. Trùng khớp với lúc lượng tử hoá bắt đầu hỏng.
2. **Theo perplexity**, hiện tượng này mượt (hàm mũ theo perplexity giảm), gợi ý rằng điều quyết định là chất lượng mô hình (dữ liệu, huấn luyện) chứ không chỉ kích thước. Số lượng outlier tăng *đơn điệu chặt* khi perplexity C4 giảm, còn theo kích thước thì không đơn điệu.
3. **Biên độ trung vị của outlier** tăng nhanh sau khi chúng có mặt ở mọi lớp: dải lượng tử quá rộng nên "phần lớn bin trống và giá trị nhỏ bị lượng tử về 0, xoá thông tin". Tác giả suy đoán cả huấn luyện FP16 cũng mất ổn định sau 6.7B vì dễ vượt giá trị tối đa FP16 (65,504; bản trích viết 65535).
4. **Tính hệ thống:** mô hình 6.7B, chuỗi 2048 có ~150,000 outlier mỗi chuỗi nhưng chỉ nằm ở **6** chiều ẩn; với 13B chỉ tối đa 7 chiều.

Outlier quan trọng thế nào (ablation xoá): đặt các chiều outlier bằng 0 làm xác suất softmax top-1 giảm từ ~40% xuống ~20% (giảm trên 20%) và perplexity tăng **600–1000%**, dù chỉ ~0.1% số đặc trưng. Xoá 7 chiều ngẫu nhiên thay vì outlier: top-1 giảm 0.02–0.3%, perplexity tăng 0.1%.

Vì sao zero-point thắng absmax trong Bảng 1: hầu hết outlier chỉ dương hoặc chỉ âm (bất đối xứng), zero-point dùng trọn dải. Nhưng khi đã tách outlier bằng decomposition, lợi thế đó biến mất (phần còn lại đối xứng). Zero-point cũng hỏng ở 13B vì outlier lớn quá nhanh. Vector-wise vẫn tốt hơn row-wise (Bảng 1), nghĩa là scale riêng cho từng cột trọng số cần cho độ chính xác.

7. Phân tích bộ nhớ và độ trễ (batch 1)

7.1 Bộ nhớ

Trọng số INT8 = 1 byte/tham số so với 2 byte FP16: giảm ≈ 2× (BLOOM-176B: 1.96×, phần dư là các cột outlier FP16 + scale + embedding/LM head và các lớp không lượng tử hoá). Paper **chỉ lượng tử hoá các lớp FFN và projection attention**; hàm attention (QK^T, PV) và KV cache giữ 16-bit (nêu ở phần Giới hạn).

7.2 Độ trễ: INT8 *không tự nhiên nhanh hơn*

Paper nói thẳng ở Phụ lục D: mục tiêu là bộ nhớ, và INT8 matmul chỉ thắng khi GPU được bão hoà bởi ma trận lớn, tức hidden ≥ 4096.

Bảng 5 (tốc độ so với FP16 cho lớp ẩn đầu tiên của FFN, GPT-3 các cỡ):

Model dim	768	1024	1536	2048	2560	4096	5140	12288
Int8 không tính overhead	0.99×	1.08×	1.43×	1.61×	1.63×	1.67×	2.13×	2.29×

Model dim	768	1024	1536	2048	2560	4096	5140	12288
Vector-wise (kernel tối ưu)	0.43×	0.49×	0.74×	0.91×	0.94×	1.18×	1.59×	2.00×
LLM.int8() (vector-wise + decomp)	0.14×	0.20×	0.36×	0.51×	0.64×	0.86×	1.22×	1.81×

Tức là LLM.int8() *chậm hơn FP16* với model dim ≤ 2560 (chỉ 13B và 175B có speedup). Nguyên nhân: lượng tử hoá/khử lượng tử activation, tìm outlier và tách ma trận chạy động mỗi lần. Paper gợi ý kernel CUDA tối ưu cho decomposition sẽ cải thiện nhiều.

Đầu-cuối (Bảng 6, BLOOM-176B, ms/token):

Batch	1	8	32
bfloat16, 8×A100-80GB	239	32	9.94
LLM.int8(), 8×A100-80GB	253	34	10.44
LLM.int8(), 4×A100-80GB	246	33	9.40
LLM.int8(), 3×A100-80GB	247	33	9.11

Ở batch 1, 8 GPU: 253 ms so với 239 ms (chậm ~6%). Lợi ích là chạy trên ít GPU hơn với latency gần tương đương (3 GPU: 247 ms).

8. Giới hạn

Tác giả nêu:

- Chỉ nghiên cứu kiểu dữ liệu Int8, không FP8 (GPU lúc đó chưa hỗ trợ).
- Chỉ đến 175B; có thể có hiện tượng nổi lên khác ở quy mô lớn hơn.
- **Không dùng Int8 cho hàm attention** (QK^T, PV); khám phá ban đầu cho thấy cần thêm kỹ thuật.
- Chỉ nghiên cứu suy luận; huấn luyện/fine-tune chỉ có phân tích sơ bộ ở Phụ lục E.
- Overhead lượng tử hoá làm chậm mô hình < 6.7B so với FP16; speedup lớn chỉ với ma trận rất lớn.

Nhận xét:

- Phân tích outlier dựa trên OPT, mô hình fairseq và GPT-J/GPT-2, đều là kiến trúc kiểu GPT cổ điển (post-/pre-LN, GELU); các kiến trúc hiện đại (RMSNorm, SwiGLU, GQA, hybrid linear attention) chưa kiểm. Giả thuyết "outlier gắn với LayerNorm/tần suất token" được các tác giả khác nêu (paper dẫn Kovaleva et al., Puccetti et al.) nhưng paper này không chứng minh cơ chế.
- Việc tìm cột outlier xảy ra động trên activation ⇒ khó kết hợp CUDA graph/kernel cố định, đây là chi phí thực tế.
- Không có tác vụ sinh dài, không có VLM/OCR.

9. Liên hệ với các bài khác

- **52 (SmoothQuant):** cùng chẩn đoán (activation có outlier theo kênh, trọng số phẳng). LLM.int8() giữ outlier ở FP16 (chính xác nhưng chậm, không hardware-friendly); SmoothQuant chia lại độ khó sang trọng số bằng phép co giãn tương đương, rồi lượng tử hoá thuần INT8. SmoothQuant báo cáo LLM.int8() "thường chậm hơn FP16".
- **53 (GPTQ):** coi RTN theo hàng (cách của LLM.int8() cho trọng số) là baseline và vượt hẳn ở 3–4 bit; GPTQ chỉ lượng tử hoá trọng số, bỏ qua vấn đề activation outlier.
- **54 (AWQ):** bảo vệ kênh trọng số quan trọng dựa vào activation, cùng họ.

- **44–47:** lượng tử hoá KV cache/attention là phần mà LLM.int8() không làm.
- **Pruning (nhóm 22–24, 29):** outlier features cũng lý giải vì sao cắt chiều ẩn (SliceGPT, bài 29) và các phương pháp dựa chuẩn của activation (Wanda, bài 24) nhạy với vài kênh khổng lồ; đây là một liên hệ suy luận.

10. Tóm tắt công thức / khái niệm

Khái niệm	Công thức / ý
Absmax INT8	$X_{i8} = \text{round}(127X/\max\|X\|)$, $s = 127/\max\|X\|$
Zero-point	co giãn + dịch sao cho $[\min, \max] \to [-127, 127]$
Vector-wise	scale từng hàng X (c_x) và từng cột W (c_w); $C \approx (X_{i8}W_{i8})/(c_x \otimes c_w)$
Outlier feature	kênh h có $\|X\| \geq 6$ ở ≥ 25% lớp, ≥ 6% vị trí
Mixed-precision decomposition	$\sum_{h\in O}$ FP16 $+ \sum_{h\notin O}$ INT8 (Eq. 8), $\alpha = 6.0$
Tỉ lệ outlier	≈ 0.1% chiều (≤ 7 chiều, đến 13B)
Giảm bộ nhớ	≈ 2× (BLOOM-176B: 1.96×)
Ngưỡng nổi lên	~6.7B tham số (phụ thuộc perplexity)

11. Kết luận ngắn

LLM.int8() cho thấy lượng tử hoá INT8 của LLM hỏng ở ~6.7B không phải vì "lỗi chung" mà vì một vài kênh activation khổng lồ và hệ thống; tách ~0.1% kênh đó sang FP16 + scale theo vector cho kết quả không mất mát đến 175B.

Link gốc: https://arxiv.org/abs/2208.07339

Bài 52

SmoothQuant: Accurate and Efficient Post-Training Quantization for Large Language Models (2023)

Tác giả: Guangxuan Xiao, *Ji Lin*, Mickael Seznec, Hao Wu, Julien Demouth, Song Han (MIT; NVIDIA; * đóng góp ngang nhau)
Venue / arXiv: arXiv:2211.10438 (v7, 29/3/2024); ICML 2023 (PMLR 202)
Nguồn: https://arxiv.org/abs/2211.10438
Chủ đề: Lượng tử hoá sau huấn luyện W8A8 (trọng số 8-bit, activation 8-bit) cho LLM bằng phép co giãn theo kênh tương đương toán học, "chuyển độ khó lượng tử hoá từ activation sang trọng số".
Vị trí trong lộ trình: Đọc ngay sau 51 (LLM.int8(): chẩn đoán outlier và tách FP16) và cần kiến thức nền lượng tử hoá ở mục 1 của bài 51. Bài này giải cùng vấn đề nhưng cho kernel INT8 thuần, hardware-friendly. Tiếp theo: 53 (GPTQ, chỉ trọng số, 3–4 bit), 54 (AWQ, cũng dùng co giãn theo kênh nhưng cho weight-only).

> Đây là ghi chú học tập bằng tiếng Việt, không phải bản dịch.

1. Bối cảnh

Để dùng phép nhân ma trận số nguyên INT8 GEMM, cả trọng số và activation cần được lượng tử hóa; cấu hình tám bit cho cả hai được gọi là W8A8. Lượng tử hóa chỉ trọng số, như ở bài 53, giảm lượng dữ liệu cần đọc nhưng vẫn cần khử lượng tử trước phép nhân FP16.

Vấn đề: ở LLM ≥ 6.7B, activation có outlier hệ thống (bài 51). ZeroQuant (per-token động + group-wise cho trọng số) vẫn hỏng với OPT-175B; LLM.int8() sửa được bằng tách FP16 nhưng khó triển khai hiệu quả trên phần cứng và nhiều khi chậm hơn FP16. Mục tiêu của paper: một cách **không cần huấn luyện lại, chính xác, và dùng được kernel INT8 chuẩn**, tốt nhất là per-tensor, static.

Thuật ngữ:

- **PTQ (post-training quantization):** lượng tử hoá sau huấn luyện, chỉ dùng một ít mẫu hiệu chuẩn (calibration).
- **Static vs dynamic quantization:** static = tính scale của activation *offline* từ mẫu calibration (nhanh, nhưng lệch nếu dữ liệu thật khác); dynamic = tính scale *lúc chạy* theo từng tensor/token (chính xác hơn, thêm chi phí).
- **Per-tensor / per-token / per-channel:** một scale cho cả tensor / mỗi token (hàng của X) / mỗi kênh đầu ra (cột của W). Xem Hình 3 của paper.
- **GEMM** = General Matrix Multiply; **BMM** = Batch Matrix Multiply (QK^T, PV trong attention).

2. Ý tưởng chính

Ba quan sát (Mục 3 của paper): (1) **Trọng số dễ lượng tử hoá, activation khó**: phân phối trọng số phẳng, đều, còn activation có outlier lớn hơn phần lớn giá trị ~100×. (2) **Outlier cố định theo kênh:** kênh nào có outlier thì xuất hiện ở mọi token. Biên độ của một kênh thay đổi ít giữa các token, nhưng biên độ giữa các kênh chênh rất lớn. (3) Vì thế **lượng tử hoá activation theo kênh** (per-channel) sẽ rất tốt, nhưng *không tương thích kernel INT8 GEMM* (scale chỉ được áp ở các chiều ngoài T và C_o, không phải chiều trong C_i).

Do đó: không lượng tử hoá theo kênh, mà **chia activation kênh j cho s_j và nhân hàng j của trọng số với s_j**. Kết quả của phép nhân không đổi (tương đương toán học), nhưng activation đã "mịn" hơn, còn trọng số "gồ ghề" hơn một chút, nhưng vì trọng số vốn dễ nên chịu được. Phép co giãn làm offline, thường gộp được vào LayerNorm/linear phía trước nên **không tốn thêm chi phí lúc chạy**.

3. Phương pháp chi tiết

3.1 Công thức lượng tử hoá cơ sở

Lượng tử hoá đối xứng INT8 (đã giải thích ở bài 51, mục 1):

$$\bar{X}^{\text{INT8}} = \left\lceil \frac{X^{\text{FP16}}}{\Delta} \right\rfloor, \qquad \Delta = \frac{\max|X|}{2^{N-1}-1} \quad (N=8).$$

Linear layer $Y = XW$ với $X \in \mathbb{R}^{T\times C_i}$ (T token, C_i kênh vào), $W \in \mathbb{R}^{C_i \times C_o}$. Điều kiện để dùng GEMM INT8 hiệu quả: scale chỉ ở chiều ngoài:

$$Y = \text{diag}(\Delta_X) \cdot (\bar{X}^{\text{INT8}} \cdot \bar{W}^{\text{INT8}}) \cdot \text{diag}(\Delta_W) \quad (2)$$

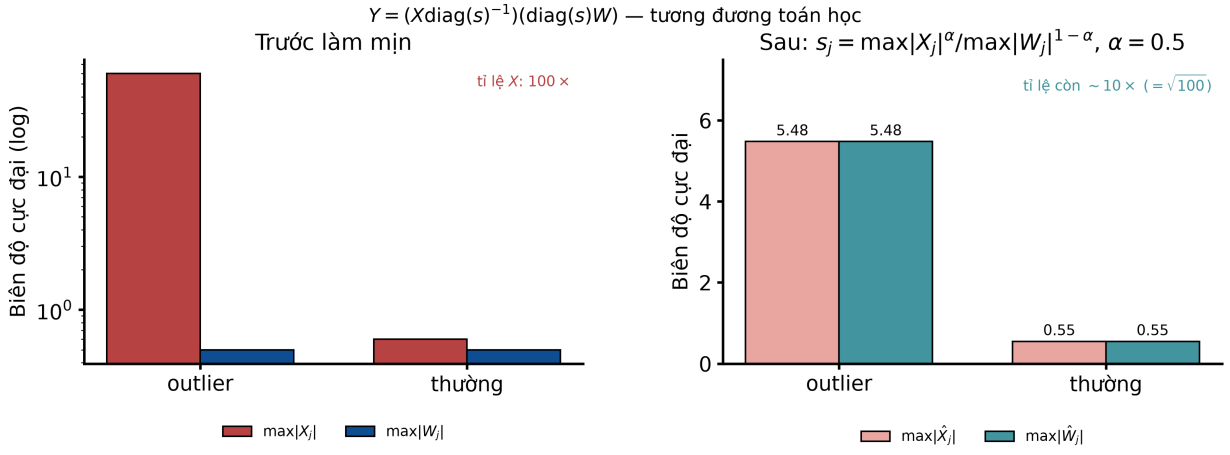
Bảng 1 (OPT, độ chính xác trung bình 4 tác vụ; mô phỏng): chỉ **per-channel activation** giữ được độ chính xác nhưng không chạy được trên INT8 GEMM:

OPT-	6.7B	13B	30B	66B	175B
FP16	64.9%	65.6%	67.9%	69.5%	71.6%
INT8 per-tensor	39.9%	33.0%	32.8%	33.1%	32.3%
INT8 per-token	42.5%	33.0%	33.1%	32.9%	31.7%
INT8 per-channel	64.8%	65.6%	68.0%	69.4%	71.4%

Phân tích "mức lượng tử hiệu dụng": nếu kênh i có max m_i và max toàn ma trận là m thì số mức hiệu dụng của kênh i là $2^8 \cdot m_i/m$; kênh không phải outlier chỉ còn **2–3 mức**. (Ví dụ tự tính: outlier 60, kênh thường $0.6 \Rightarrow 256 \times 0.01 = 2.6$ mức.)

3.2 Phép làm mịn (smoothing)

Chọn $s \in \mathbb{R}^{C_i}$ dương, và



SmoothQuant α=0.5: ví dụ outlier max $|X|$=60 → sau làm mịn biên độ X và W cùng ~5.48; tỉ lệ $100\times \to 10\times$. (minh hoạ, tự tính)

$$\boxed{Y = (X\,\text{diag}(s)^{-1}) \cdot (\text{diag}(s)\,W) = \hat{X}\hat{W}} \quad (3)$$

$\hat{X}$: chia kênh j của activation cho s_j. $\hat{W}$: nhân hàng j của trọng số với s_j (hàng j của W ứng với kênh vào j). Phép tính tương đương chính xác.

Chọn s. Nếu chọn $s_j = \max|X_j|$ thì mọi kênh activation có cùng max, nhưng đẩy hết độ khó sang trọng số (outlier chuyển thành trọng số lớn) và hỏng. Chọn $s_j = 1/\max|W_j|$ thì ngược lại. Giải pháp: cân bằng bằng **migration strength** α:

$$\boxed{s_j = \frac{\max(|X_j|)^\alpha}{\max(|W_j|)^{1-\alpha}}} \quad (4)$$

Sau làm mịn: $\max|\hat{X}_j| = \max|X_j|/s_j = \max|X_j|^{1-\alpha}\max|W_j|^{1-\alpha}$ và $\max|\hat{W}_j| = s_j\max|W_j| = \max|X_j|^\alpha\max|W_j|^\alpha$. Với $\alpha = 0.5$ cả hai bằng $\sqrt{\max|X_j|\max|W_j|}$, tức activation và trọng số ở cùng kênh có cùng biên độ cực đại và "chia sẻ cùng độ khó" (Hình 5).

Ví dụ số (tự tính): kênh outlier: $\max|X_j| = 60$, $\max|W_j| = 0.5$, $\alpha = 0.5 \Rightarrow s_j = \sqrt{60}/\sqrt{0.5} = 10.95$; $\hat{X}_j$ max $= 60/10.95 = 5.48$, $\hat{W}_j$ max $= 0.5 \times 10.95 = 5.48$. Kênh thường: $\max|X_j| = 0.6$, $\max|W_j| = 0.5 \Rightarrow s_j = 1.095$, $\hat{X}_j$ max $= 0.548$, $\hat{W}_j$ max $= 0.548$. Tỉ lệ biên độ giữa kênh outlier và kênh thường ở activation giảm từ $100\times$ xuống $10\times$ (căn bậc hai); ở trọng số tăng từ $1\times$ lên $10\times$. Độ khó được **chia đều** ($\sqrt{100}$ mỗi bên).

Giá trị α trong paper: 0.5 cho mọi OPT, BLOOM; 0.75 cho GLM-130B (activation khó hơn, ~30% outlier); cho LLaMA/Llama-2/Falcon/Mistral/Mixtral: 0.8 (LLaMA-1 Bảng 6), 0.85–0.9 (Llama-2), 0.6–0.7 (Falcon), 0.8 (Mistral, Mixtral). α tìm bằng grid search nhanh trên một tập con validation Pile.

Hiệu chuẩn. 512 câu ngẫu nhiên từ Pile (pre-training set) để tính $\max|X_j|$ (và scale tĩnh); dùng cho mọi tác vụ downstream.

3.3 Áp vào khối Transformer

Làm mịn cho **đầu vào của attention (QKV) và FFN**: gộp $1/s$ vào tham số của LayerNorm phía trước; nếu đầu vào đến từ phép cộng residual thì thêm một phép co giãn ở nhánh residual (theo Wei et al., 2022). Tất cả linear và BMM trong attention dùng INT8; các phép nhẹ element-wise (ReLU, Softmax, LayerNorm) giữ FP16 (Hình 6).

3.4 Ba mức hiệu quả (Bảng 2)

Cấu hình	Trọng số	Activation
W8A8 ngây thơ	per-tensor	per-tensor, dynamic
ZeroQuant	group-wise	per-token, dynamic
LLM.int8()	per-channel	per-token, dynamic + FP16 (outlier)
Outlier Suppression	per-tensor	per-tensor, static
SmoothQuant-O1	per-tensor	per-token, dynamic
SmoothQuant-O2	per-tensor	per-tensor, dynamic
SmoothQuant-O3	per-tensor	per-tensor, **static**

O1 → O3: càng thô càng nhanh (độ trễ thấp hơn), có thể mất chút chính xác.

3.5 Pseudo-code

```
# OFFLINE (một lần, ~512 câu calibration)
for each linear layer L (đầu vào X được tạo bởi lớp trước P):
    run calibration set -> max_X[j] = max over tokens |X[:, j]|
```

```
    max_W[j] = max over output ch |W[j, :]|
    s[j] = max_X[j]**alpha / max_W[j]**(1-alpha)
    L.W[j, :] *= s[j]                  # hàng j của W
    P.scale_output_channel(j, 1/s[j])  # gộp vào LayerNorm / linear trước (không tốn lúc
    ↪  chạy)
quantize W per-tensor INT8; (static) quantize scale của activation từ calibration

# ONLINE
X8 = quant(X_hat, delta_X)             # per-tensor (static: delta_X cố định)
Y  = INT8_GEMM(X8, W8) * (delta_X * delta_W)   # dequant ở epilogue
```

4. Thí nghiệm và kết quả chính

Mô hình: OPT (đến 175B), BLOOM-176B, GLM-130B, thêm LLaMA/Llama-2/Falcon/Mistral/Mixtral, MT-NLG 530B. **Tác vụ:** 7 tác vụ zero-shot (LAMBADA, HellaSwag, PIQA, WinoGrande, OpenBookQA, RTE, COPA) + WikiText cho OPT/BLOOM; MMLU, MNLI, QNLI, LAMBADA cho GLM-130B. Chỉ báo cáo thay đổi tương đối so với FP16.

Bảng 3, OPT-175B:

Phương pháp	LAMBADA	HellaSwag	PIQA	WinoGrande	OBQA	RTE	COPA	Trung bình	WikiText ppl
FP16	74.7%	59.3%	79.7%	72.6%	34.0%	59.9%	88.0%	66.9%	10.99
W8A8	0.0%	25.6%	53.4%	50.3%	14.0%	49.5%	56.0%	35.5%	93080
ZeroQuant	0.0%	26.0%	51.7%	49.3%	17.8%	50.9%	55.0%	35.8%	84648
LLM.int8()	74.7%	59.2%	79.7%	72.1%	34.2%	60.3%	87.0%	66.7%	11.10
Outlier Suppression	0.00%	25.8%	52.5%	48.6%	16.6%	53.4%	55.0%	36.0%	96151
SmoothQuant-O1	74.7%	59.2%	79.7%	71.2%	33.4%	58.1%	89.0%	66.5%	11.11
SmoothQuant-O2	75.0%	59.0%	79.2%	71.2%	33.0%	59.6%	88.0%	66.4%	11.14
SmoothQuant-O3	74.6%	58.9%	79.7%	71.2%	33.4%	59.9%	90.0%	66.8%	11.17

Bảng 4 (trung bình 4 tác vụ; GLM-130B: LAMBADA, MMLU, MNLI, QNLI, khác tác vụ nên không so cột): OPT-175B FP16 71.6%, W8A8 32.3%, SQ-O1/O2/O3 71.2/71.1/71.1%; BLOOM-176B FP16 68.2%, W8A8 64.2%, SQ-O1/O2/O3 68.3/68.4/67.4%; GLM-130B FP16 73.8%, W8A8 26.9%, SQ-O1/O2/O3 73.7/72.5/72.8%. LLM.int8() giữ 71.4/68.0/73.8%.

LLaMA/Llama-2/… (perplexity WikiText-2, W8A8 SmoothQuant so FP16):

Model	FP16	SQ W8A8	α
LLaMA 7B / 13B / 30B / 65B (seq 512)	11.51 / 10.05 / 7.53 / 6.17	11.56 / 10.08 / 7.56 / 6.20	0.8
Llama-2-7B	5.474	5.515	0.85
Llama-2-13B	4.950	4.929	0.85
Llama-2-70B	3.320	3.359	0.9
Falcon-7B	6.590	6.629	0.6
Falcon-40B	5.228	5.255	0.7
Mistral-7B	5.253	5.277	0.8

Model	FP16	SQ W8A8	α
Mixtral-8x7B	3.842	3.893	0.8

Instruction-tuned: OPT-IML-30B (LAMBADA/WikiText): FP16 69.12% / 14.26; SQ-O3 69.77% / 14.37; W8A8 4.21% / 576.53. MT-NLG 530B (Bảng 9): FP16 trung bình 73.1% vs INT8 73.1%.

Tác giả nhận xét LLaMA có ít vấn đề outlier hơn OPT/BLOOM nhưng SmoothQuant vẫn dùng tốt.

5. Phân tích / ablation

Migration strength α (Hình 10, OPT-175B, LAMBADA): $\alpha < 0.4 \Rightarrow$ activation vẫn khó; $\alpha > 0.6 \Rightarrow$ trọng số khó. Vùng tốt **0.4–0.6**. (Số liệu chi tiết trong hình không đọc được từ bản trích text.)

Mức lượng tử hoá (Bảng 11, PyTorch, batch 4, ms): càng thô càng nhanh; static nhanh hơn dynamic vì khỏi tính scale lúc chạy. Một số dòng đọc được: FP16 OPT-13B (seq 256/512) 152.6/296.3 ms, OPT-30B 343.0/659.9; LLM.int8() 237.1/371.5 và 387.9/654.9 (chậm hơn FP16); SQ-O1 124.5/243.3 và 246.7/490.7; SQ-O2 120.5/235.1 và 240.2/478.3; SQ-O3 112.1/223.1 và 227.6/458.4. SmoothQuant nhanh hơn FP16 ở mọi cấu hình, LLM.int8() thường chậm hơn.

O3 mất chính xác nhẹ: trên BLOOM-176B O3 thấp hơn FP16 0.8 điểm, tác giả cho là do khác biệt giữa thống kê tĩnh và thống kê thật; GLM-130B khó hơn (O3 giảm ~1 điểm). Khi calibrate scale tĩnh cho GLM-130B có cắt 2% token lớn nhất (theo Wei et al., 2022).

6. Phân tích bộ nhớ và độ trễ (batch 1)

6.1 Số đo trong paper

- **Context stage (prefill), PyTorch, batch 4, một A100-80GB:** SmoothQuant-O3 đạt tới **1.51×** nhanh hơn và **1.96×** tiết kiệm bộ nhớ (OPT-30B, seq 256); mô hình càng lớn càng được lợi. FasterTransformer: tới **1.56×** (OPT-13B, OPT-30B một GPU). OPT-66B chạy 1 GPU thay vì 2, OPT-175B 4 GPU thay vì 8 với độ trễ tương đương.
- **Decoding stage (Bảng 8, mỗi token):**

Cấu hình	Batch	SeqLen	FP16 (ms)	SQ (ms)	Speedup	Mem FP16 (GB)	Mem SQ (GB)	Tiết kiệm
OPT-30B, 1 GPU	1	512	422	314	1.35×	57	30	1.91×
OPT-30B, 1 GPU	1	1024	559	440	1.27×	58	31	1.87×
OPT-30B, 1 GPU	16	512	2488	1753	1.42×	69	44	1.59×
OPT-175B, 8 GPU	1	512	426	359	1.19×	44	23	1.87×
OPT-175B, 8 GPU	1	1024	571	475	1.20×	44	24	1.85×

(Dòng OPT-30B batch 16, seq 1024: FP16 hết bộ nhớ (OOM), SmoothQuant chạy 3947 ms với 61 GB. Dòng OPT-175B batch 16 cũng có trong Bảng 8 nhưng không trích ở đây.) Con số “ms” ở Bảng 8 là thời gian đo của paper (paper mô tả đây là độ trễ giải mã mỗi token; các con số tuyệt đối hàng trăm ms trông lớn nên hãy đối chiếu định nghĩa trong paper gốc trước khi trích dẫn).

- **MT-NLG 530B:** FP16 cần 16 GPU (1040 GB), INT8 chỉ 8 GPU (527 GB), độ trễ tương tự (seq 128: 232 ms vs 253 ms), cho phép phục vụ >500B trong một node 8×A100-80GB.

6.2 Quy luật lợi ích, hiểu bằng roofline

Prefill có nhiều token ⇒ GEMM lớn, **compute-bound**: INT8 có tốc độ tính tối đa ~2× FP16 trên Tensor Core, cộng nửa băng thông ⇒ lợi từ cả hai. Decode batch 1 là phép nhân ma trận–vector, **memory-bound**: lợi ích chỉ đến từ việc đọc ít byte hơn, tối đa ~2× với W8 (thực đo 1.19–1.42×). Phép lượng tử hoá activation (A8) *không giúp thêm gì ở* batch 1 vì activation chỉ vài KB; nó chỉ cần để dùng được INT8 GEMM khi tính toán là nút cổ chai.

7. Giới hạn

Tác giả nêu:

- Với mô hình khó (GLM-130B) cần α lớn hơn và O3 mất ~1%; kiến trúc/huấn luyện khác nhau có độ khó lượng tử khác nhau ("hy vọng khơi gợi nghiên cứu").
- Chỉ lượng tử hoá GEMM/BMM; các phép phi tuyến giữ FP16.
- Static quantization lệch giữa thống kê calibration và dữ liệu thật (BLOOM-176B O3 −0.8%).

Nhận xét:

- Hiệu quả dựa trên giả thuyết "outlier cố định theo kênh". Với kiến trúc có nhiều dạng outlier theo *token* (ví dụ "massive activations" ở token đầu) thì per-tensor static có thể kém; paper chưa phân tích.
- α và scale tĩnh calibrate trên Pile/văn bản; với OCR (phân phối token khác: chữ số, ký hiệu) cần calibrate bằng dữ liệu OCR.
- Gộp $1/s$ vào LayerNorm: với **RMSNorm có trọng số** vẫn gộp được, nhưng với các nhánh qua cổng/GatedDeltaNet, conv ngắn, hay khi đầu vào đến từ nhiều nguồn (residual cộng) thì phải xử lý riêng; paper chỉ mô tả LayerNorm/linear/residual.
- Kết quả speedup đo trên FasterTransformer + A100, mô hình ≥ 6.7B; mô hình ~1B chưa đo.
- Hầu hết thí nghiệm tập trung perplexity/zero-shot; chưa có sinh dài hoặc OCR.

8. Liên hệ với các bài khác

- **51 (LLM.int8()):** cùng chẩn đoán; thay "tách FP16" bằng phép biến đổi tương đương (không cần mixed precision ⇒ nhanh hơn). Bảng 3: cả hai khớp FP16 ở OPT-175B, nhưng latency SmoothQuant thấp hơn.
- **53 (GPTQ):** weight-only 3–4 bit dựa trên OBS; không lượng tử hoá activation, nên không có kernel INT8 thuần nhưng nén mạnh hơn nhiều. Hai bài bổ trợ: W4 cho decode, W8A8 cho prefill.
- **54 (AWQ):** cũng nhân kênh theo s nhưng cho weight-only, chọn s dựa trên biên độ activation để bảo vệ kênh trọng số quan trọng, tư tưởng gần với công thức (4).
- **10 (RMSNorm), 9 (Pre-LN):** nơi gộp $1/s$.
- **Pruning (24 Wanda):** Wanda dùng $|w| \cdot \|x_j\|$: chính là tích biên độ trọng số × biên độ activation theo kênh, cùng đại lượng cân bằng trong công thức (4) của SmoothQuant (nhận xét suy luận).

9. Tóm tắt công thức / khái niệm

Khái niệm	Công thức / ý
Lượng tử hoá	$\bar{X} = \lceil X/\Delta \rfloor$, $\Delta = \max\|X\|/(2^{N-1}-1)$
GEMM INT8 hợp lệ	scale chỉ ở chiều ngoài: $Y = \text{diag}(\Delta_X)(\bar{X}\bar{W})\text{diag}(\Delta_W)$
Làm mịn	$Y = (X\text{diag}(s)^{-1})(\text{diag}(s)W)$
Hệ số	$s_j = \max\|X_j\|^{\alpha} / \max\|W_j\|^{1-\alpha}$

Khái niệm	Công thức / ý
α	0.5 (OPT, BLOOM), 0.75 (GLM-130B), 0.8–0.9 (Llama)
O1/O2/O3	per-token dyn / per-tensor dyn / per-tensor static (activation)
Hiệu quả	≤ 1.56× nhanh, ≈ 2× bộ nhớ (OPT-175B: 8→4 GPU; MT-NLG 530B: 16→8 GPU)

10. Kết luận ngắn

SmoothQuant biến bài toán "activation có outlier theo kênh" thành bài toán dễ hơn bằng phép co giãn theo kênh tương đương, cho phép W8A8 per-tensor static thuần INT8 giữ độ chính xác từ 6.7B đến 530B và đem lại tới 1.56× tốc độ, ~2× bộ nhớ.

Link gốc: https://arxiv.org/abs/2211.10438

Bài 53

GPTQ: Accurate Post-Training Quantization for Generative Pre-trained Transformers (2023)

Tác giả: Elias Frantar, Saleh Ashkboos, Torsten Hoefler, Dan Alistarh (IST Austria; ETH Zurich; Neural-Magic)
Venue / arXiv: arXiv:2210.17323 (v2, 22/3/2023); ICLR 2023
Nguồn: https://arxiv.org/abs/2210.17323
Chủ đề: Lượng tử hoá **chỉ trọng số** (weight-only) 3–4 bit, một lần (one-shot), không retrain, dựa trên thông tin bậc hai xấp xỉ (OBS/OBQ) với ba cải tiến để mở rộng lên 175B tham số trong ~4 giờ GPU.
Vị trí trong lộ trình: Nên đọc sau 51–52 (outlier, W8A8) và **sau 22 (OBC/OBQ) và 23 (SparseGPT) trong thư mục `../pruning content/`**: GPTQ là "OBQ làm cho nhanh" và SparseGPT mượn ngược lại thuật toán này (Hessian chung, Cholesky, lazy batch) cho pruning. Tiếp theo: 54 (AWQ, đối thủ weight-only dựa trên activation).

> Đây là ghi chú học tập bằng tiếng Việt, không phải bản dịch. Dẫn xuất công thức OBS dựa vào ghi chú 22; ví dụ số 2 trọng số là phép tính minh họa.

1. Bối cảnh

GPT-3 175B cần 326 GB ở FP16 khi tính theo bội 1024, vượt dung lượng của một GPU đơn lẻ. Lượng tử hóa sau huấn luyện (PTQ) hấp dẫn vì không cần huấn luyện lại toàn bộ mô hình. Trước GPTQ, các lựa chọn có những giới hạn sau:

- Ở quy mô tỷ tham số chỉ có **round-to-nearest (RTN)**: làm tròn mỗi trọng số tới mức lượng tử gần nhất (cách LLM.int8() và ZeroQuant xử lý trọng số, bài 51). Tốt ở 8-bit, vỡ ở 3–4 bit.
- Các phương pháp chính xác hơn (AdaRound, BRECQ, AdaQuant, **OBQ**) chạy tốt cho mô hình ≤ ~100M tham số, nhưng không scale. Ví dụ OBQ lượng tử hoá ResNet-50 (25M) mất ~1 giờ; độ phức tạp $O(d_{row} \cdot d_{col}^3)$.

Thuật ngữ: **one-shot** = nén một lượt với lượng nhỏ dữ liệu hiệu chuẩn (ở đây 128 đoạn 2048 token từ C4); **lượng tử hoá theo lớp** (layer-wise) = nén từng lớp tuyến tính độc lập, tối thiểu sai lệch đầu ra của lớp đó.

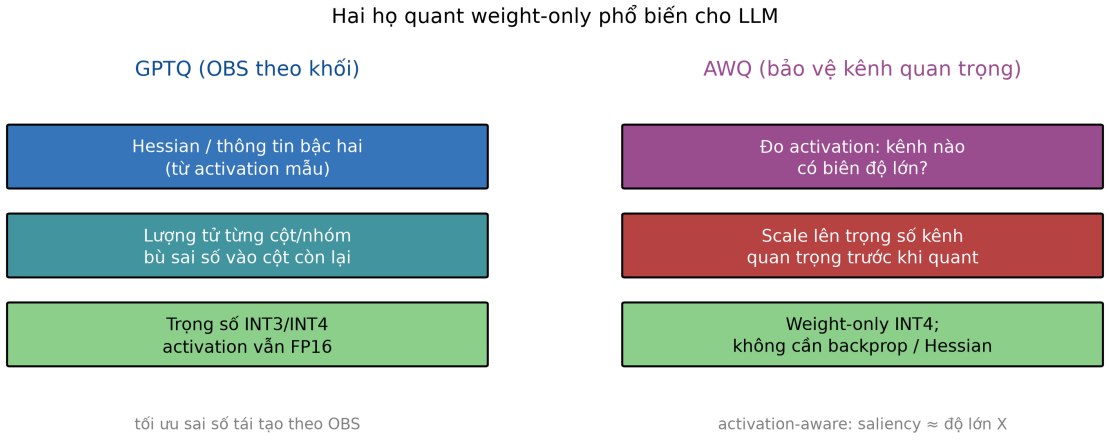
2. Ý tưởng chính

Mỗi lớp linear được xem là bài toán xấp xỉ: tìm trọng số lượng tử $\hat{W}$ sao cho $\|WX - \hat{W}X\|_2^2$ nhỏ nhất với X là đầu vào lớp từ dữ liệu hiệu chuẩn. OBQ giải bằng cách lượng tử hoá từng trọng số một và **cập nhật các trọng số chưa lượng tử hoá để bù sai số vừa gây ra** (tư tưởng của Optimal Brain Surgeon). GPTQ giữ cơ chế bù này nhưng bỏ ba chi phí lớn: (1) không cần chọn thứ tự tham lam, lượng tử hoá **mọi hàng theo cùng một thứ tự cột**, nên chỉ cần cập nhật Hessian nghịch đảo d_{col} lần thay vì $d_{row}d_{col}$ lần; (2) **lazy batch update** 128 cột một lần để dùng hết băng thông GPU; (3) **Cholesky** của H^{-1} để ổn định số học. Tổng: nhanh hơn OBQ hơn ba bậc độ lớn, đủ để lượng tử hoá OPT-175B/BLOOM-176B trong ~4 giờ trên một A100.

3. Phương pháp chi tiết

3.1 Bài toán theo lớp và OBS (nhắc lại từ 22)

Với một hàng $w \in \mathbb{R}^{d_{col}}$ của W (các hàng độc lập), sai số đầu ra khi trọng số đổi δ:



GPTQ (OBS/Hessian, bù sai số cột) vs AWQ (scale kênh theo activation) cho quant weight-only INT3/4. (minh hoạ, tự tính)

$$L(\delta) = \|\delta^\top X\|_2^2 = \tfrac{1}{2}\delta^\top H\delta, \qquad \boxed{H = 2XX^\top \in \mathbb{R}^{d_{col}\times d_{col}}}$$

H không phụ thuộc $w \Rightarrow$ **giống nhau cho mọi hàng** và chỉ phụ thuộc đầu vào X. Khi lượng tử hoá trọng số q (đặt $w_q \to \text{quant}(w_q)$) và bù các trọng số còn lại F, công thức OBS (phương trình 2 của paper):

$$\boxed{w_q^\star = \arg\min_{w_q} \frac{(\text{quant}(w_q) - w_q)^2}{[H_F^{-1}]_{qq}}, \qquad \delta_F = -\frac{w_q - \text{quant}(w_q)}{[H_F^{-1}]_{qq}} (H_F^{-1})_{:,q}}$$

Đây chính là công thức xoá trọng số của OBS (ghi chú 22, mục 3.3: $\delta_p = -\frac{w_p}{[H^{-1}]_{pp}} H^{-1}_{:,p}$) với **"xoá" được thay bằng "làm tròn"**: khi quant$(w) = 0$ thì quant$(w_q) - w_q = -w_q$ và hai công thức trùng nhau. Đây là lý do SparseGPT (23) và GPTQ chung một khung. Sau mỗi bước phải bỏ hàng/cột q khỏi H rồi cập nhật nghịch đảo một bước khử Gauss (phương trình 3):

$$H_{-q}^{-1} = \left(H^{-1} - \frac{1}{[H^{-1}]_{qq}} H^{-1}_{:,q} H^{-1}_{q,:}\right)_{-p}$$

3.2 Bước 1: thứ tự tuỳ ý (Arbitrary Order Insight)

OBQ chọn trọng số kế tiếp là cái ít làm tăng lỗi nhất (tham lam), và mỗi hàng có thứ tự riêng nên mỗi hàng cần H_F^{-1} riêng $\Rightarrow O(d_{row}\, d_{col}^3)$. Quan sát: trên các lớp lớn, thứ tự tham lam **chỉ cải thiện nhỏ** so với thứ tự tuỳ ý (có lẽ vì trọng số có lỗi lớn được lượng tử hoá cuối, lúc còn ít trọng số khác để bù, cân bằng với lợi ích tham lam). Nếu mọi hàng lượng tử hoá theo cùng thứ tự cột thì tập chưa lượng tử F và H_F^{-1} giống nhau mọi hàng $\Rightarrow$ chỉ cần cập nhật d_{col} lần: độ phức tạp còn $O(\max\{d_{row}d_{col}^2,\ d_{col}^3\})$, nhanh hơn $\min\{d_{row}, d_{col}\}$ lần.

3.3 Bước 2: Lazy batch updates

Việc cập nhật toàn bộ W và H^{-1} sau mỗi cột ít FLOP trên mỗi phần tử $\Rightarrow$ bị chặn bởi băng thông bộ nhớ (cùng bản chất với decode, một sự trùng hợp thú vị). Quan sát: quyết định làm tròn cột i chỉ bị ảnh hưởng bởi các cập nhật lên *chính cột đó*, nên có thể gom: xử lý khối $B = 128$ cột, giữ cập nhật trong khối $B \times B$, đến hết khối mới cập nhật toàn bộ phần còn lại bằng dạng đa trọng số của OBS:

$$\delta_F = -(w_Q - \text{quant}(w_Q))\,([H_F^{-1}]_{QQ})^{-1}(H_F^{-1})_{:,Q}, \qquad H_{-Q}^{-1} = H^{-1} - H^{-1}_{:,Q}([H^{-1}]_{QQ})^{-1}H^{-1}_{Q,:}$$

Không giảm số FLOP lý thuyết nhưng sửa vấn đề băng thông; "an order of magnitude speedup" với mô hình rất lớn.

3.4 Bước 3: Cholesky

Cập nhật lặp (5) tích luỹ sai số số học; H_F^{-1} có thể thành **không xác định dương**, khiến cập nhật đi sai hướng (xảy ra ở ít nhất vài lớp với mô hình > vài tỷ tham số). Cách chữa: (i) **dampening**: cộng $\lambda = 1\%$ trung bình đường chéo H vào đường chéo; (ii) chỉ cần *hàng q từ đường chéo trở đi* của $H_{F_q}^{-1}$, mà phép loại hàng/cột kiểu (3) tương đương với phân rã **Cholesky** của H^{-1} (chỉ khác việc chia hàng q cho $\sqrt{[H_{F_q}^{-1}]_{qq}}$). Do đó tính trước bằng Cholesky $H^{-1} = LL^\top$, lấy $U = L^\top$ (thừa số tam giác trên), và dùng U trong vòng lặp. Vừa ổn định vừa nhanh hơn.

3.5 Thuật toán 1 (Algorithm 1)

```
# W: [d_row, d_col], Hinv_chol = Cholesky(inverse(2*X X^T + lambda I))^T  (tam giác trên)
Q = zeros_like(W);  E = zeros(d_row, B)
for i in range(0, d_col, B):                          # từng khối B=128 cột
    for j in range(i, i+B):
        Q[:, j] = quant(W[:, j])                      # làm tròn cả cột (mọi hàng cùng
        ↪  lúc)
        E[:, j-i] = (W[:, j] - Q[:, j]) / Hinv_chol[j, j]
        W[:, j:i+B] -= E[:, j-i:j-i+1] * Hinv_chol[j, j:i+B]   # bù trong khối
    W[:, i+B:] -= E @ Hinv_chol[i:i+B, i+B:]          # cập nhật lazy phần còn lại
```

Chú ý: đây là đúng thuật toán mà SparseGPT dùng để cập nhật (xem ghi chú 23, công thức $E_{:,j-i}$), chỉ khác ở bước chọn "quant" (làm tròn vs đặt 0 theo mặt nạ).

Lưu ý triển khai: nạp **một khối Transformer (6 lớp linear) vào GPU một lần**, tích luỹ Hessian, lượng tử hoá, rồi chạy lại khối đã lượng tử để tạo đầu vào cho khối kế (nghĩa là X lấy từ mô hình đã lượng tử hoá một phần, không phải bản FP đầy đủ; cải thiện đáng kể với chi phí gần như không). Lưới lượng tử cố định trước: bất đối xứng, min-max, theo từng hàng (per-row), giống LLM.int8() (đây cũng là lưới của baseline RTN).

3.6 Ví dụ số hai trọng số (tự tính)

Xét $H = \begin{pmatrix} 2 & 1 \\ 1 & 2 \end{pmatrix}$ (giống ví dụ ở ghi chú 22), $H^{-1} = \frac{1}{3}\begin{pmatrix} 2 & -1 \\ -1 & 2 \end{pmatrix}$ (nghĩa là $XX^\top = H/2 = \begin{pmatrix} 1 & 0.5 \\ 0.5 & 1 \end{pmatrix}$: hai đầu vào tương quan dương). Lưới lượng tử: các bội của 0.5. Hàng $w = (0.3,\ 0.8)$.

- **RTN:** $0.3 \to 0.5$, $0.8 \to 1.0$. $d = w - \hat{w} = (-0.2, -0.2)$. Lỗi $= d^\top(XX^\top)d = 0.04 + 0.04 + 2(-0.2)(-0.2)(0.5) =$ **0.12**.
- **GPTQ:** cột 1: $\text{quant}(0.3) = 0.5$, sai số $w - q = -0.2$, $e = -0.2/[H^{-1}]_{11} = -0.2/(2/3) = -0.3$. Cập nhật $w_2 \leftarrow w_2 - e \cdot [H^{-1}]_{12} = 0.8 - (-0.3)(-1/3) = 0.7$ (kiểm chứng bằng công thức δ_F: $-(-0.2)/(2/3)\times(-1/3) = -0.1$). Cột 2: $\text{quant}(0.7) = 0.5$. Kết quả $\hat{w} = (0.5, 0.5)$, $d = (-0.2,\ 0.3)$. Lỗi $= 0.04 + 0.09 + 2(-0.2)(0.3)(0.5) =$ **0.07**.

Cột 1 được làm tròn lên (+0.2); do hai đầu vào tương quan dương, việc giảm cột 2 giúp bù một phần sai lệch ở đầu ra. Lỗi giảm từ 0.12 xuống 0.07 dù cả hai đều chỉ dùng lưới 0.5.

3.7 Nhóm (grouping) và các kích cỡ nhóm

Vì lưới lượng tử cố định trước nên GPTQ kết hợp tự nhiên với **group-wise quantization** (mỗi nhóm g trọng số liên tiếp trong hàng có scale/zero riêng; các tham số nhóm được xác định *trong khi* lượng tử hoá, dùng trọng số đã được cập nhật mới nhất). Chi phí: scale FP16 + zero-point mỗi nhóm; paper nói nhóm 1024 ≈ 0.02 bit/trọng số, nhóm 128 ≈ 0.15 bit thêm (cho 3-bit: $(16+3)/128 \approx 0.15$, tự kiểm).

4. Thí nghiệm và kết quả chính

Thiết lập. PyTorch, họ OPT và BLOOM; một A100 80GB; hiệu chuẩn bằng **128 đoạn 2048 token ngẫu nhiên từ C4** (không dùng dữ liệu đặc thù tác vụ ⇒ zero-shot). Lưới per-row bất đối xứng min-max; baseline RTN dùng cùng lưới. Metric: perplexity WikiText2 (cũng PTB, C4 trong phụ lục) và LAMBADA, ARC, PIQA.

Mô hình nhỏ so với phương pháp chính xác (Bảng 1, ResNet18/50 trên ImageNet; FP: 69.76% / 76.13%):

Phương pháp	RN18 4-bit	RN18 3-bit	RN50 4-bit	RN50 3-bit
AdaRound	69.34	68.37	75.84	75.14
AdaQuant	68.12	59.21	74.68	64.98
BRECQ	69.37	68.47	75.88	75.32
OBQ	69.56	68.69	75.72	75.24
GPTQ	69.37	67.88	75.71	74.87

GPTQ ngang các phương pháp chính xác ở 4-bit, hơi kém ở 3-bit, nhưng chạy < 1 phút thay vì ~1 giờ. **So với OBQ (Phụ lục A.1, Bảng 8):** BERT-base/SQuAD (FP 88.53 F1): OBQ 88.23 / 85.29, GPTQ 88.18 / 86.02 (4/3-bit). OPT-125M/WikiText2 (FP 27.66): OBQ 32.52 / 69.32, GPTQ 31.12 / 53.85 (4/3-bit): GPTQ không kém, thậm chí tốt hơn ở 3-bit.

Thời gian chạy (Bảng 2, một A100): OPT 13B 20.9 phút, 30B 44.9 phút, 66B 1.6 giờ, **175B 4.2 giờ**. BLOOM 1.7B 2.9 phút, 3B 5.2 phút, 7.1B 10.0 phút, **176B 3.8 giờ**. (ZeroQuant-LKD báo cáo 3 giờ cho 1.3B, tức ngoại suy ra hàng trăm giờ cho 175B.)

OPT, WikiText2 perplexity (Bảng 3):

Bits	125M	350M	1.3B	2.7B	6.7B	13B	30B	66B	175B
FP16	27.65	22.00	14.63	12.47	10.86	10.13	9.56	9.34	8.34
RTN 4	37.28	25.94	48.17	16.92	12.10	11.32	10.98	110	10.54
GPTQ 4	31.12	24.24	15.47	12.87	11.39	10.31	9.63	9.55	8.37
RTN 3	1.3e3	64.57	1.3e4	1.6e4	5.8e3	3.4e3	1.6e3	6.1e3	7.3e3
GPTQ 3	53.85	33.79	20.97	16.88	14.86	11.61	10.27	14.16	8.68

BLOOM (Bảng 4): FP16 560M 22.42, 1.1B 17.69, 1.7B 15.39, 3B 13.48, 7.1B 11.37, 176B 8.11; GPTQ-4: 24.03, 19.05, 16.48, 14.20, 11.73, 8.21; GPTQ-3: 32.31, 25.08, 21.11, 17.40, 13.47, 8.64; RTN-3 tới 571 ở 176B.

OPT-175B / BLOOM-176B (Bảng 5): ở 4-bit GPTQ chỉ hơn FP16 ≤ 0.25 ppl (OPT-175B Wiki2: 8.34 → 8.37, RTN 10.54); ở 3-bit RTN sập (7.3e3), GPTQ 8.68 (mất 0.3–0.6 điểm với nén > 5×). Có nhóm: OPT-175B 3-bit nhóm 1024: 8.45; nhóm 128: 8.45 (Wiki2); BLOOM-176B nhóm 128: 8.26.

2-bit và ternary (Bảng 7, Wiki2, FP16 OPT-175B 8.34 / BLOOM 8.11):

Nhóm	g128	g64	g32	3-bit (so sánh)
OPT-175B	9.58	9.18	8.94	8.68
BLOOM-176B	9.55	9.17	8.83	8.64

Nhóm 128 ≈ 2.2 bit (tăng < 1.5 ppl), nhóm 32 ≈ 2.6 bit (tăng 0.6–0.7). Nhóm 8 cho **ternary** $(-1, 0, +1)$: OPT-175B Wiki2 ppl 9.20.

Tốc độ (Bảng 6, batch 1, sinh 128 token, độ trễ trung bình mỗi token):

GPU	FP16	3-bit GPTQ	Speedup	Số GPU
A6000 48GB	589 ms	130 ms	**4.53×**	8 → 2
A100 80GB	230 ms	71 ms	**3.24×**	5 → 1

5. Phân tích / ablation

- **Mô hình càng lớn càng dễ lượng tử hoá** (Hình 1, Bảng 3): mất chính xác của GPTQ-4bit giảm từ +3.47 ppl (OPT-125M: 27.65 → 31.12) xuống +0.03 (OPT-175B). Ngoại lệ OPT-66B (RTN 110): tác giả gán cho "một tỉ lệ đáng kể đơn vị chết (dead units) ở các lớp đầu".
- **RTN vs GPTQ:** ở 4-bit RTN vẫn tạm với mô hình lớn, nhưng ở 3-bit RTN sụp hoàn toàn; GPTQ vẫn giữ được.
- **Thứ tự tuỳ ý:** hiệu ứng nhỏ trên lớp lớn (cơ sở của Bước 1). **Thứ tự tham lam** của OBQ vẫn đúng hơn chút nhưng bị cân bằng.
- **Nhóm (Hình 4, Bảng 5):** nhóm giảm mất mát cho mô hình vừa ở 4-bit và là chìa khoá cho 2-bit.
- **Ablation hệ thống thật:** nói rõ "almost all of the speedup is due to our kernels", không do truyền thông (Phụ lục A.2.2: truyền thông < 5% tổng thời gian ngay cả 8 GPU).

6. Phân tích bộ nhớ và độ trễ (batch 1)

6.1 Bộ nhớ

OPT-175B 3-bit chiếm ≈ **63 GB** (gồm embedding và output layer giữ FP16) + KV cache FP16 ≈ 9 GB cho 2048 token ⇒ vừa một A100 80GB (4-bit thì không vừa). FP16 cần 5×80GB; LLM.int8() cần 3×80GB.

6.2 Vì sao nhanh ở batch 1 mà không giảm FLOP

Decode là phép nhân **ma trận–vector** (lượng FP16 vector × ma trận lượng tử), bị chặn bởi băng thông. Kernel của paper: **dequantize trọng số "on the fly"** trong thanh ghi rồi nhân với activation FP16, không cần lượng tử activation. Tốn thêm FLOP dequant nhưng đọc ít byte hơn nhiều ⇒ nhanh. Mô hình đơn giản:

$$T_{\text{token}} \approx \frac{\text{bytes trọng số}}{B}, \qquad \text{bytes} \approx N_{\text{params}} \cdot \frac{b + \text{overhead nhóm}}{8}$$

Với $b = 3$: tối đa 16/3 ≈ 5.3× ít byte hơn FP16; thực đo 3.24–4.53× (phần còn lại là embedding/LM head FP16, KV, dequant, kernel khác). Với batch lớn hoặc prefill (compute-bound), paper nói kernel không áp dụng trực tiếp; giải pháp là **giải nén cả ma trận rồi nhân ma trận–ma trận** (< 1.5 ms trên A100 so với 76 ms cho phép nhân FC2 của OPT-175B với 16×1024 token), nên vẫn tiết kiệm GPU gần như miễn phí. Tác giả cũng nhận: *"không giảm tính toán"*, chỉ giảm di chuyển bộ nhớ.

Trong Bảng 3, mức tăng perplexity của GPTQ-4 so với FP16 là 3.47 điểm ở OPT-125M (+12.6%), 2.24 ở 350M (+10%), 0.84 ở 1.3B (+5.7%) và 0.53 ở 6.7B (+4.9%). Với OPT-125M, GPTQ-3 cho perplexity 53.85, gần gấp đôi mức FP16. Đây là các kết quả trên họ OPT, không phải dự báo cho mọi kiến trúc.

7. Giới hạn

Tác giả nêu:

- Speedup đến từ **giảm di chuyển bộ nhớ**, không giảm tính toán; thiếu hỗ trợ phần cứng cho phép nhân hỗn hợp (FP16 × INT4) nên phải dequant.
- **Không lượng tử hoá activation** (không phải nút cổ chai trong kịch bản mục tiêu); có thể kết hợp kỹ thuật khác (ZeroQuant, bài 52).
- Tập trung vào tác vụ sinh và perplexity; chưa nghiên cứu tác động lên các chỉ số phụ (độ lệch, bias) (mục Ethics).
- Perplexity "rất nghiêm" nhưng chưa đủ; zero-shot chỉ vài tác vụ.

Nhận xét:

- Chỉ OPT và BLOOM (kiến trúc cũ: GELU/ReLU, LayerNorm); chưa Llama/Qwen/SwiGLU/GQA hay lớp hồi quy (GatedDeltaNet). Tổn thất trên các kiến trúc đó đo ở nghiên cứu khác, không có ở đây.
- Calibration 128 đoạn C4 là văn bản web; với OCR (ký hiệu, chữ số, bảng) cần dữ liệu hiệu chuẩn đúng miền; Hessian $2XX^\top$ chỉ "nhìn thấy" hướng nào có trong X.
- Mục tiêu chỉ là sai số đầu ra *từng lớp*, không quan tâm lỗi tích luỹ qua các lớp/độ dài chuỗi (paper có làm hiệu chỉnh nhẹ bằng cách đưa đầu vào đã lượng tử, nhưng không tối ưu end-to-end).
- Bất ổn số (Hessian không xác định dương) cần dampening 1% và Cholesky; với mô hình nhỏ/ít dữ liệu hiệu chuẩn H có thể kém điều kiện hơn.
- Đối với mô hình đã pruning theo cấu trúc, phân bố trọng số/kích hoạt đổi; GPTQ cần chạy lại sau cùng.

8. Liên hệ với các bài khác

- **22 (OBC):** GPTQ = OBQ + (thứ tự tuỳ ý, lazy batch, Cholesky); công thức OBS ở mục 3.1 là cùng một công thức với cập nhật bù của 22. Trong 22, ExactOBS cần Hessian riêng mỗi hàng vì mask khác nhau; GPTQ tránh được vì thứ tự cột chung.
- **23 (SparseGPT):** cùng tác giả chính. SparseGPT = GPTQ với "quant" thay bằng "đặt 0 theo mặt nạ" chọn theo khối 128 cột. Cả hai chia sẻ Cholesky + lazy batch. Do đó pruning và quantization có thể xếp chồng: SparseGPT + GPTQ (nén kết hợp) là hướng tự nhiên.
- **01 (OBD/OBS):** gốc của công thức.
- **51 (LLM.int8()):** baseline RTN; GPTQ nén mạnh hơn 2.5× ở độ chính xác tương đương, nhưng không lượng tử hoá activation (không có INT8 GEMM).
- **52 (SmoothQuant):** W8A8; bổ trợ: W8A8 cho prefill, GPTQ cho decode.
- **54 (AWQ):** cũng weight-only nhưng chọn co giãn kênh dựa trên biên độ activation, không dùng cập nhật bù bằng Hessian; so sánh hai hướng nên làm sau khi đọc 54.
- **49/50 (Medusa, EAGLE):** Medusa nói train đầu trên backbone lượng tử hoá kiểu QLoRA; EAGLE + gpt-fast int4 đạt 160.4 tok/s; speculative cộng gộp với weight-only quant.

9. Tóm tắt công thức / khái niệm

Khái niệm	Công thức / ý
Mục tiêu theo lớp	$\arg\min_{\hat{W}} \|WX - \hat{W}X\|_2^2$
Hessian	$H = 2XX^\top$ (chung mọi hàng)
OBS/OBQ	$\delta_F = -\frac{w_q - \text{quant}(w_q)}{[H_F^{-1}]_{qq}} (H_F^{-1})_{:,q}$
Cập nhật nghịch đảo	$H_{-q}^{-1} = H^{-1} - \frac{1}{[H^{-1}]_{qq}} H_{:,q}^{-1} H_{q,:}^{-1}$
Độ phức tạp	OBQ $O(d_{row} d_{col}^3)$ → GPTQ $O(\max\{d_{row} d_{col}^2, d_{col}^3\})$
Lazy batch	$B = 128$ cột/khối
Dampening	$\lambda = 1\% \cdot \text{mean}(\text{diag}\, H)$
Bit hiệu dụng khi nhóm	$b + (16 + b_z)/g$
Speedup (batch 1)	3.24× (A100), 4.53× (A6000), OPT-175B 3-bit

10. Kết luận ngắn

GPTQ biến phương pháp lượng tử hoá bậc hai chính xác nhưng chậm (OBQ) thành thuật toán scale được tới 175B tham số bằng ba thủ thuật kỹ thuật, và cho ra mô hình 3–4 bit gần như không mất perplexity ở quy mô lớn, nhanh hơn 3.2–4.5× ở batch 1 nhờ giảm di chuyển bộ nhớ.

Link gốc: https://arxiv.org/abs/2210.17323

Bài 54

AWQ: Activation-aware Weight Quantization for On-Device LLM Compression and Acceleration (2024)

Tác giả: Ji Lin, *Jiaming Tang*, Haotian Tang, Shang Yang, Wei-Ming Chen, Wei-Chen Wang, Guangxuan Xiao, Xingyu Dang, Chuang Gan, Song Han (MIT, NVIDIA, ...).
Venue / arXiv: arXiv:2306.00978 (bản v6 dùng để đọc); MLSys 2024, Best Paper Award (theo trang đầu của paper).
Nguồn: https://arxiv.org/abs/2306.00978
Chủ đề: Lượng tử hóa chỉ-trọng-số (weight-only) 4-bit/3-bit cho LLM, bảo vệ "salient channels" bằng scale theo activation, và hệ thống TinyChat để biến việc giảm bộ nhớ thành tăng tốc thật ở batch 1.
Vị trí trong lộ trình: Đọc trước: 51 (LLM.int8), 52 (SmoothQuant), 53 (GPTQ). Bài này là đối thủ trực tiếp của GPTQ: không cần backprop/tái tạo, ít overfit calibration, kèm kernel thực tế. Tiếp theo: 55-56 (nén KV cache, một nguồn bộ nhớ khác), 57 (FastV).

> Đây là ghi chú học tập bằng tiếng Việt, không phải bản dịch.

1. Bối cảnh

Bài báo lấy GPT-3 175B làm ví dụ về chi phí bộ nhớ: trọng số FP16 cần 350 GB, vượt dung lượng 192 GB của một GPU B200. Hạn chế này càng rõ trên thiết bị biên như laptop, Jetson hay điện thoại, nơi bộ nhớ dành cho mô hình nhỏ hơn nhiều.

Quantization (lượng tử hóa) là lưu số thực (FP16, 16 bit) bằng số nguyên ít bit (INT4, INT3). Ví dụ INT4 chỉ có 16 mức giá trị, nên phải kèm một hệ số tỉ lệ (scale) để ánh xạ ngược về số thực.

Có hai họ:

- **QAT** (quantization-aware training): huấn luyện lại với lượng tử hóa. Quá đắt với LLM.
- **PTQ** (post-training quantization): lượng tử hóa sau khi đã huấn luyện, không (hoặc ít) huấn luyện lại. Đây là hướng thực tế.

Thêm một phân biệt quan trọng:

- **W8A8**: cả trọng số (Weight) và activation đều INT8 (LLM.int8, SmoothQuant). Phép nhân dùng INT8 tensor core.
- **W4A16** (weight-only): chỉ trọng số xuống 4 bit, activation giữ FP16. Trước khi nhân phải **dequantize** (giải lượng tử) trọng số về FP16. AWQ chọn hướng này.

Round-to-nearest (RTN): cách đơn giản nhất, cứ làm tròn từng trọng số tới mức gần nhất. Với **group size** nhỏ (ví dụ 128 trọng số dùng chung một scale) RTN đã khá mạnh, nhưng ở 3 bit thì hỏng nặng.

GPTQ (bài 53) dùng thông tin bậc hai (Hessian) để bù sai số, nhưng paper này lập luận nó có thể overfit calibration set và làm méo đặc trưng ở miền khác (Hình 8).

Vì sao weight-only đáng làm ở batch 1? Giai đoạn sinh token (generation/decode) bị giới hạn bởi băng thông bộ nhớ (memory-bound), không phải bởi FLOPs. Mỗi token mới phải đọc toàn bộ trọng số từ DRAM. Giảm số byte trọng số = giảm thời gian. Chi tiết ở mục 4.

2. Ý tưởng chính

1. Không phải mọi trọng số quan trọng như nhau. Chỉ khoảng 0,1%-1% "salient weights"; nếu giữ chúng ở FP16 thì sai số lượng tử hóa giảm mạnh.
2. Cái hay: **muốn biết trọng số nào quan trọng, hãy nhìn activation, không nhìn bản thân trọng số.** Kênh đầu vào có activation lớn thì trọng số tương ứng quan trọng. Chọn theo độ lớn trọng số (chuẩn pruning cổ điển) không tốt hơn chọn ngẫu nhiên.
3. Giữ 1% ở FP16 trộn với phần còn lại INT3 là "mixed-precision", rất khó chạy hiệu quả trên phần cứng. Thay vào đó AWQ **nhân kênh salient với s > 1 trước khi lượng tử hóa** và chia activation tương ứng cho s. Về toán học đây là phép biến đổi tương đương (không đổi kết quả nếu chưa làm tròn) nhưng sai số tương đối của kênh đó giảm khoảng s lần.
4. Hệ số s được tìm bằng một grid search cho đúng 1 siêu tham số α, chỉ dùng thống kê độ lớn activation trung bình. Không backprop, không tái tạo, nên ít overfit.
5. Kèm TinyChat: kernel dequantize-on-the-fly, đóng gói trọng số theo SIMD, hợp nhất kernel. Đạt >3x so với FP16 của HuggingFace.

3. Phương pháp chi tiết

3.1 Lượng tử hóa nhóm

Với một nhóm trọng số **w** (ví dụ 128 phần tử), N bit:

$$Q(w) = \Delta \cdot \text{Round}\Big(\frac{w}{\Delta}\Big), \qquad \Delta = \frac{\max(|\mathbf{w}|)}{2^{N-1}}$$

(Công thức (1) trong paper; phần mũ trong bản text bị mất định dạng nên ta đọc là 2^{N-1}.) Δ là bước lượng tử (scale), được xác định bởi giá trị tuyệt đối lớn nhất của nhóm.

3.2 Vì sao nhân s làm giảm sai số

Xét $y = wx$. Nhân w với $s > 1$ và chia x cho s:

$$Q(w \cdot s) \cdot \frac{x}{s} = \Delta' \cdot \text{Round}\Big(\frac{ws}{\Delta'}\Big) \cdot x \cdot \frac{1}{s}$$

Ba quan sát thực nghiệm của paper:

1. Sai số của hàm Round, $\text{RoundErr}(\cdot)$, không đổi: phân bố gần đều trong $[0, 0.5]$, trung bình 0,25.
2. Nhân một phần tử w thường không làm đổi giá trị lớn nhất của nhóm, nên $\Delta' \approx \Delta$.
3. Δ và x lưu FP16 nên không có sai số lượng tử.

Do đó:

$$\text{Err}(Q(w)x) = \Delta \cdot \text{RoundErr}\big(\tfrac{w}{\Delta}\big) \cdot x, \qquad \text{Err}\big(Q(ws)\tfrac{x}{s}\big) = \Delta' \cdot \text{RoundErr}\big(\tfrac{ws}{\Delta'}\big) \cdot x \cdot \frac{1}{s}$$

$$\boxed{\frac{\text{sai số mới}}{\text{sai số cũ}} \approx \frac{\Delta'}{\Delta} \cdot \frac{1}{s}}$$

Nếu $\Delta' \approx \Delta$ và $s > 1$ thì sai số tương đối của kênh salient nhỏ đi. Bảng 2 (OPT-6.7B, nhân 1% kênh salient):

s	1	1.25	1.5	2	4
tỉ lệ nhóm có $\Delta' \neq \Delta$	0%	2.8%	4.4%	8.2%	21.2%

s	1	1.25	1.5	2	4
trung bình Δ'/Δ	1	1.005	1.013	1.038	1.213
trung bình $\frac{\Delta'}{\Delta} \cdot \frac{1}{s}$	1	0.804	0.676	0.519	0.303
Wiki-2 PPL	23.54	12.87	12.48	11.92	12.36

Tốt nhất ở s = 2; s lớn hơn thì các kênh không salient bị thiệt (với s = 4, 21.2% nhóm bị đổi Δ và sai số kênh thường bị khuếch đại Δ'/Δ lần). Phải cân bằng hai phía.

Ví dụ nhỏ (tự dựng, minh họa trực giác, 3 bit nên $\Delta = \max/4$): nhóm $w = [1.0,\ 0.37,\ -0.5,\ 0.2]$, kênh thứ hai salient. $\Delta = 0.25$.

- RTN: $0.37/0.25 = 1.48 \to 1 \to 0.25$. Sai số trọng số 0.12, nhân với activation lớn thì thành sai số lớn.
- Nhân s = 2: $0.74/0.25 = 2.96 \to 3 \to 0.75$, chia 2 = 0.375. Sai số 0.005. Đây là ví dụ may mắn; kỳ vọng của paper là sai số làm tròn giữ nguyên (trung bình 0.25 bước) còn bước sau khi chia s thì nhỏ đi s lần.

3.3 Tìm s tối ưu

Mục tiêu (4): giữ đầu ra của lớp gần đầu ra gốc.

$$\mathbf{s}^* = \arg\min_{\mathbf{s}} \mathcal{L}(\mathbf{s}), \qquad \mathcal{L}(\mathbf{s}) = \left\| Q(W \cdot \mathrm{diag}(\mathbf{s}))\,(\mathrm{diag}(\mathbf{s})^{-1} X) - WX \right\|$$

W là trọng số FP16, X là đặc trưng đầu vào lưu từ calibration set nhỏ. Vì Q không khả vi, paper không dùng gradient xấp xỉ (STE, thấy hội tụ không ổn định) mà giới hạn không gian tìm kiếm:

$$\boxed{\mathbf{s} = \mathbf{s}_X^{\alpha}, \qquad \alpha^* = \arg\min_{\alpha \in [0,1]} \mathcal{L}(\mathbf{s}_X^{\alpha})}$$

$\mathbf{s}_X$ là độ lớn activation trung bình theo từng kênh đầu vào. $\alpha = 0$ nghĩa là không scale, $\alpha = 1$ là mạnh nhất trong không gian tìm. Grid search 20 điểm. Sau đó áp dụng **weight clipping** để tối thiểu MSE lượng tử hóa. Chia s^{-1} cho activation thường **hợp nhất (fuse) được vào toán tử phía trước** nên không tốn thêm tính toán lúc chạy.

3.4 Pseudo-code

```
AWQ_quantize(model, calib_set, N_bits, group=128):
    for each decoder block b:                       # từng khối, từ nông tới sâu
        X = cached input activations of b's linear layers  (từ calib_set, ~16 chuỗi x
        ↪  2048 token)
        for each linear layer W in b:
            sX = mean(|X|, over tokens)             # [in_channels]
            best = inf
            for alpha in linspace(0, 1, 20):
                s = sX ** alpha
                err = || Q(W * s) @ (X / s) - W @ X ||
                if err < best: best, s* = err, s
            W <- W * s*;  fuse (1/s*) into the previous op (norm / previous linear)
            clip W per group to minimise MSE;  W_q <- Q(W)
    return packed INT4 weights + FP16 scales (+ zeros)
```

3.5 TinyChat (phần hệ thống)

Theo paper (mục 4):

- **Vì sao nhanh:** giai đoạn sinh có cường độ số học (FLOPs/byte) khoảng 1 ở FP16; W4A16 nâng lên khoảng 4. Trần lý thuyết tăng ~4x.
- **Dequantize on-the-fly:** phần cứng không có phép nhân INT4 x FP16, nên dequantize ngay trong kernel nhân ma trận, tránh ghi trọng số đã giải nén ra DRAM.
- **SIMD-aware weight packing:** sắp xếp lại thứ tự bit offline để giải nén bằng AND và dịch bit trên thanh ghi SIMD (ARM NEON 128-bit: 3 lệnh SIMD cho 32 trọng số, nhanh hơn tới 1,2x). Trên GPU, đóng gói theo thứ tự $w_{0,2,4,6,1,3,5,7}$ cho mỗi 8 trọng số.
- **Kernel fusion:** hợp nhất LayerNorm thành một kernel, hợp nhất QKV, tính positional embedding ngay trong kernel, cấp phát trước KV cache và cập nhật trong kernel attention. Lý do: thời gian của mỗi kernel FP16 cỡ 0,01 ms trên 4090, ngang với chi phí khởi chạy kernel, nên giảm số kernel là tăng tốc trực tiếp.

4. Phân tích chi phí: bộ nhớ và độ trễ ở batch 1

4.1 Công thức (chung)

Cho mô hình có P tham số, bandwidth bộ nhớ B (byte/s). Sinh một token ở batch 1:

$$\text{FLOPs} \approx 2P, \qquad \text{Bytes} \approx P \cdot b_w + \text{KV}(n) + \text{activation (nhỏ)}, \qquad t_{\text{token}} \gtrsim \frac{\text{Bytes}}{B}$$

với b_w là số byte/trọng số (2 ở FP16). Cường độ số học: $\frac{2P}{Pb_w} = \frac{2}{b_w}$, tức 1 FLOP/byte ở FP16 và 4 ở 4-bit (paper nêu "≈1" và "≈4"). Roofline của 4090 trong paper: 165 TFLOPS và 1 TB/s, nên mọi tải có cường độ < 165 là memory-bound. Batch 1 nằm cực xa ngưỡng đó, nên **số byte, không phải FLOPs, quyết định thời gian**.

Dung lượng thực của W4 nhóm 128: mỗi trọng số 4 bit cộng chi phí scale (và zero-point) cho mỗi nhóm. Với scale FP16 và nhóm 128 thì thêm $16/128 = 0.125$ bit; thêm zero-point FP16 thì 0.25 bit. Vậy khoảng 4.125-4.25 bit/trọng số (tự tính; paper không nêu định dạng chính xác của scale/zero).

Ở Llama-2-7B trên RTX 4090, Hình 3a báo cáo 10 ms để xử lý prompt tóm tắt 200 token và 310 ms để sinh 20 token. Số đo này minh họa khác biệt chi phí giữa prefill và decode trong thiết lập cụ thể của bài báo; lợi ích của AWQ còn phụ thuộc kernel và phần cứng.

5. Thí nghiệm và kết quả chính

Thiết lập (mục 5.1): nhóm 128 mặc định, INT4/INT3, calibration nhỏ từ Pile, grid 20 điểm. Mô hình: LLaMA, Llama-2, OPT, Vicuna, Mistral, Mixtral, OpenFlamingo-9B, LLaVA-13B, VILA, CodeLlama. Baseline: RTN, GPTQ, GPTQ-R (có reorder). Chỉ số chính: perplexity WikiText-2.

Bảng 1 (OPT, INT3 g128; giữ x% kênh ở FP16, Wiki PPL): FP16 / RTN: OPT-1.3B 14.62 / 119.00; OPT-6.7B 10.86 / 23.54; OPT-13B 10.13 / 46.04. Với OPT-6.7B, giữ (0,1%, 1%, 3%) kênh ở FP16 chọn theo activation cho PPL 11.58, 11.39, 11.36; trong khi chọn theo trọng số: 23.41, 22.37, 22.45; chọn ngẫu nhiên: 23.54, 24.23, 24.22. Tức chọn theo trọng số không hơn ngẫu nhiên.

Bảng 3 (INT3 g128, OPT):

PPL	1.3B	2.7B	6.7B	13B	30B
FP16	14.62	12.47	10.86	10.13	9.56
RTN	119.47	298.00	23.54	46.04	18.80

PPL	1.3B	2.7B	6.7B	13B	30B
1% FP16	16.91	13.69	11.39	10.43	9.85
s=2 (scale cố định)	18.63	14.94	11.92	10.80	10.32
AWQ	16.32	13.58	11.39	10.56	9.77

Bảng 4 (Wiki-2 PPL, họ LLaMA):

				LLaMA			
Cấu hình	Llama-2 7B	13B	70B	7B	13B	30B	65B
FP16	5.47	4.88	3.32	5.68	5.09	4.10	3.53
INT3 g128 RTN	6.66	5.52	3.98	7.01	5.88	4.88	4.24
INT3 GPTQ	6.43	5.48	3.88	8.81	5.66	4.88	4.17
INT3 GPTQ-R	6.42	5.41	3.86	6.53	5.64	4.74	4.21
INT3 AWQ	6.24	5.32	3.74	6.35	5.52	4.61	3.95
INT4 g128 RTN	5.73	4.98	3.46	5.96	5.25	4.23	3.67
INT4 GPTQ	5.69	4.98	3.42	6.22	5.23	4.24	3.66
INT4 GPTQ-R	5.63	4.99	3.43	5.83	5.20	4.22	3.66
INT4 AWQ	5.60	4.97	3.41	5.78	5.19	4.21	3.62

Ở INT4 mọi phương pháp đều gần FP16; khác biệt lớn ở INT3. Mistral-7B-Instruct-v0.2 và Mixtral-8x7B-Instruct-v0.1 (Bảng 5, Wiki PPL, AWQ): Mixtral FP16 5.94, INT4-g128 6.05, INT3-g128 6.52; Mistral FP16 4.14, INT4-g128 4.30, INT3-g128 4.83.

Kết quả trên VLM:

- OpenFlamingo-9B, COCO CIDEr (Bảng 6), chỉ lượng tử hóa phần ngôn ngữ vì nó chiếm phần lớn kích thước. 32-shot: FP16 81.70; INT4-g128 RTN 77.13 (-4.57), GPTQ 74.98 (-6.72), AWQ 80.53 (-1.17). INT3-g128: RTN 64.79 (-16.91), GPTQ 64.77 (-16.93), AWQ 74.47 (-7.23).
- VILA-7B / 13B INT4-g128 trên 11 benchmark thị giác-ngôn ngữ (Bảng 7): gần như không mất, ví dụ VILA-7B VQAv2 80.3 -> 80.1, TextVQA 62.6 -> 61.9, MME 1489.4 -> 1486.3; VILA-13B VQAv2 80.5 -> 80.4. Paper tự nhận đây là nghiên cứu đầu tiên về lượng tử hóa bit thấp cho VLM.

Lập trình / toán (Bảng 8, INT4-g128): CodeLlama-7b-Instruct MBPP pass@1: FP16 38.53, RTN 37.51, GPTQ 31.97, AWQ 40.64. GSM8K Llama-2 7B/13B/70B: FP16 13.87/26.16/56.41; RTN 11.07/21.23/53.98; GPTQ 12.13/24.26/56.03; AWQ 13.57/25.25/56.40.

INT2 g64 (Bảng 9, OPT, Wiki PPL): RTN sụp đổ (hàng nghìn tới hàng trăm nghìn); GPTQ 46.67/28.15/16.65/16.74/11.75; AWQ+GPTQ 35.71/25.70/15.71/13.25/11.38 (1.3B/2.7B/6.7B/13B/30B). AWQ trực giao với GPTQ.

Tốc độ (TinyChat, batch 1, 200 token sinh, prompt 4 token):

- RTX 4090: 2.7-3.9x so với HF FP16 trên Llama-2, MPT, Falcon. Llama-2-7B: 52 -> 62 token/s nhờ fusion FP16, rồi thêm 3.1x từ kernel lượng tử.
- Laptop RTX 4070 chỉ 8 GB: chạy Llama-2-13B ở 33 token/s (FP16 thậm chí không chứa nổi 7B). Jetson Orin: tăng tới 3.5x. Tổng quát 3.2-3.3x trung bình.
- VILA-7B trên A100: 81.6 (FP16) -> 155.3 token/s (AWQ); VILA-13B: 48.5 -> 102.1.
- So hệ thống khác (Hình 10): 1.2-3.0x so với llama.cpp/exllama/AutoGPTQ trên Jetson Orin.

6. Phân tích / ablation

- **Chọn salient theo activation vs theo trọng số:** chênh lệch rất lớn (Bảng 1). Đây là kết quả trung tâm.
- **Hiệu quả dữ liệu (Hình 8a):** AWQ cần calibration nhỏ hơn 10x so với GPTQ để đạt PPL tốt (16 so với 192 chuỗi, mỗi chuỗi 2.048 token).
- **Độ bền theo miền (Hình 8b, OPT-6.7B INT3-g128):** calibrate trên PubMed, đánh giá trên Enron (hoặc ngược lại) thì AWQ chỉ xấu đi 0,5-0,6 PPL, GPTQ xấu đi 2,3-4,9. Lý do paper đưa ra: AWQ chỉ đo độ lớn trung bình theo kênh nên không overfit.
- **Instruction-tuned (Hình 5):** đánh giá bằng GPT-4 so sánh với FP16, trên Vicuna-7B/13B INT3-g128, AWQ thắng nhiều hơn RTN và GPTQ.
- **s cố định vs tìm kiếm:** s = 2 cố định cho PPL kém hơn AWQ tìm kiếm (Bảng 3), xác nhận cần cân bằng salient/không salient.

7. Giới hạn

Do tác giả nêu hoặc hàm ý:

- Chỉ weight-only; không giảm FLOPs. Ở batch lớn / prefill (compute-bound) lợi ích nhỏ vì phải dequantize.
- Tăng tốc thực phụ thuộc kernel và phần cứng riêng (TinyChat); lý thuyết 4x chỉ đạt khoảng 3,2-3,3x.
- Phạm vi cùng bậc 7B-70B, đánh giá chủ yếu perplexity.

Nhận xét:

- Không có thí nghiệm với mô hình <1B hay mô hình bị cắt tỉa. Mô hình nhỏ ít dư thừa hơn nên chịu lượng tử hóa kém hơn (giả thuyết, cần kiểm).
- Không có đánh giá OCR / bài toán đòi hỏi chép chính xác từng ký tự, nơi sai số nhỏ ở một token có thể thành lỗi ký tự.
- Không bàn đến kiến trúc lai (GatedDeltaNet / linear attention) hay vị trí hợp nhất s^{-1} trong chúng.
- Perplexity trung bình có thể che giấu việc hỏng ở đuôi phân phối (ký tự hiếm, số).

8. Liên hệ với các bài khác

- **51 LLM.int8()** và **52 SmoothQuant:** cũng dựa trên quan sát kênh outlier của activation. SmoothQuant chuyển độ khó từ activation sang weight bằng cùng loại phép scale tương đương để làm W8A8; AWQ dùng scale tương tự cho mục tiêu khác (bảo vệ weight salient, weight-only).
- **53 GPTQ:** đối thủ chính. GPTQ dùng Hessian để bù sai số, AWQ thì không; hai phương pháp kết hợp được (Bảng 9).
- **44-46 FlashAttention / PagedAttention:** AWQ giảm bộ nhớ trọng số, PagedAttention quản lý bộ nhớ KV; hai thứ cộng dồn.
- **55 StreamingLLM, 56 H2O:** nén KV cache, phần bộ nhớ còn lại. Trong cùng tập hợp bài này, AWQ nén "trọng số", 55-56 nén "bộ nhớ động".
- **08 LoRA:** QLoRA-kiểu phối hợp lượng tử hóa với fine-tuning (không có trong paper này).
- **15 GQA / 14 MQA:** AWQ được thử trên Mistral (GQA) và Mixtral (MoE), cho thấy không phụ thuộc kiến trúc attention.

9. Tóm tắt công thức / khái niệm

Khái niệm	Công thức / ý nghĩa
Lượng tử hóa nhóm	$Q(w) = \Delta\,\text{Round}(w/\Delta)$, $\Delta = \max \lvert\mathbf{w}\rvert / 2^{N-1}$
Salient weight	0,1%-1% kênh có activation lớn, chọn theo X chứ không theo W

Khái niệm	Công thức / ý nghĩa
Scale tương đương	$Q(ws) \cdot x/s$; sai số tỉ lệ $\frac{\Delta'}{\Delta} \cdot \frac{1}{s}$
Tìm s	$\mathbf{s} = \mathbf{s}_X^{\alpha}$, $\alpha \in [0, 1]$, grid 20 điểm
Cường độ số học	FP16 ≈ 1 FLOP/byte; W4A16 ≈ 4
Cận độ trễ batch 1	$t \gtrsim$ (byte trọng số + KV)/bandwidth
W4 g128 dung lượng	≈ 4,125-4,25 bit/trọng số (tự tính)
Kết quả chính	INT4: gần FP16; INT3 vượt RTN/GPTQ; VLM không mất; TinyChat >3x

10. Kết luận ngắn

AWQ cho thấy một nguyên lý đơn giản: sai số lượng tử chỉ đáng sợ ở những kênh mà activation lớn, và có thể bảo vệ chúng bằng phép scale tương đương rẻ tiền, không backprop, bền theo miền và dùng được cho mô hình thị giác-ngôn ngữ. Giá trị thực của nó nằm ở chỗ nối được "ít byte hơn" với "token/s cao hơn" ở batch 1.

Nguồn gốc: https://arxiv.org/abs/2306.00978

Bài 55

Efficient Streaming Language Models with Attention Sinks (2024)

Tác giả: Guangxuan Xiao, Yuandong Tian, Beidi Chen, Song Han, Mike Lewis (MIT, Meta AI, CMU, NVIDIA).
Venue / arXiv: arXiv:2309.17453 (v4); ICLR 2024 (theo trang đầu của paper).
Nguồn: https://arxiv.org/abs/2309.17453
Chủ đề: Hiện tượng "attention sink", cache KV dạng cuộn (rolling) giữ vài token đầu + cửa sổ gần nhất, để LLM sinh văn bản vô hạn mà bộ nhớ không đổi, không cần fine-tune.
Vị trí trong lộ trình: Đọc trước: 01 (Attention), 12 (RoPE), 13 (ALiBi), 17 (Longformer, sliding window), 44 (FlashAttention). Bài này giải thích tại sao sliding window thuần túy thất bại và sửa bằng 4 token. Tiếp theo: 56 (H2O, giữ token "nặng" theo điểm attention; so sánh trực tiếp với bài này), 57 (FastV, tỉa token thị giác).

> Đây là ghi chú học tập bằng tiếng Việt, không phải bản dịch.

1. Bối cảnh

Khi LLM sinh từng token, mỗi lớp attention cần các vector Key và Value của những token đã xử lý. KV cache lưu chúng để tránh tính lại, nhưng dung lượng tăng theo độ dài chuỗi và batch; mỗi bước decode cũng phải đọc cache tương ứng.

Bài toán streaming. Một ứng dụng liên tục bổ sung lịch sử hội thoại có thể phải xử lý chuỗi rất dài. Bài báo nêu hai khó khăn:

1. Cache KV phình ra: tốn bộ nhớ và làm chậm decode.
2. Mô hình **không ngoại suy độ dài**: hỏng khi vượt cửa sổ pre-train (ví dụ 4K của Llama-2), kể cả khi dùng RoPE hay ALiBi.

Các phương án trước đó (Hình 1):

- (a) Dense attention: giữ tất cả, độ phức tạp tổng $O(T^2)$, cache tăng mãi, hỏng khi vượt độ dài pre-train.
- (b) **Window attention** (như Longformer, bài 17): chỉ giữ L token gần nhất. Bộ nhớ không đổi, nhưng perplexity tăng mạnh khi token đầu bị loại.
- (c) Sliding window với **tính lại** (re-computation): mỗi token mới dựng lại KV cho L token gần nhất. Chất lượng tốt nhưng $O(TL^2)$, rất chậm.

Bài báo đặt câu hỏi liệu có thể phục vụ chuỗi đầu vào rất dài mà vẫn giữ được chất lượng và bộ nhớ ổn định hay không.

2. Ý tưởng chính

1. Quan sát: ở hầu hết các lớp (trừ hai lớp dưới cùng), rất nhiều attention dồn vào **các token đầu tiên**, bất kể chúng có nghĩa hay không. Paper gọi đó là **attention sink** ("hố chìm" hút attention).
2. Lý do: softmax ép tổng attention bằng 1. Khi query không khớp mạnh với token nào, mô hình vẫn phải "đổ" trọng số đi đâu đó. Token đầu nhìn thấy được bởi mọi token sau (do mặt nạ nhân quả), nên nó dễ được huấn luyện thành chỗ đổ này.

3. Khi window attention loại token đầu, mẫu số của softmax mất một phần rất lớn, phân phối attention lệch khỏi lúc huấn luyện, mô hình hỏng.
4. Sửa: **luôn giữ KV của 4 token đầu (sink) + KV của L token gần nhất**. Không fine-tune, dùng kernel chuẩn.
5. Thêm: nếu pre-train với một "sink token" học được ở đầu mọi mẫu, chỉ cần 1 token sink là đủ.

3. Phương pháp chi tiết

3.1 Vì sao window attention hỏng

Softmax: $\text{SoftMax}(x)_i = \dfrac{e^{x_i}}{e^{x_1} + \sum_{j=2}^{N} e^{x_j}}$ với $x_1 \gg x_j$ (token đầu có logit rất lớn). Bỏ token đầu là bỏ phần lớn mẫu số.

Xác nhận bằng quan sát:

- Hình 2 (Llama-2-7B, logit attention trung bình trên 256 câu dài 16): hai lớp 0 và 1 có kiểu "cục bộ"; từ các lớp sau, mô hình dồn attention vào token đầu ở mọi head.
- Phụ lục F: với chuỗi 4096 token, attention của token thứ 4096 vào token đầu "thường vượt quá một nửa tổng attention, trừ hai lớp dưới".

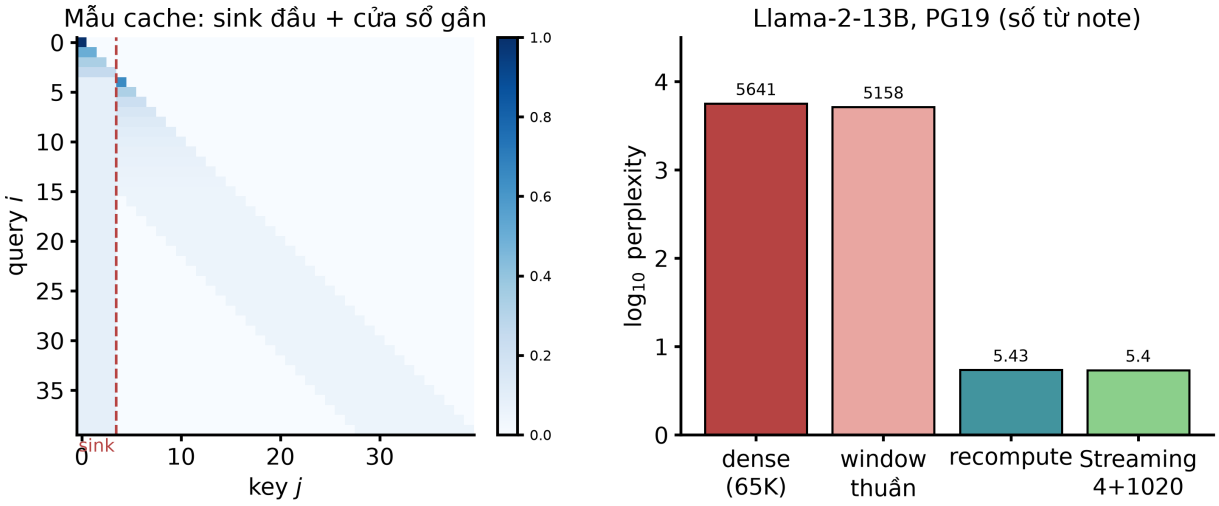
Do ngữ nghĩa hay do vị trí? Thay 4 token đầu bằng dấu xuống dòng "\n": mô hình vẫn dồn attention vào đó và khôi phục perplexity (Bảng 1: 4+1020 cho 5.40; 4"\n"+1020 cho 5.60). Kết luận: quan trọng là **vị trí tuyệt đối** của các token bắt đầu, không phải nội dung.

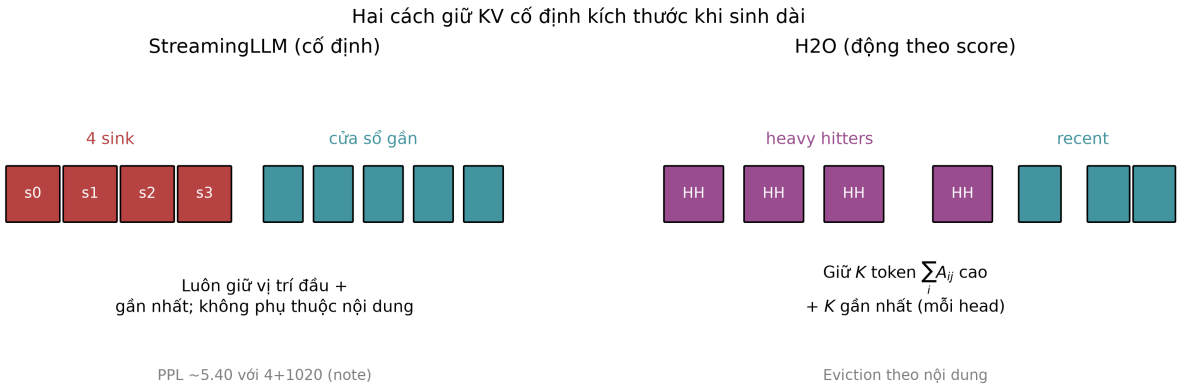
Vì sao cần tới 4 token thay vì 1? Các mô hình này khi pre-train không có token bắt đầu cố định ở mọi mẫu. Llama-2 chèn <s> trước khi cắt đoạn, nên token ở vị trí 0 phần lớn là token ngẫu nhiên. Do đó mô hình học dùng vài token đầu làm sink (đây là giải thích/giả thuyết của tác giả, được ủng hộ bởi thí nghiệm ở 3.3).

3.2 StreamingLLM: rolling KV cache với sink

Cache gồm hai phần (Hình 4):

1. **Attention sinks:** KV của 4 token đầu tiên, giữ vĩnh viễn.
2. **Rolling KV cache:** KV của $L - 4$ token gần nhất, cuộn theo thời gian.





StreamingLLM (sink cố định) vs H2O (heavy hitters động). (minh hoạ, tự tính)

StreamingLLM: giữ 4 sink + cửa sổ gần; PPL PG19 Llama-2-13B: dense/window ~5k vs Streaming 5.40. (minh hoạ, tự tính)

Vị trí được gán theo vị trí trong cache, không theo vị trí trong văn bản gốc. Ví dụ: cache đang chứa token [0,1,2,3,6,7,8] và giải mã token 9 thì vị trí là [0..7], không phải [0,1,2,3,6,7,8,9]. Đây là điểm then chốt.

- **Với RoPE** (bài 12): lưu Key *trước khi* xoay; ở mỗi bước decode áp phép xoay theo vị trí trong cache lên các Key của cache.
- **Với ALiBi** (bài 13): áp bias tuyến tính liên tục thay vì bias "nhảy cóc".

3.3 Sink token khi pre-train

Giả thuyết: nếu có một token sink chuyên dụng, mô hình không cần "mượn" nhiều token đầu. Ba mô hình 160M tham số được pre-train giống hệt, khác nhau ở chỗ:

- Vanilla (softmax thường);
- **Zero Sink:** thay softmax bằng $\text{SoftMax}_1(x)_i = \dfrac{e^{x_i}}{1+\sum_j e^{x_j}}$ (softmax-off-by-one; tương đương thêm một token có Key và Value bằng 0);
- **Learnable Sink:** thêm một token placeholder học được vào đầu mọi mẫu.

3.4 Pseudo-code

```
state: sink_kv  (KV của 4 token đầu, mỗi lớp)
       window_kv (circular buffer, kích thước L-4)

on each new token t:
    # 1. tính q,k,v cho t (K lưu trước RoPE)
    # 2. cache = concat(sink_kv, window_kv)   # độ dài <= L
    # 3. với mỗi lớp: gán vị trí 0..len(cache) cho cache + 1 cho t
    #    xoay K của cache theo vị trí đó (RoPE)  [hoặc ALiBi bias liên tục]
    # 4. attention(q_t, K_cache+k_t, V_cache+v_t)  -> output
    # 5. nếu len(window_kv) == L-4: bỏ phần tử cũ nhất của window_kv
    #    window_kv.append((k_t, v_t))
```

4. Phân tích chi phí: bộ nhớ và độ trễ

4.1 Công thức

Gọi n_ℓ số lớp full-attention, n_{kv} số KV head, d_h kích thước mỗi head, b số byte mỗi phần tử, T độ dài chuỗi, L kích thước cache.

$$\boxed{\mathrm{KV}(T) = 2\, n_\ell\, n_{kv}\, d_h\, b \cdot \min(T,\, L)} \quad (\text{StreamingLLM: } \min(T, L) = L)$$

Độ phức tạp thời gian tổng cộng (paper, Hình 1): Dense $O(T^2)$; window $O(TL)$; re-computation $O(TL^2)$; StreamingLLM $O(TL)$. Mỗi bước decode chỉ đọc L KV nên $O(L)$ mỗi token.

Chi phí thêm: xoay lại L Key bằng RoPE mỗi bước, $O(L \cdot d_h)$ mỗi head, nhỏ so với việc đọc KV.

4.2 Ví dụ số

Llama-2-7B (kiến trúc: 32 lớp, hidden 4096, MHA, tự nhớ, không từ paper; FP16): KV/token $= 2 \cdot 32 \cdot 4096 \cdot 2 = 524{,}288$ B ≈ 0.5 MiB (tự tính). Để đạt 4 triệu token (mốc paper thử), dense cần ≈ 2 TB. StreamingLLM với cache 4+2044 = 2048 token: $\approx$ 1,07 GB, cố định.

Độ trễ. Paper: trên A6000 với Llama-2-7B/13B, StreamingLLM nhanh tới **22,2x** mỗi token so với sliding-window-recompute, và bộ nhớ tương đương baseline đó. Độ trễ decode tăng gần tuyến tính theo kích thước cache (do phải đọc L KV), còn recompute tăng bậc hai.

Khi chuỗi chưa vượt kích thước cache L, StreamingLLM chưa giảm phần KV cần giữ so với attention đầy đủ. Chi phí thực tế vẫn phụ thuộc thời gian đọc cache và overhead của cách cài đặt.

5. Thí nghiệm và kết quả chính

Mô hình: Llama-2-[7,13,70]B (RoPE), MPT-[7,30]B (ALiBi), Falcon-[7,40]B, Pythia-[2.9,6.9,12]B. Dữ liệu: PG19 (100 cuốn sách ghép lại cho 4 triệu token).

Hình 1 (Llama-2-13B, cuốn sách đầu PG19, 65K token): dense PPL 5641; window 5158; sliding window recompute 5.43; StreamingLLM 5.40.

Bảng 1: Llama-2-13B, 0+1024 (window) 5158.07; 4+1020 5.40; 4 token "\n" + 1020 5.60.

Bảng 2 (PPL, số token sink khác nhau, 400K token PG19; cấu hình x+y = x token đầu + y token gần):

Mô hình	0+2048	1+2047	2+2046	4+2044	8+2040
Falcon-7B	17.90	12.12	12.12	12.12	12.12
MPT-7B	460.29	14.99	15.00	14.99	14.98
Pythia-12B	21.62	11.95	12.09	12.09	12.02

Mô hình	0+4096	1+4095	2+4094	4+4092	8+4088
Llama-2-7B	3359.95	11.88	10.51	9.59	9.54

Một hoặc hai token đầu chưa đủ cho Llama-2; 4 là ngưỡng đủ, thêm nữa lợi ích giảm dần.

Bảng 3 (160M, pre-train từ đầu, PPL cuốn sách PG19 đầu tiên):

Cache	0+1024	1+1023	2+1022	4+1020
Vanilla	27.87	18.49	18.05	18.05
Zero Sink	29214	19.90	18.27	18.01
Learnable Sink	1235	18.01	18.01	18.02

Mô hình có sink token chỉ cần 1 token sink (1+1023 cho 18.01). Phụ lục I: thêm token sink thứ hai không cải thiện thêm; với 2 sink token, mô hình cần cả hai (1+1023 cho 25.73).

Bảng 4 (zero-shot, vanilla / có sink token): ARC-c 18.6 / 19.6; ARC-e 45.2 / 45.6; HellaSwag 29.4 / 29.8; LAMBADA 39.6 / 39.9; OBQA 16.0 / 16.6; PIQA 62.2 / 62.6; Winogrande 50.1 / 50.8. Sink token không làm hại hội tụ hay chất lượng.

Hỏi đáp streaming (Bảng 5, ARC-E / ARC-C, cache 1024):

	Llama-2-7B-Chat	13B-Chat	70B-Chat
One-shot (đối chứng)	71.25 / 53.16	78.16 / 63.31	91.29 / 78.50
Dense	OOM	OOM	OOM
Window	3.58 / 1.39	0.25 / 0.34	0.12 / 0.32
StreamingLLM	71.34 / 55.03	80.89 / 65.61	91.37 / 80.20

StreamEval (Hình 9, Phụ lục C Bảng 7): StreamingLLM giữ độ chính xác hợp lý tới ~120K token; truy vấn trả lời được khi khoảng cách truy vấn-đáp án nằm trong cache. Llama-2-7B-32K-Instruct: ở khoảng cách 460 token, cache 4+2044 đạt 85.80%; ở khoảng cách 2.300 token với cache 4+2044 thì 0.00% (vượt cache), cache 4+4092 thì 61.60%.

LongBench (Phụ lục D, Bảng 8, Llama2-7B-chat): StreamingLLM 4+3496 thua truncation 1750+1750 (ví dụ NarrativeQA 11.6 vs 18.7); đặt lại 1750+1750 thì ngang (18.2).

6. Phân tích / ablation

- **Số token sink:** 4 là đủ (Bảng 2).
- **Kích thước cache (Bảng 6):** tăng cache **không** luôn giảm perplexity, tác giả nói mô hình có thể chưa tận dụng hết ngữ cảnh được cung cấp.
- **Zero Sink (softmax-off-by-one):** giúp một phần nhưng vẫn cần token đầu khác; learnable sink hiệu quả hơn.
- **Attention với sink token (Hình 7):** có sink token thì attention dồn vào nó ở mọi lớp, các token đầu khác bớt bị dùng làm sink.
- **Encoder:** BERT-base có hiện tượng tương tự ở token [SEP]; paper nhắc tới “registers” trong ViT (Darcet và cộng sự) như hiện tượng song song (Phụ lục B, H).

7. Giới hạn

Do tác giả nêu:

- Không mở rộng cửa sổ ngữ cảnh hay trí nhớ dài hạn: mô hình chỉ dùng được thông tin còn trong cache. Không phù hợp QA tài liệu dài, tóm tắt (Phụ lục A, C, D).
- Tăng cache không tăng chất lượng một cách nhất quán.

Nhận xét:

- Nhiều cách giải thích về nguồn gốc sink (softmax, số lượng token, vị trí) vẫn là giả thuyết, được kiểm bằng thí nghiệm gián tiếp (ví dụ mô hình 160M), chưa là chứng minh.
- Phép thử chủ yếu là perplexity và QA ngắn; không có tác vụ đòi hỏi chép chính xác văn bản xa (OCR kiểu “đọc lại ở đầu trang”).
- Số 22,2x so với baseline recompute (rất yếu về hiệu quả); so với dense-with-KV cache thông thường ở độ dài vừa phải thì lợi ích về tốc độ nhỏ hơn nhiều.

- Giả định chiến lược cache nhìn từ token đầu; với VLM, các token đầu thường là prompt/ảnh, nên "token đầu" có thể mang nội dung quan trọng (không được paper thử).

8. Liên hệ với các bài khác

- **17 Longformer:** window attention là nền; bài này chỉ ra window thuần túy hỏng trên mô hình đã pre-train dense và sửa bằng sink (Paper, Phụ lục B, nói Longformer/ETC/BigBird dùng attention toàn cục không hợp với mô hình tự hồi quy và phải huấn luyện lại, không áp được lên mô hình đã pre-train. Ta thấy "global tokens" của Longformer gần ý tưởng sink, nhưng đó là suy luận của ta).
- **12 RoPE, 13 ALiBi:** cần gán vị trí theo cache để hoạt động.
- **56 H2O:** cùng loại "chính sách loại KV". H2O chọn token theo điểm attention tích lũy thay vì vị trí cố định; H2O (Phụ lục C.4) nhận xét StreamingLLM mất thông tin ở giữa đầu vào, và H2O hoạt động trên 4 triệu token tốt hơn.
- **57 FastV:** tỉa token thị giác theo attention; cùng tinh thần "attention cho biết token nào bỏ được", nhưng đây là tỉa trong prefill/lớp sâu, không phải cache streaming.
- **44 FlashAttention, 46 PagedAttention:** chọn lọc KV tương thích với kernel chuẩn vì StreamingLLM không cần ma trận attention tường minh (khác H2O).
- **54 AWQ, 52 SmoothQuant:** paper có nhắc rằng ngoại lai (outlier) của lượng tử hóa liên quan đến attention sink (Xiao et al., Bondarenko et al.).
- **24 Gated DeltaNet / 18 Linear Transformers:** các lớp tuyến tính không có softmax, nên không có ràng buộc "tổng attention bằng 1" mà paper dùng để giải thích sink.

9. Tóm tắt công thức / khái niệm

Khái niệm	Nội dung
Attention sink	Token đầu nhận lượng attention lớn bất kể ngữ nghĩa, do softmax tổng = 1
Window attention	Chỉ giữ L KV gần nhất; hỏng khi token đầu bị loại
StreamingLLM	Giữ 4 KV đầu + L-4 KV gần nhất; vị trí gán theo vị trí trong cache
Kích thước KV	$2\, n_\ell n_{kv} d_h b \cdot \min(T, L)$
Độ phức tạp tổng	Dense $O(T^2)$; window/StreamingLLM $O(TL)$; recompute $O(TL^2)$
SoftMax_1	$e^{x_i}/(1+\sum_j e^{x_j})$ ("Zero Sink")
Sink token học được	Thêm 1 token vào đầu mọi mẫu khi pre-train; chỉ cần 1 sink khi streaming
Kết quả	Ổn định tới 4 triệu token; nhanh tới 22,2x so với recompute

10. Kết luận ngắn

Bài này biến một "lỗi kỳ lạ" (sliding window hỏng khi bỏ token đầu) thành một cơ chế được giải thích: softmax cần một chỗ để đổ attention thừa, và token đầu đóng vai đó. Bốn token đầu cộng một cửa sổ gần nhất là đủ để giữ bộ nhớ cố định mà không fine-tune.

Nguồn gốc: https://arxiv.org/abs/2309.17453

Bài 56

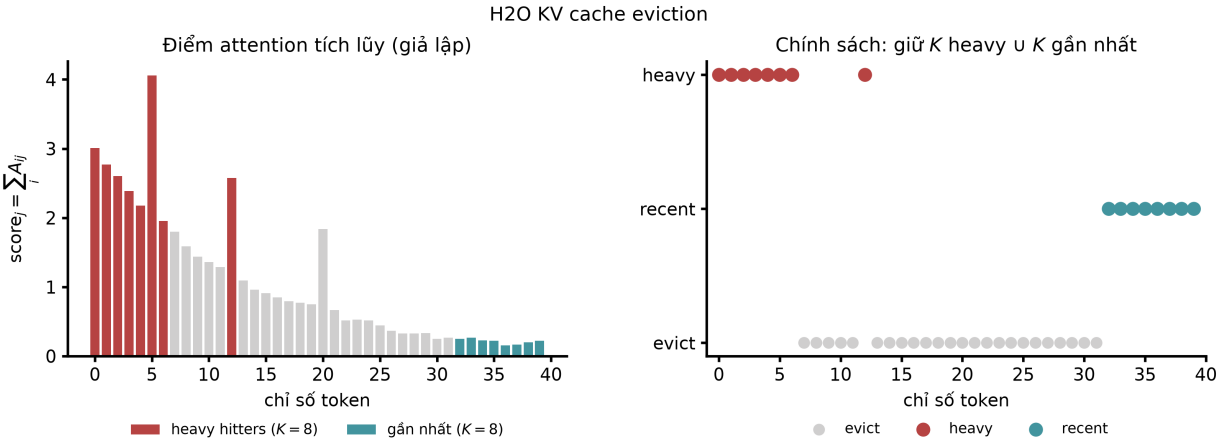
H2O: Heavy-Hitter Oracle for Efficient Generative Inference of Large Language Models (2023)

Tác giả: Zhenyu Zhang, Ying Sheng, Tianyi Zhou, Tianlong Chen, Lianmin Zheng, Ruisi Cai, Zhao Song, Yuandong Tian, Christopher Ré, Clark Barrett, Zhangyang Wang, Beidi Chen.
Venue / arXiv: arXiv:2306.14048 (v3); NeurIPS 2023 (theo trang đầu của paper).
Nguồn: https://arxiv.org/abs/2306.14048
Chủ đề: Chính sách loại bỏ (eviction) KV cache dựa trên điểm attention tích lũy (“Heavy Hitters”) cộng token gần nhất; phân tích dưới góc độ submodular; tăng thông lượng khi batch lớn.
Vị trí trong lộ trình: Đọc trước: 01, 14-15 (MQA/GQA), 44 (FlashAttention), 46 (PagedAttention), 55 (StreamingLLM). Bài này là phiên bản “thích nghi theo nội dung” của 55. Tiếp theo: 57 (FastV, tỉa token thị giác).
Phạm vi đọc: bài dài ~24K từ vì phụ lục lý thuyết (submodular, chứng minh) rất dày. Ghi chú này tập trung vào quan sát, thuật toán, thực nghiệm và hàm ý hệ thống; phần chứng minh chỉ tóm lược.

> Đây là ghi chú học tập bằng tiếng Việt, không phải bản dịch.

1. Bối cảnh

KV cache lưu Key và Value của các token đã xử lý để không phải tính lại ở mỗi bước sinh. Đổi lại, dung lượng cache tăng theo độ dài chuỗi và batch. Bài báo nêu ví dụ mô hình 30B với batch 128 và chuỗi 1024 token cần khoảng 180 GB KV cache. Với OPT-30B, 48 lớp, hidden 7168 và FP16 cho $2 \cdot 48 \cdot 7168 \cdot 2 = 1.38$ MB/token; nhân với $128 \cdot 1024$ cũng cho khoảng 180 GB.



H2O: giữ K heavy hitters theo $\sum_i A_{ij}$ và K token gần nhất; evict phần còn lại. (minh hoạ, tự tính)

Hai giai đoạn của sinh văn bản (paper):

- **Prompt phase / prefill:** xử lý toàn bộ prompt song song, tạo KV cache.
- **Token generation / decode:** sinh từng token, đọc và cập nhật KV cache.

Muốn giới hạn kích thước cache giống cache phần mềm/phần cứng (LRU, LFU, thuật toán tối ưu Belady) nhưng khó: loại nhầm KV quan trọng thì phá hỏng chất lượng vì sinh văn bản có phụ thuộc tuần tự. Các cách trước đó (Sparse Transformer, Reformer, low-rank, MQA) hoặc cần huấn luyện lại, hoặc áp thẳng vào LLM

đã pre-train thì chất lượng rơi (Hình 1). Một ứng viên lý tưởng cần: (i) cache nhỏ, (ii) tỉ lệ miss thấp, (iii) chính sách loại rẻ.

2. Ý tưởng chính

1. **Attention thưa:** dù huấn luyện dày, ma trận attention lúc suy luận thưa hơn 95% ở hầu hết các lớp (ngưỡng = 1% của giá trị lớn nhất mỗi hàng). Gợi ý có thể giảm KV tới 20x.
2. **Heavy Hitters (H2):** tổng điểm attention tích lũy của mỗi token theo mọi bước tuân theo **luật lũy thừa**: một số ít token nhận phần lớn attention. Gọi chúng là H2. Xóa H2 làm chất lượng sụp đổ; H2 tương quan với các từ xuất hiện đồng thời nhiều trong dữ liệu.
3. **Thuật toán H2O:** cache giữ ngân sách k gồm một nửa token "nặng" (điểm tích lũy cao) và một nửa token gần nhất. Mỗi bước, loại token có điểm tích lũy thấp nhất (ngoài vùng gần nhất).
4. Chỉ dùng thống kê cục bộ (tổng attention từ các token đã sinh), không cần biết tương lai, mà hiệu quả ngang với dùng thống kê toàn cục.
5. Cơ sở lý thuyết: coi đây là bài toán submodular động; tham lam gần tối ưu theo cận $(1-\alpha)(1-1/e)\max f-\beta$.

3. Phương pháp chi tiết

3.1 Bài toán có ngân sách cache

Cho $Q, K \in \mathbb{R}^{n\times d}$, ngân sách $k < n$. Tập chỉ số đang giữ trong cache khi dự đoán token i là S_i, $|S_i| = k$, và mỗi bước chỉ thay đổi tối đa 1 phần tử ($|S_i \setminus S_{i-1}| \le 1$). Attention chuẩn hóa chỉ trên các token còn lại:

$$o_i = D_i^{-1}\exp(Q_{i,*}K_{S_i,*}^\top), \qquad D_i = \big(\exp(Q_{i,*}K_{S_i,*}^\top) - \not\Vdash_{[i]\setminus S_i}\big)\cdot \mathbf{1}$$

Mục tiêu: đầu ra gần như khi không giới hạn cache.

3.2 Chính sách loại H2O

Điểm của một tập token là tổng attention nhận được:

$$F_{\text{score}}(T) = \sum_{s\in T} o_s, \qquad u = \arg\max_{v\in S_{i-1}\cup\{i\}} F_{\text{score}}(S_{i-1}\cup\{i\}\setminus\{v\})$$

nghĩa là loại token u mà việc bỏ nó làm tổng điểm của phần còn lại lớn nhất, hay đơn giản **loại token có điểm tích lũy nhỏ nhất**.

Trong hiện thực: giữ K heavy hitters và K token gần nhất; mỗi bước decode: bỏ token cũ nhất trong K cuối, loại một heavy hitter cho mỗi head, thêm token mới. **Mỗi head, mỗi lớp quyết định độc lập** (khác SpAtten, vốn cộng điểm qua head và lớp).

3.3 Pseudo-code

```
state per (layer, head): K_cache, V_cache (<= 2K entries), score[]  (điểm tích lũy)

prefill (độ dài n):
    tính attention A (n x n) song song
    score[j] = sum_i A[i,j]                       # tổng attention mỗi token nhận
    nếu n >= 2K: giữ K token có score cao nhất U K token cuối
    ngược lại: giữ hết

decode (token mới t):
    a = softmax(q_t · K_cache^T)                   # attention trên cache
```

```
out = a · V_cache
score += a;  score[t] = 0 (khởi tạo)
nếu cache đầy:
    # ứng viên loại: các token KHÔNG nằm trong K token gần nhất
    u = argmin_{j not in recent K} score[j]
    thay (K_cache[u], V_cache[u], score[u]) bằng (k_t, v_t, 0)
    # recent: dùng circular queue; vị trí token cũ nhất trong recent trở thành ứng
    ↪ viên heavy hitter
```

Ví dụ (Hình 3): ngân sách 3, sau bước giải mã thứ 4 KV của token thứ 3 (điểm tích lũy thấp nhất) bị loại và không truy cập lại được.

Chi tiết hệ thống: cài đặt trên FlexGen; bộ nhớ KV cấp phát trước, hàng đợi vòng cho K phần tử cuối, ghi đè thay vì hoán đổi bộ nhớ.

3.4 Lý thuyết (tóm lược)

Giả sử hàm attention là **submodular** (lợi ích biên giảm dần khi tập lớn hơn). Khi đó chọn tham lam top-k theo điểm đạt xấp xỉ $(1 - 1/e)$ của tối ưu. Paper mở rộng sang khung "dynamic submodular" (Định nghĩa 4.1) vì tập thay đổi từng bước, và chứng minh (Định lý 4.4, mô tả không chính thức trong thân bài) rằng

$$f(\tilde{S}_i) \geq (1 - \alpha)\left(1 - \tfrac{1}{e}\right) \max_{|S|=k} f(S) - \beta.$$

Đây là lý giải vì sao tham lam có thể tốt (dựa trên giả định "nhẹ"); ghi chú này không kiểm tra chứng minh.

4. Phân tích chi phí: bộ nhớ và độ trễ

4.1 Công thức

$$\boxed{\text{KV bytes} = 2 \cdot B \cdot T \cdot n_\ell \cdot n_{kv} \cdot d_h \cdot b}$$

với B batch, T độ dài chuỗi, n_ℓ số lớp full-attention, n_{kv} số KV head, d_h chiều mỗi head, b byte/phần tử. H2O với ngân sách tỉ lệ r (ví dụ 20%) đưa T thành rT (nhưng điểm tích lũy thêm một mảng float mỗi head).

Thời gian decode ở batch B (memory-bound):

$$t_{\text{step}} \gtrsim \frac{P\, b_w + \text{KV bytes}}{\text{bandwidth}}$$

Trọng số Pb_w chia sẻ cho cả batch (đọc một lần), còn KV tỉ lệ với B. Vì vậy:

- **Batch lớn / chuỗi dài:** KV chiếm ưu thế, giảm KV giúp thật sự (bài này: tới 29x thông lượng).
- **Batch 1 / chuỗi ngắn:** trọng số chiếm ưu thế, giảm KV gần như không giúp.

Chi phí thêm của H2O: cập nhật điểm $O(k)$ mỗi head mỗi bước và argmin (hoặc heap); cần **xác suất attention tường minh** của bước decode. Việc đó rẻ khi decode (một query), nhưng ở prefill phải tổng theo cột toàn bộ ma trận $n \times n$; điều này không tương thích tự nhiên với kernel FlashAttention vốn không trả ma trận attention (đây là suy luận; paper cài trên FlexGen).

4.2 Ví dụ số

Ví dụ của paper: OPT-30B, $B = 128$, $T = 1024$: 180 GB. Với 20% còn ~36 GB (tự tính).

5. Thí nghiệm và kết quả chính

Thiết lập: OPT (6.7B tới 175B ở phụ lục), LLaMA-7B/13B/30B, GPT-NeoX-20B; 8 tác vụ: COPA, MathQA, OpenBookQA, PiQA, RTE, Winogrande (lm-eval-harness, 5-shot) và XSUM, CNN/Daily Mail (HELM, 0-shot, 1000 mẫu); thêm AlpacaEval, MT-bench ở phụ lục. Ngân sách KV 4%, 10%, 20%, 60% độ dài prompt. GPU: A100 80GB cho độ chính xác; T4 16GB và A100 cho thông lượng. Baseline: toàn bộ cache, "Local" (chỉ token gần nhất), Sparse Transformer (strided, fixed), Top-K, SpAtten, StreamingLLM.

Kết luận chính (mục 5.1): với ngân sách <20% (giảm >5x bộ nhớ KV), H2O cho kết quả ngang cache đầy ở phần lớn tác vụ, và vượt xa "Local".

Bảng 1 (20% KV; paper không ghi rõ mô hình trong chú thích, ta chỉ trích 4 tác vụ):

Phương pháp	PiQA	COPA	OpenBookQA	Winogrande
Full	80.09	81.00	44.80	71.51
Local	57.94	56.00	28.40	51.30
H2O	79.22	85.00	43.80	71.67

(Các hàng 0-shot và 1-shot Full trong bảng bị lẫn cột trong bản text nên ta không trích; chỉ dùng hàng Full, Local, H2O như trên, những số này khớp ngữ cảnh. Thận trọng khi đối chiếu bản gốc.)

Bảng 2 (OPT-30B, 20% KV; H2 cải thiện các phương pháp thưa khác):

	COPA	OpenBookQA	PiQA	Winogrande
Full	85.00	43.20	78.51	70.24
Local (không H2)	48.00	25.20	55.82	49.17
Local + H2	84.00	43.00	78.45	69.06
Strided (không H2)	50.00	24.60	56.20	47.59
Strided + H2	83.00	42.60	78.24	69.61
Fixed (không H2)	61.00	23.80	58.60	49.88
Fixed + H2	76.00	41.40	77.80	64.96

Phụ lục C.5: Top-K cũng cải thiện thêm khi kết hợp H2 (ví dụ COPA 80.00 -> 82.00).

Thông lượng (token/s, T4 16GB; mô hình 6.7B / 30B; ngoặc là batch hiệu dụng và mức bộ nhớ thấp nhất cần):

Chuỗi (prompt + sinh)	512+32	512+512	512+1024
FlexGen 6.7B	20.2 (2, G)	16.8 (1, G)	16.9 (1, G)
H2O (20%) 6.7B	35.1 (4, G)	51.7 (4, G)	52.1 (4, G)
FlexGen 30B	8.1 (144, C)	8.5 (80, C)	7.1 (48, C)
H2O (20%) 30B	12.7 (728, C)	18.83 (416, C)	13.82 (264, C)

(Accelerate và DeepSpeed ở Bảng 3 thấp hơn rất nhiều: ví dụ 30B 0,6 token/s.) Tổng hợp: tới 29x so với DeepSpeed và Accelerate, 3x so với FlexGen. Trên XSUM thực (Bảng 4): 6.7B FlexGen 10.80 -> H2O 30.40; 30B FlexGen 3.29 -> H2O 6.70.

A100 (Bảng 5):

Cấu hình	FlexGen	H2O (20%)
30B, 7000+1024, batch 1, độ trễ	57.0 s	50.4 s
13B, 5000+5000, batch 4, độ trễ	214.2 s	155.4 s
6.7B, 2048+2048, batch 24, độ trễ	99.5 s	53.5 s
6.7B, 2048+2048, batch 24, thông lượng	494.1	918.9 token/s
6.7B, batch 64, thông lượng	OOM	1161.0 token/s

Điểm đáng chú ý cho **batch 1**: ở 30B, 7000+1024, batch 1 độ trễ chỉ giảm từ 57,0 s xuống 50,4 s (~1,13x), vì mô hình 30B ở batch 1 bị trọng số chi phối. Giảm độ trễ 1,9x chỉ ở batch lớn hơn (6.7B, batch 24: 99,5 s xuống 53,5 s). Dòng "với cùng batch size, giảm độ trễ tới 1,9x" trong abstract tương ứng ca này.

Kết hợp lượng tử hóa (Bảng 6, OPT-30B): 4-bit toàn bộ (COPA 84.00, OBQA 43.28, PiQA 78.67) và H2O + 4-bit (84.00, 43.20, 78.80), tức gần như không hỏng thêm. Bảng 7: lượng tử hóa trọng số 4-bit của FlexGen cho phép batch rất lớn (70 so với 4), thông lượng 6.7B từ 35.1 lên 50.5 (512+32) và 52.1 lên 62.3 (512+1024), dưới 2x dù batch tăng ~20 lần vì kernel lượng tử chưa tối ưu.

Dòng chảy vô hạn (Hình 5): H2O đạt 4 triệu token trên PG-19 với perplexity thấp hơn StreamingLLM ở nhiều cỡ cache (kết quả trên hình, số cụ thể không đọc được từ text).

6. Phân tích / ablation

- **H2 và recent đều cần (Bảng 9, Full / chỉ Local / chỉ H2 / Local+H2):**

	OPT-13B	OPT-30B
PiQA	77.37 / 54.62 / 76.12 / 77.26	78.51 / 55.82 / 67.25 / 78.45
OpenBookQA	41.40 / 25.60 / 30.40 / 41.20	43.20 / 25.20 / 26.60 / 43.00
MathQA	26.67 / 22.04 / 23.82 / 26.93	26.23 / 20.87 / 21.98 / 26.87
Winogrande	68.59 / 49.96 / 51.85 / 67.32	70.24 / 49.17 / 47.36 / 69.06

Chỉ H2 hay chỉ Local đều tụt 2,85% tới 22,75%; cả hai mới giữ hiệu năng. H2 nhìn chung đóng góp nhiều hơn.

- **Số shot (Bảng 10):** ở 5-shot và 10-shot, H2O cách bản đầy dưới 1,00%; "Local" mất tới 37%. 0/1-shot cần ngân sách cao hơn (30-40%) vì chuỗi ngắn (100-300 token).
- **So với SpAtten (Bảng 11, OPT-30B):** COPA 82.00 (SpAtten) vs 84.00 (H2O), OBQA 41.90 vs 43.00, PiQA 77.06 vs 78.45. Khác biệt: H2O quyết định độc lập theo head/lớp, thêm cả token gần nhất.
- **So với StreamingLLM (Phụ lục C.4):** StreamingLLM hỏng khi thông tin then chốt nằm giữa đầu vào (QA nhiều tài liệu, tóm tắt), vì chỉ giữ đầu + gần nhất; H2O giữ được nhờ điểm attention.
- **Điểm trung bình thay vì tích lũy:** thử dùng attention trung bình để chọn, kết quả tệ hơn (Phụ lục B.2). Tác giả lưu ý tổng tích lũy thiên vị token cũ (có nhiều bước để cộng dồn), nên đã thử trung bình, nhưng trung bình cho kết quả kém hơn.
- **Heavy hitter xuất hiện nhiều ở đầu câu** (Phụ lục B.2), gợi ý token đầu quan trọng (gần với sink của bài 55).
- **Đa dạng văn bản:** H2O cho văn bản ít lặp từ hơn (định tính, Phụ lục C.1).
- **H2 trong MLP (C.10):** tần suất kích hoạt neuron cũng theo luật lũy thừa; xóa H2 làm hỏng nhưng phục hồi được chỉ với 1% dữ liệu huấn luyện; H2 xuất hiện sớm khi huấn luyện ("early-bird"). Một hướng gợi ý của tác giả cho việc tiết kiệm tham số.

7. Giới hạn

Do tác giả nêu:

- Thiên lệch của điểm tích lũy theo token cũ (đã thử trung bình nhưng kém hơn).
- Phần lớn tham số nằm ở MLP; tác giả nói hướng tương lai là khai thác H2 trong MLP cho chính sách offloading.
- Token bị loại không quay lại được.

Nhận xét:

- Độ chính xác đánh giá chủ yếu bằng trắc nghiệm/few-shot ngắn và tóm tắt trên mô hình OPT/LLaMA đời đầu; ít kiểm tra tác vụ cần truy xuất chính xác thông tin ở khoảng cách xa (ngoại trừ vài thử nghiệm ở phụ lục).
- Thông lượng được đo chủ yếu so với các hệ offloading (FlexGen, DeepSpeed, Accelerate) trên T4; các hệ hiện đại (PagedAttention, bài 46) và GQA/MQA vốn đã giảm KV, nên lợi ích tương đối nhỏ hơn với mô hình GQA (không đo trong paper).
- Cần xác suất attention tường minh ở mỗi bước; khó tích hợp với kernel hợp nhất kiểu FlashAttention (suy luận); cần xử lý riêng với GQA (cộng điểm của nhiều query head vào cùng một KV head).
- Lợi ích tính toán thực chủ yếu ở batch lớn; với batch 1 rất nhỏ (xem 4.2).
- Định lý phụ thuộc giả định submodular "nhẹ" mà không được kiểm chứng trực tiếp trên mọi mô hình.

8. Liên hệ với các bài khác

- **55 StreamingLLM:** cùng bài toán, chiến lược cố định (đầu + gần nhất) so với chiến lược động theo nội dung (gần nhất + heavy hitters). H2O thừa nhận cảm hứng từ StreamingLLM cho thí nghiệm vô hạn.
- **14 MQA, 15 GQA, 16 MLA:** giảm KV theo kiến trúc (huấn luyện từ đầu). H2O giảm KV theo thời gian suy luận; hai hướng cộng dồn nhưng mô hình dùng GQA/MLA có KV đã nhỏ nên H2O ít giá trị hơn.
- **17 Longformer:** sliding window, tĩnh.
- **44-46 FlashAttention, PagedAttention:** FlashAttention không trả attention; PagedAttention quản lý KV theo trang, có thể kết hợp với chính sách loại.
- **57 FastV:** tỉa token thị giác dựa trên attention ở lớp sớm, ý tưởng gần với H2O nhưng cho prefill VLM.
- **54 AWQ:** kết hợp được (Bảng 6): lượng tử hóa trọng số + giảm KV.
- **24 Gated DeltaNet, 18, 19:** là cách tiếp cận kiến trúc để có bộ nhớ cố định (trạng thái nén), thay vì loại bỏ token rời rạc.

9. Tóm tắt công thức / khái niệm

Khái niệm	Nội dung
Heavy Hitter	Token có tổng attention tích lũy cao; phân bố luật lũy thừa
Chính sách H2O	Giữ $k/2$ heavy hitters + $k/2$ gần nhất; loại điểm thấp nhất
Điểm	$\text{score}_j = \sum_i A_{ij}$ (theo từng head, từng lớp)
Sparsity	Attention thưa >95% (ngưỡng 1% max mỗi hàng)
KV cache	$2BTn_\ell n_{kv} d_h b$ byte
Đảm bảo	$f(\tilde{S}) \geq (1-\alpha)(1-1/e)\max f - \beta$ (dưới giả định submodular)
Kết quả	5x ít KV mà giữ chất lượng; thông lượng tới 29x/29x/3x; trễ tới 1,9x (cùng batch)
Với batch 1	Lợi ích nhỏ: 57,0 s -> 50,4 s ở 30B batch 1

10. Kết luận ngắn

H2O chỉ ra rằng attention trong LLM đủ thưa và lệch để giữ khoảng 20% KV mà vẫn gần như không mất chất lượng, nhờ kết hợp token gần nhất và token "nặng" theo điểm tích lũy, với lý thuyết tham lam submodular đứng sau. Lợi ích hệ thống chủ yếu xuất hiện khi batch lớn hoặc chuỗi dài, nơi KV chiếm ưu thế.

Nguồn gốc: https://arxiv.org/abs/2306.14048

Bài 57

An Image is Worth 1/2 Tokens After Layer 2: Plug-and-Play Acceleration for VLLM Inference (FastV) (2024)

Tác giả: Liang Chen, Haozhe Zhao, Tianyu Liu, Shuai Bai, Junyang Lin, Chang Zhou, Baobao Chang (Peking University, Alibaba Group).
Venue / arXiv: arXiv:2403.06764 (ngày đầu 11/03/2024; bản v3 02/09/2024). Paper không nêu venue trong phiên bản được tham khảo.
Nguồn: https://arxiv.org/abs/2403.06764
Chủ đề: Cắt bỏ visual token trong LVLM khi suy luận, không cần huấn luyện; phân tích "attention kém hiệu quả trên ảnh" ở các lớp sâu.
Vị trí trong lộ trình: Đọc trước: 01 (Attention), 34/35 (LLaVA, LLaVA-1.5), 55 (StreamingLLM, attention sink). Bài này là một phương pháp **tối ưu suy luận cho VLM** nhắm vào số visual token; đối chiếu với 56 (H2O, loại bỏ KV theo điểm attention) và với hướng nén token ở encoder (41 GOT gộp patch 2×2).

> Ghi chú học tập bằng tiếng Việt, **không phải bản dịch**. Số đánh dấu "tự tính" được tính (giả định cấu hình như ghi chú), không phải số của paper.

1. Bối cảnh

Trong LVLM kiểu LLaVA, vision encoder như CLIP-ViT trích đặc trưng ảnh rồi chuyển chúng thành chuỗi token phù hợp với đầu vào LLM; các kiến trúc khác có thể dùng Q-Former hoặc cross-attention. Ảnh 336×336 của LLaVA-1.5 tạo 576 token, ảnh 672×672 tạo 2.304 token, còn video tám khung hình của Video-LLaVA tạo 2.048 token. Khi độ phân giải tăng, số token tăng gần theo bình phương cạnh ảnh, kéo theo chi phí tính toán và bộ nhớ.

Vấn đề: LLM xử lý token ảnh **như token văn bản** ở mọi lớp. Các kỹ thuật tối ưu LLM (FlashAttention, vLLM/PagedAttention) giữ nguyên kết quả; các kỹ thuật dựa trên mẫu attention (StreamingLLM, FastGen) chỉ thử trên LLM văn bản. Trước FastV, việc cắt token chủ yếu được nghiên cứu cho ViT (EViT, SPViT...) hoặc cần fine-tune (LLaMA-VID).

Thuật ngữ: **FLOPs** = số phép tính dấu phẩy động; **prefill** = bước xử lý toàn bộ prompt (gồm visual token) một lần trước khi sinh; **decode** = bước sinh từng token. **Attention sink / anchor token** = vài token (thường ở đầu chuỗi) nhận lượng attention rất lớn bất kể nội dung (xem 55).

2. Ý tưởng chính

Tác giả đo attention trong LLaVA-1.5 và thấy: dù ảnh chiếm ~64% số token đầu vào, từ **sau lớp 2** ảnh nhận **rất ít** attention (ở lớp sâu, hiệu quả attention của token ảnh chỉ bằng ~1/472 token system prompt). Các tác giả giả thuyết rằng thông tin ảnh đã được tập hợp vào một số token neo không phải ảnh ở lớp nông, nhờ đó các lớp sâu ít cần đọc trực tiếp token ảnh hơn. FastV tận dụng điều này: ***ở lớp K, xếp hạng token ảnh theo attention trung bình nhận được, bỏ R% token ít được chú ý nhất***; mọi lớp sau chỉ xử lý số token còn lại. Vì token bị bỏ hẳn, tiết kiệm cả **attention lẫn FFN** của các lớp sau, không cần huấn luyện lại. Với LLaVA-1.5-13B: cắt 50% token sau lớp 2 giảm ~45% FLOPs mà gần như không mất điểm.

3. Phương pháp chi tiết

3.1 Đo "attention kém hiệu quả"

Một LVLM nhận 4 loại token: system prompt (sys), ảnh (img), instruction của người dùng (ins), token sinh ra (out). Với token thứ i trong câu trả lời, ở lớp j, gọi $\alpha_{type}^{i,j}$ là tổng xác suất attention nó dành cho từng loại:

$$\alpha_{sys}^{i,j} + \alpha_{img}^{i,j} + \alpha_{ins}^{i,j} + \alpha_{out}^{i,j} = 1$$

Phân bổ attention (attention allocation) của một loại ở lớp j (cộng trên n token đầu ra):

$$\lambda_{type}^{j} = \sum_{i=1}^{n} \alpha_{type}^{i,j}$$

Hiệu quả attention (attention efficiency) = attention trung bình trên mỗi token của loại đó:

$$\boxed{\epsilon_{img}^{j} = \frac{\sum_{i=1}^{n} \alpha_{img}^{i,j}}{|img|}}$$

Trong đó $|img|$ là số token ảnh; kết quả lấy trung bình trên các head và trên N = 1.000 cặp ảnh-văn bản (mẫu từ Flickr30K, PCA-Bench, A-OKVQA, MMMU), mô hình LLaVA-1.5-7B. "Lớp nông" = 2 lớp đầu, "lớp sâu" = 30 lớp còn lại.

Phát hiện (paper):

1. Mất cân bằng attention tăng theo độ sâu. Ở lớp nông token sinh ra chủ yếu nhìn các token sinh trước đó; ở lớp sâu chúng nhìn vào system prompt.
2. **Token ảnh có hiệu quả thấp nhất ở mọi độ sâu.** Ở lớp sâu, system prompt có hiệu quả gấp **472 lần** token ảnh và chiếm **85%** tổng attention. Số trong hình 3 (độ dài trung bình): sys 35, ảnh 576, ins 135, out 150 token; tỉ lệ hiệu quả lớp sâu (so ảnh = 1×): sys 472×, ins 3×, out 12,8×; lớp nông: sys 2×, ins 3×, out 9×. Phần giới thiệu nêu: ở lớp sâu, attention của ảnh chỉ bằng 0,21% của system prompt, còn ở 2 lớp đầu thì tới 50%.
3. Bản đồ attention lớp sâu có các **vạch dọc** (một số token luôn nhận nhiều attention suốt quá trình giải mã) ở system prompt.

Giả thuyết của tác giả (không được chứng minh trực tiếp): tín hiệu thị giác dư thừa nhiều nên ở lớp nông self-attention gom thông tin ảnh có liên quan tới instruction vào một số token "neo" (không phải token ảnh); ở lớp sâu attention tập trung vào các token neo, bỏ qua token ảnh. Phù hợp với kết quả "label words are anchors" (Wang et al. 2023).

3.2 Thuật toán FastV

Hai siêu tham số: **lớp lọc K** và **tỉ lệ lọc R%**. Hàm xếp hạng ϕ:

- ϕ_{attn}: với mỗi token, điểm = **attention trung bình nó nhận được từ tất cả token khác** (dùng trong thí nghiệm chính, tính ở lớp K).
- ϕ_{rand}: bỏ ngẫu nhiên; dùng trong trường hợp cực trị K = 0 (cắt token ảnh ngay trước khi vào LLM, chưa có attention để xếp hạng).

```
# prefill, hidden: [n_tokens, d]; img_idx: chỉ số các token ảnh
for layer l in 0 .. T-1:
    if l == K:
        A = attention_probs(layer K)            # [heads, n, n] (đã trung bình theo head)
        score[j] = mean_i A[i, j]               # attention trung bình token j nhận từ
        ↪  các token khác
```

```
        drop = bottom_R%(score restricted to img_idx)
        keep = all tokens except drop
        hidden, positions = hidden[keep], positions[keep]   # các lớp sau chỉ thấy n_hat
        ↪  token
    hidden = transformer_block[l](hidden)       # attention + FFN trên tập token đã cắt
# decode: các token sinh ra chỉ attend vào tập token còn lại ở các lớp >= K
```

Chỉ token ảnh bị cắt; token văn bản được giữ. Plug-and-play: không huấn luyện lại, áp dụng cho bất kỳ LVLM dạng token nào (LLaVA, Qwen-VL, InstructBLIP, Video-LLaVA).

3.3 Ước lượng chi phí (công thức của paper)

Cho một lớp transformer với n token, hidden size d, FFN trung gian m:

$$\text{FLOPs}_{layer} = 4nd^2 + 2n^2d + 2ndm$$

($4nd^2$: bốn phép chiếu Q, K, V, O; $2n^2d$: QK^T và AV; $2ndm$: FFN với hai ma trận.) Gọi $\hat{n} = (1 - R)\, n$ sau lớp K, tổng T lớp, mức giảm FLOPs lý thuyết:

$$\boxed{1 - \frac{K(4nd^2 + 2n^2d + 2ndm) + (T - K)(4\hat{n}d^2 + 2\hat{n}^2d + 2\hat{n}dm)}{T(4nd^2 + 2n^2d + 2ndm)}}$$

Trực giác: giữ nguyên K lớp đầu, còn $T-K$ lớp sau chạy với $\hat{n}$ token; phần n^2 giảm nhanh hơn (bình phương), phần d^2 giảm tuyến tính. Công thức coi $\hat{n} = (1 - R)n$ trên *toàn bộ* n; thực tế chỉ token ảnh bị bỏ nên mức giảm thực hơi khác (xem 4).

Với n token, L_{attn} lớp lưu KV, n_{kv} head KV, kích thước head d_h và b byte mỗi phần tử, bộ nhớ KV là $2nL_{attn}n_{kv}d_hb$. Theo cơ chế của FastV, token bị loại không còn K/V trong các lớp sau điểm cắt; đây là suy luận từ phương pháp, còn bài báo chỉ báo cáo tổng bộ nhớ GPU mà không phân rã riêng KV cache.

3.4 So sánh với "huấn luyện với ít token ảnh hơn"

Cách thay thế: thêm average pooling (stride 2) sau CLIP khi huấn luyện LLaVA-1.5-7B để giảm 50% token ngay từ đầu. Paper thử cách này để so sánh công bằng (mục 5).

4. Thí nghiệm và kết quả chính

Mô hình: LLaVA-1.5-7B/13B, QwenVL-Chat-7B, InstructBLIP-Vicuna-13B (ảnh); Video-LLaVA (video). Giải mã tham lam (greedy). **Tác vụ:** captioning (Nocaps, Flickr30k; CIDEr), VQA (A-OKVQA multiple choice; OCR-VQA, Rouge-L), suy luận đa phương thức (MMMU; PCA-Bench), video QA (TGIF, MSVD, MSRVTT; dùng 1K mẫu đầu mỗi bộ, chấm bằng GPT), và các benchmark chi tiết (MME, SeedBench, MM-Vet, SciQA-IMG, AI2Diagram).

Bảng 1 (trích; "Ratio" = FLOPs so với baseline):

Model	K	R	FLOPs (B)	Ratio	Nocaps	Flickr30k	A-OKVQA	MMMU	Avg
LLaVA-1.5-7B	Baseline		99.3	100%	99.8	67.9	76.7	34.8	69.8
	2	50%	54.6	55%	99.7	67.5	77.0	34.4	69.7
	2	75%	32.8	33%	94.6	63.6	75.5	34.8	67.1
	2	90%	19.9	20%	72.1	43.7	70.1	35.0	55.2
	5	50%	59.6	60%	99.2	67.9	76.8	34.3	69.6

Model	K	R	FLOPs (B)	Ratio	Nocaps	Flickr30k	A-OKVQA	MMMU	Avg
	0	50%	51.6	52%	100.9	65.5	75.3	34.3	69.0
	0	75%	28.8	29%	27.2	61.4	72.8	35.1	49.1
	0	90%	18.9	19%	7.0	53.2	66.8	34.7	40.4
LLaVA-1.5-13B	Baseline		154.6	100%	102.8	73.0	82.0	36.4	73.6
	2	50%	84.6	55%	103.1	73.4	81.0	36.7	73.6
	2	75%	50.2	32%	100.5	72.5	80.9	38.1	73.0
	2	90%	29.7	19%	87.9	62.0	75.0	36.3	65.3
QwenVL-Chat-7B	Baseline		71.9	100%	94.9	72.5	75.6	35.8	69.7
	2	50%	39.5	55%	94.4	71.4	75.3	35.6	69.2
	2	75%	24.4	34%	90.5	67.0	75.1	35.3	67.0

Kết luận của bài báo: **K = 2, R = 50% giảm khoảng 45% FLOPs** mà gần như không mất hiệu năng cho các mô hình trên; đánh đổi hiệu năng-FLOPs "điều chỉnh được" bằng cách giảm K, tăng R. (Con số 55% của paper cao hơn một chút so với ~52% theo công thức Eq. 5 cho 7B/n≈700 (tự tính); chưa tái lập chính xác cách họ tính FLOPs thực.)

Độ trễ thực (Bảng 4; GPU A40; tác vụ A-OKVQA, chỉ xuất 1 chữ cái đáp án để loại ảnh hưởng độ dài đầu ra):

Model	Tổng thời gian	GPU memory	Score	Latency/mẫu
LLaVA-1.5-7B	6:34	19G	76,7	0,344 s
+ FastV (K=0, R=50%)	4:23	16G	75,3	0,230 s
LLaVA-1.5-13B	10:17	38G	82,0	0,539 s
+ FastV (K=0, R=50%)	6:30	30G	80,5	0,341 s

Tức 13B + FastV (0,341 s) có độ trễ tương đương 7B gốc (0,344 s) nhưng điểm cao hơn (80,5 so với 76,7), đúng như tuyên bố của tác giả. (Lưu ý: bảng này dùng K = 0, tức cắt **ngẫu nhiên** trước LLM.) Giảm tốc độ ≈ 33-37% latency (tự tính: 0,230/0,344 = 0,67; 0,341/0,539 = 0,63).

Chi tiết các tác vụ khó:

- *MME (LLaVA-1.5-13B):* tổng 1827,75 -> 1849,68 với FastV (K=2, R=50%); OCR 125,0 -> 132,5; Code 47,5 -> 57,5.
- *InstructBLIP-13B:* suy giảm nhiều hơn LLaVA với cùng cấu hình (ví dụ SeedBench 52,11 -> 50,41); đặt K = 5 thì gần như hết (51,69). Tác giả cho rằng Q-Former đã giảm token nên mất thông tin trực tiếp hơn.
- *PCA-Bench / OCR-VQA (LLaVA-1.5-13B, Bảng 5):* FastV (K=2, R=50%, FLOPs 84,6B): G-PCA mở 0,340 (gốc 0,333), đóng 0,368 (gốc 0,353); OCR-VQA Rouge-L 0,55 (gốc 0,55). K=2, R=75% (50,2B): 0,320 / 0,323, Rouge-L 0,54. K=0, R=50% (78,9B): Rouge-L 0,49.
- *Video-QA (Video-LLaVA, Bảng 6, GPT-eval):* gốc Acc/Score trung bình 0,48/3,3; FastV (K=2, R=50%, FLOPs 52,3%) 0,49/3,3; (K=5, R=50%, 57,1%) 0,49/3,4. Tác giả nhấn mạnh video dư thừa hơn (2.048 token), nên FastV còn cải thiện nhẹ TGIF (Acc 0,18 -> 0,21).

5. Phân tích / ablation

Bảng 7 (LLaVA-1.5-7B huấn luyện lại làm baseline, Nocaps / Flickr30k / A-OKVQA / MMMU):

Dòng	Cấu hình	Nocaps	Flickr30k	A-OKVQA	MMMU
	Baseline (retrained)	100,3	70,2	78,5	34,5
(a)	Huấn luyện với 50% token ảnh (pooling)	98,5	68,5	76,8	33,5
(b)	FastV K=2, R=50%	100,1	70,0	78,4	34,6
(c)	FastV K=2, R=50%, **ngẫu nhiên**	99,5	68,3	78,2	34,2
(d)	FastV bỏ token **system prompt**	89,2	64,3	69,2	33,8
(e)	bỏ **nửa đầu** system prompt	17,5	27,8	Failed	Failed
(f)	bỏ token **instruction**	77,3	50,1	56,5	29,5
(g)	áp mẫu sparse attention kiểu StreamingLLM	13,2	21,4	Failed	Failed

Rút ra:

- (a) vs (b): giảm token bằng pooling lúc huấn luyện **kém hơn** cắt lúc suy luận bằng FastV (cùng 50%), ít nhất trong cấu hình này.
- (b) vs (c): xếp hạng theo attention tốt hơn ngẫu nhiên, **nhưng chênh nhỏ ở R = 50%, K = 2** (Flickr30k 70,0 vs 68,3; Nocaps 100,1 vs 99,5). Nhận xét: ở mức cắt vừa phải, việc "cắt sau lớp 2" quan trọng hơn tiêu chí xếp hạng. Sự quan trọng của tiêu chí chỉ rõ hơn ở R cao.
- (d), (e), (f): cắt token **không phải ảnh** gây tụt mạnh; nửa đầu system prompt (các token neo) là then chốt, đúng với phát hiện về attention sink của StreamingLLM (55). Nhưng (g) cho thấy **áp thẳng pattern StreamingLLM** (chỉ giữ neo + cửa sổ gần) vào LVLM làm sập hoàn toàn, tức token ảnh khác về bản chất với token văn bản.
- **Ảnh hưởng của K và R (hình 7, LLaVA-1.5-13B trên OCR-VQA, không có số trong phần chữ):** K nhỏ thì giảm R giúp tăng điểm rõ; K lớn thì đổi R ít ảnh hưởng, chứng tỏ token ảnh dư thừa ở lớp sâu. Cũng thấy trong Bảng 1: cùng R = 75% mà K = 0 (random) cho Nocaps 27,2, còn K = 2 cho 94,6 (7B) tức **lớp 0-1 cần đủ token ảnh**.
- Trong phần giới thiệu, paper còn nêu thí nghiệm huấn luyện với các độ phân giải đặc trưng khác nhau bằng pooling để chứng minh FastV cho phép dùng ảnh phân giải cao hơn với cùng chi phí; **ghi chú này không tìm thấy bảng số tương ứng trong văn bản trích xuất**, nên không báo cáo.

6. Giới hạn

Do tác giả nêu/ngầm nêu:

- InstructBLIP (dùng Q-Former đã nén sẵn) suy giảm nhiều hơn với cùng K, R; phải tăng K.
- R cao (≥ 90%) làm tụt rõ; cần chọn K, R theo tác vụ.

Nhận xét:

- **Chỉ giảm FLOPs lý thuyết** là chính; số đo độ trễ chỉ có một cấu hình (A40, K = 0 ngẫu nhiên, đáp án 1 chữ cái). Với sinh văn bản dài thì giải mã thống trị (chi phí mỗi token gần như không phụ thuộc số visual token, trừ phần KV), nên lợi ích suy luận thực sự có thể nhỏ hơn (xem 8.2).
- Xếp hạng bằng attention trung bình **cần xác suất attention tường minh ở lớp K**, khó chạy với FlashAttention/kernel fused (cần tính riêng ở một lớp); paper không bàn đến.
- Đánh giá chủ yếu là caption/VQA ngắn, đáp án ngắn; tác vụ cần đọc chữ dày (OCR toàn trang) chỉ được thử gián tiếp qua OCR-VQA (câu trả lời ngắn).

- Thống kê attention lấy trung bình trên các bước giải mã và nhiều ảnh; nó **ẩn** việc ở từng bước token nào được cần. Giả thuyết "thông tin đã gom vào token neo" chưa được kiểm chứng (ví dụ bằng can thiệp nhân quả).
- Chỉ kiểm tra với LLM attention chuẩn (Vicuna/Qwen) và LLaVA 576 token; chưa kiểm với ảnh độ phân giải cao hàng nghìn token hay kiến trúc lai/linear attention.
- Cắt token hoàn toàn trên cả tập được chọn lúc prefill: không thể "phục hồi" nếu bước giải mã sau cần vùng ảnh đã bị bỏ.
- Phần cắt không áp dụng cho vision encoder: chi phí ViT (trên toàn bộ patch) không đổi.

7. Liên hệ với các bài khác

- **55 StreamingLLM (attention sink):** FastV thấy token neo ở system prompt nhận ~85% attention, ablation (e) xác nhận chúng thiết yếu; nhưng (g) cho thấy cách của StreamingLLM không dùng trực tiếp được cho token ảnh.
- **56 H2O:** cũng dùng điểm attention tích luỹ để loại token, nhưng ở KV cache khi sinh; FastV loại token ngay trong prefill (và áp dụng riêng cho token ảnh).
- **35 LLaVA-1.5, 34 LLaVA:** mô hình thí nghiệm chính, 576 token ảnh.
- **33 BLIP-2 (Q-Former):** InstructBLIP dùng Q-Former nên đã mất token trước, FastV hại hơn.
- **37 Qwen2-VL, 36 InternVL 1.5:** token ảnh tăng theo độ phân giải; đây là kiểu mô hình mà FastV muốn giúp.
- **41 GOT:** Giảm số token ở phía encoder là hướng bổ sung cho việc cắt token trong LLM.
- **44/45 FlashAttention, 46 vLLM:** tối ưu "chính xác" không đổi kết quả; FastV là xấp xỉ có thể mất độ chính xác.
- **24 Gated DeltaNet / 18-23 linear attention:** FastV giả định có ma trận attention softmax để xếp hạng; không áp dụng trực tiếp (xem 8).

8. Tóm tắt công thức / khái niệm

Khái niệm	Công thức / ý nghĩa
Attention allocation	$\lambda^j_{type} = \sum_i \alpha^{i,j}_{type}$
Attention efficiency	$\epsilon^j_{img} = \sum_i \alpha^{i,j}_{img} / \lvert img \rvert$
Phát hiện	lớp sâu: sys gấp 472× và chiếm 85%; ảnh ~0,21% của sys
FastV	tại lớp K bỏ R% token ảnh có attention nhận trung bình thấp nhất
FLOPs/lớp	$4nd^2 + 2n^2d + 2ndm$
Giảm FLOPs	$1 - \frac{Kf(n)+(T-K)f(\hat{n})}{Tf(n)}$, $\hat{n} = (1-R)n$
Kết quả chính	K=2, R=50%: giảm ~45% FLOPs, hiệu năng gần như không đổi (13B)
Latency (A40, A-OKVQA)	13B: 0,539 s -> 0,341 s (K=0, R=50%)
Ví dụ tự tính	d=1024, T=24, n=2.200: K=2, R=75% $\Rightarrow$ −71% FLOPs prefill LLM

9. Kết luận ngắn

FastV cho thấy trong LVLM kiểu LLaVA, token ảnh gần như bị "bỏ qua" từ sau lớp 2, nên có thể cắt một nửa (hoặc hơn) mà không huấn luyện lại, giảm ~45% FLOPs mà giữ điểm.

Link gốc: https://arxiv.org/abs/2403.06764

Bài 58

Generating Long Sequences with Sparse Transformers (2019)

Tác giả: Rewon Child, Scott Gray, Alec Radford, Ilya Sutskever (OpenAI; bản text đã đọc không in dòng affiliation, suy ra từ domain openai.com trong các đường dẫn ở bài).
Venue / arXiv: arXiv:1904.10509 (v1, 23/04/2019); paper không nêu venue trong bản text đã đọc (không có dòng "Proceedings of..."); đây là một tech report của OpenAI.
Nguồn: https://arxiv.org/abs/1904.10509
Chủ đề: Attention thưa (sparse attention) theo mẫu cố định, chọn trước khi huấn luyện (strided và fixed factorized attention), hạ độ phức tạp từ $O(n^2)$ xuống $O(n\sqrt{n})$; kèm các kỹ thuật phụ trợ (recompute để tiết kiệm bộ nhớ, kernel CUDA, kiến trúc residual sâu hơn) để huấn luyện mô hình tự hồi quy trên ảnh, văn bản, âm thanh với chuỗi dài tới hàng triệu bước.
Vị trí trong lộ trình: Đọc trước: 01 (Attention Is All You Need), 00 (preface, $O(n^2)$, prefill/decode, KV cache). Đây là **bài mở đầu của Giai đoạn 8 (Sparse attention)**: toàn bộ các mẫu attention thưa khác trong giai đoạn này (59-74) đều là biến thể hoặc mở rộng của ý tưởng "chỉ tính một tập con cặp (query, key)" mà bài này đặt nền. Đọc tiếp: 59 (Reformer, thưa hoá bằng hash nội dung), 60 (BigBird, thêm thành phần ngẫu nhiên và lý thuyết), và xa hơn 61-74 (thưa hoá thích nghi theo nội dung ở quy mô LLM hiện đại).

> Tài liệu học tập bằng tiếng Việt, là ghi chú diễn giải, không phải bản dịch. Số liệu lấy từ paper (full text arXiv, tải ngày 07/11/2026); các phép tính đánh dấu "tự tính" là của người ghi chú.

1. Bối cảnh

Self-attention chuẩn (bài 01) tính điểm cho các cặp vị trí (i, j), nên thời gian và bộ nhớ tăng theo $O(n^2)$ với độ dài chuỗi n. Chi phí này gây khó khăn cho Transformer tự hồi quy trên chuỗi rất dài, chẳng hạn ảnh độ phân giải cao khi biểu diễn từng pixel, âm thanh thô lấy mẫu nhiều nghìn lần mỗi giây hoặc văn bản ở mức ký tự.

Các hướng trước đó. PixelCNN (CNN) cần nhiều lớp để mở rộng trường nhìn (receptive field); WaveNet dùng dilated convolution để mở rộng trường nhìn theo hàm log số lớp; Transformer-XL dùng "memory" trạng thái tái sử dụng giữa các đoạn; các công trình khác dùng attention nhiều "hop", chunking, hoặc biểu diễn kích thước cố định.

Khái niệm cần biết trước.

- *Receptive field* (trường nhìn): tập vị trí đầu vào có thể ảnh hưởng tới một đại diện ở lớp cao; attention đầy đủ có trường nhìn toàn chuỗi ngay lớp 1, CNN/attention cục bộ cần nhiều lớp mới mở rộng được.
- *Connectivity pattern* $S = \{S_1, ..., S_n\}$: với mỗi vị trí đầu ra i, S_i là tập chỉ số đầu vào mà i "nhìn thấy". Attention đầy đủ (causal) có $S_i = \{j : j \leq i\}$.
- *Bits per byte / bits per dim*: log-likelihood âm trung bình trên mỗi byte (văn bản, âm thanh) hoặc kênh màu (ảnh), càng thấp càng tốt; dùng xuyên suốt Bảng 1-4.
- *Gradient checkpointing (recomputation)*: không lưu toàn bộ activation cho backward, chỉ lưu vài điểm mốc, phần còn lại tính lại khi cần gradient; đổi thời gian lấy bộ nhớ.

2. Ý tưởng chính

Paper khảo sát một mô hình 128 lớp full-attention đã huấn luyện trên CIFAR-10 và quan sát attention pattern học được (Hình 2): nhiều lớp đầu học mẫu cục bộ (giống convolution); hai lớp (19, 20) tách attention thành attention theo hàng và theo cột (tự nhân tố hoá attention toàn cục); một số lớp có mẫu toàn cục phụ thuộc dữ

liệu; phần lớn lớp 64-128 có độ thưa cao, chỉ kích hoạt cho một số input cụ thể. Quan sát này gợi ý: có thể áp một **mẫu attention thưa cố định trước khi huấn luyện** mà không mất nhiều hiệu năng, miễn là mẫu đó vẫn cho phép thông tin lan truyền giữa hai vị trí bất kỳ trong một số bước p cố định.

Cụ thể, với $p = 2$ "đầu" (head) attention nhân tố hoá, mỗi đầu chỉ nhìn một tập con các vị trí kích thước $O(\sqrt{n})$ thay vì toàn bộ n, nhưng ghép hai đầu lại thì mọi cặp (i, j) vẫn liên kết được qua tối đa $p + 1 = 3$ "chặng", chi phí giảm xuống $O(n\sqrt{n})$. Hai cách chọn tập con là **strided** (một đầu nhìn cục bộ gần, đầu còn lại nhìn cách đều theo chu kỳ l, hợp dữ liệu có cấu trúc tuần hoàn 2D như ảnh) và **fixed** (một số vị trí cố định đóng vai trò "tóm tắt" rồi truyền cho tương lai, hợp văn bản). Kèm theo là một bộ thay đổi kiến trúc (khối residual pre-activation, khởi tạo theo độ sâu, embedding vị trí nhiều chiều), kỹ thuật recompute tiết kiệm bộ nhớ, và kernel CUDA chuyên dụng.

3. Phương pháp chi tiết: attention nhân tố hoá

3.1 Attention tổng quát hoá theo connectivity pattern

Một lớp self-attention ánh xạ ma trận embedding đầu vào X thành đầu ra, tham số hoá bởi pattern $S = \{S_1, ..., S_n\}$, trong đó S_i là tập chỉ số mà vị trí đầu ra i được "nhìn":

$$\text{Attend}(X, S) = (a(x_i, S_i))_{i\in\{1,...,n\}}, \qquad a(x_i, S_i) = \text{softmax}\Big(\frac{(W_q x_i)K_{S_i}^\top}{\sqrt{d}}\Big)V_{S_i},$$

$$K_{S_i} = (W_k x_j)_{j\in S_i}, \qquad V_{S_i} = (W_v x_j)_{j\in S_i}.$$

Attention đầy đủ tự hồi quy có $S_i = \{j : j \le i\}$. Attention nhân tố hoá (factorized) thay bằng p đầu riêng, đầu m dùng tập con $A_i^{(m)} \subset \{j : j \le i\}$ với $|A_i^{(m)}| \propto p\sqrt{n}$, và $S_i = A_i^{(m)}$ cho đầu đó.

Điều kiện liên thông (connectivity). Để không mất khả năng lan truyền tín hiệu từ vị trí bất kỳ tới vị trí bất kỳ trong số bước cố định, paper yêu cầu: với mọi $j \le i$, tồn tại đường đi $(j, a, b, c, ..., i)$ dài tối đa $p + 1$ sao cho $j \in A_a^{(1)}$, $a \in A_b^{(2)}$, $b \in A_c^{(3)}$, v.v, giống đi qua p cạnh trên một đồ thị thưa để tới bất kỳ đỉnh nào. Paper ghi chú: làm "mềm" điều kiện này (chỉ dùng lớp liên kết cục bộ thuần tuý) có thể là một inductive bias tốt cho một số miền, đây chính là hướng Longformer (bài 17) theo đuổi.

$$\boxed{|A_i^{(m)}| = O(\sqrt{n}) \;\Rightarrow\; \text{chi phí mỗi vị trí } O(p\sqrt{n}) \;\Rightarrow\; \text{tổng chi phí } O(n\sqrt{n})}$$

(tự tính, $p = 2$ cố định trong paper nên $O(p\sqrt{n}) = O(\sqrt{n})$).

3.2 Hai kiểu nhân tố hoá 2D cho $p = 2$

(a) Strided attention (Hình 3b trong paper). Một đầu nhìn l vị trí gần nhất (cục bộ), đầu còn lại nhìn cách đều theo chu kỳ l:

$$A_i^{(1)} = \{t, t + 1, ..., i\},\ t = \max(0, i - l); \qquad A_i^{(2)} = \{j : (i - j) \bmod l = 0\}.$$

Stride l được chọn gần $\sqrt{n}$. Phù hợp khi dữ liệu có cấu trúc 2D tuần hoàn khớp với stride, ví dụ ảnh (coi l là chiều rộng ảnh thì đầu 2 đi theo cột) hoặc một số loại nhạc. Với văn bản không có cấu trúc tuần hoàn, paper nhận thấy mạng có thể thất bại trong việc định tuyến thông tin đúng cách bằng strided pattern, vì toạ độ không gian của một token không nhất thiết tương quan với vị trí mà thông tin đó sẽ hữu ích trong tương lai.

(b) Fixed attention (Hình 3c). Một số vị trí cố định đóng vai trò "tóm tắt" và truyền cho tương lai:

$$A_i^{(1)} = \{j : \lfloor j/l \rfloor = \lfloor i/l \rfloor\} \quad \text{(nhìn trong cùng khối kích thước } l\text{)},$$

$$A_i^{(2)} = \{j : j \bmod l \in \{t, t+1, ..., l\}\},\ t = l - c \quad \text{(nhìn } c \text{ vị trí cuối của mỗi khối)}.$$

Ví dụ cụ thể trong paper: stride $l = 128$, $c = 8$: mọi vị trí > 128 được nhìn vị trí 120-128; mọi vị trí > 256 nhìn 248-256; v.v. Với $c = 1$, biểu diễn bị bó hẹp quá mức (chỉ một vị trí "tóm tắt" cho cả khối). Paper thấy $c \in \{8, 16, 32\}$ với $l \in \{128, 256\}$ hoạt động tốt, tốn thêm chi phí theo hệ số c so với strided; nhiều head cùng kiểu fixed nên nhìn các tiểu khối (subblock) khác nhau trong l, không nên trùng nhau.

3.3 Ba cách ghép các đầu nhân tố hoá vào một lớp attention

$$\text{attention}(X) = W_p \cdot \text{Attend}(X, S), \qquad (5)$$

1. **Xen kẽ theo lớp** (interleave): mỗi khối residual thứ r dùng một đầu, luân phiên theo p:

$$\text{attention}(X) = W_p \cdot \text{Attend}(X, A^{(r \bmod p)}). \qquad (6)$$

2. **Merged head**: một đầu duy nhất nhìn hợp của các tập mà cả p đầu nhân tố hoá sẽ nhìn:

$$\text{attention}(X) = W_p \cdot \text{Attend}\Big(X, \bigcup_{m=1}^{p} A^{(m)}\Big). \qquad (7)$$

   Tốn thêm tính toán nhưng chỉ theo hệ số hằng số.
3. **Multi-head thật**: tính n_h sản phẩm attention song song (mỗi đầu dùng một A riêng) rồi ghép theo chiều đặc trưng, như multi-head attention gốc (bài 01); kích thước các ma trận trọng số trong mỗi đầu giảm theo $1/n_h$ để tổng số tham số không đổi theo n_h:

$$\text{attention}(X) = W_p \cdot \Big(\text{Attend}(X, A)^{(i)}\Big)_{i \in \{1, ..., n_h\}}. \qquad (8)$$

Paper dùng multi-head khi có thể; với chuỗi cực dài (attention chiếm phần lớn thời gian tính), tính các head tuần tự (không song song) có lợi hơn.

```
# pseudo-code (diễn giải): attention nhân tố hoá kiểu "fixed", merged head (công thức 7)
def factorized_attention(Q, K, V, l, c):
    # Q,K,V: [n, d]; mỗi vị trí i được attend theo hai tập chỉ số
    A1 = same_block(l)              # j: floor(j/l) == floor(i/l)        (head 1: trong
    ↪  cùng khối)
    A2 = last_c_of_each_block(l, c) # j: j mod l in {l-c..l}             (head 2: c vị
    ↪  trí cuối mỗi khối)
    S  = A1 union A2                # merged head
    scores = (Q @ K[S].T) / sqrt(d)
    P = softmax(scores)             # chỉ chuẩn hoá trên tập S, không phải toàn bộ n
    return P @ V[S]
```

4. Kiến trúc Sparse Transformer

4.1 Khối residual kiểu pre-activation và khởi tạo theo độ sâu

Để huấn luyện ổn định với hàng trăm lớp, paper dùng khối residual pre-activation (He et al. 2016):

$$H_0 = \text{embed}(X, W_e), \qquad H_k = H_{k-1} + \text{resblock}(H_{k-1}), \qquad y = \text{softmax}(\text{norm}(H_N)W_{out}),$$

$$\text{resblock}(H) = a(H) + b(H), \quad a(H) = \text{dropout}(\text{attention}(\text{norm}(H))), \quad b(H) = \text{dropout}(\text{ff}(\text{norm}(H + a(H)))).$$

ff$(x) = W_2 f(W_1 x + b_1) + b_2$ với f là GELU: $f(X) = X \cdot \text{sigmoid}(1.702 \cdot X)$ (xấp xỉ Hendrycks & Gimpel, cũng dùng trong GPT). Chiều ẩn W_1 mặc định $4d$ (hoặc $2d$ ở biến thể "half-size"). Vì H_N là tổng N khối residual nên mỗi khối nhận gradient trực tiếp từ output; để giữ tỉ lệ scale embedding/residual không đổi theo độ sâu N, paper **nhân khởi tạo W_2 và W_p (ma trận chiếu sau attention, công thức 5) với** $1/\sqrt{2N}$ (khởi tạo theo độ sâu, ý tưởng GPT-2 dùng lại sau).

Khởi tạo khác: $W_e \sim \mathcal{N}(0, 0.125/\sqrt{d})$; position embedding $\sim \mathcal{N}(0, 0.125/\sqrt{d \cdot n_{emb}})$; bias khởi tạo 0; trọng số còn lại $\sim \mathcal{N}(0, 0.125/\sqrt{d_{in}})$ với d_{in} là fan-in; trọng số output logits khởi tạo 0.

4.2 Embedding vị trí đa chiều cho dữ liệu khác loại

Ngoài embedding token, paper thêm n_{emb} embedding vị trí: ảnh dùng $n_{emb} = d_{data} = 3$ (hàng, cột, kênh màu); văn bản/âm thanh dùng $n_{emb} = d_{attn} = 2$ (hàng, cột trong ma trận rộng bằng stride):

$$\text{embed}(X, W_e) = \left(x_i W_e + \sum_{j=1}^{n_{emb}} o_i^{(j)} W^j \right)_{x_i \in X}. \qquad (15)$$

với x_i là one-hot token i, $o_i^{(j)}$ là one-hot vị trí theo chiều j. Paper nhấn mạnh: embedding mã hoá cấu trúc dữ liệu hoặc mã hoá chính mẫu attention là quan trọng cho hiệu năng (để mô hình biết "mình đang ở đâu" trong mẫu thưa).

4.3 Recompute, kernel block-sparse, mixed precision

Recompute. Gradient checkpointing đã biết giảm bộ nhớ khi huấn luyện mạng sâu; paper chỉ ra kỹ thuật này **đặc biệt lợi cho self-attention ở chuỗi dài** (bộ nhớ lưu activation cao hơn nhiều chi phí tính lại). Cụ thể, backward pass **tính lại cả khối attention và feedforward**; dropout chỉ áp dụng ở cuối mỗi residual, không áp dụng trong attention (khác bài 01). Chỉ nhờ recompute, paper huấn luyện được mạng dense hàng trăm lớp ở chuỗi 16,384, điều không khả thi trên phần cứng hiện có nếu không recompute.

Kernel block-sparse. Mẫu thưa 3(b)/3(c) tính bằng cách **cắt khối** từ Q, K, V rồi nhân theo khối: cục bộ tính trực tiếp; stride k tính bằng **chuyển vị ma trận rồi áp cửa sổ cục bộ**; vị trí fixed gộp lại tính theo khối. Kernel GPU riêng: softmax gộp (fuse) vào một kernel dùng register (nhanh như một phi tuyến đơn giản); **tam giác trên không bao giờ được tính**, bỏ việc cộng bias âm để mask, giảm một nửa số phép tính.

Mixed precision. Trọng số fp32, activation/gradient fp16 (tận dụng Tensor Core V100), dynamic loss scaling, gradient truyền giữa GPU ở fp16. **Khi sampling, query và key được ép về fp32** vì $QK^\top$ có thể vượt giá trị tối đa fp16.

5. Thí nghiệm và kết quả chính

CIFAR-10 (chuỗi 3,072 byte): strided, 2 head, 128 lớp, $d = 256$, FFN và Q/K "half-size", 120 epoch, lr 0.00035, dropout 0.25; 48,000 ảnh train, 2,000 validation. Đạt **2.80 bit/dim** (2.798 ± 0.004 qua 3 seed) so với SOTA trước 2.85 (PixelSNAIL).

Enwik8 (chuỗi 12,288 byte, 90M byte train, 10M validation/test): 30 lớp fixed, 8 head, $d = 512$, dropout 0.40, 80 epoch, stride $l = 128$, $c = 32$, merged head. Đạt **0.99 bit/byte** (0.992 ± 0.001 qua 3 seed), vượt Transformer-XL 88M (1.03) và khớp 277M (0.99) dù nhỏ hơn hơn một nửa.

ImageNet 64x64: 48 lớp strided, 16 head, $d = 512$, 152M tham số, stride 128, dropout 0.01, 70 epoch (7 ngày/64 V100). Đạt **3.44 bit/dim** so với SOTA trước 3.52.

Âm thanh (nhạc cổ điển, 12kHz): cùng kiến trúc 152M strided, đạt **1.97 bit/byte** cho đoạn 5 giây (65,536 bước); không có baseline so sánh trực tiếp (xử lý dữ liệu của công trình khác không công khai).

Bảng tổng hợp mật độ hoá (Bảng 1, bit/byte hoặc bit/dim, càng thấp càng tốt):

Miền dữ liệu	Mô hình	Kết quả
CIFAR-10	PixelCNN / PixelCNN++ / Image Transformer / PixelSNAIL	3.03 / 2.92 / 2.90 / 2.85
CIFAR-10	**Sparse Transformer 59M (strided)**	**2.80**
Enwik8	Deeper Self-Attention / Transformer-XL 88M / Transformer-XL 277M	1.06 / 1.03 / 0.99
Enwik8	**Sparse Transformer 95M (fixed)**	**0.99**
ImageNet 64x64	PixelCNN / Parallel Multiscale / Glow / SPN 150M	3.57 / 3.7 / 3.81 / 3.52
ImageNet 64x64	**Sparse Transformer 152M (strided)**	**3.44**
Nhạc cổ điển 12kHz, 5s	**Sparse Transformer 152M (strided)**	**1.97**

6. Phân tích / ablation

Tốc độ và chất lượng theo loại mẫu thưa (Bảng 2 của paper). Đơn vị cột "Time/Iter" không được paper ghi rõ (khả năng là giây/iteration trên cùng phần cứng); giữ nguyên để tham khảo tương đối.

Thiết lập	Bit/byte hoặc bit/dim	Time/Iter
Enwik8 (context 12,288), Dense Attention	1.00	1.31
Enwik8, Sparse Transformer (Fixed)	0.99	0.55
Enwik8, Sparse Transformer (Strided)	1.13	0.35
CIFAR-10 (context 3,072), Dense Attention	2.82	0.54
CIFAR-10, Sparse Transformer (Fixed)	2.85	0.47
CIFAR-10, Sparse Transformer (Strided)	2.80	0.38

Quan sát của paper: **mẫu thưa không chỉ nhanh hơn mà còn hội tụ loss thấp hơn** dense; strided thắng trên ảnh, fixed thắng trên văn bản (strided "thất bại định tuyến thông tin" trên Enwik8). Paper suy đoán đây có thể là inductive bias có ích từ mẫu thưa, hoặc một vấn đề tối ưu hoá chưa rõ của attention đầy đủ.

Độ dài ngữ cảnh tối thiểu lúc evaluate (Bảng 3, Enwik8, train với context 12,288). Paper thay đổi số token tối thiểu được dùng lúc eval và thấy cải thiện đơn điệu:

Context tối thiểu lúc eval	Bit/byte
6,144	0.9952
9,216	0.9936
10,752	0.9932
11,904	0.9930
12,096	0.9922
12,160	0.9908

Kết luận của paper: mạng khai thác phụ thuộc dài hạn hiệu quả, không bão hoà sớm trong phạm vi 12,288 token đã train.

Mở rộng tới chuỗi triệu bước (Bảng 4, âm thanh, giới hạn 16GB/GPU V100, không model-parallel). Paper thấy: tăng độ dài chuỗi 4 lần cần giảm dung lượng mô hình khoảng $4\sqrt{4} = 8$ lần để vẫn vừa bộ nhớ, hệ quả trực tiếp của $O(n\sqrt{n}) = O(n^{1.5})$.

Độ dài chuỗi	Số tham số	Bit/byte
65,536	152M	1.97
262,144	25M	2.17
1,048,576	3M	2.99

Nhờ vậy paper minh hoạ được việc áp self-attention cho chuỗi dài hơn 1 triệu bước là *khả thi về nguyên tắc*, dù với mô hình rất nhỏ (3M tham số) và chất lượng giảm rõ.

7. Giới hạn

Do tác giả nêu:

- Mẫu thưa **cố định, chọn trước huấn luyện**, không thích nghi theo nội dung, trong khi Hình 2 của chính bài báo cho thấy một số lớp học mẫu toàn cục phụ thuộc dữ liệu mà mẫu cố định không bắt được.
- Strided attention thất bại trên dữ liệu không tuần hoàn như văn bản; phải dùng fixed, tốn thêm hệ số c.
- Ở chuỗi hơn 1 triệu bước, paper chỉ huấn luyện được mô hình rất nhỏ (3M tham số) trên một GPU 16GB, chất lượng giảm mạnh.
- So sánh với baseline khác (PixelSNAIL, Transformer-XL...) không luôn cùng điều kiện huấn luyện.

Nhận xét:

- $O(n\sqrt{n})$ là cải thiện tiệm cận thật, nhưng ở chuỗi hàng nghìn token (nhỏ hơn nhiều so với ngữ cảnh rất dài), phần tiết kiệm so với $O(n^2)$ chưa chắc lớn hơn chi phí phép chiếu $QKVO$ và MLP.
- Paper chỉ train from scratch cho mỗi miền, không thử "cấy" mẫu thưa vào mô hình đã pretrain (cách Longformer, bài 17, làm sau).
- Không có global token tường minh (khác BigBird, bài 60); liên kết xa phải qua chuỗi 2 bước, nên tác vụ cần tổng hợp toàn chuỗi (`[CLS]`) chưa được kiểm chứng.

8. Liên hệ với các bài khác

- **01 Attention Is All You Need:** giữ nguyên công thức attention và softmax, chỉ thay *tập cặp được tính* S_i; là sparse attention đầu tiên trong lộ trình, mở Giai đoạn 8.
- **17 Longformer:** kế thừa "mẫu cố định + liên thông qua nhiều lớp", nhưng đơn giản hoá bằng sliding window + global token tường minh (dễ "cấy" vào checkpoint đã pretrain hơn) và kiểm chứng bằng fine-tune BERT-style; trích dẫn bài này là tiền đề trực tiếp.
- **55 StreamingLLM, 56 H2O:** cũng mẫu cố định (sink + window) hoặc chọn theo điểm attention, nhưng ở KV cache lúc *sinh* (decode), không phải huấn luyện từ đầu.
- **59 Reformer:** thay "mẫu cố định trước" bằng "cụm nội dung qua hashing", thích nghi theo dữ liệu, giải quyết đúng điểm yếu bài này tự nêu.
- **60 BigBird:** thêm random attention (đồ thị expander) và global token, cùng khung lý thuyết (universal approximator, Turing complete); trích dẫn bài này là một trong hai tiền đề trực tiếp.
- **44-45 FlashAttention:** giải bài toán khác (băng thông bộ nhớ của attention đầy đủ, không cắt cặp nào) bằng kernel fusion; cùng tinh thần "kernel CUDA cho attention".

8.1 Phân loại (taxonomy) sparse attention trong lộ trình (phân loại của người ghi chú, không phải từ paper này)

Bản đồ ngắn cho bài mở đầu Giai đoạn 8 (chi tiết ở 59-74). Trục chính: mẫu thưa được **cố định trước**, **chọn lúc suy luận không cần train**, hay **học cùng mô hình** (native).

Nhóm	Ý tưởng	Các bài
1. Mẫu cố định (chọn trước khi train)	Sliding window, stride, global token, random	06, 17, 55, **58 (bài này)**, 60
2. Phân cụm / hashing theo nội dung	Xấp xỉ nearest-neighbor trên key	59 (Reformer: LSH)
3. Chọn động lúc suy luận, không train	Prefill: mẫu theo head + chỉ số ước lượng online (62 MInference); decode: chọn trang KV theo chặn trên min/max của key (61 Quest)	61, 62; khảo sát so sánh: 66 (Sparse Frontier)
4. Gate học thêm trên mô hình có sẵn	Gate nhỏ dự đoán khối quan trọng, học bằng self-distillation từ attention đầy đủ	63 SeerAttention
5. Sparse "native" (học cùng mô hình, pretrain hoặc continued pretrain)	Chọn khối/token top-k bằng router hoặc indexer học được	64 NSA (ba nhánh nén/chọn/cửa sổ), 65 MoBA (router kiểu MoE theo khối), 68 DSA (lightning indexer, top-k token trên MLA), 69 GLM-5 (DSA qua continued pretrain), 72 MiniMax MSA (top-k khối riêng mỗi nhóm GQA)
6. Nén KV rồi chọn thưa	Gộp nhiều token thành một entry KV, sau đó indexer chọn thưa trên entry nén	71 DeepSeek-V4 (CSA + HCA), 74 DeepSeek-V4.1-Flash (nén theo entry, chuỗi và lớp)
7. Hybrid sparse + linear	Phần lớn lớp là linear attention/SSM, số ít lớp sparse attention giữ khả năng truy xuất	70 MiniCPM-SALA (25% InfLLM-V2 + 75% Lightning), 73 Qwen3.8-Flash-Next (GDN + QSA tỉ lệ 3:1)
8. Loại bỏ KV cache (eviction, training-free)	Giữ sink/gần nhất/điểm attention cao	55, 56 (H2O); tương tự: 57 (FastV, token thị giác)
(không thưa, nhưng đi cùng)	Cổng sigmoid sau SDPA, bỏ attention sink, ổn định huấn luyện	67 Gated Attention (Qwen)

9. Tóm tắt công thức / khái niệm

Khái niệm	Công thức / ý	Ghi chú
Connectivity pattern	$S_i \subset \{j : j \le i\}$, attention chỉ tính trên S_i	Full attention: $S_i = \{j : j \le i\}$
Điều kiện liên thông p bước	Mọi (j, i) nối qua đường dài $\le p+1$ qua các $A^{(1)}, ..., A^{(p)}$	Giữ khả năng lan truyền tín hiệu hằng số bước
Strided attention	$A_i^{(1)} = \{i-l, ..., i\}$, $A_i^{(2)} = \{j : (i-j) \bmod l = 0\}$, $l \approx \sqrt{n}$	Hợp dữ liệu có cấu trúc tuần hoàn 2D (ảnh, nhạc)
Fixed attention	$A_i^{(1)}$ = cùng khối l, $A_i^{(2)} = c$ vị trí cuối mỗi khối	Hợp văn bản; $c \in \{8, 16, 32\}$ hoạt động tốt
Ba cách ghép đầu	Xen kẽ theo lớp (6) / merged union (7) / multi-head thật (8)	Multi-head dùng khi không bị nghẽn bởi chi phí attention
Độ phức tạp	$O(n\sqrt{n})$	Tăng n gấp 4 cần giảm dung lượng mô hình $\approx 4\sqrt{4} = 8$ lần để giữ vừa bộ nhớ
Khởi tạo theo độ sâu	$W_2, W_p \sim \cdot/\sqrt{2N}$	Giữ tỉ lệ scale embedding/residual không đổi theo số lớp N
Recompute	Tính lại attention + feedforward ở backward	Đổi thời gian lấy bộ nhớ, đặc biệt lợi cho attention ở chuỗi dài

Khái niệm	Công thức / ý	Ghi chú
Kernel block-sparse	Cắt khối Q/K/V, tam giác trên không tính	Softmax fuse vào kernel, tiết kiệm nửa phép tính so với mask bằng cộng bias

10. Kết luận ngắn

Sparse Transformer đặt nền cho Giai đoạn 8: chứng minh bằng thực nghiệm rằng một **mẫu attention thưa cố định, chọn trước huấn luyện**, miễn giữ tính liên thông toàn chuỗi qua số bước hằng số, có thể đạt hoặc vượt attention đầy đủ về chất lượng với chi phí $O(n\sqrt{n})$ thay vì $O(n^2)$, đồng thời mở rộng tới chuỗi dài hơn 1 triệu bước nhờ recompute, kernel, khởi tạo sâu. Hạn chế cốt lõi, mẫu thưa tĩnh và không thích nghi theo nội dung, là động lực cho Reformer (bài 59, hashing theo nội dung) và BigBird (bài 60, thêm random attention và lý thuyết biểu diễn).

Link gốc: https://arxiv.org/abs/1904.10509

Bài 59

Reformer: The Efficient Transformer (2020)

Tác giả: Nikita Kitaev (UC Berkeley & Google Research), Łukasz Kaiser, Anselm Levskaya (Google Research).
Venue / arXiv: arXiv:2001.04451 (v2, 18/02/2020); ICLR 2020 (ghi ngay đầu bài: "Published as a conference paper at ICLR 2020").
Nguồn: https://arxiv.org/abs/2001.04451
Chủ đề: Hai kỹ thuật độc lập để Transformer rẻ hơn ở chuỗi dài: (1) **LSH attention** (locality-sensitive hashing), xấp xỉ attention đầy đủ bằng cách chỉ so khớp các key "gần" theo hash, hạ độ phức tạp từ $O(L^2)$ xuống gần $O(L \log L)$; (2) **reversible layer** (khôi phục activation từ layer sau, không cần lưu lại) và **chunked feed-forward**, loại bỏ hai nguồn tốn bộ nhớ còn lại (số lớp n_l và chiều ẩn FFN d_{ff}).
Vị trí trong lộ trình: Đọc trước: 01 (Attention), 00 (preface, $O(n^2)$, gradient checkpointing). Đọc sau 58 (Sparse Transformer, cùng là tiền đề được Reformer trích dẫn trực tiếp). Đây là bài thứ hai của Giai đoạn 8: thay "mẫu cố định trước" (58) bằng "thưa hoá thích nghi theo nội dung qua hashing xấp xỉ nearest-neighbor". Đọc tiếp 60 (BigBird, thêm random attention và lý thuyết biểu diễn), rồi 61-74 (các kỹ thuật thưa hoá hiện đại cho LLM).

> Tài liệu học tập bằng tiếng Việt, là ghi chú diễn giải, không phải bản dịch. Số liệu lấy từ paper (full text arXiv, tải ngày 07/11/2026); các phép tính đánh dấu "tự tính" là của người ghi chú.

1. Bối cảnh

Bài báo bắt đầu từ một câu hỏi về bộ nhớ: tại sao Transformer lớn khó tinh chỉnh trên một máy, dù tham số và activation của riêng một lớp có vẻ không quá lớn? Ví dụ được dẫn trong bài cho một lớp 0.5B tham số (Shazeer 2018) cần 2 GB; activation của batch 8, chuỗi 64K token và embedding 1024 cần thêm $64K \times 1K \times 8 = 0.5$B số thực, tức khoảng 2 GB. Ước tính này chưa tính ba nguồn bộ nhớ sau:

- Bộ nhớ của mô hình N lớp **lớn gấp N lần** mô hình 1 lớp, vì phải lưu activation của mọi lớp cho backward pass.
- Chiều ẩn feedforward d_{ff} thường lớn hơn hẳn d_{model} (ví dụ $d_{ff} = 4096$, $d_{model} = 1024$), chiếm phần lớn bộ nhớ activation.
- Attention trên chuỗi dài L là $O(L^2)$ cả về tính toán và bộ nhớ; với $L = 64K$, chỉ riêng ma trận $QK^\top$ ở batch 1 đã là $64K \times 64K$, tốn 16GB ở fp32.

Khái niệm cần biết trước.

- *Locality-sensitive hashing (LSH)*: một họ hàm hash h sao cho hai vector *gần nhau* có xác suất cao rơi vào cùng bucket hash, còn hai vector *xa nhau* thì không. Dùng để tìm gần-đúng-nhất-láng-giềng (approximate nearest neighbor) nhanh hơn so khớp từng cặp.
- *Memory-efficient attention*: tính attention cho từng query q_i riêng lẻ (không vật chất hoá cả ma trận $QK^\top$), rồi recompute ở backward; giảm bộ nhớ còn tuyến tính theo L, nhưng không giảm *thời gian tính* $O(L^2)$ (paper dùng kỹ thuật này làm baseline "full attention" khi so sánh).
- *Gradient checkpointing*: xem bài 58, mục 1; reversible layer (mục 4 dưới đây) là một cách đạt hiệu quả tương tự *mà không cần tính lại forward pass*.
- *Shared-QK*: dùng cùng một phép chiếu tuyến tính để tạo Query và Key (thay vì hai phép chiếu riêng như bài 01), tức $Q = K$ trước khi chuẩn hoá.

2. Ý tưởng chính

Attention đầy đủ tính $QK^\top$ cho mọi cặp, nhưng vì có softmax, phần lớn khối lượng xác suất chỉ tập trung ở vài key gần q_i nhất, phần còn lại gần như không góp vào output. Vậy với mỗi query, ta chỉ cần tìm **các key gần nó nhất** rồi attend trên tập đó. Reformer dùng **LSH** để tìm gần-đúng các key gần nhất: chiếu q_i, k_j qua các siêu phẳng ngẫu nhiên, hai vector gần nhau nhiều khả năng rơi cùng "bucket". Sắp key và query theo bucket rồi cắt thành các chunk liên tiếp, attention chỉ tính trong chunk đó và chunk ngay trước, giảm $O(L^2)$ xuống gần $O(L \log L)$ (log đến từ chi phí sắp xếp theo bucket).

Để Query và Key rơi vào đúng bucket như nhau (hai vector giống nhau phải hash giống nhau), Reformer dùng **shared-QK**: $Q = K$. Riêng kỹ thuật LSH không giải quyết hết vấn đề bộ nhớ (còn lại chi phí theo số lớp n_l và d_{ff}), nên paper thêm hai kỹ thuật độc lập: **reversible residual layer** (khôi phục activation lớp trước từ lớp sau bằng công thức nghịch, không cần lưu lại khi forward, loại bỏ hệ số n_l) và **chunking trong feedforward** (tính từng phần của d_{ff} tuần tự, loại bỏ hệ số d_{ff} khỏi bộ nhớ hoạt động). Ba kỹ thuật này **độc lập với nhau**: có thể dùng reversible + chunking mà không cần LSH (mô hình "Reversible Transformer" vẫn full attention), hoặc LSH mà không cần reversible.

3. Phương pháp chi tiết: LSH attention

3.1 Từ attention đầy đủ đến attention thưa theo tập P_i

Viết lại attention đầy đủ cho một vị trí i (bỏ qua hệ số chuẩn hoá $\sqrt{d_k}$ để gọn công thức, như paper làm):

$$o_i = \sum_{j \in P_i} \exp\left(q_i \cdot k_j - z(i, P_i)\right) v_j, \qquad P_i = \{j : i \geq j\}$$

với $z(i, P_i)$ là hệ số chuẩn hoá softmax (partition function). Để batch hoá, paper tính trên một tập lớn hơn $\tilde{P}_i \supseteq P_i$ rồi **mask** phần không hợp lệ bằng cộng $-\infty$ (hoặc 10^5):

$$o_i = \sum_{j \in \tilde{P}_i} \exp\left(q_i \cdot k_j - m(j, P_i) - z(i, P_i)\right) v_j, \qquad m(j, P_i) = \begin{cases} \infty & j \notin P_i \\ 0 & \text{khác} \end{cases}.$$

LSH attention chỉ là một cách chọn P_i khác: thay vì $P_i = \{j \leq i\}$ (toàn bộ quá khứ), đặt

$$\boxed{P_i = \{j : h(q_i) = h(k_j)\}}$$

tức i chỉ attend các vị trí mà key rơi vào **cùng bucket hash** với query của nó.

3.2 Hàm hash góc (angular LSH)

Để lấy b bucket, cố định ma trận ngẫu nhiên R kích thước $[d_k, b/2]$, định nghĩa:

$$h(x) = \arg\max\left([xR;\ -xR]\right)$$

($[u; v]$ là ghép hai vector). Đây là một lược đồ LSH đã biết (Andoni et al. 2015): chiếu x lên cả R và $-R$, bucket là chỉ số toạ độ lớn nhất trong vector ghép $2 \times (b/2) = b$ chiều. Hai vector gần nhau trên mặt cầu (theo góc) có xác suất cao rơi cùng bucket.

3.3 Batching: sắp bucket, chunk, và tập $\tilde{P}_i$

Hai vấn đề thực tế: (a) bucket có kích thước không đều, khó batch; (b) một bucket có thể có nhiều query nhưng không có key nào (hoặc ngược lại). Paper giải quyết bằng:

1. **Shared-QK, chuẩn hoá key**: đặt $k_j = q_j / \|q_j\|$, đảm bảo $h(k_j) = h(q_j)$ cho cùng vị trí j.
2. **Sắp theo (bucket, vị trí)**: sắp tất cả vị trí theo chỉ số bucket trước, theo vị trí gốc trong chuỗi sau, định nghĩa phép hoán vị $i \mapsto s_i$. Sau khi sắp, các cặp cùng bucket nằm gần đường chéo của ma trận attention đã sắp lại.
3. **Chunk kích thước** m: chia dãy đã sắp thành các chunk liên tiếp kích thước m; mỗi chunk attend chính nó và một chunk ngay trước:

$$\tilde{P}_i = \left\{ j : \left\lfloor \frac{s_i}{m} \right\rfloor - 1 \le \left\lfloor \frac{s_j}{m} \right\rfloor \le \left\lfloor \frac{s_i}{m} \right\rfloor \right\}, \qquad m = \frac{2l}{n_{buckets}}$$

(l là độ dài chuỗi). Nếu $\max_i |P_i| < m$ thì $P_i \subseteq \tilde{P}_i$ (đảm bảo không bỏ sót phần tử hợp lệ nào, với xác suất cao). Kích thước bucket trung bình là $l/n_{buckets}$; paper giả định xác suất một bucket phình gấp đôi kích thước trung bình là đủ thấp.

3.4 Multi-round LSH và masking

Vì hash chỉ là xấp xỉ, hai vector gần nhau đôi khi vẫn rơi khác bucket. Paper giảm xác suất lỗi bằng **nhiều round hash độc lập** $h^{(1)}, ..., h^{(n_{rounds})}$:

$$P_i = \bigcup_{r=1}^{n_{rounds}} P_i^{(r)}, \qquad P_i^{(r)} = \{ j : h^{(r)}(q_i) = h^{(r)}(k_j) \}.$$

Mỗi round tính $o_i^{(r)}$ độc lập (công thức (15) phụ lục A), rồi gộp lại có trừ đi hệ số đếm trùng $N_{i,j}$ (số round mà j xuất hiện trong $P_i^{(r)}$) để tránh tính j hai lần nếu nó rơi cùng bucket với i ở nhiều round; paper gộp hệ số này ngay vào số hạng mask $m_{i,j}$ cho tiện cài đặt.

Masking tự-attend. Causal attention chuẩn cho phép i attend chính nó; với shared-QK điều này có hại, vì $q_i \cdot k_i$ (tích với chính nó) hầu như luôn lớn hơn tích với vị trí khác, lấn át phần còn lại. Paper **cấm tự-attend** (đặt mask lớn nhưng hữu hạn, 10^5, không phải ∞) trừ khi vị trí đó không có đích hợp lệ nào khác (ví dụ token đầu chuỗi, chỉ có thể attend chính nó).

```
# pseudo-code (diễn giải): một round LSH attention, một head, chuỗi đã có Q=K (shared-QK)
def lsh_attention_round(Q, V, n_buckets, chunk_m):
    K = Q / norm(Q, axis=-1, keepdims=True)      # shared-QK, chuẩn hoá
    bucket = argmax(concat([Q @ R, -Q @ R], axis=-1))   # hash góc, R: [d_k, b/2]
    order  = sort_by((bucket, position))         # hoán vị s_i
    Qs, Ks, Vs = Q[order], K[order], V[order]
    for chunk in chunks(Qs, size=chunk_m):        # mỗi chunk tự attend + 1 chunk trước
        scores = chunk.Q @ concat([prev_chunk.K, chunk.K]).T
        scores = mask_self_attend(scores)          # trừ khi không còn đích khác
        P = softmax(scores)
        out[chunk] = P @ concat([prev_chunk.V, chunk.V])
    return unsort(out, order)
```

3.5 Độ phức tạp

Bảng 1 của paper (ký hiệu: l độ dài, b batch, n_h số head, n_c số chunk LSH, n_r số round hash):

Loại attention	Bộ nhớ	Thời gian
Scaled dot-product	$\max(bn_h ld_k,\ bn_h l^2)$	$\max(bn_h ld_k,\ bn_h l^2)$
Memory-efficient	$\max(bn_h ld_k,\ bn_h l^2)$	$\max(bn_h ld_k,\ bn_h l^2)$
LSH attention	$\max(bn_h ld_k,\ bn_h ln_r(4l/n_c)^2)$	$\max(bn_h ld_k,\ bn_h n_r l(4l/n_c)^2)$

Diễn giải của người ghi chú: với $n_c \propto l$ (paper đặt $n_c = l/32$ ở Bảng 3, nên $4l/n_c = 128$ không đổi theo l), số hạng $n_r(4l/n_c)^2$ trở thành $n_r \cdot 128^2$, một **hằng số không phụ thuộc** l; nhân với bn_hl ra chi phí **tuyến tính theo** l. Phần "log L" trong tên gọi $O(L \log L)$ ở abstract đến từ chi phí **sắp xếp** theo bucket (bước 3.3.2), không nằm trực tiếp trong bảng phức tạp attention này.

4. Phương pháp chi tiết: Reversible Transformer và chunking

4.1 RevNet: khôi phục activation từ lớp sau

Residual layer chuẩn $y = x + F(x)$ chỉ có một luồng vào/ra, nên phải lưu x lại để tính gradient của F. RevNet (Gomez et al. 2017) làm việc trên **cặp** $(x_1, x_2) \mapsto (y_1, y_2)$:

$$y_1 = x_1 + F(x_2), \qquad y_2 = x_2 + G(y_1)$$

và có thể **đảo ngược chính xác** mà không cần lưu gì thêm:

$$x_2 = y_2 - G(y_1), \qquad x_1 = y_1 - F(x_2).$$

Khi lan truyền ngược, người ta tính lại x_1, x_2 từ y_1, y_2 bằng công thức trên (không phải forward lại toàn mạng), rồi mới tính gradient. Áp vào Transformer: F là attention, G là feedforward, LayerNorm chuyển vào trong khối residual:

$$Y_1 = X_1 + \text{Attention}(X_2), \qquad Y_2 = X_2 + \text{FeedForward}(Y_1).$$

X_1, X_2 đều có kích thước d_{model} (cần cho phép cộng hợp lệ). Vì **không còn phải lưu activation của mỗi lớp**, hệ số n_l biến mất khỏi bộ nhớ hoạt động (vẫn còn trong bộ nhớ tham số, vì số lượng trọng số vẫn tăng theo n_l).

4.2 Chunking trong feedforward (và trong log-probability)

Phép biến đổi feedforward tính **độc lập theo từng vị trí trong chuỗi**, nên có thể chia theo chiều chuỗi thành c chunk và tính tuần tự, không mất tính đúng (chỉ là cách tổ chức tính toán khác, kết quả giống hệt):

$$Y_2 = \left[Y_2^{(1)}; ...; Y_2^{(c)}\right] = \left[X_2^{(1)} + \text{FF}(Y_1^{(1)});\ \ldots\ ; X_2^{(c)} + \text{FF}(Y_1^{(c)})\right].$$

Cách này loại d_{ff} khỏi bộ nhớ hoạt động đỉnh (chỉ cần giữ một chunk kích thước d_{ff} tại một thời điểm). Với vocab lớn hơn d_{model}, paper cũng chunk luôn phép tính log-probability ở output theo cùng cách. Phần đảo ngược (8) và backward cũng chunk tương tự.

Hoán đổi tham số CPU/GPU. Vì activation không còn phụ thuộc số lớp, bộ nhớ dùng cho activation không đổi theo n_l, nhưng *số tham số* vẫn tăng theo n_l. Paper tận dụng: vì batch × độ dài của Reformer lớn hơn hẳn Transformer thường, lượng tính toán trên mỗi lớp đủ lớn để **khấu hao chi phí chuyển trọng số CPU ↔ GPU** (vốn chậm), nên có thể giữ phần lớn tham số ở CPU và chỉ nạp lên GPU khi cần tính lớp đó.

4.3 Tổng hợp chi phí (Bảng 3 của paper)

d_{model}, d_{ff}: chiều model/feedforward; b: batch; l: độ dài; n_l: số lớp. Giữ nguyên ký hiệu như paper (có một điểm không nhất quán nhỏ giữa bld_{ff} và bn_hld_{ff} ở cột thời gian của vài dòng, paper không giải thích, chép lại đúng nguyên văn):

Kiến trúc	Bộ nhớ	Thời gian
Transformer	$\max(bld_{ff}, bn_h l^2)\, n_l$	$(bld_{ff} + bn_h l^2)\, n_l$
Reversible Transformer	$\max(bld_{ff}, bn_h l^2)$	$(bn_h l d_{ff} + bn_h l^2)\, n_l$
Chunked Reversible Transformer	$\max(bld_{model}, bn_h l^2)$	$(bn_h l d_{ff} + bn_h l^2)\, n_l$
LSH Transformer	$\max(bld_{ff}, bn_h l n_r c)\, n_l$	$(bld_{ff} + bn_h n_r l c)\, n_l$
Reformer (= reversible + chunk + LSH)	$\max(bld_{model}, bn_h l n_r c)$	$(bld_{ff} + bn_h n_r l c)\, n_l$

với $n_c = l/32$ nên $4l/n_c = 128$, $c = 128^2$ (giả định của Bảng 3). So dòng "Transformer" và "Reformer": hệ số n_l biến mất khỏi bộ nhớ (nhờ reversible), và $bld_{ff} \to bld_{model}$ (nhờ chunking, vì $d_{model} < d_{ff}$ thường $\times 4$), còn $bn_h l^2 \to bn_h l n_r c$ (nhờ LSH).

5. Thí nghiệm và kết quả chính

Thiết lập chung: $d_{model} = 1024$, $d_{ff} = 4096$, $n_{heads} = 8$, batch tổng 8 chuỗi, optimizer Adafactor, 8 thiết bị (GPU hoặc TPU v3 core). Ablation dùng mô hình 3 lớp (để so sánh được với Transformer thường, vốn tốn nhiều bộ nhớ).

Tác vụ tổng hợp (duplicate sequence): chuỗi $0w0w$ với w dài 511 (tổng 1024 token), mô hình 1 lớp, $d_{model} = d_{ff} = 256$, 4 head, train 150K bước. Attention đầy đủ giải hoàn hảo (100% acc, loss 0); đây là tác vụ **cần tra cứu không cục bộ** nên mô hình chỉ dùng attention cục bộ (span hạn chế) không giải được.

Train Eval	Full Attention	LSH-8	LSH-4	LSH-2	LSH-1
Full Attention	100%	94.8%	92.5%	76.9%	52.5%
LSH-4	0.8%	100%	99.9%	99.4%	91.9%
LSH-2	0.8%	100%	99.9%	98.1%	86.8%
LSH-1	0.8%	99.9%	99.6%	94.8%	77.9%

Đọc bảng: mô hình train với full attention vẫn dùng được LSH lúc eval (dù giảm độ chính xác); mô hình train với LSH đạt gần hoàn hảo, và **tăng số hash lúc eval cải thiện độ chính xác** ngay cả khi train với ít hash hơn (ví dụ train LSH-1, eval LSH-8 vẫn 99.9%).

Enwik8-64K và ImageNet64: so sánh hiệu ứng shared-QK và reversible riêng biệt (Hình 3): shared-QK không làm giảm chất lượng so với attention thường (ở Enwik8 còn train nhanh hơn một chút); reversible có đường cong loss gần như trùng với Transformer thường (cùng số tham số), xác nhận **tiết kiệm bộ nhớ không đổi lấy chất lượng**.

Dịch máy WMT14 En-De (Bảng 4, BLEU):

Mô hình	BLEU	sacreBLEU uncased	sacreBLEU cased
Vaswani et al. 2017, base	27.3		
Vaswani et al. 2017, big	28.4		
Ott et al. 2018, big	29.3		
Reversible Transformer (base, 100K bước)	27.6	27.4	26.9
Reversible Transformer (base, 500K bước, không chia sẻ trọng số)	28.0	27.9	27.4

Mô hình	BLEU	sacreBLEU uncased	sacreBLEU cased
Reversible Transformer (big, 300K bước, không chia sẻ trọng số)	29.1	28.9	28.4

Đáng chú ý: thí nghiệm này **không dùng LSH** (paper giải thích: câu trong WMT14 ngắn hơn 128 token, trong khi cấu hình LSH điển hình dùng chunk 128 token sau khi hash và sắp xếp, nên không có gì để hash/chunk một cách có lợi).

Mô hình Reformer lớn (tới 20 lớp) trên Enwik8/ImageNet64: nhờ reversible + chunking, paper train được tới 20 lớp trên một core, điều baseline Transformer thường "quá chậm và tốn bộ nhớ" để làm (không train được baseline ở cấu hình này). Enwik8, 12 lớp, 20K bước, dropout 0.1: **1.19 bit/dim** trên test; huấn luyện lâu hơn với tinh chỉnh thêm: **1.05 bit/dim**.

6. Phân tích / ablation

Hiệu ứng shared-QK (độc lập). Không làm giảm chất lượng so với attention thường trên cả Enwik8 và ImageNet64 (Hình 3 trái); trên Enwik8 train nhanh hơn một chút.

Hiệu ứng reversible (độc lập). Hai mô hình cùng số tham số (reversible và thường) có đường loss gần trùng nhau trên cả hai tập (Hình 3 phải): tiết kiệm bộ nhớ không đổi lấy chất lượng, đúng cho cả encoder-decoder (Bảng 4).

Số round hash n_{rounds} (LSH, Hình 4, ImageNet64). Độ chính xác xấp xỉ attention đầy đủ tăng đơn điệu theo n_{rounds}; ở $n_{rounds} = 8$ "đã gần khớp full attention" (theo lời paper). Chi phí tính toán tăng tuyến tính theo n_{rounds}, nên đây là một tham số đánh đổi tốc độ/chất lượng tường minh, có thể chọn khác nhau giữa train và eval (xem Bảng 2).

Tốc độ theo độ dài chuỗi (Hình 5 phải). Ở tổng số token cố định (tăng batch khi giảm độ dài), attention đầy đủ chậm dần khi độ dài tăng; **tốc độ LSH attention giữ bằng phẳng**, đúng với kỳ vọng từ độ phức tạp gần tuyến tính.

7. Giới hạn

Do tác giả nêu:

- LSH chỉ là *xấp xỉ*: luôn có xác suất khác không hai vector gần nhau rơi vào bucket khác nhau; phải tăng n_{rounds} để giảm xác suất lỗi, đổi lấy chi phí tính toán.
- WMT14 không áp dụng LSH được vì câu quá ngắn so với chunk size 128 mặc định; LSH chỉ có lợi ở chuỗi đủ dài.
- Bucket có thể lệch kích thước (dù trung bình $l/n_{buckets}$); phần xử lý cho trường hợp lệch (chunk liền trước) chỉ là một giải pháp "đủ tốt với xác suất cao", không có đảm bảo cứng.
- Hoán đổi tham số CPU/GPU chỉ có lợi khi lượng tính toán mỗi lớp đủ lớn để khấu hao chi phí truyền dữ liệu; paper không định lượng ngưỡng cụ thể.

Nhận xét:

- Tất cả thí nghiệm LSH đều ở chuỗi rất dài (≥ 12K cho ImageNet64, 64K cho Enwik8); paper không có thí nghiệm nào ở độ dài vài nghìn token để biết LSH có lợi ở quy mô đó không (xem mục 9, tính tay cho thấy thường là không).
- Reversible layer đổi được cách tính gradient một chút (vì cần đảo ngược chính xác), nên không "cấy" được tự do vào một mô hình đã train bằng residual thường mà giữ đúng hành vi; cần train lại hoặc fine-tune cẩn trọng.

- Shared-QK là một ràng buộc kiến trúc (bớt một phép chiếu), chưa được kiểm chứng ở mô hình decoder-only quy mô lớn hiện đại (paper chỉ thử ở quy mô nhỏ, 1-20 lớp).

8. Liên hệ với các bài khác

- **01 Attention Is All You Need:** vẫn dùng đúng công thức attention; Reformer chỉ thay *tập P_i được tính* (giống bài 58) và đổi cách lưu activation (reversible), không đổi phép toán attention cốt lõi.
- **58 Sparse Transformer:** paper trích dẫn trực tiếp là tiền đề ($O(n\sqrt{n})$ bằng mẫu cố định); Reformer thay "mẫu cố định trước" bằng "cụm nội dung qua hashing", thích nghi theo dữ liệu hơn nhưng không có đảm bảo liên thông toàn chuỗi như điều kiện p-bước của bài 58.
- **60 BigBird:** cũng trích dẫn Reformer là tiền đề; BigBird dùng random attention (lấy cảm hứng đồ thị expander) thay cho hashing, và thêm khung lý thuyết (universal approximator, Turing complete).
- **44-45 FlashAttention:** giải quyết băng thông bộ nhớ cho attention *đầy đủ, không xấp xỉ*; khác về triết lý (tối ưu kernel, không giảm số cặp tính) nhưng cùng mục tiêu "attention rẻ hơn ở chuỗi dài".
- **46 PagedAttention:** quản lý KV cache lúc *sinh* (decode); reversible/chunking của Reformer là kỹ thuật *huấn luyện*, không áp dụng trực tiếp lúc decode một token.
- **08 LoRA:** hoán đổi trọng số CPU/GPU trong Reformer khác hẳn fine-tune hạng thấp của LoRA, nhưng cùng động lực "huấn luyện mô hình lớn trên phần cứng hạn chế".

9. Tóm tắt công thức / khái niệm

Khái niệm	Công thức / ý	Ghi chú
Attention đầy đủ, dạng per-query	$o_i = \sum_{j \le i} \exp(q_i \cdot k_j - z) v_j$	Nền cho cả LSH và memory-efficient attention
Hash góc (angular LSH)	$h(x) = \arg\max([xR; -xR])$, $R \in \mathbb{R}^{d_k \times b/2}$	Vector gần nhau có xác suất cao cùng bucket
LSH attention	$P_i = \{j : h(q_i) = h(k_j)\}$	Thay cho $P_i = \{j \le i\}$ của attention đầy đủ
Shared-QK	$k_j = q_j / \|q_j\|$	Bắt buộc để $h(q_i) = h(k_i)$ cùng vị trí
Chunk sau sắp bucket	$m = 2l/n_{buckets}$, attend chunk hiện tại + 1 chunk trước	Cho phép batch hoá, không cần bucket đều
Multi-round LSH	$P_i = \bigcup_r P_i^{(r)}$	Giảm xác suất bỏ sót cặp gần nhau
Độ phức tạp LSH	$\max(bn_h l d_k,\ bn_h l n_r (4l/n_c)^2)$	Tuyến tính theo l nếu $n_c \propto l$
Reversible layer	$Y_1 = X_1 + F(X_2)$, $Y_2 = X_2 + G(Y_1)$; đảo: $X_2 = Y_2 - G(Y_1)$, $X_1 = Y_1 - F(X_2)$	Loại hệ số n_l khỏi bộ nhớ activation
Chunked feedforward	Tính FF theo từng đoạn chuỗi độc lập	Loại hệ số d_{ff} khỏi bộ nhớ activation đỉnh
Reformer	Reversible + chunking + LSH	Bộ nhớ $\max(bld_{model}, bn_h l n_r c)$, không còn hệ số n_l

10. Kết luận ngắn

Reformer tách bạch hai vấn đề bộ nhớ của Transformer và giải riêng từng cái: **LSH attention** thay tập cặp được tính bằng xấp xỉ nearest-neighbor theo hash (thích nghi theo nội dung, khác hẳn mẫu cố định của Sparse Transformer, bài 58), còn **reversible layer + chunked feedforward** loại bỏ hai hệ số n_l và d_{ff} khỏi bộ nhớ hoạt động mà không cần recompute forward. Cả ba kỹ thuật được kiểm chứng ở chuỗi rất dài (12K-64K token).

Link gốc: https://arxiv.org/abs/2001.04451

Bài 60

Big Bird: Transformers for Longer Sequences (2020)

Tác giả: Manzil Zaheer, Guru Guruganesh, Avinava Dubey, Joshua Ainslie, Chris Alberti, Santiago Ontanon, Philip Pham, Anirudh Ravula, Qifan Wang, Li Yang, Amr Ahmed (Google Research).
Venue / arXiv: arXiv:2007.14062 (v2, 08/01/2021); NeurIPS 2020 (ghi ngay trang đầu: "34th Conference on Neural Information Processing Systems (NeurIPS 2020), Vancouver, Canada").
Nguồn: https://arxiv.org/abs/2007.14062
Chủ đề: Attention thưa kết hợp ba thành phần: **window** (cục bộ), **global** (một số token nhìn/được nhìn toàn chuỗi), **random** (mỗi query nhìn thêm vài key ngẫu nhiên, lấy cảm hứng từ đồ thị expander); chi phí $O(n)$. Đi kèm khung lý thuyết chứng minh mẫu thưa này vẫn là universal approximator và Turing complete như attention đầy đủ, cộng một lower bound cho thấy việc đánh đổi này không "miễn phí".
Vị trí trong lộ trình: Đọc trước: 01 (Attention), 17 (Longformer, tiền đề trực tiếp của phần window+global), 59 (Reformer, tiền đề trực tiếp cùng Longformer theo lời paper). Đây là bài thứ ba và cuối của nhóm "sparse attention cổ điển" trong Giai đoạn 8 (cùng 58, 59); đọc tiếp 61-74 để sang các kỹ thuật thưa hoá thích nghi theo nội dung ở quy mô LLM hiện đại (block selection theo query, trainable indexer).

> Tài liệu học tập bằng tiếng Việt, là ghi chú diễn giải, không phải bản dịch. Số liệu lấy từ paper (full text arXiv, tải ngày 07/11/2026); các phép tính đánh dấu "tự tính" là của người ghi chú.

1. Bối cảnh

Self-attention đầy đủ (bài 01) có chi phí tính toán và bộ nhớ $O(n^2)$ theo độ dài n. Theo bối cảnh của bài báo, giới hạn phần cứng và kích thước mô hình thường khiến BERT/RoBERTa xử lý khoảng 512 token, gây khó khăn cho hỏi đáp trên nhiều đoạn và phân loại tài liệu dài.

Hai hướng trước đó (paper tổng hợp). (i) Giữ giới hạn độ dài, dùng cơ chế khác (retrieval, chunking) chọn một tập ngữ cảnh nhỏ rồi mới chạy transformer trên tập đó (SpanBERT, ORQA, REALM, RAG); tốn công kỹ thuật, khó huấn luyện. (ii) Bỏ attention đầy đủ: mô hình tự hồi quy một chiều (Transformer-XL, các biến thể "adaptive span") tốt cho language modeling nhưng không có ngữ cảnh hai chiều; Sparse Transformer (bài 58) hạ xuống $O(n\sqrt{n})$; Reformer (bài 59) hạ xuống $O(n \log n)$ bằng LSH; Longformer (bài 17) dùng sliding window + global token. Paper nhận định các phương pháp này phần lớn là **heuristic**, thiếu đảm bảo lý thuyết, và kế thừa trực tiếp ý tưởng "global token" từ công trình ETC (Extended Transformer Construction, Ainslie et al. 2020, cùng nhóm tác giả).

Hai câu hỏi paper đặt ra: (a) có thể đạt lợi ích thực nghiệm của attention đầy đủ bằng ít tích vô hướng (inner product) hơn không? (b) mẫu thưa đó có giữ được tính biểu diễn (expressivity) và linh hoạt của mạng gốc không?

Khái niệm cần biết trước.

- *Universal approximator*: một lớp hàm có thể xấp xỉ bất kỳ hàm liên tục nào (trên một miền compact) với sai số nhỏ tuỳ ý, nếu đủ tham số/độ sâu. Yun et al. (2019) đã chứng minh Transformer đầy đủ có tính chất này.
- *Turing complete*: có thể mô phỏng bất kỳ máy Turing nào (với giả định độ chính xác số học không hạn chế). Pérez et al. (2019) chứng minh Transformer đầy đủ Turing complete.
- *Đồ thị expander*: đồ thị thưa (ít cạnh) nhưng có đường đi ngắn giữa mọi cặp đỉnh (đường kính nhỏ theo log số đỉnh) và "trộn" thông tin nhanh qua random walk; đồ thị Erdős-Renyi (mỗi cạnh được chọn độc lập với xác suất cố định) là một ví dụ.

- *Mô hình Watts-Strogatz (small-world)*: bắt đầu từ một lưới vòng đều (mỗi đỉnh nối w hàng xóm gần), rồi thay một phần nhỏ cạnh bằng cạnh ngẫu nhiên; cân bằng giữa "đường đi ngắn toàn cục" và "tính cục bộ" (clustering coefficient cao).

2. Ý tưởng chính

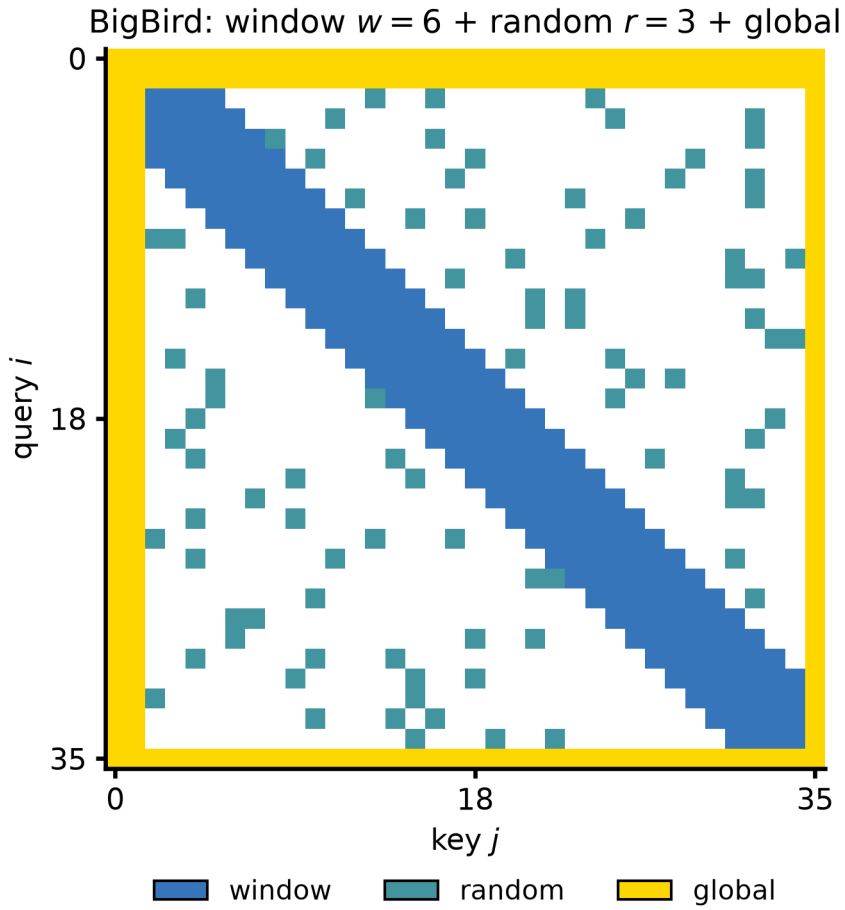
Paper nhìn self-attention như một đồ thị có hướng D trên n đỉnh: cạnh (i, j) tồn tại nếu query i attend key j; attention đầy đủ là đồ thị đầy (mọi cặp có cạnh). Giảm $O(n^2)$ xuống $O(n)$ tương đương với **thưa hoá đồ thị attention**. Hai tiêu chí cho đồ thị thưa "tốt" theo lý thuyết đồ thị: (a) đường đi ngắn giữa mọi cặp đỉnh (random graph kiểu Erdős-Renyi đạt được, với $\Theta(n)$ cạnh thì đường đi ngắn nhất là $O(\log n)$); (b) tính cục bộ cao (hầu hết thông tin liên quan nằm gần nhau, theo quan sát thực nghiệm về attention trong NLP). Mô hình Watts-Strogatz kết hợp cả hai: lưới vòng đều (cục bộ) cộng một ít cạnh ngẫu nhiên (đường đi ngắn).

BigBird hiện thực hoá trực tiếp ba thành phần tương ứng:

1. **Random attention**: mỗi query attend thêm r key ngẫu nhiên (vai trò đồ thị expander, đảm bảo thông tin lan truyền nhanh).
2. **Window (sliding) attention**: mỗi query attend w key lân cận (vai trò lưới vòng, bắt tính cục bộ; giống bài 17).
3. **Global attention**: g token đặc biệt attend toàn chuỗi và được toàn chuỗi attend (vai trò "hub" lý thuyết, chứng minh được ở mục 4).

Thực nghiệm sơ bộ của paper (Bảng 1, mục 3) cho thấy chỉ random+window chưa đạt chất lượng BERT; phải thêm global mới đủ. Paper sau đó chứng minh global token là *cần thiết về lý thuyết* (không chỉ thực nghiệm): một đồ thị thưa chứa "star graph" (toàn bộ đỉnh nối với một đỉnh trung tâm) vẫn là universal approximator và Turing complete, như attention đầy đủ. Đồng thời paper chỉ ra một bài toán cụ thể mà *bất kỳ* đồ thị thưa $\tilde{O}(n)$ cạnh đều cần nhiều lớp hơn hẳn attention đầy đủ, chứng minh việc thưa hoá không "miễn phí".

3. Phương pháp chi tiết: window, global, random



BigBird: window + random + global trên ma trận attention thưa. (minh hoạ, tự tính)

3.1 Attention tổng quát hoá theo đồ thị

Với đồ thị có hướng D trên $[n] = \{1, ..., n\}$, $N(i)$ là tập hàng xóm ra của đỉnh i:

$$\text{ATTN}_D(X)_i = x_i + \sum_{h=1}^{H} \sigma\Big(Q_h(x_i) K_h(X_{N(i)})^\top\Big) \cdot V_h(X_{N(i)}) \qquad \text{(AT)}$$

với $Q_h, K_h : \mathbb{R}^d \to \mathbb{R}^m$, $V_h : \mathbb{R}^d \to \mathbb{R}^d$, σ là hàm điểm số (softmax hoặc hardmax), H số head, $X_{N(i)}$ là ma trận chỉ gồm các dòng $\{x_j : j \in N(i)\}$. D đầy đủ cho lại attention gốc (bài 01). Viết theo ma trận kề $A \in \{0,1\}^{n \times n}$ ($A(i,j) = 1$ nếu i attend j), bài toán thưa hoá attention trở thành bài toán **thưa hoá đồ thị (graph sparsification)**.

3.2 Ba thành phần và hai cách đặt global token (ITC/ETC)

Random: $A(i, \cdot) = 1$ cho r chỉ số ngẫu nhiên (Hình 1a của paper).

Window: với độ rộng w, $A(i, i - w/2 : i + w/2) = 1$ (Hình 1b); lấy cảm hứng từ lưới vòng của Watts-Strogatz.

Global, hai cách hiện thực:

- **ITC (Internal Transformer Construction)**: chọn một tập con G ($|G| = g$) trong các token *đã có sẵn* làm global, không thêm tham số: $A(i,:) = 1$ và $A(:,i) = 1$ với mọi $i \in G$.
- **ETC (Extended Transformer Construction)**: *thêm* g token global mới (không phải token dữ liệu), tạo ma trận kề mở rộng $B \in \{0,1\}^{(N+g)\times(N+g)}$: $B(i,:) = 1$, $B(:,i) = 1$ với $i \in \{1,...,g\}$ (các token global mới), và $B(g+i, g+j) = A(i,j)$ cho phần còn lại (giữ nguyên cấu trúc cũ). ETC tốn thêm vị trí lưu ngữ cảnh (token global riêng, không "chiếm chỗ" của token dữ liệu) và cải thiện hiệu năng trong thực nghiệm (mục 6), nhưng cần thêm relative position encoding (xem Appendix E.2) và không dùng thành phần random ($r = 0$ cho ETC trong các bảng hyperparameter của paper).

BigBird: $A(i,\cdot) = 1$ cho r key ngẫu nhiên, $w/2$ bên trái và phải, và mọi $g \in G$ (ITC hoặc ETC)

3.3 Thí nghiệm sơ bộ: vì sao cần cả ba (Bảng 1 @512, dev, so với BERT-base)

Thiết lập	MLM	SQuAD	MNLI
BERT-base (full attention)	64.2	88.5	83.4
Random (R)	60.1	83.0	80.2
Window (W)	58.3	76.4	73.1
R + W	62.7	85.1	80.5

(paper không nêu cụ thể r, w dùng ở bảng sanity-check này, khác với cấu hình huấn luyện đầy đủ ở Bảng 8). Chỉ random hoặc chỉ window đều kém BERT-base rõ; cộng cả hai cải thiện nhưng vẫn chưa đạt BERT. Phần thiếu, theo paper, là **global token**, được thêm vào và biện minh bằng lý thuyết ở mục 4.

```
# pseudo-code (diễn giải): mặt nạ attention BigBird-ITC cho một dòng i (không phải
↪  global)
def bigbird_mask_row(i, n, w, r, global_idx, rng):
    allowed = set(range(max(0,i-w//2), min(n,i+w//2+1)))   # window
    allowed |= set(rng.sample(range(n), r))                  # random
    allowed |= set(global_idx)                                # global (được nhìn)
    return allowed   # nếu i in global_idx: allowed = set(range(n))  (global: nhìn tất
    ↪  cả)
```

4. Cơ sở lý thuyết

Paper chứng minh BigBird giữ được hai tính chất lý thuyết của attention đầy đủ, cộng một lower bound cho thấy cái giá phải trả.

Universal approximator (Định lý 1). Định nghĩa đồ thị hình sao (star graph) S trên $\{0,1,...,n\}$: $N(i) = \{0,i\}$ cho $i \geq 1$, $N(0) = \{1,...,n\}$ (đỉnh 0 là một token đặc biệt thêm vào, đóng vai trò global). *Định lý:* với $1 < p < \infty$, $\epsilon > 0$, với mọi hàm liên tục f (trên miền compact), tồn tại transformer attention thưa $g \in \mathcal{T}_D^{H,m,q}$ với D **là bất kỳ đồ thị chứa đồ thị sao** S, sao cho $d_p(f,g) \leq \epsilon$. Chứng minh theo 3 bước (phỏng theo khung của Yun et al. 2019): (1) xấp xỉ f bằng hàm hằng từng khúc (piecewise constant); (2) xây dựng "contextual mapping" bằng attention thưa, dùng một **sparse shift operator** (dịch các hàng của ma trận theo một khoảng do đồ thị thưa D quyết định) cộng với global token để bù lại thông tin bị mất do mỗi query chỉ nhìn ít key; (3) xấp xỉ transformer đã sửa (dùng hardmax) bằng transformer gốc (dùng softmax, ReLU). Định lý 2 chỉ là phiên bản cụ thể hoá của Định lý 1 với tham số $H = 2, m = 1, q = 4$. Vì đồ thị của BigBird (random+window+global) **chứa đồ thị sao** (nhờ thành phần global), nó thừa hưởng tính universal approximator.

Turing complete (Định lý 3, chứng minh ở Phụ lục B). Pérez et al. (2019) chứng minh transformer đầy đủ (encoder+decoder, giả định độ chính xác số học không hạn chế) mô phỏng được bất kỳ máy Turing. Paper mở

rộng kết quả này cho attention thưa: hầu hết cấu trúc chứng minh gốc vẫn dùng được, chỉ riêng lược đồ "địa chỉ hoá để quay lại một vị trí trong lịch sử" (addressing scheme, Lemma B.4 của Pérez et al.) cần thay bằng một cơ chế dùng attention thưa; paper chỉ ra cách làm điều này. Kết luận: tồn tại attention thưa với $O(n)$ tích vô hướng mà lớp Transformer dùng nó vẫn Turing complete.

Lower bound, "không có bữa ăn miễn phí" (Mệnh đề 1, mục 3.4). Xét bài toán: cho n vector đơn vị $u_1, ..., u_n$, với mỗi j tìm $j^* = \arg\max_k \|u_k - u_j\|_2^2$ (vector xa nhất). Với attention đầy đủ, một lớp đơn ($H = 1, m = 2d, q = 0$) giải được ngay (so mọi cặp tích vô hướng). Paper chứng minh: **với bất kỳ đồ thị thưa D có $\tilde{O}(n)$ cạnh** (không riêng BigBird), giải bài toán này cần $\tilde{\Omega}(n^{1-o(1)})$ lớp. Chứng minh bằng quy giảm (reduction) từ Orthogonal Vector Conjecture (OVC, giả định độ phức tạp tinh vi được dùng rộng rãi: không thể xác định tích vô hướng nhỏ nhất giữa n vector nhị phân có bằng 0 hay không trong thời gian dưới bậc hai); nếu một đồ thị thưa giải được bài toán vector-xa-nhất với ít lớp, sẽ giải được OVC nhanh hơn giả định cho phép.

Thưa hoá $\Rightarrow$ vẫn universal approximator + Turing complete, nhưng tồn tại bài toán cần $\tilde{\Omega}(n^{1-o(1)})$ lớp thay vì $O(1)$

5. Hiện thực: block-sparse attention

Vì sao không thể thưa hoá "thẳng" trên GPU/TPU. Phần cứng song song (GPU/TPU) tối ưu cho đọc liên tục theo khối (coalesced memory access); tra cứu rời rạc (sliding window hoặc random) theo từng phần tử là chậm. Giải pháp: **"blockify"**, chia Q và K thành khối kích thước b, ba thành phần được định nghĩa trên các *khối* thay vì từng phần tử.

- **Random**: mỗi khối query attend r khối key ngẫu nhiên (gather, tốn kém nhưng r nhỏ; paper nêu "$r = 3$ cho mọi thí nghiệm" ở phần hiện thực, nhưng Bảng 8 cho thấy điều này chỉ áp dụng cho cấu hình ITC, còn ETC đặt $r = 0$, xem mục 3.2 và 6).
- **Window**: số khối query và khối key bằng nhau; khối query j attend khối key $j - (w-1)/2$ đến $j + (w-1)/2$. Cách tính nhanh: tạo w bản sao của tensor khối-key đã reshape, **"cuộn" (roll)** mỗi bản sao đi một lượng khác nhau theo trục khối, rồi nhân khối-query với w bản sao đã cuộn (Hình 5 của paper); tương đương lấy cửa sổ mà không cần gather.
- **Global**: luôn gồm g khối key đầu (cố định, không cần roll hay gather).

Gọi Q', K' là Q, K đã reshape thành $\lceil n/b \rceil \times b \times d$. Khối đường chéo (phần cục bộ) tính bằng:

$$A_{jst} = \sum_u Q'_{jsu} K'_{jtu}, \qquad j = 0, ..., \lceil n/b \rceil - 1,$$

cho ra tensor $\lceil n/b \rceil \times b \times b$, chính là phần nhân khối-với-khối tương ứng. Ghép g khối global (cố định) + w bản key đã cuộn (window) + r khối key gather (random) thành một tensor **key đặc (dense)** K'' kích thước $\lceil n/b \rceil \times (g + w + r)b \times d$. Toàn bộ attention score lúc này chỉ là **một phép nhân tensor đặc** giữa Q' ($\lceil n/b \rceil \times b \times d$) và K'':

$$\boxed{\text{chi phí} = O\big(n\,(g + w + r)\,b\,d\big)}$$

thay cho $O(n^2 d)$ của attention đầy đủ, mà vẫn chỉ dùng phép nhân tensor đặc (nhanh trên GPU/TPU), gather chỉ cần cho phần random (nhỏ, $r = 3$ khối ở cấu hình ITC; ETC không dùng random, xem mục 3.2).

6. Thí nghiệm và kết quả chính

MLM pretraining (warm-start từ RoBERTa, Bảng 8 cho hyperparameter cơ bản, Bảng 9-10 cho kết quả BPC trên tập dev, càng thấp càng tốt).

Mô hình (sqln 4096 trừ RoBERTa)	Base	Large
RoBERTa (sqln 512)	1.846	1.496
Longformer	1.705	1.358
BigBird-ITC	1.678	1.456
BigBird-ETC	**1.611**	**1.274**

Hyperparameter chính (Bảng 8, base): block $b = 64$ (ITC) / $b = 84$ (ETC, large ETC $b = 169$); $g = 2b$ (ITC) hoặc 256 (ETC); $w = 3b$; $r = 3b$ (ITC) hoặc 0 (ETC); 12 lớp, 12 head, hidden 768, batch 256, Adam, lr 10^{-4}, 8×8 TPUv3.

Hỏi đáp (QA), 4 tập: Natural Questions, HotpotQA-distractor, TriviaQA-wiki, WikiHop (Bảng 2, dev, mô hình base):

Mô hình	HotpotQA Ans	HotpotQA Sup	HotpotQA Joint	NaturalQ LA	NaturalQ SA	TriviaQA Full	WikiHop MCQ
RoBERTa	73.5	83.4	63.5	-	-	74.3	72.4
Longformer	74.3	84.4	64.4	-	-	75.2	75.0
BigBird-ITC	75.7	86.8	67.7	70.8	53.3	79.5	75.9
BigBird-ETC	75.5	87.1	67.8	73.9	54.9	78.7	75.9

Trên tập test (Bảng 3, mô hình large, so với top-3 bảng xếp hạng không tính BigBird): BigBird-ETC đạt SOTA mới ở Natural Questions Long Answer, TriviaQA, WikiHop; đứng thứ ba (F1) trên HotpotQA. Paper ghi chú: BigBird nộp dưới dạng **một mô hình đơn**, trong khi top-3 khác của Natural Questions là ensemble.

Phân loại văn bản (Bảng 15, F1 micro-average, mô hình base, 4096 token):

Tập	IMDb	Yelp-5	Arxiv	Patents	Hyperpartisan
RoBERTa (512 token)	95.0	71.75	87.42	67.07	87.8
BigBird	**95.2**	**72.16**	**92.31**	**69.30**	**92.2**

Paper nhận định: lợi ích lớn nhất ở tập văn bản dài, ít dữ liệu train (Arxiv: +5 điểm so với SOTA trước; Hyperpartisan: +4.4 so với RoBERTa); với tập lớn (Patents) hoặc ngắn (IMDb), lợi ích nhỏ hoặc không thấy.

GLUE (Bảng 16, dev, base): BigBird cạnh tranh được dù các tác vụ GLUE phần lớn có ngữ cảnh ngắn (ví dụ MNLI 87.5/87.3 so với RoBERTa 87.6/-, SST-2 94.6 so với 94.8), xác nhận BigBird không "hỏng" ở ngữ cảnh ngắn.

Tóm tắt văn bản dài (Bảng 4, ROUGE-1/2/L, encoder sparse + decoder full attention):

Mô hình (baseline đều là Transformer-base/large thuần, khác nhau ở cách pretrain)	Arxiv R-1/R-2/R-L	PubMed R-1/R-2/R-L	BigPatent R-1/R-2/R-L
Transformer-base + Pegasus pretraining (dòng "+ Pegasus", không phải BigBird)	34.81/10.16/30.14	39.98/15.15/35.89	43.55/20.43/31.80
BigBird-RoBERTa (base, warm-start từ RoBERTa)	**41.22/16.43/36.96**	**43.70/19.32/39.99**	**55.69/37.27/45.56**
Pegasus-large (báo cáo gốc, Zhang et al. 2019)	44.21/16.95/38.83	45.97/20.15/41.34	52.29/33.08/41.75
BigBird-Pegasus (large, warm-start từ Pegasus)	**46.63/19.02/41.77**	**46.32/20.65/42.33**	**60.64/42.46/50.01**

(mục 4.1 của paper: chỉ dùng sparse attention ở **encoder**; decoder giữ full attention vì output tóm tắt thường ngắn hơn input nhiều, ví dụ median $\sim$200 token output so với $>$3000 token input. Bảng 4 gốc còn một dòng "Transformer" thuần và "+ RoBERTa" phía trên dòng "+ Pegasus"; ở đây chỉ trích dòng baseline gần nhất với BigBird-RoBERTa để so sánh công bằng theo cùng kích thước mô hình).

Genomics (DNA, token hoá bằng sentencepiece, mỗi token $\approx$8.78 base pair). MLM BPC (Bảng 5): SRILM 1.57; BERT (sqln 512) 1.23; **BigBird (sqln 4096) 1.12**. Dự đoán vùng promoter, F1 (Bảng 6): CNNProm 69.7; DeePromoter 95.6; **BigBird 99.9**. Dự đoán chromatin profile, AUC (Bảng 7, TF/HM/DHS): gkm-SVM 89.6 (chỉ TF); DeepSea 95.8/85.6/92.3; **BigBird 96.1/88.7/92.1** (cải thiện rõ nhất ở HM, loại được paper ghi là có tương quan tầm xa hơn).

7. Phân tích / ablation

Thành phần nào quan trọng (Bảng 1, đã trình bày ở mục 3.3): random hoặc window riêng lẻ đều kém BERT-base rõ rệt (giảm 4-6 điểm MLM, 5-12 điểm SQuAD/MNLI); cộng cả hai thu hẹp một phần khoảng cách nhưng global mới là thành phần quyết định để khớp/vượt BERT (paper không có một dòng "chỉ global" riêng để so sánh trực tiếp trong Bảng 1, nhưng toàn bộ kết quả ở mục 6 đều dùng window+global+random hoặc window+global).

ITC so với ETC: ETC tốt hơn ITC ở hầu hết tác vụ (MLM BPC, QA, mục 6), nhờ không "chiếm chỗ" của token dữ liệu để làm global và dùng relative position encoding; đổi lại cần huấn luyện token global mới từ đầu (paper dùng thêm contrastive predictive coding, CPC, làm regularizer) và không dùng thành phần random. Paper ghi chú ở mục E.2 (WikiHop): "kết nối attention thiết kế tường minh của ETC hoạt động tốt hơn một chút, nhưng ITC dựa trên kết nối ngẫu nhiên cũng khá cạnh tranh."

Độ dài ngữ cảnh ở genomics (Hình 8, Phụ lục F.1): paper chỉ báo cáo bằng đồ thị (không có bảng số cụ thể trong text đã trích): độ chính xác MLM tăng và học nhanh hơn khi tăng độ dài từ 512 lên 1024 lên 4096, nhất quán với nhận định "ngữ cảnh dài hơn giúp ích" đã thấy ở NLP.

8. Giới hạn

Do tác giả nêu:

- Lower bound (Mệnh đề 1): tồn tại bài toán mà *bất kỳ* attention thưa $\tilde{O}(n)$ cạnh cần $\tilde{\Omega}(n^{1-o(1)})$ lớp thay vì $O(1)$ của attention đầy đủ; thưa hoá không miễn phí.
- So sánh với Longformer "bị nhiễu" vì Longformer dùng ít hơn 16 lần compute tiền huấn luyện so với BigBird (paper tự ghi ở Phụ lục E.3).
- Global token tường minh (ETC) cần thêm relative position encoding và CPC loss để huấn luyện hiệu quả, tăng độ phức tạp kỹ thuật so với ITC.

- Encoder-decoder chỉ thưa hoá phía encoder; paper không thử thưa hoá decoder vì output thường ngắn (không cần).

Nhận xét:

- Thành phần random ($r = 3$ khối ở cấu hình ITC) là một siêu tham số nhỏ, cố định trong mọi thí nghiệm ITC (ETC không dùng, $r = 0$); paper không có ablation riêng cho $r \in \{1, 2, 4, ...\}$ để biết độ nhạy, và cũng không giải thích vì sao ETC không cần random trong khi ITC cần.
- Tất cả lợi ích thực nghiệm đo ở ngữ cảnh 4096 token; không có thí nghiệm ở vài nghìn token để biết ngưỡng ngữ cảnh mà lợi ích bắt đầu rõ.
- Khối (block) $b = 64$ hoặc 84 là đơn vị nhỏ nhất của mọi thao tác; ở chuỗi ngắn, số khối (n/b) nhỏ nên overhead tổ chức (reshape, roll, gather) so với tính toán thực tế chưa chắc có lợi, paper không đo riêng ở n nhỏ.

9. Liên hệ với các bài khác

- **01 Attention Is All You Need:** BigBird giữ công thức attention (AT) gần giống, chỉ thưa hoá đồ thị D; đồ thị đầy cho lại chính bài 01.
- **17 Longformer:** tiền đề trực tiếp cho window+global; paper tự so sánh ở Phụ lục E.3: BigBird-ETC khác Longformer ở (i) dùng relative position encoding, (ii) huấn luyện global token bằng CPC loss trong khi Longformer không. BigBird thêm thành phần random mà Longformer không có.
- **58 Sparse Transformer, 59 Reformer:** cả hai được trích dẫn là tiền đề trực tiếp của hướng "giảm $O(n^2)$ xuống tuyến tính/gần tuyến tính"; BigBird khác về cơ chế (random+window+global theo lý thuyết đồ thị, so với mẫu cố định của 58 hay hashing nội dung của 59) và là bài duy nhất trong ba bài có khung lý thuyết biểu diễn đầy đủ (universal approximator, Turing complete).
- **44-46 FlashAttention, PagedAttention:** giải bài toán khác (băng thông bộ nhớ của attention đầy đủ, quản lý KV cache lúc decode); BigBird thưa hoá số cặp được tính, không tối ưu kernel cho attention đầy đủ.
- **06 Mistral 7B:** sliding window của Mistral là một trường hợp riêng (không global, không random) của thành phần window trong BigBird, ở quy mô LLM hiện đại.
- **55 StreamingLLM, 56 H2O:** cùng ý tưởng "giữ vài token đặc biệt + cửa sổ gần nhất" (giống global+window) nhưng áp dụng cho KV cache lúc *sinh* (decode), không phải kiến trúc huấn luyện từ đầu như BigBird.

10. Tóm tắt công thức / khái niệm

Khái niệm	Công thức / ý	Ghi chú
Attention tổng quát theo đồ thị	$\text{ATTN}_D(X)_i = x_i + \sum_h \sigma(Q_h(x_i)K_h(X_{N(i)})^\top)V_h(X_{N(i)})$	D đầy đủ cho lại attention gốc
Ba thành phần	Random (r), Window (w), Global (g, ITC hoặc ETC)	$A(i,\cdot) = 1$ cho hợp của ba tập
ITC vs ETC	ITC: dùng lại token có sẵn. ETC: thêm g token mới, ma trận B mở rộng	ETC cần relative position encoding, CPC loss
Universal approximator	Đồ thị chứa star graph S (global) $\Rightarrow$ xấp xỉ mọi hàm liên tục	Định lý 1-2, theo khung Yun et al. 2019
Turing complete	Attention thưa $O(n)$ cạnh vẫn Turing complete	Định lý 3, mở rộng Pérez et al. 2019
Lower bound	Tác vụ vector-xa-nhất: $O(1)$ lớp (full) vs $\tilde{\Omega}(n^{1-o(1)})$ lớp (thưa $\tilde{O}(n)$ cạnh)	Mệnh đề 1, quy giảm từ OVC
Block-sparse	Reshape Q, K thành khối b; window qua "roll" key tensor, không gather	Chi phí $O(n(g+w+r)bd)$

Khái niệm	Công thức / ý	Ghi chú
Hyperparameter mặc định (base, ITC)	$b = 64$, $g = 2b$, $w = 3b$, $r = 3b$	Tổng ngân sách $8b = 512$ key/query

11. Kết luận ngắn

BigBird thưa hoá attention bằng ba thành phần lấy cảm hứng từ lý thuyết đồ thị (expander cho random, lưới vòng cho window, "hub" cho global), đạt chi phí $O(n)$ mà vẫn chứng minh được là universal approximator và Turing complete như attention đầy đủ, đồng thời chỉ ra một lower bound cho thấy cái giá lý thuyết của việc thưa hoá. Thực nghiệm xác nhận lợi ích lớn nhất ở tác vụ cần ngữ cảnh dài, ít dữ liệu (QA đa đoạn, phân loại tài liệu dài, tóm tắt, cả genomics); ở ngữ cảnh ngắn (GLUE) BigBird không thua kém. So với Longformer (bài 17), đóng góp chính của BigBird là thêm random attention và khung lý thuyết.

Link gốc: https://arxiv.org/abs/2007.14062

Bài 61

Quest: Query-Aware Sparsity for Efficient Long-Context LLM Inference (2024)

Tác giả: Jiaming Tang, Yilong Zhao, Kan Zhu, Guangxuan Xiao, Baris Kasikci, Song Han (Shanghai Jiao Tong University; MIT; University of Washington; NVIDIA). **Venue / arXiv:** arXiv:2406.10774 (v2, 26/08/2024); Proceedings of the 41st International Conference on Machine Learning (ICML 2024), Vienna, Austria, PMLR 235. **Nguồn:** https://arxiv.org/abs/2406.10774 **Chủ đề:** Chọn trang (page) KV cache để tính attention dựa trên truy vấn (query) hiện tại, dùng chặn trên điểm attention suy ra từ giá trị min/max theo từng kênh của Key trong mỗi trang, nhằm giảm lưu lượng đọc bộ nhớ khi decode ở ngữ cảnh dài mà không loại bỏ dữ liệu. **Vị trí trong lộ trình:** Đọc sau 01 (Attention Is All You Need), 17 (Longformer, sliding window), 46 (PagedAttention, khái niệm "trang" KV), 55 (StreamingLLM), 56 (H2O). Quest là lựa chọn KV theo từng bước decode, khác về bản chất với hai cơ chế loại bỏ vĩnh viễn (eviction) ở bài 55 và 56. Đọc tiếp 62 (MInference, cùng ý tưởng "ước lượng rồi chọn" nhưng áp cho prefill) và 63-65 (các cơ chế sparse attention học được hoặc huấn luyện từ đầu).

> Tài liệu học tập bằng tiếng Việt, là ghi chú diễn giải, không phải bản dịch. Số liệu lấy từ paper (full text arXiv, tải ngày 07/11/2026); các phép tính đánh dấu "tự tính" là của người ghi chú. Mục "Ứng dụng ..." là suy luận.

1. Bối cảnh

Theo cách chia của bài báo (§3.1), suy luận LLM có hai giai đoạn: prefill xử lý prompt song song và lưu Key/Value vào KV cache; decode sinh từng token và đọc lại cache đã có. Với prompt dài, việc đọc cache có thể chi phối thời gian. Trong ví dụ prompt 16K token và đầu ra 512 token, decode chiếm hơn 86% thời gian.

Vì sao decode chậm khi ngữ cảnh dài. Kích thước KV cache: $2 \times n_\ell \times T \times n_h \times d_h \times b$ (n_ℓ số lớp, T độ dài chuỗi, n_h số head, d_h chiều mỗi head, b byte/phần tử). Paper tính cho Llama-7B (32 lớp, 32 head, $d_h = 128$) ở $T = 32$K, FP16: ≈ 16 GB, cần tối thiểu 11 ms để đọc, chiếm hơn 50% độ trễ suy luận (FlashInfer FP16, RTX4090). Decode ở batch nhỏ là **memory-bound**: thời gian tỉ lệ với số byte phải đọc, không phải số phép tính.

Hướng trước đó: loại bỏ KV vĩnh viễn (eviction). H2O (bài 56) giữ token theo tổng điểm attention lịch sử; FastGen tinh chỉnh thêm loại token; TOVA chỉ dựa vào truy vấn hiện tại để quyết định loại; StreamingLLM (bài 55) giữ cố định vài token sink đầu cộng cửa sổ gần nhất. Điểm chung: các phương pháp này **quyết định xong là bỏ hẳn** một phần KV cache dựa trên lịch sử hoặc trạng thái hiện tại, nhưng token bị bỏ có thể lại quan trọng cho một truy vấn *ở tương lai*. SparQ tỉa theo kênh (channel pruning) rồi chọn token qua đó, nhưng paper nhận xét cách này chưa được kiểm chứng rộng cho tác vụ phụ thuộc xa, và sparsity theo kênh khó tăng tốc thực tế trên phần cứng.

Khái niệm cần biết trước: *KV cache* (lưu K, V để khỏi tính lại khi sinh tuần tự); *PagedAttention* (bài 46: chia KV cache thành các **trang (page)** kích thước cố định); *memory-bound* (thời gian quyết định bởi băng thông đọc, không bởi FLOPs).

2. Ý tưởng chính

Quest quan sát hai điều. Một, self-attention lúc suy luận rất thưa: hầu hết các lớp chỉ cần dưới 10% token để giữ perplexity gần như không đổi. Hai, **token nào là "quan trọng" (critical) phụ thuộc vào query hiện tại**, không cố định theo lịch sử: ví dụ với prompt "A is B. C is D. A is", token "B" chỉ nhận điểm attention cao khi truy vấn là từ "is" cuối câu, còn trước đó gần như không ai cần "B". Từ đó, Quest **không loại bỏ bất cứ token nào**; nó giữ nguyên toàn bộ KV cache nhưng quản lý theo **trang** (page, mượn từ PagedAttention),

tóm tắt mỗi trang bằng giá trị min và max theo từng kênh của các Key trong trang. Với query hiện tại, Quest dùng min/max này để tính một **chặn trên (upper bound)** cho điểm attention lớn nhất mà trang đó có thể đạt được, chọn ra Top-K trang có chặn trên cao nhất, rồi chỉ đọc KV thật của đúng các trang đó để tính attention chính xác (không xấp xỉ ở bước này). Vì chỉ đọc một phần nhỏ KV cache mỗi bước decode, Quest giảm lưu lượng bộ nhớ mà không đánh đổi thông tin: lần sau hỏi khác, trang khác có thể được chọn, không gì bị mất vĩnh viễn.

3. Phương pháp chi tiết

3.1. Hai quan sát làm nền

Sparsity cao theo lớp (Hình 3). Với mô hình LongChat-7B, paper đo "query-aware sparsity" của từng lớp: bỏ dần token KV cache sao cho perplexity trên PG19 tăng không quá 0.01. Kết quả: hai lớp đầu tiên có sparsity dưới 10% (cần giữ gần hết), còn từ lớp thứ ba trở đi sparsity trên 90% (chỉ cần chưa tới 10% token). Vì hai lớp đầu không thưa, Quest (và mọi baseline so sánh) **không áp dụng chọn lọc cho hai lớp đầu**, giữ full attention ở đó; paper nói rõ việc bỏ qua hai lớp đầu là độc lập với thuật toán chọn KV.

Tính quan trọng phụ thuộc query (Hình 2). Với ví dụ "A is B. C is D. A is", ở lớp 16 của Llama-2-7B, token "B" có điểm attention rất thấp khi các token trước "is" cuối cùng truy vấn, nhưng vừa tới token "is" thì "B" nhận điểm cao hẳn. Do đó không thể xác định trước (offline, không biết query) token nào sẽ quan trọng; phải tính theo query thực tế mỗi bước.

3.2. Quản lý KV cache theo trang và cập nhật metadata

Quest chia KV cache thành các trang kích thước S (ví dụ 16 cặp KV/trang), mượn cơ chế PagedAttention (bài 46). Mỗi trang lưu thêm 2 vector metadata theo từng kênh $i = 1, \dots, d_h$: giá trị **min** m_i và **max** M_i của Key trong trang. Khi chèn token mới vào trang (Algorithm 1 của paper, phần cập nhật):

$$M_i \leftarrow \max(M_i, k_i), \qquad m_i \leftarrow \min(m_i, k_i), \qquad i = 1, \dots, d_h$$

Chi phí cập nhật là $O(d_h)$ mỗi token, không đáng kể.

3.3. Chặn trên điểm attention của một trang (Criticality Estimation)

Câu hỏi: với vector query Q hiện tại, trang này *có thể* chứa token nào khiến điểm attention $Q \cdot K$ lớn? Ta không biết Key thật của từng token trong trang khi chưa đọc nó, chỉ biết mỗi kênh i nằm trong khoảng $[m_i, M_i]$.

Suy diễn (tự suy diễn từ phát biểu của paper). Với một kênh i cố định và $k_i \in [m_i, M_i]$:

- Nếu $Q_i \geq 0$: tích $Q_i k_i$ lớn nhất khi $k_i = M_i$, giá trị $Q_i M_i$.
- Nếu $Q_i < 0$: tích $Q_i k_i$ lớn nhất khi $k_i = m_i$ (nhân số âm đổi chiều), giá trị $Q_i m_i$.

Cả hai trường hợp đều được gộp bởi:

$$\boxed{U_i = \max(Q_i\, m_i,\ Q_i\, M_i)}$$

$U_i \geq Q_i k_i$ với **mọi** $k_i \in [m_i, M_i]$, bất kể dấu của Q_i (đúng như paper phát biểu). Cộng dồn theo mọi kênh, ta có chặn trên cho cả vector:

$$\boxed{\text{score(trang)} = \sum_{i=1}^{d_h} U_i \ \geq \ \max_{j \in \text{trang}} Q \cdot K_j}$$

Đây là chặn trên "theo hộp" (axis-aligned box bound): nó giả định (lỏng) rằng mỗi kênh có thể *đồng thời* đạt giá trị biên của mình, điều không chắc một token thật nào làm được, nên score luôn $\geq$ điểm attention thật lớn nhất trong trang, nhưng có thể lỏng hơn giá trị thật. Pseudo-code đầy đủ (Algorithm 1 của paper, cả hai phần):

```
# Khi chèn token mới k vào trang:
for i in range(dim):
    M[i] = max(M[i], k[i]);  m[i] = min(m[i], k[i])

# Khi tính attention với query Q (mỗi trang):
score = 0
for i in range(dim):
    score += max(Q[i]*M[i], Q[i]*m[i])
# score = chặn trên điểm attention lớn nhất trong trang
```

3.4. Hai giai đoạn: ước lượng rồi tính attention thật (Hình 5)

1. **Giai đoạn 1 (ước lượng).** Chỉ đọc 2 vector metadata (min, max) của mỗi trang, tính score theo công thức trên cho *tất cả* trang, chọn **Top-K** trang có score cao nhất (K là hằng số đặt trước, ví dụ 128, 256 trang · page size).
2. **Giai đoạn 2 (attention thật).** Chỉ đọc KV thật của đúng K trang đã chọn, tính softmax attention **chính xác** (không xấp xỉ) trên các token đó. Số token trong các trang được chọn gọi là **ngân sách token (token budget)**.

Khác với eviction (bài 55, 56): các trang không được chọn **vẫn còn nguyên trong bộ nhớ**, chỉ là không được đọc ở bước này; truy vấn khác ở bước sau có thể chọn lại chúng. Paper nói thẳng ở §2.2: Quest "retains all of the KV cache and selects part of the KV cache based on the current query" (giữ toàn bộ KV cache, chỉ chọn một phần theo query hiện tại) để tăng tốc attention mà không giảm độ chính xác.

Chi phí Top-K: paper đo thời gian của toán tử Top-K (dùng kernel từ thư viện tìm kiếm vector RAFT) là 5-10 µs cho chuỗi tới 128K token, "không đáng kể" nên không tính vào phân tích hiệu quả.

3.5. Phân tích lưu lượng bộ nhớ

Gọi mỗi vector K hoặc V là M byte, KV cache có L token, mỗi trang S cặp KV, chọn K trang. Lưu lượng đọc của Quest gồm: (i) đọc metadata min/max mọi trang, $\approx 2M \cdot L/S$ byte; (ii) đọc KV thật của K trang được chọn, $2M \cdot K \cdot S$ byte. So với đọc toàn bộ cache $2M \cdot L$ byte, tỉ lệ byte đọc của Quest là:

$$\boxed{\text{fraction} = \frac{1}{S} + \frac{K \cdot S}{L}}$$

Ví dụ của paper: page size $S = 16$, ngữ cảnh $L = 65536$ (64K), chọn $K = 256$ trang (ngân sách $256 \times 16 = 4096$ token). Fraction $= \frac{1}{16} + \frac{256 \times 16}{65536} = 0.0625 + 0.0625 = \frac{1}{8}$, khớp phát biểu "giảm lưu lượng bộ nhớ 8 lần". Số hạng $1/S$ (đọc metadata, cố định) là một "sàn" không giảm thêm được dù chọn ít trang hơn; tăng S hạ sàn này nhưng làm chặn trên lỏng hơn (nhiều token khác nhau gộp vào một hộp min/max, dễ chọn thừa). Lợi ích này tương thích với lượng tử hóa KV cache (paper dẫn công trình khác, Atom).

4. Thí nghiệm và kết quả chính

Mô hình: LongChat-7b-v1.5-32k, Yarn-Llama-2-7b-128k. **Baseline:** H2O, TOVA, StreamingLLM (eviction, bài 55/56), và "Full" (attention đầy đủ). **Dữ liệu:** PG19 (perplexity), passkey retrieval (Yarn), 6 tập LongBench: NarrativeQA, Qasper, MultiFieldQA, HotpotQA, GovReport, TriviaQA. Hai lớp đầu luôn giữ full attention cho mọi phương pháp (theo §3.1).

Perplexity trên PG19 (Hình 6). Với LongChat-7b-v1.5-32k, ngân sách 4096 token (~1/8 độ dài 32K): đường perplexity của Quest bám sát "Full" (oracle) trên toàn dải 0-32000 token; giá trị số chính xác không đọc được

rõ từ bản text trích xuất, chỉ nêu xu hướng định tính.

Passkey retrieval (Bảng 1, % độ chính xác lấy đúng passkey):

(i) 10K, LongChat-7b-v1.5-32k, ngân sách 32/64/128/256/512:

	32	64	128	256	512
H2O	0%	1%	1%	1%	3%
TOVA	0%	1%	1%	3%	8%
StreamingLLM	1%	1%	1%	3%	5%
Quest	65%	**99%**	**99%**	**99%**	**100%**

(ii) 100K, Yarn-Llama-2-7b-128k, ngân sách 256/512/1024/2048/4096:

	256	512	1024	2048	4096
H2O	2%	2%	2%	2%	4%
TOVA	2%	2%	2%	2%	10%
StreamingLLM	1%	1%	1%	2%	4%
Quest	88%	92%	96%	**100%**	**100%**

Lý do khoảng cách rất lớn (paper giải thích trực tiếp): H2O, TOVA, StreamingLLM **loại nhầm KV cache chứa passkey trước khi câu hỏi được hỏi**, vì chúng quyết định loại bỏ dựa trên lịch sử/vị trí, không biết trước câu hỏi cần gì. Quest không loại bỏ gì nên vẫn còn đủ dữ liệu để chọn đúng trang chứa passkey.

LongBench (Hình 7, 6 tập, nhiều ngân sách). Quest vượt mọi baseline ở mọi ngân sách trên cả 6 tập; với ngân sách 1K token đã gần bằng full cache ở hầu hết tập. Tính cả việc giữ full attention ở 2 lớp đầu, Quest đạt độ chính xác "không mất mát" với sparsity: Qasper 1/6, HotpotQA 1/6, GovReport 1/5, TriviaQA 1/10, NarrativeQA 1/5, MultiFieldQA 1/6.

Hiệu quả kernel (Hình 8, Llama2-7B, RTX4090, CUDA 12.2). (a) Criticality estimation: ở chuỗi ngắn lợi thế nhỏ (chưa bão hòa băng thông); chuỗi dài hơn, độ trễ tương đối giảm dần về $1/S$ so với FlashInfer. (b) Approximate attention: với ngân sách K cố định, độ trễ **không đổi** theo độ dài chuỗi tổng, tương đương FlashInfer ở chuỗi độ dài K.

Tổng hợp self-attention (Hình 9). Tại độ dài 32K, ngân sách 2048, Quest tăng tốc self-attention **7.03×** so với FlashInfer (số rõ nhất trong văn bản; các mức trung gian ~1.69×/2.92×/4.86× trong hình bị xáo trộn thứ tự khi trích xuất PDF, chỉ nêu tham khảo).

End-to-end (Hình 10, Ada 6000, batch 1). Tại 32K, ngân sách 2048: Quest tăng tốc **1.74×** với trọng số FP16, **2.23×** với 4-bit (AWQ), so với FlashInfer full cache. Độ trễ Quest tăng rất chậm theo độ dài chuỗi, khác FlashInfer tăng gần tuyến tính.

Cùng mức độ chính xác (Hình 11). NarrativeQA (ngữ cảnh trung bình ~24K): TOVA cần ngân sách 14K cho độ chính xác không mất mát, Quest chỉ cần 5K. Về độ trễ: Quest nhanh hơn TOVA **3.82×** trên GovReport, **4.54×** trên TriviaQA (hai số nêu rõ trong văn bản; các cột khác của Hình 11 chỉ đọc qua hình).

5. Phân tích / ablation

- **Vì sao bỏ 2 lớp đầu:** sparsity thấp (< 10%, Hình 3) khiến mọi phương pháp chọn lọc (kể cả Quest) có nguy cơ mất thông tin nếu áp dụng; giữ full attention ở đây là lựa chọn thiết kế chung cho toàn bộ so sánh.

- **Recall rate Top-10 (Hình 4, passkey 10K):** đo tỉ lệ token Top-10 điểm attention mà mỗi phương pháp giữ được ở ngân sách 64/256/1024. Quest giữ recall gần full, H2O thấp hơn hẳn vì token quan trọng bị loại ở vòng quyết định trước; số liệu cụ thể chỉ đọc qua hình.
- **Eviction vs selection (Bảng 1, LongBench Hình 7):** chênh lệch lớn và nhất quán ở mọi ngân sách khẳng định nguyên nhân gốc là **loại bỏ vĩnh viễn**, không phải chi tiết thuật toán chọn từng baseline.

6. Giới hạn

Do tác giả nêu:

- Ngân sách K (số trang chọn) là hằng số đặt trước, không tự động theo nội dung hay độ khó truy vấn.
- Hình 11(b) dùng độ trễ FlashInfer cho các baseline (vì chúng không có kernel riêng), bỏ qua chi phí vận hành khác (TOVA cần tính điểm lịch sử); paper tự nhận đây là so sánh "định tính" cho phần baseline.
- H2O cần ma trận attention $O(n^2)$ đầy đủ, không tương thích trực tiếp FlashInfer; để đánh giá công bằng ở 100K token, paper phải bắt đầu tính điểm H2O từ giai đoạn decode.

Nhận xét:

- Chặn trên theo "hộp" (§3.3) đúng nhưng có thể **lỏng**: trang có Key phân tán theo kênh vẫn có thể nhận score cao giả dù không token nào thật đạt cực trị ở mọi kênh, dẫn tới chọn thừa; paper không đo trực tiếp độ lỏng này, chỉ đo recall kết quả cuối.
- Quest **chỉ tiết kiệm băng thông đọc mỗi bước**, không giảm **dung lượng lưu trữ**: toàn bộ KV cache vẫn phải được cấp phát và giữ trong VRAM, khác H2O/StreamingLLM vốn giảm cả dung lượng. Ngữ cảnh cực dài vượt VRAM thì Quest không giải quyết được.
- Thí nghiệm chỉ ở model 7B, MHA (Llama/Yarn), độ dài 10K-128K; chưa có kết quả cho GQA/MQA, mô hình nhỏ, hay đa phương thức.
- Chọn page size S và ngân sách K là siêu tham số thủ công, không có quy tắc tự động.

7. Liên hệ với các bài khác

- **01 Attention Is All You Need:** Quest không đổi công thức attention ở giai đoạn 2, chỉ chọn tập Key/Value đưa vào softmax.
- **46 PagedAttention:** nền tảng kỹ thuật "trang" của Quest; Quest thêm metadata min/max và bước ước lượng lên trên đó.
- **17 Longformer:** cùng tinh thần "giữ softmax, giảm số cặp tính", nhưng Longformer chọn cặp theo **mẫu cố định** (cửa sổ + global) biết trước khi chạy, Quest chọn **động theo từng query** mỗi bước decode.
- **55 StreamingLLM, 56 H2O:** cả hai là **eviction**, loại bỏ vĩnh viễn KV theo vị trí cố định (55) hoặc điểm attention lịch sử (56); Quest **giữ nguyên toàn bộ KV cache**, chỉ chọn phần đọc theo query hiện tại, nên tránh được rủi ro "lỡ bỏ token sẽ cần ở tương lai" (Bảng 1). Theo cách diễn giải này: StreamingLLM/H2O đổi cả dung lượng lẫn lưu lượng để lấy tốc độ (mất thông tin); Quest chỉ đổi lưu lượng (không mất thông tin, không giảm dung lượng) - dựa trên phát biểu "retains all of the KV cache" (§2.2) của paper.
- **62 MInference:** cùng ý tưởng "ước lượng rẻ rồi tính attention thật trên phần quan trọng", nhưng nhắm vào **prefill** (song song) còn Quest nhắm **decode** (tuần tự); hai bài bù trừ hai giai đoạn của một lượt suy luận dài.
- **63 SeerAttention, 64 NSA, 65 MoBA:** các hướng **học** một cơ chế dự đoán sparsity, khác Quest là **training-free** (chỉ cần min/max tính ngay trên mô hình có sẵn).
- **58-60 (Sparse Transformer, Reformer, BigBird):** mẫu sparse attention cố định/học theo cụm thời kỳ đầu, tiền đề ý tưởng "không cần tính mọi cặp".

8. Tóm tắt công thức / khái niệm

Khái niệm	Công thức / ý nghĩa
KV cache	$2n_\ell T n_h d_h b$ byte; Quest: 16 GB cho Llama-7B ở $T = 32$K
Trang (page)	Nhóm S token liên tiếp trong KV cache, mượn từ PagedAttention (bài 46)
Metadata trang	$m_i = \min_i(k)$, $M_i = \max_i(k)$ theo từng kênh i, cập nhật khi chèn token
Chặn trên mỗi kênh	$U_i = \max(Q_i m_i,\ Q_i M_i) \geq Q_i k_i\ \forall k_i \in [m_i, M_i]$
Score trang	$\sum_i U_i \geq$ điểm attention lớn nhất thật trong trang
Hai giai đoạn	(1) ước lượng từ metadata, chọn Top-K trang; (2) attention chính xác chỉ trên K trang
Fraction lưu lượng	$\frac{1}{S} + \frac{KS}{L}$; ví dụ $S = 16, K = 256, L = 64$K $\Rightarrow 1/8$
Khác biệt cốt lõi với eviction	Giữ toàn bộ KV (không mất thông tin), chỉ chọn phần đọc theo query mỗi bước
Kết quả chính	Passkey: Quest 88-100% vs eviction ≤10%; self-attn speedup tới 7.03×; end-to-end 1.74× (FP16) / 2.23× (4-bit) ở 32K, ngân sách 2048

9. Kết luận ngắn

Quest cho thấy một cách khác để làm attention rẻ hơn ở ngữ cảnh dài: không loại bỏ token nào cả, chỉ tóm tắt mỗi trang KV bằng min/max theo kênh, dùng chặn trên đơn giản để đoán trang nào "có thể" chứa điểm attention cao cho query hiện tại, rồi chỉ đọc đúng Top-K trang đó. Vì không bao giờ mất dữ liệu, Quest tránh được thất bại nghiêm trọng của các phương pháp loại bỏ vĩnh viễn (StreamingLLM, H2O) trên bài toán truy xuất xa như passkey retrieval, với tốc độ decode tăng tới 7.03× (self-attention) ở 32K token.

Link gốc: https://arxiv.org/abs/2406.10774

Bài 62

MInference 1.0: Accelerating Pre-filling for Long-Context LLMs via Dynamic Sparse Attention (2024)

Tác giả: Huiqiang Jiang, Yucheng Li, Chengruidong Zhang, Qianhui Wu, Xufang Luo, Surin Ahn, Zhenhua Han, Amir H. Abdi, Dongsheng Li, Chin-Yew Lin, Yuqing Yang, Lili Qiu (Microsoft Corporation; University of Surrey). **Venue / arXiv:** arXiv:2407.02490 (v2, 30/10/2024); 38th Conference on Neural Information Processing Systems (NeurIPS 2024). **Nguồn:** https://arxiv.org/abs/2407.02490 **Chủ đề:** Tăng tốc giai đoạn prefill (xử lý toàn bộ prompt dài) bằng ba mẫu attention thưa (A-shape, Vertical-Slash, Block-Sparse), tìm mẫu tối ưu cho từng head ngoại tuyến, ước lượng chỉ số thưa trực tuyến với chi phí thấp, tính attention bằng kernel GPU chuyên biệt. **Vị trí trong lộ trình:** Đọc sau 01 (Attention), 17 (Longformer), 44/45 (FlashAttention), 55 (StreamingLLM, chính là trường hợp đặc biệt A-shape của bài này), 60 (BigBird). MInference là đối trọng của bài 61 (Quest): Quest tối ưu decode, MInference tối ưu prefill. Đọc tiếp 63 (SeerAttention, học mẫu sparsity thay vì tìm kiếm ngoại tuyến), 64 (NSA), 65 (MoBA).

> Tài liệu học tập bằng tiếng Việt, là ghi chú diễn giải, không phải bản dịch. Số liệu lấy từ paper (full text arXiv, tải ngày 07/11/2026); các phép tính đánh dấu "tự tính" là của người ghi chú. Mục "Ứng dụng ..." là suy luận.

1. Bối cảnh

Prefill xử lý prompt để tạo token đầu tiên; thời gian chờ đến token đó được gọi là time to first token (TTFT). Vì chi phí attention tăng bậc hai theo độ dài, prefill trở nên tốn kém ở ngữ cảnh dài. Trong phép đo của bài báo với LLaMA-3-8B trên một A100, prompt 300K token mất 6 phút và prompt 1M token mất 30 phút; attention chiếm hơn 90% độ trễ prefill.

Hướng trước đó. Attention lúc suy luận đã biết rất thưa, dẫn tới **mẫu attention thưa cố định** như Longformer (bài 17), BigBird (bài 60): hiệu quả nhưng cần huấn luyện lại, không áp trực tiếp lên LLM có sẵn. Khó hơn, phân phối attention **thay đổi mạnh theo từng input**, nên mẫu cố định không khớp mọi prompt; nếu dự đoán được mẫu thưa động một cách rẻ, có thể tính attention chỉ trên phần quan trọng mà vẫn đúng cho từng input.

Khái niệm cần biết trước: *TTFT* (thời gian tới token đầu tiên, quyết định bởi tốc độ prefill); *training-free* (áp dụng thẳng lên mô hình đã pretrain, không cần huấn luyện lại); *FLOPs-bound* (prefill xử lý nhiều token cùng lúc nên bị giới hạn bởi số phép tính, khác decode vốn memory-bound như ở bài 55/56/61).

2. Ý tưởng chính

MInference quan sát hai điều ở ma trận attention của LLM ngữ cảnh dài: (i) rất thưa, giữ top 4K cột của ma trận 128K×128K vẫn recall được 96,8% tổng điểm attention; (ii) **động**: dùng lại đúng tập chỉ số đó cho một prompt khác thì recall rơi xuống 83,7%, nghĩa là không thể xác định chỉ số thưa trước (offline) cho mọi input. Tuy động, phân phối thưa của mỗi **head** vẫn có **hình dạng ổn định** qua các prompt khác nhau, rơi vào một trong ba mẫu: **A-shape** (tập trung vào token đầu + cửa sổ gần nhất, tĩnh), **Vertical-Slash** (một số cột cố định + các đường chéo cách đều, động nhưng có cấu trúc), **Block-Sparse** (các khối rời rạc, động nhất). MInference gồm ba bước: (1) **ngoại tuyến (offline)**, xác định một lần cho mỗi head mô hình nên dùng mẫu nào; (2) **trực tuyến (online)**, với mỗi prompt mới, ước lượng rẻ chỉ số thưa cụ thể theo mẫu đã gán; (3) tính attention thưa bằng **kernel GPU** chuyên biệt cho ba mẫu. Vì chỉ áp dụng ở prefill (không sửa decode), không cần huấn luyện lại, MInference giảm tới 95% FLOPs của phép tính attention, đạt tốc độ prefill tới 10× ở 1M token mà độ chính xác downstream không giảm (có tác vụ còn nhú nhẹ).

3. Phương pháp chi tiết

3.1. Hai quan sát và ba mẫu (§2)

Hình 2 của paper: trên ma trận attention 128K×128K của LLaMA-3-8B, top-4096 cột recall 96,8% tổng attention (thưa); áp đúng tập chỉ số đó sang input khác thì recall giảm xuống 83,7% (động theo input). Dù vậy, hình dạng không gian của mỗi head ổn định qua nhiều prompt (Hình 3a), rơi vào ba loại (Bảng 1 của paper):

Mẫu	Phân bố không gian	Độ trễ GPU	Thời gian dựng chỉ số
A-shape	Tĩnh, có cấu trúc	Thấp	Bằng không
Vertical-Slash	Động, có cấu trúc	Trung bình	Nhỏ
Block-Sparse	Động, có cấu trúc	Thấp	Nhỏ
(Top-K, để so sánh)	Động, chi tiết	Cao	Lớn

A-shape: attention tập trung vào token đầu và cửa sổ cục bộ (giống attention sink bài 55), tương đối ổn định. **Vertical-Slash (VS):** attention tập trung vào một số token cố định (đường **dọc**) cộng các token cách đều nhau (đường **chéo/slash**); vị trí các đường này thay đổi theo nội dung. **Block-Sparse:** phân tán nhất nhưng vẫn giữ tính cụm theo không gian: paper đo khoảng cách tới điểm phi-zero gần nhất trong top-10 (Hình 3b) và thấy phần lớn tập trung quanh khoảng cách 5, xác nhận cụm không gian. Với cùng ngân sách FLOPs, ba mẫu này recall điểm attention cao hơn hẳn phương pháp Top-K tổng quát (Hình 3c), vì Top-K không khai thác được cấu trúc cụm của Block-Sparse.

3.2. Công thức bài toán (§3.1)

Attention thưa được viết lại bằng một **mặt nạ (mask)** $M_{i,j} \in \{0,1\}$:

$$\boxed{A(M) = \text{Softmax}\Big(\tfrac{1}{\sqrt{d}} QK^\top - c(1-M)\Big)}$$

với c là hằng số lớn (paper: 10^5) để các vị trí $M_{i,j} = 0$ gần như triệt tiêu sau softmax ($A_{i,j} \approx 0$). Mục tiêu kép:

$$\min_M \|A(M) - A_{\text{dense}}\|, \qquad \min_M \; t_{\text{sparse}}(M) + t_{\text{overhead}}(M)$$

tức vừa giữ đầu ra gần với attention đầy đủ, vừa giảm tổng thời gian tính thưa và thời gian ước lượng mặt nạ đó. Hai mục tiêu được paper giải bằng hai bước riêng: bước **offline** quyết định mẫu (ảnh hưởng nhóm 2, chi phí tính), bước **online** ước lượng chỉ số cụ thể cho từng input (ảnh hưởng cả hai).

3.3. Tìm mẫu tối ưu cho từng head, ngoại tuyến (Kernel-Aware Optimal Sparse Pattern Search, Algorithm 1)

Ý tưởng: với một ngân sách FLOPs mục tiêu t cho trước, tạo không gian tìm kiếm ρ gồm các cấu hình (mẫu + siêu tham số của mẫu, ví dụ số đường vertical/slash, hay số block top-k) có **FLOPs thật trên kernel GPU** (không phải FLOPs lý thuyết) xấp xỉ t; sau đó với một ví dụ tham chiếu, tính attention đầy đủ $y = \text{softmax}(QK^\top/\sqrt{d})$ và attention thưa y_i theo từng cấu hình trong ρ, chọn cấu hình có $\|y_i - y\|$ nhỏ nhất làm mẫu cho head đó:

```
for i in range(|sigma|):                        # dựng không gian tìm kiếm theo FLOPs thật
    while |FLOPs_in_kernel(sigma_i) - target_t| > eps:
        sigma_i = ChangeSpace(sigma_i, pattern_i)
    rho.add(sigma_i)

y = softmax(Q @ K.T / sqrt(d))                   # attention đầy đủ, ví dụ tham chiếu
```

```
best = None
for rho_i in rho:
    y_i = SparseAttention(Q, K, rho_i)
    best = argmin_recall_error(y_i, y, best)     # chọn cấu hình gần y nhất
return best                                      # mẫu + siêu tham số cho head này
```

Điểm quan trọng: FLOPs tính "kernel-aware" (đo thật trên GPU) để các cấu hình so sánh công bằng, và mục tiêu chọn là **recall đầu ra attention** (dùng FlashAttention để tính, tận dụng cả thông tin từ V), không phải một chỉ số gián tiếp. Việc này chỉ làm **một lần cho mỗi mô hình** (ngoại tuyến), không lặp lại mỗi request.

3.4. Ước lượng chỉ số thưa trực tuyến (Algorithm 2, 3; Hình 4)

Với mỗi prompt mới, chỉ số thưa cụ thể (token/đường/khối nào được giữ) phải tính **nhanh**, không được tính attention đầy đủ trước.

Head Vertical-Slash (Algorithm 2). Dùng $Q[-\text{last_q}:]$ (chỉ **last_q = 64** vector query cuối) nhân với toàn bộ K:

$$\hat{A} = \text{softmax}\left(\frac{Q[-\text{last_q}:]\,K^\top}{\sqrt{d}} + m_{\text{causal}}\right)$$

rồi cộng dồn $\hat{A}$ theo cột để chọn Top-k_v **đường dọc (vertical)**, và cộng dồn theo đường chéo để chọn Top-k_s **đường chéo (slash)**; hai tập chỉ số này được gộp vào một chỉ số thưa i_{vs} dùng để tính attention thật (sparse) trên toàn bộ Q, K, V. Trực giác: vài query cuối đủ đại diện để lộ ra đường dọc/chéo nào đang "nóng", không cần dùng mọi query.

Head Block-Sparse (Algorithm 3). Mean-pooling Q và K theo khối kích thước **block_size = 64** được $\hat{Q}, \hat{K}$; nhân $\hat{Q}\hat{K}^\top$ cho điểm attention cấp-khối ước lượng, chọn Top-k_b khối. Vì phép mean-pooling và phép nhân ma trận giao hoán được (trung bình của tích $\approx$ tích của trung bình khi các phần tử trong khối tương quan), điểm cấp-khối ước lượng này **xấp xỉ đúng** trung bình attention thật trong khối, với chi phí rẻ hơn hẳn tính đủ rồi mới pooling.

Head A-shape: không cần ước lượng gì (mặt nạ tĩnh: token đầu + cửa sổ cục bộ cố định), nên "thời gian dựng chỉ số bằng không" (Bảng 1).

3.5. Kernel

Ba kernel GPU tối ưu riêng cho ba mẫu, xây trên các compiler/thư viện tính thưa động: **PIT** (dynamic sparse compiler), **Triton**, và **FlashAttention**. Vì viết bằng Triton, paper nói kernel dễ chuyển sang phần cứng khác (H100, MI300X) mà vẫn giữ tốc độ. MInference tương thích với FlashAttention cho phần không bị mask và chỉ thêm chi phí cho phần tính chỉ số thưa.

4. Thí nghiệm và kết quả chính

Mô hình: LLaMA-3-8B-Instruct-262K, LLaMA-3-8B-Instruct-1048K (1M), GLM-4-9B-1M, Yi-9B-200K (thêm Phi-3-Mini-128K, Qwen2-7B-128K cho riêng Needle In A Haystack); GPU A100, bfloat16. Siêu tham số: A-shape dùng ngân sách FLOPs tương đương 1K token global + 4K cửa sổ cục bộ; last_q = 64; block_size = 64. **Dữ liệu:** InfiniteBench (10 tác vụ: tóm tắt, QA, mã, hội thoại, truy xuất passkey/số/KV..., độ dài trung bình ~214K); RULER (4 nhóm, 13 tác vụ, tới 128K); Needle In A Haystack (tới 1M); PG-19 (ngôn ngữ, tới 100K). **Baseline (đều training-free):** StreamingLLM (chính là A-shape, bài 55), StreamingLLM w/ dilated, StreamingLLM w/ strided, InfLLM (bộ nhớ trung gian cho streaming), "Ours w/ static" (biến thể dùng chỉ số **tĩnh** cho head VS/Block-Sparse, để đo lợi ích của việc ước lượng *động*). Tất cả baseline chỉ tính thưa ở prefill, giữ dense ở decode (giống MInference).

InfiniteBench (Bảng 2 của paper, LLaMA-3-8B-262K; trung bình 10 tác vụ):

Phương pháp	En.Sum	En.QA	Code.Debug	Retr.PassKey	Retr.KV	Trung bình
Full attention	20.2	12.4	22.1	100.0	14.4	38.2
StreamingLLM	21.0	8.2	25.9	86.8	0.8	23.8
InfLLM	24.1	7.8	19.5	100.0	1.2	34.8
Ours w/ static	19.9	8.6	20.6	92.4	0.2	31.9
MInference (Ours)	20.5	12.9	22.3	100.0	12.8	**38.8**

MInference vượt nhẹ full attention trung bình (38.8 vs 38.2) nhờ giữ tốt tác vụ tự nhiên (QA, code) và vẫn đạt 100% truy xuất passkey/số; chỉ thua nhẹ ở Retr.KV (12.8 vs 14.4). Trên GLM-4-9B-1M và Yi-9B-200K, paper báo xu hướng tương tự (MInference bám sát full attention, StreamingLLM/InfLLM tụt mạnh ở truy xuất); không trích lại toàn bộ số để tiết kiệm dung lượng.

RULER (Bảng 3, % qua các độ dài 4K-128K; "Effective" = độ dài còn giữ > 85% điểm):

Mô hình / phương pháp	Claimed	Effective	4K	32K	128K	Avg.
LLaMA-3-8B-262K (full)	262K	16K	97.2	80.8	72.2	84.4
StreamingLLM	-	4K	97.2	17.2	9.4	35.0
InfLLM	-	4K	89.4	55.6	39.5	62.9
MInference (Ours)	-	**32K**	97.7	85.0	77.6	**87.0**
GLM-4-9B-1M (full)	1M	64K	93.8	87.4	80.8	88.0
MInference (Ours)	-	**64K**	94.6	89.6	84.0	**89.6**

Điểm đáng chú ý: MInference đạt **effective context window lớn hơn cả full attention báo cáo** (32K so với 16K ở LLaMA-3, 64K bằng GLM-4), và vượt full attention ở các độ dài trên 32K; paper giải thích baseline cố định cửa sổ cục bộ (StreamingLLM) hay bộ nhớ trung gian (InfLLM) không theo kịp tác vụ multi-hop/aggregation của RULER.

Ngôn ngữ (PG-19, Hình 5). Với prompt 100K token, perplexity của MInference chỉ cao hơn full attention 0.2; thấp hơn StreamingLLM 0.25 (LLaMA-3-262K) và 0.75 (Yi-9B-200K).

Needle In A Haystack (Hình 1a so với Hình 6). MInference giữ được khả năng tìm thông tin ở mọi vị trí/độ sâu từ 1K đến 1M token (giống full attention); StreamingLLM suy giảm mạnh ngay khi thông tin nằm ngoài vùng token global + cửa sổ cục bộ.

Độ trễ (Hình 1b, 10; A100, 1M token). Tốc độ tăng theo độ dài: **1.8×** ở 100K, **4.1×** ở 300K, **6.8×** ở 500K, **10×** ở 1M, giảm thời gian prefill từ 30 phút xuống 3 phút cho prompt 1M token trên một A100; kết hợp tensor-parallel và context-parallel trên 8×A100 có thể hạ xuống 22 giây. Phân tích breakdown: 5-20% độ trễ còn lại dùng để dựng chỉ số thưa động, phần còn lại là tính attention thưa.

Kết hợp/mở rộng (Bảng 5, 6). Kết hợp với SnapKV (nén KV ở decode): điểm InfiniteBench tăng nhẹ (36.0 → 37.3) so với SnapKV đơn lẻ trên LLaMA-3-8B, cho thấy tương thích được. Ở LLaMA-3-70B-262K, MInference vẫn bám sát full attention (47.3 so với 46.5), vượt StreamingLLM (21.6) và InfLLM (39.2).

5. Phân tích / ablation

Động quan trọng hơn tĩnh (Bảng 2, 3, 4: "Ours w/ static"). Dùng chỉ số **tĩnh** cho head Vertical-Slash/Block-Sparse làm tụt hẳn hiệu năng, đặc biệt ở tác vụ rất động như KV retrieval (gần về 0), xác nhận bước ước lượng online (§3.4) là cần thiết, không chỉ việc phân loại mẫu offline.

Mỗi mẫu đóng góp riêng (Bảng 4, LLaMA-3-8B-262K, InfiniteBench):

Biến thể	Trung bình	So với đầy đủ (38.8)
Chỉ A-shape (= StreamingLLM)	23.8	chỉ nắm được thông tin trong cửa sổ cục bộ
Chỉ Block-Sparse	18.7	giảm mạnh nhất khi dùng đơn lẻ
Chỉ Vertical-Slash	37.1	gần đầy đủ nhất khi dùng đơn lẻ (cân bằng động/tĩnh)
Cả ba (MInference)	**38.8**	tốt nhất

Bỏ bất kỳ mẫu nào cũng làm giảm hiệu năng; "chỉ Vertical-Slash" giữ được nhiều nhất vì mẫu này cân bằng giữa tính động và đặc trưng kiểu StreamingLLM, nhưng vẫn kém bản đầy đủ ba mẫu.

Chi phí ước lượng động (mục 4, Latency): 5-20% độ trễ còn lại dùng để dựng chỉ số (tùy độ dài và mẫu), còn lại là tính attention thưa; cho thấy bước ước lượng rẻ hơn phần tính toán chính nó tiết kiệm được.

6. Giới hạn

Do tác giả nêu:

- Dù xấp xỉ tốt, MInference vẫn thua full attention ở tác vụ cực động như KV retrieval (Bảng 2: 12.8 so với 14.4 trên LLaMA-3; 27.6 so với 28.8 trên Yi-9B).
- Tìm kiếm mẫu offline dùng **một ví dụ tham chiếu** (Algorithm 1); không phân tích paper nêu rõ độ nhạy của mẫu được chọn với việc đổi ví dụ tham chiếu khác.
- Tất cả baseline (và cả MInference) **chỉ tính thưa ở prefill, vẫn dùng dense attention ở decode**; đây là giới hạn phạm vi chủ động của paper (chỉ nhắm TTFT), không phải thiếu sót.

Nhận xét:

- Vì chỉ tối ưu prefill, MInference **không giúp gì cho độ trễ decode batch 1**; nó bổ sung cho (không thay thế) các kỹ thuật tối ưu decode như bài 46, 55, 56, 61.
- Hiệu quả phụ thuộc kernel Triton/PIT tối ưu riêng; portability sang phần cứng khác chưa được đo kỹ ngoài A100/H100/MI300X được nêu.
- last_q=64 và block_size=64 cố định cho mọi mô hình trong thí nghiệm chính; paper không khảo sát độ nhạy khi đổi theo kích thước mô hình hoặc d_h.
- Baseline so sánh đều là phương pháp giảm context cố định theo vị trí (StreamingLLM, InfLLM); không so trực tiếp với phương pháp chọn theo nội dung ở decode (H2O, bài 56) vì khác giai đoạn.

7. Liên hệ với các bài khác

- **01, 44/45 FlashAttention:** MInference dùng FlashAttention để tính attention đầy đủ (vòng chọn mẫu offline) và làm nền kernel; công thức attention cơ bản không đổi, chỉ mặt nạ M thay đổi.
- **17 Longformer, 60 BigBird:** tiền đề "mẫu attention thưa cố định", nhưng cả hai cần huấn luyện lại; MInference không cần vì tìm mẫu + ước lượng chỉ số đều làm trên mô hình đã có.
- **55 StreamingLLM:** chính là trường hợp A-shape của MInference (dùng trực tiếp làm baseline, cùng cấu hình 1K global + 4K cục bộ); MInference thêm hai mẫu VS và Block-Sparse để bắt thông tin ngoài sink + cửa sổ.
- **61 Quest:** cặp bù trừ theo giai đoạn: Quest tối ưu **decode** (chọn trang theo query, giữ nguyên KV), MInference tối ưu **prefill** (chọn mẫu theo head, tính thưa một lần cho toàn prompt). Cùng tinh thần "ước lượng rẻ trước, tính đúng sau" nhưng ở hai giai đoạn khác nhau.
- **63 SeerAttention:** thay "tìm mẫu offline bằng ví dụ tham chiếu" bằng một **gate học được**; cùng mục tiêu nhưng cần huấn luyện nhỏ, MInference thì không.
- **64 NSA, 65 MoBA:** huấn luyện sparsity **từ đầu** (native), khác cách "gắn thêm" training-free của MInference.

- **56 H2O:** cùng ý tưởng dùng điểm attention để quyết định phần quan trọng, nhưng H2O ở decode và loại bỏ vĩnh viễn; MInference ở prefill, không loại gì khỏi KV cache, chỉ bỏ qua tính toán.

8. Tóm tắt công thức / khái niệm

Khái niệm	Công thức / ý nghĩa
Mặt nạ thưa	$A(M) = \text{softmax}(QK^\top/\sqrt{d} - c(1-M))$, c lớn (vd. 10^5)
Ba mẫu	A-shape (tĩnh, sink+local); Vertical-Slash (cột + chéo, động); Block-Sparse (khối rời rạc, động nhất)
Tìm mẫu offline	Algorithm 1: không gian tìm kiếm theo FLOPs thật trên kernel, chọn cấu hình gần attention đầy đủ nhất (một lần/mô hình)
Ước lượng VS online	$\hat{A} = \text{softmax}(Q[-\text{last_q}:]K^\top/\sqrt{d})$, last_q=64; top-$k_v$ cột, top-k_s chéo
Ước lượng BS online	Mean-pool Q, K theo block_size=64, nhân để có điểm cấp-khối, chọn top-k_b khối
Kernel	PIT + Triton + FlashAttention, ba kernel riêng theo mẫu
Kết quả chính	Giảm 95% FLOPs attention; tốc độ 1.8×/4.1×/6.8×/10× ở 100K/300K/500K/1M; effective context RULER 32K (LLaMA-3) và 64K (GLM-4), vượt cả full attention báo cáo
Phạm vi	Chỉ tối ưu **prefill**; decode vẫn dense ở mọi baseline kể cả MInference

9. Kết luận ngắn

MInference tách bài toán "attention thưa động" thành hai phần rẻ: chọn **loại mẫu** cho mỗi head một lần ngoại tuyến (dựa trên FLOPs thật và recall so với attention đầy đủ), rồi **ước lượng chỉ số cụ thể** cho mỗi input bằng vài phép tính nhỏ (last_q query cuối, hoặc mean-pooling theo khối) trước khi chạy kernel sparse chuyên biệt. Kết quả là giảm tới 95% FLOPs của attention và tăng tốc prefill tới 10× ở 1M token mà độ chính xác downstream không giảm, tất cả không cần huấn luyện lại.

Link gốc: https://arxiv.org/abs/2407.02490

Bài 63

SeerAttention: Learning Intrinsic Sparse Attention in Your LLMs (2024)

Tác giả: Yizhao Gao, Zhichen Zeng, Dayou Du, Shijie Cao, Peiyuan Zhou, Jiaxing Qi, Junjie Lai, Hayden Kwok-Hay So, Ting Cao, Fan Yang, Mao Yang (University of Hong Kong; University of Washington; Hong Kong University of Science and Technology; Microsoft Research; NVIDIA). **Venue / arXiv:** arXiv:2410.13276 (v4, 17/02/2025); paper không nêu venue hội nghị trong văn bản đã đọc. **Nguồn:** https://arxiv.org/abs/2410.13276 **Chủ đề:** Một gate (AttnGate) nhỏ, học được, đặt lên trên Q/K đã pooling, dự đoán độ thưa attention theo khối (block); huấn luyện bằng tự chưng cất (self-distillation) từ chính attention đầy đủ của mô hình gốc; chỉ áp dụng ở giai đoạn prefill trong bản này. **Vị trí trong lộ trình:** Đọc sau 01 (Attention), 12 (RoPE), 17 (Longformer), 27 (Switch Transformer, nguồn cảm hứng kiểu gate của MoE), 44/45 (FlashAttention), 55 (StreamingLLM, một thành phần của baseline DuoAttention ở đây), 61 (Quest), 62 (MInference, baseline so sánh trực tiếp). SeerAttention thay bước "tìm mẫu offline bằng ví dụ tham chiếu" của bài 62 bằng một module học được. Đọc tiếp 64 (NSA), 65 (MoBA): huấn luyện sparsity từ đầu, khác cách "gắn gate lên mô hình đã pretrain" ở đây.

> Tài liệu học tập bằng tiếng Việt, là ghi chú diễn giải, không phải bản dịch. Số liệu lấy từ paper (full text arXiv, tải ngày 07/11/2026); các phép tính đánh dấu "tự tính" là của người ghi chú. Mục "Ứng dụng ..." là suy luận.

1. Bối cảnh

Ma trận attention có độ phức tạp $O(n^2)$, nhưng ở một số head, phần lớn trọng số có thể rất nhỏ: bài báo nêu mức thưa 95–99%, thay đổi theo mô hình, đầu vào và head. Longformer và BigBird dùng mẫu cố định; MoA và MInference dùng heuristic tìm kiếm ngoại tuyến cho từng head. Từ sự phụ thuộc vào đầu vào, các tác giả liên hệ với cơ chế cổng trong Mixture-of-Experts (MoE, bài 27) và đề xuất học cách chọn phần attention cần tính.

Khái niệm cần biết trước: *block-sparse attention* (chia ma trận attention thành lưới khối $B \times B$, chỉ tính khối quan trọng, tương thích tiling của FlashAttention); *self-distillation* (ở đây là dùng **chính attention đầy đủ của mô hình gốc** làm nhãn, không phải chưng cất từ mô hình thầy lớn hơn như ở bài 28/29); *gate* (mạng nhỏ học "mở/đóng" phần tính toán, mượn từ MoE).

2. Ý tưởng chính

SeerAttention thêm vào attention chuẩn một module nhỏ gọi là **AttnGate**: pooling Q, K theo trục chuỗi, qua một lớp linear, nhân với nhau để ra **điểm gate theo khối** (mỗi khối $B \times B$ của ma trận attention một điểm), từ đó suy ra mặt nạ block-sparse nhị phân. Thay vì tự thiết kế mẫu, AttnGate được **huấn luyện bằng tự chưng cất**: nhãn (ground truth) là kết quả **2D-MaxPooling** của chính ma trận attention đầy đủ của mô hình (đã đóng băng), loss là KL-divergence. Vì chỉ huấn luyện tham số gate (vài triệu, so với hàng tỉ của backbone), quá trình rất nhẹ: với Llama-3.1-8B-Instruct, chỉ cần khoảng 0,5 tỉ token và ~40 giờ A100. Lúc suy luận, điểm gate được chuyển thành mặt nạ khối nhị phân (TopK hoặc ngưỡng), đưa vào kernel **Block-Sparse FlashAttention** chuyên biệt để bỏ qua I/O và tính toán ở khối không quan trọng. Trong bản paper này, AttnGate **chỉ áp dụng ở giai đoạn prefill**; áp dụng cho decode được paper liệt là hướng tương lai, chưa làm.

3. Phương pháp chi tiết

3.1. Thiết kế AttnGate (§3.1, Hình 1)

Với một head attention, $Q, K \in \mathbb{R}^{n \times d}$. AttnGate pooling Q, K theo trục chuỗi với kích thước khối (= bước trượt, không chồng lấn) B, được $[n/B, d]$; qua lớp linear W_q, W_k rồi nhân với nhau giống attention chuẩn:

$$\boxed{\text{score} = \text{softmax}\left(\frac{(W_q P_q(Q))(W_k P_k(K))^\top}{\sqrt{d}}\right)}$$

với P_q, P_k là phép pooling của Q, K. Kết quả có kích thước $[n/B,\ n/B]$, mỗi phần tử tương ứng một **khối** của ma trận attention đầy đủ. Với $B = 64$ (giá trị dùng trong thí nghiệm), kích thước đầu ra chỉ bằng $1/B^2 = 1/4096$ ma trận gốc, nên chi phí tính gate rất rẻ.

Chọn phép pooling (Hình 2). Paper cho phép tổ hợp nhiều kiểu pooling (avg, max, min) cho Q và K riêng (nối theo chiều ẩn trước khi vào linear nếu dùng nhiều kiểu). Quét 15 tổ hợp tốt nhất trên PG19 với Llama-3.1-8B, kết quả tốt nhất: **avg-pooling cho Q**, kết hợp **max + min + avg cho K**. Paper suy đoán: hiện tượng này liên quan tới việc K thường có nhiều outlier hơn (như trong lượng tử hóa LLM), nên pooling max/min giữ lại thông tin outlier tốt hơn avg đơn thuần.

3.2. RoPE theo khối (Block-level RoPE, §3.1, Hình 3-4)

Nếu đưa Q, K **đã áp RoPE** (bài 12) vào AttnGate rồi mới pooling, tính chất mã hóa vị trí tương đối bị phá vỡ bởi phép pooling, khiến gate không tổng quát hóa được sang độ dài chưa thấy khi huấn luyện (Hình 4: gate huấn luyện ở 8K thất bại khi đánh giá trên 64K-128K; huấn luyện ở 64K thất bại ở 128K). Giải pháp: đưa Q, K **trước khi áp RoPE** vào AttnGate, pooling, qua linear, rồi mới áp một **RoPE riêng ở cấp khối** với $\theta' = \theta/B$ (θ là theta RoPE gốc của mô hình). Với thiết kế này, AttnGate ngoại suy tốt sang độ dài dài hơn lúc huấn luyện (Hình 4 cho thấy perplexity ổn định qua 8K-128K).

3.3. Tự chưng cất AttnGate (§3.2)

Nhãn (ground truth). Lấy **2D-MaxPooling** của attention đầy đủ (đã softmax) theo từng khối $B \times B$:

$$\boxed{gt = \text{MaxPool2D}\Big(\text{softmax}(QK^\top/\sqrt{d})\Big)}$$

Lý do dùng **max** chứ không phải trung bình: một khối chỉ nên bị coi là "thưa, bỏ được" khi **toàn bộ** phần tử trong khối đều nhỏ; lấy max đúng với định nghĩa block-sparse (bỏ khối = chấp nhận bỏ cả phần tử lớn nhất trong khối đó, nên phải chắc chắn phần tử lớn nhất cũng nhỏ). Việc này không tầm thường về mặt hiện thực: các mô hình hiện đại dùng FlashAttention, không bao giờ vật chất hóa ma trận attention đầy đủ $O(n^2)$ (quá tốn bộ nhớ ở ngữ cảnh dài). Paper giải quyết bằng cách **tùy biến một kernel FlashAttention** để xuất trực tiếp kết quả 2D-max-pooled mà vẫn tái dùng luồng tính toán gốc của FlashAttention (chi tiết ở Appendix A.1 của paper, không đọc sâu trong ghi chú này).

Loss. Dùng KL-divergence (không dùng MSE):

$$\boxed{\text{score} = \text{AttnGate}(Q, K), \qquad \mathcal{L} = D_{KL}(gt \,\|\, \text{score})}$$

Vì AttnGate cũng dùng softmax ở đầu ra (mỗi hàng tổng bằng 1, giống gt), KL-divergence giúp huấn luyện tập trung khớp **phân phối** attention, không phải khớp độ lớn tuyệt đối như MSE.

Chế độ huấn luyện. Toàn bộ tham số backbone (mô hình gốc) được **đóng băng**; chỉ tham số của AttnGate (các lớp linear nhỏ) được cập nhật gradient. Vì vậy quá trình rất nhẹ so với huấn luyện lại cả mô hình hoặc huấn luyện một gate kiểu MoE từ đầu (nơi gate phải tự học cách định tuyến expert mà không có nhãn có sẵn).

3.4. Suy luận: từ điểm gate sang mặt nạ nhị phân và kernel (§3.3)

Điểm gate (số thực, dạng xác suất) được chuyển sang mặt nạ nhị phân b_{ij} bằng một trong hai cách:

$$b_{ij} = \begin{cases} 1 & j \in \text{TopK}(\text{score}_i,\, k) \\ 0 & \text{khác} \end{cases} \qquad \text{hoặc} \qquad b = \text{score} > \text{threshold}$$

Ưu điểm: sau khi huấn luyện xong một AttnGate, người dùng có thể **đổi tỉ lệ TopK hoặc ngưỡng ngay lúc suy luận** (không cần huấn luyện lại) để đánh đổi độ chính xác và tốc độ. Mặt nạ này được đưa vào **Block-Sparse FlashAttention kernel**, với kích thước khối khớp với tiling của FlashAttention (64 hoặc 128), bỏ qua hoàn toàn I/O và tính toán ở khối không được chọn. Lưu ý quan trọng (ghi rõ trong phần thí nghiệm): "AttnGate solely applies in the prefill stage" - **trong bản paper này, AttnGate chỉ hoạt động ở prefill**, decode vẫn dùng attention đầy đủ.

4. Thí nghiệm và kết quả chính

Mô hình: Llama-3.1-8B-Instruct. **Block size** $B = 64$ cho cả AttnGate và sparse kernel. **Huấn luyện gate:** dữ liệu RedPajama, chia khúc 64K token (có BOS/EOS); learning rate 10^{-3} với cosine decay; global batch 16; chỉ **500 bước**; DeepSpeed stage 2 trên GPU A100; tổng cộng ~40 giờ A100 (0,5 tỉ token) vì chỉ gate được cập nhật.

Baseline (đều training-free, không học gì): MoA (tìm mẫu tĩnh ngoại tuyến theo head; dùng cấu hình "KV sparsity" 0.5, tương ứng "attention sparsity" 0.35), MInference (bài 62, mọi head Llama-3.1-8B dùng cấu hình chính thức "Vertical-Slash"), DuoAttention (50% head "streaming" kiểu StreamingLLM/bài 55, 50% head dense; vì vậy "sparsity thật < 50%"). Tất cả chạy trên một A100.

LongBench (Bảng 1 của paper, Llama-3.1-8B-Instruct, độ chính xác trung bình theo nhóm độ dài):

Phương pháp	0-4K	4-8K	8K+	Avg. Acc.	Avg. Sparsity
Full Attention	55.32	53.98	52.9	54.07	0.0
MInference	55.23	53.78	52.18	53.73	0.31
MoA	50.74	49.84	51.89	50.82	0.35
DuoAttention	53.77	52.17	51.27	52.40	0.5*
SeerAttention	**55.43**	**54.49**	52.69	**54.20**	**0.50**

(*DuoAttention: 50% head streaming, sparsity thật nhỏ hơn 50%.) SeerAttention đạt độ chính xác trung bình **cao nhất** và sparsity trung bình **cao nhất** cùng lúc; ở hai nhóm 0-4K và 4-8K, SeerAttention còn **vượt cả full attention** (paper suy đoán: gate lọc bớt attention "nhiễu").

RULER (Bảng 2, 13 tác vụ, 4K-128K; MoA bị loại vì OOM):

Phương pháp	4K	8K	16K	32K	64K	128K	Avg. Acc.	Avg. Speedup
Full Attention	95.53	92.37	92.01	87.63	84.39	76.26	88.01	1.00
MInference	95.53	92.64	91.37	85.71	83.24	67.02	85.92	0.83
DuoAttention	95.64	92.08	90.71	84.75	83.24	75.32	86.96	1.09
SeerAttention	95.53	92.71	92.02	88.49	83.48	73.37	**87.60**	**1.41**

SeerAttention dùng một **ngưỡng cố định** 5×10^{-4} cho mọi độ dài, nhưng tự động thích nghi sparsity từ ~10% (ở 4K) lên ~85% (ở 128K) - do ngưỡng cố định trên điểm gate, còn tỉ lệ khối vượt ngưỡng tùy thuộc phân phối

attention thật ở từng độ dài. Kết quả: độ chính xác trung bình chỉ thấp hơn full attention 0,41 điểm, nhưng tốc độ trung bình **cao nhất (1.41×)**; đáng chú ý MInference **chậm hơn** full attention (0.83×, tức tốc độ < 1) do chi phí runtime tìm chỉ số thưa lớn ở các độ dài ngắn hơn 64K.

Tác vụ ngữ cảnh ngắn (Bảng 3, với ngưỡng rất cao 3×10^{-2} để ép sparsity cao):

	MMLU	HellaSwag	ARC-c	GSM-8K
Full Attention	68.1	80.1	60.7	75.7
SeerAttention	67.9	79.8	60.2	75.6
Sparsity trung bình (%)	3.4	50.4	26	52.1
Độ dài trung bình (token)	118	840	395	872

Độ chính xác giảm không đáng kể dù sparsity cao (ví dụ GSM-8K: giảm 0,1 điểm với 52,1% sparsity, độ dài trung bình chỉ 872 token). Paper tự nhận định trực tiếp: ***"với input ngữ cảnh ngắn, attention chỉ chiếm một phần nhỏ trong tổng thời gian chạy, nên sparse attention không cải thiện đáng kể độ trễ"*** *ở* nhóm tác vụ này; Bảng 3 chỉ đo xem độ chính xác có bị ảnh hưởng không, không đo tốc độ.

Hiệu quả kernel (Hình 6, A100). Chi phí chạy AttnGate gần như không đáng kể: ở 32K với sparsity 0,5, AttnGate chỉ chiếm ~1% tổng độ trễ một lớp attention; ở 128K chi phí này gần như biến mất (tỉ trọng còn nhỏ hơn). Kernel Block-Sparse FlashAttention cho tốc độ tăng gần tuyến tính theo sparsity; ở 128K với 90% sparsity, SeerAttention đạt **7.3×** so với FlashAttention-2 (số rõ nhất nêu trong văn bản và trích dẫn lại ở tóm tắt).

So kernel với MoA/MInference (Hình 7). Ở cùng mức sparsity, kernel block-sparse của SeerAttention cho tốc độ cao hơn cả hai baseline ở hầu hết trường hợp (8K/32K/128K); các giá trị cụ thể trên đồ thị không trích được chính xác từ bản text.

Tốc độ end-to-end trên RULER (Hình 9). SeerAttention đạt tốc độ prefill tới **2.43×** ở 128K (số cao nhất); MInference **chậm hơn** full attention (speedup < 1) ở độ dài dưới 64K vì chi phí tìm chỉ số lúc runtime; trung bình chung trên toàn bộ RULER, SeerAttention đạt tốc độ cao nhất (1.41×, khớp cột "Avg. Speedup" ở Bảng 2) với độ chính xác cao nhất trong các phương pháp thưa.

Trực quan hóa (Hình 8). AttnGate tự học được nhiều kiểu mẫu đa dạng mà không cần biết trước: A-shape/streaming head, đường dọc, đường chéo (có khoảng trống), khối thưa theo đường chéo chính, và cả mẫu "ngẫu nhiên" - vượt khỏi tập mẫu cố định mà MoA/MInference giả định trước (A-shape, Vertical-Slash, Block-Sparse).

5. Phân tích / ablation

- **Chọn pooling (Hình 2):** avg cho Q + (max, min, avg) cho K cho perplexity thấp nhất ở mọi mức sparsity trên PG19; paper gắn với quan sát K hay có outlier hơn Q.
- **RoPE theo khối (Hình 3, 4):** không có nó, AttnGate quá khớp (overfit) vào đúng độ dài huấn luyện và thất bại hoàn toàn khi đánh giá ở độ dài khác (huấn luyện 8K, đánh giá > 8K: perplexity tăng vọt; huấn luyện 64K, đánh giá 128K: cũng vậy); có RoPE theo khối ($\theta' = \theta/B$), AttnGate ngoại suy tốt qua 8K-128K.
- **Ngữ cảnh ngắn (Bảng 3):** cho thấy sparse attention không cải thiện đáng kể độ trễ trong phép đo với chuỗi 118–872 token, khi attention chỉ chiếm một phần nhỏ tổng thời gian chạy.
- **Ngưỡng cố định tự thích nghi theo độ dài (RULER):** cùng một ngưỡng số cho mọi độ dài vẫn tạo ra sparsity tăng dần theo độ dài (10% ở 4K lên 85% ở 128K) vì phân phối điểm gate tự nhiên lệch nhiều hơn khi ngữ cảnh dài hơn - một lợi ích "miễn phí" của thiết kế ngưỡng so với phải đặt tỉ lệ TopK cố định.

6. Giới hạn

Do tác giả nêu:

- AttnGate trong bản này **chỉ áp dụng ở prefill**; áp dụng cho **decode** (đặc biệt cho long-CoT, suy luận nhiều bước) được liệt rõ là hướng nghiên cứu tương lai, chưa thực hiện trong paper.
- Tiếp tục tiền huấn luyện (continued pre-training) với nhiều token hơn để đạt sparsity cao hơn mà không giảm độ chính xác mới chỉ có thí nghiệm "sơ bộ" ở Appendix A.2 (paper không đưa số liệu đầy đủ trong thân bài).
- Cần kernel FlashAttention tùy biến riêng để tính nhãn 2D-max-pooled hiệu quả (không thể lấy trực tiếp từ FlashAttention chuẩn).

Nhận xét:

- Toàn bộ thí nghiệm chính chỉ trên **một backbone** (Llama-3.1-8B-Instruct); chưa rõ gate tổng quát hóa thế nào sang kiến trúc khác (mô hình nhỏ, decoder lai) hay quy mô khác.
- "Self-distillation" ở đây có nghĩa riêng: nhãn lấy từ chính attention đầy đủ của mô hình đang huấn luyện gate, không phải mô hình thầy độc lập lớn hơn như bài 28/29; vẫn cần tính attention đầy đủ (qua kernel tùy biến) cho dữ liệu huấn luyện, tức có chi phí "một lần" không hoàn toàn miễn phí.
- Ngưỡng/tỉ lệ TopK được **tinh chỉnh riêng cho mỗi benchmark** (2×10^{-3} LongBench, 5×10^{-4} RULER, 3×10^{-2} tác vụ ngắn); chưa có cấu hình mặc định chung.
- Các số đo tốc độ chủ yếu ở cấp kernel/prefill cô lập; chưa đo trong pipeline phục vụ đầy đủ kết hợp cả prefill và decode (vì decode vẫn dense).

7. Liên hệ với các bài khác

- **27 Switch Transformer (MoE):** nguồn cảm hứng trực tiếp cho thiết kế gate (paper nói rõ "inspired by the gating mechanism in MoE"); khác biệt: gate SeerAttention có nhãn sẵn để chưng cất, gate MoE phải tự học định tuyến từ đầu.
- **44/45 FlashAttention:** nền kernel; paper tùy biến thêm kernel sinh nhãn 2D-max-pooled và kernel Block-Sparse FlashAttention cho suy luận.
- **12 RoPE:** nền tảng hiểu vì sao pooling trước RoPE phá vỡ tính tương đối vị trí, dẫn tới "RoPE theo khối" với θ/B.
- **17 Longformer, 60 BigBird:** đại diện hướng "mẫu cố định đặt trước", đối lập cách "học mẫu" của SeerAttention.
- **55 StreamingLLM:** thành phần "streaming head" trong baseline DuoAttention; cũng là khái niệm A-shape mà AttnGate tự học lại được (Hình 8a) không cần ai dạy.
- **61 Quest:** giải quyết decode bằng chọn trang theo query, đúng vào khoảng trống SeerAttention còn để ở phần tương lai; có thể là một hướng khả thi để lấp khoảng trống đó, dù hai cơ chế khác nhau.
- **62 MInference:** baseline so sánh trực tiếp, cùng mục tiêu (tối ưu prefill); khác biệt cốt lõi: MInference tìm kiếm ngoại tuyến bằng heuristic (Algorithm 1 của bài 62), SeerAttention học bằng gradient từ nhãn chưng cất; RULER (Bảng 2) cho thấy MInference có thể chậm hơn full attention ở độ dài vừa do chi phí runtime, còn AttnGate (đã huấn luyện trước) rẻ hơn lúc suy luận.
- **64 NSA, 65 MoBA:** huấn luyện sparsity từ đầu (native, đồng thời với backbone); SeerAttention khác biệt rõ: gắn thêm gate nhỏ lên mô hình đã pretrain xong và đóng băng, chỉ huấn luyện phần gate.
- **08 LoRA:** cùng tinh thần "thêm module nhỏ, học được, lên trên mô hình đóng băng" với chi phí huấn luyện thấp, dù LoRA nhắm thích nghi tác vụ còn AttnGate nhắm dự đoán sparsity.

8. Tóm tắt công thức / khái niệm

Khái niệm	Công thức / ý nghĩa
AttnGate	score $=$ softmax$\big((W_q P_q(Q))(W_k P_k(K))^\top/\sqrt{d}\big)$, kích thước $[n/B, n/B]$
Pooling tốt nhất	avg cho Q; (max, min, avg) nối cho K
RoPE theo khối	Pooling trước RoPE, rồi áp RoPE riêng với $\theta' = \theta/B$ sau linear layer
Nhãn tự chưng cất	$gt =$ MaxPool2D(softmax$(QK^\top/\sqrt{d})$) theo khối $B \times B$
Loss	$D_{KL}(gt \,\|\, \text{score})$, chỉ cập nhật tham số gate, backbone đóng băng
Mặt nạ nhị phân	TopK theo hàng, hoặc ngưỡng score $>$ threshold
Chi phí huấn luyện (Llama-3.1-8B)	500 bước, batch 16, ~0,5 tỉ token, ~40 giờ A100
Phạm vi trong bản này	Chỉ prefill; decode và continued-pretraining ở sparsity cao là hướng tương lai
Kết quả chính	LongBench: acc cao nhất + sparsity cao nhất (0.50); RULER: acc 87.60 (speedup 1.41×, cao nhất); kernel 7.3× ở 128K/90% sparsity
Ngữ cảnh ngắn (Bảng 3)	Sparsity cao (26-52%) nhưng giảm acc tối thiểu; paper tự nhận không cải thiện tốc độ ở độ dài ngắn

9. Kết luận ngắn

SeerAttention thay bước "đoán mẫu sparse bằng tay hoặc heuristic" bằng một gate nhỏ học được ngay trên Q/K đã pooling, huấn luyện rẻ bằng tự chưng cất từ 2D-max-pooled của chính attention đầy đủ mô hình, nên tự thích nghi độ thưa theo input và theo độ dài mà không cần ai thiết kế mẫu trước. Trong bản paper này, kỹ thuật chỉ áp dụng ở prefill, đạt độ chính xác tốt hơn các baseline training-free (MoA, MInference, DuoAttention) ở cùng hoặc sparsity cao hơn, với tốc độ kernel tới 7.3× ở 128K/90% sparsity.

Link gốc: https://arxiv.org/abs/2410.13276

Bài 64

Native Sparse Attention: Hardware-Aligned and Natively Trainable Sparse Attention (2025)

Tác giả: Jingyang Yuan và 14 đồng tác giả (DeepSeek-AI; Đại học Bắc Kinh; Đại học Washington).
Venue / arXiv: arXiv:2502.11089 (v2, 27/02/2025); paper không nêu venue hội nghị.
Nguồn: https://arxiv.org/abs/2502.11089
Chủ đề: Sparse attention "native" (học trực tiếp trong lúc huấn luyện, không phải thủ thuật chỉ áp lúc suy luận), kiến trúc ba nhánh song song (nén token, chọn block, sliding window) gộp bằng cổng học được, kernel Triton gắn chặt với GQA để đạt tốc độ thật trên phần cứng.
Vị trí trong lộ trình: Đọc trước: 01 (Attention), 15 (GQA), 17 (Longformer, sliding window + global tĩnh), 44/45 (FlashAttention, kernel nền). Đây là bài mở đầu "Giai đoạn 8: Sparse attention" của lộ trình, bàn hướng **cắt bớt cặp (query, key) ngay từ lúc huấn luyện** thay vì chỉ cắt KV-cache lúc suy luận (55 StreamingLLM, 56 H2O) hay cắt theo chiều đặc trưng (16 MLA). Đọc tiếp: 65 (MoBA, cách chọn block khác: mean-pooling thay vì điểm nén), 67 (Gated attention của Qwen, phân tích sâu hơn vai trò của cổng mà NSA dùng khá sơ sài), 68 (DeepSeek-V3.2, có khả năng là hậu duệ trực tiếp của NSA trong cùng nhóm DeepSeek).

> Tài liệu học tập bằng tiếng Việt, là ghi chú diễn giải, không phải bản dịch. Số liệu lấy từ paper (full text arXiv, tải ngày 07/11/2026); các phép tính đánh dấu "tự tính" là của người ghi chú.

1. Bối cảnh

Attention chuẩn (bài 01) tính điểm cho mọi cặp query–key, với chi phí $O(n^2)$ theo độ dài chuỗi n. Bài báo dẫn một ước tính từ công trình khác rằng attention chiếm 70–80% tổng độ trễ khi decode chuỗi 64K; đây không phải số đo riêng của NSA. Một hướng giảm chi phí là khai thác tính thưa của softmax attention và chỉ tính các cặp có vai trò quan trọng.

Hai lỗ hổng của các phương pháp sparse attention trước đó (paper tự phân loại). *"Ảo tưởng về suy luận hiệu quả"*: H2O chỉ thưa ở decode, MInference chỉ thưa ở prefill, không ai thưa ở cả hai giai đoạn nên tốc độ thực vẫn kẹt ở giai đoạn còn "đặc"; và không hợp với GQA/MQA, vì ở phương pháp như Quest mỗi head tự chọn KV riêng, nên với GQA (head trong nhóm chia sẻ KV-cache) bộ nhớ cần tải là **hợp (union)** mọi lựa chọn trong nhóm, tức dù phép tính giảm, KV phải tải vẫn gần đầy đủ. *"Huyền thoại về tính thưa có thể học được"*: áp sparsity post-hoc làm mô hình lệch quỹ đạo tối ưu (top 20% điểm attention chỉ phủ 70% tổng điểm theo một quan sát được trích dẫn, nên "retrieval head" dễ hỏng khi bị cắt lúc suy luận); thêm hai vấn đề kỹ thuật: thành phần rời rạc (k-means ở ClusterKV, SimHash ở MagicPIG) chặn gradient, và chọn theo token riêng lẻ (HashAttention) làm bộ nhớ không liên tục, không tận dụng được kernel kiểu FlashAttention nên backward chạy ở hiệu suất thấp.

Khái niệm cần biết trước. Arithmetic intensity: tỉ lệ số phép tính / số lượt truy cập bộ nhớ. Mỗi GPU có một "ngưỡng" (= FLOPS đỉnh / băng thông); trên ngưỡng là *compute-bound*, dưới ngưỡng là *memory-bound*. Huấn luyện/prefill (nhiều token song song) có arithmetic intensity cao nên compute-bound; decode (1 token/lượt nhưng tải toàn bộ KV-cache) có arithmetic intensity thấp nên memory-bound. Hai mục tiêu khác nhau: huấn luyện/prefill cần giảm **số phép tính**, decode cần giảm **lượng bộ nhớ phải tải**.

2. Ý tưởng chính

NSA thay cặp key-value gốc $k_{:t}, v_{:t}$ bằng một tập key-value "cô đọc" hơn, được xây động theo từng query: $\tilde{K}_t = f_K(q_t, k_{:t}, v_{:t})$, $\tilde{V}_t = f_V(\cdot)$, rồi tính $o_t^* = \text{Attn}(q_t, \tilde{K}_t, \tilde{V}_t)$. NSA định nghĩa ba chiến lược remapping song song, chạy đồng thời rồi gộp bằng cổng học được:

- **Nén (compression):** gộp từng khối token liên tiếp thành một vector đại diện, để nhìn bối cảnh xa với chi phí thấp (bỏ sót chi tiết mịn).
- **Chọn (selection):** dùng chính điểm attention của nhánh nén để suy ra block nào quan trọng, rồi giữ *nguyên* (không nén) các token trong block đó, theo đơn vị block liên tục (để phần cứng đọc nhanh).
- **Cửa sổ trượt (sliding window):** luôn giữ w token gần nhất, tách biệt khỏi hai nhánh trên để bối cảnh cục bộ (học nhanh, dễ chiếm ưu thế) không "cướp" gradient của hai nhánh còn lại.

Mọi phép (chọn block, nén, gate) đều khả vi, nên huấn luyện end-to-end từ đầu, không có bước rời rạc không-gradient. Đồng thời, nhánh chọn được cài đặt bằng kernel Triton thiết kế riêng, khai thác việc các head trong cùng nhóm GQA chia sẻ lựa chọn block, nên tốc độ lý thuyết chuyển thành tốc độ thực trên GPU.

3. Phương pháp chi tiết: ba nhánh và cổng

3.1 Khung tổng quát

Attention chuẩn (nhắc lại, Eq. 1-2 của paper): $o_t = \text{Attn}(q_t, k_{:t}, v_{:t})$, $\text{Attn}(q, K, V)_t = \sum_i \alpha_{t,i} v_i / \sum_j \alpha_{t,j}$, $\alpha_{t,i} = \exp(q_t^\top k_i / \sqrt{d_k})$.

NSA định nghĩa tập nhánh $\mathcal{C} = \{\text{cmp}, \text{slc}, \text{win}\}$ (nén, chọn, cửa sổ) và với mỗi nhánh c có $\tilde{K}_t^c, \tilde{V}_t^c$ riêng, rồi gộp bằng tổng có trọng số:

$$\boxed{o_t^* = \sum_{c \in \mathcal{C}} g_t^c \cdot \text{Attn}(q_t, \tilde{K}_t^c, \tilde{V}_t^c)}$$

$g_t^c \in [0, 1]$ là điểm cổng cho nhánh c, "lấy từ các đặc trưng đầu vào qua một MLP và hàm sigmoid" (paper không nêu thêm chi tiết kiến trúc MLP này, ví dụ có head-specific hay không). Gọi $N_t = \sum_c |\tilde{K}_t^c|$ là tổng số key/value "hiệu dụng"; NSA giữ $N_t \ll t$ để đảm bảo độ thưa cao.

3.2 Nhánh nén (token compression)

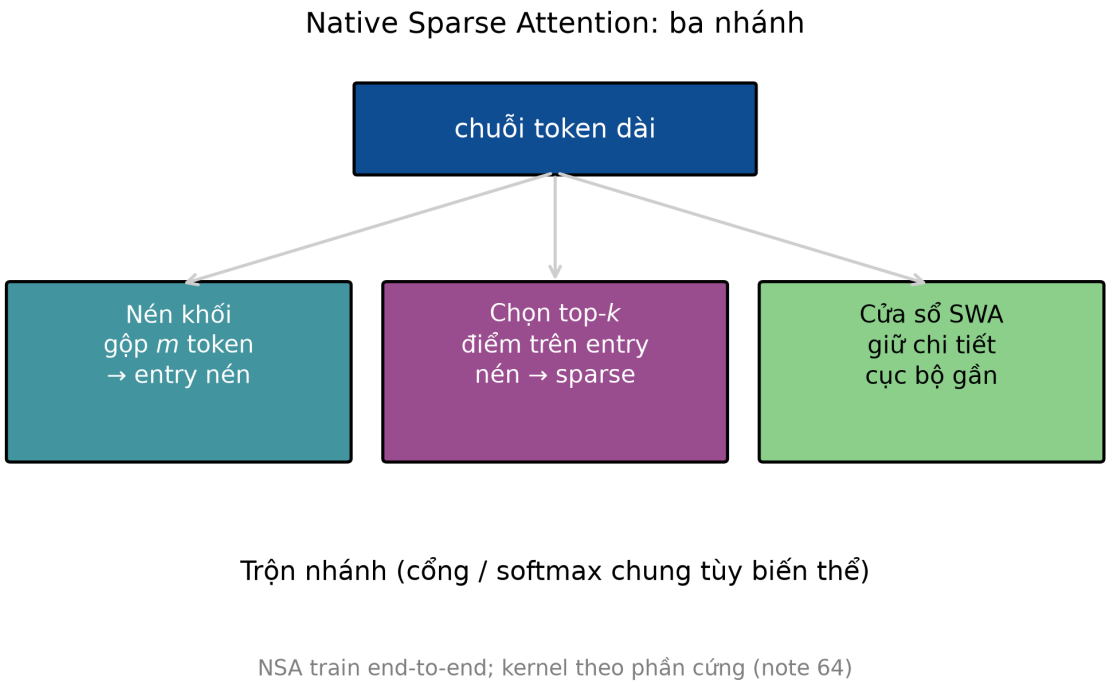
Gộp từng khối l key liên tiếp (chồng lấn, bước trượt $d_{\text{strd}} < l$; paper dùng ký hiệu d cho bước trượt, **khác** d_k ở trên) thành một key nén qua MLP học được φ có mã hoá vị trí trong khối:

$$\tilde{K}_t^{\text{cmp}} = \left\{ \varphi(k_{id_{\text{strd}}+1:id_{\text{strd}}+l}) \;\middle|\; 0 \le i \le \left\lfloor \frac{t-l}{d_{\text{strd}}} \right\rfloor \right\}$$

Số key nén tới thời điểm t là $\lfloor (t-l)/d_{\text{strd}} \rfloor + 1$. Công thức tương tự cho $\tilde{V}_t^{\text{cmp}}$ (dùng chung chỉ số khối). Vì $d_{\text{strd}} < l$ nên các khối chồng lấn, giảm hiện tượng "gãy thông tin" ở biên khối.

3.3 Nhánh chọn (token selection)

Chia khối chọn. Chia lại chuỗi thành các "khối chọn" kích thước l' (có thể khác l), paper yêu cầu $l \le l'$, $d_{\text{strd}} \mid l$, $d_{\text{strd}} \mid l'$.



NSA: ba nhánh nén khối, chọn top-k, cửa sổ SWA. (minh hoạ, tự tính)

Điểm quan trọng của khối, tận dụng attention của nhánh nén (không tính thêm attention riêng):

$$p_t^{\text{cmp}} = \text{Softmax}(q_t^\top \tilde{K}_t^{\text{cmp}})$$

Nếu $l' = l = d_{\text{strd}}$ thì $p_t^{\text{slc}} = p_t^{\text{cmp}}$ trực tiếp; khi khác, suy ra điểm khối chọn bằng tổng các điểm nén chồng lấn vào khối đó (Eq. 9). Với **GQA/MQA**, các head trong một nhóm chia sẻ KV-cache nên phải chọn cùng block; paper gộp điểm quan trọng bằng tổng trên H head cùng nhóm:

$$p_t^{\text{slc}\prime} = \sum_{h=1}^{H} p_t^{\text{slc},(h)}$$

Chọn top-n khối theo điểm đã gộp:

$$\mathcal{I}_t = \{\, i \mid \text{rank}(p_t^{\text{slc}\prime}[i]) \le n \,\}, \qquad \tilde{K}_t^{\text{slc}} = \text{Cat}\{k_{il'+1:(i+1)l'} \mid i \in \mathcal{I}_t\}$$

(tương tự cho $\tilde{V}_t^{\text{slc}}$). Việc chọn theo **block liên tục** (không chọn từng token rời) là chủ ý: phần cứng đọc block liên tục nhanh hơn đọc rải rác, và quan sát thực nghiệm của paper (mục 6.2) cho thấy điểm attention có xu hướng "cụm theo không gian" (các key gần nhau có điểm tương tự), nên chọn theo khối không mất nhiều so với chọn từng token.

3.4 Nhánh cửa sổ trượt (sliding window)

Giữ K, V của w token gần nhất: $\tilde{K}_t^{\text{win}} = k_{t-w:t}$, $\tilde{V}_t^{\text{win}} = v_{t-w:t}$. Lý do tách riêng (không gộp vào nhánh chọn): pattern cục bộ "học nhanh" và dễ chiếm ưu thế gradient, nếu không tách, hai nhánh nén/chọn có thể bị "đi tắt" (shortcut) qua nhánh cục bộ mà không học được pattern của chính nó. Ba nhánh dùng **ba bộ phép chiếu Q,K,V độc lập**, thêm một chút tham số nhưng ngăn giao thoa gradient giữa pattern gần và pattern xa.

3.5 Siêu tham số dùng trong paper

$l = 32$ (độ dài khối nén), $d_{\text{strd}} = 16$ (bước trượt nén), $l' = 64$ (độ dài khối chọn), $n = 16$ (số khối được chọn, **đã gồm** 1 khối đầu tiên và 2 khối gần nhất được **ép chọn cố định**, phần còn lại 13 khối chọn theo điểm quan trọng), $w = 512$ (cửa sổ trượt).

4. Thiết kế kernel: GQA-aware và arithmetic intensity

Nhánh nén và cửa sổ tương thích trực tiếp với kernel FlashAttention-2. Nhánh chọn cần kernel riêng: nếu nạp "query liên tục theo thời gian" kiểu FlashAttention, các query trong cùng khối có thể cần các khối KV không liên quan nhau (mỗi query chọn khối khác nhau), gây truy cập bộ nhớ rời rạc.

Giải pháp: nạp theo nhóm GQA. Vì các head cùng nhóm đã bị buộc chọn cùng khối (mục 3.3), kernel nạp tất cả head trong một nhóm tại vị trí t cùng lúc vào SRAM, cùng chỉ số khối $\mathcal{I}_t$ chung. Ba đặc điểm (Hình 3 của paper): **Group-Centric Data Loading** (nạp $Q \in \mathbb{R}^{[h,d_k]}$ của cả h head trong nhóm), **Shared KV Fetching** ($K \in \mathbb{R}^{[B_k,d_k]}, V \in \mathbb{R}^{[B_k,d_v]}$ theo $\mathcal{I}_t$, $B_k \mid l'$, tối thiểu lượt nạp), **Outer Loop on Grid** (vòng lặp query/output đưa ra grid scheduler của Triton, vì độ dài vòng lặp trong gần như đồng đều giữa các khối query):

```
// Diễn giải pseudo-code kernel (không phải code gốc)
for mỗi khối query thuộc grid (song song trên GPU):
    nạp Q[toàn bộ head trong nhóm GQA] tại vị trí t vào SRAM
    nạp chỉ số khối I_t (chung cho cả nhóm) đã tính từ bước gating
    for mỗi khối KV được chỉ định trong I_t:         // inner loop
        nạp K_block, V_block (kích thước B_k, B_k | l') vào SRAM
        tính điểm attention cục bộ trên SRAM, cộng dồn online-softmax
    ghi output ra HBM
```

Hiệu quả đạt gần tối ưu arithmetic intensity nhờ: (1) loại bỏ truyền KV dư thừa qua chia sẻ theo nhóm, (2) cân bằng tải giữa các streaming multiprocessor. MHA (mỗi head một KV riêng) vốn đã "memory-intensive và kém hiệu quả cho decode" (paper), nên NSA chỉ nhắm kiến trúc có KV chia sẻ (GQA/MQA).

5. Thí nghiệm và kết quả chính

Backbone pretraining: 27B tham số tổng, 3B kích hoạt (GQA + MoE kiểu DeepSeekMoE), 30 layer, $d_{\text{model}} = 2560$, GQA 4 nhóm, 64 head tổng (16 head/nhóm), $d_q = d_k = 192$, $d_v = 128$. MoE: 72 expert định tuyến + 2 expert chia sẻ, top-6; layer đầu dùng MLP SwiGLU thường (không MoE) để ổn định. NSA: $l = 32, d_{\text{strd}} = 16, l' = 64, n = 16, w = 512$. Cả Full Attention và NSA được tiền huấn luyện **"270B token"** chuỗi 8k (mục 4.1), tiếp tục huấn luyện + SFT ở 32k bằng YaRN để thích nghi ngữ cảnh dài. (Lưu ý: phần giới thiệu của paper lại ghi **"260B token"** cho cùng mô hình; paper nêu hai số khác nhau, ghi chú này trích cả hai, không tự chọn một số.)

Bảng 1 (benchmark tổng quát, càng cao càng tốt):

Model	MMLU	MMLU-PRO	CMMLU	BBH	GSM8K	MATH	DROP	MBPP	HumanEval	Avg
Full Attn	0.567	0.279	0.576	0.497	0.486	0.263	0.503	0.482	0.335	0.443
NSA	0.565	0.286	0.587	0.521	0.520	0.264	0.545	0.466	0.348	**0.456**

NSA thắng 7/9 chỉ số, nổi bật ở DROP (+0.042) và GSM8K (+0.034). Paper diễn giải: pretraining sparse attention buộc mô hình tập trung thông tin quan trọng, lọc bớt nhiễu từ các đường attention không liên quan.

Bảng 2 (LongBench, một phần, chuẩn hoá cùng ngân sách 2.560 token/query cho các baseline sparse):

Model	MFQA-en	MFQA-zh	Qasper	HPQ	2Wiki	GovRpt	Dur	PassR-en	PassR-zh	LCC	Avg
H2O	0.428	0.429	0.308	0.112	0.101	0.231	0.208	0.704	0.421	0.092	0.303
InfLLM	0.474	0.517	0.356	0.306	0.250	0.277	0.257	0.766	0.486	0.143	0.383
Quest	0.495	0.561	0.365	0.295	0.245	0.293	0.257	0.792	0.478	0.135	0.392
Exact-Top	0.502	0.605	0.397	0.321	0.288	0.316	0.291	0.810	0.548	0.156	0.423
Full Attn	0.512	0.623	0.409	0.350	0.305	0.324	0.294	0.830	0.560	0.163	0.437
NSA	0.503	0.624	0.432	**0.437**	**0.356**	0.307	**0.341**	**0.905**	0.550	**0.232**	**0.469**

NSA đạt điểm trung bình cao nhất (+0.032 so với Full Attention, +0.046 so với Exact-Top), nổi bật ở multi-hop QA (HPQ +0.087, 2Wiki +0.051), hiểu code (LCC +0.069), truy xuất đoạn văn (PassR-en +0.075).

Needle-in-a-Haystack (64k): NSA đạt độ chính xác truy xuất hoàn hảo ở mọi vị trí (chỉ có đồ thị, Hình 5, không có bảng số).

Bảng 3 (AIME 24, sau SFT 10B token CoT 32k chưng cất từ DeepSeek-R1, nhiệt độ 0.7, top-p 0.95, 16 mẫu/câu):

Giới hạn token sinh	Full Attention-R	NSA-R
8192	0.046	**0.121**
16384	0.092	**0.146**

Tốc độ huấn luyện (Hình 6, Triton NSA kernel vs Triton FlashAttention-2, hệ 8×A100):

Context length	8k	16k	32k	64k
Speedup forward	2.1×	3.8×	6.3×	**9.0×**
Speedup backward	1.1×	2.0×	3.4×	**6.0×**

Tốc độ decode (Bảng 4, lượng truy cập bộ nhớ theo số token tương đương):

Context length	8192	16384	32768	65536
Full Attention	8192	16384	32768	65536
NSA	2048	2560	3584	5632
Speedup kỳ vọng	4×	6.4×	9.1×	11.6×

Kiểm chứng tự tính Bảng 4: số token NSA cần tải $\approx \lfloor (s-l)/d_{\text{strd}} \rfloor + 1 + n\,l' + w$ (nén + chọn + cửa sổ). Ở $s = 65536$: $\lfloor 65504/16 \rfloor + 1 + 16 \cdot 64 + 512 = 4095 + 1024 + 512 = 5631$, khớp gần đúng số 5632 của paper (lệch 1 do quy ước làm tròn). Ở $s = 8192$: $511 + 1024 + 512 = 2047 \approx 2048$. Công thức khớp, cho thấy phần "nén" đóng góp rất nhỏ (chỉ vài trăm token) so với hai số hạng cố định $n\,l' = 1024$ và $w = 512$.

6. Phân tích / ablation

6.1 Chiến lược chọn khối thay thế, đã thử và bị loại (mô hình 3B):

- *Dựa trên clustering key* (kiểu ClusterKV): overhead phân cụm động, mất cân bằng tải giữa các cụm (đặc biệt gây lệch thời gian giữa các nhóm Expert Parallelism của MoE), và cần tái phân cụm định kỳ cùng giao thức huấn luyện theo-khúc tuần tự. Paper chỉ liệt kê lý do loại, không có số liệu.
- *Auxiliary-loss* (tương tự Quest/InfLLM/SeerAttention): học điểm khối qua một loss phụ (KL divergence với điểm attention trung bình-pool theo khối); tăng chi phí, thường giảm hiệu năng vì không khả vi trực tiếp.
- *Heuristic không tham số* (kiểu Quest: tích query với min-max toạ độ khối key): recall thấp, hiệu năng dưới chuẩn.
- Cả hai cách trên, so với NSA và Full Attention trên mô hình 3B (Hình 7), đều có loss huấn luyện cao hơn NSA (chỉ có đồ thị, không có số cụ thể).

6.2 Quan sát attention map: mô hình 27B Full Attention cho điểm attention "cụm theo không gian" (key liền kề có điểm tương tự), là động lực cho việc chọn theo khối liên tục thay vì từng token rời rạc (chỉ có hình minh hoạ, không có số liệu).

7. Giới hạn

Do tác giả nêu: NSA "có thể chưa khai thác hết lợi ích hiệu quả ở chuỗi ngắn" (tự nhận). Số 70-80% latency do attention ở 64k là trích từ ước tính lý thuyết của công trình khác, không phải số đo trực tiếp của paper này. Kernel "tập trung vào kiến trúc có KV chia sẻ như GQA/MQA"; với MHA thuần, lợi ích group-centric loading không còn ý nghĩa.

Nhận xét: mâu thuẫn nhỏ về số token tiền huấn luyện (260B ở phần giới thiệu, 270B ở mục 4.1), không rõ số nào đúng. Không có ablation tách riêng từng nhánh (bỏ nén, chỉ giữ chọn+cửa sổ...) để biết nhánh nào đóng góp bao nhiêu; so sánh chính chỉ là NSA-toàn-bộ với Full Attention và với các chiến lược chọn khối khác. Kiến trúc MLP của cổng g_t^c (head-specific hay không, kích thước) không được mô tả; bài 67 chỉ ra chính lựa chọn này ảnh hưởng rất lớn tới hiệu năng, và phê bình trực tiếp NSA "không tách bạch được đóng góp của cơ chế gate khỏi hiệu ứng của thiết kế sparse attention". MLP nén φ với "mã hoá vị trí trong khối" không được đặc tả (dạng gì, có chia sẻ giữa layer không): paper không nêu. Tốc độ báo cáo (Hình 6, Bảng 4) chỉ đo riêng lớp attention/kernel, không có số liệu thông lượng end-to-end toàn mô hình.

8. Liên hệ với các bài khác

01, 17 (Longformer): NSA giữ nguyên công thức softmax, chỉ thay tập $(\tilde{K}_t, \tilde{V}_t)$, nhưng tập này **học được và động theo query**, không có "global token" tĩnh. **06 (Mistral):** nhánh cửa sổ là một biến thể SWA quy mô LLM. **14/15 (MQA, GQA):** kernel chọn của NSA chỉ hiệu quả khi KV chia sẻ; ép các head cùng nhóm GQA chọn cùng khối (Eq. 10) là điều kiện tiên quyết. **16 (MLA):** hướng khác giảm chi phí KV, nén theo **chiều đặc trưng** (giữ mọi token, nén vector) trong khi NSA giảm theo **chiều số token**; hai hướng có thể kết hợp về lý thuyết. **44/45 (FlashAttention/-2):** nhánh nén và cửa sổ dùng trực tiếp kernel FA2, nhánh chọn là phần mở rộng riêng. **55/56 (StreamingLLM, H2O):** khối đầu tiên bị ép chọn cố định gợi nhớ attention sink, nhưng NSA còn học thêm phần động; H2O là baseline bị NSA vượt rõ trên LongBench (0.469 so với 0.303). **61-63 (Quest, MInference, SeerAttention):** Quest là baseline và cũng là một chiến lược "bị loại" ở mục 6.1; SeerAttention "tương tự về khái niệm" với biến thể auxiliary-loss bị loại; MInference là ví dụ "chỉ thưa ở prefill". **65 (MoBA):** cùng thời điểm (2/2025), cùng bài toán chọn block động nhưng tính điểm bằng mean-pooling key, không có nhánh nén/cửa sổ riêng, không có cổng. **67 (Gated attention, Qwen):** nghiên cứu sâu loại cổng NSA dùng sơ sài, chỉ ra vị trí đặt cổng và tính head-specific quan trọng thế nào. **68 (DeepSeek-V3.2, DSA):** theo tên trong lộ trình, có khả năng là bước phát triển tiếp theo của hướng này trong cùng nhóm DeepSeek.

9. Tóm tắt công thức / khái niệm

Khái niệm	Công thức / ý nghĩa	Ghi chú
Khung ba nhánh	$o_t^* = \sum_c g_t^c \,\text{Attn}(q_t, \tilde{K}_t^c, \tilde{V}_t^c)$	$\mathcal{C} = \{\text{cmp,slc,win}\}$
Nén	$\tilde{K}_t^{\text{cmp}} = \{\varphi(k_{id_{\text{strd}}+1:id_{\text{strd}}+l})\}$	$l = 32,\ d_{\text{strd}} = 16$
Điểm khối chọn	p_t^{slc} suy từ $p_t^{\text{cmp}} = \text{Softmax}(q_t^\top \tilde{K}_t^{\text{cmp}})$	tái dùng attention nén, không tính thêm
Gộp theo nhóm GQA	$p_t^{\text{slc}\prime} = \sum_h p_t^{\text{slc},(h)}$	bắt buộc để kernel hiệu quả
Top-n chọn khối	$\mathcal{I}_t = \{i \mid \text{rank}(p_t'[i]) \le n\}$	$n = 16$, gồm 1 khối đầu + 2 khối gần nhất cố định
Cửa sổ	$\tilde{K}_t^{\text{win}} = k_{t-w:t}$	$w = 512$, phép chiếu Q,K,V riêng
Arithmetic intensity	tỉ lệ compute/memory access	huấn luyện/prefill compute-bound, decode memory-bound
Kernel chọn	nạp theo nhóm GQA, shared KV fetch, outer loop on grid	chỉ lợi khi KV chia sẻ (GQA/MQA)
Sàn chi phí (tự tính)	$n\,l' + w = 1536$ token	dưới ngưỡng này NSA không lợi về băng thông
Kết quả chính	LongBench avg 0.469 (NSA) > 0.437 (Full Attn) > 0.423 (Exact-Top)	Bảng 2
Tốc độ 64k	forward 9.0×, backward 6.0×, decode 11.6×	Hình 6, Bảng 4

10. Kết luận ngắn

NSA chứng minh sparse attention có thể vừa nhanh thật trên phần cứng (nhờ kernel gắn chặt với GQA và cân bằng arithmetic intensity) vừa học được end-to-end (nhờ mọi phép đều khả vi), thay vì chỉ là thủ thuật cắt KV lúc suy luận. Ba nhánh (nén để nhìn xa rẻ, chọn block để giữ chi tiết quan trọng, cửa sổ để giữ cục bộ chính xác) cùng một cổng gộp đơn giản đạt chất lượng bằng hoặc hơn full attention trên nhiều benchmark, với tốc độ tăng mạnh khi chuỗi dài.

Link gốc: https://arxiv.org/abs/2502.11089

Bài 65

MoBA: Mixture of Block Attention for Long-Context LLMs (2025)

Tác giả: Enzhe Lu và các đồng tác giả (Moonshot AI; Tsinghua University; Zhejiang Lab/Zhejiang University).
Venue / arXiv: arXiv:2502.13189 (v1, 18/02/2025); tự gọi là "Technical Report", paper không nêu venue hội nghị.
Nguồn: https://arxiv.org/abs/2502.13189
Chủ đề: Sparse attention theo nguyên lý Mixture-of-Experts (MoE) áp vào chính cơ chế attention: chia KV thành block, dùng router top-k chọn block theo độ giống giữa query và mean-pooled key của block, chuyển đổi mượt giữa full attention và MoBA, đã triển khai thực tế cho Kimi.
Vị trí trong lộ trình: Đọc trước: 01 (Attention), 27/28 (Switch Transformer, Mixtral: nguồn cảm hứng top-k gating mà MoBA mượn từ MoE). Ra đời cùng thời điểm với 64 (NSA, 2/2025) nhưng chọn block theo cách khác (mean-pooling thay vì điểm nén) và đơn giản hơn (không có nhánh nén/cửa sổ riêng, không cổng học được). Đọc tiếp: 67 (Gated attention, Qwen).

> Tài liệu học tập bằng tiếng Việt, là ghi chú diễn giải, không phải bản dịch. Số liệu lấy từ paper (full text arXiv, tải ngày 07/11/2026); các phép tính đánh dấu "tự tính" là của người ghi chú.

1. Bối cảnh

Chi phí $O(n^2)$ của attention chuẩn (bài 01) gây trở ngại khi mở rộng ngữ cảnh của LLM. Bài báo phân loại các hướng giải quyết trước đó và chỉ ra giới hạn của từng hướng:

- **Cấu trúc định kiến cố định** (sink attention, sliding window, bài 17 Longformer): hiệu quả nhưng "task-specific", hạn chế khả năng tổng quát hoá của mô hình vì mẫu attention được *áp đặt trước*, không để mô hình tự quyết định.
- **Chọn token động lúc suy luận** (Quest, MInference, RetrievalAttention): giảm tính toán cho chuỗi dài nhưng không giảm đáng kể chi phí **huấn luyện**, khó mở rộng LLM tới ngữ cảnh hàng triệu token một cách hiệu quả.
- **Attention tuyến tính** (Mamba, RWKV, RetNet): thay hẳn softmax attention bằng xấp xỉ tuyến tính, giảm chi phí nhưng chuyển đổi từ Transformer có sẵn tốn kém hoặc phải huấn luyện lại từ đầu, và minh chứng về khả năng suy luận phức tạp còn hạn chế.

Câu hỏi nghiên cứu: làm sao thiết kế một kiến trúc attention tuân theo nguyên lý **"ít cấu trúc định kiến"** ("less structure"), để mô hình tự quyết định nhìn vào đâu, đồng thời chuyển đổi mượt giữa full attention và dạng thưa, tương thích tối đa với Transformer đã huấn luyện sẵn?

Khái niệm cần biết trước: Mixture-of-Experts (MoE), bài 27/28: một router chọn top-k trong n "expert" (thường là FFN) cho mỗi token, dựa trên điểm tương thích (affinity). MoBA mang đúng cơ chế "router + top-k" này áp vào **KV-cache** thay vì FFN: mỗi block KV đóng vai trò một "expert".

2. Ý tưởng chính

MoBA chia ngữ cảnh thành n block liên tiếp kích thước B. Với mỗi query, một **router** (gating network) tính điểm tương thích giữa query và mỗi block (bằng tích trong với mean-pooling của key trong block), rồi áp **top-k gating** (hệt MoE) để chọn ra k block "chuyên gia" mà query được phép attend. Hai ràng buộc giữ tính nhân quả (causal): (a) không bao giờ chọn block **tương lai**; (b) **block hiện tại** (chứa chính query) luôn bị ép chọn, với causal mask riêng (vì mean-pooling cả block sẽ "nhìn trước" các token chưa sinh). Paper ví vai trò

này giống **expert chia sẻ** (shared expert) trong MoE hiện đại: luôn bật, cộng thêm các "expert" được chọn động. MoBA được thiết kế để **thay thế trực tiếp** full attention (không đổi số tham số), nên có thể chuyển đổi qua lại giữa hai chế độ ngay trong lúc huấn luyện, và là một **tổng quát hoá** của cả sliding window attention và attention sink (mỗi cái là MoBA với một router đặc biệt, cố định).

3. Phương pháp chi tiết

3.1 Công thức

Attention chuẩn, một query $q \in \mathbb{R}^{1\times d}$ với N cặp key-value $K, V \in \mathbb{R}^{N\times d}$ (Eq. 1): $\text{Attn}(q, K, V) = \text{Softmax}(qK^\top)V$.

MoBA (Eq. 2) chỉ cho q attend tập con chỉ số $I \subseteq [N]$:

$$\boxed{\text{MoBA}(q, K, V) = \text{Softmax}(qK[I]^\top)V[I]}$$

Chia N thành n block bằng nhau, $B = N/n$, block i là $I_i = [(i-1)B + 1,\ iB]$ (Eq. 3). Top-k gating chọn tập hợp các block:

$$I = \bigcup_{g_i>0} I_i, \qquad g_i = \begin{cases} 1 & s_i \in \text{Topk}(\{s_j \mid j \in [n]\}, k) \\ 0 & \text{ngược lại} \end{cases}$$

Điểm tương thích s_i là tích trong giữa q và **mean-pooling** của key trong block i (Eq. 6, không có tham số học thêm cho router):

$$s_i = \langle q,\ \text{mean_pool}(K[I_i]) \rangle$$

3.2 Giữ tính nhân quả

Không attend block tương lai: với $\text{pos}(q)$ là vị trí của query, đặt $s_i = -\infty$ và $g_i = 0$ cho mọi block i mà $\text{pos}(q) < i \times B$.

Block hiện tại, ép chọn + causal mask riêng: block chứa chính vị trí query luôn được ép $g_i = 1$ (không qua router), và khi tính attention trên block này phải áp **causal mask** như bình thường, vì mean-pooling của cả block sẽ vô tình "thấy" các token ở vị trí sau query (chưa được sinh ra). Paper nhấn mạnh đây là thiết kế bắt buộc để không rò rỉ thông tin.

3.3 Hai lựa chọn thiết kế bổ sung

Phân đoạn mịn (fine-grained block segmentation). Lấy cảm hứng từ quan sát trong MoE: expert càng "mịn" (nhiều expert nhỏ hơn là ít expert to) thường cho hiệu năng tốt hơn ở cùng ngân sách tính. MoBA phân đoạn theo **chiều độ dài ngữ cảnh** (thay vì chiều ẩn của FFN), và paper kiểm chứng điều tương tự cũng đúng (mục 6).

Hybrid MoBA / full attention. Vì MoBA không đổi số tham số so với full attention, mỗi lớp attention có thể chọn full hoặc MoBA ngay từ lúc khởi tạo, và **đổi qua lại trong lúc huấn luyện** nếu cần (tương tự ý tưởng chuyển full attention sang sliding window ở một công trình khác được paper trích dẫn).

MoBA tổng quát hoá SWA và attention sink (nhận định của paper). Sliding window (bài 17) $\approx$ MoBA với router luôn chọn các block **gần nhất**. Attention sink (bài 55) $\approx$ MoBA với router luôn chọn **block đầu + block gần nhất**. Vì router của MoBA học được (không cố định), paper cho rằng MoBA có sức biểu diễn mạnh hơn hai dạng tĩnh này.

4. Hiện thực hoá: thuật toán và kernel

Cài đặt kết hợp kỹ thuật tối ưu từ FlashAttention và MoE, gồm 5 bước: (1) xác định gán query → block KV theo router + causal mask; (2) sắp lại thứ tự query theo block được gán; (3) tính attention cho mỗi block KV và các query được gán (dùng FlashAttention với độ dài thay đổi, "varlen"); (4) xếp lại output về đúng thứ tự ban đầu; (5) gộp kết quả bằng online softmax (vì một query có thể attend cả block hiện tại và nhiều block lịch sử, cần cộng dồn đúng).

```
Thuật toán 1 | MoBA (diễn giải pseudo-code gốc của paper)
Đầu vào: Q, K, V ∈ R^{N×h×d}; siêu tham số block size B, top-k; n = N/B số block
1:  K̃_i, Ṽ_i = chia_khối(K, V, B)                      // mỗi khối [B,h,d]
2:  K̄ = mean_pool(K, B) ∈ R^{n×h×d}                    // trung bình mỗi khối
3:  S = Q K̄^T ∈ R^{N×h×n}                               // điểm tương thích giữa mọi
↪   query và mọi khối
4:  M = tạo_mặt_nạ_nhân_quả(N, n)                       // cấm khối tương lai
5:  G = topk(S + M, k)                                  // chọn khối lịch sử
6:  Qs, K̃s, Ṽs = lấy_khối_tự_chú_ý(Q, K̃, Ṽ)           // block hiện tại, có causal mask
7:  Qm, K̃m, Ṽm = chọn_theo_G(Q, K̃, Ṽ, G)               // các khối lịch sử được chọn
8:  Os = flash_attention_varlen(Qs, K̃s, Ṽs, causal=True)
9:  Om = flash_attention_varlen(Qm, K̃m, Ṽm, causal=False)
10: O  = gộp_bằng_online_softmax(Os, Om)
11: trả về O
```

5. Thí nghiệm và kết quả chính

5.1 Scaling law (mục 3.1). Huấn luyện 5 mô hình (Chinchilla-optimal), seqlen 8K, MoBA block=512, top-3 (sparsity $1 - \frac{512\times 3}{8192} = 81.25\%$; cùng đoạn văn paper cũng viết lại là "sparsity up to 75%", hai số khác nhau cùng xuất hiện, ghi chú trích cả hai không tự chọn).

Tham số	Head	Layer	Hidden	Token huấn luyện	Block size	Top-k
568M	14	14	1792	10.8B	512	3
822M	16	16	2048	15.3B	512	3
1.1B	18	18	2304	20.6B	512	3
1.5B	20	20	2560	27.4B	512	3
2.1B	22	22	2816	36.9B	512	3

Đường cong scaling law khớp dạng $L(C)$ (Hình 3c): LM loss (seqlen 8K) MoBA $2.625\,C^{-0.063}$ so với Full $2.622\,C^{-0.063}$ (gần như trùng). Trailing LM loss (seqlen 32K, 2K token cuối, sparsity lên tới $1 - \frac{512\times 3}{32768} = 95.31\%$): MoBA $1.546\,C^{-0.108}$ so với Full $1.464\,C^{-0.097}$ (MoBA hơi kém hơn ở trailing nhưng khoảng cách hẹp dần theo compute).

5.2 Phân đoạn mịn (mô hình 1.5B, 32K context, Hình 4): chia 32K thành {8,16,32,64,128} block, chọn tương ứng {2,4,8,16,32} (giữ sparsity 75%). Chênh lệch LM loss khoảng 10^{-2} giữa cấu hình thô nhất (2/8) và các cấu hình mịn hơn, ủng hộ "phân đoạn mịn giúp MoE/MoBA" (chỉ đọc được xu hướng từ đồ thị, không có bảng số chi tiết).

5.3 Hybrid MoBA/full (mô hình 1.5B, 32K, 30B token): huấn luyện 90% token bằng MoBA rồi chuyển sang full attention cho 10% cuối ("MoBA/full hybrid") cho position-wise LM loss gần như trùng với full attention thuần, không có loss spike khi chuyển đổi (Hình 5a, chỉ đồ thị). Cho **SFT**: MoBA thuần cho loss kém hơn (do loss masking ở prompt làm gradient thưa, khó lan truyền qua token bị che), khắc phục bằng **layer-wise hybrid** (vài layer cuối chuyển về full attention, phần còn lại giữ MoBA) giúp giảm rõ loss (Hình 5b-c, chỉ đồ thị).

5.4 Đánh giá downstream (Llama-8B-1M-MoBA, tiếp tục tiền huấn luyện từ Llama 3.1 8B, context tăng dần 128K → 256K → 512K → 1M, MoBA cuối cùng dùng block=4096, top-12, sparsity $1 - \frac{4096 \times 12}{1M} = 95.31\%$; 3 layer cuối giữ full attention, 29 layer còn lại MoBA):

Benchmark	Llama-8B-1M-MoBA	Llama-8B-1M-Full
AGIEval [0-shot]	0.5144	0.5146
BBH [3-shot]	0.6573	0.6589
CEval [5-shot]	0.6273	0.6165
GSM8K [5-shot]	0.7278	0.7142
HellaSWAG [0-shot]	0.8262	0.8279
Loogle [0-shot]	0.4209	0.4016
Competition Math [0-shot]	0.4254	0.4324
MBPP [3-shot]	0.5380	0.5320
MBPP Sanitized [0-shot]	0.6926	0.6615
MMLU [0-shot]	0.4903	0.4904
MMLU Pro [5-shot][CoT]	0.4295	0.4328
HumanEval [0-shot][pass@1]	0.6951	0.7012
SimpleQA [0-shot]	0.0465	0.0492
TriviaQA [0-shot]	0.5673	0.5667
LongBench @32K [0-shot]	0.4828	0.4821
RULER @128K [0-shot]	0.7818	0.7849

Quan trọng về cách dùng: trong mọi đánh giá trên, "MoBA chỉ dùng cho prefill; chuyển sang full attention lúc sinh (generation) để có hiệu năng tốt hơn" (nguyên văn ý của paper, mục 3.3). Ở RULER (sparsity tới $1 - \frac{4096 \times 12}{128K} = 62.5\%$ trong khoảng đó), MoBA gần khớp Full (0.7818 so với 0.7849). Needle-in-Haystack tới 1M token: "hiệu năng khả quan" (chỉ có đồ thị, Hình 7, không có số cụ thể).

5.5 Hiệu năng và khả năng mở rộng (mục 3.4): so sánh thời gian forward riêng lớp attention giữa Llama-8B-1M-MoBA và -Full. Ở 1M token, MoBA nhanh hơn tới **6.5×**. Mở rộng tới 10M token (giữ sparsity cố định bằng cách tăng block size cùng tỉ lệ với top-k, dùng tensor parallelism theo chiều query head để vượt giới hạn bộ nhớ GPU): ở 10M token, MoBA nhanh hơn Full Attention tới **16×** (Hình 2b, chỉ đồ thị cho các điểm trung gian).

6. Phân tích / ablation

- **Scaling law khớp gần như trùng** ở seqlen 8K (chênh lệch validation loss trong khoảng 10^{-3}) dù sparsity tới 75%, nhưng **trailing LM loss ở 32K kém hơn Full** rõ hơn (dù khoảng cách hẹp dần theo compute), cho thấy phần "khó" của sparse attention nằm ở chính các token cuối chuỗi dài, không phải loss trung bình (vốn bị lệch bởi phân phối dữ liệu nghiêng về chuỗi ngắn).
- **Phân đoạn mịn có lợi** (mục 5.2): cùng tỉ lệ sparsity 75% nhưng chia nhỏ block (và tăng số block được chọn tương ứng) luôn tốt hơn hoặc bằng cấu hình thô.
- **SFT là điểm yếu riêng của attention thưa do loss masking**, không phải vấn đề của bản thân MoBA khi pretrain; layer-wise hybrid (vài layer cuối full attention) là biện pháp thực dụng, không phải giải pháp gốc rễ.

7. Giới hạn

Do tác giả nêu: MoBA-thuần cho SFT loss kém hơn do gradient thưa qua token bị masking; cần layer-wise hybrid để khắc phục. Để đạt 10M token phải dùng kỹ thuật tensor-parallel theo query head (không phải tính

năng cốt lõi của thuật toán, mà là giải pháp hạ tầng để vượt hạn chế bộ nhớ). Mô hình downstream chỉ thử nghiệm trên một base model (Llama 3.1 8B).

Nhận xét: paper có một số điểm chưa nhất quán nội bộ: tỉ lệ sparsity ở seqlen 8K được viết là "81.25%" (tính chính xác từ $1 - 512 \times 3/8192$) nhưng ngay đoạn sau lại viết "sparsity up to 75%" cho cùng cấu hình; chú thích Hình 3b ghi "last 1K tokens" nhưng tiêu đề trục của chính hình đó (và các hình liên quan ở mục 5.3) lại ghi "last 2K" : không rõ số nào đúng. Không có bảng số cho nhiều kết quả quan trọng (Hình 2, 4, 5, 7 đều chỉ là đồ thị), nên khó kiểm chứng độc lập mức chênh lệch chính xác. Không có ablation tách riêng ảnh hưởng của "không có tham số học" trong router (mean-pooling thuần) so với một router có tham số (như NSA); paper chọn mean-pooling vì đơn giản nhưng không so sánh trực tiếp với phương án có học.

8. Liên hệ với các bài khác

01 (Attention): công thức Eq. 2 chỉ thay K, V đầy đủ bằng $K[I], V[I]$, tương tự khung tổng quát của NSA (64) nhưng không có nhánh nén/cửa sổ riêng. **17 (Longformer), 55 (StreamingLLM):** theo chính nhận định của paper, cả sliding window và attention sink đều là trường hợp đặc biệt của MoBA với router cố định (không học) luôn chọn block gần nhất, hoặc block đầu + gần nhất. **27/28 (Switch Transformer, Mixtral):** nguồn gốc trực tiếp của cơ chế "router + top-k gating"; MoBA áp cùng công thức này lên trục độ dài ngữ cảnh thay vì trục FFN, và quan sát "phân đoạn mịn có lợi" của MoE cũng được paper kiểm chứng lại cho MoBA. **61 (Quest):** theo chính lời paper, "Quest có thể coi là MoBA với block nhỏ hơn và hàm biểu diễn khối đặc biệt kết hợp cả min và max pooling". **64 (NSA):** cùng bài toán (chọn block động, tháng 2/2025) nhưng NSA dùng điểm từ một nhánh nén có tham số học được (không phải mean-pooling thuần), có thêm nhánh cửa sổ riêng và cổng học được; NSA cũng phê bình ngược lại cách tiếp cận kiểu MoBA ở mục "bất tương thích với GQA" nếu mỗi head chọn khối khác nhau (MoBA không nói rõ có ép các head cùng nhóm GQA chọn chung khối như NSA không: paper không nêu). **67 (Gated attention, Qwen):** MoBA hoàn toàn không có cổng/gate ở output, khác hẳn hướng của NSA và bài 67.

9. Tóm tắt công thức / khái niệm

Khái niệm	Công thức / ý nghĩa	Ghi chú
MoBA	$\text{Softmax}(qK[I]^\top)V[I]$	I = hợp các block được chọn
Điểm tương thích	$s_i = \langle q, \text{mean_pool}(K[I_i])\rangle$	không có tham số học
Top-k gating	$g_i = 1$ nếu $s_i \in \text{Topk}(\cdot, k)$	giống router MoE
Cấm tương lai	$s_i = -\infty,\ g_i = 0$ nếu $\text{pos}(q) < iB$	giữ tính nhân quả
Block hiện tại	$g_i = 1$ ép buộc + causal mask riêng	giống "shared expert" trong MoE
SWA/sink	= MoBA với router cố định chọn block gần nhất / đầu+gần nhất	theo nhận định của paper
Hybrid layer-wise	vài layer cuối full attention, còn lại MoBA	khắc phục sparse gradient khi SFT
Cách dùng thực tế	MoBA ở prefill, full attention ở generation	mục 3.3, 5.4
Sparsity ví dụ	$1 - 512 \times 3/8192 = 81.25\%$ (cũng được paper gọi "75%")	mâu thuẫn nội bộ, trích cả hai
Tốc độ	6.5× ở 1M token, 16× ở 10M token	Hình 2, chỉ đồ thị cho điểm trung gian

10. Kết luận ngắn

MoBA mượn đúng công thức "router + top-k" của Mixture-of-Experts và áp trực tiếp lên KV-cache theo block, cho một dạng sparse attention học được, đơn giản (không tham số router, không cổng), tương thích ngược với full attention (chuyển đổi mượt, kể cả hybrid theo lớp lúc SFT), và đã được dùng thực tế để phục vụ ngữ cảnh dài của Kimi. Hai bài học quan trọng nhất cho người đọc thực dụng: (1) lợi ích chỉ rõ khi ngữ cảnh rất

dài (block × top-k $\ll T$), mức tiết kiệm phụ thuộc vào độ dài chuỗi; (2) chính nhóm tác giả dùng MoBA cho prefill và chuyển về full attention khi sinh token, nên với workload sinh tự hồi quy batch-1, giá trị trực tiếp của MoBA là hạn chế trừ khi prefill (token thị giác) rất lớn.

Link gốc: https://arxiv.org/abs/2502.13189

Bài 66

The Sparse Frontier: Sparse Attention Trade-offs in Transformer LLMs (2025)

Tác giả: Piotr Nawrot (University of Edinburgh), Robert Li (Cohere), Renjie Huang (Cohere), Sebastian Ruder (Meta), Kelly Marchisio (Cohere), Edoardo M. Ponti (University of Edinburgh). **Venue / arXiv:** arXiv:2504.17768 (v3, 22/06/2026); paper không nêu venue hội nghị trong văn bản đã đọc. **Nguồn:** https://arxiv.org/abs/2504.17768 **Chủ đề:** Nghiên cứu thực nghiệm quy mô lớn nhất tính đến thời điểm công bố về sparse attention **training-free**: xây dựng một taxonomy 4 trục để tổ chức các phương pháp, chọn 6 phương pháp tiêu biểu, đánh giá trên 3 họ mô hình (4B-72B), độ dài 16K-128K, sparsity tới 0.95, trên 9 tác vụ. **Vị trí trong lộ trình:** Đọc sau 46 (PagedAttention), 55 (StreamingLLM), 56 (H2O), 61 (Quest), 62 (MInference) vì bài này dùng lại và so sánh trực tiếp các phương pháp đó (Quest và các pattern của MInference là 2 trong 6 phương pháp được đánh giá). Bài này đóng vai trò "tổng kết, đặt tên hệ thống" cho cả giai đoạn 8: nó cho một bộ khung (4 trục thiết kế) để xếp loại mọi phương pháp sparse attention training-free đã học, và trả lời câu hỏi thực nghiệm "khi nào sparse attention đáng dùng".

> Tài liệu học tập bằng tiếng Việt, là ghi chú diễn giải, không phải bản dịch. Số liệu lấy từ paper (full text arXiv, tải ngày 07/11/2026); các phép tính đánh dấu "tự tính" là của người ghi chú. Mục "Ứng dụng ..." là suy luận.

1. Bối cảnh

Với LLM ngữ cảnh dài, attention làm tăng chi phí ở cả hai giai đoạn: phép tính prefill tăng bậc hai theo độ dài, còn decode phải đọc KV cache có kích thước tăng tuyến tính. Bài báo ghi nhận hơn 150 bài arXiv có "sparse attention" trong tiêu đề từ tháng 1/2025 đến tháng 1/2026, nhưng cho rằng các khảo sát trước (Li et al. 2025, Liu et al. 2025b, Yuan et al. 2024) chưa đánh giá một cách hệ thống trên nhiều độ dài. Nghiên cứu này so sánh ở quy mô lớn các phương pháp training-free, tức áp dụng lên mô hình đã tiền huấn luyện mà không huấn luyện lại. Các phương pháp cần huấn luyện như NSA và DeepSeek Sparse Attention nằm ngoài phạm vi đó.

2. Ý tưởng chính

Paper tổ chức "bối cảnh rối rắm" của hàng chục phương pháp sparse attention thành một **taxonomy 4 trục**: (1) đơn vị thưa hóa (unit of sparsification), (2) cách ước lượng độ quan trọng (importance estimation), (3) cách phân bổ ngân sách (budget allocation), (4) cách quản lý KV cache. Từ taxonomy này, chọn ra **6 phương pháp tiêu biểu** phủ các trục khác nhau (3 cho prefill: Vertical-Slash, FlexPrefill, Block-Sparse; 3 cho decode: SnapKV, Ada-SnapKV, Quest), đồng bộ hóa cách hiện thực để so sánh công bằng. Đánh giá trên 3 họ mô hình (Qwen 2.5, Llama 3.1, Gemma 3; 4B-72B), độ dài 16K-128K, sparsity 0-0.95, trên 9 tác vụ ngữ cảnh dài (gồm 3 tác vụ tự tạo mới dạng "truyện" để bổ sung bài kiểm tra tự nhiên, kiểm soát được). Từ đó trả lời 3 câu hỏi nghiên cứu (RQ1-3): sparse attention có hiệu quả không, phương pháp nào nên dùng, và độ dài chuỗi ảnh hưởng thế nào tới mức độ chịu được sparsity.

3. Phương pháp chi tiết

3.1. Taxonomy 4 trục thiết kế (§2)

- **Đơn vị thưa hóa (unit of sparsification):** cửa sổ cục bộ (local window), cột dọc (vertical, token toàn cục cho mọi query), slash (token ở offset cố định so với mỗi query), khối (block, ô vuông cố định như 64×64), hoặc trang (page, dùng cho decode, mượn khái niệm từ bài 46 PagedAttention). Đơn vị lớn (khối, trang) cho hiệu quả tính toán tốt hơn (memory locality); đơn vị nhỏ cho chọn lọc chính xác hơn.

- **Ước lượng độ quan trọng:** mẫu cố định (fixed, không phụ thuộc input, như StreamingLLM/bài 55) hoặt mẫu động phụ thuộc nội dung (content-aware, như tính điểm attention xấp xỉ qua pooling của MInference/bài 62, hoặc tổng điểm attention tích lũy của H2O/bài 56). Paper chỉ đánh giá các phương pháp content-aware vì công trình trước cho thấy mẫu cố định luôn thua mẫu động.
- **Phân bổ ngân sách (budget allocation):** đồng nhất (uniform, mỗi layer/head cùng ngân sách, như Block-Sparse và SnapKV), thích nghi (adaptive, như PyramidKV phân bổ nhiều hơn cho layer đầu, hoặc MoA tối ưu bằng xấp xỉ Taylor), hoặc theo ngưỡng (threshold-based, như FlexPrefill chọn đủ token để đạt 95% khối lượng attention, ngân sách tự thay đổi theo mẫu).
- **Quản lý KV cache:** loại bỏ (eviction, như H2O/bài 56, SnapKV: xóa vĩnh viễn token không quan trọng, giảm bộ nhớ nhưng mất thông tin không phục hồi được) hoặc giữ toàn bộ cache (full cache retention, như Quest/bài 61, SparQ: giữ cache đầy đủ, chỉ chọn lọc phần đọc khi tính attention, tránh mất thông tin nhưng không giảm bộ nhớ đỉnh).

3.2. Sáu phương pháp được chọn (Bảng 1)

Giai đoạn	Phương pháp	Đơn vị	Ngân sách	Quản lý KV cache
Prefill	Vertical-Slash (MInference, bài 62)	verticals và slashes	đồng nhất	không áp dụng
Prefill	FlexPrefill	verticals và slashes	theo ngưỡng	không áp dụng
Prefill	Block-Sparse (MInference, bài 62)	khối	đồng nhất	không áp dụng
Decode	SnapKV	token	đồng nhất	eviction
Decode	Ada-SnapKV	token	thích nghi	eviction
Decode	Quest (bài 61)	trang	đồng nhất	giữ toàn bộ cache

3.3. Thước đo chi phí tính toán (§3.4, Phụ lục B.1)

Paper dùng thước đo độc lập phần cứng để so sánh công bằng: **FLOPs cho prefill** (compute-bound) và **lượng truyền bộ nhớ cho decode** (memory-bound). Công thức tổng FLOPs prefill:

$$\boxed{F_{\text{prefill}} = B \cdot (F_{emb} + F_{attn} + F_{mlp} + F_{logits})}, \qquad F_{attn} = N\Big[2Ld(2d + 2d_h n_{kv}) + \rho\, hL^2(4d_h + 3)\Big]$$

với B batch size, L độ dài chuỗi, N số layer, d chiều ẩn, h, n_{kv} số head Q/KV, d_h chiều mỗi head, $\rho = 1 -$ sparsity là mật độ attention còn giữ. Công thức bộ nhớ decode:

$$\boxed{M_{\text{decode}} = M_{weights} + B \cdot M_{kv}}, \qquad M_{kv} = 2NL\, d_h n_{kv}\, \rho$$

Hai công thức này cho thấy rõ: F_{attn} (prefill) tăng theo L^2 còn phần còn lại tăng tuyến tính theo L, nên tỉ trọng attention trong tổng FLOPs tăng theo độ dài; $M_{weights}$ (decode) là hằng số không phụ thuộc L hay B trong khi M_{kv} tăng theo cả L và B, nên tỉ trọng KV cache trong tổng bộ nhớ đọc phụ thuộc mạnh vào cả độ dài VÀ batch size, không chỉ độ dài. Paper còn tính thêm chi phí ước lượng độ quan trọng của từng phương pháp (ví dụ chi phí riêng của Vertical-Slash hay của Quest khi tải trang).

3.4. Bộ 9 tác vụ đánh giá (§3.3, Bảng 2)

Chọn theo 3 chiều: **Dispersion** (thông tin cần tìm khó định vị tới đâu), **Scope** (cần xử lý bao nhiêu thông tin), **Naturalness** (ngôn ngữ tự nhiên hay tổng hợp). Gồm: QA (SQuAD, QuALITY, TOEFL - Low/Low, tự nhiên), Ruler NIAH/VT/CWE (Low/Low, High/Low, Low/High - tổng hợp), và 3 tác vụ tự tạo "Story"

(Retrieval, Multi-hop, Filtering - chuyển các tác vụ khó của Ruler sang văn bản tự nhiên dạng truyện nhiều chương). Mục đích: tránh việc chỉ đánh giá trên tác vụ truy xuất dễ (Single QA) rồi kết luận sai lệch về khả năng chịu sparsity.

4. Thí nghiệm và kết quả chính

Mô hình: Qwen 2.5 (7B, 14B, 32B, 72B, họ chính vì có dữ liệu huấn luyện nhất quán giữa các size, phù hợp so sánh theo quy mô), Llama 3.1 (8B, 70B), Gemma 3 (4B, 12B, 27B, có sẵn sliding-window ở 5/6 layer, chỉ áp sparse attention lên layer dense còn lại). Độ dài 16K/32K/64K cho cả 3 họ, 128K chỉ cho Qwen/Llama với Vertical-Slash và Quest (Gemma gần như mất hiệu năng ở 128K). Sparsity từ 0 đến 0.95 (ngân sách 1/20). Dùng vLLM, bf16.

RQ1: isoCost / Pareto (Hình 2). Sparse attention cải thiện Pareto frontier: tại cùng chi phí tính toán, mô hình sparse lớn hơn vượt mô hình dense nhỏ hơn. Với Qwen ở 128K, chỉ các cấu hình sparsity cao (0.8-0.93, tức 1/5 đến 1/15 ngân sách) mới nằm trên đường Pareto tối ưu khi prefill; sparsity 0.95 (1/20) rơi xuống dưới ranh giới tối ưu. Decode chịu được sparsity cao tốt hơn prefill, thậm chí 0.95 vẫn tối ưu hơn mô hình dense nhỏ. Với Gemma, hiệu ứng này yếu hơn ở prefill do kiến trúc sliding-window khiến tỉ trọng attention gốc đã thấp (không có nhiều để cắt thêm).

RQ2: Phân tích theo tác vụ (Hình 3). Ba phát hiện chính:

1. **Prefill và decode có độ linh hoạt khác nhau.** Chưa có phương pháp training-free nào kết hợp được bộ ước lượng độ quan trọng chi tiết (fine-grained) với kernel khai thác hiệu quả độ thưa bất quy tắc trong prefill, nên phải chọn giữa Vertical-Slash (chọn lọc toàn cục mịn) hoặc Block-Sparse (chọn khối). Không phương pháp nào thắng tuyệt đối: Vertical-Slash mạnh ở tác vụ truy xuất (Scope thấp, Dispersion thấp, ví dụ QuALITY, SQuAD); Block-Sparse mạnh ở tác vụ cần ngữ cảnh rộng hoặc suy luận nhiều bước (Scope/Dispersion cao, ví dụ Ruler VT, Story Filtering). Ở decode, chọn theo query rẻ (chỉ 1 query mỗi bước), nên Quest tổng quát hóa tốt hơn và chịu được sparsity cao hơn cả hai chiến lược prefill, trong khi vẫn giữ toàn bộ KV cache.
2. **Lợi ích của phân bổ ngân sách thích nghi phụ thuộc giai đoạn.** Ở prefill, FlexPrefill (threshold-based) chỉ bằng hoặc kém hơn Vertical-Slash (uniform), nghi do "hiện tượng attention sink" (threshold bắt được token điểm cao nhưng bỏ lỡ phần đuôi phân phối). Ở decode, Ada-SnapKV (adaptive) luôn tốt hơn SnapKV (uniform) đồng nhất, đặc biệt ở tác vụ nhiều truy vấn, nhưng cả hai vẫn kém Quest (full-cache). Điều này cho thấy loại bỏ vĩnh viễn (eviction) luôn bất lợi hơn khi token bị xóa sớm lại trở nên cần thiết sau.
3. **Mức độ chịu sparsity khác nhau rất lớn giữa tác vụ.** Tác vụ Single QA (QuALITY, SQuAD, TOEFL) chịu được sparsity 0.95 với suy giảm tối thiểu; tác vụ Multiple QA (Ruler NIAH, Story Retrieval) suy giảm đáng kể đã ở sparsity 0.8-0.9; tác vụ Scope/Dispersion cao suy giảm ngay từ sparsity 0.5-0.67. Chỉ đánh giá trên tác vụ truy xuất dễ sẽ che lấp những điểm yếu này. Quest vượt Ada-SnapKV trên truy xuất văn bản tự nhiên (Story Retrieval) nhưng kém hơn trên truy xuất tổng hợp (Ruler NIAH), vì các biểu diễn khóa (key) của token ngẫu nhiên kém phân biệt hơn, và tính hạt theo trang của Quest khuếch đại hạn chế này.

RQ3: Ảnh hưởng của độ dài chuỗi (Hình 4). Với cùng một mức ngân sách cố định, chuỗi dài hơn chịu suy giảm nhỏ hơn: ở ngân sách 1/20, lỗi tương đối (relative error, so với dense) giảm từ khoảng 0.33 (16K) xuống 0.26 (32K) và 0.20 (64K). Hệ quả: ngân sách token tối ưu nên tăng **dưới tuyến tính (sublinear)** theo độ dài chuỗi, không nên giữ cố định (vì gây suy giảm tăng dần khi chuỗi dài ra) và không cần tăng tuyến tính (lãng phí). Paper liên hệ hiện tượng này với luật Herdan (Herdan's law: thông tin mới trở nên hiếm hơn theo thời gian/vị trí trong văn bản), giải thích vì sao sparsity cao hơn khả thi ở vị trí xa hơn trong chuỗi.

Phân tích chi phí thực (Phụ lục B.2-B.3, Hình 12-13, trung bình trên Qwen 7B-72B):

- *Prefill:* tỉ trọng attention trong tổng FLOPs tăng từ 40% (16K) lên 68% (64K) và 80% (128K); tương

ứng, sparsity 5× (ngân sách 1/5) cho speedup tăng dần từ 1.5× (16K) lên 2.2× (64K) và 2.8× (128K). Độ lệch chuẩn giữa các size mô hình nhỏ (khoảng 4-6 điểm %).

- *Decode:* ở **batch size 1**, tải trọng số mô hình chiếm ưu thế: KV cache chỉ chiếm khoảng 7% lượng truyền bộ nhớ ở 16K, tăng lên khoảng 35% ở 128K (ở 64K nằm giữa, khoảng 22% theo biểu đồ). Ở batch size 8, tỉ trọng KV cache tăng lên 35-80%. Ở **batch size 64**, KV cache chiếm ưu thế (80-97%) và sparsity 5× cho speedup 2.8-4.7×. Paper nhấn mạnh: ***ở batch size 1, sparse attention mang lại lợi ích tối thiểu*** cho decode; lợi ích lớn chỉ xuất hiện ở batch size cao.

5. Phân tích / ablation

- **Adaptive không tự động tốt hơn uniform:** FlexPrefill (threshold) không vượt Vertical-Slash (uniform) ở prefill do hiện tượng attention sink; ngược lại ở decode, Ada-SnapKV (adaptive) nhất quán vượt SnapKV (uniform). Kết luận: lợi ích của phân bổ thích nghi phụ thuộc giai đoạn, không phải một quy luật chung.
- **Eviction luôn rủi ro hơn full-cache ở cùng mức sparsity:** cả SnapKV và Ada-SnapKV (eviction) đều kém Quest (giữ toàn bộ cache) dù Ada-SnapKV có phân bổ thích nghi; mất thông tin vĩnh viễn là bất lợi cố hữu của eviction.
- **Dữ liệu tự nhiên và tổng hợp phản ứng khác nhau:** Quest tốt hơn trên truy xuất tự nhiên (Story Retrieval) nhưng kém hơn trên truy xuất tổng hợp (Ruler NIAH), đối lập với Ada-SnapKV; nhấn mạnh cần benchmark cả hai loại dữ liệu để tránh kết luận một chiều.
- **Kiến trúc sẵn có sparsity làm giảm lợi ích thêm:** Gemma 3 (5/6 layer sliding-window sẵn có) cho thấy ít chồng lấp giữa đường Pareto sparse và dense ở prefill hơn Qwen/Llama (toàn layer dense), vì tỉ trọng attention gốc trong tổng chi phí đã thấp sẵn.
- **Batch size là biến ẩn quan trọng ở decode:** cùng một mức sparsity cho lợi ích rất khác nhau tùy batch size (gần như vô ích ở batch 1, mạnh ở batch 64), một biến mà nhiều bài báo sparse attention khác (bao gồm bài 61 Quest) không luôn làm rõ khi báo cáo số liệu.

6. Giới hạn

Do tác giả nêu (6 điểm, §5 Limitations):

1. Chỉ đánh giá phương pháp training-free; phương pháp training-based có thể giảm lệch huấn luyện/suy luận nhưng cần tài nguyên và dữ liệu riêng.
2. Chỉ 3 họ mô hình (Qwen 2.5, Llama 3.1, Gemma 3) đáp ứng yêu cầu phương pháp luận; chỉ instruction-tuned, chưa thử mô hình suy luận mở rộng chain-of-thought (o1, DeepSeek-R1) có thể có mẫu attention khác.
3. 9 tác vụ không phủ hết mọi tình huống; loại trừ tác vụ mở (tóm tắt) do thước đo tự động không đáng tin cậy; kết quả 128K hạn chế do hiệu năng gốc thấp.
4. Báo cáo chi phí độc lập phần cứng (FLOPs, truyền bộ nhớ) chứ không phải wall-clock thật; tốc độ thực tế phụ thuộc phần cứng, batch size, chất lượng hiện thực.
5. Không xét tương tác với kỹ thuật hiệu quả khác (lượng tử hóa, pruning, MoE). **Lưu ý:** phân tích riêng prefill và decode, nhưng hiệu quả end-to-end phụ thuộc phân bổ chung; paper tự nêu: với tải "input dài, output ngắn", prefill chiếm ưu thế chi phí nên sparsity prefill nhỏ cũng lợi hơn sparsity decode; với "output dài" thì ngược lại.
6. Ước lượng độ quan trọng tối ưu theo recall điểm attention, không trực tiếp theo loss tác vụ; khoảng cách giữa hai mục tiêu chưa được đo.

Nhận xét:

- Toàn bộ nghiên cứu ở độ dài 16K-128K; không có điểm dữ liệu dưới 16K, nên không suy ra trực tiếp số liệu cho ngữ cảnh ngắn, chỉ ngoại suy xu hướng.

- Chọn Qwen 2.5 làm họ chính vì lý do phương pháp luận hợp lý cho so sánh có kiểm soát, nhưng cũng khiến kết luận chính nghiêng về đặc thù một họ mô hình.
- Số liệu Hình 12/13 (ví dụ ~22% ở 64K cho KV cache ratio batch=1) đọc từ biểu đồ cột, không phải số trong văn bản, có thể sai số nhỏ khi trích xuất từ PDF.

7. Liên hệ với các bài khác

- **46 PagedAttention:** nguồn khái niệm "trang" làm đơn vị thưa hóa; cũng là inference engine (vLLM) chạy toàn bộ thí nghiệm.
- **55 StreamingLLM:** ví dụ kinh điển cho trục "mẫu cố định"; paper dẫn lại kết luận mẫu cố định luôn thua mẫu động, nên không đưa vào 6 phương pháp đánh giá chính.
- **56 H2O:** nguồn khái niệm "eviction" trong trục quản lý KV cache, dùng tổng điểm attention tích lũy để chọn token quan trọng.
- **61 Quest:** một trong 6 phương pháp đánh giá, đại diện "trang + giữ toàn bộ cache" ở decode; là phương pháp decode mạnh nhất, tổng quát nhất trong toàn nghiên cứu (RQ2), xác nhận độc lập kết luận của bài 61.
- **62 MInference:** cả Vertical-Slash và Block-Sparse (2/3 phương pháp prefill) đến trực tiếp từ bài 62; taxonomy dùng chính thuật ngữ MInference đặt ra.
- **63 SeerAttention:** không nằm trong 6 phương pháp đánh giá (cần huấn luyện gate, thuộc nhóm training-based bị loại khỏi phạm vi), dù về khái niệm vẫn nằm trong trục "content-aware importance estimation".
- **44/45 FlashAttention:** lý do kỹ thuật khiến prefill phải chọn giữa Vertical-Slash và Block-Sparse: mẫu thưa mịn tạo truy cập bộ nhớ bất quy tắc, không khớp kernel dạng khối của FlashAttention.

8. Tóm tắt công thức / khái niệm

Khái niệm	Công thức / ý nghĩa
Taxonomy 4 trục	Đơn vị thưa hóa; ước lượng độ quan trọng (fixed/content-aware); phân bổ ngân sách (uniform/adaptive/threshold); quản lý KV cache (eviction/full-cache)
6 phương pháp đánh giá	Prefill: Vertical-Slash, FlexPrefill, Block-Sparse. Decode: SnapKV, Ada-SnapKV, Quest
FLOPs prefill	$F_{attn} = N[2Ld(2d + 2d_h n_{kv}) + \rho h L^2(4d_h + 3)]$, $\rho = 1 -$ sparsity, tăng bậc hai theo L
Bộ nhớ decode	$M_{decode} = M_{weights} + B \cdot M_{kv}$, $M_{kv} = 2NLd_h n_{kv}\rho$ tuyến tính theo L và B
RQ1 (isoCost)	Sparse model lớn vượt dense model nhỏ ở cùng chi phí; ở 128K Qwen chỉ sparsity 0.8-0.93 tối ưu, 0.95 dưới ranh giới
RQ2 (theo tác vụ)	Prefill: Vertical-Slash tốt cho truy xuất, Block-Sparse tốt cho suy luận/tổng hợp, không ai thắng tuyệt đối. Decode: Quest (full-cache) > Ada-SnapKV > SnapKV (eviction)
RQ3 (theo độ dài)	Ở ngân sách 1/20: lỗi tương đối 0.33 (16K) -> 0.26 (32K) -> 0.20 (64K); ngân sách tối ưu nên tăng dưới tuyến tính theo L
Chi phí thực đo (Hình 12)	Prefill: attention = 40% FLOPs (16K) -> 68% (64K) -> 80% (128K); speedup 5x sparsity: 1.5x -> 2.2x -> 2.8x
Chi phí thực đo (Hình 13)	Decode batch=1: KV cache ~7% bộ nhớ (16K) -> ~35% (128K), lợi ích sparse tối thiểu; batch=64: KV cache 80-97%, speedup 2.8-4.7x

9. Kết luận ngắn

Sparse Frontier là nghiên cứu thực nghiệm quy mô lớn đầu tiên so sánh có kiểm soát các phương pháp sparse attention training-free, đóng góp một taxonomy 4 trục để hệ thống hóa cả lĩnh vực và ba kết luận thực dụng:

sparse attention cải thiện Pareto frontier chi phí/hiệu năng (nhưng chỉ ở sparsity cao, ngữ cảnh dài); không có phương pháp prefill nào thắng tuyệt đối (phải chọn theo loại tác vụ), còn ở decode Quest (giữ toàn bộ cache, chọn theo trang) tổng quát tốt nhất; và mức độ chịu sparsity tăng theo độ dài chuỗi, nên ngân sách cố định là thiết kế dưới tối ưu.

Link gốc: https://arxiv.org/abs/2504.17768

Bài 67

Gated Attention for Large Language Models: Non-linearity, Sparsity, and Attention-Sink-Free (2025)

Tác giả: Zihan Qiu, Zekun Wang, Bo Zheng, Zeyu Huang và các đồng tác giả (Qwen Team, Alibaba Group; University of Edinburgh; Stanford; MIT; Tsinghua).
Venue / arXiv: arXiv:2505.06708 (v1, 10/05/2025); paper không nêu venue hội nghị.
Nguồn: https://arxiv.org/abs/2505.06708
Chủ đề: Nghiên cứu có kiểm soát (30 biến thể, mô hình MoE 15B và dense 1.7B, huấn luyện tới 3.5T token) về việc gắn một cổng (gate) vào các vị trí khác nhau trong attention chuẩn; chỉ ra cổng sigmoid theo head, đặt sau SDPA, vừa rẻ vừa hiệu quả nhất, lý giải bằng hai cơ chế (phi tuyến hoá một phép chiếu low-rank, và gây thưa phụ thuộc query), và cho thấy nó triệt tiêu "attention sink".
Vị trí trong lộ trình: Đọc trước: 01 (Attention), 15 (GQA). Bài này khác hẳn 64 (NSA) và 65 (MoBA): không cắt bớt cặp (query, key) nào, mà **thêm một cổng rẻ** lên output của attention đầy đủ, nên không phải "sparse attention" theo nghĩa giảm số cặp tính, mà là "attention có cổng" làm đầu ra *trở nên thưa*. Đọc tiếp: không có bài tiếp theo trong "Giai đoạn 8" bàn riêng về gate, nhưng ý tưởng liên hệ trực tiếp tới 24 (Gated DeltaNet) và 55 (StreamingLLM, attention sink).

> Tài liệu học tập bằng tiếng Việt, là ghi chú diễn giải, không phải bản dịch. Số liệu lấy từ paper (full text arXiv, tải ngày 07/11/2026); các phép tính đánh dấu "tự tính" là của người ghi chú.

1. Bối cảnh

Cơ chế cổng xuất hiện trong LSTM, Highway Network, GRU, state-space model và các dạng linear attention như RetNet, GLA. Tuy vậy, tác dụng riêng của cổng thường khó tách khỏi những lựa chọn kiến trúc khác. Chẳng hạn, Switch Heads dùng sigmoid gating để chọn top-K expert attention; các tác giả nhận thấy lợi ích vẫn còn khi chỉ dùng một expert, khi cổng điều biến đầu ra mà không còn định tuyến. NSA (bài 64) cũng dùng cổng gộp ba nhánh nhưng chưa tách đóng góp của cổng khỏi thiết kế attention thưa. Bài báo vì thế nghiên cứu có kiểm soát vai trò của gating trong softmax attention.

Khái niệm cần biết trước:

- **Attention sink** (bài 55): hiện tượng token đầu chuỗi hút phần lớn trọng số attention một cách gần như vô nghĩa, do softmax luôn cần "đổ" trọng số đi đâu đó.
- **Massive activation**: một số chiều/token có giá trị kích hoạt (hidden state) lớn bất thường so với phần còn lại, được công trình khác liên hệ với attention sink.
- **Low-rank mapping**: hai phép chiếu tuyến tính liên tiếp không có hàm phi tuyến ở giữa tương đương với **một** phép chiếu tuyến tính có hạng bị chặn trên bởi chiều nhỏ nhất.

2. Ý tưởng chính

Attention chuẩn gồm 4 giai đoạn: chiếu Q, K, V (Eq. 1), SDPA + softmax (Eq. 2), ghép đa đầu (Eq. 3), chiếu ra W_O (Eq. 4). Paper thử gắn một cổng $Y' = Y \odot \sigma(XW_\theta)$ (Eq. 5) *ở* **5 vị trí khác nhau**: sau Q (G4), sau K (G3), sau V (G2), sau SDPA (G1), sau W_O (G5), và thử các biến thể về độ chi tiết (headwise/elementwise), chia sẻ hay riêng theo head, nhân hay cộng, và hàm hoạt hoá (sigmoid/SiLU). Kết luận chính: **cổng sigmoid, elementwise, riêng theo từng head (head-specific), đặt ngay sau SDPA (vị trí G1)** cho cải thiện lớn nhất (giảm tới 0.2 PPL, tăng 2 điểm MMLU), đồng thời cải thiện rõ **ổn định huấn luyện** (gần như hết loss spike, chịu được learning rate lớn hơn). Hai cơ chế giải thích hiệu quả này: **(1) phi tuyến hoá** phép chiếu low-rank

ẩn giữa W_V và W_O; **(2) gây thưa phụ thuộc query** lên output SDPA, và chính độ thưa này **triệt tiêu attention sink**.

3. Phương pháp chi tiết: các vị trí đặt cổng

3.1 Công thức cổng và 5 vị trí

$$\boxed{Y' = g(Y, X, W_\theta, \sigma) = Y \odot \sigma(XW_\theta)}$$

Y là đầu ra cần điều biến, X là đầu vào dùng để tính điểm cổng (paper lấy X = hidden state sau pre-norm), σ là hàm hoạt hoá (chủ yếu sigmoid, SiLU dùng cho cộng vì SiLU không bị chặn). Năm vị trí (Hình 1 của paper): G_2, G_3, G_4 ngay sau chiếu V, K, Q; G_1 sau SDPA (trước khi ghép đa đầu); G_5 sau W_O (toàn bộ output cuối). Ba trục biến thể khác: **độ chi tiết** (headwise: một số vô hướng/head; elementwise: một vector cùng chiều Y), **chia sẻ** (head-specific vs head-shared, cộng gộp qua trung bình các head để so sánh), **nhân hay cộng** ($Y \cdot \sigma(X\theta)$ hay $Y + \sigma(X\theta)$, chỉ dùng SiLU cho cộng).

3.2 Kết quả so sánh vị trí (mô hình MoE 15A2B, 400B token, Bảng 1 rút gọn)

Baseline: $q = 32$ head query, $k = 4$ head KV (GQA), $d_k = 128$.

Biến thể	Hàm hoạt hoá	Dạng điểm cổng	Tham số thêm (M)	Avg PPL	Hellaswag	MMLU	GSM8k	C-eval
(1) Baseline	-	-	0	6.026	73.07	58.79	52.92	60.26
(2) $k = 8$	-	-	50	5.979	73.51	59.78	52.16	62.26
(3) $q = 48$	-	-	201	5.953	73.59	58.45	53.30	59.67
(4) +4 expert	-	-	400	5.964	73.19	58.84	52.54	63.19
(5) SDPA Elementwise G_1	sigmoid	$n \times q \times d_k$	201	**5.761**	74.64	60.82	55.27	62.20
(6) v Elementwise G_2	sigmoid	$n \times k \times d_k$	25	5.820	74.38	59.17	53.97	61.00
(7) k Elementwise G_3	sigmoid	$n \times k \times d_k$	25	6.016	72.88	59.18	50.49	61.74
(8) q Elementwise G_4	sigmoid	$n \times q \times d_k$	201	5.981	73.01	58.74	53.97	62.14
(9) Dense Output G_5	sigmoid	$n \times d_{model}$	100	6.017	73.32	59.41	50.87	59.43
(10) SDPA Headwise G_1	sigmoid	$n \times q$	1.6	5.792	74.50	60.05	54.44	62.61
(12) SDPA Head-Shared G_1	sigmoid	$n \times d_k$	201	5.801	74.34	60.06	53.15	61.01
(14) SDPA Additive G_1	SiLU	$n \times q \times d_k$	201	5.821	74.81	60.06	53.30	60.98

Bốn hàng baseline bổ sung tham số (tăng k, tăng q, thêm expert) để so công bằng: dù thêm tham số tương đương hoặc nhiều hơn, chúng đều **kém** cổng SDPA (G_1). Rút ra bốn điểm: (i) cổng ở G_1/G_2 hiệu quả nhất; (ii) **head-specific quan trọng** (so hàng 10 với 12: head-shared làm giảm lợi ích dù vẫn hơn baseline); (iii) **nhân tốt hơn cộng** (hàng 5 so với 14); (iv) **sigmoid tốt hơn SiLU** ở vị trí G_1 (chi tiết ở Bảng 3, không trích riêng ở đây).

Vì sao G_5 (sau W_O) vô dụng (hàng 9, kém cả baseline ở C-eval/GSM8k): vì nó nằm *sau* phép chiếu ra cuối cùng, không chen được giữa hai phép chiếu tuyến tính W_V và W_O (xem mục 5.1), nên không giải quyết được vấn đề low-rank.

4. Thí nghiệm và kết quả chính

4.1 Mô hình dense (1.7B, Bảng 2 rút gọn), kiểm chứng SDPA Elementwise G_1 ở nhiều cấu hình, giảm bề rộng FFN để giữ nguyên số tham số:

Cấu hình	Max LR	Avg PPL	HumanEval	MMLU	GSM8k	Hellaswag
28 layer, 400B token: Baseline	4.0×10^{-3}	7.499	28.66	50.21	27.82	64.94
28 layer, 400B token: SDPA Gate	4.0×10^{-3}	**7.404**	29.27	51.15	28.28	65.48
28 layer, 3.5T token: Baseline	4.5×10^{-3}	6.180	34.15	59.10	69.07	68.02
28 layer, 3.5T token: SDPA Gate	4.5×10^{-3}	**6.130**	37.80	59.61	70.20	68.84
48 layer, 400B token: Baseline	4.0×10^{-3}	7.421	28.05	52.04	32.98	65.96
48 layer, 400B token: Baseline	8.0×10^{-3}	9.195 (phân kỳ nhẹ, PPL tăng)	21.34	44.28	15.24	57.00
48 layer, 400B token: Baseline + Sandwich Norm	8.0×10^{-3}	7.407	30.49	52.07	32.90	66.00
48 layer, 400B token: SDPA Gate	4.0×10^{-3}	7.288	31.71	52.44	32.37	66.28
48 layer, 400B token: SDPA Gate	8.0×10^{-3}	**7.325**	31.10	54.47	36.62	66.40
48 layer, 1T token, bsz 4096: Baseline	5.3×10^{-3}	7.363	29.88	54.44	32.22	65.43
48 layer, 1T token, bsz 4096: Baseline	8.0×10^{-3}	**phân kỳ, "-"**	-	-	-	-
48 layer, 1T token, bsz 4096: SDPA Gate	5.3×10^{-3}	7.101	34.15	55.70	36.69	67.17
48 layer, 1T token, bsz 4096: SDPA Gate	8.0×10^{-3}	**7.078**	31.71	56.47	39.73	67.38

Điểm nổi bật: ở LR cao (8e-3, 48 layer), **baseline phân kỳ** khi huấn luyện 1T token ("-" trong bảng paper, mô hình không hội tụ), trong khi mô hình có cổng vẫn hội tụ tốt và cho PPL thấp nhất trong toàn bộ bảng. Thêm Sandwich Norm (chuẩn hoá output attention/FFN trước khi cộng residual) cũng phục hồi được sự ổn định ở baseline nhưng cải thiện khiêm tốn hơn cổng.

4.2 Chi phí: vì tham số và FLOPs của cổng rất nhỏ, "độ trễ (wall-time) do cổng gây ra nhỏ hơn 2%" (nguyên văn).

5. Phân tích: vì sao cổng hiệu quả

5.1 Phi tuyến hoá phép chiếu low-rank (mục 4.1)

Output head k của token i (Eq. 6): $o_i^k = \sum_j S_{ij}^k \cdot X_j W_V^k W_O^k$ (gộp W_V^k và W_O^k thành một phép chiếu vì không có hàm phi tuyến ở giữa). Vì $d_k < d_{model}$ (và với GQA, W_V còn bị chia sẻ giữa các head trong nhóm), phép chiếu gộp $W_V^k W_O^k$ bị **chặn hạng (low-rank)**, giảm sức biểu diễn. Thêm phi tuyến vào giữa hai phép chiếu

này (Eq. 7, gate ở G_2; hoặc Eq. 8, gate/RMSNorm ở G_1) phá vỡ việc gộp thành một ma trận, khôi phục sức biểu diễn. Đây lý giải vì sao cả gate ở G_1 (Bảng 1 hàng 5), gate ở G_2 (hàng 6), và thậm chí **RMSNorm không tham số** đặt ở G_1 (không có trong trích ở mục 3, nhưng paper báo PPL 5.847, gần với gate) đều cải thiện: tất cả đều "chen" phi tuyến vào đúng chỗ. Ngược lại G_5 nằm *sau* W_O nên không chen được vào giữa, giải thích vì sao vô dụng.

5.2 Gây thưa phụ thuộc query (mục 4.2)

Phân tích điểm cổng trên dữ liệu test (Bảng 4 rút gọn, M-Act = kích hoạt ẩn lớn nhất trung bình qua các layer, F-Attn = tỉ lệ attention đổ vào token đầu):

Biến thể	Điểm cổng trung bình	M-Act	F-Attn	PPL
Baseline	-	1053	0.467	6.026
SDPA Elementwise G_1	0.116	94	0.048	5.761
SDPA Headwise G_1	0.172	98	0.073	5.792
SDPA Elementwise Head-shared G_1	0.271	286	0.301	5.801
v Elementwise G_2	0.221	125	0.297	5.820
SDPA Input-Independent Gate	0.335	471	0.364	5.917
SDPA Elementwise, NS-sigmoid	0.653	892	0.451	5.900

Bốn quan sát: (i) **cổng hiệu quả có điểm trung bình thấp** (0.116, phân phối dồn gần 0, tức phần lớn output SDPA bị "khoá" lại); (ii) **chia sẻ cổng giữa head làm giảm độ thưa** (0.271 so với 0.116) và giảm cả lợi ích; (iii) **gate ở G_2 (phụ thuộc key/value cũ) kém thưa hơn và kém hiệu quả hơn gate ở G_1 (phụ thuộc query hiện tại)**, cho thấy "thưa phải phụ thuộc query mới có ích", không chỉ cần thưa chung; thử cổng **không phụ thuộc đầu vào** (tham số học riêng, khởi tạo 0, không nhìn X) vẫn hơn baseline (nhờ phi tuyến) nhưng điểm cổng cao (0.335), không thưa bằng; (iv) thay sigmoid bằng **NS-sigmoid** $= 0.5 + 0.5 \cdot \sigma(x) \in [0.5, 1]$ (giữ phi tuyến, bỏ khả năng về gần 0, tức giảm thưa) cho PPL kém hơn (5.900 so với 5.761), xác nhận **chính độ thưa** (không chỉ phi tuyến) đóng góp phần lớn lợi ích.

5.3 Cổng ở SDPA triệt tiêu attention sink (mục 4.3)

Baseline: trung bình **46.7%** attention toàn layer đổ vào token đầu (Hình 2); layer 21 riêng lẻ lên tới **83%**. Sau khi gắn cổng elementwise ở G_1: tỉ lệ trung bình giảm xuống **4.8%**, layer 21 giảm xuống **4%**. Cổng ở G_2 (value) giảm được massive activation (M-Act) nhưng **không** giảm được attention sink (F-Attn vẫn 0.297 ở bảng trên) : chứng tỏ **massive activation không phải điều kiện cần** cho attention sink, chỉ gate *query-dependent* ở đúng vị trí G_1 mới triệt tiêu được sink. Cơ chế được paper phác hoạ: massive activation khởi phát từ output FFN ở layer sớm (ví dụ layer 5-6), bị cộng vào residual và lan truyền qua pre-norm ở các layer sau, gây attention sink ở các layer đó; cổng ở G_1 giữ hidden state nhỏ ngay từ các layer đầu, cắt đứt chuỗi lan truyền này. Đây cũng là lý do Sandwich Norm (chuẩn hoá output FFN trước khi vào residual) cải thiện ổn định huấn luyện: nó chặn trực tiếp đường lan truyền của giá trị lớn.

5.4 Ngoại suy ngữ cảnh dài (mục 4.4, mô hình huấn luyện 3.5T token, mở rộng RoPE base 10k→1M, tiếp tục train 32K thêm 80B token, rồi YaRN lên 128K, đánh giá RULER)

Phương pháp	4k	8k	16k	32k	64k (YaRN)	128k (YaRN)
Baseline	88.89	85.88	83.15	79.50	37.51	31.65
SDPA-Gate	90.56	87.11	84.61	79.77	**66.60**	**58.82**

Trong vùng huấn luyện gốc (32K), hai mô hình gần tương đương; khi YaRN kéo ra 64K và 128K, baseline sụt rất mạnh (ví dụ ở 32K baseline tụt 41.56 điểm so với 4k) còn mô hình có cổng sụt ít hơn hẳn (-6.89 điểm ở

cùng mốc). Giả thuyết của paper: mô hình baseline "dựa vào" attention sink để điều chỉnh phân phối attention, nên khi RoPE bị biến đổi (YaRN), sink không thích nghi kịp theo kiểu "không cần train"; mô hình có cổng không phụ thuộc sink mà dựa vào chính điểm cổng phụ thuộc đầu vào, nên bền hơn trước thay đổi RoPE.

6. Giới hạn

Do tác giả nêu: cần nghiên cứu thêm "sự tương tác giữa độ thưa, massive activation và attention sink... đặc biệt khi mở rộng tới mô hình sâu và lớn hơn". Chỉ thử trên mô hình MoE 15B và dense 1.7B, không có mô hình lớn hơn (ví dụ cỡ 30B+ dày đặc).

Nhận xét: có một điểm số chưa khớp giữa hai bảng mô tả cùng cấu hình "v Elementwise Gate" (G2): Bảng 1 ghi GSM8k $= 53.97$, còn Bảng 4 (cùng PPL 5.820, cùng Hellaswag 74.38, cùng MMLU 59.17) lại ghi GSM8k $= 51.33$, hai số khác nhau cho cùng một mô hình; không rõ số nào đúng. Dòng tiêu đề PDF ghi ngày "2025-06-11" không khớp bản v1 arXiv (10/05/2025); ghi chú này dùng ngày theo arXiv stamp + API, không theo dòng tiêu đề PDF. Paper không thử nghiệm kết hợp cổng với sparse attention thực sự (như NSA/MoBA) để xem hai hướng có cộng hưởng hay triệt tiêu lẫn nhau.

7. Liên hệ với các bài khác

01 (Attention): cổng không đổi công thức softmax, chỉ nhân thêm một hệ số sau SDPA, khác hẳn 64/65 (không giảm số cặp được tính). **15 (GQA):** baseline dùng GQA ($q = 32, k = 4$); mục 5.1 chỉ ra GQA làm W_V bị chia sẻ giữa head trong nhóm, làm trầm trọng thêm vấn đề low-rank mà gate khắc phục, nên gate có thể còn quan trọng hơn ở mô hình GQA nhóm lớn. **24 (Gated DeltaNet), 26 (Kimi Linear):** cả hai đã có cổng (forget gate / gate theo kênh) trong công thức cập nhật trạng thái của attention tuyến tính; bài 67 cho thấy ngay cả softmax attention đầy đủ cũng lợi từ cổng ở output, dù lý do (phi tuyến hoá + thưa) khác với lý do "quên theo thời gian" của SSM/linear attention. **44/45 (FlashAttention):** gate ở G_1 nằm ngay sau SDPA, trước khi ghép đầu, nên về cấu trúc có thể gộp vào kernel FlashAttention hiện có mà không phá vỡ tính liên tục bộ nhớ; paper không trình bày cài đặt kernel cụ thể. **55 (StreamingLLM), 56 (H2O):** liên hệ quan trọng nhất, vì cả hai dựa vào sự tồn tại của attention sink (StreamingLLM giữ cố định token sink; H2O giữ token "nặng" theo điểm attention tích luỹ); nếu mô hình được huấn luyện với gate kiểu bài 67 (sink giảm từ 46.7% xuống 4.8%), các heuristic này mất cơ sở và cần kiểm tra lại trên mô hình sink-free. **64 (NSA), 65 (MoBA):** cả hai không phân tách sâu vai trò cổng như bài này; NSA tự nhận bị bài 67 phê bình trực tiếp về điểm này (trích ở mục 1).

8. Tóm tắt công thức / khái niệm

Khái niệm	Công thức / ý nghĩa	Ghi chú
Cổng	$Y' = Y \odot \sigma(XW_\theta)$	X = hidden state sau pre-norm
5 vị trí	G_1 (sau SDPA), G_2 (sau V), G_3 (sau K), G_4 (sau Q), G_5 (sau W_O)	G_1 tốt nhất
Vì sao G_5 vô dụng	nằm sau W_O, không chen được vào giữa W_V, W_O	mục 5.1
Phi tuyến hoá low-rank	$o_i^k = \sum_j S_{ij}^k \cdot \text{NonLin}(X_j W_V^k) W_O^k$	Eq. 7-8
Độ thưa điểm cổng	trung bình 0.116 ở G_1 elementwise, dồn gần 0	Bảng 4
NS-sigmoid	$0.5 + 0.5\sigma(x) \in [0.5, 1]$	giữ phi tuyến, bỏ thưa, kém hơn
Attention sink	$46.7\% \to 4.8\%$ (trung bình), $83\% \to 4\%$ (layer 21)	Hình 2
Massive activation	$M\text{-}Act$ $1053 \to 94$	Bảng 4

Khái niệm	Công thức / ý nghĩa	Ghi chú
Ổn định LR	baseline 48-layer phân kỳ ở 8×10^{-3}, gate thì không	Bảng 2
Ngoại suy RULER (YaRN 128k)	baseline 31.65, gate 58.82	Bảng 5
Chi phí	tham số và FLOPs nhỏ, latency $< 2\%$	mục 3.1

9. Kết luận ngắn

Bài này không đề xuất một dạng attention mới mà **giải phẫu** một thành phần tưởng chừng phụ (cổng) và chỉ ra chính xác vị trí, độ chi tiết, và hàm hoạt hoá nào khiến nó hiệu quả: sigmoid, elementwise, riêng theo head, ngay sau SDPA. Lợi ích đến từ hai cơ chế tách bạch được bằng thực nghiệm (phi tuyến hoá phép chiếu $W_V W_O$ bị low-rank, và gây thưa phụ thuộc query), và hệ quả bất ngờ nhất là cổng này **triệt tiêu attention sink** mà không cần thiết kế gì đặc biệt cho sink. Khác NSA và MoBA (chỉ lợi ở ngữ cảnh dài), cổng này rẻ và có lợi **ngay cả ở ngữ cảnh ngắn** nhờ ổn định huấn luyện và giảm outlier, nên đáng được đánh giá cả ở ngữ cảnh ngắn. Lưu ý: paper không nói mô hình nào trong dòng Qwen đã dùng chính cơ chế này ở sản phẩm thật; nếu Qwen3-Next hay các mô hình hybrid sau này của Qwen có dùng gated attention tương tự, đó là **thông tin ngoài phạm vi bài báo này** (theo nguồn ngoài paper), không phải điều paper tự khẳng định.

Link gốc: https://arxiv.org/abs/2505.06708

Bài 68

DeepSeek-V3.2: Pushing the Frontier of Open Large Language Models (2025)

Phạm vi ghi chú: tập trung vào DeepSeek Sparse Attention (DSA): công thức lightning indexer, cách chọn token top-k, cách đặt DSA dưới MLA ở chế độ MQA, giai đoạn dense warm-up với loss KL cho indexer, giai đoạn sparse training, số token huấn luyện, đường cong chi phí prefill/decode so với DeepSeek-V3.1-Terminus, và parity evaluation. Phần RL/post-training (scaling GRPO, agentic task synthesis) chỉ tóm tắt ngắn ở mục 6, vì đã có trong lộ trình RL riêng (rl 07, 36, 39, 41, 42, 43).

Tác giả: DeepSeek-AI (research@deepseek.com); danh sách đầy đủ ở Phụ lục E của paper, không chia vai trò riêng cho DSA.
Venue / arXiv: arXiv:2512.02556 (v1, 02/12/2025); paper không nêu venue hội nghị.
Nguồn: https://arxiv.org/abs/2512.02556
Chủ đề: DeepSeek Sparse Attention (lightning indexer + chọn token top-k, đặt dưới MLA ở chế độ MQA), scaling GRPO ổn định, pipeline tổng hợp tác vụ agentic quy mô lớn.
Vị trí trong lộ trình: đọc sau 16 (MLA, DeepSeek-V2) **bắt buộc**, vì DSA được cài đặt dựa trên MLA ở chế độ MQA (Phụ lục A của paper, Hình 7) và dùng lại khái niệm latent c_t^{KV}. Có thể đọc sau 64 (NSA) để so sánh hai cách làm "sparse attention học được, train trực tiếp". Bài này đứng trước 69 (GLM-5, áp dụng lại DSA) và 71 (DeepSeek-V4, kế tiếp của DSA).

> Tài liệu học tập bằng tiếng Việt, là ghi chú diễn giải, không phải bản dịch. Số liệu lấy từ paper (full text arXiv, tải ngày 07/11/2026); các phép tính đánh dấu "tự tính" là của người ghi chú.

1. Bối cảnh

Các tác giả cho rằng mô hình mở còn tụt sau mô hình đóng như Claude, Gemini và GPT-5 vì ba nguyên nhân: attention thông thường kém hiệu quả với chuỗi dài; đầu tư tính toán cho post-training chưa đủ; và khả năng tổng quát hóa, làm theo chỉ dẫn của agent còn hạn chế. DeepSeek Sparse Attention (DSA) tập trung vào nguyên nhân thứ nhất.

Điểm khởi đầu. DeepSeek-V3.2 dùng **đúng kiến trúc** của DeepSeek-V3.2-Exp. So với DeepSeek-V3.1-Terminus (bản mới nhất của dòng V3.1, context đã mở rộng tới 128K bằng MLA như bài 16), thay đổi kiến trúc **duy nhất** là thêm DSA, thông qua **continued training** (huấn luyện tiếp từ checkpoint có sẵn), không train lại từ đầu.

Khái niệm cần nắm trước (từ bài 16): MLA nén K, V của mọi head về một latent chung nhỏ c_t^{KV}; có hai "chế độ" vận hành toán học tương đương là MHA mode (dựng lại K, V đầy đủ cho từng head, dùng khi train/prefill) và MQA mode (giữ attention trong không gian latent dùng chung cho mọi head, dùng khi decode, xem Phụ lục A của paper này). DSA cần MQA mode vì lý do sẽ nói ở mục 4.

Vấn đề của sparse attention học được nói chung (bối cảnh rộng hơn, liên hệ bài 64 NSA): muốn attention thưa mà vẫn **train được** (không chỉ áp lúc suy luận), cần một cơ chế chọn ra tập con token "quan trọng" cho mỗi query, và cơ chế chọn đó phải rẻ hơn hẳn attention đầy đủ. DSA giải quyết việc này bằng một mạng nhỏ riêng gọi là **lightning indexer**.

2. Ý tưởng chính

DSA gồm hai phần: một **lightning indexer** nhỏ, rẻ, tính điểm "liên quan" $I_{t,s}$ giữa token truy vấn t và mọi token trước đó s; và một bước **chọn top-k** giữ lại chỉ k mục key-value có điểm cao nhất, rồi attention thật

(tốn kém) chỉ chạy trên k mục này thay vì toàn bộ L token trước đó. Vì indexer có ít head, dùng ReLU (rẻ), và chạy ở FP8, chi phí của nó nhỏ hơn hẳn attention đầy đủ dù vẫn là $O(L^2)$.

Điểm khó: nếu đưa thẳng cơ chế top-k này vào một mô hình đã huấn luyện, chất lượng sẽ giảm đột ngột vì mô hình chưa từng "sống" với việc thiếu token. DeepSeek giải quyết bằng **continued pre-training hai giai đoạn** từ DeepSeek-V3.1-Terminus: giai đoạn 1 ("dense warm-up") chỉ dạy indexer bắt chước đúng phân phối attention thật mà **chưa** bật top-k; giai đoạn 2 ("sparse training") mới bật top-k và để toàn mô hình thích nghi. Vì key-value entry (ở đây là latent MLA) phải được **chia sẻ giữa nhiều query head** để kernel hiệu quả, DSA được cài đặt dưới **MQA mode của MLA** (bài 16), không phải MHA mode.

3. Phương pháp chi tiết: lightning indexer và chọn token top-k

Ký hiệu của paper: $h_t \in \mathbb{R}^d$ là token truy vấn, $h_s \in \mathbb{R}^d$ là token trước đó (preceding token), H^I là số head của indexer.

3.1 Điểm số của indexer (Eq. 1)

$$\boxed{I_{t,s} = \sum_{j=1}^{H^I} w_{t,j}^I \cdot \mathrm{ReLU}\big(q_{t,j}^I \cdot k_s^I\big)}$$

- $q_{t,j}^I \in \mathbb{R}^{d^I}$ và $w_{t,j}^I \in \mathbb{R}$: suy ra từ token truy vấn h_t (mỗi head j một vector query và một trọng số vô hướng).
- $k_s^I \in \mathbb{R}^{d^I}$: suy ra từ token trước đó h_s (dùng chung cho mọi head, giống tinh thần MQA).
- ReLU được chọn "vì lý do thông lượng" (paper, nguyên văn ý "throughput consideration"), tức rẻ hơn softmax hay các hàm phi tuyến khác trên phần cứng.
- Paper không cho số cụ thể của H^I (số head indexer) hay d^I (chiều mỗi head indexer) cho DeepSeek-V3.2. Để tham khảo, GLM-5 (bài 69) khi áp lại DSA dùng 32 head indexer, mỗi head 128 chiều, nhưng đây là số của GLM-5, **không phải** số chính thức của DeepSeek-V3.2 (paper này không nêu).
- Vì H^I nhỏ và chạy ở FP8, paper nhận định chi phí indexer "remarkable" (rất rẻ) dù công thức vẫn là tổng trên mọi $s \le t$, tức vẫn $O(L^2)$ về mặt phép tính (chỉ là hằng số nhỏ hơn rất nhiều so với attention đầy đủ).

3.2 Chọn token top-k và attention thưa (Eq. 2)

Với tập điểm số $\{I_{t,s}\}$ của mỗi token truy vấn h_t, cơ chế chọn "fine-grained" chỉ giữ lại các mục key-value $\{c_s\}$ ứng với điểm top-k:

$$\boxed{u_t = \mathrm{Attn}\big(h_t,\ \{c_s \mid I_{t,s} \in \text{Top-k}(I_{t,:})\}\big)}$$

Ở đây c_s chính là **latent MLA** (ký hiệu c_s^{KV} ở bài 16), không phải key/value thô. Tức: indexer chọn "địa chỉ" (k vị trí), còn attention thật vẫn làm việc trong không gian latent đã nén của MLA.

3.3 Pseudo-code (diễn giải, không phải code gốc)

```
def dsa_attention(h_t, cache_h, cache_c_kv, k=2048):
    # 1. Indexer: điểm liên quan giữa token t và mọi token s <= t (rẻ, FP8, H^I head)
    I_t = 0
    for j in range(H_I):
        q_I = proj_q_indexer[j](h_t)        # R^{d_I}
        w_I = proj_w_indexer[j](h_t)        # scalar
        k_I = proj_k_indexer(cache_h)       # R^{d_I}, DÙNG CHUNG mọi head
        I_t += w_I * relu(q_I @ k_I.T)      # [S] điểm cho mọi token trước đó
    # 2. Chọn top-k (giai đoạn sparse training mới bật bước này)
```

```
    idx = topk(I_t, k).indices
    # 3. Attention thật (MQA mode của MLA) chỉ trên các latent được chọn
    c_sel = cache_c_kv[idx]                 # [k, d_c]
    u_t = mla_mqa_attention(h_t, c_sel)     # xem bài 16 §3.3 (hấp thụ ma trận)
    return u_t
```

4. Phương pháp chi tiết: DSA dưới MLA, dense warm-up, sparse training

4.1 Vì sao phải đặt DSA dưới MQA mode của MLA

Paper nêu rõ: "ở cấp kernel, mỗi key-value entry phải được chia sẻ giữa nhiều query để tính toán hiệu quả" (trích dẫn công trình NSA, bài 64, cho ràng buộc kernel này). Vì vậy DSA được cài đặt dựa trên **chế độ MQA của MLA** (bài 16, Phụ lục A, Hình 7 của paper này vẽ lại sự khác biệt giữa hai chế độ): mỗi latent c_t^{KV} (một "mục key-value" của MLA) được **dùng chung cho mọi query head** của cùng một token truy vấn. Điều này khác MHA mode, nơi mỗi head có K, V riêng dựng từ latent, nên không thể chia sẻ bộ nhớ đọc giữa các head theo cách mà kernel top-k cần.

4.2 Dense warm-up: khởi tạo indexer bằng KL loss (Eq. 3)

Bắt đầu từ checkpoint DeepSeek-V3.1-Terminus (context 128K). Giai đoạn 1: **giữ nguyên dense attention** (chưa bật top-k), **đóng băng mọi tham số trừ lightning indexer**. Mục tiêu: cho indexer "nhìn" đúng những gì attention thật đang nhìn, trước khi bất kỳ token nào bị loại bỏ thật.

Cách làm: với mỗi token truy vấn t, cộng điểm attention trên **mọi head attention chính**, rồi chuẩn hóa L1 theo chiều chuỗi để ra phân phối mục tiêu $p_{t,:} \in \mathbb{R}^t$. Loss là KL-divergence giữa $p_{t,:}$ và softmax của điểm indexer:

$$\mathcal{L}_I = \sum_t D_{KL}\big(p_{t,:} \,\|\, \text{Softmax}(I_{t,:})\big)$$

Siêu tham số: learning rate 10^{-3}, **1000 bước**, mỗi bước 16 chuỗi dài 128K token. Tổng token (tự tính, khớp với mô tả của paper): $1000 \times 16 \times 131072 \approx 2.1 \times 10^9 = 2.1$ tỷ token.

4.3 Sparse training: bật top-k, train toàn mô hình (Eq. 4)

Giai đoạn 2: thêm cơ chế chọn top-k thật, tối ưu **mọi** tham số mô hình để thích nghi với mẫu thưa của DSA. Vẫn tiếp tục căn chỉnh indexer theo attention chính, nhưng chỉ xét **tập đã chọn** $S_t = \{s \mid I_{t,s} \in \text{Top-k}(I_{t,:})\}$:

$$\mathcal{L}_I = \sum_t D_{KL}\big(p_{t,S_t} \,\|\, \text{Softmax}(I_{t,S_t})\big)$$

Chi tiết quan trọng (nguyên văn ý paper): **đầu vào của indexer bị "detach" khỏi computational graph** của mô hình chính, tức tối ưu **tách biệt**: indexer chỉ nhận gradient từ $\mathcal{L}_I$; mô hình chính chỉ nhận gradient từ loss ngôn ngữ (language modeling loss). Hai đường gradient không trộn vào nhau. Siêu tham số: learning rate 7.3×10^{-6}, chọn **2048 token key-value** cho mỗi token truy vấn (tức $k = 2048$), train **cả mô hình chính và indexer** trong **15000 bước**, mỗi bước 480 chuỗi dài 128K token. Tổng token (tự tính, khớp số paper nêu 943.7B): $15000 \times 480 \times 131072 \approx 9.437 \times 10^{11} = 943.7$ tỷ token.

Nhận xét: vì $k = 2048$ là một **số tuyệt đối cố định**, với bất kỳ chuỗi có độ dài $L \le k = 2048$, top-k sẽ chọn **toàn bộ** token sẵn có: DSA khi đó **suy biến về dense attention** (không còn gì để bỏ), nhưng vẫn phải trả thêm chi phí chạy indexer. Đây là điểm mấu chốt khi đánh giá DSA có đáng dùng ở quy mô vài nghìn token hay không.

5. Chi phí suy luận và parity evaluation

Độ phức tạp. DSA giảm độ phức tạp của attention chính từ $O(L^2)$ xuống $O(Lk)$ với $k \ll L$. Lightning indexer vẫn là $O(L^2)$ nhưng "cần ít tính toán hơn nhiều" so với MLA đầy đủ của V3.1-Terminus (paper không cho số FLOPs cụ thể của indexer).

Đường cong chi phí (Hình 3 của paper). Đo trên dịch vụ thật chạy trên GPU H800, giá thuê 2 USD/giờ/GPU, trục ngang là vị trí token từ 0 đến 128K. Trục dọc: với prefill, chi phí mỗi triệu token của V3.1-Terminus và V3.2 nằm trong khoảng 0 đến khoảng 0.7 USD; với decode, khoảng 0 đến khoảng 2.4 USD. Cả hai đường đều **tăng theo vị trí token** (càng về sau, attention càng đắt vì KV cache dài hơn), nhưng đường V3.2 nằm dưới đường V3.1-Terminus rõ rệt, nhất là ở vị trí xa. **Giá trị cụ thể từng điểm trên đồ thị không đọc được từ văn bản đã trích xuất** (đây là hình, không phải bảng số), nên không ghi số chính xác ở đây; chỉ nêu xu hướng và trục. Một chi tiết kỹ thuật đáng chú ý: với prefill chuỗi ngắn, paper "cài đặc biệt một chế độ masked MHA để mô phỏng DSA", đạt hiệu quả cao hơn trong điều kiện ngữ cảnh ngắn (ngụ ý: DSA "thật" không phải lựa chọn tối ưu khi chuỗi ngắn, nên họ dùng một đường tắt khác).

Parity evaluation (không có bảng số, chỉ mô tả bằng lời trong paper):

- **Benchmark chuẩn** (tháng 9/2025): DeepSeek-V3.2-Exp so với DeepSeek-V3.1-Terminus cho hiệu năng tương tự, không suy giảm đáng kể ở cả tác vụ ngắn và dài hạn.
- **Human preference (ChatbotArena):** Elo của V3.1-Terminus và V3.2-Exp (đo 10/11/2025) "khớp nhau chặt" (closely matched), dù V3.2 có thêm sparse attention.
- **Long context:** trên benchmark AA-LCR, V3.2-Exp cao hơn V3.1-Terminus **4 điểm** ở reasoning mode (số duy nhất có trong văn bản cho mục này). Trên Fiction.liveBench, V3.2-Exp "liên tục vượt" V3.1-Terminus trên nhiều chỉ số, nhưng paper không cho số cụ thể.

6. Post-training: RL và agentic (tóm tắt ngắn)

(Phần này nằm ngoài phạm vi chính của ghi chú; chi tiết thuật toán RL có trong lộ trình RL riêng: GRPO gốc ở rl 36 (DeepSeekMath), PPO ở rl 07, DeepSeek-R1 ở rl 39, DAPO ở rl 41, Dr. GRPO ở rl 42, GSPO ở rl 43.)

- **GRPO mở rộng.** DeepSeek-V3.2 vẫn dùng GRPO (công thức giống rl 36), gộp reasoning + agent + human alignment vào **một** giai đoạn RL (tránh catastrophic forgetting của huấn luyện nhiều giai đoạn). Ba cải tiến ổn định hóa khi scale RL: (1) **Unbiased KL Estimate**: sửa K3 estimator (Schulman 2020) bằng tỉ lệ importance sampling giữa π_θ hiện tại và π_{old}, tránh gradient không giới hạn khi $\pi_\theta \ll \pi_{ref}$; (2) **Off-Policy Sequence Masking**: che (mask = 0) các chuỗi có advantage âm **và** độ lệch KL giữa π_{old} và π_θ vượt ngưỡng δ (chỉ áp cho mẫu âm, vì "mô hình học được nhiều nhất từ lỗi của chính nó", mẫu âm lệch xa lại gây hại); (3) **Keep Routing**: giữ đúng đường định tuyến expert MoE đã dùng lúc sinh (inference) khi tính gradient lúc train, tránh lệch giữa tham số được kích hoạt lúc sinh và lúc tối ưu; (4) **Keep Sampling Mask**: giữ đúng mặt nạ top-p/top-k đã dùng lúc sample để train trên cùng không gian hành động.
- **DeepSeek-V3.2-Speciale:** biến thể chỉ train trên dữ liệu reasoning, giảm length penalty, thêm dữ liệu/reward từ DeepSeekMath-V2 cho chứng minh toán; đạt **gold medal** ở IMO 2025, CMO 2025, IOI 2025, và hạng 2 ICPC World Finals 2025 (Bảng 4).
- **Agentic task synthesis:** pipeline tổng hợp hơn **1,800 môi trường** và **85,000 prompt** phức tạp (cold-start từ công thức DeepSeek-R1 hợp nhất reasoning + tool-use, rồi RL quy mô lớn trên 4 loại tác vụ: code agent, search agent, code interpreter, general agent; xem Bảng 1 của paper cho số lượng từng loại). **Context management** cho agent dài hơi: 4 chiến lược (Summary, Discard-75%, Discard-all, Parallel-fewest-step) được so sánh trên BrowseComp (Hình 6); Discard-all đạt hiệu quả/khả năng mở rộng tốt nhất trong số các chiến lược nối tiếp (serial).

7. Thí nghiệm và kết quả chính

Bảng 2 (rút gọn), so với mô hình đóng/mở, thinking mode, context 128K:

Benchmark	Claude-4.5-Sonnet	GPT-5 High	Gemini-3.0-Pro	Kimi-K2-Thinking	DeepSeek-V3.2-Thinking
MMLU-Pro (EM)	88.2	87.5	90.1	84.6	85.0
GPQA Diamond (Pass@1)	83.4	85.7	91.9	84.5	82.4
HLE (Pass@1)	13.7	26.3	37.7	23.9	25.1
LiveCodeBench (Pass@1-COT)	64.0	84.5	90.7	82.6	83.3
Codeforces (Rating)	1480	2537	2708	-	2386
AIME 2025 (Pass@1)	87.0	94.6	95.0	94.5	93.1
SWE Verified (Resolved)	77.2	74.9	76.2	71.3	73.1
τ^2-Bench (Pass@1)	84.7	80.2	85.4	74.3	80.3

Bảng 3 (hiệu suất token, DeepSeek-V3.2-Speciale so với các mô hình đóng), số trong ngoặc là số token đầu ra (nghìn):

Benchmark	GPT-5 High	Gemini-3.0-Pro	Kimi-K2-Thinking	V3.2-Thinking	**V3.2-Speciale**
AIME 2025	94.6 (13k)	95.0 (15k)	94.5 (24k)	93.1 (16k)	**96.0 (23k)**
HMMT Feb 2025	88.3 (16k)	97.5 (16k)	89.4 (31k)	92.5 (19k)	**99.2 (27k)**
Codeforces	2537 (29k)	2708 (22k)	-	2386 (42k)	**2701 (77k)**
HLE	26.3 (15k)	37.7 (15k)	23.9 (24k)	25.1 (21k)	30.6 (35k)

Speciale vượt Gemini-3.0-Pro ở nhiều mục nhưng tốn nhiều token đầu ra hơn đáng kể (ví dụ Codeforces: 77k so với 22-29k của hai mô hình đóng).

Bảng 5 (ablation tổng quát hóa agentic, Pass@K trên 50 tác vụ synthetic mẫu):

Pass@K	DeepSeek-V3.2-Exp	Sonnet-4.5	Gemini-3.0 Pro	GPT-5-Thinking
1	12%	34%	51%	62%
2	18%	47%	65%	75%
4	26%	62%	74%	82%

Dù là mô hình tự tổng hợp dữ liệu, DeepSeek-V3.2-Exp chỉ giải được 12% ở Pass@1: paper dùng đây làm luận cứ rằng dữ liệu synthetic thật sự khó, không phải "dễ với chính mô hình sinh ra nó".

8. Phân tích / ablation

- **RL trên agentic task synthetic có tổng quát hóa thật** (Hình 5): so DeepSeek-V3.2-SFT (chỉ SFT) với bản train thêm RL *chỉ* trên synthetic general-agent data (non-thinking) và với DeepSeek-V3.2-Exp (RL trên search + code). RL trên dữ liệu synthetic tạo cải thiện đáng kể trên Tau2Bench, MCP-Mark, MCP-Universe; còn RL chỉ bó hẹp ở code/search thì **không** cải thiện các benchmark này. Paper dùng kết quả này làm luận cứ cho giá trị của agentic task synthesis.

- **Context management cho search agent** (Hình 6, BrowseComp): Summary kéo số bước thực tế trung bình lên 364 bước, đạt cải thiện tới 60.2 điểm nhưng hiệu quả tổng thể thấp; Discard-all đạt 67.6 điểm, hiệu quả và khả năng mở rộng tốt (so sánh được với parallel scaling) nhưng dùng ít bước hơn.
- **Thinking vs non-thinking** (Bảng 9, Phụ lục C): non-thinking luôn kém thinking (ví dụ Terminal Bench 2.0: 37.1 so với 46.4; MCP-Mark: 26.5 so với 38.0) nhưng vẫn cạnh tranh được.

9. Giới hạn

Do tác giả nêu: (1) kiến thức thế giới (world knowledge) còn kém mô hình đóng vì tổng FLOPs tiền huấn luyện ít hơn; (2) hiệu suất token (token efficiency) kém Gemini-3.0-Pro, cần nhiều token sinh hơn để đạt chất lượng tương đương; (3) giải bài toán phức tạp vẫn kém mô hình biên giới (frontier) đóng; (4) với tool-use, model "tự kiểm tra lại dư thừa" (redundant self-verification) sinh trajectory quá dài, dễ vượt 128K context, làm giảm điểm (nhất là MCP-Mark GitHub/Playwright).

Nhận xét:

- Lightning indexer vẫn là $O(L^2)$; paper không cho số FLOPs cụ thể để so trực tiếp với MLA đầy đủ, chỉ nói "ít hơn nhiều". Không có ablation cô lập đo riêng chi phí indexer so với tổng chi phí DSA.
- Dense warm-up cần một checkpoint **dense đã huấn luyện tốt** làm "giáo viên" cho indexer (qua KL); nếu không có checkpoint dense mạnh từ trước, công thức này không áp dụng trực tiếp cho train-from-scratch (đây là lý do paper luôn gọi là "continued pre-training", không phải "pre-training với DSA từ đầu").
- Việc detach gradient của indexer khỏi mô hình chính (mục 4.3) nghĩa là chất lượng chọn token của indexer **hoàn toàn** phụ thuộc độ trung thực của KL distillation; nếu phân phối attention của mô hình chính trôi đi trong lúc sparse training, paper không có cơ chế nào buộc indexer phải "đuổi theo" ngoài việc tiếp tục tối thiểu hóa $\mathcal{L}_I$ trên tập đã chọn (một dạng circular: indexer dạy theo điểm mà chính nó vừa dùng để chọn).
- Paper **không so sánh trực tiếp DSA với NSA (bài 64) hoặc MoBA (bài 65)** ở cùng quy mô/benchmark; NSA chỉ được trích dẫn cho một ràng buộc kernel, không phải một baseline thực nghiệm.
- Hình 3 (chi phí) chỉ là đồ thị định tính trong bản text đã trích xuất, không có bảng số đi kèm để kiểm chứng lại.

10. Liên hệ với các bài khác

- **16 (MLA, DeepSeek-V2):** nền tảng bắt buộc. DSA cài trên latent c_t^{KV} của MLA, ở chế độ MQA (Phụ lục A của paper này minh họa lại sự khác biệt MHA/MQA mode mà bài 16 đã suy diễn qua "hấp thụ ma trận").
- **64 (NSA):** cùng mục tiêu "sparse attention học được, train trực tiếp"; NSA dùng ba nhánh (compress/select/window) tính sẵn mẫu thưa, DSA dùng một indexer học để chấm điểm rồi top-k. Paper V3.2 trích NSA cho ràng buộc "KV entry phải chia sẻ giữa nhiều query ở cấp kernel".
- **65 (MoBA):** một cách khác để chọn "block" quan trọng bằng độ tương tự thô (không có mạng indexer riêng); đáng so với DSA về độ phức tạp cài đặt và khả năng học.
- **17 (Longformer), 55 (StreamingLLM):** sparse attention theo **mẫu cố định** (cửa sổ, sink), đối lập với DSA là mẫu **động, theo nội dung** (indexer quyết định mỗi lần).
- **46 (PagedAttention):** trực giao; dù DSA giảm số token cần attend, hệ thống serving vẫn cần quản lý trang cho KV cache của các token được chọn.
- **69 (GLM-5):** áp lại chính DSA này (continued pretraining tương tự), có ablation so với sliding-window search-based và Gated DeltaNet ở cùng checkpoint.
- **70 (MiniCPM-SALA):** hướng khác hẳn: sparse attention (InfLLM-V2, không nén latent) xen với linear attention toàn phần (Lightning Attention), không dùng MLA.

- **rl 07 (PPO), 36 (DeepSeekMath/GRPO), 39 (DeepSeek-R1), 41 (DAPO), 42 (Dr. GRPO), 43 (GSPO):** nền RL cho phần post-training tóm tắt ở mục 6.

11. Tóm tắt công thức / khái niệm

Khái niệm	Công thức / ý nghĩa
Điểm indexer	$I_{t,s} = \sum_{j=1}^{H^I} w_{t,j}^I \cdot \text{ReLU}(q_{t,j}^I \cdot k_s^I)$
Chọn top-k	$u_t = \text{Attn}(h_t, \{c_s \mid I_{t,s} \in \text{Top-k}(I_{t,:})\})$
Độ phức tạp	$O(L^2) \to O(Lk)$ cho attention chính; indexer vẫn $O(L^2)$ nhưng rẻ
DSA dưới MLA	dùng MQA mode (bài 16); latent c_t^{KV} chia sẻ cho mọi query head
Dense warm-up	đóng băng mô hình chính, chỉ train indexer; $\mathcal{L}_I = \sum_t D_{KL}(p_{t,:} \Vert \text{Softmax}(I_{t,:}))$; 1000 bước, 2.1B token
Sparse training	train toàn mô hình + indexer (gradient tách biệt); $k = 2048$; 15000 bước, 943.7B token
Chi phí suy luận	H800, \$2/giờ-GPU; chi phí tăng theo vị trí token, V3.2 < V3.1-Terminus ở ngữ cảnh dài (Hình 3, không có bảng số)
Parity	AA-LCR: V3.2-Exp +4 điểm (reasoning mode) so với V3.1-Terminus; ChatbotArena Elo khớp nhau
RL (tóm tắt)	GRPO + Unbiased KL + Off-Policy Sequence Masking + Keep Routing + Keep Sampling Mask

12. Kết luận ngắn

DeepSeek Sparse Attention biến bài toán "attention thưa học được" thành hai module tách biệt: một lightning indexer rẻ chấm điểm mọi token trước đó, và một bước top-k giữ lại k mục để attention thật xử lý; cả hai được gắn vào MLA ở chế độ MQA và huấn luyện qua continued pre-training hai giai đoạn (dense warm-up rồi sparse training) để không phải train lại từ đầu. Theo paper, cách này giữ được chất lượng gần như ngang DeepSeek-V3.1-Terminus trong khi giảm hẳn chi phí ở ngữ cảnh dài.

Link gốc: https://arxiv.org/abs/2512.02556

Bài 69

GLM-5: from Vibe Coding to Agentic Engineering (2026)

Phạm vi ghi chú: tập trung vào **kiến trúc** của GLM-5 và cách nó áp dụng lại DSA (DeepSeek Sparse Attention, bài 68): continued pre-training hai giai đoạn, ablation DSA so với sliding-window search-based và Gated DeltaNet, kernel chuyên dụng, và kết quả ngữ cảnh dài. Phần RL (hạ tầng bất đồng bộ, thuật toán) và dữ liệu huấn luyện chỉ tóm tắt ngắn ở mục 5. Mục 10 là một phần bổ sung rõ ràng, tổng hợp từ **nguồn ngoài arXiv** (docs.z.ai, Hugging Face) về GLM-5.3 và GLM-5.3-Flash, ra đời sau paper này.

Tác giả: GLM-5 Team, Zhipu AI & Tsinghua University (danh sách đầy đủ ở mục Contribution cuối paper).
Venue / arXiv: arXiv:2602.15763 (v2, 24/02/2026); paper không nêu venue hội nghị.
Nguồn: https://arxiv.org/abs/2602.15763
Chủ đề: mô hình 744B tổng / 40B kích hoạt áp dụng DSA qua continued pre-training; ablation các cơ chế attention hiệu quả (DSA, sliding-window search-based, Gated DeltaNet/SimpleGDN); RL bất đồng bộ cho agent.
Vị trí trong lộ trình: đọc sau 68 (DeepSeek-V3.2, nguồn gốc DSA) **bắt buộc**. Nên đọc sau 16 (MLA), 20 (GLA), 23-24 (DeltaNet, Gated DeltaNet) để hiểu các phương án thay thế mà GLM-5 so sánh; sau 17 (Longformer)/06 (Mistral, sliding window) để hiểu baseline SWA. Đứng trước 70 (MiniCPM-SALA), vì cả hai bài cùng bàn về hybrid sparse + linear attention, và trước 71 (DeepSeek-V4).

> Tài liệu học tập bằng tiếng Việt, là ghi chú diễn giải, không phải bản dịch. Số liệu lấy từ paper (full text arXiv, tải ngày 07/11/2026); các phép tính đánh dấu "tự tính" là của người ghi chú. Thông tin ở mục 10 lấy từ trang tài liệu/model card chính thức (docs.z.ai, Hugging Face), không phải từ một paper, và được ghi rõ nguồn cho từng chi tiết.

1. Bối cảnh

GLM-5 kế tiếp GLM-4.5 (355B tham số tổng, 32B kích hoạt). Bài báo trình bày bốn đóng góp: áp dụng DSA để giảm chi phí huấn luyện và suy luận; hạ tầng RL bất đồng bộ slime; thuật toán Agent RL bất đồng bộ; và tối ưu cho các chip Ascend, Moore Threads, Hygon, Cambricon, Kunlunxin, MetaX và Enflame. Phần trình bày dưới đây tập trung vào DSA.

Scale kiến trúc. GLM-5 mở rộng lên **256 expert**, giảm số layer để hạn chế overhead giao tiếp expert-parallel, tạo mô hình **744B tổng tham số / 40B kích hoạt**, gấp đôi tổng tham số GLM-4.5. Văn bản phần giới thiệu nói "giảm số layer xuống 80", nhưng Bảng 10 (Phụ lục A) liệt kê 3 layer dense + 75 layer MoE + 1 layer MTP = **79** layer; cả hai số được giữ lại (xem mục 8).

Khái niệm cần nắm trước: MLA và hai chế độ MHA/MQA (bài 16); DSA, lightning indexer, dense warm-up, sparse training (bài 68); sliding-window attention (bài 06, 17); linear attention dạng Gated DeltaNet (bài 23, 24); tìm kiếm cấu trúc bằng beam search kiểu PostNAS/Jet-Nemotron (không có bài riêng trong lộ trình, paper trích [15]).

2. Ý tưởng chính

GLM-5 không tự thiết kế một cơ chế sparse attention mới: nó **áp lại DSA của DeepSeek-V3.2** (bài 68) thông qua continued pre-training, với ngân sách token nhỏ hơn hẳn (20B so với 943.7B của DeepSeek, xem mục 4). Điểm đóng góp riêng của GLM-5 là một **bộ ablation có kiểm soát**: trên cùng một checkpoint baseline (GLM-9B, full attention, đã fine-tune ngữ cảnh 128K), nhóm tác giả so sánh bốn cách giảm chi phí attention: sliding-window xen cố định, sliding-window chọn layer bằng beam search, Gated DeltaNet (GDN) và biến thể đơn giản hóa SimpleGDN, tất cả đều train bằng **continual-training từ checkpoint full-attention có sẵn**

(không train từ đầu). Kết luận chính: mọi phương án "thay layer bằng cơ chế khác" đều mất điểm ở ngữ cảnh dài vì **mất thông tin không thể phục hồi**; chỉ DSA, nhờ giữ nguyên attention đầy đủ về mặt logic (chỉ bỏ bớt token được chọn bởi indexer) và áp dụng ở **mọi** layer, mới gần như không mất gì.

3. Phương pháp chi tiết: kiến trúc nền (MLA, Muon Split, MTP)

3.1 MLA không khớp GQA khi dùng Muon: Muon Split

Nhóm tác giả quan sát: khi dùng optimizer Muon, **MLA với KV-cache latent 576 chiều không khớp hiệu năng GQA-8** (8 nhóm, KV-cache 2048 chiều). Nguyên nhân (giả thuyết của paper): Muon gốc áp **trực giao hóa** cho toàn bộ ma trận up-projection W^{UQ}, W^{UK}, W^{UV} (bài 16, §3.2) như một khối lớn, không tách theo head. Giải pháp **Muon Split**: tách các ma trận này thành khối nhỏ theo từng head, trực giao hóa **độc lập từng khối**, cho phép các head học ở tỉ lệ khác nhau.

Bảng 1 của paper (đánh giá base model, một phần):

Dataset	Hellaswag	MMLU	C-Eval	RACE	BBH	GSM8K	HumanEval
GQA-8	77.3	61.2	60.0	79.6	53.3	47.6	38.5
MLA	77.3	61.5	59.7	77.8	48.9	46.2	33.5
MLA + Muon Split	77.8	62.5	62.1	79.9	51.8	45.0	36.7
MLA-256 + Muon Split	77.4	62.0	59.9	79.6	51.3	47.5	36.6

MLA thường (không Muon Split) thua GQA-8 ở BBH (48.9 so với 53.3) và HumanEval (33.5 so với 38.5); thêm Muon Split thu hẹp phần lớn khoảng cách. Thêm lợi ích thực hành: với Muon Split, "thang đo attention logit của GLM-5 ổn định trong suốt pre-training mà không cần chiến lược clipping" (paper, diễn giải).

3.2 MLA-256: giảm chi phí decode

MLA chuẩn tính dot-product **576 chiều** lúc decode (cao hơn 128 chiều của GQA). Vì số head attention trong DeepSeek-V3 được chọn theo roofline của GPU H800, không hợp với phần cứng khác, GLM-5 **tăng head dimension từ 192 lên 256 và giảm số head đi 1/3**, giữ nguyên lượng tính toán lúc train/prefill (chế độ MHA của MLA, bài 16) nhưng giảm tính toán lúc decode. Biến thể này gọi là **MLA-256** (xem dòng cuối Bảng 1), đạt hiệu năng ngang MLA + Muon Split.

3.3 Multi-Token Prediction chia sẻ tham số

MTP (dự đoán nhiều token tiếp theo, draft model cho speculative decoding, bài 48) thường cần n layer riêng để dự đoán n token, khiến tham số và KV-cache tăng tuyến tính theo số bước. DeepSeek-V3 chỉ train 1 layer MTP, dự đoán 2 token lúc suy luận, nhưng gây lệch train/inference, giảm accept rate ở token thứ hai. GLM-5 **chia sẻ tham số giữa 3 layer MTP khi train**, giữ chi phí nhớ ngang DeepSeek-V3 nhưng tăng accept rate.

Bảng 2 của paper (accept length, 4 bước suy đoán, bộ prompt nội bộ):

Model	Accept Length
DeepSeek-V3.2	2.55
GLM-5	2.76

4. Phương pháp chi tiết: áp dụng DSA cho GLM-5

4.1 Vì sao DSA, và kết quả so với MLA (Bảng 3)

Triết lý DSA (dẫn lại từ bài 68): thay attention đặc $O(L^2)$ bằng cơ chế chọn token mịn, động theo nội dung (không phải mẫu cố định như sliding window). Paper nhấn mạnh DSA được đưa vào qua **continued pre-training từ một mô hình dense có sẵn**, tránh chi phí "thiên văn" của việc train từ đầu.

Bảng 3 của paper (so sánh MLA và DSA base model, benchmark ngữ cảnh dài 128K):

	MQ-NIAH-128k	MV-NIAH-128k	SQuAD-128k	HotpotQA-128k
MLA	100.0	95.5	79.7	66.3
DSA	100.0	97.0	86.0	63.0

Mô hình DSA đạt hiệu năng ngữ cảnh dài **gần như ngang** mô hình MLA (3/4 cột cao hơn hoặc bằng, chỉ thua ở HotpotQA-128k: 63.0 so với 66.3), dùng để kết luận "90% mục attention ở ngữ cảnh dài là dư thừa" (paper, diễn giải) và DSA giảm tính toán attention khoảng **1.5-2 lần** cho chuỗi dài.

4.2 Hai giai đoạn: dense warm-up và sparse adaptation

Giống công thức của DeepSeek (bài 68 §4.2-4.3), DSA của GLM-5 bắt đầu từ checkpoint base **ở cuối giai đoạn mid-training** (không phải ngay sau pre-training).

- **Warm-up:** 1000 bước, mỗi bước 14 chuỗi dài **202,752 token**, learning rate tối đa 5×10^{-3}. Tổng token (tự tính): $1000 \times 14 \times 202{,}752 \approx 2.84 \times 10^9 = 2.84$ tỷ token.
- **Sparse adaptation:** theo dữ liệu và siêu tham số của giai đoạn mid-training, tổng **20 tỷ token**. So với DeepSeek-V3.2 (943.7 tỷ token ở giai đoạn sparse training, bài 68 §4.3), ngân sách của GLM-5 nhỏ hơn khoảng **47 lần** (tự tính: $943.7/20 \approx 47.2$). Paper nhận định ngân sách nhỏ này **đủ** để mô hình DSA khớp hiệu năng mô hình MLA gốc.
- **Fine-tune kiểm chứng:** fine-tune cả hai mô hình (DSA và MLA) với cùng dữ liệu SFT; hai mô hình "ngang nhau cả về train loss và benchmark đánh giá" (paper, diễn giải).
- **Đường cong loss (Hình 6):** so SFT loss giữa MLA và DSA, dùng trung bình trượt cửa sổ 50; paper chỉ cho hình, phần text trích xuất không có bảng số loss cụ thể, chỉ xác nhận hai đường "gần trùng khớp" ở cuối.

4.3 Siêu tham số learning rate (Phụ lục A)

Giai đoạn warm-up DSA: learning rate giảm từ 5×10^{-3} xuống 2×10^{-4}. Giai đoạn sparse adaptation: learning rate **hằng số** 1×10^{-5}.

4.4 Lossless "by construction": vì sao DSA khác các phương án thay layer

Paper: khác các phương án "thay hẳn layer" (SWA, GDN, mục 7), DSA "lossless by construction": lightning indexer đạt độ thưa ***ở mức token*** mà không loại bỏ hẳn phụ thuộc tầm xa, áp dụng được cho **mọi** layer mà không giảm chất lượng (ablation ở mục 7).

5. Hạ tầng, kernel và RL (tóm tắt ngắn)

(Phần này nằm ngoài phạm vi chính của ghi chú DSA; chi tiết thuật toán RL/GRPO nền tảng xem lộ trình RL: rl 07 PPO, rl 36 DeepSeekMath/GRPO, rl 39 DeepSeek-R1, rl 41 DAPO, rl 43 GSPO.)

- **Kernel chuyên dụng (Ascend NPU):** ba kernel fusion: **Lightning Indexer** (gộp tính điểm + ReLU + top-k vào một kernel); **Sparse Flash Attention** (chọn top-k từ KV cache và tính attention thưa song song); **MLAPO** (gộp 13 toán tử tiền xử lý nhỏ của MLA thành một "siêu toán tử"). Lượng tử hóa hỗn hợp **W4A8** (attention/MLP ở INT8, expert MoE ở INT4) để chạy 744B tham số trên một máy Atlas 800T A3.

- **RL bất đồng bộ (tóm tắt).** Dựa trên framework "slime", tách sinh dữ liệu khỏi train để tối đa hóa GPU. Thuật toán xây trên GRPO [40] + **IcePop** [61] (giảm lệch train/inference), bỏ KL penalty, clip bất đối xứng $\epsilon_{low} = 0.2, \epsilon_{high} = 0.28$ (tinh thần "clip-higher" của DAPO, rl 41), group=batch=32, hoàn toàn on-policy.
- **DSA RL insights (liên quan trực tiếp DSA):** top-k của indexer cần **nhất quán giữa inference và training**; lưu chỉ số top-k mọi vị trí (như "routing replay" của MoE) không khả thi vì $k = 2048$ quá lớn. Giải pháp: `torch.topk` tất định thay top-k CUDA không tất định (vốn gây sụt entropy, hỏng RL sau vài bước); **đóng băng tham số indexer trong suốt RL**.
- **Post-training khác (tóm tắt):** SFT 3 nhóm dữ liệu (chat, reasoning, coding/agent), ba kiểu "thinking" (Interleaved, Preserved, Turn-level); RL tuần tự Reasoning → Agentic → General, cộng on-policy cross-stage distillation chống catastrophic forgetting.

6. Thí nghiệm và kết quả chính

Bảng 7 của paper (rút gọn, so với mô hình mở/đóng):

Benchmark	GLM-5	GLM-4.7	DeepSeek-V3.2	Claude Opus 4.5	Gemini 3 Pro
HLE	30.5	24.8	25.1	28.4	37.2
LongBench v2	64.5	59.1	59.8	64.4	68.2
SWE-bench Verified	77.8	73.8	73.1	80.9	76.2
Terminal-Bench 2.0 (Terminus-2)	56.2 / 60.7†	41.0	39.3	59.3	54.2
BrowseComp (w/ Context Manage)	75.9	67.5	67.6	57.8	59.2
τ^2-Bench	89.7	87.4	85.3	91.6	90.7

(† = verified Terminal-Bench 2.0.) Paper nhấn mạnh: GLM-5 cao nhất trong số mô hình mở ở hầu hết benchmark, vượt Gemini 3 Pro ở SWE-bench Verified, và là mô hình mở #1 trên LMArena (Hình 3). CC-Bench-V2 (Bảng 8) và SWE-rebench (Bảng 9, tóm tắt): GLM-5 vượt rõ GLM-4.7 nhưng còn khoảng cách với Claude Opus 4.5/4.6 ở tác vụ chuỗi nhiều bước (xem mục 8).

7. Phân tích / ablation: DSA so với SWA, GDN, SimpleGDN

Đây là phần thực nghiệm có giá trị nhất cho mục đích của ghi chú này: so sánh **trực tiếp, cùng checkpoint baseline** (GLM-9B, GQA ở mọi 40 layer, đã fine-tune 128K) giữa bốn cách giảm chi phí attention, tất cả bằng **continual-training conversion**.

7.1 Các phương án được so sánh

- **SWA Interleave:** xen cố định layer full-attention và layer sliding-window, tỉ lệ 1:1, cửa sổ 4096 token.
- **SWA Pattern (search-based):** lấy cảm hứng từ PostNAS, tìm bằng **beam search** (beam size 8, mỗi bước chọn 2 layer) layer nào nên chuyển sang SWA; tìm ở ngữ cảnh 16K (để giảm chi phí), áp dụng mẫu tìm được cho mọi độ dài khác. Với GLM-9B (40 layer), hội tụ sau khoảng 10 bước. Mẫu cuối (paper cho nguyên văn chuỗi 41 ký tự, 20 "F" và 21 "S", trong khi GLM-9B có 40 layer và tỉ lệ công bố là 1:1; con số không khớp hoàn toàn, có thể do lỗi trích xuất/in ấn, không tự sửa): SFSSFFSSSFFFFSSFSFFFFFFSFSFSSFSSFSFSSFSSS (S = SWA, F = full attention).
- **Gated DeltaNet (GDN, bài 24):** thay attention softmax bằng hồi quy tuyến tính có cổng, theo pipeline Jet-Nemotron.

- **SimpleGDN:** biến thể đơn giản hóa GDN để tái dùng tối đa trọng số pre-trained: bỏ hẳn Conv1d và module cổng tường minh, map trực tiếp trọng số Q, K, V đã pretrain vào công thức hồi quy tuyến tính, không thêm tham số mới.

7.2 Không train thêm (Bảng 4, RULER)

	4K	8K	16K	32K	64K	128K
GLM-9B (full attn)	95.19	93.67	92.01	91.09	85.35	75.28
SWA Interleave	94.87	54.02	25.89	12.61	8.32	6.51
SWA Pattern	95.78	92.54	88.92	82.52	70.23	53.95

Không train lại, SWA Interleave **sụp đổ** ngay ở 8K (54.02 so với 93.67); SWA Pattern (tìm layer bằng beam search) giữ được phần lớn chất lượng, dù vẫn giảm dần theo độ dài.

7.3 Có continual-training 190B token (Bảng 5, nhiều benchmark, tỉ lệ 1:1 lớp hiệu quả : lớp full attention)

	RULER 64K/128K	MRCR 64K/128K	HELMET-ICL 64K/128K	RepoQA 64K/128K
GLM-9B	85.35 / 75.28	36.53 / 35.39	77.68 / 77.36	69.00 / 65.83
SWA Interleave	65.94 / 44.93 (↓19.41 / ↓30.35)	30.03 / 28.83 (↓6.50 / ↓6.56)	75.96 / 63.52 (↓1.72 / ↓13.84)	50.33 / 39.33 (↓18.67 / ↓26.50)
SWA Pattern	83.72 / 69.59 (↓1.63 / ↓5.69)	35.02 / 33.58 (↓1.51 / ↓1.81)	76.48 / 74.60 (↓1.20 / ↓2.76)	62.33 / 51.17 (↓6.67 / ↓14.66)
GDN	76.76 / 64.00 (↓8.59 / ↓11.28)	31.72 / 30.22 (↓4.81 / ↓5.17)	76.88 / 74.84 (↓0.80 / ↓2.52)	65.50 / 56.17 (↓3.50 / ↓9.66)
SimpleGDN	81.76 / 67.03 (↓3.59 / ↓8.25)	33.03 / 31.27 (↓3.50 / ↓4.12)	79.80 / 81.84 (↑2.12 / ↑4.48)	65.50 / 58.50 (↓3.50 / ↓7.33)

Thứ tự đổi lấy chất lượng (paper, diễn giải): SWA Interleave kém nhất (dù train lại); SWA Pattern thu hẹp khoảng cách đáng kể nhờ giữ full-attention đúng chỗ cần; GDN thêm tham số nhưng vẫn mất 8-11 điểm RULER; SimpleGDN (tái dùng tối đa trọng số pretrained, không tham số mới) cân bằng tốt nhất trong nhóm thay-layer, thậm chí **vượt** baseline ở HELMET-ICL. Mọi phương án trong bảng này vẫn mất tới 5.69 điểm (RULER@128K) hoặc 14.66 điểm (RepoQA@128K) vì **mất thông tin không thể phục hồi** khi chuyển layer, dù đã giữ nửa số layer là full attention.

7.4 DSA: “lossless by construction” (Bảng 6, GLM-4.7-Flash)

Thí nghiệm quy mô nhỏ trên GLM-4.7-Flash (có MLA): warm-up indexer 1000 bước (batch 16, đóng băng mô hình chính), rồi joint-training 150 tỷ token.

	4K	8K	16K	32K	64K	128K
GLM-4.7-Flash (gốc)	97.44	96.72	95.83	92.96	85.34	79.21
+ DSA warm-up (chỉ indexer)	97.51	96.54	95.40	90.09	84.05	71.35
+ DSA (joint-training đủ 150B token)	96.77	96.25	96.69	93.45	87.06	78.86

Chỉ warm-up indexer (chưa train lại mô hình chính) đã giữ lại phần lớn chất lượng gốc, chỉ giảm rõ ở 128K (79.21 → 71.35). Sau joint-training đầy đủ, DSA **vượt** baseline ở 16K (+0.86), 32K (+0.49), 64K (+1.72), và

chỉ còn thiếu 0.35 điểm ở 128K. Đây là cơ sở cho câu "lossless by construction" (mục 4.4): khác các phương án thay layer (mục 7.2-7.3), DSA không có một khoảng mất điểm "không thể đóng" khi ngân sách training tăng lên.

8. Giới hạn

Do tác giả nêu:

- Dù vượt GLM-4.7 rõ rệt, GLM-5 vẫn thua Claude Opus 4.5 ở tác vụ **chuỗi nhiều bước** (multi-step chained tasks, tự xây từ PR đã merge): "lỗi tích lũy qua chuỗi: một sửa đổi không tối ưu ở một tác vụ có thể âm thầm phá test ở các tác vụ sau" (paper, diễn giải). Cần tiến bộ về tính nhất quán ngữ cảnh dài và tự sửa lỗi dài hạn.
- Token efficiency, world knowledge vẫn còn khoảng cách với mô hình đóng biên giới (không có số cụ thể, chỉ nêu định hướng nghiên cứu tiếp).

Nhận xét:

- **Mâu thuẫn số layer:** phần giới thiệu nói "giảm xuống 80 layer", nhưng Bảng 10 cho 3 (dense) + 75 (MoE) + 1 (MTP) = 79. Không rõ đâu là số chính xác từ paper; cả hai số được giữ lại vì chưa rõ số nào đúng số "đúng".
- **Chuỗi mẫu SWA 41 ký tự cho 40 layer:** mục 7.1 đã nêu; không khớp số layer hay tỉ lệ 1:1 được công bố. Có thể là lỗi trích xuất ký tự trùng lặp khi chuyển PDF sang text, hoặc lỗi của chính paper; không tự sửa.
- **Ablation DSA dùng hai mô hình khác nhau ở hai quy mô khác nhau:** Bảng 3 (MLA vs DSA) có vẻ dùng chính mô hình GLM-5 kích thước đầy đủ (744B/40B, vì nằm trong phần "Continued Pre-Training" chính của GLM-5), còn ablation SWA/GDN/SimpleGDN (Bảng 4, 5) và cả thí nghiệm DSA minh họa thêm (Bảng 6) dùng **GLM-9B và GLM-4.7-Flash**, hai mô hình nhỏ hơn hẳn. Vì vậy phần "DSA thắng các phương án khác" (mục 7) là một **so sánh gián tiếp** giữa hai bộ thí nghiệm ở hai quy mô khác nhau, không phải một bảng ablation duy nhất chạy hết các phương án trên cùng một mô hình 744B.
- Paper không cho số FLOPs/latency cụ thể của riêng kernel DSA trên Ascend NPU (chỉ mô tả cơ chế), nên không so sánh định lượng được với số liệu H800 của DeepSeek (bài 68).

9. Liên hệ với các bài khác

- **68 (DeepSeek-V3.2):** nguồn gốc DSA; GLM-5 lặp lại công thức hai giai đoạn với ngân sách nhỏ hơn ~47 lần.
- **16 (MLA):** nền của GLM-5 (MLA-256, Muon Split) và của DSA (indexer chọn trên latent MLA).
- **23, 24 (DeltaNet, Gated DeltaNet):** cơ sở GDN/SimpleGDN trong ablation mục 7, dùng làm baseline "thay full-attention" để so với DSA.
- **17 (Longformer), 06 (Mistral):** cơ sở sliding-window; ablation mục 7 cho thấy SWA naive là phương án tệ nhất, SWA chọn layer bằng tìm kiếm đỡ hơn nhiều.
- **48 (Speculative decoding):** MTP của GLM-5 (mục 3.3) dùng làm draft model.
- **rl 41 (DAPO), rl 43 (GSPO):** clip bất đối xứng của GLM-5 (mục 5) cùng tinh thần "clip-higher" của DAPO.
- **70 (MiniCPM-SALA):** hướng hybrid khác (sparse + linear cùng lúc, không "chọn một trong hai" như ablation của GLM-5).

10. Sau paper: GLM-5.3 và GLM-5.3-Flash (nguồn ngoài arXiv)

Mục này **không** đến từ paper GLM-5 (arXiv:2602.15763). Mọi chi tiết dưới đây lấy từ trang tài liệu chính thức (`docs.z.ai/guides/vlm/glm-5.3-flash`) và từ model card/`config.json` trên Hugging Face (`huggingface.co/zai-org/GLM-5.3`, `huggingface.co/zai-org/GLM-5.3-Flash`), truy cập 07/11/2026. Mỗi chi tiết ghi rõ nguồn; những gì không xác minh được từ nguồn chính thức được nói rõ là "chưa xác minh", không đoán số.

10.1 GLM-5.3

- **Hugging Face model card:** "dùng lại đúng base model của GLM-5.2; mọi cải tiến đến từ post-training" (so với GLM-5.2, một bản không có trong paper này; không rõ quan hệ GLM-5.2 với GLM-5).
- **Giữ DSA, xác minh bằng `config.json` (tải trực tiếp):** `architectures: GlmMoeDsaForCausalLM`; `index_topk: 2048`, `index_n_heads: 32`, `index_head_dim: 128`; không có trường `layer_types` (mọi layer dùng DSA, không hybrid). Các siêu số khác (hidden_size 6144, q_lora_rank 2048, kv_lora_rank 512, 256 expert, vocab 154,880, qk_rope_head_dim 64) khớp Bảng 10 của paper; `num_hidden_layers: 78` (+1 MTP = 79, khớp cách tính ở mục 1, không khớp số "80" trong văn bản paper).
- **Tổng tham số:** trang Hugging Face (Safetensors) hiển thị **753B**, cao hơn 744B mà paper GLM-5 công bố; không xác minh được lý do chênh lệch, không đoán.
- **License, ngày phát hành:** model card chỉ gắn tag "License: glm-5.3", không có điều khoản cụ thể; không có ngày phát hành. Theo tin tức bên ngoài (**chưa xác minh từ Zhipu chính thức**): công bố khoảng 14/08/2026, API khoảng 19/08/2026, trọng số mở cuối tháng 8/2026.

10.2 GLM-5.3-Flash

- **docs.z.ai + Hugging Face model card (khớp nhau): 320B tổng / 18B kích hoạt**; mô hình natively multimodal đầu tiên của dòng GLM; "lần đầu trong dòng GLM: kiến trúc hybrid sparse + linear attention" (nguyên văn ý). docs.z.ai: linear attention "nắm phụ thuộc cục bộ qua state modeling", sparse attention "truy xuất bối cảnh toàn cục qua một indexer nhẹ" (cùng nguyên lý lightning indexer của DSA, dù hai trang không gọi tên "DSA").
- **Xác minh bằng `config.json` (nguồn sơ cấp, tải trực tiếp):** `architectures: Glm5NextForConditionalGeneration`; `layer_types` có **45 phần tử**: **34 "linear_attention" + 11 "deepseek_sparse_attention"**, mẫu lặp cố định "3 linear rồi 1 DSA". Tỉ lệ $11/45 \approx 24\%$ sparse, $34/45 \approx 76\%$ linear: **xấp xỉ đúng 1:3** (25%/75%) mà bài 70 (MiniCPM-SALA) dùng cho InfLLM-V2 + Lightning Attention. Đây là bằng chứng sơ cấp rằng GLM-5.3-Flash dùng đúng tên layer `deepseek_sparse_attention` xen với linear attention, khác GLM-5.3 và GLM-5 (DSA ở mọi layer).
- **Chi tiết khác từ `config.json`:** `mla_use_nope: true`, `qk_rope_head_dim: 0` ở layer DSA (bỏ RoPE, giống HyPE của bài 70); `mhc: true` (Manifold-Constrained Hyper-Connections, như docs.z.ai mô tả); `index_topk: 2048`, `index_n_heads: 32`, `index_head_dim: 128`; `hidden_size: 4096`, `num_hidden_layers: 45`.
- **IndexPool (docs.z.ai):** gộp 4 vector key indexer thành 1 (weighted pooling), giảm độ trễ/bộ nhớ indexer ở 1M token. So với GLM-5.3: giảm tính toán attention và KV cache **3.01× và 4.44×** (không nêu rõ ở độ dài nào).
- **Ngữ cảnh:** 1M token đầu vào, 128K token đầu ra tối đa (docs.z.ai). **License: MIT** (model card).
- **Ngày phát hành:** không nêu trong nguồn chính thức đọc được; theo tin tức bên ngoài (chưa xác minh), khoảng 26/08/2026.
- **Không xác minh được, nêu rõ để không đoán:** docs.z.ai có câu "45 layer so với 92 layer của mô hình so sánh" nhưng không nói rõ mô hình nào; `config.json` của GLM-5.3 được tham khảo chỉ cho 78 layer, không phải 92, nên không khẳng định thêm.
- **Chi tiết phụ:** GLM-5 (paper, "Easter Eggs": ẩn danh trên OpenRouter tên "Pony Alpha") và GLM-5.3-Flash (docs.z.ai: ẩn danh trên OpenCode/OpenRouter tên "ox-alpha") đều được thử ẩn danh trước khi công bố.

11. Tóm tắt công thức / khái niệm

Khái niệm	Công thức / ý nghĩa
Scale GLM-5	744B tổng / 40B kích hoạt, 256 expert; 79 layer theo Bảng 10 (văn bản nói 80)

Khái niệm	Công thức / ý nghĩa
Muon Split	trực giao hóa riêng từng khối ma trận theo head, không theo khối lớn
MLA-256	head_dim 192 → 256, số head giảm 1/3; giảm chi phí decode, giữ chi phí train
MTP chia sẻ tham số	3 layer MTP dùng chung tham số khi train; accept length GLM-5 2.76 vs DeepSeek-V3.2 2.55
DSA cho GLM-5	continued pre-training: warm-up 2.84B token (1000×14×202,752) + sparse adaptation 20B token
So với DeepSeek	ngân sách sparse adaptation của GLM-5 ít hơn DeepSeek-V3.2 (943.7B) khoảng 47 lần
"Lossless by construction"	DSA (Bảng 6): warm-up-only mất tối đa ở 128K (79.21 → 71.35); full training vượt baseline ở 16-64K
So với SWA/GDN	mọi phương án thay layer (Bảng 4, 5) đều mất điểm không thể đóng hoàn toàn ở 64-128K, dù continual-train 190B token
RL backbone	GRPO + IcePop, bỏ KL penalty, clip bất đối xứng $\epsilon_{low} = 0.2, \epsilon_{high} = 0.28$, group=batch=32, on-policy
DSA RL insight	top-k phải tất định (`torch.topk`) lúc RL; đóng băng indexer trong RL
GLM-5.3 (nguồn ngoài arXiv)	tag `glm_moe_dsa`; "base = GLM-5.2, cải tiến chỉ từ post-training" (HF card)
GLM-5.3-Flash (nguồn ngoài arXiv)	320B/18B; `layer_types`: 34 linear_attention + 11 deepseek_sparse_attention (tỉ lệ ≈1:3); NoPE ở layer DSA; MIT

12. Kết luận ngắn

GLM-5 không phát minh cơ chế sparse attention mới; nó dùng lại DSA của DeepSeek-V3.2 (bài 68) qua continued pre-training với ngân sách nhỏ hơn hẳn. Đóng góp lớn nhất cho mục tiêu ghi chú này là một ablation có kiểm soát: so DSA với sliding-window chọn layer bằng tìm kiếm, Gated DeltaNet và SimpleGDN, qua continual-training từ cùng checkpoint full-attention. Kết luận: các phương án "thay layer" luôn mất điểm không thể đóng hoàn toàn ở ngữ cảnh dài, còn DSA gần như không mất gì.

Link gốc: https://arxiv.org/abs/2602.15763

Bài 70

MiniCPM-SALA: Hybridizing Sparse and Linear Attention for Efficient Long-Context Modeling (2026)

Phạm vi ghi chú: tập trung vào kiến trúc hybrid sparse attention (InfLLM-V2) + linear attention (Lightning Attention): tỉ lệ layer 1:3, cách chọn layer, cơ chế HyPE (RoPE chỉ ở linear, NoPE ở sparse), công thức chuyển đổi/continued training từ checkpoint Transformer có sẵn, và kết quả ngữ cảnh dài/hiệu năng suy luận.

Tác giả: MiniCPM Team (OpenBMB, phối hợp XCORE SIGMA cho một số tác giả).
Venue / arXiv: arXiv:2602.11761 (v2, 28/02/2026); paper không nêu venue hội nghị.
Nguồn: https://arxiv.org/abs/2602.11761
Chủ đề: mô hình 9B tham số, hybrid 25% sparse attention (InfLLM-V2) + 75% linear attention (Lightning Attention), chuyển đổi từ checkpoint Transformer (MiniCPM-4.0) bằng continual training, hỗ trợ ngữ cảnh tới 1M-2048K token.
Vị trí trong lộ trình: đọc sau 18 (linear attention = RNN), 20 (GLA) để hiểu Lightning Attention; sau 17 (Longformer), 64 (NSA) để hiểu sparse attention theo block; sau 24 (Gated DeltaNet), 26 (Kimi Linear) để so với các hybrid khác. Nên đọc sau 68, 69 (DSA, GLM-5) để so hai cách "kết hợp nhiều cơ chế attention": DSA áp cho mọi layer còn SALA chia layer theo tỉ lệ cố định.

> Tài liệu học tập bằng tiếng Việt, là ghi chú diễn giải, không phải bản dịch. Số liệu lấy từ paper (full text arXiv, tải ngày 07/11/2026); các phép tính đánh dấu "tự tính" là của người ghi chú.

1. Bối cảnh

Xử lý tài liệu dài, mã nguồn lớn hoặc lịch sử làm việc nhiều lượt đòi hỏi ngữ cảnh dài. Với Transformer, attention chuẩn tốn $O(N^2)$ phép tính theo độ dài N, còn KV cache phải lưu lịch sử. Bài báo lưu ý rằng ngay cả một mô hình 8B dùng GQA (bài 15) cũng có thể cần hàng chục đến hàng trăm GB cache ở ngữ cảnh cỡ triệu token.

Hai hướng xử lý đã có, mỗi hướng có giới hạn riêng (paper gọi là nghịch lý):

- **Sparse attention** (bài 17 Longformer, 55 StreamingLLM/attention sink, 64 NSA...): chỉ tính phần "nổi bật" của ma trận attention (cửa sổ trượt, global anchor...). Nhưng paper chỉ ra giới hạn "**sparse computation, dense storage**": tính ít hơn, nhưng vẫn phải **giữ nguyên toàn bộ KV-cache** để có thể truy xuất lại, nên không giảm bộ nhớ.
- **Linear attention** (bài 18, 20, 21-24): nén lịch sử vào một trạng thái hồi quy kích thước cố định, đưa độ phức tạp về $O(N)$. Nhưng đây là **nén có mất thông tin** (lossy), luôn kèm suy giảm chất lượng ở các tác vụ cần truy xuất chính xác tầm xa.

Khái niệm cần nắm trước: linear attention dạng RNN với trạng thái ma trận (bài 18), gating (bài 20), NoPE (không dùng RoPE, để lại ở bài 12), QK-Norm (chuẩn hóa Q, K trước khi nhân, bài 67 có nhắc "attention sink"), continual training/distillation từ checkpoint Transformer có sẵn (quen thuộc từ bài 16's tinh thần low-rank, nhưng ở đây là đổi *loại* layer, không chỉ nén).

2. Ý tưởng chính

MiniCPM-SALA không chọn một trong hai hướng, mà **trộn cả hai trong cùng một mô hình**: 25% layer dùng sparse attention (InfLLM-V2, không nén, không thêm tham số, chuyển được qua lại dense/sparse), 75% layer dùng linear attention (Lightning Attention, gần với softmax attention nhất trong các biến thể linear nên dễ "kế thừa" trọng số pretrained). Vị trí của 25% layer sparse không xếp xen đều mà được chọn bằng **thuật**

toán lựa chọn layer (dẫn từ công trình liên quan, Chen et al. 2026). Quan trọng hơn: mô hình không train từ đầu, mà **chuyển đổi liên tục (continual training)** từ một **checkpoint giữa đường** của MiniCPM-4.0 (đã train 7T token trong tổng số 8T token mà MiniCPM-4.0 cần để train xong), qua 5 giai đoạn cộng thêm khoảng 2T token nữa, giảm khoảng 75% chi phí training **so với train một mô hình tương đương từ đầu** (2T so với 8T; phần 7T đã train cho MiniCPM-4.0 được coi là chi phí đã trả trước, không tính lại). Chi tiết số liệu ở mục 4.2.

3. Phương pháp chi tiết: kiến trúc hybrid và HyPE

3.1 Tỉ lệ 1:3 và lý do chọn layer sparse/linear

Lấy cảm hứng từ Qwen3-Next và Kimi Linear (bài 26) cộng thực nghiệm nội bộ quy mô nhỏ, paper chọn tỉ lệ **25% layer sparse attention (InfLLM-V2) : 75% layer linear attention (Lightning Attention)**. Mỗi khối attention (sparse hoặc linear) vẫn theo sau bởi một khối FFN như Transformer chuẩn (giữ dung lượng tri thức). Vị trí các layer sparse **không xen đều**, mà xác định bằng cơ chế lựa chọn layer của Chen et al. (2026) (paper không nêu lại chi tiết thuật toán này trong văn bản, chỉ trích dẫn), cho hiệu năng downstream tốt hơn xen đều.

3.2 InfLLM-V2 (sparse) và Lightning Attention (linear): vì sao chọn hai cơ chế này

- **InfLLM-V2** (Zhao et al. 2025, bài ngoài lộ trình): ưu điểm chính được paper nêu là **không thêm tham số mới** và có thể **chuyển đổi mượt giữa chế độ dense và sparse**. Tính tương thích này giúp các layer sparse **kế thừa trực tiếp trọng số dense** khi bắt đầu chuyển đổi, không gây xung đột kiến trúc.
- **Lightning Attention** (Qin et al. 2024, bài ngoài lộ trình): được chọn vì "gần gũi về chức năng với softmax attention chuẩn" nhất trong số các linear attention, giúp việc thích nghi tham số từ checkpoint Transformer đơn giản hơn; paper cũng dẫn Chen et al. (2026) cho rằng Lightning Attention khái quát hóa độ dài tốt hơn, có thể giúp hiệu quả dữ liệu khi continual-train ngữ cảnh dài.

(Paper không đưa lại công thức toán học chi tiết của InfLLM-V2 hay Lightning Attention trong văn bản chính; cả hai được dẫn chiếu tới bài báo gốc. Phần phân tích chi phí ở mục 9 vì vậy chỉ dùng tính chất độ phức tạp đã biết: $O(N)$ cho linear attention dạng trạng thái cố định (bài 18), "thưa nhưng vẫn giữ cache đầy" cho sparse attention khối (tương tự bài 17).)

3.3 Cải tiến kiến trúc khác: QK-Norm, HyPE, output gate

- **QK-Normalization** (Henry et al. 2020): áp cho **mọi** layer attention (cả sparse và linear), tránh activation "nhọn" (spike) khi train ngữ cảnh dài, đồng thời tăng độ biểu đạt của các module linear attention.
- **HyPE (Hybrid Positional Encoding):** áp RoPE (bài 12) **chỉ** cho layer linear attention (giúp giữ thứ tự tương đối trong "bộ nhớ" toàn cục); **bỏ hẳn RoPE** ở layer sparse attention, vì RoPE làm suy giảm (decay) thông tin theo khoảng cách, cản việc truy xuất chính xác tầm xa mà sparse attention vốn mạnh ở điểm đó.
- **Output gate:** thêm sau **mỗi** khối attention (cả sparse và linear), cùng tinh thần cơ chế gated attention (bài 67): giảm hiện tượng attention sink, giúp phân phối attention linh hoạt hơn. Paper ghi nhận thực nghiệm: thêm output gate cải thiện rõ cả độ ổn định và hiệu năng.

3.4 Pseudo-code (diễn giải cấu trúc một block, không phải code gốc)

```
def sala_block(x, layer_type, pos):
    if layer_type == "sparse":          # InfLLM-V2, 25% layer
        q, k, v, z = proj_q(x), proj_k(x), proj_v(x), proj_z(x)  # z: cổng output, áp cho
↪  MỌI layer
        q, k = qk_norm(q), qk_norm(k)                  # KHÔNG áp RoPE (NoPE)
        o = infllm_v2_attention(q, k, v)               # dense<->sparse switchable
    else:                               # Lightning Attention, 75% layer
```

```
        q, k, v, z = proj_q(x), proj_k(x), proj_v(x), proj_z(x)
        q, k = rope(qk_norm(q), pos), rope(qk_norm(k), pos)  # RoPE áp cho linear
        o, state = lightning_attention(q, k, v, state)       # O(N), trạng thái cố định
    o = output_gate(o, z)                # giảm attention sink
    x = x + o
    x = x + ffn(x)                       # FFN sau mỗi khối, như Transformer chuẩn
    return x
```

4. Phương pháp chi tiết: huấn luyện chuyển đổi (Transformer-to-hybrid)

4.1 Hai cách train mô hình hybrid, và lựa chọn của SALA

Paper chia hai nhóm công thức có sẵn: (1) **train từ đầu** (Qwen3-Next, Kimi Linear...) - đơn giản, linh hoạt kiến trúc nhất nhưng đắt; (2) **chuyển đổi từ checkpoint Transformer** bằng cross-architecture distillation - tái dùng trọng số pretrained, giảm hẳn chi phí. MiniCPM-SALA chọn (2): bắt đầu từ checkpoint giữa của **MiniCPM-4.0** (checkpoint đã đi được 7T/8T token của lộ trình train MiniCPM-4.0), dùng khung **HALO** (Hybrid Attention via Layer Optimization, Chen et al. 2026) mở rộng.

4.2 Năm giai đoạn (Bảng 1 của paper)

Giai đoạn	Tham số train	Sparse attention	Độ dài chuỗi	# Token
1. Architecture Conversion (HALO)	chỉ layer linear attention	tắt	0.5K	1.3B
2. Continual Stable-Training	toàn bộ	tắt	4K	314.6B
3. Short-Decay Training	toàn bộ	tắt	4K	1006.6B
4. Long-Decay Training	toàn bộ	**mở**	32K / 160K / 520K	102.2B / 62.9B / 50.6B
5. Supervised Fine-Tuning	toàn bộ	mở	64K / 140K	204.5B / 213.3B

Giai đoạn 1 (HALO, chuyển đổi kiến trúc): khác HALO gốc ở hai điểm: (a) giữ layer đầu và layer cuối ở dạng softmax attention (không chuyển) để ổn định huấn luyện; các layer còn lại, thuật toán chọn layer của HALO quyết định layer nào **giữ lại** làm softmax attention (sẽ train thành sparse ở giai đoạn sau) và layer nào chuyển sang linear; (b) **không** làm bước fine-tune cuối của HALO gốc, mà để các giai đoạn pre-training/post-training sau đảm nhiệm. Chỉ **1.3 tỷ token**, độ dài chuỗi **512**, chỉ layer linear attention được train (mọi tham số khác đóng băng). **Giai đoạn 2-3 (ổn định + decay ngắn):** phối hợp layer linear mới với layer softmax, FFN, embedding; sparse attention vẫn **tắt** (dùng dạng dense vì InfLLM-V2 switchable), chuỗi ngắn 4K để rẻ. Giai đoạn 3 tăng trọng số dữ liệu chất lượng cao L2 và thêm nhiều PDF/L3 synthetic, khối lượng token lớn nhất toàn pipeline (1006.6 tỷ). **Giai đoạn 4 (long-decay):** mở rộng dần 4K$\rightarrow$32K$\rightarrow$160K$\rightarrow$520K, **bật sparse attention** (vì ở độ dài này lợi ích tính toán của sparse mới đáng kể), tiếp tục train **toàn bộ tham số** để học sự phối hợp sparse+linear. **Giai đoạn 5 (SFT):** dữ liệu reasoning-nặng (code, toán, tri thức, function call, hội thoại) cộng dữ liệu ngữ cảnh dài tổng hợp riêng; 64K rồi 140K; sparse attention bật suốt.

Tổng ngân sách (tự tính, kiểm lại số của paper): $1.3+314.6+1006.6+102.2+62.9+50.6+204.5+213.3 \approx 1956$ tỷ token $\approx$ 2 nghìn tỷ, đúng như paper nêu "khoảng 2T token", tức khoảng **25% của 8T token** cần để train MiniCPM-4.0 từ đầu, tức giảm khoảng 75% chi phí training (paper, Hình 1 minh họa). Paper **không nêu** số layer, số head hay d_{model} cụ thể của mô hình 9B này.

5. Thí nghiệm và kết quả chính

5.1 Đánh giá tiêu chuẩn (Bảng 2, một phần)

So với các mô hình mã nguồn mở cỡ 7-9B (Qwen3-8B, Nemotron-Nano-v2-9B, MiniCPM-4.1-8B, Ministral-3-Reasoning-8B, Falcon-H1R-7B):

Model	CMMLU	MMLU-Pro	HumanEval	MBPP	AIME24	AIME25	BBH	Trung bình
Qwen3-8B	81.68	73.26	93.90	81.32	73.33	66.67	74.17	73.45
Nemotron-Nano-v2-9B	61.59	71.79	93.90	93.39	71.67	56.67	74.28	73.82
MiniCPM-4.1-8B	84.72	72.70	91.46	91.05	80.83	72.08	82.68	76.13
Falcon-H1R-7B	63.55	70.98	96.34	91.05	86.67	81.04	63.17	76.45
MiniCPM-SALA (9B)	81.55	67.04	95.12	89.11	83.75	78.33	81.55	**76.53**

MiniCPM-SALA đạt trung bình cao nhất trong nhóm (76.53), dù MMLU-Pro (67.04) thấp hơn các mô hình full-attention (Qwen3-8B 73.26). Paper kết luận: chuyển sang hybrid **không làm giảm đáng kể** năng lực chung/ngắn hạn.

*(Lưu ý: cột "Trung bình" ở trên là của paper, tính trên **toàn bộ** các cột của Bảng 2 gốc, bao gồm nhiều benchmark không trích ở đây; không phải trung bình cộng của 7 cột hiển thị trong bảng trên, nên người đọc không nên cố tính lại từ các số đã trích.)*

5.2 Ngữ cảnh dài (Bảng 3, RULER/MRCR/NoLiMa)

	RULER 64K	RULER 128K	NoLiMa 32K	NoLiMa 64K	NoLiMa 128K	Trung bình
Qwen3-8B	80.53	71.74	43.40	23.35	11.25	32.02
Nemotron-Nano-v2-9B	88.77	68.01	19.69	11.82	5.80	25.12
Falcon-H1R-7B	56.50	36.33	14.89	9.87	4.73	16.04
MiniCPM-SALA (9B)	**92.65**	**89.37**	**54.54**	**42.95**	**23.86**	**38.97**

MiniCPM-SALA vượt rõ mọi baseline ở cả RULER và đặc biệt ở NoLiMa (benchmark đòi hỏi suy luận ngữ nghĩa, không chỉ khớp chuỗi literal): 23.86 ở 128K so với 11.25 của Qwen3-8B (gần gấp đôi). Riêng MRCR (benchmark truy xuất nhiều bản sao tương tự), MiniCPM-SALA **không** vượt trội, có lúc thua Ministral-3-R (ví dụ 128K-2N: 28.62 so với 50.30; số liệu đầy đủ trong Bảng 3 của paper).

*(Lưu ý: cột "Trung bình" ở trên là của paper, tính trên toàn bộ Bảng 3 gốc, **bao gồm cả các cột MRCR** không trích ở đây (nơi MiniCPM-SALA yếu hơn) - đây là lý do giá trị trung bình thấp hơn nhiều so với trung bình cộng của 5 cột RULER/NoLiMa hiển thị trong bảng trên.)*

5.3 Ngữ cảnh siêu dài: ngoại suy tới 2048K không cần YaRN (Bảng 4)

RULER	128K	512K	1000K	2048K
Qwen3-Next-80B-A3B-Instruct*	96.0	86.9	80.3	-

RULER	128K	512K	1000K	2048K
MiniCPM-SALA (9B)	89.4	87.1	86.3	**81.6**

(*theo tài liệu chính thức Qwen3-Next, không phải paper này tự đo lại.) Dù chỉ train tới 520K, MiniCPM-SALA **ngoại suy** tới 2048K mà không cần kỹ thuật phụ trợ như YaRN (bài 12), vẫn giữ 81.6 điểm; paper gán khả năng này cho cấu hình **NoPE ở layer sparse** (mục 3.3): KV-cache của layer sparse không gắn thông tin vị trí nên không bị giới hạn theo độ dài đã train. Ở 1000K, MiniCPM-SALA (9B) còn **vượt** Qwen3-Next-80B-A3B-Instruct (86.3 so với 80.3) dù tổng tham số ít hơn gần 9 lần (Qwen3-Next là MoE, tham số **active** mỗi token chỉ khoảng 3B, nên khoảng cách tham số active thực tế nhỏ hơn con số 9 lần này).

5.4 Tốc độ suy luận (Hình 2, 3; so với Qwen3-8B)

Đo trên NVIDIA A6000D (96GB) và RTX 5090 (32GB), input theo độ dài cho trước, sinh 1K token. Một số điểm số đọc được trực tiếp từ hình (không phải suy diễn):

Phần cứng, độ dài	Qwen3-8B TTFT	SALA TTFT	Qwen3-8B end-to-end	SALA end-to-end
A6000D, 64K (không lượng tử)	16.1 s	12.3 s	36.8 s	30.4 s
A6000D, 256K (không lượng tử)	180.8 s	51.6 s	223.8 s	69.8 s
A6000D, 1024K (không lượng tử)	**OOM**	250.3 s	**OOM**	269.1 s
5090, 64K (không lượng tử)	10.4 s	10.8 s	27.0 s	25.0 s
5090, 128K (không lượng tử)	**OOM**	22.3 s	**OOM**	36.5 s

Ở 256K trên A6000D, TTFT giảm từ 180.8s xuống 51.6s ≈ 3.5× (đúng số nêu trong abstract). Qwen3-8B hết bộ nhớ (OOM) ở 512K-1024K trên A6000D và ngay từ 128K (không lượng tử) trên RTX 5090; MiniCPM-SALA chạy được tới 1024K trên cả hai loại GPU. **Đáng chú ý ở 64K trên RTX 5090: hai mô hình ngang nhau, không rõ ai hơn** - TTFT của SALA hơi chậm hơn (10.8s so với 10.4s của Qwen3-8B) nhưng end-to-end của SALA lại hơi nhanh hơn (25.0s so với 27.0s); lợi thế tốc độ chỉ rõ rệt và nhất quán (cả hai chỉ số) từ 128K trở lên, xem phân tích ở mục 9.

6. Phân tích / ablation

Paper **không có một mục ablation riêng** tách rời kiến trúc (vd. không so "chỉ sparse", "chỉ linear", "tỉ lệ khác 1:3" trên cùng checkpoint). Các điểm phân tích nằm rải trong phần kết quả:

- **Độ lệch giữa RULER/NoLiMa và MRCR:** mô hình mạnh rõ ở truy xuất đơn giản (RULER) và truy xuất ngữ nghĩa (NoLiMa), nhưng không nổi bật ở MRCR (nhiều bản sao gây nhiễu); paper không giải thích rõ nguyên nhân, chỉ trình bày số liệu (mục 5.2).
- **Ngoại suy không cần YaRN** được paper quy cho thiết kế NoPE ở sparse layer (mục 5.3), nhưng đây là một **giải thích định tính**, không có ablation số liệu riêng (ví dụ không so với một biến thể SALA có RoPE ở sparse layer).
- **Chi phí train giảm 75%** là một phép tính tỉ lệ token (2T/8T), không phải đo trực tiếp GPU-giờ hay FLOPs; đây là ước lượng gián tiếp.

7. Giới hạn

Do tác giả nêu (gián tiếp, qua cách trình bày, không có mục "Limitations" riêng):

- MMLU-Pro của MiniCPM-SALA (67.04) thấp hơn rõ các baseline full-attention (Qwen3-8B 73.26, Falcon-H1R-7B 70.98); paper không bàn riêng về điểm yếu này.
- MRCR (truy xuất nhiều bản sao) không phải điểm mạnh của mô hình so với một số baseline (mục 5.2).
- Loại trừ MiniCPM-4.1-8B khỏi đánh giá ngữ cảnh dài vì giới hạn 64K của chính nó, không phải giới hạn của SALA, nhưng khiến bảng so sánh ngữ cảnh dài thiếu một đối chứng "cùng họ, full-attention".

Nhận xét:

- Paper không công bố số layer, d_{model}, số head cụ thể của kiến trúc 9B này, nên không tính lại được KV-cache theo từng layer (khác với bài 16, 17 là có đủ số để tự tính).
- Không có ablation tỉ lệ 1:3 so với tỉ lệ khác (ví dụ 1:4, 1:2) trên cùng checkpoint; lựa chọn 1:3 dựa trên "cảm hứng từ Qwen3-Next, Kimi Linear và thực nghiệm nhỏ nội bộ", không phải một bảng so sánh công khai trong chính paper này.
- Không có ablation tách riêng đóng góp của QK-Norm, HyPE, output gate; ba cải tiến được giới thiệu cùng lúc, chỉ có nhận xét "thực nghiệm cho thấy cải thiện ổn định và hiệu năng" chung.
- Số liệu tốc độ (Hình 2, 3) chỉ so với Qwen3-8B, không so với các hybrid khác (Jamba, Kimi Linear, GLM-5) ở cùng điều kiện phần cứng.

8. Liên hệ với các bài khác

- **18 (Linear Transformers = RNN), 20 (GLA):** nền tảng của Lightning Attention (linear attention với trạng thái cố định, $O(N)$).
- **17 (Longformer), 64 (NSA), 65 (MoBA):** họ sparse attention theo mẫu/block; InfLLM-V2 cùng tinh thần nhưng nhấn mạnh tính "switchable dense/sparse, không tham số mới".
- **24 (Gated DeltaNet), 26 (Kimi Linear):** các hybrid khác trộn linear attention với (một phần) full attention; Kimi Linear là nguồn cảm hứng trực tiếp cho tỉ lệ 1:3.
- **68 (DeepSeek-V3.2), 69 (GLM-5):** cách "kết hợp" khác hẳn: DSA áp **một cơ chế duy nhất** (chọn token động) cho **mọi** layer, trong khi SALA **chia layer** thành hai loại cơ chế cố định theo tỉ lệ. Hai triết lý khác nhau để giải cùng bài toán ngữ cảnh dài.
- **12 (RoPE):** nền để hiểu HyPE (RoPE chỉ ở linear layer, NoPE ở sparse layer) và vì sao NoPE giúp ngoại suy độ dài.
- **67 (Gated Attention/attention-sink-free):** cơ sở ý tưởng output gate.
- **46 (PagedAttention):** trực giao; dù 75% layer đã là $O(N)$, 25% layer sparse vẫn cần quản lý KV-cache lúc serving.

9. Tóm tắt công thức / khái niệm

Khái niệm	Công thức / ý nghĩa
Tỉ lệ hybrid	25% layer InfLLM-V2 (sparse) : 75% layer Lightning Attention (linear)
HyPE	RoPE chỉ ở layer linear; NoPE (bỏ RoPE) ở layer sparse để không suy giảm truy xuất tầm xa
QK-Norm	áp mọi layer attention, chống activation spike khi train dài
Output gate	sau mỗi khối attention (cả hai loại), giảm attention sink
5 giai đoạn train	HALO (1.3B, 512) → Stable (314.6B, 4K) → Short-Decay (1006.6B, 4K) → Long-Decay (102.2+62.9+50.6B, 32K/160K/520K, bật sparse) → SFT (204.5+213.3B, 64K/140K)
Tổng ngân sách	≈2T token ≈ 25% của 8T (MiniCPM-4.0 from scratch) → giảm ~75% chi phí
Ngữ cảnh siêu dài	train tới 520K, ngoại suy không YaRN tới 2048K (RULER 81.6), nhờ NoPE ở sparse layer

Khái niệm	Công thức / ý nghĩa
Tốc độ (A6000D, 256K)	TTFT Qwen3-8B 180.8s → SALA 51.6s (≈3.5×)
Ngưỡng hòa vốn (paper, mục 5.4)	ở 64K, SALA không nhanh hơn rõ; lợi ích rõ từ 128K, áp đảo (OOM baseline) từ 512K+

10. Kết luận ngắn

MiniCPM-SALA chọn hướng "chia layer" cho bài toán ngữ cảnh dài: 1/4 layer giữ sparse attention (InfLLM-V2, không nén, không tham số mới, dense↔sparse linh hoạt) để truy xuất chính xác, 3/4 layer chuyển sang linear attention (Lightning Attention, $O(N)$) để xử lý phần lớn ngữ cảnh rẻ, cộng ba chi tiết kiến trúc nhỏ nhưng quan trọng (QK-Norm, HyPE/NoPE ở sparse layer, output gate). Điểm mạnh thực nghiệm lớn nhất không phải là train from scratch mà là **continual training từ checkpoint Transformer có sẵn**, giảm ~75% chi phí, và khả năng ngoại suy tới 2048K dù chỉ train tới 520K.

Link gốc: https://arxiv.org/abs/2602.11761

Bài 71

DeepSeek-V4: Towards Highly Efficient Million-Token Context Intelligence (2026)

Phạm vi ghi chú: DeepSeek-V4 là báo cáo kỹ thuật rất dài (kiến trúc, hạ tầng, pre-training, post-training, hàng chục benchmark). Ghi chú này **tập trung vào thiết kế attention**: Compressed Sparse Attention (CSA), Heavily Compressed Attention (HCA), cách hai loại lớp xen kẽ, nhánh sliding window, lightning indexer chạy trên các mục KV đã nén, KV cache và FLOPs so với DeepSeek-V3.2 ở 1M token, và quan hệ với MLA (bài 16), NSA (bài 64), DSA (bài 68). Các thay đổi kiến trúc khác (mHC, Muon, MoE), hạ tầng, huấn luyện và post-training chỉ tóm tắt ở mục 6.

Tác giả: DeepSeek-AI (research@deepseek.com); danh sách tác giả ở Phụ lục A của paper.
Venue / arXiv: arXiv:2606.19348 (v1, 26/04/2026, theo dấu arXiv trong PDF và API arXiv); paper không nêu venue. Paper tự gọi đây là "preview version" của dòng V4.
Nguồn: https://arxiv.org/abs/2606.19348
Chủ đề: attention lai CSA + HCA (nén KV theo chiều chuỗi, rồi chọn thưa bằng lightning indexer) cho ngữ cảnh 1M token; hai mô hình MoE: DeepSeek-V4-Flash (284B tổng, 13B kích hoạt) và DeepSeek-V4-Pro (1.6T tổng, 49B kích hoạt).
Vị trí trong lộ trình: đọc sau 16 (MLA: một KV entry dùng chung cho mọi head, RoPE tách phần), 64 (NSA: nhánh nén, nhánh chọn, sliding window), 68 (DSA: lightning indexer + top-k), 55 (attention sink). Đọc tiếp 74 (DeepSeek-V4.1-Flash: dùng lại KV giữa các lớp, FP4 KV). Bài này nằm ở đây vì là điểm hội tụ: ý "nén theo chuỗi" của NSA cộng "chọn thưa bằng indexer" của DSA, đặt trên KV entry kiểu MLA ở chế độ MQA.

> Tài liệu học tập bằng tiếng Việt, là ghi chú diễn giải, không phải bản dịch. Số liệu lấy từ paper (full text arXiv, tải ngày 07/11/2026); các phép tính đánh dấu "tự tính" là của người ghi chú. Hai chỗ kiểm tra chéo dùng `config.json` công khai trên Hugging Face (nguồn ngoài paper, truy cập 07/11/2026) và được ghi rõ.

1. Bối cảnh

Các ứng dụng suy luận và agent có thể cần chuỗi dài do nhiều lượt gọi công cụ hoặc nhiều tài liệu. Attention gốc (bài 01) tốn $O(n^2)$ phép tính khi prefill; khi sinh, mỗi token phải đọc toàn bộ KV cache. DeepSeek-V3.2 (bài 68) dùng DSA giảm phần attention chính xuống $O(nk)$: lightning indexer chấm điểm token trước đó và giữ top-$k = 2048$. Dù vậy, ở 1M token, cache vẫn tăng theo độ dài vì mỗi token ở mỗi lớp lưu một latent MLA và một key indexer; bản thân indexer cũng quét toàn bộ n token cho mỗi truy vấn.

Thuật ngữ cần nắm:

- **KV entry (mục KV):** vector được lưu trong cache để query so khớp (key) và lấy thông tin (value). Ở V4, một entry vừa là key vừa là value.
- **Nén theo chiều chuỗi (sequence compression):** gộp m token liên tiếp thành 1 entry, cache nhỏ đi m lần. Đây là ý "compression branch" của NSA (bài 64), khác hẳn MLA (nén theo chiều đặc trưng, số entry vẫn bằng số token).
- **Sliding window attention (SWA):** chỉ nhìn n_{win} token gần nhất (bài 06, 17).
- **Attention sink:** cho phép tổng trọng số attention nhỏ hơn 1 (bài 55).
- **MQA:** mọi query head dùng chung một K/V (bài 14).

2. Ý tưởng chính

DeepSeek-V4 thay attention của mọi lớp (ở Flash trừ 2 lớp SWA đầu; ở Pro 2 lớp đầu là HCA) bằng một trong hai cơ chế, xếp xen kẽ theo lớp. **CSA** nén KV của mỗi $m = 4$ token thành một entry bằng một phép

pooling có trọng số học được (cửa sổ $2m$ chồng lấp), rồi chạy DSA trên các entry đã nén: lightning indexer chấm điểm các entry nén, chỉ giữ top-k (512 ở Flash, 1024 ở Pro) cho attention chính. **HCA** nén mạnh hơn nhiều ($m' = 128$ token thành 1 entry), không chọn thưa mà attention dày (dense) lên toàn bộ entry nén. Cả hai đều cộng thêm một nhánh SWA 128 token không nén để giữ chi tiết cục bộ, dùng một KV entry 512 chiều duy nhất làm cả key lẫn value cho mọi query head (MQA), RoPE một phần (64 chiều cuối), RMSNorm cho query và KV, và attention sink. Kết hợp với lưu trữ BF16/FP8 và indexer FP4, ở 1M token V4-Pro chỉ cần 27% FLOPs mỗi token và 10% KV cache so với V3.2; V4-Flash cần 10% và 7%.

3. Phương pháp chi tiết: CSA

Ký hiệu của paper: $H \in \mathbb{R}^{n\times d}$ hidden state đầu vào, n độ dài chuỗi, d hidden size, c head dim (= chiều của KV entry), m tỉ lệ nén.

3.1 Nén KV (Eq. 9-12)

Tính hai chuỗi KV thô và hai chuỗi trọng số nén:

$$C^a = HW^{aKV}, \quad C^b = HW^{bKV}, \quad Z^a = HW^{aZ}, \quad Z^b = HW^{bZ}, \qquad W^{\cdot} \in \mathbb{R}^{d\times c}.$$

Entry nén thứ i lấy m token của khối i từ nhánh a và m token của khối $i-1$ từ nhánh b, với bias vị trí học được $B^a, B^b \in \mathbb{R}^{m\times c}$:

$$\begin{aligned}
&[S^a_{mi:m(i+1)-1};\ S^b_{m(i-1):mi-1}] \\
&\quad = \text{Softmax}_{row}\big([Z^a_{mi:m(i+1)-1} + B^a;\ Z^b_{m(i-1):mi-1} + B^b]\big), \\
&C^{Comp}_i = \sum_{j=mi}^{m(i+1)-1} S^a_j \odot C^a_j \\
&\qquad\quad + \sum_{j=m(i-1)}^{mi-1} S^b_j \odot C^b_j
\end{aligned}$$

Diễn giải từng phần (của người ghi chú, dựa trên định nghĩa của paper):

- Z có cùng kích thước $n \times c$ với C, và phép nhân là Hadamard $\odot$: softmax được lấy **riêng cho từng kênh** trên $2m$ vị trí. Tức mỗi chiều trong 512 chiều có một "pooling mềm" riêng: kênh này có thể lấy chủ yếu từ token đầu khối, kênh khác từ token cuối khối.
- Cửa sổ dài $2m$, bước m: hai entry liền nhau dùng chung m token (khối $i-1$ dùng qua nhánh b cho entry i và qua nhánh a cho entry $i-1$). Vì vậy dù mỗi entry nhìn $2m$ token, số entry vẫn là n/m.
- Khi $i = 0$, phần nhánh b được đệm: Z^b bằng $-\infty$ (trọng số softmax 0) và C^b bằng 0.

Ví dụ (tự tính), $m = 4$**:** entry C^{Comp}_5 tổng hợp token 20-23 (nhánh a) và token 16-19 (nhánh b), 8 vị trí. Một query ở vị trí $t = 22$ chỉ được thấy các khối $s < \lfloor 22/4 \rfloor = 5$, tức entry 0-4 (phủ token 0-19 qua nhánh a); token 20, 21, 22 nằm trong khối của chính nó nên **không thấy được qua nhánh nén**, chỉ thấy qua nhánh SWA (mục 4.3).

3.2 Lightning indexer trên entry nén (Eq. 13-17)

Indexer key cũng được nén bằng **cùng phép nén** ở trên, nhưng với tham số riêng: $K^{IComp} \in \mathbb{R}^{(n/m)\times c_I}$. Query của indexer sinh theo kiểu low-rank, dùng chung latent query với attention chính:

$$c^Q_t = h_t W^{DQ} \in \mathbb{R}^{d_c}, \qquad [q^I_{t,1}; \ldots; q^I_{t,n^I_h}] = c^Q_t W^{IUQ}, \qquad w^I_t = h_t W^w \in \mathbb{R}^{n^I_h}$$

$$I_{t,s} = \sum_{h=1}^{n_h^I} w_{t,h}^I \cdot \mathrm{ReLU}\big(q_{t,h}^I \cdot K_s^{I Comp}\big), \qquad s < \lfloor t/m \rfloor; \qquad C_t^{SprsComp} = \{C_s^{Comp} \mid I_{t,s} \in \text{Top-k}(I_{t,:})\}$$

Đây đúng là công thức indexer của DSA (bài 68), chỉ khác là s chạy trên **khối nén** thay vì từng token, nên số điểm cần chấm giảm m lần. Kiểm tra độ phủ (tự tính): V3.2 chọn 2048 token; V4-Flash chọn 512 entry, mỗi entry tóm tắt khoảng 4 token, nên "phủ" khoảng $512 \times 4 = 2048$ token gốc; V4-Pro 1024 entry, khoảng 4096 token gốc. Số entry vào attention chính giảm 4 lần (Flash) so với V3.2.

3.3 Shared Key-Value MQA và grouped output projection (Eq. 18-19)

Query chính: $[q_{t,1}; \ldots; q_{t,n_h}] = c_t^Q W^{UQ}$. Attention chính chạy kiểu MQA, mỗi entry được chọn vừa là key vừa là value:

$$o_{t,i} = \mathrm{CoreAttn}\big(\text{query} = q_{t,i},\ \text{key} = C_t^{SprsComp},\ \text{value} = C_t^{SprsComp}\big) \in \mathbb{R}^c$$

Liên hệ MLA (diễn giải của người ghi chú): ở chế độ MQA của MLA (bài 16 §3.3, bài 68 §4.1), sau khi "hấp thụ" W^{UK} vào query và W^{UV} vào output, mọi head cùng đọc một latent c^{KV} 512 chiều cộng 64 chiều RoPE, và latent này đóng vai cả key lẫn value. V4 làm điều đó **một cách bản địa**: không còn W^{UK}, W^{UV}, chỉ một entry $c = 512$ chiều (64 chiều cuối mang RoPE) dùng chung cho mọi head.

Vì $c\,n_h$ lớn (Flash: $512 \times 64 = 32{,}768$), chiếu thẳng o_t về d rất đắt. **Grouped output projection:** chia n_h head thành g nhóm, mỗi nhóm ($c\,n_h/g$ chiều) chiếu về d_g chiều, ghép lại ($d_g g$ chiều) rồi chiếu về d. Số tham số (tự tính, Flash, $g = 8$, $d_g = 1024$, $d = 4096$): chiếu thẳng $32{,}768 \times 4096 \approx 134$M; grouped $8 \times 4096 \times 1024 + 8192 \times 4096 \approx 33.6\text{M} + 33.6\text{M} = 67$M, tức một nửa.

3.4 Pseudo-code (một lớp CSA, decode một token; đơn giản hóa)

```
def csa_decode(h_t, t, st, P):                  # st: cache của lớp, P: tham số
    # 1) Cập nhật bộ đệm token chưa đủ khối; khi đủ m token thì sinh 1 entry nén
    st.tail.append(h_t)
    if (t + 1) % P.m == 0:
        st.comp_kv.append(compress(st.tail_prev, st.tail, P.kv))   # Eq. 11-12, softmax theo
↪   kênh trên 2m vị trí
        st.comp_ik.append(compress(st.tail_prev, st.tail, P.ik))   # indexer key, cùng phép
↪   nén, tham số riêng
        st.tail_prev, st.tail = st.tail, []
    st.swa.append(rmsnorm(h_t @ P.W_swa))                          # entry không nén cho
↪   nhánh SWA
    st.swa = st.swa[-P.n_win:]
    # 2) Indexer (FP4) trên các khối s < floor(t/m) đã sẵn sàng
    cq = h_t @ P.W_DQ
    qI = (cq @ P.W_IUQ).view(P.nI, P.cI); wI = h_t @ P.W_w
    I = (wI[:, None] * relu(qI @ stack(st.comp_ik).T)).sum(0)      # Eq. 16
    sel = topk(I, min(P.k, len(I)))
    # 3) MQA: một KV entry 512 chiều làm cả K và V; nối entry nén được chọn với cửa sổ SWA
    kv = rmsnorm(cat([stack(st.comp_kv)[sel], stack(st.swa)]))
    q = rmsnorm((cq @ P.W_UQ).view(P.nh, P.c)); q, kv = rope_last64(q, kv, t)
    o = softmax_with_sink(q @ kv.T / sqrt(P.c), P.sink) @ kv       # Eq. 27
    o = rope_last64(o, -t)                                          # khử vị trí tuyệt đối
↪   ở output
    return grouped_out_proj(o, P)
```

4. Phương pháp chi tiết: HCA, nhánh sliding window và các chi tiết khác

4.1 HCA (Eq. 20-26)

HCA dùng cùng khung nhưng: chỉ một chuỗi $C = HW^{KV}$, $Z = HW^{Z}$; tỉ lệ $m' \gg m$; **không chồng lấp**; bias $B \in \mathbb{R}^{m' \times c}$:

$$S_{m'i:m'(i+1)-1} = \text{Softmax}_{row}(Z_{m'i:m'(i+1)-1} + B), \qquad C_i^{Comp} = \sum_{j=m'i}^{m'(i+1)-1} S_j \odot C_j$$

Sau đó attention **dày** (không có indexer, không top-k) của mọi query head lên toàn bộ C^{Comp} (cùng shared-KV MQA và grouped output projection như CSA). Nhờ nén 128 lần, ở 1M token mỗi query chỉ đối mặt khoảng $1{,}048{,}576/128 = 8192$ entry (tự tính).

4.2 Cấu hình và cách xen kẽ (§4.2.1)

	V4-Flash	V4-Pro
Số lớp, hidden d	43, 4096	61, 7168
2 lớp đầu	SWA thuần	HCA
Các lớp sau	CSA và HCA xen kẽ	CSA và HCA xen kẽ
CSA: m, n_h^I, c_I, top-k	4, 64, 128, 512	4, 64, 128, 1024
HCA: m'	128	128
Query heads n_h, head dim c, d_c	64, 512, 1024	128, 512, 1536
Nhóm output g, d_g	8, 1024	16, 1024
Cửa sổ SWA n_{win}	128	128
MoE	1 shared + 256 routed (dim 2048), 6 kích hoạt	1 shared + 384 routed (dim 3072), 6 kích hoạt

Văn bản chỉ nói "interleaved", không nêu tỉ lệ. Hình 6 vẽ minh họa Layer-2 CSA, Layer-3 HCA, Layer-4 CSA, Layer-5 HCA. `config.json` trên Hugging Face (nguồn ngoài paper) cho `compress_ratios` xen kẽ 4, 128, 4, 128...: Flash có 21 lớp CSA và 20 lớp HCA, Pro có 30 lớp CSA và 31 lớp HCA (tính cả 2 lớp HCA đầu).

4.3 Nhánh sliding window và vì sao cần nó

Để giữ tính nhân quả, query chỉ thấy các khối nén **đã hoàn tất trước khối của nó**, nên không thấy các token cùng khối; thêm nữa, token gần thường quan trọng nhất. Vì vậy mỗi lớp CSA/HCA có thêm $n_{win} = 128$ entry **không nén** của 128 token gần nhất, nối chung với các entry nén trong cùng một phép attention (một softmax chung, không có cổng trộn như NSA).

Nhận xét (tự tính): với HCA, khối của chính query dài tối đa 127 token, và $n_{win} = 128$ phủ đúng token $t-127..t$, nên **không có token nào "mù" hoàn toàn**: token trong khối hiện tại thấy qua SWA, token cũ hơn thấy qua bản tóm tắt 128 token. Với CSA ($m = 4$), khối hiện tại tối đa 3 token, SWA dư sức phủ; token xa chỉ được thấy nếu khối chứa nó lọt top-k.

4.4 RoPE một phần, RoPE ở output, chuẩn hóa, sink

- **QK/KV RMSNorm:** RMSNorm trên từng head query và trên head KV duy nhất ngay trước attention, tránh logit bùng nổ; nhờ đó paper không cần QK-Clip khi dùng Muon (§2.4).

- **Partial RoPE:** RoPE chỉ trên 64 chiều cuối của query và KV entry (giống tinh thần decoupled RoPE ở bài 16, nhưng nằm ngay trong entry).
- **RoPE ở output:** vì entry vừa là key vừa là value, output mang **vị trí tuyệt đối**. Diễn giải (người ghi chú): trên phần RoPE, $v_s = R_{p_s} u_s$, nên $o_t = \sum_s a_{t,s} R_{p_s} u_s$. Paper áp RoPE vị trí âm (paper viết "position $-i$"; ở đây hiểu là vị trí của token truy vấn t): $R_{-t} o_t = \sum_s a_{t,s} R_{p_s - t} u_s$, chỉ còn phụ thuộc khoảng cách $p_s - t$ (dùng tính chất $R_a R_b = R_{a+b}$ của bài 12).
- **Attention sink (Eq. 27):** mỗi head h có logit sink học được z'_h:

$$\boxed{s_{h,i,j} = \frac{\exp(z_{h,i,j})}{\sum_k \exp(z_{h,i,k}) + \exp(z'_h)}}$$

nên tổng trọng số có thể nhỏ hơn 1, thậm chí gần 0 (head "không muốn nhìn gì"), cùng ý với bài 55 nhưng sink là một logit học, không phải token thật.

5. Phân tích chi phí: KV cache và FLOPs ở 1M token

5.1 Các yếu tố paper nêu (§2.3.4)

1. **Lưu trữ trộn:** 64 chiều RoPE ở BF16, phần còn lại FP8, giảm KV cache gần một nửa so với BF16 thuần.
2. **Indexer FP4:** tính toán trong lightning indexer ở FP4.
3. **Top-k nhỏ hơn V3.2:** giúp hiệu quả ở văn bản ngắn và trung bình.
4. **Quan trọng nhất:** nén chuỗi và kiến trúc lai.

Số paper công bố (Hình 1 và §1): ở 1M token, V4-Pro cần **27%** FLOPs mỗi token (tính theo FLOPs tương đương FP8) và **10%** KV cache so với V3.2; V4-Flash **10%** và **7%**. Nhãn trên Hình 1: FLOPs thấp hơn 3.7 lần (Pro) và 9.8 lần (Flash); KV cache nhỏ hơn 9.5 lần (Pro) và 13.7 lần (Flash). So với baseline BF16 GQA8, head dim 128, KV cache của V4 còn khoảng **2%** ở 1M token. Paper không cho bảng byte/token chi tiết; số trên Hình 1 là "estimated".

5.2 Tự tính byte/token và kiểm tra chéo

Giả định (của người ghi chú): main entry 512 chiều = 64 chiều BF16 (128 B) + 448 chiều FP8 (448 B) = **576 B**, bỏ qua scale của FP8; indexer key 128 chiều FP4 kiểu MXFP4 (64 B + 4 B scale) = **68 B**.

Loại lớp	Byte cache trên mỗi token gốc
CSA	$(576 + 68)/4 = 161$
HCA	$576/128 = 4.5$
SWA (giới hạn, không tăng theo n)	576×128 B mỗi lớp cho cả chuỗi

- **V4-Flash** (21 CSA + 20 HCA): $21 \times 161 + 20 \times 4.5 = 3{,}471$ B/token. Ở 2^{20} token: khoảng 3.64 GB.
- **V4-Pro** (30 CSA + 31 HCA): $30 \times 161 + 31 \times 4.5 \approx 4{,}970$ B/token, khoảng 5.21 GB ở 2^{20} token.
- **Kiểm tra 1, baseline GQA8:** $2 \times 8 \times 128 \times 2$ B $= 4096$ B mỗi lớp mỗi token. Flash: $3471/(43 \times 4096) = 1.97\%$; Pro: $4970/(61 \times 4096) = 1.99\%$. Khớp con số "khoảng 2%" của paper.
- **Kiểm tra 2, bài 74:** paper V4.1-Flash nói 890 B/token là khoảng 1/4 của V4-Flash, tức khoảng 3.56 KB, gần 3,471 B ở trên.
- **Suy ra V3.2 (tự tính):** $3.64/0.07 \approx 52$ GB hay $3.64 \times 13.7 \approx 50$ GB; $5.21 \times 9.5 \approx 49.5$ GB. Vậy V3.2 cỡ 50 GB KV cho một chuỗi 1M token, so với 3.6-5.2 GB của V4.
- Lưu ý: tỉ lệ FLOPs của Pro "chỉ" 27% một phần vì Pro kích hoạt 49B tham số, nhiều hơn 37B của V3.2 (nhận xét của người ghi chú).

5.3 FLOPs attention mỗi token decode ở 1M (tự tính, V4-Flash)

Đếm 2 FLOPs mỗi phép nhân-cộng, chưa quy đổi theo độ chính xác:

Thành phần mỗi lớp	Công thức	FLOPs
CSA indexer	$(n/4) \times n_h^I c_I \times 2 = 262{,}144 \times 8192 \times 2$	4.29 G
CSA attention chính	$(512 + 128) \times n_h c \times 4$	0.084 G
HCA attention chính	$(8192 + 128) \times n_h c \times 4$	1.09 G

Tổng: $21 \times (4.29 + 0.084) + 20 \times 1.09 \approx 114$ GFLOPs, so với khoảng $2 \times 13\text{B} = 26$ GFLOPs của phần tham số. Bài học: ở 1M token, **indexer của CSA là khoản lớn nhất** (khoảng 90 GFLOPs), kế đến là HCA dày. Điều này giải thích vì sao paper đẩy indexer xuống FP4, lượng tử hóa điểm index từ FP32 xuống BF16 (§5.2.1: top-k selector nhanh gấp 2, recall 99.7%), và vì sao bài 74 thêm Hierarchical Sparse Indexer và bỏ HCA.

5.4 Quản lý KV cache (§3.5)

- **Hai loại cache:** "state cache" kích thước cố định cho mỗi request (KV của SWA và các token đuôi chưa đủ khối để nén, coi như trạng thái của một state-space model), và KV cache cổ điển chia block cho entry CSA/HCA. Mỗi block phủ $\text{lcm}(m, m') = \text{lcm}(4, 128) = 128$ token gốc, tức $k_1 = 32$ entry CSA và $k_2 = 1$ entry HCA (tự tính). Kiến trúc lai này phá giả định của PagedAttention (bài 46).
- **KV cache trên đĩa** cho prefix dùng chung: lưu mọi entry nén; token đuôi chưa đủ khối phải tính lại. KV của SWA lớn khoảng 8 lần entry nén (kiểm tra tự tính: $576 \times 43 = 24{,}768$ B/token so với $21 \times 144 + 20 \times 4.5 \approx 3{,}114$ B/token, tỉ lệ khoảng 7.95). Ba chiến lược: Full SWA Caching, Periodic Checkpointing mỗi p token, Zero SWA Caching (tính lại $n_{win} \cdot L$ token cuối để khôi phục chính xác). Bài 74 sẽ thay bằng "SWA Bounded Replay".

6. Các thay đổi khác, huấn luyện và post-training (tóm tắt)

- **MoE:** vẫn DeepSeekMoE; affinity đổi từ Sigmoid sang $\sqrt{\text{Softplus}(\cdot)}$; 3 lớp MoE đầu dùng Hash routing theo token ID. MTP giữ như V3 (depth 1).
- **mHC (Eq. 1-8):** residual stream rộng $n_{hc} = 4$ lần, $X_{l+1} = B_l X_l + C_l \mathcal{F}_l(A_l X_l)$; B_l chiếu lên tập ma trận ngẫu nhiên kép bằng Sinkhorn-Knopp ($t_{max} = 20$), nên $\|B_l\|_2 \le 1$, ổn định khi xếp sâu.
- **Muon:** cho đa số tham số (AdamW cho embedding, head, RMSNorm, tham số tĩnh của mHC); hybrid Newton-Schulz 8 vòng hệ số $(3.4445, -4.7750, 2.0315)$ rồi 2 vòng $(2, -1.5, 0.5)$.
- **Pre-training:** Flash 32T, Pro 33T token; lr 2.7×10^{-4} (Pro 2.0×10^{-4}) rồi cosine giảm 10 lần; độ dài 4K, 16K, 64K, 1M. **Sparse attention:** Flash train dense 1T token đầu, bật sparse ở 64K sau một giai đoạn warm-up ngắn cho indexer; Pro có giai đoạn dense dài hơn.
- **Ổn định:** Anticipatory Routing (định tuyến bằng tham số cũ $\theta_{t-\Delta t}$, chỉ bật khi có loss spike) và SwiGLU clamping (linear trong $[-10, 10]$, gate tối đa 10).
- **Post-training:** specialist từng miền (SFT rồi GRPO, reward model sinh cho tác vụ khó kiểm chứng), ba chế độ Non-think, Think High, Think Max; rồi **on-policy distillation** từ hơn 10 teacher, $\mathcal{L}_{OPD} = \sum_i w_i D_{KL}(\pi_\theta \| \pi_{E_i})$ với full-vocabulary logits. FP4 QAT cho expert và đường QK của indexer. Liên hệ distillation 28, 29.

7. Thí nghiệm và kết quả chính

Base model (Bảng 1, chọn dòng):

Benchmark	V3.2-Base (37B/671B)	V4-Flash-Base (13B/284B)	V4-Pro-Base (49B/1.6T)
MMLU (5-shot)	87.8	88.7	90.1
MMLU-Pro (5-shot)	65.5	68.3	73.5
Simple-QA verified (25-shot)	28.3	30.1	55.2
FACTS Parametric (25-shot)	27.1	33.9	62.6
HumanEval (0-shot)	62.8	69.5	76.8
BigCodeBench (3-shot)	63.9	56.8	59.2
MATH (4-shot)	60.5	57.4	64.5
LongBench-V2 (1-shot)	40.2	44.7	51.5

Ngữ cảnh 1M token và vài dòng khác (Bảng 6):

Benchmark	Opus-4.6 Max	GPT-5.4 xHigh	Gemini-3.1-Pro High	DS-V4-Pro Max
MRCR 1M (MMR)	92.9	-	76.3	83.5
CorpusQA 1M (ACC)	71.7	-	53.8	62.0
SimpleQA-Verified	46.2	45.3	75.6	57.9
Codeforces (Rating)	-	3168	3052	3206
SWE Verified	80.8	-	80.6	80.6

(GPT-5.4 không được đánh giá 1M vì API không trả lời phần lớn truy vấn, theo paper.)

Theo chế độ suy luận (Bảng 7, dòng long-context):

	Flash Non-Think	Flash High	Flash Max	Pro Non-Think	Pro High	Pro Max
MRCR 1M	37.5	76.9	78.7	44.7	83.3	83.5
CorpusQA 1M	15.5	59.3	60.5	35.6	56.5	62.0

Hình 9 (MRCR 8-needle theo độ dài 8K đến 1024K): paper mô tả truy xuất "highly stable" trong 128K và giảm thấy rõ sau 128K. Các giá trị trên hình không gán chắc được cho từng đường từ văn bản trích xuất, nên không chép lại.

8. Phân tích / ablation

- **Không có ablation riêng cho CSA/HCA:** paper không so sánh tỉ lệ CSA:HCA, m, m', top-k, n_{win}, hay "có/không nhánh SWA, có/không sink". Bằng chứng duy nhất cho thiết kế là kết quả toàn mô hình (Bảng 1, 6, 7) và chi phí ước tính (Hình 1).
- **Lượng tử hóa điểm index** FP32 sang BF16: top-k selector nhanh gấp 2, giữ recall 99.7% của KV entry (§5.2.1).
- **Lưu trữ trộn BF16/FP8:** giảm KV gần một nửa (§2.3.4), không có số chất lượng đi kèm.
- **Chế độ suy luận quan trọng cho long-context:** MRCR 1M của Flash tăng từ 37.5 (Non-think) lên 78.7 (Max). Tức khả năng dùng ngữ cảnh dài phụ thuộc mạnh vào lượng suy nghĩ, không chỉ vào kiến trúc attention (nhận xét của người ghi chú).
- **Flash so với Pro:** CorpusQA 1M ở mức High, Flash (59.3) cao hơn Pro (56.5); ở Max, Pro hơn (62.0 so với 60.5).

- **Base model:** V4-Flash-Base vượt V3.2-Base trên đa số benchmark với ít tham số hơn, nhưng thua ở BigCodeBench, MATH, BBH, TriviaQA, HellaSwag, CLUEWSC, GSM8K (Bảng 1). Không tách được phần đóng góp của attention so với dữ liệu và mHC.

9. Giới hạn

Tác giả nêu: kiến trúc khá phức tạp vì giữ nhiều thành phần và thủ thuật đã kiểm chứng sơ bộ để giảm rủi ro, sẽ tinh giản sau; cơ chế của Anticipatory Routing và SwiGLU Clamping chưa được hiểu đầy đủ; Flash yếu hơn về kiến thức do ít tham số; MRCR giảm sau 128K; đây là bản preview.

Người ghi chú nhận xét:

- Con số "27% FLOPs, 10% KV" trộn ba nguồn: kiến trúc (nén, lai), độ chính xác (FP8/FP4) và top-k nhỏ hơn. Không có thí nghiệm cô lập phần của kiến trúc.
- FLOPs và KV là ước tính, không phải latency đo thực; paper không cho bảng tốc độ prefill/decode theo độ dài như bài 68.
- HCA ở 1M token vẫn là attention dày lên khoảng 8192 entry và indexer CSA vẫn tuyến tính theo $n/4$ (mục 5.3), nên chi phí decode chưa "phẳng"; bài 74 xử lý đúng điểm này.
- Pooling nén học được làm mất chi tiết trong khối: thông tin của một token xa chỉ còn trong bản tóm tắt 4 hoặc 128 token. Tác vụ cần chép nguyên văn từ xa (như MRCR nhiều needle) có thể chịu thiệt; Opus-4.6 vẫn hơn trên MRCR 1M.
- Đánh giá 1M chỉ có MRCR và CorpusQA.

10. Liên hệ với các bài khác

- **16 (MLA):** entry 512 chiều + 64 chiều RoPE dùng chung cho mọi head chính là latent MLA ở chế độ MQA, nay thành dạng bản địa; V4 thêm nén theo chuỗi lên trên.
- **64 (NSA):** cùng bộ ba "nén, chọn, cửa sổ". Khác biệt: V4 chọn bằng indexer học riêng (kiểu DSA) thay vì dùng điểm attention nén; trộn nhánh bằng một softmax chung thay vì cổng học; pooling nén theo từng kênh có cửa sổ chồng lấp.
- **68 (DSA, V3.2):** CSA = DSA chạy trên entry nén; top-k giảm từ 2048 xuống 512/1024; indexer từ FP8 xuống FP4; quy trình dense warm-up rồi sparse vẫn giữ (nhưng train từ đầu thay vì continued training).
- **55, 67:** sink logit học được là một cách xử lý attention sink; bài 67 bỏ sink bằng cổng output. **06, 17:** sliding window. **14, 15:** MQA, GQA. **12:** RoPE.
- **46, 47:** quản lý KV dị thể, tái dùng prefix. **24, 26:** hướng lai linear + full attention.
- **65, 66, 69, 70, 72, 73:** các lựa chọn sparse/lai khác cùng giai đoạn. **74:** kế thừa trực tiếp.
- Ngoài thư mục: distillation 28, 29 (reverse KL, on-policy KD); rl 39 (DeepSeek-R1, GRPO).

11. Tóm tắt công thức / khái niệm

Khái niệm	Công thức / ý	Ghi chú
Nén CSA	$C_i^{Comp} = \sum S_j^a \odot C_j^a + \sum S_j^b \odot C_j^b$, softmax theo kênh trên $2m$ vị trí	cửa sổ $2m$, bước m, cache $1/m$
Nén HCA	$C_i^{Comp} = \sum_{j \in \text{khối } i} S_j \odot C_j$	$m' = 128$, không chồng lấp, attention dày
Indexer	$I_{t,s} = \sum_h w_{t,h}^I \text{ReLU}(q_{t,h}^I \cdot K_s^{IComp})$, $s < \lfloor t/m \rfloor$	như DSA nhưng trên entry nén, FP4
Top-k	512 (Flash), 1024 (Pro)	V3.2: 2048 token
Shared-KV MQA	entry 512 chiều vừa là K vừa là V	MLA chế độ MQA dạng bản địa
Partial RoPE	64 chiều cuối; output nhân R_{-t}	output chỉ phụ thuộc khoảng cách

Khái niệm	Công thức / ý	Ghi chú
SWA	128 entry không nén nối vào attention	phủ khối hiện tại
Sink	$\exp(z'_h)$ cộng vào mẫu softmax	tổng trọng số có thể < 1
Grouped output	g nhóm, mỗi nhóm về d_g, rồi về d	Flash: giảm khoảng một nửa tham số (tự tính)
Chi phí 1M	Pro 27% FLOPs, 10% KV; Flash 10%, 7% so với V3.2	khoảng 2% KV của BF16 GQA8
Byte/token (tự tính)	Flash khoảng 3.47 KB, Pro khoảng 4.97 KB	giả định BF16/FP8 + MXFP4
mHC	$X_{l+1} = B_l X_l + C_l \mathcal{F}_l(A_l X_l)$, B_l ngẫu nhiên kép	$n_{hc} = 4$, Sinkhorn 20 vòng

12. Kết luận ngắn

DeepSeek-V4 đi tiếp từ DSA bằng cách nén KV theo chiều chuỗi trước khi chọn thưa: CSA gộp 4 token thành một entry rồi để lightning indexer chọn top-k trên các entry nén, HCA gộp 128 token và attention dày lên tất cả, hai loại xen kẽ theo lớp, luôn kèm một cửa sổ 128 token không nén, sink và RoPE một phần. Theo paper, ở 1M token V4-Pro chỉ cần 27% FLOPs và 10% KV cache của V3.2; tính tay cho thấy KV khoảng 3.5-5 KB mỗi token, cỡ 2% của GQA8 thông thường. Paper không có ablation cho từng thành phần attention.

Link gốc: https://arxiv.org/abs/2606.19348

Bài 72

MiniMax Sparse Attention (2026)

Tác giả: Xunhao Lai, Weiqi Xu, Yufeng Yang, Qiaorui Chen, Yang Xu, Lunbin Zeng, Xiaolong Li, Haohai Sun, Haichao Zhu, Vito Zhang, Jinkai Hu, Jiayao Li, Rui Gao, Zekun Li, Songquan Zhu, Jingkai Zhou, Pengyu Zhao (MiniMax; Đại học Bắc Kinh; NVIDIA; Đại học Chiết Giang; Đại học KH&CN Hoa Trung; Đại học Nam Kinh; Đại học Điện tử Hàng Châu).
Venue / arXiv: arXiv:2606.13392 (v2, 12/06/2026; v1 ngày 11/06/2026); paper không nêu venue.
Nguồn: https://arxiv.org/abs/2606.13392
Chủ đề: MSA: sparse attention theo block xây trên GQA; Index Branch rất nhẹ chọn top-k block **riêng cho từng nhóm GQA**, Main Branch tính softmax chính xác trên các block đó; indexer học bằng KL có stop-gradient; kernel top-k không exp và attention duyệt "KV-outer". Kiểm chứng trên MoE 109B đa phương thức.
Vị trí trong lộ trình: Đọc trước: 15 (GQA), 44/45 (FlashAttention), 55 (attention sink), 64 (NSA), 65 (MoBA), 68 (DSA). MSA nằm giữa ba bài 64, 65, 68: chọn theo **block** như NSA/MoBA, **indexer riêng học bằng KL** như DSA, nhưng chia sẻ lựa chọn **theo nhóm GQA**. Đọc tiếp: 73 (QSA của Qwen, nén key của indexer, đúng điểm yếu tự tính ở mục 3.5 bài này).

> Tài liệu học tập bằng tiếng Việt, là ghi chú diễn giải, không phải bản dịch. Số liệu lấy từ paper (full text arXiv, tải ngày 07/11/2026); các phép tính đánh dấu "tự tính" là của người ghi chú.

1. Bối cảnh

Các quy trình agent dài có thể cần attention trên hàng trăm nghìn đến hàng triệu token, trong khi causal softmax attention (bài 01) tốn $\Theta(2H_qN^2d_h)$ FLOPs. Bài báo phân biệt hai hướng: kiến trúc lai thay một phần lớp softmax bằng linear attention hoặc sliding window, và làm thưa chính softmax attention. MSA chọn hướng thứ hai nhằm tận dụng hạ tầng phần mềm và phần cứng sẵn có.

Về linear attention của MiniMax. Paper **không bàn** việc MiniMax rời khỏi linear attention: nó chỉ trích MiniMax-01 và MiniMax-M1 (lightning attention) như ví dụ "hybrid stacks" trong phần mở đầu và Related Works, không giải thích vì sao chọn sparse softmax. Ghi chú không bổ sung câu chuyện đó từ nguồn ngoài.

Công trình trước. Paper phê bình các phương pháp chỉ thưa lúc suy luận (H2O, Quest, MInference...) là "kế thừa chi phí huấn luyện của full attention và để ít nhất một pha suy luận gần tốc độ full attention"; gần MSA nhất là nhóm huấn luyện native: NSA, InfLLM-V2, MoBA, DSA.

Khái niệm nền.

- **GQA** (bài 15): H_q query head chia H_{kv} nhóm, mỗi nhóm $G = H_q/H_{kv}$ head dùng chung một KV head.
- **Hai giai đoạn** (Eq. 2): $\mathcal{I}_i = \text{Index}_\phi(q_i, K_{\le i})$ (**Index Branch**), rồi $o_i = \text{Attn}(q_i, K[\mathcal{I}_i], V[\mathcal{I}_i])$ (**Main Branch**).
- **Block** (Eq. 4): $\mathcal{B}_b = \{(b-1)B_k + 1, \dots, \min(bB_k, N)\}$, $B = \lceil N/B_k \rceil$. Chọn theo block cho truy cập bộ nhớ liền mạch.
- **Tensor core MMA**: chỉ chạy hết công suất khi tile đủ lớn (ví dụ 128×128). **Arithmetic intensity**: FLOPs/byte (bài 64).

2. Ý tưởng chính

MSA gắn vào một lớp GQA chuẩn đúng **hai ma trận chiếu** (W_q^{idx}, W_k^{idx}): mỗi nhóm GQA một index query head, cả lớp một index key head. Điểm token của indexer được gộp lên block bằng **max-pooling**, mỗi nhóm

GQA tự chọn top-k block (luôn kèm block cục bộ chứa query), và Main Branch chạy softmax chính xác trên tối đa kB_k token, nên chi phí mỗi query cố định khi N tăng. Top-k không khả vi, nên indexer học bằng **KL loss** khớp với phân phối attention trung bình của nhóm, với **stop-gradient** ở đầu vào indexer và một pha **warmup** full attention. Theo "dao cạo Occam", các thành phần không cần (value head của indexer, ép sink, cửa sổ cục bộ lớn, sink học được) bị bỏ sau ablation. Kernel đồng thiết kế: top-k không exp, attention duyệt theo block KV để lấp tensor core.

3. Phương pháp chi tiết: Index Branch, Main Branch và huấn luyện

3.1 Index Branch (Eq. 5-7)

$$Q^{idx} = XW_q^{idx} \in \mathbb{R}^{N\times H_{kv}\times d_{idx}}, \qquad K^{idx} = XW_k^{idx} \in \mathbb{R}^{N\times 1\times d_{idx}}$$

$$\boxed{S_{i,j}^{idx,(r)} = \frac{(Q^{idx})_i^{(r)}(K^{idx})_j^\top}{\sqrt{d_{idx}}}, \quad M_{i,b}^{idx,(r)} = \max_{j\in\mathcal{B}_b,\, j\le i} S_{i,j}^{idx,(r)}, \quad \mathcal{I}_i^{(r)} = \mathrm{TopK}_b(M_{i,\cdot}^{idx,(r)}, k)}$$

Block không có token nhìn thấy được nhận $-\infty$; block cục bộ chứa i luôn được chọn; $\mathcal{I}_i^{(r)}$ dùng chung cho G head của nhóm r. Khác lightning indexer của DSA (bài 68): không ReLU, không trọng số theo head. Dùng max thay vì mean (diễn giải): block 128 token thường chỉ có vài token quan trọng, mean làm loãng tín hiệu đó.

3.2 Main Branch (Eq. 8)

$$\boxed{O_i^{(h)} = \mathrm{softmax}\left(\frac{Q_i^{(h)}\, K^{(r)}[\mathcal{I}_i^{(r)}]^\top}{\sqrt{d_h}}\right) V^{(r)}[\mathcal{I}_i^{(r)}], \quad h \in \mathcal{H}_r}$$

Mỗi head giữ query projection riêng, chỉ tập block dùng chung trong nhóm. Chi phí mỗi query giảm từ $O(N)$ xuống $O(kB_k)$.

3.3 Huấn luyện indexer (Eq. 9-11)

KL loss. Gọi $\mathcal{I}_{i,tok}^{(r)} = (\bigcup_{b\in\mathcal{I}_i^{(r)}} \mathcal{B}_b) \cap \{1..i\}$ là token do block đã chọn sinh ra. Trên tập này:

$$P_{i,j}^{idx,(r)} = \frac{e^{S_{i,j}^{idx,(r)}}}{\sum_{u\in\mathcal{I}_{i,tok}^{(r)}} e^{S_{i,u}^{idx,(r)}}}, \qquad P_{i,j}^{(r)} = \frac{1}{G}\sum_{\ell\in\mathcal{H}_r} \frac{e^{S_{i,j}^{(\ell)}}}{\sum_{u\in\mathcal{I}_{i,tok}^{(r)}} e^{S_{i,u}^{(\ell)}}}$$

với $S_{i,j}^{(\ell)} = Q_i^{(\ell)}(K_j^{(r)})^\top/\sqrt{d_h}$ (teacher là trung bình xác suất của G head Main Branch). Loss:

$$\boxed{\mathcal{L}_{KL} = \frac{1}{NH_{kv}}\sum_{i=1}^{N}\sum_{r=1}^{H_{kv}} D_{KL}(\mathrm{sg}(P_{i,\cdot}^{(r)}) \,\|\, P_{i,\cdot}^{idx,(r)}), \qquad \mathcal{L} = \mathcal{L}_{LM} + \lambda \sum_{\text{layers}} \mathcal{L}_{KL}}$$

Đây là KL "thuận" như forward KL trong KD: indexer bị phạt nặng khi bỏ sót token Main Branch chú ý nhiều. Paper **không nêu** λ và d_{idx}.

Gradient Detach (Eq. 11). $Q^{idx} = \mathrm{sg}(X)W_q^{idx}$, $K^{idx} = \mathrm{sg}(X)W_k^{idx}$: cùng với teacher đã detach, $\mathcal{L}_{KL}$ **chỉ** cập nhật W^{idx}, không chạm backbone (lý do ở mục 6.2).

Indexer Warmup. Vài bước đầu cả hai nhánh chạy full attention, chỉ train W^{idx} bằng KL trên toàn tiền tố; sau đó bật sparse, KL tính trên vị trí top-k. Dùng cả khi thưa hóa checkpoint dense.

Local Block. Block chứa i luôn được chọn (train và inference), $k-1$ slot còn lại do indexer quyết định.

3.4 Pseudo-code (theo Algorithm 1)

```
def msa_layer(X, Bk=128, k=16, training=True):
    Q, K, V = X @ Wq, X @ Wk, X @ Wv            # (N,Hq,dh), (N,Hkv,dh), (N,Hkv,dh)
    Qi, Ki = X.detach() @ Wq_idx, X.detach() @ Wk_idx   # (N,Hkv,d_idx), (N,1,d_idx)
    M = block_max_pool(Qi @ Ki.T / sqrt(d_idx), Bk, causal=True)   # (N,Hkv,B)
    I = topk(M, k, force_local_block=True)       # top-k block cho TỪNG nhóm GQA
    out = topk_attn(Q, K, V, I) @ Wo             # softmax chính xác trên block đã chọn
    L_kl = kl(group_mean_softmax(Q.detach(), K.detach(), I),   # teacher, Eq. 9
              softmax_over_tokens(Qi @ Ki.T, I)) if training else 0
    return out, L_kl
# warmup: I = toàn bộ tiền tố (full attention), chỉ cập nhật Wq_idx, Wk_idx
```

3.5 Độ phức tạp (Eq. 12) và tính số

$$F_{GQA}(N) = 2H_q d_h N^2, \qquad F_{MSA}(N) = \underbrace{H_{kv} d_{idx} N^2}_{\text{Index}} + \underbrace{4H_q d_h N k B_k}_{\text{Main}}$$

(GQA mang hệ số nửa vì causal; Index Branch chỉ có $QK^\top$; Main Branch tính đủ kB_k token cho $QK^\top$ và PV.)

Tính số (tự tính) với $H_q = 64, H_{kv} = 4, d_h = 128, B_k = 128, k = 16$. Paper không cho d_{idx}; giả định $d_{idx} = 128$, $N = 2^{20}$ tái tạo đúng 28.4× ($17{,}180/604.0 = 28.44$), nên nhiều khả năng là cấu hình thật (suy luận):

N	GQA (MFLOP/token)	MSA (MFLOP/token)	Giảm	Phần Index trong MSA
8K	134.2	71.3	1.88×	5.9%
32K	536.9	83.9	6.4×	20%
128K	2,147	134.2	16.0×	50%
1M (2^{20})	17,180	604.0	**28.4×**	**89%**

Hai nhận xét (tự tính): (1) **hòa vốn** khi $2H_q d_h N = H_{kv} d_{idx} N + 4H_q d_h k B_k$, tức $N \approx 4{,}228$; dưới mức này MSA về lý thuyết tốn hơn GQA; (2) ở 1M, gần 90% FLOPs của MSA nằm ở Index Branch vốn vẫn $O(N^2)$, đúng động lực của QSA (bài 73).

3.6 Ví dụ nhỏ (tự tính)

$N = 10$, $B_k = 4$ (block $\{1..4\}, \{5..8\}, \{9, 10\}$), $k = 2$, query $i = 10$, điểm index $S = [2.0, 0.1, -0.5, 0.3 \,|\, 0.2, 0.4, 1.1, -0.2 \,|\, 0.8, 1.5]$. Block max: 2.0 và 1.1; block cục bộ $\{9, 10\}$ luôn chọn, slot còn lại thuộc block 1. Tập token $\{1, 2, 3, 4, 9, 10\}$, $P^{idx} = [0.431, 0.064, 0.035, 0.079, 0.130, 0.261]$. Nếu Main Branch thật ra chú ý mạnh token 7 (block bị bỏ), KL trên tập đã chọn không thấy; chỉ warmup (KL trên toàn tiền tố) dạy được điều đó (diễn giải).

4. Thiết kế kernel

Index và TopK (4.1). Softmax bảo toàn thứ tự nên forward bỏ max/exp/sum, đưa điểm thô vào top-k. B_k lớn tăng intensity của attention; k nhỏ giữ B và k dưới "điểm ngọt" của top-k tổng quát (radix select lợi khi B lớn; bitonic sort $O(B \log_2 B)$). Chọn $B_k = 128, k = 16$: 32 lane của warp mỗi lane đọc 1/32 hàng, giữ min-heap k phần tử trong shared memory (gốc cache trong thanh ghi), cuối cùng trộn bằng k vòng shuffle; mỗi lane một bank cố định.

Bảng 1 (độ trễ top-k, µs, fp32, H800, trung vị 50 lần; index giống hệt nhau):

N	B	k	torch	TileLang	MSA	so torch	so TileLang
128K	1024	16	3970	2864	779	5.1×	3.7×
128K	2048	32	5378	3630	1991	2.7×	1.8×
512K	4096	16	33810	17779	7880	4.3×	2.3×
512K	8192	32	57659	26100	21326	2.7×	1.2×

(Triển khai dùng $B_k = 128, k = 16$; hàng $k = 32$ ứng với $B_k = 64$.)

Vì sao KV-outer (4.2). Prefill, phần tử 2 byte. *Q-outer* (Eq. 13-14): FLOPs $= 4H_qNd_hkB_k$, IO $= 4H_qNd_h + 4H_{kv}NkB_kd_h$; số hạng sau chiếm ưu thế nên (tự tính lại) FLOPs/IO $\approx H_q/H_{kv} = G$. *KV-outer* (Eq. 15-16): IO $= 4H_{kv}Nd_h + 4H_qNkd_h + 2H_qN(k+1)d_h \approx 6H_qNkd_h$, nên

$$\boxed{\text{FLOPs/IO} \approx \frac{4H_qNd_hkB_k}{6H_qNkd_h} = \frac{2}{3}B_k}$$

(pdftotext in nhầm thành "23 B_k"). Với $B_k = 128$: khoảng 85 so với $G = 16$, cao hơn khoảng 5.3 lần (tự tính). Kernel chạy persistent grid (số thread block cố định, mỗi block tự lấy việc kế tiếp) trên tile (kv_block, kv_head); reverse sparse index cho biết query nào cần nạp.

- **Pre-scheduled tile chunking:** "sink rows" (block đầu được gần như mọi query chọn, bài 55) làm lệch tải; tile bị cắt theo chiều query thành chunk tối đa khoảng $2kB_k$ query, mỗi cặp (query, chunk) được gán trước một slot $s \in [0, k)$ trong O_{buf}, nên ghi không cần atomic.
- **Two-phase forward:** kết quả riêng phần ghi vào $O_{buf} \in \mathbb{R}^{k\times n\times H_q\times d}$ và LSE_{buf}; kernel combine tính $a = \max_s LSE_s$, $LSE = a + \log\sum_s e^{LSE_s - a}$, $w_s = e^{LSE_s - LSE}$, $O = \sum_s w_s O_{buf}[s]$ (phép gộp split-K kiểu FlashDecoding).
- **Query concatenation:** một vị trí query chỉ cho $G = 16$ hàng MMA; ở KV-outer mọi query của tile chung toán hạng KV, nên ghép $\lceil 128/G \rceil = 8$ vị trí thành MMA 128×128.

Sparse KL loss (4.3). Main pass ghi luôn LSE_{main}, Index Branch rút gọn LSE theo block trên top-k để có LSE_{idx}, nên bỏ được forward KL riêng; backward dùng persistent grid với cân bằng tải động.

5. Thí nghiệm và kết quả chính

Mô hình. 41 lớp (3 dense + 38 MoE), khoảng 109B tổng / 6B kích hoạt, vocab 200K, $d_{model} = 3072$; 64 query head, 4 KV head ($G = 16$), $d_h = 128$, RoPE dim 64; 128 expert định tuyến + 1 chia sẻ, top-4. MSA: $B_k = 128$, $k = 16$ (2048 token/query/nhóm). Dữ liệu trộn text, ảnh, video.

Ngân sách 3T token. *MSA-PT*: từ đầu, 40B token warmup rồi sparse đến hết. *MSA-CPT*: từ checkpoint GQA đã train 2.6T, thay attention bằng MSA rồi train tiếp 400B (40B đầu là warmup). Learning rate, độ dài chuỗi pretraining: paper không nêu.

Động lực huấn luyện. LM loss của MSA-PT và full attention qua 3T token "gần như không phân biệt được", gradient norm cùng khoảng (Hình 2). Ở CPT, warmup hạ KL nhanh và KL vẫn thấp sau khi bật sparse. Với $\mathcal{I}^\star$ là top-k theo điểm Main Branch và $\hat{\mathcal{I}}$ là lựa chọn indexer: **block recall** $= |\mathcal{I}^\star \cap \hat{\mathcal{I}}|/|\mathcal{I}^\star|$, **score recall** $= \sum_{b\in\mathcal{I}^\star\cap\hat{\mathcal{I}}} P_b / \sum_{b\in\mathcal{I}^\star} P_b$; score recall cao hơn, tức block bị sót chủ yếu ít khối lượng (Hình 3, chỉ có đồ thị).

Bảng 2 (theo nhóm: General 7 dòng đầu, Math, Code, Retrieval, Image, Video, PPL↓):

Benchmark	Full	MSA-PT	MSA-CPT
MMLU	67.0	67.2	66.8
MMLU-Pro	38.5	38.8	39.1
BBH	67.7	66.6	66.1
GPQA Hard	25.9	26.3	26.3
ARC Challenge	82.7	82.5	82.9
TriviaQA	66.0	65.5	67.7
WinoGrande	58.3	60.9	62.0
GSM8K	76.2	77.7	73.7
MGSM	44.1	46.0	44.2
MathVista	43.8	46.8	44.5
OlymMATH Easy P@100	23.0	26.0	22.0
HumanEval	61.0	64.0	57.9
EvalPlus	59.4	61.8	60.0
BigCodeBench	44.8	44.0	45.7
MultiPL-E MBPP P@10	82.1	81.6	81.1
RULER-8K	79.8	84.2	77.2
RULER-32K	75.0	77.5	75.7
AI2D	68.3	70.6	67.3
ChartQA	75.0	75.4	71.4
MMMU	46.8	45.9	44.5
OCRBench v2	55.0	55.7	54.3
CharXiv	37.55	41.55	37.15
VisualWebBench	55.6	68.4	59.4
CVBench	57.0	59.7	58.8
EgoSchema	29.6	37.6	25.8
LongVideoBench	38.5	41.8	38.9
MLVU	44.14	46.94	43.68
MMVU	45.8	47.5	45.8
VideoMME	41.11	45.48	39.65
TemporalBench	49.4	53.4	50.6
TAU2 (PPL↓)	1.155	1.148	1.150
AgentCompany (PPL↓)	1.248	1.249	1.247
HLE (PPL↓)	1.275	1.278	1.275
SWE (PPL↓)	1.216	1.218	1.216

Paper đọc: MSA-PT mạnh nhất ở nhiều ô math, ảnh, video, retrieval (sparse native giúp biểu diễn thích nghi với mẫu thưa); MSA-CPT “bảo thủ” hơn, bám checkpoint dense ở text, code, PPL, là đường chuyển đổi thực dụng. Khoảng cách còn lại “phụ thuộc benchmark”.

Bảng 3 (MSA-CPT sau thêm khoảng 140B token mở rộng ngữ cảnh dài):

Benchmark	Subset	Full	MSA-CPT	Δ
HELMET-128K	Overall	46.53	45.93	-0.60
	ICL	70.40	72.80	+2.40
	Rerank/RAG	34.60	32.50	-2.10

Benchmark	Subset	Full	MSA-CPT	Δ
RULER-128K	Overall	72.00	72.12	+0.12
	CWE/FWE	46.35	45.00	-1.35
	MK/MQ/MV	96.63	98.87	+2.24
	QA1/QA2	47.80	46.80	-1.00
	VT	97.80	96.80	-1.00

Ở 128K, mỗi query mỗi nhóm chỉ thấy 2,048 token (1.6% ngữ cảnh, tự tính).

Hiệu năng (Hình 4; 64/4 head, $d_h = 128$, H800). FLOPs attention/token giảm **28.4×** ở 1M; đo thật **14.2× prefill**, **7.6× decode** ở 1M. Tốc độ thật thấp hơn mức giảm FLOPs do overhead (dựng index, top-k, reverse index, gom query, cân bằng tải, truy cập kém đều), nhưng tăng theo độ dài. Latency tuyệt đối chỉ có trên đồ thị.

6. Phân tích / ablation

Phụ lục B dùng pilot 10.53B tổng / 1.47B kích hoạt (16 lớp, $d_{model} = 2048$, 32 query / 4 KV head, $d_h = 128$, 64 expert top-4). Phần lớn kết quả B chỉ là đồ thị; ghi chú nêu kết luận định tính.

6.1 Nguồn gradient (B.2). Sparse từ bước 0 với (a) *chỉ LM* (thêm value projection để Index Branch đóng góp đầu ra $O' = W_o O + W_o^{idx} O^{idx}$), (b) *chỉ KL* (bỏ O^{idx}), (c) *LM + KL*. (a) giữ năng lực ngắn nhưng retrieval dài kém; (b) ngược lại; (c) cân bằng nhất.

6.2 Detach (B.3). Không detach, gradient KL vào backbone: hệ số lớn thì spike gradient và **LM loss phân kỳ trong vài trăm bước**; hệ số ổn định thì benchmark ngắn **thoái lui dần**, do **self-distillation** (backbone giảm KL bằng cách làm phân phối Main Branch đơn giản đi thay vì indexer tốt lên). Detach chữa cả hai.

6.3 Warmup (B.4). Entropy attention rơi nhanh trong vài trăm bước đầu (Hình 10), nên bật top-k từ bước 0 buộc indexer đuổi theo mục tiêu đang chạy khi chính nó chọn gần như ngẫu nhiên. Warmup cải thiện cả ngắn lẫn retrieval dài.

6.4 Sink học được (B.5), sliding window (B.6). Sink kiểu GPT-OSS không xóa hẳn sink ở token đầu và không cải thiện PPL nhất quán, bị loại (Phụ lục A: mọi head dồn khối lượng đáng kể vào token đầu dù không ép). Sliding window cùng FLOPs (block đầu + cửa sổ cùng ngân sách) có PPL agent cao hơn MSA suốt quá trình.

6.5 Kích thước block (C.1, Bảng 4; cùng tổng số token chọn):

Benchmark	Block 32	Block 64	Block 128
TAU2 / AgentCompany (PPL↓)	1.176 / 1.266	1.176 / 1.276	1.176 / 1.266
HLE / SWE (PPL↓)	1.299 / 1.233	1.299 / 1.233	1.300 / 1.233
RULER-8K	72.5	72.8	73.8
RULER-32K	66.1	65.3	64.6

6.6 Ép sink + cửa sổ cục bộ (C.2, Bảng 5, trích các dòng khác nhau rõ nhất).

Benchmark	No Forced	Forced
MMLU-Pro	32.5	33.4
GSM8K	66.0	66.9

Benchmark	No Forced	Forced
MMMU	43.6	42.9
RULER-8K	71.6	71.7
RULER-32K	61.5	65.8

MMLU (60.5/60.5), BBH (58.2/58.2) và PPL gần như không đổi. Paper kết luận sink và mẫu cục bộ tự học được, bản cuối **chỉ ép block "self" chưa đầy**.

6.7 Value head của indexer (C.3, Bảng 6, CPT). Bỏ value head không làm giảm có hệ thống: ví dụ MMLU 66.4 → 67.3, GSM8K 77.6 → 76.4, BigCodeBench 46.0 → 44.0, OCRBench v2 53.2 → 53.9, RULER-8K 84.1 → 83.0, RULER-32K 79.7 → 80.4 (with-value → no-value). Vai trò của O^{idx} là tín hiệu huấn luyện sớm, không phải dung lượng khi hội tụ; bỏ nó thì lúc suy luận indexer chỉ cần max theo block của $Q^{idx}K^{idx\top}$, không đường value, không exp.

Tóm lại (người ghi chú): pilot cần value head và ép sink/cửa sổ; ở quy mô lớn, warmup thay value head, sink và cục bộ tự học được, detach chữa spike.

7. Giới hạn

Tác giả nêu: còn khoảng cách retrieval dài (hướng: train sparse lâu hơn, ngân sách lớn hơn lúc suy luận, hàm chấm điểm giàu hơn); tốc độ thật thấp hơn mức giảm FLOPs; ablation phụ lục ít bước và tập đánh giá con; RL post-training và triển khai agentic để sau.

Người ghi chú nhận xét:

- d_{idx}, λ, learning rate, độ dài chuỗi pretraining không được nêu; $d_{idx} = 128$ ở mục 3.5 là suy ngược.
- **Benchmark ngắn không kiểm tra sparse inference:** với chuỗi $\leq kB_k = 2048$, mọi query thấy tối đa 16 block nên chọn hết, MSA trùng dense attention (tự suy từ Eq. 7). Chênh lệch ở prompt ngắn phản ánh tác động của *huấn luyện* thưa (hoặc nhiễu). Một lần chạy, không độ lệch chuẩn; vài chênh lệch rất lớn (VisualWebBench 55.6 so với 68.4, EgoSchema 29.6 so với 37.6) khó quy cho riêng attention.
- Bảng 5: RULER-32K 61.5 so với 65.8 (chênh 4.3 điểm) nhưng paper gọi là "comparable".
- Related Works nói NSA "nhắm MQA/MHA backbone", trong khi bài 64 ghi NSA thí nghiệm trên GQA 4 nhóm; ghi chú giữ cả hai mô tả.
- Index Branch vẫn $O(N^2)$ (khoảng 89% FLOPs MSA ở 1M, tự tính). MSA không giảm dung lượng KV cache, chỉ giảm lượng đọc; khác H2O (bài 56).
- Không so trực tiếp với NSA, MoBA, DSA (baseline thưa duy nhất là sliding window); kernel decode không được mô tả; chỉ đo trên H800.

8. Liên hệ với các bài khác

	NSA (64)	MoBA (65)	DSA (68)	**MSA**	QSA (73)
Backbone	GQA (theo bài 64)	GQA	MLA chế độ MQA	GQA	GQA trong hybrid GDN
Đơn vị chọn	block	block rất lớn	token	block 128	micro-block 4
Chấm điểm	từ attention nhánh nén	key trung bình	ReLU, nhiều head có trọng số	max theo block, 1 q/nhóm	key nén avg-pool, ReLU, 4 head
Chia sẻ lựa chọn	trong nhóm GQA	xem bài 65	mọi query head	**mỗi nhóm GQA một tập**	một tập mỗi query

	NSA (64)	MoBA (65)	DSA (68)	**MSA**	QSA (73)
Học selector	end-to-end qua cổng	chỉ gradient LM	KL, detach	KL, detach, warmup	KL (distill max-pool), 2 giai đoạn
Ngân sách	$n\,l' + w =$ 1536 + nén	xem bài 65	2048 token	2048 token	2048 token

(Cột MoBA theo mô tả của paper MSA.)

- **14/15 (MQA, GQA), 44/45 (FlashAttention):** chia sẻ KV trong nhóm cho phép chia sẻ lựa chọn và lấp MMA; kernel giữ khung tiled online softmax.
- **55, 56, 61, 62, 06, 17, 60:** sink tự xuất hiện (Phụ lục A); các phương pháp chỉ thưa lúc suy luận và mẫu cố định là đối chứng (B.6: cửa sổ cố định thua chọn động).
- **67 (Gated attention):** một cách khác xử lý sink; MSA thử sink học được và bỏ.
- **18-26 (linear attention, GDN, hybrid):** nhánh (i) MSA không đi; bài 73 kết hợp hybrid GDN với sparse attention.
- **distillation 15, 29:** KL của indexer là KD forward KL từ attention Main Branch sang một student nhỏ, teacher bị stop-gradient.

9. Tóm tắt công thức / khái niệm

Khái niệm	Công thức / ý	Ghi chú
Index Branch	$Q^{idx} = XW_q^{idx}$ (H_{kv} head), $K^{idx} = XW_k^{idx}$ (1 head)	chỉ 2 ma trận
Điểm block	$M_{i,b} = \max_{j\in\mathcal{B}_b, j\le i} S_{i,j}^{idx}$	max-pool, không ReLU
Chọn	TopK$(M_{i,\cdot}^{(r)}, k)$ + block cục bộ	mỗi nhóm GQA một tập
KL loss	$D_{KL}(\text{sg}(P) \,\|\, P^{idx})$, P = trung bình softmax của G head	trên token đã chọn
Detach / warmup	$Q^{idx} = \text{sg}(X)W_q^{idx}$; full attention trước	KL chỉ cập nhật indexer; 40B token
FLOPs	$2H_q d_h N^2$ vs $H_{kv} d_{idx} N^2 + 4H_q d_h N k B_k$	28.4× ở 1M
Intensity	Q-outer $\approx G$; KV-outer $\approx \frac{2}{3}B_k$	chọn KV-outer
Combine	$w_s = e^{LSE_s - LSE}$, $O = \sum_s w_s O_{buf,s}$	split-K (chia trục key thành nhiều phần rồi gộp), không atomic
Kết quả	ngang GQA ở 109B/3T; 14.2× prefill, 7.6× decode ở 1M	H800, $B_k = 128$, $k = 16$

10. Kết luận ngắn

MSA là sparse attention huấn luyện native ở dạng tối giản: indexer hai ma trận chiếu chọn 16 block 128 token cho từng nhóm GQA bằng max-pooling, học bằng KL có stop-gradient sau warmup full attention, cộng kernel top-k không exp và attention duyệt theo block KV. Ở MoE 109B đa phương thức, cả huấn luyện từ đầu lẫn chuyển đổi từ checkpoint GQA giữ chất lượng ngang full attention, với FLOPs attention giảm 28.4× và tốc độ thật 14.2× (prefill) / 7.6× (decode) ở 1M token. Hiệu quả phụ thuộc vào độ dài ngữ cảnh và chi phí indexer theo số nhóm.

Link gốc: https://arxiv.org/abs/2606.13392

Bài 73

On the Design of Qwen3.8-Next Architecture: Evaluation, Efficiency, and Training Stability (2026)

Phạm vi ghi chú: tập trung vào Qwen Sparse Attention (QSA): indexer nhẹ kiểu MQA chấm điểm **micro-block** 4 token (key nén bằng average pooling, chỉ block đã đầy), top-k block mở rộng thành mask thưa mức token (cộng đuôi block chưa đầy), so sánh với indexer token-level $O(n^2)$ của DSA (QSA: $O(n^2/r)$), đưa vào **hai giai đoạn trong CPT ở 256K** (distill indexer với backbone đứng yên, rồi sparse training toàn mô hình), và bố cục hybrid 3 Gated DeltaNet : 1 attention có cổng output. Các điểm này đã đối chiếu với paper và đều đúng; riêng gated attention paper chỉ nhắc tới (Hình 3, §2.2, §3.1), không có mục thiết kế riêng. Đánh giá kiến trúc (Bảng 1) và ổn định huấn luyện (stress test) tóm tắt ngắn; Gated Residual, n-gram embedding, Muon và scaling law chỉ tóm tắt.

Tác giả: Qwen Team; core contributors: Zihan Qiu, Zekun Wang, Xiao Li, Yanpeng Li, Yang Xu, Yixuan Wang, Huaqing Zhang, Rui Men, Bo Zheng, Dayiheng Liu (hai người cuối là tác giả liên hệ), cùng 26 contributor xếp theo alphabet. Paper không ghi tổ chức nào khác ngoài "Qwen Team".
Venue / arXiv: arXiv:2608.30320 (v1, 31/08/2026; đầu PDF ghi 2026-09-01); paper không nêu venue.
Nguồn: https://arxiv.org/abs/2608.30320
Chủ đề: Thiết kế Qwen3.8-Flash-Next (MoE 125B tổng, 6B kích hoạt, thêm 51B tham số bảng n-gram để ngoài GPU): hybrid GDN + attention 3:1, các lớp attention thay bằng QSA khi CPT, Gated Residual (residual rộng 4 nhánh, đọc qua cổng), n-gram embedding, Muon, stress test ổn định.
Vị trí trong lộ trình: Đọc trước: 24 (Gated DeltaNet), 26 (Kimi Linear, hybrid 3:1), 67 (Gated attention, cùng nhóm Qwen), 68 (DSA), 72 (MSA). Đây là bài duy nhất của giai đoạn 8 thiết kế sparse attention **cho chính decoder hybrid GDN + full attention**. Đọc tiếp: 74 (DeepSeek-V4.1-Flash, nén KV cache).

> Tài liệu học tập bằng tiếng Việt, là ghi chú diễn giải, không phải bản dịch. Số liệu lấy từ paper (full text arXiv, tải ngày 07/11/2026); các phép tính đánh dấu "tự tính" là của người ghi chú.

1. Bối cảnh

Bài báo đặt mục tiêu giữ chất lượng flagship thế hệ trước 397B-A17B, được Bảng 11 gọi là Qwen3.7-Plus-Base, với khoảng một phần ba tham số kích hoạt, một phần ba token huấn luyện và một phần chín FLOPs huấn luyện. Các thay đổi được đánh giá theo loss và benchmark downstream; chi phí huấn luyện, prefill, decode; cùng ảnh hưởng đến siêu tham số và độ ổn định. Những thước đo này không phải lúc nào cũng thay đổi cùng chiều: loss có thể giảm trong khi benchmark bão hòa, và một số tác động chỉ xuất hiện sau post-training.

Bài toán của sparse attention trong bài này. DSA (bài 68) dùng lightning indexer token-level để tạo mask thưa, nhưng "khi độ dài tăng, overhead của indexer $O(n^2)$ vẫn không thể bỏ qua". Bài 72 cho thấy cùng hiện tượng: ở 1M token, gần 90% FLOPs của MSA nằm ở indexer (tự tính). QSA nén chuỗi key **ngay trong lớp** để chi phí indexer giảm theo hệ số nén.

Khái niệm nền. GDN (bài 24): linear attention trạng thái ma trận cố định, cập nhật bằng delta rule và cổng suy giảm. **Hybrid**: xen lớp GDN với lớp softmax attention (bài 25, 26). **CPT**: huấn luyện tiếp checkpoint đã pretrain, thường ở ngữ cảnh dài hơn. **MTP**: module dự đoán thêm vài token, làm draft cho speculative decoding (bài 48). **Muon**: optimizer trực giao hóa momentum của ma trận bằng lặp Newton-Schulz (NS).

2. Ý tưởng chính

Token mixing là hybrid **3 lớp GDN : 1 lớp attention toàn cục**; khi CPT, mọi lớp full attention (cả trong module MTP) được thay bằng **QSA**. QSA có một indexer MQA nhỏ (4 query head, 1 key head chung): key

của indexer được **chia block 4 token, average-pool rồi RMSNorm** thành một key nén cho mỗi block, gắn vị trí đầu block bằng partial RoPE; mỗi query token chấm điểm các block **đã đầy** bằng tổng ReLU trên các head, chọn top-$\lceil K/r \rceil$ block, mở rộng thành tập token (cộng phần đuôi block chưa đầy) cho sparse core attention. Vì indexer chạy trên chuỗi đã nén, chi phí indexer giảm từ $O(n^2)$ xuống $O(n^2/r)$. Indexer được khởi tạo bằng **distill phân phối attention dense** (max-pool về block), sau đó cả mô hình được train thưa. Ở 1M token, module attention nhanh hơn GQA dense 7.6× ở prefill, 4.9× ở decode (mức kernel), chất lượng ngắn và dài không giảm.

3. Kiến trúc nền: hybrid GDN và gated attention

Động cơ. Full attention truy cập trực tiếp mọi token trước nhưng chi phí bậc hai và KV cache tăng tuyến tính; SWA giới hạn vùng nhìn nên thông tin ngoài cửa sổ chỉ đi gián tiếp qua độ sâu. GDN nén tiền tố vào trạng thái cố định; lớp attention xen kẽ giữ truy xuất token-level "khó cho bất kỳ bộ nhớ hữu hạn trạng thái nào tái tạo chính xác".

Gated delta rule (Eq. 1-5), trạng thái $S_t \in \mathbb{R}^{d_k \times d_v}$ (chuyển vị so với bản gốc):

$$\tilde{S}_{t-1} = \alpha_t S_{t-1}, \quad e_t = v_t - \tilde{S}_{t-1}^\top k_t, \quad S_t = \tilde{S}_{t-1} + \beta_t k_t e_t^\top, \quad y_t = S_t^\top q_t$$

Thế e_t vào (tự kiểm lại): $S_t = \alpha_t S_{t-1} + \beta_t k_t v_t^\top - \alpha_t \beta_t k_t k_t^\top S_{t-1}$, tức

$$\boxed{S_t = \alpha_t (I - \beta_t k_t k_t^\top) S_{t-1} + \beta_t k_t v_t^\top}$$

α_t điều khiển "tuổi thọ" trạng thái; delta term chỉ ghi phần sai số so với value đang gắn với k_t, nên key lặp lại cập nhật liên kết cũ thay vì cộng dồn.

Tham số hóa (Eq. 6-11): $q_t, k_t = \text{L2Norm}(\text{SiLU}(\text{ShortConv}(W x_t)))$, $v_t = \text{SiLU}(\text{ShortConv}(W_v x_t))$, $\beta_t = \sigma(W_\beta x_t)$, $\alpha_t = \exp[-\exp(A)\,\text{softplus}(W_\alpha x_t + b_\alpha)]$, đầu ra $o_t = W_o[\sigma(W_z x_t) \odot \text{RMSNorm}(y_t)]$. Khác GDN gốc: **cổng output sigmoid** thay SiLU ("cải thiện nhất quán"); **zero-centered RMSNorm** (theo Qwen3-Next) cho mọi RMSNorm để hạn chế trọng số norm phình to.

Lớp attention. Giữ **RoPE** trong lớp full attention: biến thể NoPE gần như không khác khi pretrain nhưng sau post-training **tỉ lệ sinh không dừng (endless generation) cao hơn đáng kể**. Một lớp full attention mỗi bốn lớp; "full attention định kỳ đặc biệt quan trọng cho ngữ cảnh dài". **Cổng output của attention:** paper không mô tả công thức, nhưng Hình 3 vẽ cổng sigmoid trên đầu ra sparse core attention, §2.2 viết "các cổng trong GR, gated attention và GDN đều chặn biên độ những gì ghi vào residual", và §3.1 gọi đó là "attention output gate (Qiu et al., 2025b)", chính là bài 67.

Bảng 1 (MoE 25B-A3B, 28 lớp, kiến trúc Qwen3.5; 400B token ở 4K rồi 80B token ở 32K; hai hybrid đều 1 full attention / 4 lớp, SWA cửa sổ 128):

Kiến trúc	MMLU	MMLU-Pro	SuperGPQA	MATH	GSM8K	BBH	MMMLU	EvalPlus	MultiPL-E	Avg.
Full attention	62.65	37.59	21.76	49.40	75.13	63.78	47.74	51.01	39.73	49.87
SWA hybrid	66.30	40.67	22.45	45.48	74.22	65.88	51.33	52.12	41.93	51.15
GDN hybrid	66.26	42.82	23.45	53.98	77.07	68.72	54.83	49.71	47.48	53.81

GDN hybrid hơn Transformer ở 8/9 benchmark, hơn SWA hybrid ở 7/9 (SWA nhỉnh ở MMLU và EvalPlus).

Paper tự lưu ý: kết quả ủng hộ thiết kế hybrid nhưng "không cô lập được thành phần nào gây ra cải thiện". Kernel GDN dùng **FlashQLA** (TileLang): forward nhanh 2-3×, backward khoảng 2× so với kernel Triton của FLA.

4. Phương pháp chi tiết: QSA

4.1 Compressed lightweight indexer (Eq. 12-16)

Indexer là MQA với H query head và một key head chung:

$$\hat{q}_i^h = \text{RMSNorm}(W_h^Q x_i), \qquad k_i = W^K x_i$$

Key chia block không chồng lấn r token, block b bắt đầu ở $p_b = b \cdot r$, nén bằng average pooling:

$$\hat{k}_b = \text{RMSNorm}(\text{AvgPool}(k_{p_b:p_b+r-1})), \qquad 0 \le b < \lfloor n/r \rfloor$$

Partial RoPE sau khi nén: RoPE chỉ áp lên 64/128 chiều mỗi head (khớp chiều quay của core attention); query giữ vị trí token i, key nén nhận **vị trí đầu block** p_b: $q_i^h = \text{PRoPE}(\hat{q}_i^h, i)$, $\bar{k}_b = \text{PRoPE}(\hat{k}_b, p_b)$. Nén trước rồi mới quay để "tránh lấy trung bình các biểu diễn có pha quay khác nhau" (diễn giải theo bài 12: các vector đã xoay theo góc khác nhau cộng lại sẽ triệt tiêu một phần lẫn nhau).

Điểm block-causal và chọn:

$$I_{ib} = \begin{cases} \sum_{h=1}^{H} \text{ReLU}(\langle q_i^h, \bar{k}_b \rangle), & p_b + r - 1 \le i \\ -\infty, & \text{ngược lại} \end{cases} \qquad \mathcal{B}_i = \text{TopK}_{K_B}(\{I_{ib}\}_b), \quad K_B = \lceil K/r \rceil$$

Chỉ block **đã quan sát đầy đủ** mới được chấm. Khác DSA: tổng ReLU **không** có trọng số theo head $w_{t,j}^I$. I_{ib} không có chỉ số head nên mỗi query có **một** tập chọn dùng chung cho mọi head core attention (tự suy từ công thức; khác MSA bài 72 chọn riêng từng nhóm GQA).

4.2 Mở rộng thành mask token (Eq. 19)

$$S_i = \text{Expand}(\mathcal{B}_i) \cup \left\{ r\lfloor \tfrac{i+1}{r} \rfloor, \ldots, i \right\}$$

Expand ánh xạ block thành chỉ số token gốc, cắt về ngân sách K; tập token của block cuối chưa đầy luôn được thêm. Core attention chạy softmax trên S_i (Hình 3: "micro-block sparse attention mask").

4.3 Huấn luyện hai giai đoạn trong CPT ở 256K

Stage 1: Dense Distillation (paper cũng gọi là warm-up). Teacher token-level: cộng phân phối softmax của mọi head core attention rồi chuẩn hóa L1, được $a_i \in \mathbb{R}^n$. Gộp về block bằng **max pooling** (theo SeerAttention, bài 63, và ProxyAttn) để giữ tín hiệu token nổi bật:

$$\bar{a}_{ib} = \text{MaxPool}(a_{i,p_b:p_b+r-1}), \qquad \hat{a}_i = \bar{a}_i / \|\bar{a}_i\|_1, \qquad \mathcal{L}_{KL} = \frac{1}{N} \sum_i D_{KL}(\hat{a}_{i,:} \,\|\, \text{Softmax}(I_{i,:}))$$

Chỉ block đầy đủ được tính. **Chỉ indexer được train**, 1,000 bước, lr 1×10^{-3}, mỗi bước 8 chuỗi 256K, khoảng 2B token ($1000 \times 8 \times 262{,}144 = 2.10$B, tự tính).

Stage 2: Sparse Training. Toàn bộ backbone train dưới sự dẫn đường của indexer; KL chỉ tính trên top-K_B block, teacher được chuẩn hóa lại trong $\mathcal{B}_i$: $\mathcal{L}_{KL} = \frac{1}{N}\sum_i D_{KL}(\hat{a}_{i,\mathcal{B}_i} \,\|\, \text{Softmax}(I_{i,\mathcal{B}_i}))$. 8,000 bước, lr 2.5×10^{-5}, 96 chuỗi 256K mỗi bước, khoảng 200B token (tự tính 201.3B). Paper **không nêu** đầu vào indexer có bị detach như DSA/MSA không.

So với DSA (số DSA lấy từ bài 68):

	DSA (bài 68)	QSA
Giai đoạn 1	1000 bước, lr 10^{-3}, 16 × 128K, khoảng 2.1B token	1000 bước, lr 10^{-3}, 8 × 256K, khoảng 2B token
Giai đoạn 2	15000 bước, lr 7.3×10^{-6}, 943.7B token	8000 bước, lr 2.5×10^{-5}, khoảng 200B token
Teacher	phân phối attention dense, token-level	max-pool về block, chuẩn hóa L1
Đơn vị chọn	token, $k = 2048$	micro-block 4 token, $K = 2048$

Stage 2 của QSA dùng ít token hơn DSA khoảng 4.7 lần (tự tính).

4.4 Cấu hình triển khai

Mọi lớp full attention của backbone và module MTP thay bằng QSA. Indexer MQA **4 query head**, 1 key head, partial RoPE như trên; $K = 2048$, $r = 4$, nên chọn tối đa **512 block đầy** cho mỗi query cộng phần đuôi. Kernel QSA hợp nhất tính đầu ra sparse attention và KL **không vật chất hóa** kết quả trung gian (giảm bộ nhớ). MTP nhiều bước **tái dùng chỉ số top-k** qua các bước dự đoán (theo GLM-5, bài 69) để giảm chi phí draft.

4.5 Pseudo-code

```
def qsa_layer(x, r=4, K=2048, H_idx=4):
    q_idx = prope(rmsnorm(x @ WQ_idx).view(n, H_idx, 128), pos=arange(n))
    k_blk = rmsnorm(avgpool(x @ WK_idx, r))              # (n//r, 128): NÉN TRƯỚC
    k_blk = prope(k_blk, pos=arange(n // r) * r)         # rồi quay theo p_b
    I = relu(einsum('ihd,bd->ihb', q_idx, k_blk)).sum(1) # (n, n//r), tổng trên head
    I[block_end > i] = -inf                              # chỉ block đã đầy
    S = expand(topk(I, ceil(K / r)), r)[:K] | tail_tokens(i, r)
    o = sparse_core_attention(q, k, v, S)                # softmax trên S_i
    return o * sigmoid(x @ W_gate)                       # cổng output (bài 67)
# Stage 1: đóng băng backbone, chỉ train WQ_idx, WK_idx bằng KL(maxpool(teacher) ||
↪  softmax(I))
# Stage 2: train toàn bộ, KL chỉ trên top-K_B block
```

4.6 Ví dụ và chi phí (tự tính)

Ví dụ nhỏ ($r = 4$, $K = 8$ nên $K_B = 2$, query $i = 13$, chỉ số từ 0). Block đầy: 0-3, 4-7, 8-11; block 12-15 chưa đầy nên token 12, 13 là phần đuôi. Điểm $I = [3.1, 0.4, 1.7]$ chọn block 0 và 2: $S_{13} = \{0..3, 8..11, 12, 13\}$, 10/14 token. Teacher: attention $[0.30, 0.05, 0.05, 0.10 \,|\, 0.02, 0.03, 0.05, 0.02 \,|\, 0.05, 0.20, 0.08, 0.05]$ max-pool thành $[0.30, 0.05, 0.20]$, chuẩn hóa $[0.545, 0.091, 0.364]$ (sum-pool: $[0.50, 0.12, 0.38]$); max-pool đẩy block có một token rất nổi lên.

Chi phí indexer mỗi query ở vị trí i (MAC): token-level với H^I head chiều d^I là $H^I d^I i$; QSA là $4 \cdot 128 \cdot i/4 = 128i$. Nếu so với indexer 32 head × 128 chiều (cấu hình GLM-5 theo bài 68), QSA rẻ hơn 32 lần; so với indexer MSA của bài 72 ($H_{kv} = 4$, giả định $d_{idx} = 128$: $512i$), rẻ hơn 4 lần.

5. Thí nghiệm và kết quả chính

Đường loss (Hình 4): ở Stage 2, loss của QSA bám sát full attention, chênh lệch tổng thể cỡ 10^{-4}.

Bảng 2 (Qwen3.8-Flash-Next, full attention so với QSA):

Method	MMLU-Pro	SuperGPQA	MATH	GSM8K	BBH	MMMLU	EvalPlus	MultiPL-E	Avg.
Full Attn	72.9	51.7	69.8	91.0	90.4	81.8	70.8	78.4	75.9
w/ QSA	73.7	52.1	71.6	92.2	91.6	81.1	72.3	79.8	76.8

QSA bằng hoặc hơn ở 7/8 (kém ở MMMLU).

Bảng 3 (RULER trung bình theo khoảng độ dài; MRCR 8-needle):

Method	RULER ≤128K	128-256K	256-512K	512K-1M	MRCR 128K	256K	512K	1M	Avg.
Full Attn	99.84	99.81	97.65	90.08	97.14	94.20	30.66	20.71	78.76
w/ QSA	99.89	99.62	98.95	93.00	95.98	93.00	40.53	26.44	80.93

Tương đương ở độ dài ngắn hơn, tốt hơn khi ngữ cảnh dài ra (RULER >512K: 90.08 → 93.00; MRCR 512K: 30.66 → 40.53).

Bảng 4 (độ dài chấp nhận trung bình của MTP, speculative 4 bước):

Method	MT-Bench	GSM8K	MATH	HumanEval	MBPP	Avg.
Full Attn	3.44	4.19	4.29	4.24	4.12	4.06
w/ QSA	3.47	4.20	4.30	4.26	4.13	4.07

Tái dùng chỉ số top-k qua các bước MTP không làm đổi độ dài chấp nhận.

Hiệu năng kernel (Hình 6; tốc độ tại 1M). Indexer $r = 4$ so với $r = 1$: **3.8×** ở prefill (chunk 16K cuối, batch 1), **4.4×** ở decode (batch 4, next_n = 4, tức 3 bước MTP). Module attention (indexer + sparse core) so với paged GQA của FlashInfer: **7.6×** prefill, **4.9×** decode. QSA bắt đầu nhanh hơn từ 64K (độ dài nhỏ nhất được đo). Loại GPU không nêu; số tuyệt đối chỉ có trên đồ thị.

Bảng 11 (base model; 125B/6B/51B n-gram so với 27B dense và 397B-A17B):

Benchmark	Qwen3.8-Flash-Next	Qwen3.8-27B	Qwen3.7-Plus
MMLU / MMLU-Redux	90.36 / 90.68	87.51 / 87.26	90.43 / 91.47
MMLU-Pro / SuperGPQA	73.23 / 51.36	68.60 / 44.86	70.90 / 48.42
BBH / GPQA	90.87 / 51.42	89.56 / 45.01	89.41 / 51.52
GSM8K / MATH	93.29 / 72.78	93.18 / 60.54	92.95 / 74.38
EvalPlus / MultiPL-E	78.76 / 79.09	76.05 / 74.50	78.06 / 81.68
SWEBench-Pretrain	50.99	41.66	49.24
MGSM / MMMLU / INCLUDE	89.33 / 84.86 / 78.40	86.37 / 79.74 / 74.37	85.42 / 84.53 / 78.90

Hơn 27B ở cả 14 benchmark; hơn Qwen3.7-Plus ở 8/14, kém nhiều nhất 2.6 điểm (MultiPL-E 79.09 so với 81.68).

6. Phân tích / ablation

6.1 Ablation QSA (35B-A3B, RULER tới 1M, sau Stage 2; số chỉ có trên đồ thị)

- **Tỉ lệ nén:** micro-block 2, 4, 8, 16 được so với **IndexShare** (chia sẻ chỉ số top-k giữa các lớp full attention kề nhau). QSA khớp full attention ở độ trễ indexer tương đối 0.25 (block 4), trong khi IndexShare vẫn dưới baseline ở 0.5 (một index cho hai lớp full attention cách nhau ba lớp GDN). Lý giải của paper: trong hybrid, các lớp attention cách xa nhau nên **độ tương đồng giữa lớp thấp**, chia sẻ index liên lớp kém; nén trong lớp thì không phụ thuộc điều đó.
- **Số query head của indexer** (1, 2, 4, 16): ngay sau Stage 1, dùng indexer để chạy thưa gây **tụt rõ**; một đợt train chung ngắn đưa về mức full attention. Vài head là đủ, ít hơn nhiều so với core attention; chọn 4.

6.2 Gated Residual (tóm tắt)

Residual thành $n_r = 4$ nhánh. Đọc: chuẩn hóa từng nhánh rồi cổng **sigmoid từng phần tử** dự đoán từ mọi nhánh (bottleneck hạng $d/8$), chính là GatedNorm (Eq. 29) áp lên residual rộng. Ghi: một vô hướng $s = 2\sigma(\cdot)$ mỗi nhánh. Bỏ toán tử trộn nhánh H_{res} của HC/mHC (Hyper-Connections: residual tách thành nhiều luồng có trọng số học được; mHC ràng buộc H_{res} trên một manifold) (không mất gì, bớt một lượt đọc residual mỗi block). Bảng 5 (25B-A3B, 560B token), loss / Avg.: 1.617 / 50.91 (pre-norm), 1.596 / 52.49 (mHC static), 1.594 / 54.47 (mHC dynamic), 1.590 / 54.66 (GR). Static sang dynamic: loss chỉ giảm 0.002 nhưng benchmark tăng 1.98 điểm.

Phân tích nhánh (model 20 lớp): đúng một nhánh mang đường dài (ghi chủ yếu ở lớp 0, đọc ở lớp 10-19; GDN lớp 0 → attention lớp 15 tăng từ 0.020 lên 0.138 phần đầu vào), ba nhánh còn lại cục bộ. Nơi đọc nhiều nhất từ nhánh dài **chủ yếu là lớp softmax attention**: paper gọi attention toàn cục là "hub" tích hợp ngữ cảnh dài mà GDN đã nén mất.

Suy luận: chỉ đọc 2 nhánh có cổng cao nhất thì pretrain gần như không đổi nhưng **tụt rõ sau post-training**, nên bỏ. Lưu residual ở **FP8**: các cổng (GR, gated attention, GDN) chặn biên độ nên giảm một nửa byte so với BF16 mà "gần như không mất chất lượng".

6.3 N-gram embedding, optimizer, siêu tham số (tóm tắt)

- **N-gram:** một lớp ở lớp 2 là đủ (Bảng 7). Tăng vocab n-gram 20× tới 200× (vocab gốc 250K) làm loss giảm đều 1.553 → 1.526 (không n-gram: 1.585) nhưng benchmark bão hòa (Bảng 9).
- **Muon** cho ma trận tuyến tính 2D; **AdamW** cho embedding, output head, router MoE, chiếu hạng thấp của GR, các **cổng output** (attention output gate, chiếu z của GDN). NS 8 bước (Polar Express), $\mu = 0.95$, scale $0.2\sqrt{\max(A, B)}$; tách ma trận gộp (qkv, đầu vào GDN theo head) trước khi trực giao hóa.
- **Scaling law mới** dự đoán batch và lr lớn hơn: ở 10.8B-A0.89B/4T token, $B = 25.2$M cho loss 1.5702 so với 1.5774 ($B = 12.6$M cũ); ramp batch không lợi và tốn thêm 18.8% bước. Ở 156B-A7B, điểm trung bình 60.55 so với 56.41 của công thức Qwen3.5.

6.4 Ổn định huấn luyện

Stress test (MoE 28 lớp, giữ lr hằng ở bội số của lr tối ưu, ngưỡng clip 0.5; spike = bước có loss vượt trung vị trượt 201 bước hơn 0.1):

Cấu hình	2× lr (spike/10k bước)	4× lr
Cấu trúc Qwen3.5 + AdamW	4.3	183 spike/10k; vượt ngưỡng clip 213/19,932 bước
Cấu trúc Qwen3.5 + Muon	0.2	không lần nào vượt ngưỡng clip
Muon + Gated Residual	0.2	không vượt ngưỡng; **0 spike**

Cô lập cổng (AdamW, 3× lr, bật/tắt GatedNorm): spike 32.0 → 3.2 mỗi 10k bước, số lần vượt ngưỡng 256 → 20. Không có cổng, outlier activation tăng gần tỉ lệ với lr còn tỉ lệ spike tăng nhanh hơn; paper kết luận lr cao cần một cơ chế **rescaling**, và cổng nhân cung cấp trực tiếp thay vì để mạng tự tạo outlier. **Kiểm chứng ở run thật** (276B token đầu): thêm GR giảm loss 0.026, công thức đầy đủ giảm thêm 0.032 (tổng 0.058); gradient norm trung vị / p99.9 là 0.097 / 0.298 (Muon) so với 0.053 / 0.071 và 0.043 / 0.066 (hai run có cổng); run đầy đủ không có spike, không cần qk-clip hay SwiGLU-clip.

7. Giới hạn

Tác giả nêu: nút thắt lớn nhất là thông lượng đánh giá, cần một phép thử tầm trung rẻ dự đoán được thứ hạng sau post-training; xếp hạng ở Bảng 10 chỉ là quan sát (mỗi run đánh giá một lần); Bảng 1 không cô lập thành phần.

Người ghi chú nhận xét:

- Cấu hình flagship (số lớp, d_{model}, số head core attention, số head GDN) không được nêu; mọi số ablation QSA ở Hình 5-6 chỉ có dạng đồ thị.
- Gated attention không có công thức hay ablation trong bài (phải xem bài 67). Việc mọi head dùng chung một tập chọn và việc có detach indexer ở Stage 2 hay không đều không được bàn.
- Đo kernel chỉ từ 64K, decode ở batch 4 kèm MTP; không có số cho ngữ cảnh vài nghìn token hay batch 1 không MTP. Với chuỗi $\leq K = 2048$ token, mỗi query có tối đa 512 block đầy nên chọn hết, QSA trùng dense (tự suy); các benchmark prompt ngắn ở Bảng 2 vì vậy chủ yếu đo tác động của *huấn luyện* thưa.
- Baseline full attention của Bảng 2 có vẻ là một run CPT song song (Hình 4) nhưng ngân sách token của nó không được nêu rõ.
- Tên gọi: IndexShare được trích là Bai et al. (2026), nhưng mục tham khảo tương ứng mang tên "IndexCache: ... cross-layer index reuse".

8. Liên hệ với các bài khác

- **20, 23, 24 (GLA, DeltaNet, Gated DeltaNet):** nền của lớp GDN; bài này đổi cổng output SiLU thành sigmoid, dùng zero-centered RMSNorm.
- **25 (Jamba), 26 (Kimi Linear, 3:1):** cùng triết lý hybrid; Bảng 1: GDN hybrid > SWA hybrid > Transformer ở 25B-A3B.
- **67 (Gated attention):** cổng output sigmoid của attention; ở đây cổng còn là lý do residual chịu được FP8 và lr cao.
- **68 (DSA), 69 (GLM-5):** QSA theo con đường DSA nhưng nén key indexer; tái dùng index trong MTP theo GLM-5.
- **72 (MSA):** cùng ngân sách 2048 token và KL distill; MSA chọn block 128 theo từng nhóm GQA với indexer token-level ($O(N^2)$), QSA chọn micro-block 4 cho mọi head trên key nén ($O(n^2/r)$).
- **63, 64 (SeerAttention, NSA):** max-pool teacher về block theo SeerAttention; NSA cũng dùng key nén nhưng làm nhánh attention riêng, QSA chỉ dùng để chấm điểm.
- **12 (RoPE), 48 (speculative decoding):** partial RoPE, nén trước khi quay; MTP.
- **distillation 15, 29:** Stage 1 là KD forward KL từ attention dense sang indexer nhỏ, backbone đứng yên.

9. Tóm tắt công thức / khái niệm

Khái niệm	Công thức / ý	Ghi chú
Gated delta rule	$S_t = \alpha_t(I - \beta_t k_t k_t^\top)S_{t-1} + \beta_t k_t v_t^\top$	3 GDN : 1 attention, cổng sigmoid
Key nén	$\text{PRoPE}(\text{RMSNorm}(\text{AvgPool}(k_{p_b:p_b+r-1})), p_b)$	nén trước, quay sau
Điểm block	$I_{ib} = \sum_h \text{ReLU}\langle q_i^h, \bar{k}_b\rangle$, block đã đầy	4 head, không trọng số
Chọn	$\text{TopK}_{\lceil K/r \rceil}$, Expand + đuôi	$K = 2048$, $r = 4$
Indexer	$O(n^2) \to O(n^2/r)$	3.8× prefill, 4.4× decode ở 1M
Stage 1 / 2	KL tới MaxPool teacher; rồi KL trên top-K_B	2B token lr 10^{-3} / 200B token lr 2.5×10^{-5}
Kết quả	Avg 75.9 → 76.8; RULER >512K 90.08 → 93.00	attention 7.6× / 4.9× ở 1M

10. Kết luận ngắn

Qwen3.8-Next cho thấy hybrid GDN : attention 3:1 với cổng sigmoid vừa mạnh hơn Transformer cùng cỡ, vừa ổn định ở lr cao khi kết hợp Muon và Gated Residual. QSA chữa đúng điểm yếu của DSA/MSA ở ngữ cảnh rất dài: indexer chấm điểm trên key nén 4 lần, đưa vào bằng hai giai đoạn CPT ở 256K, giữ nguyên chất lượng và tốt hơn ở 512K-1M.

Link gốc: https://arxiv.org/abs/2608.30320

Bài 74

DeepSeek-V4.1-Flash: Pushing the Limits of KV Cache Compression (2026)

Phạm vi ghi chú: báo cáo kỹ thuật dài (kiến trúc, đa phương thức, hạ tầng, pre-training, post-training agentic). Ghi chú này **tập trung vào những gì thay đổi so với DeepSeek-V4 (bài 71) về nén KV cache và attention**: Causal Encoder-Decoder (CED), Compressed Sparse Attention 2 (CSA2) với ba chế độ Full/Reindex/Reuse, Hierarchical Sparse Indexer, FP4 main KV cache, SWA Bounded Replay và quản lý persistent KV cache; phân tích chi phí (890 byte/token, 1/4 và 1/8 so với V4-Flash, FLOPs decode) và kết quả. Single-Pass mHC, Engram, DSpark, phần thị giác, optimizer, dữ liệu và post-training chỉ tóm tắt ở mục 6. Mục 10 tách riêng thông tin về bản cập nhật V4.1-Flash-1017 lấy từ nguồn ngoài arXiv.

Tác giả: DeepSeek-AI (research@deepseek.com); danh sách tác giả xếp theo chữ cái ở Phụ lục A.
Venue / arXiv: arXiv:2609.19969 (v1, 17/09/2026); paper không nêu venue.
Nguồn: https://arxiv.org/abs/2609.19969
Chủ đề: nén KV cache theo ba chiều nhân với nhau (kích thước entry, chiều chuỗi, chiều lớp); mô hình MoE đa phương thức 552B tham số backbone, kích hoạt 8B khi prefill và 16B khi decode, ngữ cảnh 1M token.
Vị trí trong lộ trình: bắt buộc đọc 71 (DeepSeek-V4: CSA, HCA, nhánh SWA, KV trên đĩa) trước; nên nắm 16 (MLA), 68 (DSA), 15 (GQA). Liên quan 48-50 (speculative decoding, DSpark), 29/31/38 (ViT, SigLIP, SmolVLM). Đây là bài cuối của "Giai đoạn 8: Sparse attention": sau khi attention đã thưa, nút cổ chai chuyển sang dung lượng và băng thông của KV cache.

> Tài liệu học tập bằng tiếng Việt, là ghi chú diễn giải, không phải bản dịch. Số liệu lấy từ paper (full text arXiv, tải ngày 07/11/2026); các phép tính đánh dấu "tự tính" là của người ghi chú. Mục 10 dùng nguồn ngoài arXiv (model card và metadata Hugging Face, trang tin api-docs.deepseek.com, truy cập 07/11/2026) và được đánh dấu rõ.

1. Bối cảnh

Các tác vụ agent dài hạn liên tục bổ sung kết quả gọi công cụ vào ngữ cảnh. Dù sparse attention (bài 68, 71) đã giảm phép tính trên chuỗi dài, bài báo vẫn nêu ba nguồn chi phí:

- **Prefill:** xử lý lại đầu vào khi cache không có dữ liệu cần dùng.
- **Runtime KV cache:** chiếm bộ nhớ HBM và giới hạn số yêu cầu đồng thời.
- **Persistent KV cache:** lưu trên SSD hoặc RAM để dùng lại tiền tố, đồng thời phát sinh chi phí nạp và di chuyển dữ liệu.

Thuật ngữ của paper: ở V4, nhánh **global** (CSA/HCA) giữ **global KV** gồm **main KV** (entry dùng cho attention chính) và **indexer K**; nhánh **SWA** giữ **local KV**, bị chặn trên bởi cửa sổ n_{win} nên không tăng theo độ dài. Với chuỗi đủ dài, global KV chiếm gần hết runtime KV.

Paper chỉ ra ba chiều nén **nhân với nhau**: (1) kích thước entry (GQA, MLA); (2) chiều chuỗi (gộp m token, như CSA/HCA); (3) chiều lớp (một lớp dùng lại cache hoặc lựa chọn của lớp khác, như Cross-Layer Attention của Brandon et al. 2024, IndexCache, YOIO, HySparse). Theo paper, các công trình trước không phủ cả ba chiều cùng lúc.

2. Ý tưởng chính

Paper nhìn V4 như "một backbone xử lý cục bộ bằng SWA, cộng ngữ cảnh toàn cục đã nén", nên giữ nguyên nhánh SWA ở mọi lớp và tối giản nhánh global. **CED** (lấy cảm hứng từ YOCO) chia 40 lớp thành 20 lớp encoder và 20 lớp decoder; global KV của decoder được chiếu thẳng từ hidden state cuối encoder, nên prefill

chỉ cần chạy nửa mô hình. **CSA2** thay cặp CSA/HCA bằng một loại attention duy nhất, cho phép các lớp **dùng chung main KV và indexer K** (chế độ Reindex) và **dùng chung luôn chỉ số top-k** (chế độ Reuse); trong decoder, **Hierarchical Sparse Indexer** cho các indexer sau chỉ chấm điểm trong một "candidate pool" do indexer đầu tạo. Main KV được lưu **FP4** (QAT trong post-training). Ở mức triển khai, **SWA Bounded Replay** khôi phục gần đúng KV của SWA bằng cách chạy lại chỉ n_{win} token, nên không cần lưu SWA KV vào persistent cache. Kết quả: global KV **890 byte/token**, khoảng 1/4 V4-Flash; persistent KV khoảng 1/8 V4-Flash; FLOPs decode chỉ tăng khoảng 1/4 khi ngữ cảnh tăng từ 4K lên 1M.

3. Phương pháp chi tiết: Causal Encoder-Decoder

Global attention. $L/2$ lớp dưới là causal encoder. Với lớp decoder $l > L/2$, KV entry và trọng số nén **không** lấy từ hidden state H_l của chính lớp mà từ $H_{L/2}$, bằng ma trận riêng của từng lớp (Eq. 1):

$$\boxed{C_l = H_{L/2} W_l^{KV}, \qquad Z_l = H_{L/2} W_l^{Z}, \qquad l > L/2}$$

Hệ quả: khi prefill, chỉ cần tính nửa dưới để có global KV cho cả nửa trên. Khác YOCO (nửa trên dùng chung **một** KV của nửa dưới), CED cho mỗi lớp decoder phép chiếu riêng (paper: tăng "dung lượng" KV và độ sâu tính toán sinh KV).

SWA vẫn theo từng lớp. KV cục bộ của mọi lớp (kể cả decoder) vẫn sinh từ H_l. Muốn có SWA KV của decoder cho các bước decode đầu, chính xác thì phải chạy decoder trên $n_{win} \times L/2$ token cuối (vì phụ thuộc SWA chồng qua từng lớp). Paper dẫn Chen et al. (2025): trường nhìn hiệu dụng của SWA nhỏ hơn nhiều so với lý thuyết, nên dùng **Decoder SWA Bounded Replay**: chỉ chạy decoder trên n_{win} token cuối của prompt, với SWA bị cắt trong đoạn replay (mục 5.4). Độ phức tạp prefill:

$$\boxed{O(NL) \;\longrightarrow\; O(NL/2 + n_{win}L/2) \approx O(NL/2) \quad (N \gg n_{win})}$$

Đây là lý do mô hình kích hoạt 8B tham số mỗi token khi prefill và 16B khi decode (decode vẫn chạy đủ 40 lớp cho từng token mới).

4. Phương pháp chi tiết: CSA2 và FP4 main KV

4.1 CSA2 đơn giản hóa CSA thế nào

So với CSA (bài 71 §3), CSA2 vẫn có indexer chấm điểm main KV, chọn top-k, và mỗi query nhìn entry được chọn cộng SWA KV của lớp. Khác biệt:

- Bộ nén **không chồng lấp** và **bỏ positional embedding tuyệt đối** trong bộ nén (diễn giải: tức các bias vị trí B^a, B^b của V4).
- **Indexer K chiếu từ main KV** thay vì có đường nén riêng từ hidden state.
- Cho phép tỉ lệ nén $m = 1$ (main KV không nén).
- **Chỉ dùng CSA2**, bỏ hẳn HCA.

4.2 Ba chế độ, gán tĩnh cho từng lớp (§2.3.1)

Chế độ	Main KV và indexer K	Indexer Q, chấm điểm	Top-k	Luôn tự tính
Full	tự tính (decoder: từ $H_{L/2}$)	có, trên mọi vị trí nhìn thấy	mới	main Q, SWA KV
Reindex	dùng lại của lớp Full gần nhất	có, chấm lại key dùng chung	mới	main Q, SWA KV

Chế độ	Main KV và indexer K	Indexer Q, chấm điểm	Top-k	Luôn tự tính
Reuse	dùng lại	không	dùng lại của lớp Full/Reindex gần nhất	main Q, SWA KV

Dùng chung main KV và indexer K giảm dung lượng cache; dùng lại top-k bỏ được phép tính indexer; Reindex giữ chia sẻ cache nhưng để lựa chọn thay đổi theo lớp.

4.3 Cấu hình V4.1-Flash (§4.2.1)

Phần	Cấu hình
Tổng	40 lớp, $d = 5120$; encoder 20 lớp, decoder 20 lớp
Encoder	lớp 0-1: SWA thuần; 18 lớp CSA2 $m = 2$, 3 nhóm 6 lớp: Full + 5 Reuse
Decoder	20 lớp CSA2 $m = 1$, 5 nhóm 4 lớp: nhóm 1 là Full + 3 Reuse, 4 nhóm sau là Reindex + 3 Reuse
Indexer	32 head, head dim 128, top-k 512
Attention chính	64 query head, head dim 512, query compression 1280; 8 nhóm output, $d_g = 1024$; $n_{win} = 128$
Hierarchical indexer	tối đa 2,048 block, mỗi block 8 vị trí, tức 16,384 vị trí ứng viên
MoE	1 shared + 384 routed (dim 2304), 6 kích hoạt; SwiGLU clamp 10
mHC	mở rộng 4 lần, Sinkhorn 20 vòng
Thị giác	ViT 32 lớp, hidden 1024, 16 head, patch 14; projector MLP 2 lớp, hidden 5120

Tức chỉ **4 lớp** sinh main KV (3 Full của encoder, 1 Full của decoder) và **8 lớp** chạy indexer (4 Full + 4 Reindex). `config.json` trên Hugging Face (nguồn ngoài paper) ghi đúng như vậy: `kv_source_layer_ids = [2, 8, 14, 20]`, `index_source_layer_ids = [2, 8, 14, 20, 24, 28, 32, 36]`, `candidate_source_layer_id = 20`.

4.4 Hierarchical Sparse Indexer (§2.3.2)

Dù dùng lại chỉ số, các indexer còn lại vẫn quét toàn bộ ngữ cảnh, là nút cổ chai ở chuỗi cực dài (bài 71 §5.3). Chỉ trong decoder: lớp Full đầu tiên chấm **mọi** vị trí, lấy top-512 cho chính nó, đồng thời chọn block: điểm của block = **max** điểm các vị trí trong block, lấy 2,048 block cao nhất (mỗi block 8 vị trí), gom thành pool 16,384 vị trí. Các lớp Reindex sau chỉ chấm điểm trong pool rồi chọn top-512 riêng. Với pool cố định, chi phí mỗi indexer sau **không phụ thuộc độ dài ngữ cảnh**; lớp Full đầu vẫn quét toàn bộ. Cơ chế được đưa vào **trong post-training**, áp dụng giống hệt khi train và khi suy luận.

4.5 FP4 main KV cache (§2.4.4)

- V4 đã dùng QAT FP4 (MXFP4 chuẩn OCP) cho query/key của indexer. V4.1 mở rộng QAT sang **main KV** để giảm dung lượng (giải lượng tử trước attention, không cần phép nhân FP4 gốc).
- Định dạng: E2M1 với **một scale E4M3 cho mỗi 16 kênh** (theo NVFP4 nhưng bỏ scale toàn cục cấp hai). Biên độ biểu diễn tối đa $448 \times 6 = 2688$.
- Lập luận vì sao bỏ scale toàn cục là an toàn: trọng số RMSNorm lớn nhất khoảng 1, nên sau RMSNorm chuẩn L2 của latent 512 kênh tối đa khoảng $\sqrt{512}$; RoPE giữ chuẩn, nên mọi phần tử không vượt quá khoảng $\sqrt{512} \approx 22.6$; thực đo khi train tối đa khoảng 10.
- Lượng tử **sau RoPE** (trước RoPE chỉ tốt hơn rất ít mà tốn thêm khi decode). Phần RoPE và không RoPE cùng định dạng. **SWA KV giữ FP8** vì nhạy với lượng tử. So với main KV FP8 của V4, dung lượng gần giảm một nửa.

4.6 Pseudo-code (một lớp CSA2, đơn giản hóa)

```
def csa2_layer(x, mode, S, P, in_decoder):          # S: trạng thái dùng chung giữa các lớp
    q, swa = main_query(x, P), swa_kv_fp8(x, P)      # mọi chế độ: Q và SWA KV của chính lớp
    if mode == "Full":
        src = S.h_enc_last if in_decoder else x      # CED: KV decoder chiếu từ H_{L/2}
        S.kv = fp4_e2m1_s16(rope(compress_no_overlap(src, P.m)))   # main KV, lượng tử sau
↪   RoPE
        S.ik = mxfp4(S.kv @ P.W_ik)                  # indexer K chiếu từ main KV
        scores = indexer(x, S.ik, cand=None)         # quét mọi vị trí nhìn thấy
        S.idx = topk(scores, 512)
        if in_decoder:                               # Hierarchical Sparse Indexer
            S.pool = positions_of(top_blocks(blockmax(scores, 8), 2048))    # 16,384 ứng
↪   viên
    elif mode == "Reindex":
        S.idx = topk(indexer(x, S.ik, cand=S.pool), 512)    # query indexer riêng, key dùng
↪   chung
    # mode == "Reuse": giữ nguyên S.kv, S.idx
    kv = cat([dequant(S.kv[S.idx]), swa])
    return attend_mqa_with_sink(q, kv, P)            # như V4: K = V, sink, output RoPE
```

5. Phân tích chi phí

5.1 Tái tạo 890 byte/token (tự tính)

Paper chỉ cho con số tổng. Giả định: main KV dùng E2M1 + một scale E4M3 cho 16 kênh (9 B mỗi 16 kênh); indexer K dùng MXFP4 chuẩn (E2M1 + scale 1 byte cho 32 phần tử, 17 B mỗi 32 kênh).

Thành phần	Nguồn	Số kênh trên mỗi token gốc	Byte/token
Main KV, encoder	3 lớp Full, $m = 2$	$3 \times 512/2 = 768$	$768 \times 9/16 = 432$
Main KV, decoder	1 lớp Full, $m = 1$	512	288
Indexer K, encoder	3 lớp, $m = 2$	$3 \times 128/2 = 192$	$192 \times 17/32 = 102$
Indexer K, decoder	1 lớp	128	68
Tổng global KV			**890**

Khớp đúng 890 B của paper, và cho thấy main KV chiếm khoảng 81% global KV. So với V4-Flash (bài 71 §5.2, tự tính khoảng 3,471 B/token): $890/3471 \approx 0.26$, khớp "khoảng 1/4". Ở 2^{20} token: $890 \times 1{,}048{,}576 \approx 0.93$ GB cho một chuỗi.

Nguồn của mức giảm, tách theo ba chiều (diễn giải của người ghi chú): chiều lớp (chỉ 4 lớp giữ main KV thay vì 41 lớp CSA/HCA), chiều chuỗi (encoder $m = 2$ nhưng decoder $m = 1$, tức thực ra nén chuỗi **yếu hơn** V4), độ chính xác (FP4 thay vì FP8/BF16). Phần lớn lợi ích đến từ chia sẻ theo lớp.

Paper nói giảm khoảng **437 lần** so với DeepSeek-V1 (Hình 1b). Kiểm tra (tự tính, cấu hình V1 không có trong paper này): nếu V1 là DeepSeek LLM 67B với 95 lớp, GQA 8 nhóm KV, head dim 128, BF16, thì $2 \times 8 \times 128 \times 95 \times 2 = 389{,}120$ B/token, và $389{,}120/890 \approx 437$.

5.2 Persistent KV cache: vì sao 1/8 (§3.2.1)

- Ở V4, SWA KV chiếm **gần một nửa** persistent cache (dù chỉ lưu n_{win} entry ở hai điểm: cuối prompt và cuối output), global KV và SWA KV đều quản lý LRU, cache SSD đủ lớn để giữ trên 72 giờ.

- V4.1: SWA KV **không** vào persistent cache nữa mà vào một memory pool phân tán dùng 10% DRAM máy chủ, TTL chỉ vài phút (SWA KV chỉ được dùng lại trong một phiên đang hoạt động). Global KV vẫn ở persistent cache, sống ít nhất 72 giờ.
- Hai thừa số: bỏ SWA (khoảng 1/2) nhân global KV còn 1/4, ra khoảng **1/8**.

5.3 SWA Bounded Replay (§3.2.2)

Dựng lại chính xác SWA KV của L lớp cần chạy lại $L \times n_{win}$ token. Bounded replay chỉ chạy lại n_{win} token cuối, và cắt SWA trong đoạn replay: replay bắt đầu ở vị trí s thì query ở vị trí i chỉ nhìn SWA key trong

$$\boxed{[\max(s,\ i - W + 1),\ i]}$$

Số token phải chạy lại (tự tính, $L = 40$, $n_{win} = 128$): chính xác $40 \times 128 = 5{,}120$ token, bounded 128 token, tức ít hơn 40 lần.

- **Encoder SWA Bounded Replay:** khi prefix hit global KV nhưng miss SWA KV, chạy lại n_{win} token cuối của prefix cùng phần suffix mới; token replay chỉ sinh lại SWA KV, không ghi đè global KV. Trạng thái là gần đúng, nên global/SWA KV của suffix phụ thuộc vị trí cache hit; paper nói chất lượng "barely compromises".
- **Decoder SWA Bounded Replay:** mỗi lần prefill, chạy decoder trên n_{win} token cuối của prompt, dùng SWA KV thu được chỉ cho decode (không đưa vào prefix cache). Chính xác thì phải chạy $L/2$ lớp decoder trên $L/2 \times n_{win} = 2{,}560$ token (tự tính). Paper mô phỏng cùng replay này trong post-training để mô hình thích nghi.

5.4 FLOPs decode theo độ dài

Hình 2: FLOPs decode mỗi token (trọng số BF16 = 1, FP8 = 0.5, FP4 = 0.25) của V4.1-Flash gần như phẳng: tăng ngữ cảnh 256 lần (4K lên 1M) chỉ làm FLOPs tăng khoảng **1/4**, ít hơn nhiều so với V4-Flash.

Ước lượng phía attention ở 1M token (tự tính, 2 FLOPs mỗi nhân-cộng, chưa quy đổi độ chính xác):

Thành phần	V4-Flash	V4.1-Flash
Vị trí indexer chấm mỗi query	$21 \times 262{,}144 \approx 5.5$M (64 head)	$3 \times 524{,}288 + 1{,}048{,}576 + 4 \times 16{,}384 \approx 2.69$M (32 head)
FLOPs indexer	khoảng 90 G	$2.69\text{M} \times 32 \times 128 \times 2 \approx 22$ G
Attention chính	$21 \times 0.084 + 20 \times 1.09 \approx 23.6$ G (HCA tăng theo n)	$38 \times (512 + 128) \times 64 \times 512 \times 4 \approx 3.2$ G (không đổi)

Phần còn tăng theo n ở V4.1 chỉ là 4 indexer của lớp Full (FP4). Bỏ HCA và dùng pool ứng viên là hai bước làm đường cong phẳng.

6. Các thay đổi khác, huấn luyện và post-training (tóm tắt)

- **Single-Pass mHC:** dịch hệ số trộn input đi một block, $X_{l+1} = B_l X_l + C_l \mathcal{F}_l(A_{l-1} X_l)$, nên trộn input không còn chờ phép rút gọn trên toàn hidden; kernel Mega-mHC đọc/ghi residual một lần, lưu lượng bộ nhớ activation từ $(4n+4)d$ (bản nhiều kernel của V4) xuống $(2n+2)d$. Paper: suy giảm chất lượng "negligible".
- **Engram:** bộ nhớ có điều kiện, 196B tham số chia hai module ở lớp 1 và 14 (đánh số từ 0), N-gram bậc 2-4, 8 hash head, mỗi head bảng khoảng 16M entry, FP8; prefetch từ RAM máy chủ qua RDMA.
- **DSpark:** speculative decoding, drafter 3 block với cửa sổ 128 token, sinh 5 vị trí nháp trong một lượt, Markov head nhẹ và confidence head để chọn độ dài kiểm tra theo tải hệ thống; train riêng sau

pre-training (backbone đóng băng), rồi train song song trong post-training không truyền gradient vào backbone. Thay MTP của V3/V4.

- **Thị giác:** DeepSeek-ViT train từ đầu (2D-RoPE, RMSNorm, SwiGLU); pixel-unshuffle 3×3 giảm visual token 9 lần, ảnh tới khoảng 1344×1344; MoE tách bias cân bằng tải cho token ảnh và văn bản. ViT học bằng SigLIP loss trên khoảng 47B cặp ảnh-văn bản, rồi fine-tune tự hồi quy với một LLM MoE 4B trên 236B token.
- **Optimizer:** Muon theo từng head cho Q và K; momentum + Sinkhorn balancing cho Engram, embedding, prediction head.
- **Pre-training:** 45T token (text-only : đa phương thức = 7:1), batch cố định 100.6M token, lr 2.6×10^{-4} rồi cosine về 2.6×10^{-5} (28T-40T). **Sparse attention train từ đầu ở 64K, không có dense warm-up** (khác V4); lên 1M ở 34T token.
- **Post-training:** SFT, RL, OPD (hơn 40 teacher), **không đổi thuật toán**; cải thiện đến từ dữ liệu (tổng hợp tác vụ agent, RL bất đồng bộ trên nhiều scaffold, model merging giữa các lượt RL). **Reasoning effort** $b \in \{1..100\}$ trong system prompt, phạt độ dài $-\min(C_{max}, k(b)\ell/L_{norm})$ với $k(b) = k_0 e^{-(b-b_{min})/\tau}$; API max/high/low ứng với $b = 100/75/50$. Liên hệ rl 00 (GRPO), distillation 29.

7. Thí nghiệm và kết quả chính

Base model (Bảng 1, chọn dòng):

Benchmark	V4-Flash-Base (13B / 284B)	V4-Pro-Base (49B / 1.6T)	V4.1-Flash-Base (8B/16B / 552B)
MMLU-Pro (5-shot)	68.3	73.5	74.1
Simple-QA verified (25-shot)	30.1	55.2	42.3
SuperGPQA (5-shot)	46.5	53.9	53.1
BBEH (1-shot)	25.4	29.8	27.2
HumanEval (0-shot)	69.5	76.8	79.4
MATH (4-shot)	57.4	64.5	61.1
MGSM (8-shot)	85.7	84.4	80.2
LongBench-V2 (1-shot)	44.7	51.5	45.2
DocVQA (LLM-Judge, 4-shot)	-	-	95.6
MMMU-Pro (4-shot)	-	-	56.5

Lưu ý: một số dòng của V4-Flash-Base khác bài 71 vì thiết lập khác (AGIEval 3-5-shot ở đây là 83.9, bài 71 0-shot là 82.6; MultiLoKo ở đây chấm bằng LLM-Judge 42.6, bài 71 dùng EM 42.2). Hình 6: V4.1-Flash-Base có bits-per-byte thấp nhất trên mọi tập held-out nội bộ (chỉ có hình).

Sau post-training (Bảng 3, chế độ Max, chọn dòng):

Benchmark	Opus-5	GPT-5.6 Sol	K3	GLM-5.3	DS-V4-Pro	DS-V4-Flash	DS-V4.1-Flash
GPQA Diamond	93.4	94.1	92.9	88.1	92.4	89.9	90.9
HLE	56.3	44.5	43.5	42.0†	42.7†	37.8†	36.8 (39.1†)
Codeforces (Rating)	-	-	-	-	3348	3289	3471
MathArena Apex	-	-	65.6	-	65.3	58.6	65.6

Benchmark	Opus-5	GPT-5.6 Sol	K3	GLM-5.3	DS-V4-Pro	DS-V4-Flash	DS-V4.1-Flash
Terminal-Bench 2.1	89.1	88.8	88.3	88.2	87.9	82.7	90.6
Terminal-Bench 4.0	51.8	39.9	12.6	37.9	12.4	7.0	31.2
DeepSWE v1.1	74.0	73.0	67.5	66.9	62.7	54.4	74.2
Automation-Bench	50.3	45.8	46.7	48.8	43.2	37.7	54.8

(† = tập con chỉ có văn bản của HLE.) Các cột DS-V4-Pro và DS-V4-Flash ở đây **khác** số trong bài 71 (ví dụ Codeforces 3348 và 3289 so với 3206 và 3052; GPQA của V4-Pro 92.4 so với 90.1); paper V4.1 không giải thích, có thể là bản V4 sau preview hoặc thiết lập đánh giá khác (suy đoán).

Reasoning effort (§5.3.2): tăng effort từ 25 lên 100: trung bình 8 benchmark reasoning từ 67.1% lên 76.3%, DeepSWE v1.1 từ 66.0% lên 74.2%, Terminal-Bench 2.1 từ 82.4% lên 90.6%, tốn khoảng 2.5 lần token output. Khoảng 60-80 đã đạt phần lớn độ chính xác với chưa tới nửa ngân sách token; bước cuối lên 100 làm trajectory agent dài thêm 1.6-1.8 lần.

Theo scaffold (Bảng 4, Max): DeepSWE v1.1 từ 65.5 (OpenCode) đến 74.2 (mini-SWE); Terminal-Bench 2.1 từ 84.1 (Codex) đến 90.6 (DeepSeek Harness Minimal). Thí nghiệm multi-agent sơ bộ (§5.3.5) cũng được báo cáo nhưng không liên quan attention.

8. Phân tích / ablation

Paper **không có bảng ablation số** cho các thành phần nén KV, chỉ có nhận định định tính:

- CED: “maintaining performance comparable to the baseline”.
- FP4 main KV: “only marginal performance degradation”; bỏ scale toàn cục: “no measurable decrease”; lượng tử trước RoPE chỉ tốt hơn rất ít.
- SWA Bounded Replay: “negligible”, “barely compromises response quality”. Single-Pass mHC: “negligible”.
- Head-wise Muon tốt hơn Muon thường; Sinkhorn-balanced update tốt hơn Adam; có định dạng FP4 chính xác hơn MXFP4 nhưng vẫn chọn MXFP4 vì tương thích phần cứng.
- Hành vi: độ chính xác tăng đơn điệu theo effort, lợi ích dồn ở khoảng thấp-trung; Hình 7-8: điểm agent tiếp tục tăng theo số bước RL.

9. Giới hạn

Tác giả nêu: các thay đổi kiến trúc tạo ra “robustness boundaries” chưa được mô tả đầy đủ; lỗi lựa chọn của CSA2 và trạng thái gần đúng của SWA Bounded Replay có thể gây suy giảm ở các trường hợp biên chưa kiểm thử (sẽ mở rộng stress test cho sparse retrieval ngữ cảnh dài và cho ranh giới khôi phục cache); vẫn còn khoảng cách với mô hình đóng lớn nhất trên tác vụ khó nhất (ví dụ Terminal-Bench 4.0, tác vụ khoa học cần kiến thức chuyên gia) và với hệ thống đóng ở tác vụ thị giác; benchmark đang bão hòa.

Người ghi chú nhận xét:

- Paper nén KV mạnh nhất từ trước tới nay nhưng **không báo cáo MRCR hay CorpusQA ở 1M** (bài 71 có); đánh giá long-context chỉ có LongBench-V2 của base model: 45.2, gần V4-Flash (44.7) và kém xa V4-Pro (51.5).
- So sánh với V4-Flash không sạch: V4.1 lớn hơn (552B so với 284B, thêm 196B Engram), nhiều dữ liệu hơn (45T so với 32T), đa phương thức. Không tách được phần do CSA2/CED.

- Decoder dùng $m = 1$ và chỉ một lớp sinh main KV; mọi lớp decoder phụ thuộc một bộ KV duy nhất chiếu từ $H_{L/2}$, nên lỗi ở bộ KV này lan ra toàn decoder.
- Bounded replay làm kết quả phụ thuộc vị trí cache hit: cùng prompt có thể cho output khác nhau tùy trạng thái cache, khó tái lập (paper tự thừa nhận không tương đương toán học).

---

10. Sau paper: bản cập nhật V4.1-Flash-1017 (nguồn ngoài arXiv)

Cảnh báo: toàn bộ mục này **không thuộc paper arXiv:2609.19969**. Nguồn: model card và API metadata của repo `deepseek-ai/DeepSeek-V4.1-Flash-1017` trên Hugging Face, và trang tài liệu api-docs.deepseek.com, truy cập 07/11/2026. Chỉ ghi điểm đã kiểm tra được.

Thông tin	Nguồn	Trạng thái
Ngày phát hành 17/10/2026	Trang tin "DeepSeek-V4.1-Flash-1017 Release 2026/10/17" (api-docs.deepseek.com/news/news261017); metadata HF `createdAt` 2026-10-17T02:40:38Z	đã xác minh
Là "a minor update to DeepSeek-V4.1-Flash", cải thiện "reasoning, coding, agentic tasks, and visual benchmarks", "reduces task execution time and output length"	Model card HF	đã xác minh
"Stronger benchmark performance, faster task completion, and shorter outputs than the previous version"; "uses reinforcement learning with self-feedback to improve end-to-end project development"	Trang tin 17/10/2026	đã xác minh
Giấy phép MIT cho repo và trọng số	Model card HF (mục License; `license: mit`)	đã xác minh
`config.json` giống hệt repo V4.1-Flash; số tham số theo metadata safetensors bằng nhau: 763,205,315,794 (cùng phân bổ BF16/F32/F8_E4M3/I8)	So sánh của người ghi chú (diff hai file, API HF)	đã xác minh; con số 763B lớn hơn 552B backbone của paper, có lẽ vì gồm cả Engram, ViT, DSpark (suy đoán)
Link "Technical Report" trỏ về PDF của V4.1-Flash; không có báo cáo riêng	Model card HF	đã xác minh
Khuyến nghị sampling: temperature 1.0, top_p 1.0, context 1M token, max_tokens ≥ 256K; không kèm chat template Jinja (dùng `encoding.py` và thư viện deepseek-recipe)	Model card HF	đã xác minh

Thông tin	Nguồn	Trạng thái
Tên API `deepseek-flash` "is now served by DeepSeek-V4.1-Flash-1017"; tên cũ `deepseek-v4-flash`, `deepseek-v4-flash-vision-exp` vẫn nhận nhưng model cũ đã ngừng	api-docs.deepseek.com (hướng dẫn gọi API)	đã xác minh
Tối ưu cho DeepSeek Harness, Tencent CodeBuddy, WorkBuddy; nên dùng Harness v0.2.1-rc.1 trở lên	Trang tin 17/10/2026	đã xác minh

Bảng benchmark trên model card (nguồn ngoài paper):

Benchmark	V4.1-Flash-1017	V4.1-Flash	Khớp Bảng 3 của paper?
HLE w/o tools	42.6 (44.0†)	36.8 (39.1†)	có
Terminal-Bench 4.0	42.3	31.2	có
DeepSWE v1.1	77.0	74.2	có
ProgramBench	29.1	20.3	có
NL2Repo-Bench	68.2	65.4	có
Automation-Bench v1.0.6	60.5	54.8	có
Agents' Last Exam	35.4	31.8	có
Apex Bench v1.1	72.1	64.6	không có trong paper (paper dùng MathArena Apex: 65.6)
Chartography (w/tools)	83.0	78.9	có
BabyVision (w/tools)	92.5	89.6	có
ZeroBench-main (w/tools)	51.0	49.0	có

Model card ghi dùng Claude Code harness cho Terminal-Bench 4.0 và Mini-SWE-V2 cho DeepSWE v1.1; paper dùng DeepSeek Harness (Minimal) cho code agent và mini-SWE cho DeepSWE, nên thiết lập có thể không trùng hoàn toàn với Bảng 3.

Không xác minh được: không có trang chính thức nào viết nguyên văn rằng 1017 là "post-training update". Các dấu hiệu (config và bố cục trọng số giống hệt, "minor update", "reinforcement learning with self-feedback") **phù hợp** với việc chỉ cập nhật trọng số bằng post-training, nhưng đó là suy luận. Không có thông tin mới nào về KV cache, attention hay kết quả long-context cho bản 1017.

---

11. Liên hệ với các bài khác

- **71 (DeepSeek-V4):** tiền thân trực tiếp: CSA thành CSA2, bỏ HCA, Zero SWA Caching thành SWA Bounded Replay, MTP thành DSpark, mHC thành Single-Pass mHC.
- **68 (DSA), 64 (NSA):** indexer + top-k; Hierarchical Sparse Indexer giống ý chọn block của NSA (điểm block = max điểm vị trí) nhưng chỉ để thu hẹp miền chấm điểm.
- **16, 14, 15 (MLA, MQA, GQA):** chiều "kích thước entry"; CSA2 thêm chiều "lớp". **06, 17, 55:** sliding window, sink.

- **46, 47:** quản lý KV và tái dùng prefix; V4.1 tách vòng đời global KV (ngày) và SWA KV (phút).
- **48-50:** speculative decoding (DSpark). **51-54:** lượng tử hóa, ở đây áp cho KV cache.
- **29, 31, 38, 57:** ViT, SigLIP, SmolVLM (pixel shuffle, còn dùng để chấm điểm dữ liệu), FastV.
- **69, 72, 73:** các thiết kế sparse/lai cùng thời. Ngoài thư mục: rl 00 (GRPO), rl 54 (agentic RL), distillation 29 (on-policy KD).

12. Tóm tắt công thức / khái niệm

Khái niệm	Công thức / ý	Ghi chú
CED	$C_l = H_{L/2}W_l^{KV}$, $Z_l = H_{L/2}W_l^{Z}$ với $l > L/2$	prefill $O(NL/2 + n_{win}L/2)$; 8B/16B kích hoạt
CSA2 Full	tự tính main KV, indexer K, top-k	4 lớp nguồn KV: 2, 8, 14, 20
CSA2 Reindex	dùng KV chung, chấm lại bằng query indexer riêng	4 lớp decoder
CSA2 Reuse	dùng chung KV và top-k	phần lớn các lớp
Hierarchical indexer	điểm block = max; 2,048 block × 8 = 16,384 ứng viên	chỉ decoder, đưa vào khi post-training
FP4 main KV	E2M1 + scale E4M3 mỗi 16 kênh, sau RoPE	SWA KV giữ FP8
Global KV	890 B/token	khoảng 1/4 V4-Flash, 437 lần nhỏ hơn V1
Persistent KV	bỏ SWA (khoảng 1/2) × global 1/4	khoảng 1/8 V4-Flash
Bounded replay	SWA key trong $[\max(s, i - W + 1), i]$	chạy lại n_{win} thay vì $L\, n_{win}$ token
Decode FLOPs	4K lên 1M chỉ tăng khoảng 1/4	trọng số BF16 1, FP8 0.5, FP4 0.25
Effort	$k(b) = k_0 e^{-(b-b_{min})/\tau}$	API 100/75/50

13. Kết luận ngắn

DeepSeek-V4.1-Flash cho thấy sau khi attention đã thưa, đòn bẩy lớn nhất còn lại là **chiều lớp**: chỉ 4 trong 40 lớp sinh main KV, các lớp khác dùng lại KV (và thường cả chỉ số top-k), cộng FP4 cho main KV, đưa global KV xuống 890 byte/token (tính tay khớp đúng), khoảng 1/4 V4-Flash; bỏ SWA khỏi persistent cache nhờ bounded replay đưa persistent KV về khoảng 1/8. CED cắt nửa prefill, Hierarchical Sparse Indexer và việc bỏ HCA làm FLOPs decode gần như phẳng theo độ dài. Paper hầu như không có ablation số cho các lựa chọn này và không đo long-context ở 1M sau post-training. Bản 1017 (nguồn ngoài arXiv) giữ nguyên kiến trúc và cải thiện benchmark agent.

Link gốc: https://arxiv.org/abs/2609.19969